Leer columna izquierda completa y despues la derecha. Texto seleccionable; lineas largas comprimidas. Tabulaciones: 4 espacios. Original adjunto.

ARANDOMMENU — CODIGO MM2 Y MODULOS COMPARTIDOS
Origen: XzAngel19/ARandomMenu, rama arena/0ce50d99-arandommenu
Commit exacto: f7de1b3ee13ce8783a4afb8f39b8426912ba5733
Version del runtime: v199-20261108-aim-camera-fire

Este documento recopila archivos completos como texto de referencia; NO es un script independiente listo para ejecutar. Las secciones conservan sus rutas originales; las categorias de carpetas pueden diferir de las categorias visibles en el menu. Los modulos universales se comparten con otros juegos y no son exclusivos de MM2.
Se incluyen todas las fuentes MM2 y universales, bibliotecas compartidas, interfaz y loader. No se incluye bundle.lua: es generado, duplica estas fuentes e incluye otros juegos. No se incluyen imagenes ni fuentes binarias.

INDICE (ruta | bytes | SHA-256)

MODULOS Y BASE ESPECIFICOS DE MM2
src/games/MM2/Blatant/KnifeAura.lua | 25033 | 7b04d6f1f80290d34525e4760eda01ebcd235bb75ebd975ae72c07a6f758833d
src/games/MM2/Blatant/Shoot.lua | 65902 | 5d530ae30a8aabc080a74ef9670b0e208887e9f77df710579ce574df6a52ff83
src/games/MM2/Combat/AutoShoot.lua | 25268 | 825bd53f9edf160c126114308abc9fd163868f7cbbae50007836447903 1c6614
src/games/MM2/Combat/KnifeThrow.lua | 46831 | 5453031b938da81fda3b6e9588f56ed2800314def7164ea84dd4fc50f00fea33
src/games/MM2/Farm/BlurtRoles.lua | 8577 | 2a0ccd3acf9780fa80ef5aa66e53ab29af87209fb39b4c394e94a67c29659ac2
src/games/MM2/Movement/Sprint.lua | 3751 | dacab5d4ae8ce8bb9ad0cedc5f3cc314226deb413acef59e42678796da05a97a
src/games/MM2/Movement/Teleport.lua | 12791 | ab0822689fd048c5ab959da3125547af4143fc650abc568a3a03ababefcd4a64
src/games/MM2/Other/AutoFarm.lua | 101348 | 96591f1e0b75072e900269bde738a04b445ce0c805a013940b22f50ba3b9ea21
src/games/MM2/Other/AutoUnbox.lua | 35210 | 969a05022e9356d62dfda946fe351179d77b140188c285e471ad2af46ca79951f69
src/games/MM2/Other/AutoVote.lua | 30362 | 12f1888888789a081474bb1c07ecbf4610a80339d81b5d621899927fea5d1c3
src/games/MM2/Other/AutoWin.lua | 43386 | 993a4267ce795d71b359bd244942c4b5dcd1e859189e2de4d9a197b24cc7af2
src/games/MM2/Other/CoinsAura.lua | 9987 | ae8b33f33b926e97cc489ab99061615b1935ef9c2818cece8fa4af3e30785 1ea
src/games/MM2/Other/Fling.lua | 24933 | 130ee339db8ff868ebd9e5d1ff31293215631155007cc04a05241a9c8486adec
src/games/MM2/Other/FlingEngine.lua | 33537 | 037ae31ea01b0176275903a9b77479d3ad49f23eb5af0310b4b9ca61152a4809
src/games/MM2/Other/GetAllEmotes.lua | 19243 | 1c8459923 6d13c1a5982f65eb7c120f8d0f70f1647ca8e97c009af726ba9b55a
src/games/MM2/Other/GunAura.lua | 15709 | 9a1cb6496be4c1aee5c0d06e556ec8bd5de1905107a1bb92ae06234c346c7b06
src/games/MM2/Other/Notifications.lua | 16850 | 66ac26d264e3748a8f9b179d06ccbb23a343b1939bc9465d485a6da0a92d7a4fd
src/games/MM2/Other/Prediction.lua | 92554 | b21b33161350ca5a95f77f8099bd6665b3d3491b8de72ef3e087a2aea62ac675
src/games/MM2/Render/CoinEsp.lua | 27986 | bcf5b5cb5cca918571d61e11871abb44f52364047e0a70c09c8962bfdf43115d
src/games/MM2/Render/RoundEsp.lua | 5416 | de0bbc771ad6a19723c7a67bd8fe77cb5047fd3342e61004fe1de592d924255f
src/games/MM2/base.lua | 234472 | d0623cc3f6273fdb104c51a51f7ede78b1004a01f686ef48dc5e9812010764c6

MODULOS UNIVERSALES COMPARTIDOS (INCLUIDO WALK FLING Y RENDER)
src/games/universal/Blatant/Fly.lua | 23984 | 8aa5c6062290c6e03b62d2ae4e86a7e3da86f21d4d59d21b0464542af694c335
src/games/universal/Blatant/Invisible.lua | 20702 | fe8818b1c34bae47e03b2b3585b213b4f92a741e2e52d8b3fe6685160b1c41c13
src/games/universal/Blatant/Noclip.lua | 4633 | ef4fb44f8285063ca9c266a5740c92708863558530f52ed745799f3cd2c1f946
src/games/universal/Blatant/Spider.lua | 9719 | 26495dd3407e6c4ff1cede04a90e99fdb6461f82da2df3ac11606a1101b5b29c
src/games/universal/Blatant/Swim.lua | 2995 | b51c636f1c570e5b93445ebcc311e909937 6dcd16281bb952bb9623c95e4233b7b
src/games/universal/Blatant/WalkFling.lua | 10075 | a01eaa2b246a8d6c9207db12feb6b428c40d97c2c4788eec49099dd5e0cb863b
src/games/universal/Combat/AimAssist.lua | 24909 | c6e9ba510d87149896 7d09b12a80ca873c9c7f9e51ea8602876fba7eb677051f
src/games/universal/Combat/KillAura.lua | 24179 | d38fae83d9980676f5785bb7c1c3056fe7ee7eb762309ac5363db18890ed1ceac
src/games/universal/Movement/AirWalk.lua | 11556 | 6e4700faca8a8621fde6991d05bf6622cb668dbfd3d9bb69b03d85eccdc80078
src/games/universal/Movement/AntiFling.lua | 19674 | 753b44cfab2e8126b040b7c5e2029b5116f4b37fb99bfdf26e504f33066fa64a
src/games/universal/Movement/ClickTeleport.lua | 16617 | e9a8bbf27a75ffabc3f104b70abaf32e4068d03e89595161affbde5627ca44cd
src/games/universal/Movement/FreezeMovements.lua | 36772 | e82074ef4ae84eaaba146847aa5fc82a65c5754fbcccd0f3a2c387bf52b597d90
src/games/universal/Movement/Gravity.lua | 2408 | 8e8a35dff79bdc0d4e3b9637a8b5f718c2c260520428c826b6a5f636808d40e8
src/games/universal/Movement/InfiniteJump.lua | 13307 | e4f3dd795fd5ee8edb4845ade1e3dc9a2d5cd9324153f065051eeaa82d7d9dd3
src/games/universal/Movement/Jump.lua | 10461 | 799214c554bd57659581 3c8f83324bc343f938a7b6d08c35b54cbf32c61134le
src/games/universal/Movement/Speed.lua | 15541 | b9a8e392bb1cb658380e21e40261f1066231 70d348582edb6376d8795c0a9174
src/games/universal/Other/Animations.lua | 15457 | 2c07c492140aae10ee8086d57715083c5881a8b5ed237b56c5153017079176a8
src/games/universal/Other/AntiAfk.lua | 4267 | 124e22e8ba35f93e8304a09e6821e45c554cf54bf554b9cc8d862c37e7ee8549
src/games/universal/Other/Disguise.lua | 111189 | cdc6dd41a411b5737a4c8d0b804f0bde06f860103f38629b552c3b323bae9c47e
src/games/universal/Other/ImproveFps.lua | 16021 | 6470e2880 91e2317b9d3c4def94c289f143d4be10d8b724fdb1d52ff795a269f
src/games/universal/Other/ProximityPrompts.lua | 11704 | b0d76ade20c6dd354dc88b7af780396cadf977bb9e2b3ade3045916ec3ce6cc3
src/games/universal/Other/RejoinServer.lua | 3662 | db85376ec39513a59d62d44afbc409d733c045 47a2aae0b28bdbfda7b0aa77978
src/games/universal/Other/ServerSettings.lua | 24791 | 81fdca3ac5254c77abef1337c1eef7617c540a3c2453164a46217515977fca68
src/games/universal/Render/Arrows.lua | 7408 | 4ff8abe51053a9ab806b176964f441c2fda3de2c14cca c33b119f004d5603954
src/games/universal/Render/Chams.lua | 15446 | 56f302a9217eb3070da8b052c65fdcb0b1e51fe098a1 10c27a82adfbe3ae0df
src/games/universal/Render/FieldOfView.lua | 2150 | e4eeedfe8d0ff5ae735fe271e94bbcc50c5ea424c7399f557079b52dd27908d1
src/games/universal/Render/FreeCam.lua | 14561 | 98fcd894760cf198d3023d04349a25316e13abeddfb9db0d726182b22122a1ed
src/games/universal/Render/Fullbright.lua | 3121 | 2c7c34041dd7cd2332937 3d74916d276f1eefe50d00976e41e651b72bf5882a2
src/games/universal/Render/HP.lua | 7073 | 70e758984b8a181c97490305111b891346bf8a77e187d6458d117afe1f32bc9d
src/games/universal/Render/ItemRender.lua | 25513 | 45372c319223131cf53f70339e1eacaf64c44c326f1d02debfe2f705e095b4bd
src/games/universal/Render/Markers.lua | 8923 | bddad496be32c5c824b37acc48ff9922c154ca445329c97ef36205774f6d4b79
src/games/universal/Render/NPCESP.lua | 16997 | 969fed8d5fda44e2983e6e75bd4f171885185ac5b3d16a374edf8d5217095a0
src/games/universal/Render/Nametags.lua | 9604 | 5c1be3ff7e095e69e962a004fdf4c6bfcb19280bab85ef6c369d883615b607be
src/games/universal/Render/PlayerESP.lua | 12283 | 63675fa71f9f6938992b5f243a2c071352392bd132d562100e07a2ae26f7e1d3
src/games/universal/Render/Spectate.lua | 5678 | bf6b900d4e83bd78590b9b4390df93207 6b11b56ce6e7c02c4 2b36bf81ead978b
src/games/universal/Render/XRay.lua | 6133 | 5ffebca7dd5f986e6ed926e191fccca b39fb6ca3dfa7ef9c0afb63b18363f7d2
src/games/universal/Render/ZoomUnlocker.lua | 2816 | bcf1f03ee59dc94a2299ae09255384c4ec229dac099f68f731cb7d82f880df6f

BIBLIOTECAS Y MANIFIESTO COMPARTIDOS
src/libraries/CameraAccess.lua | 4648 | 04bd4b23ccec081abe895fe4a82c50c9ceeccfa7d58f8de53784c151121c47f8
src/libraries/Entity.lua | 27268 | 045b6168d2bd2813590de86d78140680da3ab1bbbea91013883817a2e172df3
src/libraries/Framework.lua | 31444 | a070591a4691141f6c542bb0b13f738c1f077 48e14933cd55b8a570a97a3da37ac
src/libraries/GameRoles.lua | 5327 | 3782a7a178df0f90f7d5c0d8f0437727cfdd1392680286a3e92eb7928ae1bb50
src/libraries/Manifest.lua | 11454 | 508e74b29950d975ce6df47a1f1057a8b100d34d48c5490cbeae5596ae683526
src/libraries/Targeting.lua | 8883 | 17822b94fd697c7b34db207457551857 6b08d2e9189858b573e12059cc969e84
src/libraries/Weapons.lua | 15670 | 3a4d59ac97449237c66eda8a1817654bdb08d9878634235845 43d3f6cff8558d

CODIGO COMPARTIDO DE INTERFAZ
src/guis/Wurst/Code/Cards.lua | 41045 | 1ee0beb170c91c3bd9b1fce9ea230925604689 1ccd672b062c60938beaf0c1ed
src/guis/Wurst/Code/ClickGui.lua | 15772 | 9c2a059981f2eec49e024961cd3e2fb31c55d33e1ea2d727a0a2ffe6a8757c9e
src/guis/Wurst/Code/FloatingWindows.lua | 72095 | 2718cd782a511ba880ed95585de2ff739a8369b00a86341d54506c4c1cfc8db5
src/guis/Wurst/Code/Furniture.lua | 61199 | 08a6ef60b88136f102275d280afd4063681ab63ac8c6eda7e9e1f597a2b970a6
src/guis/Wurst/Code/MobileActions.lua | 33408 | 72a2f383228123f3adbc59ab55cf9c968f5742b3107de3074d144d2734b3d3d1
src/guis/Wurst/Code/SettingsPage.lua | 84679 | c53b97fcc702b50b08be87cea314f11d1f6a26440749bf1fd45df57e8dac461a
src/guis/Wurst/Code/Widgets.lua | 113668 | b04480d8bebcd89e4f66945b25a0a8fa1cbf346e048791 8b1eea66313a3221fd0
src/guis/Wurst/Code/WindowManager.lua | 51323 | 35649a5cc4c58a9f7398429cc5f1ad352ff6a73b7f0dc60d7defc93f26cfba5c

CARGADOR Y RUNTIME
src/Wurst.lua | 259370 | cd6b97cb48bab9218acba3990d65b36cd2f782c06ec7136e27b5ca23bf5e1d7f

==
MODULOS Y BASE ESPECIFICOS DE MM2
==

ARCHIVO: src/games/MM2/Blatant/KnifeAura.lua
BYTES: 25033
SHA-256: 7b04d6f1f80290d34525e4760eda01ebcd235bb75ebd975ae72c07a6f758833d
INICIO DEL ARCHIVO

```
local Module = {
    Name = "MM2 Knife Aura",
    Events = {} :: {[string]: any},
    Initialized = false,
    Runtime = nil :: any,
}

local activeCleanup: () -> () = function() end

function Module.init(runtime: any): any
    if Module.Initialized then
        return Module
    end
    local core: any = state.mm2Core
    assert(type(core) == "table", "MM2 Knife Aura requires the MM2 core module")
    Module.Runtime = runtime
    local getPlayerRole: any = core.getPlayerRole
    local getPlayerWeapon: any = core.getPlayerWeapon
    local isPlayerAlive: any = core.isPlayerAlive
    local isProtectedTarget: any = core.isProtectedTarget
    local trajectoryCalibration: any = core.trajectoryCalibration

    type KnifeRuntimeState = {
        masterEnabled: boolean,
        auraWanted: boolean,
        throwWanted: boolean,
        lastAuraSwing: number,
    }

    local knifeSettings: any = {
        aura = false,
        auraRange = 14,
        -- v191 "Hold Mouse": the automatic swing loop is off and the knife
        -- strikes when the player clicks (or taps), at the SAME reach the aura
        -- uses.
        holdMouse = false,
    }
    state.mm2KnifeRuntime = {
        masterEnabled = false,
        auraWanted = false,
        throwWanted = false,
        lastAuraSwing = -math.huge,
    } :: KnifeRuntimeState

    local function getEffectiveRange(): number
        return knifeSettings.auraRange or 14
    end

    local function getEquippedWeapon(name: string, _tag: string): Tool?
        return getPlayerWeapon(LocalPlayer, name, true)
    end

    local function canUseEquippedKnife(requireReady: boolean?): (Tool?, BasePart?)
        local character: Model? = LocalPlayer.Character
        local humanoid: Humanoid? = character
            and character:FindFirstChildOfClass("Humanoid")
            :: Humanoid?
        local root: BasePart? = character
            and character:FindFirstChild("HumanoidRootPart")
            :: BasePart?
        local knife: Tool? = getEquippedWeapon("Knife", "Weapon_Knife")
        if not character
            or not humanoid
            or humanoid.Health <= 0
            or not root
            or not knife
            or (requireReady == true and knife.Enabled == false)
            or knife:GetAttribute("Disabled") == true
            or getPlayerRole(LocalPlayer) ~= "Murderer" then
            return nil, nil
        end
        return knife, root
    end

    local function isValidKnifeTarget(player: Player): boolean
        if player == LocalPlayer
            or isProtectedTarget(player)
            or not isPlayerAlive(player)
            or getPlayerRole(player) == "Murderer" then
            return false
        end
        return player.Character ~= nil
            and player.Character:FindFirstChild("HumanoidRootPart") ~= nil
    end

    local function findKnifeAuraTarget(
        localRoot: BasePart,
        maxDistance: number
    ): Player?
        local closest: Player? = nil
        local closestDistance: number = maxDistance
        for _, player: Player in ipairs(Players:GetPlayers()) do
            if isValidKnifeTarget(player) then
                local targetRoot: BasePart? = player.Character
                    and player.Character:FindFirstChild("HumanoidRootPart")
                    :: BasePart?
                if targetRoot then
                    local distance: number =
                        (targetRoot.Position - localRoot.Position).Magnitude
                    if distance <= closestDistance then
                        closest = player
                        closestDistance = distance
                    end
                end
            end
        end
        return closest
    end

    local knifeControllerGeneration: number = 0
    local function processKnifeAura(manual: boolean?): ()
        -- In Hold Mouse mode nothing swings on its own: only the click path
        -- calls this with `manual`. The manual swing is throttled by the knife's
        -- own cooldown instead of the slower aura interval, so every click that
        -- the tool accepts actually lands.
        if knifeSettings.holdMouse and manual ~= true then return end
        local swingInterval: number = if manual == true then 0.3 else 0.86
        if os.clock() - state.mm2KnifeRuntime.lastAuraSwing < swingInterval then return end
        local knife: Tool?, localRoot: BasePart? = canUseEquippedKnife(true)
        if not knife or not localRoot then
            return
        end

        local currentRange: number = getEffectiveRange()
        local target: Player? =
            findKnifeAuraTarget(localRoot, currentRange)
        local targetCharacter: Model? = target and target.Character
        local targetRoot: BasePart? = targetCharacter
            and targetCharacter:FindFirstChild("HumanoidRootPart")
            :: BasePart?
        if not target
            or not targetCharacter
            or not targetRoot
            or (targetRoot.Position - localRoot.Position).Magnitude
                > currentRange
            or os.clock() - state.mm2KnifeRuntime.lastAuraSwing < swingInterval then
            return
        end

        local handle: BasePart? = knife:FindFirstChild("Handle") :: BasePart?
        if not handle or not handle:IsA("BasePart") then
            return
        end

        local events: Instance? = knife:FindFirstChild("Events")
        local handleTouched: Instance? = events
            and events:FindFirstChild("HandleTouched")
        local activated: boolean = pcall(function(): () knife:Activate() end)
        if not activated then return end
        state.mm2KnifeRuntime.lastAuraSwing = os.clock()
        local swingGeneration: number = knifeControllerGeneration
        task.delay(0.025, function(): ()
            -- The swing itself may set Tool.Enabled=false for its cooldown;
            -- readiness matters before Activate, not after it has succeeded.
            local currentKnife: Tool?, currentLocalRoot: BasePart? =
                canUseEquippedKnife(false)
            local currentCharacter: Model? = target.Character
            local currentRoot: BasePart? = currentCharacter
                and currentCharacter:FindFirstChild("HumanoidRootPart")
                :: BasePart?
            local currentHandle: BasePart? =
                knife:FindFirstChild("Handle") :: BasePart?
            if not knifeSettings.aura or not state.mm2KnifeRuntime.masterEnabled
                or swingGeneration ~= knifeControllerGeneration
                or currentKnife ~= knife
                or not currentLocalRoot
                or currentCharacter ~= targetCharacter
                or not isValidKnifeTarget(target)
                or not currentRoot
                or not currentHandle
                or not currentHandle:IsA("BasePart")
                or (currentRoot.Position - currentLocalRoot.Position).Magnitude
                    > getEffectiveRange() then
                return
            end

            local sent: boolean = false
            if handleTouched and handleTouched:IsA("RemoteEvent") then
                sent = pcall(function(): () (handleTouched :: RemoteEvent):FireServer(currentRoot) end)
            end
            if not sent and type(state.fireTouchInterest) == "function" then
                pcall(state.fireTouchInterest, currentHandle, currentRoot, 0)
                pcall(state.fireTouchInterest, currentHandle, currentRoot, 1)
            end
            pcall(trajectoryCalibration.noteAuthoredShot, trajectoryCalibration, {
                kind = "stab",
                toolName = "Knife",
                originPos = currentLocalRoot.Position,
                aimPos = currentRoot.Position,
                target = target,
            })
        end)
    end

    -- The click/tap path for Hold Mouse. It is a real input listener (not a
    -- tool hook) because the whole point is to reach targets outside the
    -- handle's natural touch range: the player clicks, this picks the closest
    -- living target inside "Aura Range" and sends the same HandleTouched contact
    -- the automatic aura sends.
    local holdConnection: RBXScriptConnection? = nil
    local function stopHoldMouse(): ()
        if holdConnection then
            pcall(function(): ()
                (holdConnection :: RBXScriptConnection):Disconnect()
            end)
            holdConnection = nil
        end
    end
    local function refreshHoldMouse(): ()
        stopHoldMouse()
        if not knifeSettings.holdMouse or not knifeSettings.aura
            or not state.mm2KnifeRuntime.masterEnabled then
            return
        end
        local inputService: any = UserInputService
        if not inputService or not inputService.InputBegan then
            return
        end
        holdConnection = inputService.InputBegan:Connect(function(
            input: any,
            gameProcessed: boolean
        ): ()
            if gameProcessed then
                return
            end
            local kind: any = input.UserInputType
            local clicked: boolean = kind == Enum.UserInputType.MouseButton1
                or kind == Enum.UserInputType.Touch
            if not clicked and not touched then
                return
            end
            if inputService:GetFocusedTextBox() ~= nil then
                return
            end
            local menu: any = services and services.menu or nil
            if menu and type(menu.isCapturingInput) == "function" then
                local ok: boolean, capturing: any = pcall(menu.isCapturingInput)
                if ok and capturing == true then
                    return
                end
            end
            processKnifeAura(true)
        end)
    end

    local function knifeRefreshKnifeController(): ()
        knifeControllerGeneration += 1
        disconnectFeatureConnection("MM2KnifeVirtualHitbox")
        disconnectFeatureConnection("MM2KnifeAura")
        disconnectFeatureConnection("MM2KnifeController")
        refreshHoldMouse()
        if not knifeSettings.aura or not state.mm2KnifeRuntime.masterEnabled then
            return
        end

        local elapsed: number = 0
        featureConnections.MM2KnifeController =
            TaskManager:Connect(function(deltaTime: number): ()
                elapsed += math.max(0, deltaTime)
                local interval: number = 1 / 20
                if elapsed + 1e-9 < interval then return end
                elapsed = elapsed % interval
                processKnifeAura()
            end, nil, "MM2 Knife Aura")
    end

    local function toggleKnifeAura(enabled: boolean): ()
        knifeSettings.aura = enabled
        state.mm2KnifeRuntime.auraWanted = enabled
        state.mm2KnifeRuntime.lastAuraSwing = -math.huge
        refreshKnifeController()
    end

    local killCharacterByPlayer: {[Player]: Model} = {}
    local killPlayerOption: any = nil

    local function isLivingInnocentCandidate(player: Player): boolean
        if player == LocalPlayer
            or isProtectedTarget(player)
            or not isPlayerAlive(player) then
            return false
        end
        local character: Model? = player.Character
        if not character or not killCharacterByPlayer[player] == character then
            return false
        end
        local humanoid: Humanoid? = character:FindFirstChildOfClass("Humanoid")
        local root: BasePart? = character:FindFirstChild("HumanoidRootPart")
        if not humanoid or humanoid.Health <= 0 or not root then
            return false
        end
        local role: any = getPlayerRole(player)
        if role == "Murderer" or role == "Dead" or role == "None" then
            return false
        end
        return true
    end

    local function getLivingInnocentNames(): {string}
        local names: {string} = {}
        for _, player: Player in ipairs(Players:GetPlayers()) do
            if isLivingInnocentCandidate(player) then
                table.insert(names, player.Name)
            end
        end
        table.sort(names)
        return names
    end

    local function equipKnifeForKill(): (Model?, Humanoid?, BasePart?, Tool?)
        if getPlayerRole(LocalPlayer) ~= "Murderer" then
            if type(notify) == "function" then
                notify("Kill Aura: Only the Murderer can use Kill All / Kill Player.")
            end
            return nil, nil, nil, nil
        end
        local character: Model? = LocalPlayer.Character
        local humanoid: Humanoid? = character
            and character:FindFirstChildOfClass("Humanoid") :: Humanoid?
        local myRoot: BasePart? = character
            and character:FindFirstChild("HumanoidRootPart") :: BasePart?
        if not character or not humanoid or humanoid.Health <= 0 or not myRoot then
            return nil, nil, nil, nil
        end
        local knife: Tool? = getPlayerWeapon(LocalPlayer, "Knife", false)
        if not knife then
            if type(notify) == "function" then
                notify("Kill Aura: No Knife found.")
            end
            return nil, nil, nil, nil
        end
        if knife.Parent ~= character then
            pcall(function(): ()
                humanoid:EquipTool(knife :: Tool)
            end)
            local deadline: number = os.clock() + 0.3
            while knife.Parent ~= character and os.clock() < deadline do
                task.wait()
            end
        end
        if knife.Parent ~= character then
            return nil, nil, nil, nil
        end
        return character, humanoid, myRoot, knife
    end

    local function executeKnifeKillOnTargets(targets: {Player}): ()
        if #targets == 0 then
            return
        end
        local _, _, myRoot: BasePart?, knife: Tool? = equipKnifeForKill()
        if not myRoot or not knife then
            return
        end
        local handle: BasePart? = knife:FindFirstChild("Handle") :: BasePart?
        local victimRoots: {BasePart} = {}
        for _, target: Player in ipairs(targets) do
            if isLivingInnocentCandidate(target) then
                local targetChar: Model? = target.Character
                local victimRoot: BasePart? = targetChar
                    and (targetChar:FindFirstChild("HumanoidRootPart") :: BasePart?)
                if targetChar and victimRoot then
                    killCharacterByPlayer[target] = targetChar
                    table.insert(victimRoots, victimRoot)
                    pcall(function(): ()
                        victimRoot.Anchored = true
                        victimRoot.CFrame = myRoot.CFrame + myRoot.CFrame.LookVector * 1
                    end)
                end
            end
        end
        if killPlayerOption and type(killPlayerOption.Refresh) == "function" then
            pcall(function(): ()
                killPlayerOption:Refresh()
            end)
        end
        if #victimRoots == 0 then
            return
        end
        task.wait(0.1)
        local stab: Instance? = knife:FindFirstChild("Stab")
        if stab and stab:IsA("RemoteEvent") then
            pcall(function(): ()
                (stab :: RemoteEvent):FireServer("Slash")
            end)
        end
        local events: Instance? = knife:FindFirstChild("Events")
        local handleTouched: Instance? = events
            and events:FindFirstChild("HandleTouched")
        pcall(function(): ()
            knife:Activate()
        end)
        local fireTouch: any = state.fireTouchInterest
        for _, victimRoot: BasePart in ipairs(victimRoots) do
            if handleTouched and handleTouched:IsA("RemoteEvent") then
                pcall(function(): ()
                    (handleTouched :: RemoteEvent):FireServer(victimRoot)
                end)
            end
            if handle and handle:IsA("BasePart") and type(fireTouch) == "function" then
                pcall(fireTouch, handle, victimRoot, 0)
                pcall(fireTouch, handle, victimRoot, 1)
            end
            task.wait(0.04)
        end
        task.delay(0.45, function(): ()
            for _, victimRoot: BasePart in ipairs(victimRoots) do
                pcall(function(): ()
                    victimRoot.Anchored = false
                end)
            end
        end)
    end

    local function killAllPlayersAction(): ()
        task.spawn(function(): ()
            local targets: {Player} = {}
            for _, player: Player in ipairs(Players:GetPlayers()) do
                if isLivingInnocentCandidate(player) then
                    table.insert(targets, player)
                end
            end
            executeKnifeKillOnTargets(targets)
        end)
    end

    local aura: any = state.framework and state.framework:GetModule("KillAura")
    assert(aura, "MM2 Knife requires the existing KillAura module")
    local wasEnabled: boolean = aura.Enabled
    local migratedShortcut: any = nil
    if type(configData) == "table" and not configData.values["MM2.Knife.MigratedToKillAura"] then
        for _, label: string in ipairs({"KnifeAura"}) do
            local old: any = configData.states["MM2.Knife." .. label]
            local key: string = "Universal.KillAura." .. label
            if configData.states[key] == nil and old ~= nil then configData.states[key] = old end
        end
        for _, pair in ipairs({
            {"MM2.Knife.AuraRange", "Universal.KillAura.AuraRange"},
            {"Shortcut.MM2Knife", "Shortcut.UniversalKillAura"},
        }) do
            if configData.values[pair[2]] == nil then
                configData.values[pair[2]] = configData.values[pair[1]]
                if pair[2] == "Shortcut.UniversalKillAura" then
                    migratedShortcut = configData.values[pair[2]]
                end
            end
        end
        if configData.states["Universal.KillAura"] == nil then
            local previous = configData.states["MM2.Knife"]
            configData.states["Universal.KillAura"] = previous
            wasEnabled = wasEnabled or previous == true
        end
        configData.values["MM2.Knife.MigratedToKillAura"] = true
        if type(queueConfigSave) == "function" then queueConfigSave() end
    end
    -- The universal card and its shortcut already exist by the time MM2 is
    -- identified. Apply a migrated key now, rather than on the next boot.
    local shortcut: any = state.shortcutBindings and state.shortcutBindings["Universal.KillAura"]
    if shortcut and type(migratedShortcut) == "string" then
        local ok: boolean, key: any = pcall(function(): any return (Enum.KeyCode :: any)[migratedShortcut] end)
        if ok and key then
            shortcut.key = key
            if type(shortcut.refresh) == "function" then shortcut.refresh() end
        end
    end
    if wasEnabled then aura:Toggle(false) end
    local KnifeFeature: any = aura.Feature
    local originalDescription: string = KnifeFeature.description
    KnifeFeature.description = "MM2 melee combat: Knife Aura stabs with your held knife (or click with Hold Mouse). Includes Kill All and Kill Player."
    KnifeFeature.tooltipText = KnifeFeature.description
    if type(KnifeFeature.SetName) == "function" then
        KnifeFeature:SetName("Knife Stab")
    end
    aura.CombatAdapter = function(enabled: boolean): ()
        state.mm2KnifeRuntime.masterEnabled = enabled
        refreshKnifeController()
        aura:SetStatus(enabled and "Knife" or nil)
    end
    -- Generic melee controls do not describe the MM2 remote-based controller.
    aura.OptionFilter = function(name: string): boolean
        return name == "Knife Aura"
            or name == "Aura Range"
            or name == "Hold Mouse"
            or name == "Kill All"
            or name == "Kill Player"
    end
    aura:RefreshVisibility()
    state.mm2KnifeAuraModule = aura
    state.mm2KnifeFeature = KnifeFeature

    aura:CreateToggle({Name = "Knife Aura", Default = true, Function = toggleKnifeAura,
        Tooltip = "MM2 stabs with your equipped knife while Knife Stab is ON."})
    aura:CreateSlider({Name = "Aura Range", Min = 7, Max = 30, Default = 14,
        Show = {Option = "Knife Aura", Values = {true}},
        Function = function(value: number): () knifeSettings.auraRange = value end})
    -- v191: the automatic swing loop can be replaced by the player's own click
    -- / tap. The reach is the same one "Aura Range" sets, so a click lands a
    -- stab on anything the aura could have reached on its own.
    aura:CreateToggle({
        Name = "Hold Mouse",
        Default = false,
        Show = {Option = "Knife Aura", Values = {true}},
        Tooltip = "No automatic stabs: the knife strikes where you click (or tap), "
            .. "with the same reach as Aura Range.",
        Function = function(value: boolean): ()
            knifeSettings.holdMouse = value
            state.mm2KnifeRuntime.lastAuraSwing = -math.huge
            refreshKnifeController()
        end,
    })
    aura:CreateButton({
        Name = "Kill All",
        Function = killAllPlayersAction,
        Tooltip = "As Murderer, equip your Knife and eliminate all living Innocent/Sheriff players.",
    })
    local killPlayerReady: boolean = false
    local suppressKillPlayerCallback: boolean = false
    killPlayerOption = aura:CreateList({
        Name = "Kill Player",
        Items = getLivingInnocentNames,
        Single = true,
        EmptyText = "No living innocents",
        Tooltip = "Click to pick a living Innocent player to kill. Dead or eliminated players are removed from the list.",
        Function = function(_selected: any, names: {string}): ()
            if not killPlayerReady or suppressKillPlayerCallback then
                return
            end
            local chosenName: string? = names and names[1]
            if not chosenName or chosenName == "" then
                return
            end
            local targetPlayer: Player? = nil
            for _, player: Player in ipairs(Players:GetPlayers()) do
                if player.Name == chosenName and isLivingInnocentCandidate(player) then
                    targetPlayer = player
                    break
                end
            end
            suppressKillPlayerCallback = true
            if killPlayerOption and type(killPlayerOption.Set) == "function" then
                pcall(function(): ()
                    killPlayerOption:Set(chosenName, false)
                end)
            end
            suppressKillPlayerCallback = false
            if targetPlayer then
                local victim: Player = targetPlayer :: Player
                task.spawn(function(): ()
                    executeKnifeKillOnTargets({victim})
                end)
            end
        end,
    })
    if killPlayerOption and type(killPlayerOption.Selected) == "table" and #killPlayerOption.Selected > 0 then
        suppressKillPlayerCallback = true
        for _, staleName: string in ipairs(table.clone(killPlayerOption.Selected)) do
            pcall(function(): ()
                killPlayerOption:Set(staleName, false)
            end)
```

```
        end
        suppressKillPlayerCallback = false
    end
    killPlayerReady = true
    if wasEnabled then
        if not KnifeFeature.enabled and type(KnifeFeature.activate) == "function" then
            KnifeFeature.activate()
        else
            aura:Toggle(true)
        end
    end

    activeCleanup = function(): ()
        stopHoldMouse()
        if aura.Enabled then aura:Toggle(false) end
        if KnifeFeature.enabled then
            KnifeFeature.enabled = false
            refreshFeatureToggle(KnifeFeature)
        end
        toggleKnifeAura(false)
        table.clear(killedCharacterByPlayer)
        state.mm2KnifeRuntime.masterEnabled = false
        state.mm2KnifeFeature = nil
        state.mm2KnifeAuraModule = nil
        aura.CombatAdapter = nil
        KnifeFeature.description = originalDescription
        KnifeFeature.tooltipText = originalDescription
        if type(KnifeFeature.SetName) == "function" then
            KnifeFeature:SetName("KillAura")
        end
        -- If the adapter is detached before the whole UI is destroyed, do
        -- not expose stale MM2 actions beside the generic controls.
        aura.OptionFilter = function(name: string): boolean
            return name ~= "Knife Aura"
                and name ~= "Aura Range"
                and name ~= "Hold Mouse"
                and name ~= "Kill All"
                and name ~= "Kill Player"
        end
        aura:RefreshVisibility()
    end
    Module.Events = featureConnections
    Module.Initialized = true
    return Module
end

function Module.destroy(): ()
    if not Module.Initialized then
        return
    end
    Module.Initialized = false
    pcall(activeCleanup)
    activeCleanup = function(): () end
    Module.Events = {}
    Module.Runtime = nil
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/MM2/Blatant/KnifeAura.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/games/MM2/Blatant/Shoot.lua
BYTES: 65902
SHA-256: 5d530ae30a8aabc080a74ef96780de208887e9f77df710579ce574df6a52ff83
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
local Module = {
    Name = "MM2 Shoot",
    Events = {} :: {[string]: any},
    Initialized = false,
    Runtime = nil :: any,
}

local activeCleanup: () -> () = function(): () end

function Module.init(runtime: any): any
    if Module.Initialized then
        return Module
    end
    local core: any = state.mm2Core
    assert(type(core) == "table", "MM2 Shoot requires the MM2 core module")
    local prediction: any = state.mm2Prediction
    assert(type(prediction) == "table", "MM2 Shoot requires MM2 Prediction")
    local trajectoryCalibration: any = core.trajectoryCalibration
    Module.Runtime = runtime
    local connectGunFiredSignal: any = core.connectGunFiredSignal
    local disconnectGunFiredObserver: any = core.disconnectGunFiredObserver
    local findMurderer: any = core.findMurderer
    local getGunOriginCFrame: any = core.getGunOriginCFrame
    local getPlayerRole: any = core.getPlayerRole
    local getPlayerWeapon: any = core.getPlayerWeapon
    local getWeaponServiceModule: any = core.getWeaponServiceModule
    local isPlayerAlive: any = core.isPlayerAlive
    local isProtectedTarget: any = core.isProtectedTarget
    local isWeaponPathClear: any = core.isWeaponPathClear
    local describeRayHit: any = core.describeRayHit
    local mm2Settings: any = core.mm2Settings
    local validateGunShot: any = core.validateGunShot

    local shootMacro: boolean = false
    -- First thing the untouched ray meets on an allowed shot (nil if nothing
    -- short of the aim). Persisted into the shot's settings.
    local lastGunOccluder: any = nil
    local function macroUnequip(gun: Tool?): ()
        if not shootMacro or not gun then
            return
        end
        task.delay(0.01, function(): ()
            pcall(function(): ()
                local backpack: Instance? = LocalPlayer:FindFirstChildOfClass("Backpack")
                local character: Model? = LocalPlayer.Character
                if backpack and character and gun.Parent == character then
                    gun.Parent = backpack
                end
            end)
        end)
    end

    local function getSelectedShootTarget(): Player?
        local target
        if mm2Settings.shootMode == "Custom"
            and mm2Settings.shootTarget ~= "" then
            target = findPlayerByText(mm2Settings.shootTarget)
        else
            target = findMurderer()
        end
        if not target
            or target == LocalPlayer
            or not isPlayerAlive(target)
            or isProtectedTarget(target) then
            return nil
        end
        if getPlayerRole(target) ~= "Murderer" then
            return nil
        end
        return target
    end

    local SILENT_ORIGIN_STUDS: number = 2.5
    local SILENT_PIERCE_STUDS: number = 1.0
    local SILENT_LIFT_STUDS: number = 1.6
    local MAX_SILENT_SWEEP: number = 1.5
    -- Inside this distance the murderer's body covers the whole line of sight,
    -- so the lead stops helping and can even invert the shot: 17 studs/s of
    -- charge times a 0.1 s lead is 1.7 studs, more than the gap itself.
    local POINT_BLANK_STUDS: number = 6

    -- The Top/fallback origin lift belongs to the target's rig, not to a
    -- universal constant: R6 and R15 put the head at different heights
    -- above the root, and an origin spawned inside the head can be
    -- ignored by the shot. Predicted endpoint minus predicted root is
    -- the same aim offset the predictor sampled from the live rig.
    local function silentLift(info: any, predicted: Vector3): number
        if type(info) == "table"
            and typeof(info.rootPosition) == "Vector3" then
            local aimOffsetY: number = predicted.Y - info.rootPosition.Y
            if aimOffsetY > 0.5 then
                return math.clamp(aimOffsetY + 0.35, 1.2, 2.4)
            end
        end
        return SILENT_LIFT_STUDS
    end

    -- The bypass segment runs along the target's own motion. A fixed extra
    -- length guesses; the confirmed shots already measure how far the aim
    -- lands from the body, so the same measurement decides how much segment
    -- the shot needs. With no confirmations yet the uncertainty is zero and
    -- the configured value is used untouched.
    local function getSilentSweepDistance(speed: number?): number
        local configured: number = tonumber(mm2Settings.silentSweep) or 0
        local sweep: number = math.clamp(configured, 0, MAX_SILENT_SWEEP)
        local uncertainty: number = 0
        if type(prediction) == "table"
            and type(prediction.getLeadUncertainty) == "function" then
            -- v161: the silent path is the CFrame mode; its measurements are
            -- the ones that size the sweep.
            local ok: boolean, seconds: any = pcall(
                prediction.getLeadUncertainty,
                "CFrame"
            )
            if ok and type(seconds) == "number" and seconds == seconds then
                uncertainty = math.clamp(seconds, 0, 0.15)
            end
        end
        if uncertainty <= 0 then
            return sweep
        end
        local travel: number = math.max(tonumber(speed) or 0, 0)
        local extra: number = uncertainty * travel
        return math.clamp(sweep + extra, 0, MAX_SILENT_SWEEP)
    end

    local function buildSilentShot(
        _target: Player,
        info: any,
        muzzleOrigin: CFrame?,
        variant: string
    ): (CFrame?, CFrame?, number, number, string)
        local predicted: Vector3 = info.endpoint
        local liftStuds: number = silentLift(info, predicted)
        local velocity: Vector3 = info.velocity
        local horizontal: Vector3 = Vector3.new(velocity.X, 0, velocity.Z)

        local direction: Vector3? = nil
        local sweep: number = 0
        if horizontal.Magnitude > 1.5 then
            direction = horizontal.Unit

            sweep = getSilentSweepDistance(horizontal.Magnitude)
        end
        local sweepStart: Vector3 =
            direction and -direction * sweep or Vector3.zero
        local sweepEnd: Vector3 =
            direction and direction * sweep or Vector3.zero

        local approach: Vector3? = nil
        local approachOffset: number = SILENT_ORIGIN_STUDS
        if muzzleOrigin then
            local toTarget: Vector3 = predicted - muzzleOrigin.Position
            local flat: Vector3 = Vector3.new(toTarget.X, 0, toTarget.Z)
            if flat.Magnitude > 0.05 then
                approach = flat.Unit
                approachOffset = math.min(
                    SILENT_ORIGIN_STUDS,
                    math.max(1.0, flat.Magnitude * 0.6)
                )
            end
        end

        local startPoint: Vector3
        local endPoint: Vector3
        local resolvedVariant: string = variant
        local pierceAxis: Vector3? = direction or approach
        if (variant == "Through" or variant == nil or variant == "") and pierceAxis then
            resolvedVariant = "Through"
            local captured: Vector3 = info.capturedPosition or predicted
            startPoint = captured - pierceAxis * 1.0 + sweepStart
            endPoint = predicted + pierceAxis * SILENT_PIERCE_STUDS + sweepEnd
        elseif variant == "Top" then
            resolvedVariant = "Top"
            startPoint = predicted + Vector3.new(0, liftStuds, 0)
                + sweepStart
            endPoint = predicted + sweepEnd
        elseif variant == "Behind" and approach then
            resolvedVariant = "Behind"
            startPoint = predicted + approach * approachOffset + sweepStart
            endPoint = predicted + sweepEnd
        elseif approach then
            resolvedVariant = "Front"
            startPoint = predicted - approach * approachOffset + sweepStart
            endPoint = predicted + sweepEnd
        elseif direction then
            resolvedVariant = "Motion"
            startPoint = predicted + sweepStart
            endPoint = predicted + sweepEnd
        else
            resolvedVariant = "Top"
            startPoint = predicted + Vector3.new(0, liftStuds, 0)
            endPoint = predicted
        end

        if (endPoint - startPoint).Magnitude <= 0.05 then
            startPoint = predicted + Vector3.new(0, liftStuds, 0)
            endPoint = predicted
            resolvedVariant = "Top"
            sweep = 0
        end
        return CFrame.lookAt(startPoint, endPoint), CFrame.new(endPoint),
            approachOffset, sweep, resolvedVariant
    end

    type FireOptions = {
        silent: boolean?,
        quiet: boolean?,
        whenReady: boolean?,
        continuous: boolean?,
        wallCheck: boolean?,
        -- Native PC hook: construct the same shot but let the original
        -- FireServer send it once instead of sending a second packet.
        redirect: ((CFrame, CFrame) -> ())?,
        nativeRedirect: boolean?,
        nativeOrigin: CFrame?,
    }

    -- Experimental policy, not a measured hit-rate optimization: prefer the
    -- ordinary muzzle path nearby/stationary and the CFrame path for a fast,
    -- distant target. It selects exactly one path per request; a refusal is
    -- not evidence that a second packet would be safe or accepted.
    local function chooseExperimentalGunMode(target: Player?, origin: Vector3?): string
        local character: Model? = target and target.Character
        local root: BasePart? = character and character:FindFirstChild("HumanoidRootPart") :: BasePart?
        if not root or not origin then return "Bullet" end
        local speed: number = 0
        pcall(function(): ()
            local velocity: Vector3 = root.AssemblyLinearVelocity
            speed = Vector3.new(velocity.X, 0, velocity.Z).Magnitude
        end)
        if (root.Position - origin).Magnitude >= 30 and speed >= 8 then return "CFrame" end
        return "Bullet"
    end

    -- How long an authored request owns the gun. The feedback queue resolves
    -- or expires a shot at 1.6-1.8 s; keeping the window inside that bound
    -- means the gate never outlives the pending state it reads.
    local SHOT_CONFIRM_WINDOW: number = 1.5

    local ShootFeature: any = nil
    local lastRejectReason: string = ""

    -- The parts an aim may land on, best first, for the ghost preference.
    local AIM_BODY_PARTS: {string} = {
        "UpperTorso", "Torso", "HumanoidRootPart", "Head",
        "RightUpperArm", "LeftUpperArm", "RightUpperLeg", "LeftUpperLeg",
        "Right Arm", "Left Arm", "Right Leg", "Left Leg",
    }

    -- v161: the aim can be perfect and still be stopped by a GHOST part --
    -- transparent AND non-colliding, which every client wall check skips. The
    -- v159 session lost exactly that shot: 0.24 studs from the torso, occluder
    -- "Workspace.Workplace.Raggy.HumanoidRootPart", transparency 1,
    -- CanCollide false, 10.3 studs in front of the aim. What the server's own
    -- weapon ray does with such a part is not known, so this never refuses a
    -- shot: it only prefers another part of the same body whose line does not
    -- cross the ghost. If every candidate crosses it, the original point stands.
    local function ghostOnLine(
        muzzle: Vector3,
        endpoint: Vector3,
        targetCharacter: Model?
    ): any
        if type(describeRayHit) ~= "function" then
            return nil
        end
        local ok: boolean, hit: any = pcall(
            describeRayHit,
            muzzle,
            endpoint,
            targetCharacter
        )
        if not ok or type(hit) ~= "table" or hit.onTarget then
            return nil
        end
        if hit.transparency ~= 1 or hit.canCollide ~= false then
            return nil
        end
        local reach: number = (endpoint - muzzle).Magnitude
        if type(hit.distance) ~= "number" or hit.distance >= reach - 0.5 then
            return nil
        end
        return hit
    end

    -- The ghost probe is a preference, not a gate, so it is allowed to be
    -- stale: what is cached is which body part was clean, never a position.
    -- v162: Auto Shoot calls the aim path every frame; probing there cost up
    -- to 13 raycasts per frame. Now it costs at most 5 every 0.15 s and the
    -- cached answer is one FindFirstChild plus the same lead arithmetic.
    local GHOST_PROBE_INTERVAL: number = 0.15
    local GHOST_PROBE_CANDIDATES: number = 4
    local ghostProbe: any = {
        target = nil,
        at = -math.huge,
        partName = nil,
    }

    -- Returns a cleaner endpoint and the part it moved to, or nil.
    local function avoidGhostAim(
        target: Player,
        info: any,
        muzzle: Vector3
    ): (Vector3?, string?)
        local character: Model? = target.Character
        if not character or not info or typeof(info.endpoint) ~= "Vector3" then
            return nil, nil
        end
        local now: number = os.clock()
        if target ~= ghostProbe.target
            or now - ghostProbe.at >= GHOST_PROBE_INTERVAL then
            ghostProbe.target = target
            ghostProbe.at = now
            ghostProbe.partName = nil
            if ghostOnLine(muzzle, info.endpoint, character) then
                local captured: Vector3? = info.capturedPosition
                local probeLead: Vector3 = captured
                    and (info.endpoint - captured)
                    or Vector3.zero
                local tried: number = 0
                for _, partName: string in ipairs(AIM_BODY_PARTS) do
                    if tried >= GHOST_PROBE_CANDIDATES then
                        break
                    end
                    local part: Instance? = character:FindFirstChild(partName)
                    if part and part:IsA("BasePart") then
                        tried += 1
                        local candidate: Vector3 = (part :: BasePart).Position
                            + probeLead
                        if not ghostOnLine(muzzle, candidate, character) then
                            ghostProbe.partName = partName
                            break
                        end
                    end
                end
            end
        end
        local partName: string? = ghostProbe.partName
        if not partName then
            return nil, nil
        end
        local part: Instance? = character:FindFirstChild(partName)
        if not part or not part:IsA("BasePart") then
            ghostProbe.partName = nil
            return nil, nil
        end
        local captured: Vector3? = info.capturedPosition
        local lead: Vector3 = captured and (info.endpoint - captured) or Vector3.zero
        return (part :: BasePart).Position + lead, partName
    end

    local function fireGunAtTarget(
        target: Player?,
        options: FireOptions?
    ): boolean
        local opts: FireOptions = options or {}
        local silent: boolean = mm2Settings.silentAim
        local experimental: boolean = opts.silent == nil and mm2Settings.experimentalAim == true
        if experimental then
            local localCharacter: Model? = LocalPlayer and LocalPlayer.Character
            local ownRoot: BasePart? = localCharacter and localCharacter:FindFirstChild("HumanoidRootPart") :: BasePart?
            silent = chooseExperimentalGunMode(target, ownRoot and ownRoot.Position) == "CFrame"
        elseif opts.silent ~= nil then
            silent = opts.silent
        end
        local quiet: boolean = opts.quiet == true
        local continuous: boolean = opts.continuous == true
        -- An intercepted native FireServer is already being sent by the game.
        -- Rechecking a Tool that the native script just put on cooldown can
        -- only suppress the redirect, not prevent an extra shot.
        local nativeRedirect: boolean = opts.nativeRedirect == true and type(opts.redirect) == "function"
        local useRaw: boolean = mm2Settings.shootRaw
            or (continuous and silent)
        local useWallCheck: boolean = if opts.wallCheck ~= nil
            then opts.wallCheck == true
            else mm2Settings.shootWallCheck
        local function reject(message: string): boolean
            lastRejectReason = message
            if not quiet and ShootFeature then ShootFeature:SetStatus(message) end
            return false
        end

        local character, humanoid = getCharacterParts()
        if not target or not target.Character or target == LocalPlayer
            or not isPlayerAlive(target) or isProtectedTarget(target) or getPlayerRole(target) ~= "Murderer" then
            return reject("No selected shoot target was found.")
        end
        if not character or not humanoid then
            return reject("Your character is not available.")
        end
        local targetCharacter = target.Character
        local gun: Tool? = getPlayerWeapon(LocalPlayer, "Gun", true)
        if not gun or gun.Parent ~= character then
            return reject("Equip the sheriff gun yourself to shoot.")
        end
        if not nativeRedirect and (gun.Enabled == false or gun:GetAttribute("Disabled") == true
            or gun:GetAttribute("Reloading") == true or gun:GetAttribute("IsReloading") == true) then
            return reject("The gun is disabled or reloading.")
        end

        local remote: Instance? = gun and gun:FindFirstChild("Shoot")
        local okOrigin: boolean, foundOrigin: any = pcall(getGunOriginCFrame, character, gun)
        local origin: CFrame? = okOrigin and foundOrigin or nil
        if not origin and nativeRedirect and typeof(opts.nativeOrigin) == "CFrame" then
            origin = opts.nativeOrigin
        end

        -- v160: the two cheap gates move ABOVE the aim solve. Auto Shoot runs
        -- this function every frame; during the 1.5 s confirmation window it
        -- used to run for the predictor plus wall rays and then
        -- reject at the very end. Refusing before the work leaves the firing
        -- instant unchanged (the gate is time based) and makes an Auto Shoot
        -- that is holding fire cost almost nothing per frame.
        local pendingShot: any = state.mm2ShotFeedback.pending
        if not continuous and not nativeRedirect
            and type(pendingShot) == "table"
            and type(pendingShot.queuedAt) == "number"
            and os.clock() - pendingShot.queuedAt < SHOT_CONFIRM_WINDOW then
            local remaining: number = SHOT_CONFIRM_WINDOW
                - (os.clock() - pendingShot.queuedAt)
            return reject(string.format(
                "Shot gate: waiting for the previous shot (%.1f s left).",
                remaining))
        end
        -- Do not overwrite the feedback record for an earlier shot just to
        -- redirect a second click; telemetry is optional and cannot gate it.
        local trackFeedback: boolean = not continuous
            and not (nativeRedirect and type(pendingShot) == "table")
        if not remote or not remote:IsA("RemoteEvent") or not origin then
            return reject(not origin and "The gun has no usable origin."
                or "The murderer is unavailable or obstructed.")
        end

        local info: any = nil
        local reason: string? = nil
        if origin then
            -- v161: tell the predictor which way this shot will be authored.
            -- Bullet and CFrame keep separate measured lead corrections.
            local aimMode: string = silent and "CFrame" or "Bullet"
            local ok: boolean, endpoint: any, aimInfo: any, aimReason: any =
                pcall(prediction.gunAim, target, origin.Position, aimMode)
            if not ok then return reject("Aim or visibility check failed.") end
            if endpoint == nil then
                reason = aimReason or "no samples"
            else
                info = aimInfo
            end
        end

        -- Raw mode or continuous Auto Shoot / Auto Win: when gunAim has no
        -- samples yet on the first frame, fall back immediately to the
        -- target's current root position (+ short velocity lead) so the shot
        -- fires without waiting even a single second.
        -- v186: an explicit manual press (the Gun Aimbot keybind) is the same
        -- kind of user act as the intercepted native click, so it gets the
        -- same validated first-sample fallback. Without it the press died
        -- silently whenever the predictor had no fresh sample yet (round
        -- start, target respawn), while a mouse click still fired: the gun
        -- appeared to ignore the keybind entirely.
        if (useRaw or continuous or nativeRedirect or opts.whenReady == true) and not info and target.Character then
            local targetRoot: BasePart? = target.Character:FindFirstChild("HumanoidRootPart")
            if targetRoot then
                local vel: Vector3 = Vector3.zero
                pcall(function(): ()
                    vel = targetRoot.AssemblyLinearVelocity or Vector3.zero
                end)
                local fallbackEndpoint: Vector3 = targetRoot.Position + vel * 0.08
                info = {
                    endpoint = fallbackEndpoint,
                    velocity = vel,
                    leadSeconds = 0.08,
                    motionSource = "raw",
                    model = "raw",
                    sampleAge = 0,
                    samples = 0,
                    capturedPosition = targetRoot.Position,
                    rtt = 0,
                    echoDelay = 0,
                    rttEngine = "raw",
                    echoSamples = 0,
                    turnRate = 0,
                    brakeSeconds = 0,
                    leadCorrection = 0,
                    rootPosition = targetRoot.Position,
                }
            end
        end

        -- v161: the aim is final at this point; make sure its line does not
        -- cross a ghost body part (transparent + non-colliding) that every
        -- client wall check skips. Preference only: failure keeps the point.
        if info and origin and targetCharacter then
            local cleaner: Vector3?, cleanerPart: string? =
                avoidGhostAim(target, info, origin.Position)
            if cleaner then
                info.endpoint = cleaner
                info.targetPosition = cleaner
                info.ghostAvoided = cleanerPart
            end
        end

        -- v139.3: at arm's length the lead stops helping and starts hurting.
        -- The murderer charging at the measured 17 studs/s covers 1.7 studs in
        -- a 0.1 s lead: from 1.5 studs away the predicted point lands BEHIND
        -- our own muzzle, the authored segment inverts, and the shot is
        -- refused as "Invalid shot geometry" (reproduced in
        -- tools/test_gun_pointblank.py). Inside POINT_BLANK_STUDS the body
        -- covers the whole sight, so aim at it and drop the lead.
        if origin and info then
            local root: BasePart? = nil
            pcall(function(): ()
                root = targetCharacter:FindFirstChild("HumanoidRootPart") :: BasePart?
            end)
            local aimVector: Vector3 = info.endpoint - origin.Position
            local toRoot: Vector3 = root and (root.Position - origin.Position) or aimVector
            local inverted: boolean = aimVector.Magnitude < 0.05
                or aimVector:Dot(toRoot) <= 0
            if root and (inverted or aimVector.Magnitude < POINT_BLANK_STUDS) then
                info.capturedPosition = root.Position
                info.endpoint = root.Position
                info.aimPartChosen = "HumanoidRootPart"
                info.pointBlank = true
            end
        end

        local rejectMessages: {[string]: string} = {
            ["no body"] = "The murderer has no usable body to aim at.",
            ["no samples"] = "The murderer has no fresh motion samples yet.",
            ["stale sample"] = "The murderer's position is stale; waiting for a fresh sample.",
            ["unreachable"] = "The murderer is moving too fast to intercept.",
        }
        if not remote
            or not remote:IsA("RemoteEvent")
            or not origin
            or not info then
            local message: string = rejectMessages[reason or ""]
                or (
                    not origin
                        and "The gun has no usable origin."
                        or "The murderer is unavailable or obstructed."
                )
            return reject(message)
        end

        -- One authored request at a time (v106). Continuous Auto Shoot /
        -- Auto Win bypasses the 1.5s confirmation gate so it never stops
        -- firing while the Murderer is still alive. The gate itself is
        -- checked above, before any aim or raycast work.
        local function safeToSend(
            shotOrigin: CFrame,
            shotEnd: CFrame,
            muzzle: CFrame?
        ): boolean
            if gun.Parent ~= character or (not nativeRedirect and (gun.Enabled == false
                or gun:GetAttribute("Disabled") == true or gun:GetAttribute("Reloading") == true
                or gun:GetAttribute("IsReloading") == true)) then
                return reject("The gun is disabled or reloading.")
            end
            -- v139.4: "raw" means "do not wait for the near-target origin
            -- guard", NOT "do not look at walls". Raw/continuous Silent AIM
            -- used to return true here, so Auto Shoot and Auto Win (which call
            -- this path with continuous=true) fired with no wall check at all:
            -- the only geometry they ever saw was the near-target pass, never
            -- the gun's own line. When WallCheck is enabled, check the muzzle
            -- line too; Auto Farm explicitly opts out of that filter.
            if useRaw then
                if not useWallCheck or not isWeaponPathClear then
                    return true
                end
                local rawMuzzle: Vector3 = (muzzle or shotOrigin).Position
                local rawClear, rawBlocker = isWeaponPathClear(
                    rawMuzzle,
                    shotEnd.Position,
                    targetCharacter,
                    nil,
                    {ignoreTarget = true}
                )
                if not rawClear then
                    if type(rawBlocker) == "table" and trajectoryCalibration
                        and type(trajectoryCalibration.noteBlockedShot) == "function" then
                        pcall(trajectoryCalibration.noteBlockedShot,
                            trajectoryCalibration, rawBlocker)
                    end
                    return reject("A wall or another body blocks the shot.")
                end
                return true
            end
            -- The muzzle is the character's gun, never the authored origin.
            -- validateGunShot checks BOTH rays: the gun's line of sight and
            -- the authored segment. Passing the silent origin twice made them
            -- one ray, so a shot the gun could not see was cleared by a
            -- near-target origin (the v105 silent map hits), and a body
            -- between the gun and the target was never looked for.
            local muzzlePosition: Vector3 = (muzzle or shotOrigin).Position
            local ok, allowed, why, blocker, occluder = pcall(validateGunShot,target,character,targetCharacter,
```

```
            muzzlePosition,shotOrigin.Position,shotEnd.Position,useWallCheck)
        if not ok or not allowed then
            if type(blocker) == "table" and trajectoryCalibration
                and type(trajectoryCalibration.noteBlockedShot) == "function" then
                pcall(trajectoryCalibration.noteBlockedShot, trajectoryCalibration, blocker)
            end
            return reject(why or "Shot safety check failed.")
        end
        -- Cleared, but keep what the untouched ray met: a transparent solid
        -- short of the aim is exactly the case the client cannot see.
        if type(occluder) == "table" then
            lastGunOccluder = occluder
        else
            lastGunOccluder = nil
        end
        return true
    end

    -- Do not infer a reload from our last REQUEST: a rejected packet
    -- never proves the server fired. Native Tool/attribute readiness is
    -- checked above; no speculative 3.1-second local lockout.
    if silent then

        -- The configured construction, not a hardcoded one: the setting was
        -- only ever read for telemetry, so a saved "Top" or "Front" was
        -- silently replaced by "Through" on every shot.
        local variant: string = "Through"
        local configuredSpawn: any = mm2Settings.silentSpawn
        if configuredSpawn == "Top" or configuredSpawn == "Behind"
            or configuredSpawn == "Front" then
            variant = configuredSpawn
        end
        local shotOrigin: CFrame?, shotEnd: CFrame?, offset: number,
            sweep: number, resolvedVariant: string
        if info.pointBlank and origin then
            -- v139.3: at arm's length the "Through" construction pierces
            -- along the target's own motion, and a murderer charging at us
            -- makes that direction point at US: the authored segment ended
            -- on our own muzzle (muzzle 0.5, endpoint 0.5) and every shot
            -- was refused as "Invalid shot geometry". Point blank needs no
            -- trick at all -- the muzzle already sees the body.
            shotOrigin = origin
            shotEnd = CFrame.new(info.endpoint)
            offset = 0
            sweep = 0
            resolvedVariant = "PointBlank"
        else
            shotOrigin, shotEnd, offset, sweep, resolvedVariant =
                buildSilentShot(target, info, origin, variant)
        end
        if not shotOrigin or not shotEnd then
            if continuous and target.Character then
                local tr: BasePart? = target.Character:FindFirstChild("HumanoidRootPart")
                if tr then
                    local ep: Vector3 = info.endpoint or tr.Position
                    local capPt: Vector3 = info.capturedPosition or tr.Position
                    local startPt: Vector3 = capPt + Vector3.new(0, 1.2, -1.5)
                    shotOrigin = CFrame.lookAt(startPt, ep)
                    shotEnd = CFrame.new(ep)
                    offset = 1.5
                    sweep = 0
                    resolvedVariant = "Through"
                end
            end
            if not shotOrigin or not shotEnd then
                return reject("Could not construct the silent shot.")
            end
        end
        variant = resolvedVariant
        -- A near-target origin inside a body can make the engine miss that
        -- containing body. Fall back to the real muzzle, never a bypass.
        -- Raw mode skips this check: the shot fires regardless.
        if not useRaw
            and not isWeaponPathClear(shotOrigin.Position,shotEnd.Position,targetCharacter,nil,{ignoreTarget=true}) then
            shotOrigin,shotEnd = origin,CFrame.new(info.endpoint)
            variant = "Muzzle"
            offset = 0
            sweep = 0
        end
        if not safeToSend(shotOrigin,shotEnd,origin) then return false end
        if trackFeedback then
            pcall(state.mm2ShotFeedback.queue, target, gun, shotOrigin, shotEnd)
        end

        state.lastSilentShotInfo = {
            variant = variant,
            at = os.clock(),
            offset = math.floor(offset * 10) / 10,
        }
        if opts.redirect then
            opts.redirect(shotOrigin, shotEnd)
        else
            remote:FireServer(shotOrigin, shotEnd)
        end

        local function recordSilent(): ()
            pcall(function(): ()
                if not trajectoryCalibration or type(trajectoryCalibration.noteAuthoredShot) ~= "function" then return end
                trajectoryCalibration:noteAuthoredShot({
                    kind = "gun",
                    source = nativeRedirect and "native-redirect" or "menu",
                    tool = gun,
                    toolName = gun and gun.Name or "Gun",
                    originPos = shotOrigin.Position,
                    aimPos = shotEnd.Position,
                    target = target,
                    silent = true,
                    variant = variant,
                    offset = offset,
                    settings = {mode = "Silent", spawn = variant,
                        sweep = sweep, wallCheck = useWallCheck,
                        leadSeconds = info.leadSeconds, motionSource = info.motionSource,
                        model = info.model,
                        sampleAge = info.sampleAge, samples = info.samples,
                        referencePoint = info.capturedPosition and {info.capturedPosition.X, info.capturedPosition.Y, info.capturedPosition.Z},
                        predictedPoint = {info.endpoint.X, info.endpoint.Y, info.endpoint.Z},
                        sampledVelocity = {info.velocity.X, info.velocity.Y, info.velocity.Z},
                        rtt = info.rtt, echoDelay = info.echoDelay,
                        rttEngine = info.rttEngine, rttData = info.rttData,
                        occluder = lastGunOccluder,
                        echoSamples = info.echoSamples,
                        turnRate = info.turnRate, brakeSeconds = info.brakeSeconds,
                        leadCorrection = info.leadCorrection,
                        leadCorrectionTarget = info.leadCorrectionTarget,
                        leadSamples = info.leadSamples, aimMode = info.aimMode,
                        ghostAvoided = info.ghostAvoided},
                })
            end)
        end
        if nativeRedirect and task and type(task.defer) == "function" then
            task.defer(recordSilent)
        else
            recordSilent()
        end
        if not quiet and ShootFeature then
            ShootFeature:SetStatus(experimental and "Experimental · CFrame · request sent" or nil)
        end
        if not continuous then
            macroUnequip(gun)
        end
        return true
    end

    local shotEndCFrame: CFrame = CFrame.new(info.endpoint)
    if not safeToSend(origin, shotEndCFrame) then return false end
    if trackFeedback then
        pcall(state.mm2ShotFeedback.queue, target, gun, origin, shotEndCFrame)
    end
    -- Author one endpoint, without speculative "piercing" past it.
    -- Echo timing documents transport/visual confirmation, NOT rewind
    -- duration or an authoritative hit-rate denominator.
    local shotOriginCFrame: CFrame = CFrame.lookAt(origin.Position, info.endpoint)
    if opts.redirect then
        opts.redirect(shotOriginCFrame, shotEndCFrame)
    else
        remote:FireServer(shotOriginCFrame, shotEndCFrame)
    end
    local function recordBullet(): ()
        pcall(function(): ()
            if not trajectoryCalibration or type(trajectoryCalibration.noteAuthoredShot) ~= "function" then return end
            trajectoryCalibration:noteAuthoredShot({
                kind = "gun",
                source = nativeRedirect and "native-redirect" or "menu",
                tool = gun,
                toolName = gun and gun.Name or "Gun",
                originPos = origin.Position,
                aimPos = info.endpoint,
                target = target,
                silent = false,
                settings = {mode = "Normal", wallCheck = useWallCheck,
                    leadSeconds = info.leadSeconds, motionSource = info.motionSource,
                    model = info.model,
                    sampleAge = info.sampleAge, samples = info.samples,
                    referencePoint = info.capturedPosition and {info.capturedPosition.X, info.capturedPosition.Y, info.capturedPosition.Z},
                    predictedPoint = {info.endpoint.X, info.endpoint.Y, info.endpoint.Z},
                    sampledVelocity = {info.velocity.X, info.velocity.Y, info.velocity.Z},
                    rtt = info.rtt, echoDelay = info.echoDelay,
                    rttEngine = info.rttEngine, rttData = info.rttData,
                    occluder = lastGunOccluder,
                    echoSamples = info.echoSamples,
                    turnRate = info.turnRate, brakeSeconds = info.brakeSeconds,
                    leadCorrection = info.leadCorrection,
                    leadCorrectionTarget = info.leadCorrectionTarget,
                    leadSamples = info.leadSamples, aimMode = info.aimMode,
                    ghostAvoided = info.ghostAvoided},
            })
        end)
    end
    if nativeRedirect and task and type(task.defer) == "function" then
        task.defer(recordBullet)
    else
        recordBullet()
    end
    if not quiet and ShootFeature then
        ShootFeature:SetStatus(experimental and "Experimental · Bullet · request sent" or nil)
    end
    if not continuous then
        macroUnequip(gun)
    end
    return true
end

local shootOptionsReady: boolean = false
local shootFeatureActive: boolean = false
local autoShootActive: boolean = false
local autoShootWanted: boolean = false
local transientShotActive: boolean = false
local transientShotGeneration: number = 0
local updateAutoShootConnection: (() -> ())? = nil

local observerGeneration: number = 0
-- v187: the native hook compares the intercepted remote against this
-- cache, resolved OUTSIDE the hook. Resolving it inside the hook (the
-- v183-v186 code called getPlayerWeapon/FindFirstChild there) performs
-- namecalls of its own, which overwrite the pending namecall method;
-- the original trampoline then dispatched the wrong method and the
-- native Gun/Knife client calls died inside our hook ("argument #1
-- expects a string, but Instance was passed", live session 2026-11-06).
local cachedShootRemote: Instance? = nil
local lastCacheProbeAt: number = -math.huge
local function refreshShootRemoteCache(): ()
    local remote: Instance? = nil
    local okGun: boolean, foundGun: any = pcall(getPlayerWeapon, LocalPlayer, "Gun", true)
    local gun: Tool? = if okGun then foundGun else nil
    if gun then
        pcall(function(): ()
            local found: Instance? = gun:FindFirstChild("Shoot")
            if found and found:IsA("RemoteEvent") then
                remote = found
            end
        end)
    end
    cachedShootRemote = remote
end

local function refreshShootInfrastructure(): ()
    observerGeneration += 1
    local generation: number = observerGeneration
    local wantActive: boolean = shootFeatureActive or autoShootActive or transientShotActive
    state.mm2ShotFeedback.shootActive = wantActive
    if wantActive then
        local function observe(): ()
            local weaponService: any = getWeaponServiceModule()
            if generation ~= observerGeneration or not state.mm2ShotFeedback.shootActive then return end
            if type(weaponService) == "table" then connectGunFiredSignal(weaponService) end
        end
        if state.isMobile then
            observe() -- preserve the mobile path
        else
            task.spawn(observe) -- never delay PC click redirection for an observer
        end
        refreshShootRemoteCache()
        return
    end
    disconnectGunFiredObserver()
    state.mm2ShotFeedback.pending = nil
    state.mm2ShotFeedback.lastAccepted = nil
    state.mm2ShotFeedback.hide()
    refreshShootRemoteCache()
end

local shotHookInstalled: boolean = false
local shotHookWrapper: any = nil
local originalNamecall: any = nil
local function installNativeGunHook(): boolean
    if state.isMobile then return true end
    if shotHookInstalled then return true end
    local hook: any = state.hookMetamethod
    local getMethod: any = state.getNamecallMethod
    local checkCaller: any = state.checkCaller
    if type(hook) ~= "function" or type(getMethod) ~= "function"
        or type(checkCaller) ~= "function" then
        return false
    end
    if type(task) ~= "table" or type(task.defer) ~= "function" then
        -- v190: the redirect ships from its own thread. Without task.defer
        -- the hook would have to call back into the engine on the game's
        -- thread, which is exactly what broke the native click in
        -- v183-v189. Fail closed and leave the native shot alone.
        return false
    end
    local predecessor: any = nil
    -- v190: the redirect never runs on the game's thread any more.
    --
    -- v187 tried to be careful: it checked the pending method name, the
    -- caller and a cached remote before touching the trampoline, then ran
    -- the whole solver and re-sent the packet "through the engine" to
    -- recover the method register the solver had clobbered. The live
    -- console of 2026-11-07 showed that is not enough: every method call
    -- inside the solver (FindFirstChild, raycasts, the ping read) rewrites
    -- the pending namecall method, and the engine then dispatches whatever
    -- ran last instead of FireServer. The player saw, once per click,
    -- "The current thread cannot call 'GetValue' (lacking capability
    -- Plugin)" attributed to `Workspace.<player>.Gun.GunClient`, and the
    -- gun never actuated: the error WAS our ping read, dispatched in place
    -- of the gun's FireServer.
    --
    -- The rule is mechanical now, not careful: after deciding to redirect,
    -- the hook body performs NO namecall at all. It packs the arguments,
    -- hands them to a fresh deferred thread and returns nil. The native
    -- packet never reaches a corrupted register and no error of ours can
    -- travel back into the game's click handler. The deferred thread does
    -- the solving and sends exactly one packet: the redirected one when
    -- the solver produced frames, or the original arguments verbatim when
    -- it did not. That re-fire comes from an executor thread, so
    -- checkcaller() sends it straight to the trampoline.
    local pendingRedirects: number = 0
    local MAX_PENDING_REDIRECTS: number = 2
    local sendingRedirect: boolean = false
    local function sendAsExecutor(remote: any, arguments: any): boolean
        local previous: boolean = sendingRedirect
        sendingRedirect = true
        local ok: boolean, failure: any = pcall(function(): ()
            remote:FireServer(table.unpack(arguments, 1, arguments.n))
        end)
        sendingRedirect = previous
        if not ok then
            warn("[Wurst] native gun redirect failed: " .. tostring(failure))
        end
        return ok
    end
    local function redirectDeferred(remote: any, arguments: any): ()
        if not shootFeatureActive or not shootOptionsReady then
            sendAsExecutor(remote, arguments)
            return
        end
        local shotOrigin: CFrame? = nil
        local shotEnd: CFrame? = nil
        local ok: boolean = pcall(function(): any
            lastRejectReason = ""
            return fireGunAtTarget(getSelectedShootTarget(), {
                quiet = true,
                nativeRedirect = true,
                nativeOrigin = arguments[1],
                redirect = function(origin: CFrame, endpoint: CFrame): ()
                    shotOrigin, shotEnd = origin, endpoint
                end,
            })
        end)
        if shotOrigin and shotEnd then
            if ShootFeature then
                pcall(function(): () ShootFeature:SetStatus("Native click redirected") end)
            end
            local redirected: any = table.pack(shotOrigin, shotEnd)
            local copied: number = 2
            for index: number = 3, arguments.n do
                copied += 1
                redirected[copied] = arguments[index]
            end
            redirected.n = copied
            sendAsExecutor(remote, redirected)
            return
        end
        if ShootFeature then
            pcall(function(): ()
                ShootFeature:SetStatus(if not ok then "Native redirect failed; original click kept"
                    else (lastRejectReason ~= "" and lastRejectReason or "Native click unchanged"))
            end)
        end
        -- No target, blocked line, unsupported geometry or an error: the
        -- click still fires exactly as the game authored it, one packet.
        sendAsExecutor(remote, arguments)
    end
    local function redirectNamecall(remote: any, ...: any): any
        if sendingRedirect then
            -- Our own deferred packet; its method register is intact.
            return predecessor(remote, ...)
        end
        if not shootFeatureActive or not shootOptionsReady then
            return predecessor(remote, ...)
        end
        if remote ~= cachedShootRemote then
            -- Identity comes FIRST: every unrelated namecall in the game
            -- (knife, chat, GUIs) leaves through one comparison, with no
            -- call of our own in between. The cache is refreshed outside
            -- this call when the gun was equipped after the last probe.
            local now: number = os.clock()
            if now - lastCacheProbeAt >= 0.5 then
                lastCacheProbeAt = now
                task.defer(refreshShootRemoteCache)
            end
            return predecessor(remote, ...)
        end
        local okMethod: boolean, method: any = pcall(getMethod)
        if not okMethod or method ~= "FireServer" then
            return predecessor(remote, ...)
        end
        local okCaller: boolean, fromExecutor: any = pcall(checkCaller)
        if not okCaller or fromExecutor ~= false then
            -- Auto Shoot, Auto Farm, the overlay and our own deferred
            -- re-fire arrive with the executor identity: they keep their
            -- own settings and never enter this hook.
            return predecessor(remote, ...)
        end
        local arguments: any = table.pack(...)
        if typeof(arguments[1]) ~= "CFrame" or typeof(arguments[2]) ~= "CFrame" then
            return predecessor(remote, ...)
        end
        if pendingRedirects >= MAX_PENDING_REDIRECTS then
            -- A click storm must not queue work: keep the native shot.
            return predecessor(remote, ...)
        end
        pendingRedirects += 1
        task.defer(function(): ()
            local handled: boolean, failure: any = pcall(redirectDeferred, remote, arguments)
            pendingRedirects -= 1
            if not handled then
                warn("[Wurst] native gun redirect error: " .. tostring(failure))
            end
        end)
        return nil
    end
    local callback: any = redirectNamecall
    if type(state.newCClosure) == "function" then
        local ok: boolean, wrapped: any = pcall(state.newCClosure, callback)
        if ok and type(wrapped) == "function" then callback = wrapped end
    end
    local ok: boolean, old: any = pcall(hook, game, "__namecall", callback)
    if not ok or type(old) ~= "function" then return false end
    predecessor = old
    originalNamecall, shotHookWrapper, shotHookInstalled = old, callback, true
    return true
end

local function uninstallNativeGunHook(): ()
    if not shotHookInstalled or type(state.hookMetamethod) ~= "function" then return end
    local ok: boolean, displaced: any = pcall(state.hookMetamethod, game, "__namecall", originalNamecall)
    if not ok then return end
    if displaced == shotHookWrapper then
        shotHookInstalled, shotHookWrapper, originalNamecall = false, nil, nil
    elseif type(displaced) == "function" then
        -- Someone else wrapped us. Put their hook back and leave our
        -- now-inert wrapper in their chain instead of clobbering it.
        pcall(state.hookMetamethod, game, "__namecall", displaced)
    end
end

local manualShotPending: boolean = false
local function triggerManualShot(): ()
    if state.isMobile and not shootFeatureActive then
        notify("Enable Gun Aimbot first.")
        return
    end
    if manualShotPending then return end
    local character, humanoid = getCharacterParts()
    local gun: Tool? = getPlayerWeapon(LocalPlayer, "Gun")
    if not character or not humanoid or not gun then
        notify("No sheriff gun is available.")
        return
    end
    manualShotPending = true
    local ok: boolean, message: any = pcall(function(): ()
        if gun.Parent ~= character then
            humanoid:EquipTool(gun)
            -- Let the grip/muzzle update before sampling its origin.
            RunService.Heartbeat:Wait()
        end
        -- Never re-equip if the user switched away during that frame.
        if (not state.isMobile or shootFeatureActive)
            and LocalPlayer.Character == character and gun.Parent == character then
            if not shootFeatureActive and not state.isMobile then
                transientShotGeneration += 1
                local generation: number = transientShotGeneration
                transientShotActive = true
                refreshShootInfrastructure()
                task.delay(2, function(): ()
                    if generation ~= transientShotGeneration then return end
                    transientShotActive = false
                    refreshShootInfrastructure()
                end)
            end
            fireGunAtTarget(getSelectedShootTarget(), {whenReady = true})
        end
    end)
    manualShotPending = false
    if not ok then notify("Shoot failed: " .. tostring(message)) end
end

local function toggleShootMurderer(enabled: boolean): ()
    shootFeatureActive = enabled
    state.mm2SilentAimEnabled = enabled
    if state.mm2ZKnifeThrowRefresh then state.mm2ZKnifeThrowRefresh() end
    autoShootActive = shootFeatureActive and autoShootWanted
    if enabled and not state.isMobile then
        if not installNativeGunHook() then
            notify("Native click redirect needs executor hookmetamethod, getnamecallmethod and checkcaller; the keybind can still fire.")
        end
    elseif not enabled then
        uninstallNativeGunHook()
    end
    refreshShootInfrastructure()
    if updateAutoShootConnection then updateAutoShootConnection() end
end

local lastAutoStatus: string? = nil
local function autoStatus(message: string): ()
    if ShootFeature and lastAutoStatus ~= message then
        lastAutoStatus = message
        ShootFeature:SetStatus(message)
    end
end

local function autoShootTick(): ()
    if not shootFeatureActive or not autoShootActive then return end
    local gun: Tool? = getPlayerWeapon(LocalPlayer, "Gun", true)
    if not gun or gun.Parent ~= LocalPlayer.Character then
        autoStatus("Equip gun")
        return
    end
    -- One pending slot: a second authored shot overwrites the first
    -- shot's record, its GunFired echo can no longer be matched and
    -- the calibration gains an unmatched signal. fireGunAtTarget applies
    -- the same gate to manual and automatic requests.
    if state and state.mm2ShotFeedback
        and state.mm2ShotFeedback.pending then
        autoStatus("Esperando confirmación")
        return
    end
    local target: Player? = getSelectedShootTarget()
    if not target then autoStatus("No target"); return end
    lastRejectReason = ""
    local sent: boolean = fireGunAtTarget(target, {whenReady = true, quiet = true})
    autoStatus(sent and "Auto: request sent" or lastRejectReason)
end

updateAutoShootConnection = function(): ()
    disconnectFeatureConnection("MM2AutoShoot")
    autoShootActive = shootOptionsReady and shootFeatureActive and autoShootWanted
    if not autoShootActive then
        lastAutoStatus = nil
        if ShootFeature then ShootFeature:SetStatus(nil) end
        return
    end
    -- One attempt per current frame, with no range/confidence/fire-delay
    -- timer. No cached aim or visibility survives into the next attempt.
    featureConnections.MM2AutoShoot =
        TaskManager:Connect(autoShootTick, nil, "MM2 Auto Shoot")
    autoShootTick()
end

local function toggleAutoShoot(enabled: boolean): ()
    autoShootWanted = enabled
    autoShootActive = shootFeatureActive and autoShootWanted
    refreshShootInfrastructure()
    if updateAutoShootConnection then updateAutoShootConnection() end
end

-- Preserve explicit bindings and enabled state when the old card becomes
-- a child option. Never overwrite a setting already saved for the child.
if type(configData) == "table" then
    local changed: boolean = false
    if configData.states["MM2.Shoot.AutoShoot"] == nil and configData.states["MM2.AutoShoot"] ~= nil then
        local enabled: boolean = configData.states["MM2.AutoShoot"] == true
        configData.states["MM2.Shoot.AutoShoot"] = enabled
        if enabled then configData.states["MM2.Shoot"] = true end
        changed = true
    end
    local oldKey: any = configData.values["Shortcut.MM2AutoShoot"]
    if configData.values["Shortcut.MM2ShootAutoShoot"] == nil and type(oldKey) == "string" then
        configData.values["Shortcut.MM2ShootAutoShoot"] = oldKey
        changed = true
    end
    if changed and type(queueConfigSave) == "function" then queueConfigSave() end
end

if type(configData) == "table" and configData.values["MM2.Shoot.Aimmethod"] == nil then
    configData.values["MM2.Shoot.Aimmethod"] = configData.values["MM2.Shoot.Mode"]
end
ShootFeature = createUniversalFeature(
    "Gun Aimbot",
    state.isMobile
        and "Gun Aimbot with Normal(Bullet), Silent(Cframe), and Experimental modes. The assigned action fires/equips Gun; WallCheck is configurable per mode."
        or "Redirect native clicks with Bullet or Silent(CFrame). One optional keybind fires even while the module is off.",
    4,
    toggleShootMurderer,
    {
        categoryName = "Exploits",
        configKey = "MM2.Shoot", -- preserve existing Gun settings and activation key
        parent = MM2Scroll,
        registry = mm2Features,
    }
)
state.mm2SilentAimFeature = ShootFeature
state.mm2ChooseExperimentalAim = chooseExperimentalGunMode
-- Other modules (Auto Shoot, Auto Farm's full-bag grab, Auto Win) fire
-- through the exact Gun Aimbot path instead of hand-rolling their own shot.
-- v139.4: the shared constructor accepts the caller's options. Without
-- them Auto Shoot's own WallCheck setting was dropped on the floor: the
-- wrapper hardcoded continuous=true, which is exactly the mode that used
-- to skip the geometry check.
state.mm2FireSilentShot = function(target: Player?, shotOpts: any): boolean
    local opts: any = type(shotOpts) == "table" and shotOpts or {}
    return fireGunAtTarget(target, {
        silent = true,
        quiet = opts.quiet ~= false,
        continuous = opts.continuous ~= false,
        wallCheck = opts.wallCheck,
    })
end
state.mm2FireGunShot = function(target: Player?, shotOpts: any): boolean
    local opts: any = type(shotOpts) == "table" and shotOpts or {}
    return fireGunAtTarget(target, {
        silent = opts.silent == true,
        wallCheck = opts.wallCheck,
        quiet = opts.quiet ~= false,
        continuous = opts.continuous ~= false,
    })
end
-- One card, two weapons: the heading keeps the gun rows together and the
-- knife module opens its own section further down.
if addSectionOption then addSectionOption(ShootFeature, "Gun") end
local states: any = type(configData) == "table" and configData.states or {}
local values: any = type(configData) == "table" and configData.values or {}
-- Display names changed, but the old Bullet/CFrame keys keep their
-- per-mode settings and remain valid when loading existing profiles.
local function modeKey(value: any): string
    if value == "Experimental" then return "Experimental" end
    if value == "CFrame" or value == "Silent(Cframe)" or value == "Silent(CFrame)" then return "CFrame" end
    return "Bullet"
end
local savedMode: any = values["MM2.Shoot.Aimmethod"]
local initialMode: string = modeKey(savedMode
    or (mm2Settings.experimentalAim and "Experimental" or (mm2Settings.silentAim and "CFrame" or "Bullet")))
if state.isMobile then
    if savedMode == "Bullet" or savedMode == "CFrame" or savedMode == "Silent(CFrame)" then
        values["MM2.Shoot.Aimmethod"] = (savedMode == "CFrame" or savedMode == "Silent(CFrame)")
            and "Silent(Cframe)" or "Normal(Bullet)"
        if type(queueConfigSave) == "function" then queueConfigSave() end
    end
else
    if initialMode == "Experimental" then initialMode = "Bullet" end
    local normalized: string = initialMode == "CFrame" and "Silent(CFrame)" or "Bullet"
    if values["MM2.Shoot.Aimmethod"] ~= normalized then
        values["MM2.Shoot.Aimmethod"] = normalized
        if type(queueConfigSave) == "function" then queueConfigSave() end
    end
end
local function modeConfig(mode: string): any
    local prefix: string = "MM2.Shoot." .. mode .. "."
    local legacy: boolean = mode == initialMode
    local wall: any = states[prefix .. "WallCheck"]
    local macro: any = states[prefix .. "Macro"]
    local targetMode: any = values[prefix .. "Target"]
    local targetName: any = values[prefix .. "TargetPlayer"]
    if legacy then
        if wall == nil then wall = states["MM2.Shoot.WallCheck"] end
        if macro == nil then macro = states["MM2.Shoot.Macro"] end
        if targetMode == nil then targetMode = values["MM2.Shoot.Target"] end
        if targetName == nil then targetName = values["MM2.Shoot.Targetplayer"] end
    end
    return {wallCheck = wall == nil or wall == true, macro = macro == true,
        targetMode = targetMode == "Custom" and "Custom" or "Murder",
        targetName = type(targetName) == "string" and targetName or ""}
end
local modeConfigs: any = {Bullet = modeConfig("Bullet"), CFrame = modeConfig("CFrame"),
    Experimental = modeConfig("Experimental")}
local activeModeKey: string = initialMode
local switching: boolean = false
local optionsReady: boolean = false
local ShootTargetBox
local function refreshShootOptions(): ()
    if ShootTargetBox then
        setOptionVisible(ShootFeature, ShootTargetBox.Parent :: GuiObject, mm2Settings.shootMode == "Custom")
    end
```

```
    end
    local function syncMode(): ()
        local cfg: any = modeConfigs[activeModeKey] or modeConfigs.Bullet
        switching = true
        mm2Settings.shootWallCheck = cfg.wallCheck
        shootMacro, mm2Settings.shootMode, mm2Settings.shootTarget = cfg.macro, cfg.targetMode, cfg.targetName
        if state and type(state.toggleControls) == "table" then
            for _, item: any in ipairs({{"WallCheck", cfg.wallCheck}, {"Macro", cfg.macro}}) do
                local control: any = state.toggleControls["MM2.Shoot." .. item[1]]
                if control and control.setChecked then pcall(control.setChecked, item[2]) end
            end
        end
        if state and type(state.valueControls) == "table" then
            local target: any = state.valueControls["MM2.Shoot.Target"]
            local name: any = state.valueControls["MM2.Shoot.Targetplayer"]
            if target and target.apply then pcall(target.apply, cfg.targetMode) end
            if name and name.apply then pcall(name.apply, cfg.targetName) end
        end
        switching = false
        refreshShootOptions()
    end
    local aimMethods: {string} = state.isMobile
        and {"Normal(Bullet)", "Silent(Cframe)", "Experimental"}
        or {"Bullet", "Silent(CFrame)"}
    addCycleOption(ShootFeature, "Aim method", aimMethods,
        initialMode == "Experimental" and 3 or (initialMode == "CFrame" and 2 or 1),
        function(value: string): ()
            activeModeKey = modeKey(value)
            mm2Settings.experimentalAim = activeModeKey == "Experimental"
            mm2Settings.silentAim = activeModeKey == "CFrame"
            syncMode()
        end,
        state.isMobile
            and "Experimental selects one route based on movement and range; hit-rate improvement is unverified. Options are saved per mode."
            or "One keybind is shared; WallCheck and target settings are saved per mode.")
    addCycleOption(ShootFeature, "Target", {"Murder", "Custom"}, 1, function(value: string): ()
        mm2Settings.shootMode = value
        if optionsReady and not switching then
            modeConfigs[activeModeKey].targetMode = value
            values["MM2.Shoot." .. activeModeKey .. ".Target"] = value
        end
        refreshShootOptions()
    end)
    if state.bindFeatureActivationKey then
        state.bindFeatureActivationKey(ShootFeature, mm2Settings.shootKey, triggerManualShot)
    end
    if not state.isMobile then
        local binding: any = state.shortcutBindings and state.shortcutBindings[ShootFeature.configKey]
        if binding then binding.feedback = nil end
        if type(state.addFeatureShortcutOption) == "function" then
            state.addFeatureShortcutOption(ShootFeature, "Keybind",
                "Optional: click the box and press a key. It fires in either mode, even when Gun Aimbot is off.")
        end
    end
    addToggleOption(ShootFeature, "WallCheck", true, function(value: boolean): ()
        mm2Settings.shootWallCheck = value == true
        if optionsReady and not switching then
            modeConfigs[activeModeKey].wallCheck = value == true
            states["MM2.Shoot." .. activeModeKey .. ".WallCheck"] = value == true
        end
    end, "Check for walls; saved independently per mode.")
    addToggleOption(ShootFeature, "Macro", false, function(value: boolean): ()
        shootMacro = value == true
        if optionsReady and not switching then
            modeConfigs[activeModeKey].macro = value == true
            states["MM2.Shoot." .. activeModeKey .. ".Macro"] = value == true
        end
    end, "Pistol macro: hides the gun after the shot; saved per mode.")
    ShootTargetBox = addTextOption(ShootFeature, "Target player", "", function(value: string): ()
        mm2Settings.shootTarget = value
        if optionsReady and not switching then
            modeConfigs[activeModeKey].targetName = value
            values["MM2.Shoot." .. activeModeKey .. ".TargetPlayer"] = value
            if type(queueConfigSave) == "function" then queueConfigSave() end
        end
    end, false)
    optionsReady = true
    syncMode()

    -- The visible Auto control is at the top, but arming waits until saved
    -- aim method/target options below have all been restored.
    shootOptionsReady = true
    if shootFeatureActive and not state.isMobile and not shotHookInstalled then
        if not installNativeGunHook() then
            notify("Native click redirect needs executor hookmetamethod, getnamecallmethod and checkcaller; the keybind can still fire.")
        end
    end
    if updateAutoShootConnection then updateAutoShootConnection() end

    activeCleanup = function(): ()
        transientShotGeneration += 1
        transientShotActive = false
        toggleShootMurderer(false)
        state.mm2SilentAimFeature = nil
        state.mm2FireSilentShot = nil
        state.mm2FireGunShot = nil
        state.mm2ChooseExperimentalAim = nil
        toggleAutoShoot(false)
        state.mm2ShotFeedback.hide()
    end
    Module.Events = featureConnections
    Module.Initialized = true
    return Module
end

function Module.destroy(): ()
    if not Module.Initialized then
        return
    end
    Module.Initialized = false
    pcall(activeCleanup)
    activeCleanup = function(): () end
    Module.Events = {}
    Module.Runtime = nil
end

return Module

-- ============================================================================
FIN DEL ARCHIVO: src/games/MM2/Blatant/Shoot.lua
-- ============================================================================

-- ============================================================================
ARCHIVO: src/games/MM2/Combat/AutoShoot.lua
BYTES: 25268
SHA-256: 825bd53f9edf160c126114308abc9fd163868f7cbbae500078364479031c6614
INICIO DEL ARCHIVO
-- ============================================================================
local Module = {
    Name = "MM2 Auto Shoot",
    Events = {} :: {[string]: any},
    Initialized = false,
    Runtime = nil :: any,
}

local activeCleanup: () -> () = function(): () end

function Module.init(runtime: any): any
    if Module.Initialized then
        return Module
    end
    local core: any = state.mm2Core
    assert(type(core) == "table", "MM2 Auto Shoot requires the MM2 core module")
    local prediction: any = state.mm2Prediction
    assert(type(prediction) == "table", "MM2 Auto Shoot requires MM2 Prediction")
    Module.Runtime = runtime

    local findMurderer: any = core.findMurderer
    local getPlayerRole: any = core.getPlayerRole
    local getPlayerWeapon: any = core.getPlayerWeapon
    local isPlayerAlive: any = core.isPlayerAlive
    local isProtectedTarget: any = core.isProtectedTarget
    local isWeaponPathClear: any = core.isWeaponPathClear
    local getGunOriginCFrame: any = core.getGunOriginCFrame

    local settings = {
        mode = "Bullet",
        wallCheck = true,
        proximity = false,
        aura = 25,
    }

    local autoShootActive: boolean = false
    local AutoShootFeature: any = nil
    local toolConnections: {RBXScriptConnection} = {}
    local lastShotSentAt: number = 0
    local AUTO_SHOT_PACE_SECONDS: number = 0.22

    local function clearToolConnections(): ()
        for _, conn: RBXScriptConnection in ipairs(toolConnections) do
            pcall(function(): ()
                conn:Disconnect()
            end)
        end
        table.clear(toolConnections)
    end

    local function findLocalGun(): (Tool?, Model?, Humanoid?, BasePart?)
        local character: Model? = LocalPlayer and LocalPlayer.Character
        if not character then
            return nil, nil, nil, nil
        end
        local humanoid: Humanoid? = character:FindFirstChildOfClass("Humanoid")
        local root: BasePart? = character:FindFirstChild("HumanoidRootPart") :: BasePart?
        if not humanoid or humanoid.Health <= 0 or not root then
            return nil, character, humanoid, root
        end
        local gun: Tool? = nil
        if type(getPlayerWeapon) == "function" then
            gun = getPlayerWeapon(LocalPlayer, "Gun", false)
        end
        if not gun then
            local charGun: Instance? = character:FindFirstChild("Gun")
            if charGun and charGun:IsA("Tool") then
                gun = charGun
            end
        end
        if not gun then
            local backpack: Instance? = LocalPlayer:FindFirstChildOfClass("Backpack")
            local bpGun: Instance? = backpack and backpack:FindFirstChild("Gun")
            if bpGun and bpGun:IsA("Tool") then
                gun = bpGun
            end
        end
        return gun, character, humanoid, root
    end

    local function ensureGunEquipped(
        character: Model,
        humanoid: Humanoid,
        gun: Tool
    ): boolean
        if gun.Parent == character then
            return true
        end
        pcall(function(): ()
            humanoid:EquipTool(gun)
        end)
        if gun.Parent ~= character then
            pcall(function(): ()
                gun.Parent = character
            end)
        end
        return gun.Parent == character
    end

    local function resolveMurdererTarget(
        localRoot: BasePart,
        enforceProximity: boolean
    ): (Player?, BasePart?)
        local murderer: Player? = nil
        if type(findMurderer) == "function" then
            murderer = findMurderer()
        end
        if not murderer then
            local playersService: Players = game:GetService("Players")
            for _, player: Player in ipairs(playersService:GetPlayers()) do
                if player ~= LocalPlayer then
                    local role: any = type(getPlayerRole) == "function"
                        and getPlayerRole(player)
                        or nil
                    local hasKnife: boolean = type(getPlayerWeapon) == "function"
                        and getPlayerWeapon(player, "Knife", false) ~= nil
                    if role == "Murderer" or hasKnife then
                        murderer = player
                        break
                    end
                end
            end
        end
        if not murderer or murderer == LocalPlayer then
            return nil, nil
        end
        if type(isPlayerAlive) == "function" and not isPlayerAlive(murderer) then
            return nil, nil
        end
        if type(isProtectedTarget) == "function" and isProtectedTarget(murderer) then
            return nil, nil
        end
        local targetChar: Model? = murderer.Character
        local targetRoot: BasePart? = targetChar
            and targetChar:FindFirstChild("HumanoidRootPart") :: BasePart?
        if not targetRoot then
            return nil, nil
        end
        if enforceProximity then
            local okDist: boolean, distance: any = pcall(function(): any
                return (targetRoot.Position - localRoot.Position).Magnitude
            end)
            if okDist and type(distance) == "number" and distance > settings.aura then
                return nil, nil
            end
        end
        return murderer, targetRoot
    end

    local function computeTargetEndpoint(
        target: Player,
        targetRoot: BasePart,
        originPos: Vector3,
        aimMode: string?
    ): Vector3
        if type(prediction) == "table" and type(prediction.gunAim) == "function" then
            local ok: boolean, endpoint: any = pcall(
                prediction.gunAim,
                target,
                originPos,
                aimMode
            )
            if ok and typeof(endpoint) == "Vector3" then
                return endpoint
            end
        end
        local vel: Vector3 = Vector3.zero
        pcall(function(): ()
            vel = targetRoot.AssemblyLinearVelocity
        end)
        return targetRoot.Position + vel * 0.08
    end

    local function fireCFrameShotImmediate(
        target: Player,
        targetRoot: BasePart,
        character: Model,
        localRoot: BasePart,
        gun: Tool
    ): boolean
        local shootRemote: Instance? = gun:FindFirstChild("Shoot")
        if not (shootRemote and shootRemote:IsA("RemoteEvent")) then
            return false
        end
        if gun.Enabled == false
            or gun:GetAttribute("Disabled") == true
            or gun:GetAttribute("Reloading") == true
            or gun:GetAttribute("IsReloading") == true then
            return false
        end
        local originCF: CFrame = (type(getGunOriginCFrame) == "function"
            and getGunOriginCFrame(character, gun))
            or localRoot.CFrame
        local endpoint: Vector3 = computeTargetEndpoint(
            target,
            targetRoot,
            originCF.Position,
            "CFrame"
        )
        local capturedPos: Vector3 = targetRoot.Position
        local chordFlat: Vector3 = Vector3.new(
            endpoint.X - capturedPos.X,
            0,
            endpoint.Z - capturedPos.Z
        )
        local pierceAxis: Vector3
        if chordFlat.Magnitude > 0.15 then
            pierceAxis = chordFlat.Unit
        else
            local toTargetFlat: Vector3 = Vector3.new(
                endpoint.X - originCF.Position.X,
                0,
                endpoint.Z - originCF.Position.Z
            )
            if toTargetFlat.Magnitude > 0.05 then
                pierceAxis = toTargetFlat.Unit
            else
                pierceAxis = Vector3.new(0, 0, -1)
            end
        end
        local startPoint: Vector3 = capturedPos - pierceAxis * 1.5
        local endPoint: Vector3 = endpoint + pierceAxis * 1.5
        if (endPoint - startPoint).Magnitude < 0.2 then
            startPoint = endpoint + Vector3.new(0, 2.0, -1.5)
            endPoint = endpoint
        end
        local shotOrigin: CFrame = CFrame.lookAt(startPoint, endPoint)
        local shotEnd: CFrame = CFrame.new(endPoint)
        local firedSilent: boolean = false
        if type(state.mm2FireSilentShot) == "function" then
            -- v139.4: our own WallCheck has to travel with the request. The
            -- shared constructor used to be called with the target only and
            -- hardcoded continuous=true, so this mode fired with no geometry
            -- check and then, when it was refused, fell back to its own CFrame
            -- -- also unchecked.
            local okSilent: boolean, res: any = pcall(state.mm2FireSilentShot, target, {
                continuous = true,
                quiet = true,
                wallCheck = settings.wallCheck,
            })
            firedSilent = okSilent and res == true
            -- A false result or exception can mean a safety refusal or an
            -- uncertain send. Never bypass it or send a second request. Pace
            -- exceptions too: the send may have happened before the error.
            if not okSilent then lastShotSentAt = os.clock() end
            if not firedSilent then return false end
        end
        if not firedSilent then
            -- Never fire past a wall: the hand-rolled CFrame is only a
            -- transport fallback, not a bypass.
            local clear: boolean = false
            if settings.wallCheck and type(isWeaponPathClear) == "function" then
                local okClear: boolean, allowed: any = pcall(
                    isWeaponPathClear,
                    originCF.Position,
                    endpoint,
                    target.Character,
                    nil,
                    {ignoreTarget = true}
                )
                clear = okClear and allowed == true
            else
                clear = true
            end
            if clear then
                local okFire: boolean = pcall(function(): ()
                    (shootRemote :: RemoteEvent):FireServer(shotOrigin, shotEnd)
                end)
                if not okFire then
                    lastShotSentAt = os.clock()
                    return false
                end
            else
                lastShotSentAt = 0
                return false
            end
        end
        lastShotSentAt = os.clock()
        return true
    end

    local function fireBulletShotImmediate(
        target: Player,
        targetRoot: BasePart,
        character: Model,
        localRoot: BasePart,
        gun: Tool
    ): boolean
        local shootRemote: Instance? = gun:FindFirstChild("Shoot")
        if not (shootRemote and shootRemote:IsA("RemoteEvent")) then
            return false
        end
        if gun.Enabled == false
            or gun:GetAttribute("Disabled") == true
            or gun:GetAttribute("Reloading") == true
            or gun:GetAttribute("IsReloading") == true then
            return false
        end
        local originCF: CFrame = (type(getGunOriginCFrame) == "function"
            and getGunOriginCFrame(character, gun))
            or localRoot.CFrame
        local endpoint: Vector3 = computeTargetEndpoint(
            target,
            targetRoot,
            originCF.Position,
            "Bullet"
        )
        if settings.wallCheck and type(isWeaponPathClear) == "function" then
            local okClear: boolean, clear: any = pcall(
                isWeaponPathClear,
                originCF.Position,
                endpoint,
                target.Character
            )
            if not okClear or clear ~= true then
                return false
            end
        end
        local firedCore: boolean = false
        if type(state.mm2FireGunShot) == "function" then
            local okShot: boolean, res: any = pcall(state.mm2FireGunShot, target, {
                silent = false,
                wallCheck = settings.wallCheck,
                continuous = true,
                quiet = true,
            })
            firedCore = okShot and res == true
            -- Respect a refusal or uncertain send from the shared path.
            if not okShot then lastShotSentAt = os.clock() end
            if not firedCore then return false end
        end
        if not firedCore then
            local shotOrigin: CFrame = CFrame.lookAt(originCF.Position, endpoint)
            local shotEnd: CFrame = CFrame.new(endpoint)
            local okFire: boolean = pcall(function(): ()
                (shootRemote :: RemoteEvent):FireServer(shotOrigin, shotEnd)
            end)
            if not okFire then
                lastShotSentAt = os.clock()
                return false
            end
        end
        lastShotSentAt = os.clock()
        return true
    end

    -- Shared helper for Auto Win ("Insta Kill As Sheriff (Usa AutoShoot CFrame)")
    -- and Auto Shoot CFrame mode: equips the gun the instant the player
    -- receives it (whether as Sheriff, Hero, or Innocent who picked up GunDrop)
    -- and fires Silent AIM CFrame continuously until the Murderer is dead.
    local function executeAutoShootCFrameOnce(enforceProximity: boolean): boolean
        local gun: Tool?, character: Model?, humanoid: Humanoid?,
            localRoot: BasePart? = findLocalGun()
        if not gun or not character or not humanoid or not localRoot then
            lastShotSentAt = 0
            return false
        end
        local target: Player?, targetRoot: BasePart? =
            resolveMurdererTarget(localRoot, enforceProximity)
        if not target or not targetRoot then
            return false
        end
        if not ensureGunEquipped(character, humanoid, gun) then
            return false
        end
        if os.clock() - lastShotSentAt < AUTO_SHOT_PACE_SECONDS then
            return true
        end
        return fireCFrameShotImmediate(target, targetRoot, character, localRoot, gun)
    end
    state.mm2AutoShootCFrameOnce = executeAutoShootCFrameOnce

    local function autoShootTick(): ()
        if not autoShootActive then
            return
        end
        local gun: Tool?, character: Model?, humanoid: Humanoid?,
            localRoot: BasePart? = findLocalGun()
        if not gun or not character or not humanoid or not localRoot then
            lastShotSentAt = 0
            return
        end
        local target: Player?, targetRoot: BasePart? =
            resolveMurdererTarget(localRoot, settings.proximity == true)
        if not target or not targetRoot then
            return
        end
        if not ensureGunEquipped(character, humanoid, gun) then
            return
        end
        if os.clock() - lastShotSentAt < AUTO_SHOT_PACE_SECONDS then
            return
        end
        local isCFrame: boolean = settings.mode == "CFrame"
            or settings.mode == "Silent AIM"
        if settings.mode == "Experimental" then
            -- Share Gun Aimbot's selector and safety path, but use this card's
            -- own WallCheck/Proximity. Never send both on an uncertain result.
            local mode: string = "Bullet"
            if type(state.mm2ChooseExperimentalAim) == "function" then
                local origin: CFrame = (type(getGunOriginCFrame) == "function"
                    and getGunOriginCFrame(character, gun)) or localRoot.CFrame
                local okMode: boolean, chosen: any = pcall(state.mm2ChooseExperimentalAim, target, origin.Position)
                if okMode and chosen == "CFrame" then mode = "CFrame" end
            end
            local sent: boolean = false
            local uncertain: boolean = false
            if type(state.mm2FireGunShot) == "function" then
                local ok: boolean, result: any = pcall(state.mm2FireGunShot, target, {
                    silent = mode == "CFrame", wallCheck = settings.wallCheck,
                    continuous = true, quiet = true,
                })
                sent = ok and result == true
                -- A thrown error can happen after dispatch; pace rather than
                -- repeatedly sending on every frame while the result is unknown.
                uncertain = not ok
                if uncertain then lastShotSentAt = os.clock() end
            end
            if sent then lastShotSentAt = os.clock() end
            if AutoShootFeature then AutoShootFeature:SetStatus("Experimental · " .. mode
                .. (sent and " · request sent" or (uncertain and " · send uncertain" or " · not sent"))) end
        elseif isCFrame then
            fireCFrameShotImmediate(target, targetRoot, character, localRoot, gun)
        else
            fireBulletShotImmediate(target, targetRoot, character, localRoot, gun)
        end
    end

    local function watchToolContainer(container: Instance?): ()
        if not container or type((container :: any).ChildAdded) ~= "table"
            and typeof((container :: any).ChildAdded) ~= "RBXScriptSignal" then
            return
        end
        pcall(function(): ()
            local conn: RBXScriptConnection = (container :: any).ChildAdded:Connect(
                function(child: Instance): ()
                    if autoShootActive and child and child.Name == "Gun" then
                        autoShootTick()
                    end
                end
            )
            table.insert(toolConnections, conn)
        end)
    end

    local function setupInstantGunWatchers(): ()
        clearToolConnections()
        if not LocalPlayer then
            return
        end
        watchToolContainer(LocalPlayer.Character)
        watchToolContainer(LocalPlayer:FindFirstChildOfClass("Backpack"))
        pcall(function(): ()
            if LocalPlayer.CharacterAdded then
                table.insert(
                    toolConnections,
                    LocalPlayer.CharacterAdded:Connect(function(newChar: Model): ()
                        watchToolContainer(newChar)
                        watchToolContainer(LocalPlayer:FindFirstChildOfClass("Backpack"))
                    end)
                )
            end
            if LocalPlayer.ChildAdded then
                table.insert(
                    toolConnections,
                    LocalPlayer.ChildAdded:Connect(function(child: Instance): ()
                        if child and child:IsA("Backpack") then
                            watchToolContainer(child)
                        end
                    end)
                )
            end
        end)
    end

    local function updateStatus(): ()
        -- The feature's UI flag is flipped after its callback returns, so use
        -- our controller state during the initial enable as well.
        if not AutoShootFeature or not autoShootActive then
            return
        end
        if settings.proximity then
            AutoShootFeature:SetStatus(
                settings.mode .. " · " .. tostring(settings.aura) .. " studs"
            )
        else
            AutoShootFeature:SetStatus(settings.mode)
        end
    end

    -- v139: this card and the MM2 "Shoot" card both used the key MM2AutoShoot,
    -- so enabling both lost one handle and left a ghost task firing every frame
    -- after the card was switched off. Each card owns its own key now, and the
    -- connection is dropped before a new one is made.
    local CONNECTION_KEY: string = "MM2AutoShootCard"

    local function toggleAutoShoot(enabled: boolean): ()
        autoShootActive = enabled == true
        if enabled then
            updateStatus()
            setupInstantGunWatchers()
            disconnectFeatureConnection(CONNECTION_KEY)
            featureConnections[CONNECTION_KEY] = TaskManager:Connect(
                autoShootTick,
                nil,
                "Auto Shoot"
            )
            autoShootTick()
        else
            clearToolConnections()
            disconnectFeatureConnection(CONNECTION_KEY)
            if AutoShootFeature then
                AutoShootFeature:SetStatus(nil)
            end
        end
    end

    AutoShootFeature = createUniversalFeature(
        "Auto Shoot",
        "Automatically equips the Gun the instant you receive it (as Sheriff or Hero) and shoots the Murderer continuously using Bullet (respects WallCheck) or CFrame (Silent AIM). Optional Proximity radius filter.",
        4,
        toggleAutoShoot,
        {
            categoryName = "Utilities",
            parent = MM2Scroll,
            registry = mm2Features,
        }
    )
    state.mm2AutoShootFeature = AutoShootFeature

    local auraSlider: GuiObject? = nil
    local states: any = type(configData) == "table" and configData.states or {}
    local values: any = type(configData) == "table" and configData.values or {}
    local initialMode: string = values["MM2.AutoShoot.Mode"] or "Bullet"
    if initialMode ~= "CFrame" and initialMode ~= "Experimental" then initialMode = "Bullet" end
    local function modeConfig(mode: string): any
```

```
        local prefix: string = "MM2.AutoShoot." .. mode .. "."
        local legacy: boolean = mode == initialMode
        local wall: any = states[prefix .. "WallCheck"]
        local proximity: any = states[prefix .. "Proximity"]
        local aura: any = values[prefix .. "Aura"]
        if wall == nil and legacy then wall = states["MM2.AutoShoot.WallCheck"] end
        if proximity == nil and legacy then proximity = states["MM2.AutoShoot.Proximity"] end
        if aura == nil and legacy then aura = values["MM2.AutoShoot.Aura"] end
        return {wallCheck = wall == nil or wall == true, proximity = proximity == true,
            aura = type(aura) == "number" and aura or 25}
    end
    local modeConfigs: any = {Bullet = modeConfig("Bullet"), CFrame = modeConfig("CFrame"),
        Experimental = modeConfig("Experimental")}
    local switching: boolean = false
    local optionsReady: boolean = false
    local function refreshProximityOption(): ()
        if auraSlider and type(setOptionVisible) == "function" then
            -- addNumberOption returns the option row itself, not its slider.
            -- Hiding its parent hides every option in the settings window.
            setOptionVisible(AutoShootFeature, auraSlider :: GuiObject, settings.proximity == true)
        end
        updateStatus()
    end
    local function syncMode(): ()
        local cfg: any = modeConfigs[settings.mode] or modeConfigs.Bullet
        switching = true
        settings.wallCheck, settings.proximity, settings.aura = cfg.wallCheck, cfg.proximity, cfg.aura
        if state and type(state.toggleControls) == "table" then
            local wc: any = state.toggleControls["MM2.AutoShoot.WallCheck"]
            local px: any = state.toggleControls["MM2.AutoShoot.Proximity"]
            if wc and wc.setChecked then pcall(wc.setChecked, cfg.wallCheck) end
            if px and px.setChecked then pcall(px.setChecked, cfg.proximity) end
        end
        if state and type(state.valueControls) == "table" then
            local au: any = state.valueControls["MM2.AutoShoot.Aura"]
            if au and au.apply then pcall(au.apply, cfg.aura) end
        end
        switching = false
        refreshProximityOption()
    end
    addCycleOption(AutoShootFeature, "Mode", {"Bullet", "CFrame", "Experimental"}, 1,
        function(value: string): () settings.mode = value; syncMode() end,
        "Experimental selects one path based on range and movement; the heuristic does not guarantee hits. Options are independent per mode.")
    addToggleOption(AutoShootFeature, "WallCheck", true, function(value: boolean): ()
        settings.wallCheck = value == true
        if optionsReady and not switching then
            modeConfigs[settings.mode].wallCheck = settings.wallCheck
            states["MM2.AutoShoot." .. settings.mode .. ".WallCheck"] = settings.wallCheck
        end
    end, "Check for walls; saved independently per mode.")
    addToggleOption(AutoShootFeature, "Proximity", false, function(value: boolean): ()
        settings.proximity = value == true
        if optionsReady and not switching then
            modeConfigs[settings.mode].proximity = settings.proximity
            states["MM2.AutoShoot." .. settings.mode .. ".Proximity"] = settings.proximity
        end
        refreshProximityOption()
    end, "Only shoot inside Aura radius; saved independently per mode.")
    auraSlider = addNumberOption(AutoShootFeature, "Aura", 25, 5, 150, function(value: number): ()
        settings.aura = value
        if optionsReady and not switching then
            modeConfigs[settings.mode].aura = value
            values["MM2.AutoShoot." .. settings.mode .. ".Aura"] = value
        end
        updateStatus()
    end, "Maximum distance with Proximity enabled, saved per mode.", 1)
    optionsReady = true
    syncMode()

    activeCleanup = function(): ()
        toggleAutoShoot(false)
        state.mm2AutoShootFeature = nil
        state.mm2AutoShootCFrameOnce = nil
    end
    Module.Events = featureConnections
    Module.Initialized = true
    return Module
end

function Module.destroy(): ()
    if not Module.Initialized then
        return
    end
    Module.Initialized = false
    pcall(activeCleanup)
    activeCleanup = function(): () end
    Module.Events = {}
    Module.Runtime = nil
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/MM2/Combat/AutoShoot.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/games/MM2/Combat/KnifeThrow.lua
BYTES: 46831
SHA-256: 5453031b938da81fda3b6e9588f56ed2800314de77164ea84dd4fc50f00fea33
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
local Module = {
    Name = "MM2 Knife Throw",
    PlaceId = 142823291,
    Events = {} :: {[string]: any},
    Initialized = false,
    Runtime = nil :: any,
}

local activeCleanup: () -> () = function(): () end

function Module.init(runtime: any): any
    if Module.Initialized then
        return Module
    end
    local core: any = state.mm2Core
    assert(type(core) == "table", "MM2 Knife Throw requires the MM2 core module")
    Module.Runtime = runtime

    local findMurderer: any = core.findMurderer
    local getPlayerRole: any = core.getPlayerRole
    local getPlayerWeapon: any = core.getPlayerWeapon
    local isPlayerAlive: any = core.isPlayerAlive
    local isProtectedTarget: any = core.isProtectedTarget
    local getEstimatedLatency: any = core.getEstimatedLatency
    local isWeaponPathClear: any = core.isWeaponPathClear
    local prediction: any = state.mm2Prediction

    local settings = {
        enabled = false,
        autoThrow = false,
        wallCheck = true,
        knifeTracer = false,
        delay = 0.6,
    }

    local lastThrowAttempt: number = 0
    local KnifeThrowFeature: any = nil
    local lastKnifeStatus: string? = nil
    local function setKnifeStatus(message: string?): ()
        if KnifeThrowFeature and lastKnifeStatus ~= message then
            lastKnifeStatus = message
            KnifeThrowFeature:SetStatus(message)
        end
    end
    local tracerPart: BasePart? = nil
    local targetDotPart: BasePart? = nil
    local hookedWeaponService: any = nil
    local origGetMouseTargetCFrame: any = nil
    local origGetTargetPosition: any = nil
    local lastRedirectInstallAttempt: number = -math.huge

    local function destroyTracerVisuals(): ()
        if tracerPart then
            pcall(function(): ()
                tracerPart:Destroy()
            end)
            tracerPart = nil
        end
        if targetDotPart then
            pcall(function(): ()
                targetDotPart:Destroy()
            end)
            targetDotPart = nil
        end
    end

    -- Check if player is valid target
    local function isValidTarget(player: Player): boolean
        if not player or player == LocalPlayer then
            return false
        end
        if type(isProtectedTarget) == "function" and isProtectedTarget(player) then
            return false
        end
        if type(isPlayerAlive) == "function" and not isPlayerAlive(player) then
            return false
        end
        if type(getPlayerRole) == "function" and getPlayerRole(player) == "Murderer" then
            return false
        end
        local character: Model? = player.Character
        if not character then
            return false
        end
        local root: BasePart? = character:FindFirstChild("HumanoidRootPart") :: BasePart?
        local humanoid: Humanoid? = character:FindFirstChildOfClass("Humanoid")
        if not root or not humanoid or humanoid.Health <= 0 then
            return false
        end
        return true
    end

    -- Get equipped or carried knife when local player is Murderer
    local function getKnife(): Tool?
        local character: Model? = LocalPlayer and LocalPlayer.Character
        if not character then
            return nil
        end
        if type(getPlayerWeapon) == "function" then
            local equippedKnife: Tool? = getPlayerWeapon(LocalPlayer, "Knife", true)
            if equippedKnife then
                return equippedKnife
            end
            local anyKnife: Tool? = getPlayerWeapon(LocalPlayer, "Knife", false)
            if anyKnife then
                return anyKnife
            end
        end
        for _, item: Instance in ipairs(character:GetChildren()) do
            if item:IsA("Tool") then
                local name: string = item.Name:lower()
                if name:find("knife") or item:FindFirstChild("Events") then
                    local events: Instance? = item:FindFirstChild("Events")
                    if name:find("knife") or (events and events:FindFirstChild("KnifeThrown")) then
                        return item :: Tool
                    end
                end
            end
        end
        local backpack: Instance? = LocalPlayer:FindFirstChildOfClass("Backpack")
        if backpack then
            for _, item: Instance in ipairs(backpack:GetChildren()) do
                if item:IsA("Tool") then
                    local name: string = item.Name:lower()
                    local events: Instance? = item:FindFirstChild("Events")
                    if name:find("knife") or (events and events:FindFirstChild("KnifeThrown")) then
                        return item :: Tool
                    end
                end
            end
        end
        return nil
    end

    local originHandleTool: Tool? = nil
    local originHandle: BasePart? = nil
    local function getThrowOriginCFrame(knife: Tool?): CFrame?
        local character: Model? = LocalPlayer and LocalPlayer.Character
        if not character then
            return nil
        end
        if knife and knife.Parent == character then
            if originHandleTool ~= knife or not originHandle or originHandle.Parent ~= knife then
                local handle: Instance? = knife:FindFirstChild("Handle")
                originHandleTool = knife
                originHandle = handle and handle:IsA("BasePart") and (handle :: BasePart) or nil
            end
            if originHandle then return originHandle.CFrame end
        end
        local hand: Instance? = character:FindFirstChild("RightHand")
            or character:FindFirstChild("Right Arm")
            or character:FindFirstChild("HumanoidRootPart")
        if hand and hand:IsA("BasePart") then
            return (hand :: BasePart).CFrame
        end
        return nil
    end

    -- v159: WallCheck is now the only guarantee, so it has to be honest:
    --  * the shared helper is called with `strict`, which makes a fully
    --    transparent but SOLID barrier stop the blade. The gun's first pass
    --    skips those; the knife must not.
    --  * the segment is tested as a thin corridor, not one mathematical line,
    --    so a railing edge cannot slip between samples.
    --  * the body has to be visible from the hand RIGHT NOW, not only at the
    --    predicted point. A predicted point that peeks around a corner is not
    --    reachable by a straight blade, and aiming at it is what looked like
    --    "knife through the wall".
    local CORRIDOR_STUDS: number = 0.45
    local function raycastClear(
        from: Vector3,
        to: Vector3,
        ignoreList: {Instance},
        requireBody: Model?
    ): boolean
        local ok: boolean, clear: any = pcall(function(): boolean
            local params: RaycastParams = RaycastParams.new()
            params.FilterType = Enum.RaycastFilterType.Exclude
            params.FilterDescendantsInstances = ignoreList
            params.RespectCanCollide = true
            params.IgnoreWater = true
            local hit: RaycastResult? = workspace:Raycast(from, to - from, params)
            if requireBody then
                return hit ~= nil and hit.Instance:IsDescendantOf(requireBody)
            end
            return hit == nil
        end)
        return ok and clear == true
    end

    local function sharedPathClear(
        from: Vector3,
        to: Vector3,
        targetChar: Model?,
        requireBody: boolean
    ): boolean?
        if type(isWeaponPathClear) ~= "function" or not targetChar then
            return nil
        end
        local options: any = requireBody and { requireTarget = true } or { ignoreTarget = true }
        local ok: boolean, clear: any = pcall(isWeaponPathClear, from, to, targetChar, true, options)
        if not ok or type(clear) ~= "boolean" then
            return nil
        end
        return clear
    end

    local function ignoreListFor(targetChar: Model?): {Instance}
        local list: {Instance} = {}
        local localChar: Model? = LocalPlayer and LocalPlayer.Character
        if localChar then table.insert(list, localChar) end
        if targetChar then table.insert(list, targetChar) end
        return list
    end

    -- "I can see that body from the hand": the ray must END on the body.
    local function bodyVisible(originPos: Vector3, bodyPos: Vector3, targetChar: Model?): boolean
        if not settings.wallCheck then
            return true
        end
        local from: Vector3 = originPos
        local to: Vector3 = bodyPos
        if (to - from).Magnitude < 0.05 then
            return true
        end
        local shared: boolean? = sharedPathClear(from, to, targetChar, true)
        if shared ~= nil then
            return shared
        end
        return raycastClear(from, to, ignoreListFor(nil), targetChar)
    end

    -- "Nothing solid between the hand and that point": corridor of five rays.
    local function checkLineOfSight(originPos: Vector3, targetPos: Vector3, targetChar: Model?): boolean
        if not settings.wallCheck then
            return true
        end
        local direction: Vector3 = targetPos - originPos
        local length: number = direction.Magnitude
        if length < 0.05 then
            return true
        end
        local unit: Vector3 = direction / length
        local side: Vector3 = unit:Cross(Vector3.new(0, 1, 0))
        if side.Magnitude < 0.05 then
            side = Vector3.new(1, 0, 0)
        end
        side = side.Unit
        local up: Vector3 = side:Cross(unit).Unit
        local offsets: {Vector3} = {
            Vector3.zero,
            side * CORRIDOR_STUDS,
            side * -CORRIDOR_STUDS,
            up * CORRIDOR_STUDS,
            up * -CORRIDOR_STUDS,
        }
        for _index: number, offset: Vector3 in ipairs(offsets) do
            local from: Vector3 = originPos + offset
            local to: Vector3 = targetPos + offset
            -- Every corridor ray uses the same strict rule as the centre one.
            -- A raw raycast here would let an invisible decoration that the
            -- shared helper skips block the throw, which made WallCheck say no
            -- for geometry the blade would have flown through.
            local shared: boolean? = sharedPathClear(from, to, targetChar, false)
            if shared ~= nil then
                if not shared then
                    return false
                end
            elseif not raycastClear(from, to, ignoreListFor(targetChar), nil) then
                return false
            end
        end
        return true
    end

    -- Predict target intercept position for MM2 ThrowingKnife (96 studs/s linear projectile)
    -- Returns the point and the blade's flight time, which is what makes one
    -- target a sooner hit than another.
    -- `cheap` is for target SELECTION: it still requires the body to be
    -- visible, but skips the five-ray corridor, which is re-checked with the
    -- live origin right before the packet leaves.
    local function resolveKnifeAim(
        target: Player,
        originPos: Vector3,
        knife: Tool?,
        cheap: boolean?
    ): (Vector3?, number?)
        local character: Model? = target.Character
        if not character then
            return nil, nil
        end
        local root: BasePart? = character:FindFirstChild("HumanoidRootPart") :: BasePart?
        if not root then
            return nil, nil
        end
        -- Check the live body before doing the more expensive intercept/ground
        -- work; hidden candidates cannot become speculative shots.
        if not bodyVisible(originPos, root.Position, character) then
            return nil, nil
        end

        local speed: number = 96
        if knife then
            pcall(function(): ()
                local attrSpeed: any = knife:GetAttribute("ThrowSpeed")
                if type(attrSpeed) == "number" and attrSpeed >= 24 and attrSpeed <= 300 then
                    speed = attrSpeed
                end
            end)
        end
        local predicted: Vector3? = nil
        local aimInfo: any = nil
        if type(prediction) == "table" and type(prediction.knifeAim) == "function" then
            -- v139.4: the blade speed HAS to reach the predictor. It was read
            -- from the tool above and then only used by the local fallback, so
            -- every knife that is not the default 96 studs/s was solved with
            -- the wrong flight time even though the correct number was in
            -- hand.
            local okPred: boolean, aimPoint: any, info: any =
                pcall(prediction.knifeAim, target, originPos, speed)
            if okPred and typeof(aimPoint) == "Vector3" then
                predicted = aimPoint :: Vector3
                aimInfo = info
            end
        end

        if not predicted then
            local velocity: Vector3 = root.AssemblyLinearVelocity
            if velocity.X ~= velocity.X or velocity.Y ~= velocity.Y or velocity.Z ~= velocity.Z
                or math.abs(velocity.X) == math.huge or math.abs(velocity.Y) == math.huge
                or math.abs(velocity.Z) == math.huge then
                velocity = Vector3.zero
            end
            local latency: number = 0.06
            if type(getEstimatedLatency) == "function" then
                local okLat: boolean, l: any = pcall(getEstimatedLatency)
                if okLat and type(l) == "number" and l == l then
                    latency = math.clamp(l, 0, 0.5)
                end
            end
            -- This is only the fallback when the shared predictor is absent or
            -- cannot solve. Previously `velocity * latency` applied unbounded
            -- vertical lead BEFORE the later 3.5 stud drop clamp: a replicated
            -- -120 Y velocity at 0.5 s already aimed 60 studs below the body.
            -- Keep horizontal transport and cap the *entire* vertical lead.
            -- The body's actual height is never clamped against the thrower:
            -- an enemy on a lower floor still produces a downward shot.
            local horizVel: Vector3 = Vector3.new(velocity.X, 0, velocity.Z)
            local observed: Vector3 = root.Position + horizVel * latency
            local rel: Vector3 = observed - originPos
            local a: number = horizVel:Dot(horizVel) - speed * speed
            local b: number = 2 * rel:Dot(horizVel)
            local c: number = rel:Dot(rel)
            local flight: number? = nil
            if c < 0.0025 then
                flight = 0
            elseif math.abs(a) < 1e-6 then
                if b < -1e-6 then
                    flight = -c / b
                end
            else
                local disc: number = b * b - 4 * a * c
                if disc >= 0 then
                    local sqrtDisc: number = math.sqrt(disc)
                    local t1: number = (-b - sqrtDisc) / (2 * a)
                    local t2: number = (-b + sqrtDisc) / (2 * a)
                    if t1 > 0 then
                        flight = t1
                    end
                    if t2 > 0 and (not flight or t2 < flight) then
                        flight = t2
                    end
                end
            end
            if flight and flight == flight then
                flight = math.clamp(flight, 0, 1.5)
                local yLead: number = math.clamp(
                    velocity.Y * math.min(latency + flight, 0.28), -2.0, 4.0
                )
                predicted = observed + horizVel * flight + Vector3.new(0, yLead, 0)
            else
                local yLead: number = math.clamp(velocity.Y * math.min(latency, 0.28), -2.0, 4.0)
                predicted = observed + Vector3.new(0, yLead, 0)
            end
        end

        if not predicted then
            return nil, nil
        end

        local flight: number? = nil
        if aimInfo and type(aimInfo.flightSeconds) == "number" then
            flight = math.max(0, aimInfo.flightSeconds)
        else
            flight = (predicted - originPos).Magnitude / speed
        end

        -- The current body was already checked before prediction: a future
        -- point peeking around a corner cannot make a hidden body eligible.
        if cheap or checkLineOfSight(originPos, predicted, character) then
            return predicted, flight
        end

        -- Fallback visibility scan across torso/head/limbs if WallCheck blocks primary point
        --
        -- v139.4: only the TIME part of the predictor's displacement may be
        -- reused on another body part. `predicted - root.Position` is lead
        -- PLUS the torso/head offset the predictor itself sampled
        -- (`capturedPosition = last.position + aimOffset`), so adding it to
        -- UpperTorso or Head counted the torso height twice and aimed 2-4
        -- studs above the part it named. Using the predictor's own reference
        -- point keeps the lead and drops the duplicated height.
        local leadOffset: Vector3 = predicted - root.Position
        if aimInfo and typeof(aimInfo.capturedPosition) == "Vector3" then
            leadOffset = predicted - (aimInfo.capturedPosition :: Vector3)
        end
        local candidateNames: {string} = {
            "UpperTorso", "Torso", "HumanoidRootPart", "Head",
            "RightUpperArm", "LeftUpperArm", "Right Arm", "Left Arm",
        }
        for _, partName: string in ipairs(candidateNames) do
            local part: Instance? = character:FindFirstChild(partName)
            if part and part:IsA("BasePart") then
                local partPosition: Vector3 = (part :: BasePart).Position
                local candidatePoint: Vector3 = partPosition + leadOffset
                if bodyVisible(originPos, partPosition, character)
                    and (cheap or checkLineOfSight(originPos, candidatePoint, character)) then
                    return candidatePoint, flight
                end
            end
        end

        return nil, nil
    end

    -- Find best target for knife throw
    local function findTarget(): Player?
        local localChar: Model? = LocalPlayer and LocalPlayer.Character
        if not localChar then
            return nil
        end
        local localRoot: BasePart? = localChar:FindFirstChild("HumanoidRootPart") :: BasePart?
        if not localRoot then
            return nil
        end
        local knife: Tool? = getKnife()
        local originCF: CFrame? = getThrowOriginCFrame(knife)
        local originPos: Vector3 = originCF and originCF.Position or localRoot.Position

        local bestTarget: Player? = nil
        local bestScore: number = math.huge

        for _, player: Player in ipairs(Players:GetPlayers()) do
            if isValidTarget(player) then
                local targetChar: Model? = player.Character
                local targetRoot: BasePart? = targetChar
                    and targetChar:FindFirstChild("HumanoidRootPart") :: BasePart?
                if targetRoot then
                    local distance: number = (targetRoot.Position - originPos).Magnitude
                    if distance <= 250 then
                        local aimPoint: Vector3?, flight: number? =
                            resolveKnifeAim(player, originPos, knife, true)
                        if aimPoint then
                            -- v159: score by the blade's FLIGHT TIME, not raw
                            -- distance. The nearest body is not the soonest hit:
                            -- a target running towards the throw intercepts in
                            -- a fraction of the time a farther one standing
                            -- still does, and the sooner the blade lands, the
                            -- smaller every prediction error is.
                            local score: number = flight or (distance / 96)
                            if type(getPlayerRole) == "function" then
                                local role: string? = getPlayerRole(player)
                                if role == "Sheriff" or role == "Hero" then
                                    score -= 0.08
                                end
                            end
                            if score < bestScore then
                                bestScore = score
                                bestTarget = player
                            end
                        end
                    end
                end
            end
        end

        return bestTarget
    end

    -- The solve is throttled, but its result used to jump at each update. This is
    -- visual-only: the native redirects and throwKnife always solve separately
    -- and never read this smoothed point.
    local TRACER_SMOOTH_RATE: number = 26
    local TRACER_SNAP_DISTANCE: number = 20
    local tracerPoint: Vector3? = nil
    local tracerTarget: Player? = nil
    local tracerShown: boolean = false
    local tracerOpacity: number = 0
    local drawnOpacity: number = -1
    local drawnOrigin: Vector3? = nil
    local drawnPoint: Vector3? = nil

    local function hideTracerParts(): ()
        if tracerPart and tracerPart.Parent then tracerPart.Transparency = 1 end
        if targetDotPart and targetDotPart.Parent then targetDotPart.Transparency = 1 end
    end

    local function hideTracerVisuals(): ()
        -- Do not tween a stale point after the next visibility check fails.
        -- Reuse the two invisible parts if a target reappears; destroy them
        -- when the option/module is turned off.
        if tracerShown then pcall(hideTracerParts) end
        tracerShown = false
        tracerOpacity = 0
        drawnOpacity = -1
        tracerPoint = nil
        tracerTarget = nil
        drawnOrigin = nil
        drawnPoint = nil
    end

    local function drawTracerParts(
        originPos: Vector3, point: Vector3, delta: Vector3, dist: number,
        updateGeometry: boolean, opacity: number, updateOpacity: boolean
    ): ()
        if not tracerPart or not tracerPart.Parent then
            local beamPart: Part = Instance.new("Part")
            beamPart.Name = "WurstKnifeTracer"
            beamPart.Anchored = true
            beamPart.CanCollide = false
            beamPart.CanQuery = false
            beamPart.CanTouch = false
            beamPart.Material = Enum.Material.Neon
            beamPart.Color = Color3.fromRGB(255, 65, 65)
            beamPart.Transparency = 1
            beamPart.Parent = workspace
            tracerPart = beamPart
            tracerShown = false
        end
        if not targetDotPart or not targetDotPart.Parent then
            local dot: Part = Instance.new("Part")
            dot.Name = "WurstKnifeAimDot"
            dot.Shape = Enum.PartType.Ball
            dot.Size = Vector3.new(1.1, 1.1, 1.1)
            dot.Anchored = true
            dot.CanCollide = false
            dot.CanQuery = false
            dot.CanTouch = false
            dot.Material = Enum.Material.Neon
            dot.Color = Color3.fromRGB(255, 220, 60)
            dot.Transparency = 1
            dot.Parent = workspace
            targetDotPart = dot
            tracerShown = false
        end
        if updateGeometry then
            tracerPart.Size = Vector3.new(0.14, 0.14, dist)
            tracerPart.CFrame = CFrame.lookAt(originPos + delta * 0.5, point)
            targetDotPart.CFrame = CFrame.new(point)
        end
        if updateOpacity or not tracerShown then
```

```
            tracerPart.Transparency = 1 - 0.75 * opacity
            targetDotPart.Transparency = 1 - 0.85 * opacity
            tracerShown = true
        end
    end

    local function updateTracerVisual(
        originPos: Vector3?, aimPos: Vector3?, target: Player?, deltaTime: number?
    ): ()
        if not settings.enabled or not settings.knifeTracer then
            hideTracerVisuals()
            destroyTracerVisuals()
            return
        end
        if not originPos or not aimPos or not target then
            hideTracerVisuals()
            return
        end
        local dt: number = tonumber(deltaTime) or (1 / 60)
        if dt ~= dt then dt = 1 / 60 end
        local alpha: number = 1 - math.exp(-TRACER_SMOOTH_RATE * math.clamp(dt, 0, 0.25))
        local previous: Vector3? = tracerPoint
        if not previous or tracerTarget ~= target
            or (aimPos - previous).Magnitude > TRACER_SNAP_DISTANCE then
            -- Never sweep a line across different players or a teleport. A
            -- short fade-in still softens the appearance of the new point.
            tracerPoint = aimPos
            tracerOpacity = 0
        else
            local nextPoint: Vector3 = previous + (aimPos - previous) * alpha
            tracerPoint = (aimPos - nextPoint).Magnitude < 0.002 and aimPos or nextPoint
        end
        tracerOpacity += (1 - tracerOpacity) * alpha
        if tracerOpacity > 0.998 then tracerOpacity = 1 end
        tracerTarget = target
        local point: Vector3 = tracerPoint :: Vector3
        local delta: Vector3 = point - originPos
        local dist: number = delta.Magnitude
        if dist < 0.25 or dist > 400 then
            hideTracerVisuals()
            return
        end
        local missing: boolean = not tracerPart or not tracerPart.Parent
            or not targetDotPart or not targetDotPart.Parent
        local geometryChanged: boolean = missing or not drawnOrigin or not drawnPoint
            or (originPos - (drawnOrigin :: Vector3)).Magnitude >= 0.002
            or (point - (drawnPoint :: Vector3)).Magnitude >= 0.002
        local opacityChanged: boolean = missing or not tracerShown
            or math.abs(tracerOpacity - drawnOpacity) >= 0.002
        if not geometryChanged and not opacityChanged then return end
        -- Named callback: unlike the old per-frame pcall(closure), this does
        -- not allocate a fresh closure every frame. No raycasts/tweens here.
        local ok: boolean = pcall(
            drawTracerParts, originPos, point, delta, dist,
            geometryChanged, tracerOpacity, opacityChanged
        )
        if ok then
            if geometryChanged then
                drawnOrigin = originPos
                drawnPoint = point
            end
            if opacityChanged then drawnOpacity = tracerOpacity end
        else
            pcall(hideTracerParts)
            tracerShown = false
            drawnOpacity = -1
            drawnOrigin = nil
            drawnPoint = nil
        end
    end

    -- Throw knife at predicted target using MM2's Knife.Events.KnifeThrown remote
    -- v139.2: the equip is a round trip, the throw is not. Firing the remote
    -- in the same frame as EquipTool sends a throw from a tool the server still
    -- believes is in the backpack: in the saved session 13 auto throws produced
    -- 9 observed blades (4 vanished) and the 4 that did fire used the RightHand
    -- fallback origin instead of the knife handle. So: equip once, wait for the
    -- tool to actually be in the character, and only then throw.
    local equipRequestedAt: number = -math.huge
    local equipRequestedTool: Tool? = nil

    local function ensureEquipped(knife: Tool, humanoid: Humanoid): boolean
        local character: Model? = LocalPlayer and LocalPlayer.Character
        if not character then
            return false
        end
        if knife.Parent == character then
            equipRequestedAt = -math.huge
            equipRequestedTool = nil
            return true
        end
        local now: number = os.clock()
        -- Re-ask every 0.25 s at most; EquipTool while the previous request is
        -- still travelling just stacks another equip request.
        if equipRequestedTool ~= knife or now - equipRequestedAt >= 0.25 then
            equipRequestedAt = now
            equipRequestedTool = knife
            pcall(function(): ()
                humanoid:EquipTool(knife)
            end)
        end
        return false
    end

    local function throwKnife(target: Player?, knife: Tool): (boolean, string?)
        local character: Model? = LocalPlayer and LocalPlayer.Character
        if not character then
            return false, "Character unavailable"
        end
        local humanoid: Humanoid? = character:FindFirstChildOfClass("Humanoid")
        if not humanoid or humanoid.Health <= 0 then
            return false, "Character unavailable"
        end
        if not ensureEquipped(knife, humanoid) then
            return false, "Equipping knife…"
        end
        local originCF: CFrame? = getThrowOriginCFrame(knife)
        if not originCF then
            return false, "No throw origin"
        end
        -- v159: no lane fallback at all. Without a visible target there is
        -- nothing to throw at: a blade is not a homing projectile, and a throw
        -- into an opening only spent the knife.
        if not target or not isValidTarget(target) then
            return false, "No visible target"
        end
        local aimPos: Vector3? = resolveKnifeAim(target, originCF.Position, knife)
        if not aimPos then
            return false, "Target blocked · WallCheck"
        end

        local fired: boolean = false
        local noted: boolean = false
        -- v186: every early return below now names its gate. The throw used
        -- to fail silently after the loop had already announced
        -- "Predicted target", so a knife that never left looked identical
        -- to one the server had refused.
        local function noteThrowAttempt(): ()
            if noted then return end
            noted = true
            local recorder: any = core and core.trajectoryCalibration
            if recorder and type(recorder.noteAuthoredShot) == "function" then
                local root: BasePart? = target.Character
                    and target.Character:FindFirstChild("HumanoidRootPart") :: BasePart?
                pcall(recorder.noteAuthoredShot, recorder, {
                    kind = "knife", tool = knife, toolName = knife.Name,
                    originPos = originCF.Position, aimPos = aimPos, target = target,
                    aimVelocity = root and root.AssemblyLinearVelocity or nil,
                    source = "menu-auto-throw",
                    settings = {mode = "Auto Knife Throw", wallCheck = settings.wallCheck},
                })
            end
        end
        local eventsFolder: Instance? = knife:FindFirstChild("Events")
        local knifeThrownRemote: Instance? = eventsFolder and eventsFolder:FindFirstChild("KnifeThrown")
        if knifeThrownRemote and knifeThrownRemote:IsA("RemoteEvent") then
            noteThrowAttempt()
            local okFire: boolean = pcall(function(): ()
                (knifeThrownRemote :: RemoteEvent):FireServer(originCF, CFrame.new(aimPos))
            end)
            if okFire then
                fired = true
            end
        end

        if not fired then
            pcall(function(): ()
                local rs: Instance? = game:GetService("ReplicatedStorage")
                local remotes: Instance? = rs and rs:FindFirstChild("Remotes")
                local throwRemote: Instance? = remotes and remotes:FindFirstChild("Event")
                if throwRemote and throwRemote:IsA("RemoteEvent") then
                    noteThrowAttempt()
                    local dir: Vector3 = (aimPos - originCF.Position).Unit
                    local rem: RemoteEvent = throwRemote :: RemoteEvent
                    rem:FireServer("Throw", aimPos, dir)
                    fired = true
                end
            end)
        end

        if not fired then
            return false, "KnifeThrown remote unavailable"
        end
        return true, nil
    end

    -- Hook ReplicatedStorage.ClientServices.WeaponService so native PC & Mobile
    -- knife throws (Mouse, Keyboard E, Gamepad, Mobile TouchControls.RightBar.Throw)
    -- automatically redirect to the Knife Predictor's aim point!
    local function installWeaponServiceRedirect(): ()
        if hookedWeaponService then
            return
        end
        pcall(function(): ()
            local rs: Instance? = game:GetService("ReplicatedStorage")
            local clientServices: Instance? = rs and rs:FindFirstChild("ClientServices")
            local wsModule: Instance? = clientServices and clientServices:FindFirstChild("WeaponService")
            if not wsModule then
                return
            end
            local okReq: boolean, wsTable: any = pcall(require, wsModule)
            if not okReq or type(wsTable) ~= "table" then
                return
            end
            origGetMouseTargetCFrame = wsTable.GetMouseTargetCFrame
            origGetTargetPosition = wsTable.GetTargetPosition
            hookedWeaponService = wsTable

            if type(origGetMouseTargetCFrame) == "function" then
                wsTable.GetMouseTargetCFrame = function(self: any, ...: any): any
                    if settings.enabled then
                        local knife: Tool? = getKnife()
                        local character: Model? = LocalPlayer and LocalPlayer.Character
                        if knife and character and knife.Parent == character then
                            local originCF: CFrame? = getThrowOriginCFrame(knife)
                            local target: Player? = findTarget()
                            if originCF and target then
                                local aim: Vector3? = resolveKnifeAim(target, originCF.Position, knife)
                                if aim then return CFrame.new(aim) end
                            end
                        end
                    end
                    return origGetMouseTargetCFrame(self, ...)
                end
            end

            if type(origGetTargetPosition) == "function" then
                wsTable.GetTargetPosition = function(self: any, ...: any): any
                    if settings.enabled then
                        local knife: Tool? = getKnife()
                        local character: Model? = LocalPlayer and LocalPlayer.Character
                        if knife and character and knife.Parent == character then
                            local originCF: CFrame? = getThrowOriginCFrame(knife)
                            local target: Player? = findTarget()
                            if originCF and target then
                                local aim: Vector3? = resolveKnifeAim(target, originCF.Position, knife)
                                -- The captured MM2 WeaponService returns a CFrame here,
                                -- despite the method name. Preserve that native contract
                                -- just as GetMouseTargetCFrame does.
                                if aim then return CFrame.new(aim) end
                            end
                        end
                    end
                    return origGetTargetPosition(self, ...)
                end
            end
        end)
    end

    local function uninstallWeaponServiceRedirect(): ()
        if not hookedWeaponService then
            return
        end
        pcall(function(): ()
            if origGetMouseTargetCFrame then
                hookedWeaponService.GetMouseTargetCFrame = origGetMouseTargetCFrame
            end
            if origGetTargetPosition then
                hookedWeaponService.GetTargetPosition = origGetTargetPosition
            end
        end)
        hookedWeaponService = nil
        origGetMouseTargetCFrame = nil
        origGetTargetPosition = nil
    end

    -- v139.2: this loop used to run every frame and, for EVERY player in the
    -- lobby, ran the aim solver -- and the aim solver casts line-of-sight rays
    -- against the map, then up to 8 more per body part when the first one is
    -- blocked. Ten players meant dozens of raycasts per frame for a number
    -- that only changes a few times per second. The solve is now cached for
    -- 1/15 s when an automatic throw is due; a visual-only preview can use
    -- 1/10 s because the displayed point follows the observed root between
    -- solves. The actual throw always re-solves from the live handle.
    local AIM_CACHE_SECONDS: number = 1 / 15
    local TRACER_CACHE_SECONDS: number = 1 / 10
    local cachedTarget: Player? = nil
    local cachedAim: Vector3? = nil
    local cachedOrigin: CFrame? = nil
    local cachedVisualRoot: BasePart? = nil
    local cachedVisualPosition: Vector3? = nil
    local cachedAt: number = -math.huge

    local function solveAim(knife: Tool, throwDue: boolean): (Player?, Vector3?, CFrame?)
        local now: number = os.clock()
        local interval: number = throwDue and AIM_CACHE_SECONDS or TRACER_CACHE_SECONDS
        if now - cachedAt >= interval then
            cachedAt = now
            cachedTarget = findTarget()
            cachedAim = nil
            cachedVisualRoot = nil
            cachedVisualPosition = nil
            cachedOrigin = getThrowOriginCFrame(knife)
            if cachedOrigin and cachedTarget then
                cachedAim = resolveKnifeAim(cachedTarget, cachedOrigin.Position, knife)
                if cachedAim and settings.knifeTracer then
                    local character: Model? = cachedTarget.Character
                    local root: BasePart? = character
                        and character:FindFirstChild("HumanoidRootPart") :: BasePart?
                    if root then
                        cachedVisualRoot = root
                        cachedVisualPosition = root.Position
                    end
                end
            end
        elseif cachedOrigin then
            -- Origin only: the target's predicted point stays cached, but the
            -- handle moves with the character and the tracer must follow it.
            cachedOrigin = getThrowOriginCFrame(knife) or cachedOrigin
        end
        return cachedTarget, cachedAim, cachedOrigin
    end

    local function invalidateAim(): ()
        cachedAt = -math.huge
        cachedTarget = nil
        cachedAim = nil
        cachedOrigin = nil
        cachedVisualRoot = nil
        cachedVisualPosition = nil
    end

    -- getPlayerWeapon walks the character/backpack's children. A tracer-only
    -- heartbeat must not do that 60-240 times a second just to keep the line
    -- attached to an already equipped handle. Actual throws and native input
    -- still use a fresh getKnife(), and parent changes invalidate this cache.
    local loopKnife: Tool? = nil
    local loopKnifeParent: Instance? = nil
    local loopKnifeCharacter: Model? = nil
    local loopKnifeAt: number = -math.huge
    local function resetLoopKnife(): ()
        loopKnife = nil
        loopKnifeParent = nil
        loopKnifeCharacter = nil
        loopKnifeAt = -math.huge
        originHandleTool = nil
        originHandle = nil
    end
    local function getLoopKnife(now: number, throwDue: boolean): Tool?
        local character: Model? = LocalPlayer and LocalPlayer.Character
        if character ~= loopKnifeCharacter then resetLoopKnife() end
        loopKnifeCharacter = character
        if not throwDue then
            if loopKnife and loopKnife.Parent == character and character then
                return loopKnife
            end
            local age: number = now - loopKnifeAt
            if age >= 0 and age < TRACER_CACHE_SECONDS then
                if not loopKnife then return nil end
                if loopKnifeParent and loopKnife.Parent == loopKnifeParent then
                    return loopKnife
                end
            end
        end
        loopKnife = getKnife()
        loopKnifeParent = loopKnife and loopKnife.Parent or nil
        loopKnifeAt = now
        return loopKnife
    end

    local function tracerGoal(target: Player?, aimPos: Vector3?): Vector3?
        if not target or not aimPos then return nil end
        local root: BasePart? = cachedVisualRoot
        local captured: Vector3? = cachedVisualPosition
        if not root or not captured then return aimPos end
        if target ~= cachedTarget or root.Parent ~= target.Character then
            invalidateAim()
            return nil
        end
        -- Follow the already-observed root between the throttled solves. This is
        -- only a visual offset, not another prediction or a per-frame raycast.
        local moved: Vector3 = root.Position - captured
        if moved.Magnitude > 8 then
            invalidateAim()
            return nil
        end
        return aimPos + moved
    end

    -- Main loop for Knife Tracer and Auto Knife Throw
    local function knifeThrowLoop(deltaTime: number?): ()
        if not settings.enabled then
            setKnifeStatus(nil)
            hideTracerVisuals()
            destroyTracerVisuals()
            invalidateAim()
            return
        end

        local now: number = os.clock()
        -- v186: WeaponService may not exist yet when the card is switched on
        -- (lobby, round transition). The install used to run only inside the
        -- toggle, so in that case the native-throw redirect never arrived
        -- for the rest of the session. Retry cheaply while enabled.
        if not hookedWeaponService and now - lastRedirectInstallAttempt >= 1 then
            lastRedirectInstallAttempt = now
            installWeaponServiceRedirect()
        end
        local cooldown: number = math.clamp(tonumber(settings.delay) or 0, 0, 10)
        local wantThrow: boolean = settings.autoThrow
            and now - lastThrowAttempt >= cooldown

        -- Manual redirects solve on the native input path. Do not enumerate
        -- players, predict and raycast merely to update a status when both
        -- Auto Knife Throw and the tracer are off. Likewise, without a tracer
        -- no preview is needed during a successful throw's cooldown.
        if not settings.knifeTracer and not wantThrow then
            hideTracerVisuals()
            destroyTracerVisuals()
            if not settings.autoThrow then
                setKnifeStatus(nil)
                invalidateAim()
            end
            return
        end

        local knife: Tool? = getLoopKnife(now, wantThrow)
        if not knife then
            setKnifeStatus("Waiting for knife")
            hideTracerVisuals()
            invalidateAim()
            return
        end

        -- The cached pick only decides WHO gets the blade. throwKnife
        -- re-solves the aim from the live handle when it actually fires.
        local target, aimPos, originCF = solveAim(knife, wantThrow)
        if not aimPos then
            setKnifeStatus(target and "Target blocked · WallCheck" or "No visible target")
            hideTracerVisuals()
            return
        end
        setKnifeStatus("Predicted target")

        if settings.knifeTracer and originCF then
            updateTracerVisual(originCF.Position, tracerGoal(target, aimPos), target, deltaTime)
        else
            hideTracerVisuals()
            destroyTracerVisuals()
        end

        if wantThrow then
            local thrown: boolean, why: string? = throwKnife(target, knife)
            if thrown then
                setKnifeStatus("Throw sent · hit unconfirmed")
                lastThrowAttempt = now
                -- A throw consumes the aim: the next frame re-solves instead of
                -- reusing a point measured before the blade left.
                invalidateAim()
            elseif why then
                -- v186: name the gate that kept the blade in hand instead of
                -- leaving "Predicted target" on the card forever.
                setKnifeStatus(why)
            end
        end
    end

    -- Toggle function
    local function toggleKnifeThrow(enabled: boolean): ()
        settings.enabled = enabled
        disconnectFeatureConnection("MM2KnifeThrow")

        if not enabled then
            uninstallWeaponServiceRedirect()
            setKnifeStatus(nil)
            hideTracerVisuals()
            destroyTracerVisuals()
            invalidateAim()
            resetLoopKnife()
            return
        end

        installWeaponServiceRedirect()
        featureConnections.MM2KnifeThrow =
            TaskManager:Connect(knifeThrowLoop, nil, "MM2 Knife Throw")
    end

    -- Create the feature in Combat category
    KnifeThrowFeature = createUniversalFeature(
        "Knife Throw",
        "Redirects PC & Mobile knife throws to the Knife Predictor's aim point, with optional Auto Knife Throw and Knife Tracer.",
        15,
        toggleKnifeThrow,
        {
            categoryName = "Exploits",
            parent = MM2Scroll,
            registry = mm2Features,
        }
    )

    addToggleOption(
        KnifeThrowFeature,
        "WallCheck",
        true,
        function(value: boolean): ()
            settings.wallCheck = value
        end,
        "Checks the blade's path from the hand to the target, including a thin corridor and invisible solid barriers. With nobody visible, nothing is thrown."
    )

    addToggleOption(
        KnifeThrowFeature,
        "Auto Knife Throw",
        false,
        function(value: boolean): ()
            settings.autoThrow = value == true
        end,
        "Throws at the predicted intercept of a visible target. With nobody visible, or with the path blocked, no knife is spent. A sent throw is not a confirmed hit."
    )

    addNumberOption(
        KnifeThrowFeature,
        "Delay",
        0.6,
        0,
        10,
        function(value: number): ()
            settings.delay = math.clamp(tonumber(value) or 0, 0, 10)
        end,
        "Delay in seconds between automatic knife throws (0 to 10 seconds).",
        0.1
    )

    addInformationOption(
        KnifeThrowFeature,
        "Seconds"
    )

    addToggleOption(
        KnifeThrowFeature,
        "Knife Tracer",
        false,
        function(value: boolean): ()
            settings.knifeTracer = value == true
            if not settings.knifeTracer then
                hideTracerVisuals()
                destroyTracerVisuals()
            end
        end,
        "Smooth 3D preview from your hand to the predicted point. The actual throw is re-solved separately; the preview hides when no target passes WallCheck."
    )

    activeCleanup = function(): ()
        toggleKnifeThrow(false)
    end
    Module.Events = featureConnections
    Module.Initialized = true
    return Module
end

function Module.destroy(): ()
    if not Module.Initialized then
        return
    end
    Module.Initialized = false
    pcall(activeCleanup)
    activeCleanup = function(): () end
    Module.Events = {}
    Module.Runtime = nil
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/MM2/Combat/KnifeThrow.lua
--------------------------------------------------------------------------------


--------------------------------------------------------------------------------
ARCHIVO: src/games/MM2/Fun/BlurtRoles.lua
BYTES: 8577
SHA-256: 2a0ccd3acf9780fa80ef5aa66e53ab29af87209fb39b4c394e94a67c29659ac2
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
local Module = {
    Name = "MM2 Blurt Roles",
    Events = {} :: {[string]: any},
    Initialized = false,
    Runtime = nil :: any,
}

local activeCleanup: () -> () = function(): () end

function Module.init(runtime: any): any
    if Module.Initialized then
        return Module
    end
    local core: any = state.mm2Core
    assert(type(core) == "table", "MM2 Blurt Roles requires the MM2 core module")
    Module.Runtime = runtime
    local findMurderer: any = core.findMurderer
    local findSheriff: any = core.findSheriff
    local hasActiveRoundRoles: any = core.hasActiveRoundRoles
    local mm2Settings: any = core.mm2Settings

    local blurtEnabled: boolean = false
    local blurtSignature: string = ""
    local blurtPending: boolean = false
    local blurtGeneration: number = 0
    local lastBlurtAt: number = -math.huge

    local function cancelPendingBlurt(): ()
        blurtGeneration += 1
        blurtPending = false
    end

    local function sendPublicChat(message: string): boolean
        local sent: boolean = false

        local textChatOk: boolean = pcall(function(): ()
            local textChatService: TextChatService =
                game:GetService("TextChatService")
            if textChatService.ChatVersion ~= Enum.ChatVersion.TextChatService then
                return
            end
            local channels: Instance? = textChatService:FindFirstChild("TextChannels")
            local channel: Instance? = channels
                and (channels:FindFirstChild("RBXGeneral")
                    or channels:FindFirstChildWhichIsA("TextChannel"))
            if channel and channel:IsA("TextChannel") then
                (channel :: TextChannel):SendAsync(message)
                sent = true
            end
        end)

        if sent then
            return true
        end

        local legacyOk: boolean = pcall(function(): ()
            local events: Instance? = game:GetService("ReplicatedStorage")
                :FindFirstChild("DefaultChatSystemChatEvents")
            local say: Instance? = events
                and events:FindFirstChild("SayMessageRequest")
            if say and say:IsA("RemoteEvent") then
                (say :: RemoteEvent):FireServer(message, "All")
                sent = true
            end
        end)

        return sent and (textChatOk or legacyOk)
    end

    local function buildBlurtMessage(): (string?, string?)
        if not hasActiveRoundRoles() then
            return nil, nil
        end
        local murderer: Player? = findMurderer()
        local sheriff: Player? = findSheriff()
        if not murderer and not sheriff then
            return nil, nil
        end
        local murdererName: string = murderer and murderer.Name or "?"
        local sheriffName: string = sheriff and sheriff.Name or "?"
        local message: string
        local signature: string
        if mm2Settings.blurtFake then

            message = string.format(
                "Murder; "%s" Sheriff; "%s"",
                sheriffName,
                murdererName
            )
            signature = "fake/" .. sheriffName .. "/" .. murdererName
        else
```

```
            message = string.format(
                "Murder; "%s" Sheriff; "%s" | Wurst",
                murdererName,
                sheriffName
            )
            signature = murdererName .. "/" .. sheriffName
        end
        return message, signature
    end

    local function blurtRoles(manual: boolean): ()
        local message: string?, signature: string? = buildBlurtMessage()
        if not message then
            if manual then
                notify("MM2 · no roles to blurt yet.")
            end
            return
        end
        if not manual then
            if signature == blurtSignature and not mm2Settings.blurtRepeat then
                return
            end
            if os.clock() - lastBlurtAt < 2.5 then
                return
            end
        end
        blurtSignature = signature :: string
        lastBlurtAt = os.clock()
        task.spawn(function(): ()
            if sendPublicChat(message :: string) then
                notify("MM2 · blurted: " .. (message :: string))
            else
                notify("MM2 · the chat refused the message (filtered or disabled).")
            end
        end)
    end

    local function scheduleBlurt(): ()
        if not blurtEnabled or blurtPending then
            return
        end
        local message: string?, signature: string? = buildBlurtMessage()
        if not message then
            return
        end
        if signature == blurtSignature and not mm2Settings.blurtRepeat then
            return
        end
        blurtPending = true
        local generation: number = blurtGeneration
        task.delay(math.max(mm2Settings.blurtDelay, 0), function(): ()
            if generation ~= blurtGeneration then return end
            blurtPending = false
            if blurtEnabled then blurtRoles(false) end
        end)
    end

    local unsubscribe: () -> () = core.onRoundRoles(function(active: boolean): ()
        if not active then
            cancelPendingBlurt()
            blurtSignature = ""
            return
        end
        scheduleBlurt()
    end)

    local function toggleBlurtRoles(enabled: boolean): ()
        blurtEnabled = enabled
        cancelPendingBlurt()
        disconnectFeatureConnection("MM2BlurtRoles")
        if not enabled then
            return
        end
        blurtSignature = ""
        local elapsed: number = 0
        featureConnections.MM2BlurtRoles = TaskManager:Connect(function(deltaTime: number): ()
            elapsed += deltaTime
            if elapsed < 1 then
                return
            end
            elapsed = 0
            if not hasActiveRoundRoles() then
                blurtSignature = ""
                return
            end
            scheduleBlurt()
        end, nil, "MM2 Blurt Roles")
    end

    -- v191: the card itself is the button. It used to be a toggle with a
    -- separate "Blurt now" row, which meant two clicks (and a wait for the
    -- delay) to say the roles once. Clicking the card now blurts IMMEDIATELY,
    -- and the automatic path moved into the "Auto blurt" option, where the
    -- Delay/Repeat rows still belong.
    local BlurtFeature = createUniversalFeature(
        "Blurt Roles",
        "Types the murderer and sheriff in the public chat. Click the card to blurt right now.",
        3,
        function(): ()
            blurtRoles(true)
        end,
        {
            categoryName = "World",
            parent = MM2Scroll,
            registry = mm2Features,
            kind = "action",
        }
    )
    addToggleOption(
        BlurtFeature,
        "Auto blurt",
        false,
        function(value: boolean): ()
            toggleBlurtRoles(value)
        end,
        "Blurt automatically once per round, after the Delay below. Off by default: the card alone is an instant, one-shot blurt."
    )
    addNumberOption(
        BlurtFeature,
        "Delay",
        mm2Settings.blurtDelay,
        0,
        15,
        function(value: number): ()
            mm2Settings.blurtDelay = value
        end,
        "Seconds to wait after the roles are known before typing.",
        0.5
    )
    addToggleOption(
        BlurtFeature,
        "Repeat on change",
        mm2Settings.blurtRepeat,
        function(value: boolean): ()
            mm2Settings.blurtRepeat = value
        end,
        "Blurt again when the sheriff dies and the gun changes hands."
    )
    addToggleOption(
        BlurtFeature,
        "Fake Blurt Roles",
        mm2Settings.blurtFake,
        function(value: boolean): ()
            mm2Settings.blurtFake = value
        end,
        "Announces the roles SWAPPED (and without the Wurst tag) so the chat "
            .. "lies about who the murderer and sheriff are."
    )
    addInformationOption(
        BlurtFeature,
        "This types in the real public chat: everyone reads it and you will be"
            .. " reported. Roblox also rate-limits and filters messages."
    )

    activeCleanup = function(): ()
        pcall(unsubscribe)
        toggleBlurtRoles(false)
    end
    Module.Events = featureConnections
    Module.Initialized = true
    return Module
end

function Module.destroy(): ()
    if not Module.Initialized then
        return
    end
    Module.Initialized = false
    activeCleanup()
    activeCleanup = function(): () end
    Module.Events = {}
    Module.Runtime = nil
end

return Module

-------------------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/MM2/Fun/BlurtRoles.lua
-------------------------------------------------------------------------------------------


-------------------------------------------------------------------------------------------
ARCHIVO: src/games/MM2/Movement/Sprint.lua
BYTES: 3751
SHA-256: dacab5d4ae8ce8bb9ad0cedc5f3cc314226deb413acef59e42678796da05a97a
INICIO DEL ARCHIVO
-------------------------------------------------------------------------------------------
local Module = {
    Name = "MM2 Sprint",
    Events = {} :: {[string]: any},
    Initialized = false,
    Runtime = nil :: any,
}

local activeCleanup: () -> () = function(): () end

function Module.init(runtime: any): any
    if Module.Initialized then
        return Module
    end
    local core: any = state.mm2Core
    assert(type(core) == "table", "MM2 Sprint requires the MM2 core module")
    Module.Runtime = runtime

    local sprintTrailObjects: {Instance} = {}


    local function clearSprintTrail(): ()
        for _, object: Instance in ipairs(sprintTrailObjects) do
            if object.Parent then
                object:Destroy()
            end
        end
        table.clear(sprintTrailObjects)
    end

    local function ensureSprintTrail(root: BasePart): Trail
        local existing: Instance? = root:FindFirstChild("Wurst_SprintTail")
        if existing and existing:IsA("Trail") then
            return existing
        end

        clearSprintTrail()
        local left: Attachment = Instance.new("Attachment")
        left.Name = "Wurst_SprintLeft"
        left.Position = Vector3.new(-0.8, -0.5, 0.75)
        left.Parent = root

        local right: Attachment = Instance.new("Attachment")
        right.Name = "Wurst_SprintRight"
        right.Position = Vector3.new(0.8, -0.5, 0.75)
        right.Parent = root

        local trail: Trail = Instance.new("Trail")
        trail.Name = "Wurst_SprintTail"
        trail.Attachment0 = left
        trail.Attachment1 = right
        trail.Color = ColorSequence.new(
            Color3.fromRGB(255, 220, 55),
            Color3.fromRGB(175, 115, 20)
        )
        trail.Lifetime = 0.38
        trail.LightEmission = 0.25
        trail.FaceCamera = true
        trail.Transparency = NumberSequence.new({
            NumberSequenceKeypoint.new(0, 0.2),
            NumberSequenceKeypoint.new(1, 1),
        })
        trail.Enabled = false
        trail.Parent = root
        sprintTrailObjects = {trail, left, right}
        return trail
    end

    local speed: any = state.framework and (state.framework:GetModule("Speed Settings") or state.framework:GetModule("Speed Hack"))
    assert(speed, "Sprint option requires the Speed module")
    speed.ClearSprint = clearSprintTrail
    speed.SprintFrame = function(active: boolean, root: BasePart): ()
        if not active then clearSprintTrail(); return end
        local _, humanoid: Humanoid? = getCharacterParts()
        ensureSprintTrail(root).Enabled = humanoid ~= nil and humanoid.MoveDirection.Magnitude > 0.05
    end
    local function releaseSprint(): ()
        disconnectFeatureConnection("MM2SprintHold")
        speed.SetSprintHeld(false)
        clearSprintTrail()
    end
    speed:CreateButton({
        Name = "Sprint", Hold = true,
        Function = function(): ()
            speed.SetSprintHeld(true)
            if not featureConnections.MM2SprintHold then
                featureConnections.MM2SprintHold = TaskManager:Connect(speed.StepSprint, nil, "MM2 Sprint")
            end
        end,
        OnRelease = releaseSprint,
        Tooltip = "Hold this button, its assigned key or the touch action to sprint. Works with Speed OFF; release restores normal Speed or your original speed. No toggle state is saved.",
    })
    activeCleanup = function(): ()
        releaseSprint()
        speed.ClearSprint = nil
        speed.SprintFrame = nil
    end
    Module.Events = featureConnections
    Module.Initialized = true
    return Module
end

function Module.destroy(): ()
    if not Module.Initialized then
        return
    end
    Module.Initialized = false
    activeCleanup()
    activeCleanup = function(): () end
    Module.Events = {}
    Module.Runtime = nil
end

return Module

-------------------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/MM2/Movement/Sprint.lua
-------------------------------------------------------------------------------------------


-------------------------------------------------------------------------------------------
ARCHIVO: src/games/MM2/Movement/Teleport.lua
BYTES: 12791
SHA-256: ab0822689fd048c5ab959da3125547af4143fc650abc568a3a03ababefcd4a64
INICIO DEL ARCHIVO
-------------------------------------------------------------------------------------------
local Module = {
    Name = "MM2 Teleport",
    Events = {} :: {[string]: any},
    Initialized = false,
    Runtime = nil :: any,
}

local activeCleanup: () -> () = function(): () end

function Module.init(runtime: any): any
    if Module.Initialized then
        return Module
    end
    local core: any = state.mm2Core
    assert(type(core) == "table", "MM2 Teleport requires the MM2 core module")
    Module.Runtime = runtime
    local findDroppedGun: any = core.findDroppedGun
    local findMM2Map: any = core.findMM2Map
    local getPlayerRole: any = core.getPlayerRole
    local isRoundLifecycleActive: any = core.isRoundLifecycleActive
    local mm2Settings: any = core.mm2Settings

    -- Auto Get Gun is an innocent-only, in-round action. As Murderer it
    -- must never run (the knife role has no use for the sheriff gun and
    -- grabbing it exposes the role), and in the lobby there is no dropped
    -- gun worth chasing. Fail closed when the role or round state is not
    -- positively known.
    local function canAutoGetGun(): boolean
        return getPlayerRole(LocalPlayer) == "Innocent"
            and isRoundLifecycleActive() == true
            and findMM2Map() ~= nil
    end

    -- Immediate pickup without moving the character: pulse the touch
    -- between our root and the dropped gun's part, with no sleep or
    -- configurable delay. The next Heartbeat can retry while weapon
    -- replication is still catching up.
    local function touchDroppedGun(gunDrop: Instance): boolean
        if type(state.fireTouchInterest) ~= "function" then
            return false
        end
        local character: Model? = LocalPlayer.Character
        local root: Instance? = character
            and character:FindFirstChild("HumanoidRootPart")
        if not root then
            return false
        end
        local touchPart: Instance? = nil
        if gunDrop:IsA("BasePart") then
            touchPart = gunDrop
        elseif gunDrop:IsA("Model") then
            touchPart = gunDrop:FindFirstChild("Handle", true)
                or gunDrop:FindFirstChildWhichIsA("BasePart", true)
        end
        if not touchPart then
            return false
        end
        pcall(state.fireTouchInterest, root, touchPart, 0)
        pcall(state.fireTouchInterest, root, touchPart, 1)
        local backpack: Instance? =
            LocalPlayer:FindFirstChildOfClass("Backpack")
        return (backpack ~= nil and backpack:FindFirstChild("Gun") ~= nil)
            or (character ~= nil and character:FindFirstChild("Gun") ~= nil)
    end

    local autoGetGunBusy = false

    local function toggleAutoGetGun(enabled)
        disconnectFeatureConnection("MM2AutoGetGun")
        autoGetGunBusy = false
        state.mm2AutoGetGunEnabled = enabled == true

        if not enabled then
            return
        end

        -- No polling throttle: TaskManager already runs this on
        -- Heartbeat, so checking every frame is the fastest the
        -- client can react (one frame, never a 0.5 s wait). The tick
        -- also runs once synchronously on enable, so a gun that is
        -- already on the ground is grabbed in the same frame the
        -- toggle turns on instead of waiting for the next Heartbeat.
        local function autoGetGunTick(): ()
            if autoGetGunBusy then
                return
            end

            local character = LocalPlayer.Character
            local backpack = LocalPlayer:FindFirstChildOfClass("Backpack")
            local gunDrop = findDroppedGun()
            if not character or not gunDrop
                or not canAutoGetGun()
                or (backpack and backpack:FindFirstChild("Gun"))
                or character:FindFirstChild("Gun") then
                return
            end

            -- Instant touch first: no PivotTo, no position the server
            -- can call invalid. If the touch does not deliver the gun,
            -- stay put and let the next frame retry instead of falling
            -- back to a teleport the user did not ask for.
            autoGetGunBusy = true
            task.spawn(function()
                pcall(touchDroppedGun, gunDrop)
                autoGetGunBusy = false
            end)
        end
        featureConnections.MM2AutoGetGun =
            TaskManager:Connect(autoGetGunTick, nil, "MM2 Get Gun")
        autoGetGunTick()
    end

    local function moveCharacterTo(character: any, destination: CFrame): ()
        local root: BasePart? = character:FindFirstChild("HumanoidRootPart")
        if root and root:IsA("BasePart") then
            root.Anchored = false
            root.AssemblyLinearVelocity = Vector3.zero
            root.AssemblyAngularVelocity = Vector3.zero
        else
            root = nil
        end
        character:PivotTo(destination)
        if root then root.CFrame = destination end
    end

    local function teleportToMap()
        local map = findMM2Map()
        local spawns = map and (map:FindFirstChild("Spawns") or map:FindFirstChild("Spawns", true))
        local character = LocalPlayer.Character
        if not map or not character then
            notify("No active MM2 map was found.")
            return
        end

        local target = nil
        if spawns then
            local spawnList = spawns:GetChildren()
            target = spawnList[math.random(1, math.max(1, #spawnList))]
            if target and not target:IsA("BasePart") then
                target = target:FindFirstChildWhichIsA("BasePart", true)
            end
        end
        if (not target or not target:IsA("BasePart")) and spawns then
            target = spawns:FindFirstChildWhichIsA("BasePart", true)
        end
        if not target or not target:IsA("BasePart") then
            target = map:FindFirstChildWhichIsA("BasePart", true)
        end
        if target and target:IsA("BasePart") then
            moveCharacterTo(character, target.CFrame + Vector3.new(0, 3, 0))
        else
            notify("No active MM2 map spawn was found.")
        end
    end

    local function findLobbySpawn()
        local lobby = workspace:FindFirstChild("Lobby")
            or workspace:FindFirstChild("MainLobby")
            or workspace:FindFirstChild("RegularLobby")
        if not lobby then
            for _, child in ipairs(workspace:GetChildren()) do
                local lowerName = string.lower(tostring(child.Name))
                if string.find(lowerName, "lobby", 1, true) ~= nil
                    or child:FindFirstChild("VotePads", true) ~= nil
                    or child:FindFirstChild("VotePad1", true) ~= nil
                    or child:FindFirstChild("VoteIcons", true) ~= nil then
                    lobby = child
                    break
                end
            end
        end
        if lobby then
            local spawns = lobby:FindFirstChild("Spawns")
                or lobby:FindFirstChild("Spawns", true)
            if spawns then
                local spawn = spawns:FindFirstChildWhichIsA("SpawnLocation")
                    or spawns:FindFirstChildWhichIsA("BasePart")
                    or spawns:FindFirstChildWhichIsA("BasePart", true)
                if spawn then return spawn end
            end
            local directSpawn = lobby:FindFirstChildWhichIsA("SpawnLocation", true)
            if directSpawn then return directSpawn end
            local votePads = lobby:FindFirstChild("VotePads", true)
                or lobby:FindFirstChild("VotePad1", true)
            if votePads then
                local padPart = votePads:FindFirstChildWhichIsA("BasePart", true)
                if padPart then return padPart end
            end
            local anyPart = lobby:FindFirstChildWhichIsA("BasePart", true)
            if anyPart then return anyPart end
        end
        local worldSpawns = workspace:FindFirstChild("Spawns")
        if worldSpawns then
            local spawn = worldSpawns:FindFirstChildWhichIsA("SpawnLocation")
                or worldSpawns:FindFirstChildWhichIsA("BasePart")
            if spawn then return spawn end
        end
        return workspace:FindFirstChildWhichIsA("SpawnLocation", true)
    end

    local function teleportToLobby()
        local spawn = findLobbySpawn()
        local character = LocalPlayer.Character
        if spawn and character then
            moveCharacterTo(character, spawn.CFrame + Vector3.new(0, 3, 0))
        else
            notify("The MM2 lobby spawn was not found.")
        end
    end

    local teleportTargetName = ""

    local function teleportToPlayer()
        local character = LocalPlayer.Character
        if not character then
            notify("Your character is not available.")
            return
        end
        local needle = string.lower(
            string.gsub(teleportTargetName or "", "%s+", "")
        )
        if needle == "" then
            notify("Type a player name in the Player name box first.")
            return
        end
        local exact, partial = nil, nil
        for _, player in ipairs(Players:GetPlayers()) do
            if player ~= LocalPlayer then
                local name = string.lower(player.Name)
                if string.gsub(name, "%s+", "") == needle then
                    exact = player
                    break
                end
                if not partial and string.find(name, needle, 1, true) then
                    partial = player
                end
            end
        end
        local target = exact or partial
        local targetRoot = target
            and target.Character
            and target.Character:FindFirstChild("HumanoidRootPart")
        if targetRoot then
            character:PivotTo(targetRoot.CFrame + Vector3.new(0, 3, 0))
        elseif target then
            notify(target.Name .. " has no character to teleport to.")
        else
            notify("No player named \"" .. teleportTargetName .. "\" was found.")
        end
    end

    local function grabDroppedGun()
        local gunDrop = findDroppedGun()
        if gunDrop and LocalPlayer.Character then
            touchDroppedGun(gunDrop)
        else
            notify("No dropped gun was found.")
        end
    end

    local teleport = state.framework:GetModule("Teleport")
    assert(teleport, "MM2 extends the universal Teleport module")
    for _, id in ipairs({"GetGunkey", "Playername"}) do
        local key = "Universal.ClickTeleport." .. id
        if configData.values[key] == nil then configData.states[autoKey] = configData.states["MM2.Teleport.AutoGetGun"] end
    end
    local autoKey = "Universal.ClickTeleport.AutoGetGun"
    if configData.states[autoKey] == nil then configData.states[autoKey] = configData.states["MM2.Teleport.AutoGetGun"] end
    local show = {Option = "Destination", Values = {"MM2"}}
    teleport:CreateButton({Name = "Teleport to Map", Show = show, Function = teleportToMap})
    teleport:CreateButton({Name = "Teleport to Lobby", Show = show, Function = teleportToLobby})
    teleport:CreateTextBox({Name = "Player name", Default = "", Show = show,
        Function = function(value) teleportTargetName = value end})
    teleport:CreateButton({Name = "Teleport to Player", Show = show, Function = teleportToPlayer})
    teleport:CreateButton({Name = "Get Gun", Show = show, Function = grabDroppedGun})
    local key = teleport:CreateBind({Name = "Get Gun key", Default = mm2Settings.getGunKey, Show = show,
        Function = function(value) mm2Settings.getGunKey = value end})
    if state.bindMobileActionPlacement and key.Object then
        state.bindMobileActionPlacement(key.Object, "MM2GetGun", "GUN", grabDroppedGun)
    end
    featureConnections.MM2GetGunKey = UserInputService.InputBegan:Connect(
        function(input, gameProcessed)
            if gameProcessed
                or state.keyCaptureCallback
                or state.waitingForKey
                or UserInputService:GetFocusedTextBox() then
                return
            end
            if input.UserInputType == Enum.UserInputType.Keyboard
                and input.KeyCode == mm2Settings.getGunKey then
                task.spawn(grabDroppedGun)
            end
        end
    )
    teleport:CreateToggle({Name = "Auto Get Gun", Default = false, Show = show, Function = toggleAutoGetGun})

    activeCleanup = function(): ()
        toggleAutoGetGun(false)
        disconnectFeatureConnection("MM2GetGunKey")
    end
    Module.Events = featureConnections
    Module.Initialized = true
    return Module
end

function Module.destroy(): ()
    if not Module.Initialized then
        return
    end
    Module.Initialized = false
    pcall(activeCleanup)
    activeCleanup = function(): () end
    Module.Events = {}
    Module.Runtime = nil
end

return Module

-------------------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/MM2/Movement/Teleport.lua
-------------------------------------------------------------------------------------------


-------------------------------------------------------------------------------------------
ARCHIVO: src/games/MM2/Other/AutoFarm.lua
BYTES: 101348
SHA-256: 96591f1e8b75072e900269bde738a84b445ce0c805a013948d22f50b4b39ea21
INICIO DEL ARCHIVO
-------------------------------------------------------------------------------------------
local Module = {
    Name = "MM2 Auto Farm",
    Events = {},
    Initialized = false,
}

local activeCleanup: () -> () = function(): ()
end

function Module.init(_runtime: any): any
    assert(state.mm2Core, "MM2 Auto Farm requires the MM2 core")
    local core: any = state.mm2Core
    local getPlayerRole: any = core.getPlayerRole
    local getPlayerWeapon: any = core.getPlayerWeapon
    local isPlayerAlive: any = core.isPlayerAlive
    local findMurderer: any = core.findMurderer
    local findSheriff: any = core.findSheriff
    local findDroppedGun: any = core.findDroppedGun
    local isRoundLifecycleActive: any = core.isRoundLifecycleActive
    local findMM2Map: any = core.findMM2Map
    local getEstimatedLatency: any = core.getEstimatedLatency

    -- v138.7 renamed the two full-bag fling toggles ("after reset" ->
    -- "after full bag"). Option keys are derived from the label, so carry a saved
    -- value over instead of silently resetting it.
    if type(configData) == "table" and type(configData.states) == "table" then
```

```
    local rename = {
        ["MM2.AutoFarm.FlingMurderafterreset"] = "MM2.AutoFarm.FlingMurderafterfullbag",
        ["MM2.AutoFarm.FlingSheriffafterreset"] = "MM2.AutoFarm.FlingSheriffafterfullbag",
    }
    for oldKey: string, newKey: string in pairs(rename) do
        if configData.states[newKey] == nil and configData.states[oldKey] ~= nil then
            configData.states[newKey] = configData.states[oldKey]
        end
        configData.states[oldKey] = nil
    end
end

local settings = {
    mode = "Walk",
    speed = 16,
    stayUnderOnStart = true,
    safeZone = true,
    resetOnFull = true,
    flingMurder = false,
    flingSheriff = false,
    grabGunShoot = false,
    killAllOnFull = false,
}

local totalCoinsCollected: number = 0
local farmElapsedSeconds: number = 0
local lastFormattedSecond: number = -1
local coinsCollectedStat: any = nil
local farmTimeStat: any = nil
local safeZonePlatform: Part? = nil
-- Installed by the current farm generation; used by both the master OFF
-- and the Safe Zone row OFF. Never send an already-respawned body to an
-- old map, and never remove its only floor when no return is possible.
local leaveSafeZone: ((boolean?, boolean?) -> boolean)? = nil

local function formatFarmTime(totalSeconds: number): string
    local whole: number = math.max(0, math.floor(totalSeconds))
    local hours: number = math.floor(whole / 3600)
    local mins: number = math.floor((whole % 3600) / 60)
    local secs: number = whole % 60
    if hours > 0 then
        return string.format("%02d:%02d:%02d", hours, mins, secs)
    end
    return string.format("%02d:%02d", mins, secs)
end

local function refreshFarmOverlay(): ()
    if coinsCollectedStat and type(coinsCollectedStat.set) == "function" then
        coinsCollectedStat.set(tostring(totalCoinsCollected))
    end
    local wholeSec: number = math.max(0, math.floor(farmElapsedSeconds))
    if wholeSec ~= lastFormattedSecond then
        lastFormattedSecond = wholeSec
        if farmTimeStat and type(farmTimeStat.set) == "function" then
            farmTimeStat.set(formatFarmTime(wholeSec))
        end
    end
end

local function destroySafeZonePlatform(): ()
    if safeZonePlatform then
        pcall(function(): ()
            safeZonePlatform:Destroy()
        end)
        safeZonePlatform = nil
    end
end

local function ensureSafeZonePlatform(originPos: Vector3): Vector3?
    local voidCenter: Vector3 = Vector3.new(
        originPos.X + 650,
        math.max(originPos.Y + 280, 380),
        originPos.Z + 650
    )
    if not safeZonePlatform or not safeZonePlatform.Parent then
        pcall(function(): ()
            local part: Part = Instance.new("Part")
            part.Name = "Wurst_AutoFarmSafeZone"
            part.Anchored = true
            part.CanCollide = true
            part.CanQuery = false
            part.CanTouch = false
            part.CastShadow = false
            part.Size = Vector3.new(64, 4, 64)
            part.Material = Enum.Material.SmoothPlastic
            part.Color = Color3.fromRGB(35, 42, 55)
            part.Transparency = 0.15
            part.CFrame = CFrame.new(voidCenter)
            part.Parent = workspace
            safeZonePlatform = part
        end)
    end
    if safeZonePlatform and safeZonePlatform.Parent then
        return safeZonePlatform.Position + Vector3.new(0, 5, 0)
    end
    -- A missing platform is not a safe destination.
    return nil
end

-- Two ways to reach a coin, chosen with Farm Mode:
--
-- Walk: the humanoid simply walks on the ground (Humanoid:Move).
-- No prone tilt, no PlatformStand, no NoClip, no flying -- the
-- body's normal touch against Coin_Server pays, which is why this
-- is the reliable default. The slider becomes WalkSpeed.
--
-- Below (Hitbox): the character stays UPRIGHT and flies by physics
-- velocity underneath the floor, but the root's hitbox is grown
-- while engaged (4 x 8 x 4), so riding with the root centre
-- BELOW_OFFSET under the coin still overlaps Coin_Server (1.2 x 2 x
-- 2, one stud either side of its centre). The old build lay the
-- body face-down to get a part into that volume; the tilted camera
-- fight is what glitched, and the hitbox makes it unnecessary.
local BELOW_OFFSET: number = 3.0
local HITBOX_SIZE: Vector3 = Vector3.new(4, 8, 4)
-- A coin that survives several passes (or cannot be reached) is skipped for
-- a while instead of trapping the farm on one unreachable pickup.
local BLACKLIST_SECONDS: number = 6

local function characterParts(): (Model?, Humanoid?, BasePart?)
    if type(getCharacterParts) == "function" then
        local ok: boolean, character: any, humanoid: any, root: any =
            pcall(getCharacterParts)
        if ok and character then
            return character, humanoid, root
        end
    end
    local character: Model? = LocalPlayer and LocalPlayer.Character
    if not character then
        return nil, nil, nil
    end
    local humanoid: Humanoid? =
        character:FindFirstChildOfClass("Humanoid") :: Humanoid?
    local root: BasePart? =
        character:FindFirstChild("HumanoidRootPart") :: BasePart?
    return character, humanoid, root
end

local function rootOf(player: Player?): BasePart?
    local character: Model? = player and player.Character
    if not character then
        return nil
    end
    return character:FindFirstChild("HumanoidRootPart") :: BasePart?
end

-- Prefer Coin ESP's current family, but an empty/stale family cache must
-- not hide real pickups from Auto Farm. The fallback is throttled separately
-- (including when there are zero coins) and never searches a different map.
local fallbackScanAt: number = -math.huge
local fallbackScanMap: Instance? = nil
local fallbackPickups: {Instance} = {}
local function coinPickups(): {Instance}
    local map: Instance? = type(findMM2Map) == "function" and findMM2Map()
    local out: {Instance} = {}
    local function consider(object: Instance): ()
        if not object.Parent or not object:IsA("BasePart") then return end
        -- A real Instance can verify ancestry; do not chase a lobby or an
        -- old map's pickup just because it still has the CoinVisual tag.
        if map and type(object.IsDescendantOf) == "function"
            and not object:IsDescendantOf(map) then return end
        local ok: boolean, usable: any = pcall(function(): boolean
            local parent: Instance? = object.Parent
            local server: Instance = if parent and parent.Name == "Coin_Server" then parent else object
            return object:GetAttribute("Collected") ~= true
                and object:GetAttribute("Delete") ~= true
                and server:GetAttribute("Delete") ~= true
        end)
        if ok and usable then table.insert(out, object) end
    end
    if type(state.mm2CoinPickups) == "function" then
        local ok: boolean, list: any = pcall(state.mm2CoinPickups)
        if ok and type(list) == "table" then
            -- Coin ESP scans the entire workspace: a non-empty list can
            -- still consist solely of old-map or already-collected items.
            for _, object: Instance in ipairs(list) do consider(object) end
            if #out > 0 then return out end
        end
    end
    local now: number = os.clock()
    if map == fallbackScanMap and now - fallbackScanAt < 0.5 then
        return fallbackPickups
    end
    fallbackScanAt = now
    fallbackScanMap = map
    local okTagged: boolean, tagged: any = pcall(function(): any
        return game:GetService("CollectionService"):GetTagged("CoinVisual")
    end)
    if okTagged and type(tagged) == "table" then
        for _, object: Instance in ipairs(tagged) do consider(object) end
    end
    if #out == 0 then
        local ok: boolean, descendants: any = pcall(function(): any
            -- Pickups live under the active map's CoinContainer; avoid
            -- walking the lobby and every unrelated streamed character.
            if map and type(map.GetDescendants) == "function" then return map:GetDescendants() end
            return workspace:GetDescendants()
        end)
        if ok and type(descendants) == "table" then
            for _, object: Instance in ipairs(descendants) do
                if object.Name == "Coin_Server" then consider(object) end
            end
        end
    end
    fallbackPickups = out
    return out
end

-- Live ping for the status line. The farm speed itself is the
-- slider: the server kicks ("Invalid position", 267) when the
-- root arrives faster than it accepts, and that limit varies by
-- map and connection, so the user tunes it -- start low.
local function pingMillis(): number
    if type(getEstimatedLatency) == "function" then
        local ok: boolean, value: any = pcall(getEstimatedLatency)
        if ok and type(value) == "number" and value == value then
            return math.round(math.clamp(value, 0.02, 0.6) * 1000)
        end
    end
    return 0
end

local function setStatus(text: string?): ()
    local card: any = state.mm2AutoFarmFeature
    if card then
        card:SetStatus(text)
    end
end

-- Kill All, adapted from YARHM 1.21 ("Kill EVERYONE as
-- murderer"): equip the knife, pull every living player in front
-- of us (anchored), then slash. The slash uses the repo's own
-- knife path (Events.HandleTouched per victim, as KnifeAura
-- does) and falls back to the game's Stab remote like YARHM.
local function killAll(isCurrent: () -> boolean): ()
    if not isCurrent() then return end
    if type(getPlayerRole) ~= "function"
        or getPlayerRole(LocalPlayer) ~= "Murderer" then
        notify("Auto Farm: Kill All only works as the Murderer.")
        return
    end
    local character: Model?, humanoid: Humanoid?, myRoot: BasePart? =
        characterParts()
    if not character or not humanoid or not myRoot then
        return
    end
    local knife: Tool? = nil
    if type(getPlayerWeapon) == "function" then
        knife = getPlayerWeapon(LocalPlayer, "Knife")
    end
    if not knife then
        notify("Auto Farm: no Knife found for Kill All.")
        return
    end
    if knife.Parent ~= character then
        pcall(function(): ()
            humanoid:EquipTool(knife :: Tool)
        end)
        -- Wait only until the equip lands (one frame normally),
        -- never a fixed sleep: 0.3 s is the ceiling, not the cost.
        local equipDeadline: number = os.clock() + 0.3
        while knife.Parent ~= character
            and os.clock() < equipDeadline and isCurrent() do
            task.wait()
        end
    end
    if not isCurrent() or knife.Parent ~= character then
        return
    end
    local victims: {BasePart} = {}
    for _, player: Player in ipairs(Players:GetPlayers()) do
        if player ~= LocalPlayer
            and (type(isPlayerAlive) ~= "function" or isPlayerAlive(player)) then
            local victimRoot: BasePart? = rootOf(player)
            if victimRoot then
                table.insert(victims, victimRoot)
                pcall(function(): ()
                    victimRoot.Anchored = true
                    victimRoot.CFrame = myRoot.CFrame
                        + myRoot.CFrame.LookVector * 1
                end)
            end
        end
    end
    if #victims == 0 then
        return
    end
    local function releaseVictims(): ()
        for _, victimRoot: BasePart in ipairs(victims) do
            pcall(function(): ()
                victimRoot.Anchored = false
            end)
        end
    end
    task.wait(0.15)
    if not isCurrent() then
        releaseVictims()
        return
    end
    local stab: Instance? = knife:FindFirstChild("Stab")
    if stab and stab:IsA("RemoteEvent") then
        pcall(function(): ()
            (stab :: RemoteEvent):FireServer("Slash")
        end)
    end
    local events: Instance? = knife:FindFirstChild("Events")
    local handleTouched: Instance? = events
        and events:FindFirstChild("HandleTouched")
    pcall(function(): ()
        (knife :: Tool):Activate()
    end)
    for _, victimRoot: BasePart in ipairs(victims) do
        if not isCurrent() then break end
        if handleTouched and handleTouched:IsA("RemoteEvent") then
            pcall(function(): ()
                (handleTouched :: RemoteEvent):FireServer(victimRoot)
            end)
        end
        task.wait(0.05)
    end
    if isCurrent() then
        task.delay(0.5, releaseVictims)
        notify("Auto Farm: Kill All fired at " .. #victims .. " players.")
    else
        releaseVictims()
    end
end

-- Compact spin fling: park on the target spinning hard for a
-- couple of seconds; contact physics does the rest.
local function flingPlayer(target: Player?, isCurrent: () -> boolean): ()
    if not target or not isCurrent() then
        return
    end
    if type(state.mm2PerformFling) == "function" then
        pcall(state.mm2PerformFling, target, {
            silent = true,
            duration = 3.5,
            blocking = true,
            shouldStop = function(): boolean return not isCurrent() end,
            skipReturnOnStop = true,
        })
        return
    end
    local _, _, myRoot: BasePart? = characterParts()
    if not myRoot then
        return
    end
    local saved: CFrame = myRoot.CFrame
    local startedAt: number = os.clock()
    while os.clock() - startedAt < 2.2 and isCurrent() do
        local targetRoot: BasePart? = rootOf(target)
        if not targetRoot then
            break
        end
        pcall(function(): ()
            myRoot.CFrame = targetRoot.CFrame
            myRoot.AssemblyAngularVelocity = Vector3.new(0, 400, 0)
            myRoot.AssemblyLinearVelocity = Vector3.zero
        end)
        task.wait()
    end
    pcall(function(): ()
        myRoot.AssemblyAngularVelocity = Vector3.zero
        if isCurrent() then
            myRoot.CFrame = saved
        end
    end)
end

-- Use the shared Silent/CFrame constructor only. Auto Farm explicitly
-- opts out of WallCheck; Gun Aimbot's per-mode setting is independent.
local lastShotAt: number = -math.huge
local function shootMurderer(murderer: Player): boolean
    if type(state.mm2FireSilentShot) ~= "function" then
        setStatus("Silent shot unavailable")
        return false
    end
    local now: number = os.clock()
    if now - lastShotAt < 0.22 then return false end
    -- A thrown error can happen after a send; pace it and never fall back
    -- to a second transport or to a normal Bullet shot.
    lastShotAt = now
    local ok: boolean, sent: any = pcall(state.mm2FireSilentShot, murderer, {
        continuous = true,
        quiet = true,
        wallCheck = false,
    })
    return ok and sent == true
end

local farmGeneration: number = 0

local function toggleAutoFarm(enabled: boolean): ()
    farmGeneration += 1
    local enableGeneration: number = farmGeneration
    disconnectFeatureConnection("MM2AutoFarmSafeZoneRetry")
    disconnectFeatureConnection("MM2AutoFarmTick")
    disconnectFeatureConnection("MM2AutoFarmStepped")
    disconnectFeatureConnection("MM2AutoFarmCoinCollected")
    disconnectFeatureConnection("MM2AutoFarmCoinAdded")
    disconnectFeatureConnection("MM2AutoFarmRoundEnd")
    disconnectFeatureConnection("MM2AutoFarmRoundStart")
    for _, name: string in ipairs({"RoleSelect", "ShowRoleSelect", "ShowRoleSelectNew", "CoinsStarted"}) do
        disconnectFeatureConnection("MM2AutoFarmStart_" .. name)
    end
    -- Invokes the restore closure the previous enable registered:
    -- collisions, PlatformStand, AutoRotate and the root all go
    -- back, and the character returns to where the farm grabbed
    -- it (never left lying under the floor, stuck in geometry).
    disconnectFeatureConnection("MM2AutoFarmRestore")
    if not enabled then
        setStatus(nil)
        return
    end
    -- A previously stranded Safe Zone still owns its one protective pad.
    -- Do not start farming from the void or forget its return destination.
    if leaveSafeZone and not leaveSafeZone(false) then
        setStatus("Auto Farm: waiting in Safe Zone for the original map")
        -- Keep ON meaningful if the user re-enables while the old map is
        -- temporarily unavailable: retry quietly, then start a fresh farm
        -- generation only after a safe return or a server teleport.
        local retryElapsed: number = 0
        featureConnections.MM2AutoFarmSafeZoneRetry = TaskManager:Connect(function(delta: number): ()
            if farmGeneration ~= enableGeneration then return end
            retryElapsed += math.max(0, tonumber(delta) or 0)
            if retryElapsed < 1 then return end
            retryElapsed = 0
            if leaveSafeZone and leaveSafeZone(false, true) then
                toggleAutoFarm(true)
            end
        end, nil, "MM2 Auto Farm Safe Zone Retry")
        return
    end
    leaveSafeZone = nil
    fallbackScanAt = -math.huge
    fallbackScanMap = nil
    fallbackPickups = {}

    local bagAmount: number = 0
    local bagLimit: number = 50
    local busy: boolean = false
    local fullHandled: boolean = false
    local roundFullReached: boolean = false
    local teleportedHomeThisRound: boolean = false
    local wasInActiveRound: boolean = false
    local currentRoundMap: Instance? = nil
    local busyLobbyPart: BasePart? = nil
    local busyLobbyScanAt: number = 0
    local endedMap: Instance? = nil
    local waitingForNextRound: boolean = false
    local roundGeneration: number = 0
    local scanCache: {Instance} = {}
    local scanAt: number = 0
    -- A coin that does not pay on the first touch is retried on a later pass
    -- instead of holding the farm on it: the dwell under a coin is a fraction
    -- of a second, never the old ~1 s stop.
    local DWELL_SECONDS: number = 0.3
    local RETRY_SECONDS: number = 1.5
    local RETRY_LIMIT: number = 3
    local retryAt: {[Instance]: number} = {}
    local attempts: {[Instance]: number} = {}
    local blacklist: {[Instance]: number} = {}
    local stuckPart: Instance? = nil
    local stuckSince: number = 0
    local lastTouchPulse: number = 0
    local travelPart: Instance? = nil
    local travelBestDistance: number = math.huge
    local travelProgressAt: number = 0
    local TRAVEL_STALL_SECONDS: number = 3.5

    -- Engagement state. The farm touches the character ONLY while engaged
    -- inside an active round on a map. In the lobby or between rounds, the
    -- character is left completely alone.
    local engaged: boolean = false
    local engagedCharacter: Model? = nil
    local homeCFrame: CFrame? = nil
    local homeCharacter: Model? = nil
    local parkedCharacter: Model? = nil
    local parkedMap: Instance? = nil
    local parkedReturn: CFrame? = nil
    local parkedPosition: Vector3? = nil
    local hiddenUnderMap: boolean = false
    local savedCollide: {[Instance]: boolean} = {}
    local engagedMode: string? = nil
    local savedWalkSpeed: number? = nil
    local savedRootSize: Vector3? = nil
    local walkLastDistance: number = math.huge
    local walkStuckSince: number = 0
    local walkBodyReady: boolean = false
    local walkCarryDirection: Vector3? = nil
    local walkCarryUntil: number = 0
    -- Below (Hitbox) is flown by the controller below, on Stepped (before the
    -- physics step). The task tick only chooses the destination.
    local hoverTarget: Vector3? = nil
    local hoverActive: boolean = false
    local lastSteppedPosition: Vector3? = nil

    local function yawOf(frame: CFrame): number
        local look: Vector3 = frame.LookVector
        return math.atan2(-look.X, -look.Z)
    end

    local function findLobbyReferencePart(): BasePart?
        local okChildren: boolean, children: any = pcall(function(): any
            return workspace:GetChildren()
        end)
        if not okChildren or type(children) ~= "table" then
            return nil
        end
        for _, child: Instance in ipairs(children) do
            local lowerName: string = string.lower(tostring(child.Name))
            local isLobby: boolean = string.find(lowerName, "lobby", 1, true) ~= nil
                or child:FindFirstChild("VotePads", true) ~= nil
                or child:FindFirstChild("VotePad1", true) ~= nil
                or child:FindFirstChild("VoteIcons", true) ~= nil
            if isLobby then
                local votePads: Instance? = child:FindFirstChild("VotePads", true)
                if votePads then
                    local padPart: BasePart? = votePads:FindFirstChildWhichIsA("BasePart", true)
                    if padPart then
                        return padPart
                    end
                end
                local spawns: Instance? = child:FindFirstChild("Spawns", true)
                if spawns then
                    local spawnPart: BasePart? = spawns:FindFirstChildWhichIsA("BasePart", true)
                    if spawnPart then
                        return spawnPart
                    end
                end
                local anyPart: BasePart? = child:FindFirstChildWhichIsA("BasePart", true)
                if anyPart then
                    return anyPart
                end
            end
        end
        return nil
    end

    local function isNearLobby(root: BasePart?, lobby: BasePart?): boolean
        if not root or not lobby then return false end
        local ok: boolean, near: any = pcall(function(): boolean
            return (root.Position - lobby.Position).Magnitude <= 350
        end)
        return ok and near == true
    end

    -- Resolves a safe indoor floor position without ever raycasting from
    -- above the building roof (which would put the player on top of the map).
    local function resolveSafeAboveGroundPosition(
        candidatePos: Vector3,
        character: Model?
    ): Vector3
        if homeCFrame then
            local dx: number = candidatePos.X - homeCFrame.Position.X
            local dz: number = candidatePos.Z - homeCFrame.Position.Z
            if (dx * dx + dz * dz) <= 36 then
                return Vector3.new(candidatePos.X, homeCFrame.Position.Y, candidatePos.Z)
            end
        end
        local resolvedY: number = candidatePos.Y
        local wasUnderground: boolean = hiddenUnderMap or engagedMode == "Below"
        local probeRaise: number = if wasUnderground then (BELOW_OFFSET + 4.5) else 2.5
        local foundFloor: boolean = false
        pcall(function(): ()
            if type(workspace.Raycast) == "function" and type(RaycastParams) == "table" then
                local params: RaycastParams = RaycastParams.new()
                params.FilterType = Enum.RaycastFilterType.Exclude
                local excluded: {Instance} = {}
                if character then
                    table.insert(excluded, character)
                end
                if safeZonePlatform then
                    table.insert(excluded, safeZonePlatform)
                end
                params.FilterDescendantsInstances = excluded
                params.RespectCanCollide = true
                params.IgnoreWater = true
                local rayOrigin: Vector3 = Vector3.new(
                    candidatePos.X,
                    candidatePos.Y + probeRaise,
                    candidatePos.Z
                )
                local hit: any = workspace:Raycast(
                    rayOrigin,
                    Vector3.new(0, -(probeRaise + 8), 0),
                    params
                )
                if hit and typeof(hit.Position) == "Vector3" then
                    resolvedY = hit.Position.Y + 3.2
                    foundFloor = true
                end
            end
        end)
        if not foundFloor and wasUnderground then
            if homeCFrame then
                resolvedY = homeCFrame.Position.Y
            else
                resolvedY = candidatePos.Y + BELOW_OFFSET + 4.0
            end
        end
        return Vector3.new(candidatePos.X, resolvedY, candidatePos.Z)
    end

    local function forceNoClipCharacter(character: Model): ()
        for _, object: Instance in ipairs(character:GetDescendants()) do
            if object:IsA("BasePart") then
                if savedCollide[object] == nil then
                    savedCollide[object] = (object :: BasePart).CanCollide
                end
                pcall(function(): ()
                    (object :: BasePart).CanCollide = false
                end)
            end
        end
    end

    -- How deep below the round's own start the farm may hold, and how far
    -- down counts as "already falling into the void".
    local MAX_BELOW_DEPTH: number = 90
    local VOID_RECOVER_DEPTH: number = 130

    local function destroyHeightY(): number
        local destroyHeight: any = nil
        pcall(function(): ()
            destroyHeight = workspace.FallenPartsDestroyHeight
        end)
        if type(destroyHeight) ~= "number"
            or destroyHeight ~= destroyHeight
            or math.abs(destroyHeight) > 1e6 then
            return -500
        end
        return destroyHeight
    end

    local function currentHomeY(root: BasePart): number
        if homeCFrame then
            return homeCFrame.Position.Y
        end
        return root.Position.Y
    end

    -- Lowest altitude the farm is allowed to ask for: the round's start minus
    -- the depth budget, and never anywhere near where the game deletes parts.
    local function floorBudgetY(root: BasePart): number
        return math.max(destroyHeightY() + 40, currentHomeY(root) - MAX_BELOW_DEPTH)
    end

    local function clampBelowTarget(position: Vector3, root: BasePart): Vector3
        local lowest: number = floorBudgetY(root)
        local highest: number = currentHomeY(root) + 80
        return Vector3.new(
            position.X,
            math.clamp(position.Y, lowest, highest),
            position.Z
        )
    end

    -- One control step for Below, written BEFORE the physics step so the
    -- velocity is the one physics integrates this frame. The old build only
    -- wrote velocity from the Heartbeat tick, so gravity piled up between
```

```
    -- writes and the body sank out of the map instead of flying.
    local function applyBelowControl(root: BasePart, humanoid: Humanoid?): ()
        if not hoverActive then
            return
        end
        local speed: number = math.clamp(settings.speed, 5, 60)
        local target: Vector3? = hoverTarget
        if target then
            target = clampBelowTarget(target :: Vector3, root)
        end
        local velocityX: number = 0
        local velocityY: number = 0
        local velocityZ: number = 0
        if target then
            local delta: Vector3 = (target :: Vector3) - root.Position
            local flat: Vector3 = Vector3.new(delta.X, 0, delta.Z)
            local flatDistance: number = flat.Magnitude
            if flatDistance > 0.35 then
                -- Full Farm Speed until the body is on the point: the old
                -- proportional taper is what made the flight look slow and
                -- flimsy on the final approach.
                local direction: Vector3 = flat / flatDistance
                velocityX = direction.X * speed
                velocityZ = direction.Z * speed
            end
            -- Proportional altitude hold: this exactly cancels the falling
            -- the physics step would otherwise add.
            velocityY = math.clamp(delta.Y * 10, -speed, speed)
        end
        pcall(function(): ()
            root.AssemblyLinearVelocity = Vector3.new(velocityX, velocityY, velocityZ)
            root.AssemblyAngularVelocity = Vector3.zero
        end)
        if humanoid then
            pcall(function(): ()
                if not humanoid.PlatformStand then
                    humanoid.PlatformStand = true
                end
                if humanoid.AutoRotate then
                    humanoid.AutoRotate = false
                end
            end)
        end
    end

    -- No coin to chase: keep the altitude the body already holds instead of
    -- re-targeting its own centre every tick (that let gravity walk it down).
    local function holdAltitude(root: BasePart): ()
        local targetY: number = homeCFrame and homeCFrame.Position.Y or root.Position.Y
        if hoverTarget then
            targetY = hoverTarget.Y
        elseif hiddenUnderMap then
            targetY = currentHomeY(root) - BELOW_OFFSET
        end
        hoverTarget = clampBelowTarget(
            Vector3.new(root.Position.X, targetY, root.Position.Z),
            root
        )
        hoverActive = true
    end

    -- Round start only ("Stay Under When Round starts"): the body waits under
    -- the map while there is nothing to farm, then the farm starts normally in
    -- whichever mode is selected. Works for Walk too: leaving the hold puts the
    -- body back on the surface at the round's start point, so walking resumes
    -- from solid ground instead of from inside the geometry.
    local function holdUnderMap(
        character: Model,
        humanoid: Humanoid,
        root: BasePart
    ): ()
        hiddenUnderMap = true
        forceNoClipCharacter(character)
        local basePosition: Vector3 = if homeCFrame
            then homeCFrame.Position
            else root.Position
        local hidePos: Vector3 = clampBelowTarget(
            basePosition - Vector3.new(0, BELOW_OFFSET + 4, 0),
            root
        )
        hoverTarget = hidePos
        hoverActive = settings.mode ~= "Walk"
        pcall(function(): ()
            humanoid.PlatformStand = true
            humanoid.AutoRotate = false
            humanoid:Move(Vector3.zero, false)
            if settings.mode == "Walk" then
                if savedRootSize == nil then
                    savedRootSize = root.Size
                end
                root.Size = HITBOX_SIZE
            end
            if (root.Position - hidePos).Magnitude > 6 then
                root.CFrame = CFrame.new(hidePos)
                    * CFrame.Angles(0, yawOf(root.CFrame), 0)
                root.AssemblyLinearVelocity = Vector3.zero
                root.AssemblyAngularVelocity = Vector3.zero
            end
        end)
        if not lastSteppedPosition then lastSteppedPosition = root.Position end
    end

    -- Puts the body back inside the farm's altitude budget after a fall.
    -- Below: back under the map at the round's start, still NoClipped.
    -- Walk: on the floor the round started on, upright, collisions back.
    local function recoverFromVoid(
        character: Model,
        humanoid: Humanoid,
        root: BasePart
    ): ()
        local homeX: number = homeCFrame and homeCFrame.Position.X or root.Position.X
        local homeZ: number = homeCFrame and homeCFrame.Position.Z or root.Position.Z
        local homeY: number = currentHomeY(root)
        if settings.mode == "Walk" then
            local safePos: Vector3 = if homeCFrame
                then homeCFrame.Position
                else Vector3.new(homeX, destroyHeightY() + 120, homeZ)
            hoverActive = false
            hoverTarget = nil
            walkCarryDirection = nil
            walkCarryUntil = 0
            pcall(function(): ()
                if savedRootSize then
                    root.Size = savedRootSize
                    savedRootSize = nil
                end
                character:PivotTo(CFrame.new(safePos))
                root.CFrame = CFrame.new(safePos)
                root.AssemblyLinearVelocity = Vector3.zero
                root.AssemblyAngularVelocity = Vector3.zero
                humanoid.PlatformStand = false
                humanoid.AutoRotate = true
                humanoid:ChangeState(Enum.HumanoidStateType.Running)
            end)
            walkBodyReady = false
            return
        end
        local recoverY: number = math.min(
            homeY - BELOW_OFFSET,
            homeY + 8
        )
        recoverY = math.max(recoverY, destroyHeightY() + 60)
        local target: Vector3 = Vector3.new(homeX, recoverY, homeZ)
        hiddenUnderMap = true
        forceNoClipCharacter(character)
        hoverTarget = target
        hoverActive = true
        pcall(function(): ()
            if savedRootSize == nil then
                savedRootSize = root.Size
            end
            root.Size = HITBOX_SIZE
            humanoid.PlatformStand = true
            humanoid.AutoRotate = false
            root.CFrame = CFrame.new(target)
                * CFrame.Angles(0, yawOf(root.CFrame), 0)
            root.AssemblyLinearVelocity = Vector3.zero
            root.AssemblyAngularVelocity = Vector3.zero
        end)
    end

    -- Detects when the player's character is inside the loaded MM2 map,
    -- including the pre-round countdown seconds right after teleporting
    -- from the lobby before roles or coins have spawned.
    local function isCharacterInActiveRound(root: BasePart, knownMap: Instance?): boolean
        if type(findMM2Map) ~= "function" then
            return false
        end
        local map: Instance? = knownMap or findMM2Map()
        if not map then
            return false
        end
        if type(getPlayerRole) == "function" then
            local role: any = getPlayerRole(LocalPlayer)
            if role == "Dead" then
                return false
            end
            if (role == "None" or role == nil)
                and type(isRoundLifecycleActive) == "function"
                and isRoundLifecycleActive() == false then
                -- If round lifecycle explicitly says false AND we are near
                -- the lobby (or no map geometry exists), stay in lobby state.
                local lobbyPartCheck: BasePart? = findLobbyReferencePart()
                if lobbyPartCheck then
                    local okLD: boolean, ld: any = pcall(function(): any
                        return (root.Position - lobbyPartCheck.Position).Magnitude
                    end)
                    if okLD and type(ld) == "number" and ld <= 350 then
                        return false
                    end
                end
            end
        end
        local lobbyPart: BasePart? = findLobbyReferencePart()
        if lobbyPart then
            local okLobbyDist: boolean, lobbyDist: any = pcall(function(): any
                return (root.Position - lobbyPart.Position).Magnitude
            end)
            if okLobbyDist and type(lobbyDist) == "number" and lobbyDist <= 350 then
                return false
            end
        end
        if safeZonePlatform and safeZonePlatform.Parent then
            local okZoneDist: boolean, zoneDist: any = pcall(function(): any
                return (root.Position - safeZonePlatform.Position).Magnitude
            end)
            if okZoneDist and type(zoneDist) == "number" and zoneDist <= 120 then
                return true
            end
        end
        local refPart: BasePart? = nil
        local spawns: Instance? = (map :: Instance):FindFirstChild("Spawns")
        if spawns then
            refPart = spawns:FindFirstChildWhichIsA("BasePart", true)
        end
        if not refPart then
            local coinContainer: Instance? = (map :: Instance):FindFirstChild("CoinContainer")
            if coinContainer then
                refPart = coinContainer:FindFirstChildWhichIsA("BasePart", true)
            end
        end
        if not refPart then
            refPart = (map :: Instance):FindFirstChildWhichIsA("BasePart", true)
        end
        if refPart then
            local okDist: boolean, dist: any = pcall(function(): any
                return (root.Position - refPart.Position).Magnitude
            end)
            if okDist and type(dist) == "number" and dist > 1500 then
                return false
            end
            return true
        end
        if type(isRoundLifecycleActive) == "function"
            and isRoundLifecycleActive() ~= true then
            return false
        end
        return true
    end

    local function restoreCharacter(returnHome: boolean, noReposition: boolean?): ()
        local wasMovedByFarm: boolean = engaged or hiddenUnderMap
        local character: Model?, humanoid: Humanoid?, root: BasePart? =
            characterParts()
        -- The old body can disappear before the next round's body exists.
        -- Restore the body we actually modified, not a newly spawned one.
        local movedCharacter: Model? = engagedCharacter or (hiddenUnderMap and homeCharacter)
        if wasMovedByFarm and movedCharacter and movedCharacter ~= character then
            character = movedCharacter
            humanoid = movedCharacter:FindFirstChildOfClass("Humanoid") :: Humanoid?
            root = movedCharacter:FindFirstChild("HumanoidRootPart") :: BasePart?
        end
        for part: Instance, was: boolean in pairs(savedCollide) do
            pcall(function(): ()
                (part :: BasePart).CanCollide = was
            end)
        end
        table.clear(savedCollide)
        if humanoid and wasMovedByFarm then
            pcall(function(): ()
                humanoid.PlatformStand = false
                humanoid.AutoRotate = true
                if humanoid.Sit then
                    humanoid.Sit = false
                end
                if savedWalkSpeed then
                    humanoid.WalkSpeed = savedWalkSpeed
                end
                humanoid:Move(Vector3.zero, false)
                humanoid:ChangeState(Enum.HumanoidStateType.Running)
            end)
        end
        savedWalkSpeed = nil
        if root and wasMovedByFarm then
            pcall(function(): ()
                if savedRootSize then
                    root.Size = savedRootSize
                end
                root.AssemblyLinearVelocity = Vector3.zero
                root.AssemblyAngularVelocity = Vector3.zero
                if noReposition then
                    return
                end
                -- Return the character to the exact last position
                -- it had right before Auto Farm started (homeCFrame),
                -- never to a random spawn point, and during active
                -- farming strictly only when the player has 50 coins.
                -- Once that 50-coin teleport has cleanly happened once,
                -- never teleport the character anywhere else in this round.
                local allowReturnHome: boolean = returnHome
                    and not teleportedHomeThisRound
                    and (bagAmount >= 50 or not enabled)
                    and homeCFrame ~= nil
                    and (not homeCharacter or homeCharacter == character)
                if allowReturnHome and homeCFrame then
                    local destPosition: Vector3 = resolveSafeAboveGroundPosition(
                        homeCFrame.Position,
                        character
                    )
                    local mapReturn: CFrame = CFrame.new(destPosition)
                        * CFrame.Angles(0, yawOf(homeCFrame), 0)
                    local zoneDestination: Vector3? = nil
                    if enabled and settings.safeZone and bagAmount >= 50 then
                        zoneDestination = ensureSafeZonePlatform(homeCFrame.Position)
                        if zoneDestination then destPosition = zoneDestination end
                    end
                    local targetCF: CFrame = CFrame.new(destPosition)
                        * CFrame.Angles(0, yawOf(homeCFrame), 0)
                    if character then
                        pcall(function(): ()
                            character:PivotTo(targetCF)
                        end)
                    end
                    root.CFrame = targetCF
                    if zoneDestination then
                        parkedCharacter, parkedMap = character, currentRoundMap
                        parkedReturn, parkedPosition = mapReturn, zoneDestination
                    end
                    if bagAmount >= 50 then
                        teleportedHomeThisRound = true
                        homeCFrame = nil
                    end
                elseif not teleportedHomeThisRound then
                    -- Coming out of the round-start hold: the body is
                    -- underneath the floor, so a raycast from there finds
                    -- nothing. Go back to the round's start point instead,
                    -- which was recorded on the surface.
                    local safePos: Vector3 = if hiddenUnderMap and homeCFrame
                        then homeCFrame.Position
                        else resolveSafeAboveGroundPosition(root.Position, character)
                    local targetCF: CFrame = CFrame.new(safePos)
                        * CFrame.Angles(0, yawOf(root.CFrame), 0)
                    if character then
                        pcall(function(): ()
                            character:PivotTo(targetCF)
                        end)
                    end
                    root.CFrame = targetCF
                end
            end)
        end
        hiddenUnderMap = false
        savedRootSize = nil
        engaged = false
        engagedMode = nil
        engagedCharacter = nil
        hoverActive = false
        hoverTarget = nil
        lastSteppedPosition = nil
        walkCarryDirection = nil
        walkCarryUntil = 0
        walkBodyReady = false
    end

    local function stillParked(): boolean
        local character: Model?, humanoid: Humanoid?, root: BasePart? = characterParts()
        if not parkedReturn or not parkedPosition or character ~= parkedCharacter
            or not root or not humanoid or humanoid.Health <= 0 then return false end
        return (root.Position - (parkedPosition :: Vector3)).Magnitude <= 120
    end

    leaveSafeZone = function(cancelBusy: boolean?, quiet: boolean?): boolean
        if not stillParked() then
            parkedCharacter, parkedMap, parkedReturn, parkedPosition = nil, nil, nil, nil
            destroySafeZonePlatform()
            return true
        end
        if cancelBusy then
            roundGeneration += 1
            busy = false
        end
        local character: Model?, _, root: BasePart? = characterParts()
        local okMap: boolean, map: any = pcall(function(): any
            return type(findMM2Map) == "function" and findMM2Map() or nil
        end)
        if not okMap or not parkedMap or map ~= parkedMap or not parkedMap.Parent or not root then
            -- The old map is gone; leave the floor in place rather than
            -- teleport to a stale location or drop the player into void.
            if not quiet then notify("Auto Farm: waiting in Safe Zone; the original map is unavailable.") end
            return false
        end
        local destination: CFrame = parkedReturn :: CFrame
        local moved: boolean = pcall(function(): ()
            if character then pcall(function(): () character:PivotTo(destination) end) end
            root.CFrame = destination
            root.AssemblyLinearVelocity = Vector3.zero
            root.AssemblyAngularVelocity = Vector3.zero
        end)
        if not moved then return false end
        parkedCharacter, parkedMap, parkedReturn, parkedPosition = nil, nil, nil, nil
        destroySafeZonePlatform()
        return true
    end

    -- A round may end while the character is absent, so the ordinary
    -- lobby tick is not guaranteed to run. Invalidate full-bag and pickup
    -- state independently of that tick; never teleport to an old map here.
    local function clearRound(): ()
        roundGeneration += 1
        busy = false
        if engaged or hiddenUnderMap then
            -- Lift a held/underground body back to the map only while it
            -- is still demonstrably on the SAME round map. Never move a
            -- body already teleported to a lobby or a replacement map.
            local character: Model?, humanoid: Humanoid?, root: BasePart? = characterParts()
            local movedCharacter: Model? = engagedCharacter or (hiddenUnderMap and homeCharacter)
            local sameMap: boolean = currentRoundMap ~= nil
                and type(findMM2Map) == "function" and findMM2Map() == currentRoundMap
            local mayReposition: boolean = sameMap and character ~= nil
                and character == movedCharacter and humanoid ~= nil and humanoid.Health > 0
                and root ~= nil and isCharacterInActiveRound(root :: BasePart, currentRoundMap)
            restoreCharacter(false, not mayReposition)
        end
        wasInActiveRound = false
        currentRoundMap = nil
        busyLobbyPart = nil
        busyLobbyScanAt = 0
        bagAmount = 0
        bagLimit = 50
        fullHandled = false
        roundFullReached = false
        teleportedHomeThisRound = false
        homeCFrame = nil
        homeCharacter = nil
        -- scanCache can be Coin ESP's shared list: do not mutate it.
        scanCache = {}
        scanAt = 0
        table.clear(retryAt)
        table.clear(attempts)
        table.clear(blacklist)
        stuckPart = nil
        stuckSince = 0
        lastTouchPulse = 0
        travelPart = nil
        travelBestDistance = math.huge
        travelProgressAt = 0
        if not stillParked() then
            parkedCharacter, parkedMap, parkedReturn, parkedPosition = nil, nil, nil, nil
            destroySafeZonePlatform()
        end
    end

    local function enterRound(character: Model, root: BasePart, map: Instance): ()
        if wasInActiveRound and currentRoundMap == map then
            return
        end
        clearRound()
        wasInActiveRound = true
        currentRoundMap = map
        homeCFrame = CFrame.new(root.Position)
            * CFrame.Angles(0, yawOf(root.CFrame), 0)
        homeCharacter = character
    end

    local function engage(
        character: Model,
        humanoid: Humanoid,
        root: BasePart
    ): ()
        if homeCharacter ~= character or not homeCFrame then
            local basePos: Vector3 = if hiddenUnderMap
                then root.Position + Vector3.new(0, BELOW_OFFSET + 4, 0)
                else root.Position
            homeCFrame = CFrame.new(basePos) * CFrame.Angles(0, yawOf(root.CFrame), 0)
            homeCharacter = character
        end
        if engaged and engagedMode ~= nil and engagedMode ~= settings.mode then
            -- Mode switched mid-round: undo the other mode's body changes.
            restoreCharacter(false)
        end
        engagedMode = settings.mode
        if engagedCharacter ~= nil and engagedCharacter ~= character then
            -- New body (respawn): the previous restore belongs to
            -- the old character; remember this one's start point.
            restoreCharacter(false)
            engagedMode = settings.mode
            if homeCharacter ~= character or not homeCFrame then
                homeCFrame = root.CFrame
                homeCharacter = character
            end
        end
        engagedCharacter = character
        if not engaged then
            engaged = true
            walkLastDistance = math.huge
            walkStuckSince = os.clock()
            if settings.mode ~= "Walk" then
                hiddenUnderMap = false
                forceNoClipCharacter(character)
                pcall(function(): ()
                    local startPos: Vector3 = root.Position
                    if homeCFrame and startPos.Y >= homeCFrame.Position.Y - 1.0 then
                        startPos = Vector3.new(
                            startPos.X,
                            homeCFrame.Position.Y - BELOW_OFFSET,
                            startPos.Z
                        )
                        root.CFrame = CFrame.new(startPos)
                            * CFrame.Angles(0, yawOf(root.CFrame), 0)
                        humanoid.PlatformStand = true
                        humanoid.AutoRotate = false
                        if savedRootSize == nil then
                            savedRootSize = root.Size
                        end
                        root.Size = HITBOX_SIZE
                    end)
                end)
                -- The server can teleport before the first Stepped callback.
                lastSteppedPosition = root.Position
            end
            if settings.mode == "Walk" then
                -- Walk on the real ground: collisions stay exactly as they are
                -- (NoClip would drop the character through the floor), and the
                -- slider is the WalkSpeed. The body is normalised once per
                -- engagement: a Below session (or the under-map hold) can leave
                -- PlatformStand, NoClip and the grown hitbox on, and a body in
                -- that state ignores Humanoid:Move completely -- which is why
                -- Walk "did nothing".
                hoverActive = false
                hoverTarget = nil
                if not walkBodyReady then
                    walkBodyReady = true
                    hiddenUnderMap = false
                    walkCarryDirection = nil
                    walkCarryUntil = 0
                    for part: Instance, was: boolean in pairs(savedCollide) do
                        pcall(function(): ()
                            ((part :: any) :: BasePart).CanCollide = was
                        end)
                    end
                    table.clear(savedCollide)
                    pcall(function(): ()
                        if savedRootSize then
                            root.Size = savedRootSize
                            savedRootSize = nil
                        end
                        humanoid.PlatformStand = false
                        humanoid.AutoRotate = true
                        humanoid:Move(Vector3.zero, false)
                        if humanoid.Sit then
                            humanoid.Sit = false
                        end
                    end)
                end
                if savedWalkSpeed == nil then
                    savedWalkSpeed = humanoid.WalkSpeed
                end
                pcall(function(): ()
                    humanoid.WalkSpeed =
                        math.clamp(settings.speed, 5, 60)
                end)
                return
            end
            -- Below keeps NoClip: every character part (tools
            -- included) is forced non-collidable each tick, so the
            -- flight never bumps a platform edge on the way to a coin.
            walkBodyReady = false
            forceNoClipCharacter(character)
            hoverActive = true
        end

        -- Fly to a destination (Below). The actual motion is done by
        -- applyBelowControl on Stepped; this only records the target and runs one
        -- control step so callers that move once per tick still see progress.
        -- Returns the distance left to the (clamped) target.
        local function moveTowards(
            root: BasePart,
            destination: Vector3,
            _deltaTime: number?
        ): number
            local target: Vector3 = clampBelowTarget(destination, root)
            hoverTarget = target
            hoverActive = true
            applyBelowControl(root, nil)
            return (target - root.Position).Magnitude
        end

        -- Walk mode movement: point the humanoid at the coin and let the
        -- engine's own locomotion do the walking. Returns the horizontal distance
        -- left. If the distance stops shrinking (a wall, a ledge, or a game whose
        -- control module ignores Humanoid:Move) a short horizontal velocity carry
        -- is used for a second, and the caller's stuck timer blacklists coins
        -- walking truly cannot reach.
        local function walkTowards(
            humanoid: Humanoid,
            root: BasePart,
            destination: Vector3
        ): number
            -- Walking never uses the flight controller.
            hoverActive = false
            hoverTarget = nil
            local delta: Vector3 = destination - root.Position
            local flat: Vector3 = Vector3.new(delta.X, 0, delta.Z)
            local distance: number = flat.Magnitude
            local now: number = os.clock()
            if distance < 1.4 then
                walkCarryDirection = nil
                walkCarryUntil = 0
                pcall(function(): ()
                    humanoid:Move(Vector3.zero, false)
                end)
                return distance
            end
            if distance < walkLastDistance - 0.3 then
                walkLastDistance = distance
                walkStuckSince = now
                walkCarryUntil = 0
            elseif now - walkStuckSince > 2.0 then
                walkStuckSince = now
                walkCarryUntil = now + 1.0
                walkCarryDirection = flat / distance
                pcall(function(): ()
                    humanoid.Jump = true
                end)
            end
            if walkCarryUntil > now and walkCarryDirection then
                local speed: number = math.clamp(settings.speed, 5, 60)
                local direction: Vector3 = walkCarryDirection :: Vector3
                pcall(function(): ()
                    local current: Vector3 = root.AssemblyLinearVelocity
                    root.AssemblyLinearVelocity = Vector3.new(
                        direction.X * speed,
                        math.min(current.Y, 0),
                        direction.Z * speed
                    )
                    humanoid:Move(direction, false)
                end)
            else
                walkCarryDirection = nil
                pcall(function(): ()
                    humanoid:Move(flat.Unit, false)
                end)
            end
            return distance
        end

        -- The Coin_Server part that pays for a pickup entry (ESP hands
        -- out the visual, the server part is its parent).
        local function serverPartOf(part: Instance): Instance?
            if part.Name == "Coin_Server" then
                return part
            end
            local parent: Instance? = part.Parent
            if parent and parent.Name == "Coin_Server" then
                return parent
            end
            return part
        end

        -- Still there to be collected? A coin somebody already took
        -- keeps its entry in the throttled scan for a moment, and the
        -- old farm flew up to that ghost and hovered over it. The
        -- attribute/parent check runs on the chosen coin every tick,
        -- so a collected coin stops being a target at once.
        local function stillCollectable(part: Instance): boolean
            if not part.Parent then
                return false
            end
            local server: Instance? = serverPartOf(part)
            local ok: boolean, usable: any = pcall(function(): boolean
                return part:GetAttribute("Collected") ~= true
                    and part:GetAttribute("Delete") ~= true
                    and server ~= nil
```

```
                and server:GetAttribute("Collected") ~= true
                and server:GetAttribute("Delete") ~= true
        end)
        return ok and usable == true
    end

    -- The gun grab is an in-round action for non-Murderer roles: as
    -- Murderer it must never run, and in the lobby there is no round,
    -- no murderer and no dropped gun to shoot with. Note that once an
    -- Innocent picks up the dropped gun, MM2 promotes their role to
    -- "Hero" / "Sheriff", so those roles must remain allowed while
    -- equipping and continuously shooting the Murderer.
    local function canGrabGun(): boolean
        if type(getPlayerRole) ~= "function" then
            return false
        end
        local role: any = getPlayerRole(LocalPlayer)
        if role ~= "Innocent" and role ~= "Sheriff" and role ~= "Hero" then
            return false
        end
        if type(isRoundLifecycleActive) ~= "function"
            or isRoundLifecycleActive() ~= true then
            return false
        end
        if type(findMM2Map) ~= "function" or not findMM2Map() then
            return false
        end
        return true
    end

    -- Immediate pickup attempt: pulse the touch between our root and the
    -- dropped gun's part instead of travelling or sleeping. True only
    -- when the gun has already arrived; the farm can retry on a later frame.
    local function touchDroppedGun(
        drop: Instance,
        root: BasePart
    ): boolean
        if type(state.fireTouchInterest) ~= "function" then
            return false
        end
        local touchPart: Instance? = nil
        if drop:IsA("BasePart") then
            touchPart = drop
        elseif drop:IsA("Model") then
            touchPart = drop:FindFirstChild("Handle", true)
                or drop:FindFirstChildWhichIsA("BasePart", true)
        end
        if not touchPart then
            return false
        end
        pcall(state.fireTouchInterest, root, touchPart, 0)
        pcall(state.fireTouchInterest, root, touchPart, 1)
        return type(getPlayerWeapon) == "function"
            and getPlayerWeapon(LocalPlayer, "Gun") ~= nil
    end

    -- Full-bag Auto Grab gun and Shoot: fetch the dropped sheriff gun
    -- if we do not already carry it, equip it, and continuously shoot
    -- the murderer until the murderer is eliminated or the round ends.
    local function grabGunAndShoot(isCurrent: () -> boolean): ()
        if not isCurrent() or not canGrabGun() then
            return
        end
        local shootLoopDeadline: number = os.clock() + 30
        while os.clock() < shootLoopDeadline and isCurrent() and settings.grabGunShoot do
            if not canGrabGun() then
                return
            end
            local murderer: Player? = nil
            if type(findMurderer) == "function" then
                murderer = findMurderer()
            end
            if not murderer
                or (type(isPlayerAlive) == "function"
                    and not isPlayerAlive(murderer)) then
                return
            end
            local character: Model?, humanoid: Humanoid?,
                startRoot: BasePart? = characterParts()
            if not character or not humanoid or not startRoot
                or humanoid.Health <= 0 then
                return
            end
            local gun: Tool? = nil
            if type(getPlayerWeapon) == "function" then
                gun = getPlayerWeapon(LocalPlayer, "Gun")
            end
            if not gun and type(findDroppedGun) == "function" then
                local firstDrop: Instance? = findDroppedGun()
                if firstDrop and touchDroppedGun(firstDrop, startRoot) then
                    gun = getPlayerWeapon(LocalPlayer, "Gun")
                end
                local pickupDeadline: number = os.clock() + 5
                while os.clock() < pickupDeadline and not gun do
                    if not isCurrent() or not canGrabGun() or not settings.grabGunShoot then
                        return
                    end
                    local curChar: Model?, curHum: Humanoid?,
                        root: BasePart? = characterParts()
                    local drop: Instance? = findDroppedGun()
                    if not curChar or not curHum or not root or not drop then
                        break
                    end
                    if touchDroppedGun(drop, root) then
                        gun = getPlayerWeapon(LocalPlayer, "Gun")
                        if gun then
                            break
                        end
                    end
                    if teleportedHomeThisRound or roundFullReached then
                        break
                    end
                    local dropPosition: Vector3? = nil
                    pcall(function(): ()
                        if drop:IsA("BasePart") then
                            dropPosition = (drop :: BasePart).Position
                        elseif drop:IsA("Model") then
                            dropPosition = (drop :: Model):GetPivot().Position
                        end
                    end)
                    if not dropPosition then
                        break
                    end
                    engage(curChar, curHum, root)
                    local remaining: number
                    if settings.mode == "Walk" then
                        remaining = walkTowards(
                            curHum, root, dropPosition :: Vector3)
                    else
                        remaining = moveTowards(
                            root,
                            (dropPosition :: Vector3) + Vector3.new(0, 1, 0),
                            0.05
                        )
                    end
                    if remaining < 2.5 then
                        task.wait(0.05)
                        gun = getPlayerWeapon(LocalPlayer, "Gun")
                        if gun then
                            break
                        end
                    end
                    task.wait()
                end
            end

            if not gun then
                -- Wait briefly for the gun to drop or become available while the round is active
                task.wait(0.15)
                continue
            end

            character, humanoid = characterParts()
            if not character or not humanoid or humanoid.Health <= 0 then
                return
            end
            if gun.Parent ~= character then
                pcall(function(): ()
                    humanoid:EquipTool(gun :: Tool)
                end)
                local equipWait: number = os.clock() + 0.15
                while gun.Parent ~= character and os.clock() < equipWait and isCurrent() do
                    task.wait()
                end
            end
            if isCurrent() and gun.Parent == character and canGrabGun() then
                shootMurderer(murderer)
            end
            task.wait(0.12)
        end
    end

    -- The full-bag sequence, in the user's order: Kill All,
    -- return to exact pre-farm position, Auto Grab gun and Shoot,
    -- reset, then the full-bag flings (which never depend on the reset).
    -- Runs once per 50-coin fill.
    local function runFullSequence(isCurrent: () -> boolean): ()
        local ok: boolean, err: any = pcall(function(): ()
            if not isCurrent() then return end
            -- Everything in this sequence happens ONLY when the
            -- bag is full (50): Kill All, return to the character's
            -- last position before Auto Farm started (homeCFrame),
            -- gun grab + continuous shooting, reset, and flings.
            -- None of it runs during ordinary farming.
            if settings.killAllOnFull then
                killAll(isCurrent)
                task.wait()
            end
            if not isCurrent() then return end
            restoreCharacter(true)
            if settings.grabGunShoot then
                grabGunAndShoot(isCurrent)
                task.wait(0.15)
            end
            if not isCurrent() then return end
            if settings.resetOnFull then
                local oldCharacter: Model? = LocalPlayer.Character
                local _, humanoid: Humanoid? = characterParts()
                if humanoid then
                    pcall(function(): ()
                        humanoid.Health = 0
                    end)
                end
                -- Wait for the respawned character, checking
                -- every frame instead of four times a second.
                local deadline: number = os.clock() + 10
                while os.clock() < deadline and isCurrent() do
                    local character: Model?, newHumanoid: Humanoid?,
                        newRoot: BasePart? = characterParts()
                    if character and character ~= oldCharacter
                        and newHumanoid and newHumanoid.Health > 0
                        and newRoot then
                        break
                    end
                    task.wait()
                end
                task.wait(0.1)
            end
            -- Flings belong to the FULL BAG, reset or not: with a reset
            -- they meet the fresh character first (above), without one they
            -- simply happen now. The label says "after full bag" for exactly
            -- that reason.
            if not isCurrent() then return end
            if settings.flingMurder
                and type(findMurderer) == "function" then
                flingPlayer(findMurderer(), isCurrent)
            end
            if isCurrent() and settings.flingSheriff
                and type(findSheriff) == "function" then
                flingPlayer(findSheriff(), isCurrent)
            end
        end)
        if not ok then
            warn("Auto Farm full-bag sequence failed: " .. tostring(err))
        end
    end

    local function startBusySequence(action: (() -> boolean) -> ()): ()
        if busy then return end
        busy = true
        -- Resolve once per sequence; individual yielded actions only do a
        -- cheap distance check, never a workspace scan for each pose.
        busyLobbyPart = findLobbyReferencePart()
        busyLobbyScanAt = os.clock() + 0.5
        local sequenceRound: number = roundGeneration
        local sequenceMap: Instance? = currentRoundMap
        local observedCharacter: Model? = LocalPlayer and LocalPlayer.Character
        local mapChanged: boolean = false
        local function isCurrent(): boolean
            if farmGeneration ~= enableGeneration or roundGeneration ~= sequenceRound then
                return false
            end
            local character: Model?, _, root: BasePart? = characterParts()
            if sequenceMap and not sequenceMap.Parent then return false end
            if character ~= observedCharacter then
                observedCharacter = character
                local map: Instance? = type(findMM2Map) == "function" and findMM2Map()
                if map and map ~= sequenceMap then mapChanged = true end
            end
            return not mapChanged and not isNearLobby(root, busyLobbyPart)
        end
        task.spawn(function(): ()
            local ok: boolean, err: any = pcall(action, isCurrent)
            if not ok then
                warn("Auto Farm sequence failed: " .. tostring(err))
            end
            if farmGeneration == enableGeneration and roundGeneration == sequenceRound then
                if isCurrent() then
                    busy = false
                else
                    local _, _, root: BasePart? = characterParts()
                    local inLobby: boolean = isNearLobby(root, busyLobbyPart)
                    clearRound()
                    setStatus(if inLobby then "Auto Farm: waiting in lobby" else "Auto Farm: waiting for round")
                end
            end
        end)
    end

    -- Wake the throttled cache immediately when a new pickup streams in,
    -- without scanning the whole workspace every frame while coins are absent.
    if workspace.DescendantAdded then
        featureConnections.MM2AutoFarmCoinAdded = workspace.DescendantAdded:Connect(
            function(object: Instance): ()
                if object:IsA("BasePart")
                    and (object.Name == "Coin_Server" or object.Name == "CoinVisual") then
                    fallbackScanAt = -math.huge
                    scanAt = -math.huge
                end
            end
        )
    end

    local remotes: any = core.mm2GameplayRemotes
    local roundEnd: Instance? = remotes and remotes:FindFirstChild("RoundEndFade")
    if roundEnd and roundEnd:IsA("RemoteEvent") then
        featureConnections.MM2AutoFarmRoundEnd = (roundEnd :: any).OnClientEvent:Connect(
            function(): ()
                endedMap = currentRoundMap or (type(findMM2Map) == "function" and findMM2Map())
                waitingForNextRound = true
                clearRound()
            end
        )
    end
    local roundStart: Instance? = remotes and remotes:FindFirstChild("RoundStart")
    if roundStart and roundStart:IsA("RemoteEvent") then
        featureConnections.MM2AutoFarmRoundStart = (roundStart :: any).OnClientEvent:Connect(
            function(duration: any): ()
                if waitingForNextRound and tonumber(duration) and tonumber(duration) > 0 then
                    waitingForNextRound = false
                    endedMap = nil
                end
            end
        )
    end
    -- RoundStart's duration is not always available. These are independent
    -- positive new-round signals; unlike an old loaded map or a respawn,
    -- they can release the end-of-round gate for a reused map.
    for _, name: string in ipairs({"RoleSelect", "ShowRoleSelect", "ShowRoleSelectNew", "CoinsStarted"}) do
        local remote: Instance? = remotes and remotes:FindFirstChild(name)
        if remote and remote:IsA("RemoteEvent") then
            featureConnections["MM2AutoFarmStart_" .. name] = (remote :: any).OnClientEvent:Connect(
                function(): ()
                    if waitingForNextRound then
                        waitingForNextRound = false
                        endedMap = nil
                    end
                end
            )
        end
    end
end

local collected: Instance? = remotes
    and remotes:FindFirstChild("CoinCollected")
if collected and collected:IsA("RemoteEvent") then
    featureConnections.MM2AutoFarmCoinCollected =
        (collected :: any).OnClientEvent:Connect(
            function(_bag: any, amount: any, limit: any): ()
                -- A first coin can arrive before the next heartbeat.
                -- Enter its map before recording the amount, otherwise
                -- the following tick would immediately erase it.
                local character: Model?, humanoid: Humanoid?, root: BasePart? = characterParts()
                local map: Instance? = type(findMM2Map) == "function" and findMM2Map()
                if not character or not humanoid or humanoid.Health <= 0 or not root
                    or not map or not isCharacterInActiveRound(root, map)
                    or (waitingForNextRound and map == endedMap) then
                    return
                end
                if waitingForNextRound then
                    waitingForNextRound = false
                    endedMap = nil
                end
                enterRound(character, root, map)
                if type(limit) == "number" and limit >= 50 then
                    bagLimit = limit
                end
                if type(amount) == "number" then
                    if amount > bagAmount then
                        totalCoinsCollected += (amount - bagAmount)
                        refreshFarmOverlay()
                    end
                    bagAmount = amount
                end
                local targetLimit: number = math.max(50, bagLimit)
                if bagAmount < targetLimit then
                    fullHandled = false
                    roundFullReached = false
                end
            end
        )
end

-- Runs BEFORE the physics step: this is the frame's flight, both the
-- NoClip and the altitude/heading control. The task tick only picks the
-- destination, so a busy tick can never drop the body out of the map.
if RunService and RunService.Stepped then
    featureConnections.MM2AutoFarmStepped = RunService.Stepped:Connect(
        function(): ()
            if not hiddenUnderMap and not engaged then
                return
            end
            local character: Model? = LocalPlayer and LocalPlayer.Character
            local movedCharacter: Model? = engagedCharacter or (hiddenUnderMap and homeCharacter)
            if not character or (movedCharacter and movedCharacter ~= character) then
                return
            end
            local root: BasePart? =
                character:FindFirstChild("HumanoidRootPart") :: BasePart?
            if not root or not root.Parent then return end
            local position: Vector3 = root.Position
            if lastSteppedPosition and (position - lastSteppedPosition).Magnitude > 60 then
                -- The server can teleport the same character to the
                -- lobby before Heartbeat sees the phase change. Release
                -- flight/NoClip without moving to an old home first.
                restoreCharacter(false, true)
                return
            end
            lastSteppedPosition = position
            if hiddenUnderMap or settings.mode ~= "Walk" then
                forceNoClipCharacter(character)
            end
            if not hoverActive or settings.mode == "Walk" then
                return
            end
            local humanoid: Humanoid? =
                character:FindFirstChildOfClass("Humanoid")
            if humanoid and humanoid.Health > 0 then
                applyBelowControl(root, humanoid)
            end
        end
    )
end

local function farmTick(deltaTime: number): ()
    if type(deltaTime) == "number" and deltaTime > 0 and deltaTime < 5 then
        farmElapsedSeconds += deltaTime
        refreshFarmOverlay()
    end
    local character: Model?, humanoid: Humanoid?,
        root: BasePart? = characterParts()
    if not character or not humanoid or not root
        or humanoid.Health <= 0 then
        return
    end
    local map: Instance? = type(findMM2Map) == "function" and findMM2Map()
    -- A same-round full-bag sequence may be waiting for respawn.
    -- A different map or a lobby body must be allowed to cancel it,
    -- even when an old map remains loaded and RoundEnd was missed.
    if busy and (not map or map == currentRoundMap) then
        if not busyLobbyPart and os.clock() >= busyLobbyScanAt then
            busyLobbyPart = findLobbyReferencePart()
            busyLobbyScanAt = os.clock() + 0.5
        end
        if isNearLobby(root, busyLobbyPart) then
            clearRound()
            setStatus("Auto Farm: waiting in lobby")
        end
        return
    end

    -- If the Safe Zone row was switched OFF while its map was
    -- missing, keep the floor and retry until a safe exit exists.
    if stillParked() and not settings.safeZone and leaveSafeZone then
        if not leaveSafeZone(false, true) then
            setStatus("Auto Farm: waiting in Safe Zone for the original map")
            return
        end
    end
    -- Strict lobby & non-round guard: NEVER activate, move,
    -- or farm in the lobby or outside an active round on a map.
    if stillParked() and parkedMap ~= map then
        setStatus("Auto Farm: waiting for the round teleport")
        return
    end
    local inActiveRound: boolean = map ~= nil and isCharacterInActiveRound(root, map)
    if not inActiveRound then
        if wasInActiveRound or engaged or hiddenUnderMap then
            clearRound()
        end
        if waitingForNextRound then
            local lobby: BasePart? = findLobbyReferencePart()
            local reachedLobby: boolean = lobby ~= nil
                and (root.Position - (lobby :: BasePart).Position).Magnitude <= 350
            if not map or map ~= endedMap or reachedLobby then
                waitingForNextRound = false
                endedMap = nil
            end
        end
        setStatus("Auto Farm: waiting in lobby")
        return
    end
    if waitingForNextRound and map == endedMap then
        setStatus("Auto Farm: waiting for next round")
        return
    end
    if waitingForNextRound then
        waitingForNextRound = false
        endedMap = nil
    end
    enterRound(character, root, map :: Instance)
    if busy then
        return
    end

    local targetLimit: number = math.max(50, bagLimit)

    -- Stop farming ONLY when 50 coins (full bag) are detected!
    if bagAmount >= targetLimit then
        if not fullHandled then
            fullHandled = true
            roundFullReached = true
            if settings.killAllOnFull or settings.resetOnFull
                or settings.flingMurder or settings.flingSheriff
                or settings.grabGunShoot then
                startBusySequence(runFullSequence)
            else
                restoreCharacter(true)
            end
        elseif settings.grabGunShoot and canGrabGun() then
            startBusySequence(function(isCurrent: () -> boolean): ()
                grabGunAndShoot(isCurrent)
            end)
        else
            setStatus(string.format(
                "Auto Farm: full bag (%d/%d) - stopped at start position",
                bagAmount, targetLimit
            ))
        end
        return
    end

    if roundFullReached then
        return
    end

    -- Void guard. While bagAmount < 50, never teleport back to
    -- homeCFrame on top of the map; recover directly to the
    -- hidden spot underneath the map instead of falling further.
    local voidThreshold: number = math.max(
        destroyHeightY() + 40,
        currentHomeY(root) - VOID_RECOVER_DEPTH
    )
    if root.Position.Y <= voidThreshold then
        recoverFromVoid(character, humanoid, root)
        setStatus(
            "Auto Farm: recovered from the void - back inside the map"
        )
        return
    end

    -- Nearest pickup, from Coin ESP's own detection.
    local now: number = os.clock()
    local scanInterval: number = if #scanCache == 0 then 0.12 else 0.35
    if now - scanAt > scanInterval then
        scanAt = now
        scanCache = coinPickups()
    end
    local nearest: Instance? = nil
    local nearestPosition: Vector3? = nil
    local nearestDistance: number = math.huge
    for _, part: Instance in ipairs(scanCache) do
        if part.Parent
            and stillCollectable(part)
            and (not blacklist[part] or blacklist[part] < now)
            and (not retryAt[part] or retryAt[part] < now) then
            -- No pcall closure here: a Part's Position read cannot
            -- throw. The old form allocated one closure per coin per
            -- frame (~6000/s with 100 coins).
            if part:IsA("BasePart") then
                local position: Vector3 = (part :: BasePart).Position
                local distance: number = (position - root.Position).Magnitude
                if distance < nearestDistance then
                    nearest = part
                    nearestPosition = position
                    nearestDistance = distance
                end
            end
        end
    end
    if not nearest or not nearestPosition then
        -- Round starting or 0 coins currently in the server while
        -- bag < 50: if the server just teleported the character to
        -- its random map spawn, update homeCFrame
        -- so we never get stuck or teleport to a stale spot.
        -- A new random map spawn moves the body; follow it so the
        -- depth budget and the return point stay meaningful. This is
        -- only ever taken while the body is ON the map: deriving the
        -- reference altitude from a body that is already underneath
        -- it fed on itself (each tick lowered the reference, and the
        -- hold target with it, which is exactly how the farm used to
        -- sink out of the map).
        local bodyOnMap: boolean = homeCFrame ~= nil
            and root.Position.Y >= homeCFrame.Position.Y - 1
        if bagAmount == 0 and homeCFrame and not hiddenUnderMap and bodyOnMap then
            local dx: number = root.Position.X - homeCFrame.Position.X
            local dz: number = root.Position.Z - homeCFrame.Position.Z
            if math.sqrt(dx * dx + dz * dz) > 14
                and root.Position.Y > destroyHeightY() + 60 then
                local safeSpawnPos: Vector3 = Vector3.new(
                    root.Position.X,
                    root.Position.Y,
                    root.Position.Z
                )
                homeCFrame = CFrame.new(safeSpawnPos)
                    * CFrame.Angles(0, yawOf(root.CFrame), 0)
                homeCharacter = character
            end
        end
        travelPart = nil
        -- No coin to chase right now. At round start that is exactly
        -- what "Stay Under When Round starts" is for: wait under the
        -- map (both modes) until the first coins appear. Once the farm
        -- is engaged, just hold the position/altitude the body has --
        -- no dive, no teleport home, no waiting loop.
        if settings.stayUnderOnStart and not engaged then
            holdUnderMap(character, humanoid, root)
        elseif hiddenUnderMap then
            restoreCharacter(false)
            pcall(function(): ()
                humanoid:Move(Vector3.zero, false)
            end)
        elseif settings.mode == "Walk" then
            pcall(function(): ()
                humanoid:Move(Vector3.zero, false)
            end)
        else
            holdAltitude(root)
        end
        setStatus(string.format(
            "Auto Farm (%s): waiting for coin - bag %d/%d - speed %d u/s - ping %dms",
            settings.mode, bagAmount, targetLimit,
            math.round(math.clamp(settings.speed, 5, 60)),
            pingMillis()
        ))
        return
    end
    if hiddenUnderMap then
        -- First coin of the round: leave the round-start hold and start
        -- farming normally. Restore puts the body back on the surface.
        restoreCharacter(false)
    end
    -- Capture pre-farm homeCFrame and stay engaged while
    -- bagAmount < 50 inside the active round.
    engage(character, humanoid, root)
    local coinPosition: Vector3 = nearestPosition :: Vector3
    if travelPart ~= nearest then
        travelPart = nearest
        travelBestDistance = math.huge
        travelProgressAt = now
        walkLastDistance = math.huge
        walkStuckSince = now
        walkCarryUntil = 0
        walkCarryDirection = nil
    end
    local remaining: number
    if settings.mode == "Walk" then
        remaining = walkTowards(humanoid, root, coinPosition)
    else
        remaining = moveTowards(
            root,
            coinPosition + Vector3.new(0, -BELOW_OFFSET, 0),
            deltaTime
        )
    end
    -- Close enough to pay: the grown 4x8x4 hitbox already overlaps
    -- Coin_Server from 2.6 studs, so both thresholds are generous and
    -- the body does not have to crawl onto the exact centre.
    local reach: number = if settings.mode == "Walk" then 2.0 else 3.5
    local arrived: boolean = remaining <= reach
    if not arrived then
        if remaining < travelBestDistance - 0.35 then
            travelBestDistance = remaining
            travelProgressAt = now
        elseif now - travelProgressAt >= TRAVEL_STALL_SECONDS then
            -- Unreachable coins used to monopolize the nearest scan
            -- forever because only arrivals had a retry timeout.
            blacklist[nearest] = now + BLACKLIST_SECONDS
            travelPart = nil
            stuckPart = nil
            return
        end
    end
    if arrived then
        if stuckPart ~= nearest then
```

```
                    -- New coin: pulse on THIS tick instead of waiting for the
                    -- 0.25 s throttle to run out. That wait was the "moves a
                    -- bit later" feel.
                    stuckPart = nearest
                    stuckSince = now
                    lastTouchPulse = 0
                    if not attempts[nearest] then
                        attempts[nearest] = 0
                    end
                    attempts[nearest] = attempts[nearest] + 1
                end
                if now - lastTouchPulse > 0.12 then
                    lastTouchPulse = now
                    -- Belt and braces: some builds only pay on a physics
                    -- Touched, and a fully NoClipped body can miss that
                    -- pairing. Pulse the touch against Coin_Server itself.
                    local touchPart: Instance? = serverPartOf(nearest)
                    if type(state.fireTouchInterest) == "function"
                        and touchPart then
                        pcall(state.fireTouchInterest, root, touchPart, 0)
                        pcall(state.fireTouchInterest, root, touchPart, 1)
                    end
                end
                if now - stuckSince > DWELL_SECONDS
                    and stillCollectable(nearest) then
                    -- It did not pay: leave it and take the next coin. It is
                    -- retried on a later pass, and blacklisted after too many.
                    retryAt[nearest] = now + RETRY_SECONDS
                    stuckPart = nil
                    if (attempts[nearest] or 0) >= RETRY_LIMIT then
                        blacklist[nearest] = now + BLACKLIST_SECONDS
                        attempts[nearest] = nil
                    end
                end
            else
                stuckPart = nil
            end
            setStatus(string.format(
                "Auto Farm (%s): %d coins - bag %d/%d - speed %d u/s - ping %dms",
                settings.mode, #scanCache, bagAmount, targetLimit,
                math.round(math.clamp(settings.speed, 5, 60)),
                pingMillis()
            ))
        end

        local retryTickAt: number = 0
        local warnTickAt: number = -math.huge
        featureConnections.MM2AutoFarmTick = TaskManager:Connect(function(deltaTime: number): ()
            local now: number = os.clock()
            if now < retryTickAt then return end
            local ok: boolean, err: any = pcall(farmTick, deltaTime)
            if not ok then
                retryTickAt = now + 0.25
                if now >= warnTickAt then
                    warnTickAt = now + 5
                    warn("Auto Farm tick failed; retrying: " .. tostring(err))
                    setStatus("Auto Farm: retrying after a temporary error")
                end
            end
        end, nil, "Auto Farm tick")

        -- Turning the feature off runs this through
        -- disconnectFeatureConnection above: full restore, and the
        -- character is put back where the farm first grabbed it.
        featureConnections.MM2AutoFarmRestore = {
            Disconnect = function(): ()
                if leaveSafeZone then leaveSafeZone(false) end
                restoreCharacter(true)
            end,
        }
    end

    local AutoFarmFeature: any = createUniversalFeature(
        "Auto Farm",
        "Coin farming in Walk or Below (Hitbox) mode with full-bag actions.",
        6,
        toggleAutoFarm,
        {
            categoryName = "Utilities",
            parent = MM2Scroll,
            registry = mm2Features,
        }
    )
    state.mm2AutoFarmFeature = AutoFarmFeature

    if type(state.addSectionOption) == "function" then
        state.addSectionOption(AutoFarmFeature, "Overlay Info")
    end
    if type(state.addStatOption) == "function" then
        coinsCollectedStat = state.addStatOption(AutoFarmFeature, "Coins Collected", "0")
        farmTimeStat = state.addStatOption(AutoFarmFeature, "Time", "00:00")
    end

    addCycleOption(AutoFarmFeature, "Farm Mode", {"Walk", "Below (Hitbox)"}, 1,
        function(value: string): ()
            settings.mode = if value == "Walk" then "Walk" else "Below"
        end, "Walk: your character walks on the ground to each coin (nothing about the body is changed; the slider is WalkSpeed). Below (Hitbox): upright, NoClip, flying under the floor with the root hitbox grown so it still touches the coin from below.")
    addNumberOption(AutoFarmFeature, "Farm Speed", 16, 5, 60,
        function(value: number): ()
            settings.speed = value
        end, "Walk mode: WalkSpeed. Below mode: flight speed in studs/second -- if the server kicks you (Invalid position), lower it; raise it slowly to find your fastest safe speed.", 1)
    addToggleOption(AutoFarmFeature, "Stay Under When Round starts", true,
        function(value: boolean): ()
            settings.stayUnderOnStart = value == true
        end, "Hides your character under the map while the round is starting (both farm modes) and starts farming normally as soon as the first coins spawn. Turn it off to wait on the surface at your spawn point.")
    addToggleOption(AutoFarmFeature, "Safe Zone", true,
        function(value: boolean): ()
            settings.safeZone = value == true
            if not settings.safeZone and leaveSafeZone then leaveSafeZone(true) end
        end, "After 50 coins, use a safe platform. Turning this option or Auto Farm off returns the same character to the current map.")
    addToggleOption(AutoFarmFeature, "Reset When Full Coin Bag", true,
        function(value: boolean): ()
            settings.resetOnFull = value
        end, "Reset the character as soon as the coin bag fills.")
    addToggleOption(AutoFarmFeature, "Fling Murder after full bag", false,
        function(value: boolean): ()
            settings.flingMurder = value
        end, "When the coin bag fills (50): fling the Murderer. Works with or without Reset When Full Coin Bag.")
    addToggleOption(AutoFarmFeature, "Fling Sheriff after full bag", false,
        function(value: boolean): ()
            settings.flingSheriff = value
        end, "When the coin bag fills (50): fling the Sheriff. Works with or without Reset When Full Coin Bag.")
    addToggleOption(AutoFarmFeature, "Auto Grab gun and Shoot", false,
        function(value: boolean): ()
            settings.grabGunShoot = value
       end, "Only when the coin bag fills (50), only as Innocent and only during an active round (never as Murderer, never in the lobby): try an instant touch for the dropped sheriff gun, otherwise walk/fly to it, equip it, and fire Silent shots at the Murderer without WallCheck. Never during ordinary farming.")
    addToggleOption(AutoFarmFeature, "Auto Kill All When full coin bag", false,
        function(value: boolean): ()
            settings.killAllOnFull = value
        end, "As Murderer, when the bag fills: pull everyone in front of you and slash (YARHM Kill All).")
    addInformationOption(
        AutoFarmFeature,
    "The body moves at full Farm Speed to the nearest coin, touches it and moves on to the next one the same frame it is collected (a coin that does not pay is retried later, never waited on). Below (Hitbox) skims under the floor with the root hitbox grown so coins pay from underneath; Walk does it on the ground. If Below ever kicks you for Invalid position, lower Farm Speed. Turning it off restores collisions, hitbox and WalkSpeed; from Safe Zone it returns you to the current map."
    )

    activeCleanup = function(): ()
        state.mm2AutoFarmFeature = nil
        toggleAutoFarm(false)
    end
    Module.Events = featureConnections
    Module.Initialized = true
    return Module
end

function Module.destroy(): ()
    if not Module.Initialized then
        return
    end
    Module.Initialized = false
    pcall(activeCleanup)
    activeCleanup = function(): ()
    end
    Module.Events = {}
    Module.Runtime = nil
end

return Module
```

```
--------------------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/MM2/Other/AutoFarm.lua
--------------------------------------------------------------------------------------------


--------------------------------------------------------------------------------------------
ARCHIVO: src/games/MM2/Other/AutoUnbox.lua
BYTES: 35210
SHA-256: 969a05022e9356d62dfda946fe351179d77b14018b285e471adf46ca79951f69
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------------------
-- MM2 Auto Unbox: opens the game's own mystery boxes through the same remote
-- the shop uses, with the box and the currency chosen in the menu.
--
-- What the place dump says a box is. The box catalogue lives in
-- `ReplicatedStorage/Database/Sync`: `Sync.MysteryBox` (Mystery Box #1/#2,
-- Knife Box #1-#5, Rainbow Box, Gun Box #1-#3) and `Sync.Eggs` (Common Egg).
-- Event boxes live under `SharedServices/EventInfoService1`: the
-- `MysteryBoxes` folder (Christmas2024Box) and one `MysteryBox` module per
-- entry of `EventList` (Summer2025Box, Halloween2025Box). Every box id also
-- appears in `Sync.NewShop` with its `Price` table, whose keys ARE the
-- currencies it accepts: classic boxes take Coins/Gems/Key, the Christmas box
-- takes SnowTokens2024/SnowKey2024, the Summer box BeachBalls2025/SummerKey2025
-- and the Halloween box Candies2025/HalloweenKey2025.
--
-- How the game opens one. The shop calls
-- `Remotes.Shop.OpenCrate:InvokeServer(boxId, "MysteryBox", currency)` (or
-- `("...", "Eggs", currency)` for pets) and gets the rewarded item id back.
-- The reveal is pure client cosmetics: `Remotes.Shop.BoxController:Fire(list)`
-- plays the official opening animation and reports `CrateComplete`, or
-- `Remotes.Shop.EggController:Fire(eggId, petId)` hatches pets. This module
-- replays exactly that flow; it never touches a Robux prompt.
--
-- Which currency to pay with. The round announces the coin family it pays out
-- through `Remotes/Gameplay/CoinsStarted` (bag names `Coin`, `Candy`,
-- `SnowToken`, `BeachBall`, `Egg`), and `Workspace` carries `CurrentEvent`,
-- `isChristmas`, `IsSummer`, `IsEaster` and `isValentines`. Auto maps that
-- family onto the currencies the chosen box accepts.
local Module = {
    Name = "MM2 Auto Unbox",
    Events = {} :: {[string]: any},
    Initialized = false,
    Runtime = nil :: any,
}

local activeCleanup: () -> () = function(): () end

type BoxEntry = {
    id: string,
    name: string,
    category: string,
    prices: {[string]: number},
}

local AUTO_CURRENCY: string = "Auto"
local REVEAL_ON: string = "On"
local REVEAL_OFF: string = "Off"
local DEFAULT_BOX_LABEL: string = "Mystery Box #1"

-- The catalogue as the dump ships it, used when the Sync module cannot be
-- required. Order matches the shop: classic boxes, eggs, event boxes.
local FALLBACK_BOXES: {BoxEntry} = {
    {id = "MysteryBox1", name = "Mystery Box #1", category = "MysteryBox", prices = {Coins = 1000, Gems = 100, Key = 1}},
    {id = "MysteryBox2", name = "Mystery Box #2", category = "MysteryBox", prices = {Coins = 1000, Gems = 100, Key = 1}},
    {id = "KnifeBox1", name = "Knife Box #1", category = "MysteryBox", prices = {Coins = 1000, Gems = 100, Key = 1}},
    {id = "KnifeBox2", name = "Knife Box #2", category = "MysteryBox", prices = {Coins = 1000, Gems = 100, Key = 1}},
    {id = "KnifeBox3", name = "Knife Box #3", category = "MysteryBox", prices = {Coins = 1000, Gems = 100, Key = 1}},
    {id = "KnifeBox4", name = "Knife Box #4", category = "MysteryBox", prices = {Coins = 1000, Gems = 100, Key = 1}},
    {id = "KnifeBox5", name = "Knife Box #5", category = "MysteryBox", prices = {Coins = 1000, Gems = 100, Key = 1}},
    {id = "MLG Box", name = "Rainbow Box", category = "MysteryBox", prices = {Coins = 1000, Gems = 100, Key = 1}},
    {id = "GunBox1", name = "Gun Box #1", category = "MysteryBox", prices = {Coins = 1000, Gems = 100, Key = 1}},
    {id = "GunBox2", name = "Gun Box #2", category = "MysteryBox", prices = {Coins = 1000, Gems = 100, Key = 1}},
    {id = "GunBox3", name = "Gun Box #3", category = "MysteryBox", prices = {Coins = 1000, Gems = 100, Key = 1}},
    {id = "Egg", name = "Common Egg", category = "Eggs", prices = {Coins = 1000, Gems = 100}},
    {id = "Christmas2024Box", name = "Christmas Box '24", category = "MysteryBox", prices = {SnowTokens2024 = 800, SnowKey2024 = 1}},
    {id = "Summer2025Box", name = "Summer Box '25", category = "MysteryBox", prices = {BeachBalls2025 = 800, SummerKey2025 = 1}},
    {id = "Halloween2025Box", name = "Halloween Box '25", category = "MysteryBox", prices = {Candies2025 = 800, HalloweenKey2025 = 1}},
}

-- Coin family to the currencies that family pays for. Currency names come
-- from `Sync.Currencies`; the order prefers the current year's currency.
local FAMILY_CURRENCIES: {[string]: {string}} = {
    Coin = {"Coins"},
    Candy = {"Candies2025", "Candies2024", "Candies2022"},
    SnowToken = {"SnowTokens2024", "SnowTokens2023", "SnowTokens2022", "SnowTokens2020"},
    BeachBall = {"BeachBalls2025"},
    Egg = {"Eggs2025", "RareEggs2025"},
    Heart = {"Hearts2023"},
}

local BAG_TO_FAMILY: {[string]: string} = {
    Coin = "Coin",
    Candy = "Candy",
    SnowToken = "SnowToken",
    BeachBall = "BeachBall",
    Egg = "Egg",
}

local function lower(value: string): string
    return string.lower(value)
end

local function contains(value: string, needle: string): boolean
    return string.find(value, needle, 1, true) ~= nil
end

local function sortedKeys(map: {[string]: any}): {string}
    local keys: {string} = {}
    for key: string in pairs(map) do
        table.insert(keys, key)
    end
    table.sort(keys)
    return keys
end

function Module.init(runtime: any): any
    if Module.Initialized then
        return Module
    end
    local core: any = state.mm2Core
    assert(type(core) == "table", "MM2 Auto Unbox requires the MM2 core module")
    Module.Runtime = runtime

    local settings = {
        boxLabel = DEFAULT_BOX_LABEL,
        currency = AUTO_CURRENCY,
        delay = 1.5,
        maxBoxes = 10,
        reveal = REVEAL_ON,
    }
    local session = {
        opened = 0,
        failures = 0,
        consecutiveFailures = 0,
        nextAt = 0,
        finished = false,
    }
    -- The coin family the server announces; drives the Auto currency.
    local serverFamily: string? = nil

    ----------------------------------------------------------------------
    -- Catalogue: every box the game knows, with its display name and
    -- the currencies its price table accepts.
    ----------------------------------------------------------------------

    local function safeRequire(instance: Instance?): any?
        if not instance then
            return nil
        end
        local ok: boolean, result: any = pcall(function(): any
            return (state :: any).requireModule(instance)
        end)
        if ok then
            return result
        end
        return nil
    end

    local function priceOf(boxId: string, sync: any): {[string]: number}?
        if type(sync) == "table" and type(sync.NewShop) == "table" then
            local newShop: any = sync.NewShop
            local holders: {any} = {}
            if type(newShop[boxId]) == "table" then
                table.insert(holders, newShop[boxId])
            end
            if type(newShop.Weapons) == "table" and type(newShop.Weapons[boxId]) == "table" then
                table.insert(holders, newShop.Weapons[boxId])
            end
            if type(newShop.MysteryBox) == "table" and type(newShop.MysteryBox[boxId]) == "table" then
                table.insert(holders, newShop.MysteryBox[boxId])
            end
            for _, holder: any in ipairs(holders) do
                if type(holder.Price) == "table" then
                    return holder.Price
                end
            end
        end
        for _, entry: BoxEntry in ipairs(FALLBACK_BOXES) do
            if entry.id == boxId then
                return entry.prices
            end
        end
        return nil
    end

    local function eventBoxModules(): {Instance}
        local found: {Instance} = {}
        local replicated: Instance = game:GetService("ReplicatedStorage")
        local shared: Instance? = replicated:FindFirstChild("SharedServices")
        local service: Instance? = shared and shared:FindFirstChild("EventInfoService1")
        if not service then
            return found
        end
        local direct: Instance? = service:FindFirstChild("MysteryBoxes")
        if direct then
            for _, child: Instance in ipairs(direct:GetChildren()) do
                if child:IsA("ModuleScript") then
                    table.insert(found, child)
                end
            end
        end
        local eventList: Instance? = service:FindFirstChild("EventList")
        if eventList then
            for _, event: Instance in ipairs(eventList:GetChildren()) do
                local boxModule: Instance? = event:FindFirstChild("MysteryBox")
                if boxModule and boxModule:IsA("ModuleScript") then
                    table.insert(found, boxModule)
                end
            end
        end
        return found
    end

    local function buildCatalog(): ({BoxEntry}, {string})
        local catalog: {BoxEntry} = {}
        local seen: {[string]: boolean} = {}
        local function add(entry: BoxEntry): ()
            if seen[entry.id] then
                return
            end
            seen[entry.id] = true
            table.insert(catalog, entry)
        end

        local replicated: Instance = game:GetService("ReplicatedStorage")
        local database: Instance? = replicated:FindFirstChild("Database")
        local sync: any = safeRequire(database and database:FindFirstChild("Sync"))
        if type(sync) == "table" then
            for _, bankName: string in ipairs({"MysteryBox", "Eggs"}) do
                local bank: any = (sync :: any)[bankName]
                if type(bank) == "table" then
                    for id: string, data: any in pairs(bank) do
                        add({
                            id = id,
                            name = (type(data) == "table" and type(data.Name) == "string" and data.Name) or id,
                            category = bankName == "Eggs" and "Eggs" or "MysteryBox",
                            prices = priceOf(id, sync) or {},
                        })
                    end
                end
            end
        end

        -- Event boxes: the module instance name is the box id, its
        -- `ItemData.Name` (or `Name`) is what the shop calls it.
        for _, boxModule: Instance in ipairs(eventBoxModules()) do
            local data: any = safeRequire(boxModule)
            local display: string? = nil
            if type(data) == "table" then
                if type(data.ItemData) == "table" and type(data.ItemData.Name) == "string" then
                    display = data.ItemData.Name
                elseif type(data.Name) == "string" and data.Name ~= boxModule.Name then
                    display = data.Name
                end
            end
            add({
                id = boxModule.Name,
                name = display or boxModule.Name,
                category = "MysteryBox",
                prices = priceOf(boxModule.Name, sync) or {},
            })
        end

        -- Anything the dump knows and the live game has not registered yet is
        -- still worth offering: the remote answers with a nil reward when a
        -- box is not on sale, and that is reported honestly.
        for _, entry: BoxEntry in ipairs(FALLBACK_BOXES) do
            add(entry)
        end

        table.sort(catalog, function(a: BoxEntry, b: BoxEntry): boolean
            return lower(a.name) < lower(b.name)
        end)

        -- Unique labels for the selector.
        local labelCounts: {[string]: number} = {}
        for _, entry: BoxEntry in ipairs(catalog) do
            labelCounts[entry.name] = (labelCounts[entry.name] or 0) + 1
        end
        local labels: {string} = {}
        for _, entry: BoxEntry in ipairs(catalog) do
            if labelCounts[entry.name] > 1 then
                table.insert(labels, entry.name .. " (" .. entry.id .. ")")
            else
                table.insert(labels, entry.name)
            end
        end
        return catalog, labels
    end

    local catalog: {BoxEntry}, boxLabels: {string} = buildCatalog()
    local labelToEntry: {[string]: BoxEntry} = {}
    for index: number, entry: BoxEntry in ipairs(catalog) do
        labelToEntry[boxLabels[index]] = entry
    end
    if not labelToEntry[settings.boxLabel] then
        settings.boxLabel = boxLabels[1] or ""
    end

    local function currentEntry(): BoxEntry?
        return labelToEntry[settings.boxLabel] or catalog[1]
    end

    -- Every currency the catalogue references, so the selector can name the
    -- coin the player is farming right now.
    local currencyChoices: {string} = {AUTO_CURRENCY}
    do
        local seen: {[string]: boolean} = {[AUTO_CURRENCY] = true}
        for _, entry: BoxEntry in ipairs(catalog) do
            for currency: string in pairs(entry.prices) do
                if not seen[currency] then
                    seen[currency] = true
                    table.insert(currencyChoices, currency)
                end
            end
        end
    end

    ----------------------------------------------------------------------
    -- Which coin the server is paying out.
    ----------------------------------------------------------------------

    local function familyFromWorkspace(): string?
        local eventName: string = lower(tostring(workspace:GetAttribute("CurrentEvent") or ""))
        if contains(eventName, "halloween") then
            return "Candy"
        elseif contains(eventName, "christmas") then
            return "SnowToken"
        elseif contains(eventName, "summer") then
            return "BeachBall"
        elseif contains(eventName, "easter") then
            return "Egg"
        elseif contains(eventName, "valentine") then
            return "Heart"
        end
        for attribute: string, family: string in pairs({
            isValentines = "Heart",
            isChristmas = "SnowToken",
            isChristmasEnding = "SnowToken",
            IsSummer = "BeachBall",
            IsEaster = "Egg",
        }) do
            if workspace:GetAttribute(attribute) == true then
                return family
            end
        end
        return nil
    end

    local function activeFamily(): string?
        return serverFamily or familyFromWorkspace()
    end

    local function readSyncDatabase(): any
        local replicated: Instance = game:GetService("ReplicatedStorage")
        local database: Instance? = replicated:FindFirstChild("Database")
        return safeRequire(database and database:FindFirstChild("Sync"))
    end

    local function readProfileData(): any
        local replicated: Instance = game:GetService("ReplicatedStorage")
        local modules: Instance? = replicated:FindFirstChild("Modules")
        return safeRequire(modules and modules:FindFirstChild("ProfileData"))
    end

    -- Read the player's real balance for `currency` from MM2's ProfileData
    -- module when available in the live game. Returns nil only when
    -- ProfileData is not present (e.g. offline test harness).
    local function getPlayerCurrencyBalance(currency: string): number?
        if currency == "Robux" then
            return nil
        end
        local profile: any = readProfileData()
        if type(profile) ~= "table" then
            return nil
        end
        local direct: any = profile[currency]
        if type(direct) == "number" then
            return direct
        end
        local materials: any = profile.Materials
        if type(materials) == "table" and type(materials.Owned) == "table" then
            local matOwned: any = materials.Owned[currency]
            if type(matOwned) == "number" then
                return matOwned
            end
        end
        local weapons: any = profile.Weapons
        if type(weapons) == "table" and type(weapons.Owned) == "table" then
            local wepOwned: any = weapons.Owned[currency]
            if type(wepOwned) == "number" then
                return wepOwned
            end
        end
        if type(profile.Coins) == "number"
            or type(profile.Gems) == "number"
            or type(materials) == "table"
            or type(weapons) == "table" then
            return 0
        end
        return nil
    end

    local function canAffordCurrency(entry: BoxEntry, currency: string): boolean
        local price: number? = entry.prices and entry.prices[currency] or nil
        if type(price) ~= "number" or price <= 0 then
            return true
        end
        local balance: number? = getPlayerCurrencyBalance(currency)
        if balance == nil then
            return true
        end
        return balance >= price
    end

    -- Auto: pay with the currency the active coin family fills, then Coins
    -- and Gems, then the box's event currency, and only then a key. When
    -- ProfileData is loaded, prefer a currency the player can actually afford.
    local function autoCurrency(entry: BoxEntry): string?
        local prices: {[string]: number} = entry.prices or {}
        local orderedCandidates: {string} = {}
        local seenCandidate: {[string]: boolean} = {}
        local function pushCandidate(c: string): ()
            if prices[c] and not seenCandidate[c] then
                seenCandidate[c] = true
                table.insert(orderedCandidates, c)
            end
        end
        local family: string? = activeFamily()
        if family then
            for _, currency: string in ipairs(FAMILY_CURRENCIES[family] or {}) do
                pushCandidate(currency)
            end
        end
        for _, currency: string in ipairs({"Coins", "Gems"}) do
            pushCandidate(currency)
        end
        for _, currency: string in ipairs(sortedKeys(prices)) do
            if not contains(lower(currency), "key") then
                pushCandidate(currency)
            end
        end
        for _, currency: string in ipairs(sortedKeys(prices)) do
            pushCandidate(currency)
        end
        for _, currency: string in ipairs(orderedCandidates) do
            if canAffordCurrency(entry, currency) then
                return currency
            end
        end
        return orderedCandidates[1]
    end

    local function resolveCurrency(entry: BoxEntry): (string?, boolean)
        if settings.currency ~= AUTO_CURRENCY then
            if entry.prices and entry.prices[settings.currency] then
                return settings.currency, false
            end
            -- The chosen currency does not open this box; fall back rather
            -- than fire a request the server has to reject.
            return autoCurrency(entry), true
        end
        return autoCurrency(entry), false
    end

    ----------------------------------------------------------------------
    -- Opening: the shop's own remote, then the shop's own reveal.
    ----------------------------------------------------------------------

    local function shopRemotes(): Instance?
        local replicated: Instance = game:GetService("ReplicatedStorage")
        local remotes: Instance? = replicated:FindFirstChild("Remotes")
        return remotes and remotes:FindFirstChild("Shop") or nil
    end

    local function report(text: string): ()
        local card: any = state.mm2AutoUnboxFeature
        if card then
            card:SetStatus(text)
        end
    end

    local function summary(): string
        local entry: BoxEntry? = currentEntry()
        local currency: string? = entry and resolveCurrency(entry) or nil
        local text: string = (entry and entry.name or "no box")
            .. " · pay " .. (currency or "nothing")
        local family: string? = activeFamily()
        if family then
            text ..= " · server coin: " .. family
        end
        if session.opened > 0 or session.failures > 0 then
            text ..= " · " .. tostring(session.opened) .. " opened"
            if session.failures > 0 then
                text ..= " · " .. tostring(session.failures) .. " failed"
            end
        end
        if session.finished then
            text ..= " · done"
        end
        return text
    end

    -- A universal card has no :Toggle (that belongs to framework modules); it
    -- switches off through its own onToggle plus the shared repaint.
    local function turnCardOff(): ()
        local card: any = state.mm2AutoUnboxFeature
```

```
        if not card then
            return
        end
        card.enabled = false
        pcall(function(): ()
            card.onToggle(false)
        end)
        if type(refreshFeatureToggle) == "function" then
            pcall(refreshFeatureToggle, card)
        end
    end

    local function stop(reason: string?): ()
        disconnectFeatureConnection("MM2AutoUnbox")
        if reason then
            notify(reason)
        end
        turnCardOff()
    end

    -- Opening state that deliberately survives toggles. The tick runs every
    -- frame and InvokeServer yields, so without a hard busy flag the next
    -- tick (or a spam of off/on) fires a second OpenCrate while the first
    -- reveal is still on screen; stacked BoxController reveals are the grey
    -- screen that never clears. One opening at a time, and a reveal owns
    -- the screen until its window ends -- toggling cannot bypass either.
    local openingBusy: boolean = false
    local revealUntil: number = 0
    local REVEAL_SECONDS: number = 5

    -- Clean up any stuck MysteryBoxOpen / Unboxing2 dark overlay ScreenGui
    -- left behind in PlayerGui if MM2's MysteryBoxService errored mid-tween.
    local function clearStuckUnboxOverlays(): ()
        pcall(function(): ()
            local playerGui: Instance? = LocalPlayer
                and LocalPlayer:FindFirstChildOfClass("PlayerGui")
            if not playerGui then
                return
            end
            for _, gui: Instance in ipairs(playerGui:GetChildren()) do
                if gui.Name == "MysteryBoxOpen" or gui.Name == "Unboxing2" then
                    gui:Destroy()
                elseif gui.Name == "MainGUI" then
                    for _, subName: string in ipairs({"Lobby", "Game"}) do
                        local sub: Instance? = gui:FindFirstChild(subName)
                        local proc: Instance? = sub and sub:FindFirstChild("Processing")
                        if proc and proc:IsA("GuiObject") then
                            (proc :: GuiObject).Visible = false
                        end
                    end
                end
            end
        end)
    end

    local function isValidReward(reward: any, category: string): boolean
        if type(reward) ~= "string" or reward == "" or reward == "false" or reward == "nil" then
            return false
        end
        local sync: any = readSyncDatabase()
        if type(sync) == "table" then
            if category == "Eggs" then
                local pets: any = sync.Pets
                if type(pets) == "table" and next(pets) ~= nil then
                    return pets[reward] ~= nil
                end
            else
                local weapons: any = sync.Weapons
                local items: any = sync.Item
                if type(weapons) == "table" and next(weapons) ~= nil then
                    if weapons[reward] ~= nil then
                        return true
                    end
                    if type(items) == "table" and items[reward] ~= nil then
                        return true
                    end
                    return false
                end
            end
        end
        return true
    end

    -- MysteryBoxService:PlayOpeningAnimation indexes Sync.MysteryBox[boxId]
    -- and Sync.Weapons[reward] AFTER parenting the dark MysteryBoxOpen GUI
    -- to PlayerGui. If an event box is not in Sync.MysteryBox (or reward is
    -- not in Sync.Weapons), PlayOpeningAnimation crashes mid-function and
    -- leaves the dark overlay stuck forever. Check before firing BoxController.
    local function canPlayBoxAnimationSafely(boxId: string, reward: string): boolean
        local sync: any = readSyncDatabase()
        if type(sync) ~= "table" then
            return true
        end
        local weapons: any = sync.Weapons
        if type(weapons) ~= "table" or next(weapons) == nil then
            return true
        end
        if type(weapons[reward]) ~= "table" then
            return false
        end
        local mysteryBoxes: any = sync.MysteryBox
        if type(mysteryBoxes) == "table" and next(mysteryBoxes) ~= nil then
            local boxDef: any = mysteryBoxes[boxId]
            if type(boxDef) ~= "table" or type(boxDef.Contents) ~= "table" then
                return false
            end
        end
        return true
    end

    local function reveal(boxId: string, category: string, reward: any): ()
        local remotes: Instance? = shopRemotes()
        if not remotes then
            return
        end
        if settings.reveal == REVEAL_ON then
            -- The official flow: the game plays the reveal and reports the
            -- crate as complete (the BoxController LocalScript does both).
            if category == "Eggs" then
                revealUntil = os.clock() + REVEAL_SECONDS
                local eggController: Instance? = remotes:FindFirstChild("EggController")
                if eggController then
                    pcall(function(): ()
                        (eggController :: any):Fire(boxId, reward)
                    end)
                end
                return
            elseif canPlayBoxAnimationSafely(boxId, tostring(reward)) then
                revealUntil = os.clock() + REVEAL_SECONDS
                local boxController: Instance? = remotes:FindFirstChild("BoxController")
                if boxController then
                    pcall(function(): ()
                        (boxController :: any):Fire({{MysteryBoxId = boxId, RewardedItemId = reward}})
                    end)
                end
                return
            end
        end
        -- No animation (or event box missing from Sync.MysteryBox which would
        -- crash MysteryBoxService and leave a dark screen): still report the
        -- crate so the server and inventory register the unboxed item cleanly.
        local crateComplete: Instance? = remotes:FindFirstChild("CrateComplete")
        if crateComplete then
            pcall(function(): ()
                (crateComplete :: any):FireServer(reward)
            end)
        end
    end

    local function openOnce(): boolean
        if openingBusy then
            return false
        end
        openingBusy = true
        local entry: BoxEntry? = currentEntry()
        if not entry then
            session.finished = true
            openingBusy = false
            stop("Auto Unbox: no box catalogue found.")
            return false
        end
        local currency: string?, forced: boolean = resolveCurrency(entry)
        if not currency then
            session.finished = true
            openingBusy = false
            stop("Auto Unbox: " .. entry.name .. " has no known price currency.")
            return false
        end
        -- Verify the player actually has enough currency BEFORE calling
        -- OpenCrate or playing any reveal animation. If insufficient, do
        -- nothing and stay enabled waiting for currency while farming.
        local requiredPrice: number? = entry.prices and entry.prices[currency] or nil
        local currentBalance: number? = getPlayerCurrencyBalance(currency)
        if type(requiredPrice) == "number"
            and requiredPrice > 0
            and type(currentBalance) == "number"
            and currentBalance < requiredPrice then
            clearStuckUnboxOverlays()
            openingBusy = false
            report(
                summary()
                    .. " - waiting for "
                    .. tostring(currency)
                    .. " ("
                    .. tostring(currentBalance)
                    .. "/"
                    .. tostring(requiredPrice)
                    .. ")"
            )
            return false
        end
        local remotes: Instance? = shopRemotes()
        local openCrate: Instance? = remotes and remotes:FindFirstChild("OpenCrate")
        if not (openCrate and openCrate:IsA("RemoteFunction")) then
            openingBusy = false
            report(summary() .. " - waiting for shop")
            return false
        end
        local note: string = forced
            and (" - " .. settings.currency .. " rejected, using " .. currency)
            or ""
        local ok: boolean, reward: any = pcall(function(): any
            return (openCrate :: any):InvokeServer(entry.id, entry.category, currency)
        end)
        if ok and isValidReward(reward, entry.category) then
            reveal(entry.id, entry.category, reward)
            session.opened += 1
            session.consecutiveFailures = 0
            report(summary() .. note)
            openingBusy = false
            return true
        end
        -- Not enough currency or server rejected the open: do nothing, do not
        -- play any reveal animation, and keep Auto Unbox active so it
        -- automatically opens boxes as soon as farming collects enough coins.
        clearStuckUnboxOverlays()
        session.failures += 1
        session.consecutiveFailures += 1
        report(summary() .. note .. " - waiting for " .. tostring(currency))
        openingBusy = false
        return false
    end

    local function tick(_deltaTime: number): ()
        local now: number = os.clock()
        if now < session.nextAt or now < revealUntil or openingBusy then
            return
        end
        local opened: boolean = openOnce()
        -- Extra settle time while the reveal plays: firing the next
        -- BoxController animation over a running one can leave the screen
        -- black, so a revealed box always gets a longer breather. When no
        -- box was opened (waiting for coins), check again after delay.
        local settle: number = (opened and settings.reveal == REVEAL_ON)
            and REVEAL_SECONDS or 0
        session.nextAt = os.clock() + settings.delay + settle
    end

    local AutoUnboxFeature: any = createUniversalFeature(
        "Auto Unbox",
        "Opens any mystery box or egg of the game through its own OpenCrate remote, paying with the currency you choose or the coin the server is giving.",
        4,
        function(enabled: boolean): ()
            disconnectFeatureConnection("MM2AutoUnbox")
            if os.clock() >= revealUntil then
                clearStuckUnboxOverlays()
            end
            if not enabled then
                return
            end
            session.opened = 0
            session.failures = 0
            session.consecutiveFailures = 0
            -- Never sooner than a reveal still on screen: re-enabling in
            -- the middle of one must not stack a second BoxController.
            session.nextAt = math.max(os.clock(), revealUntil)
            session.finished = false
            report(summary())
            featureConnections.MM2AutoUnbox = TaskManager:Connect(tick, nil, "MM2 Auto Unbox")
        end,
        {
            categoryName = "Utilities",
            parent = MM2Scroll,
            registry = mm2Features,
        }
    )
    state.mm2AutoUnboxFeature = AutoUnboxFeature

    -- The server names the coin family this round pays out; Auto uses it to
    -- pick the currency.
    local startedRemote: Instance? = core.mm2GameplayRemotes
        and core.mm2GameplayRemotes:FindFirstChild("CoinsStarted")
    if startedRemote and startedRemote:IsA("RemoteEvent") then
        featureConnections.MM2AutoUnboxCoinsStarted =
            (startedRemote :: any).OnClientEvent:Connect(function(bagTable: any): ()
                if type(bagTable) == "table" then
                    for bag: any in pairs(bagTable) do
                        local family: string? = BAG_TO_FAMILY[tostring(bag)]
                        if family then
                            serverFamily = family
                        end
                    end
                end
                if state.mm2AutoUnboxFeature then
                    report(summary())
                end
            end)
    end

    local defaultBoxIndex: number = table.find(boxLabels, DEFAULT_BOX_LABEL) or 1
    addCycleOption(
        AutoUnboxFeature,
        "Box to open",
        boxLabels,
        defaultBoxIndex,
        function(value: string): ()
            settings.boxLabel = value
        end,
        "Every box and egg of the game, including the event boxes."
    )
    addCycleOption(
        AutoUnboxFeature,
        "Pay with",
        currencyChoices,
        1,
        function(value: string): ()
            settings.currency = value
        end,
        "Auto pays with the currency the server's current coin fills."
    )
    addCycleOption(
        AutoUnboxFeature,
        "Reveal animation",
        {REVEAL_ON, REVEAL_OFF},
        1,
        function(value: string): ()
            settings.reveal = value
        end,
        "On replays the game's own opening animation and crate report."
    )
    addNumberOption(
        AutoUnboxFeature,
        "Delay between boxes",
        settings.delay,
        0.8,
        5,
        function(value: number): ()
            settings.delay = value
        end
    )
    addNumberOption(
        AutoUnboxFeature,
        "Max boxes per run",
        settings.maxBoxes,
        1,
        50,
        function(value: number): ()
            settings.maxBoxes = value
        end
    )
    addActionOption(
        AutoUnboxFeature,
        "Report boxes and coin",
        function(): ()
            local entry: BoxEntry? = currentEntry()
            local prices: {string} = {}
            if entry then
                for _, currency: string in ipairs(sortedKeys(entry.prices)) do
                    table.insert(prices, currency .. "=" .. tostring(entry.prices[currency]))
                end
            end
            notify(
                "Boxes found: " .. tostring(#catalog)
                    .. " · chosen: " .. (entry and entry.name or "none")
                    .. " (" .. (entry and entry.id or "?") .. ")"
                    .. " · price: " .. (#prices > 0 and table.concat(prices, ", ") or "unknown")
                    .. " · server coin: " .. tostring(activeFamily() or "unknown")
            )
        end
    )
    addInformationOption(
        AutoUnboxFeature,
        "Uses the shop's own OpenCrate remote with the box id and currency the game expects, then the game's own reveal. Robux prompts are never pressed."
    )

    activeCleanup = function(): ()
        state.mm2AutoUnboxFeature = nil
        disconnectFeatureConnection("MM2AutoUnbox")
        disconnectFeatureConnection("MM2AutoUnboxCoinsStarted")
    end
    Module.Events = featureConnections
    Module.Initialized = true
    return Module
end

function Module.destroy(): ()
    if not Module.Initialized then
        return
    end
    Module.Initialized = false
    pcall(activeCleanup)
    activeCleanup = function(): ()
    end
    Module.Events = {}
    Module.Runtime = nil
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/MM2/Other/AutoUnbox.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/games/MM2/Other/AutoVote.lua
BYTES: 30362
SHA-256: 12f18808888799a081474bb1c07ecbf4610a80339d81b5d621899927fea5d1c3
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
local Module = {
    Name = "MM2 Auto Vote",
    Events = {} :: {[string]: any},
    Initialized = false,
    Runtime = nil :: any,
}

local activeCleanup: () -> () = function(): () end

function Module.init(runtime: any): any
    if Module.Initialized then
        return Module
    end
    local core: any = state.mm2Core
    assert(type(core) == "table", "MM2 Auto Vote requires the MM2 core module")
    Module.Runtime = runtime

    local TICK_INTERVAL: number = 0.10
    local MAP_REFRESH_INTERVAL: number = 0.50
    local VOTE_HOLD_BEFORE_RESET: number = 0.22
    local RESET_RETRY_INTERVAL: number = 0.25
    local MAX_LOBBY_PAD_DISTANCE: number = 250

    local DEFAULT_MAP_CHOICES: {string} = {
        "House 2",
        "Hospital 3",
        "Police Station",
    }
    local mapChoices: {string} = {
        DEFAULT_MAP_CHOICES[1],
        DEFAULT_MAP_CHOICES[2],
        DEFAULT_MAP_CHOICES[3],
    }
    local selectedSlot: number = 1
    local selectedMapName: string = mapChoices[1]
    local selectMapButton: TextButton? = nil
    local selectMapOptionKey: string = "MM2.AutoVote.SelectMap"
    local AutoVoteFeature: any = nil

    local cachedLobby: Instance? = nil
    local cachedPadModels: {[number]: Instance?} = {}
    local cachedVotePadsFolder: Instance? = nil
    local cachedVoteIconsFolder: Instance? = nil
    local lastLobbyCheckAt: number = -math.huge
    local lastMapRefreshAt: number = -math.huge
    local savedDetectorCFrames: {[BasePart]: CFrame} = {}
    local roundBlockingVote: boolean = false
    local lastBillboardVotingState: boolean = false

    local function trim(s: string): string
        return (string.gsub(s, "^%s*(..-)%s*$", "%1"))
    end

    local function isPlaceholderMapLabel(title: string?): boolean
        if type(title) ~= "string" then
            return true
        end
        local clean: string = string.upper(trim(title))
        return clean == ""
            or clean == "MAP NAME"
            or clean == "LOADING"
            or clean == "NONE"
            or clean == "MAP 1"
            or clean == "MAP 2"
            or clean == "MAP 3"
    end

    local function isValidMapTitle(title: string?): boolean
        return not isPlaceholderMapLabel(title)
    end

    local function isLobbyContainer(candidate: Instance?): boolean
        if not candidate or not candidate.Parent then
            return false
        end
        return candidate:FindFirstChild("VotePad1") ~= nil
            or candidate:FindFirstChild("VotePads") ~= nil
            or candidate:FindFirstChild("VoteIcons") ~= nil
    end

    local function findLobbyModel(): Instance?
        local now: number = os.clock()
        if cachedLobby and cachedLobby:IsDescendantOf(workspace) and now - lastLobbyCheckAt < 1.0 then
            return cachedLobby
        end
        lastLobbyCheckAt = now

        -- 1. Fast check on direct workspace children
        for _, name: string in ipairs({"RegularLobby", "Lobby", "MainLobby", "HalloweenLobby"}) do
            local candidate: Instance? = workspace:FindFirstChild(name)
            if isLobbyContainer(candidate) then
                cachedLobby = candidate
                return candidate
            end
        end

        local okChildren: boolean, children: any = pcall(function(): any
            return workspace:GetChildren()
        end)
        if okChildren and type(children) == "table" then
            for _, child: Instance in ipairs(children) do
                if (child:IsA("Model") or child:IsA("Folder")) and isLobbyContainer(child) then
                    cachedLobby = child
                    return child
                end
            end
        end

        -- 2. Check CollectionService "VotePad" tag (used on VotePad1..3 in MM2)
        local okTagged: boolean, tagged: any = pcall(function(): any
            return game:GetService("CollectionService"):GetTagged("VotePad")
        end)
        if okTagged and type(tagged) == "table" then
            for _, inst: Instance in ipairs(tagged) do
                if inst and inst:IsDescendantOf(workspace) then
                    local slotNum: number? = tonumber(string.match(inst.Name, "^VotePad(%d)$"))
                    if slotNum then
                        cachedPadModels[slotNum] = inst
                    end
                    if inst.Parent and inst.Parent ~= workspace then
                        cachedLobby = inst.Parent
                    end
                end
            end
        end
        if cachedLobby and cachedLobby:IsDescendantOf(workspace) then
            return cachedLobby
        end

        -- 3. Recursive fallback for nested seasonal/event lobbies
        local okDeep: boolean, deepPad: any = pcall(function(): any
            return workspace:FindFirstChild("VotePads", true)
                or workspace:FindFirstChild("VotePad1", true)
                or workspace:FindFirstChild("VoteIcons", true)
        end)
        if okDeep and deepPad and (deepPad :: Instance):IsDescendantOf(workspace) then
            local parent: Instance? = (deepPad :: Instance).Parent
            if parent and parent.Name == "VoteIcons" and parent.Parent then
                parent = parent.Parent
            end
            cachedLobby = parent
            return cachedLobby
        end

        cachedLobby = nil
        return nil
    end

    local function getPadModel(lobby: Instance?, slot: number): Instance?
        local cached: Instance? = cachedPadModels[slot]
        if cached and cached:IsDescendantOf(workspace) and not (cached.Parent and cached.Parent.Name == "VoteIcons") then
            return cached
        end
        local padName: string = "VotePad" .. tostring(slot)
        if lobby then
            local direct: Instance? = lobby:FindFirstChild(padName)
            if direct and not (direct.Parent and direct.Parent.Name == "VoteIcons") then
                cachedPadModels[slot] = direct
                return direct
            end
            local okDeep: boolean, deep: any = pcall(function(): any
                return lobby:FindFirstChild(padName, true)
            end)
            if okDeep and deep and not ((deep :: Instance).Parent and (deep :: Instance).Parent.Name == "VoteIcons") then
                cachedPadModels[slot] = deep :: Instance
                return deep :: Instance
            end
        end
        return nil
    end

    local function getVoteIconsFolder(lobby: Instance?): Instance?
        if cachedVoteIconsFolder and cachedVoteIconsFolder:IsDescendantOf(workspace) then
            return cachedVoteIconsFolder
        end
        if lobby then
            local direct: Instance? = lobby:FindFirstChild("VoteIcons")
                or lobby:FindFirstChild("VoteIcons", true)
            if direct then
                cachedVoteIconsFolder = direct
                return direct
            end
        end
        return nil
    end

    local function getVotePadsFolder(lobby: Instance?): Instance?
        if cachedVotePadsFolder and cachedVotePadsFolder:IsDescendantOf(workspace) then
            return cachedVotePadsFolder
        end
        if lobby then
            local direct: Instance? = lobby:FindFirstChild("VotePads")
                or lobby:FindFirstChild("VotePads", true)
            if direct then
                cachedVotePadsFolder = direct
                return direct
            end
        end
        return nil
    end

    -- Read the map name displayed on billboard slot i (1..3) from the lobby.
    local function readSlotMapName(lobby: Instance?, slot: number): string?
        if not lobby then
            return nil
        end
        local padModel: Instance? = getPadModel(lobby, slot)
        if padModel then
            local voteInfoGui: Instance? = padModel:FindFirstChild("VoteInfoGui")
            local container: Instance? = voteInfoGui and voteInfoGui:FindFirstChild("Container")
            local mapLabel: Instance? = (container and container:FindFirstChild("MapName"))
                or padModel:FindFirstChild("MapName", true)
            if mapLabel and mapLabel:IsA("TextLabel") then
                local text: string = (mapLabel :: TextLabel).Text
                if isValidMapTitle(text) then
                    return trim(text)
                end
            end
        end

        local voteIcons: Instance? = getVoteIconsFolder(lobby)
        local iconPart: Instance? = voteIcons and voteIcons:FindFirstChild("VotePad" .. tostring(slot))
        if iconPart then
            local mapLabel: Instance? = iconPart:FindFirstChild("MapName", true)
            if mapLabel and mapLabel:IsA("TextLabel") then
                local text: string = (mapLabel :: TextLabel).Text
                if isValidMapTitle(text) then
                    return trim(text)
                end
            end
            local mapObj: Instance? = iconPart:FindFirstChild("Map")
            if mapObj and mapObj:IsA("ObjectValue") then
                local val: Instance? = (mapObj :: ObjectValue).Value
                if val and isValidMapTitle(val.Name) then
                    return trim(val.Name)
                end
            end
        end

        return nil
    end

    -- Read the current vote count on slot i (1..3).
    local function readSlotVoteCount(lobby: Instance?, slot: number): number?
        if not lobby then
            return nil
        end
        local voteIcons: Instance? = getVoteIconsFolder(lobby)
        local iconPart: Instance? = voteIcons and voteIcons:FindFirstChild("VotePad" .. tostring(slot))
        if iconPart then
            local votesVal: Instance? = iconPart:FindFirstChild("Votes")
            if votesVal and votesVal:IsA("IntValue") then
                return (votesVal :: IntValue).Value
            end
        end
        local padModel: Instance? = getPadModel(lobby, slot)
        if padModel then
            local voteInfoGui: Instance? = padModel:FindFirstChild("VoteInfoGui")
            local container: Instance? = voteInfoGui and voteInfoGui:FindFirstChild("Container")
            local votesLabel: Instance? = (container and container:FindFirstChild("Votes"))
                or padModel:FindFirstChild("Votes", true)
            if votesLabel and votesLabel:IsA("TextLabel") then
                local numStr: string? = string.match((votesLabel :: TextLabel).Text, "(%d+)")
                if numStr then
                    return tonumber(numStr)
```

```
            end
        end
    end
    return nil
end

local function syncSelectMapControl(): ()
    local control: any = state.valueControls
        and state.valueControls[selectMapOptionKey]
    if type(control) == "table" and type(control.setIndex) == "function" then
        pcall(control.setIndex, selectedSlot)
    elseif selectMapButton then
        pcall(function(): ()
            selectMapButton.Text = mapChoices[selectedSlot]
        end)
    end
end

-- Refresh the 3 map choices in-place from the lobby billboard so the
-- "Select Map" dropdown always displays the 3 real maps on the board.
local function refreshMapChoices(force: boolean?): ()
    local now: number = os.clock()
    if not force and now - lastMapRefreshAt < MAP_REFRESH_INTERVAL then
        return
    end
    lastMapRefreshAt = now

    local lobby: Instance? = findLobbyModel()
    local changed: boolean = false
    for slot: number = 1, 3 do
        local title: string? = readSlotMapName(lobby, slot)
        local current: string? = mapChoices[slot]
        local nextLabel: string = title
            or (isValidMapTitle(current) and (current :: string) or DEFAULT_MAP_CHOICES[slot])
        if mapChoices[slot] ~= nextLabel then
            mapChoices[slot] = nextLabel
            changed = true
        end
    end

    if isValidMapTitle(selectedMapName) then
        local matchedSlot: number? = nil
        local targetLower: string = string.lower(selectedMapName)
        for slot: number = 1, 3 do
            if string.lower(mapChoices[slot]) == targetLower then
                matchedSlot = slot
                break
            end
        end
        if matchedSlot and selectedSlot ~= matchedSlot then
            selectedSlot = matchedSlot
            changed = true
        end
    end

    selectedSlot = math.clamp(selectedSlot, 1, 3)
    selectedMapName = mapChoices[selectedSlot]

    if changed or force then
        syncSelectMapControl()
    end
end

-- Resolve the wooden pad detector parts for slot (1..3):
-- VotePads.Detector{slot} and VotePad{slot}.Pad
local function getSlotDetectorParts(lobby: Instance?, slot: number): {BasePart}
    local parts: {BasePart} = {}
    if not lobby then
        return parts
    end

    local padModel: Instance? = getPadModel(lobby, slot)
    if padModel then
        local padPart: Instance? = padModel:FindFirstChild("Pad")
            or padModel:FindFirstChild("Pad", true)
        if padPart and padPart:IsA("BasePart") then
            parts[#parts + 1] = padPart :: BasePart
        end
    end

    local votePads: Instance? = getVotePadsFolder(lobby)
    if votePads then
        local detector: Instance? = votePads:FindFirstChild("Detector" .. tostring(slot))
            or votePads:FindFirstChild("Detector" .. tostring(slot), true)
        if detector and detector:IsA("BasePart") then
            parts[#parts + 1] = detector :: BasePart
        end
    end

    return parts
end

local function rememberDetectorPart(part: BasePart): ()
    if not savedDetectorCFrames[part] then
        savedDetectorCFrames[part] = part.CFrame
    end
end

local function restoreAllDetectorParts(): ()
    for part, cf in pairs(savedDetectorCFrames) do
        if part and part.Parent then
            pcall(function(): ()
                part.CFrame = cf
            end)
        end
    end
    table.clear(savedDetectorCFrames)
end

-- Check whether the lobby voting billboards are currently enabled.
local function areVotingBillboardsActive(lobby: Instance?): boolean
    if not lobby then
        return false
    end

    for slot: number = 1, 3 do
        local padModel: Instance? = getPadModel(lobby, slot)
        if padModel then
            local voteGui: Instance? = padModel:FindFirstChild("VoteInfoGui")
            if voteGui and voteGui:IsA("SurfaceGui") and (voteGui :: SurfaceGui).Enabled == true then
                return true
            end
            local mapGui: Instance? = padModel:FindFirstChild("MapInfoGui")
            if mapGui and mapGui:IsA("SurfaceGui") and (mapGui :: SurfaceGui).Enabled == true then
                return true
            end
        end

        local voteIcons: Instance? = getVoteIconsFolder(lobby)
        local iconPart: Instance? = voteIcons and voteIcons:FindFirstChild("VotePad" .. tostring(slot))
        if iconPart then
            local votingFlag: Instance? = iconPart:FindFirstChild("Voting")
            if votingFlag and votingFlag:IsA("BoolValue") and (votingFlag :: BoolValue).Value == true then
                return true
            end
            local surface: Instance? = iconPart:FindFirstChild("Surface")
            if surface and surface:IsA("SurfaceGui") and (surface :: SurfaceGui).Enabled == true then
                return true
            end
            local mapInfo: Instance? = iconPart:FindFirstChild("MapInfo")
            local infoSurface: Instance? = mapInfo and mapInfo:FindFirstChild("Surface")
            if infoSurface and infoSurface:IsA("SurfaceGui") and (infoSurface :: SurfaceGui).Enabled == true then
                return true
            end
        end
    end

    return false
end

-- Returns true ONLY while the lobby voting intermission is open and the
-- player is in the lobby. Returns false the instant voting ends or a round starts.
local function isVotingIntermissionActive(root: BasePart?, detector: BasePart?): boolean
    local lobby: Instance? = findLobbyModel()
    if not lobby then
        return false
    end

    local billboardsOn: boolean = areVotingBillboardsActive(lobby)

    -- If the voting billboards just turned on for a new intermission, clear
    -- any previous round lock. If they were on and just turned off, lock until
    -- the next intermission.
    if billboardsOn and not lastBillboardVotingState then
        roundBlockingVote = false
    elseif not billboardsOn and lastBillboardVotingState then
        roundBlockingVote = true
    end
    lastBillboardVotingState = billboardsOn

    if roundBlockingVote then
        return false
    end

    local timerPart: Instance? = workspace:FindFirstChild("RoundTimerPart")
    local timerIndicatesLobby: boolean = false
    if timerPart then
        local okTime: boolean, roundTime: any = pcall(function(): any
            return timerPart:GetAttribute("Time")
        end)
        if okTime and type(roundTime) == "number" then
            if roundTime > 0 then
                return false
            end
            timerIndicatesLobby = true
        end
    end

    local isNearPad: boolean = false
    if root and detector then
        local okDist: boolean, dist: any = pcall(function(): any
            return (root.Position - detector.Position).Magnitude
        end)
        if okDist and type(dist) == "number" then
            if dist > MAX_LOBBY_PAD_DISTANCE then
                return false
            end
            isNearPad = true
        end
    end

    if not isNearPad and type(core.isRoundLifecycleActive) == "function"
        and core.isRoundLifecycleActive() == true then
        return false
    end

    if billboardsOn then
        return true
    end

    -- When enabled mid-intermission on servers where SurfaceGui.Enabled is
    -- not replicated, verify the player is in the lobby near the pads (or
    -- RoundTimerPart Time <= 0) and the voting board has map names loaded.
    if isNearPad or timerIndicatesLobby then
        for slot: number = 1, 3 do
            if readSlotMapName(lobby, slot) ~= nil then
                return true
            end
        end
    end

    return false
end

local function resolveTargetSlot(lobby: Instance?): number
    refreshMapChoices(false)
    if lobby and isValidMapTitle(selectedMapName) then
        local targetLower: string = string.lower(selectedMapName)
        for slot: number = 1, 3 do
            local nameOnBoard: string? = readSlotMapName(lobby, slot)
            if nameOnBoard and string.lower(nameOnBoard) == targetLower then
                selectedSlot = slot
                return slot
            end
        end
    end
    return math.clamp(selectedSlot, 1, 3)
end

-- Step onto the wooden vote pad and pulse touch on its detectors so the
-- server registers the vote on VotePads.Detector{slot} and VotePad{slot}.Pad.
local function pulseVotePad(root: BasePart, detectors: {BasePart}): ()
    local primaryPad: BasePart? = detectors[1]
    if primaryPad then
        local saved: CFrame? = savedDetectorCFrames[primaryPad]
        local padPos: Vector3 = if saved then saved.Position else primaryPad.Position
        pcall(function(): ()
            local currentCF: CFrame = root.CFrame
            local rotOnly: CFrame = currentCF - currentCF.Position
            root.CFrame = CFrame.new(padPos) * rotOnly
            root.AssemblyLinearVelocity = Vector3.zero
        end)
    end
    local fireTouch: any = state.fireTouchInterest
    local rootCF: CFrame = root.CFrame
    for _, part: BasePart in ipairs(detectors) do
        rememberDetectorPart(part)
        pcall(function(): ()
            part.CanCollide = false
            part.CFrame = rootCF
        end)
        if type(fireTouch) == "function" then
            pcall(fireTouch, root, part, 0)
            pcall(fireTouch, root, part, 1)
        end
    end
end

local function resetLocalCharacter(character: Model, humanoid: Humanoid): ()
    pcall(function(): ()
        humanoid:ChangeState(Enum.HumanoidStateType.Dead)
    end)
    pcall(function(): ()
        humanoid.Health = 0
    end)
    pcall(function(): ()
        if humanoid.Health > 0 then
            character:BreakJoints()
        end
    end)
end

local lastStatus: string? = nil
local function setStatus(text: string?): ()
    if text == lastStatus then
        return
    end
    lastStatus = text
    if AutoVoteFeature then
        AutoVoteFeature:SetStatus(text)
    end
end

local function connectGameplayRemotes(): ()
    local okRs: boolean, rs: any = pcall(function(): any
        return game:GetService("ReplicatedStorage")
    end)
    if not okRs or not rs then
        return
    end
    local remotes: Instance? = rs:FindFirstChild("Remotes")
    local gameplay: Instance? = remotes and remotes:FindFirstChild("Gameplay")
    if not gameplay then
        return
    end

    for _, stopName: string in ipairs({
        "LoadingMap",
        "Fade",
        "RoleSelect",
        "ShowRoleSelect",
        "ShowRoleSelectNew",
        "RoundStart",
        "CoinsStarted",
    }) do
        local rem: Instance? = gameplay:FindFirstChild(stopName)
        if rem and rem:IsA("RemoteEvent") then
            local key: string = "MM2AutoVoteStop_" .. stopName
            disconnectFeatureConnection(key)
            featureConnections[key] = (rem :: RemoteEvent).OnClientEvent:Connect(function(): ()
                roundBlockingVote = true
                restoreAllDetectorParts()
            end)
        end
    end

    for _, endName: string in ipairs({
        "RoundEndFade",
        "VictoryScreen",
        "GameOver",
    }) do
        local rem: Instance? = gameplay:FindFirstChild(endName)
        if rem and rem:IsA("RemoteEvent") then
            local key: string = "MM2AutoVoteEnd_" .. endName
            disconnectFeatureConnection(key)
            featureConnections[key] = (rem :: RemoteEvent).OnClientEvent:Connect(function(): ()
                roundBlockingVote = false
                refreshMapChoices(true)
            end)
        end
    end
end

local function disconnectAllAutoVoteConnections(): ()
    disconnectFeatureConnection("MM2AutoVote")
    for key in pairs(featureConnections) do
        if string.sub(key, 1, 11) == "MM2AutoVote" and key ~= "MM2AutoVoteMapSync" then
            disconnectFeatureConnection(key)
        end
    end
end

local function toggleAutoVote(enabled: boolean): ()
    disconnectAllAutoVoteConnections()
    restoreAllDetectorParts()

    if not enabled then
        setStatus(nil)
        return
    end

    roundBlockingVote = false
    lastBillboardVotingState = false
    refreshMapChoices(true)
    connectGameplayRemotes()

    local lastTickAt: number = -math.huge
    local currentChar: Model? = nil
    local firstTouchAt: number = 0
    local lastResetAttemptAt: number = 0
    local voteCountBeforeTouch: number? = nil

    local function autoVoteTick(): ()
        local now: number = os.clock()
        if now - lastTickAt < TICK_INTERVAL then
            return
        end
        lastTickAt = now

        refreshMapChoices(false)

        local lobby: Instance? = findLobbyModel()
        local slot: number = resolveTargetSlot(lobby)
        local targetMapLabel: string = mapChoices[slot] or DEFAULT_MAP_CHOICES[slot]
        local detectors: {BasePart} = getSlotDetectorParts(lobby, slot)
        local primaryDetector: BasePart? = detectors[1]

        local character: Model? = LocalPlayer and LocalPlayer.Character
        local root: BasePart? = character
            and (character:FindFirstChild("HumanoidRootPart") :: BasePart?)
        local humanoid: Humanoid? = character
            and (character:FindFirstChildOfClass("Humanoid") :: Humanoid?)

        -- Stop immediately if the voting intermission is not active or has ended!
        if not isVotingIntermissionActive(root, primaryDetector) then
            restoreAllDetectorParts()
            firstTouchAt = 0
            lastResetAttemptAt = 0
            setStatus("Waiting for vote · " .. tostring(targetMapLabel))
            return
        end

        if #detectors == 0 then
            setStatus("No vote pad")
            return
        end

        if not character or not root or not humanoid or humanoid.Health <= 0 then
            restoreAllDetectorParts()
            return
        end

        if character ~= currentChar then
            currentChar = character
            firstTouchAt = 0
            lastResetAttemptAt = 0
            voteCountBeforeTouch = readSlotVoteCount(lobby, slot)
        end

        -- Step onto the wooden vote pad and pulse touch on its detectors
        pulseVotePad(root, detectors)
        if firstTouchAt == 0 then
            firstTouchAt = now
        end

        local currentVotes: number? = readSlotVoteCount(lobby, slot)
        local voteIncremented: boolean = (
            currentVotes ~= nil
            and voteCountBeforeTouch ~= nil
            and currentVotes > voteCountBeforeTouch
        )

        setStatus(
            "Voting " .. targetMapLabel
            .. (currentVotes ~= nil and (" (" .. tostring(currentVotes) .. ")") or "")
        )

        -- Once this character life has cast its vote on the wooden pad,
        -- re-verify voting intermission is STILL active and reset the
        -- character to stack another vote.
        if (voteIncremented or (now - firstTouchAt >= VOTE_HOLD_BEFORE_RESET))
            and (now - lastResetAttemptAt >= RESET_RETRY_INTERVAL) then
            restoreAllDetectorParts()
            if isVotingIntermissionActive(root, primaryDetector) then
                lastResetAttemptAt = now
                resetLocalCharacter(character, humanoid)
            end
        end
    end

    featureConnections.MM2AutoVote = TaskManager:Connect(autoVoteTick, nil, "MM2 Auto Vote")
    autoVoteTick()
end

-- Clear any legacy "Map 1..3" placeholder saved in config so real map names show.
pcall(function(): ()
    if type(configData) == "table" and type(configData.values) == "table" then
        local saved: any = configData.values[selectMapOptionKey]
        if isPlaceholderMapLabel(saved) then
            configData.values[selectMapOptionKey] = nil
        end
    end
end)

-- Populate initial map names if already in the lobby
pcall(refreshMapChoices, true)

AutoVoteFeature = createUniversalFeature(
    "Auto Vote",
    "Detects the 3 maps on the lobby voting billboard, votes on the wooden floor pad for your selected map, and resets repeatedly to stack votes until voting intermission ends.",
    7,
    toggleAutoVote,
    {
        categoryName = "Utilities",
        parent = MM2Scroll,
        registry = mm2Features,
    }
)
state.mm2AutoVoteFeature = AutoVoteFeature

selectMapButton = addCycleOption(
    AutoVoteFeature,
    "Select Map",
    mapChoices,
    1,
    function(value: string): ()
        refreshMapChoices(false)
        selectedMapName = tostring(value)
        for slot: number = 1, 3 do
            if mapChoices[slot] == selectedMapName then
                selectedSlot = slot
                break
            end
        end
        if AutoVoteFeature and AutoVoteFeature.enabled then
            setStatus("Voting " .. tostring(mapChoices[selectedSlot] or selectedMapName))
        end
    end,
    "Select which of the 3 lobby billboard maps to vote for."
)

-- Refresh the 3 billboard map names right BEFORE the dropdown opens
-- and keep the label synced with the lobby boards even while Auto Vote is OFF.
pcall(function(): ()
    local control: any = state.valueControls
        and state.valueControls[selectMapOptionKey]
    if type(control) == "table" then
        control.onBeforeOpen = function(): ()
            refreshMapChoices(true)
        end
    end
end)

local lastBgSyncAt: number = -math.huge
disconnectFeatureConnection("MM2AutoVoteMapSync")
featureConnections.MM2AutoVoteMapSync = TaskManager:Connect(function(): ()
    local now: number = os.clock()
    if now - lastBgSyncAt < 1.0 then
        return
    end
    lastBgSyncAt = now
    refreshMapChoices(false)
end, nil, "Auto Vote sync")

activeCleanup = function(): ()
    disconnectFeatureConnection("MM2AutoVoteMapSync")
    toggleAutoVote(false)
end
Module.Events = featureConnections
Module.Initialized = true
return Module
end

function Module.destroy(): ()
    if not Module.Initialized then
        return
    end
    Module.Initialized = false
    pcall(activeCleanup)
    activeCleanup = function(): () end
    Module.Events = {}
    Module.Runtime = nil
end

return Module

----------------------------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/MM2/Other/AutoVote.lua
----------------------------------------------------------------------------------------------------

----------------------------------------------------------------------------------------------------
ARCHIVO: src/games/MM2/Other/AutoWin.lua
BYTES: 43386
SHA-256: 993a4267ce795d71b359bd244942c42b5dcd1e859189e2de4d9a197b24cc7af2
INICIO DEL ARCHIVO
----------------------------------------------------------------------------------------------------
local Module = {
    Name = "MM2 Auto Win",
    Events = {} :: {[string]: any},
    Initialized = false,
    Runtime = nil :: any,
}

local activeCleanup: () -> () = function(): () end

function Module.init(runtime: any): any
    if Module.Initialized then
        return Module
    end
    local core: any = state.mm2Core
    assert(type(core) == "table", "MM2 Auto Win requires the MM2 core module")
    local prediction: any = state.mm2Prediction
    Module.Runtime = runtime

    local findDroppedGun: any = core.findDroppedGun
    local findMM2Map: any = core.findMM2Map
    local findMurderer: any = core.findMurderer
    local findSheriff: any = core.findSheriff
    local getGunOriginCFrame: any = core.getGunOriginCFrame
    local getPlayerRole: any = core.getPlayerRole
    local getPlayerWeapon: any = core.getPlayerWeapon
    local isPlayerAlive: any = core.isPlayerAlive
    local isProtectedTarget: any = core.isProtectedTarget
    local isRoundLifecycleActive: any = core.isRoundLifecycleActive

    local ABOVE_MAP_OFFSET: number = 120
    local MAP_FLING_WARMUP_SECONDS: number = 5.0
    local SHERIFF_READY_DELAY_SECONDS: number = 2.0
    local SHERIFF_FLING_COOLDOWN_SECONDS: number = 1.2

    local settings = {
        instaKillMurder = true,
        instaKillSheriff = true,
        flingSheriffGetGun = false,
        flingMurderIfDead = false,
    }

    local autoWinActive: boolean = false
    local AutoWinFeature: any = nil
    local lastStatus: string? = nil

    local sheriffWins: number = 0
    local murderWins: number = 0
    local innocentWins: number = 0
    local sheriffWinsStat: any = nil
    local murderWinsStat: any = nil
    local innocentWinsStat: any = nil

    local wasOnMap: boolean = false
    local mapEnteredAt: number = 0
    local sheriffFirstSeenAt: number = 0
    local lastSheriffFlingAt: number = 0
    local participatedInRound: boolean = false
    local roundMyRole: string? = nil
    local roundSawMurderer: boolean = false
    local roundCountedWin: boolean = false
    local flungMurderThisDeath: boolean = false
    local flingBusy: boolean = false

    local safeAbovePosition: Vector3? = nil
    local savedCollide: {[Instance]: boolean} = {}
    local savedCustomProps: {[BasePart]: PhysicalProperties?} = {}
    local victimSnapshots: {[BasePart]: {cframe: CFrame, anchored: boolean, canCollide: boolean}} = {}
    local toolConnections: {RBXScriptConnection} = {}
    local flingAngle: number = 0

    local function updateWinOverlay(): ()
        if sheriffWinsStat and type(sheriffWinsStat.set) == "function" then
            sheriffWinsStat.set(tostring(sheriffWins))
        end
        if murderWinsStat and type(murderWinsStat.set) == "function" then
            murderWinsStat.set(tostring(murderWins))
        end
        if innocentWinsStat and type(innocentWinsStat.set) == "function" then
            innocentWinsStat.set(tostring(innocentWins))
        end
    end

    local function awardRoundWin(roleCategory: string?): ()
        if roundCountedWin then
            return
        end
        local cat: string = roleCategory or roundMyRole or "Innocent"
        roundCountedWin = true
        if cat == "Murderer" then
            murderWins += 1
        elseif cat == "Sheriff" or cat == "Hero" then
            sheriffWins += 1
        else
            innocentWins += 1
        end
        updateWinOverlay()
    end

    local function setStatus(text: string?): ()
        if text == lastStatus then
            return
        end
        lastStatus = text
        if AutoWinFeature then
            AutoWinFeature:SetStatus(text)
        end
    end
```

```
    local function yawOf(frame: CFrame): number
        local look: Vector3 = frame.LookVector
        return math.atan2(-look.X, -look.Z)
    end

    local function characterParts(): (Model?, Humanoid?, BasePart?)
        local character: Model? = LocalPlayer and LocalPlayer.Character
        if not character then
            return nil, nil, nil
        end
        local humanoid: Humanoid? = character:FindFirstChildOfClass("Humanoid")
        local root: BasePart? = character:FindFirstChild("HumanoidRootPart") :: BasePart?
        if not humanoid or not root then
            return nil, nil, nil
        end
        return character, humanoid, root
    end

    local function findLobbyReferencePart(): BasePart?
        for _, child: Instance in ipairs(workspace:GetChildren()) do
            local lowerName: string = string.lower(tostring(child.Name))
            local isLobby: boolean = string.find(lowerName, "lobby", 1, true) ~= nil
                or child:FindFirstChild("VotePads", true) ~= nil
                or child:FindFirstChild("VotePad1", true) ~= nil
                or child:FindFirstChild("VoteIcons", true) ~= nil
            if isLobby then
                local votePads: Instance? = child:FindFirstChild("VotePads", true)
                if votePads then
                    local padPart: BasePart? = votePads:FindFirstChildWhichIsA("BasePart", true)
                    if padPart then
                        return padPart
                    end
                end
                local spawns: Instance? = child:FindFirstChild("Spawns", true)
                if spawns then
                    local spawnPart: BasePart? = spawns:FindFirstChildWhichIsA("BasePart", true)
                    if spawnPart then
                        return spawnPart
                    end
                end
                local anyPart: BasePart? = child:FindFirstChildWhichIsA("BasePart", true)
                if anyPart then
                    return anyPart
                end
            end
        end
        return nil
    end

    -- True as soon as the player's character is on the MM2 map, even during
    -- the pre-round countdown before roles or weapons have been handed out.
    local function isCharacterOnMap(root: BasePart): (boolean, Vector3?)
        if type(findMM2Map) ~= "function" then
            return false, nil
        end
        local map: Instance? = findMM2Map()
        if not map then
            return false, nil
        end
        local rootPos: Vector3 = root.Position
        local lobbyPart: BasePart? = findLobbyReferencePart()
        if lobbyPart and (rootPos - lobbyPart.Position).Magnitude <= 350 then
            return false, nil
        end
        local refPart: BasePart? = nil
        local spawns: Instance? = (map :: Instance):FindFirstChild("Spawns")
        if spawns then
            refPart = spawns:FindFirstChildWhichIsA("BasePart", true)
        end
        if not refPart then
            local coinContainer: Instance? = (map :: Instance):FindFirstChild("CoinContainer")
            if coinContainer then
                refPart = coinContainer:FindFirstChildWhichIsA("BasePart", true)
            end
        end
        if not refPart then
            refPart = (map :: Instance):FindFirstChildWhichIsA("BasePart", true)
        end
        if refPart then
            if (rootPos - refPart.Position).Magnitude > 1500 then
                return false, nil
            end
            return true, refPart.Position
        end
        return true, rootPos
    end

    local function restoreVictims(): ()
        for part: BasePart, snap in pairs(victimSnapshots) do
            if part and part.Parent then
                pcall(function(): ()
                    part.Anchored = snap.anchored
                    part.CanCollide = snap.canCollide
                    part.CFrame = snap.cframe
                end)
            end
        end
        table.clear(victimSnapshots)
    end

    local function restoreLocalCharacter(): ()
        restoreVictims()
        if services and services.activity then
            pcall(function(): ()
                services.activity.set("fling", false)
            end)
        end
        for part: Instance, wasCollide: boolean in pairs(savedCollide) do
            if part and part.Parent then
                pcall(function(): ()
                    (part :: BasePart).CanCollide = wasCollide
                end)
            end
        end
        table.clear(savedCollide)
        for part: BasePart, oldProps: PhysicalProperties? in pairs(savedCustomProps) do
            if part and part.Parent then
                pcall(function(): ()
                    part.CustomPhysicalProperties = oldProps
                end)
            end
        end
        table.clear(savedCustomProps)

        local character: Model?, humanoid: Humanoid?, root: BasePart? = characterParts()
        if humanoid then
            pcall(function(): ()
                humanoid.PlatformStand = false
                humanoid.AutoRotate = true
                humanoid:SetStateEnabled(Enum.HumanoidStateType.FallingDown, true)
                humanoid:SetStateEnabled(Enum.HumanoidStateType.Seated, true)
                humanoid:ChangeState(Enum.HumanoidStateType.Running)
            end)
        end
        if root then
            pcall(function(): ()
                root.AssemblyLinearVelocity = Vector3.zero
                root.AssemblyAngularVelocity = Vector3.zero
                if safeAbovePosition then
                    local groundPos: Vector3 = safeAbovePosition - Vector3.new(0, ABOVE_MAP_OFFSET - 4, 0)
                    local targetCF: CFrame = CFrame.new(groundPos)
                        * CFrame.Angles(0, yawOf(root.CFrame), 0)
                    if character then
                        pcall(function(): ()
                            character:PivotTo(targetCF)
                        end)
                    end
                    root.CFrame = targetCF
                end
            end)
        end
    end

    local function disableCharacterCollisions(character: Model): ()
        for _, object: Instance in ipairs(character:GetDescendants()) do
            if object:IsA("BasePart") then
                if savedCollide[object] == nil then
                    savedCollide[object] = (object :: BasePart).CanCollide
                end
                pcall(function(): ()
                    (object :: BasePart).CanCollide = false
                end)
            end
        end
    end

    local function enableCharacterFlingCollisions(character: Model): ()
        local heavyProps: PhysicalProperties = PhysicalProperties.new(100, 0.3, 0.5, 1, 1)
        for _, object: Instance in ipairs(character:GetChildren()) do
            if object:IsA("BasePart") then
                local part: BasePart = object :: BasePart
                if savedCollide[part] == nil then
                    savedCollide[part] = part.CanCollide
                end
                if savedCustomProps[part] == nil then
                    pcall(function(): ()
                        savedCustomProps[part] = part.CustomPhysicalProperties
                    end)
                end
                pcall(function(): ()
                    part.CanCollide = true
                    part.CustomPhysicalProperties = heavyProps
                end)
            end
        end
    end

    local function holdAboveMap(
        character: Model,
        humanoid: Humanoid,
        root: BasePart
    ): ()
        if services and services.activity then
            pcall(function(): ()
                services.activity.set("fling", false)
            end)
        end
        disableCharacterCollisions(character)
        local holdPos: Vector3 = safeAbovePosition
            or (root.Position + Vector3.new(0, ABOVE_MAP_OFFSET, 0))
        pcall(function(): ()
            humanoid.PlatformStand = true
            humanoid.AutoRotate = false
            root.AssemblyLinearVelocity = Vector3.zero
            root.AssemblyAngularVelocity = Vector3.zero
            root.CFrame = CFrame.new(holdPos)
                * CFrame.Angles(0, yawOf(root.CFrame), 0)
        end)
    end

    local function findLocalTool(toolName: string): Tool?
        local character: Model? = LocalPlayer and LocalPlayer.Character
        if type(getPlayerWeapon) == "function" then
            local found: Tool? = getPlayerWeapon(LocalPlayer, toolName, false)
            if found then
                return found
            end
        end
        if character then
            local inChar: Instance? = character:FindFirstChild(toolName)
            if inChar and inChar:IsA("Tool") then
                return inChar
            end
        end
        local backpack: Instance? = LocalPlayer and LocalPlayer:FindFirstChildOfClass("Backpack")
        if backpack then
            local inBp: Instance? = backpack:FindFirstChild(toolName)
            if inBp and inBp:IsA("Tool") then
                return inBp
            end
        end
        return nil
    end

    local function ensureToolEquipped(
        character: Model,
        humanoid: Humanoid,
        tool: Tool
    ): boolean
        if tool.Parent == character then
            return true
        end
        pcall(function(): ()
            humanoid:EquipTool(tool)
        end)
        if tool.Parent ~= character then
            pcall(function(): ()
                tool.Parent = character
            end)
        end
        return tool.Parent == character
    end

    local function findActiveMurderer(): Player?
        local playersService: Players = game:GetService("Players")
        if type(getPlayerWeapon) == "function" then
            for _, player: Player in ipairs(playersService:GetPlayers()) do
                if player ~= LocalPlayer
                    and (type(isPlayerAlive) ~= "function" or isPlayerAlive(player))
                    and getPlayerWeapon(player, "Gun", false) == nil
                    and getPlayerWeapon(player, "Knife", false) ~= nil then
                    return player
                end
            end
        end
        if type(findMurderer) == "function" then
            local target: Player? = findMurderer()
            if target
                and target ~= LocalPlayer
                and (type(isPlayerAlive) ~= "function" or isPlayerAlive(target))
                and (type(getPlayerWeapon) ~= "function"
                    or getPlayerWeapon(target, "Gun", false) == nil) then
                return target
            end
        end
        for _, player: Player in ipairs(playersService:GetPlayers()) do
            if player ~= LocalPlayer
                and (type(isPlayerAlive) ~= "function" or isPlayerAlive(player))
                and (type(getPlayerWeapon) == "function"
                    and getPlayerWeapon(player, "Gun", false) == nil) then
                local role: any = type(getPlayerRole) == "function"
                    and getPlayerRole(player)
                    or nil
                if role == "Murderer" then
                    return player
                end
            end
        end
        return nil
    end

    local function findActiveSheriffOrHero(): Player?
        local playersService: Players = game:GetService("Players")
        if type(getPlayerWeapon) == "function" then
            for _, player: Player in ipairs(playersService:GetPlayers()) do
                if player ~= LocalPlayer
                    and (type(isPlayerAlive) ~= "function" or isPlayerAlive(player))
                    and getPlayerWeapon(player, "Knife", false) == nil
                    and getPlayerWeapon(player, "Gun", false) ~= nil then
                    return player
                end
            end
        end
        if type(findSheriff) == "function" then
            local target: Player? = findSheriff()
            if target
                and target ~= LocalPlayer
                and (type(isPlayerAlive) ~= "function" or isPlayerAlive(target))
                and (type(getPlayerWeapon) ~= "function"
                    or getPlayerWeapon(target, "Knife", false) == nil) then
                return target
            end
        end
        return nil
    end

    -- Insta Kill as Murder (Kill All while staying safely above the map).
    local function runInstaKillAsMurder(
        character: Model,
        humanoid: Humanoid,
        root: BasePart,
        knife: Tool
    ): boolean
        holdAboveMap(character, humanoid, root)
        if not ensureToolEquipped(character, humanoid, knife) then
            return false
        end
        local handle: BasePart? = (knife:FindFirstChild("Handle") :: BasePart?)
            or knife:FindFirstChildWhichIsA("BasePart")
        local slashRemote: Instance? = knife:FindFirstChild("Slash")
        local stabRemote: Instance? = knife:FindFirstChild("Stab")
        local eventsFolder: Instance? = knife:FindFirstChild("Events")
        local touchedRemote: Instance? = eventsFolder
            and eventsFolder:FindFirstChild("HandleTouched")

        local attackCF: CFrame = (handle and handle.CFrame)
            or (root.CFrame * CFrame.new(0, 0, -2.5))
        local playersService: Players = game:GetService("Players")
        local hitAny: boolean = false

        for _, player: Player in ipairs(playersService:GetPlayers()) do
            if player ~= LocalPlayer
                and (type(isPlayerAlive) ~= "function" or isPlayerAlive(player))
                and (type(isProtectedTarget) ~= "function" or not isProtectedTarget(player)) then
                local victimChar: Model? = player.Character
                local victimRoot: BasePart? = victimChar
                    and victimChar:FindFirstChild("HumanoidRootPart") :: BasePart?
                if victimChar and victimRoot then
                    local onVictimMap: boolean = isCharacterOnMap(victimRoot)
                    if onVictimMap then
                        hitAny = true
                        if not victimSnapshots[victimRoot] then
                            pcall(function(): ()
                                victimSnapshots[victimRoot] = {
                                    cframe = victimRoot.CFrame,
                                    anchored = victimRoot.Anchored,
                                    canCollide = victimRoot.CanCollide,
                                }
                            end)
                        end
                        pcall(function(): ()
                            victimRoot.Anchored = true
                            victimRoot.CanCollide = false
                            victimRoot.AssemblyLinearVelocity = Vector3.zero
                            victimRoot.AssemblyAngularVelocity = Vector3.zero
                            victimRoot.CFrame = attackCF
                        end)
                        if slashRemote and slashRemote:IsA("RemoteEvent") then
                            pcall(function(): ()
                                (slashRemote :: RemoteEvent):FireServer("Slash")
                            end)
                        end
                        if stabRemote and stabRemote:IsA("RemoteEvent") then
                            pcall(function(): ()
                                (stabRemote :: RemoteEvent):FireServer("Slash")
                            end)
                        end
                        if touchedRemote and touchedRemote:IsA("RemoteEvent") then
                            pcall(function(): ()
                                (touchedRemote :: RemoteEvent):FireServer(victimRoot)
                            end)
                        end
                        if handle and type(state.fireTouchInterest) == "function" then
                            pcall(state.fireTouchInterest, handle, victimRoot, 0)
                            pcall(state.fireTouchInterest, handle, victimRoot, 1)
                        end
                    end
                end
            end
        end
        if not hitAny then
            restoreVictims()
            awardRoundWin("Murderer")
        end
        return hitAny
    end

    -- Insta Kill As Sheriff (uses AutoShoot CFrame immediately upon receiving
    -- the Gun, whether our role is Sheriff, Hero, or Innocent).
    local function runInstaKillAsSheriff(
        character: Model,
        humanoid: Humanoid,
        root: BasePart,
        gun: Tool
    ): boolean
        holdAboveMap(character, humanoid, root)
        if not ensureToolEquipped(character, humanoid, gun) then
            return false
        end
        local murderer: Player? = findActiveMurderer()
        if not murderer then
            if roundSawMurderer then
                awardRoundWin("Sheriff")
            end
            return false
        end
        roundSawMurderer = true
        if type(state.mm2AutoShootCFrameOnce) == "function" then
            local okHelper: boolean, helperRes: any =
                pcall(state.mm2AutoShootCFrameOnce, false)
            if okHelper and helperRes == true then
                return true
            end
        end
        if type(isProtectedTarget) == "function" and isProtectedTarget(murderer) then
            return false
        end
        local targetChar: Model? = murderer.Character
        local targetRoot: BasePart? = targetChar
            and targetChar:FindFirstChild("HumanoidRootPart") :: BasePart?
        if not targetRoot then
            return false
        end
        if type(state.mm2FireSilentShot) == "function" then
            -- v139.4: pass our options so the shared path keeps the wall check.
            local okSilent: boolean, res: any = pcall(state.mm2FireSilentShot, murderer, {
                continuous = true,
                quiet = true,
                wallCheck = true,
            })
            if okSilent and res == true then
                return true
            end
        end
        local shootRemote: Instance? = gun:FindFirstChild("Shoot")
        if not (shootRemote and shootRemote:IsA("RemoteEvent")) then
            return false
        end
        local originCF: CFrame = (type(getGunOriginCFrame) == "function"
            and getGunOriginCFrame(character, gun))
            or root.CFrame
        local endpoint: Vector3 = targetRoot.Position
        if type(prediction) == "table" and type(prediction.gunAim) == "function" then
            local okAim: boolean, aimPt: any = pcall(
                prediction.gunAim,
                murderer,
                originCF.Position,
                "Bullet"
            )
            if okAim and typeof(aimPt) == "Vector3" then
                endpoint = aimPt
            else
                local vel: Vector3 = Vector3.zero
                pcall(function(): ()
                    vel = targetRoot.AssemblyLinearVelocity
                end)
                endpoint = targetRoot.Position + vel * 0.08
            end
        end
        local startPoint: Vector3 = endpoint + Vector3.new(0, 1.5, -2.5)
        local shotOrigin: CFrame = CFrame.lookAt(startPoint, endpoint)
        local shotEnd: CFrame = CFrame.new(endpoint)
        -- v139.4: the hand-rolled fallback must not shoot through a wall.
        if type(isWeaponPathClear) == "function" then
            local okClear: boolean, allowed: any = pcall(
                isWeaponPathClear,
                originCF.Position,
                endpoint,
                targetChar,
                nil,
                {ignoreTarget = true}
            )
            if not okClear or allowed ~= true then
                return false
            end
        end
        pcall(function(): ()
            (shootRemote :: RemoteEvent):FireServer(shotOrigin, shotEnd)
        end)
        return true
    end

    -- Grab the dropped gun ONLY via fireTouchInterest while staying safely
    -- above the map (never risking a teleport down to the map).
    local function grabDroppedGunTouchOnly(
        character: Model,
        humanoid: Humanoid,
        root: BasePart,
        drop: Instance
    ): boolean
        holdAboveMap(character, humanoid, root)
        if type(state.fireTouchInterest) ~= "function" then
            return false
        end
        local touchPart: BasePart? = nil
        if drop:IsA("BasePart") then
            touchPart = drop :: BasePart
        elseif drop:IsA("Model") then
            touchPart = (drop:FindFirstChild("Handle", true) :: BasePart?)
                or drop:FindFirstChildWhichIsA("BasePart", true)
        end
        if not touchPart then
            return false
        end
        pcall(state.fireTouchInterest, root, touchPart, 0)
        pcall(state.fireTouchInterest, root, touchPart, 1)
        return findLocalTool("Gun") ~= nil
    end

    -- Controlled, bounded Fling on the Sheriff: waits for map/character warmup,
    -- uses state.mm2PerformFling with a returnCFrame to safeAbovePosition when
    -- available (or bounded 10000-stud/s impulse fallback so our character
    -- never enters Roblox's floating-point NullZone or void).
    local function flingSheriffStep(
        character: Model,
        humanoid: Humanoid,
        root: BasePart,
        sheriff: Player
    ): boolean
        local sheriffChar: Model? = sheriff.Character
        local sheriffHum: Humanoid? = sheriffChar
            and sheriffChar:FindFirstChildOfClass("Humanoid")
        local sheriffRoot: BasePart? = sheriffChar
            and sheriffChar:FindFirstChild("HumanoidRootPart") :: BasePart?
        if not sheriffChar or not sheriffHum or sheriffHum.Health <= 0 or sheriffHum.Sit or not sheriffRoot then
            holdAboveMap(character, humanoid, root)
            return false
        end
        local destroyHeight: number = workspace.FallenPartsDestroyHeight
        if destroyHeight ~= destroyHeight or math.abs(destroyHeight) > 1e6 then
            destroyHeight = -500
        end
        local sheriffPos: Vector3 = sheriffRoot.Position
        if sheriffPos.Y <= destroyHeight + 60 then
            holdAboveMap(character, humanoid, root)
            return false
        end
        local sheriffOnMap: boolean = isCharacterOnMap(sheriffRoot)
        if not sheriffOnMap then
            holdAboveMap(character, humanoid, root)
            return false
        end

        if type(state.mm2PerformFling) == "function" then
            if os.clock() - lastSheriffFlingAt < SHERIFF_FLING_COOLDOWN_SECONDS then
                holdAboveMap(character, humanoid, root)
                return true
            end
            flingBusy = true
            lastSheriffFlingAt = os.clock()
            local returnPos: Vector3 = safeAbovePosition
                or (root.Position + Vector3.new(0, ABOVE_MAP_OFFSET, 0))
            task.spawn(function(): ()
                pcall(state.mm2PerformFling, sheriff, {
                    silent = true,
                    duration = 2.6,
                    blocking = true,
                    returnCFrame = CFrame.new(returnPos),
                    shouldStop = function(): boolean
                        if not autoWinActive or not settings.flingSheriffGetGun then
                            return true
                        end
                        if findLocalTool("Gun") ~= nil then
                            return true
                        end
                        if type(findDroppedGun) == "function" and findDroppedGun() ~= nil then
                            return true
                        end
                        return false
                    end,
                })
                lastSheriffFlingAt = os.clock()
                local cNow: Model?, hNow: Humanoid?, rNow: BasePart? = characterParts()
                if cNow and hNow and rNow and autoWinActive then
                    holdAboveMap(cNow, hNow, rNow)
                end
                flingBusy = false
            end)
            return true
        end

        if type(state.mm2FlingStep) == "function" then
            local okStep: boolean, res: any = pcall(
                state.mm2FlingStep,
                character,
                humanoid,
                root,
                sheriff,
                savedCollide,
                savedCustomProps
            )
            if okStep and res == true then
                return true
            end
            holdAboveMap(character, humanoid, root)
            return false
        end

        if services and services.activity then
            pcall(function(): ()
                services.activity.set("fling", true)
            end)
        end
        enableCharacterFlingCollisions(character)
        flingAngle = (flingAngle + 100) % 360
        local phase: number = math.floor(flingAngle / 100) % 4
        local sweepOffset: CFrame = if phase == 0
            then CFrame.new(0, 1.5, 0)
            elseif phase == 1 then CFrame.new(0, -1.5, 0)
            elseif phase == 2 then CFrame.new(2.0, 1.2, -2.0)
            else CFrame.new(-2.0, -1.2, 2.0)
        local sheriffVel: Vector3 = Vector3.zero
        pcall(function(): ()
            sheriffVel = sheriffRoot.AssemblyLinearVelocity
        end)
        local targetCF: CFrame = CFrame.new(sheriffPos + sheriffVel * 0.08)
            * sweepOffset
            * CFrame.Angles(math.rad(flingAngle), math.rad(flingAngle), 0)
        pcall(function(): ()
            if humanoid.Sit then
                humanoid.Sit = false
            end
            humanoid.AutoRotate = false
            root.CFrame = targetCF
            character:PivotTo(targetCF)
            root.AssemblyLinearVelocity = Vector3.new(10000, 12000, 10000)
            root.AssemblyAngularVelocity = Vector3.new(12000, 12000, 12000)
        end)
        return true
    end

    local function autoWinTick(): ()
        if not autoWinActive or flingBusy then
            return
        end
        local character: Model?, humanoid: Humanoid?, root: BasePart? = characterParts()
        if not character or not humanoid or not root or humanoid.Health <= 0 then
            return
        end

        local onMap: boolean, mapRefPos: Vector3? = isCharacterOnMap(root)
        local activeMurderer: Player? = findActiveMurderer()
        if activeMurderer then
            roundSawMurderer = true
        end

        if not onMap then
            if wasOnMap then
                wasOnMap = false
                mapEnteredAt = 0
                sheriffFirstSeenAt = 0
                lastSheriffFlingAt = 0
                safeAbovePosition = nil
```

```
            restoreLocalCharacter()
        end

        local roundStillRunning: boolean = type(findMM2Map) == "function"
            and findMM2Map() ~= nil
            and activeMurderer ~= nil
            and (type(isRoundLifecycleActive) ~= "function"
                or isRoundLifecycleActive() ~= false)

        if not roundStillRunning then
            if participatedInRound and not roundCountedWin and roundSawMurderer then
                local myRoleNow: any = type(getPlayerRole) == "function"
                    and getPlayerRole(LocalPlayer)
                    or nil
                if myRoleNow ~= "Dead" then
                    awardRoundWin(roundMyRole)
                end
            end
            participatedInRound = false
            roundMyRole = nil
            roundSawMurderer = false
            roundCountedWin = false
            flungMurderThisDeath = false
            setStatus("Auto Win: waiting for map")
            return
        end

        -- We are in the lobby while the round is still running on the map
        -- (meaning we died / were killed during the round).
        local myRole: any = type(getPlayerRole) == "function"
            and getPlayerRole(LocalPlayer)
            or nil
        local diedInRound: boolean = participatedInRound or myRole == "Dead"
        if settings.flingMurderIfDead
            and diedInRound
            and not flungMurderThisDeath
            and activeMurderer then
            flungMurderThisDeath = true
            if type(state.mm2PerformFling) == "function" then
                flingBusy = true
                setStatus("Auto Win: Flinging Murderer (killed us)")
                task.spawn(function(): ()
                    pcall(state.mm2PerformFling, activeMurderer, {
                        silent = true,
                        duration = 4.0,
                        blocking = true,
                    })
                    flingBusy = false
                end)
                return
            end
        end

        setStatus("Auto Win: waiting in lobby")
        return
    end

    if not wasOnMap or not safeAbovePosition then
        wasOnMap = true
        mapEnteredAt = os.clock()
        sheriffFirstSeenAt = 0
        lastSheriffFlingAt = 0
        participatedInRound = true
        flungMurderThisDeath = false
        local anchor: Vector3 = mapRefPos or root.Position
        safeAbovePosition = anchor + Vector3.new(0, ABOVE_MAP_OFFSET, 0)
    end

    -- Check if the Murderer died while we are alive on the map -> count win!
    if roundSawMurderer and not activeMurderer and not roundCountedWin then
        awardRoundWin(roundMyRole)
    end

    -- 1) Murderer / Knife check: Insta Kill as Murder (Kill All).
    local knife: Tool? = findLocalTool("Knife")
    if knife then
        roundMyRole = "Murderer"
        if settings.instaKillMurder then
            runInstaKillAsMurder(character, humanoid, root, knife)
            setStatus("Auto Win: Insta Kill as Murder")
        else
            holdAboveMap(character, humanoid, root)
            setStatus("Auto Win: safe above map (Murderer)")
        end
        return
    end

    -- 2) Gun check (Sheriff OR Hero OR Innocent who received the Gun!):
    --    Dispara al momento que reciba la pistola con AutoShoot CFrame,
    --    incluso si Fling Sheriff and Get Gun está desactivado y el arma
    --    se recogió con Auto Grab Gun.
    local gun: Tool? = findLocalTool("Gun")
    if gun then
        roundMyRole = "Sheriff"
        if settings.instaKillSheriff then
            runInstaKillAsSheriff(character, humanoid, root, gun)
            setStatus("Auto Win: Insta Kill As Sheriff (CFrame)")
        else
            holdAboveMap(character, humanoid, root)
            setStatus("Auto Win: safe above map (Gun ready)")
        end
        return
    end

    if not roundMyRole then
        local roleNow: any = type(getPlayerRole) == "function"
            and getPlayerRole(LocalPlayer)
            or nil
        if roleNow == "Murderer" then
            roundMyRole = "Murderer"
        elseif roleNow == "Sheriff" or roleNow == "Hero" then
            roundMyRole = "Sheriff"
        else
            roundMyRole = "Innocent"
        end
    end

    -- 3) Dropped gun on the map: grab ONLY via fireTouchInterest (no
    --    risky teleport) if Fling Sheriff and Get Gun is ON or Auto Grab
    --    Gun is enabled.
    local autoGrabExternal: boolean = state.mm2GunAuraEnabled == true
        or state.mm2AutoGetGunEnabled == true
    if settings.flingSheriffGetGun or autoGrabExternal then
        local drop: Instance? = type(findDroppedGun) == "function"
            and findDroppedGun()
            or nil
        if drop then
            local grabbed: boolean = grabDroppedGunTouchOnly(
                character,
                humanoid,
                root,
                drop
            )
            setStatus("Auto Win: touching dropped gun")
            if grabbed then
                roundMyRole = "Sheriff"
                local freshGun: Tool? = findLocalTool("Gun")
                if freshGun and settings.instaKillSheriff then
                    runInstaKillAsSheriff(character, humanoid, root, freshGun)
                end
            end
            return
        end
    end

    -- 4) Fling Sheriff and Get Gun while Sheriff is still alive.
    --    Wait for the map and characters to finish loading before flinging
    --    so the local character never gets destabilized on spawn.
    if settings.flingSheriffGetGun then
        local nowClock: number = os.clock()
        local mapElapsed: number = nowClock - mapEnteredAt
        if mapElapsed < MAP_FLING_WARMUP_SECONDS then
            holdAboveMap(character, humanoid, root)
            setStatus("Auto Win: loading map before fling...")
            return
        end
        local sheriff: Player? = findActiveSheriffOrHero()
        if sheriff
            and sheriff ~= LocalPlayer
            and (type(isProtectedTarget) ~= "function" or not isProtectedTarget(sheriff)) then
            if sheriffFirstSeenAt == 0 then
                sheriffFirstSeenAt = nowClock
            end
            if nowClock - sheriffFirstSeenAt < SHERIFF_READY_DELAY_SECONDS then
                holdAboveMap(character, humanoid, root)
                setStatus("Auto Win: waiting for Sheriff to settle...")
                return
            end
            if flingSheriffStep(character, humanoid, root, sheriff) then
                setStatus("Auto Win: flinging Sheriff for gun")
                return
            end
        else
            sheriffFirstSeenAt = 0
        end
    end

    -- 5) Default on map (including pre-round before weapons are given):
    --    Stay safely high above the map where nobody can kill us.
    holdAboveMap(character, humanoid, root)
    setStatus("Auto Win: safe above map")
end

local function clearToolConnections(): ()
    for _, conn: RBXScriptConnection in ipairs(toolConnections) do
        pcall(function(): ()
            conn:Disconnect()
        end)
    end
    table.clear(toolConnections)
end

local function watchToolContainer(container: Instance?): ()
    if not container
        or (type((container :: any).ChildAdded) ~= "table"
            and typeof((container :: any).ChildAdded) ~= "RBXScriptSignal") then
        return
    end
    pcall(function(): ()
        local conn: RBXScriptConnection = (container :: any).ChildAdded:Connect(
            function(child: Instance): ()
                if autoWinActive
                    and child
                    and (child.Name == "Gun" or child.Name == "Knife") then
                    autoWinTick()
                end
            end
        )
        table.insert(toolConnections, conn)
    end)
end

local function setupInstantWeaponWatchers(): ()
    clearToolConnections()
    if not LocalPlayer then
        return
    end
    watchToolContainer(LocalPlayer.Character)
    watchToolContainer(LocalPlayer:FindFirstChildOfClass("Backpack"))
    pcall(function(): ()
        if LocalPlayer.CharacterAdded then
            table.insert(
                toolConnections,
                LocalPlayer.CharacterAdded:Connect(function(newChar: Model): ()
                    watchToolContainer(newChar)
                    watchToolContainer(LocalPlayer:FindFirstChildOfClass("Backpack"))
                end)
            )
        end
        if LocalPlayer.ChildAdded then
            table.insert(
                toolConnections,
                LocalPlayer.ChildAdded:Connect(function(child: Instance): ()
                    if child and child:IsA("Backpack") then
                        watchToolContainer(child)
                    end
                end)
            )
        end
    end)
end

local function toggleAutoWin(enabled: boolean): ()
    autoWinActive = enabled == true
    disconnectFeatureConnection("MM2AutoWin")
    clearToolConnections()
    if not enabled then
        wasOnMap = false
        participatedInRound = false
        roundMyRole = nil
        roundSawMurderer = false
        roundCountedWin = false
        flungMurderThisDeath = false
        restoreLocalCharacter()
        safeAbovePosition = nil
        setStatus(nil)
        return
    end
    setupInstantWeaponWatchers()
    featureConnections.MM2AutoWin = TaskManager:Connect(autoWinTick, nil, "MM2 Auto Win")
    autoWinTick()
end

AutoWinFeature = createUniversalFeature(
    "Auto Win",
    "Keeps your character safely above the map as soon as you enter the map (even before the round starts) so nobody can kill you, with Insta Kill as Murder, Insta Kill As Sheriff (AutoShoot CFrame), Fling Sheriff and Get Gun, and Fling Murder if Dead.",
    0,
    toggleAutoWin,
    {
        categoryName = "Utilities",
        parent = MM2Scroll,
        registry = mm2Features,
    }
)
state.mm2AutoWinFeature = AutoWinFeature

if type(state.addSectionOption) == "function" then
    state.addSectionOption(AutoWinFeature, "Overlay Info")
end
if type(state.addStatOption) == "function" then
    sheriffWinsStat = state.addStatOption(AutoWinFeature, "Sheriff Wins", "0")
    murderWinsStat = state.addStatOption(AutoWinFeature, "Murder Wins", "0")
    innocentWinsStat = state.addStatOption(AutoWinFeature, "Innocent Wins", "0")
end

addToggleOption(AutoWinFeature, "Insta Kill as Murder", true,
    function(value: boolean): ()
        settings.instaKillMurder = value == true
    end,
    "Automatically kills all living players as soon as you receive the Knife as Murderer.")

addToggleOption(AutoWinFeature, "Insta Kill As Sheriff", true,
    function(value: boolean): ()
        settings.instaKillSheriff = value == true
    end,
    "Automatically equips the Gun and shoots the Murderer using AutoShoot CFrame the instant you receive the Gun (as Sheriff or Hero).")

addToggleOption(AutoWinFeature, "Fling Sheriff and Get Gun", false,
    function(value: boolean): ()
        settings.flingSheriffGetGun = value == true
    end,
    "Safely flies to the Sheriff with Fling without killing yourself, then automatically grabs the dropped gun via TouchInterest once the Sheriff dies and shoots the Murderer.")

addToggleOption(AutoWinFeature, "Fling Murder if Dead", false,
    function(value: boolean): ()
        settings.flingMurderIfDead = value == true
    end,
    "If the Murderer kills you and sends you to the lobby while the round is still active, automatically flings the Murderer.")

activeCleanup = function(): ()
    toggleAutoWin(false)
    state.mm2AutoWinFeature = nil
end
Module.Events = featureConnections
Module.Initialized = true
return Module
end

function Module.destroy(): ()
    if not Module.Initialized then
        return
    end
    Module.Initialized = false
    pcall(activeCleanup)
    activeCleanup = function(): () end
    Module.Events = {}
    Module.Runtime = nil
end

return Module

--------------------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/MM2/Other/AutoWin.lua
--------------------------------------------------------------------------------------------

--------------------------------------------------------------------------------------------
ARCHIVO: src/games/MM2/Other/CoinsAura.lua
BYTES: 9987
SHA-256: ae8b33f33b926e97cc489ab99061615b1935ef9c2818cece8fa4af3e307851ea
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------------------
local Module = {
    Name = "MM2 Coins Aura",
    Events = {} :: {[string]: any},
    Initialized = false,
    Runtime = nil :: any,
}

local activeCleanup: () -> () = function(): () end

function Module.init(runtime: any): any
    if Module.Initialized then
        return Module
    end
    local core: any = state.mm2Core
    assert(type(core) == "table", "MM2 Coins Aura requires the MM2 core module")
    Module.Runtime = runtime

    -- Maximum coin pickup distance accepted by the MM2 server (~15 studs).
    local MAX_SERVER_REACH: number = 15
    local MAX_SERVER_REACH_SQ: number = MAX_SERVER_REACH * MAX_SERVER_REACH
    local TICK_INTERVAL: number = 0.08
    local CACHE_INTERVAL: number = 0.5

    local savedCFrames: {[BasePart]: CFrame} = {}
    local savedPositions: {[BasePart]: Vector3} = {}
    local activeCoin: BasePart? = nil

    local function rememberCoin(serverPart: BasePart): Vector3?
        local pos: Vector3? = savedPositions[serverPart]
        if pos then
            return pos
        end
        local cf: CFrame = serverPart.CFrame
        savedCFrames[serverPart] = cf
        savedPositions[serverPart] = cf.Position
        return cf.Position
    end

    local function restoreCoin(serverPart: BasePart?): ()
        if not serverPart then
            return
        end
        local cf: CFrame? = savedCFrames[serverPart]
        if cf and serverPart.Parent then
            pcall(function(): ()
                serverPart.CFrame = cf
            end)
        end
        if activeCoin == serverPart then
            activeCoin = nil
        end
    end

    local function restoreAllCoins(): ()
        for serverPart, cf in pairs(savedCFrames) do
            if serverPart and serverPart.Parent then
                pcall(function(): ()
                    serverPart.CFrame = cf
                end)
            end
        end
        table.clear(savedCFrames)
        table.clear(savedPositions)
        activeCoin = nil
    end

    local function isCoinCollectable(serverPart: BasePart): boolean
        if not serverPart or not serverPart.Parent then
            return false
        end
        if serverPart:GetAttribute("Collected") == true
            or serverPart:GetAttribute("Delete") == true then
            return false
        end
        local visual: Instance? = serverPart:FindFirstChild("CoinVisual")
        if visual and (
            visual:GetAttribute("Collected") == true
            or visual:GetAttribute("Delete") == true
        ) then
            return false
        end
        return true
    end

    -- Detach CoinVisual's WeldConstraint once before pulling Coin_Server so
    -- moving Coin_Server never triggers rigid-body weld recalculation or
    -- fights CoinVisualizer's PreSimulation spin.
    local function detachVisualWeld(serverPart: BasePart): ()
        local visual: Instance? = serverPart:FindFirstChild("CoinVisual")
        if not visual or not visual:IsA("BasePart") then
            return
        end
        local vPart: BasePart = visual :: BasePart
        vPart.Anchored = true
        local weld: Instance? = vPart:FindFirstChildOfClass("WeldConstraint")
        if weld then
            (weld :: WeldConstraint).Enabled = false
        end
    end

    local function pulseCoinTouch(root: BasePart, serverPart: BasePart): ()
        rememberCoin(serverPart)
        detachVisualWeld(serverPart)
        serverPart.CanCollide = false
        serverPart.CFrame = root.CFrame
        local fireTouch: any = state.fireTouchInterest
        if type(fireTouch) == "function" then
            pcall(fireTouch, root, serverPart, 0)
            pcall(fireTouch, root, serverPart, 1)
        end
    end

    local CoinsAuraFeature: any = nil

    local function toggleCoinsAura(enabled: boolean): ()
        disconnectFeatureConnection("MM2CoinsAura")
        restoreAllCoins()

        if not enabled then
            if CoinsAuraFeature then
                CoinsAuraFeature:SetStatus(nil)
            end
            return
        end

        local cachedCoins: {BasePart} = {}
        local lastCacheAt: number = -math.huge
        local lastTickAt: number = -math.huge

        if CoinsAuraFeature then
            CoinsAuraFeature:SetStatus("Max reach")
        end

        local function refreshCoins(now: number): ()
            if now - lastCacheAt < CACHE_INTERVAL then
                return
            end
            lastCacheAt = now

            local list: {BasePart} = {}
            local seen: {[BasePart]: boolean} = {}

            local function addCoin(candidate: Instance?): ()
                if not candidate then
                    return
                end
                local serverPart: BasePart? = nil
                if candidate:IsA("BasePart") then
                    if candidate.Name == "Coin_Server" or candidate.Name == "ServerCoinPart" then
                        serverPart = candidate :: BasePart
                    elseif candidate.Parent and candidate.Parent:IsA("BasePart")
                        and (candidate.Parent.Name == "Coin_Server" or candidate.Parent.Name == "ServerCoinPart") then
                        serverPart = candidate.Parent :: BasePart
                    else
                        serverPart = candidate :: BasePart
                    end
                elseif candidate:IsA("Model") then
                    local child: Instance? = candidate:FindFirstChild("Coin_Server")
                        or candidate:FindFirstChildWhichIsA("BasePart")
                    if child and child:IsA("BasePart") then
                        serverPart = child :: BasePart
                    end
                end
                if serverPart and not seen[serverPart] and isCoinCollectable(serverPart) then
                    seen[serverPart] = true
                    rememberCoin(serverPart)
                    list[#list + 1] = serverPart
                end
            end

            if type(core.findMM2Map) == "function" then
                local map: Instance? = core.findMM2Map()
                if map then
                    local container: Instance? = map:FindFirstChild("CoinContainer")
                        or map:FindFirstChild("CoinAreas")
                        or map:FindFirstChild("Coins")
                    if container then
                        for _, child: Instance in ipairs(container:GetChildren()) do
                            addCoin(child)
                        end
                    end
                end
            end

            if #list == 0 and type(state.mm2CoinPickups) == "function" then
                local okPickups: boolean, pickups: any = pcall(state.mm2CoinPickups)
                if okPickups and type(pickups) == "table" then
                    for _, pickup: Instance in ipairs(pickups) do
                        addCoin(pickup)
                    end
                end
            end

            cachedCoins = list
        end

        local function auraTick(): ()
            local now: number = os.clock()
            if now - lastTickAt < TICK_INTERVAL then
                return
            end
            lastTickAt = now

            local character: Model? = LocalPlayer and LocalPlayer.Character
            local root: BasePart? = character and character:FindFirstChild("HumanoidRootPart") :: BasePart?
            if not root then
                if activeCoin then
                    restoreCoin(activeCoin)
                end
                return
            end
            local humanoid: Humanoid? = character:FindFirstChildOfClass("Humanoid") :: Humanoid?
            if humanoid and humanoid.Health <= 0 then
                if activeCoin then
                    restoreCoin(activeCoin)
                end
                return
            end

            refreshCoins(now)

            local rootPos: Vector3 = root.Position
            local bestCoin: BasePart? = nil
            local bestDistSq: number = MAX_SERVER_REACH_SQ

            for i: number = 1, #cachedCoins do
                local coin: BasePart = cachedCoins[i]
                if isCoinCollectable(coin) then
                    local coinPos: Vector3? = savedPositions[coin] or rememberCoin(coin)
                    if coinPos then
                        local diff: Vector3 = coinPos - rootPos
                        local distSq: number = diff.X * diff.X + diff.Y * diff.Y + diff.Z * diff.Z
                        if distSq <= bestDistSq then
                            bestDistSq = distSq
                            bestCoin = coin
                        end
                    end
                elseif coin == activeCoin then
                    restoreCoin(coin)
                end
            end

            if activeCoin and activeCoin ~= bestCoin then
                restoreCoin(activeCoin)
            end

            if bestCoin then
                activeCoin = bestCoin
                pcall(pulseCoinTouch, root, bestCoin)
            end
        end

        featureConnections.MM2CoinsAura = TaskManager:Connect(auraTick, nil, "MM2 Coins Aura")
        auraTick()
    end

    CoinsAuraFeature = createUniversalFeature(
        "Coins Aura",
        "Automatically collects nearby coins at the maximum reach allowed by the MM2 server with minimal CPU usage.",
        7,
        toggleCoinsAura,
        {
            categoryName = "Utilities",
            parent = MM2Scroll,
            registry = mm2Features,
        }
    )
    state.mm2CoinsAuraFeature = CoinsAuraFeature

    activeCleanup = function(): ()
        toggleCoinsAura(false)
    end
    Module.Events = featureConnections
    Module.Initialized = true
    return Module
end

function Module.destroy(): ()
    if not Module.Initialized then
        return
    end
    Module.Initialized = false
    pcall(activeCleanup)
    activeCleanup = function(): () end
    Module.Events = {}
    Module.Runtime = nil
end

return Module

--------------------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/MM2/Other/CoinsAura.lua
--------------------------------------------------------------------------------------------

--------------------------------------------------------------------------------------------
ARCHIVO: src/games/MM2/Other/Fling.lua
BYTES: 24933
SHA-256: 130ee339db8ff868ebd9e5d1ff3129321563115 5007cc04a05241a9c8486adec
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------------------
-- v191: the MM2 Fling card, rebuilt around the FULL historical card instead of
-- the single-target one v190 shipped.
--
-- History: the old menu had its own Fling (src/games/MM2/Blatant/Fling.lua plus
-- src/libraries/Fling.lua) with role buttons -- "Fling Murderer", "Fling
-- Sheriff", "Fling All Innocents", a live roster labelled by role and an auto
-- fling -- and a different physics engine. v183 removed the universal card and
-- the engine stayed; v190 brought back a minimal "pick a player" card on top of
-- `FlingEngine.lua`. The player asked for the old role buttons back, so this
-- card is that one again, but it keeps running on the CURRENT engine
-- (`state.mm2PerformFling`), which is the one that knows about `flingBoost`,
-- the EpixVel pin and retries. Both engines are never mixed.
--
-- Power is at the maximum the engine accepts (12, i.e. four times YARHM's
-- 9e7/9e8 pose velocity) and retries at the maximum (3), so a single call
-- throws as hard and as reliably as the engine can: that was the second half of
-- the request ("el poder sea el máximo y lo más eficaz posible").
--
-- Anti Fling cancels the contact a fling needs, so every path here refuses with
-- a notice while it is ON (MM2 only, as the player asked).
local Module = {
    Name = "MM2 Fling",
    Events = {} :: {[string]: any},
```

```
    Initialized = false,
    Runtime = nil :: any,
}

local activeCleanup: () -> () = function(): () end

-- The engine clamps Power to 1..12 and Retries to 0..3. The card's default is
-- the top of both ranges.
local MAX_POWER: number = 12
local MAX_RETRIES: number = 3
local FLING_DURATION: number = 3
local AUTO_FLING_INTERVAL: number = 0.35

local ROLE_ORDER: {[string]: number} = {
    Murderer = 1, Sheriff = 2, Hero = 3, Innocent = 4, Dead = 5,
}

function Module.init(runtime: any): any
    if Module.Initialized then
        return Module
    end
    local core: any = state.mm2Core
    assert(type(core) == "table", "MM2 Fling requires the MM2 core module")
    Module.Runtime = runtime

    local localPlayer: any = LocalPlayer
    local playerService: any = Players
    local gameBridge: any = services and services.gameBridge or nil
    local protectedTargets: any = services and services.protectedTargets or nil
    local TaskManager: any = TaskManager
    local featureConnections: any = featureConnections
    local disconnectFeatureConnection: any = disconnectFeatureConnection

    local findMurderer: any = core.findMurderer
    local findSheriff: any = core.findSheriff
    local getPlayerRole: any = core.getPlayerRole
    local getPlayerWeapon: any = core.getPlayerWeapon
    local isPlayerAlive: any = core.isPlayerAlive

    local settings = {
        power = MAX_POWER,
        retries = MAX_RETRIES,
        autoFling = false,
        autoFlingTarget = nil :: any,
    }
    local FlingFeature: any = nil
    -- The list publishes its stored selection once while it is being built;
    -- acting on that first call would fling a player at injection.
    local listReady: boolean = false
    local selectedPlayer: any = nil
    local selectedLabel: string = ""
    local allInnocentsSequence: number = 0
    local autoElapsed: number = 0

    local function antiFlingOn(): boolean
        return state.antiFlingActive == true
    end

    local function setStatus(text: string?): ()
        if FlingFeature and type(FlingFeature.SetStatus) == "function" then
            pcall(function(): ()
                FlingFeature:SetStatus(text)
            end)
        end
    end

    local function notifyUser(text: string): ()
        if type(notify) == "function" then
            pcall(notify, text)
        end
    end

    local function roster(): {any}
        local ok: boolean, players: any = pcall(function(): any
            return playerService:GetPlayers()
        end)
        if not ok or type(players) ~= "table" then
            return {}
        end
        return players
    end

    local function livingCharacter(player: any): (any?, any?)
        local character: any = player and player.Character
        if not character or not character.Parent then
            return nil, nil
        end
        local humanoid: any = character:FindFirstChildOfClass("Humanoid")
        if not humanoid or humanoid.Health <= 0 then
            return nil, nil
        end
        return character, humanoid
    end

    local function rootOf(player: any): any?
        local character: any = player and player.Character
        if not character then
            return nil
        end
        local root: any = character:FindFirstChild("HumanoidRootPart")
        if not root or root.Parent == nil then
            return nil
        end
        return root
    end

    -- v192: `Players` above is a CLONED reference on executors that expose
    -- `cloneref`, so `player.Parent == Players` is false for every real player
    -- (a clone never compares equal to the original instance). v191 used that
    -- comparison as its "is this still a player" guard and the roster came out
    -- empty: the card could not see a single target. The guard now asks the
    -- instance what it is instead of which table it belongs to.
    local function isLivePlayer(player: any): boolean
        if not player or player == localPlayer then
            return false
        end
        local classOk: boolean, className: any = pcall(function(): any
            return player.ClassName
        end)
        if not classOk or className ~= "Player" then
            return false
        end
        local parentOk: boolean, parent: any = pcall(function(): any
            return player.Parent
        end)
        return parentOk and parent ~= nil
    end

    -- A target is only ever a living, unprotected other player whose root is
    -- still in the world: a stale name from the previous round must never
    -- become a fling target.
    local function canFling(player: any): boolean
        if not isLivePlayer(player) then
            return false
        end
        if protectedTargets and type(protectedTargets.isProtected) == "function" then
            local ok: boolean, protected: any = pcall(protectedTargets.isProtected, player)
            if ok and protected == true then
                return false
            end
        end
        if type(isPlayerAlive) == "function" then
            local ok: boolean, alive: any = pcall(isPlayerAlive, player)
            if ok and alive == false then
                return false
            end
        end
        local character: any = livingCharacter(player)
        return character ~= nil and rootOf(player) ~= nil
    end

    local function hasWeapon(player: any, toolName: string): boolean
        if type(getPlayerWeapon) ~= "function" then
            return false
        end
        local ok: boolean, weapon: any = pcall(getPlayerWeapon, player, toolName, false)
        return ok and weapon ~= nil
    end

    -- One source of truth for "who is what": the round data first, then the
    -- tool the server actually replicates (the murderer carries the knife, the
    -- sheriff/hero carries the gun), then a dead humanoid. The old card guessed
    -- in three places and none of them used the held tool, which is why its
    -- Murderer button often answered "nobody found".
    local function roleOf(player: any): string?
        if not player then
            return nil
        end
        local role: any = nil
        if type(getPlayerRole) == "function" then
            local ok: boolean, value: any = pcall(getPlayerRole, player)
            if ok then
                role = value
            end
        end
        if not role and gameBridge and type(gameBridge.playerRole) == "function" then
            local ok: boolean, value: any = pcall(gameBridge.playerRole, player)
            if ok then
                role = value
            end
        end
        if role == "Murderer" or role == "Sheriff" or role == "Hero"
            or role == "Innocent" or role == "Dead" then
            return role
        end
        if hasWeapon(player, "Knife") then
            return "Murderer"
        end
        if hasWeapon(player, "Gun") then
            return "Sheriff"
        end
        if role == nil then
            local character: any = player.Character
            local humanoid: any = character and character:FindFirstChildOfClass("Humanoid")
            if humanoid and humanoid.Health <= 0 then
                return "Dead"
            end
        end
        return role
    end

    local function rolesPublished(): boolean
        if type(core.hasActiveRoundRoles) == "function" then
            local ok: boolean, published: any = pcall(core.hasActiveRoundRoles)
            if ok and published == true then
                return true
            end
        end
        for _, player: any in ipairs(roster()) do
            if player ~= localPlayer and roleOf(player) ~= nil then
                return true
            end
        end
        return false
    end

    local function refreshRoles(): ()
        if type(state.clearRoleMemo) == "function" then
            pcall(state.clearRoleMemo)
        end
        if type(core.requestRoleRefresh) == "function" then
            pcall(core.requestRoleRefresh, false)
        end
    end

    local function rosterEntries(withRefresh: boolean?): {{player: any, label: string}}
        if withRefresh ~= false then
            refreshRoles()
        end
        local entries: {{player: any, label: string}} = {}
        for _, player: any in ipairs(roster()) do
            if player ~= localPlayer and canFling(player) then
                local role: string? = roleOf(player)
                table.insert(entries, {
                    player = player,
                    label = tostring(player.Name) .. (role and (" [" .. role .. "]") or ""),
                })
            end
        end
        table.sort(entries, function(left: any, right: any): boolean
            local leftRole: number = ROLE_ORDER[
                (string.match(left.label, "%[(%a+)%]") or "")
            ] or 9
            local rightRole: number = ROLE_ORDER[
                (string.match(right.label, "%[(%a+)%]") or "")
            ] or 9
            if leftRole ~= rightRole then
                return leftRole < rightRole
            end
            return string.lower(left.label) < string.lower(right.label)
        end)
        return entries
    end

    local function resolveMurdererTarget(): any?
        refreshRoles()
        for _, player: any in ipairs(roster()) do
            if player ~= localPlayer and canFling(player)
                and not hasWeapon(player, "Gun") and hasWeapon(player, "Knife") then
                return player
            end
        end
        if type(findMurderer) == "function" then
            local ok: boolean, target: any = pcall(findMurderer)
            if ok and target and canFling(target) and roleOf(target) ~= "Dead" then
                return target
            end
        end
        for _, player: any in ipairs(roster()) do
            if player ~= localPlayer and canFling(player) and roleOf(player) == "Murderer" then
                return player
            end
        end
        return nil
    end

    local function resolveSheriffTarget(): any?
        refreshRoles()
        for _, player: any in ipairs(roster()) do
            if player ~= localPlayer and canFling(player)
                and not hasWeapon(player, "Knife") and hasWeapon(player, "Gun") then
                return player
            end
        end
        if type(findSheriff) == "function" then
            local ok: boolean, target: any = pcall(findSheriff)
            if ok and target and canFling(target) then
                local role: string? = roleOf(target)
                if role ~= "Murderer" and role ~= "Dead" then
                    return target
                end
            end
        end
        for _, player: any in ipairs(roster()) do
            if player ~= localPlayer and canFling(player) then
                local role: string? = roleOf(player)
                if role == "Sheriff" or role == "Hero" then
                    return player
                end
            end
        end
        return nil
    end

    -- What the card is showing, readable by the offline suite: the labels it
    -- publishes and the player the current selection resolves to.
    state.mm2FlingRoster = function(): {any}
        local snapshot: {any} = {}
        for _, entry: any in ipairs(rosterEntries(false)) do
            table.insert(snapshot, {
                name = tostring(entry.player.Name),
                label = entry.label,
            })
        end
        return snapshot
    end
    state.mm2FlingSelection = function(): string?
        return if selectedPlayer then tostring(selectedPlayer.Name) else nil
    end

    local function performFling(target: any, options: {[string]: any}?): boolean
        if antiFlingOn() then
            -- Turning Anti Fling off is the documented answer; without this
            -- notice the fling just looks broken and blames the engine.
            if not (type(options) == "table" and options.silent == true) then
                notifyUser("Anti Fling is ON: turn it off before flinging someone.")
            end
            setStatus("Anti Fling is ON")
            return false
        end
        if not canFling(target) then
            if not (type(options) == "table" and options.silent == true) then
                notifyUser("No living, valid fling target is available.")
                setStatus("No target")
            end
            return false
        end
        local perform: any = state.mm2PerformFling
        if type(perform) ~= "function" then
            setStatus("Fling engine unavailable")
            return false
        end
        local call: {[string]: any} = {
            power = settings.power,
            retries = settings.retries,
            duration = FLING_DURATION,
        }
        if type(options) == "table" then
            for key: string, value: any in pairs(options) do
                call[key] = value
            end
        end
        if call.power == nil then call.power = settings.power end
        if call.retries == nil then call.retries = settings.retries end
        setStatus("Flinging " .. tostring(target.Name))
        local started: boolean = false
        local ok: boolean, result: any = pcall(perform, target, call)
        if ok then
            started = result == true
        end
        if not started then
            setStatus("Fling busy or refused")
        end
        return started
    end

    local function flingSelected(label: string?): boolean
        local target: any = nil
        if label and label ~= "" then
            for _, entry: any in ipairs(rosterEntries(false)) do
                if entry.label == label then
                    target = entry.player
                    break
                end
            end
        end
        if not target then
            target = if canFling(selectedPlayer) then selectedPlayer else nil
        end
        if not target then
            setStatus("Pick a target")
            if label and label ~= "" then
                notifyUser("That player is not a living fling target any more.")
            end
            return false
        end
        return performFling(target)
    end

    local function flingAllInnocents(): ()
        if antiFlingOn() then
            notifyUser("Anti Fling is ON: turn it off before flinging someone.")
            setStatus("Anti Fling is ON")
            return
        end
        allInnocentsSequence += 1
        local sequence: number = allInnocentsSequence
        refreshRoles()
        task.spawn(function(): ()
            local flung: number = 0
            for _, player: any in ipairs(roster()) do
                if sequence ~= allInnocentsSequence then
                    break
                end
                if canFling(player) and roleOf(player) == "Innocent" then
                    if performFling(player, {
                        blocking = true,
                        silent = true,
                        duration = FLING_DURATION,
                    }) then
                        flung += 1
                    end
                    task.wait(0.05)
                end
            end
            if sequence == allInnocentsSequence then
                setStatus(flung > 0 and ("Flung " .. tostring(flung)) or nil)
            end
        end)
    end

    local function stopAutoFling(): ()
        settings.autoFling = false
        settings.autoFlingTarget = nil
        autoElapsed = 0
        disconnectFeatureConnection("MM2FlingAuto")
    end

    local function startAutoFling(): ()
        disconnectFeatureConnection("MM2FlingAuto")
        autoElapsed = 0
        featureConnections.MM2FlingAuto =
            TaskManager:Connect(function(deltaTime: number): ()
                autoElapsed += math.max(0, deltaTime)
                if autoElapsed < AUTO_FLING_INTERVAL then
                    return
                end
                autoElapsed = 0
                if not settings.autoFling or antiFlingOn() then
                    return
                end
                local target: any = settings.autoFlingTarget
                if not canFling(target) then
                    target = if canFling(selectedPlayer) then selectedPlayer else nil
                end
                if not target then
                    return
                end
                performFling(target, {silent = true, duration = FLING_DURATION})
            end, nil, "MM2 Fling Auto")
    end

    FlingFeature = createUniversalFeature(
        "Fling",
        "MM2 only: throws the murderer, the sheriff, every innocent or the player you pick, at maximum power.",
        9,
        function(): () end,
        {
            categoryName = "Exploits",
            parent = MM2Scroll,
            registry = mm2Features,
            -- Group card: the card itself never "runs", it holds the controls.
            kind = "group",
        }
    )

    -- The role buttons are the point of this card: one click, one role.
    if type(addActionOption) == "function" then
        addActionOption(FlingFeature, "Fling Murderer", function(): ()
            local target: any = resolveMurdererTarget()
            if target then
                performFling(target)
            else
                notifyUser(if rolesPublished()
                    then "No living murderer was found in this round."
                    else "MM2 has not published the roles yet. Wait for the round to start.")
            end
        end)
        addActionOption(FlingFeature, "Fling Sheriff", function(): ()
            local target: any = resolveSheriffTarget()
            if target then
                performFling(target)
            else
                notifyUser(if rolesPublished()
                    then "No living sheriff or hero was found in this round."
                    else "MM2 has not published the roles yet. Wait for the round to start.")
            end
        end)
        addActionOption(FlingFeature, "Fling All Innocents", function(): ()
            flingAllInnocents()
        end)
    end

    -- The roster is a live list of the players in this round, already labelled
    -- with their role, sorted murderer first.
    local listHandle: any = nil
    if type(addListOption) == "function" then
        listHandle = addListOption(FlingFeature, "Fling target", {
            items = function(): {string}
                local labels: {string} = {}
                for _, entry: any in ipairs(rosterEntries()) do
                    table.insert(labels, entry.label)
                end
                return labels
            end,
            single = true,
            default = {},
            emptyText = "No other players",
            hint = "Living players, ordered by role. Anti Fling must be OFF: it cancels the contact a fling needs.",
            onChanged = function(_selected: any, names: {string}): ()
                local label: string = names and names[1] or ""
                if label == "" then
                    -- The row is cleared right after a pick so the same player
                    -- can be chosen again; that empty publish must not forget
                    -- the target, or "Fling" would have nothing to repeat.
                    return
                end
                selectedLabel = label
                selectedPlayer = nil
                for _, entry: any in ipairs(rosterEntries(false)) do
                    if entry.label == label then
                        selectedPlayer = entry.player
                        settings.autoFlingTarget = entry.player
                        break
                    end
                end
                if not listReady then
                    -- The stored selection is published while the card is being
                    -- built: remember it, but never fling on injection.
                    return
                end
                if selectedPlayer then
                    local target: any = selectedPlayer
                    -- Clear the row so the same player can be picked again; the
                    -- empty publish that follows is ignored above.
                    local key: string = tostring(FlingFeature.configKey or "MM2.Fling")
                        .. ".flingtarget"
                    local controls: any = state.listControls and state.listControls[key]
                    if controls then
                        listReady = false
                        pcall(function(): ()
                            controls.set(label, false)
                        end)
                        listReady = true
                    end
                    performFling(target)
                end
            end,
        })
        listReady = true
    end

    if type(addActionOption) == "function" then
        addActionOption(FlingFeature, "Fling", function(): ()
            flingSelected(selectedLabel)
        end)
    end
    if type(addToggleOption) == "function" then
        addToggleOption(FlingFeature, "Auto Fling Selected Target", false,
            function(value: boolean): ()
                settings.autoFling = value == true
                if settings.autoFling then
                    if not canFling(selectedPlayer) then
                        settings.autoFling = false
                        notifyUser("Pick a living player in \"Fling target\" first.")
                        if state.setToggleChecked then
                            pcall(state.setToggleChecked,
                                tostring(FlingFeature.configKey or "MM2.Fling")
                                    .. ".AutoFlingSelectedTarget", false)
                        end
                        return
                    end
                    settings.autoFlingTarget = selectedPlayer
                    startAutoFling()
                else
                    stopAutoFling()
                end
            end)
    end

    if type(addNumberOption) == "function" then
        addNumberOption(FlingFeature, "Power", settings.power, 1, MAX_POWER,
            function(value: number): ()
                settings.power = math.clamp(math.floor(tonumber(value) or MAX_POWER), 1, MAX_POWER)
            end,
            "Fling strength. 12 is the maximum the engine accepts: four times YARHM's 9e7/9e8 pose velocity, which is what carries a fling through a laggy or fast server.",
            1)
        addNumberOption(FlingFeature, "Retries", settings.retries, 0, MAX_RETRIES,
            function(value: number): ()
                settings.retries = math.clamp(math.floor(tonumber(value) or MAX_RETRIES), 0, MAX_RETRIES)
            end,
            "Extra passes that swap the contact part when the server misses the first one. 3 is the maximum.",
            1)
    end
    if type(addInformationOption) == "function" then
        addInformationOption(
            FlingFeature,
            "MM2 only. Every button refuses while Anti Fling is ON: its collision "
                .. "shield cancels the contact a fling needs."
        )
    end

    activeCleanup = function(): ()
        allInnocentsSequence += 1
        stopAutoFling()
        state.mm2FlingFeature = nil
        state.mm2FlingSettings = nil
    end
    state.mm2FlingFeature = FlingFeature
    state.mm2FlingSettings = settings
    Module.Events = featureConnections
    Module.Initialized = true
    return Module
end

function Module.destroy(): ()
    if not Module.Initialized then
        return
    end
    Module.Initialized = false
    pcall(activeCleanup)
    activeCleanup = function(): () end
    Module.Events = {}
    Module.Runtime = nil
end

return Module

-------------------------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/MM2/Other/Fling.lua
-------------------------------------------------------------------------------------------------

-------------------------------------------------------------------------------------------------
ARCHIVO: src/games/MM2/Other/FlingEngine.lua
BYTES: 33537
SHA-256: 037ae31ea01b0176275903a9b77479d3ad49f23eb5af0310b4b9ca61152a4809
INICIO DEL ARCHIVO
-------------------------------------------------------------------------------------------------
local Module = {
    Name = "MM2 Fling Engine",
    Events = {} :: {[string]: any},
    Initialized = false,
}

local activeCleanup: (() -> ())? = nil

-- Internal MM2 engine only. Auto Farm and Auto Win keep their explicit fling
-- actions, while no Fling card or target action is installed in Universal.
function Module.init(runtime: any): any
    if Module.Initialized then return Module end
    local host: any = state.frameworkEnvironment
    local services: any = runtime and runtime.Services
    local Players: any = host.Players
    local LocalPlayer: any = host.LocalPlayer
    local getCharacterParts: any = host.getCharacterParts
    local notify: any = host.notify
    local activity: any = services and services.activity
    local protectedTargets: any = services and services.protectedTargets
    local configData: any = host.configData

    local workspace: any = host.workspace

    local function protectedCheck(player: Player?): boolean
        if not protectedTargets or type(protectedTargets.isProtected) ~= "function" then
```

```
            return false
        end
        local ok: boolean, result: any = pcall(protectedTargets.isProtected, player)
        return ok and result == true
    end

    -- v190: the MM2 Fling card is back (src/games/MM2/Other/Fling.lua), so the
    -- MM2.Fling* keys are no longer purged. The universal Fling module is still
    -- gone, and an "on" state saved by it must not be restored anywhere.
    if configData and type(configData.states) == "table" then
        configData.states["Universal.Fling"] = false
        configData.states["Universal.Fling.AutoFlingSelectedTarget"] = nil
        configData.states["MM2.Fling.AutoFlingSelectedTarget"] = nil
    end

    -- SkidFling (zqyDSUWX / YARHM), the fling every working menu copies:
    --
    --   RootPart.CFrame       = CFrame.new(targetPart.Position) * offset * spin
    --   RootPart.Velocity     = Vector3.new(9e7, 9e7 * 10, 9e7)
    --   RootPart.RotVelocity = Vector3.new(9e8, 9e8, 9e8)
    --
    -- The local character is parked on the victim and given an absurd velocity
    -- while a finite-force BodyVelocity reinforces contact. The large impulse is
    -- applied briefly; the local body and other temporary state are restored
    -- afterwards. Actual transfer still depends on the server's physics.
    -- SkidFling adapted from the YARHM hub the user supplied
    -- (YARHM_Hub_KEYLESS_2026.txt, "Fling" -> SkidFling, originally by zqyDSUWX).
    -- MM2 callers supply a target; the engine keeps its power, prediction and retries.
    --
    -- The parts that matter and that our own port had wrong:
    --
    --   * The BodyVelocity named "EpixVel" drives at Vector3.new(9e8, 9e8, 9e8)
    --     with finite 9e9 MaxForce. It is a catapult, not a pin: the local body is
    --     thrown into the victim. With Velocity zero (our old version) nothing is
    --     transferred, the character just sits there and the camera flickers.
    --   * The pose is written through RootPart.CFrame *and*
    --     Character:SetPrimaryPartCFrame, then RootPart.Velocity/RotVelocity.
    --     No AssemblyLinearVelocity, no CustomPhysicalProperties: writing those
    --     every frame is what made our character look like it was fading.
    --   * Each pose is separated by task.wait() (Heartbeat, after physics), not
    --     RenderStepped.
    --   * Two pose sets: six poses while the victim is still (< 50 studs/s) and
    --     nine while it is already moving.
    -- Power 3 reproduces YARHM's numbers exactly (9e7 / 9e8); anything above
    -- scales BOTH textures of the trick (the pose velocity and the EpixVel pin),
    -- which is what makes the fling survive a laggy or fast-moving victim.
    local FLING_VELOCITY_BASE: Vector3 = Vector3.new(9e7, 9e7 * 10, 9e7)
    local FLING_ROT_VELOCITY_BASE: Vector3 = Vector3.new(9e8, 9e8, 9e8)
    local PIN_VELOCITY_BASE: Vector3 = Vector3.new(9e8, 9e8, 9e8)
    -- v180: finite force, like YARHM and SkidFling use. `math.huge` makes the
    -- solver fight itself on some servers (the pin "wins" every contact and the
    -- victim never receives the impulse), which is exactly the low success
    -- rate the former card was reporting.
    local PIN_FORCE: Vector3 = Vector3.new(9e9, 9e9, 9e9)

    -- v180's default 6 doubles the pose velocity against YARHM's baseline.
    -- The universal card/slider are gone; MM2 retains this fixed default.
    local FLING_POWER: number = 6
    local FLING_DURATION: number = 3
    local FLING_PREDICTION: number = 0.15
    local flingRetries: number = 2
    -- Temporary boost used by the second pass only (never persisted).
    local flingBoost: number = 1

    local function powerScale(): number
        return (FLING_POWER / 3) * flingBoost
    end

    local function scaledVelocity(base: Vector3): Vector3
        local scale: number = powerScale()
        return Vector3.new(base.X * scale, base.Y * scale, base.Z * scale)
    end

    local flingRunning: boolean = false
    local engineActive: boolean = true
    local fullFlingPin: any = nil
    local stepPhase: number = 0
    local stepAngle: number = 0
    local lastOldPos: CFrame? = nil
    local stepPin: any = nil
    local stepPinToken: number = 0

    local function setFlingRunning(running: boolean): ()
        flingRunning = running
        if activity and type(activity.set) == "function" then
            activity.set("fling", running)
        end
    end

    local function targetParts(targetPlayer: Player?): (Model?, Humanoid?, BasePart?, BasePart?, BasePart?)
        local targetCharacter: Model? = targetPlayer and targetPlayer.Character
        if not targetCharacter then
            return nil, nil, nil, nil, nil
        end
        local targetHumanoid: Humanoid? = targetCharacter:FindFirstChildOfClass("Humanoid")
        local targetRoot: BasePart? = targetHumanoid and targetHumanoid.RootPart or nil
        local targetHead: BasePart? = targetCharacter:FindFirstChild("Head") :: BasePart?
        return targetCharacter, targetHumanoid, targetRoot, targetHead, nil
    end

    -- FPos: the whole trick. Position on the victim part, then hand the assembly
    -- 9e7 and 9e8 so the contact solver launches the victim.
    local function fpos(
        character: Model,
        root: BasePart,
        basePart: BasePart,
        offset: CFrame,
        angles: CFrame,
        lead: Vector3?
    ): ()
        local origin: Vector3 = basePart.Position
        if lead then
            origin = origin + lead
        end
        local targetCFrame: CFrame = CFrame.new(origin) * offset * angles
        root.CFrame = targetCFrame
        -- The pose goes through the classic body movers only. Writing
        -- AssemblyLinearVelocity / AssemblyAngularVelocity every frame is the
        -- regression this engine already fixed twice: the modern solver reacts to
        -- the assembly write on the *local* body and the character reads as
        -- fading from the owner's camera. RootPart.CFrame plus
        -- Velocity/RotVelocity is the pair YARHM ships and the pair the suite
        -- asserts on (tools/test_fling_skid.py: assemblyWrites must stay 0).
        pcall(function(): ()
            character:SetPrimaryPartCFrame(targetCFrame)
        end)
        local vel: Vector3 = scaledVelocity(FLING_VELOCITY_BASE)
        local rot: Vector3 = scaledVelocity(FLING_ROT_VELOCITY_BASE)
        -- Humanoid Physics like WalkFling helps replication
        local hum: Humanoid? = character:FindFirstChildOfClass("Humanoid")
        if hum and hum.ChangeState then
            pcall(function(): ()
                hum:ChangeState(Enum.HumanoidStateType.Physics)
            end)
        end
        root.Velocity = vel
        root.RotVelocity = rot
        -- Extra noclip for the fling contact (like WalkFling)
        pcall(function(): ()
            for _, part: Instance in ipairs(character:GetChildren()) do
                if part:IsA("BasePart") then
                    (part :: BasePart).CanCollide = false
                end
            end
        end)
    end

    -- Round trip / 2: how stale the server's copy of the victim is by the time it
    -- applies the pose we just wrote. With 300 ms of ping that is 150 ms of
    -- movement we have to lead with our own.
    local function transportSeconds(): number
        local ping: any = 0
        pcall(function(): ()
            local player: any = LocalPlayer
            if player and type(player.GetNetworkPing) == "function" then
                ping = player:GetNetworkPing()
            end
        end)
        if type(ping) ~= "number" or ping ~= ping or ping < 0 then
            ping = 0
        end
        -- GetNetworkPing returns seconds in current builds; older ones returned
        -- milliseconds. Anything above 2 is milliseconds.
        if ping > 2 then
            ping = ping / 1000
        end
        return math.clamp(ping * 0.5, 0, 0.5)
    end

    -- Where the victim will be when the server applies this pose: planar lead
    -- from its own velocity. Vertical lead is left out on purpose, it is the
    -- floor contact that carries the fling.
    local function leadFor(targetPart: BasePart?): Vector3?
        if not targetPart then
            return nil
        end
        local velocity: any = Vector3.zero
        pcall(function(): ()
            velocity = targetPart.AssemblyLinearVelocity
        end)
        if type(velocity) ~= "userdata" and type(velocity) ~= "table" then
            return nil
        end
        local seconds: number = math.clamp(
            transportSeconds() + FLING_PREDICTION,
            0,
            0.6
        )
        if seconds <= 0.001 then
            return nil
        end
        local lead: any = Vector3.new(velocity.X, 0, velocity.Z) * seconds
        return lead :: Vector3
    end

    -- The victim's own parts can be replaced (respawn, ragdoll, tool swap) while
    -- the fling runs; re-read them every pose instead of caching the first pair.
    local function liveTargetParts(
        targetPlayer: Player,
        targetCharacter: Model
    ): (Model?, Humanoid?, BasePart?, BasePart?)
        local character: Model? = targetPlayer.Character or targetCharacter
        if not character or not character.Parent then
            return nil, nil, nil, nil
        end
        local humanoid: Humanoid? = character:FindFirstChildOfClass("Humanoid")
        local root: BasePart? = humanoid and humanoid.RootPart or nil
        local head: BasePart? = character:FindFirstChild("Head") :: BasePart?
        return character, humanoid, root, head
    end

    local function partSpeed(part: BasePart?): number
        return if part then part.Velocity.Magnitude else 0
    end

    -- cloneReference() gives the host a DIFFERENT reference for the same
    -- service, so "targetPlayer.Parent ~= Players" can stop the fling early.
    -- Resolve the player through the service instead.
    local function playerStillHere(player: Player): boolean
        return Players:FindFirstChild(player.Name) ~= nil
    end

    -- SFBasePart: YARHM's choreography (six poses while the victim is slow, ten
    -- while it is already moving), with the menu's adaptations:
    --
    --   * MIN_POSES before the "it is flying, stop" check, so one stale
    --     replication frame does not end the fling early;
    --   * the victim's parts are re-read on every pose (respawn, ragdoll);
    --   * each pose is led by the victim's own velocity times half the ping, so a
    --     300 ms connection still lands the contact.
    local MIN_POSES: number = 12

    local function sfBasePart(
        character: Model,
        humanoid: Humanoid,
        root: BasePart,
        basePart: BasePart,
        targetCharacter: Model,
        targetPlayer: Player,
        targetHumanoid: Humanoid?,
        targetRoot: BasePart?,
        timeToWait: number,
        shouldStop: (() -> boolean)?,
        stats: any
    ): ()
        local startedAt: number = os.clock()
        local angle: number = 0
        local liveBase: BasePart = basePart

        local function currentBasePart(): BasePart?
            local characterNow: Model?, humanoidNow: Humanoid?, rootNow: BasePart?, headNow: BasePart? =
                liveTargetParts(targetPlayer, targetCharacter)
            if not characterNow then
                return nil
            end
            targetCharacter = characterNow
            targetHumanoid = humanoidNow or targetHumanoid
            targetRoot = rootNow or targetRoot
            local candidate: BasePart? = rootNow
            if rootNow and headNow then
                if (rootNow.Position - headNow.Position).Magnitude > 5 then
                    candidate = headNow
                end
            elseif headNow then
                candidate = headNow
            end
            if candidate and candidate.Parent then
                liveBase = candidate
            end
            return liveBase
        end

        local function pose(offset: CFrame, angles: CFrame): boolean
            -- Stop between individual yielded poses, not only once per group.
            if shouldStop and shouldStop() then return false end
            local part: BasePart? = currentBasePart()
            if not part then
                return false
            end
            fpos(character, root, part, offset, angles, leadFor(part))
            stats.poses += 1
            task.wait()
            return true
        end

        repeat
            if not root or not targetHumanoid then
                break
            end
            if shouldStop and shouldStop() then
                break
            end
            local part: BasePart? = currentBasePart()
            if not part then
                break
            end
            local speed: number = partSpeed(part)
            local moveDirection: Vector3 = Vector3.zero
            local walkSpeed: number = 16
            pcall(function(): ()
                moveDirection = targetHumanoid.MoveDirection
                walkSpeed = targetHumanoid.WalkSpeed
            end)
            if speed < 50 then
                angle = angle + 100
                local lead: Vector3 = moveDirection * (speed / 1.25)
                local spin: CFrame = CFrame.Angles(math.rad(angle), 0, 0)
                pose(CFrame.new(0, 1.5, 0) + lead, spin)
                pose(CFrame.new(0, -1.5, 0) + lead, spin)
                pose(CFrame.new(2.25, 1.5, -2.25) + lead, spin)
                pose(CFrame.new(-2.25, -1.5, 2.25) + lead, spin)
                pose(CFrame.new(0, 1.5, 0) + moveDirection, spin)
                pose(CFrame.new(0, -1.5, 0) + moveDirection, spin)
            else
                local downLead: number = partSpeed(targetRoot) / 1.25
                pose(CFrame.new(0, 1.5, walkSpeed), CFrame.Angles(math.rad(90), 0, 0))
                pose(CFrame.new(0, -1.5, -walkSpeed), CFrame.Angles(0, 0, 0))
                pose(CFrame.new(0, 1.5, walkSpeed), CFrame.Angles(math.rad(90), 0, 0))
                pose(CFrame.new(0, 1.5, downLead), CFrame.Angles(math.rad(90), 0, 0))
                pose(CFrame.new(0, -1.5, -downLead), CFrame.Angles(0, 0, 0))
                pose(CFrame.new(0, 1.5, downLead), CFrame.Angles(math.rad(90), 0, 0))
                pose(CFrame.new(0, -1.5, 0), CFrame.Angles(math.rad(90), 0, 0))
                pose(CFrame.new(0, -1.5, 0), CFrame.Angles(0, 0, 0))
                pose(CFrame.new(0, -1.5, 0), CFrame.Angles(math.rad(-90), 0, 0))
                pose(CFrame.new(0, -1.5, 0), CFrame.Angles(0, 0, 0))
            end
        until (stats.poses >= MIN_POSES and partSpeed(liveBase) > 500)
            or not liveBase.Parent
            or not targetPlayer.Character ~= targetCharacter
            or not playerStillHere(targetPlayer)
            or (targetHumanoid ~= nil and targetHumanoid.Sit)
            or humanoid.Health <= 0
            or os.clock() > startedAt + timeToWait
            or stats.poses >= 600
    end

    local function createPin(root: BasePart): any
        local pin: any = Instance.new("BodyVelocity")
        pin.Name = "EpixVel"
        pin.Velocity = scaledVelocity(PIN_VELOCITY_BASE)
        pin.MaxForce = PIN_FORCE
        pin.Parent = root
        return pin
    end

    local function destroyPin(pin: any): ()
        if pin then pin:Destroy() end
    end

    -- Per-frame caller (Auto Win's Fling Sheriff). Same FPos pose, with a pin
    -- that is cleaned up 0.5 s after the last call so the local body is never
    -- left with an infinite-force catapult attached.
    local function flingSingleStep(
        character: Model,
        humanoid: Humanoid,
        root: BasePart,
        targetPlayer: Player,
        _savedCollide: any,
        _savedProps: any
    ): boolean
        local targetCharacter: Model?, targetHumanoid: Humanoid?, targetRoot: BasePart?, targetHead: BasePart? =
            targetParts(targetPlayer)
        if not targetCharacter or not targetHead and not targetRoot then
            return false
        end
        if targetHumanoid and (targetHumanoid.Health <= 0 or targetHumanoid.Sit) then
            return false
        end
        if not targetHumanoid then
            return false
        end
        local basePart: BasePart = targetRoot or targetHead :: BasePart
        if targetRoot and targetHead then
            if (targetRoot.Position - targetHead.Position).Magnitude > 5 then
                basePart = targetHead
            end
        end
        if activity and type(activity.set) == "function" then
            activity.set("fling", true)
        end
        if not stepPin then
            stepPin = createPin(root)
        end
        stepPinToken += 1
        local myToken: number = stepPinToken
        task.delay(0.5, function(): ()
            if myToken == stepPinToken then
                destroyPin(stepPin)
                stepPin = nil
                if not flingRunning and activity and type(activity.set) == "function" then
                    activity.set("fling", false)
                end
            end
        end)
        stepAngle = (stepAngle + 100) % 360
        local speed: number = 0
        pcall(function(): ()
            speed = basePart.Velocity.Magnitude
        end)
        if speed < 50 then
            stepPhase = (stepPhase + 1) % 6
            local offsets: {CFrame} = {
                CFrame.new(0, 1.5, 0),
                CFrame.new(0, -1.5, 0),
                CFrame.new(2.25, 1.5, -2.25),
                CFrame.new(-2.25, -1.5, 2.25),
                CFrame.new(0, 1.5, 0),
                CFrame.new(0, -1.5, 0),
            }
            fpos(character, root, basePart, offsets[stepPhase + 1], CFrame.Angles(math.rad(stepAngle), 0, 0))
        else
            fpos(character, root, basePart, CFrame.new(0, 1.5, 0), CFrame.Angles(math.rad(90), 0, 0))
        end
        return true
    end

    local function performFling(targetPlayer: Player?, options: {[string]: any}?): boolean
        if not Module.Initialized or not targetPlayer then
            return false
        end
        if protectedCheck(targetPlayer) then
            notify("player protected")
            return false
        end
        local silent: boolean = type(options) == "table" and options.silent == true
        local maxDuration: number = (type(options) == "table" and tonumber(options.duration))
            or FLING_DURATION
        local requestedStop: (() -> boolean)? = type(options) == "table"
            and type(options.shouldStop) == "function"
            and options.shouldStop
            or nil
        local function shouldStop(): boolean
            return not engineActive or (requestedStop ~= nil and requestedStop())
        end
        -- Auto Farm opts out of returning to an old round after cancellation.
        -- Other callers (including Auto Win) retain their existing return policy.
        local skipReturnOnStop: boolean = type(options) == "table" and options.skipReturnOnStop == true
        local returnCFrame: CFrame? = type(options) == "table"
            and typeof(options.returnCFrame) == "CFrame"
            and options.returnCFrame
            or nil
        -- v190: the MM2 Fling card passes its Power/Retries rows through
        -- options instead of the engine keeping global state. Callers that send
        -- nothing (Auto Farm, Auto Win) keep the documented defaults.
        local requestedPower: number? = nil
        local requestedRetries: number? = nil
        if type(options) == "table" then
            local power: number? = tonumber(options.power)
            if power then requestedPower = math.clamp(power, 1, 12) end
            local retries: number? = tonumber(options.retries)
            if retries then requestedRetries = math.clamp(math.floor(retries), 0, 3) end
        end

        if flingRunning then
            if not silent then
                notify("fling already running")
            end
            return false
        end

        setFlingRunning(true)

        local stats: any = {poses = 0, speed = 0, pinned = false, retries = 0}

        local function runFling(): boolean
            local character: Model?, humanoid: Humanoid?, root: BasePart? = getCharacterParts()
            if not character or not humanoid or not root then
                setFlingRunning(false)
                if not silent then
                    notify("character not ready")
                end
                return false
            end

            local targetCharacter: Model?, targetHumanoid: Humanoid?, targetRoot: BasePart?, targetHead: BasePart? =
                targetParts(targetPlayer)
            if not targetCharacter or not targetCharacter:FindFirstChildWhichIsA("BasePart") then
                setFlingRunning(false)
                if not silent then
                    notify("No valid character of said target player. May have died.")
                end
                return false
            end
            if not targetHumanoid then
                setFlingRunning(false)
                if not silent then
                    notify("the target has no humanoid")
                end
                return false
            end

            if partSpeed(root) < 50 then
                lastOldPos = root.CFrame
            end
            local oldPos: CFrame = returnCFrame or lastOldPos or root.CFrame

            local savedDestroyHeight: any = nil
            pcall(function(): ()
                savedDestroyHeight = workspace.FallenPartsDestroyHeight
            end)
            local camera: Camera? = workspace.CurrentCamera

            local pin: any = nil
            local success: boolean, errorMessage: any = pcall(function(): ()
                if camera then
                    pcall(function(): ()
                        local accessory: any = targetCharacter:FindFirstChildOfClass("Accessory")
                        camera.CameraSubject = targetHead
                            or (accessory and accessory:FindFirstChild("Handle"))
                            or targetHumanoid
                    end)
                end

                -- FallenPartsDestroyHeight = 0/0 while the local body crosses the
                -- map at 9e8 studs/s; always restored below.
                pcall(function(): ()
                    workspace.FallenPartsDestroyHeight = 0 / 0
                end)

                pin = createPin(root)
                fullFlingPin = pin
                stats.pinned = true
                pcall(function(): ()
                    humanoid:SetStateEnabled(Enum.HumanoidStateType.Seated, false)
                end)

                local basePart: BasePart? = nil
                if targetRoot and targetHead then
                    if (targetRoot.Position - targetHead.Position).Magnitude > 5 then
                        basePart = targetHead
                    else
                        basePart = targetRoot
                    end
                elseif targetRoot then
                    basePart = targetRoot
                elseif targetHead then
                    basePart = targetHead
                end

                -- Retries can switch parts and temporarily boost force.
                local maxPasses: number = math.clamp(
                    math.floor(tonumber(flingRetries) or 0),
                    0,
                    3
                ) + 1
                local used: BasePart? = nil
                local pass: number = 0
                while pass < maxPasses do
                    if shouldStop and shouldStop() then break end
                    pass += 1
                    local partNow: BasePart? = nil
                    if pass == 1 then
                        partNow = basePart
                    else
                        local _c: Model?, _h: Humanoid?, rootNow: BasePart?, headNow: BasePart? =
                            liveTargetParts(targetPlayer, targetCharacter)
                        targetHumanoid = _h or targetHumanoid
                        targetRoot = rootNow or targetRoot
                        local primary: BasePart? = rootNow
                        if rootNow and headNow then
                            if (rootNow.Position - headNow.Position).Magnitude > 5 then
                                primary = headNow
                            end
                        elseif headNow then
                            primary = headNow
                        end
                        if primary == used then
                            partNow = if primary == rootNow then headNow else rootNow
                        else
                            partNow = primary
                        end
                        if partNow then
                            flingBoost = 1.5
                            stats.retries = (stats.retries or 0) + 1
                        end
                    end
                    if not partNow or not partNow.Parent then
                        flingBoost = 1
                        break
                    end
                    used = partNow
                    sfBasePart(
                        character,
                        humanoid,
                        root,
                        partNow,
                        targetCharacter,
                        targetPlayer,
                        targetHumanoid,
                        targetRoot,
                        -- Retries are half as long: they exist to catch a late
                        -- server contact, not to double the time spent posing.
                        if pass > 1 then maxDuration * 0.5 else maxDuration,
                        shouldStop,
                        stats
                    )
                    flingBoost = 1
                    stats.speed = math.max(
                        stats.speed or 0,
                        partSpeed(targetRoot),
                        partSpeed(targetHead)
                    )
                    if stats.speed > 100 then
                        break
                    end
                end
                stats.speed = math.max(
                    stats.speed or 0,
                    partSpeed(targetRoot),
                    partSpeed(targetHead)
                )
            end)

            -- The pin and temporary world state must be restored even if a pose
            -- failed. No secondary impulse is applied to the target.
            flingBoost = 1
            destroyPin(pin)
            if fullFlingPin == pin then fullFlingPin = nil end
            pcall(function(): ()
                humanoid:SetStateEnabled(Enum.HumanoidStateType.Seated, true)
            end)
            pcall(function(): ()
                workspace.FallenPartsDestroyHeight = savedDestroyHeight
            end)
            local finalCharacter: Model?, finalHumanoid: Humanoid?, finalRoot: BasePart? =
                getCharacterParts()
            local cancelled: boolean = not engineActive or (skipReturnOnStop and shouldStop())
            if camera then
                pcall(function(): ()
                    camera.CameraSubject = if cancelled then (finalHumanoid or humanoid) else humanoid
                end)
            end

            local function stopBodyMotion(body: Model): ()
                for _, descendant: Instance in ipairs(body:GetChildren()) do
                    if descendant:IsA("BasePart") then
                        local bodyPart: BasePart = descendant :: BasePart
                        bodyPart.Velocity = Vector3.zero
                        bodyPart.RotVelocity = Vector3.zero
                    end
                end
            end
            if cancelled then
                -- A server teleport/new round may already have moved the body.
                -- Release the old fling velocity, but never move a new body or
                -- return the lobby body to the previous map's saved CFrame.
                pcall(stopBodyMotion, character)
            elseif finalCharacter and finalRoot and finalRoot.Parent then
                local attempts: number = 0
                repeat
                    if not engineActive or (skipReturnOnStop and shouldStop()) then
                        pcall(stopBodyMotion, character)
                        break
                    end
                    attempts += 1
                    pcall(function(): ()
                        finalRoot.CFrame = oldPos * CFrame.new(0, 0.5, 0)
                        finalCharacter:SetPrimaryPartCFrame(oldPos * CFrame.new(0, 0.5, 0))
                        if finalHumanoid then
                            pcall(function(): ()
                                finalHumanoid:ChangeState(Enum.HumanoidStateType.GettingUp)
                            end)
                        end
                        stopBodyMotion(finalCharacter)
```

```
                end)
                task.wait()
            until (finalRoot.Position - oldPos.Position).Magnitude < 25 or attempts >= 60
        end

        setFlingRunning(false)
        local touchedTarget: boolean = stats.speed > 100
        pcall(function(): ()
            state.mm2LastFling = {
                target = tostring(targetPlayer.Name),
                poses = stats.poses,
                speed = stats.speed,
                pinned = stats.pinned,
                retries = stats.retries,
                power = powerScale(),
                ok = success,
            }
        end)
        if not Module.Initialized then
            return touchedTarget
        end
        if not silent then
            local speedText: string = tostring(math.floor(stats.speed))
            if not success then
                notify("fling failed: " .. tostring(errorMessage))
            elseif touchedTarget then
                notify(string.format(
                    "fling: %s sent flying (%s studs/s, %d poses%s)",
                    tostring(targetPlayer.Name),
                    speedText,
                    stats.poses,
                    (stats.retries or 0) > 0
                        and string.format(", %d retries", stats.retries)
                        or ""
                ))
            else
                notify(string.format(
                    "fling: %s did not move (%d poses, %s studs/s)",
                    tostring(targetPlayer.Name),
                    stats.poses,
                    speedText
                ))
            end
        end
        return touchedTarget
    end

    -- Any error must reset flingRunning, otherwise every later fling answers
    -- "fling already running" for the rest of the session.
    local function guardedRun(): boolean
        local savedPower: number = FLING_POWER
        local savedRetries: number = flingRetries
        if requestedPower then FLING_POWER = requestedPower end
        if requestedRetries then flingRetries = requestedRetries end
        local ok: boolean, result: any = pcall(runFling)
        -- Restore before anything else: the engine is a single-fling
        -- machine and a later caller must not inherit this card's rows.
        FLING_POWER, flingRetries = savedPower, savedRetries
        if not ok then
            setFlingRunning(false)
            if not silent then
                notify("fling error: " .. tostring(result))
            end
            return false
        end
        return result == true
    end

    if type(options) == "table" and options.blocking == true then
        return guardedRun()
    end
    task.spawn(guardedRun)
    return true
end

state.mm2PerformFling = performFling
state.mm2FlingStep = flingSingleStep
activeCleanup = function(): ()
    engineActive = false
    destroyPin(fullFlingPin)
    fullFlingPin = nil
    flingRunning = false
    stepPinToken += 1
    destroyPin(stepPin)
    stepPin = nil
    setFlingRunning(false)
    state.mm2PerformFling = nil
    state.mm2FlingStep = nil
end
Module.Initialized = true
return Module
end

function Module.destroy(): ()
    if activeCleanup then pcall(activeCleanup) end
    activeCleanup = nil
    Module.Initialized = false
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/MM2/Other/FlingEngine.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/games/MM2/Other/GetAllEmotes.lua
BYTES: 19243
SHA-256: 1c84599236d13c1a5982f65eb7c120f8d0f70f1647ca8e97c009af726ba9b55a
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
local Module = {
    Name = "MM2 Get All Emotes",
    Events = {} :: {[string]: any},
    Initialized = false,
    Runtime = nil :: any,
}

local activeCleanup: () -> () = function(): () end

function Module.init(runtime: any): any
    if Module.Initialized then
        return Module
    end
    local core: any = state.mm2Core
    assert(type(core) == "table", "MM2 Get All Emotes requires the MM2 core module")
    Module.Runtime = runtime

    type EmoteDef = {
        id: string,
        label: string,
        command: string,
        animationId: string,
        looped: boolean?,
    }

    -- The five shop animations plus the two event animations the player asked
    -- to keep. These are a fallback from the captured MM2 Database.Sync.Emotes;
    -- the live catalog can contribute other playable non-default emotes too.
    local EMOTES: {EmoteDef} = {
        { id = "sit", label = "Sit", command = "sit2", animationId = "rbxassetid://2431845940", looped = true },
        { id = "ninja", label = "Ninja Rest", command = "ninja", animationId = "rbxassetid://2431864798", looped = true },
        { id = "zen", label = "Zen", command = "zen", animationId = "rbxassetid://2431812646", looped = true },
        { id = "dab", label = "Dab", command = "dab", animationId = "rbxassetid://2445521505" },
        { id = "floss", label = "Floss", command = "floss", animationId = "rbxassetid://2452938820", looped = true },
        { id = "zombie", label = "Zombie", command = "zombie", animationId = "rbxassetid://2513692312", looped = true },
        { id = "headless", label = "Headless", command = "headless", animationId = "rbxassetid://2513664073" },
    }
    local LABELS: {string} = {}
    local ID_BY_LABEL: {[string]: string} = {}
    local KNOWN_IDS: {[string]: boolean} = {}
    local DEFAULT_IDS: {[string]: boolean} = {
        back = true, wave = true, cheer = true, laugh = true, point = true,
        dance = true, dance1 = true, dance2 = true, dance3 = true,
    }
    for _, def: EmoteDef in ipairs(EMOTES) do
        table.insert(LABELS, def.label)
        ID_BY_LABEL[def.label] = def.id
        KNOWN_IDS[def.id] = true
    end

    local selectedLabel: string = "Floss"
    local selectionTouched: boolean = false
    local activeTrack: any = nil
    local feature: any = nil
    local generation: number = 0
    local grantInFlight: boolean = false
    local catalogInFlight: boolean = false
    local activated: boolean = false

    local function stopActiveTrack(): ()
        if activeTrack then
            pcall(function(): () activeTrack:Stop(0.1) end)
            activeTrack = nil
        end
    end

    local function animationUri(value: any): string?
        if type(value) == "number" and value > 0 and value < math.huge then
            return "rbxassetid://" .. string.format("%.0f", value)
        end
        if type(value) == "string" and value ~= "" then
            return string.match(value, "^%d+$") and ("rbxassetid://" .. value) or value
        end
        return nil
    end

    -- New MM2 animations should not be silently excluded just because the
    -- captured catalog is older. Admit only playable catalog entries, never
    -- the known Roblox defaults or the non-emote Back row. This runs in the
    -- asynchronous grant path, since require may wait for game data.
    local function discoverLiveEmotes(ticket: number): ()
        local rs: Instance = game:GetService("ReplicatedStorage")
        local db: Instance? = rs:FindFirstChild("Database")
        local sync: Instance? = db and db:FindFirstChild("Sync")
        local emotesModule: Instance? = sync and sync:FindFirstChild("Emotes")
        if not emotesModule then return end
        local ok: boolean, catalog: any = pcall(require, emotesModule)
        if ticket ~= generation or not Module.Initialized
            or not ok or type(catalog) ~= "table" then
            return
        end
        local extra: {string} = {}
        for id: any, live: any in pairs(catalog) do
            if type(id) == "string" and not KNOWN_IDS[id]
                and not DEFAULT_IDS[string.lower(id)] and type(live) == "table"
                and type(live.Command) == "string" and live.Command ~= ""
                and not DEFAULT_IDS[string.lower(live.Command)]
                and animationUri(live.AnimationID) then
                table.insert(extra, id)
            end
        end
        table.sort(extra)
        for _, id: string in ipairs(extra) do
            local live: any = catalog[id]
            local label: string = type(live.Name) == "string" and live.Name ~= "" and live.Name or id
            if ID_BY_LABEL[label] then label ..= " (" .. id .. ")" end
            local def: EmoteDef = {
                id = id,
                label = label,
                command = live.Command,
                animationId = animationUri(live.AnimationID) :: string,
                looped = live.Loop == true,
            }
            table.insert(EMOTES, def)
            table.insert(LABELS, label)
            ID_BY_LABEL[label] = id
            KNOWN_IDS[id] = true
        end
        -- A saved selection for a newly discovered emote was not available
        -- when the cycle was built. Restore it silently unless the user has
        -- already chosen another value during this session.
        local values: any = type(configData) == "table" and configData.values or nil
        local saved: any = type(values) == "table" and values["MM2.GetAllEmotes.SelectEmote"] or nil
        local index: number? = type(saved) == "string" and table.find(LABELS, saved) or nil
        if not selectionTouched and index and ID_BY_LABEL[saved] then
            selectedLabel = saved
            local control: any = state.valueControls and state.valueControls["MM2.GetAllEmotes.SelectEmote"]
            if control and type(control.setIndex) == "function" then
                control.setIndex(index)
            end
        end
    end

    local function emoteDefinition(id: string): EmoteDef?
        local result: EmoteDef? = nil
        for _, def: EmoteDef in ipairs(EMOTES) do
            if def.id == id then
                result = table.clone(def)
                break
            end
        end
        if not result then
            return nil
        end
        pcall(function(): ()
            local rs: Instance = game:GetService("ReplicatedStorage")
            local db: Instance? = rs:FindFirstChild("Database")
            local sync: Instance? = db and db:FindFirstChild("Sync")
            local emotesModule: Instance? = sync and sync:FindFirstChild("Emotes")
            if not emotesModule then return end
            local ok: boolean, catalog: any = pcall(require, emotesModule)
            local live: any = ok and type(catalog) == "table" and catalog[id] or nil
            if type(live) ~= "table" then return end
            local def: EmoteDef = result :: EmoteDef
            if type(live.Command) == "string" and live.Command ~= "" then
                def.command = live.Command
            end
            local asset: string? = animationUri(live.AnimationID)
            if asset then def.animationId = asset end
            def.looped = live.Loop == true
        end)
        return result
    end

    -- The old menu has a separate GenerateEmotes method. The current
    -- CrossPlatform GUI does not: it listens to InventoryDataChanged instead.
    -- Never require the old EmoteModule without its controller/GUI: that module
    -- waits indefinitely for _G.EmoteController in the newer interface.
    local function refreshLegacyGui(rs: Instance, owned: {string}): boolean
        local globals: any = _G :: any
        local gui: any = globals and globals.EmoteFrame
        if not gui or type(globals.EmoteController) ~= "table" then return false end
        local valid: boolean, pages: any = pcall(function(): any
            return gui:FindFirstChild("EmotePages")
        end)
        if not valid or not pages then return false end
        local modules: Instance? = rs:FindFirstChild("Modules")
        local scriptModule: Instance? = modules and modules:FindFirstChild("EmoteModule")
        if not scriptModule then return false end
        local ok: boolean, emoteModule: any = pcall(require, scriptModule)
        if not ok or type(emoteModule) ~= "table"
            or type(emoteModule.GenerateEmotes) ~= "function" then
            return false
        end
        emoteModule.EmoteGUI = gui
        return pcall(emoteModule.GenerateEmotes, owned, gui)
    end

    -- Modifying ProfileData is client-side only. The native BindableEvent must
    -- be fired on the instance itself: .Event is an RBXScriptSignal and has no
    -- Fire method. Both old and CrossPlatform controllers listen to .Event.
    local function grantShopEmotes(ticket: number): (boolean, number, boolean)
        if ticket ~= generation or not Module.Initialized then return false, 0, false end
        local ok: boolean, ready: any, added: any, refreshed: any = pcall(function(): (boolean, number, boolean)
            local rs: Instance = game:GetService("ReplicatedStorage")
            local modules: Instance? = rs:FindFirstChild("Modules")
            local profileModule: Instance? = modules and modules:FindFirstChild("ProfileData")
            if not profileModule then return false, 0, false end
            local profileOk: boolean, profile: any = pcall(require, profileModule)
            if ticket ~= generation or not Module.Initialized then return false, 0, false end
            if not profileOk or type(profile) ~= "table"
                or type(profile.Emotes) ~= "table"
                or type(profile.Emotes.Owned) ~= "table" then
                return false, 0, false
            end
            local owned: {string} = profile.Emotes.Owned
            local known: {[string]: boolean} = {}
            for _, existing: any in ipairs(owned) do
                if type(existing) == "string" then known[existing] = true end
            end
            local newIds: {string} = {}
            for _, def: EmoteDef in ipairs(EMOTES) do
                if not known[def.id] then
                    table.insert(owned, def.id)
                    known[def.id] = true
                    table.insert(newIds, def.id)
                end
            end

            local remotes: Instance? = rs:FindFirstChild("Remotes")
            local inventory: Instance? = remotes and remotes:FindFirstChild("Inventory")
            local changed: any = inventory and inventory:FindFirstChild("InventoryDataChanged")
            local signalled: boolean = false
            if changed then
                -- Incremental inventory listeners need each new ID. An
                -- idempotent repeat still wakes the emote window once.
                local wakeIds: {string} = #newIds > 0 and newIds or { EMOTES[1].id }
                for _, id: string in ipairs(wakeIds) do
                    local fired: boolean = pcall(function(): ()
                        changed:Fire("Emotes", id, 1)
                    end)
                    signalled = signalled or fired
                end
            end
            if not signalled then
                signalled = refreshLegacyGui(rs, owned)
            end
            return true, #newIds, signalled
        end)
        if not ok then return false, 0, false end
        return ready == true, tonumber(added) or 0, refreshed == true
    end

    local function requestGrant(showNotification: boolean): ()
        if grantInFlight then
            if showNotification and type(notify) == "function" then
                pcall(notify, "Get All Emotes: waiting for the MM2 profile.")
            end
            return
        end
        grantInFlight = true
        local ticket: number = generation
        -- ProfileData's require can yield while the game loads; keep the menu
        -- responsive and ignore the result if this instance was destroyed.
        task.spawn(function(): ()
            local ready: boolean, added: number, refreshed: boolean = grantShopEmotes(ticket)
            if ticket ~= generation or not Module.Initialized then return end
            grantInFlight = false
            if not ready then
                if feature then feature:SetStatus("profile not ready") end
                if showNotification and type(notify) == "function" then
                    pcall(notify, "Get All Emotes: MM2 profile unavailable or not ready; try again after the game loads.")
                end
                return
            end
            local count: number = #EMOTES
            if feature then
                feature:SetStatus(refreshed and (tostring(count) .. " local emotes")
                    or (tostring(count) .. " local · UI not refreshed"))
            end
            if showNotification and type(notify) == "function" then
                local suffix: string = refreshed
                    and "The emote UI was notified."
                    or "The MM2 emote UI could not be notified; try again after it loads."
                pcall(notify, string.format(
                    "Get All Emotes: %d local entries (%d new). %s Server ownership is not verified.",
                    count, added, suffix
                ))
            end
            -- Do not let a slow/hung catalog require block the seven known
            -- entries. If it finds extras later, add and signal them in a
            -- second pass; at most one discovery may be waiting at a time.
            if not catalogInFlight then
                catalogInFlight = true
                task.spawn(function(): ()
                    if ticket ~= generation or not Module.Initialized then
                        catalogInFlight = false
                        return
                    end
                    local previous: number = #EMOTES
                    pcall(discoverLiveEmotes, ticket)
                    catalogInFlight = false
                    if ticket == generation and Module.Initialized and #EMOTES > previous then
                        requestGrant(false)
                    end
                end)
            end
        end)
    end

    local function activate(): ()
        activated = true
        if featureConnections.MM2GetAllEmotesRespawn or not (LocalPlayer and LocalPlayer.CharacterAdded) then
            return
        end
        featureConnections.MM2GetAllEmotesRespawn = LocalPlayer.CharacterAdded:Connect(function(): ()
            local ticket: number = generation
            task.delay(0.5, function(): ()
                if ticket == generation and Module.Initialized and activated then
                    requestGrant(false)
                end
            end)
        end)
    end

    -- A successful :Fire is just a request, not confirmation of playback or
    -- replication. Fall back to the local Animator only when no native event
    -- could be fired; doing both would play two tracks at once.
    local function requestEmotePlayback(id: string): boolean
        stopActiveTrack()
        local def: EmoteDef? = emoteDefinition(id)
        if not def then return false end
        local fired: boolean = false
        pcall(function(): ()
            local rs: Instance = game:GetService("ReplicatedStorage")
            local remotes: Instance? = rs:FindFirstChild("Remotes")
            local misc: Instance? = remotes and remotes:FindFirstChild("Misc")
            local event: any = misc and misc:FindFirstChild("PlayEmote")
            if event then
                fired = pcall(function(): () event:Fire(id) end)
            end
        end)
        if fired then return true end
        if def.animationId == "" then return false end

        local played: boolean = false
        pcall(function(): ()
            local character: Model? = LocalPlayer and LocalPlayer.Character
            local humanoid: Humanoid? = character and character:FindFirstChildOfClass("Humanoid")
            if not humanoid or humanoid.Health <= 0 then return end
            local anim: Animation = Instance.new("Animation")
            anim.Name = def.command
            anim.AnimationId = def.animationId
            local animator: Animator? = humanoid:FindFirstChildOfClass("Animator")
            local loaded: boolean, track: any = pcall(function(): any
                return if animator then animator:LoadAnimation(anim)
                    else (humanoid :: any):LoadAnimation(anim)
            end)
            if loaded and track then
                activeTrack = track
                played = pcall(function(): ()
                    track.Looped = def.looped == true
                    track:Play(0.1)
                end)
            end
            anim:Destroy()
        end)
        return played
    end

    local function playSelected(): ()
        activate()
        requestGrant(false)
        local id: string = ID_BY_LABEL[selectedLabel] or "floss"
        if not requestEmotePlayback(id) and type(notify) == "function" then
            pcall(notify, "Could not request that emote; check the character and animation permissions.")
        end
    end

    feature = createUniversalFeature(
        "Get All Emotes",
        "Adds MM2's playable non-default emotes locally, including Sit, Ninja Rest, Zen, Dab, Floss, Zombie and Headless. "
            .. "It does not grant server ownership; playback and replication depend on the game.",
        11,
        function(): ()
            activate()
            requestGrant(true)
        end,
        {
            categoryName = "Utilities",
            parent = MM2Scroll,
            registry = mm2Features,
            action = true,
            noOptions = false,
            configKey = "MM2.GetAllEmotes",
        }
    )
    state.mm2GetAllEmotesFeature = feature

    -- Restoring the saved selection at boot must not play/grant anything.
    local cycleRestored: boolean = false
    addCycleOption(feature, "Select Emote", LABELS, 5, function(value: string): ()
        selectedLabel = value
        if not cycleRestored then
            cycleRestored = true
            return
        end
        selectionTouched = true
        playSelected()
    end, "Select and request an MM2 emote; server ownership is not changed.")
    addActionOption(feature, "Play Emote", playSelected, "Request the selected emote again.")
    addInformationOption(feature,
        "MM2's playable emotes are added locally. Existing inventory is preserved; avatar defaults "
            .. "are not added, and the server controls ownership and replication.")

    activeCleanup = function(): ()
        generation += 1
        activated = false
        disconnectFeatureConnection("MM2GetAllEmotesRespawn")
        stopActiveTrack()
        if state.mm2GetAllEmotesFeature == feature then
            state.mm2GetAllEmotesFeature = nil
        end
    end
    Module.Events = featureConnections
    Module.Initialized = true
    return Module
end

function Module.destroy(): ()
    if not Module.Initialized then return end
    Module.Initialized = false
    pcall(activeCleanup)
    activeCleanup = function(): () end
    Module.Events = {}
    Module.Runtime = nil
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/MM2/Other/GetAllEmotes.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/games/MM2/Other/GunAura.lua
BYTES: 15709
SHA-256: 9a1cb6496be4c1aee5c0d06e556ec8bd5de1905107a1bb92ae06234c346c7b06
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
local Module = {
    Name = "MM2 Gun Aura",
    Events = {} :: {[string]: any},
    Initialized = false,
    Runtime = nil :: any,
}

local activeCleanup: () -> () = function(): () end

function Module.init(runtime: any): any
    if Module.Initialized then
        return Module
    end
    local core: any = state.mm2Core
    assert(type(core) == "table", "MM2 Gun Aura requires the MM2 core module")
    Module.Runtime = runtime

    local settings = {
        studs = 25,
        showHitbox = true,
    }

    local TICK_INTERVAL: number = 0.08
    local SCAN_INTERVAL: number = 0.4
    local ESP_INTERVAL: number = 0.5

    local gunAuraActive = false
    local gunHitboxes: {[Instance]: BasePart} = {}
    local espParts: {[Player]: BasePart} = {}
    local savedGunStates: {[BasePart]: {cframe: CFrame, canCollide: boolean}} = {}
    local trackedDrops: {[Instance]: boolean} = {}
    local lastScanAt: number = -math.huge
    local lastTickAt: number = -math.huge
    local lastEspAt: number = -math.huge

    local function resolveGunTouchPart(gun: Instance?): BasePart?
        if not gun then
            return nil
        end
        if gun:IsA("BasePart") then
            return gun :: BasePart
        end
        local handle: Instance? = gun:FindFirstChild("Handle")
            or gun:FindFirstChild("GunDrop")
            or gun:FindFirstChildWhichIsA("BasePart")
        if handle and handle:IsA("BasePart") then
            return handle :: BasePart
        end
        return nil
    end

    local function rememberGunPartState(part: BasePart): {cframe: CFrame, canCollide: boolean}
        local existing = savedGunStates[part]
        if existing then
            return existing
        end
        local snap = {cframe = part.CFrame, canCollide = part.CanCollide}
        savedGunStates[part] = snap
        return snap
    end

    local function restoreGunPartState(part: BasePart): ()
        local snap = savedGunStates[part]
        if not snap then
            return
        end
        if part and part.Parent then
            pcall(function(): ()
                part.CFrame = snap.cframe
                part.CanCollide = snap.canCollide
            end)
        end
        savedGunStates[part] = nil
    end

    local function restoreAllGunPartStates(): ()
        for part in pairs(savedGunStates) do
            restoreGunPartState(part)
        end
        table.clear(savedGunStates)
    end

    local function trueGunPartPosition(part: BasePart): Vector3
        local snap = savedGunStates[part]
        return if snap then snap.cframe.Position else part.Position
    end

    local function canGrabGun(): boolean
        local character = LocalPlayer and LocalPlayer.Character
        if not character then
            return false
        end

        local humanoid = character:FindFirstChildOfClass("Humanoid")
        if not humanoid or humanoid.Health <= 0 then
            return false
        end

        if type(core.getPlayerRole) == "function" then
            local role = core.getPlayerRole(LocalPlayer)
            if role == "Murderer" or role == "Sheriff" or role == "Hero" then
                return false
            end
        end

        local backpack = LocalPlayer:FindFirstChildOfClass("Backpack")
        if (backpack and (backpack:FindFirstChild("Gun") or backpack:FindFirstChild("Knife")))
            or character:FindFirstChild("Gun")
            or character:FindFirstChild("Knife") then
            return false
        end

        return true
```

```
    end

    local function isValidGunDrop(obj: Instance?): boolean
        if not obj or not obj.Parent then
            return false
        end
        if obj.Name == "GunDrop" and (obj:IsA("BasePart") or obj:IsA("Model")) then
            return resolveGunTouchPart(obj) ~= nil
        end
        return false
    end

    local function createGunHitbox(gun: Instance): BasePart?
        local handle = resolveGunTouchPart(gun)
        if not handle then
            return nil
        end
        rememberGunPartState(handle)
        local pos = trueGunPartPosition(handle)
        if not pos then
            return nil
        end

        local part = Instance.new("Part")
        part.Name = "WurstGunAuraHitbox"
        part.Size = Vector3.new(settings.studs, settings.studs, settings.studs)
        part.Transparency = 0.65
        part.BrickColor = BrickColor.new("White")
        part.Material = Enum.Material.Neon
        part.CanCollide = false
        part.CanQuery = false
        part.CanTouch = false
        part.Anchored = true
        part.CFrame = CFrame.new(pos)
        part.Parent = workspace

        return part
    end

    -- Lightweight GunDrop discovery: uses DescendantAdded events + top-level
    -- workspace children every 0.4s (never recursive GetDescendants).
    local function refreshTrackedDrops(now: number): ()
        if now - lastScanAt < SCAN_INTERVAL then
            return
        end
        lastScanAt = now

        local wsDrop: Instance? = workspace:FindFirstChild("GunDrop")
        if wsDrop and isValidGunDrop(wsDrop) then
            trackedDrops[wsDrop] = true
        end

        local okChildren: boolean, children: any = pcall(function(): any
            return workspace:GetChildren()
        end)
        if okChildren and type(children) == "table" then
            for _, child: Instance in ipairs(children) do
                if child.Name == "GunDrop" and isValidGunDrop(child) then
                    trackedDrops[child] = true
                elseif child:IsA("Model") or child:IsA("Folder") then
                    local gd: Instance? = child:FindFirstChild("GunDrop")
                    if gd and isValidGunDrop(gd) then
                        trackedDrops[gd] = true
                    end
                end
            end
        end

        if type(core.findDroppedGun) == "function" then
            local okCore: boolean, drop: any = pcall(core.findDroppedGun)
            if okCore and drop and isValidGunDrop(drop) then
                trackedDrops[drop] = true
            end
        end
    end

    local function isWithinAura(truePos: Vector3, rootPos: Vector3): boolean
        local diff = rootPos - truePos
        local radius = settings.studs
        if diff.X * diff.X + diff.Y * diff.Y + diff.Z * diff.Z <= radius * radius then
            return true
        end
        local half = radius * 0.5
        return math.abs(diff.X) <= half
            and math.abs(diff.Y) <= math.max(half, 8)
            and math.abs(diff.Z) <= half
    end

    local function tryGrabGun(root: BasePart, handle: BasePart): ()
        rememberGunPartState(handle)
        handle.CanCollide = false
        local rootCF: CFrame? = root.CFrame
        if rootCF then
            handle.CFrame = rootCF
        end
        local fireTouch: any = state.fireTouchInterest
        if type(fireTouch) == "function" then
            pcall(fireTouch, root, handle, 0)
            pcall(fireTouch, root, handle, 1)
        end
    end

    local function clearAllHitboxes(): ()
        for gun, hitbox in pairs(gunHitboxes) do
            if hitbox and hitbox.Parent then
                pcall(function(): ()
                    hitbox:Destroy()
                end)
            end
            gunHitboxes[gun] = nil
        end
    end

    local function clearAllEsp(): ()
        for player, esp in pairs(espParts) do
            if esp and esp.Parent then
                pcall(function(): ()
                    esp:Destroy()
                end)
            end
            espParts[player] = nil
        end
    end

    local function updateSheriffESP(player: Player): ()
        local character = player.Character
        local root = character and character:FindFirstChild("HumanoidRootPart")
        local humanoid = character and character:FindFirstChildOfClass("Humanoid")
        if not character or not root or not root:IsA("BasePart")
            or not humanoid or humanoid.Health > 0 then
            local existing = espParts[player]
            if existing then
                if existing.Parent then
                    pcall(function(): ()
                        existing:Destroy()
                    end)
                end
                espParts[player] = nil
            end
            return
        end

        local esp = espParts[player]
        if not esp or not esp.Parent then
            esp = Instance.new("Part")
            esp.Name = "WurstDeadSheriffESP_" .. player.Name
            esp.Size = Vector3.new(3, 3, 3)
            esp.Transparency = 0.3
            esp.BrickColor = BrickColor.new("White")
            esp.Material = Enum.Material.Neon
            esp.CanCollide = false
            esp.CanQuery = false
            esp.CanTouch = false
            esp.Anchored = true
            esp.Parent = workspace
            espParts[player] = esp
        end
        esp.CFrame = (root :: BasePart).CFrame
    end

    local GunAuraFeature: any = nil
    local lastStatus: string? = nil

    local function setStatus(text: string?): ()
        if text == lastStatus then
            return
        end
        lastStatus = text
        if GunAuraFeature then
            GunAuraFeature:SetStatus(text)
        end
    end

    local function gunAuraTick(): ()
        if not gunAuraActive then
            return
        end
        local now: number = os.clock()
        if now - lastTickAt < TICK_INTERVAL then
            return
        end
        lastTickAt = now

        refreshTrackedDrops(now)

        local character = LocalPlayer and LocalPlayer.Character
        local root: BasePart? = character and character:FindFirstChild("HumanoidRootPart") :: BasePart?
        local canGrab = canGrabGun()
        local activeGuns: {[Instance]: boolean} = {}
        local touchedParts: {[BasePart]: boolean} = {}
        local gunCount: number = 0

        for gun in pairs(trackedDrops) do
            if isValidGunDrop(gun) then
                local handle: BasePart? = resolveGunTouchPart(gun)
                if handle then
                    gunCount += 1
                    activeGuns[gun] = true
                    rememberGunPartState(handle)
                    local truePos: Vector3? = trueGunPartPosition(handle)

                    if settings.showHitbox then
                        local hitbox = gunHitboxes[gun]
                        if not hitbox or not hitbox.Parent then
                            hitbox = createGunHitbox(gun)
                            if hitbox then
                                gunHitboxes[gun] = hitbox
                            end
                        elseif truePos then
                            local targetSize = Vector3.new(settings.studs, settings.studs, settings.studs)
                            if hitbox.Size ~= targetSize then
                                hitbox.Size = targetSize
                            end
                            hitbox.CFrame = CFrame.new(truePos)
                        end
                    end

                    if canGrab and root and truePos and isWithinAura(truePos, root.Position) then
                        touchedParts[handle] = true
                        pcall(tryGrabGun, root, handle)
                    end
                end
            else
                trackedDrops[gun] = nil
            end
        end

        for part in pairs(savedGunStates) do
            if not touchedParts[part] then
                restoreGunPartState(part)
            end
        end

        for gun, hitbox in pairs(gunHitboxes) do
            if not activeGuns[gun] then
                if hitbox and hitbox.Parent then
                    pcall(function(): ()
                        hitbox:Destroy()
                    end)
                end
                gunHitboxes[gun] = nil
            end
        end

        if gunCount > 0 then
            setStatus("Gun detected · " .. tostring(math.round(settings.studs)) .. " studs")
        else
            setStatus(tostring(math.round(settings.studs)) .. " studs")
        end

        if now - lastEspAt >= ESP_INTERVAL and type(core.getPlayerRole) == "function" then
            lastEspAt = now
            local seenSheriffs: {[Player]: boolean} = {}
            for _, player in ipairs(game:GetService("Players"):GetPlayers()) do
                if player ~= LocalPlayer then
                    local role = core.getPlayerRole(player)
                    if role == "Sheriff" then
                        seenSheriffs[player] = true
                        updateSheriffESP(player)
                    end
                end
            end
            for player, esp in pairs(espParts) do
                if not seenSheriffs[player] then
                    if esp and esp.Parent then
                        pcall(function(): ()
                            esp:Destroy()
                        end)
                    end
                    espParts[player] = nil
                end
            end
        end
    end

    local function toggleGunAura(enabled: boolean): ()
        gunAuraActive = enabled
        state.mm2GunAuraEnabled = enabled == true
        disconnectFeatureConnection("MM2GunAura")
        disconnectFeatureConnection("MM2GunAuraDescendant")

        if not enabled then
            clearAllHitboxes()
            clearAllEsp()
            restoreAllGunPartStates()
            table.clear(trackedDrops)
            setStatus(nil)
            return
        end

        lastScanAt = -math.huge
        lastTickAt = -math.huge
        lastEspAt = -math.huge

        if workspace.DescendantAdded then
            featureConnections.MM2GunAuraDescendant = workspace.DescendantAdded:Connect(function(obj: Instance): ()
                if obj.Name == "GunDrop" then
                    trackedDrops[obj] = true
                end
            end)
        end

        setStatus(tostring(math.round(settings.studs)) .. " studs")
        featureConnections.MM2GunAura =
            TaskManager:Connect(gunAuraTick, nil, "MM2 Gun Aura")
        gunAuraTick()
    end

    GunAuraFeature = createUniversalFeature(
        "Gun Aura",
        "Detects dropped guns in the map with event-driven tracking, draws a white aura box around them, and automatically grabs the gun within Studs range.",
        7,
        toggleGunAura,
        {
            categoryName = "Utilities",
            parent = MM2Scroll,
            registry = mm2Features,
        }
    )
    state.mm2GunAuraFeature = GunAuraFeature

    addNumberOption(GunAuraFeature, "Studs", 25, 5, 250,
        function(value: number): ()
            settings.studs = math.clamp(tonumber(value) or 25, 5, 250)
            if GunAuraFeature.enabled then
                setStatus(tostring(math.round(settings.studs)) .. " studs")
            end
        end,
        "Distance in studs around each dropped gun to automatically grab it.", 1)
    addToggleOption(GunAuraFeature, "Show Hitbox", true,
        function(value: boolean): ()
            settings.showHitbox = value == true
            if not settings.showHitbox then
                clearAllHitboxes()
            end
        end,
        "Shows or hides the white aura hitbox square around dropped guns.")

    activeCleanup = function(): ()
        toggleGunAura(false)
    end
    Module.Events = featureConnections
    Module.Initialized = true
    return Module
end

function Module.destroy(): ()
    if not Module.Initialized then
        return
    end
    Module.Initialized = false
    pcall(activeCleanup)
    activeCleanup = function(): () end
    Module.Events = {}
    Module.Runtime = nil
end

return Module

-------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/MM2/Other/GunAura.lua
-------------------------------------------------------------------------------

-------------------------------------------------------------------------------
ARCHIVO: src/games/MM2/Other/Notifications.lua
BYTES: 16850
SHA-256: 66ac26d264e3748a8f9b179d06ccb23a343b1939bc9465d485a6da0a92d7a4fd
INICIO DEL ARCHIVO
-------------------------------------------------------------------------------
local Module = {
    Name = "MM2 Notifications",
    Events = {} :: {[string]: any},
    Initialized = false,
    Runtime = nil :: any,
}

local activeCleanup: () -> () = function(): () end

function Module.init(runtime: any): any
    if Module.Initialized then return Module end
    local core: any = state.mm2Core
    assert(type(core) == "table", "MM2 Notifications requires the MM2 core")
    Module.Runtime = runtime

    -- Instant Roles used to be an independent Blurt Roles checkbox. Preserve
    -- both its value and its old independent-on behavior, without overriding a
    -- newer Notifications master setting on subsequent boots.
    if type(configData) == "table" and type(configData.states) == "table" then
        local states: any = configData.states
        local oldKey: string = "MM2.BlurtRoles.InstantRoles"
        local newKey: string = "MM2.Notifications.InstantRoles"
        local old: any = states[oldKey]
        local changed: boolean = false
        if states[newKey] == nil and type(old) == "boolean" then
            states[newKey] = old
            if old and states["MM2.Notifications"] == nil then
                states["MM2.Notifications"] = true
            end
            changed = true
        end
        if old ~= nil then states[oldKey] = nil; changed = true end
        if changed and type(queueConfigSave) == "function" then queueConfigSave() end
    end

    local settings: any = {
        instantRoles = false,
        gun = true,
        murderClose = true,
        yourRole = true,
        roundEvents = true,
        murderDefeated = true,
        sheriffDown = true,
        closeRadius = 30,
        closeCooldown = 8,
    }
    local enabled: boolean = false
    local phase: string = type(core.getRoundPhase) == "function" and core.getRoundPhase() or "unknown"
    local elapsed: number = 0
    local murdererSeen: boolean = false
    local sheriffSeen: boolean = false
    local lastLocalRole: string? = nil
    local lastMurderer: Player? = nil
    local lastSheriff: Player? = nil
    local defeatNotified: boolean = false
    local sheriffNotified: boolean = false
    local nearMurderer: boolean = false
    local lastCloseAt: number = -math.huge
    local gunWasDropped: boolean = false
    local gunLastSeenAt: number = -math.huge
    local gunDropPreviousHolder: Player? = nil
    local lastGunHolder: Player? = nil
    local gunHolderInitialized: boolean = false

    local function resetRound(): ()
        murdererSeen = false
        sheriffSeen = false
        lastLocalRole = nil
        lastMurderer = nil
        lastSheriff = nil
        defeatNotified = false
        sheriffNotified = false
        nearMurderer = false
        lastCloseAt = -math.huge
        gunWasDropped = false
        gunLastSeenAt = -math.huge
        gunDropPreviousHolder = nil
        lastGunHolder = nil
        gunHolderInitialized = false
    end

    local function inRound(): boolean
        if type(core.isRoundLifecycleActive) == "function"
            and core.isRoundLifecycleActive() == false then return false end
        if phase == "lobby" then return false end
        return phase == "map" or (type(core.hasActiveRoundRoles) == "function" and core.hasActiveRoundRoles())
    end

    local function rootOf(player: Player?): BasePart?
        local character: Model? = player and player.Character
        return character and character:FindFirstChild("HumanoidRootPart") :: BasePart? or nil
    end

    local function isAlive(player: Player?): boolean
        if not player then return false end
        if type(core.isPlayerAlive) == "function" then return core.isPlayerAlive(player) == true end
        return rootOf(player) ~= nil
    end

    local function showRoles(murderer: Player?, sheriff: Player?): ()
        if not enabled or not settings.instantRoles or not inRound() then return end
        if not murdererSeen and not sheriffSeen and murderer and sheriff then
            murdererSeen = true
            sheriffSeen = true
            notify("Roles ready - Murderer: " .. murderer.Name .. " | Sheriff/Hero: " .. sheriff.Name)
            return
        end
        if murderer and not murdererSeen then
            murdererSeen = true
            notify("Murderer: " .. murderer.Name)
        end
        if sheriff and not sheriffSeen then
            sheriffSeen = true
            notify("Sheriff/Hero: " .. sheriff.Name)
        end
    end

    local function gunDistance(drop: Instance?): number?
        local root: BasePart? = rootOf(LocalPlayer)
        if not root or not drop then return nil end
        local ok: boolean, distance: any = pcall(function(): number
            local position: Vector3 = if drop:IsA("BasePart")
                then (drop :: BasePart).Position
                else (drop :: Model):GetPivot().Position
            return (position - root.Position).Magnitude
        end)
        return if ok and type(distance) == "number" then distance else nil
    end

    local function updateGun(sheriff: Player?): ()
        if not settings.gun or type(core.findDroppedGun) ~= "function" then return end
        local now: number = os.clock()
        local drop: Instance? = core.findDroppedGun()
        local holder: Player? = nil
        if sheriff and isAlive(sheriff) and type(core.getPlayerWeapon) == "function"
            and core.getPlayerWeapon(sheriff, "Gun") ~= nil then holder = sheriff end
        if drop then
            if not gunWasDropped then
                -- The previous owner's Tool may still replicate briefly after
                -- the drop appears. It is not evidence they picked it back up.
                gunDropPreviousHolder = lastGunHolder or holder
                local distance: number? = gunDistance(drop)
                notify("Gun dropped" .. (distance and (" · " .. tostring(math.round(distance)) .. " studs away") or ""), "danger")
            end
            gunWasDropped = true
            gunLastSeenAt = now
        elseif gunWasDropped and holder and holder ~= gunDropPreviousHolder then
            notify("Gun picked up by " .. holder.Name, "good")
            gunWasDropped = false
            gunDropPreviousHolder = nil
        elseif gunWasDropped and now - gunLastSeenAt >= 2 then
            -- Missing/streamed-out is not proof of a pickup.
            gunWasDropped = false
            gunDropPreviousHolder = nil
        elseif holder and gunHolderInitialized and lastGunHolder and holder ~= lastGunHolder then
            notify("Gun is now held by " .. holder.Name)
        end
        if holder then
            lastGunHolder = holder
            gunHolderInitialized = true
        end
    end

    local function confirmedDead(player: Player?): boolean
        if not player then return false end
        -- getPlayerRole can infer Dead when a character streams out. Only
        -- explicit round data or a visible zero-health humanoid is evidence.
        if type(core.getRoundRole) == "function" and core.getRoundRole(player) == "Dead" then return true end
        local character: Model? = player.Character
        local humanoid: Humanoid? = character and character:FindFirstChildOfClass("Humanoid") :: Humanoid?
        return humanoid ~= nil and humanoid.Health <= 0
    end

    -- v191: the "murderer close" alert says WHERE as well as how far. Roblox's
    -- own compass is +Z = north and -X = west, so a plain world-space bearing
    -- is enough to say "NE" without a camera dependency (and it stays correct
    -- while the camera spins).
    local COMPASS: {string} = {"N", "NE", "E", "SE", "S", "SW", "W", "NW"}
    local function compassOf(x: number?, z: number?): string
        local flatX: number = tonumber(x) or 0
        local flatZ: number = tonumber(z) or 0
        if math.abs(flatX) + math.abs(flatZ) < 1e-3 then
            return "here"
        end
        local angle: number = math.deg(math.atan2(flatX, flatZ))
        local index: number = math.floor((angle + 360 + 22.5) % 360) / 45) + 1
        return COMPASS[index]
    end

    local function updateProximity(murderer: Player?): ()
        if not settings.murderClose or not murderer or murderer == LocalPlayer
            or not isAlive(LocalPlayer) or not isAlive(murderer) then
            nearMurderer = false
            return
        end
        local localRoot: BasePart? = rootOf(LocalPlayer)
        local murderRoot: BasePart? = rootOf(murderer)
        if not localRoot or not murderRoot then nearMurderer = false; return end
        local delta: Vector3 = murderRoot.Position - localRoot.Position
        local distance: number = delta.Magnitude
        local radius: number = math.clamp(tonumber(settings.closeRadius) or 30, 10, 120)
        local cooldown: number = math.clamp(tonumber(settings.closeCooldown) or 8, 2, 30)
        local now: number = os.clock()
        -- Still-close re-warns after the cooldown (the v190 behaviour the
        -- suite pins); leaving the radius re-arms it immediately.
        if distance <= radius and now - lastCloseAt >= cooldown then
            nearMurderer = true
            lastCloseAt = now
            notify(
                "Murderer close: " .. murderer.Name
                    .. " · " .. tostring(math.round(distance)) .. " studs"
                    .. " · " .. compassOf(delta.X, delta.Z)
                    .. (distance <= radius * 0.4 and " · VERY close" or ""),
                "danger"
            )
        elseif distance > radius * 1.25 then
            nearMurderer = false
        end
    end

    local function sample(): ()
        if not enabled or not inRound() then return end
        local murderer: Player? = type(core.findMurderer) == "function" and core.findMurderer() or nil
        local sheriff: Player? = type(core.findSheriff) == "function" and core.findSheriff() or nil
        if murderer then lastMurderer = murderer end
        if sheriff then lastSheriff = sheriff end
        showRoles(murderer, sheriff)
        if settings.yourRole and type(core.getPlayerRole) == "function" then
            local myRole: any = core.getPlayerRole(LocalPlayer)
            if (myRole == "Innocent" or myRole == "Sheriff" or myRole == "Hero" or myRole == "Murderer")
                and myRole ~= lastLocalRole then
                lastLocalRole = myRole
                notify("Your role: " .. myRole)
            end
        end
        updateGun(sheriff)
        updateProximity(murderer)
        if settings.murderDefeated and not defeatNotified and confirmedDead(lastMurderer) then
            defeatNotified = true
            notify("Murderer defeated: " .. (lastMurderer :: Player).Name, "good")
        end
        -- v191: the sheriff (or the hero who picked the gun up) is worth its
        -- own alert: their death is when the gun hits the floor.
        if settings.sheriffDown and not sheriffNotified and lastSheriff ~= nil
            and lastSheriff ~= LocalPlayer and confirmedDead(lastSheriff) then
            sheriffNotified = true
            notify("Sheriff down: " .. (lastSheriff :: Player).Name .. " · the gun is loose", "danger")
        end
    end

    local nextWarningAt: number = -math.huge
    local sampling: boolean = false
    local sampleAgain: boolean = false
    local function trySample(): ()
        -- A role lookup can rebuild the core index and synchronously emit
        -- onRoundRoles. Coalesce that callback instead of recursing into a
        -- partially updated sample; the regular tick handles further changes.
        if sampling then sampleAgain = true; return end
        sampling = true
        for _pass: number = 1, 2 do
            sampleAgain = false
            local ok: boolean, err: any = pcall(sample)
            if not ok and os.clock() >= nextWarningAt then
                nextWarningAt = os.clock() + 5
                warn("MM2 Notifications: " .. tostring(err))
            end
            if not sampleAgain or not enabled then break end
        end
        sampling = false
    end

    local unsubscribeRoles: () -> () = core.onRoundRoles(function(active: boolean): ()
        if active and enabled then trySample() end
    end)
    local unsubscribePhase: () -> () = core.onRoundPhase(function(nextPhase: string, reason: string): ()
        local previous: string = phase
        phase = nextPhase
        if nextPhase == "lobby" and previous ~= "lobby" then
            if enabled and settings.roundEvents and previous == "map" and reason ~= "subscribe" then
                notify("Round ended")
            end
            resetRound()
        elseif nextPhase == "map" and previous ~= "map" then
            resetRound()
            if enabled and settings.roundEvents and reason ~= "subscribe" then
                notify("Entered the map - round starting")
            end
            if enabled then trySample() end
        end
    end)

    local function toggleNotifications(value: boolean): ()
        enabled = value == true
        disconnectFeatureConnection("MM2NotificationsTick")
        elapsed = 0
```

```
        if not enabled then return end
        trySample()
        featureConnections.MM2NotificationsTick = TaskManager:Connect(function(deltaTime: number): ()
            elapsed += math.max(0, deltaTime)
            if elapsed < 0.25 then return end
            elapsed = 0
            trySample()
        end, nil, "MM2 Notifications")
    end

    local feature: any = createUniversalFeature(
        "Notifications", "Private MM2 alerts for roles, the gun, nearby danger and rounds.",
        4, toggleNotifications,
        {categoryName = "Utilities", parent = MM2Scroll, registry = mm2Features}
    )
    addToggleOption(feature, "Instant Roles", false, function(value: boolean): ()
        settings.instantRoles = value
        core.mm2Settings.instantRoleNotify = value
        if enabled and value then trySample() end
    end, "Private role alerts as soon as each role is known; never sends public chat.")
    addToggleOption(feature, "Gun Notification", true, function(value: boolean): ()
        settings.gun = value
        if not value then
            gunWasDropped = false
            gunDropPreviousHolder = nil
            lastGunHolder = nil
            gunHolderInitialized = false
        elseif enabled then
            trySample()
        end
    end, "Announces a dropped gun and a confirmed new holder.")
    addToggleOption(feature, "Murder Close", true, function(value: boolean): ()
        settings.murderClose = value
        if enabled and value then trySample() end
    end, "Warns when the living Murderer gets close, with the distance and the compass side. Off = no proximity alerts.")
    addNumberOption(feature, "Close radius", 30, 10, 120, function(value: number): ()
        settings.closeRadius = math.clamp(tonumber(value) or 30, 10, 120)
        nearMurderer = false
        lastCloseAt = -math.huge
        if enabled and settings.murderClose then trySample() end
    end, "Studs at which the Murder Close warning fires. The alert repeats when you get closer or after the cooldown.", 10, true)
    addNumberOption(feature, "Close cooldown", 8, 2, 30, function(value: number): ()
        settings.closeCooldown = math.clamp(tonumber(value) or 8, 2, 30)
    end, "Minimum seconds between two Murder Close warnings.", 1, true)
    addToggleOption(feature, "Sheriff Down", true, function(value: boolean): ()
        settings.sheriffDown = value
        if enabled and value then trySample() end
    end, "Announces the known Sheriff/Hero death: that is when the gun is loose.")
    addToggleOption(feature, "Your Role", true, function(value: boolean): ()
        settings.yourRole = value
        if enabled and value then trySample() end
    end, "Notifies you when your role becomes known or changes.")
    addToggleOption(feature, "Round Events", true, function(value: boolean): ()
        settings.roundEvents = value
    end, "Private alerts for entering the map and the round ending.")
    addToggleOption(feature, "Murderer Defeated", true, function(value: boolean): ()
        settings.murderDefeated = value
        if enabled and value then trySample() end
    end, "Warns only when the known Murderer is confirmed dead.")
    addInformationOption(feature, "These alerts are private. Blurt Roles is the separate public-chat feature.")

    activeCleanup = function(): ()
        toggleNotifications(false)
        pcall(unsubscribeRoles)
        pcall(unsubscribePhase)
        core.mm2Settings.instantRoleNotify = false
    end
    Module.Events = featureConnections
    Module.Initialized = true
    return Module
end

function Module.destroy(): ()
    if not Module.Initialized then return end
    Module.Initialized = false
    activeCleanup()
    activeCleanup = function(): () end
    Module.Events = {}
    Module.Runtime = nil
end

return Module

-------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/MM2/Other/Notifications.lua
-------------------------------------------------------------------------------

-------------------------------------------------------------------------------
ARCHIVO: src/games/MM2/Prediction.lua
BYTES: 92554
SHA-256: b21b33161350ca5a95f77f8099bd6665b3d3491b8de72ef3e087a2aea62ac675
INICIO DEL ARCHIVO
-------------------------------------------------------------------------------
local Module = {
    Name = "MM2 Prediction",
    Events = {} :: {[string]: any},
    Initialized = false,
    Runtime = nil :: any,
}

--[[
    Kinematic prediction core for MM2 gun shots and knife throws.

    What this model is based on (TestingStuff evidence, not guesses):

      * GunFired echoes carry (handle, origin, endpoint, hitPart). The time
        between our FireServer and that echo is the only transport number we
        can measure in-game; the saved calibration recorded 297 confirmed
        shots with a mean of ~123.5 ms.
      * ThrowingKnifeVisuals advances the blade linearly:
            CFrame + Direction * ThrowSpeed * dt
        with ThrowSpeed 96 by default and no gravity term. A blade therefore
        needs an intercept solve, not a ballistic one.
      * A remote character's replicated position is older than the server
        state the gun ray is checked against. The aim point advances by
        the full measured transport plus the replication interpolation
        window (the saved sessions score hits at the confirmation
        instant, i.e. the server behaves as if it rewound by the whole
        ping; the one-way half was tried in v85 and led short).

    What this model deliberately does NOT do:

      * No speculative openings or unconstrained extrapolation. Turn and
        braking terms come from weighted fits, capped at 45 degrees / the stop
        distance. Knife landing additionally uses a bounded client-side support
        probe; its cache, rig and slope thresholds are conservative estimates,
        not measured server behaviour.
      * No unconstrained auto-tuned lead. Transport is the median of at least
        three fresh echoes inside sane pairing limits, with smoothed RTT as
        fallback; the separate residual correction is median-filtered and
        capped at 60 ms.
      * No invented movement when the measured velocity is zero but
        MoveDirection is not. The server only knows replicated motion.
      * No distance/confidence gate before firing. Freshness of the sample,
        WallCheck and weapon readiness are the real conditions.
]]

type Sample = {
    at: number,
    position: Vector3,
    velocity: Vector3,
    state: string,
    grounded: boolean,
    aimOffset: Vector3,
    aimPart: string,
    -- Replicated control input, recorded so an offline replay can weigh an
    -- input-based turn against the measured velocity turn.
    moveDirection: Vector3,
}

type Track = {
    character: Model,
    humanoid: Humanoid,
    root: BasePart,
    samples: {Sample},
    -- v139 cache: reused for PART_REFRESH_SECONDS instead of re-searched on
    -- every 60 Hz sample.
    aimPart: BasePart?,
    aimName: string?,
    partsAt: number?,
    -- v139: recycled sample tables. At 60 Hz x 12 players this was creating
    -- ~700 tables per second, which is exactly the kind of steady garbage that
    -- shows up later as a hitch when the collector catches up.
    spare: {Sample}?,
    -- Up to two short-lived support probes for the knife only. These never
    -- change the shared gun prediction or its historical floor clamp.
    knifeFloorCache: {any}?,
    knifeFloorNext: number?,
    knifeFloorSerial: number?,
}

type Fit = {
    velocity: Vector3,
    residual: number,
    count: number,
    span: number,
    -- Weighted mean sample time. The slope is the chord over the window, so
    -- this is where the measured velocity actually points.
    center: number,
}

type Motion = {
    velocity: Vector3,
    horizontal: Vector3,
    vertical: number,
    airborne: boolean,
    sampleAge: number,
    samples: number,
    residual: number,
    source: string,
    -- Signed yaw rate in rad/s measured between two fits. Zero when the
    -- measurement is not trustworthy (slow, noisy or too few samples).
    turnRate: number,
    -- Longitudinal acceleration along the current direction, clamped. Used
    -- only to keep the lead length honest while the target speeds up or stops.
    longitudinalAccel: number,
    -- Full horizontal acceleration vector measured between two fits,
    -- magnitude-clamped. The calibration replays (85 scored confirmations)
    -- picked a constant-acceleration model over constant velocity 57 to 19,
    -- so the vector is applied over the horizon when it is meaningful.
    accelX: number,
    accelZ: number,
    -- Seconds until the target stops when it is braking hard, else nil.
    brakeSeconds: number?,
    -- How far behind the newest position sample the fitted direction sits.
    -- predictedRoot starts at that sample, not at the later gunAim call.
    fitLag: number,
    -- True only when the measured vertical speed is worth extrapolating. The
    -- humanoid state alone is not enough: a replicated "Jumping" pose can be a
    -- character standing at the apex of a jump.
    verticalLead: boolean,
    -- Replicated control input and instantaneous velocity on the newest sample.
    moveDirection: Vector3?,
    reportedHorizontal: Vector3?,
    hasInputHistory: boolean?,
}

type AimInfo = {
    endpoint: Vector3,
    targetPosition: Vector3,
    capturedPosition: Vector3,
    rootPosition: Vector3,
    velocity: Vector3,
    leadSeconds: number,
    turnRate: number,
    brakeSeconds: number?,
    sampleAge: number,
    samples: number,
    residual: number,
    motionSource: string,
    model: string,
    rtt: number,
    -- Raw engine readings at aim time, so a calibration can tell which of the
    -- two sources produced the lead instead of having to reconstruct it.
    rttEngine: number?,
    rttData: number?,
    echoDelay: number?,
    -- How many echoes back the recorded delay. Zero means the horizon used
    -- the ping, which the calibration can distinguish from a real median.
    echoSamples: number,
    -- Seconds added to the horizon by the confirmed-shot feedback for this
    -- aim mode. Zero until three confirmations of that mode were measured.
    leadCorrection: number,
    -- What the measured median asks for, before the per-sample slew limit.
    -- Reporting only: the aim already uses `leadCorrection`.
    leadCorrectionTarget: number?,
    leadSamples: number?,
    aimMode: string?,
    flightSeconds: number?,
}

local MAX_SAMPLE_AGE: number = 0.35
local FIT_WINDOW: number = 0.16
local FIT_TAU: number = 0.055
local HISTORY_SECONDS: number = 0.6
-- Remote characters replicate around 20 Hz. 60 Hz keeps three observations per
-- replicated pose (so a fit is never built from a single packet) without
-- inventing detail that the network never sent. Raising it only re-weights the
-- same poses and costs CPU; the dominant error is the transport horizon.
local SAMPLE_HZ: number = 60
local MAX_HORIZONTAL_SPEED: number = 60
local DEFAULT_KNIFE_SPEED: number = 96
local MAX_ECHOES: number = 8
-- A median of two echoes is just the mean of two mismatches. The echo has one
-- source (the confirmation path), and the v64 session matched the server clock
-- to within 8 ms on 13 of 13 shots, so three samples are enough to trust the
-- round trip. Requiring six left a 189 s session on an engine reading that was
-- half the measured round trip.
local MIN_ECHO_SAMPLES: number = 3
-- Pairing sanity limits. An echo only reaches noteEcho after base.lua matched
-- the same gun, an origin within 3.5 studs and a direction within ~15 degrees,
-- so a late sample is our own shot, not a crossed one. The upper bound mirrors
-- the 1.6 s pending-shot window in base.lua; the v18 session has a confirmed
-- 1007.7 ms echo at 270.3 ms ping, which an 800 ms cut would have thrown away.
local MIN_ECHO_DELAY: number = 0.05
local MAX_ECHO_DELAY: number = 1.6
-- Echo housekeeping (v105). The window holds samples, not history: an echo
-- older than this no longer describes the current route, so it expires even
-- when too few shots were fired to push it out by count -- in a quiet round
-- the v104 window could survive minutes past the connection it measured.
local ECHO_MAX_AGE: number = 30
-- A healthy echo median sits at ~1.0-1.1x the RTT (both are the same round
-- trip). If the median climbs past 1.5x, the window is poisoned by late
-- pairings again; clearing it drops the lead back to the ping fallback until
-- three fresh echoes rebuild it, instead of waiting for eight new samples
-- to rotate the stale ones out.
local ECHO_DIVERGENCE_RATIO: number = 1.5
-- Transport + manual lead. A full second only happens when the session ping
-- or the manual offset asks for it; the status line shows the real number.
local MAX_HORIZON: number = 1.0
-- Automatic lead never exceeds this. At 16 studs/s a 1 s lead is 16 studs of
-- overshoot; past 0.6 s the sample is stale enough that refusing to invent
-- more distance is safer. Manual offset is still allowed up to MAX_HORIZON.
local MAX_AUTO_LEAD: number = 0.6
-- Turn and braking corrections are measured, never guessed:
--   * two weighted fits 0.10 s apart give a signed yaw rate;
--   * the same pair gives longitudinal acceleration for the stop cap.
local TURN_SPAN: number = 0.10
local TURN_MIN_SPEED: number = 2.0
local TURN_MAX_RATE: number = 10.0
-- Below ~23 deg/s the two fits are mostly replication jitter; a correction
-- that small only adds noise to the endpoint.
local TURN_MIN_RATE: number = 0.4
local TURN_MAX_ANGLE: number = math.rad(45)
-- Two fits over replicated poses disagree by jitter as much as by turning.
-- At 0.5 the recorded session produced 224 deg and 152 deg turns on a
-- straight run; at 0.20 the shots with a >100 deg turn drop from five to two
-- and the measured aim improves, so this is where the noise stops.
local TURN_RESIDUAL: number = 0.20
-- Vertical extrapolation needs a real vertical speed. A humanoid reported as
-- Jumping at its apex has almost no vertical velocity; dropping the aim by a
-- full ballistic term there is what put shot 6 2.3 studs under the target.
local VERTICAL_MIN_SPEED: number = 8.0
-- v161: how far the floor clamp is allowed to RAISE a downward prediction.
-- The clamp exists so a landing target is not aimed at below the ground; it
-- used to apply without limit, and the archived sessions show the cost: in
-- one of them it lifted the aim by 20.9 studs (the target was falling off
-- something, not landing), turning a small dip into a huge miss. One jump
-- height is the honest bound: anything deeper is a fall, and a fall should be
-- led, not cancelled. Replay over 411 airborne samples of the 13 saved
-- sessions: 3.07 -> 2.55 studs of mean vertical error, and the worst session
-- drops from 17.62 to 3.15.
local MAX_FLOOR_LIFT_STUDS: number = 6
-- Robust transport fallback: a single GetNetworkPing spike (0.31 s against a
-- 0.107 s baseline in the 2026-10-18 session) must not move the next aim.
local RTT_SAMPLES: number = 8
local RTT_SAMPLE_INTERVAL: number = 0.25
local BRAKE_MIN_ACCEL: number = 6.0
local BRAKE_MAX_ACCEL: number = 120.0
-- Closed-loop lead feedback. Every confirmed shot measures where the target
-- actually was when the server scored it and compares that with the point we
-- aimed at, projected on the target's own horizontal velocity. The median of
-- the last few shots is added to the horizon, bounded so one bad measurement
-- can never turn the lead into a guess. This is the only part of the model
-- that learns from the server, and it learns from confirmations we already
-- receive: no extra session and no extra instrument are needed.
local LEAD_FEEDBACK_MIN_SAMPLES: number = 3
local LEAD_FEEDBACK_MAX_SAMPLES: number = 8
-- A single measurement this large is a teleport, a target swap or a respawn,
-- not a lead error: it is dropped instead of stored.
local LEAD_FEEDBACK_SAMPLE_LIMIT: number = 0.15
-- The applied correction. At 18 studs/s this is ~1.1 studs either way; the
-- systematic bias it removes (full round trip used as a one-way delay) is
-- around half the echo, which is far larger than the cap.
local MAX_LEAD_CORRECTION: number = 0.06
-- v161: the measured correction is APPLIED (it was computed, written to
-- telemetry and discarded since v120). It moves at most this much per new
-- confirmation, so a single bad pairing cannot swing the aim and the loop
-- cannot oscillate: the error it corrects is measured after the previous
-- correction was already in place.
local LEAD_SLEW_PER_SAMPLE: number = 0.02

local settings = {
    -- Automatic lead is the default: the measured FireServer -> GunFired echo
    -- (fallback: smoothed ping) already includes the device/network timing.
    -- Manual offset is only used when autoLead is turned off.
    autoLead = true,
    -- Manual calibration in seconds. Positive aims further ahead.
    extraLead = 0,
    -- Half of the default 20 Hz replication interval. This is the window the
    -- client interpolates between two replicated character states.
    interpolation = 1 / 40,
}

local activeCleanup: () -> () = function(): () end

local function finiteNumber(value: any): boolean
    return type(value) == "number" and value == value and math.abs(value) < math.huge
end

local function finiteVector(value: any): boolean
    return value ~= nil
        and finiteNumber(value.X)
        and finiteNumber(value.Y)
        and finiteNumber(value.Z)
end

local function humanoidStateName(humanoid: Humanoid): string
    local state: any = humanoid:GetState()
    return state.Name or tostring(state)
end

-- v158: the predictor is asked only about live bodies. A ragdoll on the floor
-- still has a HumanoidRootPart and a last velocity, so `knifeAim`/`gunAim`
-- happily solved a lead for a corpse that cannot move. This is the same test
-- base.lua applies to "alive", evaluated for the target we are about to solve
-- rather than trusting a cached track.
local function bodyIsAlive(humanoid: Humanoid?): boolean
    if not humanoid or humanoid.Health <= 0 then return false end
    local currentState: Enum.HumanoidStateType = humanoid:GetState()
    if currentState == Enum.HumanoidStateType.Dead
        or currentState == Enum.HumanoidStateType.Physics then
        return false
    end
    if humanoid:GetAttribute("Dead") == true then return false end
    local parent: Instance? = humanoid.Parent
    return not parent or parent:GetAttribute("Dead") ~= true
end

local function targetHumanoid(target: Player): any
    local ok: boolean, humanoid: any = pcall(function(): any
        local character = target.Character
        return character and character:FindFirstChildOfClass("Humanoid")
    end)
    if not ok then
        return nil
    end
    return humanoid
end

local AIM_PARTS: {string} = {
    "UpperTorso",
    "Torso",
    "HumanoidRootPart",
    "Head",
}

local function findAimPart(character: Model): (BasePart?, string)
    for _, partName: string in ipairs(AIM_PARTS) do
        local part: Instance? = character:FindFirstChild(partName)
        if part and part:IsA("BasePart") then
            return part :: BasePart, partName
        end
    end
    local root: Instance? = character:FindFirstChild("HumanoidRootPart")
    if root and root:IsA("BasePart") then
        return root :: BasePart, "HumanoidRootPart"
    end
    return nil, ""
end

function Module.init(runtime: any): any
    if Module.Initialized then
        return Module
    end
    local core: any = state.mm2Core
    assert(type(core) == "table", "MM2 Prediction requires the MM2 core module")
    local predictionMath: any = core.predictionMath
    assert(type(predictionMath) == "table", "MM2 Prediction requires predictionMath")
    Module.Runtime = runtime

    local tracks: {[Player]: Track} =
        setmetatable({}, {__mode = "k"}) :: any
    local echoes: {number} = {}
    local echoTimes: {number} = {}
    local elapsed: number = 0

    -- v139: the sampler runs 60 times a second for COLOUR every other player, so
    -- the per-sample lookups are cached per track and refreshed at 2 Hz (or at
    -- once when the character/part disappears). Before this, every single sample
    -- did FindFirstChildOfClass("Humanoid") + FindFirstChild("HumanoidRootPart")
    -- + a 3-4 lookup aim-part search, ~2000 instance lookups per second.
    local PART_REFRESH_SECONDS: number = 0.5

    local function refreshTrackParts(
        track: Track,
        character: Model,
        now: number
    ): boolean
        local refresh: boolean = now - (track.partsAt or -math.huge) >= PART_REFRESH_SECONDS
            or track.root == nil
            or track.humanoid == nil
        if not refresh then
            return true
        end
        local humanoid: Humanoid? = character:FindFirstChildOfClass("Humanoid")
            :: Humanoid?
        local root: BasePart? = character:FindFirstChild("HumanoidRootPart")
            :: BasePart?
        if not humanoid or not root or not bodyIsAlive(humanoid) then
            return false
        end
        track.humanoid = humanoid
        track.root = root
        local aimPart: BasePart?, aimName: string = findAimPart(character)
        track.aimPart = aimPart
        track.aimName = aimName
        track.partsAt = now
        return true
    end

    local function ensureTrack(player: Player, now: number): Track?
        local character: Model? = player.Character
        if not character then
            tracks[player] = nil
            return nil
        end
        local track: Track? = tracks[player]
        if not track or track.character ~= character then
            track = {
                character = character,
                humanoid = nil,
                root = nil,
                samples = {},
                partsAt = -math.huge,
            }
            tracks[player] = track
        end
        if not refreshTrackParts(track, character, now) then
            tracks[player] = nil
            return nil
        end
        return track
    end

    local function samplePlayer(player: Player, now: number): ()
        local track: Track? = ensureTrack(player, now)
        if not track then
            return
        end
        local humanoid: Humanoid? = track.humanoid
        if not bodyIsAlive(humanoid) then
            tracks[player] = nil
            return
        end
        local root: BasePart = track.root
        local position: Vector3 = root.Position
        local velocity: Vector3 = root.AssemblyLinearVelocity
        if not finiteVector(velocity) then
            velocity = Vector3.zero
        end
        local stateName: string = humanoidStateName(track.humanoid)
        -- FloorMaterial often reads Air on remote characters that are walking
        -- on flat ground. The humanoid state is the reliable signal.
        local grounded: boolean = stateName ~= "Jumping"
            and stateName ~= "Freefall"
        -- Reading MoveDirection off a live Humanoid cannot throw: no pcall, no
        -- closure. A nil humanoid was already rejected by ensureTrack.
        local move: any = track.humanoid.MoveDirection
        local moveDirection: Vector3 = if typeof(move) == "Vector3"
            and finiteVector(move)
            then move
            else Vector3.zero
        -- Reused from the 2 Hz cache: a destroyed part still answers Position,
        -- and a respawn replaces the whole track, so there is nothing to check
        -- per sample.
        local aimPart: BasePart? = track.aimPart
        local aimPartName: string = track.aimName or ""
        local aimOffset: Vector3 = aimPart
            and (aimPart.Position - position)
            or Vector3.zero

        local samples: {Sample} = track.samples
        local last: Sample? = samples[#samples]
        if last then
            local delta: number = now - last.at
            -- Respawns, teleports and ownership changes must not seed a fit.
            if delta > 0 and delta < 0.4
                and (position - last.position).Magnitude > 8 + 120 * delta then
                local spare: {Sample} = track.spare or {}
                track.spare = spare
                for _, dropped: Sample in ipairs(samples) do
                    if #spare < 8 then
                        table.insert(spare, dropped)
                    end
                end
                table.clear(samples)
            end
        end
        local spare: {Sample} = track.spare or {}
        track.spare = spare
        local sample: Sample = table.remove(spare) or {} :: Sample
        sample.at = now
        sample.position = position
        sample.velocity = velocity
        sample.state = stateName
        sample.grounded = grounded
        sample.aimOffset = aimOffset
        sample.aimPart = aimPartName
        sample.moveDirection = moveDirection
        table.insert(samples, sample)
        while #samples > 2 and now - samples[1].at > HISTORY_SECONDS do
            local retired: Sample = table.remove(samples, 1)
            if #spare < 8 then
                table.insert(spare, retired)
            end
        end
    end

    local function fitVelocity(track: Track, reference: number): Fit?
        local samples: {Sample} = track.samples
        if #samples < 3 then
            return nil
        end
        -- Collect the window that ends at `reference`, not necessarily at the
        -- newest sample: the same weights applied to an older window give the
        -- velocity direction 0.10 s ago, which is what the turn rate needs.
        local collected: {Sample} = {}
        for index: number = #samples, 1, -1 do
            local sample: Sample = samples[index]
            if sample.at <= reference + 1e-4 then
                if reference - sample.at > FIT_WINDOW then
                    break
                end
                table.insert(collected, 1, sample)
            end
        end
        if #collected < 3 then
            return nil
        end
        local anchor: number = collected[#collected].at
        local sw, st, stt: number = 0, 0, 0
        local sx, sy, sz: number = 0, 0, 0
        local stx, sty, stz: number = 0, 0, 0
        for _, sample: Sample in ipairs(collected) do
            local dt: number = sample.at - anchor
            local age: number = math.max(0, reference - sample.at)
            local weight: number = math.exp(-age / FIT_TAU)
            sw += weight
            st += weight * dt
            stt += weight * dt * dt
            sx += weight * sample.position.X
            sy += weight * sample.position.Y
            sz += weight * sample.position.Z
            stx += weight * dt * sample.position.X
            sty += weight * dt * sample.position.Y
            stz += weight * dt * sample.position.Z
        end
        if sw <= 0 then
            return nil
        end
        local denominator: number = stt - st * st / sw
        if denominator <= 1e-8 then
            return nil
        end
        local meanT: number = st / sw
        local meanX: number = sx / sw
        local meanY: number = sy / sw
        local meanZ: number = sz / sw
        local vx: number = (stx - st * sx / sw) / denominator
        local vy: number = (sty - st * sy / sw) / denominator
        local vz: number = (stz - st * sz / sw) / denominator
        local squared: number = 0
        for _, sample: Sample in ipairs(collected) do
            local dt: number = sample.at - anchor
            local dx: number = sample.position.X - (meanX + vx * (dt - meanT))
            local dy: number = sample.position.Y - (meanY + vy * (dt - meanT))
            local dz: number = sample.position.Z - (meanZ + vz * (dt - meanT))
            squared += dx * dx + dy * dy + dz * dz
        end
        return {
            velocity = Vector3.new(vx, vy, vz),
            residual = math.sqrt(squared / #collected),
            count = #collected,
            span = collected[#collected].at - collected[1].at,
            -- `dt` is relative to the newest sample in this window, not to
            -- `reference`. Using reference here shifts the two fit centers by
            -- different amounts on uneven frames and biases their measured
            -- acceleration (a 24 studs/s^2 probe read 27.33 before this fix).
            center = anchor + meanT,
        }
    end

    -- Two fits 0.10 s apart: their directions give the signed yaw rate, their
    -- magnitudes give the longitudinal braking. Everything is gated on a clean
    -- fit and a real speed, so noise and near-stationary targets stay at zero.
    local function measureTurn(
        track: Track,
        now: number,
        fit: Fit
    ): (number, number, number?, number, number, number)
        local newest: Sample? = track.samples[#track.samples]
        -- The extrapolation starts at the newest position sample. Measuring
        -- from `now` instead would add sampleAge to every acceleration and
```

Leer columna izquierda completa y despues la derecha. Texto seleccionable; lineas largas comprimidas. Tabulaciones: 4 espacios. Original adjunto.

```
        -- turn correction even though that age is already in the horizon.
        local fitLag: number = newest
            and math.max(0, newest.at - fit.center)
            or 0
        if fit.count < 3 or fit.span < 0.06 then
            return 0, 0, nil, fitLag, 0, 0
        end
        local previous: Fit? = fitVelocity(track, now - TURN_SPAN)
        if not previous or previous.count < 3 or previous.span < 0.04 then
            return 0, 0, nil, fitLag, 0, 0
        end
        -- Soft residual gate: a hard turn makes both fits noisier, and
        -- the old all-or-nothing cut dropped the measured acceleration
        -- exactly in the shots that needed it (v91 shot 5 turned ~90
        -- degrees inside the fit window and was missed 3.3 studs
        -- sideways). Between one and 2.25x the clean limit the turn and
        -- acceleration are applied at half strength instead of zero.
        local worstResidual: number = math.max(fit.residual, previous.residual)
        local confidence: number = 1
        if worstResidual > TURN_RESIDUAL then
            if worstResidual > TURN_RESIDUAL * 2.25 then
                return 0, 0, nil, fitLag, 0, 0
            end
            confidence = 0.5
        end
        local v0: Vector3 = Vector3.new(previous.velocity.X, 0, previous.velocity.Z)
        local v1: Vector3 = Vector3.new(fit.velocity.X, 0, fit.velocity.Z)
        if v0.Magnitude < TURN_MIN_SPEED or v1.Magnitude < TURN_MIN_SPEED then
            return 0, 0, nil, fitLag, 0, 0
        end
        local dt: number = math.max(0.04, fit.center - previous.center)
        local cross: number = v0.Z * v1.X - v0.X * v1.Z
        local dot: number = v0.X * v1.X + v0.Z * v1.Z
        local rate: number = math.clamp(
            math.atan2(cross, dot) / dt,
            -TURN_MAX_RATE,
            TURN_MAX_RATE
        )
        if math.abs(rate) < TURN_MIN_RATE then
            rate = 0
        end
        local ax: number = (v1.X - v0.X) / dt * confidence
        local az: number = (v1.Z - v0.Z) / dt * confidence
        rate *= confidence
        if math.abs(rate) < TURN_MIN_RATE then
            rate = 0
        end
        local speed: number = v1.Magnitude
        local along: number = math.clamp(
            (ax * v1.X + az * v1.Z) / speed,
            -BRAKE_MAX_ACCEL,
            BRAKE_MAX_ACCEL
        )
        local brakeSeconds: number? = nil
        if along <= -BRAKE_MIN_ACCEL then
            local currentSpeed: number = math.max(0, speed + along * fitLag)
            brakeSeconds = currentSpeed / -along
        end
        local accelMagnitude: number = math.sqrt(ax * ax + az * az)
        if accelMagnitude > BRAKE_MAX_ACCEL then
            local scale: number = BRAKE_MAX_ACCEL / accelMagnitude
            ax *= scale
            az *= scale
        end
        return rate, along, brakeSeconds, fitLag, ax, az
    end

    local function estimateMotion(track: Track, now: number): Motion?
        local samples: {Sample} = track.samples
        local last: Sample? = samples[#samples]
        if not last then
            return nil
        end
        local age: number = math.max(0, now - last.at)
        if age > MAX_SAMPLE_AGE then
            return nil
        end
        local fit: Fit? = fitVelocity(track, now)
        local reported: Vector3 = last.velocity
        local horizontal: Vector3
        local source: string
        local turnRate: number = 0
        local longitudinalAccel: number = 0
        local accelX: number = 0
        local accelZ: number = 0
        local brakeSeconds: number? = nil
        local fitLag: number = 0
        local verticalLead: boolean = false
        if fit and fit.count >= 3 and fit.span >= 0.04 and fit.residual <= 1.5 then
            -- Relaxed gates (v118): the old residual <= 0.5 and span >= 0.06
            -- dropped too many valid acceleration samples, causing false
            -- "unreachable" rejects. The calibration shows good shots
            -- correlate with short trajectory spans (~0.23s), so letting
            -- more data through improves hit rate without adding noise.
            horizontal = Vector3.new(fit.velocity.X, 0, fit.velocity.Z)
            source = "history"
        else
            horizontal = Vector3.new(reported.X, 0, reported.Z)
            source = "replicated"
        end
        if not finiteVector(horizontal) then
            horizontal = Vector3.zero
        elseif horizontal.Magnitude > MAX_HORIZONTAL_SPEED then
            horizontal = horizontal.Unit * MAX_HORIZONTAL_SPEED
        end
        -- Airborne is a physics fact before it is a state name: in the
        -- v91 session one target fell at -46 studs/s while its state
        -- still read "Running" (it had just left a ledge), so no
        -- ballistic lead was applied and the shot went over its head.
        -- A grounded runner cannot have |vy| anywhere near 15, so fast
        -- vertical velocity alone proves the body is off the ground.
        local airborne: boolean = last.state == "Jumping"
            or last.state == "Freefall"
            or math.abs(reported.Y) >= 15
        local vertical: number = 0
        if airborne then
            vertical = reported.Y
            if not finiteNumber(vertical) then
                vertical = 0
            elseif math.abs(vertical) < 0.5 and fit then
                vertical = fit.velocity.Y
            end
            vertical = math.clamp(vertical, -120, 120)
        end
        if fit then
            turnRate, longitudinalAccel, brakeSeconds, fitLag, accelX, accelZ =
                measureTurn(track, now, fit)
        end
        if airborne and math.abs(vertical) >= VERTICAL_MIN_SPEED then
            verticalLead = true
        end
        local reportedHorizontal: Vector3 = Vector3.new(reported.X, 0, reported.Z)
        if not finiteVector(reportedHorizontal) then
            reportedHorizontal = Vector3.zero
        elseif reportedHorizontal.Magnitude > MAX_HORIZONTAL_SPEED then
            reportedHorizontal = reportedHorizontal.Unit * MAX_HORIZONTAL_SPEED
        end
        local moveDir: Vector3 = last.moveDirection or Vector3.zero
        local moveHorizontal: Vector3 = Vector3.new(moveDir.X, 0, moveDir.Z)
        if not finiteVector(moveHorizontal) then
            moveHorizontal = Vector3.zero
        end
        local hasInputHistory: boolean = moveHorizontal.Magnitude > 0.08
        if not hasInputHistory then
            for idx: number = math.max(1, #samples - 15), #samples do
                local s: Sample = samples[idx]
                local md: Vector3 = s.moveDirection or Vector3.zero
                if math.sqrt(md.X * md.X + md.Z * md.Z) > 0.08 then
                    hasInputHistory = true
                    break
                end
            end
        end
        return {
            velocity = Vector3.new(horizontal.X, vertical, horizontal.Z),
            horizontal = horizontal,
            vertical = vertical,
            airborne = airborne,
            sampleAge = age,
            samples = #samples,
            residual = fit and fit.residual or math.huge,
            source = source,
            turnRate = turnRate,
            longitudinalAccel = longitudinalAccel,
            accelX = accelX,
            accelZ = accelZ,
            brakeSeconds = brakeSeconds,
            fitLag = fitLag,
            verticalLead = verticalLead,
            moveDirection = moveHorizontal,
            reportedHorizontal = reportedHorizontal,
            hasInputHistory = hasInputHistory,
        }
    end

    -- Lead displacement for a horizon.
    -- When live Roblox Humanoid.MoveDirection control input is present, we
    -- integrate the exact Roblox Humanoid exponential velocity-relaxation law:
    --   dv/dt = (v_cmd - v(t)) / tau
    --   dx(T) = v_cmd * T + (v_0 - v_cmd) * tau * (1 - exp(-T / tau))
    -- Validated across all 17 live MM2 calibration shots (0.83 studs mean XZ
    -- error and 100% hit rate). When no MoveDirection history is present (e.g.
    -- synthetic trajectories), falls back to the fitted chord/braking solver.
    local function leadHorizontal(motion: Motion, horizon: number): Vector3
        if horizon <= 0 then
            return Vector3.zero
        end
        local moveDir: Vector3? = motion.moveDirection
        local moveMag: number = moveDir and moveDir.Magnitude or 0
        if motion.hasInputHistory then
            local repH: Vector3 = motion.reportedHorizontal or motion.horizontal
            local fitH: Vector3 = motion.horizontal
            local v0: Vector3 = if repH.Magnitude > 0.05
                then repH * 0.65 + fitH * 0.35
                else fitH
            local v0Mag: number = v0.Magnitude
            local effHorizon: number = horizon * 0.88
            local tau: number = motion.airborne and 0.17 or 0.11
            if moveMag > 0.08 and moveDir then
                local unitMove: Vector3 = moveDir / moveMag
                local targetSpeed: number = math.clamp(math.max(16, v0Mag), 16, 24)
                local vCmd: Vector3 = unitMove * targetSpeed
                local alpha: number = tau * (1 - math.exp(-effHorizon / tau))
                return vCmd * effHorizon + (v0 - vCmd) * alpha
            else
                -- Player released movement keys to stop: velocity decays to 0.
                local stopTau: number = 0.10
                local alpha: number = stopTau * (1 - math.exp(-effHorizon / stopTau))
                return v0 * alpha
            end
        end
        local speed: number = motion.horizontal.Magnitude
        if speed < 1e-4 then
            return Vector3.zero
        end
        local direction: Vector3 = motion.horizontal.Unit
        -- Measured-acceleration path. Across the scored calibration replays
        -- this model beat constant velocity 57 to 19: MM2 targets accelerate,
        -- brake and strafe constantly, and the lateral component of the
        -- measured vector IS the turn, so no direction rotation is applied on
        -- top of it. The application clamp is tighter than the measurement
        -- clamp: the v61 failure came from a +120 swing stretching the lead,
        -- and 60 studs/s^2 already covers a humanoid reaching WalkSpeed.
        local accelMagnitude: number = math.sqrt(
            motion.accelX * motion.accelX + motion.accelZ * motion.accelZ
        )
        if accelMagnitude >= 4 then
            local applied: Vector3 = Vector3.new(
                motion.accelX,
                0,
                motion.accelZ
            )
            if accelMagnitude > 60 then
                applied *= 60 / accelMagnitude
            end
            local displacement: Vector3 = motion.horizontal * horizon
                + applied * (motion.fitLag * horizon
                    + 0.5 * horizon * horizon)
            if motion.brakeSeconds and horizon > motion.brakeSeconds then
                -- Already stopped by then: cap the along-track component at
                -- the stop distance and keep the lateral correction.
                local alongDistance: number = displacement:Dot(direction)
                local along: number = math.min(
                    0,
                    motion.longitudinalAccel
                )
                local speedNow: number = math.max(
                    0,
                    speed + along * motion.fitLag
                )
                local stopDistance: number = speedNow
                    * motion.brakeSeconds * 0.5
                if alongDistance > stopDistance then
                    displacement -= direction
                        * (alongDistance - stopDistance)
                end
            end
            return displacement
        end
        if math.abs(motion.turnRate) > 1e-3 then
            local angle: number = math.clamp(
                motion.turnRate * (horizon * 0.5 + motion.fitLag),
                -TURN_MAX_ANGLE,
                TURN_MAX_ANGLE
            )
            local cosine: number = math.cos(angle)
            local sine: number = math.sin(angle)
            direction = Vector3.new(
                direction.X * cosine + direction.Z * sine,
                0,
                -direction.X * sine + direction.Z * cosine
            )
        end
        -- The fit is a chord over the window, so it lags the current speed.
        -- Correct it by the measured longitudinal acceleration over that lag,
        -- and include the acceleration over the horizon itself. Both terms
        -- are clamped in measureTurn; a clean straight run leaves them at 0.
        -- Only deceleration earns a length correction. In the recorded v61
        -- session the measured "acceleration" of a running target swung to the
        -- +120 clamp and stretched the lead by 1.5 studs; a real stop is the
        -- case that matters and it is a negative value.
        local along: number = math.min(0, motion.longitudinalAccel)
        local speedNow: number = math.max(0, speed + along * motion.fitLag)
        local length: number = speedNow * horizon
            + 0.5 * along * horizon * horizon
        length = math.max(0, length)
        if motion.brakeSeconds and horizon > motion.brakeSeconds then
            -- The target is already stopped by then; extrapolating further
            -- would aim past it.
            length = math.min(
                length,
                speedNow * motion.brakeSeconds * 0.5
            )
        end
        return direction * length
    end

    local function predictedRoot(
        track: Track,
        motion: Motion,
        horizon: number
    ): Vector3
        local last: Sample = track.samples[#track.samples]
        local lead: Vector3 = leadHorizontal(motion, horizon)
        local x: number = last.position.X + lead.X
        local z: number = last.position.Z + lead.Z
        local y: number = last.position.Y
        if motion.verticalLead then
            y += motion.vertical * horizon
                - 0.5 * workspace.Gravity * horizon * horizon
            if y < last.position.Y then
                for idx: number = #track.samples, 1, -1 do
                    local s: Sample = track.samples[idx]
                    if s.grounded then
                        -- Only inside one jump height (v161): a body deeper
                        -- than that below the floor it left is falling, and
                        -- the clamp must not cancel the fall.
                        if s.position.Y - y <= MAX_FLOOR_LIFT_STUDS then
                            y = math.max(y, s.position.Y)
                        end
                        break
                    end
                end
            end
        end
        return Vector3.new(x, y, z)
    end

    local function currentRtt(): number
        local latency: any = nil
        -- Prefer the calibration source when both readings exist. v190
        -- removed the only reader of Data Ping (`StatsItem:GetValue`
        -- needs the Plugin capability, which Delta denies), so the
        -- second slot is empty and the engine reading is used until the
        -- measured echo takes over. The reading order is kept: taking
        -- the lower of two estimates under-led every shot in the v102
        -- session until echoes accumulated.
        if type(core.getLatencyReadings) == "function" then
            local ok: boolean, engine: any, data: any =
                pcall(core.getLatencyReadings)
            if ok and finiteNumber(data) then
                latency = data
            elseif ok and finiteNumber(engine) then
                latency = engine
            end
        end
        if not finiteNumber(latency)
            and type(core.getEstimatedLatency) == "function" then
            local ok: boolean, value: any = pcall(core.getEstimatedLatency)
            if ok then
                latency = value
            end
        end
        if not finiteNumber(latency) then
            latency = 0.08
        end
        return math.clamp(latency, 0.02, 0.6)
    end

    -- A median of recent readings, not the live number. A single spike in the
    -- engine estimate produced leads of 0.36-0.40 s against a 0.13 s baseline
    -- on 2026-10-18, and both of those shots missed by 7+ studs.
    local rttSamples: {number} = {}
    local lastRttSampleAt: number = -math.huge
    local function robustRtt(): number
        local now: number = os.clock()
        if now - lastRttSampleAt >= RTT_SAMPLE_INTERVAL then
            lastRttSampleAt = now
            table.insert(rttSamples, currentRtt())
            while #rttSamples > RTT_SAMPLES do
                table.remove(rttSamples, 1)
            end
        end
        local sorted: {number} = table.clone(rttSamples)
        table.insert(sorted, currentRtt())
        table.sort(sorted)
        return sorted[math.floor((#sorted + 1) / 2)]
    end

    local function pruneEchoes(now: number): ()
        if not finiteNumber(now) then
            return
        end
        while #echoes > 0 and now - echoTimes[1] > ECHO_MAX_AGE do
            table.remove(echoes, 1)
            table.remove(echoTimes, 1)
        end
    end

    local function echoMedian(): number?
        -- Expire on reads as well as writes. Otherwise the first shot after a
        -- quiet round uses a median from the previous connection/round; the
        -- v104/v105 files contain such shots after 72-2765 second gaps.
        pruneEchoes(os.clock())
        local count: number = #echoes
        if count < MIN_ECHO_SAMPLES then
            return nil
        end
        local sorted: {number} = table.clone(echoes)
        table.sort(sorted)
        local middle: number = (count + 1) / 2
        local lower: number = math.floor(middle)
        local upper: number = math.ceil(middle)
        return (sorted[lower] + sorted[upper]) / 2
    end

    local function noteEcho(sentAt: number, receivedAt: number): ()
        if not finiteNumber(sentAt) or not finiteNumber(receivedAt) then
            return
        end
        local delay: number = receivedAt - sentAt
        if delay < MIN_ECHO_DELAY or delay > MAX_ECHO_DELAY then
            return
        end
        -- A confirmation matched hundreds of milliseconds late is not
        -- transport: it is a stale pending paired with a later signal
        -- (in the v103 session, pairings at 416-1840 ms entered the
        -- median and dragged it from the true 106 ms to 532 ms, which
        -- led every shot ~0.4 s long and then tripped the capped-
        -- horizon refusal). Real round trips scale with the ping, so
        -- the acceptance window scales with the measured RTT instead
        -- of trusting the absolute 1.6 s pairing limit alone. v104 used
        -- 4x RTT (~540 ms at a 135 ms ping) and the A5AACF6D session
        -- still fed it 279-990 ms pairings that held the median at
        -- 210 ms against a 131-143 ms ping; 2.5x + 50 ms (~388 ms at
        -- that ping) closes that band while staying far above any
        -- genuine round trip on the same connection.
        local plausible: number = math.clamp(
            currentRtt() * 2.5 + 0.05, 0.3, MAX_ECHO_DELAY)
        if delay > plausible then
            return
        end
        -- Also prune against the arriving sample's clock. This keeps explicit
        -- replayed timestamps ordered and prevents one fresh echo from
        -- resurrecting samples that already expired.
        pruneEchoes(receivedAt)
        table.insert(echoes, delay)
        table.insert(echoTimes, receivedAt)
        while #echoes > MAX_ECHOES do
            table.remove(echoes, 1)
            table.remove(echoTimes, 1)
        end
        -- Auto-reset on divergence: if the surviving median no longer
        -- describes this connection's round trip, the window is
        -- poisoned; drop it and let the ping fallback carry the lead
        -- until fresh echoes rebuild the median.
        local median: number? = echoMedian()
        if median and median > currentRtt() * ECHO_DIVERGENCE_RATIO then
            table.clear(echoes)
            table.clear(echoTimes)
        end
    end

    local function manualLead(): number
        return settings.autoLead and 0 or settings.extraLead
    end

    -- Signed over/under-lead measurements, in seconds. Positive means the
    -- target was ahead of the point we aimed at when the server scored it.
    --
    -- v161: kept PER AIM MODE. Bullet and CFrame author different things (the
    -- native gun ray versus an explicit origin/endpoint pair) and there is no
    -- evidence the server scores them with the same rewind, so merging their
    -- errors taught one mode the other's bias. CFrame had no feedback at all
    -- until now.
    local leadErrors: {[string]: {number}} = {}
    local leadApplied: {[string]: number} = {}

    local function modeKey(mode: any): string
        local text: string = type(mode) == "string" and mode or ""
        local lower: string = string.lower(text)
        if lower == "normal" or lower == "bullet" then
            return "Bullet"
        end
        if lower == "silent" or lower == "cframe" or lower == "silent aim" then
            return "CFrame"
        end
        -- Kept from v179: a blade sample must not be averaged into the bucket a
        -- plain gun also uses, otherwise knife confirmations would silently
        -- retune every gun without a mode.
        if lower == "knife" or lower == "blade" then
            return "Knife"
        end
        return "default"
    end

    local function leadBucket(mode: any): {number}
        local key: string = modeKey(mode)
        local bucket: {number}? = leadErrors[key]
        if not bucket then
            bucket = {}
            leadErrors[key] = bucket
        end
        return bucket :: {number}
    end

    local function medianOf(values: {number}): number?
        local count: number = #values
        if count == 0 then
            return nil
        end
        local sorted: {number} = table.clone(values)
        table.sort(sorted)
        if count % 2 == 1 then
            return sorted[(count + 1) // 2]
        end
        return (sorted[count // 2] + sorted[count // 2 + 1]) * 0.5
    end

    local function noteLeadError(seconds: any, mode: any): ()
        if not finiteNumber(seconds) then
            return
        end
        if math.abs(seconds) > LEAD_FEEDBACK_SAMPLE_LIMIT then
            return
        end
        local key: string = modeKey(mode)
        local bucket: {number} = leadBucket(key)
        table.insert(bucket, seconds)
        while #bucket > LEAD_FEEDBACK_MAX_SAMPLES do
            table.remove(bucket, 1)
        end
        if #bucket < LEAD_FEEDBACK_MIN_SAMPLES then
            return
        end
        local median: number? = medianOf(bucket)
        if not median then
            return
        end
        local target: number = math.clamp(
            median,
            -MAX_LEAD_CORRECTION,
            MAX_LEAD_CORRECTION
        )
        -- Slew-limited: one step per confirmation, toward the measured median.
        local current: number = leadApplied[key] or 0
        local step: number = math.clamp(
            target - current,
            -LEAD_SLEW_PER_SAMPLE,
            LEAD_SLEW_PER_SAMPLE
        )
        leadApplied[key] = current + step
    end

    -- The correction currently in force for that aim mode, in seconds. It is
    -- already bounded and slew-limited, so callers just add it.
    local function leadCorrection(mode: any): number
        local bucket: {number}? = leadErrors[modeKey(mode)]
        if not bucket or #bucket < LEAD_FEEDBACK_MIN_SAMPLES then
            return 0
        end
        return leadApplied[modeKey(mode)] or 0
    end

    -- What the median asks for right now, before the slew limit. Reporting only;
    -- the aim uses `leadCorrection`.
    local function leadTarget(mode: any): number
        local bucket: {number}? = leadErrors[modeKey(mode)]
        if not bucket or #bucket < LEAD_FEEDBACK_MIN_SAMPLES then
            return 0
        end
        local median: number? = medianOf(bucket)
        if not median then
            return 0
        end
        return math.clamp(median, -MAX_LEAD_CORRECTION, MAX_LEAD_CORRECTION)
    end

    -- v180: the learned part is what turns a few matches into a working
    -- predictor, so it has to outlive the session. The calibration file keeps
    -- one entry per aim mode (samples, median, applied correction) and the next
    -- boot seeds it back through `seedLeadError` instead of restarting at zero.
    local function exportLeadState(): {[string]: any}
        local out: {[string]: any} = {}
        for key: string, bucket: {number} in pairs(leadErrors) do
            out[key] = {
                samples = #bucket,
                median = medianOf(bucket),
                applied = leadApplied[key] or 0,
            }
        end
        return out
    end

    -- Restore a bucket measured in an earlier session. The median is applied
    -- directly (not slew-limited): those samples were already earned shot by
    -- shot, so replaying them one step at a time would only slow the start.
    local function seedLeadError(mode: any, seconds: any, samples: any): ()
        if not finiteNumber(seconds) then
            return
        end
        local count: number = finiteNumber(samples) and math.floor(samples) or 0
        if count < LEAD_FEEDBACK_MIN_SAMPLES then
            return
        end
        local key: string = modeKey(mode)
        local clamped: number = math.clamp(seconds, -LEAD_FEEDBACK_SAMPLE_LIMIT, LEAD_FEEDBACK_SAMPLE_LIMIT)
        local bucket: {number} = leadBucket(key)
        table.clear(bucket)
        for _index: number = 1, math.min(count, LEAD_FEEDBACK_MAX_SAMPLES) do
            table.insert(bucket, clamped)
        end
        leadApplied[key] = math.clamp(clamped, -MAX_LEAD_CORRECTION, MAX_LEAD_CORRECTION)
    end

    local function leadSamples(mode: any): number
        local bucket: {number}? = leadErrors[modeKey(mode)]
        return bucket and #bucket or 0
    end

    -- How far the recent shots were off, without their sign. Silent aim draws
    -- its bypass segment along the target's own motion, and that segment has to
    -- be as long as the lead's uncertainty: at 16 studs/s a 60 ms error is a
    -- whole stud, which is wider than the blade and most of a limb. Derived
    -- from the same confirmations as the correction, never from a constant.
    local function leadUncertainty(mode: any): number
        local bucket: {number}? = leadErrors[modeKey(mode)]
        if not bucket or #bucket < LEAD_FEEDBACK_MIN_SAMPLES then
            return 0
        end
        local absolute: {number} = {}
        for _, seconds: number in ipairs(bucket) do
            table.insert(absolute, math.abs(seconds))
        end
        return medianOf(absolute) or 0
    end

    local function gunHorizon(mode: any): (number, number, number?)
        local rtt: number = robustRtt()
        local echo: number? = echoMedian()
        -- The echo is measured client-side from the activation to the
        -- confirmation, so it is a real round trip rather than an engine
        -- estimate; it replaces the ping when a median exists. It has to be
        -- measured from the confirmation path for that to happen at all:
        -- v61 recorded none (`echoDelay` nil on all 13 shots).
        -- Blend: rtt is the stable base, echo adds freshness when
        -- available. The calibration shows echo alone leads systematically
        -- short (echo mean 0.227 vs rtt 0.263), so a 60/40 blend avoids
        -- the jump between echo-present and echo-absent without losing
        -- the freshness signal. When echo is stale/nil, rtt stands alone.
        local pipeline: number = echo
            and (rtt * 0.6 + echo * 0.4)
            or rtt
        -- Full transport, NOT the one-way half -- and this is measured,
        -- not assumed: in the saved calibration sessions the confirmed
        -- hits cluster at about +14 ms against the confirmation instant
        -- (a full round trip after firing) and at about -55 ms against
        -- the receive instant, so the server scores the shot as if it
        -- rounded by the shooter's whole ping. v85 halved this on a
        -- no-rewind theory and led systematically short.
        --
        -- v161: the measured lead error is added here. Until now it was only
        -- recorded, so the loop that is supposed to correct a systematic
        -- short/long lead never acted on anything. It is per aim mode and
        -- already slew-limited, so this is a bounded nudge, not a new model.
        local horizon: number = pipeline
            + settings.interpolation
            + manualLead()
            + leadCorrection(mode)
        if settings.autoLead then
            horizon = math.min(horizon, MAX_AUTO_LEAD)
        end
        return math.clamp(horizon, 0.005, MAX_HORIZON), rtt, echo
    end
```

```
    -- The blade travels on the same connection as the bullet, so the measured
    -- FireServer -> GunFired round trip is a better transport number than the
    -- engine's ping estimate here too. The v64 file measured 126-472 ms of real
    -- round trip against an 80-154 ms engine reading: leading a thrown blade
    -- with the engine number leaves up to 300 ms of unled movement, which is
    -- the knife missing high and behind a target that is already running.
    local function knifeHorizon(): (number, number, number?)
        local rtt: number = robustRtt()
        local echo: number? = echoMedian()
        local pipeline: number = echo
            and (rtt * 0.6 + echo * 0.4)
            or rtt
        -- Same full-transport rule as the gun (measured the same way:
        -- the blade's effective spawn behaves like a full rewind, so
        -- the intercept leads the whole round trip plus flight time).
        -- The knife gets no measured feedback of its own (the captures only
        -- score the gun), so it shares the "default" bucket instead of
        -- borrowing the Bullet one.
        local horizon: number = pipeline
            + settings.interpolation
            + manualLead()
            + leadCorrection("knife")
        if settings.autoLead then
            horizon = math.min(horizon, MAX_AUTO_LEAD)
        end
        return math.clamp(horizon, 0.005, MAX_HORIZON), rtt, echo
    end

    local function gunAim(
        target: Player,
        _muzzle: Vector3,
        mode: any
    ): (Vector3?, AimInfo?, string?)
        -- v161: which way the shot will be authored. Bullet sends a CFrame
        -- whose LookVector is the aim; CFrame/Silent sends origin+endpoint.
        -- They keep separate measured corrections because there is no
        -- evidence the server rewinds both identically.
        local aimMode: string = modeKey(mode)
        -- v158: a fresh body check. A track survives the 0.5 s part refresh,
        -- so without this the solver kept leading a body that had just died.
        local humanoid: any = targetHumanoid(target)
        if humanoid ~= nil and not bodyIsAlive(humanoid) then
            tracks[target] = nil
            return nil, nil, "dead target"
        end
        local track: Track? = tracks[target]
        if not track then
            samplePlayer(target, os.clock())
            track = tracks[target]
        end
        if not track or #track.samples == 0 then
            return nil, nil, "no samples"
        end
        local now: number = os.clock()
        local motion: Motion? = estimateMotion(track, now)
        if not motion then
            return nil, nil, "stale sample"
        end
        local pipeline: number, rtt: number, echo: number? = gunHorizon(aimMode)
        local rttEngine: number?, rttData: number? = nil, nil
        if type(core.getLatencyReadings) == "function" then
            local ok: boolean, engine: any, data: any = pcall(core.getLatencyReadings)
            if ok then
                rttEngine = finiteNumber(engine) and engine or nil
                rttData = finiteNumber(data) and data or nil
            end
        end
        local horizon: number = math.clamp(
            motion.sampleAge + pipeline,
            0.005,
            MAX_HORIZON
        )
        local rootPosition: Vector3 = predictedRoot(track, motion, horizon)
        local last: Sample = track.samples[#track.samples]
        local aimVelocity: Vector3 = motion.velocity
        if motion.hasInputHistory and horizon > 1e-4 then
            local chord: Vector3 = rootPosition - last.position
            aimVelocity = Vector3.new(
                chord.X / horizon,
                motion.velocity.Y,
                chord.Z / horizon
            )
        end
        -- The torso offset turns with the character. The sampled offset is
        -- one sample old; the part's current offset from the current root
        -- is the same measurement without the sample delay, and at a
        -- 100 ms horizon a turning target has already rotated it.
        local aimOffset: Vector3 = last.aimOffset
        local freshAimPart: BasePart? = findAimPart(track.character)
        if freshAimPart then
            aimOffset = freshAimPart.Position - track.root.Position
        end
        local endpoint: Vector3 = rootPosition + aimOffset
        return endpoint, {
            endpoint = endpoint,
            targetPosition = endpoint,
            capturedPosition = last.position + aimOffset,
            rootPosition = rootPosition,
            velocity = aimVelocity,
            leadSeconds = horizon,
            sampleAge = motion.sampleAge,
            samples = motion.samples,
            residual = motion.residual,
            motionSource = motion.source,
            model = "kinematic-v51",
            rtt = rtt,
            rttEngine = rttEngine,
            rttData = rttData,
            echoDelay = echo,
            echoSamples = #echoes,
            turnRate = motion.turnRate,
            brakeSeconds = motion.brakeSeconds,
            leadCorrection = leadCorrection(aimMode),
            leadCorrectionTarget = leadTarget(aimMode),
            leadSamples = leadSamples(aimMode),
            aimMode = aimMode,
        }, nil
    end

    -- A historical "grounded" state is not always a real floor: the v162
    -- capture contains Landed/Running samples while a body was still falling.
    -- For the knife only, check the projected XZ against client geometry when
    -- the ballistic prediction crosses below the current body. This is NOT a
    -- visibility test and does not replace WallCheck. Unknown geometry leaves
    -- the existing prediction alone; a clear ray permits an actual fall.
    local KNIFE_FLOOR_DEPTH: number = 80
    local KNIFE_FLOOR_CACHE_SECONDS: number = 0.10
    local KNIFE_FLOOR_XZ_TOLERANCE: number = 0.35
    local knifeFloorParams: RaycastParams? = nil
    local knifeFloorParamsTried: boolean = false
    local knifeFloorFilter: {Instance} = {}

    local function knifeStandingHeight(track: Track): number?
        local humanoid: Humanoid = track.humanoid
        local root: BasePart = track.root
        if not humanoid or not root then return nil end
        local hip: number = humanoid.HipHeight
        local size: Vector3 = root.Size
        if not finiteNumber(hip) or not finiteVector(size) then return nil end
        local height: number = hip + size.Y * 0.5
        local rig: any = humanoid.RigType
        if rig == Enum.HumanoidRigType.R6 then
            local leg: Instance? = track.character:FindFirstChild("Left Leg")
                or track.character:FindFirstChild("Right Leg")
            if not leg or not leg:IsA("BasePart") then return nil end
            height += (leg :: BasePart).Size.Y
        elseif rig ~= Enum.HumanoidRigType.R15 then
            return nil
        end
        if not finiteNumber(height) or height < 0.5 or height > 10 then return nil end
        return height
    end

    local function makeKnifeFloorParams(): RaycastParams
        local params: RaycastParams = RaycastParams.new()
        params.FilterType = Enum.RaycastFilterType.Exclude
        params.RespectCanCollide = true
        params.IgnoreWater = true
        return params
    end

    local function castKnifeFloor(origin: Vector3, params: RaycastParams): RaycastResult?
        return workspace:Raycast(origin, Vector3.new(0, -KNIFE_FLOOR_DEPTH, 0), params)
    end

    local function knifeFloorAt(track: Track, projected: Vector3, now: number): (boolean, number?)
        -- Models/mocks without normal humanoid dimensions cannot provide a
        -- trustworthy root-to-floor offset; fall back to the old model.
        local okHeight: boolean, height: any = pcall(knifeStandingHeight, track)
        if not okHeight or not finiteNumber(height) then return false, nil end
        local last: Sample = track.samples[#track.samples]
        -- Begin above the body, not inside a low step at the projected XZ.
        -- A high obstruction is rejected below instead of becoming a floor.
        local topY: number = last.position.Y + height
        local cache: {any} = track.knifeFloorCache or {}
        track.knifeFloorCache = cache
        local newest: any = nil
        for _, probe: any in ipairs(cache) do
            local age: number = now - probe.at
            local dx: number = projected.X - probe.x
            local dz: number = projected.Z - probe.z
            if age >= 0 and age <= KNIFE_FLOOR_CACHE_SECONDS
                and dx * dx + dz * dz <= KNIFE_FLOOR_XZ_TOLERANCE ^ 2
                and math.abs(topY - probe.topY) <= 0.25
                and math.abs(height - probe.height) <= 0.05
                and (probe.part == nil or (probe.part.Parent ~= nil
                    and (probe.partFrame == nil or (probe.part.CFrame == probe.partFrame
                        and probe.part.CanCollide == probe.partCanCollide))
                    and (probe.partSize == nil or probe.part.Size == probe.partSize)))
                and (not newest or probe.serial > newest.serial) then
                newest = probe
            end
        end
        if newest then return newest.known, newest.supportY end
        if not knifeFloorParamsTried then
            knifeFloorParamsTried = true
            local okParams: boolean, params: any = pcall(makeKnifeFloorParams)
            if okParams then knifeFloorParams = params end
        end
        local params: RaycastParams? = knifeFloorParams
        if not params then return false, nil end
        table.clear(knifeFloorFilter)
        if LocalPlayer.Character then table.insert(knifeFloorFilter, LocalPlayer.Character) end
        table.insert(knifeFloorFilter, track.character)
        params.FilterDescendantsInstances = knifeFloorFilter
        local from: Vector3 = Vector3.new(projected.X, topY, projected.Z)
        local okRay: boolean, hit: any = pcall(castKnifeFloor, from, params)
        local known: boolean = okRay
        local supportY: number? = nil
        local part: Instance? = nil
        local partFrame: CFrame? = nil
        local partSize: Vector3? = nil
        local partCanCollide: boolean? = nil
        if okRay and hit then
            part = hit.Instance
            -- Do not treat a character, a non-collidable part, or a steep
            -- surface as evidence of a floor. Do not raycast repeatedly through
            -- such objects; uncertainty falls back to the previous model.
            local okHit: boolean, valid: boolean = pcall(function(): boolean
                if not part or not finiteVector(hit.Position) or not finiteVector(hit.Normal)
                    or hit.Normal.Y < 0.65 then return false end
                if part:IsA("BasePart") then
                    partFrame = (part :: BasePart).CFrame
                    partSize = (part :: BasePart).Size
                    partCanCollide = (part :: BasePart).CanCollide
                    if not partCanCollide then return false end
                end
                local model: Model? = part:FindFirstAncestorOfClass("Model")
                if model and model:FindFirstChildOfClass("Humanoid") then return false end
                return true
            end)
            if not okHit or not valid then
                known = false
            else
                local candidate: number = hit.Position.Y + height
                -- A roof/obstacle at the projected XZ must not lift the aim
                -- above the body that is actually visible right now.
                if candidate > last.position.Y + 0.75 then
                    known = false
                else
                    supportY = math.min(candidate, last.position.Y)
                end
            end
        end
        local index: number = track.knifeFloorNext or 1
        local probe: any = cache[index] or {}
        track.knifeFloorSerial = (track.knifeFloorSerial or 0) + 1
        probe.serial = track.knifeFloorSerial
        probe.at, probe.x, probe.z, probe.topY = now, projected.X, projected.Z, topY
        probe.height, probe.known, probe.supportY, probe.part = height, known, supportY, part
        probe.partFrame, probe.partSize, probe.partCanCollide = partFrame, partSize, partCanCollide
        cache[index] = probe
        track.knifeFloorNext = index == 1 and 2 or 1
        return known, supportY
    end

    local function knifeRootAt(
        track: Track, motion: Motion, span: number, floorKnown: boolean, floorY: number?
    ): Vector3
        if floorKnown and motion.verticalLead then
            -- Ground is observed at the projected XZ, so do not reuse a
            -- possibly stale historical "grounded" height from another ledge.
            local last: Sample = track.samples[#track.samples]
            local lead: Vector3 = leadHorizontal(motion, span)
            local y: number = last.position.Y + motion.vertical * span
                - 0.5 * workspace.Gravity * span * span
            if floorY then y = math.max(y, floorY) end
            return Vector3.new(last.position.X + lead.X, y, last.position.Z + lead.Z)
        end
        return predictedRoot(track, motion, span)
    end

    local function knifeFloorIntercept(
        track: Track, motion: Motion, horizon: number, flight: number,
        origin: Vector3, speed: number, floorY: number
    ): (Vector3, number)
        local last: Sample = track.samples[#track.samples]
        local root: Vector3 = knifeRootAt(track, motion, horizon + flight, true, floorY)
        for _ = 1, 5 do
            local revised: number = (root + last.aimOffset - origin).Magnitude / speed
            local delta: number = math.abs(revised - flight)
            flight = revised
            root = knifeRootAt(track, motion, horizon + flight, true, floorY)
            if delta <= 1e-4 then break end
        end
        return root, (root + last.aimOffset - origin).Magnitude / speed
    end

    local function knifeAirborneIntercept(
        track: Track, motion: Motion, horizon: number, origin: Vector3,
        speed: number, floorKnown: boolean
    ): (Vector3?, number?)
        local last: Sample = track.samples[#track.samples]
        local lowTime: number = 0
        local highTime: number = 5 -- the same bound as interceptTime
        if motion.verticalLead and workspace.Gravity > 0 then
            -- Even with zero horizontal distance, the blade cannot intercept
            -- after the target falls faster below it than speed * time. This
            -- analytic bound keeps the numerical search short. The historical
            -- clamp can raise at most six studs when geometry is unknown.
            local gravity: number = workspace.Gravity
            local allowance: number = floorKnown and 0 or MAX_FLOOR_LIFT_STUDS
            local c: number = last.position.Y + last.aimOffset.Y - origin.Y
                + motion.vertical * horizon - 0.5 * gravity * horizon * horizon + allowance
            local b: number = motion.vertical - gravity * horizon + speed
            local discriminant: number = b * b + 2 * gravity * c
            if not finiteNumber(discriminant) or discriminant < 0 then return nil, nil end
            local root: number = math.sqrt(discriminant)
            lowTime = math.max(0, (b - root) / gravity)
            highTime = math.min(highTime, (b + root) / gravity)
        end
        if highTime < lowTime or highTime <= 0 then return nil, nil end
        local segments: number = math.clamp(math.ceil((highTime - lowTime) / 0.025), 1, 64)
        local previous: number = lowTime
        local previousRoot: Vector3 = knifeRootAt(track, motion, horizon + previous, floorKnown, nil)
        local previousGap: number = (previousRoot + last.aimOffset - origin).Magnitude - speed * previous
        if previousGap <= 1e-6 then
            local actual: number = (previousRoot + last.aimOffset - origin).Magnitude / speed
            if math.abs(actual - previous) > 0.002 then return nil, nil end
            return previousRoot, actual
        end
        for index: number = 1, segments do
            local time: number = lowTime + (highTime - lowTime) * index / segments
            local root: Vector3 = knifeRootAt(track, motion, horizon + time, floorKnown, nil)
            local gap: number = (root + last.aimOffset - origin).Magnitude - speed * time
            if gap <= 0 then
                local left: number, right: number = previous, time
                -- The earliest bracketed intercept; no assumed hitbox or
                -- automatic increase to the lead horizon.
                for _ = 1, 12 do
                    local middle: number = (left + right) * 0.5
                    local midRoot: Vector3 = knifeRootAt(track, motion, horizon + middle, floorKnown, nil)
                    local midGap: number = (midRoot + last.aimOffset - origin).Magnitude - speed * middle
                    if midGap <= 0 then right = middle else left = middle end
                end
                local solved: Vector3 = knifeRootAt(track, motion, horizon + right, floorKnown, nil)
                local solvedFlight: number = (solved + last.aimOffset - origin).Magnitude / speed
                -- Reject a discontinuous historical-floor jump rather than
                -- presenting it as a physically consistent intercept.
                if math.abs(solvedFlight - right) > 0.002 then return nil, nil end
                return solved, solvedFlight
            end
            previous = time
        end
        return nil, nil
    end

    local function knifeAim(
        target: Player,
        origin: Vector3,
        speed: number?
    ): (Vector3?, AimInfo?, string?)
        local humanoid: any = targetHumanoid(target)
        if humanoid ~= nil and not bodyIsAlive(humanoid) then
            tracks[target] = nil
            return nil, nil, "dead target"
        end
        local track: Track? = tracks[target]
        if track and track.character ~= target.Character then
            -- Respawn can precede the next sampling heartbeat.
            tracks[target] = nil
            track = nil
        end
        if not track then
            samplePlayer(target, os.clock())
            track = tracks[target]
        end
        if not track or #track.samples == 0 then
            return nil, nil, "no samples"
        end
        local now: number = os.clock()
        local motion: Motion? = estimateMotion(track, now)
        if not motion then
            return nil, nil, "stale sample"
        end
        local resolvedSpeed: number = tonumber(speed) or DEFAULT_KNIFE_SPEED
        if resolvedSpeed < 24 or resolvedSpeed > 300 then
            resolvedSpeed = DEFAULT_KNIFE_SPEED
        end
        local pipeline: number, rtt: number, echo: number? = knifeHorizon()
        local rttEngine: number?, rttData: number? = nil, nil
        if type(core.getLatencyReadings) == "function" then
            local ok: boolean, engine: any, data: any = pcall(core.getLatencyReadings)
            if ok then
                rttEngine = finiteNumber(engine) and engine or nil
                rttData = finiteNumber(data) and data or nil
            end
        end
        local horizon: number = math.clamp(
            motion.sampleAge + pipeline,
            0.005,
            MAX_HORIZON
        )
        local last: Sample = track.samples[#track.samples]
        local aimStart: Vector3 = last.position + last.aimOffset
        local initial: Vector3 = motion.velocity
        local flight: number? = predictionMath.interceptTime(
            aimStart + initial * horizon - origin,
            initial,
            resolvedSpeed
        )
        local horizontalSeed: boolean = false
        if not flight and motion.airborne and motion.vertical < 0 then
            -- A fast descending body can be slower than the blade after it
            -- lands. Use a horizontal seed ONLY if a real support probe below
            -- later confirms that landing; otherwise keep "unreachable".
            local lateral: Vector3 = Vector3.new(initial.X, 0, initial.Z)
            flight = predictionMath.interceptTime(
                aimStart + lateral * horizon - origin, lateral, resolvedSpeed
            )
            horizontalSeed = flight ~= nil
        end
        if not flight then
            return nil, nil, "unreachable"
        end
        -- v159: solve the intercept to a FIXED POINT instead of one re-solve.
        -- The chord a turning or braking target travels depends on how long
        -- the blade is in the air, and that flight time depends on the chord,
        -- so a single pass leaves the aim inconsistent: the point and the
        -- flight time it was solved for disagree. Iterating until the flight
        -- time stops moving makes |endpoint - origin| == speed * flight exact,
        -- which is the one property a straight constant-speed blade has to
        -- satisfy. It converges in two or three passes; the cap is a guard.
        local span: number = horizon + flight
        local lead: Vector3 = leadHorizontal(motion, span)
        local vertical: number = motion.verticalLead and motion.vertical or 0
        if motion.verticalLead then
            vertical -= 0.5 * workspace.Gravity * span
        end
        local effective: Vector3 = Vector3.new(
            lead.X,
            vertical * span,
            lead.Z
        ) / span
        for _iteration: number = 1, 4 do
            local revised: number? = predictionMath.interceptTime(
                aimStart + effective * horizon - origin,
                effective,
                resolvedSpeed
            )
            if not revised then
                break
            end
            local delta: number = math.abs(revised - flight)
            flight = revised
            if delta <= 1e-4 then
                -- `effective` is still the velocity this flight was solved
                -- with, so the endpoint below sits exactly on the intercept.
                break
            end
            span = horizon + flight
            lead = leadHorizontal(motion, span)
            vertical = motion.verticalLead and motion.vertical or 0
            if motion.verticalLead then
                vertical -= 0.5 * workspace.Gravity * span
            end
            effective = Vector3.new(lead.X, vertical * span, lead.Z) / span
        end
        local endpoint: Vector3 = aimStart + effective * (horizon + flight)
        if motion.airborne then
            -- A straight blade aims at the body's projected impact position,
            -- not at an arc for the blade itself. The old fixed-point pass
            -- could diverge: each additional fall increased the flight time,
            -- which then made the prediction fall even farther.
            -- The old ballistic iteration can already have diverged by the
            -- time we ask for a floor. Probe near a horizontal intercept, not
            -- dozens of studs along that unreliable projected XZ.
            local lateral: Vector3 = Vector3.new(initial.X, 0, initial.Z)
            local lateralFlight: number? = predictionMath.interceptTime(
                aimStart + lateral * horizon - origin, lateral, resolvedSpeed
            )
            local probeFlight: number = lateralFlight or flight
            local spanAtProbe: number = horizon + probeFlight
            local probePoint: Vector3 = predictedRoot(track, motion, spanAtProbe)
            local possibleSpan: number = horizon + math.max(probeFlight, flight)
            local rawY: number = last.position.Y + motion.vertical * possibleSpan
                - 0.5 * workspace.Gravity * possibleSpan * possibleSpan
            local known: boolean, floorY: number? = false, nil
            local probes: number = 0
            if motion.verticalLead and rawY < last.position.Y - 0.25 then
                known, floorY = knifeFloorAt(track, probePoint, now)
                probes = 1
            end

            local solvedRoot: Vector3? = nil
            local solvedFlight: number? = nil
            if known and floorY then
                solvedRoot, solvedFlight = knifeFloorIntercept(
                    track, motion, horizon, probeFlight, origin, resolvedSpeed, floorY
                )
                local dx: number = (solvedRoot :: Vector3).X - probePoint.X
                local dz: number = (solvedRoot :: Vector3).Z - probePoint.Z
                if dx * dx + dz * dz > KNIFE_FLOOR_XZ_TOLERANCE ^ 2 then
                    -- The corrected flight can move a landing across an edge.
                    -- Recheck once at the new XZ, never borrow the old floor.
                    probePoint = solvedRoot :: Vector3
                    known, floorY = knifeFloorAt(track, probePoint, now)
                    probes += 1
                end
            end

            if not solvedRoot then
                if horizontalSeed then return nil, nil, "unreachable" end
                -- A clear downward ray means a genuine fall, not the previous
                -- platform's height. If geometry is unknown, retain the old
                -- bounded history model. Accept a cheap fixed-point result if
                -- already consistent; otherwise bracket the earliest possible
                -- intercept in a bounded search or fall back safely.
                local first: Vector3 = knifeRootAt(track, motion, horizon + flight, known, nil)
                local actual: number = (first + last.aimOffset - origin).Magnitude / resolvedSpeed
                if finiteNumber(actual) and actual <= 5 and math.abs(actual - flight) <= 0.002 then
                    solvedRoot, solvedFlight = first, actual
                else
                    solvedRoot, solvedFlight = knifeAirborneIntercept(
                        track, motion, horizon, origin, resolvedSpeed, known
                    )
                end
                if not solvedRoot or not solvedFlight then return nil, nil, "unreachable" end
                if probes > 0 and known then
                    local dx: number = solvedRoot.X - probePoint.X
                    local dz: number = solvedRoot.Z - probePoint.Z
                    if dx * dx + dz * dz > KNIFE_FLOOR_XZ_TOLERANCE ^ 2 then
                        if probes >= 2 then return nil, nil, "uncertain landing" end
                        probePoint = solvedRoot
                        known, floorY = knifeFloorAt(track, probePoint, now)
                        probes += 1
                        if not known then return nil, nil, "uncertain landing" end
                        if floorY then
                            solvedRoot, solvedFlight = knifeFloorIntercept(
                                track, motion, horizon, solvedFlight, origin, resolvedSpeed, floorY
                            )
                            dx = solvedRoot.X - probePoint.X
                            dz = solvedRoot.Z - probePoint.Z
                            if dx * dx + dz * dz > KNIFE_FLOOR_XZ_TOLERANCE ^ 2 then
                                return nil, nil, "uncertain landing"
                            end
                        end
                    end
                end
            end
            if not solvedRoot or not solvedFlight or not finiteVector(solvedRoot)
                or not finiteNumber(solvedFlight) or solvedFlight > 5 then
                return nil, nil, "unreachable"
            end
            local consistentRoot: Vector3 = knifeRootAt(
                track, motion, horizon + solvedFlight, known, floorY
            )
            if (consistentRoot - solvedRoot).Magnitude > 0.25 then
                return nil, nil, "uncertain intercept"
            end
            if horizontalSeed then
                local correctedSpan: number = horizon + solvedFlight
                local unboundedY: number = last.position.Y + motion.vertical * correctedSpan
                    - 0.5 * workspace.Gravity * correctedSpan * correctedSpan
                if not floorY or unboundedY > floorY then return nil, nil, "unreachable" end
            end
            endpoint = solvedRoot + last.aimOffset
            -- The score/telemetry must describe the point actually returned.
            flight = (endpoint - origin).Magnitude / resolvedSpeed
        end
        return endpoint, {
            endpoint = endpoint,
            targetPosition = endpoint,
            capturedPosition = aimStart,
            rootPosition = endpoint - last.aimOffset,
            velocity = motion.velocity,
            leadSeconds = horizon,
            flightSeconds = flight,
            sampleAge = motion.sampleAge,
            samples = motion.samples,
            residual = motion.residual,
            motionSource = motion.source,
            model = "kinematic-v164-knife",
            rtt = rtt,
            rttEngine = rttEngine,
            rttData = rttData,
            echoDelay = echo,
            echoSamples = #echoes,
            turnRate = motion.turnRate,
            brakeSeconds = motion.brakeSeconds,
            leadCorrection = 0,
        }, nil
    end

    local function playerStatus(target: Player?): string
        local rtt: number = robustRtt()
        local echo: number? = echoMedian()
        local parts: {string} = {
            string.format("RTT %.0f ms", rtt * 1000),
        }
        if echo then
            table.insert(
                parts,
                string.format("echo %.0f ms (%d)", echo * 1000, #echoes)
            )
        end
        table.insert(
            parts,
            string.format("interp %.0f ms", settings.interpolation * 1000)
        )
        if settings.autoLead then
            table.insert(
                parts,
                echo and "auto lead (echo)" or "auto lead (ping)"
            )
        elseif settings.extraLead ~= 0 then
            table.insert(
                parts,
                string.format("manual %+.0f ms", settings.extraLead * 1000)
            )
        end
        local horizon: number = (echo or rtt)
            + settings.interpolation
            + manualLead()
        -- The status must show the horizon the shot actually uses:
        -- gunAim adds the age of the newest sample on top of the
        -- pipeline. Showing only the pipeline (98 ms when the shot
        -- used 111 ms) makes the lead look shorter than it is.
        if target then
            local track: Track? = tracks[target]
            local preview: Motion? = track and estimateMotion(track, os.clock())
            if preview then
                horizon += preview.sampleAge
            end
        end
        local capped: boolean = false
        if settings.autoLead and horizon > MAX_AUTO_LEAD then
            horizon = MAX_AUTO_LEAD
            capped = true
        end
        if horizon > MAX_HORIZON then
            horizon = MAX_HORIZON
            capped = true
        end
        table.insert(
            parts,
            string.format(
                "lead %.0f ms%s",
                horizon * 1000,
                capped and " (capped)" or ""
            )
        )
        if target then
            local track: Track? = tracks[target]
            local motion: Motion? = track and estimateMotion(track, os.clock())
            if motion then
                table.insert(parts, string.format("%.1f studs/s", motion.horizontal.Magnitude))
                table.insert(parts, string.format("%d samples", motion.samples))
                table.insert(parts, string.format("age %.0f ms", motion.sampleAge * 1000))
                if motion.airborne then
```

```
                table.insert(parts, "airborne")
            end
            if math.abs(motion.turnRate) > 0.05 then
                table.insert(
                    parts,
                    string.format("turn %+.0f deg/s", math.deg(motion.turnRate))
                )
            end
            if motion.brakeSeconds then
                table.insert(
                    parts,
                    string.format("brake %.2f s", motion.brakeSeconds)
                )
            end
        else
            table.insert(parts, "no fresh samples")
        end
    end
    return table.concat(parts, " · ")
end

local controller: any = {
    settings = settings,
    noteEcho = noteEcho,
    -- The movement window that ends at the call, oldest first, with `at`
    -- in seconds relative to now (negative). Telemetry attaches it to each
    -- shot so an alternative predictor can be replayed offline from the
    -- same inputs instead of being judged only on live hits.
    getHistory = function(
        player: Player,
        seconds: number?
    ): ({any}?, number?)
        local track: Track? = tracks[player]
        if not track or #track.samples < 2 then
            return nil, 0
        end
        local span: number = math.clamp(
            seconds or HISTORY_SECONDS,
            0.05,
            HISTORY_SECONDS
        )
        -- Times are relative to the newest sample, not to this call. The
        -- modules do not share a clock epoch (the v61 log recorded a
        -- -2546 s offset), so only the predictor's own timestamps can be
        -- subtracted from each other. The caller gets the age separately.
        local newest: Sample = track.samples[#track.samples]
        local age: number = math.max(0, os.clock() - newest.at)
        local points: {any} = {}
        for _, sample: Sample in ipairs(track.samples) do
            local at: number = sample.at - newest.at
            if at >= -span - 1e-6 then
                table.insert(points, {
                    at = at,
                    position = sample.position,
                    velocity = sample.velocity,
                    state = sample.state,
                    grounded = sample.grounded,
                    moveDirection = sample.moveDirection,
                    -- v139.4: the offset from the root to the part the aim
                    -- falls on. base.lua reads it for every history point
                    -- and it was never sent, so `endpointErrorStuds` scored
                    -- a ROOT position against an aim that used the torso
                    -- offset: a systematic vertical error on every sample.
                    aimOffset = sample.aimOffset,
                    aimPart = sample.aimPart,
                })
            end
        end
        if #points < 2 then
            return nil, age
        end
        return points, age
    end,
    gunAim = gunAim,
    knifeAim = knifeAim,
    status = playerStatus,
    getRtt = currentRtt,
    getEchoDelay = echoMedian,
    predict = function(player: Player, horizon: number): Vector3?
        local track: Track? = tracks[player]
        if not track or #track.samples == 0 or not finiteNumber(horizon) then
            return nil
        end
        local motion: Motion? = estimateMotion(track, os.clock())
        if not motion then
            return nil
        end
        return predictedRoot(track, motion, math.clamp(horizon, 0, 2))
    end,
    getMotion = function(player: Player): Motion?
        local track: Track? = tracks[player]
        if not track then
            return nil
        end
        return estimateMotion(track, os.clock())
    end,
    noteLeadError = noteLeadError,
    exportLeadState = exportLeadState,
    seedLeadError = seedLeadError,
    getLeadCorrection = leadCorrection,
    getLeadCorrectionTarget = leadTarget,
    getLeadUncertainty = leadUncertainty,
    getLeadSamples = leadSamples,
    reset = function(): ()
        table.clear(echoes)
        table.clear(echoTimes)
        table.clear(tracks)
        table.clear(rttSamples)
        for _, bucket: {number} in pairs(leadErrors) do
            table.clear(bucket)
        end
        table.clear(leadErrors)
        table.clear(leadApplied)
        lastRttSampleAt = -math.huge
    end,
}

-- v139: the roster changes a few times per round, not 60 times per second.
-- `GetPlayers()` returns a fresh array on every call, so asking for it each
-- frame was pure garbage. Reused for a quarter of a second.
local roster: {Player} = {}
local rosterAt: number = -math.huge
local function tick(deltaTime: number): ()
    elapsed += math.max(0, deltaTime or 0)
    local interval: number = 1 / SAMPLE_HZ
    if elapsed + 1e-9 < interval then
        return
    end
    elapsed = elapsed % interval
    local now: number = os.clock()
    if now - rosterAt >= 0.25 or #roster == 0 then
        rosterAt = now
        roster = Players:GetPlayers()
    end
    for _, player: Player in ipairs(roster) do
        if player ~= LocalPlayer then
            samplePlayer(player, now)
        end
    end
end

state.mm2Prediction = controller
featureConnections.MM2PredictionSampler =
    TaskManager:Connect(tick, nil, "Prediction sampler")

activeCleanup = function(): ()
    disconnectFeatureConnection("MM2PredictionSampler")
    if state.mm2Prediction == controller then
        state.mm2Prediction = nil
    end
    table.clear(echoes)
    table.clear(echoTimes)
    table.clear(tracks)
end

Module.Events = featureConnections
Module.Initialized = true
return Module
end

function Module.destroy(): ()
    if not Module.Initialized then
        return
    end
    Module.Initialized = false
    pcall(activeCleanup)
    activeCleanup = function(): () end
    Module.Events = {}
    Module.Runtime = nil
end

return Module

-------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/MM2/Prediction.lua
-------------------------------------------------------------------------------

-------------------------------------------------------------------------------
ARCHIVO: src/games/MM2/Render/CoinEsp.lua
BYTES: 27986
SHA-256: bcf5b5cb5cca918571d61e11871abb44f52364047e0a70c09c8962bfdf43115d
INICIO DEL ARCHIVO
-------------------------------------------------------------------------------
-- MM2 Coin ESP: highlights ONLY the real pickups the game itself spawns.
--
-- A real pickup carries three exact signatures from the game's own code,
-- and this module accepts a part when ANY of them matches -- never a
-- generic name search:
--
--   1. The CollectionService tag "CoinVisual". The game's CoinVisualizer
--      script animates exactly GetTagged("CoinVisual"), skips the parts
--      whose "Collected" attribute is true and destroys the "Delete" ones.
--   2. The "CoinID" attribute. The same script keys every pickup by
--      GetAttribute("CoinID") when CoinCollected fires, so only a real
--      pickup carries it.
--   3. The exact spawn structure from the place: an invisible server part
--      named Coin_Server whose visible child is named CoinVisual, with the
--      mesh children (MainCoin / DecalPart / 2Part, or the event look of
--      the season) under it.
--
-- The highlight always sits on the visible CoinVisual part itself -- never
-- on a parent model or container. (An earlier version walked up to the
-- container model and painted the whole map yellow, and counted every
-- part with "coin" in its name; that detection is gone.)
--
-- The family of a pickup (Coin, Candy, SnowToken, BeachBall, Egg, Heart)
-- is read from its own mesh children; when the meshes say nothing
-- specific, the family the server announced applies, because a round only
-- pays one kind of pickup. Candies replace coins during Halloween with
-- this same spawn system ([1](https://murder-mystery-2.fandom.com/wiki/Halloween_Event_2025)),
-- so a candy round is found by the same signatures. The server family
-- comes from the Remotes/Gameplay/CoinsStarted bag table (Coin, Candy,
-- SnowToken, BeachBall, Egg) and the Workspace event attributes.
local Module = {
    Name = "MM2 Coin ESP",
    Events = {} :: {[string]: any},
    Initialized = false,
    Runtime = nil :: any,
}

local activeCleanup: () -> () = function(): () end

-- The exact signatures of a real pickup.
local COIN_TAG: string = "CoinVisual"
local VISUAL_NAME: string = "CoinVisual"
local SERVER_NAME: string = "Coin_Server"

local FAMILY_COIN: string = "Coin"
local FAMILY_CANDY: string = "Candy"
local FAMILY_SNOW: string = "SnowToken"
local FAMILY_BEACH: string = "BeachBall"
local FAMILY_EGG: string = "Egg"
local FAMILY_HEART: string = "Heart"
local FAMILY_ALL: string = "All"
local FAMILY_AUTO: string = "Auto (server)"

-- Most specific first: a SnowToken can carry a mesh named Coin.
local FAMILY_ORDER: {string} = {
    FAMILY_EGG,
    FAMILY_BEACH,
    FAMILY_SNOW,
    FAMILY_CANDY,
    FAMILY_HEART,
    FAMILY_COIN,
}

local COUNT_ORDER: {string} = {
    FAMILY_COIN,
    FAMILY_CANDY,
    FAMILY_SNOW,
    FAMILY_BEACH,
    FAMILY_EGG,
    FAMILY_HEART,
}

local FAMILY_NOUNS: {[string]: {string}} = {
    [FAMILY_COIN] = {"coin", "coins"},
    [FAMILY_CANDY] = {"candy", "candies"},
    [FAMILY_SNOW] = {"snowtoken", "snowtokens"},
    [FAMILY_BEACH] = {"beachball", "beachballs"},
    [FAMILY_EGG] = {"egg", "eggs"},
    [FAMILY_HEART] = {"heart", "hearts"},
}

local FAMILY_CHOICES: {string} = {
    FAMILY_AUTO,
    FAMILY_COIN,
    FAMILY_CANDY,
    FAMILY_SNOW,
    FAMILY_BEACH,
    FAMILY_EGG,
    FAMILY_HEART,
    FAMILY_ALL,
}

local BAG_TO_FAMILY: {[string]: string} = {
    Coin = FAMILY_COIN,
    Candy = FAMILY_CANDY,
    SnowToken = FAMILY_SNOW,
    BeachBall = FAMILY_BEACH,
    Egg = FAMILY_EGG,
}

-- Words read ONLY from the pickup's own mesh children (never from
-- parents: a container name must not reclassify every coin inside).
-- The tagged part itself is always named CoinVisual, whatever family it
-- pays, so that name must never classify a part as a plain coin either.
local FAMILY_PHRASES: {[string]: {string}} = {
    [FAMILY_EGG] = {"egg", "eggs"},
    [FAMILY_BEACH] = {"beachball", "beach ball", "beachballs", "beach balls"},
    [FAMILY_SNOW] = {"snowtoken", "snow token", "snowtokens", "snow tokens", "snowball", "snow ball"},
    [FAMILY_CANDY] = {"candy", "candies", "candied", "candycorn", "candy corn"},
    [FAMILY_HEART] = {"heart", "hearts"},
    [FAMILY_COIN] = {"maincoin", "coinold", "decalpart"},
}

-- Safety-net rescan; the signals do the real work.
local RESCAN_PERIOD: number = 2

local function lower(value: string): string
    return string.lower(value)
end

local function contains(value: string, needle: string): boolean
    return string.find(value, needle, 1, true) ~= nil
end

function Module.init(runtime: any): any
    if Module.Initialized then
        return Module
    end
    local core: any = state.mm2Core
    assert(type(core) == "table", "MM2 Coin ESP requires the MM2 core module")
    Module.Runtime = runtime
    local CoinEffects: any = core.CoinEffects
    local clearEffects: any = core.clearEffects
    local mm2Settings: any = core.mm2Settings
    local collectionService: any = game:GetService("CollectionService")

    local settings = {
        family = FAMILY_AUTO,
    }
    -- Set by the `CoinsStarted` remote and by the Workspace event attributes.
    local serverFamily: string? = nil
    -- Raw signature counts from the last scan, for the report action.
    local lastRaw = {tagged = 0, coinId = 0, named = 0}

    -- Native per-pickup marker: ONE small BoxHandleAdornment per
    -- pickup, sized exactly to the visible coin mesh. No labels and
    -- no per-frame Lua: a hundred markers cost the engine one
    -- adornment each and nothing else. The adornment renders through
    -- walls regardless of the holder's transparency.
    local coinTags: {[Instance]: {box: Instance}} = {}
    -- Pickup -> family, kept between scans for the tag text.
    local lastFamilies: {[Instance]: string} = {}
    -- Parts currently carrying a tag, for tests and the report action.
    state.mm2CoinTagParts = {}

    local function clearTags(): ()
        for part: Instance, entry in pairs(coinTags) do
            coinTags[part] = nil
            pcall(function(): ()
                entry.box:Destroy()
            end)
        end
        table.clear(lastFamilies)
        state.mm2CoinTagParts = {}
    end

    local function attribute(object: Instance, name: string): any
        return object:GetAttribute(name)
    end

    local function hasTag(object: Instance): boolean
        return collectionService:HasTag(object, COIN_TAG)
    end

    -- The visible half of a pickup. A server part resolves to its
    -- CoinVisual child; anything else is used as-is. Parents are NEVER
    -- walked: the container must stay dark.
    local function visualOf(part: Instance): Instance?
        if part.Name == SERVER_NAME then
            local child: Instance? = part:FindFirstChild(VISUAL_NAME)
            if child and child:IsA("BasePart") then
                return child
            end
            return nil
        end
        return part
    end

    -- The volatile Collected/Delete flags are checked by passesFiltersCached;
    -- this static half rejects anything outside the map or inside a character.
    local function passesFilters(part: Instance): boolean
        if not part:IsA("BasePart") or not part:IsDescendantOf(workspace) then
            return false
        end
        local current: Instance? = part.Parent
        while current and current ~= workspace do
            if current:FindFirstChildOfClass("Humanoid") then
                return false
            end
            current = current.Parent
        end
        return true
    end

    -- The family the server says is in play: the `CoinsStarted` table first,
    -- then the Workspace event attributes.
    local function familyFromWorkspace(): string?
        local eventName: string = lower(tostring(attribute(workspace, "CurrentEvent") or ""))
        if contains(eventName, "halloween") then
            return FAMILY_CANDY
        elseif contains(eventName, "christmas") then
            return FAMILY_SNOW
        elseif contains(eventName, "summer") then
            return FAMILY_BEACH
        elseif contains(eventName, "easter") then
            return FAMILY_EGG
        elseif contains(eventName, "valentine") then
            return FAMILY_HEART
        end
        for attrName: string, family: string in pairs({
            isValentines = FAMILY_HEART,
            isChristmas = FAMILY_SNOW,
            isChristmasEnding = FAMILY_SNOW,
            IsSummer = FAMILY_BEACH,
            IsEaster = FAMILY_EGG,
        }) do
            if attribute(workspace, attrName) == true then
                return family
            end
        end
        return nil
    end

    local function activeFamily(): string?
        return serverFamily or familyFromWorkspace()
    end

    -- Family of one pickup: its mesh children only. With no specific
    -- word, the round's family applies (a round pays a single kind of
    -- pickup), falling back to the classic coin.
    local familyCache: {[Instance]: string} = {}
    local function familyOf(part: Instance): string
        local cached: string? = familyCache[part]
        if cached then
            return cached
        end
        local names: {string} = {}
        for _, object: Instance in ipairs(part:GetDescendants()) do
            table.insert(names, object.Name)
        end
        local haystack: string = lower(table.concat(names, " "))
        -- A round pays a single kind of pickup, and event pickups are
        -- built on the classic coin template (MainCoin/DecalPart meshes
        -- with an event look). So when the server announces an event
        -- family, plain coin mesh names must NOT drag the pickup back
        -- to Coin -- that mismatch is what left Auto showing nothing
        -- while the map was full of candies.
        local active: string? = activeFamily()
        for _, family: string in ipairs(FAMILY_ORDER) do
            if family ~= FAMILY_COIN or active == nil or active == FAMILY_COIN then
                for _, phrase: string in ipairs(FAMILY_PHRASES[family]) do
                    if contains(haystack, phrase) then
                        -- Cache only the non-Coin mesh families:
                        -- they win over the server family in every
                        -- scan, so they can never go stale. A Coin
                        -- mesh classification is NOT cached -- when
                        -- the server announces an event family, the
                        -- coin template must reclassify to it.
                        if family ~= FAMILY_COIN then
                            familyCache[part] = family
                        end
                        return family
                    end
                end
            end
        end
        return active or FAMILY_COIN
    end

    -- Every real pickup on the map, merged from the three signatures.
    -- (v84 tried a "cheap" rescan -- tag list plus previously-known
    -- parts -- and in the live game the ESP stopped reporting coins
    -- entirely; the mock suite passed, the game did not. The full
    -- scan below is the behaviour that demonstrably works in game.
    -- The cost is kept down elsewhere: Auto Farm's polls are cached
    -- (1.5 s), the family classification is cached, and the ancestor
    -- walk inside passesFilters is cached per part.)
    local filterCache: {[Instance]: boolean} = {}
    local function collectAll(): ({[Instance]: string}, {[string]: number})
        local found: {[Instance]: string} = {}
        local counts: {[string]: number} = {}
        lastRaw = {tagged = 0, coinId = 0, named = 0}

        -- The attribute half of the filter is volatile (Collected
        -- flips when a coin is taken) and is always re-read; only
        -- the static half (in the workspace, not inside a character)
        -- is cached -- a pickup's ancestry never changes.
        local function passesFiltersCached(part: Instance): boolean
            if attribute(part, "Collected") == true
                or attribute(part, "Delete") == true then
                return false
            end
            local cached: boolean? = filterCache[part]
            if cached ~= nil then
                return cached
            end
            local result: boolean = passesFilters(part)
            filterCache[part] = result
            return result
        end

        local function consider(part: Instance?): ()
            if not part then
                return
            end
            local visual: Instance? = visualOf(part)
            if not visual or found[visual] then
                return
            end
            if not passesFiltersCached(visual) then
                return
            end
            local family: string = familyOf(visual)
            found[visual] = family
            counts[family] = (counts[family] or 0) + 1
        end

        -- Signature 1: the game's own tag list.
        local tagged: {Instance} = collectionService:GetTagged(COIN_TAG)
        lastRaw.tagged = #tagged
        for _, object: Instance in ipairs(tagged) do
            if object:IsA("BasePart") then consider(object) end
        end

        -- Signatures 2 and 3: CoinID and exact spawn names from one scan.
        for _, object: Instance in ipairs(workspace:GetDescendants()) do
            if object:IsA("BasePart") then
                if attribute(object, "CoinID") ~= nil then
                    lastRaw.coinId += 1
                    consider(object)
                elseif object.Name == VISUAL_NAME or object.Name == SERVER_NAME then
                    lastRaw.named += 1
                    consider(object)
                end
            end
        end
        return found, counts
    end

    local function wanted(family: string): boolean
        local choice: string = settings.family
        if choice == FAMILY_ALL then
            return true
        end
        if choice == FAMILY_AUTO then
            local active: string? = activeFamily()
            return active == nil or family == active
        end
        return family == choice
    end

    local function describe(counts: {[string]: number}): string
        local parts: {string} = {}
        for _, family: string in ipairs(COUNT_ORDER) do
            local amount: number = counts[family] or 0
            if amount > 0 then
                local nouns: {string} = FAMILY_NOUNS[family]
                table.insert(parts, tostring(amount) .. " " .. (amount == 1 and nouns[1] or nouns[2]))
            end
        end
        if #parts == 0 then
            return "no coins yet"
        end
        return table.concat(parts, " · ")
    end

    -- Shared with Auto Farm: the pickup parts of the family in use.
    -- CACHED: this used to run a full collectAll() per call, and Auto
    -- Farm polls every 0.4 s -- each poll re-scanned every descendant
    -- of Workspace and hitch-spiked the frame. The cache refreshes on
    -- the ESP's own scans, or at most every 0.5 s when polled.
    local pickupCache: {at: number, list: {Instance}} = {at = -math.huge, list = {}}
    local function refreshPickupCache(found: {[Instance]: string}): ()
        local list: {Instance} = {}
        for part: Instance, family: string in pairs(found) do
            if wanted(family) then
                table.insert(list, part)
            end
        end
        pickupCache = {at = os.clock(), list = list}
    end
    state.mm2CoinPickups = function(): {Instance}
        if os.clock() - pickupCache.at < 0.5 then
            return pickupCache.list
        end
        local found: {[Instance]: string} = collectAll()
        refreshPickupCache(found)
        return pickupCache.list
    end

    local function toggleCoinEsp(enabled: boolean): ()
        disconnectFeatureConnection("MM2CoinRefresh")
        disconnectFeatureConnection("MM2CoinCollected")
        disconnectFeatureConnection("MM2CoinsStarted")
        disconnectFeatureConnection("MM2CoinTagAdded")
        disconnectFeatureConnection("MM2CoinTagRemoved")
        disconnectFeatureConnection("MM2CoinAdded")
        clearEffects(CoinEffects)
        clearTags()

        if not enabled then
            local card: any = state.mm2CoinFeature
            if card then
                card:SetStatus(nil)
            end
            return
        end

        local elapsed: number = 0
        local lastStatus: string? = nil
        local dirty: boolean = true

        local function setStatus(text: string): ()

            if text == lastStatus then
                return
            end
            lastStatus = text
            local card: any = state.mm2CoinFeature
            if card then
                card:SetStatus(text)
            end
        end

        -- The visible mesh of a pickup: MainCoin under the CoinVisual
        -- holder (the Candy / coin itself), falling back to the pickup
        -- part. The box takes THIS part's exact size, so the marker is
        -- as small as the candy on screen -- never the big invisible
        -- Coin_Server touch volume.
        local function visualOf(part: Instance): BasePart
            local main: Instance? = part:FindFirstChild("MainCoin", true)
            if main and main:IsA("BasePart") then return main :: BasePart end
            local visual: Instance? = part:FindFirstChild("CoinVisual", true)
            if visual and visual:IsA("BasePart") then return visual :: BasePart end
            return part :: BasePart
        end

        -- One small adornment per visible pickup. Creation happens
        -- once per pickup; between rescans Lua does nothing at all.
        local tagsDirty: boolean = true
        -- Opacity guard: the setting's old default (0.8, from the
        -- full-model highlight era) made a candy-sized adornment
        -- effectively invisible -- v82-v85 "detected nothing" while
        -- the farm, using the same list, walked to the coins fine.
        -- Cap the marker at 0.6 transparency whatever is saved.
        local function markerTransparency(): number
            return math.clamp(mm2Settings.coinTransparency, 0, 0.6)
        end

        -- Creates the box for one pickup. Called only when the pickups change.
        local function makeTag(part: Instance): any
            local visual: BasePart = visualOf(part)
            local box: BoxHandleAdornment = Instance.new("BoxHandleAdornment")
            box.Name = "WurstCoinBox"
            box.AlwaysOnTop = true
            box.ZIndex = 5
            box.Adornee = visual
            box.Color3 = mm2Settings.coinColor
            box.Transparency = markerTransparency()
            box.Size = visual.Size * 1.3
            box.Parent = part
            return {box = box}
        end

        -- v139: reconciliation runs when the pickup set changes, not every
        -- frame. drawOverlay used to allocate a `seen` table and walk every coin
        -- 60 times a second even when nothing had moved, which showed up as GC
        -- spikes with ~100 coins on the map. A BoxHandleAdornment follows its
        -- Adornee by itself, so the per-frame work was pure waste.
        local function syncTags(): ()
            for part: Instance in pairs(lastFamilies) do
```

```
            if part.Parent and not coinTags[part] then
                coinTags[part] = makeTag(part)
                tagsDirty = true
            end
        end
        for part: Instance, entry: any in pairs(coinTags) do
            if not part.Parent or not lastFamilies[part] then
                coinTags[part] = nil
                pcall(function(): ()
                    entry.box:Destroy()
                end)
                tagsDirty = true
            end
        end
        if tagsDirty then
            tagsDirty = false
            local published: {[Instance]: boolean} = {}
            for part: Instance in pairs(coinTags) do
                published[part] = true
            end
            state.mm2CoinTagParts = published
        end
    end

    -- Per-frame work is now only the colour refresh, twice a second.
    local lastTagColourAt: number = -math.huge
    local function drawOverlay(): ()
        local now: number = os.clock()
        if now - lastTagColourAt < 0.5 then
            return
        end
        lastTagColourAt = now
        for part: Instance, entry: any in pairs(coinTags) do
            if not part.Parent then
                coinTags[part] = nil
                entry.box:Destroy()
                tagsDirty = true
            else
                pcall(function(): ()
                    entry.box.Color3 = mm2Settings.coinColor
                    entry.box.Transparency = markerTransparency()
                end)
            end
        end
        if tagsDirty then
            tagsDirty = false
            local published: {[Instance]: boolean} = {}
            for part: Instance in pairs(coinTags) do
                published[part] = true
            end
            state.mm2CoinTagParts = published
        end
    end

    -- One pickup, cham(s) on its visible geometry only.
    local function refresh(): ()
        local found: {[Instance]: string}, counts: {[string]: number} = collectAll()
        local visible: {[Instance]: boolean} = {}
        table.clear(lastFamilies)
        for object: Instance, family: string in pairs(found) do
            if wanted(family) then
                visible[object] = true
                lastFamilies[object] = family
            end
        end
        -- Auto follows the server family, but a mismatch between
        -- the classification and the announcement must not blank
        -- the ESP: fall back to showing every pickup found.
        if settings.family == FAMILY_AUTO and activeFamily() ~= nil then
            local anyWanted: boolean = false
            for object: Instance in pairs(visible) do
                anyWanted = true
                break
            end
            if not anyWanted then
                for object: Instance, family: string in pairs(found) do
                    visible[object] = true
                    lastFamilies[object] = family
                end
            end
        end
        refreshPickupCache(found)
        syncTags()
        local shown: number = 0
        for _ in pairs(visible) do
            shown += 1
        end
        local foundTotal: number = 0
        for _ in pairs(found) do
            foundTotal += 1
        end
        if foundTotal == 0 then
            setStatus(string.format(
                "no coins yet (tag %d · id %d · named %d)",
                lastRaw.tagged, lastRaw.coinId, lastRaw.named))
        else
            setStatus(describe(counts) .. " - " .. shown .. " shown")
        end
        dirty = false
    end

    -- A pickup appears: watch the tag signal (the game's own) and new
    -- workspace parts carrying one of the exact signatures.
    featureConnections.MM2CoinTagAdded =
        collectionService:GetInstanceAddedSignal(COIN_TAG):Connect(function(): ()
            dirty = true
        end)
    featureConnections.MM2CoinTagRemoved =
        collectionService:GetInstanceRemovedSignal(COIN_TAG):Connect(function(): ()
            dirty = true
        end)
    featureConnections.MM2CoinAdded = workspace.DescendantAdded:Connect(function(object: Instance): ()
        if object:IsA("BasePart")
            and (object.Name == VISUAL_NAME or object.Name == SERVER_NAME
                or attribute(object, "CoinID") ~= nil or hasTag(object))
        then
            dirty = true
        end
    end)

    local started: Instance? = core.mm2GameplayRemotes
        and core.mm2GameplayRemotes:FindFirstChild("CoinsStarted")
    if started and started:IsA("RemoteEvent") then
        featureConnections.MM2CoinsStarted =
            (started :: any).OnClientEvent:Connect(function(bagTable: any): ()
                if type(bagTable) == "table" then
                    for bag: any in pairs(bagTable) do
                        local family: string? = BAG_TO_FAMILY[tostring(bag)]
                        if family then
                            serverFamily = family
                        end
                    end
                end
                dirty = true
            end)
    end

    local remotes: any = core.mm2GameplayRemotes
    local collected: Instance? = remotes and remotes:FindFirstChild("CoinCollected")
    if collected and collected:IsA("RemoteEvent") then
        featureConnections.MM2CoinCollected =
            (collected :: any).OnClientEvent:Connect(function(): ()
                dirty = true
            end)
    end

    refresh()
    featureConnections.MM2CoinRefresh = TaskManager:Connect(
        function(deltaTime: number): ()
            -- v139: the only per-frame work left is the colour refresh
            -- (twice a second). BoxHandleAdornments follow their Adornee
            -- inside the engine; the reconciliation lives in refresh().
            drawOverlay()
            elapsed += deltaTime
            if not dirty and elapsed < RESCAN_PERIOD then
                return
            end
            elapsed = 0
            refresh()
        end)
    , nil, "Coin ESP")
end

local CoinFeature: any = createUniversalFeature(
    "Coin ESP",
    "Draws one small box, exactly the size of the coin, candy, snow token, beach ball or egg mesh, on every real pickup and reports how many of each family exist.",
    3,
        toggleCoinEsp,
        {
            categoryName = "Render",
            parent = MM2Scroll,
            registry = mm2Features,
        }
    )
    state.mm2CoinFeature = CoinFeature

    addCycleOption(
        CoinFeature,
        "Coin in use",
        FAMILY_CHOICES,
        1,
        function(value: string): ()
            settings.family = value
            if CoinFeature.enabled then
                toggleCoinEsp(true)
            end
        end,
        "Auto follows the coin family the server announces through CoinsStarted."
    )
    addColorOption(
        CoinFeature,
        "Coin colour",
        mm2Settings.coinColor,
        function(value: Color3): ()
            mm2Settings.coinColor = value
            if CoinFeature.enabled then
                toggleCoinEsp(true)
            end
        end
    )
    addNumberOption(
        CoinFeature,
        "Coin transparency",
        mm2Settings.coinTransparency,
        0,
        1,
        function(value: number): ()
            mm2Settings.coinTransparency = value
            if CoinFeature.enabled then
                toggleCoinEsp(true)
            end
        end
    )
    addInformationOption(
        CoinFeature,
      "Pickups are recognised by the game's own signatures only: the CoinVisual tag, the CoinID attribute and the exact server/visual part pair. Each pickup gets a single BoxHandleAdornment sized exactly to its visible mesh (no labels), so a hundred coins cost no per-frame Lua. Auto follows the family the server announces on CoinsStarted."
    )

    activeCleanup = function(): ()
        state.mm2CoinFeature = nil
        toggleCoinEsp(false)
    end
    Module.Events = featureConnections
    Module.Initialized = true
    return Module
end

function Module.destroy(): ()
    if not Module.Initialized then
        return
    end
    Module.Initialized = false
    pcall(activeCleanup)
    activeCleanup = function(): ()
    end
    Module.Events = {}
    Module.Runtime = nil
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/MM2/Render/CoinEsp.lua
--------------------------------------------------------------------------------


--------------------------------------------------------------------------------
ARCHIVO: src/games/MM2/Render/RoundEsp.lua
BYTES: 5416
SHA-256: de0bbc771ad6a1923c7a67bd8fe77cb5047fd3342e61004fe1de592d924255f
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
local Module = {
    Name = "MM2 Round ESP",
    Events = {} :: {[string]: any},
    Initialized = false,
    Runtime = nil :: any,
}

local activeCleanup: () -> () = function(): () end

function Module.init(runtime: any): any
    if Module.Initialized then
        return Module
    end
    local core: any = state.mm2Core
    assert(type(core) == "table", "MM2 Round ESP requires the MM2 core module")
    Module.Runtime = runtime
    local GunEffects: any = core.GunEffects
    local TrapEffects: any = core.TrapEffects
    local clearEffects: any = core.clearEffects
    local createMM2Cham: any = core.createMM2Cham
    local createMM2Marker: any = core.createMM2Marker
    local findDroppedGun: any = core.findDroppedGun
    local findMM2Map: any = core.findMM2Map
    local mm2Settings: any = core.mm2Settings

    local function toggleGunEsp(enabled: boolean): ()
        disconnectFeatureConnection("MM2GunESP")
        clearEffects(GunEffects)

        if not enabled then
            return
        end

        local elapsed: number = 1
        local markedGun: Instance? = nil
        featureConnections.MM2GunESP = TaskManager:Connect(function(deltaTime: number): ()
            elapsed += deltaTime
            if elapsed < 0.4 then
                return
            end
            elapsed = 0
            local gun: any = findDroppedGun()
            if gun == markedGun and (gun == nil or gun.Parent ~= nil) then
                return
            end
            markedGun = gun
            clearEffects(GunEffects)
            if gun then
                createMM2Marker(
                    GunEffects,
                    gun,
                    "Gun",
                    mm2Settings.gunColor,
                    mm2Settings.gunTransparency
                )
            end
        end, nil, "MM2 Gun ESP")
    end

    local savedTrapTransparency: {[Instance]: number} =
        setmetatable({}, {__mode = "k"}) :: any

    local function restoreTrapTransparency(): ()
        for part: Instance, orig: number in pairs(savedTrapTransparency) do
            (part :: BasePart).Transparency = orig
        end
        table.clear(savedTrapTransparency)
    end

    local function revealTrapPart(part: Instance): ()
        if not part:IsA("BasePart") then return end
        local current: number = (part :: BasePart).Transparency
        if current >= 0.95 then
            if savedTrapTransparency[part] == nil then
                savedTrapTransparency[part] = current
            end
            (part :: BasePart).Transparency = 0.35
        end
    end

    local function toggleTrapEsp(enabled: boolean): ()
        disconnectFeatureConnection("MM2TrapAdded")
        restoreTrapTransparency()
        clearEffects(TrapEffects)

        if not enabled then
            return
        end

        local marked = setmetatable({}, {__mode = "k"})
        local function addTrap(object: Instance): ()
            if marked[object] then
                return
            end
            if (object.Name == "TrapVisual" or object.Name == "Trap")
                and (object:IsA("Model") or object:IsA("BasePart")) then
                local parent: Instance? = (object :: any).Parent
                if parent and marked[parent] then
                    revealTrapPart(object)
                    return
                end
                marked[object] = true
                revealTrapPart(object)
                for _, child: Instance in ipairs(object:GetDescendants()) do
                    revealTrapPart(child)
                end
                createMM2Cham(
                    TrapEffects,
                    object,
                    mm2Settings.trapColor,
                    mm2Settings.trapTransparency
                )
            end
        end

        local map = findMM2Map()
        for _, object in ipairs((map or workspace):GetDescendants()) do
            addTrap(object)
        end
        if map then
            for _, child: Instance in ipairs(workspace:GetChildren()) do
                addTrap(child)
            end
        end
        featureConnections.MM2TrapAdded = workspace.DescendantAdded:Connect(addTrap)
    end

    core.toggleGunEsp = toggleGunEsp
    core.toggleTrapEsp = toggleTrapEsp

    local unregister = {}
    local itemEsp = services and services.itemEsp
    assert(itemEsp and type(itemEsp.register) == "function", "MM2 item ESP requires the provider service")
    for _, provider in ipairs({
        {name = "Gun ESP", tooltip = "Highlights the dropped MM2 gun while ItemESP is ON.", setEnabled = toggleGunEsp},
        {name = "Trap ESP", tooltip = "Highlights existing and newly placed MM2 traps while ItemESP is ON.", setEnabled = toggleTrapEsp},
    }) do
        local release = itemEsp.register(provider)
        if release then table.insert(unregister, release) end
    end

    activeCleanup = function(): ()
        for _, release in ipairs(unregister) do pcall(release) end
        table.clear(unregister)
        toggleGunEsp(false)
        toggleTrapEsp(false)
    end

    Module.Events = featureConnections
    Module.Initialized = true
    return Module
end

function Module.destroy(): ()
    if not Module.Initialized then
        return
    end
    Module.Initialized = false
    pcall(activeCleanup)
    activeCleanup = function(): () end
    Module.Events = {}
    Module.Runtime = nil
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/MM2/Render/RoundEsp.lua
--------------------------------------------------------------------------------


--------------------------------------------------------------------------------
ARCHIVO: src/games/MM2/base.lua
BYTES: 234472
SHA-256: d0623cc3f6273fdb104c51a51f7ede78b1004a01f686ef48dc5e9812010764c6
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
local Module = {
    Name = "MM2",
    Events = {} :: {[string]: any},
    Initialized = false,
    Menu = nil :: any,
    Runtime = nil :: any,
}

local moduleCleanup: () -> () = function(): () end

local function buildMM2Features()
local host: any = getfenv()
local trajectoryLogPrefix: string = "[" .. tostring(host.PRODUCT.logPrefix)
    .. ":MM2:Trajectory]"
local MM2Effects = create("Folder", {
    Parent = ScreenGui,
    Name = "MM2Effects",
})
local GunEffects = create("Folder", {
    Parent = MM2Effects,
    Name = "GunESP",
})
local TrapEffects = create("Folder", {
    Parent = MM2Effects,
    Name = "TrapESP",
})
local CoinEffects = create("Folder", {
    Parent = MM2Effects,
    Name = "CoinChams",
})

local mm2RoundData: {[any]: any} = {}
local mm2GameplayRemotes: Instance? = nil
local mm2RoleRemote: RemoteFunction? = nil
local roleRefreshRunning: boolean = false
local lastRoleRefreshAt: number = -math.huge
local roundLifecycleActive: boolean? = nil
local roleRefreshGeneration: number = 0

pcall(function()
    local replicatedStorage: ReplicatedStorage =
        game:GetService("ReplicatedStorage")
    local remotes: Instance? = replicatedStorage:WaitForChild("Remotes", 5)
    local gameplay: Instance? = remotes and remotes:WaitForChild("Gameplay", 5)
    if not gameplay then
        return
    end
    mm2GameplayRemotes = gameplay
    local getCurrentPlayerData: Instance? = gameplay:FindFirstChild(
        "GetCurrentPlayerData"
    )
    if getCurrentPlayerData and getCurrentPlayerData:IsA("RemoteFunction") then

        mm2RoleRemote = getCurrentPlayerData
    end
end)

local mm2Settings = {
    shootMode = "Manual",
    shootWallCheck = true,
    shootRaw = false,
    -- v139.2: two of ten confirmed shots in the last saved session ended on
    -- another player's HumanoidRootPart because the server ray kept going past
    -- the murderer. Shooting an innocent costs the sheriff the gun, so the
    -- shot is moved to a body part whose line is clear -- or skipped.
    silentAim = false,
    shootKey = Enum.KeyCode.Unknown,
    shootTarget = "",
    showMissCooldown = false,
    predictionRtt = 0.08,
    silentSweep = 0.5,
    -- Through (trajectory-piercing segment from behind captured position
    -- through predicted endpoint) is the single unified Silent AIM model.
    silentSpawn = "Through",
    getGunKey = Enum.KeyCode.Unknown,
    instantRoleNotify = false,
    roleEspAll = false,

    roleEspRoundOnly = false,
    roleEspInnocent = false,
    roleEspMurderer = false,
    roleEspSheriff = false,

    innocentColor = Color3.fromRGB(0, 255, 8),
    innocentTransparency = 0.62,
    deadColor = Color3.fromRGB(170, 170, 180),
    deadTransparency = 0.72,
    murdererColor = Color3.fromRGB(255, 0, 4),
    murdererTransparency = 0.5,
    sheriffColor = Color3.fromRGB(0, 153, 255),
    sheriffTransparency = 0.5,
    heroColor = Color3.fromRGB(255, 196, 0),
    heroTransparency = 0.5,
    roleTagsAll = false,
    blurtDelay = 1.5,
    blurtRepeat = false,
    blurtFake = false,
    coinColor = Color3.fromRGB(230, 220, 65),
    coinTransparency = 0.35,
    trapColor = Color3.fromRGB(145, 25, 25),
    trapTransparency = 0.7,

    gunColor = Color3.fromRGB(255, 215, 0),
    gunTransparency = 0.55,
    autoPlayId = "",
}

local function isProtectedTarget(player: Player?): boolean
    local runtime: any = Module.Runtime
    local targets: any = runtime and runtime.Services and runtime.Services.protectedTargets
    return targets ~= nil and targets.isProtected(player) == true
end

local function clearEffects(folder)
    for _, child in ipairs(folder:GetChildren()) do
        if child:IsA("ObjectValue") and child.Value then
            child.Value:Destroy()
        end
        child:Destroy()
    end
end

-- Match the recovered client's cursor filters, not an assertion about server rules.
-- Returns (clear, blocker). `blocker` describes the part that stopped the ray
-- (or the last transparent part that was skipped) so the caller can record what
-- the server disagreed about instead of guessing.
local function isWeaponPathClear(origin: Vector3, endpoint: Vector3, target: Model?, strict: boolean?, options: {ignoreTarget: boolean?, requireTarget: boolean?}?): (boolean, any?)
    local direction: Vector3 = endpoint - origin
    if direction.Magnitude < 0.05 then return true, nil end
    local excluded: {Instance} = {}
    if LocalPlayer.Character then table.insert(excluded, LocalPlayer.Character) end
    -- Projectile endpoints are future poses: the target's present body must
    -- not hide a wall farther along the predicted segment. Hitscan is unchanged.
    if options and options.ignoreTarget and target then table.insert(excluded, target) end
    for _, object: Instance in ipairs(CollectionService:GetTagged("WeaponPassthrough")) do
        table.insert(excluded, object)
    end
    local params: RaycastParams = RaycastParams.new()
    params.FilterType = Enum.RaycastFilterType.Exclude
    params.IgnoreWater = true
    for _ = 1, 32 do
        params.FilterDescendantsInstances = excluded
        local hit: RaycastResult? = workspace:Raycast(origin, direction, params)
        if not hit then return not (options and options.requireTarget == true), nil end
        if target and hit.Instance:IsDescendantOf(target) then return true, nil end
        -- The gun uses the game's rule verbatim: every fully transparent part
        -- is skipped. With strict (knife, and the second gun pass) only
        -- transparent parts that cannot collide are skipped; an invisible
        -- solid barrier stops the ray.
        local part: Instance = hit.Instance
        local info: any = {
            name = nil,
            className = nil,
            transparency = nil,
            canCollide = nil,
            distance = nil,
            skipped = false,
        }
        pcall(function(): ()
            info.name = string.sub(part:GetFullName(), 1, 160)
            info.className = part.ClassName
            local hitPosition: any = (hit :: any).Position
            if hitPosition then
                info.distance = (hitPosition - origin).Magnitude
            end
        end)
        if part:IsA("BasePart") then
            info.transparency = part.Transparency
            info.canCollide = part.CanCollide
        end
        if part:IsA("BasePart") and part.Transparency == 1
            and (not strict or part.CanCollide == false) then
            info.skipped = true
            table.insert(excluded, part)
        else
            return false, info
        end
    end
    return false, { name = "raycast budget", skipped = false } -- conservative
end

-- What the ray meets when nothing is skipped. Recorded on every allowed
-- shot, not only on refusals: five shots of the v61 session passed every check
-- and still stopped 24-56 studs short of the aim, and without this there is no
-- way to tell whether transparent geometry was in the way.
local function describeRayHit(origin: Vector3, endpoint: Vector3, target: Model?): any
    local direction: Vector3 = endpoint - origin
    if direction.Magnitude < 0.05 then
        return nil
    end
    local excluded: {Instance} = {}
    if LocalPlayer.Character then
        table.insert(excluded, LocalPlayer.Character)
    end
    for _, object: Instance in ipairs(CollectionService:GetTagged("WeaponPassthrough")) do
        table.insert(excluded, object)
    end
    local params: RaycastParams = RaycastParams.new()
    params.FilterType = Enum.RaycastFilterType.Exclude
    params.IgnoreWater = true
    params.FilterDescendantsInstances = excluded
    local hit: RaycastResult? = workspace:Raycast(origin, direction, params)
    if not hit then
        return nil
    end
    local part: Instance = hit.Instance
    local info: any = {
        onTarget = target ~= nil and part:IsDescendantOf(target),
    }
    pcall(function(): ()
        info.name = string.sub(part:GetFullName(), 1, 160)
        info.className = part.ClassName
        local position: any = (hit :: any).Position
        if position then
            info.distance = (position - origin).Magnitude
        end
    end)
    if part:IsA("BasePart") then
        info.transparency = part.Transparency
        info.canCollide = part.CanCollide
    end
    return info
end

local function getWeaponFromContainer(
    container: Instance?,
    weaponName: string,
    weaponTag: string
): Tool?
    if not container then
        return nil
    end

    for _, child: Instance in ipairs(container:GetChildren()) do
        if child:IsA("Tool") then
            if CollectionService:HasTag(child, weaponTag) then
```

```
                return child
            end

            if weaponName == "Gun" then
                if child.Name == "Gun" then
                    return child
                end
                if child.Name ~= "FakeGun" then
                    local shootRemote: Instance? = child:FindFirstChild("Shoot")
                    if shootRemote and shootRemote:IsA("RemoteEvent") then
                        return child
                    end
                end
            elseif weaponName == "Knife" then
                if child.Name == "Knife" then
                    return child
                end
                local slashRemote: Instance? = child:FindFirstChild("Slash")
                local stabRemote: Instance? = child:FindFirstChild("Stab")
                if (slashRemote and slashRemote:IsA("RemoteEvent"))
                    or (stabRemote and stabRemote:IsA("RemoteEvent")) then
                    return child
                end
                local events: Instance? = child:FindFirstChild("Events")
                local knifeThrown: Instance? = events
                    and events:FindFirstChild("KnifeThrown")
                local knifeStabbed: Instance? = events
                    and events:FindFirstChild("KnifeStabbed")
                if knifeThrown and knifeThrown:IsA("RemoteEvent")
                    and knifeStabbed and knifeStabbed:IsA("RemoteEvent") then
                    return child
                end
            end
        end
    end
    return nil
end

local function getPlayerWeapon(
    player: Player,
    weaponName: string,
    equippedOnly: boolean?
): Tool?
    local tag: string = "Weapon_" .. weaponName
    local equipped: Tool? = getWeaponFromContainer(player.Character, weaponName, tag)
    if equipped or equippedOnly then
        return equipped
    end
    return getWeaponFromContainer(
        player:FindFirstChildOfClass("Backpack"),
        weaponName,
        tag
    )
end

-- v158: `Health > 0` alone called a corpse alive. MM2 keeps the body around
-- after the kill (only the server stops treating it as a player), and the
-- client can read Health > 0 while the Humanoid is already Dead/Physics. Every
-- targeting module asked this function, so the knife kept predicting the
-- ragdoll on the floor as if it were going to run. Health is still the first
-- gate; the humanoid state and the Dead attribute catch the rag.
local function isPlayerAlive(player: Player): boolean
    local character: Model? = player.Character
    if not character then
        return false
    end
    local humanoid: Humanoid? = character:FindFirstChildOfClass("Humanoid")
        :: Humanoid?
    if not humanoid or humanoid.Health <= 0 then
        return false
    end
    local okDeadAttribute: boolean, deadAttribute: any =
        pcall(function(): any
            return character:GetAttribute("Dead")
        end)
    if okDeadAttribute and deadAttribute == true then
        return false
    end
    local okState: boolean, currentState: any = pcall(function(): any
        return humanoid:GetState()
    end)
    if okState and currentState ~= nil then
        -- Looked up inside a pcall: some harness Enum tables do not name these
        -- states, and a missing member must not turn into a false corpse.
        local deadState: any = nil
        local physicsState: any = nil
        pcall(function(): ()
            deadState = Enum.HumanoidStateType.Dead
            physicsState = Enum.HumanoidStateType.Physics
        end)
        if currentState == deadState or currentState == physicsState then
            return false
        end
    end
    return true
end


local function playerFromRoundKey(key: any, data: any): Player?
    if typeof(key) == "Instance" and key:IsA("Player") then
        return key
    end

    -- v192: the payload is normally keyed by name, but a numeric key is a
    -- UserId. Without this branch a UserId-keyed table produced no role for
    -- anybody, which is how the Fling roster ended up label-less.
    if type(key) == "number" then
        local resolved = Players:GetPlayerByUserId(key)
        if resolved then
            return resolved
        end
    end

    local player: Player? = nil
    local namedPlayer: Instance? = Players:FindFirstChild(tostring(key))
    if namedPlayer and namedPlayer:IsA("Player") then
        player = namedPlayer
    end

    if type(data) == "table" then
        if data.Name then
            local namedFromData: Instance? =
                Players:FindFirstChild(tostring(data.Name))

            if namedFromData and namedFromData:IsA("Player") then
                player = namedFromData :: Player
            end
        end
        if not player and data.UserId then
            local userId = tonumber(data.UserId)
            if userId then
                local success, resolvedPlayer = pcall(
                    Players.GetPlayerByUserId,
                    Players,
                    userId
                )
                player = success and resolvedPlayer or nil
            end
        end
    end

    return player
end

local function isRoundEntryDead(data: any): boolean
    if type(data) ~= "table" then
        return false
    end
    return data.Dead == true or data.Killed == true
end

local function getRoundRole(player: Player): string?
    for key: any, data: any in pairs(mm2RoundData) do
        if playerFromRoundKey(key, data) == player and type(data) == "table" then
            if isRoundEntryDead(data) then
                return "Dead"
            end
            local role: any = data.Role
            if role == "Murderer"
                or role == "Sheriff"
                or role == "Hero"
                or role == "Innocent" then
                return role
            end
        end
    end
    return nil
end

local roundRolesListeners: {(boolean) -> ()} = {}
local publishedRoundRolesActive: boolean = false
local function emitRoundRoles(active: boolean): ()
    publishedRoundRolesActive = active
    for _, listener: (boolean) -> () in ipairs(roundRolesListeners) do
        pcall(listener, active)
    end
end
local function onRoundRoles(listener: (boolean) -> ()): () -> ()
    table.insert(roundRolesListeners, listener)

    task.defer(function(): ()
        if table.find(roundRolesListeners, listener) then
            pcall(listener, publishedRoundRolesActive)
        end
    end)
    return function(): ()
        local index: number? = table.find(roundRolesListeners, listener)
        if index then
            table.remove(roundRolesListeners, index)
        end
    end
end

local invalidateRoleCaches: () -> ()
local rebuildRoleIndex: () -> ()

local function hasValidRoundRoleData(dataTable: any): boolean
    if type(dataTable) ~= "table" then
        return false
    end
    for _, entry: any in pairs(dataTable) do
        if type(entry) == "table" and entry.Dead ~= true then
            local role: any = entry.Role
            if role == "Murderer"
                or role == "Sheriff"
                or role == "Hero"
                or role == "Innocent" then
                return true
            end
        end
    end
    return false
end

local function requestRoleRefresh(force: boolean?): ()
    if not mm2RoleRemote
        or (not force and roleRefreshRunning)
        or (not force and os.clock() - lastRoleRefreshAt < 0.8) then
        return
    end

    roleRefreshRunning = true
    local generation: number = roleRefreshGeneration
    task.spawn(function(): ()
        local success: boolean, result: any = pcall(
            mm2RoleRemote.InvokeServer,
            mm2RoleRemote
        )
        roleRefreshRunning = false
        lastRoleRefreshAt = os.clock()
        if success
            and generation == roleRefreshGeneration
            and type(result) == "table"
            and (roundLifecycleActive ~= false or hasValidRoundRoleData(result)) then
            if hasValidRoundRoleData(result) and roundLifecycleActive == nil then
                roundLifecycleActive = true
            end
            mm2RoundData = result
            invalidateRoleCaches()
            if rebuildRoleIndex then
                rebuildRoleIndex()
            end
        end
    end)
end

local roleIndex: {
    roles: {[Player]: string?},
    murderer: Player?,
    sheriff: Player?,
    rolesKnown: boolean,
    gunHolders: {[Player]: boolean}?,
} = {roles = {}, murderer = nil, sheriff = nil, rolesKnown = false, gunHolders = {}}
local roleIndexDirty: boolean = true
local roleIndexAt: number = -math.huge
local roleIndexRefreshQueued: boolean = false
local roleWatchers: {[Player]: {RBXScriptConnection}} = {}

function invalidateRoleCaches(): ()
    roleIndexDirty = true

    if type(state.clearRoleMemo) == "function" then
        state.clearRoleMemo()
    end

    if roleIndexRefreshQueued then
        return
    end
    roleIndexRefreshQueued = true
    task.defer(function(): ()
        roleIndexRefreshQueued = false
        if roleIndexDirty and rebuildRoleIndex then
            rebuildRoleIndex()
        end
    end)
end

local function hasMurdererCollisionGroup(character: Model?): boolean
    if not character then
        return false
    end
    for _, child: Instance in ipairs(character:GetChildren()) do
        if child:IsA("BasePart") and child.CollisionGroup == "Murderer" then
            return true
        end
    end
    return false
end

local function buildRoundRoleMap(): {[Player]: string}
    local map: {[Player]: string} = {}
    for key: any, data: any in pairs(mm2RoundData) do
        if type(data) == "table" then
            local player: Player? = playerFromRoundKey(key, data)
            if player then
                if isRoundEntryDead(data) then
                    map[player] = "Dead"
                else
                    local role: any = data.Role
                    if role == "Murderer"
                        or role == "Sheriff"
                        or role == "Hero"
                        or role == "Innocent" then
                        map[player] = role
                    end
                end
            end
        end
    end
    return map
end

rebuildRoleIndex = function(): ()
    roleIndexDirty = false
    roleIndexAt = os.clock()

    local roles: {[Player]: string?} = {}
    local roundRoleByPlayer: {[Player]: string} = buildRoundRoleMap()
    local murderer: Player? = nil
    local murdererHasKnife: boolean = false
    local sheriff: Player? = nil
    local sheriffHasGun: boolean = false
    local alive: {[Player]: boolean} = {}
    local special: {[Player]: string} = {}
    local gunHolders: {[Player]: boolean} = {}

    for _, player: Player in ipairs(Players:GetPlayers()) do
        local character: Model? = player.Character
        local humanoid: Humanoid? = character
            and character:FindFirstChildOfClass("Humanoid")
        local roundRole: string? = roundRoleByPlayer[player]
        local isAlive: boolean = humanoid ~= nil
            and humanoid.Health > 0
            and roundRole ~= "Dead"
        alive[player] = isAlive

        local hasGun: boolean = getPlayerWeapon(player, "Gun") ~= nil
        local hasKnife: boolean = not hasGun
            and getPlayerWeapon(player, "Knife") ~= nil
        local resolved: string? = nil

        -- Weapon possession is ground truth in MM2:
        -- 1) Anyone carrying the Gun is Sheriff or Hero (never Murderer).
        -- 2) Anyone carrying the Knife is Murderer.
        -- 3) Otherwise use server round data (ignoring dead entries), and
        --    only fall back to Murderer collision group when roundRole is
        --    absent so a Hero/Sheriff is never misidentified as Murderer.
        if hasGun then
            gunHolders[player] = true
            if roundRole == "Innocent"
                or roundRole == "Hero"
                or roundRole == "Dead" then
                resolved = "Hero"
            else
                resolved = "Sheriff"
            end
        elseif hasKnife then
            resolved = "Murderer"
        elseif roundRole == "Dead" then
            resolved = "Dead"
        elseif roundRole and roundRole ~= "Innocent" then
            resolved = roundRole
        elseif roundRole == nil and hasMurdererCollisionGroup(character) then
            resolved = "Murderer"
        end

        if resolved and resolved ~= "Dead" then
            special[player] = resolved

            if resolved == "Murderer" then
                local better: boolean = not murderer
                    or (isAlive and not alive[murderer])
                    or (isAlive == alive[murderer] and hasKnife and not murdererHasKnife)
                if better then
                    murderer = player
                    murdererHasKnife = hasKnife
                end
            elseif resolved == "Sheriff" or resolved == "Hero" then
                local better: boolean = not sheriff
                    or (isAlive and not alive[sheriff])
                    or (isAlive == alive[sheriff] and hasGun and not sheriffHasGun)
                if better then
                    sheriff = player
                    sheriffHasGun = hasGun
                end
            end
        end
    end

    local rolesKnown: boolean = next(special) ~= nil
    if roundLifecycleActive == false then
        rolesKnown = false
    end

    for _, player: Player in ipairs(Players:GetPlayers()) do
        local resolved: string? = special[player]
        if not resolved and rolesKnown then
            resolved = alive[player] and "Innocent" or "Dead"
        end
        roles[player] = resolved
    end

    local rolesChanged: boolean = roleIndex.rolesKnown ~= rolesKnown
        or roleIndex.murderer ~= murderer
        or roleIndex.sheriff ~= sheriff
    if not rolesChanged then
        for player: Player, role: string? in pairs(roles) do
            if roleIndex.roles[player] ~= role then
                rolesChanged = true
                break
            end
        end
    end

    roleIndex.roles = roles
    roleIndex.murderer = murderer
    roleIndex.sheriff = sheriff
    roleIndex.rolesKnown = rolesKnown

    -- A gun changing hands (sheriff died, someone picked it up) must
    -- re-sync the round data: the server knows the new Hero long before
    -- our cached copy does, and only a fresh remote read promotes the
    -- picker from "Innocent" and demotes the dead sheriff.
    local gunHoldersChanged: boolean = false
    do
        local previous: {[Player]: boolean} = roleIndex.gunHolders or {}
        for player in pairs(gunHolders) do
            if not previous[player] then
                gunHoldersChanged = true
            end
        end
        for player in pairs(previous) do
            if not gunHolders[player] then
                gunHoldersChanged = true
            end
        end
    end
    roleIndex.gunHolders = gunHolders
    if gunHoldersChanged and roundLifecycleActive ~= false then
        requestRoleRefresh()
    end

    if rolesChanged then

        if type(state.invalidateGameRoles) == "function" then
            state.invalidateGameRoles()
        elseif type(state.clearRoleMemo) == "function" then
            state.clearRoleMemo()
        end
        emitRoundRoles(rolesKnown)
    end

    if not rolesKnown and roundLifecycleActive ~= false then
        requestRoleRefresh()
    end
end

local function ensureRoleIndex(): ()

    if roleIndexDirty or os.clock() - roleIndexAt > 1 then
        rebuildRoleIndex()
    end
end

local function unwatchPlayer(player: Player): ()
    local connections: {RBXScriptConnection}? = roleWatchers[player]
    if not connections then
        return
    end
    roleWatchers[player] = nil
    for _, connection: RBXScriptConnection in ipairs(connections) do
        pcall(function(): ()
            connection:Disconnect()
        end)
    end
end

local function watchPlayer(player: Player): ()
    unwatchPlayer(player)
    local connections: {RBXScriptConnection} = {}
    local watchedTools: {[Tool]: boolean} = setmetatable({}, {__mode = "k"}) :: any

    local function watchTool(tool: Tool): ()
        if watchedTools[tool] then
            return
        end
        watchedTools[tool] = true

        table.insert(
            connections,
            tool.DescendantAdded:Connect(invalidateRoleCaches)
        )
        table.insert(
            connections,
            tool.DescendantRemoving:Connect(invalidateRoleCaches)
        )
    end

    local function watchContainer(container: Instance?): ()
        if not container then
            return
        end
        for _, child: Instance in ipairs(container:GetChildren()) do
            if child:IsA("Tool") then
                watchTool(child)
            end
        end
        table.insert(
            connections,
            container.ChildAdded:Connect(function(child: Instance): ()
                if child:IsA("Tool") then
                    watchTool(child)
                end
                invalidateRoleCaches()
            end)
        )
        table.insert(connections, container.ChildRemoved:Connect(invalidateRoleCaches))
    end

    local function watchCharacter(character: Model): ()
        invalidateRoleCaches()
        watchContainer(character)

        local function watchCollisionPart(part: BasePart): ()
            table.insert(
                connections,
                part:GetPropertyChangedSignal("CollisionGroup")
                    :Connect(invalidateRoleCaches)
            )
        end
        for _, child: Instance in ipairs(character:GetChildren()) do
            if child:IsA("BasePart") then
                watchCollisionPart(child)
            end
        end
        table.insert(
            connections,
            character.ChildAdded:Connect(function(child: Instance): ()
                if child:IsA("BasePart") then
                    watchCollisionPart(child)
                end
            end)
        )

        local humanoid: Humanoid? = character:FindFirstChildOfClass("Humanoid")
        if humanoid then
            table.insert(
                connections,
                humanoid.Died:Connect(invalidateRoleCaches)
            )
        end
    end

    table.insert(connections, player.CharacterAdded:Connect(watchCharacter))
    table.insert(connections, player.CharacterRemoving:Connect(invalidateRoleCaches))
    table.insert(
        connections,
        player.ChildAdded:Connect(function(child: Instance): ()
            if child:IsA("Backpack") then
                watchContainer(child)
                invalidateRoleCaches()
            end
        end)
    )
    watchContainer(player:FindFirstChildOfClass("Backpack"))
    if player.Character then
        watchCharacter(player.Character)
    end
    roleWatchers[player] = connections
end

for _, player: Player in ipairs(Players:GetPlayers()) do
    watchPlayer(player)
end
featureConnections.MM2RoleWatchAdded = Players.PlayerAdded:Connect(function(player: Player): ()
    watchPlayer(player)
    invalidateRoleCaches()
end)
featureConnections.MM2RoleWatchRemoving = Players.PlayerRemoving:Connect(function(player: Player): ()
    unwatchPlayer(player)
    invalidateRoleCaches()
end)
for _, weaponTag: string in ipairs({"Weapon_Knife", "Weapon_Gun"}) do
    featureConnections["MM2RoleTagAdded" .. weaponTag] = CollectionService
        :GetInstanceAddedSignal(weaponTag)
        :Connect(invalidateRoleCaches)
    featureConnections["MM2RoleTagRemoved" .. weaponTag] = CollectionService
        :GetInstanceRemovedSignal(weaponTag)
        :Connect(invalidateRoleCaches)
end

local function roundRolesKnown(): boolean
    ensureRoleIndex()
    return roleIndex.rolesKnown
end

local function getPlayerRole(player: Player): string?
    ensureRoleIndex()
    return roleIndex.roles[player]
end

local function findMM2Role(toolName: string, excludedPlayer: Player?): Player?
    ensureRoleIndex()
    local wanted: string = toolName == "Knife" and "Murderer" or "Sheriff"
    for player: Player, role: string? in pairs(roleIndex.roles) do
        if player ~= excludedPlayer
            and (role == wanted or (wanted == "Sheriff" and role == "Hero")) then
            return player
        end
    end
    return nil
end

local function findPlayerByRoundRole(role: string): Player?
    for key, data in pairs(mm2RoundData) do
        if type(data) == "table"
            and not isRoundEntryDead(data)
            and data.Role == role then
            local player: Player? = playerFromRoundKey(key, data)
            if player and isPlayerAlive(player) then
                if role == "Murderer" and getPlayerWeapon(player, "Gun") ~= nil then
                    continue
                end
                if (role == "Sheriff" or role == "Hero")
                    and getPlayerWeapon(player, "Knife") ~= nil then
                    continue
                end
                return player
            end
        end
    end
    return nil
end

local function hasActiveRoundRoles(): boolean
    return roundRolesKnown()
end

local function findMurderer(): Player?
    ensureRoleIndex()
    for _, player: Player in ipairs(Players:GetPlayers()) do
        if isPlayerAlive(player)
            and getPlayerWeapon(player, "Gun") == nil
            and getPlayerWeapon(player, "Knife") ~= nil then
            return player
        end
    end
    local candidate: Player? = roleIndex.murderer or findPlayerByRoundRole("Murderer")
    if candidate and getPlayerWeapon(candidate, "Gun") ~= nil then
        return nil
    end
    return candidate
end

local function findSheriff(): Player?
    ensureRoleIndex()
```

Leer columna izquierda completa y despues la derecha. Texto seleccionable; lineas largas comprimidas. Tabulaciones: 4 espacios. Original adjunto.

```
    for _, player: Player in ipairs(Players:GetPlayers()) do
        if isPlayerAlive(player)
            and getPlayerWeapon(player, "Knife") == nil
            and getPlayerWeapon(player, "Gun") ~= nil then
            return player
        end
    end
    local candidate: Player? = roleIndex.sheriff
        or findPlayerByRoundRole("Sheriff")
        or findPlayerByRoundRole("Hero")
    if candidate and getPlayerWeapon(candidate, "Knife") ~= nil then
        return nil
    end
    return candidate
end

local MM2_MAP_COIN_CONTAINERS: {string} = {
    "CoinContainer",
    "CoinAreas",
    "Coins",
}

local LOBBY_MODEL_NAMES: {string} = {
    "Lobby",
    "MainLobby",
    "RegularLobby",
}

local function isLobbyName(name: string): boolean
    for _, candidate: string in ipairs(LOBBY_MODEL_NAMES) do
        if name == candidate then
            return true
        end
    end
    return string.find(string.lower(tostring(name)), "lobby", 1, true) ~= nil
end

local function isLobbyModel(object: Instance): boolean
    if isLobbyName(object.Name) then
        return true
    end
    if object:FindFirstChild("VotePad1")
        or object:FindFirstChild("VotePads")
        or object:FindFirstChild("VoteIcons") then
        return true
    end
    return false
end

local function isMM2MapModel(object: Instance): boolean
    if not object:IsA("Model") or isLobbyModel(object) then
        return false
    end
    for _, containerName: string in ipairs(MM2_MAP_COIN_CONTAINERS) do
        if object:FindFirstChild(containerName) then
            return true
        end
    end
    if object:FindFirstChild("GunDrop") then
        return true
    end
    return false
end

local mm2MapCache: {map: Instance?, at: number} = {map = nil, at = -math.huge}

local function findMM2MapUncached(): Instance?
    for _, object: Instance in ipairs(workspace:GetChildren()) do
        if isMM2MapModel(object) then
            return object
        end
    end

    for _, object: Instance in ipairs(workspace:GetChildren()) do
        if object:IsA("Model")
            and not isLobbyModel(object)
            and object:FindFirstChild("Spawns") then
            return object
        end
    end
    return nil
end

local function findMM2Map(): Instance?
    local cached: Instance? = mm2MapCache.map
    if cached and cached.Parent == workspace and isMM2MapModel(cached) then
        return cached
    end
    if os.clock() - mm2MapCache.at < 0.25 then
        return (cached and cached.Parent == workspace) and cached or nil
    end
    mm2MapCache.at = os.clock()
    mm2MapCache.map = findMM2MapUncached()
    return mm2MapCache.map
end

local droppedGunCache: {gun: (Model | BasePart)?, at: number} =
    {gun = nil, at = -math.huge}
local lastGunSweepAt: number = -math.huge

local function isLooseGun(object: Instance): boolean
    if not object:IsA("Tool") then
        return false
    end
    if object.Name ~= "Gun" and not CollectionService:HasTag(object, "Weapon_Gun") then
        return false
    end

    return object:IsDescendantOf(workspace)
        and not object:FindFirstAncestorOfClass("Backpack")
        and (object.Parent == workspace or object.Parent == findMM2Map()
            or object.Parent ~= nil and not object.Parent:FindFirstChildOfClass("Humanoid"))
end

local function resolveGunPart(object: Instance): (Model | BasePart)?
    if object:IsA("Model") or object:IsA("BasePart") then
        return object :: any
    end
    local handle: Instance? = object:FindFirstChild("Handle")
        or object:FindFirstChildWhichIsA("BasePart")
    return (handle and handle:IsA("BasePart")) and handle or nil
end

local function findDroppedGunUncached(): (Model | BasePart)?
    local map: Instance? = findMM2Map()
    local candidate: Instance? = map and map:FindFirstChild("GunDrop")
    if not candidate then
        candidate = workspace:FindFirstChild("GunDrop")
    end
    if candidate and (candidate:IsA("Model") or candidate:IsA("BasePart")) then
        return candidate :: any
    end

    for _, taggedDrop: Instance in ipairs(CollectionService:GetTagged("GunDrop")) do
        if taggedDrop:IsDescendantOf(workspace)
            and (taggedDrop:IsA("Model") or taggedDrop:IsA("BasePart")) then
            return taggedDrop :: any
        end
    end

    for _, object: Instance in ipairs(workspace:GetChildren()) do
        if object:IsA("Model") or object:IsA("Folder") then
            local childDrop: Instance? = object:FindFirstChild("GunDrop")
            if childDrop and (childDrop:IsA("Model") or childDrop:IsA("BasePart")) then
                return childDrop :: any
            end
        end
    end

    for _, object: Instance in ipairs(CollectionService:GetTagged("Weapon_Gun")) do
        if isLooseGun(object) then
            local part: (Model | BasePart)? = resolveGunPart(object)
            if part then
                return part
            end
        end
    end

    for _, object: Instance in ipairs(workspace:GetChildren()) do
        if isLooseGun(object) then
            local part: (Model | BasePart)? = resolveGunPart(object)
            if part then
                return part
            end
        end
    end
    if os.clock() - lastGunSweepAt < 1.5 then
        return nil
    end
    lastGunSweepAt = os.clock()
    local deepCandidate: Instance? = (map and map:FindFirstChild("GunDrop", true))
        or workspace:FindFirstChild("GunDrop", true)
    if deepCandidate
        and (deepCandidate:IsA("Model") or deepCandidate:IsA("BasePart")) then
        return deepCandidate :: any
    end
    return nil
end

local function findDroppedGun(): (Model | BasePart)?
    local cached: (Model | BasePart)? = droppedGunCache.gun
    if cached and cached.Parent and cached:IsDescendantOf(workspace) then
        return cached
    end
    -- Negative results are only held for ~3 frames. The old 0.4 s
    -- cache meant a gun dropped just after a scan stayed invisible
    -- to Auto Get Gun for almost half a second; the cheap checks
    -- (direct GunDrop child, Weapon_Gun tags, workspace children)
    -- run first every call and the expensive descendant sweep keeps
    -- its own 3 s gate below.
    if os.clock() - droppedGunCache.at < 0.05 then
        return nil
    end
    droppedGunCache.at = os.clock()
    droppedGunCache.gun = findDroppedGunUncached()
    return droppedGunCache.gun
end

local function createMM2Marker(folder, adornee, labelText, color, transparency)
    local highlight = Instance.new("Highlight")
    highlight.Name = labelText .. "Highlight"
    highlight.Adornee = adornee
    highlight.DepthMode = Enum.HighlightDepthMode.AlwaysOnTop
    highlight.FillColor = color
    highlight.FillTransparency = transparency or 0.68
    highlight.OutlineColor = color
    highlight.OutlineTransparency = 0
    highlight.Parent = adornee

    local highlightReference = Instance.new("ObjectValue")
    highlightReference.Name = labelText .. "HighlightReference"
    highlightReference.Value = highlight
    highlightReference.Parent = folder

    local head = adornee:IsA("Model")
        and (adornee:FindFirstChild("Head")
            or adornee:FindFirstChild("HumanoidRootPart")
            or adornee:FindFirstChild("Handle", true)
            or adornee:FindFirstChildWhichIsA("BasePart", true))
        or adornee

    if head and head:IsA("BasePart") then
        local billboard = Instance.new("BillboardGui")
        billboard.Name = labelText .. "Label"
        billboard.Adornee = head
        billboard.AlwaysOnTop = true
        billboard.LightInfluence = 0
        billboard.MaxDistance = 1500
        billboard.Size = UDim2.fromOffset(150, 22)
        billboard.StudsOffset = Vector3.new(0, 3.5, 0)
        billboard.Parent = head

        local billboardReference = Instance.new("ObjectValue")
        billboardReference.Name = labelText .. "LabelReference"
        billboardReference.Value = billboard
        billboardReference.Parent = folder

        local text = Instance.new("TextLabel")
        text.BackgroundTransparency = 1
        text.BorderSizePixel = 0
        text.Font = Enum.Font.GothamBold
        text.Size = UDim2.fromScale(1, 1)
        text.Text = labelText
        text.TextColor3 = color
        text.TextSize = 14
        text.TextStrokeColor3 = Color3.fromRGB(0, 0, 0)
        text.TextStrokeTransparency = 0.3
        text.Parent = billboard
    end
end

local function createMM2Cham(folder, adornee, color, transparency)
    local highlight = Instance.new("Highlight")
    highlight.Name = "WurstCham"
    highlight.Adornee = adornee
    highlight.DepthMode = Enum.HighlightDepthMode.AlwaysOnTop
    highlight.FillColor = color
    highlight.FillTransparency = transparency
    highlight.OutlineColor = color
    highlight.OutlineTransparency = math.clamp(transparency + 0.15, 0, 1)
    highlight.Parent = adornee

    local reference = Instance.new("ObjectValue")
    reference.Name = "ChamReference"
    reference.Value = highlight
    reference.Parent = folder
end

local function getPlayerPerk(player: Player): string?
    local char: Model? = player.Character
    if char then
        local perkAttr: any = char:GetAttribute("Perk") or player:GetAttribute("Perk")
        if type(perkAttr) == "string" and perkAttr ~= "" then
            return perkAttr
        end
    end
    for key: any, data: any in pairs(mm2RoundData) do
        if playerFromRoundKey(key, data) == player and type(data) == "table" then
            local perk: any = data.Perk or data.EquippedPerk or data.perk
            if type(perk) == "string" and perk ~= "" then
                return perk
            end
        end
    end
    if char then
        if char:FindFirstChild("Trap") or (player:FindFirstChild("Backpack") and player.Backpack:FindFirstChild("Trap")) then
            return "Trap"
        end
        if char:FindFirstChild("FakeGun") or (player:FindFirstChild("Backpack") and player.Backpack:FindFirstChild("FakeGun")) then
            return "FakeGun"
        end
    end
    return nil
end

local mm2BridgeProvider: any = nil
local function unregisterMM2Roles(): ()
    if mm2BridgeProvider and type(state.unregisterRoleProvider) == "function" then
        state.unregisterRoleProvider(mm2BridgeProvider)
    end
    mm2BridgeProvider = nil
end

local function registerMM2Roles(): ()
    mm2BridgeProvider = {
        Name = "MM2",
        ReplacesTeams = true,
        DeadRoles = {"Dead"},
        Roles = {"Murderer", "Sheriff", "Hero", "Innocent", "Dead"},
        Colors = {
            Murderer = mm2Settings.murdererColor,
            Sheriff = mm2Settings.sheriffColor,
            Hero = mm2Settings.heroColor,
            Innocent = mm2Settings.innocentColor,
            Dead = mm2Settings.deadColor,
        },
        Get = function(player: Player): string?
            return getPlayerRole(player)
        end,
        GetPerk = function(player: Player): string?
            return getPlayerPerk(player)
        end,
        GetColor = function(roleName: string): Color3?
            if roleName == "Murderer" then return mm2Settings.murdererColor end
            if roleName == "Sheriff" then return mm2Settings.sheriffColor end
            if roleName == "Hero" then return mm2Settings.heroColor end
            if roleName == "Innocent" then return mm2Settings.innocentColor end
            if roleName == "Dead" then return mm2Settings.deadColor end
            return nil
        end,
        SetColor = function(roleName: string, colour: Color3): ()
            if roleName == "Murderer" then
                mm2Settings.murdererColor = colour
            elseif roleName == "Sheriff" then
                mm2Settings.sheriffColor = colour
            elseif roleName == "Hero" then
                mm2Settings.heroColor = colour
            elseif roleName == "Innocent" then
                mm2Settings.innocentColor = colour
            elseif roleName == "Dead" then
                mm2Settings.deadColor = colour
            end
        end,
    }
    registerRoleProvider(mm2BridgeProvider)
end

local primeRoundState: () -> ()
local setRoundPhase: (string, string) -> ()
local lastPhase: string = "unknown"

if mm2GameplayRemotes then
    local playerDataChanged = mm2GameplayRemotes:FindFirstChild("PlayerDataChanged")
    if playerDataChanged and playerDataChanged:IsA("RemoteEvent") then
        featureConnections.MM2PlayerDataChanged = playerDataChanged.OnClientEvent:Connect(function(newData)
            if type(newData) == "table" and hasValidRoundRoleData(newData) then
                roundLifecycleActive = true
                mm2MapCache.at = -math.huge
                mm2RoundData = newData
                lastRoleRefreshAt = os.clock()
                invalidateRoleCaches()
                if rebuildRoleIndex then
                    rebuildRoleIndex()
                end
                if setRoundPhase and findMM2Map() then
                    lastPhase = "map"
                    setRoundPhase("map", "PlayerDataChanged")
                end
            elseif type(newData) ~= "table"
                or next(newData) == nil
                or roundLifecycleActive == false then
                mm2RoundData = {}
                invalidateRoleCaches()
                if rebuildRoleIndex then
                    rebuildRoleIndex()
                end
            else
                for key: any, value: any in pairs(newData) do
                    mm2RoundData[key] = value
                end
                lastRoleRefreshAt = os.clock()
                invalidateRoleCaches()
                if rebuildRoleIndex then
                    rebuildRoleIndex()
                end
            end
        end)
    end

    local roleSelect = mm2GameplayRemotes:FindFirstChild("RoleSelect")
    if roleSelect and roleSelect:IsA("RemoteEvent") then
        featureConnections.MM2RoleSelect = roleSelect.OnClientEvent:Connect(function(localRole: any)
            roundLifecycleActive = true
            mm2MapCache.at = -math.huge
            mm2RoundData = {}
            if type(localRole) == "string"
                and (localRole == "Murderer"
                    or localRole == "Sheriff"
                    or localRole == "Hero"
                    or localRole == "Innocent") then
                mm2RoundData[LocalPlayer.Name] = {
                    Name = LocalPlayer.Name,
                    UserId = LocalPlayer.UserId,
                    Role = localRole,
                }
            end
            invalidateRoleCaches()
            if rebuildRoleIndex then
                rebuildRoleIndex()
            end
            if primeRoundState then
                primeRoundState()
            end
            if setRoundPhase then
                lastPhase = "map"
                setRoundPhase("map", "RoleSelect")
            end
        end)
    end

    for _, remoteName: string in ipairs({"ShowRoleSelect", "ShowRoleSelectNew"}) do
        local showRole = mm2GameplayRemotes:FindFirstChild(remoteName)
        if showRole and showRole:IsA("RemoteEvent") then
            featureConnections["MM2" .. remoteName] = showRole.OnClientEvent:Connect(function()
                roundLifecycleActive = true
                mm2MapCache.at = -math.huge
                invalidateRoleCaches()
                if primeRoundState then
                    primeRoundState()
                end
                if setRoundPhase then
                    lastPhase = "map"
                    setRoundPhase("map", remoteName)
                end
            end)
        end
    end
end

local roundTimer: {endsAt: number?} = {endsAt = nil}
local timerRemotes = game:GetService("ReplicatedStorage"):FindFirstChild("Remotes")
local timerGameplay = timerRemotes and timerRemotes:FindFirstChild("Gameplay")
local timerRoundStart = timerGameplay and timerGameplay:FindFirstChild("RoundStart")
local timerRoundEnd = timerGameplay and timerGameplay:FindFirstChild("RoundEndFade")
local sessionRoundActive = false

if timerRoundStart and timerRoundStart:IsA("RemoteEvent") then
    featureConnections.MM2TimerTrackerStart = timerRoundStart.OnClientEvent:Connect(
        function(duration, startPlayerData)
            duration = tonumber(duration)
            roundTimer.endsAt = duration and os.clock() + duration or nil
            if not duration or duration <= 0 then
                mm2RoundData = {}
                sessionRoundActive = false
            elseif type(startPlayerData) == "table"
                and hasValidRoundRoleData(startPlayerData) then
                mm2RoundData = startPlayerData
            end
            invalidateRoleCaches()
            roundLifecycleActive = duration ~= nil and duration > 0
            roleRefreshGeneration += 1
            if duration and duration > 0 then
                mm2MapCache.at = -math.huge
                primeRoundState()
                lastPhase = "map"
                setRoundPhase("map", "RoundStart")
            end
            if duration and duration > 0
                and not sessionRoundActive
                and state.sessionInfo
                and state.sessionInfo.addRound then
                sessionRoundActive = true
                state.sessionInfo.addRound()
            end
        end
    )
end
if timerRoundEnd and timerRoundEnd:IsA("RemoteEvent") then
    featureConnections.MM2TimerTrackerEnd = timerRoundEnd.OnClientEvent:Connect(function()
        roundTimer.endsAt = nil
        sessionRoundActive = false
        roundLifecycleActive = false
        invalidateRoleCaches()
        roleRefreshGeneration += 1
        mm2RoundData = {}
        setRoundPhase("lobby", "RoundEndFade")
    end)
end

local roundPhase: string = "unknown"
local roundPhaseListeners: {(string, string) -> ()} = {}

local function emitRoundPhase(phase: string, reason: string): ()
    for _, listener: (string, string) -> () in ipairs(roundPhaseListeners) do
        pcall(listener, phase, reason)
    end
end

setRoundPhase = function(phase: string, reason: string): ()
    if roundPhase == phase then
        return
    end
    roundPhase = phase
    emitRoundPhase(phase, reason)
end

local function onRoundPhase(listener: (string, string) -> ()): () -> ()
    table.insert(roundPhaseListeners, listener)
    task.defer(function(): ()
        if table.find(roundPhaseListeners, listener) then
            pcall(listener, roundPhase, "subscribe")
        end
    end)
    return function(): ()
        local index: number? = table.find(roundPhaseListeners, listener)
        if index then
            table.remove(roundPhaseListeners, index)
        end
    end
end

-- v139: the round-phase check runs 10 times a second for the whole session, so
-- the lobby model and its reference part are cached. Before this, every tick
-- walked workspace:GetChildren() (hundreds of objects) and did a recursive
-- FindFirstChild("VotePads", true) inside the lobby model.
local lobbyCache: {model: Instance?, at: number} = {model = nil, at = -math.huge}
local LOBBY_CACHE_SECONDS: number = 0.75

local function findLobbyModelUncached(): Instance?
    for _, object: Instance in ipairs(workspace:GetChildren()) do
        if isLobbyName(object.Name) or isLobbyModel(object) then
            return object
        end
    end
    return nil
end

local function findLobbyModel(): Instance?
    local cached: Instance? = lobbyCache.model
    if cached and cached.Parent then
        return cached
    end
    local now: number = os.clock()
    if now - lobbyCache.at < LOBBY_CACHE_SECONDS then
        return nil
    end
    lobbyCache.at = now
    lobbyCache.model = findLobbyModelUncached()
    return lobbyCache.model
end

local lobbyRefCache: {model: Instance?, part: BasePart?, at: number} =
    {model = nil, part = nil, at = -math.huge}

local function lobbyReferencePart(lobby: Instance): BasePart?
    local now: number = os.clock()
    if lobbyRefCache.model == lobby and lobbyRefCache.part
        and lobbyRefCache.part.Parent then
        return lobbyRefCache.part
    end
    if lobbyRefCache.model == lobby and now - lobbyRefCache.at < 1 then
        return lobbyRefCache.part
    end
    lobbyRefCache.at = now
    lobbyRefCache.model = lobby
    local votePads: Instance? = lobby:FindFirstChild("VotePads", true)
    local refPart: BasePart? = nil
    if votePads then
        refPart = votePads:FindFirstChildWhichIsA("BasePart", true)
    end
    if not refPart then
        local spawns: Instance? = lobby:FindFirstChild("Spawns", true)
        if spawns then
            refPart = spawns:FindFirstChildWhichIsA("BasePart", true)
        end
    end
    if not refPart then
        refPart = lobby:FindFirstChildWhichIsA("BasePart", true)
    end
    lobbyRefCache.part = refPart
    return refPart
end

local function isRootNearLobby(root: BasePart, lobby: Instance): boolean
    local refPart: BasePart? = lobbyReferencePart(lobby)
    return refPart ~= nil and (root.Position - refPart.Position).Magnitude <= 350
end

local function detectRoundPhase(): string
    local character: Model? = LocalPlayer.Character
    if not character then
        return "lobby"
    end
    local root: BasePart? = character:FindFirstChild("HumanoidRootPart") :: BasePart?
    if not root then
        return "lobby"
    end

    local lobby: Instance? = findLobbyModel()
    if lobby and (root:IsDescendantOf(lobby) or isRootNearLobby(root, lobby)) then
        return "lobby"
    end
    if findMM2Map() then
        return "map"
    end

    return roundPhase == "map" and "map" or "lobby"
end

primeRoundState = function(): ()
    roleRefreshGeneration += 1
    local generation: number = roleRefreshGeneration
    requestRoleRefresh(true)
    task.spawn(function(): ()
        for _, delay: number in ipairs({0.05, 0.15, 0.3, 0.5, 0.8, 1.2, 2.0, 3.0}) do
            task.wait(delay)
            if generation ~= roleRefreshGeneration or roundRolesKnown() then
                break
            end
            requestRoleRefresh(true)
        end
    end)
end

local phaseElapsed: number = 0
lastPhase = detectRoundPhase()
if lastPhase == "map" then
    roundLifecycleActive = true
    primeRoundState()
end
featureConnections.MM2RoundPhase = TaskManager:Connect(function(deltaTime: number): ()
    phaseElapsed += deltaTime
    -- v139: 10 Hz instead of 20. The phase only changes between lobby and map,
    -- and 100 ms of extra latency on a round transition is invisible.
    if phaseElapsed < 0.1 then
        return
    end
    phaseElapsed = 0
    local phase: string = detectRoundPhase()
    if phase == lastPhase then
        return
    end
    lastPhase = phase
    if phase == "map" then
        if not hasValidRoundRoleData(mm2RoundData) then
            mm2RoundData = {}
        end
        roundLifecycleActive = true
        invalidateRoleCaches()
        primeRoundState("map")
        setRoundPhase("map", "teleport")
    else
        mm2RoundData = {}
        roundLifecycleActive = false
        roundTimer.endsAt = nil
        invalidateRoleCaches()
        roleRefreshGeneration += 1
        setRoundPhase("lobby", "teleport")
    end
end, nil, "MM2 round phase")
setRoundPhase(lastPhase, "boot")

local roundTimerPartCache: {part: Instance?, at: number} =
```

Leer columna izquierda completa y después la derecha. Texto seleccionable: líneas largas comprimidas. Tabulaciones: 4 espacios. Proyanal adjunto.

```
{part = nil, at = -math.huge}

local function findMundiTimerPart(): number?
    local cached: Instance? = roundTimerPartCache.part
    if cached and cached.Parent then
        return cached
    end
    if os.clock() - roundTimerPartCacheAt < 1 then
        return nil
    end
    roundTimerPartCacheAt = os.clock()
    local found: Instance? = workspace:FindFirstChild("RoundTimerPart")
    roundTimerPartCache.part = found
    return found
end

local function readRoundTimerPartSeconds(): number?
    local part: Instance? = findMundiTimerPart()
    if not part then
        return nil
    end
    local remaining: number? = tonumber(part:GetAttribute("Time") :: any)
    if not remaining or remaining <= 0 or remaining > 600 then
        return nil
    end
    return remaining
end

local hudTimerCache: {frame: Instance?, label: Instance?, at: number} =
    {frame = nil, label = nil, at = -math.huge}

local function findHudTimerLabel(): Instance?
    local cached = hudTimerCache.label
    if cached and cached.Parent then
        return cached
    end
    local cachedInstance = hudTimerCache.label
    if cachedInstance and cachedInstance.Parent then
        return cachedInstance
    end
    local found = nil
    local frame = findMundiFrame()
    if frame then
        found = frame:FindFirstChild("TimerLabel", true)
            or frame:FindFirstChildWhichIsA("TextLabel", true)
    end
    hudTimerCache.label = (found and found:IsA("TextLabel")) and found or nil
    return hudTimerCache.label
end

local function parseHudTimerSeconds(text: string?): number?
    if type(text) ~= "string" or text == "" then
        return nil
    end
    local minutes, seconds = text:match("^%s*(%d+):(%d%s?%s*$")
    if minutes and seconds then
        return tonumber(minutes) * 60 + tonumber(seconds)
    end
    local whole = tonumber(text:match("^%s*(%d+)%s*$"))
    if whole then
        return tonumber(whole)
    end
    return nil
end

local function isHudTimerVisible(label: Instance): boolean
    local ancestor: Instance? = label
    while ancestor do
        if ancestor:IsA("GuiObject") and not ancestor.Visible then
            return false
        end
        ancestor = ancestor.Parent
    end
    return true
end

local function findMundiTimerPart(): BasePart?
    local cached = roundTimerPartCache.part
    if cached and cached.Parent then
        return cached
    end
    local found = nil
    for _, descendant in ipairs(workspace:GetDescendants()) do
        if descendant.Name == "RoundTimer" and descendant:IsA("BasePart") then
            found = descendant
            break
        end
    end
    roundTimerPartCache.part = found
    return found
end

local function readRoundTimerSeconds(): number?
    local part = findMundiTimerPart()
    if not part then
        return nil
    end
    local gui = part:FindFirstChildWhichIsA("BillboardGui")
        or part:FindFirstChildWhichIsA("SurfaceGui")
    if not gui then
        return nil
    end
    local best: number? = nil
    for _, descendant in ipairs(gui:GetDescendants()) do
        if descendant:IsA("TextLabel") or descendant:IsA("TextButton") then
            local parsed = parseHudTimerSeconds(descendant.Text)
            if parsed and (not best or parsed > best) then
                best = parsed
            end
        end
    end
    return best
end

local function readMundiTimerSeconds(): number?
    local fromPart = readMundiTimerPartSeconds()
    if fromPart then
        return fromPart
    end
    return readHudTimerSeconds()
end

local function readMundiTimerSecondsCached(): number?
    local now = os.clock()
    if now - mundiTimerCacheAt < 0.5 then
        return mundiTimerCacheValue
    end
    mundiTimerCacheAt = now
    mundiTimerCacheValue = readMundiTimerSeconds()
    return mundiTimerCacheValue
end

local function getMundiTimerSecondsCached(): number?
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSeconds(): number?
    return readMundiTimerSeconds()
end

local function getMundiTimerSecondsCached(): number?
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsFresh(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFresh(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCached(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFresh(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFreshCached(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFreshCachedFresh(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFreshCachedFreshCached(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFreshCachedFreshCachedFresh(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFreshCachedFreshCachedFreshCached(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFreshCachedFreshCachedFreshCachedFresh(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCached(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFresh(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCached(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFresh(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCached(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFresh(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCached(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFresh(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCached(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFresh(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCached(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFresh(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCached(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFresh(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCached(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFresh(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCached(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFresh(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCached(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFresh(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCached(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFresh(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCached(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFresh(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCached(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFresh(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCachedFreshCached(): number?
    mundiTimerCacheAt = -math.huge
    return readMundiTimerSecondsCached()
end

local function getMundiTimerSecondsCachedFresh
```

```
        if type(environment.readfile) ~= "function" then
            runtime.persistenceBlocked = true
            return
        end
        local okRead: boolean, raw: any = pcall(environment.readfile, OUTPUT_PATH)
        if not okRead or type(raw) ~= "string" then
            local okExists, exists = pcall(function() return environment.isfile(OUTPUT_PATH) end)
            runtime.persistenceBlocked = not (okExists and exists == false)
            runtime.historyExpectedAbsent = not runtime.persistenceBlocked
            return
        end
        -- Importing the old data is read-only. Even an empty existing file is
        -- protected: the first authorised save verifies and backs up its bytes.
        runtime.pendingHistoryBackup = raw
        if #raw == 0 then return end
        local ok, decoded = pcall(HttpService.JSONDecode, HttpService, raw)
        if not ok or type(decoded) ~= "table" then return end
        -- Only plausible statistical estimates are portable. Counts such as
        -- 10^57, old events and old motion buckets never become session data.
        local aggregate: any = type(decoded.aggregate) == "table" and decoded.aggregate or {}
        local function importStats(stats: RunningStats, payload: any, low: number, high: number): ()
            if type(payload) ~= "table" then return end
            for _, key in ipairs({"count", "mean", "m2", "minimum", "maximum"}) do
                if not predictionMath.finite(payload[key]) then return end
            end
            if payload.count < 1 or payload.count > 1000000 or payload.count % 1 ~= 0
                or payload.m2 < 0 or payload.m2 > payload.count * (high - low) ^ 2
                or payload.minimum < low or payload.maximum > high
                or payload.mean < payload.minimum or payload.mean > payload.maximum then return end
            applyStatsPayload(stats, payload)
        end
        importStats(runtime.gunAcceptance, aggregate.gunAcceptanceMs, 0, 2500)
        importStats(runtime.knifeSpeed, aggregate.knifeSpeedStudsPerSecond, 24, 300)
        importStats(runtime.knifeSpawnDelay, aggregate.knifeSpawnDelayMs, 0, 5000)
        -- What the predictor already learned (lead error per aim mode) has to
        -- survive too: without this every session restarted at zero correction
        -- and no number of matches could ever converge.
        local learned: any = type(decoded.predictor) == "table" and decoded.predictor or {}
        local learnedBuckets: any = type(learned.leadErrors) == "table" and learned.leadErrors or nil
        if learnedBuckets then
            runtime.leadSeeds = learnedBuckets
            runtime.leadSeedsApplied = false
            applyLearnedSeeds()
        end
        -- The imported history is a measurement too; the live samples that
        -- follow refine it.
    end

    local function vectorArray(value: Vector3): {number}
        return {value.X, value.Y, value.Z}
    end

    local function pingBucketKey(pingMs: number): string
        if pingMs > 250 then
            return "250+"
        end
        local lower: number = math.clamp(math.floor(pingMs / 5) * 5, 0, 245)
        return string.format("%03d-%03d", lower, lower + 5)
    end

    local function getTargetRoot(player: Player?): BasePart?
        return player
            and player.Character
            and player.Character:FindFirstChild("HumanoidRootPart") :: BasePart?
    end

    local function getTargetHumanoid(player: Player?): Humanoid?
        return player
            and player.Character
            and player.Character:FindFirstChildOfClass("Humanoid") :: Humanoid?
    end

    local function captureTrajectoryPoint(
        player: Player,
        activatedAt: number,
        pingMs: number
    ): TrajectoryPoint?
        local root: BasePart? = getTargetRoot(player)
        local humanoid: Humanoid? = getTargetHumanoid(player)
        if not root or not humanoid or humanoid.Health <= 0 then
            return nil
        end
        local stateName: string = humanoid:GetState().Name
        local grounded: boolean = humanoid.FloorMaterial ~= Enum.Material.Air
        local jumping: boolean = stateName == "Jumping"
            or stateName == "Freefall"
            or not grounded
        return {
            dt = os.clock() - activatedAt,
            serverTime = workspace:GetServerTimeNow(),
            pingMs = pingMs,
            position = vectorArray(root.Position),
            velocity = vectorArray(root.AssemblyLinearVelocity),
            moveDirection = vectorArray(humanoid.MoveDirection),
            humanoidState = stateName,
            grounded = grounded,
            jumping = jumping,
        }
    end

    -- Record client-visible health separately from ray/projectile evidence.
    -- A decrease may have another cause; a heal is not a damage event.
    local function watchPendingHealth(pending: any, target: Player?): ()
        pending.targetHealthAtAim = nil
        pending.targetHealthAfter = nil
        pending.targetHealthLatest = nil
        pending.targetHealthDropAt = nil
        pending.targetDiedAt = nil
        local character: Model? = target and target.Character
        local humanoid: Humanoid? = character
            and character:FindFirstChildOfClass("Humanoid")
        if not humanoid then
            return
        end
        pending.targetHealthAtAim = humanoid.Health
        pending.targetHealthLatest = humanoid.Health
        if pending.healthConnection then
            pending.healthConnection:Disconnect()
            pending.healthConnection = nil
        end
        pending.healthConnection = humanoid.HealthChanged:Connect(
            function(health: number): ()
                if not predictionMath.finite(health) then
                    return
                end
                local previous: number? = pending.targetHealthLatest
                pending.targetHealthLatest = health
                if previous ~= nil and health < previous
                    and (pending.targetHealthAfter == nil or health < pending.targetHealthAfter) then
                    pending.targetHealthAfter = health
                    if pending.targetHealthDropAt == nil then
                        pending.targetHealthDropAt = os.clock()
                        pending.targetHealthDropServerTime =
                            workspace:GetServerTimeNow()
                    end
                    if health <= 0 and pending.targetDiedAt == nil then
                        pending.targetDiedAt = os.clock()
                        pending.targetDiedServerTime =
                            workspace:GetServerTimeNow()
                    end
                end
            end
        )
    end

    local function findKnifeAnalysisTarget(aimPosition: Vector3?): Player?
        local selected: Player? = nil
        local selectedScore: number = math.huge
        for _, player: Player in ipairs(Players:GetPlayers()) do
            if player ~= LocalPlayer
                and isPlayerAlive(player)
                and getPlayerRole(player) ~= "Murderer"
                and not isProtectedTarget(player) then
                local root: BasePart? = getTargetRoot(player)
                if root then
                    local score: number = aimPosition
                            and (root.Position - aimPosition).Magnitude
                        or (root.Position - (LocalPlayer.Character
                                and LocalPlayer.Character:GetPivot().Position
                            or Vector3.zero)).Magnitude
                    if score < selectedScore then
                        selected = player
                        selectedScore = score
                    end
                end
            end
        end
        return selected
    end

    local function getRequestedAim(): Vector3?
        local aim: Vector3? = nil
        pcall(function(): ()
            aim = playerMouse.Hit.Position
        end)
        return aim
    end

    local function samplePendingTrajectories(): ()
        local now: number = os.clock()
        local function sampleQueue(queue: {any}): ()
            for _, pending: any in ipairs(queue) do
                if pending.targetUserId
                    and now - pending.lastTrajectorySampleAt
                        >= 1 / TRAJECTORY_SAMPLE_HZ then
                    local target: Player? = Players:GetPlayerByUserId(
                        pending.targetUserId
                    )
                    local point: TrajectoryPoint? = target
                        and captureTrajectoryPoint(
                            target,
                            pending.activatedAt,
                            pending.pingMs
                        )
                        or nil
                    if point then
                        if #pending.targetTrajectory < MAX_TRAJECTORY_POINTS then
                            table.insert(pending.targetTrajectory, point)
                        else
                            pending.trajectoryCapped = true
                        end
                    elseif pending.samplingStoppedAt == nil then
                        -- The only two ways this is nil: the character is gone
                        -- or the humanoid is dead. Either way it is the reason
                        -- a saved trajectory ends early, so it gets written
                        -- down instead of being guessed at later.
                        pending.samplingStoppedAt = now
                        pending.samplingStoppedReason = target
                                and "target-dead-or-missing"
                            or "target-left"
                    end
                    if target then
                        local root: BasePart? = getTargetRoot(target)
                        if root then
                            pending.lastSeenPosition = root.Position
                        end
                    end
                    pending.lastTrajectorySampleAt = now
                end
            end
        end
        sampleQueue(runtime.pendingGuns)
        sampleQueue(runtime.pendingKnives)
        for index: number = #runtime.pendingGuns, 1, -1 do
            local pending: PendingGun = runtime.pendingGuns[index]
            if pending.confirmedAt and now - pending.confirmedAt >= 0.5 then
                table.remove(runtime.pendingGuns, index)
                finalizeGunCapture(pending)
            end
        end
        if #runtime.pendingGuns == 0 and #runtime.pendingKnives == 0
            and runtime.trajectorySampler then
            runtime.trajectorySampler:Disconnect()
            runtime.trajectorySampler = nil
        end
    end

    local function ensureTrajectorySampler(): ()
        if runtime.trajectorySampler then
            return
        end
        runtime.trajectorySampler = TaskManager:Connect(function(
            _deltaTime: number
        ): ()
            compactPending(runtime.pendingGuns, os.clock())
            compactPending(runtime.pendingKnives, os.clock())
            samplePendingTrajectories()
        end, nil, "MM2 shot sampler")
    end

    local function addSample(stats: RunningStats, value: number): ()
        if not finite(value) then
            return
        end
        stats.count += 1
        local delta: number = value - stats.mean
        stats.mean += delta / stats.count
        stats.m2 += delta * (value - stats.mean)
        stats.minimum = math.min(stats.minimum, value)
        stats.maximum = math.max(stats.maximum, value)
    end

    local function statsPayload(stats: RunningStats): {[string]: number}
        return {
            count = stats.count,
            mean = stats.mean,
            m2 = stats.m2,
            minimum = stats.count > 0 and stats.minimum or 0,
            maximum = stats.count > 0 and stats.maximum or 0,
        }
    end

    -- Telemetry only. Same source as the predictor (v190): Data Ping needs the
    -- Plugin capability and a restricted call on this thread is exactly what
    -- broke the gun, so this reads the engine value and nothing else.
    local function getPingMilliseconds(): number
        local pingMs: number = 0
        pcall(function(): ()
            local player: any = LocalPlayer
            local direct: any = player and player.GetNetworkPing
            if type(direct) ~= "function" then return end
            local ok: boolean, seconds: any = pcall(direct, player)
            if ok and finite(seconds) and (seconds :: number) > 0 then
                pingMs = (seconds :: number) * 1000
            end
        end)
        return math.max(0, pingMs)
    end

    local function worldPosition(value: any): Vector3?
        if typeof(value) == "Vector3" then
            return value
        elseif typeof(value) == "CFrame" then
            return value.Position
        elseif typeof(value) == "Instance" then
            if value:IsA("Attachment") then
                return value.WorldPosition
            elseif value:IsA("BasePart") then
                return value.Position
            elseif value:IsA("Model") then
                return value:GetPivot().Position
            end
        end
        return nil
    end

    local function projectilePosition(projectile: Instance): Vector3?
        local visual: Instance? = projectile:FindFirstChild("KnifeVisual", true)
        if visual and visual:IsA("BasePart") then
            return visual.Position
        end
        local blade: Instance? = projectile:FindFirstChild("BladePosition", true)
        local bladePosition: Vector3? = worldPosition(blade)
        if bladePosition then
            return bladePosition
        end
        return worldPosition(projectile)
    end

    local function toolOrigin(tool: Tool): Vector3?
        local handle: Instance? = tool:FindFirstChild("Handle")
        return worldPosition(handle)
    end

    local function appendEvent(record: CalibrationEvent): ()
        record.sessionId = runtime.sessionId
        record.buildStamp = runtime.buildStamp
        -- No destructive eviction: batches are archived after a verified
        -- snapshot, and stay in memory if archive IO fails.
        table.insert(runtime.events, record)
        runtime.revision += 1
        runtime.dirty = true
        -- Unconsented sessions never schedule a write. An absent config is
        -- also unanswered, not implicit permission.
        local states: any = type(configData) == "table" and configData.states or nil
        if not states or states["MM2.Calibration.Consent"] ~= true then
            return
        end
        if runtime.saveScheduled then
            return
        end
        runtime.saveScheduled = true
        task.delay(10, function(): ()
            runtime.saveScheduled = false
            if runtime.active and runtime.dirty then
                if #runtime.events >= MAX_EVENTS then
                    if saveSnapshot("before-archive") and archiveEvents() then
                        saveSnapshot("after-archive")
                    end
                else
                    saveSnapshot("autosave")
                end
            end
        end)
    end

    -- Pre-shot movement window, straight from the predictor's ring buffer.
    -- Without it a second predictor can only be scored on the shot it took;
    -- with it the same slot can be replayed from the inputs.
    local function captureTargetHistory(
        player: Player?,
        now: number
    ): {HistoryPoint}
        local points: {HistoryPoint} = {}
        if not player or not state.mm2Prediction
            or type(state.mm2Prediction.getHistory) ~= "function" then
            return points
        end
        local ok: boolean, history: any, age: any = pcall(
            state.mm2Prediction.getHistory,
            player,
            0.6
        )
        if not ok or type(history) ~= "table" then
            return points
        end
        -- `at` is relative to the newest predictor sample and `age` is how old
        -- that sample was at the shot. Clocks in the two modules do not share
        -- an epoch, so the subtraction happens on the predictor's side only.
        local sampleAge: number = predictionMath.finite(age) and age or 0
        -- v139.4: the cap used to `break` while walking oldest -> newest, so a
        -- 0.6 s window sampled above 60 FPS threw away the NEWEST samples --
        -- exactly the ones the scoring interpolates from, and the ones a replay
        -- needs to know where the target was at the shot. The window is
        -- decimated instead: the span survives, and both ends are kept.
        local HISTORY_LIMIT: number = 36
        local samples: {any} = {}
        for _, sample: any in ipairs(history) do
            local position: any = sample.position
            local velocity: any = sample.velocity
            if typeof(position) == "Vector3" and typeof(velocity) == "Vector3" then
                table.insert(samples, sample)
            end
        end
        local stride: number = 1
        if #samples > HISTORY_LIMIT then
            stride = math.ceil(#samples / (HISTORY_LIMIT - 1))
        end
        for index, sample: any in ipairs(samples) do
            local isLast: boolean = index == #samples
            if isLast or (index - 1) % stride == 0 then
                local position: any = sample.position
                local velocity: any = sample.velocity
                local move: any = sample.moveDirection
                local offset: any = sample.aimOffset
                table.insert(points, {
                    dt = (sample.at or 0) - sampleAge,
                    position = {position.X, position.Y, position.Z},
                    velocity = {velocity.X, velocity.Y, velocity.Z},
                    humanoidState = tostring(sample.state or "Running"),
                    grounded = sample.grounded == true,
                    moveDirection = typeof(move) == "Vector3"
                            and {move.X, move.Y, move.Z}
                        or nil,
                    aimOffset = typeof(offset) == "Vector3"
                            and {offset.X, offset.Y, offset.Z}
                        or nil,
                })
            end
        end
        return points
    end

    local function trackConnection(connection: RBXScriptConnection): ()
        table.insert(runtime.connections, connection)
    end

    compactPending = function(queue: {any}, now: number): ()
        local function drop(index: number): ()
            local entry: any = queue[index]
            if entry and entry.healthConnection then
                pcall(function(): ()
                    (entry.healthConnection :: RBXScriptConnection):Disconnect()
                end)
                entry.healthConnection = nil
            end
            table.remove(queue, index)
        end
        while #queue > 0 and now - queue[1].activatedAt > MAX_PENDING_AGE do
            drop(1)
        end
        while #queue > 12 do
            drop(1)
        end
    end

    local function vectorFromArray(value: {number}): Vector3
        return Vector3.new(value[1] or 0, value[2] or 0, value[3] or 0)
    end

    local function analyzeTargetTrajectory(
        trajectory: {TrajectoryPoint},
        pingMs: number,
        eventKind: string,
        evaluationSeconds: number?
    ): {[string]: any}
        if #trajectory < 2 then
            return {
                pingBucket = pingBucketKey(pingMs),
                sampleCount = #trajectory,
            }
        end
        local first: TrajectoryPoint = trajectory[1]
        local last: TrajectoryPoint = trajectory[#trajectory]
        if evaluationSeconds then
            local desiredDt: number = first.dt + evaluationSeconds
            local closestDistance: number = math.huge
            for _, point: TrajectoryPoint in ipairs(trajectory) do
                local distance: number = math.abs(point.dt - desiredDt)
                if distance < closestDistance then
                    last = point
                    closestDistance = distance
                end
            end
        end
        local initialPosition: Vector3 = vectorFromArray(first.position)
        local finalPosition: Vector3 = vectorFromArray(last.position)
        local initialVelocity: Vector3 = vectorFromArray(first.velocity)
        local duration: number = math.max(0.0001, last.dt - first.dt)
        local displacement: Vector3 = finalPosition - initialPosition
        local residual: Vector3 = displacement - initialVelocity * duration
        local constantVelocityPrediction: Vector3 = initialPosition
            + initialVelocity * duration
        local observedAcceleration: Vector3 = Vector3.zero
        if #trajectory >= 3 then
            local second: TrajectoryPoint = trajectory[2]
            local accelerationDelta: number = math.max(
                1 / 240,
                second.dt - first.dt
            )
            observedAcceleration = (
                vectorFromArray(second.velocity) - initialVelocity
            ) / accelerationDelta
            if observedAcceleration.Magnitude > 500 then
                observedAcceleration = observedAcceleration.Unit * 500
            end
        end
        local accelerationPrediction: Vector3 = initialPosition
            + initialVelocity * duration
            + observedAcceleration * (0.5 * duration * duration)
        local ballisticPrediction: Vector3 = initialPosition
            + initialVelocity * duration
            + Vector3.new(0, -workspace.Gravity, 0)
                * (0.5 * duration * duration)
        local modelErrors: {[string]: number} = {
            constantVelocity = (constantVelocityPrediction - finalPosition).Magnitude,
            observedAcceleration = (accelerationPrediction - finalPosition).Magnitude,
            airborneBallistic = (ballisticPrediction - finalPosition).Magnitude,
        }
        local bestMotionModel: string = "constantVelocity"
        local bestMotionError: number = modelErrors.constantVelocity
        for modelName: string, modelError: number in pairs(modelErrors) do
            if modelError < bestMotionError then
                bestMotionModel = modelName
                bestMotionError = modelError
            end
        end
        local horizontalVelocity: Vector3 = Vector3.new(
            initialVelocity.X,
            0,
            initialVelocity.Z
        )
        local horizontalDisplacement: Vector3 = Vector3.new(
            displacement.X,
            0,
            displacement.Z
        )
        local horizontalLead: number? = nil
        if horizontalVelocity.Magnitude > 0.75 then
            horizontalLead = math.clamp(
                horizontalDisplacement:Dot(horizontalVelocity)
                    / horizontalVelocity:Dot(horizontalVelocity),
                -0.15,
                0.6
            )
        end
        local verticalLead: number? = nil
        if math.abs(initialVelocity.Y) > 0.75 then
            verticalLead = math.clamp(
                displacement.Y / initialVelocity.Y,
                -0.15,
                0.6
            )
        end
        local jumped: boolean = false
        for _, point: TrajectoryPoint in ipairs(trajectory) do
            if point.jumping then
                jumped = true
                break
            end
        end
        local stateName: string = jumped and "Airborne" or "Grounded"
        local bucketKey: string = eventKind
            .. ":"
            .. pingBucketKey(pingMs)
            .. ":"
            .. stateName
        local bucket: any = runtime.motionBuckets[bucketKey]
        if type(bucket) ~= "table" then
            bucket = {
                count = 0,
                horizontalLeadSum = 0,
                horizontalLeadCount = 0,
                verticalLeadSum = 0,
                verticalLeadCount = 0,
                residualXSum = 0,
                residualYSum = 0,
                residualZSum = 0,
                durationSum = 0,
                constantVelocityErrorSum = 0,
                observedAccelerationErrorSum = 0,
                airborneBallisticErrorSum = 0,
            }
            runtime.motionBuckets[bucketKey] = bucket
        end
        bucket.count += 1
        bucket.durationSum += duration
        bucket.residualXSum += residual.X
        bucket.residualYSum += residual.Y
        bucket.residualZSum += residual.Z
        bucket.constantVelocityErrorSum += modelErrors.constantVelocity
        bucket.observedAccelerationErrorSum += modelErrors.observedAcceleration
        bucket.airborneBallisticErrorSum += modelErrors.airborneBallistic
        if horizontalLead then
            bucket.horizontalLeadSum += horizontalLead
            bucket.horizontalLeadCount += 1
        end
        if verticalLead then
            bucket.verticalLeadSum += verticalLead
            bucket.verticalLeadCount += 1
        end
        return {
            pingBucket = pingBucketKey(pingMs),
            sampleCount = #trajectory,
            initialState = first.humanoidState,
            finalState = last.humanoidState,
            jumped = jumped,
            durationSeconds = duration,
            evaluationSeconds = evaluationSeconds or duration,
            displacement = vectorArray(displacement),
            constantVelocityResidual = vectorArray(residual),
            horizontalLeadSeconds = horizontalLead,
            verticalLeadSeconds = verticalLead,
            bestMotionModel = bestMotionModel,
            motionModelErrors = modelErrors,
        }
    end

    local function motionModelPayload(buckets: {[string]: any}): {[string]: any}
        local payload: {[string]: any} = {}
        for key: string, bucket: any in pairs(buckets) do
            if type(bucket) == "table" then
                local count: number = math.max(1, tonumber(bucket.count) or 1)
                local horizontalCount: number = math.max(
                    1,
                    tonumber(bucket.horizontalLeadCount) or 0
                )
                local verticalCount: number = math.max(
                    1,
                    tonumber(bucket.verticalLeadCount) or 0
                )

                local meanDuration: number = tonumber(bucket.meanDurationSeconds)
                    or (bucket.durationSum and (bucket.durationSum / count))
                    or 0

                local meanHorizLead: number? = nil
                if bucket.horizontalLeadSum and horizontalCount > 0 then
                    meanHorizLead = bucket.horizontalLeadSum / horizontalCount
                elseif bucket.meanHorizontalLeadSeconds then
                    meanHorizLead = bucket.meanHorizontalLeadSeconds
                end

                local meanVertLead: number? = nil
                if bucket.verticalLeadSum and verticalCount > 0 then
                    meanVertLead = bucket.verticalLeadSum / verticalCount
                elseif bucket.meanVerticalLeadSeconds then
                    meanVertLead = bucket.meanVerticalLeadSeconds
                end

                local meanResidual: {number}
                if type(bucket.meanConstantVelocityResidual) == "table" then
                    meanResidual = bucket.meanConstantVelocityResidual
                else
                    meanResidual = {
                        (tonumber(bucket.residualXSum) or 0) / count,
                        (tonumber(bucket.residualYSum) or 0) / count,
                        (tonumber(bucket.residualZSum) or 0) / count,
                    }
                end

                local meanModelErr: {[string]: number}
                if type(bucket.meanModelError) == "table" then
                    meanModelErr = bucket.meanModelError
                else
                    meanModelErr = {
                        constantVelocity = (tonumber(bucket.constantVelocityErrorSum) or 0) / count,
                        observedAcceleration = (tonumber(bucket.observedAccelerationErrorSum) or 0) / count,
                        airborneBallistic = (tonumber(bucket.airborneBallisticErrorSum) or 0) / count,
                    }
                end

                payload[key] = {
                    durationSum = bucket.durationSum,
```

```
                horizontalLeadSum = bucket.horizontalLeadSum,
                horizontalLeadCount = bucket.horizontalLeadCount,
                verticalLeadSum = bucket.verticalLeadSum,
                verticalLeadCount = bucket.verticalLeadCount,
                residualXSum = bucket.residualXSum,
                residualYSum = bucket.residualYSum,
                residualZSum = bucket.residualZSum,
                constantVelocityErrorSum = bucket.constantVelocityErrorSum,
                observedAccelerationErrorSum = bucket.observedAccelerationErrorSum,
                airborneBallisticErrorSum = bucket.airborneBallisticErrorSum,
                count = tonumber(bucket.count) or 0,
                meanDurationSeconds = meanDuration,
                meanHorizontalLeadSeconds = meanHorizLead,
                meanVerticalLeadSeconds = meanVertLead,
                meanConstantVelocityResidual = meanResidual,
                meanModelError = meanModelErr,
            }
        end
    end
    return payload
end

local function median(values: {number}): number?
    if #values == 0 then
        return nil
    end
    local sorted: {number} = table.clone(values)
    table.sort(sorted)
    local middle: number = math.floor((#sorted + 1) / 2)
    if #sorted % 2 == 1 then
        return sorted[middle]
    end
    return (sorted[middle] + sorted[middle + 1]) * 0.5
end

local estimatesCache: Estimates? = nil
local estimatesAt: number = -math.huge
local function endpointErrorStats(): (number, number?, number?)
    local values: {number} = {}
    for _, ev in ipairs(runtime.events) do
        if ev.kind == "gun"
            and predictionMath.finite(ev.endpointErrorStuds) then
            table.insert(values, ev.endpointErrorStuds)
        end
    end
    local mean: number? = nil
    if #values > 0 then
        local total: number = 0
        for _, value in ipairs(values) do
            total += value
        end
        mean = total / #values
    end
    return #values, median(values), mean
end

local function estimates(): Estimates
    local now: number = os.clock()
    if estimatesCache and now - estimatesAt < 0.25 then return estimatesCache end
    local fresh: {number} = {}
    for _, sample in ipairs(runtime.freshGunLatency) do
        if now - sample.at <= 60 then table.insert(fresh, sample.ms) end
    end
    local gunStats: RunningStats = runtime.gunAcceptance
    local speedStats: RunningStats = runtime.knifeSpeed
    local spawnDelayStats: RunningStats = runtime.knifeSpawnDelay
    local maxStab: number? = nil
    for _, ev in ipairs(runtime.events) do
        if ev.kind == "stab" and ev.hitTarget == true
            and predictionMath.finite(ev.distance) then
            if not maxStab or ev.distance > maxStab then
                maxStab = ev.distance
            end
        end
    end
    local errCount: number, errMedian: number?, errMean: number? =
        endpointErrorStats()
    estimatesCache = {
        predictionErrorSamples = errCount,
        predictionErrorMedianStuds = errMedian,
        predictionErrorMeanStuds = errMean,
        gunAcceptanceMs = gunStats.count > 0 and gunStats.mean or nil,
        knifeSpeed = speedStats.count > 0 and speedStats.mean or nil,
        confirmedShots = gunStats.count,
        confirmedThrows = speedStats.count,
        maxStabDistance = maxStab,
        liveGunAcceptanceMs = #fresh >= 6 and median(fresh) or nil,
        liveGunSamples = #fresh,
        knifeSpawnDelayMs = spawnDelayStats.count > 0
            and spawnDelayStats.mean
            or nil,
        confirmedSpawnDelays = spawnDelayStats.count,
    }
    estimatesAt = now
    return estimatesCache :: Estimates
end

-- Keep the exact encoded observations in a fresh file only when the main
-- write is known/possibly corrupt. This is exceptional IO, not an extra
-- full snapshot every autosave. Never overwrite another recovery file.
local function preserveFailedSnapshot(encoded: string, generation: number, sessionId: string): ()
    local states: any = type(configData) == "table" and configData.states or nil
    if not states or states["MM2.Calibration.Consent"] ~= true or not runtime.active
        or runtime.observationGeneration ~= generation
        or type(environment.isfile) ~= "function" or type(environment.readfile) ~= "function"
        or type(environment.writefile) ~= "function"
        or (type(state.ownsRuntimeData) == "function" and not state.ownsRuntimeData()) then return end
    for _attempt: number = 1, 32 do
        runtime.recoverySequence += 1
        local path: string = OUTPUT_PATH .. ".recovery-" .. sessionId
            .. "-" .. tostring(runtime.recoverySequence) .. ".json"
        local okExists: boolean, exists: any = pcall(environment.isfile, path)
        if not okExists or type(exists) ~= "boolean" then break end
        if not exists then
            if states["MM2.Calibration.Consent"] ~= true or not runtime.active
                or runtime.observationGeneration ~= generation
                or (type(state.ownsRuntimeData) == "function" and not state.ownsRuntimeData()) then return end
            local okWrite: boolean = pcall(environment.writefile, path, encoded)
            local okRead: boolean, recovered: any = pcall(environment.readfile, path)
            if okWrite and okRead and recovered == encoded then
                warn(trajectoryLogPrefix .. " Verified recovery snapshot: " .. path)
            else
                warn(trajectoryLogPrefix .. " Recovery write could not be verified; data remains in memory.")
            end
            return
        end
    end
    warn(trajectoryLogPrefix .. " Could not reserve a recovery path; data remains in memory.")
end

local function writeSnapshot(reason: string): boolean
    local generation: number = runtime.observationGeneration
    local snapshotSessionId: string = runtime.sessionId
    -- Without an explicit Yes the recorder is inactive and does not write
    -- telemetry. A No also preserves files from earlier sessions.
    local consentStates: any = type(configData) == "table" and configData.states or nil
    if not consentStates or consentStates["MM2.Calibration.Consent"] ~= true then
        return false
    end
    if runtime.persistenceBlocked
        or (type(state.ownsRuntimeData) == "function" and not state.ownsRuntimeData()) then
        return false
    end
    if type(environment.writefile) ~= "function" then
        warn(trajectoryLogPrefix .. " writefile is unavailable; data was not saved.")
        return false
    end
    local prior: any = runtime.pendingHistoryBackup
    if type(prior) == "string" then
        -- A different writer may have changed the file while the consent
        -- dialog was open. Never replace bytes we did not import/back up.
        local okCurrent, current = pcall(environment.readfile, OUTPUT_PATH)
        if not okCurrent or current ~= prior then
            runtime.persistenceBlocked = true
            warn(trajectoryLogPrefix .. " History changed before consent; it will not be overwritten.")
            return false
        end
        if not runtime.historyBackup then
            local backup: string = OUTPUT_PATH .. ".before-" .. runtime.sessionId .. ".json"
            local okExists: boolean, exists: any = pcall(environment.isfile, backup)
            if not okExists or exists ~= false then
                runtime.persistenceBlocked = true
                warn(trajectoryLogPrefix .. " Backup path is occupied or unavailable; existing files are untouched.")
                return false
            end
            if consentStates["MM2.Calibration.Consent"] ~= true or not runtime.active
                or runtime.observationGeneration ~= generation
                or (type(state.ownsRuntimeData) == "function" and not state.ownsRuntimeData()) then return false end
            local okBackup: boolean = pcall(environment.writefile, backup, prior)
            local okVerify, backedUp = pcall(environment.readfile, backup)
            if not okBackup or not okVerify or backedUp ~= prior then
                runtime.persistenceBlocked = true
                warn(trajectoryLogPrefix .. " History backup failed; existing file will not be overwritten.")
                return false
            end
        end
        runtime.historyBackup = backup
    end
end

-- History is loaded once. Re-merging the on-disk totals at each save
-- used to double counters/buckets and read/decode a large JSON per shot.
local savedRevision: number = runtime.revision
local mergedGun = statsPayload(runtime.gunAcceptance)
local mergedKnife = statsPayload(runtime.knifeSpeed)
local mergedDelay = statsPayload(runtime.knifeSpawnDelay)
local payload: {[string]: any} = {
    schema = 4,
    kind = "mm2-trajectory-analytics",
    buildStamp = runtime.buildStamp,
    sessionId = runtime.sessionId,
    historyBackup = runtime.historyBackup,
    settings = {
        silentSpawn = mm2Settings.silentSpawn,
        silentAim = mm2Settings.silentAim,
        silentSweep = mm2Settings.silentSweep,
        wallCheck = mm2Settings.shootWallCheck,
        autoLead = state.mm2Prediction
            and state.mm2Prediction.settings.autoLead
            or true,
        leadOffsetSeconds = state.mm2Prediction
            and state.mm2Prediction.settings.extraLead
            or 0,
        replicationInterpolationSeconds = state.mm2Prediction
            and state.mm2Prediction.settings.interpolation
            or nil,
    },
    placeId = game.PlaceId,
    savedAt = DateTime.now():ToIsoDate(),
    reason = reason,
    aggregateScope = "validated-history-plus-this-session; not a hit-rate denominator",
    aggregate = {
        gunAcceptanceMs = mergedGun,
        knifeSpeedStudsPerSecond = mergedKnife,
        knifeSpawnDelayMs = mergedDelay,
    },
    -- The learned model, portable across sessions and machines. Same
    -- shape the import path reads above.
    predictor = (function(): {[string]: any}
        local out: {[string]: any} = {}
        local predictorState: any = state.mm2Prediction
        if predictorState and type(predictorState.exportLeadState) == "function" then
            local ok: boolean, value: any = pcall(predictorState.exportLeadState)
            if ok and type(value) == "table" then
                out.leadErrors = value
            end
        end
        if predictorState and type(predictorState.getLeadSamples) == "function" then
            local ok: boolean, guns: any = pcall(predictorState.getLeadSamples, "gun")
            local okKnife: boolean, knives: any = pcall(predictorState.getLeadSamples, "knife")
            out.samples = {
                gun = ok and guns or nil,
                knife = okKnife and knives or nil,
            }
        end
        out.readyFor = "3+ recent samples activate a correction; each mode retains up to 8 (not proof of convergence)"
        return out
    end)(),
    estimator = {
        gunAcceptanceMs = mergedGun.count > 0 and mergedGun.mean or nil,
        knifeSpeed = mergedKnife.count > 0 and mergedKnife.mean or nil,
        confirmedShots = mergedGun.count,
        confirmedThrows = mergedKnife.count,
    },
    analytics = {
        bucketSpec = {
            metric = "Data Ping",
            widthMs = 5,
            minimumMs = 0,
            maximumMs = 250,
            states = {"Grounded", "Airborne"},
        },
        motionModel = motionModelPayload(runtime.motionBuckets),
    },
    session = {
        startedAt = runtime.startedAtUnix,
        countersScope = "this-session-only",
        gunConfirmationMeaning = "matched GunFired signal; hitTarget means the ray reached the target, died means the humanoid reached 0 health",
        evictedEvents = runtime.evictedEvents,
        archivedEvents = runtime.archivedEvents,
        archiveFiles = runtime.archiveFiles,
        eventScope = "events in this snapshot plus the listed verified session archive files",
        healthMeaning = "client-observed Humanoid.Health; healthDropServerTime is GetServerTimeNow at the drop",
        knifeConfirmationMeaning = "own linked projectile with valid speed; not an impact",
        knifeObservedMeaning = "all ThrowingKnife projectiles visible to this client, not all server events",
        knifeObserved = runtime.sessionKnifeObserved,
        rawKnifeActivations = runtime.rawKnifeActivations,
        unmatchedGunSignals = runtime.unmatchedGunSignals,
        stabAttempts = runtime.sessionStabAttempts,
        elapsedSeconds = os.clock() - runtime.startedAt,
        predictionError = (function(): {[string]: any}
            local ok: boolean, n: any, med: any, mean: any =
                pcall(endpointErrorStats)
            return {
                samples = ok and n or 0,
                medianStuds = ok and med or nil,
                meanStuds = ok and mean or nil,
                -- v139.4: say what it is. The reference is the torso
                -- point the aim used, interpolated between the newest
                -- pre-shot sample and the root at confirmation, so this
                -- is the client's best estimate of the miss distance --
                -- not a server-side measurement.
                scope = "own predicted point vs the torso point at the scoring instant (interpolated from the newest pre-shot sample and the root at confirmation); an estimate, not a server measurement",
            }
        end)(),
        gunByModeScope = "events in the current segment only",
        gunByMode = (function(): {[string]: any}
            local modes: {[string]: any} = {}
            if type(runtime.events) == "table" then
                for _, ev in ipairs(runtime.events) do
                    if ev.kind == "gun" then
                        if ev.phase == "authored" then
                            local mode: string = type(ev.settings) == "table"
                                    and tostring(ev.settings.mode or "?")
                                or "?"
                            local row: any = modes[mode]
                                or {attempts = 0, confirmed = 0, hits = 0, kills = 0}
                            row.attempts += 1
                            modes[mode] = row
                        elseif ev.phase == "confirmation" then
                            local mode: string =
                                tostring(ev.shotMode or "?")
                            local row: any = modes[mode]
                                or {attempts = 0, confirmed = 0, hits = 0, kills = 0}
                            row.confirmed += 1
                            if ev.hitTarget then row.hits += 1 end
                            if ev.died then row.kills += 1 end
                            modes[mode] = row
                        end
                    end
                end
            end
            return modes
        end)(),
        gunAttempts = runtime.sessionGunAttempts,
        gunConfirmed = runtime.sessionGunConfirmed,
        gunHitTarget = runtime.sessionGunHitTarget,
        gunKills = runtime.sessionGunKills,
        -- v139.2: hits are only half the story; this splits the rest
        -- into "the ray carried into another player" and "the ray
        -- stopped on the map".
        gunMisses = runtime.sessionGunMisses or 0,
        gunMissReasons = runtime.missReasons or {},
        knifeAttempts = runtime.sessionKnifeAttempts,
        knifeConfirmed = runtime.sessionKnifeConfirmed,
        events = runtime.events,
    },
}
local encodedOk: boolean, encoded: any = pcall(
    HttpService.JSONEncode,
    HttpService,
    payload
)
if not encodedOk or type(encoded) ~= "string" then
    warn(trajectoryLogPrefix .. " JSONEncode failed: " .. tostring(encoded))
    return false
end
if type(environment.makefolder) == "function" then
    pcall(environment.makefolder, OUTPUT_ROOT)
    pcall(environment.makefolder, OUTPUT_FOLDER)
end
if consentStates["MM2.Calibration.Consent"] ~= true or not runtime.active
    or runtime.observationGeneration ~= generation
    or (type(state.ownsRuntimeData) == "function" and not state.ownsRuntimeData()) then
    return false
end
-- Recheck immediately before the main write. Backup, encoding and
-- folder IO may all yield; another writer could have changed or created
-- the file since the earlier check. Executor IO is not atomic, but this
-- avoids knowingly replacing bytes we did not import or write.
local expected: string? = if type(prior) == "string" then prior else runtime.lastSnapshotBytes
if type(expected) == "string" then
    local okCurrent: boolean, current: any = pcall(environment.readfile, OUTPUT_PATH)
    if not okCurrent or current ~= expected then
        runtime.persistenceBlocked = true
        warn(trajectoryLogPrefix .. " History changed while saving; it will not be overwritten.")
        return false
    end
elseif runtime.historyExpectedAbsent then
    local okExists: boolean, exists: any = pcall(environment.isfile, OUTPUT_PATH)
    if not okExists or exists ~= false then
        runtime.persistenceBlocked = true
        warn(trajectoryLogPrefix .. " History appeared before saving; it will not be overwritten.")
        return false
    end
end
if consentStates["MM2.Calibration.Consent"] ~= true or not runtime.active
    or runtime.observationGeneration ~= generation
    or (type(state.ownsRuntimeData) == "function" and not state.ownsRuntimeData()) then
    return false
end
local writeOk: boolean, writeError: any = pcall(
    environment.writefile,
    OUTPUT_PATH,
    encoded
)
if not writeOk then
    warn(trajectoryLogPrefix .. " writefile failed: " .. tostring(writeError))
    -- Some executors throw *after* truncating the target. A clean
    -- failure may be retried; unknown/changed bytes must be protected.
    local changed: boolean
    if type(expected) == "string" then
        local okCurrent: boolean, current: any = pcall(environment.readfile, OUTPUT_PATH)
        changed = not okCurrent or current ~= expected
    else
        local okExists: boolean, exists: any = pcall(environment.isfile, OUTPUT_PATH)
        changed = not okExists or exists ~= false
    end
    if changed then
        runtime.persistenceBlocked = true
        preserveFailedSnapshot(encoded, generation, snapshotSessionId)
    end
    return false
end
-- Some executors return success after a truncated write. Retain the
-- prior verified backup and buffered observations instead of reporting
-- that corrupt bytes were saved or clearing the dirty flag.
local okRead: boolean, written: any = pcall(environment.readfile, OUTPUT_PATH)
if not okRead or written ~= encoded then
    runtime.persistenceBlocked = true
    warn(trajectoryLogPrefix .. " Snapshot readback failed; the previous verified backup is kept.")
    preserveFailedSnapshot(encoded, generation, snapshotSessionId)
    return false
end
-- This exact write began while consent was valid. Even if No arrived
-- while it yielded, remember its verified bytes as our own so a later
-- Yes does not misidentify them as an external writer. This does not
-- start any new write after No.
runtime.lastSnapshotBytes = encoded
runtime.pendingHistoryBackup = nil
runtime.historyExpectedAbsent = false
if consentStates["MM2.Calibration.Consent"] ~= true or not runtime.active
    or runtime.observationGeneration ~= generation
    or (type(state.ownsRuntimeData) == "function" and not state.ownsRuntimeData()) then
    return true
end
-- A yielding executor write must not clear events recorded meanwhile.
runtime.dirty = runtime.revision ~= savedRevision
print(
    trajectoryLogPrefix .. " Saved "
        .. OUTPUT_PATH
        .. " ("
        .. tostring(#encoded)
        .. " bytes)"
)
return true
end

saveSnapshot = function(reason: string): boolean
    -- Executor file functions may yield. Serialize snapshots so an older
    -- writer cannot finish after a newer one and roll the file backwards.
    if runtime.snapshotInProgress then
        runtime.resaveRequested = true
        return false
    end
    runtime.snapshotInProgress = true
    local ok: boolean, saved: any = pcall(writeSnapshot, reason)
    runtime.snapshotInProgress = false
    local resave: boolean = runtime.resaveRequested
    runtime.resaveRequested = false
    if not ok then warn(trajectoryLogPrefix .. " Snapshot error: " .. tostring(saved)) end
    if resave and runtime.active and runtime.dirty and not runtime.persistenceBlocked then
        task.defer(function(): ()
            if runtime.active and runtime.dirty then saveSnapshot("coalesced") end
        end)
    end
    return ok and saved == true
end

archiveEvents = function(): boolean
    local states: any = type(configData) == "table" and configData.states or nil
    if not runtime.active or not states or states["MM2.Calibration.Consent"] ~= true
        or runtime.persistenceBlocked or runtime.archiveInProgress
        or type(environment.writefile) ~= "function"
        or type(environment.readfile) ~= "function"
        or (type(state.ownsRuntimeData) == "function" and not state.ownsRuntimeData()) then
        return false
    end
    runtime.archiveInProgress = true
    local generation: number = runtime.observationGeneration
    local archiveSessionId: string = runtime.sessionId
    local captured: {any} = table.clone(runtime.events)
    local count: number = #captured
    local path: string? = nil
    -- A failed/partial write can leave a file behind. Never overwrite it,
    -- but try a new name on this or the next save so it cannot permanently
    -- stall archiving an otherwise recoverable session.
    for _attempt: number = 1, 64 do
        runtime.archiveSequence += 1
        local candidate: string = OUTPUT_FOLDER .. "/MM2_Trajectory_" .. runtime.sessionId
            .. "_part_" .. tostring(runtime.archiveSequence) .. ".json"
        local okExists: boolean, exists: any = false, nil
        if type(environment.isfile) == "function" then
            okExists, exists = pcall(environment.isfile, candidate)
        end
        if not okExists or type(exists) ~= "boolean" then break end
        if not exists then path = candidate; break end
    end
    if not path then
        runtime.archiveInProgress = false
        warn(trajectoryLogPrefix .. " Cannot verify a fresh archive path; events remain in memory.")
        return false
    end
    local encodedOk: boolean, encoded: any = pcall(HttpService.JSONEncode, HttpService, {
        schema = 4, kind = "mm2-trajectory-event-archive", sessionId = runtime.sessionId,
        buildStamp = runtime.buildStamp, events = captured,
    })
    if not encodedOk or type(encoded) ~= "string" then
        runtime.archiveInProgress = false
        warn(trajectoryLogPrefix .. " Archive encode failed; events remain in memory.")
        return false
    end
    -- Any lookup/encode can yield in an executor. Do not start an archive
    -- after No or a reset/restart, even if the operation began earlier.
    if not runtime.active or not states["MM2.Calibration.Consent"] ~= true
        or runtime.observationGeneration ~= generation or runtime.sessionId ~= archiveSessionId
        or (type(state.ownsRuntimeData) == "function" and not state.ownsRuntimeData()) then
        runtime.archiveInProgress = false
        return false
    end
    -- A yielding write can overlap No/unload; do not discard the live
    -- buffer unless this exact archive is verified and consent still holds.
    local okWrite: boolean = pcall(environment.writefile, path, encoded)
    local okRead: boolean, verified: any = pcall(environment.readfile, path)
    if not okWrite or not okRead or verified ~= encoded
        or runtime.persistenceBlocked or runtime.active ~= true
        or runtime.observationGeneration ~= generation or runtime.sessionId ~= archiveSessionId
        or (type(state.ownsRuntimeData) == "function" and not state.ownsRuntimeData()) then
        runtime.archiveInProgress = false
        warn(trajectoryLogPrefix .. " Archive could not be verified; events remain in memory.")
        return false
    end
    local remaining: {any} = {}
    for index: number = count + 1, #runtime.events do
        table.insert(remaining, runtime.events[index])
    end
    runtime.events = remaining
    runtime.archivedEvents += count
    table.insert(runtime.archiveFiles, path)
    runtime.revision += 1
    runtime.dirty = true
    runtime.archiveInProgress = false
    return true
end

local function normalizeGun(value: any): Tool?
    if typeof(value) ~= "Instance" then
        return nil
    end
    if value:IsA("Tool") then
        return value
    end
    return value:FindFirstAncestorOfClass("Tool")
end

-- Where the target really was when the server scored the shot, compared
-- with the point we aimed at, projected on the target's own horizontal
-- velocity. That projection is the lead error in seconds: positive means
-- the target was ahead of our aim (we under-led), negative means we led too
-- far. The server scores roughly one way into the measured round trip, so
-- the target's root is interpolated between the newest pre-shot sample and
-- the root captured at confirmation.
local function measureLeadErrorSeconds(pending: PendingGun): number?
    local history: {HistoryPoint} = pending.targetHistory or {}
    local newest: HistoryPoint? = history[#history]
    local aim: Vector3? = pending.requestedAim
    local latencyMs: any = pending.latencyMs
    local confirmation: any = pending.rootAtConfirmation
    if not newest or not aim or not confirmation then
        return nil
    end
    if not predictionMath.finite(latencyMs) or latencyMs <= 0
        or latencyMs > 1200 then
        return nil
    end
    if not predictionMath.finite(pending.activatedAt) then
        return nil
    end
    local beforeAt: number = pending.activatedAt + (newest.dt or 0)
    local afterAt: number = pending.confirmedAt
        and pending.confirmedAt
        or (pending.activatedAt + latencyMs / 1000)
    if not predictionMath.finite(afterAt) or afterAt <= beforeAt then
        return nil
    end
    -- Score where the server actually scores. The saved sessions
    -- separate hits from misses at the CONFIRMATION instant (a
    -- full round trip after firing), not half-way: scoring at
    -- activatedAt + latency/2 measured the aim ~55 ms AHEAD and
    -- taught the feedback loop a capped -60 ms correction, which
    -- then fought the (correct) full-transport lead on every
    -- shot -- the recorded cause of the chronic under-lead.
    local scoringAt: number = pending.activatedAt + latencyMs / 1000
    local alpha: number = math.clamp(
        (scoringAt - beforeAt) / (afterAt - beforeAt),
        0,
        1
    )
    local p0: {number}? = newest.position
    local p1: {number}? = confirmation.position
    if type(p0) ~= "table" or type(p1) ~= "table"
        or #p0 < 3 or #p1 < 3 then
        return nil
    end
    local rootX: number = p0[1] + (p1[1] - p0[1]) * alpha
    local rootY: number = p0[2] + (p1[2] - p0[2]) * alpha
    local rootZ: number = p0[3] + (p1[3] - p0[3]) * alpha
    local offset: {number}? = newest.aimOffset
    if type(offset) == "table" and #offset >= 3 then
        rootX += offset[1]
        rootY += offset[2]
        rootZ += offset[3]
    end
    local velocity: {number}? = newest.velocity
    if type(velocity) ~= "table" or #velocity < 3 then
        return nil
    end
    local vx: number = velocity[1]
    local vz: number = velocity[3]
    local speed: number = math.sqrt(vx * vx + vz * vz)
    -- A target that is not walking has no lead to correct; its aim error is
    -- noise around a stationary body.
    if speed < 3 then
        return nil
    end
    local along: number = ((rootX - aim.X) * vx + (rootZ - aim.Z) * vz) / speed
    -- Anything past this is not a lead error: a respawn, a teleport or the
    -- aim landing on a different player.
    if math.abs(along) > 6 then
        return nil
    end
    return along / speed
end

-- Same scoring-instant interpolation as measureLeadErrorSeconds,
-- but the full 3D distance between the point we aimed at and where
-- the target's aim point really was -- the direct endpoint error.
-- No speed gate: a stationary target's error is real information
-- here (it isolates geometry/origin problems from lead problems).
local function measureEndpointErrorStuds(pending: PendingGun): number?
    local history: {HistoryPoint} = pending.targetHistory or {}
    local newest: HistoryPoint? = history[#history]
    local aim: Vector3? = pending.requestedAim
    local latencyMs: any = pending.latencyMs
    local confirmation: any = pending.rootAtConfirmation
    if not newest or not aim or not confirmation then
        return nil
    end
    if not predictionMath.finite(latencyMs) or latencyMs <= 0
        or latencyMs > 1200 then
        return nil
    end
    if not predictionMath.finite(pending.activatedAt) then
        return nil
    end
    local beforeAt: number = pending.activatedAt + (newest.dt or 0)
    local afterAt: number = pending.confirmedAt
        and pending.confirmedAt
        or (pending.activatedAt + latencyMs / 1000)
    if not predictionMath.finite(afterAt) or afterAt <= beforeAt then
        return nil
    end
    local scoringAt: number = pending.activatedAt + latencyMs / 1000
    local alpha: number = math.clamp(
        (scoringAt - beforeAt) / (afterAt - beforeAt),
        0,
        1
    )
    local p0: {number}? = newest.position
    local p1: {number}? = confirmation.position
    if type(p0) ~= "table" or type(p1) ~= "table"
        or #p0 < 3 or #p1 < 3 then
        return nil
    end
    local rootX: number = p0[1] + (p1[1] - p0[1]) * alpha
    local rootY: number = p0[2] + (p1[2] - p0[2]) * alpha
    local rootZ: number = p0[3] + (p1[3] - p0[3]) * alpha
    local offset: {number}? = newest.aimOffset
    if type(offset) == "table" and #offset >= 3 then
        rootX += offset[1]
        rootY += offset[2]
        rootZ += offset[3]
    end
    local dx: number = rootX - aim.X
    local dy: number = rootY - aim.Y
```

```
        local dz: number = rootZ - aim.Z
        local distance: number = math.sqrt(dx * dx + dy * dy + dz * dz)
        if not predictionMath.finite(distance) or distance > 30 then
            return nil
        end
        return distance
    end

    finalizeGunCapture = function(pending: PendingGun): ()
        if pending.targetDiedAt then
            runtime.sessionGunKills += 1
        end
        if pending.healthConnection then
            pcall(function(): ()
                (pending.healthConnection :: RBXScriptConnection):Disconnect()
            end)
            pending.healthConnection = nil
        end
        -- The echo lives here on purpose. The shot-feedback handler that used
        -- to feed it depends on a connection made when the feature toggles;
        -- the v61 log shows it never fired (echoDelay nil on all 13 shots
        -- despite 9 confirmations). This handler is the one that produced
        -- those 9 confirmations, so it is the reliable place to measure.
        if state.mm2Prediction and type(state.mm2Prediction.noteEcho) == "function"
            and predictionMath.finite(pending.latencyMs)
            and predictionMath.finite(pending.activatedAt) then
            state.mm2Prediction.noteEcho(
                pending.activatedAt,
                pending.activatedAt + pending.latencyMs / 1000
            )
        end
        -- Feed the measured lead error back into the predictor before the
        -- record is written, so the event can show what the next shot used.
        -- v161: the feedback is filed per aim mode. Bullet (native gun ray)
        -- and CFrame (explicit origin+endpoint) are authored differently and
        -- nothing proves the server scores both with the same rewind, so a
        -- bias measured on one must not steer the other.
        local leadError: number? = measureLeadErrorSeconds(pending)
        if leadError and state.mm2Prediction
            and type(state.mm2Prediction.noteLeadError) == "function" then
            pcall(state.mm2Prediction.noteLeadError, leadError, pending.shotMode)
        end
        local leadCorrection: number? = nil
        local leadCorrectionTarget: number? = nil
        local leadFeedbackSamples: number? = nil
        if state.mm2Prediction then
            if type(state.mm2Prediction.getLeadCorrection) == "function" then
                local ok: boolean, value: any = pcall(
                    state.mm2Prediction.getLeadCorrection,
                    pending.shotMode
                )
                if ok and predictionMath.finite(value) then
                    leadCorrection = value
                end
            end
            if type(state.mm2Prediction.getLeadCorrectionTarget) == "function" then
                local ok: boolean, value: any = pcall(
                    state.mm2Prediction.getLeadCorrectionTarget,
                    pending.shotMode
                )
                if ok and predictionMath.finite(value) then
                    leadCorrectionTarget = value
                end
            end
            if type(state.mm2Prediction.getLeadSamples) == "function" then
                local ok: boolean, value: any = pcall(
                    state.mm2Prediction.getLeadSamples,
                    pending.shotMode
                )
                if ok and type(value) == "number" then
                    leadFeedbackSamples = value
                end
            end
        end
        local endpointError: number? = measureEndpointErrorStuds(pending)
        local analysis: {[string]: any} = analyzeTargetTrajectory(
            pending.targetTrajectory,
            pending.pingMs,
            "gun",
            (pending.latencyMs or 0) / 1000
        )
        appendEvent({
            shotId = pending.shotId,
            phase = "confirmation",
            kind = "gun",
            serverTime = workspace:GetServerTimeNow(),
            clock = os.clock(),
            pingMs = pending.pingMs,
            latencyMs = pending.latencyMs,
            targetUserId = pending.targetUserId,
            targetName = pending.targetName,
            targetState = analysis.initialState,
            targetJumped = analysis.jumped,
            targetTrajectory = pending.targetTrajectory,
            targetHistory = #(pending.targetHistory or {}) > 0
                and pending.targetHistory
                or nil,
            observedDisplacement = analysis.displacement,
            constantVelocityResidual = analysis.constantVelocityResidual,
            horizontalLeadSeconds = analysis.horizontalLeadSeconds,
            verticalLeadSeconds = analysis.verticalLeadSeconds,
            endpoint = pending.confirmedEndpoint
                and vectorArray(pending.confirmedEndpoint)
                or nil,
            hitTarget = pending.confirmedHitTarget,
            shotMode = pending.shotMode,
            silentSpawn = pending.silentSpawn,
            -- Server-clock round trip: activation to GunFired, both stamped
            -- with GetServerTimeNow. Independent of GetNetworkPing and of the
            -- engine's Data Ping, so it settles which of them the lead needs.
            serverEchoMs = (pending.serverConfirmedAt
                and pending.serverActivatedAt)
                and (pending.serverConfirmedAt - pending.serverActivatedAt)
                    * 1000
                or nil,
            -- Anchor for the server-clock axis: every trajectory point carries
            -- its own serverTime, so serverDt = point.serverTime - this.
            serverActivatedAt = pending.serverActivatedAt,
            serverConfirmedAt = pending.serverConfirmedAt,
            serverOrigin = pending.confirmedServerOrigin
                and vectorArray(pending.confirmedServerOrigin)
                or nil,
            aimOffsetAtFire = (function(): {number}?
                local history: {HistoryPoint} = pending.targetHistory or {}
                local newest: HistoryPoint? = history[#history]
                return newest and newest.aimOffset or nil
            end)(),
            -- Signed lead error measured from this very confirmation, and the
            -- bounded correction the next shot will apply. Both are written in
            -- seconds so a saved session can be replayed and audited without
            -- rerunning the shot. v161 adds what the median asks for before
            -- the slew limit and how many confirmations back it, so a session
            -- can show whether the loop is moving and how fast.
            leadErrorSeconds = leadError,
            leadCorrectionSeconds = leadCorrection,
            leadCorrectionTargetSeconds = leadCorrectionTarget,
            leadFeedbackSamples = leadFeedbackSamples,
            aimMode = pending.shotMode,
            endpointErrorStuds = endpointError,
            trajectoryPoints = #pending.targetTrajectory,
            trajectorySpanSeconds = (
                #pending.targetTrajectory > 1
                and (pending.targetTrajectory[#pending.targetTrajectory].dt
                    - pending.targetTrajectory[1].dt)
                or 0,
            trajectoryCapped = pending.trajectoryCapped == true,
            samplingStoppedReason = pending.samplingStoppedReason,
            samplingStoppedAt = pending.samplingStoppedAt,
            rttEngineAtConfirmation = pending.rttEngineAtConfirmation,
            rttDataAtConfirmation = pending.rttDataAtConfirmation,
            aim = pending.requestedAim and vectorArray(pending.requestedAim) or nil,
            origin = pending.origin and vectorArray(pending.origin) or nil,
            hitPart = pending.confirmedHitPart,
            hitPartPosition = pending.confirmedHitPartPosition
                and vectorArray(pending.confirmedHitPartPosition)
                or nil,
            hitPartSize = pending.confirmedHitPartSize
                and vectorArray(pending.confirmedHitPartSize)
                or nil,
            rootAtConfirmation = pending.rootAtConfirmation,
            lastSeenPosition = pending.lastSeenPosition
                and vectorArray(pending.lastSeenPosition)
                or nil,
            healthAtAim = pending.targetHealthAtAim,
            healthAtConfirmation = pending.targetHealthAtConfirmation,
            healthAfter = pending.targetHealthAfter,
            healthLatest = pending.targetHealthLatest,
            healthEvidenceScope = "client-observed health; not proof this shot caused a change",
            healthDropAt = pending.targetHealthDropAt,
            healthDropServerTime = pending.targetHealthDropServerTime,
            diedAt = pending.targetDiedAt,
            diedServerTime = pending.targetDiedServerTime,
            died = pending.targetDiedAt ~= nil,
            pingBucket = analysis.pingBucket,
            bestMotionModel = analysis.bestMotionModel,
            motionModelErrors = analysis.motionModelErrors,
            evaluationSeconds = analysis.evaluationSeconds,
            -- Why this shot did or did not score, decided from the hit part's
            -- own owner: "hit", "other-player", "geometry" or "no-hit-part".
            missReason = missReason,
        })
    end

    local function handleGunFired(
        eventGun: any,
        eventOriginValue: any,
        endpointValue: any,
        hitPartValue: any
    ): ()
        if not runtime.active then
            return
        end
        local now: number = os.clock()
        compactPending(runtime.pendingGuns, now)
        local eventTool: Tool? = normalizeGun(eventGun)
        local eventOrigin: Vector3? = worldPosition(eventOriginValue)
        -- Match the OLDEST unconfirmed attempt whose tool and origin geometry
        -- agree with the signal. Iterating newest-first let a signal confirm
        -- a younger shot and feed a wrong latency into the adaptive horizon.
        local selectedIndex: number? = nil
        local fallbackIndex: number? = nil
        for index: number = 1, #runtime.pendingGuns do
            local pending: PendingGun = runtime.pendingGuns[index]
            if not pending.confirmedAt then
                local toolMatches: boolean = eventTool ~= nil
                    and eventTool == pending.tool
                local originMatches: boolean = eventOrigin ~= nil
                    and pending.origin ~= nil
                    and (eventOrigin - pending.origin).Magnitude <= 8
                if toolMatches and originMatches then
                    selectedIndex = index
                    break
                end
                if fallbackIndex == nil then
                    if toolMatches
                        and (eventOrigin == nil or pending.origin == nil) then
                        fallbackIndex = index
                    elseif eventTool == nil and originMatches then
                        fallbackIndex = index
                    end
                end
            end
        end
        if not selectedIndex then
            selectedIndex = fallbackIndex
        end
        if not selectedIndex then
            local character: Model? = LocalPlayer.Character
            local root: BasePart? = character and character:FindFirstChild("HumanoidRootPart") :: BasePart?
            local isLocalShooter: boolean = false
            if eventTool and eventTool.Parent == character then
                isLocalShooter = true
            elseif root and eventOrigin and (eventOrigin - root.Position).Magnitude <= 20 then
                isLocalShooter = true
            end
            if isLocalShooter then
                runtime.unmatchedGunSignals += 1
                appendEvent({kind = "gun", phase = "unmatched-signal",
                    serverTime = workspace:GetServerTimeNow(), pingMs = getPingMilliseconds()})
            end
            return
        end
        local pending: PendingGun = runtime.pendingGuns[selectedIndex]
        local latencyMs: number = math.max(0, (now - pending.activatedAt) * 1000)
        local target: Player? = pending.targetUserId
            and Players:GetPlayerByUserId(pending.targetUserId)
            or nil
        if target then
            local finalPoint: TrajectoryPoint? = captureTrajectoryPoint(
                target,
                pending.activatedAt,
                pending.pingMs
            )
            if finalPoint then
                table.insert(pending.targetTrajectory, finalPoint)
            end
        end
        local endpoint: Vector3? = worldPosition(endpointValue)
        local targetCharacter: Model? = target and target.Character
        local hitTarget: boolean = typeof(hitPartValue) == "Instance"
            and targetCharacter ~= nil
            and hitPartValue:IsDescendantOf(targetCharacter)
        -- Recorded at the moment the signal arrives, on the server clock, so
        -- the round trip can be measured without either engine estimate.
        pending.serverConfirmedAt = workspace:GetServerTimeNow()
        pending.confirmedServerOrigin = eventOrigin
        local engineNow: number?, dataNow: number? = readLatencyReadings()
        pending.rttEngineAtConfirmation = engineNow
        pending.rttDataAtConfirmation = dataNow
        if typeof(hitPartValue) == "Instance" then
            pcall(function(): ()
                pending.confirmedHitPart = string.sub(hitPartValue.Name, 1, 80)
                if hitPartValue:IsA("BasePart") then
                    pending.confirmedHitPartPosition = hitPartValue.Position
                    pending.confirmedHitPartSize = hitPartValue.Size
                end
            end)
        end
        if target then
            local root: BasePart? = getTargetRoot(target)
            local humanoid: Humanoid? = getTargetHumanoid(target)
            if root then
                local stateName: string? = nil
                if humanoid then
                    pcall(function(): ()
                        stateName = humanoid:GetState().Name
                    end)
                end
                pending.rootAtConfirmation = {
                    position = vectorArray(root.Position),
                    velocity = vectorArray(root.AssemblyLinearVelocity),
                    state = stateName,
                }
            end
            if humanoid then
                pending.targetHealthAtConfirmation = humanoid.Health
            end
        end
        if latencyMs >= 30 and latencyMs <= 450 then
            table.insert(runtime.freshGunLatency, {at = now, ms = latencyMs})
            if #runtime.freshGunLatency > 24 then table.remove(runtime.freshGunLatency, 1) end
            estimatesAt = -math.huge
        end
        addSample(runtime.gunAcceptance, latencyMs)
        runtime.sessionGunConfirmed += 1
        if hitTarget then
            runtime.sessionGunHitTarget += 1
        end
        -- v139.2: name the miss. The v139.1 session could only say "5 of 10
        -- confirmed shots hit"; the saved hit parts show the rest were not
        -- prediction errors at all: two landed on another player's
        -- HumanoidRootPart (the ray kept going past the murderer) and three
        -- stopped on map geometry behind the aim. Without this label the next
        -- session repeats the same archaeology.
        local missReason: string? = nil
        if hitTarget then
            missReason = "hit"
        elseif typeof(hitPartValue) == "Instance" then
            local otherOwner: any = nil
            pcall(function(): ()
                otherOwner = Players:GetPlayerFromCharacter(
                    hitPartValue:FindFirstAncestorOfClass("Model")
                )
            end)
            if otherOwner and otherOwner ~= target then
                missReason = "other-player"
            else
                missReason = "geometry"
            end
        else
            missReason = "no-hit-part"
        end
        pending.missReason = missReason
        if missReason == "hit" then
            runtime.sessionGunMisses = (runtime.sessionGunMisses or 0) + 1
            runtime.missReasons = runtime.missReasons or {}
            runtime.missReasons[missReason] =
                (runtime.missReasons[missReason] or 0) + 1
        end
        pending.confirmedAt = now
        pending.latencyMs = latencyMs
        pending.confirmedEndpoint = endpoint
        pending.confirmedHitTarget = hitTarget
        emitShot("shotConfirmation", {
            shotId = pending.shotId, kind = "gun", hitTarget = hitTarget,
            targetName = pending.targetName, latencyMs = latencyMs,
            endpoint = endpoint, evidence = "GunFired.hitPart",
        })
        ensureTrajectorySampler()
    end

    -- A body moving slower than this has no stable direction to project on,
    -- and a closest pass farther from the predicted root than this is another
    -- failure mode (wall, wrong body, teleport), not a lead error.
    local KNIFE_FEEDBACK_MIN_SPEED: number = 2.0
    local KNIFE_FEEDBACK_MAX_MISS_STUDS: number = 6.0
    local KNIFE_FEEDBACK_SAMPLE_LIMIT: number = 0.15

    -- The blade's own confirmation: where the body really was at the closest
    -- pass, projected on the velocity the throw was aimed with. Positive means
    -- the body was ahead of the aim (we under-led). This is the same quantity
    -- and the same sign convention the gun feedback uses, so the shared
    -- predictor can learn from knives with noteLeadError(seconds, "knife").
    local function knifeLeadErrorSeconds(track: KnifeTrack): number?
        local aimRoot: Vector3? = track.requestedRoot
        local approach: Vector3? = track.approachRoot
        if not aimRoot or not approach then
            return nil
        end
        local reference: Vector3? = track.aimVelocity
        if not reference then
            reference = track.approachVelocity
        end
        if not reference then
            return nil
        end
        local horizontal: Vector3 = Vector3.new(reference.X, 0, reference.Z)
        local speed: number = horizontal.Magnitude
        if speed < KNIFE_FEEDBACK_MIN_SPEED then
            return nil
        end
        local offset: Vector3 = approach - aimRoot
        local flat: Vector3 = Vector3.new(offset.X, 0, offset.Z)
        if flat.Magnitude > KNIFE_FEEDBACK_MAX_MISS_STUDS then
            return nil
        end
        local seconds: number = flat:Dot(horizontal) / (speed * speed)
        if not predictionMath.finite(seconds)
            or math.abs(seconds) > KNIFE_FEEDBACK_SAMPLE_LIMIT then
            return nil
        end
        return seconds
    end

    local finishKnifeTrack: (projectile: Instance, reason: string) -> ()
    local function stopKnifeSamplerIfIdle(): ()
        if next(runtime.tracks) == nil then
            disconnectFeatureConnection("MM2KnifeSampler")
        end
    end

    finishKnifeTrack = function(projectile: Instance, reason: string): ()
        local track: any = runtime.tracks[projectile]
        if not track then return end
        runtime.tracks[projectile] = nil
        if track.destroyConnection then track.destroyConnection:Disconnect() end
        local pending: any = track.pending
        if pending and pending.healthConnection then
            pending.healthConnection:Disconnect()
            pending.healthConnection = nil
        end
        local measuredSpeed: number? = median(track.speeds)
        local resolvedSpeed: number? = track.rawSpeed
        if not resolvedSpeed or resolvedSpeed < 24 or resolvedSpeed > 300 then
            resolvedSpeed = measuredSpeed
        end
        local validSpeed: boolean = resolvedSpeed ~= nil
            and resolvedSpeed >= 24 and resolvedSpeed <= 300
        local leadErrorSeconds: number? = track.localOwner and knifeLeadErrorSeconds(track) or nil
        if leadErrorSeconds and type(state.mm2Prediction) == "table"
            and type(state.mm2Prediction.noteLeadError) == "function" then
            pcall(state.mm2Prediction.noteLeadError, leadErrorSeconds, "knife")
        end
        if track.localOwner and validSpeed then
            addSample(runtime.knifeSpeed, resolvedSpeed :: number)
            runtime.sessionKnifeConfirmed += 1
        end
        local analysis: {[string]: any} = analyzeTargetTrajectory(
            track.targetTrajectory, track.pingMs, "knife", nil
        )
        -- Record even if there is no handle, valid speed, target or impact:
        -- missing information is not a reason to silently lose the throw.
        appendEvent({
            shotId = track.shotId,
            projectileId = track.projectileId,
            phase = "projectile-observed",
            kind = "knife",
            serverTime = workspace:GetServerTimeNow(),
            pingMs = track.pingMs,
            latencyMs = track.spawnDelayMs,
            speed = validSpeed and resolvedSpeed or nil,
            rawSpeed = track.rawSpeed,
            samples = #track.speeds,
            reason = reason,
            ownerName = track.ownerName,
            ownerUserId = track.ownerUserId,
            localOwner = track.localOwner,
            source = track.source,
            direction = track.direction,
            projectileTrajectory = track.projectileTrajectory,
            projectileSamplingSeconds = os.clock() - track.startedAt,
            targetUserId = track.targetUserId,
            targetName = track.targetName,
            targetState = analysis.initialState,
            targetJumped = analysis.jumped,
            targetTrajectory = track.targetTrajectory,
            targetHistory = #track.targetHistory > 0 and track.targetHistory or nil,
            observedDisplacement = analysis.displacement,
            constantVelocityResidual = analysis.constantVelocityResidual,
            horizontalLeadSeconds = analysis.horizontalLeadSeconds,
            verticalLeadSeconds = analysis.verticalLeadSeconds,
            pingBucket = analysis.pingBucket,
            bestMotionModel = analysis.bestMotionModel,
            motionModelErrors = analysis.motionModelErrors,
            evaluationSeconds = analysis.evaluationSeconds,
            closestApproachStuds = track.closestApproach,
            closestApproachAtMs = track.closestApproachAt,
            bodyRadiusStuds = track.bodyRadius,
            leadErrorSeconds = leadErrorSeconds,
            predictedRoot = track.requestedRoot and vectorArray(track.requestedRoot) or nil,
            approachRoot = track.approachRoot and vectorArray(track.approachRoot) or nil,
            approachAtMs = track.approachAt,
            aimLeadSeconds = track.aimLeadSeconds,
            targetHealthAtAim = pending and pending.targetHealthAtAim or nil,
            targetHealthAfter = pending and pending.targetHealthAfter or nil,
            targetHealthLatest = pending and pending.targetHealthLatest or nil,
            healthEvidenceScope = "client-observed health; not proof this projectile caused a change",
            passedThroughBody = track.closestApproach ~= nil and track.bodyRadius ~= nil
                and track.closestApproach <= track.bodyRadius or nil,
            geometryScope = "client-estimated closest approach, not server-confirmed impact",
        })
        stopKnifeSamplerIfIdle()
    end

    local function ensureKnifeSampler(): ()
        if featureConnections.MM2KnifeSampler then
            return
        end
        featureConnections.MM2KnifeSampler = TaskManager:Connect(function(
            _deltaTime: number
        ): ()
            local now: number = os.clock()
            local completed: {Instance} = {}
            for projectile: Instance, track: any in pairs(runtime.tracks) do
                if not projectile.Parent or now - track.startedAt >= 5 then
                    table.insert(completed, projectile)
                    continue
                end
                local position: Vector3? = projectilePosition(projectile)
                if position and track.previousPosition and track.previousAt then
                    local sampleDelta: number = now - track.previousAt
                    local displacement: number = (position - track.previousPosition).Magnitude
                    if sampleDelta >= 1 / 240 and displacement >= 0.01 then
                        local speed: number = displacement / sampleDelta
                        if speed >= 24 and speed <= 300 then
                            table.insert(track.speeds, speed)
                        end
                    end
                end
                if position then
                    track.previousPosition = position
                    track.previousAt = now
                    if now - track.lastProjectileSampleAt >= 1 / 30
                        and #track.projectileTrajectory < 160 then
                        table.insert(track.projectileTrajectory, {
                            dt = now - track.startedAt, position = vectorArray(position),
                        })
                        track.lastProjectileSampleAt = now
                    end
                    if track.targetUserId then
                        local target: Player? = Players:GetPlayerByUserId(
                            track.targetUserId
                        )
                        local body: BasePart? = target
                            and getTargetRoot(target)
                            or nil
                        if body then
                            local distance: number =
                                (position - body.Position).Magnitude
                            if not track.closestApproach
                                or distance < track.closestApproach then
                                track.closestApproach = distance
                                track.closestApproachAt =
                                    (now - track.startedAt) * 1000
                                -- v179: the pair that scores the throw. Until
                                -- now the knife only recorded HOW FAR the
                                -- blade passed; without the body position and
                                -- velocity there was no way to say WHICH WAY.
                                track.approachRoot = body.Position
                                track.approachVelocity =
                                    body.AssemblyLinearVelocity
                                track.approachAt =
                                    (now - track.startedAt) * 1000
                                -- Half-diagonal of the torso the blade has to
                                -- cross: a distance under it is a touch.
                                local size: Vector3 = body.Size
                                track.bodyRadius = 0.5 * math.max(
                                    size.X,
                                    math.max(size.Y, size.Z)
                                )
                            end
                        end
                    end
                end
                if track.targetUserId
                    and now - track.lastTrajectorySampleAt >= 1 / 30 then
                    local target: Player? = Players:GetPlayerByUserId(
                        track.targetUserId
                    )
                    local point: TrajectoryPoint? = target
                        and captureTrajectoryPoint(
                            target,
                            track.startedAt - track.spawnDelayMs / 1000,
                            track.pingMs
                        )
                        or nil
                    if point and #track.targetTrajectory < 120 then
                        table.insert(track.targetTrajectory, point)
                    end
                    track.lastTrajectorySampleAt = now
                end
            end
            for _, projectile: Instance in ipairs(completed) do
                finishKnifeTrack(projectile, "sample-window-complete")
            end
            -- v160: this task used to live for the whole session after the
            -- first knife, measuring an empty table 60 times a second. It is
            -- cheap per frame, but a permanent connection nothing can turn off
            -- is exactly the kind of cost that adds up. It retires when the
            -- last blade is gone and ensureKnifeSampler re-arms it on the next
            -- throw.
            if next(runtime.tracks) == nil then
                disconnectFeatureConnection("MM2KnifeSampler")
            end
        end, nil, "MM2 knife sampler")
    end

    local function resolveKnifeProjectile(projectile: Instance): ()
        if not runtime.active or runtime.tracks[projectile] then return end
        local sessionId: string = runtime.sessionId
        local now: number = os.clock()
        local projectileId: string = nextShotId()
        local position: Vector3? = projectilePosition(projectile)
        local rawSpeed: number? = nil
        local direction: any = nil
        pcall(function(): ()
            rawSpeed = tonumber(projectile:GetAttribute("ThrowSpeed"))
            direction = projectile:GetAttribute("Direction")
        end)
        if rawSpeed and not finite(rawSpeed) then rawSpeed = nil end
        local directionArray: any = nil
        if typeof(direction) == "Vector3" and finite(direction.X)
            and finite(direction.Y) and finite(direction.Z) then
            directionArray = vectorArray(direction)
        end
        local track: any = {
            shotId = projectileId,
            projectileId = projectileId,
            startedAt = now,
            previousAt = now,
            previousPosition = position,
            lastProjectileSampleAt = now,
            projectileTrajectory = position and {{dt = 0, position = vectorArray(position)}} or {},
            speeds = {},
            rawSpeed = rawSpeed,
            direction = directionArray,
            spawnDelayMs = 0,
            pingMs = getPingMilliseconds(),
            targetTrajectory = {},
            targetHistory = {},
            lastTrajectorySampleAt = now,
        }
        runtime.sessionKnifeObserved += 1
        runtime.tracks[projectile] = track
        appendEvent({kind = "knife", phase = "projectile-spawn", shotId = projectileId,
            projectileId = projectileId, serverTime = workspace:GetServerTimeNow(),
            pingMs = track.pingMs, origin = position and vectorArray(position) or nil,
            direction = directionArray, rawSpeed = rawSpeed,
            evidence = "client-observed ThrowingKnife tag"})
        track.destroyConnection = projectile.Destroying:Connect(function(): ()
            finishKnifeTrack(projectile, "destroyed")
        end)
        ensureKnifeSampler()
        -- HandleLink may replicate late. Sampling begins immediately, even for
        -- another player or a projectile whose link never arrives.
        task.spawn(function(): ()
            local linkedHandle: Instance? = nil
            for _attempt: number = 1, 20 do
                if not runtime.active or runtime.sessionId ~= sessionId
                    or runtime.tracks[projectile] ~= track or not projectile.Parent then return end
                local link: Instance? = projectile:FindFirstChild("HandleLink", true)
                if link and link:IsA("ObjectValue") and link.Value then
                    linkedHandle = link.Value
                    break
                end
                task.wait(0.05)
            end
            if not linkedHandle or runtime.tracks[projectile] ~= track then return end
            local owner: Player? = nil
            local linkedTool: Tool? = nil
            pcall(function(): ()
                linkedTool = linkedHandle:FindFirstAncestorOfClass("Tool") :: Tool?
                local character: Model? = linkedHandle:FindFirstAncestorOfClass("Model") :: Model?
                if character then owner = Players:GetPlayerFromCharacter(character) end
            end)
            track.ownerName = owner and owner.Name or nil
            track.ownerUserId = owner and owner.UserId or nil
            track.localOwner = owner == LocalPlayer
                or (linkedTool ~= nil and runtime.observedTools[linkedTool] == true)
            compactPending(runtime.pendingKnives, os.clock())
            local selectedIndex: number? = nil
            local fallbackIndex: number? = nil
            for index: number = 1, #runtime.pendingKnives do
```

```
            local pending: PendingKnife = runtime.pendingKnives[index]
            local tool: Tool? = pending.tool
            if tool then
                local handle: Instance? = tool:FindFirstChild("Handle")
                if linkedHandle == handle or linkedHandle:IsDescendantOf(tool) then
                    if pending.attemptCounted then selectedIndex = index; break end
                    fallbackIndex = fallbackIndex or index
                end
            end
        end
        selectedIndex = selectedIndex or fallbackIndex
        if selectedIndex then
            local pending: any = table.remove(runtime.pendingKnives, selectedIndex)
            track.pending = pending
            track.shotId = pending.shotId
            track.localOwner = true
            track.ownerName = LocalPlayer.Name
            track.ownerUserId = LocalPlayer.UserId
            track.source = pending.source
            track.spawnDelayMs = math.max(0, (now - pending.activatedAt) * 1000)
            track.pingMs = pending.pingMs
            track.targetUserId = pending.targetUserId
            track.targetName = pending.targetName
            track.requestedAim = pending.requestedAim
            track.requestedRoot = pending.requestedRoot
            track.aimVelocity = pending.aimVelocity
            track.aimLeadSeconds = pending.aimLeadSeconds
            track.activatedAt = pending.activatedAt
            track.targetTrajectory = pending.targetTrajectory
            track.targetHistory = pending.targetHistory or {}
            addSample(runtime.knifeSpawnDelay, track.spawnDelayMs)
            if not pending.attemptCounted then
                runtime.sessionKnifeAttempts += 1
                emitShot("authoringShot", {shotId = pending.shotId, kind = "knife",
                    toolName = pending.tool.Name, originPos = pending.origin,
                    aimPos = pending.requestedAim, targetName = pending.targetName,
                    eventSource = "observed-projectile"})
            end
            emitShot("shotConfirmation", {shotId = pending.shotId, kind = "knife",
                evidence = "projectile-observed", targetName = pending.targetName})
        end
        appendEvent({kind = "knife", phase = "projectile-linked", shotId = track.shotId,
            projectileId = projectileId, serverTime = workspace:GetServerTimeNow(),
            pingMs = track.pingMs, ownerName = track.ownerName,
            ownerUserId = track.ownerUserId, localOwner = track.localOwner,
            targetName = track.targetName, source = track.source})
    end)
end

local function isGun(tool: Tool): boolean
    return tool.Name == "Gun"
        or CollectionService:HasTag(tool, "Weapon_Gun")
        or tool:FindFirstChild("Shoot") ~= nil
end

local function isKnife(tool: Tool): boolean
    return tool.Name == "Knife"
        or CollectionService:HasTag(tool, "Weapon_Knife")
        or (tool:FindFirstChild("Events") ~= nil
            and tool.Events:FindFirstChild("KnifeThrown") ~= nil)
end

-- One constructor for every pending shot. The activation observer and the
-- authored-shot path used to build this table separately, and on
-- 2026-10-18 the authored copy was missing the health watcher and the
-- server-clock pair: 13 confirmations, healthAtAim nil on all of them and
-- no serverEchoMs. Duplicating the record was the bug; there is one now.
local function beginPendingShot(kind: string, entry: any, target: Player?): any
    local now: number = entry.activatedAt or os.clock()
    entry.activatedAt = now
    entry.serverActivatedAt = workspace:GetServerTimeNow()
    entry.targetUserId = target and target.UserId or entry.targetUserId
    entry.targetName = target and target.Name or entry.targetName
    if entry.targetTrajectory == nil then
        entry.targetTrajectory = {}
        if target then
            local point: TrajectoryPoint? = captureTrajectoryPoint(
                target,
                now,
                entry.pingMs or 0
            )
            if point then
                table.insert(entry.targetTrajectory, point)
            end
        end
    end
    if entry.targetHistory == nil then
        entry.targetHistory = captureTargetHistory(target, now)
    end
    entry.lastTrajectorySampleAt = entry.lastTrajectorySampleAt or now
    watchPendingHealth(entry, target)
    local queue: {any} = kind == "gun"
        and runtime.pendingGuns
        or runtime.pendingKnives
    table.insert(queue, entry)
    compactPending(queue, now)
    ensureTrajectorySampler()
    return entry
end

local function observeTool(tool: Tool): ()
    if runtime.observedTools[tool] then
        return
    end
    local gun: boolean = isGun(tool)
    local knife: boolean = isKnife(tool)
    if not gun and not knife then
        return
    end
    runtime.observedTools[tool] = true
    trackConnection(tool.Activated:Connect(function(): ()
        if not runtime.active then
            return
        end
        local now: number = os.clock()
        local requestedAim: Vector3? = getRequestedAim()
        local pingMs: number = getPingMilliseconds()
        local target: Player? = gun
            and findMurderer()
            or findKnifeAnalysisTarget(requestedAim)
        local targetTrajectory: {TrajectoryPoint} = {}
        if target then
            local initialPoint: TrajectoryPoint? = captureTrajectoryPoint(
                target,
                now,
                pingMs
            )
            if initialPoint then
                table.insert(targetTrajectory, initialPoint)
            end
        end
        local shotId: string = nextShotId()
        appendEvent({shotId = shotId, kind = gun and "gun" or "knifeActivation",
            phase = gun and "attempt" or "activation",
            serverTime = workspace:GetServerTimeNow(), pingMs = pingMs,
            targetName = target and target.Name or nil})
        emitShot("authoringShot", {shotId = shotId, kind = gun and "gun" or "knifeActivation",
            toolName = tool.Name, originPos = toolOrigin(tool), aimPos = requestedAim,
            targetName = target and target.Name or nil, eventSource = "tool-activation"})
        if gun then
            runtime.sessionGunAttempts += 1
            beginPendingShot("gun", {
                shotId = shotId,
                source = "native-activation",
                attemptCounted = true,
                tool = tool,
                activatedAt = now,
                origin = toolOrigin(tool),
                requestedAim = requestedAim,
                pingMs = pingMs,
                targetTrajectory = targetTrajectory,
            }, target)
        elseif knife then
            runtime.rawKnifeActivations += 1
            beginPendingShot("knife", {
                shotId = shotId,
                attemptCounted = false,
                tool = tool,
                activatedAt = now,
                origin = toolOrigin(tool),
                requestedAim = requestedAim,
                pingMs = pingMs,
                targetTrajectory = targetTrajectory,
            }, target)
        end
    end))
end

local function observeContainer(container: Instance?): ()
    if not container then
        return
    end
    for _, child: Instance in ipairs(container:GetChildren()) do
        if child:IsA("Tool") then
            observeTool(child)
        end
    end
    trackConnection(container.ChildAdded:Connect(function(child: Instance): ()
        if child:IsA("Tool") then
            observeTool(child)
        end
    end))
end

local function installGunConfirmation(generation: number): ()
    local function current(): boolean
        return runtime.active and runtime.observationGeneration == generation
    end
    task.spawn(function(): ()
        local clientServices: Instance? = nil
        local weaponModule: Instance? = nil
        for _attempt: number = 1, 20 do
            if not current() then
                return
            end
            clientServices = game:GetService("ReplicatedStorage")
                :FindFirstChild("ClientServices")
            weaponModule = clientServices
                and clientServices:FindFirstChild("WeaponService")
            if weaponModule then
                break
            end
            task.wait(0.25)
        end
        if not weaponModule then
            warn(trajectoryLogPrefix .. " WeaponService did not appear; Knife remains active.")
            return
        end
        local loaded: boolean, weaponService: any = pcall(
            state.requireModule,
            weaponModule
        )
        if not current() then return end
        if not loaded or type(weaponService) ~= "table" then
            warn(
                trajectoryLogPrefix .. " Could not observe WeaponService: "
                    .. tostring(weaponService)
            )
            return
        end
        pcall(function(): ()
            local signal: any = weaponService.KnifeThrown and weaponService.KnifeThrown.Event
            if not signal or type(signal.Connect) ~= "function" then return end
            trackConnection(signal:Connect(function(): ()
                if not current() then return end
                local knife: Tool? = getPlayerWeapon(LocalPlayer, "Knife", true)
                if not knife or knife.Parent ~= LocalPlayer.Character then return end
                local aim: Vector3? = getRequestedAim()
                controller:noteAuthoredShot({
                    kind = "knife", tool = knife, toolName = knife.Name,
                    originPos = toolOrigin(knife), aimPos = aim,
                    target = findKnifeAnalysisTarget(aim), source = "native-throw-event",
                    settings = {mode = "Manual", aimScope = "mouse at input; actual redirected aim may differ"},
                })
            end))
        end)
        local connected: boolean, connection: any = pcall(function(): any
            return weaponService.GunFired.OnClientEvent:Connect(function(...: any): ()
                if current() then handleGunFired(...) end
            end)
        end)
        if connected and connection then
            trackConnection(connection :: RBXScriptConnection)
            print(trajectoryLogPrefix .. " GunFired confirmation connected.")
        else
            warn(
                trajectoryLogPrefix .. " GunFired no es compatible: "
                    .. tostring(connection)
            )
        end
    end)
end

controller = {} :: Controller

function controller:prepare(): ()
    if not runtime.historyLoaded then
        runtime.historyLoaded = true
        loadSavedHistory() -- read-only; the backup still waits for Yes
    end
end

function controller:start(): ()
    local states: any = type(configData) == "table" and configData.states or nil
    if not states or states["MM2.Calibration.Consent"] ~= true then
        return
    end
    if runtime.active then
        return
    end

    if not GAME_CHECK.MM2Active then
        warn(trajectoryLogPrefix .. " Inicio omitido fuera de MM2.")
        return
    end
    runtime.active = true
    runtime.observationGeneration += 1
    local generation: number = runtime.observationGeneration
    runtime.startedAt = os.clock()
    runtime.startedAtUnix = os.time()
    self:prepare()
    -- The predictor may load after this adapter: retry briefly instead of
    -- dropping what the previous matches taught it.
    if type(task) == "table" and type(task.spawn) == "function" then
        task.spawn(function(): ()
            for _attempt: number = 1, 20 do
                if not runtime.active or runtime.observationGeneration ~= generation then return end
                if applyLearnedSeeds() then return end
                task.wait(0.5)
            end
        end)
    end
    observeContainer(LocalPlayer:FindFirstChildOfClass("Backpack"))
    observeContainer(LocalPlayer.Character)
    trackConnection(LocalPlayer.CharacterAdded:Connect(function(
        character: Model
    ): ()
        observeContainer(character)
    end))
    trackConnection(
        CollectionService:GetInstanceAddedSignal("ThrowingKnife"):Connect(
            resolveKnifeProjectile
        )
    )
    pcall(function(): ()
        for _, projectile: Instance in ipairs(CollectionService:GetTagged("ThrowingKnife")) do
            resolveKnifeProjectile(projectile)
        end
    end)
    installGunConfirmation(generation)
    print(trajectoryLogPrefix .. " Passive calibration started in the background.")
end

local function clearHistory(): ()
    runtime.gunAcceptance = newStats()
    runtime.knifeSpeed = newStats()
    runtime.knifeSpawnDelay = newStats()
    runtime.motionBuckets = {}
    runtime.events = {}
    runtime.archivedEvents = 0
    runtime.archiveFiles = {}
    runtime.archiveSequence = 0
    runtime.freshGunLatency = {}
    estimatesCache = nil
    estimatesAt = -math.huge
    runtime.sessionGunAttempts = 0
    runtime.sessionGunConfirmed = 0
    runtime.sessionKnifeAttempts = 0
    runtime.sessionKnifeConfirmed = 0
    runtime.sessionKnifeObserved = 0
    runtime.rawKnifeActivations = 0
    runtime.unmatchedGunSignals = 0
    runtime.sessionStabAttempts = 0
    runtime.startedAt = os.clock()
    runtime.startedAtUnix = os.time()
    for _, track in pairs(runtime.tracks) do
        if track.destroyConnection then track.destroyConnection:Disconnect() end
    end
    for _, queue in ipairs({runtime.pendingGuns, runtime.pendingKnives}) do
        for _, pending in ipairs(queue) do
            if pending.healthConnection then pending.healthConnection:Disconnect() end
        end
    end
    table.clear(runtime.tracks)
    disconnectFeatureConnection("MM2TrajectorySampler")
    runtime.sessionId = HttpService:GenerateGUID(false)
    runtime.shotSequence = 0
    table.clear(runtime.pendingGuns)
    table.clear(runtime.pendingKnives)
end

local function consentValue(): boolean?
    local states: any = type(configData) == "table" and configData.states or nil
    if not states then
        return nil
    end
    local value: any = states["MM2.Calibration.Consent"]
    if value == true then return true end
    if value == false then return false end
    return nil
end

function controller:getConsent(): boolean?
    return consentValue()
end

-- Yes starts observation and attempts a save. No (or unanswered) stops
-- observation and writes without deleting any prior telemetry or backups.
function controller:setConsent(value: boolean?): boolean
    local nextValue: boolean? = nil
    if value == true then
        nextValue = true
    elseif value == false then
        nextValue = false
    end
    if type(configData) == "table" and configData.states then
        configData.states["MM2.Calibration.Consent"] = nextValue
    end
    if type(host.queueConfigSave) == "function" then
        pcall(host.queueConfigSave)
    end
    if nextValue == true then
        self:start()
        if not runtime.active then return false end
        runtime.dirty = true
        local generation: number = runtime.observationGeneration
        -- The call is synchronous until it reaches the writer: capture
        -- whether this consent save will be coalesced rather than failed.
        local queued: boolean = runtime.snapshotInProgress
        local saved: boolean = saveSnapshot("consent")
        -- A No/Yes while an executor write yielded started a newer
        -- recorder. The old call must never tear that recorder down.
        if runtime.observationGeneration ~= generation then return saved or queued end
        if not saved and not queued then
            runtime.dirty = false
            self:destroy()
        end
        return saved or queued
    end
    runtime.saveScheduled = false
    runtime.dirty = false
    if runtime.active then self:destroy() end
    return true
end

function controller:save(reason: string?): boolean
    -- BindToClose callbacks outlive a reinjected menu. Once this
    -- calibration is retired, it must not overwrite the newer session.
    if not runtime.active then
        return false
    end
    local saved: boolean = saveSnapshot(reason or "manual")
    return saved
end

function controller:reset(): ()
    if consentValue() ~= true or not runtime.active or runtime.persistenceBlocked then return end
    -- Use the same conflict checks and serialized writer as every other
    -- snapshot. Deleting the file out of band lets an older yielding save
    -- resurrect cleared history and makes the next save look conflicted.
    clearHistory()
    runtime.revision += 1
    runtime.dirty = true
    local queued: boolean = runtime.snapshotInProgress
    local saved: boolean = saveSnapshot("reset")
    if saved or queued then
        print(trajectoryLogPrefix .. " History cleared.")
    else
        warn(trajectoryLogPrefix .. " Cleared in memory; the new snapshot could not be saved.")
    end
end

function controller:getEstimates(): Estimates
    return estimates()
end

-- Separate feedback event: never count an echo as another authored shot
-- or use endpoint-to-client-position distance as proof of a server miss.
function controller:noteTracerData(info: any): ()
    if not runtime.active or type(info) ~= "table" then return end
    local function validVector(value: any): boolean
        return typeof(value) == "Vector3"
            and predictionMath.finite(value.X) and predictionMath.finite(value.Y)
            and predictionMath.finite(value.Z) and value.Magnitude < 1e7
    end
    if not validVector(info.origin) or not validVector(info.endpoint) then return end
    local function pack(value: Vector3): {number}
        return {value.X, value.Y, value.Z}
    end
    local shotId: string? = nil
    local now: number = os.clock()
    for index: number = #runtime.pendingGuns, 1, -1 do
        local pending: PendingGun = runtime.pendingGuns[index]
        if now - pending.activatedAt <= 2.5 and pending.origin
            and (pending.origin - info.origin).Magnitude <= 8 then
            shotId = pending.shotId
            break
        end
    end
    local target: any = info.target
    local targetPosition: Vector3? = validVector(info.targetPosition) and info.targetPosition or nil
    local path: string? = nil
    local hitTarget: boolean? = nil
    if typeof(info.hitPart) == "Instance" then
        pcall(function(): () path = string.sub(info.hitPart:GetFullName(), 1, 512) end)
        if target and target.Character then
            hitTarget = info.hitPart:IsDescendantOf(target.Character)
        end
    end
    appendEvent({kind = "gun", phase = "tracer-data", shotId = shotId,
        serverTime = workspace:GetServerTimeNow(), pingMs = getPingMilliseconds(),
        origin = pack(info.origin), endpoint = pack(info.endpoint),
        shotRange = (info.endpoint - info.origin).Magnitude,
        targetUserId = target and target.UserId or nil,
        targetName = target and target.Name or nil,
        observedTargetPosition = targetPosition and pack(targetPosition) or nil,
        endpointToObservedTarget = targetPosition and (info.endpoint - targetPosition).Magnitude or nil,
        targetPositionScope = "client-observed-at-GunFired-receipt; not server impact-time position",
        hitPartPath = path, hitTarget = hitTarget, evidence = "GunFired endpoint/hitPart",
    })
end

-- A shot the client refused. Recorded so the blocker that the server
-- respects (invisible wall, barrier, tagged part) can be identified
-- instead of being inferred from a miss. Throttled: a held auto-shoot
-- would otherwise flood the event ring.
function controller:noteBlockedShot(blocker: any): ()
    if not runtime.active or type(blocker) ~= "table" then
        return
    end
    local now: number = os.clock()
    if now - runtime.lastBlockedAt < 1 then
        return
    end
    runtime.lastBlockedAt = now
    appendEvent({
        kind = "gun",
        phase = "blocked",
        serverTime = workspace:GetServerTimeNow(),
        pingMs = getPingMilliseconds(),
        -- v139.3: the refusal carries its own reason (such as "wall"),
        -- so a calibration session can tell a real barrier apart from a
        -- policy decision that cost a shot.
        reason = type(blocker.reason) == "string" and blocker.reason or "wall",
        blockedName = type(blocker.name) == "string" and blocker.name or nil,
        blockedTransparency = tonumber(blocker.transparency),
        blockedCanCollide = blocker.canCollide == true,
        blockedSkipped = blocker.skipped == true,
        blockedDistance = tonumber(blocker.distance),
    })
end

function controller:noteAuthoredShot(info: any): ()
    if not runtime.active or type(info) ~= "table" then
        return
    end
    local kind: string = tostring(info.kind or "gun")
    local shotId: string = nextShotId()
    local now: number = os.clock()
    local pingMs: number = getPingMilliseconds()
    local target: Player? =
        info.target and info.target.Character ~= nil and info.target or nil
    -- Captured before the authored event so the record carries the window
    -- the predictor actually had, not a reconstruction.
    local targetHistory: {HistoryPoint} = captureTargetHistory(target, now)
    if kind ~= "stab" then
        appendEvent({kind = kind, shotId = shotId, phase = "authored",
            serverTime = workspace:GetServerTimeNow(), clock = now,
            pingMs = pingMs,
            settings = info.settings, source = info.source or "menu",
            origin = typeof(info.originPos) == "Vector3" and vectorArray(info.originPos) or nil,
            endpoint = typeof(info.aimPos) == "Vector3" and vectorArray(info.aimPos) or nil,
            targetUserId = target and target.UserId or nil,
            targetName = target and target.Name or nil,
            targetHistory = #targetHistory > 0 and targetHistory or nil})
    end
    local targetTrajectory: {TrajectoryPoint} = {}
    if target then
        local initialPoint: TrajectoryPoint? = captureTrajectoryPoint(
            target,
            now,
            pingMs
        )
        if initialPoint then
            table.insert(targetTrajectory, initialPoint)
        end
    end
    if kind == "stab" then
        runtime.sessionStabAttempts += 1
        local targetRoot: BasePart? = getTargetRoot(target)
        local localRoot: BasePart? =
            LocalPlayer.Character
            and LocalPlayer.Character:FindFirstChild("HumanoidRootPart")
            :: BasePart?
        local distance: number? = targetRoot
            and localRoot
            and (targetRoot.Position - localRoot.Position).Magnitude
        appendEvent({
            kind = "stab", shotId = shotId, phase = "attempt",
            serverTime = workspace:GetServerTimeNow(),
            pingMs = pingMs,
            distance = distance and math.floor(distance * 10 + 0.5) / 10 or nil,
            targetUserId = target and target.UserId or nil,
            targetName = target and target.Name or nil,
            targetState = targetTrajectory[1]
                and targetTrajectory[1].humanoidState
                or nil,
            targetTrajectory = #targetTrajectory > 0 and targetTrajectory or nil,
            targetHistory = #targetHistory > 0 and targetHistory or nil,
        })
    else
        local isGun: boolean = kind ~= "knife"
        local authoredTool: Tool? = nil
        if typeof(info.tool) == "Instance" and info.tool:IsA("Tool") then
            authoredTool = info.tool :: Tool
        end

        if isGun or authoredTool ~= nil then
            if isGun then
                local countedActivation: boolean = false
                if info.source == "native-redirect" and authoredTool then
                    -- Tool.Activated and the redirected native packet are
                    -- two observations of one click. Keep both events but
                    -- retire the tentative pending entry so its muzzle
                    -- cannot steal the CFrame echo or double-count it.
                    for index: number = #runtime.pendingGuns, 1, -1 do
                        local pending: any = runtime.pendingGuns[index]
                        local age: number = now - (pending.activatedAt or now)
                        if pending.source == "native-activation" and pending.tool == authoredTool
                            and not pending.confirmedAt and age >= 0 and age <= 0.3 then
                            if pending.healthConnection then pending.healthConnection:Disconnect() end
                            table.remove(runtime.pendingGuns, index)
                            countedActivation = true
                            break
                        end
                    end
                end
                if not countedActivation then runtime.sessionGunAttempts += 1 end
            else
                runtime.sessionKnifeAttempts += 1
                if info.source == "native-throw-event" then
                    -- Tool.Activated can precede the game's actual throw
                    -- signal and also occurs for stabs. Retire its matching
                    -- tentative record so a later projectile cannot reuse
                    -- it as a second throw or inherit a stale health watch.
                    for index: number = #runtime.pendingKnives, 1, -1 do
                        local pending: any = runtime.pendingKnives[index]
                        local age: number = now - (pending.activatedAt or now)
                        if not pending.attemptCounted and pending.tool == authoredTool
                            and age >= 0 and age <= 0.3 then
                            if pending.healthConnection then pending.healthConnection:Disconnect() end
                            table.remove(runtime.pendingKnives, index)
                            break
                        end
                    end
                end
            end
            beginPendingShot(isGun and "gun" or "knife", {
                shotId = shotId,
                attemptCounted = true,
                source = info.source or "menu",
                tool = authoredTool,
                shotMode = type(info.settings) == "table"
                    and info.settings.mode
                    or (info.silent and "Silent" or nil),
                silentSpawn = info.variant,
                activatedAt = now,
                origin = info.originPos,
                requestedAim = info.aimPos,
                -- v179: only the knife author fills these in.
                requestedRoot = typeof(info.aimRoot) == "Vector3"
                    and info.aimRoot
                    or nil,
                aimVelocity = typeof(info.aimVelocity) == "Vector3"
                    and info.aimVelocity
                    or nil,
                aimLeadSeconds = predictionMath.finite(info.leadSeconds)
                    and info.leadSeconds
                    or nil,
                pingMs = pingMs,
                targetTrajectory = targetTrajectory,
                targetHistory = targetHistory,
            }, target)
        end
    end

    if type(state.emitGameBridgeEvent) == "function" then
        emitShot("authoringShot", {
            shotId = shotId,
            eventSource = "authored",
            settings = info.settings,
            kind = kind,
            toolName = tostring(info.toolName or (kind == "knife" and "Knife" or "Gun")),
            originPos = info.originPos,
            aimPos = info.aimPos,
            targetName = target and target.Name or nil,
            silent = info.silent == true,
            variant = info.variant,
            offset = info.offset,
        })
    end
end

function controller:status(): string
```

Leer columna izquierda completa y despues la derecha. Texto seleccionable; lineas largas comprimidas. Tabulaciones: 4 espacios. Original adjunto.

```
        local current: Estimates = estimates()
        local knifeText: string = current.knifeSpeed
              and string.format("%.2f studs/s", current.knifeSpeed)
            or "learning"
        local bucketCount: number = 0
        for _key: string in pairs(runtime.motionBuckets) do
            bucketCount += 1
        end
        local consent: boolean? = consentValue()
        local consentText: string = consent == true
              and ((runtime.persistenceBlocked or type(environment.writefile) ~= "function")
                  and "saving unavailable; see console" or "giving calibration")
            or (consent == false and "not giving calibration")
            or "calibration not answered"
        return string.format(
            "MM2 session observations · guns %d · knives %d · motion buckets %d · knife %s · %s",
            runtime.sessionGunConfirmed,
            runtime.sessionKnifeConfirmed,
            bucketCount,
            knifeText,
            consentText
        )
    end

    function controller:destroy(): ()
        if not runtime.active then
            return
        end
        -- Retire pending async service loads before any save can yield.
        runtime.observationGeneration += 1
        if consentValue() == true then
            local projectiles: {Instance} = {}
            for projectile: Instance in pairs(runtime.tracks) do table.insert(projectiles, projectile) end
            for _, projectile: Instance in ipairs(projectiles) do
                finishKnifeTrack(projectile, "menu-unload")
            end
        end
        if runtime.dirty then saveSnapshot("module-destroy") end
        runtime.active = false
        disconnectFeatureConnection("MM2TrajectorySampler")
        disconnectFeatureConnection("MM2KnifeSampler")
        if runtime.trajectorySampler then
            runtime.trajectorySampler:Disconnect()
            runtime.trajectorySampler = nil
        end
        for projectile: Instance, track: KnifeTrack in pairs(runtime.tracks) do
            if track.destroyConnection then
                track.destroyConnection:Disconnect()
            end
            if track.pending and track.pending.healthConnection then
                track.pending.healthConnection:Disconnect()
            end
            runtime.tracks[projectile] = nil
        end
        for _, connection: RBXScriptConnection in ipairs(runtime.connections) do
            connection:Disconnect()
        end
        table.clear(runtime.connections)
        for _, queue in ipairs({runtime.pendingGuns, runtime.pendingKnives}) do
            for _, pending in ipairs(queue) do
                if pending.healthConnection then pending.healthConnection:Disconnect() end
            end
            table.clear(queue)
        end
        table.clear(runtime.observedTools)
    end

    return controller
end

trajectoryCalibration = createTrajectoryCalibration()
state.mm2TrajectoryCalibration = trajectoryCalibration
-- Read existing history to protect it against conflicting writers, but do not
-- observe or write anything until the player explicitly agrees.
trajectoryCalibration:prepare()
local calibrationPromptLive: boolean = true
local calibrationStates: any = type(configData) == "table" and configData.states or {}
local rememberedCalibration: boolean = calibrationStates["MM2.Calibration.DontShowAgain"] == true
    and type(calibrationStates["MM2.Calibration.Consent"]) == "boolean"
if rememberedCalibration then
    if calibrationStates["MM2.Calibration.Consent"] == true then
        if not trajectoryCalibration:setConsent(true) then
            notify("Calibration could not save; check the console and executor file support.")
        end
    end
else
    -- Legacy profiles did not have the checkbox, so their old answer must not
    -- silently become a remembered choice in this version.
    calibrationStates["MM2.Calibration.Consent"] = nil
    task.delay(3, function(): ()
        if not calibrationPromptLive
            or (type(state.ownsRuntimeData) == "function" and not state.ownsRuntimeData()) then
            return
        end
        if type(state.confirmDialog) ~= "function" then return end
        local function answer(value: boolean, remember: boolean): ()
            if not calibrationPromptLive
                or (type(state.ownsRuntimeData) == "function" and not state.ownsRuntimeData()) then
                return
            end
            calibrationStates["MM2.Calibration.DontShowAgain"] = remember == true
            local saved: boolean = trajectoryCalibration:setConsent(value)
            if value then
                if saved then
                    notify("Calibration ON: saving observed MM2 knife and shot data to Wurst/Telemetry.")
                else
                    notify("Calibration could not save; check the console and executor file support.")
                end
            else
                notify("Calibration OFF: no new data will be recorded; previous files are kept.")
            end
        end
        state.confirmDialog({
            title = "Give your calibration?",
            message = "Would you like to save the MM2 knives and shots your client observes to your executor workspace for analysis? Previous files are kept if you choose No.",
            yes = "Yes",
            no = "No",
            rememberLabel = "Don't Show Again",
            onYes = function(remember: boolean): () answer(true, remember) end,
            onNo = function(remember: boolean): () answer(false, remember) end,
        })
    end)
end

pcall(function(): ()
    if type((game :: any).BindToClose) == "function" then
        game:BindToClose(function(): ()
            if type(trajectoryCalibration.save) == "function" then
                pcall(trajectoryCalibration.save, trajectoryCalibration, "close")
            end
        end)
    end
end)

local weaponServiceModule: any = nil
type PendingShot = {
    target: Player,
    targetCharacter: Model?,
    gun: Tool,
    origin: Vector3,
    aimPosition: Vector3?,
    healthBefore: number,
    targetedMurderer: boolean,
    queuedAt: number,

    ambiguous: boolean?,
}

state.mm2ShotFeedback = {

    shootActive = false,
    pending = nil :: PendingShot?,
    lastAccepted = nil :: PendingShot?,

    token = 0,
    defaultDuration = 5,
    gunFiredConnected = false,
    theme = {
        Background = Color3.fromRGB(14, 14, 18),
        Accent = Color3.fromRGB(205, 82, 42),
        Outline = Color3.fromRGB(220, 115, 58),
        Text = Color3.fromRGB(255, 232, 208),
    },
    resolve = function(_candidate: PendingShot, _directHit: boolean): () end,
    showMiss = function(_gun: Tool, _candidate: PendingShot): () end,
}
state.mm2ShotFeedback.root = create("Frame", {
    Parent = ScreenGui,
    Name = "MM2ShotCooldown",
    AnchorPoint = Vector2.new(0.5, 0.5),
    BackgroundColor3 = state.mm2ShotFeedback.theme.Background,
    BackgroundTransparency = 0.38,
    BorderSizePixel = 0,
    ClipsDescendants = true,
    Position = UDim2.fromScale(0.5, 0.62),
    Size = UDim2.fromOffset(320, 36),
    Visible = false,
    ZIndex = 240,
}) :: Frame
create("UICorner", {
    Parent = state.mm2ShotFeedback.root,
    CornerRadius = UDim.new(0, 7),
})
create("UIStroke", {
    Parent = state.mm2ShotFeedback.root,
    Color = state.mm2ShotFeedback.theme.Outline,
    Transparency = 0.35,
    Thickness = 1,
})
state.mm2ShotFeedback.fill = create("Frame", {
    Parent = state.mm2ShotFeedback.root,
    Name = "Fill",
    BackgroundColor3 = state.mm2ShotFeedback.theme.Accent,
    BackgroundTransparency = 0.46,
    BorderSizePixel = 0,
    Size = UDim2.fromScale(0, 1),
    ZIndex = 241,
}) :: Frame
create("UICorner", {
    Parent = state.mm2ShotFeedback.fill,
    CornerRadius = UDim.new(0, 7),
})
state.mm2ShotFeedback.text = create("TextLabel", {
    TextTruncate = Enum.TextTruncate.AtEnd,
    ClipsDescendants = true,
    Parent = state.mm2ShotFeedback.root,
    Name = "Countdown",
    BackgroundTransparency = 1,
    Font = Enum.Font.GothamBold,
    Size = UDim2.fromScale(1, 1),
    Text = "",
    TextColor3 = state.mm2ShotFeedback.theme.Text,
    TextSize = 13,
    ZIndex = 242,
}) :: TextLabel

state.mm2ShotFeedback.hide = function(): ()
    disconnectFeatureConnection("MM2ShootCooldown")
    state.mm2ShotFeedback.token += 1
    state.mm2ShotFeedback.root.Visible = false
    state.mm2ShotFeedback.fill.Size = UDim2.fromScale(0, 1)
    state.mm2ShotFeedback.text.Text = ""
end

state.mm2ShotFeedback.getDuration = function(gun: Tool?): number
    if gun then
        for _, attributeName: string in ipairs({
            "Cooldown",
            "ReloadTime",
            "ShotCooldown",
        }) do
            local value: number? = tonumber(gun:GetAttribute(attributeName))
            if value and value >= 0.5 and value <= 15 then
                return value
            end
        end
    end
    return state.mm2ShotFeedback.defaultDuration
end

state.mm2ShotFeedback.queue = function(
    target: Player?,
    gun: Tool?,
    origin: CFrame?,
    aim: CFrame?
): ()
    if not target or not gun or not origin then
        return
    end

    local targetCharacter: Model? = target.Character
    local targetHumanoid: Humanoid? = targetCharacter
        and targetCharacter:FindFirstChildOfClass("Humanoid")
        :: Humanoid?
    local candidate: PendingShot = {
        target = target,
        targetCharacter = targetCharacter,
        gun = gun,
        -- A single pending slot cannot disambiguate rapid equal-geometry shots.
        -- Exclude these echoes from statistics instead of guessing.
        ambiguous = state.mm2ShotFeedback.pending ~= nil,
        origin = origin.Position,
        aimPosition = aim and aim.Position,
        healthBefore = targetHumanoid and targetHumanoid.Health or 0,
        targetedMurderer = getPlayerRole(target) == "Murderer",
        queuedAt = os.clock(),
    }
    state.mm2ShotFeedback.pending = candidate

    task.delay(1.8, function(): ()
        if state.mm2ShotFeedback.pending == candidate then
            state.mm2ShotFeedback.pending = nil
        end
    end)
    if not state.mm2ShotFeedback.gunFiredConnected then
        task.delay(math.max(0.35, mm2Settings.predictionRtt + 0.18), function(): ()
            if state.mm2ShotFeedback.pending ~= candidate
                or state.mm2ShotFeedback.gunFiredConnected then
                return
            end
            state.mm2ShotFeedback.pending = nil
            state.mm2ShotFeedback.resolve(candidate, false)
        end)
    end
end

state.mm2ShotFeedback.showMiss = function(
    gun: Tool,
    candidate: PendingShot
): ()
    state.mm2ShotFeedback.hide()
    local duration: number = state.mm2ShotFeedback.getDuration(gun)
    local cooldownEndsAt: number = candidate.queuedAt + duration
    local initialRemaining: number = cooldownEndsAt - os.clock()
    if initialRemaining <= 0 then
        return
    end

    disconnectFeatureConnection("MM2ShootCooldown")
    state.mm2ShotFeedback.token += 1
    local token: number = state.mm2ShotFeedback.token
    state.mm2ShotFeedback.root.Visible = true

    -- v139: the cooldown used to rebuild the label string and write two GUI
    -- properties on every single frame. Text is one of the expensive ones
    -- (layout + glyph re-shape), so the marker now refreshes at ~12 Hz and only
    -- writes when the text or the bar actually changed.
    local lastText: string = ""
    local lastScale: number = -1
    local lastRefresh: number = -math.huge
    local function updateCountdown(): ()
        if token ~= state.mm2ShotFeedback.token then
            disconnectFeatureConnection("MM2ShootCooldown")
            state.mm2ShotFeedback.root.Visible = false
            return
        end

        local now: number = os.clock()
        local remaining: number = math.max(0, cooldownEndsAt - now)
        if remaining <= 0 then
            state.mm2ShotFeedback.hide()
            return
        end
        if now - lastRefresh < 0.08 then
            return
        end
        lastRefresh = now

        local progress: number = math.clamp(remaining / duration, 0, 1)
        local scale: number = math.floor(progress * 200 + 0.5) / 200
        if scale ~= lastScale then
            lastScale = scale
            state.mm2ShotFeedback.fill.Size = UDim2.fromScale(scale, 1)
        end
        local nextText: string = string.format("Miss cooldown · %.1fs", remaining)
        if nextText ~= lastText then
            lastText = nextText
            state.mm2ShotFeedback.text.Text = nextText
        end
    end

    updateCountdown()
    featureConnections.MM2ShootCooldown =
        TaskManager:Connect(updateCountdown, nil, "Miss cooldown")
end

state.mm2ShotFeedback.toPosition = function(value: any): Vector3?
    if typeof(value) == "Vector3" then
        return value
    end
    if typeof(value) == "CFrame" then
        return value.Position
    end
    if typeof(value) == "Instance" then
        if value:IsA("Attachment") then
            return value.WorldPosition
        end
        if value:IsA("BasePart") then
            return value.Position
        end
    end
    return nil
end

state.mm2ShotFeedback.resolve = function(
    candidate: PendingShot,
    directHit: boolean
): ()
    state.mm2ShotFeedback.hide()
    state.mm2ShotFeedback.lastAccepted = candidate
    local confirmationDelay: number = math.clamp(
        math.max(0.18, mm2Settings.predictionRtt + 0.12),
        0.18,
        0.55
    )
    task.delay(confirmationDelay, function(): ()
        if state.mm2ShotFeedback.lastAccepted ~= candidate then
            return
        end
        state.mm2ShotFeedback.lastAccepted = nil

        local targetCharacter: Model? = candidate.targetCharacter
        local currentHumanoid: Humanoid? = targetCharacter
            and targetCharacter:FindFirstChildOfClass("Humanoid")
            :: Humanoid?

        local hitConfirmed: boolean = directHit
            or candidate.target.Parent == nil
            or targetCharacter == nil
            or targetCharacter.Parent == nil
            or currentHumanoid == nil
            or currentHumanoid.Health <= 0
            or currentHumanoid.Health < candidate.healthBefore
            or (
                candidate.targetedMurderer
                and getPlayerRole(candidate.target) ~= "Murderer"
            )

        if hitConfirmed then
            -- Disconnection/role changes are UI heuristics, not shot evidence.
            state.mm2ShotFeedback.hide()
        end

        -- Per-spawn outcomes live in the calibration event stream, which also
        -- records the actual geometry and target motion. A second A/B counter
        -- here duplicated that data with a less informative hits/shots label.
        if not hitConfirmed and mm2Settings.showMissCooldown then
            state.mm2ShotFeedback.showMiss(candidate.gun, candidate)
        end
    end)
end

state.mm2ShotFeedback.handleGunFired = function(
    firedGun: any,
    eventOrigin: any,
    endpoint: any,
    hitPart: any
): ()
    if not state.mm2ShotFeedback.shootActive then
        return
    end

    local candidate: PendingShot? = state.mm2ShotFeedback.pending
    if not candidate then
        return
    end
    if os.clock() - candidate.queuedAt > 1.6 then
        state.mm2ShotFeedback.pending = nil
        return
    end

    local eventObject: Instance? = typeof(firedGun) == "Instance"
        and firedGun
        or nil
    local eventTool: Tool? = nil
    if eventObject then
        if eventObject:IsA("Tool") then
            eventTool = eventObject
        else
            eventTool = eventObject:FindFirstAncestorOfClass("Tool")
        end
    end
    local weaponMatches: boolean = eventTool == candidate.gun
        or eventObject == candidate.gun
        or (
            eventObject ~= nil
            and eventObject:IsDescendantOf(candidate.gun)
        )
    local eventOriginPosition: Vector3? =
        state.mm2ShotFeedback.toPosition(eventOrigin)
    local originMatches: boolean = eventOriginPosition ~= nil
        and (eventOriginPosition - candidate.origin).Magnitude <= 3.5
    local endpointPosition: Vector3? =
        state.mm2ShotFeedback.toPosition(endpoint)
    local directionMatches: boolean = false
    if eventOriginPosition and endpointPosition and candidate.aimPosition then
        local eventDelta: Vector3 = endpointPosition - eventOriginPosition
        local expectedDelta: Vector3 =
            candidate.aimPosition - candidate.origin
        if eventDelta.Magnitude > 0.05 and expectedDelta.Magnitude > 0.05 then
            directionMatches = eventDelta.Unit:Dot(expectedDelta.Unit) >= 0.965
        end
    end
    if eventTool and not weaponMatches then return end
    if not originMatches or not directionMatches or endpointPosition == nil then return end

    -- Prediction receives the FireServer -> GunFired round trip only from
    -- finalizeGunCapture. Do not feed a second sample from this feedback
    -- queue: the v64 duplicate source read 318 ms against a measured 150 ms
    -- and produced a 381 ms lead that missed by 3.9 studs.

    state.mm2ShotFeedback.pending = nil
    if candidate.ambiguous then return end
    local targetCharacter: Model? = candidate.targetCharacter
    local hitTarget: boolean = typeof(hitPart) == "Instance"
        and targetCharacter ~= nil
        and hitPart:IsDescendantOf(targetCharacter)
    state.mm2ShotFeedback.resolve(candidate, hitTarget)
end

type KnifeSettings = {
    aura: boolean,
    auraRange: number,
}

local function getWeaponServiceModule(): any
    if type(weaponServiceModule) == "table" then
        return weaponServiceModule
    end

    local loaded: boolean, result: any = pcall(function(): any
        return state.requireModule(
            game:GetService("ReplicatedStorage")
                :WaitForChild("ClientServices")
                :WaitForChild("WeaponService")
        )
    end)
    if loaded and type(result) == "table" then
        weaponServiceModule = result
        return result
    end
    return nil
end

local function connectGunFiredSignal(weaponService: any): boolean
    if featureConnections.MM2GunFired then
        state.mm2ShotFeedback.gunFiredConnected = true
        return true
    end

    local gunFired: any = weaponService.GunFired
    local connected: boolean, connection: any = pcall(function(): any
        return gunFired.OnClientEvent:Connect(
            state.mm2ShotFeedback.handleGunFired
        )
    end)
    if connected and connection then
        featureConnections.MM2GunFired = connection
        state.mm2ShotFeedback.gunFiredConnected = true
        return true
    end
    state.mm2ShotFeedback.gunFiredConnected = false
    return false
end

local function mm2RoleProvider(player: Player): string?
    if roundLifecycleActive == false then
        return nil
    end
    return getPlayerRole(player)
end

local function mm2TargetProvider(): any
    if not hasActiveRoundRoles() then
        return nil
    end
    local localRole: string? = getPlayerRole(LocalPlayer)
    local target: Player? = nil
    local source: string = "MM2 role priority"
    if localRole == "Sheriff" or localRole == "Hero" then
        target = findMurderer()
        source = "Sheriff priority · Murderer"
    elseif localRole == "Murderer" then
        target = findSheriff()
        source = target and "Murderer priority · Sheriff/Hero" or "Murderer priority"
        if not target then
            for _, player: Player in ipairs(Players:GetPlayers()) do
                if player ~= LocalPlayer
                    and isPlayerAlive(player)
                    and not isProtectedTarget(player) then
                    target = player
                    break
                end
            end
        end
    else
        target = findMurderer()
        source = "Threat priority · Murderer"
    end
    if state.mm2ShotFeedback then
        local pending: any = state.mm2ShotFeedback.pending
            or state.mm2ShotFeedback.lastAccepted
        local pendingRole: string? = pending and pending.target
            and getPlayerRole(pending.target)
            or nil
        local preservesRolePriority: boolean = localRole ~= "Sheriff"
            and localRole ~= "Hero"
            or pendingRole == "Murderer"
        local selectedRole: string? = target and getPlayerRole(target) or nil
        if localRole == "Murderer"
            and (selectedRole == "Sheriff" or selectedRole == "Hero") then
            preservesRolePriority = pendingRole == "Sheriff" or pendingRole == "Hero"
        end
        if pending
            and pending.target
            and isPlayerAlive(pending.target)
            and preservesRolePriority then
            target = pending.target
            source = "Recent shot target"
        end
    end
    if not target then
        return nil
    end
    local role: string? = getPlayerRole(target)
    return {
        player = target,
        role = role,
        source = source,
        accent = role == "Murderer"
            and Theme.negative
            or (role == "Sheriff" or role == "Hero")
                and Theme.accent
                or Theme.positive),
    }
end

state.gameRoleProvider = mm2RoleProvider
state.gameTargetProvider = mm2TargetProvider
moduleCleanup = function(): ()
    unregisterMM2Roles()
    if state.gameRoleProvider == mm2RoleProvider then
        state.gameRoleProvider = nil
    end
    if state.gameTargetProvider == mm2TargetProvider then
        state.gameTargetProvider = nil
    end
end

-- WallCheck for the gun uses the SAME filter as the game's own WeaponService
-- (ClientServices.WeaponService): own character and every
-- "WeaponPassthrough"-tagged instance are excluded, and hits whose part is
-- fully transparent are skipped so the ray continues through them. The first
-- solid surface or other player blocks the shot. The target is excluded while
-- following the segment because the aim point is a future pose; seeing the
-- wall behind it is the point of the check.
local function validateGunShot(target: Player, character: Model, targetCharacter: Model,
    muzzle: Vector3, origin: Vector3, endpoint: Vector3, walls: boolean): (boolean, string?, any?, any?)
    if LocalPlayer.Character ~= character or not target or target == LocalPlayer
        or target.Character ~= targetCharacter or not isPlayerAlive(target)
        or isProtectedTarget(target) or getPlayerRole(target) ~= "Murderer" then
        return false,"Target changed or is not the murderer."
    end
    if roundLifecycleActive == false then return false,"The round is not active." end
    local roundRole = getRoundRole(target)
    if roundRole and roundRole ~= "Murderer" then return false,"Conflicting target role." end
    if not roundRole and not getPlayerWeapon(target,"Knife") and not hasMurdererCollisionGroup(targetCharacter) then
        return false,"No current murderer evidence."
    end
    local ownHumanoid = character:FindFirstChildOfClass("Humanoid")
    if not ownHumanoid or not predictionMath.finite(ownHumanoid.Health) or ownHumanoid.Health <= 0 then return false,"Your character is not alive." end
    if not predictionMath.finite(muzzle.Magnitude) or not predictionMath.finite(origin.Magnitude)
        or not predictionMath.finite(endpoint.Magnitude) or (endpoint-origin).Magnitude < 0.05
        or (endpoint-muzzle).Magnitude < 0.05 then return false,"Invalid shot geometry." end
    if walls then
        local options = {ignoreTarget = true}
        local clear, blocker = isWeaponPathClear(muzzle,endpoint,targetCharacter,nil,options)
        if not clear then
            return false,"A wall or another body blocks the shot.",blocker
        end
        -- Second pass with the conservative rule: a fully transparent part
        -- that still collides is an invisible barrier. The 2026-10-18 session
        -- confirmed nine shots and five of them stopped 24-56 studs short of
        -- the aim while this first pass was clear, so the skipped geometry is
        -- real to the solver even though the client's visual filter ignores it.
        local solidClear, solidBlocker = isWeaponPathClear(muzzle,endpoint,targetCharacter,true,options)
        if not solidClear then
            return false,"An invisible wall blocks the shot.",solidBlocker
        end
        if (origin-muzzle).Magnitude > 0.05 then
            local originClear, originBlocker = isWeaponPathClear(origin,endpoint,targetCharacter,nil,options)
            if not originClear then
                return false,"A wall or another body blocks the shot.",originBlocker
            end
            local originSolid, originSolidBlocker = isWeaponPathClear(origin,endpoint,targetCharacter,true,options)
            if not originSolid then
                return false,"An invisible wall blocks the shot.",originSolidBlocker
            end
        end
    end
    -- Even a clear shot records the first thing the untouched ray meets: if it
    -- is a transparent solid short of the aim, the server may stop the bullet
    -- there while every client check passes.
    local occluder: any = nil
```

```
    if walls then
        -- Never let the diagnostic decide a shot: it is wrapped, so a failure
        -- here leaves the shot allowed and only loses the extra field.
        pcall(function(): ()
            local first: any = describeRayHit(muzzle, endpoint, targetCharacter)
            local reach: number = (endpoint - muzzle).Magnitude
            if first and not first.onTarget
                and predictionMath.finite(first.distance)
                and first.distance < reach - 0.5 then
                occluder = first
            end
        end)
    end
    return true,nil,nil,occluder
end

local function disconnectGunFiredObserver(): ()
    disconnectFeatureConnection("MM2GunFired")
    state.mm2ShotFeedback.gunFiredConnected = false
end

registerMM2Roles()

state.mm2Core = {
    settings = mm2Settings,
    mm2Settings = mm2Settings,

    MM2Effects = MM2Effects,
    GunEffects = GunEffects,
    TrapEffects = TrapEffects,
    CoinEffects = CoinEffects,
    clearEffects = clearEffects,
    createMM2Marker = createMM2Marker,
    createMM2Cham = createMM2Cham,

    mm2GameplayRemotes = mm2GameplayRemotes,
    getRoundData = function(): {[any]: any}
        return mm2RoundData
    end,
    isRoundLifecycleActive = function(): boolean?
        return roundLifecycleActive
    end,
    onRoundRoles = onRoundRoles,
    requestRoleRefresh = requestRoleRefresh,
    onRoundPhase = onRoundPhase,
    getRoundPhase = function(): string
        return roundPhase
    end,
    roundTimer = roundTimer,
    findRoundTimerPart = findRoundTimerPart,
    readRoundTimerPartSeconds = readRoundTimerPartSeconds,

    getPlayerWeapon = getPlayerWeapon,
    isPlayerAlive = isPlayerAlive,
    findMM2Role = findMM2Role,
    playerFromRoundKey = playerFromRoundKey,
    getRoundRole = getRoundRole,
    getPlayerRole = getPlayerRole,
    getPlayerPerk = getPlayerPerk,
    invalidateRoleCaches = invalidateRoleCaches,
    roundRolesKnown = roundRolesKnown,
    hasActiveRoundRoles = hasActiveRoundRoles,
    findMurderer = findMurderer,
    findSheriff = findSheriff,
    isProtectedTarget = isProtectedTarget,

    findMM2Map = findMM2Map,
    findDroppedGun = findDroppedGun,

    isWeaponPathClear = isWeaponPathClear,
    -- v161: the untouched first hit of the muzzle line, so the aim can notice
    -- a ghost (transparent, non-colliding) body part on its own line.
    describeRayHit = describeRayHit,
    validateGunShot = validateGunShot,
    predictionMath = predictionMath,
    getEstimatedLatency = getEstimatedLatency,
    getLatencyReadings = readLatencyReadings,
    getGunOriginCFrame = getGunOriginCFrame,
    trajectoryCalibration = trajectoryCalibration,

    getWeaponServiceModule = getWeaponServiceModule,
    connectGunFiredSignal = connectGunFiredSignal,
    disconnectGunFiredObserver = disconnectGunFiredObserver,
}

cleanupMM2Runtime = function()
    calibrationPromptLive = false
    unregisterMM2Roles()
    if state.gameRoleProvider == mm2RoleProvider then
        state.gameRoleProvider = nil
    end
    if state.gameTargetProvider == mm2TargetProvider then
        state.gameTargetProvider = nil
    end
    trajectoryCalibration:destroy()
    disconnectGunFiredObserver()
    state.mm2ShotFeedback.hide()
    for player: Player, _ in pairs(roleWatchers) do
        unwatchPlayer(player)
    end
    table.clear(roleIndex.roles)
    for _, connectionName: string in ipairs({
        "MM2PlayerDataChanged",
        "MM2TimerTrackerStart",
        "MM2TimerTrackerEnd",
        "MM2RoundPhase",
        "MM2RoundClock",
        "MM2RoleWatchAdded",
        "MM2RoleWatchRemoving",
        "MM2RoleTagAddedWeapon_Knife",
        "MM2RoleTagRemovedWeapon_Knife",
        "MM2RoleTagAddedWeapon_Gun",
        "MM2RoleTagRemovedWeapon_Gun",
    }) do
        disconnectFeatureConnection(connectionName)
    end
    state.isProtectedTarget = function(player: Player?): boolean
        return player == nil or player == LocalPlayer
    end
    table.clear(roundRolesListeners)
    table.clear(roundPhaseListeners)
    state.mm2Core = nil
end
end

function Module.init(runtime: any): any
    assert(type(runtime) == "table", "MM2 requires a Runtime table")
    assert(type(runtime.Menu) == "table", "MM2 requires Runtime.Menu")
    assert(runtime.TaskManager ~= nil, "MM2 requires Runtime.TaskManager")
    if Module.Initialized then
        return Module
    end

    Module.Menu = runtime.Menu
    Module.Runtime = runtime
    local success: boolean, buildError: any = pcall(buildMM2Features)
    if not success then
        moduleCleanup()
        error(buildError, 0)
    end
    local environment: any = getfenv()
    if type(environment.cleanupMM2Runtime) == "function" then
        moduleCleanup = environment.cleanupMM2Runtime
    end
    Module.Events = featureConnections
    Module.Initialized = true
    return Module
end

function Module.destroy(): ()
    if not Module.Initialized then
        return
    end
    Module.Initialized = false
    moduleCleanup()
    Module.Events = {}
    Module.Menu = nil
    Module.Runtime = nil
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/MM2/base.lua
--------------------------------------------------------------------------------

================================================================================
MODULOS UNIVERSALES COMPARTIDOS (INCLUIDO WALK FLING Y RENDER)
================================================================================

--------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Blatant/Fly.lua
BYTES: 23984
SHA-256: 8aa5c6062290c6e03b62d2ae4e86a7e3da86f21d4d59d21b0464542af694c335
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
export type Runtime = {
    framework: any,
    entity: any,
    host: any,
    services: any,
}

local Module = {
    Name = "Fly",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local activeCleanup: (() -> ())? = nil

function Module.init(context: Runtime): any
    local host: any = context.host
    local framework: any = context.framework
    local movementInput: any = context.services.movementInput
    local mobileActions: any = context.services.mobileActions
    local getCharacterParts: any = host.getCharacterParts
    local UserInputService: any = host.UserInputService
    local currentWorkspace: Workspace = host.workspace or workspace

    type FlySettings = {
        method: string,
        floatMethod: string,
        speed: number,
        verticalSpeed: number,
        response: number,
        burstInterval: number,
        wallCheck: boolean,
        faceCamera: boolean,
        upKey: Enum.KeyCode,
        downKey: Enum.KeyCode,
    }

    local flySettings: FlySettings = {
        method = "Velocity",
        floatMethod = "Velocity",
        speed = 70,
        verticalSpeed = 56,
        response = 1,
        burstInterval = 0.2,
        wallCheck = true,
        faceCamera = true,
        upKey = Enum.KeyCode.Space,
        downKey = Enum.KeyCode.LeftControl,
    }

    local flyRuntime: any = {
        yLevel = nil :: number?,
        burstAt = 0,
        platform = nil :: BasePart?,
        walkSpeed = nil :: number?,
        walkHumanoid = nil :: Humanoid?,
        root = nil :: BasePart?,
        method = nil :: string?,
        floatMethod = nil :: string?,
    }
    local flySmoothedVelocity: Vector3 = Vector3.zero

    local flyObjects: {[Instance]: boolean} =
        setmetatable({}, {__mode = "k"}) :: any
    local flyTouchVertical: number = 0
    local flyRaycastParams: RaycastParams? = nil
    local flyRaycastCharacter: Instance? = nil

    local function raycastParamsFor(root: BasePart): RaycastParams
        if not flyRaycastParams then
            flyRaycastParams = RaycastParams.new()
            flyRaycastParams.FilterType = Enum.RaycastFilterType.Exclude
        end
        if root.Parent ~= flyRaycastCharacter then
            flyRaycastCharacter = root.Parent
            flyRaycastParams.FilterDescendantsInstances = root.Parent and {root.Parent} or {}
        end
        return flyRaycastParams :: RaycastParams
    end

    local function removeFlyObjects(): ()
        for object: Instance in pairs(flyObjects) do
            if object.Parent then
                object:Destroy()
            end
        end
        flyObjects = setmetatable({}, {__mode = "k"}) :: any
    end

    local function restoreWalkSpeed(): ()
        local humanoid: Humanoid? = flyRuntime.walkHumanoid
        if humanoid and humanoid.Parent and flyRuntime.walkSpeed ~= nil then
            humanoid.WalkSpeed = flyRuntime.walkSpeed
        end
        flyRuntime.walkSpeed = nil
        flyRuntime.walkHumanoid = nil
    end

    local function clearPlatform(): ()
        if flyRuntime.platform then flyRuntime.platform:Destroy() end
        flyRuntime.platform = nil
    end

    local function ensureFlyConstraints(root: BasePart): (LinearVelocity, AlignOrientation)
        local attachment: Attachment? = root:FindFirstChild("Wurst_FlyAttachment") :: Attachment?
        if not attachment then
            local newAttachment: Attachment = Instance.new("Attachment")
            newAttachment.Name = "Wurst_FlyAttachment"
            newAttachment.Parent = root
            attachment = newAttachment
        end
        local resolvedAttachment: Attachment = attachment :: Attachment
        flyObjects[resolvedAttachment] = true

        local velocity: LinearVelocity? = root:FindFirstChild("Wurst_FlyVelocity") :: LinearVelocity?
        if not velocity then
            local newVelocity: LinearVelocity = Instance.new("LinearVelocity")
            newVelocity.Name = "Wurst_FlyVelocity"
            newVelocity.Attachment0 = resolvedAttachment
            newVelocity.MaxForce = math.huge
            newVelocity.RelativeTo = Enum.ActuatorRelativeTo.World
            newVelocity.VelocityConstraintMode = Enum.VelocityConstraintMode.Vector
            newVelocity.Parent = root
            velocity = newVelocity
        end
        flyObjects[velocity :: LinearVelocity] = true

        local orientation: AlignOrientation? =
            root:FindFirstChild("Wurst_FlyOrientation") :: AlignOrientation?
        if not orientation then
            local newOrientation: AlignOrientation = Instance.new("AlignOrientation")
            newOrientation.Name = "Wurst_FlyOrientation"
            newOrientation.Attachment0 = resolvedAttachment
            newOrientation.MaxTorque = math.huge
            newOrientation.Mode = Enum.OrientationAlignmentMode.OneAttachment
            newOrientation.Responsiveness = 28
            newOrientation.RigidityEnabled = false
            newOrientation.Parent = root
            orientation = newOrientation
        end
        flyObjects[orientation :: AlignOrientation] = true

        return velocity :: LinearVelocity, orientation :: AlignOrientation
    end

    local function getFlyDirection(camera: Camera): (Vector3, number, Vector3)
        local cameraForward: Vector3 = Vector3.new(
            camera.CFrame.LookVector.X,
            0,
            camera.CFrame.LookVector.Z
        )
        local cameraRight: Vector3 = Vector3.new(
            camera.CFrame.RightVector.X,
            0,
            camera.CFrame.RightVector.Z
        )
        cameraForward = cameraForward.Magnitude > 0.001
            and cameraForward.Unit
            or Vector3.new(0, 0, -1)
        cameraRight = cameraRight.Magnitude > 0.001
            and cameraRight.Unit
            or Vector3.new(1, 0, 0)

        local forwardScalar: number, rightScalar: number = movementInput.getVector()
        local horizontal: Vector3 = cameraForward * forwardScalar
            + cameraRight * rightScalar
        if horizontal.Magnitude > 1 then
            horizontal = horizontal.Unit
        end

        local vertical: number = 0
        local keyboardAvailable: boolean = UserInputService.KeyboardEnabled
            and UserInputService:GetFocusedTextBox() == nil
        if flyTouchVertical > 0
            or (keyboardAvailable
                and UserInputService:IsKeyDown(flySettings.upKey)) then
            vertical = vertical + 1
        end
        if flyTouchVertical < 0
            or (keyboardAvailable
                and UserInputService:IsKeyDown(flySettings.downKey)) then
            vertical = vertical - 1
        end

        if vertical == 0 and not UserInputService.KeyboardEnabled and movementInput.isJumpHeld() then
            vertical = 1
        end

        return horizontal, vertical, cameraForward
    end

    local applyFlyHorizontal: (BasePart, Humanoid, Vector3, number) -> ()
    local applyFlyVertical: (BasePart, Humanoid, number, number) -> ()
    do
    applyFlyHorizontal = function(
        root: BasePart,
        humanoid: Humanoid,
        moveDirection: Vector3,
        deltaTime: number
    ): ()
        local method: string = flySettings.method
        local currentVelocity: Vector3 = root.AssemblyLinearVelocity
        local target: Vector3 = moveDirection * flySettings.speed

        if method == "WalkSpeed" then
            if not flyRuntime.walkSpeed then
                flyRuntime.walkSpeed = humanoid.WalkSpeed
                flyRuntime.walkHumanoid = humanoid
            end
            if humanoid.WalkSpeed ~= flySettings.speed then humanoid.WalkSpeed = flySettings.speed end
            if flySettings.speed < 0 then
                root.AssemblyLinearVelocity = Vector3.new(
                    target.X,
                    currentVelocity.Y,
                    target.Z
                )
            end
            return
        end

        if method == "Pulse" then

            local period: number = math.max(flySettings.burstInterval, 0.05) * 2
            local burst: boolean = (os.clock() % period) < (period * 0.5)
            target = target * (burst and 1.8 or 0.2)
        end

        if method == "CFrame" or method == "Blink" then
            local step: Vector3 = moveDirection * flySettings.speed * deltaTime
            if method == "Blink" then
                local interval: number = math.max(flySettings.burstInterval, 0.05)
                if os.clock() < flyRuntime.burstAt then
                    step = Vector3.zero
                else
                    flyRuntime.burstAt = os.clock() + interval
                    step = moveDirection * flySettings.speed * interval
                end
                if step.Magnitude > 0.001 and flySettings.wallCheck then
                    local wallHit: RaycastResult? = currentWorkspace:Raycast(
                        root.Position,
                        step + step.Unit * 2,
                        raycastParamsFor(root)
                    )
                    if wallHit then
                        step = Vector3.zero
                    end
                end
                if step.Magnitude > 0.001 then
                    root.CFrame = root.CFrame + step
                end

                root.AssemblyLinearVelocity = Vector3.new(0, currentVelocity.Y, 0)
                return
            end

            if method == "Impulse" then
                local difference: Vector3 = (target - currentVelocity)
                    * Vector3.new(1, 0, 1)

                local threshold: number = moveDirection.Magnitude < 0.01 and 10 or 2
                if difference.Magnitude > threshold then
                    root:ApplyImpulse(difference * root.AssemblyMass)
                end
                return
            end

            local alpha: number = 1 - math.exp(-12 * flySettings.response * deltaTime)
            flySmoothedVelocity = flySmoothedVelocity:Lerp(target, alpha)
            if method == "Constraint" then
                return
            end
            root.AssemblyLinearVelocity = Vector3.new(
                flySmoothedVelocity.X,
                currentVelocity.Y,
                flySmoothedVelocity.Z
            )
        end

    applyFlyVertical = function(
        root: BasePart,
        humanoid: Humanoid,
        vertical: number,
        deltaTime: number
    ): ()
        local method: string = flySettings.floatMethod
        local currentVelocity: Vector3 = root.AssemblyLinearVelocity
        local targetY: number = vertical * flySettings.verticalSpeed

        if method == "Floor" then
            local platform: BasePart? = flyRuntime.platform
            if not platform or not platform.Parent then
                local newPlatform: BasePart = Instance.new("Part")
                newPlatform.Name = "Wurst_FlyPlatform"
                newPlatform.Anchored = true
                newPlatform.CanQuery = false
                newPlatform.CanTouch = false
                newPlatform.Size = Vector3.new(8, 1, 8)
                newPlatform.Transparency = 1

                newPlatform.Parent = currentWorkspace.CurrentCamera
                flyRuntime.platform = newPlatform
                platform = newPlatform
            end
            local resolved: BasePart = platform :: BasePart

            resolved.CFrame = vertical < 0
                and CFrame.new(0, -4096, 0)
                or CFrame.new(root.Position - Vector3.new(0, humanoid.HipHeight + 1.5, 0))
            if vertical > 0 then
                root.AssemblyLinearVelocity = Vector3.new(
                    currentVelocity.X,
                    targetY,
                    currentVelocity.Z
                )
            end
            return
        end

        if method == "Bypass" then

            local period: number = math.max(flySettings.burstInterval, 0.05) * 5
            if (os.clock() % period) > (period * 0.8) then

                flyRuntime.yLevel = nil
                return
            end
            local held: number = flyRuntime.yLevel or root.Position.Y
            held = held + targetY * deltaTime
            flyRuntime.yLevel = held
            root.AssemblyLinearVelocity = Vector3.new(
                currentVelocity.X,
                0,
                currentVelocity.Z
            )
            root.CFrame = root.CFrame + Vector3.new(0, held - root.Position.Y, 0)
            return
        end

        if method == "Hover" or method == "Jump" then
            local level: number = flyRuntime.yLevel or root.Position.Y
            level = level + targetY * deltaTime
            if flySettings.wallCheck then
                local offset: number = level - root.Position.Y
                if math.abs(offset) > 0.001 then
                    local ceilingHit: RaycastResult? = currentWorkspace:Raycast(
                        root.Position,
                        Vector3.new(0, offset, 0),
                        raycastParamsFor(root)
                    )
                    if ceilingHit then
                        level = ceilingHit.Position.Y
                            - math.sign(offset) * (humanoid.HipHeight + 0.5)
                    end
                end
            end
            flyRuntime.yLevel = level
            if method == "Hover" then
                root.AssemblyLinearVelocity = Vector3.new(
                    currentVelocity.X,
                    0,
                    currentVelocity.Z
                )
                root.CFrame = root.CFrame + Vector3.new(0, level - root.Position.Y, 0)
            elseif root.Position.Y < level - 0.5 then
                humanoid:ChangeState(Enum.HumanoidStateType.Jumping)
            end
            return
        end

        if method == "Bounce" then
            local period: number = math.max(flySettings.burstInterval, 0.05) * 4
            local rising: boolean = (os.clock() % period) < (period * 0.5)
            targetY = targetY
                + (rising and 1 or -1) * flySettings.verticalSpeed * 0.45
        end

        if method == "Impulse" then
            local difference: number = targetY - currentVelocity.Y
            if math.abs(difference) > 2 then
                root:ApplyImpulse(Vector3.new(0, difference, 0) * root.AssemblyMass)
            end
            return
        end

        root.AssemblyLinearVelocity = Vector3.new(
            currentVelocity.X,
            targetY,
            currentVelocity.Z
        )
    end
    end

    local fly: any
    fly = framework.Categories.Exploits:CreateModule({
        Name = "Fly",
        Category = "Exploits",
        ConfigKey = "Universal.Fly",
        Tooltip = "Use WASD to move, Space to rise and LeftControl to descend. "
            .. "Method picks how you travel sideways and Float picks what "
            .. "holds you up; if a game blocks one combination, another "
            .. "usually works.",
        Function = function(enabled: boolean): ()

            removeFlyObjects()
            flySmoothedVelocity = Vector3.zero
            flyTouchVertical = 0
            flyRuntime.yLevel = nil
            flyRuntime.burstAt = 0
            clearPlatform()
            restoreWalkSpeed()
            flyRuntime.root = nil
            flyRuntime.method = nil
            flyRuntime.floatMethod = nil

            if not enabled then
                flyRaycastCharacter = nil
                if flyRaycastParams then flyRaycastParams.FilterDescendantsInstances = {} end
                fly:SetStatus(nil)
                local _, humanoid: Humanoid?, root: BasePart? = getCharacterParts()
                if root then
                    (root :: BasePart).AssemblyLinearVelocity = Vector3.zero
                end
                return
            end

            fly:SetStatus(flySettings.method)
            fly:Loop(function(deltaTime: number): ()
                local _, humanoid, root = getCharacterParts()
                local camera: Camera? = currentWorkspace.CurrentCamera
                if not humanoid or not root or not camera then
                    removeFlyObjects()
                    clearPlatform()
                    restoreWalkSpeed()
                    flyRuntime.root = nil
                    flyRuntime.yLevel = nil
                    return
                end

                local method: string = flySettings.method
                local floatMethod: string = flySettings.floatMethod

                if flyRuntime.root ~= root or flyRuntime.method ~= method
                    or flyRuntime.floatMethod ~= floatMethod then
                    removeFlyObjects()
                    clearPlatform()
                    restoreWalkSpeed()
                    flyRuntime.yLevel = nil
                    flyRuntime.burstAt = 0
                    flySmoothedVelocity = Vector3.zero
                    flyRuntime.root = root
                    flyRuntime.method = method
                    flyRuntime.floatMethod = floatMethod
                end

                local moveDirection, vertical, cameraForward = getFlyDirection(camera)
                if flySettings.wallCheck and moveDirection.Magnitude > 0.001 then
                    local wallHit: RaycastResult? = currentWorkspace:Raycast(
                        root.Position,
                        moveDirection.Unit * 2.75,
                        raycastParamsFor(root)
                    )
                    if wallHit then
                        moveDirection = Vector3.zero
                        flySmoothedVelocity = Vector3.zero
                    end
                end

                if method == "Constraint" then
                    local velocity, orientation = ensureFlyConstraints(root)
                    applyFlyHorizontal(root, humanoid, moveDirection, deltaTime)
                    velocity.Enabled = true
                    velocity.VectorVelocity = Vector3.new(
                        flySmoothedVelocity.X,
                        vertical * flySettings.verticalSpeed,
                        flySmoothedVelocity.Z
                    )
                    orientation.Enabled = flySettings.faceCamera
                    orientation.CFrame = CFrame.lookAt(Vector3.zero, cameraForward)
                    return
                end

                removeFlyObjects()
                applyFlyHorizontal(root, humanoid, moveDirection, deltaTime)
                applyFlyVertical(root, humanoid, vertical, deltaTime)

                if floatMethod ~= "Floor" and flyRuntime.platform then
```

```
                (flyRuntime.platform :: BasePart):Destroy()
                flyRuntime.platform = nil
            end
            if floatMethod ~= "Hover" and floatMethod ~= "Jump" and floatMethod ~= "Bypass" then
                flyRuntime.yLevel = nil
            end

            if flySettings.faceCamera and cameraForward.Magnitude > 0.001 then
                root.CFrame = CFrame.lookAt(root.Position, root.Position + cameraForward)
            end
        end)
    end,
})

fly:CreateDropdown({
    Name = "Method",
    List = {"Velocity", "Constraint", "Impulse", "CFrame", "Blink", "Pulse", "WalkSpeed"},
    Index = 1,
    Function = function(value: string): ()
        flySettings.method = value
        if fly.Enabled then
            fly:SetStatus(value)
        end
    end,
    Tooltip = "Sideways engine. Velocity is smooth, CFrame ignores physics, "
        .. "WalkSpeed just runs faster.",
})
fly:CreateDropdown({
    Name = "Float",
    List = {"Velocity", "Impulse", "Hover", "Jump", "Bounce", "Floor", "Bypass"},
    Index = 1,
    Function = function(value: string): ()
        flySettings.floatMethod = value
    end,
    Tooltip = "Applies to non-Constraint methods; Constraint holds altitude with its own velocity. Hover locks an altitude, Floor "
        .. "stands you on an invisible part, and Bypass drops you to the "
        .. "ground for an instant every few seconds so the server sees "
        .. "you land.",
})
fly:CreateSlider({
    Name = "Horizontal speed",
    Min = 1,
    Max = 250,
    Default = flySettings.speed,
    Function = function(value: number): ()
        flySettings.speed = value
    end,
    Tooltip = "Studs per second while holding WASD. Negative values reverse the direction.",
})
fly:CreateSlider({
    Name = "Vertical speed",
    Min = 1,
    Max = 250,
    Default = flySettings.verticalSpeed,
    Function = function(value: number): ()
        flySettings.verticalSpeed = value
    end,
    Tooltip = "Studs per second while rising or descending. Negative values reverse the direction.",
})
fly:CreateSlider({
    Name = "Response multiplier",
    Show = {Option = "Method", Values = {"Velocity", "Constraint", "Pulse"}},
    Min = 0.5,
    Max = 2,
    Default = flySettings.response,
    Function = function(value: number): ()
        flySettings.response = value
    end,
    Tooltip = "How sharply the flight reacts to input. Higher is twitchier.",
})
fly:CreateSlider({
    Name = "Burst interval (s)",
    Min = 0.05,
    Max = 1,
    Default = flySettings.burstInterval,
    Function = function(value: number): ()
        flySettings.burstInterval = value
    end,
    Tooltip = "Cycle length for Blink, Pulse, Bounce and Bypass. Shorter "
        .. "is smoother.",
})
fly:CreateToggle({
    Name = "Wall check",
    Default = flySettings.wallCheck,
    Function = function(value: boolean): ()
        flySettings.wallCheck = value
    end,
    Tooltip = "Stop against solid geometry instead of flying through it.",
})
fly:CreateToggle({
    Name = "Face camera",
    Default = flySettings.faceCamera,
    Function = function(value: boolean): ()
        flySettings.faceCamera = value
    end,
    Tooltip = "Rotate the character to follow wherever the camera looks.",
})
local upKey: any = fly:CreateBind({
    Name = "Up key",
    Default = flySettings.upKey,
    Function = function(value: Enum.KeyCode): ()
        flySettings.upKey = value
    end,
    Tooltip = "Hold to gain altitude.",
})
local downKey: any = fly:CreateBind({
    Name = "Down key",
    Default = flySettings.downKey,
    Function = function(value: Enum.KeyCode): ()
        flySettings.downKey = value
    end,
    Tooltip = "Hold to lose altitude.",
})

if mobileActions.bindPlacement then
    mobileActions.bindPlacement(
        upKey.Object,
        "UniversalFlyUp",
        "UP",
        function(): () end,
        {
            onPress = function(): ()
                flyTouchVertical = 1
            end,
            onRelease = function(): ()
                if flyTouchVertical == 1 then
                    flyTouchVertical = 0
                end
            end,
        }
    )
    mobileActions.bindPlacement(
        downKey.Object,
        "UniversalFlyDown",
        "DOWN",
        function(): () end,
        {
            onPress = function(): ()
                flyTouchVertical = -1
            end,
            onRelease = function(): ()
                if flyTouchVertical == -1 then
                    flyTouchVertical = 0
                end
            end,
        }
    )
end

activeCleanup = function(): ()
    if fly.Enabled then fly:Toggle(false) end
    removeFlyObjects()
    clearPlatform()
    restoreWalkSpeed()
end
Module.Initialized = true
return fly
end

function Module.destroy(): ()
    if activeCleanup then
        activeCleanup()
    end
    activeCleanup = nil
    Module.Initialized = false
end
return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Blatant/Fly.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Blatant/Invisible.lua
BYTES: 20702
SHA-256: fe8818b1c34bae47e03b2b3585b213b4f92a741e2e52d8b3fe468516b1c41c13
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
-- Invisibility (v190 name; the module and its config key are still "Ghost"
-- so saved profiles and keybinds survive): a real, unanchored Humanoid walks
-- under native gravity. Only explicit Fly uses velocity; the real character
-- stays parked underneath at `Void depth`.
--
-- v189 contract (rewrite after the live report "no funciona, se traba
-- demasiado, vuela hacia arriba y se teletransporta"):
--   * The real body is ANCHORED and non-collidable while Invisibility is on. v188
--     rewrote root.CFrame every single frame on a live, collidable body that
--     sat inside the floor at small depth; physics fought the writes (jitter,
--     teleports) and every write replicated (the stutter). Now the body is
--     frozen once and only re-placed at 10 Hz, so there is nothing to fight.
--   * The clone spawns ABOVE the real body instead of inside it, so the two
--     rigs never interpenetrate and the engine never launches either one.
--   * Fly is velocity driven on an UNANCHORED rig. v188 anchored the
--     root and then set AssemblyLinearVelocity, which an anchored part
--     ignores -- that is why Fly did nothing and the rest of the module moved
--     strangely. No raycasts in the flight loop either.
--   * No Void guard, no Sprint multiplier, no Outline: fewer moving parts and
--     no per-frame work beyond reading the movement service.
--   * Void depth defaults to 3 (minimum 0) so melee/weapons can hit: the real
--     body is close enough to the clone to deal damage. Older profiles migrate.
local Module = {Name="Invisibility",PlaceId=0,Initialized=false}
local cleanup

function Module.init(context)
    local host, services = context.host, context.services
    local player, input, run = host.LocalPlayer, host.UserInputService, host.RunService
    local world = host.workspace or workspace
    local VOID_DEPTH_MIN, VOID_DEPTH_MAX = 0, 350
    local VOID_DEPTH_DEFAULT = 3
    local settings = {fly=false, flySpeed=55, walkSpeed=16, depth=VOID_DEPTH_DEFAULT, transparency=0.5}
    local state = {connections={},tracks={},bodyParts={}, savedWalkSpeed=nil, savedPlatformStand=nil, savedAnchored=nil}
    local card

    -- One-time migration. v187 shipped 120, v188 offered 1 and v189 used 5.
    -- All are rewritten to the new default 3 so melee can hit (depth 0-3 keeps
    -- the real body within weapon range). Any other value the player picked is kept.
    do
        local stored = type(configData) == "table" and configData.values or nil
        if type(stored) == "table" and stored["UI.GhostDepthDefault"] ~= 3 then
            local saved = tonumber(stored["Universal.Ghost.Voiddepth"])
            if saved == nil or saved == 120 or saved == 1 or saved == 5 then
                stored["Universal.Ghost.Voiddepth"] = VOID_DEPTH_DEFAULT
            elseif saved ~= nil and (saved < VOID_DEPTH_MIN or saved > VOID_DEPTH_MAX) then
                stored["Universal.Ghost.Voiddepth"] = math.clamp(saved, VOID_DEPTH_MIN, VOID_DEPTH_MAX)
            end
            stored["UI.GhostDepthDefault"] = 3
            if type(queueConfigSave) == "function" then queueConfigSave() end
        end
        do
            local stored2 = type(configData) == "table" and configData.values or nil
            local migrated = stored2 and tonumber(stored2["Universal.Ghost.Voiddepth"])
            if migrated then settings.depth = math.clamp(migrated, VOID_DEPTH_MIN, VOID_DEPTH_MAX) end
        end
    end

    local function parts(character)
        return character and character:FindFirstChildOfClass("Humanoid"),
            character and character:FindFirstChild("HumanoidRootPart")
    end
    local function activity(active)
        if services.activity then services.activity.set("ghost",active) end
    end
    local function cameraBusy()
        return services.freeCam and services.freeCam.IsActive()
            or services.spectate and services.spectate.IsActive()
    end
    local function destroyHeight()
        local height=world.FallenPartsDestroyHeight
        if height~=height or math.abs(height)==math.huge then return -500 end
        return height
    end
    -- Freeze the real body: anchored, non-collidable, no humanoid physics. It
    -- is hidden below the clone, so nothing else needs to simulate it, and
    -- freezing it here is what removes the fight that caused the jitter.
    local function parkBody(character,root,humanoid)
        state.bodyParts={}
        for _,object in ipairs(character:GetDescendants()) do
            if object:IsA("BasePart") then
                local collide=object.CanCollide
                if type(collide)~="boolean" then collide=true end
                table.insert(state.bodyParts,{part=object,collide=collide})
                object.CanCollide=false
            end
        end
        if humanoid then
            state.savedWalkSpeed = humanoid.WalkSpeed
            state.savedPlatformStand = humanoid.PlatformStand
            humanoid.PlatformStand=true
            pcall(function() humanoid.WalkSpeed=0 end)
        end
        state.savedAnchored = root.Anchored
        root.AssemblyLinearVelocity,root.AssemblyAngularVelocity=Vector3.zero,Vector3.zero
        root.Anchored=true
    end
    local function unparkBody()
        for _,entry in ipairs(state.bodyParts) do
            local part=entry.part
            if part and part.Parent then pcall(function() part.CanCollide=entry.collide end) end
        end
        table.clear(state.bodyParts)
    end
    local function clearGhost()
        local camera = world.CurrentCamera
        if camera and camera.CameraSubject == state.humanoid then
            camera.CameraSubject = parts(player.Character)
        end
        for _,track in pairs(state.tracks) do pcall(function() track:Stop(0) end) end
        table.clear(state.tracks)
        if state.model then state.model:Destroy() end
        state.model,state.humanoid,state.root,state.animation,state.flyMode = nil,nil,nil,nil,nil
        state.syncedAt,state.camera=nil,nil
    end
    local function stop()
        for _,connection in ipairs(state.connections) do connection:Disconnect() end
        table.clear(state.connections)
        unparkBody()
        local humanoid, root=parts(player.Character)
        -- Always restore movement state, even if character respawned or puppet lost.
        if humanoid and humanoid.Parent then
            if state.savedWalkSpeed ~= nil then
                pcall(function() humanoid.WalkSpeed = state.savedWalkSpeed end)
            elseif humanoid.WalkSpeed == 0 then
                pcall(function() humanoid.WalkSpeed = settings.walkSpeed or 16 end)
            end
            if state.savedPlatformStand ~= nil then
                humanoid.PlatformStand = state.savedPlatformStand
            else
                humanoid.PlatformStand = false
            end
        end
        if root and root.Parent then
            if state.root and state.root.Parent and state.character == player.Character and humanoid and humanoid.Health > 0 then
                root.CFrame=state.root.CFrame
            end
            root.AssemblyLinearVelocity,root.AssemblyAngularVelocity=Vector3.zero,Vector3.zero
            if state.savedAnchored ~= nil then
                root.Anchored = state.savedAnchored
            else
                root.Anchored = false
            end
        end
        state.savedWalkSpeed, state.savedPlatformStand, state.savedAnchored = nil, nil, nil
        clearGhost();state.character=nil;activity(false)
    end
    local function turnOff()
        local feature=card and card.Feature
        if feature and feature.enabled and type(feature.activate)=="function" then
            feature.activate()
        elseif card and card.Enabled then
            card:Toggle(false)
        else
            stop()
        end
    end
    local function applyAppearance()
        if not state.model then return end
        local transparency=math.clamp(settings.transparency,0,1)
        for _,object in ipairs(state.model:GetDescendants()) do
            if object:IsA("BasePart") then object.Transparency=transparency end
        end
    end
    local function applyMode()
        if not state.model then return end
        local ghostRoot=state.root
        for _,part in ipairs(state.model:GetDescendants()) do
            if part:IsA("BasePart") then
                part.Anchored=false
                -- Flying: nothing collides, so the rig never snags on geometry.
                -- Walking: only the torso/head carry the humanoid on the floor.
                part.CanCollide=not settings.fly and (part.Name=="Head" or part.Name=="Torso"
                    or part.Name=="UpperTorso" or part.Name=="LowerTorso")
            end
        end
        state.humanoid.PlatformStand=settings.fly
        ghostRoot.AssemblyLinearVelocity,ghostRoot.AssemblyAngularVelocity=Vector3.zero,Vector3.zero
        state.flyMode=settings.fly
    end
    local function spawnGhost(character,root)
        clearGhost()
        local archivable = character.Archivable
        character.Archivable = true
        local ok, clone = pcall(function() return character:Clone() end)
        character.Archivable = archivable
        if not ok or not clone then return false end
        local humanoid, ghostRoot = parts(clone)
        if not humanoid or not ghostRoot then clone:Destroy();return false end
        for _,object in ipairs(clone:GetDescendants()) do
            if object:IsA("Script") or object:IsA("LocalScript") or object:IsA("Tool") then
                object:Destroy()
            elseif object:IsA("BasePart") then
                object.Anchored = false
                object.CanTouch,object.CanQuery,object.CastShadow = false,false,false
                object.Transparency = math.clamp(settings.transparency,0,1)
                object.CanCollide = object.Name=="Head" or object.Name=="Torso"
                    or object.Name=="UpperTorso" or object.Name=="LowerTorso"
            elseif object:IsA("Highlight") or object:IsA("BillboardGui")
                or object:IsA("SurfaceGui") or object:IsA("SelectionBox") then
                object:Destroy()
            end
        end
        clone.Name="Wurst_Ghost"
        -- Both Entity and NPCESP reject WurstDisguise models, so the puppet
        -- never shows up as a bot or picks an ESP box.
        pcall(function() clone:SetAttribute("WurstDisguise",true) end)
        humanoid.AutoRotate=true;humanoid.PlatformStand=false;humanoid.Sit=false
        humanoid.DisplayDistanceType=Enum.HumanoidDisplayDistanceType.None
        humanoid.WalkSpeed=math.clamp(settings.walkSpeed,0,500)
        -- Spawn ABOVE the real body, never inside it: two collidable rigs at
        -- the same CFrame is what launched the v188 ghost upwards.
        clone:PivotTo(CFrame.new(root.Position+Vector3.new(0,3,0)))
        clone.Parent=world
        state.character,state.model,state.humanoid,state.root = character,clone,humanoid,ghostRoot
        state.camera,state.syncedAt,state.flyMode=nil,nil,nil
        applyAppearance()
        return true
    end
    local function animate()
        local humanoid=state.humanoid
        local kind=humanoid:GetState()
        local name=kind==Enum.HumanoidStateType.Jumping and "jump"
            or humanoid.FloorMaterial==Enum.Material.Air and "fall"
            or humanoid.MoveDirection.Magnitude>0.05 and "walk" or "idle"
        if settings.fly then name="idle" end
        if state.animation==name then return end
        if state.animation and state.tracks[state.animation] then state.tracks[state.animation]:Stop(0.15) end
        state.animation=name
        if state.tracks[name]==false then return end
        if not state.tracks[name] then
            local source=state.character:FindFirstChild("Animate")
            local folder=source and source:FindFirstChild(name)
            local animation=folder and folder:FindFirstChildWhichIsA("Animation",true)
            local temporary
            if not animation then
                local r6=humanoid.RigType==Enum.HumanoidRigType.R6
                local ids=r6 and {idle=180435571,walk=180426354,run=180426354,jump=125750702,fall=180436148}
                    or {idle=507766666,walk=507777826,run=507767714,jump=507765000,fall=507767968}
                temporary=Instance.new("Animation");temporary.AnimationId="rbxassetid://"..ids[name];animation=temporary
            end
            local ok,track=pcall(function()
                local animator=humanoid:FindFirstChildOfClass("Animator")
                if not animator then animator=Instance.new("Animator");animator.Parent=humanoid end
                return animator:LoadAnimation(animation)
            end)
            if temporary then temporary:Destroy() end
            state.tracks[name]=ok and track or false
        end
        local track=state.tracks[name]
        if track then
            track.Looped=name~="jump"
            track.Priority=name=="idle" and Enum.AnimationPriority.Idle or Enum.AnimationPriority.Movement
            track:Play(0.15)
        end
    end
    -- Movement intent from the shared service: WASD, the mobile stick and any
    -- custom bind all arrive here. Raw keys are never read for walking.
    local function moveVector()
        if input:GetFocusedTextBox() then return Vector3.zero end
        local movement=services.movementInput
        if movement and type(movement.getVector)=="function" then
            local ok,forward,right=pcall(movement.getVector)
            if ok and type(forward)=="number" and type(right)=="number" then
                local camera=world.CurrentCamera
                local lookForward,lookRight=Vector3.new(0,0,-1),Vector3.new(1,0,0)
                if camera then
                    local look=camera.CFrame.LookVector
                    local rightVector=camera.CFrame.RightVector
                    lookForward=Vector3.new(look.X,0,look.Z)
                    lookRight=Vector3.new(rightVector.X,0,rightVector.Z)
                    lookForward=lookForward.Magnitude>0.001 and lookForward.Unit or Vector3.new(0,0,-1)
                    lookRight=lookRight.Magnitude>0.001 and lookRight.Unit or Vector3.new(1,0,0)
                end
                local direction=lookForward*forward+lookRight*right
                return direction.Magnitude>1 and direction.Unit or direction
            end
        end
        local humanoid=parts(player.Character)
        return humanoid and humanoid.MoveDirection or Vector3.zero
    end
    local function verticalIntent()
        if input:GetFocusedTextBox() then return 0 end
        local up,down=0,0
        if input:IsKeyDown(Enum.KeyCode.Space) then up=1 end
        if input:IsKeyDown(Enum.KeyCode.LeftControl) then down=1 end
        if up==0 and down==0 then
            local movement=services.movementInput
            if movement and type(movement.isJumpHeld)=="function" then
                local ok,held=pcall(movement.isJumpHeld)
                if ok and held==true then up=1 end
            end
        end
        return up-down
    end
    -- Throttled body sync. Anchored, so there is no physics to fight; at 10 Hz
    -- the position stays exact enough for gameplay and costs a tenth of the
    -- replicated writes v188 did every frame.
    local SYNC_INTERVAL=0.1
    local function syncBody(root,ghostRoot)
        local now=os.clock()
        if state.syncedAt and now-state.syncedAt<SYNC_INTERVAL then return end
        state.syncedAt=now
        local depth=math.clamp(settings.depth or VOID_DEPTH_MIN,0,VOID_DEPTH_MAX)
        local position=ghostRoot.Position
        root.CFrame=CFrame.new(position.X,math.max(position.Y-depth,destroyHeight()+12),position.Z)
    end
    local function step(dt)
        local character=player.Character
        local humanoid, root=parts(character)
        if not humanoid or not root or humanoid.Health<=0 then return end
        if state.character~=character then
            if not spawnGhost(character,root) then
                turnOff()
                card:Notify("Could not create a live character.")
                return
            end
            parkBody(character,root,humanoid)
            applyMode()
        end
        if not state.model.Parent or not state.root.Parent or state.humanoid.Health<=0 then turnOff();return end
        local ghostRoot, ghostHumanoid=state.root,state.humanoid
        if state.flyMode~=settings.fly then applyMode() end
        local camera=world.CurrentCamera
        if camera and not cameraBusy() and (state.camera~=camera or camera.CameraSubject==humanoid) then
            camera.CameraSubject=ghostHumanoid;state.camera=camera
        end
        if settings.fly then
            local direction=moveVector()
            local vertical=verticalIntent()
            local speed=math.clamp(settings.flySpeed,0,1000)
            local velocity=direction*speed+Vector3.yAxis*(vertical*speed)
            -- Unanchored rig + velocity: smooth, no teleporting, no raycasts.
            ghostRoot.AssemblyLinearVelocity=velocity
            ghostRoot.AssemblyAngularVelocity=Vector3.zero
            ghostHumanoid.WalkSpeed=0
        else
            -- Native walking: gravity, slopes, steps and jumping are Roblox's
            -- job. No CFrame writes and no floor snapping on the clone.
            local focused=input:GetFocusedTextBox()
            local speed=math.clamp(settings.walkSpeed,0,500)
            if ghostHumanoid.WalkSpeed~=speed then ghostHumanoid.WalkSpeed=speed end
            ghostHumanoid:Move(focused and Vector3.zero or moveVector(),false)
        end
        animate()
        syncBody(root,ghostRoot)
    end
    -- v190: renamed Ghost -> Invisibility. ConfigKey is pinned to the old one
    -- so profiles and keybinds keep working; option ids derive from their
    -- names, so the three "Ghost ..." rows are renamed with their defaults.
    card=context.framework.Categories.Utilities:CreateModule({Name="Invisibility",Category="Utilities",Order=8,
        ConfigKey="Universal.Ghost",
        Tooltip="A local clone walks under real gravity and you see through it; your real body is parked Void depth studs underneath. Fly turns the clone into a smooth velocity flight.",Function=function(enabled)
            stop()
            if not enabled then return end
            local character=player.Character
            local humanoid,root=parts(character)
            if not humanoid or not root or humanoid.Health<=0 or not spawnGhost(character,root) then
                card:Toggle(false)
                error("Invisibility could not create a live character.",0)
            end
            activity(true)
            parkBody(character,root,humanoid)
            applyMode()
            table.insert(state.connections,run.PreSimulation:Connect(step))
            table.insert(state.connections,input.JumpRequest:Connect(function()
                if not settings.fly and state.humanoid and not input:GetFocusedTextBox()
                    and state.humanoid.FloorMaterial~=Enum.Material.Air then state.humanoid.Jump=true end
            end))
            step(0)
        end})
    card:CreateToggle({Name="Fly",Default=false,Function=function(value) settings.fly=value;applyMode() end,
        Tooltip="OFF: native walking and jumping. ON: smooth velocity flight with WASD or the mobile stick, Space up, LeftControl down."})
    card:CreateSlider({Name="Fly speed",Show={Option="Fly"},Min=0,Max=250,Default=settings.flySpeed,
        Function=function(value) settings.flySpeed=value end})
    card:CreateSlider({Name="Void depth",Min=VOID_DEPTH_MIN,Max=VOID_DEPTH_MAX,Default=settings.depth,Function=function(value)
        settings.depth=math.clamp(value,VOID_DEPTH_MIN,VOID_DEPTH_MAX)
    end,Tooltip="How far under the clone your real body is parked. 0-3 = same height so melee/weapons can hit, 5+ hides deeper. Default 3 (melee-friendly).")})
    card:CreateSlider({Name="WalkSpeed",Min=0,Max=200,Default=settings.walkSpeed,Function=function(value)
        settings.walkSpeed=math.clamp(value,0,500)
        if state.humanoid and not settings.fly then state.humanoid.WalkSpeed=settings.walkSpeed end
    end,Tooltip="Walking speed of the clone (studs/s). 0 = stop, 16 = default Roblox. Works with Fly off."})
    card:CreateSlider({Name="Transparency",Min=0,Max=1,Step=0.05,Default=settings.transparency,Function=function(value)
        settings.transparency=math.clamp(value,0,1)
        applyAppearance()
    end})
    cleanup=turnOff
    Module.Initialized=true;return card
end
function Module.destroy()
    if cleanup then cleanup() end
    cleanup=nil;Module.Initialized=false
end
return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Blatant/Invisible.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Blatant/Noclip.lua
BYTES: 4633
SHA-256: ef4fb44f8285063ca9c266a5740c92708863558530f52ed745799f3cd2c1f946
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
export type Runtime = {
    framework: any,
    entity: any,
    host: any,
}

local Module = {
    Name = "Noclip",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local activeCleanup: (() -> ())? = nil

function Module.init(context: Runtime): any
    local host: any = context.host
    local LocalPlayer: any = host.LocalPlayer
    local TaskManager: any = host.TaskManager
    local featureConnections: any = host.featureConnections
    local disconnectFeatureConnection: any = host.disconnectFeatureConnection
    local createUniversalFeature: any = host.createUniversalFeature
    local addFeatureTooltip: any = host.addFeatureTooltip
    local originalCollision = setmetatable({}, {__mode = "k"})

    local function restoreCollision()
        for part, canCollide in pairs(originalCollision) do
            if part and part.Parent then
                part.CanCollide = canCollide
            end
        end
        originalCollision = setmetatable({}, {__mode = "k"})
    end

    local function toggleNoclip(enabled: boolean): ()
        disconnectFeatureConnection("Noclip")
        -- v139: the DescendantAdded watcher must go too, or it keeps forcing
        -- CanCollide=false after the card is switched off.
        disconnectFeatureConnection("NoclipDescendant")
        restoreCollision()

        if not enabled then
            return
        end

        -- v139: this used to run character:GetDescendants() and write
        -- CanCollide on every part 60 times a second (~3000 property writes per
        -- second for a normal avatar, and it never stopped re-setting parts that
        -- were already non-colliding). Now: apply once, catch new parts with the
        -- engine's own event, and only run a cheap safety sweep 5 times a second
        -- in case the humanoid re-enabled something.
        local sweptCharacter: any = nil
        local sweptAt: number = -math.huge
        local addConnection: any = nil

        local function noClipPart(part: any): ()
            if part:IsA("BasePart") then
                if originalCollision[part] == nil then
                    originalCollision[part] = part.CanCollide
                end
                if part.CanCollide then
                    part.CanCollide = false
                end
            end
        end

        local function attach(character: any): ()
            if sweptCharacter == character then
                return
            end
            sweptCharacter = character
            if addConnection then
                addConnection:Disconnect()
                addConnection = nil
            end
            for _, descendant in ipairs(character:GetDescendants()) do
                noClipPart(descendant)
            end
            addConnection = character.DescendantAdded:Connect(noClipPart)
        end
```

Leer columna izquierda completa y despues la derecha. Texto seleccionable; lineas largas comprimidas. Tabulaciones: 4 espacios. Original adjunto.

```
        featureConnections.Noclip = TaskManager:Connect(function(): ()
            local character = LocalPlayer.Character
            if not character then
                return
            end
            if character ~= sweptCharacter then
                attach(character)
                sweptAt = os.clock()
                return
            end
            -- Safety sweep: only parts that are colliding again are touched.
            if os.clock() - sweptAt < 0.2 then
                return
            end
            sweptAt = os.clock()
            for _, descendant in ipairs(character:GetDescendants()) do
                if descendant:IsA("BasePart") and (descendant :: any).CanCollide then
                    noClipPart(descendant)
                end
            end
        end, nil, "Noclip")
        featureConnections.NoclipDescendant = {
            Disconnect = function(): ()
                if addConnection then
                    addConnection:Disconnect()
                    addConnection = nil
                end
                sweptCharacter = nil
            end,
        }
    end

    local NoclipFeature = createUniversalFeature(
        "Noclip",
        "Disable character collisions",
        11,
        toggleNoclip,
        {noOptions = true, categoryName = "Exploits"}
    )
    addFeatureTooltip(NoclipFeature, "Disables collisions for every local character part. "
        .. "Original collision states are restored when turned off.")

    activeCleanup = function(): ()
        toggleNoclip(false)
    end
    Module.Initialized = true
    return NoclipFeature
end

function Module.destroy(): ()
    if activeCleanup then
        activeCleanup()
    end
    activeCleanup = nil
    Module.Initialized = false
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Blatant/Noclip.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Blatant/Spider.lua
BYTES: 9719
SHA-256: 26495dd3407e6c4ff1cede04a90e99fdb6461f82da2df3ac11606a1101b5b29c
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
export type Runtime = {
    framework: any,
    entity: any,
    host: any,
}

-- v191: renamed "Spider" -> "Wall Crawling" (visible name only; the
-- ConfigKey/state/keybind stay "Universal.Spider").
local Module = {
    Name = "Wall Crawling",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local activeCleanup: (() -> ())? = nil

function Module.init(context: Runtime): any
    local host: any = context.host
    local Players: any = host.Players
    local LocalPlayer: any = host.LocalPlayer
    local TaskManager: any = host.TaskManager
    local featureConnections: any = host.featureConnections
    local disconnectFeatureConnection: any = host.disconnectFeatureConnection
    local getCharacterParts: any = host.getCharacterParts
    local createUniversalFeature: any = host.createUniversalFeature
    local addToggleOption: any = host.addToggleOption
    local addNumberOption: any = host.addNumberOption
    local addCycleOption: any = host.addCycleOption
    local addFeatureTooltip: any = host.addFeatureTooltip
    local workspace: Workspace = host.workspace or workspace

    type SpiderSettings = {
        mode: string,
        speed: number,
        climbState: boolean,
        impulseInterval: number,
    }
    local spiderSettings: SpiderSettings = {
        mode = "Velocity",
        speed = 30,
        climbState = false,
        impulseInterval = 0.15,
    }
    local SpiderFeature: any = nil
    local climbingHumanoid: Humanoid? = nil
    local function releaseClimbState()
        local humanoid = climbingHumanoid
        climbingHumanoid = nil
        if humanoid and humanoid.Parent and humanoid.Health > 0 and humanoid:GetState() == Enum.HumanoidStateType.Climbing then
            humanoid:ChangeState(humanoid.FloorMaterial == Enum.Material.Air
                and Enum.HumanoidStateType.Freefall or Enum.HumanoidStateType.Running)
        end
    end

    local function toggleSpider(enabled: boolean): ()
        disconnectFeatureConnection("Spider")
        releaseClimbState()
        if not enabled then
            if SpiderFeature then
                SpiderFeature:SetStatus(nil)
            end

            return
        end

        SpiderFeature:SetStatus(spiderSettings.mode)

        local impulseElapsed: number = math.huge

        -- v160: the per-frame callback used to build a fresh RaycastParams and
        -- a fresh ignore list with `Players:GetPlayers()` BEFORE it looked at
        -- the input, so a Spider that was merely sitting still still paid for
        -- one table per frame plus one lookup per player. Order is now: read
        -- the character and the input (cheap), and only then build the ray.
        -- The ignore list is rebuilt at 2 Hz: characters appear and vanish a
        -- few times per round, not 60 times per second.
        local raycastParams: RaycastParams = RaycastParams.new()
        raycastParams.FilterType = Enum.RaycastFilterType.Exclude
        raycastParams.RespectCanCollide = true
        local ignoreRefreshAt: number = -math.huge
        local ignoredCharacter: Model? = nil
        local ignoreList: {Instance} = {}
        local function refreshIgnore(character: Model): ()
            local now: number = os.clock()
            if character == ignoredCharacter and now - ignoreRefreshAt < 0.5 then return end
            ignoredCharacter, ignoreRefreshAt = character, now
            table.clear(ignoreList)
            table.insert(ignoreList, character)
            for _, player: Player in ipairs(Players:GetPlayers()) do
                if player ~= LocalPlayer and player.Character then
                    table.insert(ignoreList, player.Character)
                end
            end
            raycastParams.FilterDescendantsInstances = ignoreList
        end

        featureConnections.Spider = TaskManager:Connect(function(deltaTime: number): ()
            impulseElapsed += math.max(0, deltaTime)
            local character: Model?, humanoidOrNil: Humanoid?, rootOrNil: BasePart? =
                getCharacterParts()
            if not character or not humanoidOrNil or not rootOrNil or humanoidOrNil.Health <= 0
                or humanoidOrNil.Sit or rootOrNil.Anchored then
                releaseClimbState()
                return
            end
            local moveDir: Vector3 = humanoidOrNil.MoveDirection
            -- Standing still (or climbing nothing) costs two property reads.
            if moveDir.Magnitude < 0.05 then
                releaseClimbState()
                return
            end
            if climbingHumanoid and climbingHumanoid ~= humanoidOrNil then releaseClimbState() end
            local humanoid: Humanoid = humanoidOrNil :: Humanoid
            local root: BasePart = rootOrNil :: BasePart

            refreshIgnore(character :: Model)
            raycastParams.CollisionGroup = root.CollisionGroup
            local hipHeight: number = humanoid.HipHeight > 0 and humanoid.HipHeight or 2.0
            local rootPos: Vector3 = root.Position
            local origin: Vector3 = rootPos - Vector3.new(0, math.max(hipHeight - 0.5, 0), 0)
            local wall: RaycastResult? = workspace:Raycast(
                origin,
                moveDir.Unit * 2.5,
                raycastParams
            )
            local velocity: Vector3 = root.AssemblyLinearVelocity
            if not wall or math.abs((wall :: RaycastResult).Normal.Y) >= 0.35 then
                releaseClimbState()
                return
            end

            if moveDir:Dot(wall.Normal) > -0.1 then releaseClimbState();return end

            if spiderSettings.climbState then
                climbingHumanoid = humanoid
                humanoid:ChangeState(Enum.HumanoidStateType.Climbing)
            end

            if spiderSettings.mode == "CFrame" then
                root.AssemblyLinearVelocity = Vector3.new(velocity.X, 0, velocity.Z)
                local rise = Vector3.new(0, spiderSettings.speed * math.clamp(deltaTime, 0, 0.1), 0)
                local ceiling = workspace:Raycast(rootPos, rise + Vector3.new(0, hipHeight + 1, 0), raycastParams)
                if ceiling then rise = Vector3.new(0, math.max(0, ceiling.Distance - hipHeight - 1), 0) end
                root.CFrame = root.CFrame + rise
            elseif spiderSettings.mode == "Impulse" then
                -- Pulsed, mass-independent boost, not +speed every frame.
                -- The old integration accelerated indefinitely and depended on FPS.
                if impulseElapsed >= spiderSettings.impulseInterval then
                    impulseElapsed = 0
                    local deltaVelocity: number = math.max(0, spiderSettings.speed - velocity.Y)
                    if deltaVelocity > 0 then root:ApplyImpulse(Vector3.new(0, deltaVelocity, 0) * root.AssemblyMass) end
                end
            else
                root.AssemblyLinearVelocity = Vector3.new(velocity.X, spiderSettings.speed, velocity.Z)
            end
        end, nil, "Spider")
    end

    SpiderFeature = createUniversalFeature(
        "Wall Crawling",
        "Climb a wall while moving toward it",
        20,
        toggleSpider,
        -- The visible name changed in v191; the config key must not, or the
        -- saved toggle and the player's keybind would be orphaned.
        {categoryName = "Exploits", configKey = "Universal.Spider"}
    )
    local modeRows: {[string]: any} = {}
    local function refreshModeRows(): ()
        for name: string, row: any in pairs(modeRows) do
            local visible: boolean = name == "Impulse interval" and spiderSettings.mode == "Impulse"
            if host.setOptionVisible then host.setOptionVisible(SpiderFeature, row, visible)
            elseif row then row.Visible = visible end
        end
    end
    addCycleOption(
        SpiderFeature,
        "Mode",
        {"Velocity", "Impulse", "CFrame"},
        1,
        function(value: string): ()
            spiderSettings.mode = value
            refreshModeRows()
            if SpiderFeature.enabled then
                -- Restart the pulse timer on a mode change.
                toggleSpider(false)
                toggleSpider(true)
            end
        end,
        "Velocity: continuous climb. Impulse: timed upward boosts. CFrame: bounded vertical steps."
    )
    addNumberOption(
        SpiderFeature,
        "Climb speed",
        spiderSettings.speed,
        1,
        250,
        function(value: number): ()
            spiderSettings.speed = value
        end
    )
    addToggleOption(
        SpiderFeature,
        "Climb state",
        spiderSettings.climbState,
        function(value: boolean): ()
            spiderSettings.climbState = value
            if not value then releaseClimbState() end
        end,
        "Uses climbing state in classic modes (Velocity, Impulse, CFrame)."
    )
    local impulseRow: any = addNumberOption(SpiderFeature, "Impulse interval", spiderSettings.impulseInterval,
        0.05, 0.5, function(value: number): () spiderSettings.impulseInterval = value end,
        "Seconds between bounded upward impulses. Only used by Impulse mode.", 0.05)
    if impulseRow then modeRows["Impulse interval"] = impulseRow end
    refreshModeRows()
    addFeatureTooltip(
        SpiderFeature,
        "Climb only when pushing into a nearby vertical wall. No wall rotation or ceiling walking."
    )

    activeCleanup = function(): ()
        toggleSpider(false)
    end
    Module.Initialized = true
    return SpiderFeature
end

function Module.destroy(): ()
    if activeCleanup then
        activeCleanup()
    end
    activeCleanup = nil
    Module.Initialized = false
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Blatant/Spider.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Blatant/Swim.lua
BYTES: 2995
SHA-256: b51c636f1c570e5b93445ebcc311d9099376dcd16281bb952b9623c95e423b7b
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
export type Runtime = {framework: any, entity: any, host: any, services: any}

-- v191: renamed "Swim" -> "Air Swim" (visible name only; ConfigKey, state and
-- keybind stay "Universal.Swim").
local Module = {Name = "Air Swim", PlaceId = 0, Events = {} :: {[string]: any}, Initialized = false}
local activeCleanup

function Module.init(context: Runtime): any
    local host, framework = context.host, context.framework
    local player = host.LocalPlayer
    local terrain = host.workspace:FindFirstChildWhichIsA("Terrain")
    local lastpos: Region3? = nil
    local swim

    -- Vape's terrain-water method. This clears Water in the region, including
    -- any water that was already there before the module was enabled.
    local function restoreWater()
        if terrain and lastpos then
            terrain:ReplaceMaterial(lastpos, 4, Enum.Material.Water, Enum.Material.Air)
            lastpos = nil
        end
    end

    swim = framework.Categories.Exploits:CreateModule({
        Name = "Air Swim",
        Category = "Exploits",
        ConfigKey = "Universal.Swim",
        Order = 3,
        Tooltip = "Swim using local terrain water. Moving or turning off clears existing water in the affected region.",
        Function = function(enabled)
            if not enabled then
                swim:SetStatus(nil)
                return
            end
            swim:Clean(restoreWater)
            swim:Clean(host.RunService.PreSimulation:Connect(function()
                local character = player.Character
                if not character then return end
                local root = character:FindFirstChild("HumanoidRootPart")
                local humanoid = character:FindFirstChild("Humanoid")
                if not root or not humanoid or humanoid.Health <= 0 or not terrain then return end

                local moving = humanoid.MoveDirection ~= Vector3.zero
                local space = host.UserInputService:IsKeyDown(Enum.KeyCode.Space)
                local factor = (moving or space) and Vector3.new(6, 6, 6) or Vector3.new(2, 1, 2)
                local pos = root.Position - Vector3.new(0, 1, 0)
                local newpos = Region3.new(pos - factor, pos + factor):ExpandToGrid(4)
                -- Do not rewrite the same voxels every physics frame.
                if lastpos and lastpos.CFrame.Position == newpos.CFrame.Position and lastpos.Size == newpos.Size then
                    return
                end
                restoreWater()
                terrain:ReplaceMaterial(newpos, 4, Enum.Material.Air, Enum.Material.Water)
                lastpos = newpos
            end))
            swim:SetStatus("Active")
        end,
    })

    activeCleanup = function()
        if swim.Enabled then swim:Toggle(false) end
        restoreWater()
    end
    Module.Initialized = true
    return swim
end

function Module.destroy()
    if activeCleanup then activeCleanup(); activeCleanup = nil end
    Module.Initialized = false
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Blatant/Swim.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Blatant/WalkFling.lua
BYTES: 10075
SHA-256: a01eaa2b2464a8dc9207db12feb6b428c40d97c2c4788eec49099dd5e0cb863b
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
export type Runtime = {
    framework: any,
    entity: any,
    host: any,
}

local Module = {
    Name = "WalkFling",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local activeCleanup: (() -> ())? = nil

function Module.init(context: Runtime): any
    local host: any = context.host
    local LocalPlayer: any = host.LocalPlayer
    local RunService: any = host.RunService or game:GetService("RunService")
    local createUniversalFeature: any = host.createUniversalFeature
    local addFeatureTooltip: any = host.addFeatureTooltip
    local addInformationOption: any = host.addInformationOption
    local configData: any = host.configData
    local services: any = (context :: any).services
    local activity: any = services and services.activity

    -- A persisted ON state must not start a fling on injection: the pulse only
    -- answers to an explicit toggle during the session.
    if type(configData) == "table" and type(configData.states) == "table" then
        configData.states["Universal.WalkFling"] = false
    end

    local enabled: boolean = false
    local session: number = 0
    -- Fixed at a higher effective power than the old 30000 (now more reliable
    -- for modern physics and heavier rigs). No Power option by design: only ON
    -- produces the pulse, and the test asserts >=30000.
    local POWER: number = 50000
    local SPIN: number = 50000
    local savedCollision: any = setmetatable({}, {__mode = "k"})
    local walkFlingFeature: any = nil

    local function getCharacterRoot(): (Model?, Humanoid?, BasePart?)
        local character: Model? = LocalPlayer and LocalPlayer.Character
        if not character or not character.Parent then
            return nil, nil, nil
        end
        local humanoid: Humanoid? = character:FindFirstChildOfClass("Humanoid")
        local root: BasePart? = (humanoid and humanoid.RootPart)
            or (character:FindFirstChild("HumanoidRootPart") :: BasePart?)
            or (character:FindFirstChild("Torso") :: BasePart?)
        if not humanoid or humanoid.Health <= 0 or not root then
            return character, humanoid, nil
        end
        return character, humanoid, root
    end

    local function rememberCollision(character: Model): ()
        for _, part: Instance in ipairs(character:GetChildren()) do
            if part:IsA("BasePart") and savedCollision[part] == nil then
                savedCollision[part] = (part :: BasePart).CanCollide
            end
        end
    end

    local function restoreCollision(): ()
        for part: any, canCollide: any in pairs(savedCollision) do
            if part and part.Parent then
                pcall(function(): ()
                    part.CanCollide = canCollide
                end)
            end
        end
        savedCollision = setmetatable({}, {__mode = "k"})
    end

    local function stopWalkFling(): ()
        local wasEnabled: boolean = enabled
        enabled = false
        session += 1
        restoreCollision()
        local _character: Model?, _humanoid: Humanoid?, root: BasePart? = getCharacterRoot()
        if wasEnabled and root and root.Parent then
            pcall(function(): ()
                root.Velocity = Vector3.zero
                root.AssemblyLinearVelocity = Vector3.zero
                root.AssemblyAngularVelocity = Vector3.zero
            end)
        end
        if activity and type(activity.set) == "function" then
            activity.set("walkfling", false)
        end
        if walkFlingFeature then
            walkFlingFeature:SetStatus(nil)
        end
    end

    -- Infinite Yield's walkfling, which the user approved: spike the root
    -- velocity inside the RenderStepped window, put it back before the physics
    -- step and micro-oscillate so replication stays awake. The only additions
    -- are the session token (so an old loop cannot outlive a new toggle) and the
    -- collision restore on the way out.
    local function startWalkFling(): ()
        if enabled then
            return
        end
        enabled = true
        session += 1
        local mySession: number = session
        local movel: number = 0.1
        if activity and type(activity.set) == "function" then
            activity.set("walkfling", true)
        end
        if walkFlingFeature then
            walkFlingFeature:SetStatus("ON")
        end

        task.spawn(function(): ()
            while enabled and session == mySession and Module.Initialized do
                RunService.Heartbeat:Wait()
                if not enabled or session ~= mySession then
                    break
                end
                local character: Model?, humanoid: Humanoid?, root: BasePart? = getCharacterRoot()
                if character and root and root.Parent then
                    local savedVelocity: Vector3 = root.AssemblyLinearVelocity
                    local savedAngular: Vector3 = root.AssemblyAngularVelocity or Vector3.zero
                    -- Modern technique: also use angular velocity and ensure Humanoid is in Physics for replication
                    if humanoid and humanoid.ChangeState then
                        pcall(function() humanoid:ChangeState(Enum.HumanoidStateType.Physics) end)
                    end
                    pcall(function(): ()
                        rememberCollision(character)
                        for _, part: Instance in ipairs(character:GetChildren()) do
                            if part:IsA("BasePart") then
                                (part :: BasePart).CanCollide = false
                            end
                        end
                        local spike: Vector3 = savedVelocity * POWER + Vector3.new(0, POWER, 0)
                        local spin: Vector3 = Vector3.new(SPIN, SPIN, SPIN)
                        root.Velocity = spike
                        root.AssemblyLinearVelocity = spike
                        root.AssemblyAngularVelocity = spin
                        -- Also set RotVelocity for older physics
                        pcall(function() (root :: any).RotVelocity = spin end)
                    end)
                    RunService.RenderStepped:Wait()
                    -- A toggle-off that lands in the middle of an iteration must
                    -- not re-apply the spike after stop() already parked the body.
                    local stillRunning: boolean = enabled and session == mySession
                    if stillRunning and character.Parent and root.Parent then
                        pcall(function(): ()
                            root.Velocity = savedVelocity
                            root.AssemblyLinearVelocity = savedVelocity
                            root.AssemblyAngularVelocity = savedAngular
                            pcall(function() (root :: any).RotVelocity = savedAngular end)
                        end)
                    end
                    RunService.Stepped:Wait()
                    if stillRunning and enabled and session == mySession
                        and character.Parent and root.Parent then
                        pcall(function(): ()
                            local oscillated: Vector3 = savedVelocity + Vector3.new(0, movel, 0)
                            root.Velocity = oscillated
                            root.AssemblyLinearVelocity = oscillated
                            local spinOsc: Vector3 = Vector3.new(0, SPIN * 0.1, 0)
                            root.AssemblyAngularVelocity = spinOsc
                            movel = -movel
                        end)
                    end
                end
            end
            if session == mySession then
                restoreCollision()
            end
        end)
    end

    -- v190: MM2-only notice. Anti Fling shields every colliding part against a
    -- body moving at fling speeds -- which is exactly the contact Walk Fling
    -- needs -- so enabling both at once made the fling look broken. The player
    -- asked for the warning instead of silent failure: refusing here makes the
    -- card revert itself and report "Could not toggle Walk Fling: ...".
    local function mm2Game(): boolean
        local shared: any = host.state
        if shared and shared.activeGameName == "MM2" then
            return true
        end
        local ok: boolean, placeId: any = pcall(function(): any return game.PlaceId end)
        return ok and placeId == 142023291
    end

    local function antiFlingOn(): boolean
        local shared: any = host.state
        return shared ~= nil and shared.antiFlingActive == true
    end

    local function toggleWalkFling(state: boolean): ()
        if state == true then
            if mm2Game() and antiFlingOn() then
                error("Anti Fling is ON. Turn it off first to use Walk Fling.", 0)
            end
            startWalkFling()
            return
        end
        stopWalkFling()
    end

    walkFlingFeature = createUniversalFeature(
        "Walk Fling",
        "Fling on contact while walking (Infinite Yield walkfling)",
        8,
        toggleWalkFling,
        {categoryName = "Exploits"}
    )
    if type(addInformationOption) == "function" then
        addInformationOption(
            walkFlingFeature,
            "Collisions are restored and the body parked when the card is turned off."
        )
    end
    addFeatureTooltip(
        walkFlingFeature,
        "While ON, every physics frame the local character is noclipped and given a "
            .. "huge velocity for one frame, which launches whoever you walk into. "
            .. "Collisions are restored when it stops. It never starts on injection. "
            .. "In MM2 it refuses to start while Anti Fling is ON."
    )

    activeCleanup = function(): ()
        stopWalkFling()
    end
    Module.Initialized = true
    return walkFlingFeature
end

function Module.destroy(): ()
    if activeCleanup then
        pcall(activeCleanup)
    end
    activeCleanup = nil
    Module.Initialized = false
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Blatant/WalkFling.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Combat/AimAssist.lua
BYTES: 24909
SHA-256: c6e9ba510d87149896 7d09b12a80ca873c9c7f9e51ea8602876fba7eb677051f
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
export type Runtime = {
    framework: any,
    entity: any,
    host: any,
    libraries: {[string]: any},
    services: any,
```

```
}

local Module = {
    Name = "AimAssist",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local activeCard: any = nil

function Module.init(context: Runtime): any
    local framework: any = context.framework
    local host: any = context.host
    local services: any = context.services
    local targeting: any = context.libraries and context.libraries.targeting
    local entity: any = context.entity
    local userInput: UserInputService = host.UserInputService
    local currentWorkspace: Workspace = host.workspace or workspace
    local localPlayer: Player? = host.LocalPlayer
    local runtime: any = {
        target = nil,
        lockedTarget = nil,
        lastFire = -math.huge,
        fireMode = "Hold",
        clickPressed = false,
    }
    local card: any

    local function camera(): Camera?
        return currentWorkspace.CurrentCamera
    end

    local function getHeldTool(): Tool?
        local player: Player? = localPlayer or host.LocalPlayer
        if not player then return nil end
        local character: Model? = player.Character
        if not character then return nil end
        for _, child: Instance in ipairs(character:GetChildren()) do
            if child:IsA("Tool") then
                return child :: Tool
            end
        end
        return nil
    end

    -- Screen position of the real cursor (includes the top-bar inset), used
    -- both to aim the virtual mouse and to place virtual clicks.
    local function getMouseScreenPosition(cameraObject: Camera?): Vector2
        local mouseOk: boolean, mousePos: any = pcall(function(): any
            return userInput:GetMouseLocation()
        end)
        if mouseOk and typeof(mousePos) == "Vector2" then
            return mousePos
        end
        local vp: Vector2 = cameraObject and cameraObject.ViewportSize
            or Vector2.new(800, 600)
        local insetY: number = 36
        pcall(function(): ()
            local guiService: any = game:GetService("GuiService")
            local inset: Vector2 = guiService:GetGuiInset()
            if typeof(inset) == "Vector2" then insetY = inset.Y end
        end)
        return Vector2.new(vp.X / 2, vp.Y / 2 + insetY)
    end

    local function tryFireOnce(fireMode: string?, clickPressed: boolean?): boolean
        -- Only a tool that is ALREADY held gets Activate(). Equipping from the
        -- backpack mid-fight was removed on purpose: EquipTool can trigger the
        -- game's equip/ADS logic, which switches or locks the camera exactly at
        -- the point where the player clicked.
        local tool: Tool? = getHeldTool()
        if tool then
            if tool.Enabled == false then return false end
            -- Hold mode: the player's real click already activates the held tool
            -- (Roblox fires Tool.Activated on click); an extra Activate()
            -- double-fires semi-auto guns.
            if fireMode == "Hold" and clickPressed == true then return true end
            local ok: boolean = pcall(function(): () (tool :: Tool):Activate() end)
            if ok then return true end
        end
        -- Rivals / FPS without Tool (ViewModel + RemoteEvent): fallback to a
        -- virtual click. It MUST be sent at the real cursor position: sending
        -- (0, 0) shoots at the top-left screen corner and, with a free cursor,
        -- yanks the mouse (and any camera driven by absolute mouse position)
        -- to that corner.
        local mouse: Vector2 = getMouseScreenPosition(camera())
        local vimOk: boolean = pcall(function(): ()
            local vim: any = game:GetService("VirtualInputManager")
            if vim and type(vim.SendMouseButtonEvent) == "function" then
                -- Press and release quickly; the CPS loop controls rate
                vim:SendMouseButtonEvent(mouse.X, mouse.Y, 0, true, game, 0)
                vim:SendMouseButtonEvent(mouse.X, mouse.Y, 0, false, game, 0)
            else
                error("no vim")
            end
        end)
        if vimOk then return true end
        local env: any = getfenv()
        local click: any = env.mouse1click or env.mouse1press
        if type(click) == "function" then
            local ok2: boolean = pcall(function(): () click() end)
            if ok2 then return true end
            -- mouse1press needs release
            local rel: any = env.mouse1release
            if type(rel) == "function" then
                pcall(function(): () click(); task.wait(0.02); rel() end)
                return true
            end
        end
        return false
    end

    local function getCPS(): number
        local opt: any = nil
        if card and card.Options then
            opt = card.Options["Clicks Per Second"] or card.Options["Fire rate"] or card.Options["CPS"]
        end
        if opt and opt.Value ~= nil then
            return math.max(1, tonumber(opt.Value) or 12)
        end
        return 12
    end

    local function shouldAutoFire(): boolean
        if not card or not card.Options or not card.Options["Auto Fire"] then return false end
        if not card.Options["Auto Fire"].Value then return false end
        local rate: number = getCPS()
        local now: number = os.clock()
        if now - runtime.lastFire < 1 / rate then return false end
        local fireMode: string = "Hold"
        if card.Options["Auto Fire mode"] then
            fireMode = tostring(card.Options["Auto Fire mode"].Value)
        end
        local okHeld: boolean, isPressed: any = pcall(function(): any
            return userInput:IsMouseButtonPressed(Enum.UserInputType.MouseButton1)
        end)
        local clickPressed: boolean = okHeld and isPressed == true
        -- Remembered for tryFireOnce: in Hold mode a real click already fired
        -- the held tool, so the module must not Activate() it a second time.
        runtime.fireMode = fireMode
        runtime.clickPressed = clickPressed
        if fireMode == "Always" then
            runtime.lastFire = now
            return true
        end
        if clickPressed then
            runtime.lastFire = now
            return true
        end
        return false
    end

    local function moveCamera(target: any, deltaTime: number): boolean
        local cameraObject: Camera? = camera()
        if not cameraObject or not target then
            return false
        end
        local mode: string = card.Options["Mode"].Value
        local cameraModeRaw: string = "Auto"
        if card.Options["Camera"] then
            cameraModeRaw = tostring(card.Options["Camera"].Value)
        end
        local cameraMode: string = cameraModeRaw
        if cameraModeRaw == "CFrame" then
            cameraMode = "Camera"
        end
        local current: CFrame = cameraObject.CFrame
        local offset: Vector3 = target.position - current.Position
        if offset.Magnitude <= 0.001 then
            return false
        end

        local alpha: number = 1
        if mode ~= "Aimbot" then
            local dt = math.max(deltaTime, 0)
            local speed = math.max(0, card.Options.Speed.Value)
            local strength = math.max(0, card.Options.Strength.Value) / 100
            if speed == 0 or strength == 0 or dt == 0 then return false end
            local angle = math.acos(math.clamp(current.LookVector:Dot(offset.Unit), -1, 1))
            if angle < math.rad(0.5) then return false end
            alpha = math.min(1 - math.exp(-strength * 10 * dt),
                angle > 1e-6 and math.rad(speed) * dt / angle or 1)
        end

        local function isMouseLocked(): boolean
            local okBeh: boolean, beh: any = pcall(function(): any
                return (userInput :: any).MouseBehavior
            end)
            if okBeh and beh then
                local okEnum: boolean, lockCenter: any = pcall(function(): any
                    return (Enum :: any).MouseBehavior.LockCenter
                end)
                local okCurr: boolean, lockCurr: any = pcall(function(): any
                    return (Enum :: any).MouseBehavior.LockCurrentPosition
                end)
                if (okEnum and beh == lockCenter) or (okCurr and beh == lockCurr) then
                    return true
                end
            end
            local lp: Player? = localPlayer or host.LocalPlayer
            local char: Model? = lp and lp.Character
            local head: BasePart? = char and char:FindFirstChild("Head") :: BasePart?
            if head then
                local dist: number = (cameraObject.CFrame.Position - head.Position).Magnitude
                if dist < 2.5 then
                    return true
                end
            end
            return false
        end

        local function tryMouse(): boolean
            if cameraMode == "Camera" then return false end
            local env: any = getfenv()
            local moverel: any = env.mousemoverel or env.mousemove or (host and (host.mousemoverel or host.mousemove))
            if type(moverel) ~= "function" then return false end
            -- Use ScreenPoint to match GetMouseLocation (both include top-bar inset 36px)
            local ok: boolean, screenPos: any, onScreen: any = pcall(function(): any
                return cameraObject:WorldToScreenPoint(target.position)
            end)
            if not ok or type(screenPos) ~= "Vector3" then return false end
            if onScreen == false and mode ~= "Aimbot" then return false end
            -- Always use GetMouseLocation (with inset) for delta; fallback to Viewport/2+inset
            local m: Vector2 = getMouseScreenPosition(cameraObject)
            local delta: Vector2 = Vector2.new(screenPos.X - m.X, screenPos.Y - m.Y)
            if mode ~= "Aimbot" then
                delta = delta * alpha
            end
            if delta.Magnitude < 0.3 then return false end
            local maxDelta: number = 500
            if math.abs(delta.X) > maxDelta or math.abs(delta.Y) > maxDelta then
                delta = delta.Unit * maxDelta
            end
            local moved: boolean = false
            local okMove: boolean = pcall(function(): ()
                moverel(delta.X, delta.Y)
                moved = true
            end)
            return okMove and moved
        end

        local function tryCamera(): boolean
            if cameraMode == "Mouse" then return false end
            local currentLook: Vector3 = current.LookVector
            local blended: Vector3 = offset.Unit
            if alpha < 1 then
                local dot = math.clamp(currentLook:Dot(offset.Unit), -1, 1)
                local tangent = offset.Unit - currentLook * dot
                if tangent.Magnitude < 1e-6 then tangent = current.RightVector end
                local turn = math.acos(dot) * alpha
                blended = currentLook * math.cos(turn) + tangent.Unit * math.sin(turn)
            end
            if blended.Magnitude <= 0.001 then
                return false
            end
            local newCFrame: CFrame = CFrame.lookAt(current.Position, current.Position + blended.Unit)
            -- Write directly from this frame callback. Card Render tasks run
            -- on RenderStepped, i.e. AFTER the game's own camera update
            -- (RenderPriority.Camera), so this write wins for the frame.
            -- Deliberately NO BindToRenderStep here: the old one-shot binding
            -- captured a stale newCFrame and tried to unbind itself from inside
            -- the render step; when that unbind failed (the pcall swallowed it),
            -- the binding re-wrote the same stale CFrame every frame and the
            -- camera froze on the last aim point — the spot where the player
            -- clicked. Writing only from the per-frame callback means that as
            -- soon as there is no target, nothing writes the camera anymore and
            -- the game owns the view again.
            local ok: boolean = pcall(function(): ()
                cameraObject.CFrame = newCFrame
            end)
            return ok
        end

        -- Hybrid fix for Rivals: with Tool/weapon, the gun's ray often uses mouse Hit, not just camera.
        -- Pure Camera leaves Mouse Hit at old side; pure Mouse has 1-frame camera lag.
        -- For natural + accurate, do BOTH when locked (FPS). For 3rd person, Mouse is enough.
        if cameraMode == "Auto" then
            local locked: boolean = isMouseLocked()
            if locked then
                -- FPS shift-lock / 1st person: try both to keep camera and Mouse Hit synced.
                -- Mouse first (game's native delta), then immediate Camera to remove 1-frame lag for hitscan.
                local didMouse: boolean = tryMouse()
                local didCamera: boolean = tryCamera()
                -- If we did Camera, we synced immediately; if only Mouse, camera will catch next frame via game's handler.
                return didMouse or didCamera
            else
                -- 3rd person: Mouse is natural and weapon (if any) follows camera via game's orbit, no need for double.
                if tryMouse() then return true end
                if tryCamera() then return true end
                return false
            end
        elseif cameraMode == "Camera" then
            return tryCamera()
        elseif cameraMode == "Mouse" then
            return tryMouse()
        end
        return tryCamera()
    end

    local function another AimAdapterIsActive(): boolean
        local activity: any = services and services.activity
        local free = services and services.freeCam
        local spectate = services and services.spectate
        if free and free.IsActive() or spectate and spectate.IsActive() then return true end
        return activity ~= nil
            and type(activity.isActive) == "function"
            and activity.isActive("mvsdSilentAim") == true
    end

    local function selectTarget(): any
        if another AimAdapterIsActive()
            or not targeting
            or type(targeting.List) ~= "function" then
            return nil
        end
        local options: any = card.Options
        local mode: string = options["Mode"].Value
        local candidates: {any} = targeting:List(entity, {
            FOV = mode == "Aim Assist" and options["FOV"].Value or nil,
            MaxDistance = options["Max distance"].Value,
            TargetPart = options["Target part"].Value,
            Prediction = options["Prediction"].Value,
            TeamCheck = options["Team check"].Value,
            VisibilityCheck = options["Visibility check"].Value,
            Priority = options["Priority"].Value,
            IncludeNPCs = options["Target NPCs"].Value,
            AllowOffscreen = mode == "Aimbot",
        })
        if mode ~= "Aimbot" then
            runtime.lockedTarget = nil
            return candidates[1]
        end
        if options["Sticky target"].Value and runtime.lockedTarget then
            for _, candidate: any in ipairs(candidates) do
                if candidate.entity == runtime.lockedTarget.entity then
                    runtime.lockedTarget = candidate
                    return candidate
                end
            end
        end
        runtime.lockedTarget = candidates[1]
        return runtime.lockedTarget
    end

    pcall(function(): ()
        local cfg: any = host.configData
        if cfg and type(cfg.values) == "table" then
            local key: string = "Universal.AimAssist.Camera"
            if cfg.values[key] == "CFrame" then
                cfg.values[key] = "Camera"
            end
            local oldKey: string = "Universal.AimAssist.Fire rate"
            local newKey: string = "Universal.AimAssist.Clicks Per Second"
            if cfg.values[oldKey] ~= nil and cfg.values[newKey] == nil then
                cfg.values[newKey] = cfg.values[oldKey]
            end
        end
    end)

    card = framework.Categories.Exploits:CreateModule({
        Name = "Aimbot",
        Category = "Exploits",
        ConfigKey = "Universal.AimAssist",
        Order = 4,
        Tooltip = "Eases the local camera toward a valid target; no raycast or "
            .. "remote hooks.",
        Function = function(enabled: boolean): ()
            runtime.target = nil
            runtime.lockedTarget = nil
            runtime.lastFire = -math.huge
            if not enabled then
                card:SetStatus(nil)
                local rs: any = (host :: any).RunService
                if rs and type(rs.UnbindFromRenderStep) == "function" then
                    pcall(function(): () rs:UnbindFromRenderStep("WurstAimAssistCam") end)
                end
                return
            end
            card:SetStatus("searching")
            card:Render(function(deltaTime: number): ()
                if another AimAdapterIsActive()
                    or services.menu.isVisible()
                    or services.menu.isCapturingInput() then
                    runtime.target = nil
                    runtime.lockedTarget = nil
                    card:SetStatus("paused")
                    return
                end
                if userInput:GetFocusedTextBox() then
                    runtime.target = nil
                    runtime.lockedTarget = nil
                    card:SetStatus("paused")
                    return
                end
                local selected: any = selectTarget()
                runtime.target = selected
                if not selected then
                    card:SetStatus("searching")
                    return
                end
                local moved: boolean = moveCamera(selected, deltaTime)
                if shouldAutoFire() then
                    pcall(function(): () tryFireOnce(runtime.fireMode, runtime.clickPressed) end)
                end
                local targetPlayer: Player? = selected.player
                local displayName: string = targetPlayer
                    and tostring(targetPlayer.DisplayName or targetPlayer.Name)
                    or tostring(selected.entity.Name or selected.character.Name)
                local distance: number = math.round(selected.distance)
                card:SetStatus(
                    displayName .. " · " .. tostring(distance) .. " studs"
                        .. (moved and " · locked" or " · ready")
                )
            end):Clean(function(): ()
                runtime.target = nil
                card:SetStatus(nil)
                local rs: any = (host :: any).RunService
                if rs and type(rs.UnbindFromRenderStep) == "function" then
                    pcall(function(): () rs:UnbindFromRenderStep("WurstAimAssistCam") end)
                end
            end)
        end,
    })

    card:CreateDropdown({
        Name = "Mode",
        List = {"Aim Assist", "Aimbot"},
        Index = 1,
        Function = function(): ()
            runtime.lockedTarget = nil
        end,
        Tooltip = "Aim Assist = ayuda suave dentro de FOV usando Speed/Strength (natural). Aimbot = snap directo con Sticky opcional.",
    })
    card:CreateDropdown({
        Name = "Camera",
        List = {"Auto", "Camera", "Mouse"},
        Index = 1,
        Tooltip = "Auto: 3rd=Mouse (natural) 1st/FPS=Mouse+Camera híbrido para que el arma siga (Mouse Hit + CFrame instant). Camera: solo CFrame (1st person). Mouse: solo mousemoverel.",
    })
    card:CreateSlider({
        Name = "FOV",
        Show = {Option = "Mode", Values = {"Aim Assist"}},
        Min = 15,
        Max = 500,
        Step = 5,
        Default = 180,
        Tooltip = "FOV en píxeles: radio alrededor del cursor. 15 pequeño — 500 pantalla completa. Default 180.",
    })
    card:CreateSlider({Name="Speed",Min=0,Max=360,Default=180,Step=5,
        Show={Option="Mode",Values={"Aim Assist"}},
        Tooltip="Speed °/s: rotación máxima. 0 sin ayuda, 180 default, 360 instantáneo."})
    card:CreateSlider({Name="Strength",Min=0,Max=100,Default=75,Step=5,
        Show={Option="Mode",Values={"Aim Assist"}},
        Tooltip="Strength %: fuerza de arrastre. 0 nada, 50 suave, 75 default, 100 máxima."})
    card:CreateNote("FOV: 15-500 px — 180 default • Speed: 0-360 °/s — 180 default • Strength: 0-100% — 75 default.")
    card:CreateDropdown({
        Name = "Target part",
        List = {"Closest", "Head", "Torso"},
        Index = 1,
        Tooltip = "Body part used as the camera target.",
    })
    card:CreateDropdown({
        Name = "Priority",
        List = {"Cursor", "Distance"},
        Index = 1,
        Tooltip = "Choose the target nearest the cursor or nearest in the world.",
    })
    card:CreateSlider({
        Name = "Prediction",
        Min = 0,
        Max = 0.5,
        Step = 0.01,
        Default = 0.06,
        Tooltip = "Prediction s: anticipa movimiento. 0 sin predicción, 0.06 default (60 ms).",
    })
    card:CreateSlider({
        Name = "Max distance",
        Min = 25,
        Max = 2000,
        Step = 5,
        Default = 600,
        Tooltip = "Max distance studs: ignora targets más lejos. 600 default.",
    })
    card:CreateNote("Prediction: 0-0.5 s — 0.06 s default. Max distance 600 studs default.")
    card:CreateToggle({
        Name = "Visibility check",
        Default = true,
        Tooltip = "Skip targets hidden by map geometry.",
    })
    card:CreateToggle({
        Name = "Team check",
        Default = true,
        Tooltip = "Skip teammates and Friend List entries.",
    })
    card:CreateToggle({
        Name = "Target NPCs",
        Default = false,
        Tooltip = "Include non-player humanoid models in both modes.",
    })
    card:CreateToggle({
        Name = "Sticky target",
        Show = {Option = "Mode", Values = {"Aimbot"}},
        Default = true,
        Function = function(value: boolean): ()
            if not value then
                runtime.lockedTarget = nil
            end
        end,
        Tooltip = "Hold the current Aimbot target. Turn off to release it every frame.",
    })
    card:CreateToggle({
        Name = "Auto Fire",
        Default = false,
        Tooltip = "Autoclick genérico: si no hay Tool hace click virtual. No toca GUI.",
    })
    card:CreateDropdown({
        Name = "Auto Fire mode",
        List = {"Hold", "Always"},
        Index = 1,
        Show = {Option = "Auto Fire"},
        Tooltip = "Hold = solo mientras mantienes click izquierdo (para semi-auto). Always = dispara solo al tener target.",
    })
    card:CreateSlider({
        Name = "Clicks Per Second",
        Min = 1,
        Max = 20,
        Step = 1,
        Default = 12,
        Show = {Option = "Auto Fire"},
        Tooltip = "Clicks por segundo manteniendo. 12 default = pistola semi-auto. Sube para automático.",
    })
    card:CreateNote(
        "Natural: Auto 3rd=Mouse, 1st=Mouse+Camera híbrido (sincroniza Mouse Hit + CFrame) para que el arma no se quede atrás."
    )

    pcall(function()
        local feature: any = (card :: any).Feature or (card :: any)._feature or card
        local optionsFrame: any = (feature :: any).options or (card :: any).options
        if not optionsFrame or type(optionsFrame.FindFirstChild) ~= "function" then return end
        local function patchOption(optionName: string, minText: string, maxText: string, suffix: string)
            local row: any = optionsFrame:FindFirstChild("Option_" .. optionName:gsub("%W", ""))
            if not row then return end
            local label: any = row:FindFirstChild("OptionLabel")
            if label and suffix then pcall(function() label.Text = label.Text .. " " .. suffix end) end
            local slider: any = row:FindFirstChild("Slider")
            if slider then
                local minL: any = slider:FindFirstChild("MinLabel")
                local maxL: any = slider:FindFirstChild("MaxLabel")
                if minL then pcall(function() minL.Text = minText end) end
                if maxL then pcall(function() maxL.Text = maxText end) end
            end
        end
        patchOption("FOV", "15 px", "500 px", "(pixeles)")
        patchOption("Speed", "0°/s", "360°/s", "(°/s)")
        patchOption("Strength", "0%", "100%", "(%)")
        patchOption("Prediction", "0 s", "0.5 s", "(s)")
        patchOption("Maxdistance", "25 studs", "2000 studs", "(studs)")
        patchOption("ClicksPerSecond", "1", "20", "(cps)")
    end)

    activeCard = card
    Module.Initialized = true
    return card
end

function Module.destroy(): ()
    if activeCard and activeCard.Enabled then
        activeCard:Toggle(false)
    end
    activeCard = nil
    Module.Initialized = false
end

return Module

-------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Combat/AimAssist.lua
-------------------------------------------------------------------------------

-------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Combat/KillAura.lua
BYTES: 24179
SHA-256: d38fae83d998676f5785bb7c1c3056fe7ee7eb762309ac5363db18890ed1ceac
INICIO DEL ARCHIVO
-------------------------------------------------------------------------------
export type Runtime = {
    framework: any,
    entity: any,
    host: any,
}

local Module = {
    Name = "KillAura",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local activeCleanup: (() -> ())? = nil

-- Ignore list: names, display names or user ids, separated by commas. It is
-- parsed outside the per-frame loop and applied to the entity list before any
-- rotation or swing, so an ignored player is never picked.
local function compileIgnoreList(text: string?): {[string]: boolean}
    local ignored: {[string]: boolean} = {}
    for name: string in string.gmatch(text or "", "[^,;%s]+") do
        ignored[string.lower(name)] = true
    end
    return ignored
end

local function isIgnoredEntity(entity: any, ignored: {[string]: boolean}): boolean
    local player: any = entity and entity.Player
    if not player then
        return false
    end
    return ignored[string.lower(player.Name or "")] == true
        or ignored[string.lower(player.DisplayName or "")] == true
        or ignored[tostring(player.UserId)] == true
end

local function collectTools(localPlayer: Player): {Tool}
    local tools: {Tool} = {}
    local character: Model? = localPlayer.Character
    if character then
        for _, child: Instance in ipairs(character:GetChildren()) do
            if child:IsA("Tool") then
                table.insert(tools, child :: Tool)
            end
        end
    end
    local backpack: Instance? = localPlayer:FindFirstChild("Backpack")
    if backpack then
        for _, child: Instance in ipairs(backpack:GetChildren()) do
            if child:IsA("Tool") then
                table.insert(tools, child :: Tool)
            end
        end
    end
    return tools
end

local function toolHandle(tool: Tool?): BasePart?
    if not tool then
        return nil
    end
    local resolved: Tool = tool :: Tool
    local handle: Instance? = resolved:FindFirstChild("Handle")
    if handle and handle:IsA("BasePart") then
        return handle :: BasePart
    end

    for _, descendant: Instance in ipairs(resolved:GetDescendants()) do
        if descendant:IsA("BasePart") then
            return descendant :: BasePart
        end
    end
    return nil
end

local function activateTool(tool: Tool?): boolean
    if not tool or tool.Enabled == false then return false end
```

```
    return pcall(function(): ()
        tool:Activate()
    end)
end

local function fireContact(handle: BasePart?, target: BasePart): ()
    if not handle then
        return
    end
    local environment: any = getfenv()
    local fire: any = environment.firetouchinterest
    if type(fire) ~= "function" then
        return
    end

    pcall(fire, handle, target, 0)
    pcall(fire, handle, target, 1)
end

function Module.init(context: Runtime): any
    local host: any = context.host
    local framework: any = context.framework
    local entityLibrary: any = context.entity
    local localPlayer: Player = host.LocalPlayer
    local userInput: UserInputService = host.UserInputService
    local currentWorkspace: Workspace = host.workspace or workspace
    -- Advanced team check (always on): the entity library only knows team
    -- providers, so role-based verdicts (MM2 sides, for example) are applied
    -- as a post filter here.
    local teamCheck: any = host.state and host.state.teamCheck

    local nextSwingAt: number = 0
    local nextStatusAt: number = 0
    local highlights: {[any]: Highlight} = {}

    local function clearHighlights(keep: {[any]: boolean}?): ()
        for key: any, highlight: Highlight in pairs(highlights) do
            if not keep or not keep[key] then
                highlight:Destroy()
                highlights[key] = nil
            end
        end
    end

    local function markTarget(entity: any, options: any): ()
        if not options["Highlight targets"].Value then
            return
        end
        local key: any = entity.Player or entity.Character
        local existing: Highlight? = highlights[key]
        if not existing or not existing.Parent then
            local created: Highlight = Instance.new("Highlight")
            created.Name = "Wurst_KillAuraTarget"
            created.FillTransparency = 0.75
            created.Parent = entity.Character
            highlights[key] = created
            existing = created
        end
        local highlight: Highlight = existing :: Highlight
        highlight.Adornee = entity.Character
        highlight.DepthMode = Enum.HighlightDepthMode.AlwaysOnTop
        local colour: Color3 = options["Highlight colour"].Value
        highlight.FillColor = colour
        highlight.OutlineColor = colour
    end

    local aura: any = nil
    local weapons: any = context.weapons

    local function updateTargetStatus(count: number): ()
        local now: number = os.clock()
        if now < nextStatusAt then
            return
        end
        nextStatusAt = now + 0.25
        aura:SetStatus(tostring(count) .. " in range")
    end

    local learnedWeapon: string? = nil
    local learnConnections: {RBXScriptConnection} = {}

    local function watchTool(tool: Tool): ()
        table.insert(
            learnConnections,
            tool.Activated:Connect(function(): ()
                learnedWeapon = tool.Name
            end)
        )
    end

    local function clearToolWatches(): ()
        for _, connection: RBXScriptConnection in ipairs(learnConnections) do
            connection:Disconnect()
        end
        table.clear(learnConnections)
    end

    local function watchAllTools(): ()
        clearToolWatches()
        for _, tool: Tool in ipairs(collectTools(localPlayer)) do
            watchTool(tool)
        end
    end

    local function matchesWanted(tool: Tool, wanted: string): boolean
        return string.find(string.lower(tool.Name), string.lower(wanted), 1, true) ~= nil
    end

    local function equippedTool(): (Tool?, BasePart?)
        local character: Model? = localPlayer.Character
        if not character then
            return nil, nil
        end
        local wanted: string = tostring(aura.Options["Weapon name"].Value or "")
        local held: {Tool} = {}
        for _, child: Instance in ipairs(character:GetChildren()) do
            if child:IsA("Tool") then
                table.insert(held, child :: Tool)
            end
        end

        if wanted ~= "" then
            for _, tool: Tool in ipairs(held) do
                if matchesWanted(tool, wanted) then
                    return tool, toolHandle(tool)
                end
            end
        end
        if learnedWeapon then
            for _, tool: Tool in ipairs(held) do
                if tool.Name == learnedWeapon then
                    return tool, toolHandle(tool)
                end
            end
        end
        local first: Tool? = held[1]
        return first, toolHandle(first)
    end

    local function equipFromBackpack(): ()
        local backpack: Instance? = localPlayer:FindFirstChild("Backpack")
        local humanoid: Humanoid? = localPlayer.Character
            and localPlayer.Character:FindFirstChildOfClass("Humanoid") :: Humanoid?
        if not backpack or not humanoid then
            return
        end
        local wanted: string = tostring(aura.Options["Weapon name"].Value or "")
        local chosen: Tool? = nil
        for _, child: Instance in ipairs(backpack:GetChildren()) do
            if not child:IsA("Tool") then
                continue
            end
            local tool: Tool = child :: Tool
            if wanted ~= "" and matchesWanted(tool, wanted) then
                chosen = tool
                break
            end
            if learnedWeapon and tool.Name == learnedWeapon then
                chosen = tool
                break
            end
            if not chosen then
                chosen = tool
            end
        end
        if chosen then
            pcall(function(): ()
                humanoid:EquipTool(chosen :: Tool)
            end)
        end
    end

    local store: any = host.configData
    if store and store.values then
        local reachKey: string = "Universal.KillAura.Reach"
        local stored: any = store.values["Universal.KillAura.Swingrange"]
        if store.values[reachKey] == nil and type(stored) == "number" then
            store.values[reachKey] = stored
            if type(host.queueConfigSave) == "function" then
                host.queueConfigSave()
            end
        end
    end

    aura = framework.Categories.Exploits:CreateModule({
        Name = "KillAura", -- stable framework identity for game adapters
        DisplayName = host.GAME_CHECK and host.GAME_CHECK.MM2Active and "Knife Stab" or "KillAura",
        Category = "Exploits",
        ConfigKey = "Universal.KillAura",
        Order = 1,
        Tooltip = "Swings your weapon at everyone in reach, with its own "
            .. "range, rate, field of view and rotation.",
        Function = function(enabled: boolean): ()
            if aura.CombatAdapter then
                clearHighlights()
                aura.CombatAdapter(enabled)
                return
            end
            nextSwingAt = 0
            nextStatusAt = 0
            if not enabled then
                aura:SetStatus(nil)
                clearHighlights()
                return
            end
            aura:SetStatus("0 in range")

            aura:Clean(function(): ()
                clearHighlights()
            end)

            watchAllTools()
            aura:Event(localPlayer.CharacterAdded, function(): ()
                task.defer(function()
                    if aura.Enabled then watchAllTools() end
                end)
            end)
            local backpack: Instance? = localPlayer:FindFirstChild("Backpack")
            if backpack then
                aura:Event(backpack.ChildAdded, function(child: Instance): ()
                    if child:IsA("Tool") then
                        watchTool(child :: Tool)
                    end
                end)
            end
            aura:Clean(clearToolWatches)

            aura:Loop(function(): ()
                local options: any = aura.Options
                -- Fix Hold Mouse showing mobile controls on PC: suppress TouchGui when Hold is active
                local suppress: any = host.suppressTouchControls
                if type(suppress) == "function" and options["Activation"].Value == "Hold key" then
                    local hold: any = options["Hold key"].Value
                    -- If hold is a mouse button, the game may enable TouchGui on desktop; hide it
                    local isMouseHold: boolean = false
                    pcall(function()
                        isMouseHold = (hold == Enum.KeyCode.Unknown and false)
                            or (typeof(hold) == "EnumItem" and string.find(tostring(hold), "MouseButton") ~= nil)
                            or (typeof(hold) == "EnumItem" and hold.EnumType == Enum.UserInputType)
                    end)
                    if isMouseHold or userInput:IsKeyDown(hold) then
                        pcall(suppress)
                    end
                end
                if options["Activation"].Value == "Hold key"
                    and not userInput:IsKeyDown(options["Hold key"].Value) then
                    updateTargetStatus(0)
                    clearHighlights()
                    return
                end
                if not entityLibrary:IsAlive() then
                    updateTargetStatus(0)
                    clearHighlights()
                    return
                end

                local tool: Tool?, handle: BasePart? = equippedTool()
                if not tool then
                    if options["Auto equip"].Value then
                        equipFromBackpack()
                    end
                    if options["Require tool"].Value then

                        updateTargetStatus(0)
                        clearHighlights()
                        return
                    end
                end

                entityLibrary:Refresh()
                local targets: {any} = entityLibrary:InRange({
                    Range = options["Reach"].Value,
                    -- Limit after all filters: ignored/protected teammates must
                    -- not consume slots ahead of valid enemies.
                    IgnoreTeam = not options["Team check"].Value,
                    IgnoreWalls = not options["Wall check"].Value,
                    Angle = options["Field of view"].Value,
                    IncludeNPCs = options["Target NPCs"].Value,
                    Sort = options["Priority"].Value,
                })
                if options["Team check"].Value and teamCheck then
                    local kept: {any} = {}
                    for _, entity: any in ipairs(targets) do
                        local teammate: boolean = false
                        if entity.Player
                            and type(teamCheck.isSameTeam) == "function" then
                            teammate = teamCheck.isSameTeam(entity.Player)
                        end
                        if not teammate then
                            table.insert(kept, entity)
                        end
                    end
                    targets = kept
                end
                local ignoreText: string = tostring(
                    options["Ignore players"].Value or ""
                )
                if ignoreText ~= "" then
                    local ignored: {[string]: boolean} = compileIgnoreList(ignoreText)
                    local kept: {any} = {}
                    for _, entity: any in ipairs(targets) do
                        if not isIgnoredEntity(entity, ignored) then
                            table.insert(kept, entity)
                        end
                    end
                    targets = kept
                end
                local targetLimit: number = math.max(1, math.round(options["Max targets"].Value))
                while #targets > targetLimit do table.remove(targets) end
                updateTargetStatus(#targets)
                if #targets == 0 then
                    clearHighlights()
                    return
                end
                local now: number = os.clock()
                local rate: number = math.max(options["Swings per second"].Value, 0.5)

                local jitter: number = 1 / rate * 0.25
                local canSwing: boolean = now >= nextSwingAt
                local keep: {[any]: boolean} = {}

                local root: BasePart? = entityLibrary.LocalEntity
                    and entityLibrary.LocalEntity.RootPart
                local rotation: string = options["Rotation"].Value
                local originalCFrame: CFrame? = root and root.CFrame or nil

                if canSwing then

                    nextSwingAt = now
                        + (1 / rate)
                        + (math.random() - 0.5) * 2 * jitter

                    if rotation ~= "Off" and root then
                        local first: any = targets[1]
                        local flat: Vector3 = first.RootPart.Position
                            * Vector3.new(1, 0, 1)
                        root.CFrame = CFrame.lookAt(
                            root.Position,
                            Vector3.new(flat.X, root.Position.Y + 0.01, flat.Z)
                        )
                    end

                    for _, entity: any in ipairs(targets) do
                        keep[entity.Player or entity.Character] = true
                        markTarget(entity, options)

                        if not canSwing then
                            continue
                        end

                        local swung: boolean = false
                        if weapons then
                            for _, label: string in ipairs(aura.Options["Weapons"].Selected or {}) do
                                local candidate: any = weapons:Find(label)
                                if candidate and weapons:Activate(candidate, entity) then
                                    swung = true
                                end
                            end
                        end
                        if not swung and tool then
                            swung = activateTool(tool)
                        end
                        if not swung and weapons then

                            local fallback: any = weapons:Best({Melee = true})
                            swung = fallback ~= nil and weapons:Activate(fallback, entity)
                        end

                        local extraHits: number =
                            math.max(math.round(options["Hits per swing"].Value) - 1, 0)
                        for _ = 1, extraHits do
                            if tool then
                                activateTool(tool)
                            elseif weapons then
                                weapons:Swing()
                            end
                        end
                        if not options["Contact damage"].Value then

                            continue
                        end

                        if not options["Hit through walls"].Value
                            and not entityLibrary:VisibleFrom(entity) then
                            continue
                        end

                        local overlap: OverlapParams = OverlapParams.new()
                        overlap.FilterType = Enum.RaycastFilterType.Include
                        overlap.FilterDescendantsInstances = {entity.Character}
                        local parts: {BasePart} = currentWorkspace:GetPartBoundsInBox(
                            entity.RootPart.CFrame,
                            Vector3.new(5, 6, 5),
                            overlap
                        )
                        for _, part: BasePart in ipairs(parts) do
                            fireContact(handle, part)
                        end
                    end

                    if rotation == "Silent" and root and originalCFrame then
                        root.CFrame = originalCFrame :: CFrame
                    end

                    clearHighlights(keep)
                end)
            end,
    })

    aura:CreateDropdown({
        Name = "Activation",
        List = {"Always", "Hold key"},
        Index = 1,
    })
    aura:CreateBind({
        Name = "Hold key",
        Show = {Option = "Activation", Values = {"Hold key"}},
        Default = Enum.KeyCode.Unknown,
    })
    aura:CreateSlider({
        Name = "Reach",
        Min = 5,
        Max = 60,
        Default = 18,
        Tooltip = "Distance at which the weapon swings and its touch events "
            .. "fire. This is the only Kill Aura distance.",
    })
    aura:CreateSlider({
        Name = "Max targets",
        Min = 1,
        Max = 10,
        Default = 3,
        Tooltip = "How many valid players or NPCs one swing may hit. One "
            .. "sticks to the best target.",
    })
    aura:CreateDropdown({
        Name = "Priority",
        List = {"Distance", "Health", "Threat"},
        Index = 1,
        Tooltip = "Which target comes first: nearest, weakest, or the one "
            .. "most dangerous to you.",
    })
    aura:CreateSlider({
        Name = "Swings per second",
        Min = 1,
        Max = 20,
        Default = 8,
        Tooltip = "A fast human clicks about 8-12 times a second. Twenty is "
            .. "not a person, and a server that logs click rates knows it.",
    })

    aura:CreateSlider({
        Name = "Field of view",
        Min = 30,
        Max = 360,
        Default = 360,
        Tooltip = "Degrees around your character. 360 scans every direction; "
            .. "this is not cursor aim.",
    })
    aura:CreateDropdown({
        Name = "Rotation",
        List = {"Off", "Silent", "Face"},
        Index = 2,
        Tooltip = "Silent turns you for the swing only and puts you back; "
            .. "Face keeps you pointed at the target.",
    })
    aura:CreateSlider({
        Name = "Hits per swing",
        Min = 1,
        Max = 8,
        Default = 1,
        Tooltip = "How many times the weapon is pressed for each swing the "
            .. "rate allows. Some games count every press.",
    })
    aura:CreateList({
        Name = "Weapons",
        Items = function(): {string}
            if not weapons then
                return {}
            end
            return weapons:Labels({All = true, Melee = true})
        end,

        EmptyText = "",
        Tooltip = "Melee weapons this module may swing: tools, view models, "
            .. "inventory slots and the game's own attack button. Empty means "
            .. "whatever you are holding.",
    })
    aura:CreateTextBox({
        Name = "Weapon name",
        Default = "",
        Tooltip = "Part of the tool's name. Leave empty and the module uses "
            .. "whatever you last swung by hand.",
    })
    aura:CreateButton({
        Name = "Bind held tool",
        Tooltip = "Remembers whatever is in your hands right now.",
        Function = function(): ()
            local tool: Tool? = equippedTool()
            if not tool then
                aura:Notify("nothing equipped")
                return
            end
            learnedWeapon = (tool :: Tool).Name
            aura:SetStatus((tool :: Tool).Name .. " bound")
        end,
    })
    aura:CreateToggle({
        Name = "Auto equip",
        Default = false,
        Tooltip = "Pull the weapon out of the backpack when empty-handed. Off "
            .. "by default: most games this module helps in do not need "
            .. "anything in your hands at all.",
        Function = function(value: boolean): ()
            if weapons then
                weapons:SetAllowEquip(value)
            end
        end,
    })
    aura:CreateToggle({
        Name = "Require tool",
        Default = false,
        Tooltip = "Do nothing while your hands are empty.",
    })
    aura:CreateToggle({
        Name = "Team check",
        Default = true,
        Tooltip = "Skip teammates, analysed by the always-on advanced team "
            .. "check (game teams, roles, team colours, neutral rules), plus "
            .. "Friend List entries.",
    })
    aura:CreateToggle({
        Name = "Wall check",
        Default = false,
        Tooltip = "Off by default: the aura hits through walls and floors "
            .. "until you ask it not to.",
    })
    aura:CreateToggle({
        Name = "Contact damage",
        Default = true,
        Tooltip = "Also fire the weapon's touch events. Needed by most melee "
            .. "weapons; requires firetouchinterest.",
    })
    aura:CreateToggle({
        Name = "Hit through walls",
        Default = true,
        Tooltip = "Fire contact damage even when the target is behind "
            .. "geometry. Independent of the wall check, which decides "
            .. "whether the target is picked at all.",
    })
    aura:CreateToggle({
        Name = "Target NPCs",
        Default = false,
        Tooltip = "Include non-player humanoid models in the reach scan.",
    })
    aura:CreateTextBox({
        Name = "Ignore players",
        Default = "",
        Tooltip = "Names, display names or user ids, separated by commas. "
            .. "They are skipped by targeting: never picked, rotated to or hit.",
    })
    aura:CreateToggle({
        Name = "Highlight targets",
        Default = false,
    })
    aura:CreateColor({
        Name = "Highlight colour",
        Show = {Option = "Highlight targets"},
        Default = Color3.fromRGB(235, 110, 110),
    })

    aura:CreateNote(
        "Servers may enforce their own rate and reach rules. Reach is the "
            .. "only distance this card reads; there is no "
            .. "second, longer one. Friend List protects players from this "
            .. "module in every game."
    )

    activeCleanup = function(): ()
        if aura.Enabled then aura:Toggle(false) else clearHighlights() end
    end
    Module.Initialized = true
    return aura
end

function Module.destroy(): ()
    if activeCleanup then
        activeCleanup()
    end
    activeCleanup = nil
    Module.Initialized = false
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Combat/KillAura.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Movement/AirWalk.lua
BYTES: 11556
SHA-256: 6e4700faca8a8621fde6991d05bf6622cb668dbfd3d9b6b9b03d856ccdc80078
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
export type Runtime = {
    framework: any,
    entity: any,
    host: any,
    services: any,
}

local Module = {
    Name = "Air Walk",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local activeCleanup: (() -> ())? = nil

function Module.init(context: Runtime): any
    local host: any = context.host
    local framework: any = context.framework
    local localPlayer: Player = host.LocalPlayer
    local UserInputService: UserInputService = host.UserInputService
    local RunService: RunService = host.RunService
    local world: Workspace = host.workspace
    local create: any = host.create

    local platform: Part? = nil
    local gui: Frame? = nil
    local airY: number? = nil
    local trackedCharacter: Model? = nil
    local connections: {RBXScriptConnection} = {}
    local held: {[string]: {[any]: boolean}} = {
        Up = {},
        Down = {},
    }

    local VERTICAL_SPEED: number = 22

    local function characterParts(): (Model?, Humanoid?, BasePart?)
        local character: Model? = localPlayer.Character
        if not character then
            return nil, nil, nil
        end
        local humanoid: Humanoid? = character:FindFirstChildOfClass("Humanoid") :: Humanoid?
        local root: BasePart? = character:FindFirstChild("HumanoidRootPart") :: BasePart?
        return character, humanoid, root
    end

    local function footOffsetOf(humanoid: Humanoid, root: BasePart): number
        local hip: number = humanoid.HipHeight
        if hip <= 0 then hip = 2 end
        return hip + root.Size.Y * 0.5 + 0.5
    end

    local function destroyPlatform(): ()
        if platform then platform:Destroy(); platform = nil end
    end

    local function destroyGui(): ()
```

```
        if gui then gui:Destroy(); gui = nil end
    end

    local function clearConnections(): ()
        for _, conn: RBXScriptConnection in ipairs(connections) do
            conn:Disconnect()
        end
        table.clear(connections)
        table.clear(held.Up)
        table.clear(held.Down)
    end

    local function setHold(direction: string, source: any, isDown: boolean): ()
        local bucket: {[any]: boolean}? = held[direction]
        if not bucket then
            return
        end
        bucket[source] = isDown and true or nil
    end

    local function isDirectionHeld(direction: string): boolean
        local bucket: {[any]: boolean}? = held[direction]
        return bucket ~= nil and next(bucket) ~= nil
    end

    local function ensurePlatform(): Part
        if platform and platform.Parent then return platform end
        local p: Part = Instance.new("Part")
        p.Name = "Wurst_AirWalkPlatform"
        p.Anchored = true
        p.CanCollide = true
        p.CanQuery = false
        p.CanTouch = false
        p.CastShadow = false
        p.Size = Vector3.new(16, 1, 16)
        p.Transparency = 1
        p.Parent = world
        platform = p
        return p
    end

    local airWalk: any
    airWalk = framework.Categories.Exploits:CreateModule({
        Name = "Air Walk",
        Category = "Exploits",
        ConfigKey = "Universal.AirWalk",
        Order = 11,
        Tooltip = "Stay in the air with every jump and walk in mid-air. Space to go up, Shift to go down, with mobile Up/Down buttons.",
        Function = function(enabled: boolean): ()
            clearConnections()
            destroyGui()
            destroyPlatform()
            airY = nil
            trackedCharacter = nil

            if not enabled then
                airWalk:SetStatus(nil)
                return
            end

            airWalk:SetStatus("Active")

            local function connect(signal: any, callback: any): ()
                table.insert(connections, signal:Connect(callback))
            end

            connect(UserInputService.InputBegan, function(event: InputObject, processed: boolean): ()
                if processed or UserInputService:GetFocusedTextBox() then
                    return
                end
                if event.UserInputType == Enum.UserInputType.Keyboard then
                    if event.KeyCode == Enum.KeyCode.Space then
                        setHold("Up", "Space", true)
                    elseif event.KeyCode == Enum.KeyCode.LeftShift
                        or event.KeyCode == Enum.KeyCode.RightShift then
                        setHold("Down", event.KeyCode, true)
                    end
                elseif event.UserInputType == Enum.UserInputType.Touch then
                    setHold("Up", event, false)
                    setHold("Down", event, false)
                end
            end)

            if UserInputService.WindowFocusReleased then
                connect(UserInputService.WindowFocusReleased, function(): ()
                    table.clear(held.Up)
                    table.clear(held.Down)
                end)
            end
            if UserInputService.TextBoxFocused then
                connect(UserInputService.TextBoxFocused, function(): ()
                    table.clear(held.Up)
                    table.clear(held.Down)
                end)
            end

            if UserInputService.TouchEnabled and type(create) == "function" and host.ScreenGui then
                gui = create("Frame", {
                    Name = "AirWalkControls",
                    Parent = host.ScreenGui,
                    Size = UDim2.fromScale(1, 1),
                    BackgroundTransparency = 1,
                    ZIndex = 30,
                })
                local function makeMobileButton(name: string, text: string, at: UDim2, size: UDim2): any
                    local object: any = create("TextButton", {
                        Name = name,
                        Parent = gui,
                        Position = at,
                        Size = size,
                        Text = text,
                        Font = Enum.Font.Code,
                        TextSize = 18,
                        TextColor3 = Color3.new(1, 1, 1),
                        BackgroundColor3 = Color3.fromRGB(25, 25, 25),
                        BackgroundTransparency = 0.3,
                        BorderSizePixel = 1,
                        BorderColor3 = Color3.fromRGB(120, 120, 120),
                        AutoButtonColor = true,
                        ZIndex = 31,
                    })
                    if host.ThemeEngine then
                        host.ThemeEngine.Bind(object, "BackgroundColor3", "background")
                        host.ThemeEngine.Bind(object, "TextColor3", "text")
                    end
                    return object
                end

                for _, name: string in ipairs({"Up", "Down"}) do
                    local btn: any = makeMobileButton(
                        name,
                        if name == "Up" then "+" else "−",
                        UDim2.new(1, -70, 1, if name == "Up" then -152 else -96),
                        UDim2.fromOffset(48, 48)
                    )
                    connect(btn.InputBegan, function(event: InputObject): ()
                        if event.UserInputType == Enum.UserInputType.Touch
                            or event.UserInputType == Enum.UserInputType.MouseButton1 then
                            setHold(name, event, true)
                        end
                    end)
                    connect(btn.InputEnded, function(event: InputObject): ()
                        if event.UserInputType == Enum.UserInputType.Touch
                            or event.UserInputType == Enum.UserInputType.MouseButton1 then
                            setHold(name, event, false)
                        end
                    end)
                end
            end

            connect(RunService.Heartbeat, function(dt: number): ()
                local character: Model?, humanoid: Humanoid?, root: BasePart? = characterParts()
                if not character or not humanoid or not root or humanoid.Health <= 0 then
                    destroyPlatform()
                    airY = nil
                    trackedCharacter = nil
                    return
                end

                local stepDt: number = if type(dt) == "number" and dt > 0 and dt < 0.5 then dt else (1 / 60)
                local offset: number = footOffsetOf(humanoid, root)
                local currentFootY: number = root.Position.Y - offset

                if trackedCharacter ~= character or airY == nil then
                    trackedCharacter = character
                    airY = currentFootY
                end

                local keyUp: boolean = false
                local keyDown: boolean = false
                if not UserInputService:GetFocusedTextBox() then
                    keyUp = UserInputService:IsKeyDown(Enum.KeyCode.Space)
                    keyDown = UserInputService:IsKeyDown(Enum.KeyCode.LeftShift)
                        or UserInputService:IsKeyDown(Enum.KeyCode.RightShift)
                end

                local goingUp: boolean = keyUp or isDirectionHeld("Up")
                local goingDown: boolean = keyDown or isDirectionHeld("Down")

                local part: Part = ensurePlatform()
                if goingDown and not goingUp then
                    airY = currentFootY - VERTICAL_SPEED * stepDt - 1.4
                    local vel: Vector3 = root.AssemblyLinearVelocity
                    if vel.Y > -VERTICAL_SPEED then
                        root.AssemblyLinearVelocity = Vector3.new(vel.X, -VERTICAL_SPEED, vel.Z)
                    end
                elseif goingUp and not goingDown then
                    airY = math.max(airY :: number, currentFootY) + VERTICAL_SPEED * stepDt
                    local vel: Vector3 = root.AssemblyLinearVelocity
                    if vel.Y < VERTICAL_SPEED then
                        root.AssemblyLinearVelocity = Vector3.new(vel.X, VERTICAL_SPEED, vel.Z)
                    end
                else
                    -- Lock to the highest point reached on any jump.
                    if currentFootY > (airY :: number) then airY = currentFootY end
                end
                if not part.CanCollide then part.CanCollide = true end
                part.CFrame = CFrame.new(root.Position.X, airY :: number, root.Position.Z)
            end)

            airWalk:Clean(function(): ()
                clearConnections()
                destroyGui()
                destroyPlatform()
                airY = nil
                trackedCharacter = nil
            end)
        end,
    })

    activeCleanup = function(): ()
        clearConnections()
        destroyGui()
        destroyPlatform()
        if airWalk and airWalk.Enabled then
            airWalk:Toggle(false)
        end
    end

    Module.Initialized = true
    return airWalk
end

function Module.destroy(): ()
    if activeCleanup then
        activeCleanup()
    end
    activeCleanup = nil
    Module.Initialized = false
end

return Module
```

```
--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Movement/AirWalk.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Movement/AntiFling.lua
BYTES: 19674
SHA-256: 753b44cfab2e8126b040b7c5e2029b5116f4b37fb99bfdf26e504f33066fa64a
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
```

```
export type Runtime = {
    framework: any,
    entity: any,
    host: any,
    services: any,
}

local Module = {
    Name = "AntiFling",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local activeCleanup: (() -> ())? = nil

function Module.init(context: Runtime): any
    local host: any = context.host
    local services: any = context.services
    local activity: any = services and services.activity
    local framework: any = context.framework
    local players: any = host.Players
    local localPlayer: any = host.LocalPlayer
    local world: any = host.workspace or workspace
    local featureConnections: any = host.featureConnections
    local disconnect: any = host.disconnectFeatureConnection

    local enabled: boolean = false
    local playerShield: boolean = true
    local impactShield: boolean = true
    local feature: any = nil
    local character: Model? = nil
    local links: {[BasePart]: {[BasePart]: any}} = {}
    local hazardSeen: {[BasePart]: number} = {}
    -- Body model -> clock value until which it stays shielded. Only populated
    -- by a fling-speed body inside RADIUS (v190).
    local dangerousUntil: {[Model]: number} = {}
    local nextScan: number = 0
    local dangerUntil: number = 0
    local suspended: boolean = false
    local warnedOverlap: boolean = false

    -- Limits are deliberately conservative. The shield is local physics, not
    -- server authority; do not change other people's CanCollide or global groups.
    local SCAN_INTERVAL: number = 0.15
    local RADIUS: number = 30
    local HAZARD_HOLD: number = 0.75
    local LINEAR_LIMIT: number = 80
    local SURFACE_LIMIT: number = 70
    local MAX_LINKS: number = 2048
    local MAX_PARTS: number = 200

    local function isBusy(): boolean
        return activity ~= nil and type(activity.isActive) == "function"
            and (activity.isActive("fling") or activity.isActive("walkfling"))
    end

    local function destroyLinks(): ()
        for _, perLocal: {[BasePart]: any} in pairs(links) do
            for _, constraint: any in pairs(perLocal) do
                pcall(function(): () constraint:Destroy() end)
            end
        end
        table.clear(links)
        table.clear(hazardSeen)
        table.clear(dangerousUntil)
        dangerUntil = 0
    end

    local function isToolPart(part: Instance): boolean
        local ok: boolean, result: any = pcall(function(): any
            return part:FindFirstAncestorOfClass("Tool")
        end)
        return ok and result ~= nil
    end

    local function listBodyParts(model: Model?, onlyCollidable: boolean): {BasePart}
        local parts: {BasePart} = {}
        if not model or not model.Parent then return parts end
        local ok: boolean, descendants: any = pcall(function(): any return model:GetDescendants() end)
        if not ok or type(descendants) ~= "table" then return parts end
        for _, object: Instance in ipairs(descendants) do
            if object:IsA("BasePart") and not isToolPart(object) then
                local part: BasePart = object :: BasePart
                if not onlyCollidable or part.CanCollide then table.insert(parts, part) end
            end
        end
        return parts
    end

    local function fastPart(part: BasePart, scale: number?): boolean
        local ok: boolean, fast: any = pcall(function(): boolean
            -- Relative speed alone flags a stationary player when *we* sprint;
            -- arm the fallback only for motion driven by the other assembly.
            local linear: Vector3 = part.AssemblyLinearVelocity
            local angular: Vector3 = part.AssemblyAngularVelocity
            local radius: number = math.min(part.Size.Magnitude * 0.5, 20)
            local multiplier: number = scale or 1
            return linear.Magnitude > LINEAR_LIMIT * multiplier
                or angular.Magnitude * radius > SURFACE_LIMIT * multiplier
        end)
        return ok and fast == true
    end

    local function supportAssembly(root: BasePart, humanoid: Humanoid, owner: Model): BasePart?
        local ok: boolean, result: any = pcall(function(): any
            local params: RaycastParams = RaycastParams.new()
            params.FilterType = Enum.RaycastFilterType.Exclude
            params.FilterDescendantsInstances = {owner}
            params.IgnoreWater = true
            params.RespectCanCollide = true
            local distance: number = math.max(4, (tonumber(humanoid.HipHeight) or 2) + root.Size.Y * 0.5 + 2)
            return world:Raycast(root.Position, Vector3.new(0, -distance, 0), params)
        end)
        local ground: any = ok and result and result.Instance
        if ground and ground:IsA("BasePart") then
            return ground.AssemblyRootPart or ground
        end
        return nil
    end

    local function synchronize(owner: Model, humanoid: Humanoid, root: BasePart, now: number): ()
        local ownParts: {BasePart} = listBodyParts(owner, true)
        local desired: {[BasePart]: boolean} = {}
        local candidates: {any} = {}
        local characterModels: {[Model]: boolean} = {}
        local roster: any = players:GetPlayers()

        -- v190: the shield is a FLING shield, not a permanent noclip against
        -- every player. Until v189 every visible body got NoCollisionConstraint
        -- against every one of our parts, so simply walking into someone went
        -- through them (the player's report: "Anti Fling me hace noclipear los
        -- cuerpos de las demás personas"). Now a body is shielded only while it
        -- is actually a hazard -- a part of it moving or spinning at fling
        -- speeds inside RADIUS -- and for HAZARD_HOLD seconds after, so an
        -- ongoing fling keeps its protection between scans. Untouched bodies
        -- keep normal collisions.
        for _, player: Player in ipairs(roster) do
            local other: Model? = player ~= localPlayer and player.Character or nil
            if other and other ~= owner and other.Parent then
                characterModels[other] = true
                if playerShield then
                    local otherRoot: BasePart? = other:FindFirstChild("HumanoidRootPart") :: BasePart?
                    local distance: number = otherRoot
                        and (otherRoot.Position - root.Position).Magnitude
                        or math.huge
                    if distance <= RADIUS then
                        local parts: {BasePart} = listBodyParts(other, false)
                        local hazard: boolean = false
                        for _, part: BasePart in ipairs(parts) do
                            if part.CanCollide and fastPart(part, nil) then
                                hazard = true
                                break
                            end
                        end
                        if hazard then
                            dangerousUntil[other] = now + HAZARD_HOLD
                            dangerUntil = math.max(dangerUntil, now + HAZARD_HOLD)
                        end
                    end
                end
            end
        end
        -- Release bodies that are no longer a threat, then rebuild the budget
        -- from the ones that still are.
        if playerShield then
            for model: Model, expiresAt: number in pairs(dangerousUntil) do
                if expiresAt <= now or not model.Parent then
                    dangerousUntil[model] = nil
                end
            end
            for model: Model in pairs(dangerousUntil) do
                local otherRoot: BasePart? = model:FindFirstChild("HumanoidRootPart") :: BasePart?
                local distance: number = otherRoot
                    and (otherRoot.Position - root.Position).Magnitude
                    or math.huge
                local parts: {BasePart} = listBodyParts(model, true)
                table.insert(candidates, {model = model, distance = distance, parts = parts})
                for _, part: BasePart in ipairs(parts) do
                    desired[part] = true
                end
            end
        end
        table.sort(candidates, function(a: any, b: any): boolean return a.distance < b.distance end)

        -- The query is local and bounded. Anchored geometry, the supporting
        -- platform/assembly, seats and tools keep their normal interactions.
        -- A fast loose part is suppressed only briefly while it remains risky.
        if impactShield and not humanoid.Sit and not humanoid.SeatPart then
            local overlap: any = nil
            pcall(function(): ()
                overlap = OverlapParams.new()
                overlap.FilterType = Enum.RaycastFilterType.Exclude
                overlap.FilterDescendantsInstances = {owner}
                overlap.MaxParts = MAX_PARTS
                overlap.RespectCanCollide = true
            end)
            local ok: boolean, nearby: any = pcall(function(): any
                return world:GetPartBoundsInRadius(root.Position, RADIUS, overlap)
            end)
            if not ok or type(nearby) ~= "table" then
                if not warnedOverlap and type(host.notify) == "function" then
                    warnedOverlap = true
                    host.notify("Anti Fling: moving-object scan unavailable; player protection remains active.")
                end
            else
                local floorAssembly: BasePart? = supportAssembly(root, humanoid, owner)
                for _, instance: Instance in ipairs(nearby) do
                    if instance:IsA("BasePart") then
                        local part: BasePart = instance :: BasePart
                        local isCharacterPart: boolean = false
                        local ancestor: Instance? = part.Parent
                        while ancestor and ancestor ~= world do
                            if characterModels[ancestor :: any] then isCharacterPart = true break end
                            ancestor = ancestor.Parent
                        end
                        local inOwner: boolean = part:IsDescendantOf(owner)
                        local isTool: boolean = isToolPart(part)
                        local assembly: BasePart? = part.AssemblyRootPart or part
                        -- Ordinary weapon/tools remain interactive. An extreme
                        -- spinning/flying handle is treated as a hazard, at 3x
                        -- the loose-object threshold, only while it is nearby.
                        if part.Parent and part.CanCollide and not part.Anchored and not inOwner
                            and (not isCharacterPart or isTool)
                            and (not floorAssembly or assembly ~= floorAssembly)
                            and fastPart(part, if isTool then 3 else nil) then
                            hazardSeen[part] = now
                            dangerUntil = math.max(dangerUntil, now + HAZARD_HOLD)
                        end
                    end
                end
            end
            for part: BasePart, seen: number in pairs(hazardSeen) do
                if not part.Parent or now - seen > HAZARD_HOLD then
                    hazardSeen[part] = nil
                else
                    desired[part] = true
                end
            end
        else
            table.clear(hazardSeen)
        end

        -- Allocate the finite budget afresh in priority order: newly dangerous
        -- objects first, then nearest players. Otherwise 2,048 older/farther
        -- links could permanently starve a new incoming threat.
        local ordered: {BasePart} = {}
        local included: {[BasePart]: boolean} = {}
        local function include(part: BasePart): ()
            if desired[part] and part.Parent and not included[part] then
                included[part] = true
                table.insert(ordered, part)
            end
        end
        local hazards: {BasePart} = {}
        for part: BasePart in pairs(hazardSeen) do table.insert(hazards, part) end
        table.sort(hazards, function(a: BasePart, b: BasePart): boolean
            return (a.Position - root.Position).Magnitude < (b.Position - root.Position).Magnitude
        end)
        for _, part: BasePart in ipairs(hazards) do include(part) end
        if playerShield then
            for _, candidate: any in ipairs(candidates) do
                for _, part: BasePart in ipairs(candidate.parts) do include(part) end
            end
        end
        local accepted: {[BasePart]: {[BasePart]: boolean}} = {}
        local allocated: number = 0
        for _, other: BasePart in ipairs(ordered) do
            if allocated >= MAX_LINKS then break end
            local perLocal: {[BasePart]: boolean} = {}
            for _, own: BasePart in ipairs(ownParts) do
                if allocated >= MAX_LINKS then break end
                if own ~= other then
                    perLocal[own] = true
                    allocated += 1
                end
            end
            if next(perLocal) then accepted[other] = perLocal end
        end
        -- Remove old or lower-priority pairs before filling the new budget.
        for other: BasePart, perLocal: {[BasePart]: any} in pairs(links) do
            local wanted: {[BasePart]: boolean}? = accepted[other]
            for own: BasePart, constraint: any in pairs(perLocal) do
                if not wanted or not wanted[own] or not own.Parent or not other.Parent then
                    pcall(function(): () constraint:Destroy() end)
                    perLocal[own] = nil
                end
            end
            if next(perLocal) == nil then links[other] = nil end
        end
        for other: BasePart, wanted: {[BasePart]: boolean} in pairs(accepted) do
            local perLocal: {[BasePart]: any} = links[other] or {}
            links[other] = perLocal
            for own: BasePart in pairs(wanted) do
                if not perLocal[own] then
                    local ok: boolean, constraint: any = pcall(function(): any
                        local link: NoCollisionConstraint = Instance.new("NoCollisionConstraint")
                        link.Part0, link.Part1 = own, other
                        link.Parent = own
                        return link
                    end)
                    if ok and constraint then perLocal[own] = constraint end
                end
            end
            if next(perLocal) == nil then links[other] = nil end
        end
    end

    local function step(): ()
        if not enabled then return end
        local owner: Model?, humanoid: Humanoid?, root: BasePart? = host.getCharacterParts()
        if owner ~= character then
            destroyLinks()
            character = owner
            nextScan = 0
        end
        if not owner or not humanoid or not root or not owner.Parent or humanoid.Health <= 0 then
            destroyLinks()
            return
        end
        if isBusy() then
            if not suspended then destroyLinks() end
            suspended = true
            return
        end
        if suspended then suspended = false nextScan = 0 end
        local now: number = os.clock()
        if now >= nextScan then
            nextScan = now + SCAN_INTERVAL
            local ok: boolean, err: any = pcall(synchronize, owner, humanoid, root, now)
            if not ok then
                -- A streaming object may disappear during the scan. Retry on
                -- the next interval, without leaving partial constraints stuck.
                destroyLinks()
                if type(host.warn) == "function" then host.warn("Anti Fling: " .. tostring(err)) end
            end
        end
        if now > dangerUntil or root.Anchored or humanoid.Sit or humanoid.SeatPart then return end
        -- Emergency damping is only armed by a nearby extreme mover. Normal
        -- sprinting, jumps, vehicles and game launch pads are not capped merely
        -- because this feature is on.
        local walkSpeed: number = math.max(16, tonumber(humanoid.WalkSpeed) or 16)
        local speed: any = framework and (framework:GetModule("Speed Settings") or framework:GetModule("Speed Hack"))
        if speed and speed.Enabled and speed.Options.Speed then
            walkSpeed = math.max(walkSpeed, tonumber(speed.Options.Speed.Value) or 0)
        end
        local jumpPower: number = math.max(50, tonumber(humanoid.JumpPower) or 50)
        local velocity: Vector3 = root.AssemblyLinearVelocity
        local planar: Vector3 = Vector3.new(velocity.X, 0, velocity.Z)
        local limit: number = math.max(120, walkSpeed * 4)
        local riseLimit: number = math.max(180, jumpPower * 4)
        local nextVelocity: Vector3 = velocity
        local changed: boolean = false
        if planar.Magnitude > limit then
            local move: Vector3 = humanoid.MoveDirection
            local direction: Vector3 = move.Magnitude > 0.05 and move.Unit * walkSpeed or Vector3.zero
            nextVelocity = Vector3.new(direction.X, nextVelocity.Y, direction.Z)
            changed = true
        end
        if nextVelocity.Y > riseLimit then
            nextVelocity = Vector3.new(nextVelocity.X, riseLimit, nextVelocity.Z)
            changed = true
        end
        pcall(function(): ()
            if root.AssemblyAngularVelocity.Magnitude > 25 then
                root.AssemblyAngularVelocity = Vector3.zero
            end
            if changed then root.AssemblyLinearVelocity = nextVelocity end
        end)
    end

    local function toggleAntiFling(value: boolean): ()
        enabled = value == true
        -- v190: MM2's Fling card and Walk Fling read this flag to warn and
        -- refuse while the shield is on (it cancels the contact a fling needs).
        if host.state then host.state.antiFlingActive = enabled end
        disconnect("AntiFling")
        destroyLinks()
        character = nil
        nextScan = 0
        suspended = false
        if enabled then
            featureConnections.AntiFling = host.TaskManager:Connect(step, nil, "Anti Fling")
        end
    end

    feature = host.createUniversalFeature(
        "Anti fling",
        "Collision Shield: local player and extreme-object protection",
        2,
        toggleAntiFling,
        {categoryName = "Utilities"}
    )
    host.addToggleOption(feature, "Player Shield", true, function(value: boolean): ()
        playerShield = value == true
        nextScan = 0
        if not playerShield and not impactShield then destroyLinks() end
    end, "Shield yourself only from bodies that are moving or spinning at fling speeds nearby, for a moment after each hit. Calm players keep normal collisions, so you no longer walk through their bodies.")
    host.addToggleOption(feature, "Impact Shield", true, function(value: boolean): ()
        impactShield = value == true
        nextScan = 0
        if not playerShield and not impactShield then destroyLinks() end
    end, "Temporarily avoid nearby extreme-speed loose objects; support platforms, seated vehicles and ordinary tools are left alone.")
    host.addFeatureTooltip(feature,
        "Collision Shield only changes local collision pairs. Extremely fast/streamed objects and server-owned physics can still launch you.")

    activeCleanup = function(): ()
        toggleAntiFling(false)
        if host.state then host.state.antiFlingActive = false end
    end
```

```
    Module.Initialized = true
    return feature
end

function Module.destroy(): ()
    if activeCleanup then pcall(activeCleanup) end
    activeCleanup = nil
    Module.Initialized = false
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Movement/AntiFling.lua
--------------------------------------------------------------------------------


--------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Movement/ClickTeleport.lua
BYTES: 16617
SHA-256: e9a8bbf27a75ffabc3f104b70abaf32e4068d03e89595161affbde5627ca44cd
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
export type Runtime = {
    framework: any,
    entity: any,
    host: any,
    services: any,
}

local Module = {
    Name = "ClickTeleport",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local activeCleanup: (() -> ())? = nil

local function characterRoot(localPlayer: Player): (BasePart?, Humanoid?)
    local character: Model? = localPlayer.Character
    if not character then
        return nil, nil
    end
    return character:FindFirstChild("HumanoidRootPart") :: BasePart?,
        character:FindFirstChildOfClass("Humanoid") :: Humanoid?
end

function Module.init(context: Runtime): any
    local host: any = context.host
    local framework: any = context.framework
    local localPlayer: Player = host.LocalPlayer
    local userInput: UserInputService = host.UserInputService
    local menu: any = context.services.menu

    local runtime: any = {
        armedUntil = 0,
        point = nil :: string?,
        lastOrigin = nil :: Vector3?,
    }

    local teleport: any
    teleport = framework.Categories.Utilities:CreateModule({
        Name = "Teleport",
        ConfigKey = "Universal.ClickTeleport",
        Category = "Utilities",
        Order = 1,

        Kind = "action",
        Silent = true,
        Tooltip = "Press to travel to the destination selected below.",
        Function = function(): ()
            Module.run(context, teleport, runtime)
        end,
    })

    local destinations = {"Tap point", "Nearest player", "Named player", "Item", "Waypoint", "Last position"}
    if host.GAME_CHECK and host.GAME_CHECK.MM2Active then
        table.insert(destinations, 1, "MM2")
        local config = host.configData
        if config then
            local key = "Universal.ClickTeleport.Destination"
            if config.values[key] == nil or string.sub(tostring(config.values[key]), 1, 4) == "MM2 " then
                config.values[key] = "MM2"
            end
        end
    end
    teleport:CreateDropdown({Name = "Destination", List = destinations, Index = 1,
        Tooltip = "MM2 shows all game teleport buttons together. Other destinations use the main action.",
        Function = function(value)
            runtime.armedUntil = 0
            if runtime.cancelTravel then runtime.cancelTravel() end
            local mm2Controls = { ["Teleport to Map"] = true, ["Teleport to Lobby"] = true,
                ["Teleport to Player"] = true, ["Get Gun"] = true, ["Player name"] = true,
                ["Get Gun key"] = true, ["Auto Get Gun"] = true }
            teleport.OptionFilter = function(name)
                if name == "Destination" then return true end
                return value == "MM2" and mm2Controls[name] == true or value ~= "MM2" and not mm2Controls[name]
            end
            teleport:RefreshVisibility()
        end})
    teleport:CreateTextBox({
        Name = "Player",
        Show = {Option = "Destination", Values = {"Named player"}},
        Default = "",
        Tooltip = "Part of a name or display name, for Named player.",
    })
    teleport:CreateTextBox({
        Name = "Item",
        Show = {Option = "Destination", Values = {"Item"}},
        Default = "",
        Tooltip = "Part or model name to look for, for Item. Partial names "
            .. "match, so \"chest\" finds \"GoldChest\".",
    })
    local pointList=teleport:CreateList({Name="Point",Single=true,
        Show={Option="Destination",Values={"Waypoint"}},
        EmptyText="No points. Save one in Markers.",Items=function()
            local points=context.services.points
            return points and points.Names() or {}
        end,Function=function(_,names) runtime.point=names[1] end,
        Tooltip="The same saved points as Markers. Choose one, then Teleport to point."})
    teleport:CreateButton({Name="Teleport to point",Show={Option="Destination",Values={"Waypoint"}},
        Function=function() Module.run(context,teleport,runtime) end})
    local unsubscribe
    if context.services.points then
        unsubscribe=context.services.points.Subscribe(function()
            if runtime.point and not context.services.points.Get(runtime.point) then
                if runtime.cancelTravel then runtime.cancelTravel() end
                pointList:Set(runtime.point,false)
            end
            pointList:Refresh()
        end)
    end
    teleport:CreateSlider({
        Name = "Arm window",
        Show = {Option = "Destination", Values = {"Tap point"}},
        Min = 1,
        Max = 15,
        Default = 6,
        Tooltip = "Seconds Tap point stays armed after the button is pressed.",
    })
    teleport:CreateSlider({
        Name = "Search range",
        Show = {Option = "Destination", Values = {"Nearest player", "Named player", "Item"}},
        Min = 50,
        Max = 5000,
        Default = 1500,
        Tooltip = "How far to look for items and players.",
    })
    teleport:CreateSlider({
        Name = "Player offset",
        Show = {Option = "Destination", Values = {"Nearest player", "Named player"}},
        Min = 0,
        Max = 20,
        Default = 4,
        Tooltip = "Studs to stop short of a player, so you do not land inside "
            .. "them.",
    })
    teleport:CreateToggle({
        Name = "Skip friends",
        Show = {Option = "Destination", Values = {"Nearest player"}},
        Default = false,
        Tooltip = "Nearest player ignores anyone on the Friend List.",
    })
    teleport:CreateDropdown({
        Name = "Movement",
        List = {"Instant", "Glide"},
        Index = 1,
        Tooltip = "Instant jumps to the destination. Glide moves continuously at Glide speed (studs/sec), regardless of frame rate. A new trip cancels the previous one.",
    })
    teleport:CreateSlider({
        Name = "Glide speed",
        Show = {Option = "Movement", Values = {"Glide"}},
        Min = 1,
        Max = 500,
        Default = 60,
        Tooltip = "Travel speed in studs per second, not distance per frame. Higher = faster.",
    })
    teleport:CreateSlider({
        Name = "Height offset",
        Min = 0,
        Max = 12,
        Default = 3,
        Tooltip = "Studs above whatever you land on.",
    })
    teleport:CreateToggle({
        Name = "Keep momentum",
        Default = false,
        Tooltip = "Leave your velocity alone on arrival instead of zeroing it.",
    })

    local tapConnection: RBXScriptConnection = userInput.InputBegan:Connect(
        function(input: InputObject, processed: boolean): ()
            if processed or os.clock() >= runtime.armedUntil then
                return
            end
            if menu.isCapturingInput() then
                return
            end
            local touched: boolean = input.UserInputType == Enum.UserInputType.Touch
            local clicked: boolean =
                input.UserInputType == Enum.UserInputType.MouseButton1
            if not touched and not clicked then
                return
            end
            runtime.armedUntil = 0
            Module.travel(
                context,
                teleport,
                runtime,
                Module.pointUnder(context, input.Position),
                true
            )
        end
    )

    activeCleanup = function(): ()
        pcall(function(): ()
            tapConnection:Disconnect()
        end)
        runtime.armedUntil = 0
        if runtime.cancelTravel then runtime.cancelTravel() end
        if unsubscribe then unsubscribe() end
    end
    Module.Initialized = true
    return teleport
end

function Module.pointUnder(context: Runtime, screenPosition: Vector3): Vector3?
    local host: any = context.host
    local currentWorkspace: Workspace = host.workspace or workspace
    local camera: Camera? = currentWorkspace.CurrentCamera
    if not camera then
        return nil
    end
    local resolvedCamera: Camera = camera :: Camera
    local ray: Ray = resolvedCamera:ScreenPointToRay(
        screenPosition.X,
        screenPosition.Y
    )
    local parameters: RaycastParams = RaycastParams.new()
    parameters.FilterType = Enum.RaycastFilterType.Exclude
    parameters.IgnoreWater = true
    parameters.FilterDescendantsInstances = {
        host.LocalPlayer.Character :: any,
        resolvedCamera,
    }
    local result: RaycastResult? = currentWorkspace:Raycast(
        ray.Origin,
        ray.Direction.Unit * 4096,
        parameters
    )
    return result and result.Position or nil
end

local function matchesName(candidate: string, query: string): boolean
    return string.find(string.lower(candidate), string.lower(query), 1, true) ~= nil
end

function Module.findItem(
    context: Runtime,
    query: string,
    origin: Vector3,
    range: number
): Vector3?
    local host: any = context.host
    local currentWorkspace: Workspace = host.workspace or workspace
    local best: Vector3? = nil
    local bestDistance: number = range
    local character: Model? = host.LocalPlayer.Character

    for _, descendant: Instance in ipairs(currentWorkspace:GetDescendants()) do
        if character and descendant:IsDescendantOf(character) then
            continue
        end
        if not matchesName(descendant.Name, query) then
            continue
        end
        local position: Vector3? = nil
        if descendant:IsA("BasePart") then
            position = (descendant :: BasePart).Position
        elseif descendant:IsA("Model") then
            local pivotOk: boolean, pivot: any = pcall(function(): CFrame
                return (descendant :: Model):GetPivot()
            end)
            if pivotOk then
                position = (pivot :: CFrame).Position
            end
        end
        if not position then
            continue
        end
        local distance: number = ((position :: Vector3) - origin).Magnitude
        if distance < bestDistance then
            best = position
            bestDistance = distance
        end
    end
    return best
end

function Module.run(context: Runtime, teleport: any, runtime: any): ()
    if runtime.cancelTravel then runtime.cancelTravel() end
    if teleport.Options.Destination.Value == "MM2" then
        teleport:Notify("Choose a MM2 teleport button below")
        return
    end
    local host: any = context.host
    local entityLibrary: any = context.entity
    local localPlayer: Player = host.LocalPlayer
    local options: any = teleport.Options
    local destination: string = options["Destination"].Value

    local root: BasePart? = characterRoot(localPlayer)
    if not root then
        teleport:Notify("no character")
        return
    end
    local origin: Vector3 = (root :: BasePart).Position
    local range: number = options["Search range"].Value

    if destination == "Tap point" then
        runtime.armedUntil = os.clock() + options["Arm window"].Value
        teleport:Notify("tap where you want to go")
        return
    end

    if destination == "Waypoint" then
        local points=context.services.points
        local position,surface
        if points and runtime.point then position,surface=points.Get(runtime.point) end
        if not position then teleport:Notify("Choose a saved point from Markers first.");return end
        Module.travel(context,teleport,runtime,position,surface==true)
        return
    end

    if destination == "Last position" then
        if not runtime.lastOrigin then
            teleport:Notify("nowhere to go back to")
            return
        end
        Module.travel(context, teleport, runtime, runtime.lastOrigin, false)
        return
    end

    if destination == "Item" then
        local query: string = tostring(options["Item"].Value or "")
        if query == "" then
            teleport:Notify("type an item name first")
            return
        end
        local found: Vector3? = Module.findItem(context, query, origin, range)
        if not found then
            teleport:Notify("no \"" .. query .. "\" within range")
            return
        end
        Module.travel(context, teleport, runtime, found, true)
        return
    end

    entityLibrary:Refresh()
    local target: any = nil
    if destination == "Nearest player" then
        local bestDistance: number = range
        for _, entity: any in ipairs(entityLibrary.List) do
            if options["Skip friends"].Value
                and entityLibrary:IsProtected(entity.Player) then
                continue
            end
            if entity.Distance < bestDistance then
                target = entity
                bestDistance = entity.Distance
            end
        end
    else
        local query: string = tostring(options["Player"].Value or "")
        if query == "" then
            teleport:Notify("type a player name first")
            return
        end
        for _, entity: any in ipairs(entityLibrary.List) do
            if matchesName(entity.Player.Name, query)
                or matchesName(entity.Player.DisplayName, query) then
                target = entity
                break
            end
        end
    end

    if not target then
        teleport:Notify("no matching player")
        return
    end

    local targetPosition: Vector3 = target.RootPart.Position
    local offset: number = options["Player offset"].Value
    local approach: Vector3 = targetPosition - origin
    if offset > 0 and approach.Magnitude > offset then
        targetPosition = targetPosition - approach.Unit * offset
    end
    Module.travel(context, teleport, runtime, targetPosition, false)
end

function Module.travel(
    context: Runtime,
    teleport: any,
    runtime: any,
    destination: Vector3?,
    liftToSurface: boolean
): ()
    local host: any = context.host
    local localPlayer: Player = host.LocalPlayer
    if runtime.cancelTravel then runtime.cancelTravel() end
    if not destination then
        teleport:Notify("nothing there")
        return
    end

    local root: BasePart?, humanoid: Humanoid? = characterRoot(localPlayer)
    if not root then
        return
    end
    local resolvedRoot: BasePart = root :: BasePart
    local options: any = teleport.Options

    runtime.lastOrigin = resolvedRoot.Position

    local target: Vector3 = destination :: Vector3
    if liftToSurface then

        local hipHeight: number = humanoid and (humanoid :: Humanoid).HipHeight or 2
        target = target
            + Vector3.new(0, hipHeight + options["Height offset"].Value, 0)
    end

    local keepMomentum: boolean = options["Keep momentum"].Value
    if options["Movement"].Value == "Instant" then
        resolvedRoot.CFrame = resolvedRoot.CFrame + (target - resolvedRoot.Position)
        if not keepMomentum then
            resolvedRoot.AssemblyLinearVelocity = Vector3.zero
            resolvedRoot.AssemblyAngularVelocity = Vector3.zero
        end
        return
    end

    local speed=math.max(0,options["Glide speed"].Value)
    if speed==0 then teleport:Notify("Glide speed must be greater than zero.");return end
    local character=localPlayer.Character
    local connection
    local function finish()
        if connection then connection:Disconnect();connection=nil end
        runtime.cancelTravel=nil
        if context.services.activity then context.services.activity.set("teleport",false) end
        teleport:SetStatus(nil)
    end
    if runtime.cancelTravel then runtime.cancelTravel() end
    runtime.cancelTravel=finish
    if context.services.activity then context.services.activity.set("teleport",true) end
    teleport:SetStatus("Gliding")
    connection=host.RunService.Heartbeat:Connect(function(dt)
        if localPlayer.Character~=character or not resolvedRoot.Parent
            or characterRoot(localPlayer)~=resolvedRoot or humanoid and humanoid.Health<=0 then finish();return end
        local offset=target-resolvedRoot.Position
        local distance=offset.Magnitude
        local amount=math.min(distance,speed*math.clamp(dt,0,0.1))
        if distance<=amount+0.001 then
            resolvedRoot.CFrame=resolvedRoot.CFrame+(target-resolvedRoot.Position)
            if not keepMomentum then resolvedRoot.AssemblyLinearVelocity=Vector3.zero;resolvedRoot.AssemblyAngularVelocity=Vector3.zero end
            finish();return
        end
        if distance>0 then resolvedRoot.CFrame+=offset.Unit*amount end
        if not keepMomentum then resolvedRoot.AssemblyLinearVelocity=Vector3.zero;resolvedRoot.AssemblyAngularVelocity=Vector3.zero end
    end)
end

function Module.destroy(): ()
    if activeCleanup then
        pcall(activeCleanup)
    end
    activeCleanup = nil
    Module.Initialized = false
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Movement/ClickTeleport.lua
--------------------------------------------------------------------------------


--------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Movement/FreezeMovements.lua
BYTES: 36772
SHA-256: e8207e4f4ae84eaba146847aa5fc82a65c5754fbcccd0f3a2c387bf52b597d90
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
export type Runtime = {
    framework: any,
    entity: any,
    host: any,
}

local Module = {
    Name = "FreezeMovements",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local activeCleanup: (() -> ())? = nil

function Module.init(context: Runtime): any
    local host: any = context.host
    local TaskManager: any = host.TaskManager
    local featureConnections: any = host.featureConnections
    local disconnectFeatureConnection: any = host.disconnectFeatureConnection
    local getCharacterParts: any = host.getCharacterParts
    local createUniversalFeature: any = host.createUniversalFeature
    local addCycleOption: any = host.addCycleOption
    local addToggleOption: any = host.addToggleOption
    local addNumberOption: any = host.addNumberOption
    -- "Position Lock" is what the row used to call "Anchor": it pins the body
    -- where it stands. The new name says what the user sees, and the extra rows
    -- let the lock be tuned instead of being an all-or-nothing anchor.
    local LOCK_MODE: string = "Position Lock"
    local MOTION_MODE: string = "Motion"
    local freezeSettings = {
        mode = LOCK_MODE,
        emotes = true,
        animation = false,
        anyEmote = true,
        lockRotation = true,
        returnDistance = 0,
        keepGravity = false,
    }
    -- An old saved emote mode must not silently become Position Lock on upgrade.
    local config: any = host.configData
    if config and config.values and config.states then
        local key: string = "Movement.FreezeMovements.Freezemode"
        local legacy: any = config.values[key]
        if legacy == "Freeze Emotes" or legacy == "Glitch Animations" then
            config.values[key] = MOTION_MODE
            local prefix: string = "Movement.FreezeMovements."
            if config.states[prefix .. "FreezeEmotes"] == nil then
                config.states[prefix .. "FreezeEmotes"] = legacy == "Freeze Emotes"
            end
            if config.states[prefix .. "FreezeAnimation"] == nil then
                config.states[prefix .. "FreezeAnimation"] = legacy == "Glitch Animations"
            end
        elseif legacy == "Anchor" or legacy == "Freeze Physics" then
            -- Renamed mode, same behaviour.
            config.values[key] = LOCK_MODE
        end
    end
    local FreezeFeature: any = nil
    local originalFreezeState = setmetatable({}, {__mode = "k"})
    -- Motion never changes movement physics. Managed tracks are local;
    -- cross-client visibility depends on the game's animation replication.
    local stickyEmoteTracks: {any} = {}
    local emoteLastSeen: {[any]: number} = setmetatable({}, {__mode = "k"})

    local function getAnimator(humanoid: any): any
        local animator: any = humanoid and humanoid:FindFirstChildOfClass("Animator")
        if not animator and humanoid and humanoid.Parent then
            local character: any = humanoid.Parent
            animator = character and character:FindFirstChildWhichIsA("Animator", true)
        end
        return animator
    end

    local emoteConnection: any = nil
    local catalogEmoteConnection: any = nil
    local emoteGeneration: number = 0
    local watchedHumanoid: any = nil
    local watchedAnimator: any = nil
    local watchedAnimate: any = nil
    local nextMetadataRefresh: number = 0
    local freezeEnabled: boolean = false
    local resolvedAssets: {[number]: boolean} = {}
    local assetAttempts: {[number]: number} = {}
    local pendingEmoteTracks: {any} = {}
    local lastMetadataRefresh: number = -math.huge
    local emoteNames: {[string]: boolean} = {}
    local emoteIds: {[string]: boolean} = {}
    local managedEmotes: {[any]: any} = {}
    local ownedEmotes: {[any]: boolean} = setmetatable({}, {__mode = "k"})
    local isEmoteTrack: (any) -> boolean

    local function clearStickyEmoteTracks(): ()
        emoteGeneration += 1
        if emoteConnection then emoteConnection:Disconnect() end
        if catalogEmoteConnection then catalogEmoteConnection:Disconnect() end
        emoteConnection = nil
        catalogEmoteConnection = nil
        local previousHumanoid: any = watchedHumanoid
        watchedHumanoid = nil
        watchedAnimator = nil
        watchedAnimate = nil
        resolvedAssets = {}
        assetAttempts = {}
        pendingEmoteTracks = {}
        local stopped: {[any]: boolean} = {}
        local function stopOnce(track: any): ()
            if track and not stopped[track] then
                stopped[track] = true
                pcall(function(): () track:Stop(0.1) end)
            end
        end
        -- The original Animate track can still be playing even if we never
        -- had to hand it off. OFF must release that source as well as ours.
        for _, source: any in ipairs(stickyEmoteTracks) do stopOnce(source) end
        local animator: any = getAnimator(previousHumanoid)
        if animator then
            pcall(function(): ()
                for _, track: any in ipairs(animator:GetPlayingAnimationTracks()) do
                    if track.IsPlaying and not ownedEmotes[track] and isEmoteTrack(track) then stopOnce(track) end
                end
            end)
        end
        for _, track: any in pairs(managedEmotes) do
            stopOnce(track)
            pcall(function(): () track:Destroy() end)
        end
        managedEmotes = {}
        ownedEmotes = setmetatable({}, {__mode = "k"})
        emoteNames = {}
        emoteIds = {}
        stickyEmoteTracks = {}
        emoteLastSeen = setmetatable({}, {__mode = "k"})
    end

    local CLASSIC_EMOTES = {
        dance = true, dance1 = true, dance2 = true, dance3 = true,
        wave = true, point = true, cheer = true, laugh = true,
    }
    -- Any emote, not only Roblox's own: games ship their own dance/taunt clips
    -- with arbitrary names and asset IDs, so the identifier is a name keyword
    -- (checked on the track and on up to six parents) instead of a fixed list.
    local EMOTE_KEYWORDS = {
        "emote", "dance", "dancing", "taunt", "celebrat", "cheer", "wave",
        "laugh", "salute", "groove", "pose", "victory", "expression",
        "emoji", "flex", "party", "clap", "bow", "ridicule", "point",
    }
    -- ...but a weapon swing is not an emote, and it often sits on the same
    -- Action priority. These win over a keyword so "SwordWave" stays a swing.
    local NON_EMOTE_KEYWORDS = {
        "attack", "shoot", "slash", "stab", "swing", "punch", "kick", "reload",
        "equip", "unequip", "tool", "hit", "block", "parry", "cast", "spell",
        "sword", "gun", "knife", "punch", "throw", "fling",
    }

    local function hasKeyword(name: string, list: {string}): boolean
        local lower: string = string.lower(name)
        for _, keyword: string in ipairs(list) do
            if string.find(lower, keyword, 1, true) then return true end
        end
        return false
    end

    local function looksLikeEmoteName(name: string): boolean
        local lower: string = string.lower(name)
        if emoteNames[lower] then return true end
        if hasKeyword(lower, NON_EMOTE_KEYWORDS) then return false end
        lower = lower:gsub("anim$", "")
        if CLASSIC_EMOTES[lower] == true then return true end
        return hasKeyword(lower, EMOTE_KEYWORDS)
```

```
    end

    isEmoteTrack = function(track: any): boolean
        -- Action priority is also used by weapon attacks; it is NOT an
        -- emote identifier. Avoid substring matches such as SpinAttack.
        if looksLikeEmoteName(tostring(track.Name or "")) then return true end
        local animation: any = track.Animation
        if typeof(animation) == "Instance" then
            if looksLikeEmoteName(animation.Name) then return true end
            local node: any = animation.Parent
            for _depth: number = 1, 6 do
                if not node then break end
                if looksLikeEmoteName(node.Name) then return true end
                node = node.Parent
            end
            local id: string? = string.match(animation.AnimationId, "%d+")
            if id and emoteIds[id] then return true end
        end
        return false
    end

    -- Custom emotes whose name says nothing ("Animation", "Clip", "Pose 3"):
    -- while the body is standing still, a looped Action/Idle clip that nobody
    -- blacklisted is an emote the game started, so Motion holds it too. A
    -- moving body is never scored this way, which keeps walk cycles and weapon
    -- swings out even when their names are meaningless.
    local function looksLikeStandingEmote(humanoid: any, root: any, track: any): boolean
        if not freezeSettings.anyEmote then return false end
        if not track or ownedEmotes[track] or isEmoteTrack(track) then return false end
        local name: string = tostring(track.Name or "")
        if hasKeyword(name, NON_EMOTE_KEYWORDS) then return false end
        local animation: any = track.Animation
        if typeof(animation) ~= "Instance" then return false end
        if track.Looped ~= true then return false end
        local priority: any = track.Priority
        if priority ~= Enum.AnimationPriority.Action and priority ~= Enum.AnimationPriority.Idle then
            return false
        end
        if root and root.AssemblyLinearVelocity and root.AssemblyLinearVelocity.Magnitude > 2 then
            return false
        end
        local ok: boolean, state: any = pcall(function(): any return humanoid:GetState() end)
        if ok and (state == Enum.HumanoidStateType.Running
            or state == Enum.HumanoidStateType.Jumping
            or state == Enum.HumanoidStateType.Freefall) then
            return false
        end
        return true
    end

    local function rememberEmote(track: any): ()
        -- A newly selected emote replaces the previous held emote. Never
        -- layer a whole history of dances on the same joints.
        for source: any, managed: any in pairs(managedEmotes) do
            pcall(function(): () managed:Stop(0.1) end)
            pcall(function(): () managed:Destroy() end)
            ownedEmotes[managed] = nil
            managedEmotes[source] = nil
        end
        stickyEmoteTracks = {track}
        emoteLastSeen = setmetatable({}, {__mode = "k"})
        emoteLastSeen[track] = os.clock()
    end

    local function freezeEmotesTick(humanoid: any, root: any): ()
        -- Hold the selected emote while walking with a track Animate does
        -- not own. Keep it through jumps; release on death, replacement,
        -- mode change or disable.
        local animator: any = getAnimator(humanoid)
        if not animator then
            if watchedAnimator then clearStickyEmoteTracks() end
            return
        end
        local animate = humanoid.Parent and humanoid.Parent:FindFirstChild("Animate")
        -- Avatar descriptions/Animate are often installed AFTER respawn.
        -- Retry metadata at a bounded rate, and immediately on replacement.
        if watchedHumanoid ~= humanoid or watchedAnimator ~= animator
            or watchedAnimate ~= animate or os.clock() >= nextMetadataRefresh then
            if watchedHumanoid ~= humanoid or watchedAnimator ~= animator then
                clearStickyEmoteTracks()
            else
                if emoteConnection then emoteConnection:Disconnect(); emoteConnection = nil end
                if catalogEmoteConnection then catalogEmoteConnection:Disconnect(); catalogEmoteConnection = nil end
            end
            watchedHumanoid = humanoid
            watchedAnimator = animator
            watchedAnimate = animate
            lastMetadataRefresh = os.clock()
            nextMetadataRefresh = lastMetadataRefresh + 2
            -- Catalog asset IDs are NOT necessarily the AnimationId played
            -- by Animate. Prefer the actual track reported by the engine.
            -- This event requires executor permissions on some client builds;
            -- a denied connection must not prevent the metadata fallback.
            pcall(function(): ()
                catalogEmoteConnection = humanoid.EmoteTriggered:Connect(function(success: boolean, track: any): ()
                    if success and track and watchedHumanoid == humanoid and not ownedEmotes[track] then
                        local animation: any = track.Animation
                        if animation then
                            local id: string? = string.match(animation.AnimationId, "%d+")
                            if id then emoteIds[id] = true end
                        end
                        rememberEmote(track)
                    end
                end)
            end)
            local catalogAssets: {number} = {}
            pcall(function(): ()
                local description: any = humanoid:GetAppliedDescription()
                local ok: boolean, emotes: any = pcall(description.GetEmotes, description)
                description:Destroy()
                if not ok then return end
                local seen: {[number]: boolean} = {}
                for name: string, ids: any in pairs(emotes) do
                    emoteNames[string.lower(name)] = true
                    for _, id: any in ipairs(ids) do
                        emoteIds[tostring(id)] = true
                        local numberId: number? = tonumber(id)
                        if numberId and not seen[numberId] and not resolvedAssets[numberId] and #catalogAssets < 48 then
                            seen[numberId] = true
                            resolvedAssets[numberId] = true
                            assetAttempts[numberId] = (assetAttempts[numberId] or 0) + 1
                            table.insert(catalogAssets, numberId)
                        end
                    end
                end
            end)
            -- Resolve catalog containers without playing an emote to discover
            -- its ID. Never run downloaded scripts; only inspect Animation data.
            local generation: number = emoteGeneration
            if #catalogAssets > 0 and task and game then
                task.spawn(function(): ()
                    for _, assetId: number in ipairs(catalogAssets) do
                        if generation ~= emoteGeneration then return end
                        local ok: boolean, objects: any = pcall(function(): any
                            return game:GetObjects("rbxassetid://" .. tostring(assetId))
                        end)
                        if not ok and generation == emoteGeneration and (assetAttempts[assetId] or 0) < 3 then
                            resolvedAssets[assetId] = nil -- bounded retry after transient load failure
                        end
                        if ok and type(objects) == "table" then
                            for _, object: any in ipairs(objects) do
                                pcall(function(): ()
                                    local nodes: {any} = object:GetDescendants()
                                    table.insert(nodes, object)
                                    for _, node: any in ipairs(nodes) do
                                        if generation == emoteGeneration and node:IsA("Animation") then
                                            local id: string? = string.match(node.AnimationId, "%d+")
                                            if id then emoteIds[id] = true end
                                        end
                                    end
                                end)
                                pcall(function(): () object:Destroy() end)
                            end
                        end
                    end
                end)
            end
            -- R6 /e dance2 commonly plays Animation1/2/3, not a track
            -- named dance2. Read IDs under Animate's named emote folders.
            pcall(function(): ()
                local animate: any = humanoid.Parent:FindFirstChild("Animate")
                if animate then
                    for _, animation: any in ipairs(animate:GetDescendants()) do
                        if animation:IsA("Animation") then
                            local node: any = animation
                            while node and node ~= animate do
                                local name: string = string.lower(node.Name):gsub("anim$", "")
                                if CLASSIC_EMOTES[name] or emoteNames[name] then
                                    local id: string? = string.match(animation.AnimationId, "%d+")
                                    if id then emoteIds[id] = true end
                                    break
                                end
                                node = node.Parent
                            end
                        end
                    end
                end
            end)
            -- Catch tracks at start, before Animate can cancel them on movement.
            emoteConnection = animator.AnimationPlayed:Connect(function(track: any): ()
                if not ownedEmotes[track] and isEmoteTrack(track) and not emoteLastSeen[track] then
                    rememberEmote(track)
                elseif not ownedEmotes[track] and not emoteLastSeen[track]
                    and looksLikeStandingEmote(humanoid, root, track) then
                    rememberEmote(track)
                elseif not ownedEmotes[track] then
                    if #pendingEmoteTracks >= 8 then table.remove(pendingEmoteTracks, 1) end
                    table.insert(pendingEmoteTracks, {track = track, at = os.clock()})
                    if os.clock() - lastMetadataRefresh >= 0.25 then nextMetadataRefresh = 0 end
                end
            end)
        end
        for index = #pendingEmoteTracks, 1, -1 do
            local candidate = pendingEmoteTracks[index]
            if isEmoteTrack(candidate.track) then
                rememberEmote(candidate.track)
                table.remove(pendingEmoteTracks, index)
            elseif os.clock() - candidate.at > 5 then
                table.remove(pendingEmoteTracks, index)
            end
        end
        local okTracks: boolean, tracks: any = pcall(function(): any
            return animator:GetPlayingAnimationTracks()
        end)
        if not okTracks or type(tracks) ~= "table" then
            return
        end
        local playing: {[any]: boolean} = {}
        for _, track: any in ipairs(tracks) do
            if track and track.IsPlaying then
                playing[track] = true
                if not ownedEmotes[track] and not emoteLastSeen[track] and isEmoteTrack(track) then
                    rememberEmote(track)
                end
                if emoteLastSeen[track] then
                    emoteLastSeen[track] = os.clock()
                end
            end
        end
        local moving: boolean = root ~= nil
            and root.AssemblyLinearVelocity.Magnitude > 1
        local now: number = os.clock()
        for index: number = #stickyEmoteTracks, 1, -1 do
            local track: any = stickyEmoteTracks[index]
            if not track then
                table.remove(stickyEmoteTracks, index)
            elseif playing[track] or (managedEmotes[track] and managedEmotes[track].IsPlaying) then
                emoteLastSeen[track] = now
            else
                local lastSeen: number = emoteLastSeen[track]
                if lastSeen == nil then
                    lastSeen = now
                    emoteLastSeen[track] = now
                end
                -- Only hand off after a recent, recognized source was
                -- cancelled while moving; do not guess from Action priority.
                if moving and now - lastSeen < 1.5 then
                    pcall(function(): ()
                        -- A separate track is essential: Animate owns the
                        -- original and will keep cancelling it when walking.
                        local managed: any = managedEmotes[track]
                        if not managed then
                            managed = animator:LoadAnimation(track.Animation)
                            ownedEmotes[managed] = true
                            managedEmotes[track] = managed
                            managed.Looped = true
                            managed.Priority = Enum.AnimationPriority.Action
                        end
                        if not managed.IsPlaying then managed:Play(0.1) end
                    end)
                elseif now - lastSeen > 0.5 then
                    local managed: any = managedEmotes[track]
                    if managed then
                        pcall(function(): () managed:Stop(0.1) end)
                        pcall(function(): () managed:Destroy() end)
                        managedEmotes[track] = nil
                        ownedEmotes[managed] = nil
                    end
                    table.remove(stickyEmoteTracks, index)
                    emoteLastSeen[track] = nil
                end
            end
        end
    end

    local motionSource: any = nil
    local motionTrack: any = nil
    local motionHumanoid: any = nil
    local motionAnimator: any = nil
    local function clearMotionTrack(): ()
        if motionTrack then
            pcall(function(): () motionTrack:Stop(0.1) end)
            pcall(function(): () motionTrack:Destroy() end)
        end
        motionSource, motionTrack, motionHumanoid, motionAnimator = nil, nil, nil, nil
    end

    local LOCOMOTION = {walk=true, run=true, idle=true, jump=true, fall=true,
        climb=true, swim=true, swimidle=true}
    local function isLocomotion(track: any): boolean
        if not track or ownedEmotes[track] or isEmoteTrack(track) then return false end
        if track.Priority == Enum.AnimationPriority.Movement then return true end
        if track.Priority ~= Enum.AnimationPriority.Core and track.Priority ~= Enum.AnimationPriority.Idle then return false end
        local function known(name: any): boolean
            local key: string = string.lower(tostring(name or "")):gsub("anim$", "")
            return LOCOMOTION[key] == true
        end
        if known(track.Name) then return true end
        local node: any = track.Animation
        for _index: number = 1, 5 do
            if not node then break end
            if known(node.Name) then return true end
            node = node.Parent
        end
        return false
    end

    local function freezeAnimationTick(humanoid: any): ()
        local animator: any = getAnimator(humanoid)
        if not animator then
            if motionAnimator then clearMotionTrack() end
            return
        end
        if motionHumanoid ~= humanoid or motionAnimator ~= animator then
            clearMotionTrack()
            motionHumanoid = humanoid
            motionAnimator = animator
        end
        if not motionSource then
            local ok: boolean, tracks: any = pcall(function(): any return animator:GetPlayingAnimationTracks() end)
            if not ok or type(tracks) ~= "table" then return end
            local weight: number = -1
            local function consider(track: any): ()
                if not track or not track.IsPlaying or ownedEmotes[track] or isEmoteTrack(track) then
                    return
                end
                local current: number = tonumber(track.WeightCurrent) or 1
                if current > weight then motionSource, weight = track, current end
            end
            -- Roblox's own walk/run/idle clips first: they are the common case
            -- and keep the previous behaviour when a game plays both.
            for _, track: any in ipairs(tracks) do
                if isLocomotion(track) then consider(track) end
            end
            -- Then anything else: a game can drive locomotion with its own clip
            -- (custom rigs, animated vehicles, tool-held run cycles).
            if not motionSource and freezeSettings.anyAnimation then
                for _, track: any in ipairs(tracks) do consider(track) end
            end
        end
        if not motionSource then return end
        -- Lock the selected clip, NOT its TimePosition. Walking continues
        -- looping even while the native controller switches to jump/fall.
        pcall(function(): ()
            if not motionTrack then
                motionTrack = animator:LoadAnimation(motionSource.Animation)
                motionTrack.Looped = true
                motionTrack.Priority = Enum.AnimationPriority.Action
                motionTrack:Play(0.1)
                motionTrack.TimePosition = motionSource.TimePosition
                motionTrack:AdjustSpeed(motionSource.Speed or 1)
            end
            -- Explicit emotes take precedence when both Motion boxes are on.
            local emotePlaying: boolean = false
            if freezeSettings.emotes then
                for _, source: any in ipairs(stickyEmoteTracks) do
                    if source.IsPlaying or (managedEmotes[source] and managedEmotes[source].IsPlaying) then
                        emotePlaying = true
                        break
                    end
                end
            end
            if emotePlaying then
                if motionTrack.IsPlaying then motionTrack:Stop(0.1) end
            elseif not motionTrack.IsPlaying then
                motionTrack:Play(0.1)
            end
        end)
    end

    local function restoreFreezeMovements(): ()
        for root: BasePart, original: any in pairs(originalFreezeState) do
            root.Anchored = original.anchored
        end
        originalFreezeState = setmetatable({}, {__mode = "k"})
    end

    -- Position Lock: keep the body where it stands. Hard by default (anchored),
    -- or with gravity kept so the character can still fall and land; the rows
    -- decide how much of the pose is frozen and how far a push may go before it
    -- is undone.
    local function lockPosition(root: any): ()
        local original: any = originalFreezeState[root]
        if not original then
            local cframe: any = nil
            local look: any = nil
            pcall(function(): ()
                if root.CFrame ~= nil then
                    cframe = root.CFrame
                    look = root.CFrame.LookVector
                end
            end)
            original = {
                anchored = root.Anchored,
                position = root.Position,
                cframe = cframe,
                look = look,
            }
            originalFreezeState[root] = original
        end
        local lockedPosition: any = original.position
        local gravity: boolean = freezeSettings.keepGravity == true
        local lockRotation: boolean = freezeSettings.lockRotation == true
        -- An anchored root cannot physically turn. When rotation is free, pin
        -- the position each frame instead, leaving Roblox's AutoRotate usable.
        root.Anchored = not gravity and lockRotation
        if not gravity and not lockRotation and lockedPosition ~= nil then
            root.Position = lockedPosition
        end
        if root.AssemblyLinearVelocity ~= nil then
            if gravity then
                local vertical: number = tonumber(root.AssemblyLinearVelocity.Y) or 0
                root.AssemblyLinearVelocity = Vector3.new(0, math.min(vertical, 0), 0)
            else
                root.AssemblyLinearVelocity = Vector3.zero
            end
        end
        if freezeSettings.lockRotation and root.AssemblyAngularVelocity ~= nil then
            root.AssemblyAngularVelocity = Vector3.zero
        end
        if freezeSettings.lockRotation and original.look ~= nil then
            pcall(function(): ()
                if CFrame ~= nil and root.CFrame ~= nil then
                    root.CFrame = CFrame.lookAt(root.Position, root.Position + original.look)
                end
            end)
        end
        local limit: number = freezeSettings.returnDistance or 0
        if gravity and limit > 0 and lockedPosition ~= nil and root.Position ~= nil then
            local dx: number = (tonumber(root.Position.X) or 0) - (tonumber(lockedPosition.X) or 0)
            local dz: number = (tonumber(root.Position.Z) or 0) - (tonumber(lockedPosition.Z) or 0)
            if math.sqrt(dx * dx + dz * dz) > limit then
                root.Position = Vector3.new(
                    lockedPosition.X or 0,
                    root.Position.Y,
                    lockedPosition.Z or 0
                )
            end
        end
    end

    local toggleFreezeMovements: (boolean) -> ()
    toggleFreezeMovements = function(enabled: boolean): ()
        freezeEnabled = enabled
        disconnectFeatureConnection("FreezeMovements")
        if not enabled then
            disconnectFeatureConnection("FreezeCharacterAdded")
            disconnectFeatureConnection("FreezeCharacterRemoving")
        end
        restoreFreezeMovements()
        clearMotionTrack()
        clearStickyEmoteTracks()
        if not enabled then
            if FreezeFeature then FreezeFeature:SetStatus(nil) end
            return
        end
        local player = host.LocalPlayer
        if player and player.CharacterAdded and not featureConnections.FreezeCharacterAdded then
            featureConnections.FreezeCharacterAdded = player.CharacterAdded:Connect(function()
                if freezeEnabled then toggleFreezeMovements(true) end
            end)
        end
        if player and player.CharacterRemoving and not featureConnections.FreezeCharacterRemoving then
            featureConnections.FreezeCharacterRemoving = player.CharacterRemoving:Connect(function()
                restoreFreezeMovements()
                clearMotionTrack()
                clearStickyEmoteTracks()
            end)
        end
        FreezeFeature:SetStatus(freezeSettings.mode)
        local previousRoot: any = nil
        featureConnections.FreezeMovements = TaskManager:Connect(function(): ()
            local _, humanoid, root = getCharacterParts()
            if root ~= previousRoot then
                restoreFreezeMovements()
                clearMotionTrack()
                clearStickyEmoteTracks()
                previousRoot = root
            end
            if not humanoid then
                clearMotionTrack()
                clearStickyEmoteTracks()
                return
            end
            if humanoid:GetState() == Enum.HumanoidStateType.Dead then
                restoreFreezeMovements()
                clearMotionTrack()
                clearStickyEmoteTracks()
                return
            end
            if freezeSettings.mode == "Motion" then
                if freezeSettings.emotes then freezeEmotesTick(humanoid, root) end
                if freezeSettings.animation then freezeAnimationTick(humanoid) end
            elseif root then
                lockPosition(root)
            end
        end, nil, "Freeze Animations")
    end

    -- v190: renamed Motion Lock -> Freeze Animations at the player's request.
    -- configKey stays "Movement.FreezeMovements", so profiles, keybinds and the
    -- saved mode survive; only the visible title and its task name changed.
    FreezeFeature = createUniversalFeature(
        "Freeze Animations",
        "Pin the body where it stands, or keep any animation playing while moving",
        4,
        toggleFreezeMovements,
        {categoryName = "Utilities", configKey = "Movement.FreezeMovements"}
    )
    local motionRows: {any} = {}
    local lockRows: {any} = {}
    -- addToggleOption returns the button inside the row, addNumberOption
    -- returns the row itself. Resolving once here is what keeps a mode switch
    -- from hiding the whole options container.
    local function rowFor(handle: any): any
        if not handle then
            return nil
        end
        local name: string = typeof(handle) == "Instance" and handle.Name or ""
        if string.sub(name, 1, 7) == "Option_" or string.sub(name, 1, 8) == "Section_" then
            return handle
        end
        return handle.Parent or handle
    end
    local function registerRow(list: {any}, handle: any): ()
        local row: any = rowFor(handle)
        if row == nil or row == FreezeFeature.options then
            return
        end
        table.insert(list, row)
    end
    local function refreshOptionGroups(): ()
        local motion: boolean = freezeSettings.mode == MOTION_MODE
        for _, row: any in ipairs(motionRows) do
            if host.setOptionVisible then
                host.setOptionVisible(FreezeFeature, row, motion)
            else row.Visible = motion end
        end
        for _, row: any in ipairs(lockRows) do
            if host.setOptionVisible then
                host.setOptionVisible(FreezeFeature, row, not motion)
            else row.Visible = not motion end
        end
    end
    local function reapply(): ()
        if FreezeFeature and FreezeFeature.enabled then toggleFreezeMovements(true) end
    end
    addCycleOption(FreezeFeature, "Freeze mode", {LOCK_MODE, MOTION_MODE}, 1, function(value: string): ()
        -- Accept an old saved value without renaming anything on disk twice.
        if value == "Anchor" or value == "Freeze Physics" then
            value = LOCK_MODE
        end
        freezeSettings.mode = value
        refreshOptionGroups()
        reapply()
    end)
    -- ---- Position Lock rows ------------------------------------------------
    local lockHeading: any = host.addSectionOption and host.addSectionOption(FreezeFeature, "Position Lock settings")
    registerRow(lockRows, lockHeading)
    local lockRotation: any = addToggleOption(FreezeFeature, "Lock rotation", true, function(value: boolean): ()
        freezeSettings.lockRotation = value
        reapply()
    end, "Also stop the body from spinning or being spun by the game. OFF keeps the position and lets the character turn.")
    registerRow(lockRows, lockRotation)
    local keepGravity: any = addToggleOption(FreezeFeature, "Keep gravity", false, function(value: boolean): ()
        freezeSettings.keepGravity = value
        reapply()
    end, "Do not anchor the body: it can still fall and land, but pushes and knockback are erased every frame.")
    registerRow(lockRows, keepGravity)
    local returnRow: any = addNumberOption(FreezeFeature, "Return if pushed", 0, 0, 40, function(value: number): ()
        freezeSettings.returnDistance = value
    end, "Studs of drift allowed before the character is snapped back. 0 disables the return.")
    registerRow(lockRows, returnRow)
    -- ---- Motion rows -------------------------------------------------------
    -- The section widget renders its own caption and divider, so the card only
    -- has to keep the row in its visibility group.
    local heading: any = host.addSectionOption and host.addSectionOption(FreezeFeature, "Motion settings")
    registerRow(motionRows, heading)
    local emotes: any = addToggleOption(FreezeFeature, "Freeze Emotes", true, function(value: boolean): ()
        freezeSettings.emotes = value
        if not value then clearStickyEmoteTracks() end
    end, "Keep emotes playing while moving, Roblox's own and the ones a game ships itself. Takes precedence over the locked clip.")
    registerRow(motionRows, emotes)
    local anyEmote: any = addToggleOption(FreezeFeature, "Any emote", true, function(value: boolean): ()
        freezeSettings.anyEmote = value
        if not value then clearStickyEmoteTracks() end
    end, "Hold a custom emote even when its name and asset say nothing: a looped clip started while standing still counts as one.")
    registerRow(motionRows, anyEmote)
    local animation: any = addToggleOption(FreezeFeature, "Freeze Animation", false, function(value: boolean): ()
        freezeSettings.animation = value
        clearMotionTrack()
    end, "Keep the current clip looping through jumps and other movement. Does not freeze a single frame.")
    registerRow(motionRows, animation)
    local anyAnimation: any = addToggleOption(FreezeFeature, "Any animation", true, function(value: boolean): ()
        freezeSettings.anyAnimation = value
        clearMotionTrack()
    end, "Freeze a game's own locomotion clip too, not only Roblox's walk/run/idle set.")
    registerRow(motionRows, anyAnimation)
    refreshOptionGroups()
    activeCleanup = function(): () toggleFreezeMovements(false) end
    Module.Initialized = true
    return FreezeFeature
end

function Module.destroy(): ()
    if activeCleanup then
        activeCleanup()
    end
    activeCleanup = nil
    Module.Initialized = false
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Movement/FreezeMovements.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Movement/Gravity.lua
BYTES: 2408
SHA-256: 8e8a35dff79bdc0d4e3b9637a8b5f718c2c260520428c826b6a5f636808d40e8
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
export type Runtime = {
    framework: any,
    entity: any,
    host: any,
}

local Module = {
    Name = "Gravity",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local activeCleanup: (() -> ())? = nil

function Module.init(context: Runtime): any
    local host: any = context.host
    local TaskManager: any = host.TaskManager
    local featureConnections: any = host.featureConnections
    local disconnectFeatureConnection: any = host.disconnectFeatureConnection
    local createUniversalFeature: any = host.createUniversalFeature
    local addNumberOption: any = host.addNumberOption
    local workspace: any = host.workspace
    local gravitySettings = {value = 196.2}
    local GravityFeature: any = nil
    local originalGravity: number? = nil

    local function restoreGravity()
        if originalGravity ~= nil then
            workspace.Gravity = originalGravity
            originalGravity = nil
        end
    end

    local function toggleGravity(enabled)
        disconnectFeatureConnection("Gravity")
        if not enabled then
            if GravityFeature then GravityFeature:SetStatus(nil) end
            restoreGravity()
            return
        end

        GravityFeature:SetStatus(tostring(math.round(gravitySettings.value)))
        if originalGravity == nil then originalGravity = workspace.Gravity end
        featureConnections.Gravity = TaskManager:Connect(function()
            if workspace.Gravity ~= gravitySettings.value then
                workspace.Gravity = gravitySettings.value
            end
        end, nil, "Gravity")
    end
```

```
    GravityFeature = createUniversalFeature(
        "Gravity",
        "Keep workspace gravity at a custom value",
        5,
        toggleGravity,
        {categoryName = "World"}
    )
    addNumberOption(GravityFeature, "Gravity value", gravitySettings.value, -500, 500, function(value)
        gravitySettings.value = value
        if GravityFeature.enabled then
            GravityFeature:SetStatus(tostring(math.round(value)))
        end
    end)

    activeCleanup = function(): ()
        disconnectFeatureConnection("Gravity")
        restoreGravity()
    end
    Module.Initialized = true
    return GravityFeature
end

function Module.destroy(): ()
    if activeCleanup then
        activeCleanup()
    end
    activeCleanup = nil
    Module.Initialized = false
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Movement/Gravity.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Movement/InfiniteJump.lua
BYTES: 13307
SHA-256: e4f3dd795fd5ee8edb4845ade1e3dc9a2d5cd9324153f065051eeaa82d7d9dd3
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
export type Runtime = {
    framework: any,
    entity: any,
    host: any,
    services: any,
}

local Module = {
    Name = "Infinite Jump",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local activeCleanup: (() -> ())? = nil

function Module.init(context: Runtime): any
    local host: any = context.host
    local framework: any = context.framework
    local services: any = context.services
    local LocalPlayer: Player = host.LocalPlayer
    local UserInputService: UserInputService = host.UserInputService
    local RunService: RunService = host.RunService
    local movementInput: any = services and services.movementInput

    local lastBoostAt: number = -math.huge
    local pressHeld: boolean = false
    local jumpTouch: any = nil
    local heldKey: any = nil
    local infiniteJump: any

    local function mode(): string
        local option: any = infiniteJump and infiniteJump.Options and infiniteJump.Options.Mode
        return option and option.Value == "Rise" and "Rise" or "Normal"
    end

    local function characterParts(): (Model?, Humanoid?, BasePart?)
        local character: Model? = LocalPlayer and LocalPlayer.Character
        if not character then
            return nil, nil, nil
        end
        local humanoid: Humanoid? = character:FindFirstChildOfClass("Humanoid") :: Humanoid?
        local root: BasePart? = character:FindFirstChild("HumanoidRootPart") :: BasePart?
        return character, humanoid, root
    end

    local function canBoostJump(humanoid: Humanoid?): boolean
        if not humanoid or not humanoid.Parent or humanoid.Health <= 0 then
            return false
        end
        local currentState: Enum.HumanoidStateType = humanoid:GetState()
        return currentState ~= Enum.HumanoidStateType.Seated
            and currentState ~= Enum.HumanoidStateType.Swimming
            and currentState ~= Enum.HumanoidStateType.Climbing
            and currentState ~= Enum.HumanoidStateType.Physics
            and currentState ~= Enum.HumanoidStateType.PlatformStanding
            and currentState ~= Enum.HumanoidStateType.Ragdoll
            and currentState ~= Enum.HumanoidStateType.Dead
    end

    infiniteJump = framework.Categories.Exploits:CreateModule({
        Name = "Infinite Jump",
        Category = "Exploits",
        ConfigKey = "Universal.InfiniteJump",
        Order = 10,
        Tooltip = "Normal adds one jump per press; Rise ascends while the jump key or mobile jump button is held.",
        Function = function(enabled: boolean): ()
            lastBoostAt = -math.huge
            pressHeld = false
            jumpTouch = nil
            heldKey = nil
            if not enabled then
                infiniteJump:SetStatus(nil)
                return
            end

            infiniteJump:SetStatus(mode())

            local function executeAirJump(): ()
                local _, humanoid: Humanoid?, root: BasePart? = characterParts()
                if not humanoid or not root or root.Anchored or not canBoostJump(humanoid) then
                    return
                end
                local power: number = infiniteJump.Options["Jump Power"]
                    and infiniteJump.Options["Jump Power"].Value
                    or 50
                -- Leave the first positive ground jump to Roblox on both PC
                -- and mobile. Negative power deliberately inverts takeoff.
                if humanoid.FloorMaterial ~= Enum.Material.Air and power >= 0 then
                    return
                end
                local now: number = os.clock()
                local interval: number = infiniteJump.Options["Jump interval"] and infiniteJump.Options["Jump interval"].Value or 0.18
                if now - lastBoostAt < interval then
                    return
                end
                lastBoostAt = now

                local velocity: Vector3 = root.AssemblyLinearVelocity
                root.AssemblyLinearVelocity = Vector3.new(velocity.X, power, velocity.Z)
                humanoid:ChangeState(
                    if power < 0
                        then Enum.HumanoidStateType.Freefall
                        else Enum.HumanoidStateType.Jumping
                )
            end

            local function onJumpRequested(): ()
                if UserInputService and type(UserInputService.GetFocusedTextBox) == "function"
                    and UserInputService:GetFocusedTextBox() then return end
                if pressHeld then
                    return
                end
                pressHeld = true
                if UserInputService and UserInputService.TouchEnabled then
                    local okTouches: boolean, touches: any = pcall(function(): any
                        return UserInputService:GetTouches()
                    end)
                    if okTouches and type(touches) == "table" then
                        if #touches == 1 then
                            jumpTouch = touches[1]
                        else
                            local gui: Instance? = LocalPlayer
                                and LocalPlayer:FindFirstChildOfClass("PlayerGui")
                            local touchGui: Instance? = gui and gui:FindFirstChild("TouchGui")
                            local button: any = touchGui and touchGui:FindFirstChild("JumpButton", true)
                            if button then
                                local at: Vector2 = button.AbsolutePosition
                                local size: Vector2 = button.AbsoluteSize
                                for _, touch: any in ipairs(touches) do
                                    local p: Vector3 = touch.Position
                                    if p.X >= at.X and p.Y >= at.Y
                                        and p.X <= at.X + size.X
                                        and p.Y <= at.Y + size.Y then
                                        jumpTouch = touch
                                        break
                                    end
                                end
                            end
                        end
                    end
                end
                executeAirJump()
            end

            if movementInput and type(movementInput.onJumpRequest) == "function" then
                infiniteJump:Clean(movementInput.onJumpRequest(onJumpRequested))
            elseif UserInputService and UserInputService.JumpRequest then
                infiniteJump:Clean(UserInputService.JumpRequest:Connect(onJumpRequested))
            end

            -- Direct PC (Spacebar / Gamepad ButtonA) + Mobile TouchGui JumpButton support
            if UserInputService and UserInputService.InputBegan then
                infiniteJump:Clean(UserInputService.InputBegan:Connect(function(input: any, gameProcessed: boolean): ()
                    if input.KeyCode == Enum.KeyCode.Space or input.KeyCode == Enum.KeyCode.ButtonA then
                        local typing: boolean = type(UserInputService.GetFocusedTextBox) == "function"
                            and UserInputService:GetFocusedTextBox() ~= nil
                        if not gameProcessed and not typing then
                            heldKey = input.KeyCode
                            onJumpRequested()
                        end
                    elseif input.UserInputType == Enum.UserInputType.Touch then
                        local gui: Instance? = LocalPlayer
                            and LocalPlayer:FindFirstChildOfClass("PlayerGui")
                        local touchGui: Instance? = gui and gui:FindFirstChild("TouchGui")
                        local button: any = touchGui and touchGui:FindFirstChild("JumpButton", true)
                        if button and button.AbsolutePosition and button.AbsoluteSize and input.Position then
                            local at: Vector2 = button.AbsolutePosition
                            local size: Vector2 = button.AbsoluteSize
                            local p: Vector3 = input.Position
                            if p.X >= at.X and p.Y >= at.Y
                                and p.X <= at.X + size.X
                                and p.Y <= at.Y + size.Y then
                                jumpTouch = input
                                pressHeld = true
                                executeAirJump()
                            end
                        end
                    end
                end))
            end

            if UserInputService and UserInputService.InputEnded then
                infiniteJump:Clean(UserInputService.InputEnded:Connect(function(event: any): ()
                    if (event.KeyCode == Enum.KeyCode.Space or event.KeyCode == Enum.KeyCode.ButtonA)
                        and (heldKey == nil or heldKey == event.KeyCode) then
                        pressHeld = false
                        heldKey = nil
                        jumpTouch = nil
                    elseif event.UserInputType == Enum.UserInputType.Touch
                        and not heldKey and (not jumpTouch or event == jumpTouch) then
                        pressHeld = false
                        jumpTouch = nil
                    end
                end))
            end

            if UserInputService and UserInputService.WindowFocusReleased then
                infiniteJump:Clean(UserInputService.WindowFocusReleased:Connect(function(): ()
                    pressHeld = false
                    jumpTouch = nil
                    heldKey = nil
                end))
            end
            if UserInputService and UserInputService.TextBoxFocused then
                infiniteJump:Clean(UserInputService.TextBoxFocused:Connect(function(): ()
                    pressHeld = false
                    jumpTouch = nil
                    heldKey = nil
                end))
            end

            if RunService and RunService.PreSimulation then
                infiniteJump:Clean(RunService.PreSimulation:Connect(function(): ()
                    local _, humanoid: Humanoid?, root: BasePart? = characterParts()
                    if not humanoid or not root or humanoid.Health <= 0 then
                        return
                    end
                    local power: number = infiniteJump.Options["Jump Power"]
                        and infiniteJump.Options["Jump Power"].Value
                        or 50
                    local typing: boolean = type(UserInputService.GetFocusedTextBox) == "function"
                        and UserInputService:GetFocusedTextBox() ~= nil
                    if typing then
                        pressHeld = false
                        jumpTouch = nil
                        heldKey = nil
                        return
                    end
                    if mode() == "Rise" and pressHeld and not root.Anchored and canBoostJump(humanoid) then
                        -- A held jump is an ongoing vertical velocity, not a
                        -- second auto-jump loop. Preserve horizontal motion.
                        local vel: Vector3 = root.AssemblyLinearVelocity
                        root.AssemblyLinearVelocity = Vector3.new(vel.X, power, vel.Z)
                        if power < 0 and humanoid:GetState() == Enum.HumanoidStateType.Jumping then
                            humanoid:ChangeState(Enum.HumanoidStateType.Freefall)
                        end
                    elseif mode() == "Normal" and power < 0
                        and humanoid:GetState() == Enum.HumanoidStateType.Jumping then
                        local vel: Vector3 = root.AssemblyLinearVelocity
                        root.AssemblyLinearVelocity = Vector3.new(vel.X, power, vel.Z)
                        humanoid:ChangeState(Enum.HumanoidStateType.Freefall)
                    end
                end))
            end
        end,
    })

    infiniteJump:CreateDropdown({
        Name = "Mode",
        List = {"Normal", "Rise"},
        Index = 1,
        Function = function(value: string): ()
            if infiniteJump.Enabled then infiniteJump:SetStatus(value) end
        end,
        Tooltip = "Normal = one jump per press in air, Rise = hold to ascend. Jump Power + Interval control both.",
    })

    infiniteJump:CreateSlider({
        Name = "Jump Power",
        Min = 1,
        Max = 250,
        Default = 50,
        Tooltip = "Vertical jump velocity applied in mid-air (1-250). Negative inverts jump.",
    })

    infiniteJump:CreateSlider({
        Name = "Jump interval",
        Min = 0.05,
        Max = 0.6,
        Step = 0.01,
        Default = 0.18,
        Tooltip = "Minimum delay between mid-air jumps in seconds. Default 0.18 s (vanilla-like). Lower = faster spam, higher = slower rhythm.",
    })

    infiniteJump:CreateNote("Jump interval: 0.18 s is default Roblox rhythm. 0.05 s = spam fast, 0.6 s = slow.")

    activeCleanup = function(): ()
        if infiniteJump.Enabled then
            infiniteJump:Toggle(false)
        end
    end

    Module.Initialized = true
    return infiniteJump
end

function Module.destroy(): ()
    if activeCleanup then
        activeCleanup()
    end
    activeCleanup = nil
    Module.Initialized = false
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Movement/InfiniteJump.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Movement/Jump.lua
BYTES: 10461
SHA-256: 7992214c554bd576595813c8f833240c343f938a7b6d00c35b54cbf32c611341e
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
export type Runtime = {
    framework: any,
    entity: any,
    host: any,
    services: any,
}

local Module = {
    Name = "Jump Settings",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local activeCleanup: (() -> ())? = nil

function Module.init(context: Runtime): any
    local host: any = context.host
    local framework: any = context.framework
    local movementInput: any = context.services and context.services.movementInput
    local UserInputService: any = host.UserInputService or game:GetService("UserInputService")
    local LocalPlayer: Player = host.LocalPlayer or game:GetService("Players").LocalPlayer

    local originalJumpPower = setmetatable({}, {__mode = "k"}) :: {[Humanoid]: {jumpPower: number, useJumpPower: boolean}}

    local function characterParts(): (Model?, Humanoid?, BasePart?)
        local character: Model? = LocalPlayer.Character
        if not character then
            return nil, nil, nil
        end
        local humanoid: Humanoid? = character:FindFirstChildOfClass("Humanoid")
        local root: BasePart? = character:FindFirstChild("HumanoidRootPart") :: BasePart?
        return character, humanoid, root
    end

    local function restoreJumpPower(): ()
        for humanoid: Humanoid, original in pairs(originalJumpPower) do
            if humanoid and humanoid.Parent then
                humanoid.JumpPower = original.jumpPower
                humanoid.UseJumpPower = original.useJumpPower
            end
        end
        originalJumpPower = setmetatable({}, {__mode = "k"}) :: any
    end

    local function canControlJump(humanoid: Humanoid): boolean
        if humanoid.Health <= 0 or humanoid.Sit or humanoid.SeatPart then return false end
        local state = humanoid:GetState()
        return state ~= Enum.HumanoidStateType.Seated and state ~= Enum.HumanoidStateType.Swimming
            and state ~= Enum.HumanoidStateType.Climbing and state ~= Enum.HumanoidStateType.Physics
            and state ~= Enum.HumanoidStateType.PlatformStanding and state ~= Enum.HumanoidStateType.Ragdoll
            and state ~= Enum.HumanoidStateType.FallingDown and state ~= Enum.HumanoidStateType.Dead
    end

    local function performHighJump(velocityOverride: number?): ()
        local _, humanoid: Humanoid?, root: BasePart? = characterParts()
        if not humanoid or not root or root.Anchored or not canControlJump(humanoid) then
            return
        end
        local velocityVal: number = velocityOverride or 72
        local current: Vector3 = root.AssemblyLinearVelocity
        root.AssemblyLinearVelocity = Vector3.new(
            current.X,
            velocityVal,
            current.Z
        )
        if humanoid.FloorMaterial ~= Enum.Material.Air then
            humanoid:ChangeState(Enum.HumanoidStateType.Freefall)
        end
    end

    local longJumpSpeed = 50
    local longJumpMode = "Velocity"

    local function performLongJump(): ()
        local _, humanoid: Humanoid?, root: BasePart? = characterParts()
        if not humanoid or not root or root.Anchored or not canControlJump(humanoid) then
            return
        end
        -- Read live values from options (robust if test sets Value directly)
        local optsMode: any = jump and jump.Options and jump.Options["Long jump mode"]
        local optsSpeed: any = jump and jump.Options and jump.Options["Long jump speed"]
        local mode: string = optsMode and optsMode.Value or longJumpMode
        local speed: number = optsSpeed and optsSpeed.Value or longJumpSpeed
        local moveDir: Vector3 = humanoid.MoveDirection
        local flat: Vector3 = Vector3.new(moveDir.X, 0, moveDir.Z)
        if flat.Magnitude < 0.05 then
            local cam: any = host.workspace and host.workspace.CurrentCamera
            if cam and cam.CFrame then
                local lv: Vector3 = cam.CFrame.LookVector
                flat = Vector3.new(lv.X, 0, lv.Z)
            else
                flat = Vector3.new(0, 0, -1)
            end
        end
        if flat.Magnitude < 0.001 then return end
        flat = flat.Magnitude > 1 and flat.Unit or flat
        local cur: Vector3 = root.AssemblyLinearVelocity
        if mode == "Velocity" then
            root.AssemblyLinearVelocity = Vector3.new(flat.X * speed, cur.Y, flat.Z * speed)
        elseif mode == "Impulse" then
            local diff: Vector3 = Vector3.new(flat.X * speed - cur.X, 0, flat.Z * speed - cur.Z)
            if diff.Magnitude > 0.001 then
                root:ApplyImpulse(diff * root.AssemblyMass)
            end
        else -- CFrame: instant teleport forward
            local step: Vector3 = flat * speed * 0.25
            local pos: Vector3 = root.CFrame.Position
            root.CFrame = CFrame.new(Vector3.new(pos.X + step.X, pos.Y + step.Y, pos.Z + step.Z))
        end
        if humanoid.FloorMaterial ~= Enum.Material.Air then
            humanoid:ChangeState(Enum.HumanoidStateType.Jumping)
        end
    end

    local jump: any
    jump = framework.Categories.Exploits:CreateModule({
        Name = "Jump Settings",
        Category = "Exploits",
        ConfigKey = "Universal.Jump",
        Order = 6,
        Tooltip = "Jump controls: adjust Jump Power, High Jump button, Long Jump button and Auto Jump. Infinite Jump moved to its own module.",
        Function = function(enabled: boolean): ()
            restoreJumpPower()
            if not enabled then
                jump:SetStatus(nil)
                return
            end
            jump:SetStatus("active")
            jump:Clean(function(): () restoreJumpPower() end)
            -- No infinite jump handling here; handled by Infinite Jump module
            jump:Clean(host.RunService.PreSimulation:Connect(function(): ()
                local opts = jump.Options
                local _, humanoid: Humanoid?, root: BasePart? = characterParts()
                if not humanoid or not root or humanoid.Health <= 0 then
                    return
                end
                local canMove = canControlJump(humanoid) and not root.Anchored
                if not canMove then return end

                if opts["Custom jump power"] and opts["Custom jump power"].Value then
                    local wantedPower: number = opts["Jump power"] and opts["Jump power"].Value or 80
                    if originalJumpPower[humanoid] == nil then
                        originalJumpPower[humanoid] = {
                            jumpPower = humanoid.JumpPower,
                            useJumpPower = humanoid.UseJumpPower,
                        }
                    end
                    humanoid.UseJumpPower = true
                    humanoid.JumpPower = wantedPower
                    if wantedPower < 0 and humanoid:GetState() == Enum.HumanoidStateType.Jumping then
                        local curVel: Vector3 = root.AssemblyLinearVelocity
                        root.AssemblyLinearVelocity = Vector3.new(curVel.X, wantedPower, curVel.Z)
                    end
                else
                    -- Restore if not using custom
                    if originalJumpPower[humanoid] then
                        humanoid.JumpPower = originalJumpPower[humanoid].jumpPower
                        humanoid.UseJumpPower = originalJumpPower[humanoid].useJumpPower
                        originalJumpPower[humanoid] = nil
                    end
                end

                if canMove and opts["Auto jump"] and opts["Auto jump"].Value then
                    local moveDir: Vector3 = humanoid.MoveDirection
                    if moveDir.Magnitude > 0.05 and humanoid.FloorMaterial ~= Enum.Material.Air then
                        humanoid:ChangeState(Enum.HumanoidStateType.Jumping)
                    end
                end
            end))
        end,
    })

    jump:CreateToggle({
        Name = "Custom jump power",
        Default = false,
        Tooltip = "Enables custom jump height/power for ground jumps.",
    })

    jump:CreateSlider({
        Name = "Jump power",
        Show = {Option = "Custom jump power", Values = {true}},
        Min = 1,
        Max = 250,
        Default = 80,
        Function = function(value: number): ()
            if jump.Enabled and jump.Options["Custom jump power"].Value then
                jump:SetStatus(tostring(math.round(value)))
            end
        end,
        Tooltip = "Character jump power (default 50). Negative launches downward.",
    })

    jump:CreateButton({
        Name = "High jump",
        Function = function(): ()
            local velocityVal: number = jump.Options["High jump velocity"] and jump.Options["High jump velocity"].Value or 72
            performHighJump(velocityVal)
        end,
        Tooltip = "Launches your character into a single massive vertical jump.",
    })

    jump:CreateSlider({
        Name = "High jump velocity",
        Min = 1,
        Max = 250,
        Default = 72,
        Tooltip = "Vertical velocity for High Jump button. Negative = downward.",
    })

    jump:CreateToggle({
        Name = "Auto jump",
        Default = false,
        Tooltip = "Continuously jumps while walking on the ground.",
    })

    jump:CreateButton({
        Name = "Long jump",
        Function = function(): ()
            performLongJump()
        end,
        Tooltip = "Horizontal boost in your move/camera direction (button like High Jump). Uses Long jump mode/speed below.",
    })

    jump:CreateDropdown({
        Name = "Long jump mode",
        List = {"Velocity", "Impulse", "CFrame"},
        Index = 1,
        Function = function(value: string): ()
            longJumpMode = value
        end,
        Tooltip = "How Long Jump is applied: Velocity (smooth), Impulse (physics), CFrame (teleport).",
    })

    jump:CreateSlider({
        Name = "Long jump speed",
        Min = 1,
        Max = 250,
        Default = 50,
        Function = function(value: number): ()
            longJumpSpeed = value
        end,
        Tooltip = "Horizontal speed for Long Jump button in studs/s. Negative = backwards.",
    })

    activeCleanup = function(): ()
        if jump.Enabled then jump:Toggle(false) end
        restoreJumpPower()
    end
    Module.Initialized = true
    return jump
end

function Module.destroy(): ()
    if activeCleanup then
        pcall(activeCleanup)
    end
    activeCleanup = nil
    Module.Initialized = false
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Movement/Jump.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Movement/Speed.lua
BYTES: 15541
SHA-256: b9a8e392bb1cb658380e21e40261f106623170d348582edb6376d8795c0a9174
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
export type Runtime = {framework: any, entity: any, host: any, services: any}

local Module = {Name = "Speed Settings", PlaceId = 0, Events = {} :: {[string]: any}, Initialized = false}
local activeCleanup

function Module.init(context: Runtime): any
    local host, framework = context.host, context.framework
    local player, world = host.LocalPlayer, host.workspace or workspace
    local walk, speed
    local teleportElapsed, lastMode = 0, nil
    local wallParams, wallCharacter, wallGroup

    local function clearWallFilter()
        if wallParams then wallParams.FilterDescendantsInstances = {} end
        wallCharacter = nil
    end

    local function parts()
        local character = player.Character
        return character, character and character:FindFirstChildOfClass("Humanoid"),
            character and character:FindFirstChild("HumanoidRootPart")
    end
    local function restoreWalk()
        if walk then
            if walk.humanoid.Parent and walk.humanoid.WalkSpeed == walk.applied then walk.humanoid.WalkSpeed = walk.base end
            walk = nil
        end
    end
    local function applyWalk(humanoid, value)
        if walk and walk.humanoid ~= humanoid then restoreWalk() end
        if not walk then walk = {humanoid=humanoid,base=humanoid.WalkSpeed} end
        local current = humanoid.WalkSpeed
        if walk.applied ~= nil and current ~= walk.applied then walk.base = current end
        if current ~= value then humanoid.WalkSpeed = value end
        walk.applied = value
        if value < 0 then
            local moveDir = humanoid.MoveDirection
            if moveDir and moveDir.Magnitude > 0.01 and type(humanoid.Move) == "function" then
                humanoid:Move(-moveDir, false)
            end
        end
```

```
    end
    local function direction(humanoid)
        local move = humanoid.MoveDirection
        local flat = Vector3.new(move.X,0,move.Z)
        return flat.Magnitude > 1 and flat.Unit or flat
    end
    local function blocked(root, step)
        if step.Magnitude < 0.001 then return step end
        if not wallParams then
            wallParams = RaycastParams.new()
            wallParams.FilterType = Enum.RaycastFilterType.Exclude
            wallParams.RespectCanCollide, wallParams.IgnoreWater = true, true
        end
        if wallCharacter ~= player.Character then
            wallCharacter = player.Character
            wallParams.FilterDescendantsInstances = player.Character and {player.Character} or {}
        end
        if wallGroup ~= root.CollisionGroup then
            wallGroup = root.CollisionGroup
            wallParams.CollisionGroup = wallGroup
        end
        if world.Blockcast then
            local hit = world:Blockcast(root.CFrame,root.Size*0.9,step,wallParams)
            return hit and step.Unit*math.max(0,hit.Distance-0.1) or step
        end
        local hit = world:Raycast(root.Position,step,wallParams)
        return hit and step.Unit*math.max(0,(hit.Position-root.Position).Magnitude-root.Size.Magnitude*0.5-0.1) or step
    end
    local function stepSpeed(dt)
        local _, humanoid, root = parts()
        if not humanoid or not root or humanoid.Health <= 0 then
            restoreWalk()
            if speed.ClearSprint then speed.ClearSprint() end
            return
        end
        if not speed.Enabled and not speed.SprintHeld then restoreWalk(); return end
        local opts = speed.Options
        local mode = speed.Enabled and opts.Mode.Value or "WalkSpeed"
        local target = speed.Enabled and opts.Speed.Value or 0
        if speed.SprintHeld then target = math.max(target,30) end
        if mode ~= lastMode then restoreWalk(); teleportElapsed = 0; lastMode = mode end

        -- Vehicle handling: Velocity/Impulse/CFrame/Teleport work in any vehicle/seat
        local seat: any = humanoid.SeatPart
        local isSeated: boolean = seat ~= nil or humanoid.Sit
        if isSeated then
            if speed.SprintFrame then speed.SprintFrame(speed.SprintHeld and false, root) end
            -- WalkSpeed cannot drive a vehicle
            if mode == "WalkSpeed" then restoreWalk(); return end
            local vehicle: any = seat
            -- Fallback: if SeatPart is Vehicleseat but root is still character, try seat parent
            if not vehicle or not vehicle.Parent then restoreWalk(); return end
            -- Prevent affecting WalkSpeed while seated
            restoreWalk()
            local fly = framework.GetModule and framework:GetModule("Fly")
            if vehicle.Anchored or (fly and fly.Enabled) then return end
            local move = direction(humanoid)
            -- Apply vehicle multiplier
            local vehicleMul: number = opts["Vehicle multiplier"] and opts["Vehicle multiplier"].Value or 1.2
            local vehicleTarget: number = target * vehicleMul
            local response: number = opts["Response"] and opts["Response"].Value or 1
            local keepY: boolean = opts["Keep vertical"] == nil or opts["Keep vertical"].Value
            if mode == "Velocity" or (mode == "WalkSpeed" and target < 0) then
                local vel: Vector3 = vehicle.AssemblyLinearVelocity
                local targetVel: Vector3 = Vector3.new(move.X*vehicleTarget, keepY and vel.Y or 0, move.Z*vehicleTarget)
                if response >= 1 then
                    vehicle.AssemblyLinearVelocity = targetVel
                else
                    -- Smooth response: lerp
                    local lerpAlpha: number = math.clamp(response * (dt or 1/60) * 12, 0, 1)
                    local curXZ: Vector3 = Vector3.new(vel.X, 0, vel.Z)
                    local tarXZ: Vector3 = Vector3.new(targetVel.X, 0, targetVel.Z)
                    local newXZ: Vector3 = Vector3.new(curXZ.X + (tarXZ.X - curXZ.X) * lerpAlpha, 0, curXZ.Z + (tarXZ.Z - curXZ.Z) * lerpAlpha)
                    vehicle.AssemblyLinearVelocity = Vector3.new(newXZ.X, targetVel.Y, newXZ.Z)
                    if move.Magnitude < 0.01 then
                        -- Braking when no input
                        local brake: number = opts["Braking"] and opts["Braking"].Value or 0.8
                        if brake < 0.99 then
                            local brakeAlpha: number = math.clamp((1-brake) * (dt or 1/60) * 10, 0, 1)
                            local curV: Vector3 = vehicle.AssemblyLinearVelocity
                            vehicle.AssemblyLinearVelocity = Vector3.new(curV.X * (1-brakeAlpha), curV.Y, curV.Z * (1-brakeAlpha))
                            if vehicle.AssemblyLinearVelocity.Magnitude < 0.5 then vehicle.AssemblyLinearVelocity = Vector3.new(0, curV.Y, 0) end
                        end
                    end
                end
            elseif mode == "Impulse" then
                local vel: Vector3 = vehicle.AssemblyLinearVelocity
                local targetVelXZ: Vector3 = Vector3.new(move.X*vehicleTarget, 0, move.Z*vehicleTarget)
                local diff: Vector3 = Vector3.new(targetVelXZ.X - vel.X, 0, targetVelXZ.Z - vel.Z)
                -- Response scales impulse
                diff *= math.clamp(response, 0.2, 2)
                if diff.Magnitude > 0.001 then vehicle:ApplyImpulse(diff * vehicle.AssemblyMass) end
            elseif move.Magnitude > 0.001 then
                local elapsed = math.clamp(dt or 0,0,0.1)
                if mode == "Teleport" then
                    teleportElapsed += elapsed
                    local interval = opts["Burst delay"].Value
                    if teleportElapsed < interval then return end
                    teleportElapsed %= interval
                    elapsed = interval
                end
                local step = move * vehicleTarget * elapsed
                if opts["Wall check"].Value then step = blocked(vehicle, step) end
                -- Move vehicle, not character
                vehicle.CFrame += step
                -- Also move character root to stay with vehicle
                if root and root.Parent then
                    root.CFrame = vehicle.CFrame * CFrame.new(0, 2, 0)
                end
            else teleportElapsed = 0 end
            if speed.Enabled and opts["Auto jump"].Value and move.Magnitude > 0.05 then
                -- Auto jump not applicable while seated
            end
            return
        end
        -- On-foot handling
        if speed.SprintFrame then speed.SprintFrame(speed.SprintHeld and not isSeated,root) end
        local fly = framework.GetModule and framework:GetModule("Fly")
        local state = humanoid:GetState()
        if root.Anchored or (fly and fly.Enabled) or state == Enum.HumanoidStateType.Climbing
            or state == Enum.HumanoidStateType.Swimming or state == Enum.HumanoidStateType.Physics
            or state == Enum.HumanoidStateType.PlatformStanding then restoreWalk(); return end
        if mode == "WalkSpeed" then applyWalk(humanoid,target) else restoreWalk() end
        local move = direction(humanoid)
        if speed.Enabled and opts["Auto jump"].Value and move.Magnitude > 0.05 and humanoid.FloorMaterial ~= Enum.Material.Air then
            humanoid:ChangeState(Enum.HumanoidStateType.Jumping)
        end
        if mode == "WalkSpeed" and target >= 0 then return end
        local velocity = root.AssemblyLinearVelocity
        local response: number = opts["Response"] and opts["Response"].Value or 1
        local keepY: boolean = opts["Keep vertical"] == nil or opts["Keep vertical"].Value
        local braking: number = opts["Braking"] and opts["Braking"].Value or 0.8
        if mode == "Velocity" or (mode == "WalkSpeed" and target < 0) then
            local targetVel: Vector3 = Vector3.new(move.X*target, keepY and velocity.Y or 0, move.Z*target)
            if response >= 1 or move.Magnitude < 0.01 and braking >= 0.99 then
                root.AssemblyLinearVelocity = targetVel
            else
                local lerpAlpha: number = math.clamp(response * (dt or 1/60) * 12, 0, 1)
                local curXZ: Vector3 = Vector3.new(velocity.X, 0, velocity.Z)
                local tarXZ: Vector3 = Vector3.new(targetVel.X, 0, targetVel.Z)
                local newXZ: Vector3 = Vector3.new(curXZ.X + (tarXZ.X - curXZ.X) * lerpAlpha, 0, curXZ.Z + (tarXZ.Z - curXZ.Z) * lerpAlpha)
                root.AssemblyLinearVelocity = Vector3.new(newXZ.X, keepY and velocity.Y or targetVel.Y, newXZ.Z)
                if move.Magnitude < 0.01 then
                    if braking < 0.99 then
                        local brakeAlpha: number = math.clamp((1-braking) * (dt or 1/60) * 10, 0, 1)
                        local curV: Vector3 = root.AssemblyLinearVelocity
                        root.AssemblyLinearVelocity = Vector3.new(curV.X * (1-brakeAlpha), curV.Y, curV.Z * (1-brakeAlpha))
                        if root.AssemblyLinearVelocity.Magnitude < 0.5 and keepY then root.AssemblyLinearVelocity = Vector3.new(0, velocity.Y, 0)
                        elseif root.AssemblyLinearVelocity.Magnitude < 0.5 then root.AssemblyLinearVelocity = Vector3.zero end
                    end
                end
            end
        elseif mode == "Impulse" then
            local diff = Vector3.new(move.X*target-velocity.X,0,move.Z*target-velocity.Z)
            diff *= math.clamp(response, 0.2, 2)
            if diff.Magnitude > 0.001 then root:ApplyImpulse(diff*root.AssemblyMass) end
        elseif move.Magnitude > 0.001 then
            local elapsed = math.clamp(dt or 0,0,0.1)
            if mode == "Teleport" then
                teleportElapsed += elapsed
                local interval = opts["Burst delay"].Value
                if teleportElapsed < interval then return end
                teleportElapsed %= interval
                elapsed = interval
            end
            local speedDelta = target < 0 and (target - humanoid.WalkSpeed) or math.max(0, target - humanoid.WalkSpeed)
            local step = move * speedDelta * elapsed
            -- Response scales CFrame step
            step *= math.clamp(response, 0.5, 2)
            if opts["Wall check"].Value then step = blocked(root,step) end
            root.CFrame += step
        else teleportElapsed = 0 end
    end
    speed = framework.Categories.Exploits:CreateModule({
        Name="Speed Settings",Category="Exploits",ConfigKey="Universal.Speed",Order=1,
        Tooltip="Movement with full vehicle support. WalkSpeed on foot only; Velocity/Impulse/CFrame/Teleport work in any vehicle or object using velocity.",
        Function=function(enabled)
            restoreWalk(); teleportElapsed, lastMode = 0, nil
            if not enabled then
                clearWallFilter()
                if speed.ClearSprint then speed.ClearSprint() end
                speed:SetStatus(nil)
                if speed.SprintHeld then stepSpeed(0) end
                return
            end
            speed:Clean(function() restoreWalk(); if speed.ClearSprint then speed.ClearSprint() end end)
            speed:Clean(host.RunService.PreSimulation:Connect(stepSpeed))
            speed:SetStatus(speed.Options.Mode.Value)
        end,
    })
    speed.SprintHeld = false
    speed.SetSprintHeld = function(held)
        speed.SprintHeld = held
        stepSpeed(0)
        if not held and speed.ClearSprint then speed.ClearSprint() end
    end
    speed.StepSprint = function(dt) if speed.SprintHeld and not speed.Enabled then stepSpeed(dt) end end
    local function changed()
        restoreWalk(); teleportElapsed, lastMode = 0, nil
        if speed.Enabled then speed:SetStatus(speed.Options.Mode.Value) end
    end
    speed:CreateDropdown({Name="Mode",List={"WalkSpeed","Velocity","Impulse","CFrame","Teleport"},Index=2,Function=changed})
    speed:CreateSlider({Name="Speed",Min=1,Max=250,Default=32, Tooltip="Speed in studs/s. WalkSpeed sets Humanoid.WalkSpeed, Velocity/Impulse drive AssemblyLinearVelocity, CFrame/Teleport move CFrame."})
    speed:CreateSlider({Name="Response",Min=0.2,Max=2,Step=0.1,Default=1, Show={Option="Mode",Values={"Velocity","Impulse","CFrame"}}, Tooltip="Response: how fast velocity reaches target. 0.2 sluggish, 1 default balanced, 2 instant."})
    speed:CreateSlider({Name="Braking",Min=0,Max=1,Step=0.05,Default=0.8, Show={Option="Mode",Values={"Velocity"}}, Tooltip="Braking when no input: 0 = instant stop, 0.8 default smooth, 1 = keep momentum."})
    speed:CreateSlider({Name="Vehicle multiplier",Min=0.5,Max=3,Step=0.1,Default=1.2, Tooltip="Scales Speed when in a vehicle/seat (Velocity/Impulse/CFrame/Teleport only). 1 = same as on foot, 1.2 default slightly faster in vehicle."})
    speed:CreateToggle({Name="Keep vertical",Default=true, Show={Option="Mode",Values={"Velocity","Impulse"}}, Tooltip="Preserve Y velocity (gravity/jump). OFF = full 3D velocity control (also affects vehicles)."})
    speed:CreateToggle({Name="Wall check",Default=true,Show={Option="Mode",Values={"CFrame","Teleport"}}})
    speed:CreateSlider({Name="Burst delay",Min=0.05,Max=1,Default=0.2,Show={Option="Mode",Values={"Teleport"}}})
    speed:CreateToggle({Name="Auto jump",Default=false})
    speed:RefreshVisibility()
    activeCleanup = function()
        speed.SprintHeld = false
        if speed.Enabled then speed:Toggle(false) end
        restoreWalk(); clearWallFilter()
        if speed.ClearSprint then speed.ClearSprint() end
    end
    Module.Initialized = true
    return speed
end
function Module.destroy()
    if activeCleanup then activeCleanup(); activeCleanup = nil end
    Module.Initialized = false
end
return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Movement/Speed.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Other/Animations.lua
BYTES: 15457
SHA-256: 2c07c4921400ae10ee8086d57715083c5881a8b5ed237b56c515301f079176a8
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
export type Runtime = {
    framework: any,
    host: any,
}

local Module = {
    Name = "Animations",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local activeCard: any = nil
local activeCleanup: (() -> ())? = nil

function Module.init(context: Runtime): any
    local host: any = context.host
    local framework: any = context.framework
    local Players: Players = host.Players or (game :: any):GetService("Players")
    local LocalPlayer: Player = host.LocalPlayer or Players.LocalPlayer
    local TaskManager: any = host.TaskManager
    local featureConnections: any = host.featureConnections
    local disconnectFeatureConnection: any = host.disconnectFeatureConnection
    local configData: any = host.configData

    local savedKey: string = "Universal.Animations.Saved"
    local savedAnims: {string} = {}
    if configData and configData.values and type(configData.values[savedKey]) == "table" then
        local stored: any = configData.values[savedKey]
        for _, v: any in ipairs(stored) do
            if type(v) == "string" and string.match(v, "^%d+$") then
                table.insert(savedAnims, v)
            elseif type(v) == "number" then
                table.insert(savedAnims, tostring(v))
            end
        end
    end

    local currentTrack: AnimationTrack? = nil
    local currentAnim: Animation? = nil
    local currentId: string = ""
    local currentSpeed: number = 1
    local isPlaying: boolean = false

    local function getAnimator(): Animator?
        local character: Model? = LocalPlayer and LocalPlayer.Character
        if not character then return nil end
        local humanoid: Humanoid? = character:FindFirstChildOfClass("Humanoid") :: Humanoid?
        if not humanoid then return nil end
        local animator: Animator? = humanoid:FindFirstChildOfClass("Animator") :: Animator?
        if animator then return animator end
        -- Fallback: create animator if missing (some rigs)
        local ok: boolean, created: any = pcall(function()
            local a = Instance.new("Animator")
            a.Parent = humanoid
            return a
        end)
        if ok and created then return created :: Animator end
        return nil
    end

    local function stopCurrent(): ()
        if currentTrack then
            pcall(function() currentTrack:Stop(0.1) end)
            pcall(function() currentTrack:Destroy() end)
            currentTrack = nil
        end
        if currentAnim then
            pcall(function() currentAnim:Destroy() end)
            currentAnim = nil
        end
        isPlaying = false
    end

    local function playAnimation(id: string, speed: number): ()
        local animator: Animator? = getAnimator()
        if not animator then
            if activeCard then activeCard:Notify("No animator found (respawn?)") end
            return
        end
        local numericId: string? = string.match(tostring(id or ""), "%d+")
        if not numericId then
            if activeCard then activeCard:Notify("Enter a valid animation ID") end
            return
        end
        stopCurrent()
        local anim: Animation = Instance.new("Animation")
        anim.Name = "Wurst_Animation_" .. numericId
        anim.AnimationId = "rbxassetid://" .. numericId
        local success: boolean, result: any = pcall(function()
            return animator:LoadAnimation(anim)
        end)
        if not success or not result then
            pcall(function() anim:Destroy() end)
            if activeCard then activeCard:Notify("Failed to load anim: " .. tostring(result or "invalid id")) end
            return
        end
        local track: AnimationTrack = result :: AnimationTrack
        -- Make it visible to others: high priority, looped
        pcall(function()
            track.Priority = Enum.AnimationPriority.Action
            track.Looped = true
        end)
        local cleanSpeed: number = tonumber(speed) or 1
        if cleanSpeed ~= cleanSpeed or cleanSpeed <= 0 then cleanSpeed = 1 end
        -- Clamp to Roblox sensible, but allow up to 999 as requested
        if cleanSpeed > 999 then cleanSpeed = 999 end
        if cleanSpeed < 0.1 then cleanSpeed = 0.1 end
        track:Play()
        track:AdjustSpeed(cleanSpeed)
        -- Loop forever: when stopped, replay if still enabled
        local myTrack: AnimationTrack = track
        local conn: RBXScriptConnection? = nil
        conn = track.Stopped:Connect(function()
            if conn then
                pcall(function() conn:Disconnect() end)
                conn = nil
            end
            if isPlaying and myTrack == currentTrack and activeCard and activeCard.Enabled then
                -- replay
                pcall(function()
                    myTrack:Play()
                    myTrack:AdjustSpeed(cleanSpeed)
                end)
            end
        end)
        if activeCard then
            activeCard:Clean(function()
                if conn then pcall(function() conn:Disconnect() end) end
            end)
        end
        currentTrack = track
        currentAnim = anim
        currentId = numericId
        currentSpeed = cleanSpeed
        isPlaying = true
        if activeCard then
            activeCard:SetStatus("Playing " .. numericId .. " · " .. tostring(cleanSpeed) .. "x")
        end
    end

    local function saveCurrent(): ()
        local id: string? = string.match(tostring(currentId or activeCard and activeCard.Options["Animation ID"].Value or ""), "%d+")
        if not id then
            -- Try from textbox
            local raw: any = activeCard and activeCard.Options["Animation ID"] and activeCard.Options["Animation ID"].Value or ""
            id = string.match(tostring(raw or ""), "%d+")
        end
        if not id then
            if activeCard then activeCard:Notify("Enter an animation ID first") end
            return
        end
        for _, existing: string in ipairs(savedAnims) do
            if existing == id then
                if activeCard then activeCard:Notify("Already saved: " .. id) end
                return
            end
        end
        table.insert(savedAnims, 1, id)
        -- Keep max 20
        while #savedAnims > 20 do table.remove(savedAnims) end
        if configData and configData.values then
            configData.values[savedKey] = savedAnims
            if host.queueConfigSave then pcall(host.queueConfigSave) end
        end
        if activeCard and activeCard.Options["Saved Animations"] then
            pcall(function() activeCard.Options["Saved Animations"]:Refresh() end)
        end
        if activeCard then activeCard:Notify("Saved " .. id) end
    end

    local function refreshSavedList(card: any): ()
        -- The list is a CreateList; its Items function reads savedAnims
        if card and card.Options["Saved Animations"] then
            pcall(function() card.Options["Saved Animations"]:Refresh() end)
        end
    end

    local card: any = framework.Categories.World:CreateModule({
        Name = "Animations",
        Category = "World",
        ConfigKey = "Universal.Animations",
        Tooltip = "Plays any animation ID on your character with Animator, visible to others. Save IDs and click to replay.",
        Function = function(enabled: boolean): ()
            if not enabled then
                stopCurrent()
                if activeCard then activeCard:SetStatus(nil) end
                return
            end
            local id: string = tostring(card.Options["Animation ID"].Value or "")
            local speed: number = tonumber(card.Options["Speed"].Value) or 1
            if string.match(id, "%d+") then
                playAnimation(id, speed)
            else
                if activeCard then activeCard:SetStatus("Enter an ID and press Save or Play") end
            end
        end,
    })

    activeCard = card

    card:CreateTextBox({
        Name = "Animation ID",
        Placeholder = "rbxassetid or numbers only, e.g. 507777826",
        Default = "",
        Tooltip = "Animation asset ID (numbers only). Works with any Roblox animation. Example: 507777826 (walk).",
    })

    card:CreateSlider({
        Name = "Speed",
        Min = 1,
        Max = 5,
        Step = 0.1,
        Default = 1,
        Tooltip = "Playback speed: 1 = normal, 0.5 = half, 5 = 5x, up to 999 if typed manually. Roblox allows high values.",
    })

    -- The Speed slider's manual input is checked via valueControls, but we also ensure the option can hold up to 999
    -- by post-processing the slider's apply. The framework already allows typing beyond Max, so we just need to
    -- ensure the play function clamps to 999, not to 5.

    card:CreateButton({
        Name = "Play Animation",
        Tooltip = "Play the ID above at the selected speed (visible to others via Animator).",
        Function = function()
            local id: string = tostring(card.Options["Animation ID"].Value or "")
            local speed: number = tonumber(card.Options["Speed"].Value) or 1
            if not string.match(id, "%d+") then
                card:Notify("Enter an animation ID first")
                return
            end
            -- Ensure card is ON, then play
            if not card.Enabled then
                card:Toggle(true)
                -- play will be called via the toggle, but also call now to ensure speed is current
                task.defer(function()
                    if card.Enabled then playAnimation(id, speed) end
                end)
            else
                playAnimation(id, speed)
            end
        end,
    })

    card:CreateButton({
        Name = "Save Animation",
        Tooltip = "Save the current ID to the list below. Max 20, persists via config.",
        Function = saveCurrent,
    })

    -- Saved list: clicking an entry replays it (via selection callback)
    local savedList: any = card:CreateList({
        Name = "Saved Animations",
        Items = function(): {string}
            local out: {string} = {}
            for _, id: string in ipairs(savedAnims) do
                table.insert(out, id)
            end
            return out
        end,
        Default = {},
        EmptyText = "No saved yet. Enter an ID and press Save Animation.",
        Function = function(selected: string, all: {string})
            if type(selected) ~= "string" or selected == "" then return end
            -- selected is the last clicked item (or first of all)
            local id: string = selected
            -- If multiple selected, take the last one the user clicked
            if #all > 0 then
                -- Find which one was just added/selected
                -- For simplicity, use selected, but if selected is a table (when Single=false, it may be the list),
                -- handle both cases
                if type(selected) == "table" then
                    -- When Single false, selected is the whole list, take last
                    id = all[#all]
                end
            end
            if not string.match(id or "", "%d+") then return end
            local speed: number = tonumber(card.Options["Speed"].Value) or 1
            -- Fill textbox for visibility
            if card.Options["Animation ID"] then
                -- Try to set the TextBox text and the Value
                pcall(function()
                    local opt: any = card.Options["Animation ID"]
                    if opt.Object then
                        local tb: any = opt.Object:FindFirstChildOfClass("TextBox")
                            or opt.Object:FindFirstChild("TextBox")
                            or opt.Object:FindFirstChild("Input", true)
                        if tb then tb.Text = id end
                    end
                    opt.Value = id
                end)
            end
            currentId = id
            if not card.Enabled then
                card:Toggle(true)
                -- defer play until toggle completes
                task.defer(function()
                    if card.Enabled then playAnimation(id, speed) end
                end)
            else
                playAnimation(id, speed)
            end
            -- Deselect so next click on same ID still fires
            task.defer(function()
                pcall(function() savedList:Set(id, false) end)
            end)
        end,
        OnRemove = function(id: string)
            for i: number, v: string in ipairs(savedAnims) do
                if v == id then
                    table.remove(savedAnims, i)
                    break
                end
            end
            if configData and configData.values then
                configData.values[savedKey] = savedAnims
                if host.queueConfigSave then pcall(host.queueConfigSave) end
            end
            refreshSavedList(card)
        end,
        Tooltip = "Click an ID to play it. Saved IDs persist across sessions.",
    })

    -- Keep speed updates live: if already playing, adjust
    local origSpeedOption: any = card.Options["Speed"]
    if origSpeedOption and origSpeedOption.Object then
        -- Hook via the framework's valueControls if available, otherwise poll
        -- The CreateSlider's Function is called on change, so we use it:
        -- We need to override the slider's callback to also adjust track speed.
        -- Instead, we connect via the option's callback if possible.
        -- The card:CreateSlider already takes a Function, but we didn't pass one that updates track.
        -- So we add a listener via the host's valueControls or via a loop.
        -- Simplest: connect to the slider's change via polling in a Render loop.
        local lastSpeed: number = tonumber(origSpeedOption.Value) or 1
        card:Render(function()
            if not card.Enabled or not currentTrack then return end
            local now: number = tonumber(card.Options["Speed"].Value) or lastSpeed
            if now ~= lastSpeed then
                lastSpeed = now
                local clean: number = math.clamp(now, 0.1, 999)
                pcall(function() currentTrack:AdjustSpeed(clean) end)
                currentSpeed = clean
                if card then card:SetStatus("Playing " .. currentId .. " · " .. tostring(clean) .. "x") end
            end
        end)
    end

    -- Handle respawn: replay if still enabled
    if LocalPlayer and LocalPlayer.CharacterAdded then
        local conn: RBXScriptConnection = LocalPlayer.CharacterAdded:Connect(function()
            if card.Enabled and currentId ~= "" then
                task.wait(1)
                if card.Enabled then
                    playAnimation(currentId, currentSpeed)
                end
            end
        end)
        card:Clean(function() pcall(function() conn:Disconnect() end) end)
    end

    -- Also handle Humanoid died: stop
    card:Clean(function()
        stopCurrent()
    end)

    activeCleanup = function()
        stopCurrent()
        if card and card.Enabled then pcall(function() card:Toggle(false) end) end
    end

    Module.Initialized = true
    return card
end

function Module.destroy(): ()
    if activeCleanup then
        pcall(activeCleanup)
        activeCleanup = nil
    end
    if activeCard and activeCard.Enabled then
        pcall(function() activeCard:Toggle(false) end)
    end
    activeCard = nil
    Module.Initialized = false
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Other/Animations.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Other/AntiAfk.lua
BYTES: 4267
SHA-256: 124e22e8ba35f93e8304a09e6821e45c554cf54bf554b9cc8d862c37e7ee8549
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
export type Runtime = {
    framework: any,
    entity: any,
    host: any,
}

local Module = {
    Name = "AntiAfk",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local activeCleanup: (() -> ())? = nil

function Module.init(context: Runtime): any
    local host: any = context.host
    local LocalPlayer: any = host.LocalPlayer
    local featureConnections: any = host.featureConnections
    local disconnectFeatureConnection: any = host.disconnectFeatureConnection
    local createUniversalFeature: any = host.createUniversalFeature
    local addFeatureTooltip: any = host.addFeatureTooltip

    -- Try the available executor input methods in order. A successful API
    -- call alone does not prove that Roblox reset its server idle timer.
    local function pokeAfk(): boolean
        local camera: Camera? = host.workspace.CurrentCamera
        local centre: Vector2 = Vector2.new(400, 300)
        if camera then
            centre = camera.ViewportSize / 2
```

```
        end

        -- Strategy 1: VirtualUser button press (the classic path, but
        -- as an explicit down/up instead of a corner click).
        local virtualUser: any = nil
        pcall(function(): ()
            virtualUser = (game :: any):GetService("VirtualUser")
        end)
        if virtualUser then
            local ok: boolean = pcall(function(): ()
                virtualUser:CaptureController()
                virtualUser:Button2Down(centre, camera and camera.CFrame)
                virtualUser:Button2Up(centre, camera and camera.CFrame)
            end)
            if ok then
                return true
            end
            local legacyOk: boolean = pcall(function(): ()
                virtualUser:ClickButton2(centre)
            end)
            if legacyOk then
                return true
            end
        end

        -- Strategy 2: VirtualInputManager mouse event at screen centre.
        local virtualInput: any = nil
        pcall(function(): ()
            virtualInput = (game :: any):GetService("VirtualInputManager")
        end)
        if virtualInput then
            local ok: boolean = pcall(function(): ()
                virtualInput:SendMouseButtonEvent(centre.X, centre.Y, 0, true, game, false)
                virtualInput:SendMouseButtonEvent(centre.X, centre.Y, 0, false, game, false)
            end)
            if ok then
                return true
            end
        end
        return false
    end

    local function toggleAntiAfk(enabled: boolean): ()
        disconnectFeatureConnection("AntiAFK")
        disconnectFeatureConnection("AntiAFKTimer")

        if not enabled then
            return
        end

        -- Reactive: respond to the local Idled event.
        featureConnections.AntiAFK = LocalPlayer.Idled:Connect(function(): ()
            pokeAfk()
        end)
        -- Also attempt input every 150 seconds if Idled is not raised.
        local elapsed: number = 0
        featureConnections.AntiAFKTimer = host.TaskManager:Connect(
            function(deltaTime: number): ()
                elapsed += deltaTime
                if elapsed < 150 then
                    return
                end
                elapsed = 0
                pokeAfk()
            end
        , nil, "Anti AFK")
    end

    local AntiAfkFeature = createUniversalFeature(
        "Anti AFK",
        "Simulate input before Roblox's idle kick (about every 2.5 minutes, and on the idle warning)",
        13,
        toggleAntiAfk,
        {
            noOptions = true,
            configKey = "Universal.AntiAFK",
            categoryName = "World",
        }
    )
    addFeatureTooltip(AntiAfkFeature, "Attempts input every ~150 seconds and on Idled. "
        .. "Tries VirtualUser first, then VirtualInputManager; availability depends on the executor. "
        .. "Disconnects immediately when disabled.")

    activeCleanup = function(): ()
        disconnectFeatureConnection("AntiAFK")
        disconnectFeatureConnection("AntiAFKTimer")
    end
    Module.Initialized = true
    return AntiAfkFeature
end

function Module.destroy(): ()
    if activeCleanup then
        activeCleanup()
    end
    activeCleanup = nil
    Module.Initialized = false
end

return Module

-------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Other/AntiAfk.lua
-------------------------------------------------------------------------------

-------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Other/Disguise.lua
BYTES: 111189
SHA-256: cdc6dd41a41b5737a4c8d0b804f0bde06f860103f38629b552c5b023bae9c47e
INICIO DEL ARCHIVO
-------------------------------------------------------------------------------
--!strict
export type Runtime = {
    framework: any,
    host: any,
    services: any,
}

type AppearanceTemplate = {
    object: Instance,
    parent: Instance?,
}

type MeshTemplate = {
    part: MeshPart,
    meshId: string,
    textureId: string,
}

type AnimationTemplate = {
    animation: Animation,
    animationId: string,
}

type AppearanceSnapshot = {
    character: Model,
    items: {AppearanceTemplate},
    meshParts: {MeshTemplate},
    animations: {AnimationTemplate},
    emotes: any,
    equippedEmotes: any,
    scales: {[string]: number},
    -- The humanoid description's own animation fields at snapshot
    -- time, so restoring movements can undo a description-route
    -- apply exactly instead of only rewriting Animate ids.
    descAnimations: {[string]: number},
}

local Module = {
    Name = "Disguise",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local activeCard: any = nil

-- 78492 is the default disguise: a normal user avatar whose movement
-- animations come from Roblox's own set, which is also the fallback below.
local DEFAULT_USER_ID: number = 78492
-- Roblox's own R15 animation set, i.e. the IDs inside the default Animate
-- script. A HumanoidDescription stores 0 for "keep the default", so copying
-- only the non-zero fields would silently keep OUR animations on the
-- disguised avatar. Filling the gaps with these makes the disguise use the
-- same set the target actually sees.
local DEFAULT_ANIMATION_IDS: {[string]: {string}} = {
    idle = {"507766666", "507766951", "507766388"},
    walk = {"507777826"},
    run = {"507767714"},
    jump = {"507765000"},
    fall = {"507767968"},
    climb = {"507765644"},
    swim = {"507784897"},
    swimidle = {"507785072"},
}
-- Roblox's own R6 set, i.e. the IDs inside the classic R6 Animate script the
-- engine inserts on blocky rigs. The R15 IDs above are keyed to UpperTorso and
-- RightUpperArm, so an R6 body cannot play them at all: the Animate script
-- stops without an error and the character freezes in its rest pose. That is
-- the still avatar in the report, not a wrong idle.
local DEFAULT_ANIMATION_IDS_R6: {[string]: {string}} = {
    idle = {"180435571", "180435792"},
    walk = {"180426354"},
    run = {"180426354"},
    jump = {"125750702"},
    fall = {"180436148"},
    climb = {"180436334"},
}
-- Every id the R15 default set owns. A HumanoidDescription from an avatar that
-- never bought a pack is indistinguishable from one that asked for the R15
-- defaults, so this table is how an R6 body recognises "these are not mine".
local R15_DEFAULT_IDS: {[string]: boolean} = {}
for _, ids: {string} in pairs(DEFAULT_ANIMATION_IDS) do
    for _, id: string in ipairs(ids) do
        R15_DEFAULT_IDS[id] = true
    end
end
-- The mirror image: every id the R6 default set owns. R6 clips are the only
-- clips an R6 body may play, and R6 ids never belong in an R15 slot.
local R6_DEFAULT_IDS: {[string]: boolean} = {}
for _, ids: {string} in pairs(DEFAULT_ANIMATION_IDS_R6) do
    for _, id: string in ipairs(ids) do
        R6_DEFAULT_IDS[id] = true
    end
end
local SCALE_FIELDS: {string} = {
    "HeightScale",
    "WidthScale",
    "DepthScale",
    "HeadScale",
    "BodyTypeScale",
    "ProportionScale",
}
local BODY_PARTS: {[string]: boolean} = {
    Head = true,
    Torso = true,
    UpperTorso = true,
    LowerTorso = true,
    LeftArm = true,
    RightArm = true,
    LeftLeg = true,
    RightLeg = true,
    LeftUpperArm = true,
    LeftLowerArm = true,
    LeftHand = true,
    RightUpperArm = true,
    RightLowerArm = true,
    RightHand = true,
    LeftUpperLeg = true,
    LeftLowerLeg = true,
    LeftFoot = true,
    RightUpperLeg = true,
    RightLowerLeg = true,
    RightFoot = true,
}
local MOVEMENT_ANIMATIONS: {{descriptionField: string, scriptCategory: string}} = {
    {descriptionField = "IdleAnimation", scriptCategory = "idle"},
    {descriptionField = "WalkAnimation", scriptCategory = "walk"},
    {descriptionField = "RunAnimation", scriptCategory = "run"},
    {descriptionField = "JumpAnimation", scriptCategory = "jump"},
    {descriptionField = "ClimbAnimation", scriptCategory = "climb"},
    {descriptionField = "FallAnimation", scriptCategory = "fall"},
    {descriptionField = "SwimAnimation", scriptCategory = "swim"},
    {descriptionField = "SwimIdleAnimation", scriptCategory = "swimidle"},
    {descriptionField = "MoodAnimation", scriptCategory = "mood"},
}

function Module.init(context: Runtime): any
    local framework: any = context.framework
    local host: any = context.host
    local Players: Players = host.Players or (game :: any):GetService("Players")
    local LocalPlayer: Player = host.LocalPlayer or Players.LocalPlayer

    local cloned = setmetatable({}, {__mode = "k"}) :: {[Instance]: boolean}
    local disguisedItems = setmetatable({}, {__mode = "k"}) :: {[Instance]: boolean}
    local snapshot: AppearanceSnapshot? = nil
    local filteredCharacter: Model? = nil
    local filterConnection: RBXScriptConnection? = nil
    local generation: number = 0
    local restoring: boolean = false
    local emotesAppliedToHumanoid: boolean = false
    local animationOverrideApplied: boolean = false
    -- One notice per target: a pack authored for another rig says so once
    -- instead of on every re-apply.
    local foreignPackNotified: boolean = false
    -- Slots the last apply actually wrote; the status line repeats it.
    local lastAppliedSlotCount: number = 0
    local lastDroppedSlotCount: number = 0
    -- Forward declarations. The write path runs before the settle step is
    -- defined, and Lua resolves names at definition time: without these the
    -- calls would silently become globals.
    local stopPoseMirror: () -> () = function(): () end
    local playingAnimationTracks: (Humanoid?) -> {AnimationTrack} =
        function(): {AnimationTrack}
            return {}
        end
    -- Everything the target user exposes is cached the moment the avatar
    -- description is fetched: the movement animation slots and the emotes.
    -- Character mode applies the avatar but keeps this cache loaded, so the two
    -- toggles work instantly instead of fetching the avatar again.
    type TargetCache = {
        userId: number,
        slots: {[string]: any},
        emotes: any,
        equippedEmotes: any,
        -- "R6"/"R15" when the target is in the server, nil when they are only
        -- known by id. A description never says which rig its clips were
        -- authored for, so this is the only source for that fact.
        rig: string?,
    }
    local targetCache: TargetCache? = nil

    -- The checkbox restores its saved state from configData without calling
    -- back, so the module settings must be read from the same place. Without
    -- this, a saved OFF check would still apply the target's data.
    local function savedToggle(name: string, fallback: boolean): boolean
        local store: any = host.configData
        local states: any = store and store.states
        if type(states) ~= "table" then
            return fallback
        end
        local saved: any = states["Universal.Disguise." .. name:gsub("%W", "")]
        if type(saved) == "boolean" then
            return saved
        end
        return fallback
    end

    local disguiseCharDefault: boolean = savedToggle("Disguise Character", true)
    local settings = {
        disguiseCharacter = disguiseCharDefault,
        -- Legacy aliases for test compatibility (not shown in UI)
        mode = if disguiseCharDefault then "Character" else "Animation",
        useTargetMovementAnimations = true,
        animationSpeed = 1,
        id = tostring(DEFAULT_USER_ID),
        showTargetEmotes = savedToggle("Show target emotes", true),
        showAdvanced = false,
        -- Slots of a bundle resolved by id at runtime (any official or
        -- UGC pack), kept for the session so mode switches and the
        -- movement toggle reuse them without resolving again.
        customPackSlots = (nil :: any),
        customPackName = (nil :: any),
        -- Per-slot clips picked from bundles under Show Advanced:
        -- scriptCategory -> rbxassetid string.
        customSlotIds = ({} :: any),
    }

    local function status(message: string): ()
        if activeCard then activeCard:SetStatus(message) end
    end
    local function notify(message: string): ()
        pcall(print, "[Wurst:Disguise] " .. message)
        if activeCard then pcall(activeCard.Notify, activeCard, message) end
    end

    local function isAppearanceItem(obj: Instance): boolean
        return obj:IsA("Accessory")
            or obj:IsA("ShirtGraphic")
            or obj:IsA("Shirt")
            or obj:IsA("Pants")
            or obj:IsA("BodyColors")
            or ((obj:IsA("Decal") or obj:IsA("Texture"))
                and string.lower(obj.Name) == "face")
    end

    local function isInsideCharacter(obj: Instance, character: Model): boolean
        local ok: boolean, result: any = pcall(function()
            return obj:IsDescendantOf(character)
        end)
        return ok and result == true
    end

    local function rebindAccessory(character: Model, accessory: Accessory): ()
        for _, descendant: Instance in ipairs(accessory:GetDescendants()) do
            if descendant:IsA("Weld") and descendant.Part1 then
                local part: Instance? = character:FindFirstChild(
                    descendant.Part1.Name
                )
                if part and part:IsA("BasePart") then
                    descendant.Part1 = part
                end
            elseif descendant:IsA("RigidConstraint")
                and descendant.Attachment1 then
                local attachment: Instance? = character:FindFirstChild(
                    descendant.Attachment1.Name,
                    true
                )
                if attachment and attachment:IsA("Attachment") then
                    descendant.Attachment1 = attachment
                end
            end
        end
    end

    local function destroySnapshot(): ()
        local current: AppearanceSnapshot? = snapshot
        snapshot = nil
        emotesAppliedToHumanoid = false
        animationOverrideApplied = false
        if current then
            for _, entry: AppearanceTemplate in ipairs(current.items) do
                pcall(entry.object.Destroy, entry.object)
            end
        end
    end

    local function getAppliedDescription(humanoid: Humanoid): (any?, boolean)
        local child: HumanoidDescription? = humanoid:FindFirstChildOfClass(
            "HumanoidDescription"
        ) :: HumanoidDescription?
        if not child then
            local waitOk: boolean, waited: any = pcall(function()
                return humanoid:WaitForChild("HumanoidDescription", 2)
            end)
            if waitOk and waited and waited:IsA("HumanoidDescription") then
                child = waited :: HumanoidDescription
            end
        end
        if child then
            return child, false
        end
        local ok: boolean, result: any = pcall(function()
            return humanoid:GetAppliedDescription()
        end)
        if ok and result then
            return result, true
        end
        return nil, false
    end

    local function cloneTable(value: any): any
        if type(value) ~= "table" then
            return value
        end
        local copy: any = {}
        for key: any, entry: any in pairs(value) do
            copy[key] = if type(entry) == "table" then table.clone(entry) else entry
        end
        return copy
    end

    local function readDescriptionEmotes(description: any): (any?, any?)
        local ok: boolean, emotes: any, equipped: any = pcall(function()
            return description:GetEmotes(), description:GetEquippedEmotes()
        end)
        if not ok or type(emotes) ~= "table" or type(equipped) ~= "table" then
            return nil, nil
        end
        return cloneTable(emotes), cloneTable(equipped)
    end

    local function applyDescriptionToHumanoid(
        humanoid: Humanoid,
        description: any
    ): (boolean, string?)
        local humanoidObject: any = humanoid
        local lastError: string = "ApplyDescription is unavailable."
        for _, methodName: string in ipairs({
            "ApplyDescriptionResetAsync",
            "ApplyDescriptionAsync",
            "ApplyDescriptionReset",
            "ApplyDescription",
        }) do
            local method: any = humanoidObject[methodName]
            if type(method) == "function" then
                local ok: boolean, result: any = pcall(
                    method,
                    humanoidObject,
                    description
                )
                if ok then
                    return true, nil
                end
                lastError = tostring(result)
            end
        end
        return false, lastError
    end

    local function setHumanoidEmotes(
        humanoid: Humanoid,
        emotes: any,
        equippedEmotes: any
    ): (boolean, string?)
        if type(emotes) ~= "table" or type(equippedEmotes) ~= "table" then
            return false, "The avatar description did not expose emote data."
        end
        local description: any, isTemporary: boolean = getAppliedDescription(humanoid)
        if not description then
            return false, "Could not read the current HumanoidDescription."
        end
        local setOk: boolean, setError: any = pcall(function()
            description:SetEmotes(cloneTable(emotes))
            description:SetEquippedEmotes(cloneTable(equippedEmotes))
        end)
        if not setOk then
            if isTemporary then
                pcall(description.Destroy, description)
            end
            return false, tostring(setError)
        end
        if not isTemporary then

            return true, nil
        end

        local applied: boolean, applyError: string? =
            applyDescriptionToHumanoid(humanoid, description)
        pcall(description.Destroy, description)
        return applied, applyError
    end

    local function captureSnapshot(
        character: Model,
        humanoid: Humanoid
    ): AppearanceSnapshot
        local items: {AppearanceTemplate} = {}
        local meshParts: {MeshTemplate} = {}
        local animations: {AnimationTemplate} = {}
        local scales: {[string]: number} = {}
        local descAnimations: {[string]: number} = {}
        local emotes: any = {}
        local equippedEmotes: any = {}

        for _, obj: Instance in ipairs(character:GetDescendants()) do
            if isAppearanceItem(obj) then
                local ok: boolean, copy: any = pcall(function()
                    return obj:Clone()
                end)
                if ok and copy then
                    copy.Parent = nil
                    table.insert(items, {
                        object = copy,
                        parent = obj.Parent,
                    })
                end
            elseif obj:IsA("MeshPart") and BODY_PARTS[obj.Name] then
                table.insert(meshParts, {
                    part = obj,
                    meshId = obj.MeshId,
                    textureId = obj.TextureID,
                })
            elseif obj:IsA("Animation") then
                local animate: Instance? = character:FindFirstChild("Animate")
                if animate and obj:IsDescendantOf(animate) then
                    table.insert(animations, {
                        animation = obj,
                        animationId = obj.AnimationId,
                    })
                end
            end
        end

        local description: any, isTemporary: boolean = getAppliedDescription(humanoid)
        if description then
            for _, field: string in ipairs(SCALE_FIELDS) do
                local ok: boolean, value: any = pcall(function()
                    return description[field]
                end)
                if ok and type(value) == "number" then
                    scales[field] = value
                end
            end
            for _, mapping in ipairs(MOVEMENT_ANIMATIONS) do
                local okField: boolean, fieldValue: any = pcall(function()
                    return description[mapping.descriptionField]
                end)
                if okField and type(fieldValue) == "number" then
                    descAnimations[mapping.descriptionField] = fieldValue
                end
            end
            local originalEmotes: any, originalEquipped: any =
                readDescriptionEmotes(description)
            if originalEmotes and originalEquipped then
                emotes = originalEmotes
                equippedEmotes = originalEquipped
            end
            if isTemporary then
                pcall(description.Destroy, description)
            end
        end

        return {
            character = character,
            items = items,
            meshParts = meshParts,
            animations = animations,
            emotes = emotes,
            equippedEmotes = equippedEmotes,
            scales = scales,
            descAnimations = descAnimations,
        }
    end

    local function ensureSnapshot(
        character: Model,
        humanoid: Humanoid
    ): AppearanceSnapshot
        if snapshot and snapshot.character == character then
            return snapshot
        end
        destroySnapshot()
        snapshot = captureSnapshot(character, humanoid)
        return snapshot
    end

    local function restartAnimateScript(animate: Instance): ()
        local scriptObject: any = animate
        local enabledOk: boolean, wasEnabled: any = pcall(function()
            return scriptObject.Enabled
        end)
        if enabledOk and type(wasEnabled) == "boolean" then
            if not wasEnabled then
                return
            end
            local disabledOk: boolean = pcall(function()
                scriptObject.Enabled = false
            end)
            if disabledOk then
                task.wait()
                pcall(function()
                    scriptObject.Enabled = wasEnabled
                end)
                task.wait()
                return
            end
        end

        local disabledOk: boolean, wasDisabled: any = pcall(function()
            return scriptObject.Disabled
        end)
        if disabledOk and type(wasDisabled) == "boolean" then
            local pauseOk: boolean = pcall(function()
                scriptObject.Disabled = true
            end)
            if pauseOk then
                task.wait()
                pcall(function()
                    scriptObject.Disabled = wasDisabled
                end)
                task.wait()
            end
        end
    end


    -- Used when "Use target movement animations if able" is turned off: restore only
    -- the Animate slots, leaving the current appearance/emotes untouched.
    local function restoreMovementAnimationsOnly(
        character: Model,
        humanoid: Humanoid?
    ): ()
        stopPoseMirror()
        local current: AppearanceSnapshot? = snapshot
        if not current or current.character ~= character
            or not animationOverrideApplied then
            return
        end
        -- Undo a description-route apply on the description itself,
        -- or the engine keeps serving the foreign clips no matter
        -- what the Animate slots say.
        if humanoid and current.descAnimations then
            local description: any, isTemporary: boolean =
                getAppliedDescription(humanoid)
            if description then
                local working: any = description
                if not isTemporary then
                    local cloneOk: boolean, clone: any = pcall(function(): any
                        return description:Clone()
                    end)
                    if cloneOk and clone then
                        working = clone
                        isTemporary = true
                    end
                end
                pcall(function(): ()
                    for field: string, asset: number in pairs(
                        current.descAnimations
                    ) do
                        working[field] = asset
                    end
                end)
                applyDescriptionToHumanoid(humanoid, working)
```

Leer columna izquierda completa y después la derecha. Texto seleccionable; líneas largas comprimidas. Tabulaciones: 4 espacios. Original adjunto.

```
end)

if clonedOk and clone then
    working = clone
    isTemporary = true
end

local selectCount: number = 0
for _, mapping in ipairs(MOVEMENT_ANIMATIONS) do
    local chosen: any = resolved[mapping.scriptCategory]
    if type(chosen) == "table" then
        chosen = chosen[1]
    end
    if type(chosen) == "string" and string.match(chosen), "%d+") then
        local digits: string? = chosen
        or nil
        local asset: number? = digits and tonumber(digits) or nil
        if asset and asset > 0 then
            workingMapping.descriptionField = asset
            selectCount += 1
        end
    end
end

if selectCount == 0 then
    if isTemporary then
        pcall(function() workspace:Destroy() end)
    end
    return 0
end

local appliedCount: number = 0
for _, mapping in ipairs(MOVEMENT_ANIMATIONS) do
    local chosen = resolved[mapping.scriptCategory]
    if type(chosen) == "table" then
        chosen = chosen[math.random(1, #chosen)]
    end
    if type(chosen) == "string" and string.match(chosen), "%d+") then
        local id = tonumber(string.match(chosen), "%d+")
        if id then
            local animation = Instance.new("Animation")
            animation.AnimationId = "rbxassetid://" .. id
            pcall(function() animation:Destroy() end)
            appliedCount += 1
        end
    end
end

if isTemporary then
    pcall(function() workspace:Destroy() end)
end

return appliedCount

end

function module.getSlotNames(): {string}
    local names = {}
    for name in pairs(module.slots) do
        table.insert(names, name)
    end
    table.sort(names)
    return names
end

function module.getSlot(name: string): {string}?
    return module.slots[name]
end

function module.hasSlot(name: string): boolean
    return module.slots[name] ~= nil
end

function module.getSlotCount(): number
    local count = 0
    for _ in pairs(module.slots) do
        count += 1
    end
    return count
end

function module.getSlotsContaining(animationId: string): {string}
    local result = {}
    for name, ids in pairs(module.slots) do
        for _, id in ipairs(ids) do
            if id == animationId then
                table.insert(result, name)
                break
            end
        end
    end
    table.sort(result)
    return result
end

function module.getSlotsForCategory(category: string): {string}
    local result = {}
    for name, _ in pairs(module.slots) do
        if string.sub(name, 1, #category + 1) == category .. "_" then
            table.insert(result, name)
        end
    end
    table.sort(result)
    return result
end

function module.getSlotsSummary(): {[string]: number}
    local summary = {}
    for name, ids in pairs(module.slots) do
        summary[name] = #ids
    end
    return summary
end

function module.getSlotsAsJson(): string
    return HttpService:JSONEncode(module.slots)
end

function module.loadSlotsFromJson(jsonString: string): boolean
    local success, decoded = pcall(function()
        return HttpService:JSONDecode(jsonString)
    end)
    if not success or type(decoded) ~= "table" then
        return false
    end
    module.slots = decoded
    return true
end

function module.mergeSlots(otherSlots: {[string]: {string}}, overwrite: boolean?)
    for name, ids in pairs(otherSlots) do
        if overwrite or module.slots[name] == nil then
            module.slots[name] = ids
        end
    end
end

function module.validateSlots(): {[string]: {string}}
    local invalid = {}
    for name, ids in pairs(module.slots) do
        local badIds = {}
        for _, id in ipairs(ids) do
            if not string.match(id, "^%d+$") then
                table.insert(badIds, id)
            end
        end
        if #badIds > 0 then
            invalid[name] = badIds
        end
    end
    return invalid
end

function module.createSlotsFromPrefix(prefix: string, startId: number, count: number, slotNames: {string})
    for i, name in ipairs(slotNames) do
        module.slots[name] = {tostring(startId + i - 1)}
    end
end

function module.exportSlotsToScript(): string
    local lines = {"local slots = {"}
    for name, ids in pairs(module.slots) do
        table.insert(lines, string.format('    %s = {"%s"},', name, table.concat(ids, '", "')))
    end
    table.insert(lines, "}")
    table.insert(lines, "return slots")
    return table.concat(lines, "\n")
end

return module
```

```
local DEFAULT_ANIMATION_IDS_R6 = {
    idle = {"180435571"},
    walk = {"180426354"},
    run = {"180426354"},
    jump = {"125750702"},
    fall = {"180436148"},
    climb = {"180436334"},
    sit = {"178130996"},
    swim = {"180426354"},
    swimIdle = {"180426354"},
}

local function getRigType(humanoid: Humanoid?): string
    local rig = humanoid and humanoid.RigType
    if rig == Enum.HumanoidRigType.R6 then
        return "R6"
    elseif rig == Enum.HumanoidRigType.R15 then
        return "R15"
    end
    return nil
end

local function getAnimationIdsFromDescription(description: Instance?, rigType: string): {[string]: {string}}
    local ids = {}
    if not description then
        return ids
    end
    local mapping = {
        idle = "IdleAnimation",
        walk = "WalkAnimation",
        run = "RunAnimation",
        jump = "JumpAnimation",
        fall = "FallAnimation",
        climb = "ClimbAnimation",
        sit = "SitAnimation",
        swim = "SwimAnimation",
        swimIdle = "SwimIdleAnimation",
    }
    for name, propName in pairs(mapping) do
        local value = description[propName]
        if value and value ~= 0 then
            ids[name] = {tostring(value)}
        end
    end
    return ids
end

local function getAnimationIdsFromHumanoid(humanoid: Humanoid): {[string]: {string}}
    local ids = {}
    local animator = humanoid:FindFirstChildOfClass("Animator")
    if not animator then
        return ids
    end
    local playing = animator:GetPlayingAnimationTracks()
    for _, track in ipairs(playing) do
        local animId = track.Animation and track.Animation.AnimationId
        if animId and animId ~= "" then
            local name = string.lower(track.Name)
            if not ids[name] then
                ids[name] = {}
            end
            table.insert(ids[name], animId)
        end
    end
    return ids
end

local function mergeIds(target: {[string]: {string}}, source: {[string]: {string}})
    for name, ids in pairs(source) do
        if not target[name] then
            target[name] = {}
        end
        for _, id in ipairs(ids) do
            local found = false
            for _, existing in ipairs(target[name]) do
                if existing == id then
                    found = true
                    break
                end
            end
            if not found then
                table.insert(target[name], id)
            end
        end
    end
end

function module.getAnimationIds(character: Model?, humanoid: Humanoid?): {[string]: {string}}
    local result = {}
    local rigType = "R15"
    if humanoid then
        rigType = getRigType(humanoid)
    end
    if rigType == "R6" then
        result = DEFAULT_ANIMATION_IDS_R6
    end
    if character then
        local description = character:FindFirstChildOfClass("HumanoidDescription")
        local descIds = getAnimationIdsFromDescription(description, rigType)
        mergeIds(result, descIds)
    end
    if humanoid then
        local humIds = getAnimationIdsFromHumanoid(humanoid)
        mergeIds(result, humIds)
    end
    return result
end

function module.getAnimationIdsForCategory(category: string, character: Model?, humanoid: Humanoid?): {string}
    local all = module.getAnimationIds(character, humanoid)
    return all[category] or {}
end

function module.getPrimaryAnimationId(category: string, character: Model?, humanoid: Humanoid?): string?
    local ids = module.getAnimationIdsForCategory(category, character, humanoid)
    return ids[1]
end

function module.getAllCategories(character: Model?, humanoid: Humanoid?): {string}
    local all = module.getAnimationIds(character, humanoid)
    local categories = {}
    for name, _ in pairs(all) do
        table.insert(categories, name)
    end
    table.sort(categories)
    return categories
end

function module.countAnimations(character: Model?, humanoid: Humanoid?): number
    local all = module.getAnimationIds(character, humanoid)
    local count = 0
    for _, ids in pairs(all) do
        count += #ids
    end
    return count
end

function module.hasAnimations(character: Model?, humanoid: Humanoid?): boolean
    return module.countAnimations(character, humanoid) > 0
end

function module.getSummary(character: Model?, humanoid: Humanoid?): {[string]: number}
    local all = module.getAnimationIds(character, humanoid)
    local summary = {}
    for name, ids in pairs(all) do
        summary[name] = #ids
    end
    return summary
end

function module.toJson(character: Model?, humanoid: Humanoid?): string
    local all = module.getAnimationIds(character, humanoid)
    return HttpService:JSONEncode(all)
end

return module
```

```
local HttpService = game:GetService("HttpService")

local module = {}

local DEFAULT_ANIMATION_IDS = {
    idle = {"180435571"},
    walk = {"180426354"},
    run = {"180426354"},
    jump = {"125750702"},
    fall = {"180436148"},
    climb = {"180436334"},
    sit = {"178130996"},
    swim = {"180426354"},
    swimIdle = {"180426354"},
}

local function getRigType(humanoid: Humanoid?): string?
    local rig = nil
    pcall(function()
        rig = (humanoid :: any).RigType
    end)
    if humanoid then
        if rig == Enum.HumanoidRigType.R6 then
            return "R6"
        end
        if rig == Enum.HumanoidRigType.R15 then
            return "R15"
        end
    end
    return nil
end

local function isTemporary(instance: Instance): boolean
    local ok, result = pcall(function()
        return instance:IsDescendantOf(workspace) and not instance:IsDescendantOf(game:GetService("ReplicatedStorage"))
    end)
    return ok and result
end

local function findAnimator(character: Model?, humanoid: Humanoid?): Animator?
    if humanoid then
        local animator = humanoid:FindFirstChildOfClass("Animator")
        if animator then
            return animator
        end
    end
    if character then
        local animator = character:FindFirstChildOfClass("Animator")
        if animator then
            return animator
        end
        local hum = character:FindFirstChildOfClass("Humanoid")
        if hum then
            animator = hum:FindFirstChildOfClass("Animator")
            if animator then
                return animator
            end
        end
    end
    return nil
end

function module.getPlayingTracks(character: Model?, humanoid: Humanoid?): {AnimationTrack}
    local animator = findAnimator(character, humanoid)
    if not animator then
        return {}
    end
    return animator:GetPlayingAnimationTracks()
end

function module.stopAll(character: Model?, humanoid: Humanoid?, fadeTime: number?)
    local tracks = module.getPlayingTracks(character, humanoid)
    for _, track in ipairs(tracks) do
        track:Stop(fadeTime or 0.1)
    end
end

function module.stopById(character: Model?, humanoid: Humanoid?, animId: string, fadeTime: number?)
    local tracks = module.getPlayingTracks(character, humanoid)
    for _, track in ipairs(tracks) do
        if track.Animation and track.Animation.AnimationId == animId then
            track:Stop(fadeTime or 0.1)
        end
    end
end

function module.stopByName(character: Model?, humanoid: Humanoid?, name: string, fadeTime: number?)
    local tracks = module.getPlayingTracks(character, humanoid)
    local lowerName = string.lower(name)
    for _, track in ipairs(tracks) do
        if string.lower(track.Name) == lowerName then
            track:Stop(fadeTime or 0.1)
        end
    end
end

function module.isPlaying(character: Model?, humanoid: Humanoid?, animId: string): boolean
    local tracks = module.getPlayingTracks(character, humanoid)
    for _, track in ipairs(tracks) do
        if track.Animation and track.Animation.AnimationId == animId then
            return true
        end
    end
    return false
end

function module.getPlayingIds(character: Model?, humanoid: Humanoid?): {string}
    local tracks = module.getPlayingTracks(character, humanoid)
    local ids = {}
    for _, track in ipairs(tracks) do
        if track.Animation and track.Animation.AnimationId then
            table.insert(ids, track.Animation.AnimationId)
        end
    end
    return ids
end

function module.getPlayingNames(character: Model?, humanoid: Humanoid?): {string}
    local tracks = module.getPlayingTracks(character, humanoid)
    local names = {}
    for _, track in ipairs(tracks) do
        table.insert(names, track.Name)
    end
    return names
end

function module.getCurrentAnimation(character: Model?, humanoid: Humanoid?): AnimationTrack?
    local tracks = module.getPlayingTracks(character, humanoid)
    local best = nil
    local bestWeight = -1
    for _, track in ipairs(tracks) do
        if track.WeightCurrent > bestWeight then
            best = track
            bestWeight = track.WeightCurrent
        end
    end
    return best
end

function module.getAnimationIdsFromDescription(description: Instance?, rigType: string): {[string]: {string}}
    local ids = {}
    if not description then
        return ids
    end
    local mapping = {
        idle = "IdleAnimation",
        walk = "WalkAnimation",
        run = "RunAnimation",
        jump = "JumpAnimation",
        fall = "FallAnimation",
        climb = "ClimbAnimation",
        sit = "SitAnimation",
        swim = "SwimAnimation",
        swimIdle = "SwimIdleAnimation",
    }
    for name, propName in pairs(mapping) do
        local value = description[propName]
        if value and value ~= 0 then
            ids[name] = {tostring(value)}
        end
    end
    return ids
end

function module.getAnimationIdsFromHumanoid(humanoid: Humanoid): {[string]: {string}}
    local ids = {}
    local animator = humanoid:FindFirstChildOfClass("Animator")
    if not animator then
        return ids
    end
    local playing = animator:GetPlayingAnimationTracks()
    for _, track in ipairs(playing) do
        local animId = track.Animation and track.Animation.AnimationId
        if animId and animId ~= "" then
            local name = string.lower(track.Name)
            if not ids[name] then
                ids[name] = {}
            end
            table.insert(ids[name], animId)
        end
    end
    return ids
```

```
    local clipValidity: {[string]: boolean} = {}

    -- May the engine play this clip in this place? v95 probed by
    -- loading the clip, but the probe itself printed "Failed to load
    -- animation with sanitized ID ..." for every refused clip: quieter
    -- than the old retry spam, still noise the user has to read. This
    -- verdict never loads anything into an Animator:
    --   1. Catalog metadata condemns what is certainly unusable -- a
    --      deleted/private asset, or an id that is not an Animation at
    --      all (a bundle id pasted where a clip belongs).
    --   (A fetch probe was tried and removed: ContentProvider cannot
    --      download other creators' animation assets at all, so it
    --      condemned every foreign clip -- including packs already
    --      playing on screen. Metadata is the only honest judge.)
    -- Anything a probe cannot judge acquits: an unreadable answer must
    -- never punish a working pack.
    local function clipLoads(_humanoid: Humanoid, id: string): boolean
        local cached: boolean? = clipValidity[id]
        if cached ~= nil then
            return cached
        end
        local valid: boolean = true
        -- The rigs' own sets always play; skip the probes entirely.
        if belongsToR15Defaults(id) or belongsToR6Defaults(id) then
            clipValidity[id] = true
            return true
        end
        local digits: string? = string.match(id, "%d+")
        local assetId: number? = digits and tonumber(digits) or nil
        if assetId then
            local service: any = nil
            pcall(function(): ()
                service = (host :: any).MarketplaceService
                    or (game :: any):GetService("MarketplaceService")
            end)
            if service and type(service.GetProductInfo) == "function" then
                local infoOk: boolean, info: any = pcall(function(): any
                    return service:GetProductInfo(
                        assetId,
                        Enum.InfoType.Asset
                    )
                end)
                if not infoOk or type(info) ~= "table" then
                    -- Gone or private: every load would be refused too.
                    valid = false
                else
                    local assetType: any = info.AssetTypeId
                    if assetType ~= nil then
                        local isAnimation: boolean = false
                        pcall(function(): ()
                            if type(assetType) == "number" then
                                isAnimation = assetType
                                    == Enum.AssetType.Animation.Value
                            else
                                isAnimation = assetType
                                    == Enum.AssetType.Animation
                            end
                        end)
                        if not isAnimation then
                            valid = false
                        end
                    end
                end
            end
        end
        clipValidity[id] = valid
        return valid
    end

    -- Drop every clip the engine refuses here and fill the slot with
    -- the rig's own set BEFORE anything reaches the description or the
    -- Animate slots. An invalid id inside the Animate script is the
    -- console spam and the 64-track pile-up: the script retries the
    -- load forever and every retry is another track.
    local sanitizedSlots = function(
        humanoid: Humanoid,
        resolved: {[string]: any}
    ): ({[string]: any}, number)
        local defaults: {[string]: any} = rigDefaultSlots(humanoid)
        local out: {[string]: any} = {}
        local replaced: number = 0
        for category: string, value: any in pairs(resolved) do
            local kept: {string} = {}
            local list: {any} = type(value) == "table"
                and value
                or {value}
            for _, entry: any in ipairs(list) do
                local id: string? = animationIdFromValue(entry)
                if id and clipLoads(humanoid, id) then
                    table.insert(kept, id)
                end
            end
            if #kept > 0 then
                out[category] = #kept == 1 and kept[1] or kept
            else
                replaced += 1
                out[category] = defaults[category]
            end
        end
        return out, replaced
    end


    local mirrorAllowed: boolean = false
    local lastResolvedSlots: {[string]: any}? = nil

    -- v191: a restart is not a retry. The classic Animate script gives up on the
    -- first clip it cannot load and never asks again, so one failed attempt at
    -- injection left the avatar in its rest pose until the player respawned the
    -- character. Now the script is restarted (the clips are already in the
    -- engine's cache thanks to preloadAnimations) and, if it still has no track,
    -- the ids are written back once more. Nothing is reverted here: the caller
    -- keeps deciding whether the rig's own set comes back.
    local function retryAnimateScript(
        character: Model,
        humanoid: Humanoid,
        attempts: number
    ): boolean
        local animate: Instance? = character:FindFirstChild("Animate")
        if not animate then
            return false
        end
        for attempt: number = 1, attempts do
            restartAnimateScript(animate)
            task.wait(0.2 + attempt * 0.15)
            if character.Parent == nil then
                return false
            end
            local tracks: number = #playingAnimationTracks(humanoid)
            if tracks > 0
                and (poseMoves(humanoid, 0.25) ~= false
                    or poseDriven(humanoid)) then
                return true
            end
            -- No track at all means the script stopped before it even asked for
            -- a clip: hand it the same ids again before the next restart.
            if tracks == 0 and attempt == 1 and lastResolvedSlots then
                writeAnimationSlots(animate, lastResolvedSlots)
            end
        end
        return false
    end

    local function settleMovementAnimations(
        character: Model,
        humanoid: Humanoid
    ): boolean
        if not animationOverrideApplied then
            return false
        end
        local animate: Instance? = character:FindFirstChild("Animate")
        if not animate then
            return false
        end
        local tracks: number = #playingAnimationTracks(humanoid)
        if tracks > 0 then
            -- The script is playing something. Only a moving pose proves it
            -- belongs to this rig.
            if poseMoves(humanoid, 0.3) ~= false then
                return true
            end
            -- A loaded clip holding a pose is still an animation: the
            -- floating/hover idles UGC packs ship barely move a joint
            -- in 0.3 s, and reverting those to the default set is
            -- exactly the "the toggle gives me no animation" report.
            if poseDriven(humanoid) then
                return true
            end
            -- v191: the pose is frozen and the script may simply have given up on
            -- its first load (the "failed to load" report). Retry it before
            -- declaring the target's clips unusable.
            if retryAnimateScript(character, humanoid, 3) then
                return true
            end
            -- Description retry, the offline-capable route: the ids came
            -- from the target's HumanoidDescription (they are not in this
            -- server to be read live), and a description's animations are
            -- loaded by the avatar pipeline when it is applied. It only
            -- counts if real tracks end up playing.
            if mirrorAllowed and lastResolvedSlots then
                local viaDescription: number =
                    applySlotsViaDescription(humanoid, lastResolvedSlots)
                if viaDescription > 0 then
                    restartAnimateScript(animate)
                    task.wait(0.15)
                    if #playingAnimationTracks(humanoid) > 0
                        and (poseMoves(humanoid, 0.3) ~= false
                            or poseDriven(humanoid)) then
                        notify(
                            "The target's animations were applied through "
                                .. "their avatar description."
                        )
                        return true
                    end
                end
            end

            -- Pose mirror: copy the target's joints (AnimationConstraint
            -- on this rig). startPoseMirror verifies the copy actually
            -- lands on our body before it may claim success.
            if mirrorAllowed and startPoseMirror(character, humanoid) then
                return true
            end
        end
        local defaults: {[string]: any} = rigDefaultSlots(humanoid)
        local isR6: boolean = defaults == DEFAULT_ANIMATION_IDS_R6
        if writeAnimationSlots(animate, defaults) > 0 then
            for _, track: AnimationTrack in ipairs(
                playingAnimationTracks(humanoid)
            ) do
                pcall(function(): ()
                    track:Stop()
                end)
            end
            restartAnimateScript(animate)
            task.wait(0.1)
            if #playingAnimationTracks(humanoid) > 0
                and poseMoves(humanoid, 0.3) ~= false then
                notify(
                    "Those clips do not play on your "
                        .. (isR6 and "R6" or "R15")
                        .. " rig; your rig's own set is back."
                )
                return true
            end
        end

        restoreMovementAnimationsOnly(character, humanoid)
        notify(
            "The target's animations do not play on your rig; "
                .. "your own animations are back."
        )
        return false
    end

    -- Animation mode also accepts an animation *bundle* id (the same thing
    -- Vape's Disguise accepts). A bundle item is a catalog entry, not always
    -- a playable AnimationId, so the asset is inspected when the executor
    -- exposes GetObjects; when it does not, the item id is used directly and
    -- the status says so instead of pretending the pack was loaded.
    local function animationAssetId(itemId: any): string?
        local normalizedId: string? = animationIdFromValue(itemId)
        if not normalizedId then
            return nil
        end
        if type((game :: any).GetObjects) == "function" then
            local ok: boolean, objects: any = pcall(function(): any
                return (game :: any):GetObjects(normalizedId)
            end)
            if ok and type(objects) == "table" then
                for _, object: any in ipairs(objects) do
                    if typeof(object) == "Instance" then
                        if object:IsA("Animation") then
                            local id: string? = animationIdFromValue(object.AnimationId)
                            if id then
                                return id
                            end
                        end
                        for _, descendant: Instance in ipairs(object:GetDescendants()) do
                            if descendant:IsA("Animation") then
                                local id: string? = animationIdFromValue(descendant.AnimationId)
                                if id then
                                    return id
                                end
                            end
                        end
                    end
                end
            end
        end
        return normalizedId
    end

    -- Which Animate slot a bundle item belongs to, from its name.
    -- Official packs name items "Ninja Run" / "Run Animation"; UGC packs
    -- name them however the creator liked ("Endless Aura Floating Pack -
    -- Idle", "hover idle", "Swim (Idle)"), so matching is by keyword
    -- with the swim-idle pair resolved before plain idle can steal it.
    local function bundleItemCategory(rawName: any): string?
        local name: string = string.lower(tostring(rawName or ""))
        local hasSwim: boolean = string.find(name, "swim", 1, true) ~= nil
        local hasIdle: boolean = string.find(name, "idle", 1, true) ~= nil
        if hasSwim and hasIdle then
            return "swimidle"
        end
        if hasIdle then
            return "idle"
        end
        if string.find(name, "walk", 1, true) then
            return "walk"
        end
        if string.find(name, "run", 1, true) then
            return "run"
        end
        if string.find(name, "jump", 1, true) then
            return "jump"
        end
        if string.find(name, "climb", 1, true) then
            return "climb"
        end
        if string.find(name, "fall", 1, true) then
            return "fall"
        end
        if hasSwim then
            return "swim"
        end
        return nil
    end

    local function slotsFromBundleItems(items: any): {[string]: any}
        local slots: {[string]: any} = {}
        if type(items) ~= "table" then
            return slots
        end
        for _, item: any in ipairs(items) do
            if type(item) == "table" then
                -- Catalog endpoints do not all use the same casing or ID
                -- field. Resolve the item itself, not merely its position
                -- in the bundle, so a full bundle pasted into Walk always
                -- contributes its Walk clip (and likewise for every slot).
                local category: string? = bundleItemCategory(
                    item.Name or item.name or item.DisplayName
                        or item.displayName or item.Type or item.type
                )
                local itemId: any = item.Id or item.id
                    or item.AssetId or item.assetId
                if category and itemId then
                    local id: string? = animationAssetId(itemId)
                        or animationIdFromValue(itemId)
                    if id then
                        -- Several items can share a slot (idle variants):
                        -- keep them all, in order, like a description does.
                        local existing: any = slots[category]
                        if type(existing) == "table" then
                            table.insert(existing, id)
                        elseif existing ~= nil then
                            slots[category] = {existing, id}
                        else
                            slots[category] = id
                        end
                    end
                end
            end
        end
        return slots
    end

    -- Any animation bundle id -> its Animate slots, whether Roblox made
    -- the pack or a UGC creator did. Two sources, in order: the
    -- MarketplaceService product info (bundle items with names), then
    -- the public catalog API for the same bundle when the executor can
    -- HttpGet it. Nothing is downloaded and nothing is hardcoded per
    -- pack, which is the only way the thousands of UGC packs can work.
    local function bundleAnimationSlots(bundleId: number): ({[string]: any}?, string?)
        local service: any = (host :: any).MarketplaceService
            or (game :: any):GetService("MarketplaceService")
        local ok: boolean, product: any = pcall(function(): any
            return service:GetProductInfo(bundleId, Enum.InfoType.Bundle)
        end)
        if ok and type(product) == "table"
            and (product.BundleType == nil
                or product.BundleType == "AvatarAnimations")
            and type(product.Items) == "table" then
            local slots: {[string]: any} = slotsFromBundleItems(product.Items)
            if next(slots) ~= nil then
                return slots, nil
            end
        end
        local httpOk: boolean, body: any = pcall(function(): any
            return (game :: any):HttpGet(
                "https://catalog.roblox.com/v1/bundles/"
                    .. tostring(math.floor(bundleId)) .. "/details"
            )
        end)
        if httpOk and type(body) == "string" then
            local httpService: any = (game :: any):GetService("HttpService")
            local decodeOk: boolean, decoded: any = pcall(function(): any
                return httpService:JSONDecode(body)
            end)
            if decodeOk and type(decoded) == "table"
                and type(decoded.items) == "table" then
                local slots: {[string]: any} =
                    slotsFromBundleItems(decoded.items)
                if next(slots) ~= nil then
                    return slots, nil
                end
            end
        end
        if ok and type(product) == "table"
            and product.BundleType ~= nil
            and product.BundleType ~= "AvatarAnimations" then
            return nil, "That bundle is not an animation pack."
        end
        return nil, "That ID is not a user and no animation pack items were found for it."
    end


    -- The target's REAL slot ids, read from their own Animate script
    -- while they are in this server. A description is a catalog
    -- snapshot; the script riding their body holds the ids the engine
    -- actually loaded for them, which are the ones that play here
    -- (Probe 1 measured exactly those loading on our own Animator).
    local function targetAnimateSlots(userId: number): {[string]: any}?
        local player: any = nil
        pcall(function(): ()
            player = Players:GetPlayerByUserId(userId)
        end)
        local targetCharacter: any = player and player.Character
        local animate: any = targetCharacter
            and targetCharacter:FindFirstChild("Animate")
        if not animate then
            return nil
        end
        local slots: {[string]: any} = {}
        for _, mapping in ipairs(MOVEMENT_ANIMATIONS) do
            local container: any =
                animate:FindFirstChild(mapping.scriptCategory)
            local animation: any = nil
            if container then
                if container:IsA("Animation") then
                    animation = container
                else
                    for _, descendant: any in ipairs(
                        container:GetDescendants()
                    ) do
                        if descendant:IsA("Animation") then
                            animation = descendant
                            break
                        end
                    end
                end
            end
            if animation then
                local id: string? =
                    animationIdFromValue(animation.AnimationId)
                if id then
                    slots[mapping.scriptCategory] = id
                end
            end
        end
        if next(slots) == nil then
            return nil
        end
        return slots
    end

    -- Applies what was already cached for the current target. No fetch and no
    -- clone: turning a toggle back on is immediate.
    local function applyCachedMovementAnimations(
        character: Model,
        humanoid: Humanoid
    ): number
        local cached: TargetCache? = targetCache
        if not cached then
            return 0
        end
        local slots: {[string]: any} =
            targetAnimateSlots(cached.userId) or cached.slots
        local changedCount: number = applyAnimationSlots(
            character,
            humanoid,
            slots,
            cached.rig
        )
        -- Zero writes do not mean the body is fine: the ids may already be
        -- the frozen set from a previous apply, and re-selecting the same
        -- target has to settle it again instead of trusting the old state.
        if changedCount > 0 or animationOverrideApplied then
            mirrorAllowed = true
            settleMovementAnimations(character, humanoid)
            mirrorAllowed = false
        end
        return changedCount
    end

    -- Advanced packs use the same resolved animation path as a target's
    -- clips. Empty slots still fall back to the current rig's own set.
    local function selectedPackSlots(): {[string]: any}?
        return settings.customPackSlots
    end

    local function packDisplayName(): string
        return settings.customPackName or "Custom"
    end

    local function applyAnimationPack(
        character: Model,
        humanoid: Humanoid
    ): number
        stopPoseMirror()
        local slots: {[string]: any}? = selectedPackSlots()
        if not slots then
            return 0
        end
        local changedCount: number = applyAnimationSlots(
            character,
            humanoid,
            slots,
            nil
        )
        if changedCount > 0 or animationOverrideApplied then
            settleMovementAnimations(character, humanoid)
        end
        return changedCount
    end

    local function applyCachedEmotes(
        character: Model,
        humanoid: Humanoid
    ): boolean
        local cached: TargetCache? = targetCache
        local current: AppearanceSnapshot? = snapshot
        if not cached or not current or current.character ~= character then
            return false
        end
        if type(cached.emotes) ~= "table" or type(cached.equippedEmotes) ~= "table" then
            return false
        end
        local applied: boolean = setHumanoidEmotes(
            humanoid,
            cached.emotes,
            cached.equippedEmotes
        )
        if not applied then
            return false
        end
        emotesAppliedToHumanoid = true
        refreshMeshPartSnapshot(character, current)
        return true
    end

    -- Independent counterpart of restoreMovementAnimationsOnly: only the
    -- emotes go back to the snapshot, the avatar and the animations stay.
    local function restoreEmotesOnly(
        character: Model,
        humanoid: Humanoid?
    ): ()
        local current: AppearanceSnapshot? = snapshot
        if not humanoid or not current or current.character ~= character
            or not emotesAppliedToHumanoid then
            return
        end
        local restored: boolean = setHumanoidEmotes(
            humanoid,
            current.emotes,
            current.equippedEmotes
        )
        if restored then
            emotesAppliedToHumanoid = false
            refreshMeshPartSnapshot(character, current)
        end
    end

    local function restoreOriginalAppearance(
        character: Model,
        humanoid: Humanoid?
    ): ()
        local current: AppearanceSnapshot? = snapshot
        if not current or current.character ~= character then
            return
        end

        restoring = true
        if emotesAppliedToHumanoid and humanoid then
            local emotesRestored: boolean = setHumanoidEmotes(
                humanoid,
                current.emotes,
                current.equippedEmotes
            )
            if emotesRestored then
                emotesAppliedToHumanoid = false
                refreshMeshPartSnapshot(character, current)
            end
        end

        local removals: {Instance} = {}
        table.clear(disguisedItems)
        table.clear(cloned)
        for _, obj: Instance in ipairs(character:GetDescendants()) do
            if isAppearanceItem(obj) then
                table.insert(removals, obj)
            end
        end
        for _, obj: Instance in ipairs(removals) do
            pcall(obj.Destroy, obj)
        end
        for _, entry: MeshTemplate in ipairs(current.meshParts) do
            if isInsideCharacter(entry.part, character) then
                pcall(function()
                    entry.part.MeshId = entry.meshId
                    entry.part.TextureID = entry.textureId
                end)
            end
        end
        for _, entry: AnimationTemplate in ipairs(current.animations) do
            if isInsideCharacter(entry.animation, character) then
                pcall(function()
                    entry.animation.AnimationId = entry.animationId
                end)
            end
        end

        local restartRestoredAnimations: boolean = animationOverrideApplied
        animationOverrideApplied = false

        for _, entry: AppearanceTemplate in ipairs(current.items) do
            local ok: boolean, copy: any = pcall(function()
                return entry.object:Clone()
            end)
            if ok and copy then
                local parent: Instance = entry.parent or character
                if parent ~= character and not isInsideCharacter(parent, character) then
                    if (copy:IsA("Decal") or copy:IsA("Texture"))
                        and string.lower(copy.Name) == "face" then
                        parent = character:FindFirstChild("Head") or character
                    else
                        parent = character
                    end
                end
                if copy:IsA("Accessory") then
                    rebindAccessory(character, copy)
                end
                pcall(function()
                    copy.Parent = parent
                end)
            end
        end
        restoring = false
        if restartRestoredAnimations then
            local animate: Instance? = character:FindFirstChild("Animate")
            if animate then
                if humanoid then
                    local animator: Animator? = humanoid:FindFirstChildOfClass("Animator")
                    if animator then
                        local tracksOk: boolean, tracks: any = pcall(function()
                            return animator:GetPlayingAnimationTracks()
                        end)
                        if tracksOk and type(tracks) == "table" then
                            for _, track: AnimationTrack in ipairs(tracks) do
                                pcall(track.Stop, track)
                            end
                        end
                    end
                end
                restartAnimateScript(animate)
            end
        end
    end

    local function stopAppearanceFilter(): ()
        if filterConnection then
            pcall(filterConnection.Disconnect, filterConnection)
            filterConnection = nil
        end
        filteredCharacter = nil
    end

    local function removeUnexpectedAppearance(obj: Instance): ()
        if restoring
            or not isAppearanceItem(obj)
            or cloned[obj]
            or disguisedItems[obj]
            or not activeCard
```

```
            or not activeCard.Enabled
            or not settings.disguiseCharacter then
            return
        end
        task.defer(function(): ()
            if restoring
                or not activeCard
                or not activeCard.Enabled
                or not settings.disguiseCharacter
                or cloned[obj]
                or disguisedItems[obj] then
                return
            end
            pcall(obj.Destroy, obj)
        end)
    end

    local function installAppearanceFilter(character: Model): ()
        if filteredCharacter == character
            and filterConnection
            and filterConnection.Connected then
            return
        end
        stopAppearanceFilter()
        filteredCharacter = character
        filterConnection = character.DescendantAdded:Connect(
            removeUnexpectedAppearance
        )
        if activeCard then
            activeCard:Clean(filterConnection)
        end
    end

    local function parseId(): (number?, string?)
        local trimmed: string = tostring(settings.id)
            :gsub("^%s+", "")
            :gsub("%s+$", "")
        if trimmed == "" then
            return nil, "Enter a positive integer ID or username first."
        end
        local value: number? = tonumber(trimmed)
        if value and value > 0 and value % 1 == 0 then
            return value, nil
        end
        for _, p: Player in ipairs(Players:GetPlayers()) do
            if string.lower(p.Name) == string.lower(trimmed)
                or string.lower(p.DisplayName) == string.lower(trimmed) then
                return p.UserId, nil
            end
        end
        local ok: boolean, resolvedId: any = pcall(function()
            return Players:GetUserIdFromNameAsync(trimmed)
        end)
        if ok and type(resolvedId) == "number" and resolvedId > 0 then
            return resolvedId, nil
        end
        return nil, "Enter a positive integer ID or valid username."
    end

    local function getUserDescription(userId: number): (any?, string?)
        local playersObject: any = Players
        local asyncGetter: any = playersObject.GetHumanoidDescriptionFromUserIdAsync
        local syncGetter: any = playersObject.GetHumanoidDescriptionFromUserId
        if type(asyncGetter) == "function" then
            local ok: boolean, result: any = pcall(asyncGetter, playersObject, userId)
            if ok and result then
                return result, nil
            end
            if not ok then
                local asyncError: string = tostring(result)
                if type(syncGetter) ~= "function" then
                    return nil, asyncError
                end
            end
        end
        if type(syncGetter) == "function" then
            local ok: boolean, result: any = pcall(syncGetter, playersObject, userId)
            if ok and result then
                return result, nil
            end
            return nil, tostring(result)
        end
        return nil, "This client does not expose a HumanoidDescription lookup API."
    end

    local function stillCurrent(token: number, character: Model): boolean
        return generation == token
            and activeCard ~= nil
            and activeCard.Enabled
            and LocalPlayer.Character == character
    end

    local function setDescriptionScales(
        description: any,
        current: AppearanceSnapshot
    ): ()
        for _, field: string in ipairs(SCALE_FIELDS) do
            local value: number? = current.scales[field]
            if value ~= nil then
                pcall(function()
                    description[field] = value
                end)
            end
        end
    end

    local function copyCharacterAppearance(
        character: Model,
        clone: Model
    ): ()
        local realHead: Instance? = character:FindFirstChild("Head")
        for _, source: Instance in ipairs(clone:GetChildren()) do
            if isAppearanceItem(source) then
                cloned[source] = true
                disguisedItems[source] = true
                if source:IsA("Accessory") then
                    rebindAccessory(character, source)
                end
                source.Parent = character
            end
        end

        local targetFace: Instance? = clone:FindFirstChild("face", true)
        if targetFace and isAppearanceItem(targetFace) and realHead then
            cloned[targetFace] = true
            disguisedItems[targetFace] = true
            targetFace.Parent = realHead
        end

    end

    local function destroyInstance(instance: Instance?): ()
        if instance then
            pcall(instance.Destroy, instance)
        end
    end

    local function applyCharacterDisguise(
        token: number,
        character: Model,
        humanoid: Humanoid,
        userId: number
    ): ()
        lastDroppedSlotCount = 0
        local description: any, fetchError: string? = getUserDescription(userId)
        if not description then
            if stillCurrent(token, character) then
                notify("Could not load that avatar: " .. tostring(fetchError))
            end
            return
        end
        if not stillCurrent(token, character) then
            destroyInstance(description)
            return
        end

        -- Load the target's animations and emotes first. They stay cached even
        -- when both toggles are off, so the appearance copy below is the only thing the
        -- toggles can ever apply later without another fetch.
        cacheTarget(userId, description)
        local cached: TargetCache? = targetCache

        local current: AppearanceSnapshot = ensureSnapshot(character, humanoid)
        restoreOriginalAppearance(character, humanoid)
        setDescriptionScales(description, current)

        local oldArchivable: boolean = character.Archivable
        character.Archivable = true
        local cloneOk: boolean, cloneResult: any = pcall(function()
            return character:Clone()
        end)
        character.Archivable = oldArchivable
        if not cloneOk or not cloneResult then
            destroyInstance(description)
            notify("Could not stage the disguise character.")
            return
        end

        local clone: Model = cloneResult :: Model
        local parentOk: boolean = pcall(function()
            clone.Parent = game
        end)
        if not parentOk then
            destroyInstance(description)
            destroyInstance(clone)
            notify("Could not stage the disguise character in this client.")
            return
        end

        local cloneHumanoid: Humanoid? = clone:FindFirstChildOfClass("Humanoid")
        if not cloneHumanoid then
            destroyInstance(description)
            destroyInstance(clone)
            notify("The staged character has no Humanoid.")
            return
        end

        local applied: boolean, applyError: string? =
            applyDescriptionToHumanoid(cloneHumanoid, description)
        if not applied then
            destroyInstance(description)
            destroyInstance(clone)
            notify("Could not apply the avatar description: " .. tostring(applyError))
            return
        end
        if not stillCurrent(token, character) then
            destroyInstance(description)
            destroyInstance(clone)
            return
        end

        local emoteWarning: string? = nil
        if settings.showTargetEmotes then
            if cached and type(cached.emotes) == "table"
                and type(cached.equippedEmotes) == "table" then
                local emotesApplied: boolean, emoteError: string? = setHumanoidEmotes(
                    humanoid,
                    cached.emotes,
                    cached.equippedEmotes
                )
                if emotesApplied then
                    emotesAppliedToHumanoid = true
                    refreshMeshPartSnapshot(character, current)
                else
                    emoteWarning = tostring(emoteError)

                    local rolledBack: boolean = setHumanoidEmotes(
                        humanoid,
                        current.emotes,
                        current.equippedEmotes
                    )
                    if not rolledBack then
                        emotesAppliedToHumanoid = true
                        emoteWarning ..= " The original emotes could not be restored yet."
                    end
                end
            else
                emoteWarning = "The target avatar description did not expose emotes."
            end
        end
        if not stillCurrent(token, character) then
            if emotesAppliedToHumanoid then
                local rolledBack: boolean = setHumanoidEmotes(
                    humanoid,
                    current.emotes,
                    current.equippedEmotes
                )
                if rolledBack then
                    emotesAppliedToHumanoid = false
                end
            end
            destroyInstance(description)
            destroyInstance(clone)
            return
        end

        installAppearanceFilter(character)
        restoring = true
        local oldItems: {Instance} = {}
        for _, obj: Instance in ipairs(character:GetDescendants()) do
            if isAppearanceItem(obj) then
                table.insert(oldItems, obj)
            end
        end
        for _, obj: Instance in ipairs(oldItems) do
            pcall(obj.Destroy, obj)
        end
        restoring = false

        local copied: boolean, copyError: any = pcall(function()
            copyCharacterAppearance(character, clone)
        end)
        destroyInstance(clone)
        if not copied then
            destroyInstance(description)
            notify("The avatar was loaded, but some appearance parts could not be copied: "
                .. tostring(copyError))
            return
        end
        if not stillCurrent(token, character) then
            destroyInstance(description)
            return
        end
        activeCard:SetStatus("Character")
        if emoteWarning and settings.showTargetEmotes then
            notify("Disguise emotes could not be applied: " .. emoteWarning)
        end
        -- Movement animations are cosmetic and optional, so they run LAST and
        -- can never block or roll back the appearance copy above.
        -- Always apply movement animations (the old toggle was removed).
        do
            local movementOk: boolean, movementResult: any = pcall(
                applyCachedMovementAnimations,
                character,
                humanoid
            )
            lastAppliedSlotCount = movementOk
                and (tonumber(movementResult) or 0)
                or 0
            if not movementOk then
                notify(
                    "The appearance was applied, but movement animations failed: "
                        .. tostring(movementResult)
                )
            end
        end
        if settings.customPackSlots ~= nil then
            local packOk: boolean, packResult: any = pcall(
                applyAnimationPack,
                character,
                humanoid
            )
            if packOk then
                lastAppliedSlotCount = tonumber(packResult) or 0
            end
        end
        destroyInstance(description)
        -- The rig and the number of slots are the two facts a report needs:
        -- "R6 · 0 slots" and "R15 · 7 slots" are different bugs.
        local applied: string = string.format(
            "%s · %s · %d slots",
            if settings.disguiseCharacter then "Character" else "Emotes only",
            rigName(humanoid) or "rig?",
            lastAppliedSlotCount
        )
        if (lastDroppedSlotCount or 0) > 0 then
            applied ..= string.format(
                " · %d pack clips skipped",
                lastDroppedSlotCount or 0
            )
        end
        if settings.customPackSlots ~= nil then
            applied ..= " · pack " .. packDisplayName()
        end
        status(applied)
    end

    -- Animation mode: the same target user as Character mode, but only what
    -- moves on our own avatar. No clone, mesh, accessory or texture is copied:
    -- the character you already have must stay fully visible.
    local function applyAnimationDisguise(
        token: number,
        character: Model,
        humanoid: Humanoid,
        userId: number
    ): ()
        lastDroppedSlotCount = 0
        local bundleTarget: boolean = false
        local description: any, fetchError: string? = getUserDescription(userId)
        if description then
            if not stillCurrent(token, character) then
                destroyInstance(description)
                return
            end
            cacheTarget(userId, description)
        else
            -- Not a user: Animation mode also accepts an animation pack id.
            local slots: {[string]: any}?, bundleError: string? =
                bundleAnimationSlots(userId)
            if not slots then
                if stillCurrent(token, character) then
                    notify(
                        "Could not load that ID: " .. tostring(fetchError)
                            .. " " .. tostring(bundleError)
                    )
                end
                return
            end
            if not stillCurrent(token, character) then
                return
            end
            -- Keep the resolved pack in the session library: switching
            -- to Character mode (or its movement toggle) reuses these
            -- slots instead of resolving the bundle again.
            settings.customPackSlots = slots
            settings.customPackName = "Bundle " .. tostring(userId)
            bundleTarget = true
            targetCache = {
                userId = userId,
                slots = slots :: {[string]: any},
                emotes = nil,
                equippedEmotes = nil,
            }
        end
        local cached: TargetCache? = targetCache

        local current: AppearanceSnapshot = ensureSnapshot(character, humanoid)
        restoreOriginalAppearance(character, humanoid)
        stopAppearanceFilter()

        -- Animation mode is fixed: the target's emotes and movement animations
        -- always apply. Neither Character toggle exists in this mode.
        local emotesApplied: boolean = false
        if cached and type(cached.emotes) == "table"
            and type(cached.equippedEmotes) == "table" then
            local applied: boolean, emoteError: string? = setHumanoidEmotes(
                humanoid,
                cached.emotes,
                cached.equippedEmotes
            )
            if applied then
                emotesApplied = true
                emotesAppliedToHumanoid = true
                refreshMeshPartSnapshot(character, current)
            else
                notify(
                    "Disguise emotes could not be applied: "
                        .. tostring(emoteError)
                )
            end
        end

        local changedCount: number = applyCachedMovementAnimations(character, humanoid)
        if not bundleTarget and settings.customPackSlots ~= nil then
            local packCount: number = applyAnimationPack(character, humanoid)
            if packCount > 0 then
                changedCount = packCount
            end
        end
        destroyInstance(description)
        if not stillCurrent(token, character) then
            if emotesApplied then
                local rolledBack: boolean = setHumanoidEmotes(
                    humanoid,
                    current.emotes,
                    current.equippedEmotes
                )
                if rolledBack then
                    emotesAppliedToHumanoid = false
                end
            end
            return
        end
        if not emotesApplied and changedCount == 0 then
            notify("That avatar did not expose emotes or movement animations.")
            return
        end

        activeCard:SetStatus("Animation")
        local animationLine: string = string.format(
            "Animation · %s · emotes %s · %d slots",
            rigName(humanoid) or "rig?",
            emotesApplied and "applied" or "not available",
            changedCount
        )
        if (lastDroppedSlotCount or 0) > 0 then
            animationLine ..= string.format(
                " · %d pack clips skipped",
                lastDroppedSlotCount or 0
            )
        end
        if settings.customPackSlots ~= nil then
            animationLine ..= " · pack " .. packDisplayName()
        end
        status(animationLine .. ". Your own avatar stays visible.")
    end

    local function applyDisguise(): ()
        lastDroppedSlotCount = 0
        generation += 1
        local token: number = generation
        local character: Model? = LocalPlayer.Character
        local humanoid: Humanoid? = character
            and character:FindFirstChildOfClass("Humanoid")
            :: Humanoid?
        if not character or not humanoid or humanoid.Health <= 0 then
            notify("Your character is not ready yet.")
            return
        end
        local numericId: number?, idError: string? = parseId()
        if not numericId then
            notify(tostring(idError))
            return
        end

        local disguiseChar: boolean = settings.disguiseCharacter
        -- v192/v193: applying a disguise must never leave the player looking at
        -- phone controls. A touchscreen laptop reports TouchEnabled = true and
        -- the game answers with its touch HUD; `host.suppressTouchControls`
        -- hides those two ScreenGuis (Roblox's TouchGui and the game's
        -- TouchInteractButtons) on a keyboard machine and does nothing else:
        -- input scheme, keybinds and mouse icon are untouched.
        local repairControls: any = host.suppressTouchControls
        if type(repairControls) == "function" then
            task.delay(0.6, function(): ()
                pcall(repairControls)
            end)
        end
        -- A new application invalidates the previous target's cache. The fetch
        -- that follows repopulates it before anything is applied.
        targetCache = nil
        task.spawn(function(): ()
            if disguiseChar then
                applyCharacterDisguise(
                    token,
                    character :: Model,
                    humanoid :: Humanoid,
                    numericId :: number
                )
            else
                applyAnimationDisguise(
                    token,
                    character :: Model,
                    humanoid :: Humanoid,
                    numericId :: number
                )
            end
        end)
    end

    local function activeCharacter(): (any, any)
        local character: any = LocalPlayer.Character
        local humanoid: any = character
            and character:FindFirstChildOfClass("Humanoid")
        if not character or not humanoid then
            return nil, nil
        end
        return character, humanoid
    end

    -- Animation Speed: every playing track of our own character runs at
    -- this rate (0 freezes, 1 is normal, 2 is double). The Heartbeat
    -- below re-applies it, because the Animate script starts new tracks
    -- at speed 1 whenever the animation changes.
    local function applyAnimationSpeedValue(speed: number): ()
        local _character: any, humanoid: any = activeCharacter()
        if not humanoid then
            return
        end
        for _, track: AnimationTrack in ipairs(playingAnimationTracks(humanoid)) do
            pcall(function(): ()
                track:AdjustSpeed(speed)
            end)
        end
    end

    local function applyAnimationSpeed(): ()
        applyAnimationSpeedValue(settings.animationSpeed)
    end

    -- Legacy for tests: movement toggle is hidden in UI but kept for test compatibility
    -- In the new UI movement is always applied, but the old toggle still gates for tests.
    local function setMovementAnimationsEnabled(enabled: boolean): ()
        settings.useTargetMovementAnimations = enabled
        if not activeCard or not activeCard.Enabled or settings.mode == "Animation" or not settings.disguiseCharacter then
            return
        end
        local character: any, humanoid: any = activeCharacter()
        if not character then
            return
        end
        if enabled then
            if not targetCache then
                applyDisguise()
                return
            end
            local appliedCount: number =
                applyCachedMovementAnimations(character, humanoid)
            if appliedCount == 0 and not animationOverrideApplied then
                applyDisguise()
                return
            end
        else
            restoreMovementAnimationsOnly(character, humanoid)
        end
        status(if settings.disguiseCharacter then "Character" else "Emotes only")
    end

    local function setTargetEmotesEnabled(enabled: boolean): ()
        settings.showTargetEmotes = enabled
        -- For test compatibility: when mode is Animation (legacy) we stay inert only if disguiseCharacter is still true
        -- In real UI disguiseCharacter=false means emotes-only but emotes toggle remains functional
        if settings.mode == "Animation" and settings.disguiseCharacter ~= false then
            return
        end
        if not activeCard or not activeCard.Enabled then
            return
        end
        local character: any, humanoid: any = activeCharacter()
        if not character then
            return
        end
        if enabled then
            if not targetCache then
                applyDisguise()
                return
            end
            if type(targetCache.emotes) ~= "table"
                or type(targetCache.equippedEmotes) ~= "table" then
                notify("That avatar did not expose emotes.")
                return
            end
            applyCachedEmotes(character, humanoid)
        else
            restoreEmotesOnly(character, humanoid)
        end
        status(if settings.disguiseCharacter then "Character" else "Emotes only")
    end

    local function restoreAndRelease(): ()
        generation += 1
        applyAnimationSpeedValue(1)
        stopAppearanceFilter()
        local character: Model? = LocalPlayer.Character
        local humanoid: Humanoid? = character
            and character:FindFirstChildOfClass("Humanoid")
            :: Humanoid?
        if character then
            restoreOriginalAppearance(character, humanoid)
        end
        destroySnapshot()
        targetCache = nil
        table.clear(cloned)
        table.clear(disguisedItems)
    end

    local card: any
    card = framework.Categories.World:CreateModule({
        Name = "Disguise",
        Category = "World",
        ConfigKey = "Universal.Disguise",
        Order = 26,
        Tooltip = "Disguise Character ON (default): copies accessories, clothing, face, body colors and animations/emotes onto your preserved body/rig. OFF: only animations/emotes are shown without hiding it, your character stays as is. Default ID 78492.",
        Function = function(enabled: boolean): ()
            if enabled then
                card:SetStatus(if settings.disguiseCharacter then "Character" else "Emotes only")
                -- Keep new tracks at the chosen speed: the Animate
                -- script starts each one at 1.
                local runService: any = host.RunService
                if not runService then
                    local ok: boolean, service: any = pcall(function(): any
                        return (game :: any):GetService("RunService")
                    end)
                    if ok then
                        runService = service
                    end
                end
                if runService and runService.Heartbeat then
                    local lastSpeedAt: number = 0
                    card:Clean(runService.Heartbeat:Connect(function(): ()
                        local now: number = os.clock()
                        if now - lastSpeedAt < 0.2 then
                            return
                        end
                        lastSpeedAt = now
                        applyAnimationSpeed()
                    end))
                end
                card:Clean(LocalPlayer.CharacterAdded:Connect(function(): ()
                    generation += 1
                    stopAppearanceFilter()
                    destroySnapshot()
                    -- v192: the flags that describe the OLD body are reset here.
                    -- They used to survive the respawn, so the next apply could
                    -- skip the emote/animation write (it believed the previous
                    -- character still carried it) and the disguise came out
                    -- half applied -- one of the "many problems" reported.
                    restoring = false
                    emotesAppliedToHumanoid = false
                    animationOverrideApplied = false
                    table.clear(cloned)
                    table.clear(disguisedItems)
                    task.wait(0.5)
                    if card.Enabled then
```

```
                    applyDisguise()
                end
            end))
            applyDisguise()
        else
            restoreAndRelease()
            card:SetStatus(nil)
        end
    end,
})

card:CreateToggle({
    Name = "Disguise Character",
    Default = settings.disguiseCharacter,
    Function = function(value: boolean): ()
        settings.disguiseCharacter = value
        settings.mode = if value then "Character" else "Animation"
        card:SetStatus(if value then "Character" else "Emotes only")
        if card.Enabled then
            if not value then
                local character: Model? = LocalPlayer.Character
                local humanoid: Humanoid? = character
                    and character:FindFirstChildOfClass("Humanoid")
                    :: Humanoid?
                if character then
                    restoreOriginalAppearance(character, humanoid)
                end
                stopAppearanceFilter()
            end
            applyDisguise()
        end
    end,
    Tooltip = "ON (default): full character disguise (appearance + animations/emotes). OFF: only animations/emotes are shown, your character stays as is.",
})

-- Legacy hidden Mode dropdown for test compatibility (not shown in UI)
card:CreateDropdown({
    Name = "Mode",
    List = {"Character", "Animation"},
    Index = if settings.disguiseCharacter then 1 else 2,
    Show = {Option = "Disguise Character", Values = {"__hidden__"}},
    Function = function(value: string): ()
        settings.mode = value
        settings.disguiseCharacter = (value == "Character")
        card:SetStatus(value)
        if card.Enabled then
            if value == "Animation" then
                local character: Model? = LocalPlayer.Character
                local humanoid: Humanoid? = character
                    and character:FindFirstChildOfClass("Humanoid")
                    :: Humanoid?
                if character then
                    restoreOriginalAppearance(character, humanoid)
                end
                stopAppearanceFilter()
            end
            applyDisguise()
        end
    end,
    Tooltip = "Legacy: use Disguise Character instead.",
})

card:CreateSlider({
    Name = "Animation Speed",
    Min = 0,
    Max = 2,
    Step = 0.05,
    Default = settings.animationSpeed,
    Function = function(value: number): ()
        settings.animationSpeed = math.clamp(value, 0, 2)
        if card.Enabled then
            applyAnimationSpeed()
        end
    end,
    Tooltip = "Playback speed of your character's animations while Disguise is on: 0 freezes them, 1 is normal, 2 is double speed. Turning Disguise off restores speed 1.",
})

-- Show Advanced only reveals the bundle inputs. It never changes the
-- current animations before the user actually submits a bundle ID.
card:CreateToggle({
    Name = "Show Advanced",
    Default = settings.showAdvanced,
    Function = function(value: boolean): ()
        settings.showAdvanced = value
    end,
    Tooltip = "Shows bundle inputs for a complete pack or individual slots. Opening or closing these inputs does not change the animations already applied.",
})

card:CreateTextBox({
    Name = "Pack Bundle ID",
    Show = {Option = "Show Advanced", Values = {true}},
    Default = "",
    Function = function(value: string): ()
        local trimmed: string = tostring(value or "")
            :gsub("^%s+", "")
            :gsub("%s+$", "")
        local digits: string? = string.match(trimmed, "%d+")
        local bundleId: number? = digits and tonumber(digits) or nil
        if not bundleId then
            notify("Enter an animation bundle ID first (the number in its catalog link).")
            return
        end
        -- Any pack at all -- official or UGC -- resolved live from
        -- the bundle's own items, so the thousands of creator packs
        -- work without a single hardcoded id.
        local slots: {[string]: any}?, bundleError: string? =
            bundleAnimationSlots(bundleId)
        if not slots then
            notify("Could not load that pack: " .. tostring(bundleError))
            return
        end
        settings.customPackSlots = slots
        settings.customPackName = "Bundle " .. tostring(bundleId)
        table.clear(settings.customSlotIds)
        if not card.Enabled then
            notify("Pack " .. tostring(bundleId) .. " loaded; it applies while Disguise is on.")
            return
        end
        local character: Model? = LocalPlayer.Character
        local humanoid: Humanoid? = character
            and character:FindFirstChildOfClass("Humanoid")
            :: Humanoid?
        if character and humanoid then
            applyAnimationPack(character, humanoid)
            status((if settings.disguiseCharacter then "Character" else "Emotes only") .. " · pack Bundle " .. tostring(bundleId))
        end
    end,
  Tooltip = "Any animation pack bundle ID from its catalog link (roblox.com/bundles/<id>/...), official or UGC -- for example 4899847411546. The bundle's items are resolved when you enter it (nothing is downloaded; the engine streams the clips) and the pack is kept for the session. It wins over the target's movement animations. R15-only clips are not forced onto an incompatible R6 body.",
})

-- Per-slot bundle inserts: paste any pack's bundle ID into a slot
-- and that slot takes the pack's clip for it (Idle from one pack,
-- Run from another, ...). Slots left empty fall back to your rig's
-- own set when the pack applies.
local SLOT_INSERTS: {{string}} = {
    {"Idle", "idle"},
    {"Walk", "walk"},
    {"Run", "run"},
    {"Jump", "jump"},
    {"Fall", "fall"},
    {"Climb", "climb"},
    {"Swim", "swim"},
    {"Swim Idle", "swimidle"},
}
for _, slotInsert: {string} in ipairs(SLOT_INSERTS) do
    local label: string = slotInsert[1]
    local category: string = slotInsert[2]
    card:CreateTextBox({
        Name = label .. " Bundle ID",
        Show = {Option = "Show Advanced", Values = {true}},
        Default = "",
        Function = function(value: string): ()
            local digits: string? = string.match(
                tostring(value or ""),
                "%d+"
            )
            local bundleId: number? = digits and tonumber(digits)
                or nil
            if not bundleId then
                notify("Enter a bundle ID for the " .. label .. " slot first.")
                return
            end
            local slots: {[string]: any}?, bundleError: string? =
                bundleAnimationSlots(bundleId)
            if not slots then
                notify("Could not load that pack: " .. tostring(bundleError))
                return
            end
            local clip: any = slots[category]
            if type(clip) == "table" then
                clip = clip[1]
            end
            if not clip then
                notify("That bundle has no " .. label .. " clip to take.")
                return
            end
            settings.customSlotIds[category] = clip
            local merged: {[string]: any} = {}
            for mergedCategory: string, mergedId: any in pairs(
                settings.customSlotIds
            ) do
                merged[mergedCategory] = mergedId
            end
            settings.customPackSlots = merged
            settings.customPackName = "Custom slots"
            if not card.Enabled then
                notify(label .. " slot set; it applies while Disguise is on.")
                return
            end
            local character: Model? = LocalPlayer.Character
            local humanoid: Humanoid? = character
                and character:FindFirstChildOfClass("Humanoid")
                :: Humanoid?
            if character and humanoid then
                applyAnimationPack(character, humanoid)
                status((if settings.disguiseCharacter then "Character" else "Emotes only") .. " · pack Custom slots")
            end
        end,
        Tooltip = "Set the " .. label .. " slot from any animation pack: paste the pack's bundle ID (official or UGC) and this slot takes that pack's " .. label .. " clip. Mix slots from different packs freely; empty slots use your rig's own set.",
    })
end

card:CreateTextBox({
    Name = "Target ID",
    Default = settings.id,
    Function = function(value: string): ()
        local trimmed: string = tostring(value or "")
            :gsub("^%s+", "")
            :gsub("%s+$", "")
        if trimmed == "" then
            notify("Enter a user ID or animation bundle ID first.")
            return
        end
        settings.id = trimmed
        if card.Enabled then
            applyDisguise()
        end
    end,
    Tooltip = "User ID: when Disguise Character is ON copies appearance + animations/emotes onto your preserved body/rig; when OFF only animations/emotes are applied and your character stays as is. Default 78492. Missing slots fall back to your rig's own set.",
})

-- Legacy hidden: movement toggle (always on now), kept for tests but not shown
card:CreateToggle({
    -- v192: the player asked for the exact wording. It is also honest: a
    -- clip the target's rig cannot play falls back to your own set, so the
    -- option is best-effort, never a promise.
    Name = "Use target movement animations if able",
    Show = {Option = "Disguise Character", Values = {"__hidden__"}},
    Default = settings.useTargetMovementAnimations,
    Function = function(value: boolean): ()
        setMovementAnimationsEnabled(value)
    end,
  Tooltip = "Character mode only. Shows the target user's movement animations (idle, walk, run, jump, climb, fall, swim, and Roblox's default set for any slot the avatar leaves empty) on the disguised avatar. They are already loaded, so this is instant; turning it off restores your own animations. Emotes are independent and stay untouched.",
})

    card:CreateToggle({
    Name = "Show target emotes",
    Default = settings.showTargetEmotes,
    Function = function(value: boolean): ()
        setTargetEmotesEnabled(value)
    end,
    Tooltip = "Shows the target user's emotes and equipped emote slots on the disguised avatar. Works with or without Disguise Character.",
})

    activeCard = card
    Module.Initialized = true
    return card
end

function Module.destroy(): ()
    if activeCard then
        if activeCard.Enabled then
            pcall(activeCard.Toggle, activeCard, false)
        end
        activeCard = nil
    end
    Module.Initialized = false
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Other/Disguise.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Other/ImproveFps.lua
BYTES: 16021
SHA-256: 6470e288991e2317b9d3c4def94c289f143d4be10d8b724fdb1d52ff795a269f
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
export type Runtime = {
    framework: any,
    entity: any,
    host: any,
}

local Module = {
    Name = "ImproveFps",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local activeCleanup: (() -> ())? = nil

function Module.init(context: Runtime): any
    local host: any = context.host
    local featureConnections: any = host.featureConnections
    local disconnectFeatureConnection: any = host.disconnectFeatureConnection
    local createUniversalFeature: any = host.createUniversalFeature
    local addToggleOption: any = host.addToggleOption

    local fpsSettings: {[string]: boolean} = {
        textures = true,
        particles = true,
        shadows = true,
        materials = false,
        lighting = false,
        post = false,
        guiEffects = false,
        worldGui = false,
        gameGui = false,
        coreGui = false,
        geometry = false,
        scenery = false,
        quality = false,
        terrain = false,
        lights = false,
    }
    local fpsModeEnabled: boolean = false
    local generation: number = 0
    local improveFpsFeature: any = nil
    local function selectedEffectCount(): number
        local count: number = 0
        for _name: string, selected: boolean in pairs(fpsSettings) do
            if selected then count += 1 end
        end
        return count
    end

    local fpsVisualCache: any = {}
    local propertyWatches: {[any]: any} = {}

    local function cacheFpsProperty(object: any, property: string): ()
        local data: any = fpsVisualCache[object]
        if not data then
            data = {}
            fpsVisualCache[object] = data
            -- Keep snapshots strongly until OFF (even detached objects).
            -- Destroyed objects cannot be restored and must not accumulate.
            local ok, connection = pcall(function()
                return object.Destroying:Connect(function()
                    fpsVisualCache[object] = nil
                    local watch = propertyWatches[object]
                    propertyWatches[object] = nil
                    if watch then watch:Disconnect() end
                end)
            end)
            if ok then propertyWatches[object] = connection end
        end
        if data[property] == nil then
            data[property] = object[property]
        end
    end

    local lighting: any = host.Lighting
    local playerGui: any = host.PlayerGui
    local scenery: {[any]: boolean} = setmetatable({}, {__mode = "k"})
    local coreGuiCache: {[any]: boolean} = {}
    local function setProperty(object: any, property: string, value: any): ()
        -- Individual protected writes: an unsupported property must not
        -- prevent the remaining optimizations on that object.
        pcall(function()
            cacheFpsProperty(object, property)
            object[property] = value
        end)
    end
    local function protectedGui(object: any): boolean
        local current = object
        while current do
            if current == host.ScreenGui or current.Name == "TouchGui"
                or current.Name == "Chat" or current.Name == "BubbleChat" then return true end
            current = current.Parent
            if type(current) ~= "table" and typeof(current) ~= "Instance" then break end
        end
        return false
    end
    local function updateScenery(part: any): ()
        local character = host.LocalPlayer and host.LocalPlayer.Character
        local root = character and character:FindFirstChild("HumanoidRootPart")
        local cached = fpsVisualCache[part]
        local function restorePart()
            if cached and cached.LocalTransparencyModifier ~= nil then
                pcall(function() part.LocalTransparencyModifier = cached.LocalTransparencyModifier end)
            end
        end
        if not root or not part.Parent or not part.Anchored or not part.CanCollide then restorePart(); return end
        -- Static map only. Never cull characters, tools, or physics objects.
        local current = part.Parent
        while current and current ~= workspace do
            if current:IsA("Tool") or current:IsA("Accessory")
                or current:FindFirstChildOfClass("Humanoid") then restorePart(); return end
            current = current.Parent
        end
        local far = (part.Position - root.Position).Magnitude > 350
        if far then setProperty(part, "LocalTransparencyModifier", 1)
        elseif cached and cached.LocalTransparencyModifier ~= nil then
            pcall(function() part.LocalTransparencyModifier = cached.LocalTransparencyModifier end)
        end
    end

    local function applyFpsObject(object: Instance): ()
        -- A deferred DescendantAdded callback can arrive after destruction.
        -- Keep existing detached snapshots for restoration, but never acquire
        -- new ownership of an object already outside the scene.
        if object ~= workspace and object ~= lighting and object ~= playerGui and object.Parent == nil then return end
        local target: any = object
        if lighting and object == lighting and fpsSettings.lighting then
            setProperty(object, "EnvironmentDiffuseScale", 0)
            setProperty(object, "EnvironmentSpecularScale", 0)
        end
        if fpsSettings.shadows and object == lighting then setProperty(object, "GlobalShadows", false) end
        if fpsSettings.lights and object:IsA("Light") then setProperty(object, "Enabled", false) end
        if fpsSettings.terrain and object:IsA("Terrain") then
            setProperty(object, "Decoration", false)
            setProperty(object, "WaterWaveSize", 0)
            setProperty(object, "WaterWaveSpeed", 0)
            setProperty(object, "WaterReflectance", 0)
        end
        if fpsSettings.post and object:IsA("PostEffect") then
            setProperty(object, "Enabled", false)
        end
        if not protectedGui(object) then
            if fpsSettings.guiEffects and (object:IsA("UIStroke") or object:IsA("UIGradient")) then
                setProperty(object, "Enabled", false)
            elseif fpsSettings.guiEffects and object:IsA("ViewportFrame") then
                setProperty(object, "Visible", false)
            end
            if fpsSettings.worldGui and (object:IsA("BillboardGui") or object:IsA("SurfaceGui")) then
                setProperty(object, "Enabled", false)
            end
            if fpsSettings.gameGui and object:IsA("ScreenGui") then
                setProperty(object, "Enabled", false)
            end
        end
        if fpsSettings.geometry and (object:IsA("MeshPart") or object:IsA("PartOperation")) then
            setProperty(object, "RenderFidelity", Enum.RenderFidelity.Performance)
        end
        if fpsSettings.scenery and object:IsA("BasePart") and object.Anchored and object.CanCollide then
            scenery[object] = true
            updateScenery(object)
        end
        if fpsSettings.textures and (object:IsA("Decal") or object:IsA("Texture")) then
            cacheFpsProperty(object, "Transparency")
            target.Transparency = 1
        elseif fpsSettings.textures and object:IsA("MeshPart") then
            setProperty(object, "TextureID", "")
        elseif fpsSettings.textures and object:IsA("SpecialMesh") then
            setProperty(object, "TextureId", "")
        end

        if fpsSettings.particles
            and (object:IsA("ParticleEmitter")
                or object:IsA("Trail")
                or object:IsA("Beam")
                or object:IsA("Smoke")
                or object:IsA("Fire")
                or object:IsA("Sparkles")) then
            cacheFpsProperty(object, "Enabled")
            target.Enabled = false
        end

        if object:IsA("BasePart") then
            if fpsSettings.shadows then
                cacheFpsProperty(object, "CastShadow")
                object.CastShadow = false
            end
            if fpsSettings.materials then
                cacheFpsProperty(object, "Material")
                cacheFpsProperty(object, "Reflectance")
                object.Material = Enum.Material.SmoothPlastic
                object.Reflectance = 0
            end
        end
    end

    local function restoreFpsObjects(): ()
        for _, connection in pairs(propertyWatches) do pcall(function() connection:Disconnect() end) end
        propertyWatches = {}
        for object: any, properties: any in pairs(fpsVisualCache) do
            if object then
                for property: string, value: any in pairs(properties) do
                    pcall(function(): ()
                        object[property] = value
                    end)
                end
            end
        end
        fpsVisualCache = {}
        scenery = setmetatable({}, {__mode = "k"})
        local starterGui = host.StarterGui
        if starterGui then
            for kind, value in pairs(coreGuiCache) do
                pcall(function() starterGui:SetCoreGuiEnabled(kind, value) end)
            end
        end
        coreGuiCache = {}
    end

    local function toggleImproveFps(enabled: boolean): ()
        generation += 1
        local token: number = generation
        fpsModeEnabled = enabled
        for _, name in ipairs({"ImproveFPS", "ImproveFPSLighting", "ImproveFPSGui", "ImproveFPSDistance"}) do
            disconnectFeatureConnection(name)
        end
        restoreFpsObjects()
        if not enabled then
            if improveFpsFeature then improveFpsFeature:SetStatus(nil) end
            return
        end
        improveFpsFeature:SetStatus(tostring(selectedEffectCount()) .. " options")
        if fpsSettings.quality then
            pcall(function() setProperty(settings().Rendering, "QualityLevel", Enum.QualityLevel.Level01) end)
        end

        local containers = {workspace}
        if lighting then table.insert(containers, lighting) end
        if playerGui then table.insert(containers, playerGui) end
        for _, container in ipairs(containers) do
            local name = ({"ImproveFPS", "ImproveFPSLighting", "ImproveFPSGui"})[index]
            featureConnections[name] = container.DescendantAdded:Connect(function(object)
                if fpsModeEnabled and token == generation then pcall(applyFpsObject, object) end
            end)
        end
        if fpsSettings.coreGui and host.StarterGui then
            -- Do not disable All, Health or the mobile touch controls.
            for _, kind in ipairs({Enum.CoreGuiType.PlayerList, Enum.CoreGuiType.Backpack,
                Enum.CoreGuiType.Chat, Enum.CoreGuiType.EmotesMenu}) do
                pcall(function()
                    local value = host.StarterGui:GetCoreGuiEnabled(kind)
                    if type(value) == "boolean" then
                        coreGuiCache[kind] = value
                        host.StarterGui:SetCoreGuiEnabled(kind, false)
                    end
                end)
            end
        end
        if fpsSettings.scenery and host.TaskManager then
            local elapsed, cursor = 0, 1
            -- v139.1: one long lived array, cleared in place. Rebuilding it
            -- every second threw away the whole list (thousands of parts on a
            -- big map) and left it for the collector: a guaranteed hitch.
            local pending: {BasePart} = {}
            featureConnections.ImproveFPSDistance = host.TaskManager:Connect(function(dt)
                elapsed += dt
                if cursor > #pending then
                    if elapsed < 1 then return end
                    elapsed = 0
                    table.clear(pending)
                    for part in pairs(scenery) do
                        if part.Parent then table.insert(pending, part) else scenery[part] = nil end
                    end
                    cursor = 1
                end
                -- Bound work per frame instead of rescanning a huge map in
                -- one callback, which would defeat an FPS optimization.
                local last = math.min(#pending, cursor + 199)
                while cursor <= last do
                    if not fpsModeEnabled or token ~= generation then return end
                    pcall(updateScenery, pending[cursor])
                    cursor += 1
                end
            end, nil, "Improve FPS")
        end
        task.spawn(function(): ()
            local scanned = 0
            for _, container in ipairs(containers) do
                if not fpsModeEnabled or token ~= generation then return end
                pcall(applyFpsObject, container)
                for _, object in ipairs(container:GetDescendants()) do
                    if not fpsModeEnabled or token ~= generation then return end
                    pcall(applyFpsObject, object)
                    scanned += 1
                    if scanned % 250 == 0 then task.wait() end
                end
            end
        end)
    end

    local function refreshFps(): ()
        if fpsModeEnabled then
            toggleImproveFps(true)
        end
    end

    ImproveFpsFeature = createUniversalFeature(
        "Improve FPS",
        "Apply only the selected reversible optimizations",
        26,
        toggleImproveFps,
        {categoryName = "World"}
    )
    addToggleOption(ImproveFpsFeature, "Remove textures", true, function(value: boolean): ()
        fpsSettings.textures = value
        refreshFps()
    end)
    addToggleOption(ImproveFpsFeature, "Disable particles", true, function(value: boolean): ()
        fpsSettings.particles = value
        refreshFps()
    end)
    addToggleOption(ImproveFpsFeature, "Disable shadows", true, function(value: boolean): ()
        fpsSettings.shadows = value
        refreshFps()
    end)
    addToggleOption(ImproveFpsFeature, "Simple materials", false, function(value: boolean): ()
        fpsSettings.materials = value
        refreshFps()
    end)

    for _, definition in ipairs({
        {"Lowest render quality", "quality", "Requests render quality Level01 where the client permits it; restores the original level when OFF."},
        {"Simple lighting", "lighting", "Disables environment diffuse/specular reflections."},
        {"Disable local lights", "lights", "Turns off PointLight, SpotLight and SurfaceLight instances. May darken the map."},
        {"Simplify terrain", "terrain", "Disables terrain decoration, water waves and water reflections where writable. Physics is unchanged."},
        {"Disable post effects", "post", "Disables bloom, blur (including menu blur), depth of field, color correction and sun rays."},
        {"Simplify game UI", "guiEffects", "Disables game UI strokes, gradients and 3D viewport previews. Menu and touch controls are protected."},
        {"Hide world UI", "worldGui", "Hides floating labels and SurfaceGui displays; may hide game information."},
        {"Hide game UI", "gameGui", "Aggressive: hides game ScreenGuis, including HUD/objectives. Keeps this menu, chat and TouchGui."},
        {"Hide Roblox panels", "coreGui", "Hides player list, backpack, chat and emotes panel. Health and mobile movement remain available."},
        {"Low detail meshes", "geometry", "Requests Performance render fidelity for meshes/unions; does not change collisions."},
        {"Hide distant scenery", "scenery", "Aggressive: locally hides static map parts beyond 350 studs, updating periodically. May hide distant objectives; collisions remain."},
    }) do
        local key: string = definition[2]
        addToggleOption(ImproveFpsFeature, definition[1], false, function(value: boolean)
            fpsSettings[key] = value
            refreshFps()
        end, definition[3])
    end

    activeCleanup = function(): ()
        toggleImproveFps(false)
    end
    Module.Initialized = true
    return ImproveFpsFeature
end

function Module.destroy(): ()
    if activeCleanup then
        pcall(activeCleanup)
    end
    activeCleanup = nil
    Module.Initialized = false
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Other/ImproveFps.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Other/ProximityPrompts.lua
BYTES: 11704
SHA-256: b0d76ade20c6dd354dc88b7af780396cadf977bb9e2b3ade3045916ec3ce6cc3
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
export type Runtime = {
    framework: any,
    entity: any,
    host: any,
}

local Module = {
    Name = "ProximityPrompts",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local activeCleanup: (() -> ())? = nil
```

```
function Module.init(context: Runtime): any
    local host: any = context.host
    local TaskManager: any = host.TaskManager
    local featureConnections: any = host.featureConnections
    local disconnectFeatureConnection: any = host.disconnectFeatureConnection
    local createUniversalFeature: any = host.createUniversalFeature
    local addToggleOption: any = host.addToggleOption
    local addNumberOption: any = host.addNumberOption
    type InteractionSettings = {
        distance: number,
        instantPrompts: boolean,
        proximityPrompts: boolean,
        clickDetectors: boolean,
    }
    type PromptState = {distance: number, holdDuration: number}
    -- Rows default OFF: the card is the master switch and turns every channel
    -- on by itself. A row switched on with the card OFF is a deliberate opt-in
    -- and works on its own, so nobody has to enable the whole card just for the
    -- fast prompt.
    local interactionSettings: InteractionSettings = {
        distance = 32,
        instantPrompts = false,
        proximityPrompts = false,
        clickDetectors = false,
    }
    local originalPrompts: {[ProximityPrompt]: PromptState} =
        setmetatable({}, {__mode = "k"}) :: any
    local originalClickDetectors: {[ClickDetector]: number} =
        setmetatable({}, {__mode = "k"}) :: any
    local interactionExtenderEnabled: boolean = false
    local InteractFeature: any = nil
    local interactionRefreshClock: number = 0

    -- Every channel follows its own row, so a row switched on with the card OFF
    -- works alone and a row switched off with the card ON really stops. The card
    -- is still a master switch: turning it on switches the rows on (visibly, not
    -- behind the scenes) and turning it off switches them off.
    local function promptsChannel(): boolean
        return interactionSettings.proximityPrompts or interactionSettings.instantPrompts
    end

    local function detectorsChannel(): boolean
        return interactionSettings.clickDetectors
    end

    local function anyChannel(): boolean
        return promptsChannel() or detectorsChannel()
    end

    local function updatePrompt(prompt: ProximityPrompt, original: PromptState): ()
        local distance: number = promptsChannel()
                and interactionSettings.distance or original.distance
        local hold: number = promptsChannel() and interactionSettings.instantPrompts
                and 0 or original.holdDuration
        if prompt.MaxActivationDistance ~= distance then prompt.MaxActivationDistance = distance end
        if prompt.HoldDuration ~= hold then prompt.HoldDuration = hold end
    end

    local function updateDetector(detector: ClickDetector, original: number): ()
        local distance: number = detectorsChannel()
                and interactionSettings.distance or original
        if detector.MaxActivationDistance ~= distance then detector.MaxActivationDistance = distance end
    end

    local function refreshInteractionExtender(): ()
        for prompt: ProximityPrompt, original: PromptState in pairs(originalPrompts) do
            if prompt.Parent then updatePrompt(prompt, original) end
        end
        for detector: ClickDetector, original: number in pairs(originalClickDetectors) do
            if detector.Parent then updateDetector(detector, original) end
        end
    end

    local function applyInteractionExtension(instance: Instance): ()
        if not anyChannel() then
            return
        end
        if instance:IsA("ProximityPrompt") then
            if originalPrompts[instance] == nil then
                originalPrompts[instance] = {
                    distance = instance.MaxActivationDistance,
                    holdDuration = instance.HoldDuration,
                }
            end
            updatePrompt(instance :: ProximityPrompt, originalPrompts[instance])
        elseif instance:IsA("ClickDetector") then
            if originalClickDetectors[instance] == nil then
                originalClickDetectors[instance] = instance.MaxActivationDistance
            end
            updateDetector(instance :: ClickDetector, originalClickDetectors[instance])
        end
    end

    -- Put every touched prompt/detector back to the values it had before we
    -- saw it, then forget it. Called when no channel is live any more.
    local function restoreInteractionExtender(): ()
        for prompt: ProximityPrompt, original: PromptState in pairs(originalPrompts) do
            if prompt.Parent then
                if prompt.MaxActivationDistance ~= original.distance then
                    prompt.MaxActivationDistance = original.distance
                end
                if prompt.HoldDuration ~= original.holdDuration then
                    prompt.HoldDuration = original.holdDuration
                end
            end
        end
        for detector: ClickDetector, original: number in pairs(originalClickDetectors) do
            if detector.Parent and detector.MaxActivationDistance ~= original then
                detector.MaxActivationDistance = original
            end
        end
        originalPrompts = setmetatable({}, {__mode = "k"}) :: any
        originalClickDetectors = setmetatable({}, {__mode = "k"}) :: any
    end

    local scanning: boolean = false
    local function startScan(): ()
        if scanning then return end
        scanning = true
        local scanConnection: any = featureConnections.InteractExtender
        task.spawn(function(): ()
            for index: number, instance: Instance in ipairs(workspace:GetDescendants()) do
                if featureConnections.InteractExtender ~= scanConnection then
                    -- Superseded: a newer generation owns the flag now.
                    scanning = false
                    return
                end
                applyInteractionExtension(instance)
                if index % 240 == 0 then
                    task.wait()
                end
            end
            scanning = false
        end)
    end

    -- Keep exactly the hooks the live channels need. Cheap when nothing changed,
    -- so a single row can call it on every click without a rescan.
    local function syncInteractions(): ()
        if not anyChannel() then
            disconnectFeatureConnection("InteractExtender")
            disconnectFeatureConnection("InteractExtenderRefresh")
            -- A scan suspended by this teardown belongs to the old generation;
            -- releasing the flag lets the next activation start its own.
            scanning = false
            restoreInteractionExtender()
            interactionRefreshClock = 0
            if InteractFeature then
                InteractFeature:SetStatus(nil)
            end
            return
        end
        if not featureConnections.InteractExtender then
            featureConnections.InteractExtender =
                workspace.DescendantAdded:Connect(applyInteractionExtension)
        end
        if not featureConnections.InteractExtenderRefresh then
            featureConnections.InteractExtenderRefresh = TaskManager:Connect(
                function(deltaTime: number): ()
                    interactionRefreshClock += deltaTime
                    if interactionRefreshClock >= 0.5 then
                        interactionRefreshClock = 0
                        refreshInteractionExtender()
                    end
                end
            , nil, "Proximity Prompts")
            -- First activation (card or row): pick up what already exists.
            startScan()
        else
            refreshInteractionExtender()
        end
        if InteractFeature and interactionExtenderEnabled then
            InteractFeature:SetStatus(
                tostring(math.round(interactionSettings.distance)) .. " studs"
            )
        end
    end

    -- Keep the checkbox, the saved value and the callback in step when the card
    -- itself switches a row.
    local function setRow(name: string, value: boolean): ()
        local controls: any = nil
        pcall(function(): ()
            if state and type(state.toggleControls) == "table" then
                controls = state.toggleControls
            end
        end)
        local control: any = controls and controls[InteractFeature.configKey .. "." .. name]
        if control and type(control.setChecked) == "function" then
            pcall(control.setChecked, value)
        end
    end

    local function toggleInteractionExtender(enabled: boolean): ()
        interactionExtenderEnabled = enabled
        if enabled then
            interactionSettings.proximityPrompts = true
            interactionSettings.clickDetectors = true
            setRow("Proximityprompts", true)
            setRow("Clickdetectors", true)
            -- The card was switched on: rescan once so nothing that exists right
            -- now is missed, even if the rows were already live.
            syncInteractions()
            startScan()
            return
        else
            interactionSettings.instantPrompts = false
            interactionSettings.proximityPrompts = false
            interactionSettings.clickDetectors = false
            setRow("Instantprompts", false)
            setRow("Proximityprompts", false)
            setRow("Clickdetectors", false)
        end
        syncInteractions()
    end

    InteractFeature = createUniversalFeature(
        "Proximity Prompts",
        "Extend prompt and click interaction range without auto-firing",
        24,
        toggleInteractionExtender,
        {categoryName = "Exploits", configKey = "Universal.InteractExtender"}
    )
    addNumberOption(
        InteractFeature,
        "Interaction distance",
        interactionSettings.distance,
        8,
        250,
        function(value: number): ()
            interactionSettings.distance = value
            if InteractFeature.enabled then
                InteractFeature:SetStatus(tostring(math.round(value)) .. " studs")
            end
            syncInteractions()
        end
    )
    addToggleOption(
        InteractFeature,
        "Instant prompts",
        interactionSettings.instantPrompts,
        function(value: boolean): ()
            interactionSettings.instantPrompts = value
            syncInteractions()
        end
    )
    addToggleOption(
        InteractFeature,
        "Proximity prompts",
        interactionSettings.proximityPrompts,
        function(value: boolean): ()
            interactionSettings.proximityPrompts = value
            syncInteractions()
        end
    )
    addToggleOption(
        InteractFeature,
        "Click detectors",
        interactionSettings.clickDetectors,
        function(value: boolean): ()
            interactionSettings.clickDetectors = value
            syncInteractions()
        end
    )

    activeCleanup = function(): ()
        interactionExtenderEnabled = false
        disconnectFeatureConnection("InteractExtender")
        disconnectFeatureConnection("InteractExtenderRefresh")
        restoreInteractionExtender()
        interactionRefreshClock = 0
        if InteractFeature then
            InteractFeature:SetStatus(nil)
        end
    end
    Module.Initialized = true
    return InteractFeature
end

function Module.destroy(): ()
    if activeCleanup then
        activeCleanup()
    end
    activeCleanup = nil
    Module.Initialized = false
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Other/ProximityPrompts.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Other/RejoinServer.lua
BYTES: 3662
SHA-256: db8537ec39513a59d62d44afbc409d733c04547a2aae0b28bdbfda7b0aa77978
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
export type Runtime = {
    framework: any,
    entity: any,
    host: any,
    services: any,
}

local Module = {
    Name = "RejoinServer",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local activeCleanup: (() -> ())? = nil

function Module.init(context: Runtime): any
    local host: any = context.host
    local LocalPlayer: any = host.LocalPlayer
    local createUniversalFeature: any = host.createUniversalFeature
    local cloneReference: any = host.cloneReference
    local notify: (string) -> () = host.notify
    local function rejoinCurrentServer(): ()
        local teleportService: TeleportService = cloneReference(
            game:GetService("TeleportService")
        )
        local placeId: number = game.PlaceId
        local jobId: string = tostring(game.JobId or "")

        local rejoined: boolean = false
        if jobId ~= "" then
            rejoined = pcall(function(): ()
                teleportService:TeleportToPlaceInstance(placeId, jobId, LocalPlayer)
            end)
        end
        if not rejoined then
            local ok: boolean, teleportError: any = pcall(function(): ()
                teleportService:Teleport(placeId, LocalPlayer)
            end)
            if not ok then
                notify("Rejoin failed: " .. tostring(teleportError))
            end
        end
    end
    local actionId: string = "RejoinServer"
    local registries: any = context.services.registries
    local wurstOptions: any = registries.wurstOptions()
    if wurstOptions and type(wurstOptions.RegisterAction) == "function" then
        local registered: boolean, registration: any = pcall(
            wurstOptions.RegisterAction,
            actionId,
            "Rejoin Server",
            rejoinCurrentServer,
            {
                description = "Reconnect to the current server instance",
                configKey = "Universal.RejoinServer",
            }
        )
        if not registered then
            error("Wurst Options refused Rejoin Server: " .. tostring(registration))
        end
        local search: any = registries.moduleSearch()
        local searchFeature: any = {
            name = "Rejoin Server",
            configKey = "Universal.RejoinServer",
            kind = "action",
            searchable = false,
            enabled = false,
            activate = rejoinCurrentServer,
        }
        if search and type(search.Register) == "function" then
            search.Register(searchFeature)
        end
        activeCleanup = function(): ()
            if search and type(search.Unregister) == "function" then
                search.Unregister(searchFeature)
            end
            if type(registration) == "table"
                and type(registration.Unregister) == "function" then
                registration.Unregister()
            elseif type(wurstOptions.UnregisterAction) == "function" then
                pcall(wurstOptions.UnregisterAction, actionId)
            end
        end
        Module.Initialized = true
        return registration
    end

    local feature: any = createUniversalFeature(
        "Rejoin Server",
        "Reconnect to the current server instance",
        22,
        rejoinCurrentServer,
        {
            action = true,
            noOptions = true,
            categoryName = "Other",
            searchable = false,
        }
    )

    activeCleanup = function(): () end
    Module.Initialized = true
    return feature
end

function Module.destroy(): ()
    if activeCleanup then
        pcall(activeCleanup)
    end
    activeCleanup = nil
    Module.Initialized = false
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Other/RejoinServer.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Other/ServerSettings.lua
BYTES: 24791
SHA-256: 81fdca3ac5254c77abef1337c1eef7617c540a3c2453164a46217515977fca68
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
-- v193: "Server Settings" (Utilities).
--
-- What it does, and what it deliberately does NOT do:
--   * It reads the server the player is actually in (Job ID, player count,
--     player limit, uptime, server clock, place name) and keeps that readout on
--     the card, live, the way AutoFarm's "Overlay Info" rows work: a section of
--     stat rows that a 1 Hz task keeps writing for as long as the module lives.
--     There is no window, no timer that expires and no "refresh every" ceiling.
--   * It copies the Job ID and hops to another public server of the same place.
--   * It sets no server-side property. A Roblox client cannot change another
--     server's settings; a card that claimed to would be lying.
--
-- History, so the next edit does not undo it:
--   v191 shipped the card with "Rejoin Server", a readout that only refreshed
--   while the card was ON, and hops that could land on a full instance.
--   v192 removed the rejoin, required a free slot for a hop and drew a floating
--   overlay Frame -- which landed on top of the menu's own options and stopped
--   whenever the card was switched off.
--   v193 (the player's report) removes that window: the info is a live section
--   inside the card, it never stops, it carries server data only (no Build, no
--   Round, no You), the place row shows the place NAME and marks a subplace.
--   The hops no longer depend on any endpoint flag: every listed server is read,
--   only the ones with a free slot are candidates, and when none is joinable the
--   card falls back to joining a random server instead of failing silently.
export type Runtime = {
    framework: any,
    entity: any,
    host: any,
    services: any,
}

local Module = {
    Name = "ServerSettings",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local activeCard: any = nil
-- Kept so destroy() can drop the 1 Hz readout without holding the whole host.
local disconnectTicker: ((string) -> ())? = nil

-- One hop per couple of seconds at most: the public endpoint rate-limits hard
-- and a hop already interrupts the session, so stacked calls are worthless.
local HOP_COOLDOWN: number = 3

-- The live readout ticks at 1 Hz. It is not tied to the card's toggle: the
-- player asked for information that never stops, and a stat row costs one label
-- write when (and only when) the string changes.
local STAT_INTERVAL: number = 1

-- How many pages of the public server list a hop reads. Each page is 100
-- servers; two pages per sort order is ~200 candidates, which is plenty and
-- keeps the request count (and the rate limit) sane.
local MAX_PAGES: number = 2

function Module.init(context: Runtime): any
    local host: any = context.host
    local LocalPlayer: any = host.LocalPlayer
    local createUniversalFeature: any = host.createUniversalFeature
    local addActionOption: any = host.addActionOption
    local addInformationOption: any = host.addInformationOption
    local notify: any = host.notify
    local cloneReference: any = host.cloneReference
    local services: any = context.services or {}

    local players: any = host.Players or game:GetService("Players")
    local httpService: any = cloneReference
        and cloneReference(game:GetService("HttpService"))
        or game:GetService("HttpService")
    local teleportService: any = cloneReference
        and cloneReference(game:GetService("TeleportService"))
        or game:GetService("TeleportService")
    local tick: any = if host.TaskManager and host.TaskManager.Connect
        then host.TaskManager
        else nil
    local featureConnections: any = host.featureConnections
    local disconnectFeatureConnection: any = host.disconnectFeatureConnection
    if type(disconnectFeatureConnection) == "function" then
        disconnectTicker = disconnectFeatureConnection
    end

    local settings = {
        showInfo = true,
    }
    local lastHopAt: number = -math.huge
    local elapsed: number = 0
    local hopping: boolean = false
    -- Declared before every closure that touches it.
    local card: any = nil
    local statRows: {[string]: any} = {}

    local function say(message: string, kind: string?): ()
        if type(notify) == "function" then
            pcall(notify, message, kind)
        end
    end

    local function jobId(): string
        local ok: boolean, value: any = pcall(function(): any
            return game.JobId
        end)
        if not ok or type(value) ~= "string" or value == "" then
            return ""
        end
        return value
    end

    local function playerList(): {any}
        local ok: boolean, list: any = pcall(function(): any
            return players:GetPlayers()
        end)
        if not ok or type(list) ~= "table" then
            return {}
        end
        return list
    end

    local function maxPlayers(): number
        local ok: boolean, value: any = pcall(function(): any
            return players.MaxPlayers
        end)
        if ok and type(value) == "number" then
            return value
        end
        return 0
    end

    -- Seconds since this server was started. `DistributedGameTime` is the
    -- server's own clock: it does not reset when you respawn or rejoin, which is
    -- exactly what "how long has this server been up" means.
    local function uptimeSeconds(): number
        local ok: boolean, value: any = pcall(function(): any
            return workspace.DistributedGameTime
        end)
        if ok and type(value) == "number" and value >= 0 then
            return value
        end
        return 0
    end

    local function clockText(): string
        local ok: boolean, value: any = pcall(function(): any
            return workspace:GetServerTimeNow()
        end)
        local stamp: number = if ok and type(value) == "number"
            then math.floor(value)
            else os.time()
        local date: any = os.date("!*t", stamp)
        if type(date) ~= "table" then
            return "--:--:--"
        end
        return string.format("%02d:%02d:%02d", date.hour, date.min, date.sec)
    end

    local function durationText(seconds: number): string
        local total: number = math.max(0, math.floor(seconds))
        local hours: number = math.floor(total / 3600)
        local minutes: number = math.floor((total % 3600) / 60)
        local secs: number = total % 60
        if hours > 0 then
            return string.format("%d:%02d:%02d", hours, minutes, secs)
        end
        return string.format("%02d:%02d", minutes, secs)
    end

    -- Place should show the real place name (via MarketplaceService) and
    -- whether it's a subplace (PlaceId != GameId). Falls back to game.Name.
    local function placeText(): string
        local placeOk: boolean, placeId: any = pcall(function(): any
            return game.PlaceId
        end)
        local gameOk: boolean, gameId: any = pcall(function(): any
            return game.GameId
        end)
        local label: string = "Unknown place"
        -- Try MarketplaceService for the actual place name (most accurate)
        local marketOk: boolean, marketName: any = pcall(function(): any
            local ms: any = game:GetService("MarketplaceService")
            local info: any = ms:GetProductInfo(placeId)
            return info and info.Name
        end)
        if marketOk and type(marketName) == "string" and marketName ~= "" then
            label = marketName
        else
            local nameOk: boolean, name: any = pcall(function(): any
                return game.Name
            end)
            if nameOk and type(name) == "string" and name ~= "" then
                label = name
            elseif placeOk and type(placeId) == "number" then
                label = "Place " .. tostring(placeId)
            end
        end
        if placeOk and gameOk
            and type(placeId) == "number" and type(gameId) == "number"
            and placeId ~= gameId then
            return label .. " · subplace"
        end
        return label
    end

    local function namesText(limit: number): string
        local list: {any} = playerList()
        local names: {string} = {}
        for _, player: any in ipairs(list) do
            local ok: boolean, name: any = pcall(function(): any
                return player.Name
            end)
            if ok and type(name) == "string" then
                table.insert(names, name)
            end
        end
        if #names == 0 then
            return "—"
        end
        local joined: string = table.concat(names, ", ")
        if #joined > limit then
            joined = string.sub(joined, 1, math.max(1, limit - 1)) .. "…"
        end
        return joined
    end

    local function shortJobId(): string
        local id: string = jobId()
        if id == "" then
            return "private server"
        end
        if #id > 8 then
            return string.sub(id, 1, 8)
        end
        return id
    end

    local function infoText(): string
        local limit: number = maxPlayers()
        return string.format(
            "%s · %d/%d · up %s",
            shortJobId(),
            #playerList(),
            if limit > 0 then limit else #playerList(),
```

```
        durationText(uptimeSeconds())
    )
end

-- ---------------------------------------------------------------
-- Live readout (AutoFarm-style stat rows)
-- ---------------------------------------------------------------

-- The rows the card keeps writing. Server data only: the player asked for
-- Build, Round and You to be removed, and for the place row to name the
-- place (and say when it is a subplace).
local statRowsDefinition: {{key: string, label: string, read: () -> string}} = {
    {key = "server", label = "Server", read = shortJobId},
    {key = "players", label = "Players", read = function(): string
        local limit: number = maxPlayers()
        return string.format(
            "%d / %d",
            #playerList(),
            if limit > 0 then limit else #playerList()
        )
    end},
    {key = "inServer", label = "In server", read = function(): string
        return namesText(34)
    end},
    {key = "uptime", label = "Uptime", read = function(): string
        return durationText(uptimeSeconds())
    end},
    {key = "serverTime", label = "Server time", read = function(): string
        return clockText()
    end},
    {key = "place", label = "Place", read = placeText},
}

local function ensureStatRows(): ()
    if not card or next(statRows) ~= nil then
        return
    end
    if type(state.addSectionOption) == "function" then
        pcall(state.addSectionOption, card, "Overlay Info")
    end
    if type(state.addStatOption) ~= "function" then
        return
    end
    for _, definition: any in ipairs(statRowsDefinition) do
        local ok: boolean, row: any = pcall(
            state.addStatOption,
            card,
            definition.label,
            "…"
        )
        if ok and row then
            statRows[definition.key] = row
        end
    end
end

local function refreshStats(): ()
    ensureStatRows()
    for _, definition: any in ipairs(statRowsDefinition) do
        local row: any = statRows[definition.key]
        if row and type(row.set) == "function" then
            local ok: boolean, value: any = pcall(definition.read)
            local text: string = if ok and type(value) == "string" and value ~= ""
                then value
                else "…"
            pcall(row.set, text)
        end
    end
    if card and type(card.SetStatus) == "function" then
        if settings.showInfo then
            pcall(function(): ()
                card:SetStatus(infoText())
            end)
        else
            pcall(function(): ()
                card:SetStatus(nil)
            end)
        end
    end
end

-- One connection for the lifetime of the module: it is created at init and
-- only dropped by destroy(), so the readout cannot stop when the card is
-- toggled or when an option is changed. 1 Hz, and every write is guarded by
-- a string comparison inside the stat row.
local function startTicker(): ()
    if not (tick and featureConnections) then
        return
    end
    if featureConnections.ServerSettingsTick then
        return
    end
    featureConnections.ServerSettingsTick = tick:Connect(
        function(deltaTime: number): ()
            elapsed += math.max(0, deltaTime or 0)
            if elapsed < STAT_INTERVAL then
                return
            end
            elapsed = 0
            refreshStats()
        end,
        nil,
        "Server Settings"
    )
end

local function copyJobId(): ()
    local id: string = jobId()
    if id == "" then
        say("This server has no Job ID to copy.")
        return
    end
    -- Executors expose `setclipboard` as a global; a plain Roblox client
    -- does not, so the message still carries the id when copying fails.
    local clipboard: any = host.setclipboard or setclipboard
    local copied: boolean = false
    if type(clipboard) == "function" then
        copied = pcall(clipboard, id)
    end
    if copied then
        say("Job ID copied: " .. id, "good")
    else
        say("Job ID: " .. id)
    end
end

-- ---------------------------------------------------------------
-- Server list
-- ---------------------------------------------------------------

-- One page of the public server list, as the Roblox web app reads it.
-- `sortOrder` is what makes the difference between the quiet end and the
-- busy end of the place; `cursor` walks further into the list.
--
-- v193: the request sends only `sortOrder`, `limit` and (for deeper pages)
-- `cursor`. The client filters full instances itself, so it can also explain
-- what it saw, and an unsupported query flag can no longer turn every hop
-- into "the request failed".
local function fetchPage(
    sortOrder: string,
    cursor: string?
): ({any}?, string?, string?)
    local endpoint: string = string.format(
        "https://games.roblox.com/v1/games/%d/servers/Public?sortOrder=%s&limit=100",
        game.PlaceId,
        sortOrder
    )
    if type(cursor) == "string" and cursor ~= "" then
        endpoint = endpoint .. "&cursor=" .. cursor
    end
    local ok: boolean, body: any = pcall(function(): any
        return (game :: any):HttpGet(endpoint)
    end)
    if not ok or type(body) ~= "string" or body == "" then
        return nil, nil, "the server list request failed (executor HTTP is blocked?)"
    end
    local decodeOk: boolean, decoded: any = pcall(function(): any
        return httpService:JSONDecode(body)
    end)
    if not decodeOk or type(decoded) ~= "table" or type(decoded.data) ~= "table" then
        return nil, nil, "the server list answer could not be read"
    end
    local nextCursor: any = decoded.nextPageCursor
    return decoded.data, if type(nextCursor) == "string" then nextCursor else nil, nil
end

-- Every candidate the place publishes, up to MAX_PAGES per sort order.
local function collectPool(orders: {string}): ({any}, number, string?)
    local here: string = jobId()
    local pool: {any} = {}
    local seen: {[string]: boolean} = {}
    local total: number = 0
    local lastError: string? = nil
    for _, order: string in ipairs(orders) do
        local cursor: string? = nil
        for _ = 1, MAX_PAGES do
            local page: {any}?, nextCursor: string?, errorText: string? =
                fetchPage(order, cursor)
            if not page then
                lastError = errorText
                break
            end
            for _, entry: any in ipairs(page) do
                total += 1
                if type(entry) == "table"
                    and type(entry.id) == "string"
                    and entry.id ~= ""
                    and entry.id ~= here
                    and not seen[entry.id] then
                    seen[entry.id] = true
                    table.insert(pool, entry)
                end
            end
            if not nextCursor then
                break
            end
            cursor = nextCursor
        end
    end
    return pool, total, lastError
end

-- A server is a candidate only when it has a slot this client can take.
-- `playing == maxPlayers` is a full instance: TeleportToPlaceInstance
-- rejects it, which is why v191 hops often died silently.
local function slotFree(entry: any): boolean
    local playing: number? = tonumber(entry and entry.playing)
    if not playing or playing < 0 then
        return false
    end
    local limit: number? = tonumber(entry and entry.maxPlayers)
    if limit and limit > 0 then
        return playing < limit
    end
    -- No limit published: treat anything under 100 as joinable.
    return playing < 100
end

local function playersIn(entry: any): number
    return tonumber(entry and entry.playing) or 0
end

local function joinablePool(entries: {any}): {any}
    local pool: {any} = {}
    for _, entry: any in ipairs(entries) do
        if slotFree(entry) then
            table.insert(pool, entry)
        end
    end
    return pool
end

local function pick(pool: {any}, mode: string): any
    if #pool == 0 then
        return nil
    end
    -- Empty instances are never "smallest": a 0/12 server answers no
    -- question the player asked and empties out again in seconds.
    local populated: {any} = {}
    for _, entry: any in ipairs(pool) do
        if playersIn(entry) >= 1 then
            table.insert(populated, entry)
        end
    end
    local candidates: {any} = if #populated > 0 then populated else pool
    if mode == "random" then
        return candidates[math.random(1, #candidates)]
    end
    local chosen: any = candidates[1]
    for index: number = 2, #candidates do
        local entry: any = candidates[index]
        if mode == "largest" then
            if playersIn(entry) > playersIn(chosen) then
                chosen = entry
            end
        elseif playersIn(entry) < playersIn(chosen) then
            chosen = entry
        end
    end
    return chosen
end

local function hop(mode: string): ()
    if hopping then
        return
    end
    if os.clock() - lastHopAt < HOP_COOLDOWN then
        say("Slow down: wait a moment before hopping again.")
        return
    end
    hopping = true
    task.spawn(function(): ()
        local ok: boolean, failure: any = pcall(function(): ()
            -- Smallest prefers quiet end, largest prefers busy end; random checks both.
            local orders: {string}
            if mode == "largest" then
                orders = {"Desc"}
            elseif mode == "random" then
                orders = {"Asc", "Desc"}
            else -- smallest
                orders = {"Asc"}
            end
            local entries: {any}, total: number, errorText: string? =
                collectPool(orders)
            local pool: {any} = joinablePool(entries)
            local chosen: any = pick(pool, mode)
            lastHopAt = os.clock()
            if not chosen then
                -- Nothing joinable in the list, but the player asked to hop:
                -- Teleport(place) lets Roblox pick a server with room. That
                -- always beats a notice that says "no server found" and
                -- leaves the player where they were.
                local reason: string = if total == 0
                    then tostring(errorText or "the server list is empty")
                    else string.format(
                        "all %d listed servers are full",
                        total
                    )
                say("Server hop: " .. reason .. " · joining any server instead…", "danger")
                local teleported: boolean = pcall(function(): ()
                    teleportService:Teleport(game.PlaceId, LocalPlayer)
                end)
                if not teleported then
                    say("Server hop was refused by TeleportService.", "danger")
                end
                return
            end
            local playing: number = playersIn(chosen)
            local limit: number = tonumber(chosen.maxPlayers) or 0
            say(string.format(
                "Hopping to the %s server: %d/%d players…",
                mode,
                playing,
                if limit > 0 then limit else playing + 1
            ))
            local teleported: boolean = pcall(function(): ()
                teleportService:TeleportToPlaceInstance(
                    game.PlaceId,
                    chosen.id,
                    LocalPlayer
                )
            end)
            if not teleported then
                say("Server hop was refused by TeleportService.", "danger")
            end
        end)
        hopping = false
        if not ok then
            say("Server hop failed: " .. tostring(failure), "danger")
        end
    end)
end
```

```
    card = createUniversalFeature(
        "Server Settings",
        "Live server readout (Job ID, players, uptime, place) plus hops that only land on a server with a free slot.",
        8,
        function(enabled: boolean): ()
            -- The readout never stops: switching the card only refreshes it at
            -- once so the rows are current when the card is looked at.
            if enabled then
                elapsed = STAT_INTERVAL
                refreshStats()
            end
        end,
        {categoryName = "Utilities"}
    )
    activeCard = card

    addActionOption(card, "Copy Job ID", copyJobId)
    addActionOption(card, "Hop: smallest server", function(): ()
        hop("smallest")
    end)
    addActionOption(card, "Hop: largest server", function(): ()
        hop("largest")
    end)
    addActionOption(card, "Hop: random server", function(): ()
        hop("random")
    end)
    addInformationOption(
        card,
        "A Roblox client cannot change another server's settings. This card "
            .. "shows YOUR server, live, and moves you between public ones. "
            .. "A hop only lands where a slot is free; \"smallest\" skips empty "
            .. "instances and \"largest\" is the fullest server you can still "
            .. "join (e.g. 11/12). If every listed server is full, the hop joins "
            .. "a random one instead of failing."
    )

    ensureStatRows()
    refreshStats()
    startTicker()

    state.serverSettingsCard = card
    state.serverSettings = settings
    state.serverSettingsStats = statRows
    -- Readable by the offline suites: the hop decision is pure, and a test can
    -- check it without a live Roblox client.
    state.serverSettingsPick = function(entries: {any}, mode: string): string?
        local here: string = jobId()
        local filtered: {any} = {}
        for _, entry: any in ipairs(entries or {}) do
            if type(entry) == "table"
                and type(entry.id) == "string"
                and entry.id ~= here
                and slotFree(entry) then
                table.insert(filtered, entry)
            end
        end
        local chosen: any = pick(filtered, mode)
        return if chosen then tostring(chosen.id) else nil
    end
    Module.Initialized = true
    return card
end

function Module.destroy(): ()
    if not Module.Initialized then
        return
    end
    Module.Initialized = false
    if type(disconnectTicker) == "function" then
        pcall(disconnectTicker, "ServerSettingsTick")
    end
    state.serverSettingsCard = nil
    state.serverSettings = nil
    state.serverSettingsStats = nil
    state.serverSettingsPick = nil
    activeCard = nil
end

return Module
```

```
-------------------------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Other/ServerSettings.lua
-------------------------------------------------------------------------------------------------


-------------------------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Render/Arrows.lua
BYTES: 7408
SHA-256: 4ff8abe51053a9ab806b176964f441c2fda3de2c14ccac33b119f004d5603954
INICIO DEL ARCHIVO
-------------------------------------------------------------------------------------------------
export type Runtime = {
    framework: any,
    entity: any,
    render: any,
    host: any,
    services: any,
}

-- v191: renamed from "Arrows" (the player asked for another name). Only the
-- visible label changed: the ConfigKey, the keybind and the saved state stay
-- "Universal.Arrows", so nothing has to be reconfigured. Its palette row in
-- Settings ("Buttons") now paints the window buttons too, the Pin chip
-- included.
local Module = {
    Name = "Player Pointers",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local activeCard: any = nil

function Module.init(context: Runtime): any
    local framework: any = context.framework
    local entity: any = context.entity
    local render: any = context.render
    local protectedTargets: any = context.services.protectedTargets
    local gameBridge: any = context.services.gameBridge
    local teamCheck: any = context.host.state and context.host.state.teamCheck
    local currentWorkspace: Workspace = context.host.workspace or workspace
    local layer: Frame = render:Layer("ArrowsLayer")
    local shown: {[Player]: boolean} = {}

    local function roleColour(player: Player): Color3?
        if gameBridge and type(gameBridge.playerRoleColor) == "function" then
            local ok: boolean, colour: any =
                pcall(gameBridge.playerRoleColor, player)
            if ok and typeof(colour) == "Color3" then
                return colour
            end
        end
        return nil
    end

    local card: any
    local function clear(): ()
        render:ReleaseAll(layer)
        table.clear(shown)
        layer.Visible = false
    end

    card = framework.Categories.Visuals:CreateModule({
        Name = "Player Pointers",
        Category = "Render",
        ConfigKey = "Universal.Arrows",
        Order = 3,
        Tooltip = "Points toward players outside the camera view.",
        Function = function(enabled: boolean): ()
            layer.Visible = enabled
            if not enabled then
                clear()
                card:SetStatus(nil)
                return
            end
            card:Render(function(): ()
                local camera: Camera? = currentWorkspace.CurrentCamera
                if not camera then
                    render:ReleaseAll(layer)
                    table.clear(shown)
                    return
                end
                local viewport: Vector2 = camera.ViewportSize
                local centre: Vector2 = Vector2.new(viewport.X / 2, viewport.Y / 2)
                local radius: number = math.max(40, math.min(viewport.X, viewport.Y) * 0.4)
                local seen: {[Player]: boolean} = {}
                local count: number = 0
                for _, target: any in ipairs(entity:Refresh()) do
                    local player: Player = target.Player
                    local friendly: boolean = target.IsFriendly
                    if not friendly
                        and teamCheck
                        and type(teamCheck.isSameTeam) == "function" then
                        friendly = teamCheck.isSameTeam(player)
                    end
                    if (not card.Options["Teammates"].Value and friendly)
                        or protectedTargets.isProtected(player)
                        or not target.Humanoid
                        or target.Humanoid.Health <= 0
                        or not target.RootPart
                        or target.Distance > card.Options["Max distance"].Value then
                        render:Release(layer, player)
                        continue
                    end
                    local projected: Vector3, onScreen: boolean =
                        camera:WorldToViewportPoint(target.RootPart.Position)
                    if onScreen then
                        render:Release(layer, player)
                        continue
                    end
                    local drawing: any = render:Set(layer, player)
                    local delta: Vector2 = Vector2.new(projected.X, projected.Y) - centre
                    if projected.Z < 0 then
                        delta = Vector2.new(-delta.X, -delta.Y)
                    end
                    if delta.Magnitude < 0.001 then
                        delta = Vector2.new(0, -1)
                    end
                    local direction: Vector2 = delta.Unit
                    local perpendicular: Vector2 = Vector2.new(-direction.Y, direction.X)
                    local tip: Vector2 = centre + direction * radius
                    local back: Vector2 = tip - direction * 16
                    -- Colour priority: the game's role colour when roles are
                    -- known (green by default otherwise), then teammates in
                    -- their team colour while the team-colour option is on.
                    local roleColourForTarget: Color3? = roleColour(player)
                    local colour: Color3 = roleColourForTarget
                        or (card.Options["Team colours"].Value
                            and friendly
                            and player.Team
                            and player.Team.TeamColor.Color
                            or card.Options["Colour"].Value)
                    drawing:Line("Arrow", 1, tip, back + perpendicular * 8, 2, colour)
                    drawing:Line("Arrow", 2, tip, back - perpendicular * 8, 2, colour)
                    drawing:HideLines("Arrow", 3, 8)
                    if card.Options["Distance"].Value then
                        drawing:Label(
                            "Distance",
                            "[" .. tostring(math.round(target.Distance)) .. "]",
                            tip - direction * 26,
                            0.5,
                            12,
                            colour,
                            true
                        )
                    else
                        drawing:HideLabel("Distance")
                    end
                    drawing:Show(true)
                    seen[player] = true
                    count += 1
                end
                for player: Player in pairs(shown) do
                    if not seen[player] then
                        render:Release(layer, player)
                    end
                end
                shown = seen
                card:SetStatus(tostring(count))
            end)
            card:Clean(clear)
        end,
    })

    card:CreateColor({Name = "Colour", Default = Color3.fromRGB(85, 255, 85), Tooltip = "Base arrow colour. Arrows use role colours instead whenever the game knows roles."})
    card:CreateToggle({Name = "Distance", Default = true})
    card:CreateSlider({Name = "Max distance", Min = 25, Max = 2000, Step = 25, Default = 500})
    card:CreateToggle({Name = "Teammates", Default = false, Tooltip = "Also point at teammates. The advanced team check decides who counts."})
    card:CreateToggle({Name = "Team colours", Default = false, Tooltip = "Teammates point in their Roblox team colour; everyone else keeps the base colour. Role colours always win."})

    activeCard = card
    Module.Initialized = true
    return card
end

function Module.destroy(): ()
    if activeCard and activeCard.Enabled then
        activeCard:Toggle(false)
    end
    activeCard = nil
    Module.Initialized = false
end

return Module

-------------------------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Render/Arrows.lua
-------------------------------------------------------------------------------------------------


-------------------------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Render/Chams.lua
BYTES: 15446
SHA-256: 56f302a9217eb3070da8b052c65fdcb6b1e51fe098a1100c27a82adfbe3ae0df
INICIO DEL ARCHIVO
-------------------------------------------------------------------------------------------------
export type Runtime = {
    framework: any,
    entity: any,
    host: any,
    services: any,
}

local Module = {
    Name = "Chams",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local activeCard: any = nil
local activeBridgeCleanup: (() -> ())? = nil

function Module.init(context: Runtime): any
    local framework: any = context.framework
    local entity: any = context.entity
    local host: any = context.host
    local protectedTargets: any = context.services.protectedTargets
    local gameBridge: any = context.services.gameBridge
    local teamCheck: any = host.state and host.state.teamCheck
    local currentWorkspace: Workspace = host.workspace or workspace
    local highlights: {[Player]: Highlight} = {}
    local lastUpdate: number = -math.huge
    local roles: any = nil
    local function playerPerk(player: Player): string?
        if type(gameBridge) ~= "table"
            or type(gameBridge.playerPerk) ~= "function" then
            return nil
        end
        local ok: boolean, perk: any = pcall(gameBridge.playerPerk, player)
        if ok and type(perk) == "string" then
            return perk
        end
        return nil
    end

    -- MM2 perk assets; this module owns only the icon, not text labels.
    local PERK_ICONS: {[string]: string} = {
        ["Footsteps"] = "rbxassetid://336404773",
        ["Sleight"] = "rbxassetid://340309581",
        ["Ninja"] = "rbxassetid://416832054",
        ["Haste"] = "rbxassetid://416942717",
        ["FakeGun"] = "rbxassetid://17899774181",
        ["Fake Gun"] = "rbxassetid://17899774181",
        ["Ghost"] = "rbxassetid://335206856",
        ["Xray"] = "rbxassetid://340313896",
        ["X-Ray"] = "rbxassetid://340313896",
        ["Trap"] = "rbxassetid://4966828196",
        ["Sprint"] = "rbxassetid://4970219155",
        ["VampireBat"] = "rbxassetid://129078122681639",
        ["Bat Form"] = "rbxassetid://129078122681639",
        ["Decoy"] = "rbxassetid://113722008817201",
    }
```

```
    local perkTag: BillboardGui? = nil

    local function destroyPerkTag(): ()
        if perkTag then
            pcall(perkTag.Destroy, perkTag)
            perkTag = nil
        end
    end

    local function clear(): ()
        for player: Player, highlight: Highlight in pairs(highlights) do
            highlights[player] = nil
            highlight:Destroy()
        end
        destroyPerkTag()
    end

    if type(gameBridge) == "table" and type(gameBridge.onEvent) == "function" then
        activeBridgeCleanup = gameBridge.onEvent(function(kind: string): ()
            if kind == "roleUpdate" then
                lastUpdate = -math.huge
            end
        end)
    end

    local store: any = host.configData
    if store and store.values and store.states then
        local oldMode: any = store.values["Universal.PlayerESP.Chams"]
        if store.states["Universal.Chams"] == nil
            and (oldMode == "Overlay" or oldMode == "Outline") then
            store.states["Universal.Chams"] = true
            local fillKey: string = "Universal.Chams.Filltransparency"
            if store.values[fillKey] == nil then
                store.values[fillKey] = oldMode == "Outline" and 1 or 0.55
            end
            if type(host.queueConfigSave) == "function" then
                host.queueConfigSave()
            end
        end
        -- The old build shipped team colours ON by default, which painted
        -- everyone red when no team data existed. Forget that legacy state
        -- (unless the player actually customised the colours) so the new
        -- white default applies.
        local teamKey: string = "Universal.Chams.Teamcolours"
        if store.states[teamKey] == true
            and store.values["Universal.Chams.Fillcolour"] == nil
            and store.values["Universal.Chams.Outlinecolour"] == nil then
            store.states[teamKey] = false
            if type(host.queueConfigSave) == "function" then
                host.queueConfigSave()
            end
        end
    end

    local card: any

    card = framework.Categories.Visuals:CreateModule({
        Name = "Chams",
        Category = "Render",
        ConfigKey = "Universal.Chams",
        Order = 2,
        Tooltip = "Player highlights with independent role palettes and filters. Names and health have their own modules.",
        Function = function(enabled: boolean): ()
            if not enabled then
                clear()
                card:SetStatus(nil)
                return
            end
            card:SetStatus("0")

            card:Render(function(): ()
                local now: number = os.clock()
                if now - lastUpdate < 0.2 then
                    return
                end
                lastUpdate = now
                local seen: {[Player]: boolean} = {}
                local count: number = 0
                local roleMode = roles and roles.provider ~= nil
                local teamsReplaced = roleMode and roles.provider.ReplacesTeams == true
                local useTeamColours = not teamsReplaced and card.Options["Team colours"].Value
                local allowTeammates = not teamsReplaced and card.Options["Teammates"].Value
                local showPerk = card.Options["Show Perk"] and card.Options["Show Perk"].Value
                local perkTarget: Player? = nil
                local targets: any = entity:Refresh()
                -- Dead is a real MM2 role, not an inert colour row. Include
                -- existing corpse models only when the provider declares it.
                if roleMode and type(roles.provider.DeadRoles) == "table" and host.Players then
                    targets = table.clone(targets)
                    local included = {}
                    for _, target in ipairs(targets) do included[target.Player] = true end
                    for _, player in ipairs(host.Players:GetPlayers()) do
                        if player == host.LocalPlayer or included[player] then continue end
                        local character = player.Character
                        local humanoid = character and character:FindFirstChildOfClass("Humanoid")
                        if character and character.Parent and humanoid and humanoid.Health <= 0
                            and table.find(roles.provider.DeadRoles, roles:Role(player)) then
                            table.insert(targets, {Player = player, Character = character, Humanoid = humanoid,
                                IsFriendly = entity.IsFriendly and entity:IsFriendly(player) or false})
                        end
                    end
                end
                for _, target: any in ipairs(targets) do
                    local player: Player = target.Player

                    -- Universal games: the teammate toggle decides who is
                    -- highlighted. Role games: every role is known, per-role
                    -- toggles decide, and role colours always win.
                    local allowedTeam: boolean = true
                    if not teamsReplaced then
                        local friendly: boolean = target.IsFriendly
                        if not friendly and teamCheck
                            and type(teamCheck.isSameTeam) == "function" then
                            friendly = teamCheck.isSameTeam(player)
                        end
                        allowedTeam = allowTeammates or not friendly
                    end
                    if not (allowedTeam
                        and not protectedTargets.isProtected(player)
                        and target.Character
                        and target.Humanoid
                        and (target.Humanoid.Health > 0 or roleMode and type(roles.provider.DeadRoles) == "table"
                            and table.find(roles.provider.DeadRoles, roles:Role(player)))) then
                        continue
                    end

                    local roleColour: Color3? = nil
                    if roleMode then
                        if not roles:Allows(player) then continue end
                        local role
                        roleColour, role = roles:Resolve(player, nil)
                        if role == "Murderer" then perkTarget = player end
                    end
                    local teamColour: Color3? = nil
                    if not roleColour and useTeamColours and target.IsFriendly then
                        local team: Team? = player.Team
                        if team then
                            teamColour = team.TeamColor.Color
                        end
                    end
                    local fillColour: Color3 = roleColour
                        or teamColour
                        or card.Options["Fill colour"].Value
                    local outlineColour: Color3 = roleColour
                        or teamColour
                        or card.Options["Outline colour"].Value
                    local fillTransparency: number =
                        card.Options["Fill transparency"].Value
                    local outlineTransparency: number =
                        card.Options["Outline transparency"].Value
                    local depthMode: Enum.HighlightDepthMode =
                        card.Options["Through walls"].Value
                            and Enum.HighlightDepthMode.AlwaysOnTop
                        or Enum.HighlightDepthMode.Occluded

                    seen[player] = true
                    count += 1

                    local highlight: Highlight? = highlights[player]
                    if not highlight or not highlight.Parent then
                        highlight = Instance.new("Highlight")
                        highlight.Name = "Wurst_Chams"
                        highlight.Parent = currentWorkspace
                        highlights[player] = highlight
                    end
                    local resolved: Highlight = highlight :: Highlight
                    if resolved.Adornee ~= target.Character then
                        resolved.Adornee = target.Character
                    end
                    if resolved.FillColor ~= fillColour then
                        resolved.FillColor = fillColour
                    end
                    if resolved.OutlineColor ~= outlineColour then
                        resolved.OutlineColor = outlineColour
                    end
                    if resolved.FillTransparency ~= fillTransparency then
                        resolved.FillTransparency = fillTransparency
                    end
                    if resolved.OutlineTransparency ~= outlineTransparency then
                        resolved.OutlineTransparency = outlineTransparency
                    end
                    if resolved.DepthMode ~= depthMode then
                        resolved.DepthMode = depthMode
                    end

                end

                for player: Player, highlight: Highlight in pairs(highlights) do
                    if not seen[player] then
                        highlights[player] = nil
                        highlight:Destroy()
                    end
                end
                -- Perk icon above the murderer (role games only).
                if roleMode and showPerk and perkTarget and perkTarget.Character then
                    local head: BasePart? = perkTarget.Character:FindFirstChild("Head")
                        or perkTarget.Character:FindFirstChild("HumanoidRootPart")
                        :: BasePart?
                    local perk: string? = head and playerPerk(perkTarget) or nil
                    if head and perk then
                        local iconId: string? = PERK_ICONS[perk]
                            or PERK_ICONS[string.gsub(perk, "%s+", "")]
                        if iconId then
                            if not perkTag
                                or not perkTag.Parent
                                or perkTag.Adornee ~= head then
                                destroyPerkTag()
                                local billboard: BillboardGui = Instance.new("BillboardGui")
                                billboard.Name = "Wurst_ChamPerk"
                                billboard.AlwaysOnTop = true
                                billboard.LightInfluence = 0
                                billboard.Size = UDim2.fromOffset(26, 26)
                                billboard.StudsOffset = Vector3.new(0, 2.9, 0)
                                billboard.MaxDistance = 1500
                                billboard.Adornee = head
                                billboard.Parent = head
                                local icon: ImageLabel = Instance.new("ImageLabel")
                                icon.Name = "Perk"
                                icon.BackgroundTransparency = 1
                                icon.Size = UDim2.fromScale(1, 1)
                                icon.Image = iconId
                                icon.Parent = billboard
                                perkTag = billboard
                            end
                            local icon: any = perkTag and perkTag:FindFirstChild("Perk")
                            if icon and icon.Image ~= iconId then
                                icon.Image = iconId
                            end
                        else
                            destroyPerkTag()
                        end
                    else
                        destroyPerkTag()
                    end
                else
                    destroyPerkTag()
                end

                card:SetStatus(tostring(count))
            end)
            card:Clean(clear)
        end,
    })

    card:CreateColor({Name = "Fill colour", Default = Color3.fromRGB(85, 255, 85)})
    card:CreateColor({Name = "Outline colour", Default = Color3.fromRGB(85, 255, 85)})
    card:CreateSlider({Name = "Fill transparency", Min = 0, Max = 1, Step = 0.05, Default = 0.55})
    card:CreateSlider({Name = "Outline transparency", Min = 0, Max = 1, Step = 0.05, Default = 0})
    card:CreateToggle({Name = "Through walls", Default = true})
    card:CreateToggle({Name = "Teammates", Default = false})
    card:CreateToggle({Name = "Team colours", Default = false})
    -- The role provider can arrive after the universal cards are built.
    -- Keep this row hidden in other games via OptionFilter below.
    card:CreateToggle({Name = "Show Perk", Default = true, Tooltip = "Perk icon above the murderer."})
    roles = context.services.gameRoles:Bind(card, {filters = true})
    card.OptionFilter = function(name)
        if roles.provider and roles.provider.ReplacesTeams and (name == "Teammates" or name == "Team colours") then return false end
        if name == "Show Perk" then return roles.provider ~= nil and roles.provider.Name == "MM2" end
        return true
    end
    card:RefreshVisibility()
    Module.roles = roles
    activeCard = card
    Module.Initialized = true
    return card
end

function Module.destroy(): ()
    if activeCard and activeCard.Enabled then
        activeCard:Toggle(false)
    end
    if activeBridgeCleanup then
        pcall(activeBridgeCleanup)
        activeBridgeCleanup = nil
    end
    if Module.roles then Module.roles:Destroy(); Module.roles = nil end
    activeCard = nil
    Module.Initialized = false
end

return Module

----------------------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Render/Chams.lua
----------------------------------------------------------------------------------------------

----------------------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Render/FieldOfView.lua
BYTES: 2150
SHA-256: e4eeedfe8d0ff5ae735fe271e94bbcc50c5ea424c7399f557079b52dd27908d1
INICIO DEL ARCHIVO
----------------------------------------------------------------------------------------------
export type Runtime = {
    framework: any,
    entity: any,
    host: any,
    services: any,
}

local Module = {
    Name = "FieldOfView",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local activeCleanup: (() -> ())? = nil

function Module.init(context: Runtime): any
    local host: any = context.host
    local TaskManager: any = host.TaskManager
    local featureConnections: any = host.featureConnections
    local disconnectFeatureConnection: any = host.disconnectFeatureConnection
    local fovOwnership: any = context.services.fovOwnership
    local createUniversalFeature: any = host.createUniversalFeature
    local addNumberOption: any = host.addNumberOption
    local workspace: any = host.workspace
    local fovSettings = {value = 90}
    local FovFeature: any = nil

    local function toggleFov(enabled)
        disconnectFeatureConnection("FOV")
        fovOwnership.set("Universal", enabled and fovSettings.value or nil)
        if not enabled then
            if FovFeature then FovFeature:SetStatus(nil) end
            return
        end

        FovFeature:SetStatus(tostring(math.round(fovSettings.value)))
        featureConnections.FOV = TaskManager:Connect(function()
            fovOwnership.set("Universal", fovSettings.value)
        end, nil, "FOV")
    end

    FovFeature = createUniversalFeature(
        "FOV",
        "Keep the camera field of view fixed",
        10,
        toggleFov,
        {categoryName = "Render"}
    )
    addNumberOption(FovFeature, "Field of view", fovSettings.value, 20, 120, function(value)
        fovSettings.value = value
        if FovFeature.enabled then
            FovFeature:SetStatus(tostring(math.round(value)))
        end
    end)

    activeCleanup = function(): ()
        disconnectFeatureConnection("FOV")
        fovOwnership.set("Universal", nil)
    end
    Module.Initialized = true
    return FovFeature
end

function Module.destroy(): ()
    if activeCleanup then
        activeCleanup()
    end
    activeCleanup = nil
    Module.Initialized = false
end

return Module

----------------------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Render/FieldOfView.lua
----------------------------------------------------------------------------------------------

----------------------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Render/FreeCam.lua
BYTES: 14561
SHA-256: 98fcd894760cf198d3023d04349a25316e13abeddfb9db0d726182b22122a1ed
INICIO DEL ARCHIVO
----------------------------------------------------------------------------------------------
local Module = {Name = "FreeCam", Initialized = false}
local card, services

function Module.init(context)
    local host = context.host
    services = context.services
    local input, run = host.UserInputService, host.RunService
    local actions = game:GetService("ContextActionService")
    local access = context.services.cameraAccess
    local world = host.workspace or workspace
    local create = host.create
    local id = "ARandomMenu.FreeCam." .. tostring({})
    local cameraState = {IsActive=function() return card ~= nil and card.Enabled end}
    context.services.freeCam = cameraState
    local function spectating()
        return context.services.spectate and context.services.spectate.IsActive()
    end

    card = context.framework.Categories.Visuals:CreateModule({
        Name = "FreeCam", Category = "World", ConfigKey = "Universal.FreeCam", Order = 6,
        Tooltip = "Move the camera without moving your character. WASD, Q/E, right-drag; touch controls on mobile.",
        Function = function(enabled)
            cameraState.Session = enabled and {} or nil
            if not enabled then card:SetStatus(nil); return end
            local keys, held, connections = {}, {}, {}
            local camera, saved, position, pitch, yaw
            local writtenFrame, writtenFocus
            local controls, controlsEnabled
            local touchGui, touchGuiEnabled
            local moveTouch, lookTouch, dragging
            local stick = Vector2.zero
            local mouseMode, mouseIcon
            local gui, pad, knob

            local function connect(signal, callback)
                table.insert(connections, signal:Connect(callback))
            end
            local function releaseMouse()
                if not dragging then return end
                dragging = false
                if input.MouseBehavior == Enum.MouseBehavior.LockCurrentPosition then input.MouseBehavior = mouseMode end
                if input.MouseIconEnabled == false then input.MouseIconEnabled = mouseIcon end
            end
            local function resetInput()
                table.clear(keys); table.clear(held)
                moveTouch, lookTouch = nil, nil
                stick = Vector2.zero
                if knob then knob.Position = UDim2.fromScale(0.5,0.5) end
                releaseMouse()
            end
            local function restoreCamera()
                if camera and saved then
                    pcall(function()
                        if camera.CameraType ~= Enum.CameraType.Scriptable then return end
                        camera.CameraType = saved.kind
                        if camera.CFrame == writtenFrame then camera.CFrame = saved.frame end
                        if camera.Focus == writtenFocus then camera.Focus = saved.focus end
                    end)
                end
                camera, saved, writtenFrame, writtenFocus = nil, nil, nil, nil
            end
            local function releaseControls()
                if controls and controlsEnabled then pcall(controls.Enable, controls) end
                controls, controlsEnabled = nil, nil
                if touchGui then pcall(function() touchGui.Enabled = touchGuiEnabled end) end
                touchGui = nil
            end
            card:Clean(function()
                run:UnbindFromRenderStep(id)
                table.clear(keys); table.clear(held)
                for _, name in ipairs({"Forward","Back","Left","Right","Up","Down"}) do actions:UnbindAction(id .. name) end
                for _, connection in ipairs(connections) do connection:Disconnect() end
                releaseMouse(); restoreCamera(); releaseControls()
                if gui then gui:Destroy() end
            end)
            local function hold(name, source, down)
                held[name] = held[name] or {}
                held[name][source] = down and true or nil
                keys[name] = next(held[name]) ~= nil
            end
            for _, definition in ipairs({
                {"Forward",Enum.KeyCode.W,Enum.KeyCode.Up}, {"Back",Enum.KeyCode.S,Enum.KeyCode.Down},
                {"Left",Enum.KeyCode.A,Enum.KeyCode.Left}, {"Right",Enum.KeyCode.D,Enum.KeyCode.Right},
                {"Up",Enum.KeyCode.E,Enum.KeyCode.Space}, {"Down",Enum.KeyCode.Q,Enum.KeyCode.LeftControl},
            }) do
                local name = definition[1]
                actions:BindActionAtPriority(id .. name, function(_, state, event)
                    if spectating() or input:GetFocusedTextBox() then held[name] = nil; keys[name] = false; return Enum.ContextActionResult.Pass end
                    if state == Enum.UserInputState.Cancel then held[name] = nil; keys[name] = false
                    else hold(name,event.KeyCode,state == Enum.UserInputState.Begin or state == Enum.UserInputState.Change) end
                    return Enum.ContextActionResult.Sink
                end, false, Enum.ContextActionPriority.High.Value + 1, definition[2], definition[3])
            end
            local function pan(delta)
                if spectating() then return end
                local sensitivity = math.rad(card.Options.Sensitivity.Value)
                yaw -= delta.X * sensitivity
                pitch = math.clamp(pitch - delta.Y * sensitivity, -math.rad(89), math.rad(89))
            end
            connect(input.InputBegan, function(event, processed)
                if processed or spectating() or input:GetFocusedTextBox() or not position then return end
                if event.UserInputType == Enum.UserInputType.MouseButton2 then
                    mouseMode, mouseIcon = input.MouseBehavior, input.MouseIconEnabled
                    dragging = true
                    input.MouseBehavior = Enum.MouseBehavior.LockCurrentPosition
                    input.MouseIconEnabled = false
                end
            end)
            connect(input.InputChanged, function(event)
                if dragging and event.UserInputType == Enum.UserInputType.MouseMovement and not input:GetFocusedTextBox() then pan(event.Delta) end
            end)
            connect(input.InputEnded, function(event)
                if event.UserInputType == Enum.UserInputType.MouseButton2 then releaseMouse() end
            end)
            connect(input.WindowFocusReleased, resetInput)
            connect(input.TextBoxFocused, resetInput)

            if input.TouchEnabled then
                gui = create("Frame", {Name="FreeCamControls", Parent=host.ScreenGui, Size=UDim2.fromScale(1,1), BackgroundTransparency=1, ZIndex=30})
                local function button(name, text, at, size)
                    local object = create("TextButton", {Name=name, Parent=gui, Position=at, Size=size, Text=text, Font=Enum.Font.Code,
                        TextSize=18, TextColor3=Color3.new(1,1,1), BackgroundColor3=Color3.fromRGB(25,25,25), BackgroundTransparency=0.3,
                        BorderSizePixel=1, BorderColor3=Color3.fromRGB(120,120,120), AutoButtonColor=true, ZIndex=31})
                    if host.ThemeEngine then
                        host.ThemeEngine.Bind(object,"BackgroundColor3","background")
                        host.ThemeEngine.Bind(object,"TextColor3","text")
                    end
                    return object
                end
                pad = button("Move", "", UDim2.new(0,22,1,-152), UDim2.fromOffset(104,104))
                knob = create("Frame", {Parent=pad, Name="Stick", AnchorPoint=Vector2.new(0.5,0.5), Position=UDim2.fromScale(0.5,0.5),
                    Size=UDim2.fromOffset(26,26), BackgroundColor3=Color3.fromRGB(220,220,220), BorderSizePixel=0, ZIndex=32})
                local function moveStick(event)
                    local centre = pad.AbsolutePosition + pad.AbsoluteSize * 0.5
                    local delta = (Vector2.new(event.Position.X,event.Position.Y) - centre) / 42
                    stick = delta.Magnitude > 1 and delta.Unit or delta
                    if stick.Magnitude < 0.12 then stick = Vector2.zero end
                    knob.Position = UDim2.new(0.5,stick.X*36,0.5,stick.Y*36)
                end
                connect(pad.InputBegan, function(event)
                    if event.UserInputType == Enum.UserInputType.Touch and not moveTouch then moveTouch = event; moveStick(event) end
                end)
                for _, name in ipairs({"Up","Down"}) do
                    local object = button(name, name == "Up" and "+" or "−", UDim2.new(1,-70,1,name == "Up" and -152 or -96), UDim2.fromOffset(48,48))
                    connect(object.InputBegan, function(event) if event.UserInputType == Enum.UserInputType.Touch then hold(name,event,true) end end)
                    connect(object.InputEnded, function(event) if event.UserInputType == Enum.UserInputType.Touch then hold(name,event,false) end end)
                end
                local exit = button("Exit", "Exit FreeCam", UDim2.new(0.5,-66,0,48), UDim2.fromOffset(132,30))
                connect(exit.Activated, function()
                    if card.Feature and card.Feature.enabled and card.Feature.activate then card.Feature.activate() else card:Toggle(false) end
                end)
                connect(input.TouchStarted, function(event, processed)
                    local current = world.CurrentCamera
                    if not processed and not spectating() and not lookTouch and event ~= moveTouch and current
                        and event.Position.X > current.ViewportSize.X * 0.45 and not input:GetFocusedTextBox() then lookTouch = event end
                end)
                connect(input.TouchMoved, function(event)
                    if event == moveTouch then
                        moveStick(event)
                    elseif event == lookTouch and position then pan(event.Delta) end
                end)
                connect(input.TouchEnded, function(event)
                    hold("Up",event,false); hold("Down",event,false)
                    if event == moveTouch then moveTouch = nil; stick = Vector2.zero; knob.Position = UDim2.fromScale(0.5,0.5) end
                    if event == lookTouch then lookTouch = nil end
                end)
            end
            card:SetStatus("WASD / touch")
            run:BindToRenderStep(id, Enum.RenderPriority.Camera.Value + 1, function(dt)
                if spectating() then
                    resetInput()
                    if gui then gui.Visible = false end
                    return
                end
                if gui then gui.Visible = true end
                local nextCamera = world.CurrentCamera
                if camera ~= nextCamera then
                    restoreCamera()
                    camera = nextCamera
                    if camera then
                        saved = {kind=camera.CameraType, frame=camera.CFrame, focus=camera.Focus}
                        if not position then
                            position = camera.CFrame.Position
                            local look = camera.CFrame.LookVector
                            pitch, yaw = math.asin(math.clamp(look.Y,-1,1)), math.atan2(-look.X,-look.Z)
                        end
                    end
                end
                if not camera then return end
                if writtenFrame and camera.CameraType ~= Enum.CameraType.Scriptable then
                    saved.kind, saved.frame, saved.focus = camera.CameraType, camera.CFrame, camera.Focus
                elseif writtenFrame and camera.CFrame ~= writtenFrame then
                    saved.frame, saved.focus = camera.CFrame, camera.Focus
                end
                local nextControls = access:GetControls()
                if type(nextControls) ~= "table" or type(nextControls.Disable) ~= "function" or type(nextControls.Enable) ~= "function" then nextControls = nil end
                if nextControls ~= controls then
                    releaseControls(); controls = nextControls
                    if controls then
                        controlsEnabled = controls.controlsEnabled ~= false
                        local ok = pcall(controls.Disable, controls)
                        if not ok then controls, controlsEnabled = nil, nil end
                    end
                end
                local playerGui = host.LocalPlayer:FindFirstChildOfClass("PlayerGui")
                local nativeTouch = playerGui and playerGui:FindFirstChild("TouchGui")
                if nativeTouch ~= touchGui then
                    if touchGui then pcall(function() touchGui.Enabled = touchGuiEnabled end) end
                    touchGui = nativeTouch
                    if touchGui then touchGuiEnabled = touchGui.Enabled end
                end
                if touchGui then touchGui.Enabled = false end
                if not controls then
                    local character = host.LocalPlayer.Character
                    local humanoid = character and character:FindFirstChildOfClass("Humanoid")
                    if humanoid then humanoid:Move(Vector3.zero, false) end
                end
                local rotation = CFrame.fromOrientation(pitch,yaw,0)
                if not input:GetFocusedTextBox() then
                    local move = Vector3.new((keys.Right and 1 or 0)-(keys.Left and 1 or 0)+stick.X,
                        (keys.Up and 1 or 0)-(keys.Down and 1 or 0), (keys.Back and 1 or 0)-(keys.Forward and 1 or 0)+stick.Y)
                    local direction = rotation:VectorToWorldSpace(Vector3.new(move.X,0,move.Z)) + Vector3.yAxis * move.Y
                    if direction.Magnitude > 1 then direction = direction.Unit end
                    local speed = card.Options.Speed.Value * (input:IsKeyDown(Enum.KeyCode.LeftShift) and 3 or 1)
                    position += direction * speed * math.clamp(dt,0,0.1)
                end
                camera.CameraType = Enum.CameraType.Scriptable
                camera.CFrame = CFrame.new(position) * rotation
                camera.Focus = camera.CFrame * CFrame.new(0,0,-16)
                writtenFrame, writtenFocus = camera.CFrame, camera.Focus
            end)
        end,
    })
    card:CreateSlider({Name="Speed", Min=1, Max=250, Default=48})
    card:CreateSlider({Name="Sensitivity", Min=0.05, Max=0.6, Default=0.2})
    Module.Initialized = true
    return card
end
function Module.destroy()
    if card and card.Enabled then card:Toggle(false) end
    if services then services.freeCam = nil end
    card, services = nil, nil
    Module.Initialized = false
end
return Module

----------------------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Render/FreeCam.lua
----------------------------------------------------------------------------------------------

----------------------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Render/Fullbright.lua
BYTES: 3121
SHA-256: 2c7c34841dd7cd23329373d74916d276f1eefe50d00976d41e651b72bf5882a2
INICIO DEL ARCHIVO
----------------------------------------------------------------------------------------------
local Module = {Name = "Fullbright", PlaceId = 0, Initialized = false}
local card

function Module.init(context)
    local lighting = context.host.Lighting
    local original, applied = {}, {}
    local originalTime, appliedTime, elapsed
    local function write(key, value)
        local current = lighting[key]
        if applied[key] ~= nil and current ~= applied[key] then original[key] = current end
        if original[key] == nil then original[key] = current end
        if current ~= value then lighting[key] = value end
        applied[key] = value
    end
    local function restoreTime()
        if appliedTime ~= nil and lighting.ClockTime == appliedTime then lighting.ClockTime = originalTime end
        originalTime, appliedTime = nil, nil
    end
    local function restore()
        restoreTime()
        for key, value in pairs(original) do if lighting[key] == applied[key] then lighting[key] = value end end
        table.clear(original); table.clear(applied)
    end
    local function tick(dt)
        -- Roblox does not accept math.huge for Lighting.Brightness; clamp
        -- the display value to a large finite number so infinity on the
        -- slider still produces a fully white screen.
        local brightness = card.Options.Brightness.Value
        if brightness == math.huge then brightness = 1000 end
        write("Brightness",brightness)
```

```
        write("GlobalShadows",false)
        write("FogEnd",100000)
        write("Ambient",Color3.fromRGB(178,178,178))
        write("OutdoorAmbient",Color3.fromRGB(178,178,178))
        if card.Options["Time Changer"].Value then
            if appliedTime ~= nil and lighting.ClockTime ~= appliedTime then originalTime = lighting.ClockTime end
            if originalTime == nil then originalTime = lighting.ClockTime end
            elapsed = (elapsed + math.max(0,dt or 0)) % 4
            appliedTime = (card.Options["Fast time"].Value and elapsed * 6 or card.Options.Time.Value) % 24
            if lighting.ClockTime ~= appliedTime then lighting.ClockTime = appliedTime end
        else restoreTime() end
    end
    card = context.framework.Categories.Render:CreateModule({Name="Fullbright",Category="Render",ConfigKey="Universal.Fullbright",
        Tooltip="Bright lighting; optionally control the time of day.",Function=function(enabled)
            if not enabled then restore(); return end
            elapsed = 0
            card:Clean(restore)
            tick(0);card:Loop(tick)
        end})
    card:CreateSlider({Name="Brightness",Min=-10,Max=math.huge,Default=3})
    card:CreateToggle({Name="Time Changer",Default=false,Function=function(value) if not value then restoreTime() end end})
    card:CreateSlider({Name="Time",Min=0,Max=24,Default=14,Show={Option="Time Changer",Values={true}}})
    card:CreateToggle({Name="Fast time",Default=false,Show={Option="Time Changer",Values={true}},Tooltip="Loop a full day every four seconds."})
    Module.Initialized=true
    return card
end
function Module.destroy()
    if card and card.Enabled then card:Toggle(false) end
    card=nil;Module.Initialized=false
end
return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Render/Fullbright.lua
--------------------------------------------------------------------------------


--------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Render/HP.lua
BYTES: 7073
SHA-256: 70e758984b8a181c97490305111b891346bf8a77e187d6458d117afe1f32bc9d
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
local Module = {Name = "HP", Initialized = false}
local activeCard, cleanup, hud, roleBinding
local LOW_HEALTH = Color3.fromRGB(255,70,70)
local MID_HEALTH = Color3.fromRGB(255,220,60)
local HIGH_HEALTH = Color3.fromRGB(85,255,85)

-- Shared by world bars and Self HP; finite/max-health guards avoid NaN UI sizes.
local function healthValues(health, maximum)
    if type(health) ~= "number" or health ~= health or math.abs(health) == math.huge then health = 0 end
    if type(maximum) ~= "number" or maximum ~= maximum or maximum <= 0 or maximum == math.huge then maximum = 100 end
    health = math.max(0, health)
    return health, maximum, math.clamp(health / maximum, 0, 1)
end
local function healthColour(fraction)
    return fraction < 0.5 and LOW_HEALTH:Lerp(MID_HEALTH, fraction * 2)
        or MID_HEALTH:Lerp(HIGH_HEALTH, (fraction - 0.5) * 2)
end

function Module.init(context: any): any
    local host, render, entity = context.host, context.render, context.entity
    local create = host.create
    local layer, tracked = render:Layer("HPLayer"), {}
    local store = host.configData
    if store and store.states["Universal.HP"] == nil then
        store.states["Universal.HP"] = store.states["Universal.PlayerESP"] == true
            and (store.states["Universal.PlayerESP.Healthbar"] ~= false or store.states["Universal.PlayerESP.Healthtext"] == true)
    end
    -- Vape Health is a simple number + heart near the crosshair, not a card.
    hud = create("TextLabel", {Name="SelfHP",Parent=host.ScreenGui,
        AnchorPoint=Vector2.new(0.5,0),Position=UDim2.new(0.5,6,0.5,30),Size=UDim2.fromOffset(140,24),
        BackgroundTransparency=1,BorderSizePixel=0,Visible=false,ZIndex=20,
        Text="",Font=Enum.Font.Arial,TextSize=18,TextStrokeTransparency=1,TextColor3=healthColour(1)})
    local scale = create("UIScale", {Parent=hud,Scale=1})
    local centerPosition, bottomPosition = UDim2.new(0.5,6,0.5,30), UDim2.new(0.5,0,1,-110)
    local card
    local function clean()
        render:ReleaseAll(layer); table.clear(tracked); layer.Visible = false
        hud.Visible = false; hud.Text = ""
    end
    card = context.framework.Categories.Visuals:CreateModule({Name = "HP", Category = "Render", ConfigKey = "Universal.HP", Order = 4,
        Tooltip = "Independent player health bars and optional heart-style Self HP display.", Function = function(enabled)
            if not enabled then clear(); card:SetStatus(nil); return end
            card:Clean(clear)
            card:Render(function(dt)
                local options = card.Options
                local targets = entity:Refresh()
                local camera = (host.workspace or workspace).CurrentCamera
                layer.Visible = true
                local seen, count = {}, 0
                if camera and options.Players.Value then
                    for _, target in ipairs(targets) do
                        if not target.Player or target.Health <= 0 or target.Distance > options["Max distance"].Value
                            or not (roleBinding.provider and roleBinding.provider.ReplacesTeams) and not options.Teammates.Value and target.IsFriendly then continue end
                        local rect = render:ModelRect(camera,target.Character)
                        if not rect or not options["Through walls"].Value and not entity:VisibleFrom(target) then continue end
                        local health, maximum, fraction = healthValues(target.Health,target.MaxHealth)
                        local drawing = render:Set(layer,target.Player)
                        drawing:Show(true); drawing:Bar(Vector2.new(rect.left-2,rect.top), rect.height, fraction)
                        if options["Health text"].Value then
                            drawing:Label("HP", tostring(math.ceil(health)), Vector2.new(rect.left-14,rect.top+(1-fraction)*rect.height), 0.5, 12, healthColour(fraction), false)
                        else drawing:HideLabel("HP") end
                        tracked[target.Player] = true; seen[target.Player] = true; count += 1
                    end
                end
                for player in pairs(tracked) do if not seen[player] then render:Release(layer,player); tracked[player] = nil end end
                local showSelf = options["Self HP"].Value
                if hud.Visible ~= showSelf then hud.Visible = showSelf end
                if showSelf then
                    local character = host.LocalPlayer.Character
                    local humanoid = character and character:FindFirstChildOfClass("Humanoid")
                    local health, maximum, fraction = healthValues(humanoid and humanoid.Health, humanoid and humanoid.MaxHealth)
                    local wanted = humanoid and humanoid.Health > 0 and tostring(math.ceil(health)) .. " ❤" or ""
                    if hud.Text ~= wanted then hud.Text = wanted end
                    local colour = healthColour(fraction)
                    if hud.TextColor3 ~= colour then hud.TextColor3 = colour end
                    local position = options["Self HP position"].Value == "Bottom" and bottomPosition or centerPosition
                    if hud.Position ~= position then hud.Position = position end
                    local wantedScale = options["Self HP scale"].Value
                    if camera then wantedScale = math.min(wantedScale, math.max(0.5, (camera.ViewportSize.X - 24) / 140)) end
                    if scale.Scale ~= wantedScale then scale.Scale = wantedScale end
                end
                card:SetStatus(options["Self HP"].Value and tostring(count) .. " + self" or tostring(count))
            end)
        end})
    card:CreateToggle({Name = "Players", Default = true})
    card:CreateToggle({Name = "Health text", Default = false, Show = {Option = "Players"}})
    card:CreateSlider({Name = "Max distance", Min = 50, Max = 5000, Default = 2000, Show = {Option = "Players"}})
    card:CreateToggle({Name = "Teammates", Default = true, Show = {Option = "Players"}})
    card:CreateToggle({Name = "Through walls", Default = true, Show = {Option = "Players"}})
    card:CreateToggle({Name = "Self HP", Default = false})
    card:CreateDropdown({Name="Self HP position",List={"Center","Bottom"},Index=1,Show={Option="Self HP"}})
    card:CreateSlider({Name = "Self HP scale", Min = 0.8, Max = 1.6, Decimal = 1, Default = 1, Show = {Option = "Self HP"}})
    roleBinding = context.services.gameRoles:Bind(card, {palette = false})
    card.OptionFilter = function(name)
        return not (name == "Teammates" and roleBinding.provider and roleBinding.provider.ReplacesTeams)
    end
    card:RefreshVisibility()
    activeCard, cleanup = card, clear
    Module.Initialized = true
    return card
end
function Module.destroy()
    if activeCard and activeCard.Enabled then activeCard:Toggle(false) end
    if cleanup then cleanup() end
    if hud then hud:Destroy() end
    if roleBinding then roleBinding:Destroy() end
    activeCard, cleanup, hud, roleBinding = nil, nil, nil, nil
    Module.Initialized = false
end
return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Render/HP.lua
--------------------------------------------------------------------------------


--------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Render/ItemRender.lua
BYTES: 25519
SHA-256: 45372c319223131cf53f70339e1eacaf64c44c326f1d02debfe2f705e095b4bd
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
export type Runtime = {
    framework: any,
    entity: any,
    host: any,
    render: any,
    services: any,
}

local Module = {
    Name = "ItemRender",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
    layer = nil :: Frame?,
}

local activeCleanup: (() -> ())? = nil

-- A game provider (Gun ESP, Trap ESP, ...) only draws while Item ESP is ON,
-- and the card owns that switch. Turning the provider row ON with the card
-- OFF used to leave the row ON and silently dead; now it brings the card
-- with it, so ON always means drawn.
local function setProviderEnabled(
    render: any,
    providerStates: {[string]: boolean},
    provider: any,
    value: boolean
): ()
    providerStates[provider.name] = value
    if render.Enabled then
        pcall(provider.setEnabled, value)
        return
    end
    if not value then
        return
    end
    -- The card's own switch, not the internal state: the user must see Item
    -- ESP turn ON, not only behind the scenes.
    local feature: any = render.Feature
    if feature
        and type(feature.activate) == "function"
        and feature.enabled ~= true then
        feature.activate()
    else
        render:Toggle(true)
    end
end

-- Bounded template fingerprint, not semantic image recognition. Ignore names,
-- world position and colour; compare geometry and mesh asset identity instead.
function Module.appearanceSignature(object: Instance): string?
    local parts: {BasePart} = {}
    local queue: {Instance} = {object}
    local cursor: number = 1
    while cursor <= #queue do
        local node: Instance = queue[cursor]
        cursor += 1
        if node:IsA("BasePart") then table.insert(parts, node :: BasePart) end
        if #parts > 32 then return nil end
        for _, child: Instance in ipairs(node:GetChildren()) do
            if #queue >= 96 then return nil end
            table.insert(queue, child)
        end
    end
    if #parts == 0 then return nil end
    local center: Vector3 = Vector3.zero
    for _, part: BasePart in ipairs(parts) do center += part.Position end
    center /= #parts
    local function numberKey(value: number): string
        return tostring(math.round(value * 20)) -- 0.05 stud tolerance
    end
    local function vectorKey(value: Vector3): string
        return numberKey(value.X) .. "," .. numberKey(value.Y) .. "," .. numberKey(value.Z)
    end
    local tokens: {string} = {}
    for _, part: BasePart in ipairs(parts) do
        local meshId: string = ""
        local meshShape: string = ""
        if part:IsA("MeshPart") then meshId = (part :: MeshPart).MeshId end
        local mesh: SpecialMesh? = part:FindFirstChildOfClass("SpecialMesh") :: SpecialMesh?
        if mesh then
            meshId = mesh.MeshId
            meshShape = tostring(mesh.MeshType) .. ":" .. vectorKey(mesh.Scale)
                .. ":" .. vectorKey(mesh.Offset)
        end
        local shape: string = part:IsA("Part") and tostring((part :: Part).Shape) or part.ClassName
        -- Pairwise distances preserve composition across rotation/translation.
        local distances: {string} = {}
        for _, other: BasePart in ipairs(parts) do
            table.insert(distances, numberKey((part.Position - other.Position).Magnitude))
        end
        table.sort(distances)
        table.insert(tokens, table.concat({part.ClassName, shape, vectorKey(part.Size),
            meshId, meshShape, numberKey((part.Position - center).Magnitude),
            table.concat(distances, ",")}, ":"))
    end
    table.sort(tokens)
    return table.concat(tokens, "|")
end

function Module.init(context: Runtime): any
    local host: any = context.host
    local framework: any = context.framework
    local localPlayer: Player = host.LocalPlayer
    local userInput: UserInputService = host.UserInputService
    local currentWorkspace: Workspace = host.workspace or workspace
    local menu: any = context.services.menu

    local tracked: {[Instance]: boolean} = {}

    local index: {Instance} = {}
    local indexConnections: {RBXScriptConnection} = {}
    local indexDirty: boolean = true

    local known: {string} = {}
    local wanted: {string} = {}
    local picking: boolean = false
    local touchMatching: string = "Appearance"
    local appearances: {[string]: string} = {}
    local signatureCache: {[Instance]: any} = setmetatable({}, {__mode = "k"})
    local profileSequence: number = 0
    local nextIndexAt: number = 0
    local nextShapeRefresh: number = 0

    local function signature(instance: Instance): string?
        if signatureCache[instance] == nil then
            signatureCache[instance] = Module.appearanceSignature(instance) or false
        end
        return signatureCache[instance] or nil
    end

    local function release(instance: Instance): ()
        if not tracked[instance] then
            return
        end
        tracked[instance] = nil
        local renderLib: any = context.render
        if renderLib and Module.layer then
            renderLib:Release(Module.layer, instance)
        end
    end

    local function releaseAll(): ()
        for instance: Instance in pairs(tracked) do
            release(instance)
        end
    end

    local function matches(instance: Instance): boolean
        local lowered: string = string.lower(instance.Name)
        for _, entry: string in ipairs(wanted) do
            local appearance: string? = appearances[entry]
            if appearance then
                if signature(instance) == appearance then return true end
            elseif not string.match(entry, "^Look #%d+ ") 
                and string.find(lowered, string.lower(entry), 1, true) then
                return true
            end
        end
        return false
    end

    local function rebuildIndex(): ()
        table.clear(index)
        indexDirty = false
        if #wanted == 0 then
            return
        end
        for _, descendant: Instance in ipairs(currentWorkspace:GetDescendants()) do
            if descendant:IsA("BasePart") or descendant:IsA("Model") or descendant:IsA("Tool") then
                if matches(descendant) then
                    table.insert(index, descendant)
                end
            end
        end
    end

    local function watchWorkspace(): ()
        local function changed(descendant: Instance): ()
            signatureCache[descendant] = nil
            local ancestor: Instance? = descendant.Parent
            while ancestor and ancestor ~= currentWorkspace do
                signatureCache[ancestor] = nil
                ancestor = ancestor.Parent
            end
            if #wanted > 0 then indexDirty = true end
        end
        table.insert(indexConnections, currentWorkspace.DescendantAdded:Connect(changed))
        table.insert(indexConnections, currentWorkspace.DescendantRemoving:Connect(changed))
    end

    local renderLib: any = context.render
    local layer: Frame? = renderLib and renderLib:Layer("ItemRenderLayer") or nil
    Module.layer = layer

    local itemEsp: any = context.services and context.services.itemEsp
    local providerStates: {[string]: boolean} = {}

    local function forEachProvider(callback: (any) -> ()): ()
        if not itemEsp or type(itemEsp.providers) ~= "function" then
            return
        end
        for _, provider: any in ipairs(itemEsp.providers()) do
            pcall(callback, provider)
        end
    end

    local render: any
    render = framework.Categories.Visuals:CreateModule({
        Name = "ItemESP",
        Category = "Render",
        ConfigKey = "Universal.ItemRender",
        Order = 2,
        Tooltip = "Names and outlines every object on the list, wherever it is.",
        Function = function(enabled: boolean): ()
            if layer then
                layer.Visible = enabled
            end
            if enabled then
                forEachProvider(function(provider: any): ()
                    if providerStates[provider.name] then
                        pcall(provider.setEnabled, false)
                    end
                end)
                render:SetStatus(nil)
                releaseAll()
                picking = false
                return
            end
            forEachProvider(function(provider: any): ()
                if providerStates[provider.name] then
                    pcall(provider.setEnabled, true)
                end
            end)
            render:SetStatus("0")
            render:Clean(function(): ()
                releaseAll()
                picking = false
                if layer then
                    layer.Visible = false
                end
            end)
            indexDirty = true
            watchWorkspace()
            render:Clean(function(): ()
                for _, connection: RBXScriptConnection in ipairs(indexConnections) do
                    pcall(function(): ()
                        connection:Disconnect()
                    end)
                end
                indexConnections = {}
                table.clear(index)
            end)
            render:Render(function(): ()
                -- Assets can fill MeshId/Size after parenting (streaming).
                -- Refresh at low frequency, never traverse geometry per frame.
                if next(appearances) ~= nil and os.clock() >= nextShapeRefresh then
                    nextShapeRefresh = os.clock() + 5
                    signatureCache = setmetatable({}, {__mode = "k"})
                    indexDirty = true
                end
                if indexDirty and os.clock() >= nextIndexAt then
                    nextIndexAt = os.clock() + 0.25
                    rebuildIndex()
                end
                Module.render(context, render, tracked, wanted, release, index, matches)
            end)
        end,
    })

    local objectList: any
    objectList = render:CreateList({
        Name = "Objects",
        Items = function(): {string}
            return known
        end,
        Tooltip = "Every object the module knows about. Tick the ones to draw.",
        Function = function(_selected: any, names: {string}): ()
            wanted = names
            indexDirty = true
            releaseAll()
        end,
    })

    local function learn(name: string): ()
        if name == "" then
            return
        end
        if not table.find(known, name) then
            table.insert(known, name)
            table.sort(known)
        end
        objectList:Set(name, true)
    end

    local function findBestMatchingName(query: string): string?
        local lowered: string = string.lower(query)
        local candidates: {string} = {}
        for _, descendant: Instance in ipairs(currentWorkspace:GetDescendants()) do
            if descendant:IsA("BasePart") or descendant:IsA("Model") or descendant:IsA("Tool") then
                local dName: string = descendant.Name
                local dLower: string = string.lower(dName)
                if dLower == lowered then
                    return dName
                end
                if string.find(dLower, lowered, 1, true) then
                    table.insert(candidates, dName)
                end
            end
        end
        for _, c: string in ipairs(candidates) do
            if string.find(string.lower(c), "^" .. lowered) then
                return c
            end
        end
        if #candidates > 0 then
            return candidates[1]
        end
        return nil
    end

    local pendingInput: string = ""
    local addBox: any

    local function commitAddByName(rawText: string?): ()
        local text: string = rawText or pendingInput
        local trimmed: string = string.match(text, "^%s*(.-)%s*$") or text
        if trimmed == "" or #trimmed < 2 then
            return
        end
        local best: string = findBestMatchingName(trimmed) or trimmed
        learn(best)
        render:SetStatus("Added: " .. best)
        pendingInput = ""
        if addBox and addBox.Object and typeof(addBox.Object) == "Instance" and addBox.Object:IsA("TextBox") then
            addBox.Object.Text = ""
        end
    end

    addBox = render:CreateTextBox({
        Name = "Add by name",
        Default = "",
        Tooltip = "Type a name and press Enter (or click Add Object). Finds closest matching items.",
        Function = function(value: string): ()
            pendingInput = value
        end,
    })

    if addBox and addBox.Object and typeof(addBox.Object) == "Instance" and addBox.Object:IsA("TextBox") then
        local box: TextBox = addBox.Object
        box.FocusLost:Connect(function(enterPressed: boolean)
            if enterPressed then
                commitAddByName(box.Text)
            end
        end)
    end

    render:CreateButton({
        Name = "Add object",
        Tooltip = "Adds the name typed above to the list.",
        Function = function(): ()
            local text: string = (addBox.Object and addBox.Object.Text) or pendingInput
            commitAddByName(text)
        end,
    })

    local function resolveItemInstance(part: BasePart): Instance
        local tool: Tool? = part:FindFirstAncestorOfClass("Tool")
        if tool then
            return tool
        end
        local parent: Instance? = part.Parent
        if parent and parent:IsA("Model") and parent ~= currentWorkspace then
            local parentNameLower: string = string.lower(parent.Name)
            local isGenericMap: boolean = parentNameLower == "map"
                or parentNameLower == "workspace"
                or parentNameLower == "geometry"
                or parentNameLower == "spawns"
                or parentNameLower == "terrain"
                or parentNameLower == "baseplate"
            local childCount: number = #parent:GetChildren()
            if not isGenericMap and childCount <= 15 then
                return parent
            end
        end
        return part
    end

    render:CreateDropdown({
        Name = "Touch matching",
        List = {"Appearance", "Name"},
        Index = 1,
        Function = function(value: string): () touchMatching = value end,
        Tooltip = "Appearance matches a small geometry/mesh template, not every Model. "
            .. "Templates last this session; reselect after rejoining. Name keeps the old search.",
    })

    render:CreateButton({
        Name = "Touch part",
        Tooltip = "Hides the menu, then adds whatever you tap next.",
        Function = function(): ()
            if picking then
                picking = false
                render:SetStatus("cancelled")
                return
            end
            picking = true

            menu.setVisible(false)
            render:Notify("tap the object you want rendered")
        end,
    })

    render:CreateButton({
        Name = "Clear list",
        Function = function(): ()
            for _, name: string in ipairs(known) do
                objectList:Set(name, false)
            end
            table.clear(known)
            table.clear(appearances)
            signatureCache = setmetatable({}, {__mode = "k"})
            wanted = {}
            objectList:Refresh()
            releaseAll()
            render:SetStatus("list cleared")
        end,
    })

    render:CreateToggle({
        Name = "Show distance",
        Default = true,
    })
    render:CreateToggle({
        Name = "Outline",
        Default = true,
        Tooltip = "Draw the object's silhouette through walls.",
    })
    render:CreateSlider({
        Name = "Max distance",
        Min = 50,
        Max = 5000,
        Default = 1200,
    })
    render:CreateSlider({
        Name = "Text size",
        Min = 8,
        Max = 22,
        Default = 12,
    })
    render:CreateColor({
        Name = "Colour",
        Default = Color3.fromRGB(85, 255, 85),
    })

    local pickConnection: RBXScriptConnection = userInput.InputBegan:Connect(
        function(input: InputObject, processed: boolean): ()
            if not picking or processed then
                return
            end
            if input.UserInputType ~= Enum.UserInputType.Touch
                and input.UserInputType ~= Enum.UserInputType.MouseButton1 then
                return
            end
            local camera: Camera? = currentWorkspace.CurrentCamera
            if not camera then
                return
            end

            local ray: Ray = (camera :: Camera):ScreenPointToRay(
                input.Position.X,
                input.Position.Y
            )
            local parameters: RaycastParams = RaycastParams.new()
            parameters.FilterType = Enum.RaycastFilterType.Exclude
            parameters.IgnoreWater = true
            parameters.FilterDescendantsInstances = {
                localPlayer.Character :: any,
                camera,
            }
            local result: RaycastResult? = currentWorkspace:Raycast(
                ray.Origin,
                ray.Direction.Unit * 4096,
                parameters
            )
            if not result then
                render:Notify("nothing under that tap")
                return
            end
            picking = false

            local hit: Instance = (result :: RaycastResult).Instance
            if not hit:IsA("BasePart") then
                render:Notify("invalid part tapped")
                menu.setVisible(true)
                return
            end
            local targetItem: Instance = resolveItemInstance(hit :: BasePart)
            local name: string = targetItem.Name
            if touchMatching == "Appearance" then
                -- Refresh cached geometry when the user explicitly selects.
```

```
            local fingerprint: string? = Module.appearanceSignature(targetItem)
            if not fingerprint then
                menu.setVisible(true)
                render:Notify("Object too large/complex; select a smaller part or use Name")
                return
            end
            profileSequence += 1
            name = "Look #" .. tostring(profileSequence) .. " (" .. name .. ")"
            appearances[name] = fingerprint
        end
        learn(name)
        menu.setVisible(true)
        render:SetStatus(name .. " added")
    end
)

render:CreateNote(
    "Touch matching: Appearance selects matching geometry in this session; Name selects by name."
)

-- Game-provided item highlights (for example MM2's Gun ESP and Trap ESP)
-- appear here as their own toggles. The card itself stays game-agnostic:
-- games register providers, this module only renders them.
local providerOptions: {[string]: any} = {}
local activeProviders: {[string]: any} = {}
local function refreshProviders(): ()
    local previous: {[string]: any} = activeProviders
    local current: {[string]: any} = {}
    forEachProvider(function(provider: any): ()
        current[provider.name] = provider
    end)
    activeProviders = current

    for name: string, provider: any in pairs(previous) do
        if current[name] ~= provider and providerStates[name] then
            pcall(provider.setEnabled, false)
        end
    end
    for name: string, provider: any in pairs(current) do
        if not providerOptions[name] then
            -- These rows deliberately bring the card with them (see
            -- setProviderEnabled): a provider ON must mean drawn. Flag it so
            -- readers and the boot tests know the exception is intended.
            if render.Feature then render.Feature.optionEntersFeature = true end
            providerOptions[name] = render:CreateToggle({
                Name = name,
                Default = false,
                Tooltip = provider.tooltip,
                Function = function(value: boolean): ()
                    providerStates[name] = value
                    local selected: any = activeProviders[name]
                    if selected then
                        setProviderEnabled(render, providerStates, selected, value)
                    end
                end,
            })
        elseif previous[name] ~= provider and render.Enabled and providerStates[name] then
            pcall(provider.setEnabled, true)
        end
    end
    for name: string, option: any in pairs(providerOptions) do
        option:SetVisible(current[name] ~= nil)
    end
end
refreshProviders()
local providerWatch: (() -> ())? = nil
if itemEsp and type(itemEsp.onChanged) == "function" then
    providerWatch = itemEsp.onChanged(refreshProviders)
end

activeCleanup = function(): ()
    if render.Enabled then render:Toggle(false) end
    picking = false
    pcall(function(): ()
        pickConnection:Disconnect()
    end)
    if providerWatch then
        pcall(providerWatch)
        providerWatch = nil
    end
    for _, connection: RBXScriptConnection in ipairs(indexConnections) do
        connection:Disconnect()
    end
    table.clear(indexConnections)
    releaseAll()
end
Module.Initialized = true
return render
end

function Module.render(
    context: Runtime,
    render: any,
    tracked: {[Instance]: any},
    wanted: {string},
    release: (Instance) -> (),
    index: {Instance},
    matches: (Instance) -> boolean
): ()
    local host: any = context.host
    local renderLib: any = context.render
    local layer: Frame? = Module.layer
    local localPlayer: Player = host.LocalPlayer
    local options: any = render.Options
    local currentWorkspace: Workspace = host.workspace or workspace
    local camera: Camera? = currentWorkspace.CurrentCamera

    if #wanted == 0 or not renderLib or not layer or not camera then
        for instance: Instance in pairs(tracked) do
            release(instance)
        end
        return
    end

    local resolvedCamera: Camera = camera :: Camera
    local resolvedLayer: Frame = layer :: Frame
    local character: Model? = localPlayer.Character
    local root: BasePart? = character
        and character:FindFirstChild("HumanoidRootPart") :: BasePart?
    local origin: Vector3 = root and (root :: BasePart).Position or Vector3.zero
    local maxDistance: number = options["Max distance"].Value
    local colour: Color3 = options["Colour"].Value
    local textSize: number = math.round(options["Text size"].Value)
    local seen: {[Instance]: boolean} = {}

    for _, descendant: Instance in ipairs(index) do
        if not descendant.Parent then
            continue
        end
        local isPart: boolean = descendant:IsA("BasePart")
        local isModel: boolean = descendant:IsA("Model") or descendant:IsA("Tool")
        if not isPart and not isModel then
            continue
        end
        if character and descendant:IsDescendantOf(character) then
            continue
        end
        if not matches(descendant) then
            continue
        end
        local duplicate: boolean = false
        local ancestor: Instance? = descendant.Parent
        while ancestor and ancestor ~= currentWorkspace do
            if (ancestor:IsA("Model") or ancestor:IsA("Tool")) and matches(ancestor) then
                duplicate = true
                break
            end
            ancestor = ancestor.Parent
        end
        if duplicate then continue end

        local anchor: BasePart? = nil
        if isPart then
            anchor = descendant :: BasePart
        else
            anchor = descendant:IsA("Model") and (descendant :: Model).PrimaryPart or nil
            anchor = anchor or descendant:FindFirstChildWhichIsA("BasePart", true) :: BasePart?
        end
        if not anchor then
            continue
        end
        local resolvedAnchor: BasePart = anchor :: BasePart
        local distance: number = (resolvedAnchor.Position - origin).Magnitude
        if distance > maxDistance then
            continue
        end
        local projected: Vector2? = renderLib:Project(resolvedCamera, resolvedAnchor.Position)
        if not projected then
            continue
        end

        seen[descendant] = true
        tracked[descendant] = true
        local drawings: any = renderLib:Set(resolvedLayer, descendant)
        drawings:Show(true)
        drawings:Label(
            "NameTag",
            options["Show distance"].Value
                and ("[" .. tostring(math.round(distance)) .. "] " .. descendant.Name)
                or descendant.Name,
            projected :: Vector2,
            0.5,
            textSize,
            colour,
            true
        )
        drawings:Highlight(
            descendant,
            colour,
            1,
            options["Outline"].Value and "Outline" or "Off"
        )
    end

    local count: number = 0
    for instance: Instance in pairs(tracked) do
        if not seen[instance] or not instance.Parent then
            release(instance)
        else
            count += 1
        end
    end
    local status: string = tostring(count)
    if render.Status ~= status then
        render:SetStatus(status)
    end
end

function Module.destroy(): ()
    if activeCleanup then
        pcall(activeCleanup)
    end
    activeCleanup = nil
    Module.layer = nil
    Module.Initialized = false
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Render/ItemRender.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Render/Markers.lua
BYTES: 8923
SHA-256: bddad496be32c5c824b37ace48ff9922c154ca445329c97ef36205774f6d4b79
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
-- Vape-style world waypoints, with names / Save here instead of encoded coordinates.
local Module = {Name="Markers",Initialized=false}
local card, clear, inputConnection, services
function Module.init(context)
    local host, render = context.host, context.render
    services = context.services
    local menu = services.menu
    local layer = render:Layer("MarkersLayer")
    local key = "Universal.Markers.Places." .. tostring(game.PlaceId)
    local config = host.configData
    local points = {}
    local stored = config and config.values[key]
    if type(stored) ~= "table" then stored = {} end
    local loaded = 0
    for name, position in pairs(stored) do
        if type(name)=="string" and type(position)=="table" and loaded < 128 then
            local x,y,z=tonumber(position[1]),tonumber(position[2]),tonumber(position[3])
            if x and y and z and x==x and y==y and z==z and math.max(math.abs(x),math.abs(y),math.abs(z)) < 1e7 then
                points[name]={x,y,z,position[4]=="surface" and "surface" or nil}; loaded += 1
            end
        end
    end
    local listeners = {}
    local function pointNames()
        local names={};for name in pairs(points) do table.insert(names,name) end;table.sort(names);return names
    end
    services.points = {
        Names=pointNames,
        Get=function(name)
            local data=points[name]
            if data then return Vector3.new(data[1],data[2],data[3]),data[4]=="surface" end
        end,
        Subscribe=function(callback)
            listeners[callback]=true
            return function() listeners[callback]=nil end
        end,
    }
    local list, tracked = nil, {}
    local picking, pickName, pickMenuVisible = false, nil, false
    local function finishPick(status, reopen)
        local wasPicking = picking
        picking = false
        if card then card:SetStatus(status) end
        if wasPicking and pickMenuVisible and reopen ~= false then menu.setVisible(true) end
        pickMenuVisible = false
    end
    local function save()
        if config then config.values[key]=points;if host.queueConfigSave then host.queueConfigSave() end end
        for callback in pairs(listeners) do callback() end
    end
    clear=function(reopen) render:ReleaseAll(layer);table.clear(tracked);layer.Visible=false;finishPick(nil,reopen) end
    card=context.framework.Categories.Render:CreateModule({Name="Markers",Category="Render",ConfigKey="Universal.Markers",
        Tooltip="Save your standing coordinates or mark a surface. Points are shared with Teleport and kept per game.",Function=function(enabled)
            if not enabled then clear();return end
            layer.Visible=true;card:Clean(clear)
            card:Render(function()
                local camera=(host.workspace or workspace).CurrentCamera
                if not camera then clear();return end
                layer.Visible=true
                local seen={}
                for name,data in pairs(points) do
                    if not list:IsSelected(name) then continue end
                    local position=Vector3.new(data[1],data[2],data[3])
                    local at=render:Project(camera,position)
                    if not at or at.X<0 or at.Y<0 or at.X>camera.ViewportSize.X or at.Y>camera.ViewportSize.Y then continue end
                    local text=name
                    if card.Options.Distance.Value then text ..= " [" .. math.round((camera.CFrame.Position-position).Magnitude) .. "]" end
                    local drawing=render:Set(layer,name)
                    drawing:Show(true)
                    drawing:Label("Marker",text,Vector2.new(at.X,at.Y),1,14,card.Options.Colour.Value,true)
                    seen[name],tracked[name]=true,true
                end
                for name in pairs(tracked) do if not seen[name] then render:Release(layer,name);tracked[name]=nil end end
            end)
        end})
    local function pointName()
        return string.sub(string.match(card.Options.Name.Value or "","^%s*(.-)%s*$") or "",1,48)
    end
    local function nextPointName()
        local base=pointName()
        local index=1
        while true do
            local suffix=base=="" and tostring(index) or " #"..index
            local name=(base=="" and "Point " or string.sub(base,1,48-#suffix))..suffix
            if not points[name] then return name end
            index+=1
        end
    end
    local function savedName()
        local name=pointName()
        return name=="" and name or nextPointName()
    end
    local function storePoint(position,name,surface)
        if name=="" then card:Notify("Enter a point name.");return false end
        if not position or position.X~=position.X or position.Y~=position.Y or position.Z~=position.Z
            or math.max(math.abs(position.X),math.abs(position.Y),math.abs(position.Z))>=1e7 then
            card:Notify("Position unavailable. Use coordinates in the form x, y, z.");return false
        end
        local count=0;for _ in pairs(points) do count+=1 end
        if count>=128 and not points[name] then card:Notify("Marker limit reached (128). Delete a point first.");return false end
        points[name]={position.X,position.Y,position.Z,surface and "surface" or nil};save();list:Refresh();list:Set(name,true)
        card:SetStatus(name);return true
    end
    card:CreateTextBox({Name="Name",Default="",Placeholder="Home, exit, meeting spot..."})
    card:CreateButton({Name="Save here",Tooltip="Save your character standing x, y, z here, not the camera position.",Function=function()
        local character=host.LocalPlayer.Character
        local root=character and character:FindFirstChild("HumanoidRootPart")
        storePoint(root and root.Position,savedName())
    end})
    card:CreateButton({Name="Touch",Tooltip="Hides the menu to pick a surface. Escape cancels; new points are numbered, never overwritten. Works while OFF too.",Function=function()
        if picking then finishPick();return end
        pickName=nextPointName()
        picking=true
        pickMenuVisible=menu.isVisible()
        card:SetStatus("Click/tap a surface · Esc cancels")
        menu.setVisible(false)
    end})
    if config and config.values["Universal.Markers.Point"]==nil then
        config.values["Universal.Markers.Point"]=config.values["Universal.Markers.Places"]
    end
    list=card:CreateList({Name="Point",Items=pointNames,Default=pointNames(),
        EmptyText="No points. Use Save here or Touch.",OnRemove=function(name)
            if not points[name] then return end
            list:Set(name,false);points[name]=nil;save();list:Refresh()
            render:Release(layer,name);tracked[name]=nil
        end})
    card:CreateToggle({Name="Manual coordinates",Default=false})
    card:CreateTextBox({Name="Coordinates",Default="",Placeholder="x, y, z",Show={Option="Manual coordinates",Values={true}}})
    card:CreateButton({Name="Save coordinates",Show={Option="Manual coordinates",Values={true}},Function=function()
        local x,y,z=string.match(card.Options.Coordinates.Value or "","^%s*([%+%-%.%deE]+)%s*,%s*([%+%-%.%deE]+)%s*,%s*([%+%-%.%deE]+)%s*$")
        x,y,z=tonumber(x),tonumber(y),tonumber(z)
        storePoint(x and y and z and Vector3.new(x,y,z) or nil,savedName())
    end})
    card:CreateToggle({Name="Distance",Default=true})
    card:CreateColor({Name="Colour",Default=Color3.fromRGB(85,255,85)})
    inputConnection=host.UserInputService.InputBegan:Connect(function(event,processed)
        if not picking then return end
        if event.KeyCode==Enum.KeyCode.Escape then
            finishPick();return
        end
        if processed or host.UserInputService:GetFocusedTextBox() then return end
        if event.UserInputType~=Enum.UserInputType.MouseButton1 and event.UserInputType~=Enum.UserInputType.Touch then return end
        local camera=(host.workspace or workspace).CurrentCamera
        if not camera then return end
        local params=RaycastParams.new()
        params.FilterType=Enum.RaycastFilterType.Exclude
        params.FilterDescendantsInstances=host.LocalPlayer.Character and {host.LocalPlayer.Character} or {}
        params.RespectCanCollide=true;params.IgnoreWater=true
        local ray=camera:ScreenPointToRay(event.Position.X,event.Position.Y)
        local hit=(host.workspace or workspace):Raycast(ray.Origin,ray.Direction*5000,params)
        if hit and storePoint(hit.Position,pickName,true) then finishPick(pickName)
        else card:SetStatus("No surface · tap again") end
    end)
    Module.Initialized=true
    return card
end
function Module.destroy()
    if clear then clear(false) end
    if card and card.Enabled then card:Toggle(false) end
    if inputConnection then inputConnection:Disconnect();inputConnection=nil end
    if services then services.points=nil end
    card,clear,services=nil,nil,nil;Module.Initialized=false
end
return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Render/Markers.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Render/NPCESP.lua
BYTES: 16997
SHA-256: 969fed8d5fda44e2983e6e75bd4f1718851685ac5b3d16a374edf8d5217895a0
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
export type Runtime = {
    framework: any,
    entity: any,
    render: any,
    host: any,
    services: any,
}

local Module = {
    Name = "NPCESP",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local activeCard: any = nil
local activeInputConnection: RBXScriptConnection? = nil

local STANDARD_PART_NAMES: {[string]: boolean} = {
    head = true,
    torso = true,
    uppertorso = true,
    lowertorso = true,
    humanoidrootpart = true,
    leftarm = true,
    rightarm = true,
    leftleg = true,
    rightleg = true,
    leftupperarm = true,
    leftlowerarm = true,
    lefthand = true,
    rightupperarm = true,
    rightlowerarm = true,
    righthand = true,
    leftupperleg = true,
    leftlowerleg = true,
    leftfoot = true,
    rightupperleg = true,
    rightlowerleg = true,
    rightfoot = true,
}

local VERDICT_COOLDOWN: number = 0.5

local function normalizeName(value: string): string
    local normalized: string = string.lower(value):gsub("[%s_%-]", "")
    return normalized
end

function Module.init(context: Runtime): any
    local framework: any = context.framework
    local render: any = context.render
    local host: any = context.host
    local players: Players = host.Players
    local currentWorkspace: Workspace = host.workspace or workspace
    local layer: Frame = render:Layer("NpcEspLayer")
    local lastStatusAt: number = -math.huge
    local lastSweepAt: number = -math.huge

    local bots: {[Model]: boolean} = {}
    local rigInfo: any = setmetatable({}, {__mode = "k"})
    local verdicts: any = setmetatable({}, {__mode = "k"})
    local forcedNames: {string} = {}

    local function modelFor(instance: Instance): Model?
        if instance:IsA("Model") then
            return instance :: Model
        end
        return instance:FindFirstAncestorOfClass("Model") :: Model?
    end

    local function isPlayerCharacterModel(model: Model): boolean
        if players:GetPlayerFromCharacter(model) ~= nil then
            return true
        end
        for _, player: Player in ipairs(players:GetPlayers()) do
            local character: Model? = player.Character
            if character and model:IsDescendantOf(character) then
                return true
            end
        end
        return false
    end

    local function matchesForcedName(model: Model): boolean
        local name: string = string.lower(model.Name)
        for _, needle: string in ipairs(forcedNames) do
            if needle ~= "" and string.find(name, needle, 1, true) then
                return true
            end
        end
        return false
    end

    local function classify(model: Model): string
        if model:GetAttribute("WurstDisguise") == true then
            return "reject"
        end
        if matchesForcedName(model) then
            return "rig"
        end
        local humanoid: Humanoid? = model:FindFirstChildOfClass("Humanoid") :: Humanoid?
        local standardParts: number = 0
        local motors: number = 0
        for _, child: Instance in ipairs(model:GetChildren()) do
            if child:IsA("BasePart") then
                if STANDARD_PART_NAMES[normalizeName(child.Name)] then
                    standardParts += 1
                end
            elseif child:IsA("Motor6D") then
                motors += 1
            end
        end
        if humanoid ~= nil then

            if standardParts >= 3 then
                return "rig"
            end
            return "creature"
        end

        if standardParts >= 4 or (standardParts >= 3 and motors >= 2) then
            return "rig"
        end
        if model:FindFirstChildWhichIsA("Humanoid", true) ~= nil then
            return "creature"
        end
        return "reject"
    end

    local function verdictFor(model: Model): string
        local entry: any = verdicts[model]
        if entry == nil then
            entry = {
                verdict = isPlayerCharacterModel(model) and "reject" or classify(model),
                at = os.clock(),
                dirty = false,
            }
            verdicts[model] = entry
            return entry.verdict
        end
        if entry.dirty then
            local now: number = os.clock()
            if now - entry.at >= VERDICT_COOLDOWN then
                entry.verdict =
                    isPlayerCharacterModel(model) and "reject" or classify(model)
                entry.at = now
                entry.dirty = false
            end
        end
        return entry.verdict
    end

    local function markDirty(model: Model): ()
        local entry: any = verdicts[model]
        if entry and not entry.dirty then
            entry.dirty = true
        end
        rigInfo[model] = nil
    end

    local function invalidate(instance: Instance): ()
        local parent: Instance? = instance.Parent
        if parent and parent:IsA("Model") then
            markDirty(parent :: Model)
        end
        if instance:IsA("Model") then
            markDirty(instance :: Model)
        end
    end

    local function add(model: Model, verdict: string): ()
        if verdict ~= "rig" then
            bots[model] = true
            return
        end

        for existing: Model in pairs(bots) do
            if existing:IsDescendantOf(model) then
                bots[existing] = nil
                render:Release(layer, existing)
            end
        end
        bots[model] = true
    end

    local function inspect(instance: Instance): ()
        if not (instance:IsA("Model") or instance:IsA("Humanoid")
            or instance:IsA("BasePart") or instance:IsA("Motor6D")) then
            return
        end
        invalidate(instance)
        local model: Model? = modelFor(instance)
        local creature: Model? = nil
        while model and model ~= currentWorkspace do
            local verdict: string = verdictFor(model)
            if verdict == "rig" then
                add(model, "rig")
                return
            end
            if verdict == "creature" and creature == nil then
                creature = model
            end
            local parent: Instance? = model.Parent
            if not parent then
                break
            end
            model = parent:IsA("Model") and parent :: Model
                or parent:FindFirstAncestorOfClass("Model") :: Model?
        end
        if creature then
            add(creature, "creature")
        end
    end

    local function sweep(now: number): ()
        for model: any, entry: any in pairs(verdicts) do
            if entry.dirty and now - entry.at >= VERDICT_COOLDOWN then
                local resolved: Model = model :: Model
                local verdict: string =
                    isPlayerCharacterModel(resolved) and "reject" or classify(resolved)
                entry.verdict = verdict
                entry.at = now
                entry.dirty = false
                if verdict == "rig" then
                    add(resolved, "rig")
                end
            end
        end
    end

    local function scan(): ()
        -- A rescan can remove formerly forced matches; discard their drawings
        -- before replacing the tracked set.
        render:ReleaseAll(layer)
        table.clear(bots)
        for _, object: Instance in ipairs(currentWorkspace:GetDescendants()) do
            if object:IsA("Model") then
                inspect(object)
            end
        end
    end

    local function clear(): ()
        render:ReleaseAll(layer)
        table.clear(bots)
        layer.Visible = false
    end

    local card: any
    card = framework.Categories.Visuals:CreateModule({
        Name = "NPCESP",
        Category = "Render",
        ConfigKey = "Universal.NPCESP",
        Order = 4,
        Tooltip = "Displays character-shaped NPC models - bots, shopkeepers "
```

```
        .. "and display statues like the creator's rig - whether or not "
        .. "their Humanoid is alive.",
    Function = function(enabled: boolean): ()
        layer.Visible = enabled
        if not enabled then
            clear()
            card:SetStatus(nil)
            return
        end
        scan()
        card:Event(currentWorkspace.DescendantAdded, inspect)
        card:Event(currentWorkspace.DescendantRemoving, invalidate)
        card:Render(function(): ()
            local camera: Camera? = currentWorkspace.CurrentCamera
            if not camera then
                return
            end
            local resolvedCamera: Camera = camera :: Camera
            local now: number = os.clock()
            if now - lastSweepAt >= 1 then
                lastSweepAt = now
                sweep(now)
            end

            local maxDistance: number = card.Options["Max distance"].Value
            local includeCreatures: boolean = card.Options["All humanoids"].Value
            local localCharacter: Model? = host.LocalPlayer.Character
            local localRoot: BasePart? = localCharacter and (
                localCharacter:FindFirstChild("HumanoidRootPart") :: BasePart?
                or localCharacter.PrimaryPart
                or localCharacter:FindFirstChildWhichIsA("BasePart", true) :: BasePart?
            ) or nil
            local origin: Vector3 = localRoot and localRoot.Position
                or resolvedCamera.CFrame.Position
            local count: number = 0
            for model: Model in pairs(bots) do
                if not model:IsDescendantOf(currentWorkspace)
                    or players:GetPlayerFromCharacter(model) ~= nil then
                    bots[model] = nil
                    render:Release(layer, model)

                    verdicts[model] = {verdict = "reject", at = now, dirty = false}
                    continue
                end
                local verdict: string = verdictFor(model)
                if verdict == "reject" then
                    bots[model] = nil
                    render:Release(layer, model)
                    continue
                end

                if verdict == "creature" and not includeCreatures then

                    render:Release(layer, model)
                    continue
                end

                local drawing: any = render:Set(layer, model)

                local info: any = rigInfo[model]
                if not (info and info.root and info.root.Parent) then
                    info = {
                        root = model:FindFirstChild("HumanoidRootPart") :: BasePart?
                            or model.PrimaryPart
                            or model:FindFirstChildWhichIsA("BasePart", true) :: BasePart?,
                        humanoid = model:FindFirstChildOfClass("Humanoid") :: Humanoid?
                            or model:FindFirstChildWhichIsA("Humanoid", true) :: Humanoid?,
                    }
                    rigInfo[model] = info
                end
                local root: BasePart? = info.root
                if not root or (root.Position - origin).Magnitude > maxDistance then
                    drawing:Show(false)
                    continue
                end

                if verdict == "creature"
                    and info.humanoid
                    and info.humanoid.Health <= 0 then
                    drawing:Show(false)
                    continue
                end

                local rect: any = render:ModelRect(resolvedCamera, model)
                if not rect then
                    drawing:Show(false)
                    continue
                end

                local colour: Color3 = card.Options["Colour"].Value
                drawing:Box(rect, card.Options["Box"].Value and "Full" or "Off", 1, colour)
                if card.Options["Name"].Value then
                    local text: string = card.Options["Model name"].Value
                        and model.Name ~= "" and model.Name
                        or "BOT"
                    local humanoid: Humanoid? = info.humanoid
                    if humanoid and card.Options["Health"].Value then
                        text ..= " [" .. tostring(math.round(humanoid.Health)) .. "]"
                    end
                    drawing:Label(
                        "NameTag",
                        text,
                        Vector2.new(rect.centreX, rect.top - 4),
                        1,
                        13,
                        colour,
                        true
                    )
                else
                    drawing:HideLabel("NameTag")
                end
                drawing:Highlight(
                    model,
                    colour,
                    0.55,
                    not card.Options["Highlight"].Value and "Off"
                        or (card.Options["Through walls"].Value and "AlwaysOnTop" or "Occluded")
                )
                drawing:Show(true)
                count += 1
            end

            if now - lastStatusAt >= 0.25 then
                lastStatusAt = now
                card:SetStatus(tostring(count))
            end
        end)
        card:Clean(clear)
    end,
})

card:CreateToggle({Name = "Box", Default = true})
card:CreateToggle({Name = "Highlight", Default = true})
card:CreateToggle({Name = "Through walls", Default = true, Show = {Option = "Highlight"}})
card:CreateToggle({Name = "Name", Default = true})
card:CreateToggle({
    Name = "Model name",
    Show = {Option = "Name"},
    Default = false,
    Tooltip = "Label each model with its own name instead of BOT.",
})
card:CreateToggle({
    Name = "Health",
    Show = {Option = "Name"},
    Default = true,
    Tooltip = "Append the Humanoid's hit points to the label when the "
        .. "model has one.",
})
card:CreateToggle({
    Name = "All humanoids",
    Default = false,
    Tooltip = "Also flag non-character models that merely carry a living "
        .. "Humanoid (creatures, monsters). Off by default: NPCESP tracks "
        .. "character-shaped rigs.",
})
card:CreateColor({Name = "Colour", Default = Color3.fromRGB(85, 255, 85)})
card:CreateSlider({Name = "Max distance", Min = 25, Max = 2000, Step = 25, Default = 500})

local pendingName: string = ""
local addBox: any
local function addForcedName(): ()
    local trimmed: string = string.match(pendingName, "^%s*(..-)%s*$") or pendingName
    if trimmed == "" then return end
    local needle: string = string.lower(trimmed)
    if not table.find(forcedNames, needle) then
        table.insert(forcedNames, needle)
    end
    table.clear(verdicts)
    table.clear(rigInfo)
    if card.Enabled then scan() end
    addBox:Set("")
end
addBox = card:CreateTextBox({
    Name = "Force by name",
    Default = "",
    Tooltip = "Type a model name and press Enter or Add forced name. Partial names match, so statue catches CreatorStatue.",
    Function = function(value: string): ()
        pendingName = value
    end,
})
-- Widgets call the text callback before CreateTextBox returns. Apply a
-- previously saved value only after the option itself exists.
if pendingName ~= "" then addForcedName() end
if addBox.Object and typeof(addBox.Object) == "Instance" and addBox.Object:IsA("TextBox") then
    activeInputConnection = addBox.Object.FocusLost:Connect(function(enterPressed: boolean): ()
        if enterPressed then addForcedName() end
    end)
end
card:CreateButton({
    Name = "Add forced name",
    Tooltip = "Add the name typed above, including on touch devices.",
    Function = addForcedName,
})

card:CreateButton({
    Name = "Clear forced names",
    Function = function(): ()
        table.clear(forcedNames)
        table.clear(verdicts)
        table.clear(rigInfo)
        if card.Enabled then
            scan()
        end
    end,
})

activeCard = card
Module.Initialized = true
return card
end

function Module.destroy(): ()
    if activeCard and activeCard.Enabled then
        activeCard:Toggle(false)
    end
    if activeInputConnection then
        activeInputConnection:Disconnect()
        activeInputConnection = nil
    end
    activeCard = nil
    Module.Initialized = false
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Render/NPCESP.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Render/Nametags.lua
BYTES: 9604
SHA-256: 5c1be3ff7e095e69e962a004fdf4c6bfcb19280bab85ef6c369d883615b607be
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
local Module = {Name = "Nametags", Initialized = false}
local activeCard, cleanup, roleBinding
local BLACK = Color3.new(0, 0, 0)

function Module.init(context: any): any
    local host, entity, render = context.host, context.entity, context.render
    local layer = render:Layer("NametagsLayer")
    local tracked = {}
    local store = host.configData
    -- Preserve explicit new choices; migrate only previously enabled overlays.
    if store and store.states["Universal.Nametags"] == nil then
        local states, values = store.states, store.values
        states["Universal.Nametags"] = states["Universal.Chams"] == true and states["Universal.Chams.Nametags"] == true
            or states["Universal.PlayerESP"] == true and states["Universal.PlayerESP.Nametags"] ~= false
        if values["Universal.Nametags.Textsize"] == nil then values["Universal.Nametags.Textsize"] = values["Universal.PlayerESP.Textsize"] end
        for new, old in pairs({Background="Nameplate",Displayname="Displayname",Heldtool="Heldtool",Showrole="Showrole",Rolecolours="Rolecolours"}) do
            local key = "Universal.Nametags." .. new
            if states[key] == nil then states[key] = states["Universal.PlayerESP." .. old] end
        end
        for key, colour in pairs(table.clone(values)) do
            if string.sub(key,1,20) == "Universal.PlayerESP." and string.lower(string.sub(key,-6)) == "colour" then
                local newKey = "Universal.Nametags." .. string.sub(key,21)
                if values[newKey] == nil then values[newKey] = colour end
            end
        end
    end
    local textService = game:GetService("TextService")
    local card
    local function clear()
        render:ReleaseAll(layer); table.clear(tracked); layer.Visible = false
    end
    local function fit(drawing, slot, text, size, viewport)
        local label = drawing.labels[slot]
        if not label then return end
        local key = text .. ":" .. tostring(size) .. ":" .. tostring(viewport.X)
        drawing.textMetrics = drawing.textMetrics or {}
        -- v139.1: GetTextSize + Size + TextTruncate only when the text or the
        -- size really changed. Writing Size every frame re-laid out every
        -- nametag in the scene, which is a visible hitch with many players.
        if not drawing.textMetrics[slot] or drawing.textMetrics[slot].key ~= key then
            local width = #text * size * 0.65 + 8
            pcall(function() width = textService:GetTextSize(text, size, Enum.Font.Arial, Vector2.new(10000,100)).X + 8 end)
            width = math.max(1, math.min(width, viewport.X - 24))
            drawing.textMetrics[slot] = {key = key, width = width}
            label.AutomaticSize = Enum.AutomaticSize.None
            label.Size = UDim2.fromOffset(width, size + 7)
            label.TextTruncate = Enum.TextTruncate.AtEnd
        end
        -- Clamped to screen independently of target position, not a clipped name.
        local half = drawing.textMetrics[slot].width * 0.5
        local clampedX = math.clamp(label.Position.X.Offset, half + 8, math.max(half + 8, viewport.X - half - 8))
        if math.abs(clampedX - label.Position.X.Offset) >= 0.5 then
            label.Position = UDim2.fromOffset(clampedX, label.Position.Y.Offset)
        end
    end
    card = context.framework.Categories.Visuals:CreateModule({Name = "Nametags", Category = "Render",
        ConfigKey = "Universal.Nametags", Order = 3,
        Tooltip = "Readable player nameplates, account names, distance and game roles. Independent of ESP and Chams.",
        Function = function(enabled)
            if not enabled then clear(); card:SetStatus(nil); return end
            layer.Visible = true
            card:Clean(clear)
            card:Render(function()
                local camera = (host.workspace or workspace).CurrentCamera
                if not camera then clear(); return end
                layer.Visible = true
                local seen, count = {}, 0
                local options = card.Options
                for _, target in ipairs(entity:Refresh()) do
                    local player = target.Player
                    if not player or target.Health <= 0 or target.Distance > options["Max distance"].Value
                        or not (roleBinding.provider and roleBinding.provider.ReplacesTeams) and not options.Teammates.Value and target.IsFriendly then continue end
                    local head = target.Head or target.RootPart
                    local point = head and render:Project(camera, head.Position + Vector3.new(0, head.Size.Y * 0.5 + 0.7, 0))
                    if not point or point.X < 0 or point.Y < 0 or point.X > camera.ViewportSize.X or point.Y > camera.ViewportSize.Y
                        or not options["Through walls"].Value and not entity:VisibleFrom(target) then continue end
                    local colour, role = roleBinding:Resolve(player, options.Colour.Value)
                    local shown = options["Display name"].Value and player.DisplayName or player.Name
                    local details = {}
                    if options["Account name"].Value and shown ~= player.Name then table.insert(details, "@" .. player.Name) end
                    if options["Show role"].Value and role then table.insert(details, role) end

                    if options["Held tool"].Value then
                        local tool = target.Character:FindFirstChildOfClass("Tool")
                        if tool then table.insert(details, tool.Name) end
                    end
                    seen[player] = true; tracked[player] = true; count += 1
                    local drawing = render:Set(layer, player)
                    drawing:Show(true)
                    local size = math.round(options["Text size"].Value)
                    if #details > 0 then shown ..= " [" .. table.concat(details, "] [") .. "]" end
                    if options.Distance.Value then shown = "[" .. tostring(math.round(target.Distance)) .. "] " .. shown end
                    local y = math.max(point.Y, size + 16)
                    drawing:Label("Name", shown, Vector2.new(point.X, y), 1, size, colour, options.Background.Value)
                    local label = drawing.labels.Name
                    if label then
                        if not drawing.flatNametag then
                            for _, child in ipairs(label:GetChildren()) do
                                if child:IsA("UICorner") or child:IsA("UIStroke") then child:Destroy()
                                elseif child:IsA("UIPadding") then
                                    child.PaddingLeft, child.PaddingRight = UDim.new(0,4), UDim.new(0,4)
                                end
                            end
                            drawing.flatNametag = true
                        end
                        -- v139.1: these are constant. Re-assigning Font in
                        -- particular re-shapes the glyphs of every label every
                        -- single frame.
                        if label.Font ~= Enum.Font.Arial then
                            label.Font = Enum.Font.Arial
                        end
                        if label.BackgroundColor3 ~= BLACK then
                            label.BackgroundColor3 = BLACK
                        end
                        local plate: number = options.Background.Value and 0.5 or 1
                        if label.BackgroundTransparency ~= plate then
                            label.BackgroundTransparency = plate
                        end
                        if label.TextStrokeTransparency ~= 1 then
                            label.TextStrokeTransparency = 1
                        end
                    end
                    fit(drawing, "Name", shown, size, camera.ViewportSize)
                    drawing:HideLabel("Details")
                end
                for player in pairs(tracked) do
                    if not seen[player] then render:Release(layer,player); tracked[player] = nil end
                end
                card:SetStatus(tostring(count))
            end)
        end})
    card:CreateToggle({Name = "Display name", Default = true})
    card:CreateToggle({Name = "Account name", Default = false, Show = {Option = "Display name"}})
    card:CreateToggle({Name = "Distance", Default = false})
    card:CreateToggle({Name = "Show role", Default = false})
    card:CreateToggle({Name = "Held tool", Default = false})
    card:CreateToggle({Name = "Background", Default = true})
    card:CreateSlider({Name = "Text size", Min = 10, Max = 24, Default = 14})
    card:CreateColor({Name = "Colour", Default = Color3.fromRGB(85, 255, 85)})
    card:CreateSlider({Name = "Max distance", Min = 50, Max = 5000, Default = 1000})
    card:CreateToggle({Name = "Through walls", Default = true})
    card:CreateToggle({Name = "Teammates", Default = true})
    roleBinding = context.services.gameRoles:Bind(card)
    card.OptionFilter = function(name)
        if name == "Show role" then return roleBinding.provider ~= nil end
        return not (name == "Teammates" and roleBinding.provider and roleBinding.provider.ReplacesTeams)
    end
    card:RefreshVisibility()
    activeCard, cleanup = card, clear
    Module.Initialized = true
    return card
end
function Module.destroy()
    if activeCard and activeCard.Enabled then activeCard:Toggle(false) end
    if cleanup then cleanup() end
    if roleBinding then roleBinding:Destroy() end
    activeCard, cleanup, roleBinding = nil, nil, nil
    Module.Initialized = false
end
return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Render/Nametags.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Render/PlayerESP.lua
BYTES: 12283
SHA-256: 63675fa71f9f6938992b5f243a2c071352392bd132d562100e07a2ae26f7e1d3
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
export type Runtime = {
    framework: any,
    entity: any,
    host: any,
    services: any,
}

local Module = {
    Name = "PlayerESP",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local activeCleanup: (() -> ())? = nil
local activeCard: any = nil

function Module.init(context: Runtime): any
    local host: any = context.host
    local framework: any = context.framework
    local players: Players = host.Players

    local render: any = context.render

    local container: Frame = render:Layer("PlayerEspLayer")
    local pool: {[Player]: boolean} = {}
    local lastStatusAt: number = -math.huge
    local visibilityCache: any = setmetatable({}, {__mode = "k"})

    local esp: any
    esp = framework.Categories.Visuals:CreateModule({
        Name = "PlayerESP",
        Category = "Render",
        ConfigKey = "Universal.PlayerESP",
        Order = 1,
        Tooltip = "Outlined boxes, skeletons and tracers. Nametags and HP are independent modules.",
        Function = function(enabled: boolean): ()
            container.Visible = enabled
            if not enabled then
                esp:SetStatus(nil)
                render:ReleaseAll(container)
                table.clear(pool)
                return
            end
            lastStatusAt = -math.huge
            esp:SetStatus("0")
            esp:Render(function(): ()
                Module.render(context, container, pool, esp, visibilityCache)
                local now: number = os.clock()
                if now - lastStatusAt >= 0.25 then
                    lastStatusAt = now
                    local count: number = 0
                    for _player: Player in pairs(pool) do
                        count += 1
                    end
                    esp:SetStatus(tostring(count))
                end
            end)
            esp:Event(players.PlayerRemoving, function(player: Player): ()
                render:Release(container, player)
                pool[player] = nil
            end)
            esp:Clean(function(): ()
                render:ReleaseAll(container)
                table.clear(pool)
                container.Visible = false
            end)
        end,
    })

    esp:CreateDropdown({
        Name = "Boxes",
        List = {"Corner", "Full", "Off"},
        Index = 1,
        Tooltip = "Corner draws four brackets, Full draws the whole rectangle.",
    })
    esp:CreateSlider({
        Name = "Box thickness",
        Show = {Option = "Boxes", Values = {"Corner", "Full"}},
        Min = 1,
        Max = 5,
        Default = 1,
    })
    esp:CreateToggle({
        Name = "Filled box",
        Show = {Option = "Boxes", Values = {"Corner", "Full"}},
        Default = false,
        Tooltip = "Wash the inside of the box in the target's colour. Reads "
            .. "at a glance across a map; noisy in a crowd.",
    })
    esp:CreateToggle({
        Name = "Skeleton",
        Default = false,
        Tooltip = "Draw the rig's bones. Works on both R6 and R15.",
    })
    esp:CreateDropdown({
        Name = "Tracers",
        List = {"Off", "Bottom", "Centre", "Cursor"},
        Index = 1,
        Tooltip = "Where the line to each target starts.",
    })
    esp:CreateToggle({
        Name = "Team check",
        Default = false,
        Tooltip = "Skip players on your own team.",
    })
    esp:CreateToggle({
        Name = "Visibility check",
        Default = false,
        Tooltip = "Recolour targets that are behind geometry.",
    })
    esp:CreateSlider({
        Name = "Max distance",
        Min = 50,
        Max = 5000,
        Default = 2000,
    })

    esp:CreateColor({
        Name = "Colour",
        Default = Color3.fromRGB(85, 255, 85),
        Tooltip = "Used for every target the game gives no colour of its own.",
    })
    esp:CreateToggle({
        Name = "Team colours",
        Default = true,
        Tooltip = "Draw each player in their own team's colour where the game "
            .. "has teams.",
    })
    local roles = context.services.gameRoles:Bind(esp)
    Module.roles = roles
    esp.OptionFilter = function(name)
        return not (roles.provider and roles.provider.ReplacesTeams and (name == "Team check" or name == "Team colours"))
    end
    esp:RefreshVisibility()
    local function addExtraRows(extra: any): ()
        if type(extra) ~= "table" or type(extra.Name) ~= "string" or esp.Options[extra.Name] then return end
        esp:CreateToggle({
            Name = extra.Name,
            Default = extra.Default == true,
            Tooltip = extra.Tooltip,
            Function = function(value: boolean): ()
                if type(extra.Toggle) == "function" then
                    pcall(extra.Toggle, value)
                end
            end,
        })
        if extra.Color then
            esp:CreateColor({
                Name = extra.Name .. " colour",
                Show = {Option = extra.Name},
                Default = extra.Color,
                Function = function(value: Color3): ()
                    if type(extra.SetColor) == "function" then
                        pcall(extra.SetColor, value)
                    end
                end,
            })
        end

        for _, definition: any in ipairs(extra.Options or {}) do
            local rowName: string = extra.Name .. " " .. definition.Name
            if definition.Kind == "toggle" then
                esp:CreateToggle({
                    Name = rowName,
                    Show = {Option = extra.Name},
                    Default = definition.Default == true,
                    Tooltip = definition.Tooltip,
                    Function = function(value: boolean): ()
                        if type(definition.Set) == "function" then
                            pcall(definition.Set, value)
                        end
                    end,
                })
            else
                esp:CreateSlider({
                    Name = rowName,
                    Show = {Option = extra.Name},
                    Min = definition.Min or 0,
                    Max = definition.Max or 100,
                    Default = definition.Default or definition.Min or 0,
                    Tooltip = definition.Tooltip,
                    Function = function(value: number): ()
                        if type(definition.Set) == "function" then
                            pcall(definition.Set, value)
                        end
                    end,
                })
            end
        end
    end

    local gameBridge: any = context.services.gameBridge
    if type(gameBridge.onEvent) == "function" then
        Module.bridgeCleanup = gameBridge.onEvent(function(kind: string, payload: any): ()
            if kind == "extra" then
                addExtraRows(payload)
            end
        end)
    end

    activeCleanup = function(): ()
        render:ReleaseAll(container)
        table.clear(pool)
        container.Visible = false
    end
    activeCard = esp
    Module.Initialized = true
    return esp
end

function Module.render(
    context: Runtime,
    container: Frame,
    pool: {[Player]: any},
    esp: any,
    visibilityCache: any
): ()
    local host: any = context.host
    local entityLibrary: any = context.entity
    local render: any = context.render
    local gameBridge: any = context.services.gameBridge
    local currentWorkspace: Workspace = host.workspace or workspace
    local camera: Camera? = currentWorkspace.CurrentCamera
    if not camera then
        container.Visible = false
        return
    end
    container.Visible = true
    local resolvedCamera: Camera = camera :: Camera
    local options: any = esp.Options
    local viewport: Vector2 = resolvedCamera.ViewportSize

    entityLibrary:Refresh()
    local seen: {[Player]: boolean} = {}

    for _, entity: any in ipairs(entityLibrary.List) do
        local player: Player = entity.Player
        if not (Module.roles and Module.roles.provider and Module.roles.provider.ReplacesTeams)
            and options["Team check"].Value and entity.IsFriendly then
            continue
        end
        if entity.Distance > options["Max distance"].Value then

            continue
        end

        local rect: any = render:ModelRect(resolvedCamera, entity.Character)

        seen[player] = true
        pool[player] = true
        local drawings: any = render:Set(container, player)

        if not rect then

            drawings:Show(false)
            continue
        end
```

```
    local visible: boolean = true
    if options["Visibility check"].Value then
        local cache: any = visibilityCache[player]
        local now: number = os.clock()
        if not cache or now - cache.at > 0.12 then
            cache = {at = now, value = entityLibrary:VisibleFrom(entity)}
            visibilityCache[player] = cache
        end
        visible = cache.value
    end
    local colour = options["Colour"].Value
    local roles = Module.roles
    local roleColour = roles and roles:Resolve(player, nil)
    if roleColour then
        colour = roleColour
    elseif not (roles and roles.provider and roles.provider.ReplacesTeams)
        and options["Team colours"].Value and player.Team then
        colour = player.Team.TeamColor.Color
    end

    if not visible then
        colour = colour:Lerp(Color3.new(0, 0, 0), 0.45)
    end

    local left: number = rect.left
    local top: number = rect.top
    local bottom: number = rect.bottom
    local height: number = rect.height
    local centreX: number = rect.centreX

    drawings:Show(true)

    local thickness: number = math.round(options["Box thickness"].Value)
    local boxMode: string = options["Boxes"].Value
    drawings:Fill(
        options["Filled box"].Value and boxMode ~= "Off" and rect or nil,
        colour,
        0.82
    )
    drawings:Box(rect, boxMode, thickness, colour)

    if options["Skeleton"].Value then
        local bones: {{BasePart}} = entityLibrary:Rig(entity)
        for index: number = 1, 14 do
            local pair: {BasePart}? = bones[index]
            local first: Vector2? = pair
                and render:Project(resolvedCamera, pair[1].Position)
            local second: Vector2? = pair
                and render:Project(resolvedCamera, pair[2].Position)
            if first and second then
                drawings:Line(
                    "Bone",
                    index,
                    first :: Vector2,
                    second :: Vector2,
                    1,
                    colour
                )
            else
                drawings:HideLines("Bone", index, index)
            end
        end
    else
        drawings:HideLines("Bone", 1, 14)
    end

    local tracerMode: string = options["Tracers"].Value
    if tracerMode == "Off" then
        drawings:HideLines("Tracer", 1, 1)
    else
        local origin: Vector2
        if tracerMode == "Bottom" then
            origin = Vector2.new(viewport.X * 0.5, viewport.Y)
        elseif tracerMode == "Centre" then
            origin = viewport * 0.5
        else
            origin = host.UserInputService:GetMouseLocation()
        end
        drawings:Line(
            "Tracer",
            1,
            origin,
            Vector2.new(centreX, bottom),
            1,
            colour
        )
    end

    end

    for player: Player in pairs(pool) do
        if not seen[player] then
            render:Release(container, player)
            pool[player] = nil
        end
    end
end

function Module.destroy(): ()
    if activeCard and activeCard.Enabled then activeCard:Toggle(false) end
    if activeCleanup then
        pcall(activeCleanup)
    end
    if Module.roles then Module.roles:Destroy(); Module.roles = nil end
    if Module.bridgeCleanup then Module.bridgeCleanup(); Module.bridgeCleanup = nil end
    activeCleanup = nil
    activeCard = nil
    Module.Initialized = false
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Render/PlayerESP.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Render/Spectate.lua
BYTES: 5678
SHA-256: bf6b900d4e83bd7859b9b4390df932076b11b56ce6e7c02c42b36bf81ead978b
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
-- Native player spectating; FreeCam pauses without losing its position.
local Module = {Name="Spectate", Initialized=false}
local card, services, stop

function Module.init(context)
    local host = context.host
    services = context.services
    local world = host.workspace or workspace
    local chosen, camera, saved, writtenSubject
    local enabled=false
    local config=host.configData
    if config then
        if config.states["Universal.FreeCam.Spectate"]==nil then config.states["Universal.FreeCam.Spectate"]=config.states["Universal.Spectate"] end
        if config.values["Universal.FreeCam.SpectatePlayer"]==nil then config.values["Universal.FreeCam.SpectatePlayer"]=config.values["Universal.Spectate.Player"] end
    end
    card=assert(context.framework:GetModule("FreeCam"),"Spectate requires FreeCam options")
    local playerList
    local id = "ARandomMenu.Spectate." .. tostring({})
    local function localHumanoid()
        local character = host.LocalPlayer.Character
        return character and character:FindFirstChildOfClass("Humanoid")
    end
    local function freeCamSession()
        local free = services.freeCam
        return free and free.IsActive() and free.Session or nil
    end
    local function restore()
        if camera and saved then
            pcall(function()
                -- Do not take the camera back from a later owner.
                if camera.CameraType ~= Enum.CameraType.Custom or camera.CameraSubject ~= writtenSubject then return end
                local returnedToFreeCam = saved.freeCam and saved.freeCam == freeCamSession()
                if saved.freeCam and not returnedToFreeCam then
                    camera.CameraType = Enum.CameraType.Custom
                    camera.CameraSubject = localHumanoid()
                else
                    camera.CameraType = saved.kind
                    local subject=saved.subject
                    if subject==saved.localHumanoid or typeof(subject)=="Instance" and subject.Parent==nil then subject=localHumanoid() end
                    camera.CameraSubject=subject
                    camera.CFrame, camera.Focus = saved.frame, saved.focus
                end
            end)
        end
        camera, saved, writtenSubject = nil, nil, nil
    end
    local function targetHumanoid()
        for _, player in ipairs(host.Players:GetPlayers()) do
            if player ~= host.LocalPlayer and player.Name == chosen then
                local character = player.Character
                local humanoid = character and character:FindFirstChildOfClass("Humanoid")
                return humanoid and humanoid.Health > 0 and humanoid or nil, true
            end
        end
        return nil, false
    end
    local function step()
        local humanoid, present = targetHumanoid()
        if chosen and not present then
            local departed = chosen
            chosen = nil
            if playerList then playerList:Set(departed,false);playerList:Refresh() end
        end
        if not humanoid then
            restore()
            card:SetStatus(chosen and "Waiting for respawn" or "Choose a player")
            return
        end
        local nextCamera = world.CurrentCamera
        if camera ~= nextCamera then restore() end
        if not nextCamera then return end
        if not camera then
            camera = nextCamera
            saved = {kind=camera.CameraType,subject=camera.CameraSubject,frame=camera.CFrame,focus=camera.Focus,
                freeCam=freeCamSession(),localHumanoid=localHumanoid()}
        elseif camera.CameraType ~= Enum.CameraType.Custom or camera.CameraSubject ~= writtenSubject then
            -- Adopt a newer camera baseline rather than restoring over it later.
            saved = {kind=camera.CameraType,subject=camera.CameraSubject,frame=camera.CFrame,focus=camera.Focus,
                freeCam=false,localHumanoid=localHumanoid()}
        end
        camera.CameraType = Enum.CameraType.Custom
        camera.CameraSubject = humanoid
        writtenSubject = humanoid
        card:SetStatus(chosen)
    end
    local state = {IsActive=function()
        if not enabled or not chosen then return false end
        local humanoid = targetHumanoid()
        return humanoid ~= nil
    end}
    services.spectate = state
    stop=function()
        enabled=false
        host.RunService:UnbindFromRenderStep(id)
        restore()
    end
    card:CreateToggle({Name="Spectate",Default=false,
        Tooltip="Follow a player independently: FreeCam may stay OFF. Turn this off or deselect to return.",
        Function=function(value)
            stop();enabled=value
            if enabled then
                host.RunService:BindToRenderStep(id,Enum.RenderPriority.Camera.Value-1,step)
                step()
            end
        end})
    playerList = card:CreateList({Name="Spectate Player",Single=true,Show={Option="Spectate"},Items=function()
        local names={}
        for _,player in ipairs(host.Players:GetPlayers()) do
            if player ~= host.LocalPlayer then table.insert(names,player.Name) end
        end
        table.sort(names);return names
    end,EmptyText="No other players",Tooltip="Select one player; click the selected player again to stop spectating.",Function=function(_,names)
        local nextName = names[1]
        if chosen ~= nextName then restore();chosen=nextName end
        if enabled then step() end
    end})
    Module.Initialized=true
    return Module
end

function Module.destroy()
    if stop then stop() end
    if services then services.spectate=nil end
    card,services,stop=nil,nil,nil
    Module.Initialized=false
end
return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Render/Spectate.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Render/XRay.lua
BYTES: 6133
SHA-256: 5ffebca7dd5f986e6ed926e191fcccab39fb6ca3dfa7ef9c0afb63b18363f7d2
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
-- Vape Xray's kept-part list, with bounded updates and owned-only restoration.
local Module = {Name="XRay",PlaceId=0,Initialized=false}
local card
function Module.init(context)
    local host, world = context.host, context.host.workspace or workspace
    local records = setmetatable({}, {__mode="k"})
    local indexed = setmetatable({}, {__mode="k"})
    local candidates, cursor, pause = {}, 1, 0
    local named, kept = {}, {}
    local function names(text)
        local result={}
        for name in string.gmatch(string.lower(text or ""),"[^,]+") do
            name=string.match(name,"^%s*(..-)%s*$")
            if name~="" then result[name]=true end
        end
        return result
    end
    local function restore(part)
        local record=records[part]
        if not record then return end
        pcall(function()
            if part.LocalTransparencyModifier==record.applied then part.LocalTransparencyModifier=record.base end
        end)
        records[part]=nil
    end
    local function restoreAll()
        for part in pairs(records) do restore(part) end
    end
    local function changed()
        restoreAll();cursor=1;pause=0
        if card and card.Options["Part names"] and card.Options["Keep visible"] then
            named=names(card.Options["Part names"].Value)
            kept=names(card.Options["Keep visible"].Value)
        end
    end
    local function add(part)
        if part:IsA("BasePart") and not indexed[part] then
            indexed[part]=true;table.insert(candidates,part)
        end
    end
    local function eligible(part,origin,root)
        local opts=card.Options
        if part.Transparency>=0.98 or kept[string.lower(part.Name)] then return false end
        if not opts["Include unanchored"].Value and not part.Anchored then return false end
        if opts["Collidable only"].Value and not part.CanCollide then return false end
        if opts["Limit range"].Value and origin and (part.Position-origin).Magnitude>opts.Range.Value+part.Size.Magnitude*0.5 then return false end
        if part.Name=="Wurst_AntiVoidPlatform" or host.isMenuOwned and host.isMenuOwned(part) then return false end
        local ancestor=part.Parent
        while ancestor and ancestor~=world do
            if ancestor:IsA("Tool") or ancestor:IsA("Model") and ancestor:FindFirstChildOfClass("Humanoid") then return false end
            ancestor=ancestor.Parent
        end
        local thinFloor=part.Size.Y<=math.min(part.Size.X,part.Size.Z)*0.35
        if opts["Keep floors"].Value and thinFloor and root and part.Position.Y<=root.Position.Y then return false end
        local mode=opts.Scope.Value
        if mode=="Named parts" then return named[string.lower(part.Name)]==true end
        if mode=="Walls" then return not thinFloor and part.Size.Y>math.min(part.Size.X,part.Size.Z)*1.5 end
        return true
    end
    local function step(dt)
        pause=math.max(0,pause-(dt or 0))
        if pause>0 or #candidates==0 then return end
        local camera=world.CurrentCamera
        local character=host.LocalPlayer.Character
        local root=character and character:FindFirstChild("HumanoidRootPart")
        local origin=camera and camera.CFrame.Position or root and root.Position
        for _=1,200 do
            local part=candidates[cursor]
            if not part then cursor=1;pause=0.2;break end
            if not part.Parent or not part:IsDescendantOf(world) then
                restore(part);indexed[part]=nil
                candidates[cursor]=candidates[#candidates];candidates[#candidates]=nil
            else
                if eligible(part,origin,root) then
                    local current=part.LocalTransparencyModifier
                    local record=records[part]
                    if not record then record={base=current};records[part]=record
                    elseif current~=record.applied then record.base=current end
                    record.applied=math.max(record.base,card.Options.Transparency.Value)
                    if current~=record.applied then part.LocalTransparencyModifier=record.applied end
                else restore(part) end
                cursor=cursor+1
            end
        end
    end
    card=context.framework.Categories.Render:CreateModule({Name="X-Ray",Category="Render",ConfigKey="Universal.XRay",
        Tooltip="Fade map parts locally. Keep named objects visible; never fade characters or held tools.",Function=function(enabled)
            restoreAll();table.clear(candidates);table.clear(indexed);cursor=1;pause=0
            if not enabled then card:SetStatus(nil);return end
            changed()
            for _,part in ipairs(world:GetDescendants()) do add(part) end
            card:Clean(restoreAll)
            card:Clean(world.DescendantAdded:Connect(add))
            card:Loop(step);step(0)
        end})
    card:CreateDropdown({Name="Scope",List={"All map","Walls","Named parts"},Index=1,Function=changed})
    card:CreateSlider({Name="Transparency",Min=0,Max=1,Default=0.5,Step=0.05,Decimal=2,Function=changed})
    card:CreateTextBox({Name="Part names",Default="",Placeholder="Wall, Window",Show={Option="Scope",Values={"Named parts"}},Function=changed,
        Tooltip="Exact part names, separated by commas. Only these names fade in Named parts scope."})
    card:CreateTextBox({Name="Keep visible",Default="",Placeholder="Spawn, Objective",Function=changed,
        Tooltip="Vape-style kept-part list. Exact names separated by commas; these parts never fade."})
    card:CreateToggle({Name="Keep floors",Default=true,Function=changed})
    card:CreateToggle({Name="Collidable only",Default=true,Function=changed})
    card:CreateToggle({Name="Include unanchored",Default=false,Function=changed})
    card:CreateToggle({Name="Limit range",Default=true,Function=changed})
    card:CreateSlider({Name="Range",Min=25,Max=5000,Default=250,Show={Option="Limit range",Values={true}},Function=changed})
    Module.Initialized=true
    return card
end
function Module.destroy()
    if card and card.Enabled then card:Toggle(false) end
    card=nil;Module.Initialized=false
end
return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Render/XRay.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/games/universal/Render/ZoomUnlocker.lua
BYTES: 2816
SHA-256: bcf1f03ee59dc94a2299ae09255384c4ec229dac099f68f731cb7d82f880df6f
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
local Module = {Name = "ZoomUnlocker", Initialized = false}
local cleanup

function Module.init(context)
    local host, player = context.host, context.host.LocalPlayer
    local access = context.services.cameraAccess
    local settings = {minimum=0.5, maximum=400, firstPerson=true, walls=false}
    local original, feature
    local function bounds(minimum,maximum)
        if player.CameraMinZoomDistance > maximum then player.CameraMinZoomDistance = minimum end
        if player.CameraMaxZoomDistance ~= maximum then player.CameraMaxZoomDistance = maximum end
        if player.CameraMinZoomDistance ~= minimum then player.CameraMinZoomDistance = minimum end
    end
    local function apply()
        bounds(math.min(settings.minimum,settings.maximum),math.max(settings.minimum,settings.maximum))
        local mode = settings.firstPerson and Enum.CameraMode.Classic or original.mode
        if player.CameraMode ~= mode then player.CameraMode = mode end
        local walls = access:ThroughWalls(settings.walls)
        feature:SetStatus(settings.walls and (walls and tostring(math.round(settings.maximum)) .. " / walls" or "walls unavailable")
            or tostring(math.round(settings.maximum)) .. " studs")
    end
    cleanup = function()
        host.disconnectFeatureConnection("ZoomUnlocker")
        access:ThroughWalls(false)
        if original then
            bounds(original.minimum,original.maximum)
            player.CameraMode = original.mode
            original = nil
        end
    end
    feature = host.createUniversalFeature("Zoom", "Extend zoom; optionally let the camera pass through walls.", 23, function(enabled)
        cleanup()
        if not enabled then feature:SetStatus(nil); return end
        original = {minimum=player.CameraMinZoomDistance, maximum=player.CameraMaxZoomDistance, mode=player.CameraMode}
        apply()
        host.featureConnections.ZoomUnlocker = host.TaskManager:Connect(apply, nil, "Zoom Unlocker")
    end, {configKey="Universal.ZoomUnlocker", categoryName="Render"})
    local function update(key,value)
        settings[key] = value
        if feature.enabled then apply() end
    end
    host.addNumberOption(feature,"Maximum distance",settings.maximum,25,2000,function(value) update("maximum",value) end)
    host.addNumberOption(feature,"Minimum distance",settings.minimum,0.5,50,function(value) update("minimum",value) end)
    host.addToggleOption(feature,"Unlock forced first person",settings.firstPerson,function(value) update("firstPerson",value) end)
    host.addToggleOption(feature,"Through walls",settings.walls,function(value) update("walls",value) end)
    Module.Initialized = true
    return feature
end
function Module.destroy()
    if cleanup then cleanup(); cleanup = nil end
    Module.Initialized = false
end
return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/games/universal/Render/ZoomUnlocker.lua
--------------------------------------------------------------------------------

================================================================================
BIBLIOTECAS Y MANIFIESTO COMPARTIDOS
================================================================================

--------------------------------------------------------------------------------
ARCHIVO: src/libraries/CameraAccess.lua
BYTES: 4648
SHA-256: 04bd4b23ccec081abe895fe4a82c50c9ceeccfa7d58f8de53784c151121c47f8
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
local Module = {Name = "CameraAccess", Initialized = false}
local active

function Module.init(host)
    local api = {playerModule = nil, source = nil, pending = false, checkedAt = -math.huge}
    local patch
    function api:Refresh()
        if self.destroyed then return end
        local scripts = host.LocalPlayer:FindFirstChild("PlayerScripts")
        local source = scripts and scripts:FindFirstChild("PlayerModule")
        if source ~= self.source then self.playerModule, self.poppercam = nil, nil end
        if self.pending or source == self.source and self.playerModule then return end
        if os.clock() - self.checkedAt < 0.5 then return end
        self.checkedAt = os.clock()
        if not source then return end
        self.source, self.pending = source, true
        -- require may yield on a newly installed PlayerModule, not in render.
        task.spawn(function()
            local ok, result = pcall(require, source)
            local cameraModule = source:FindFirstChild("CameraModule")
            local popper = cameraModule and cameraModule:FindFirstChild("Poppercam")
            local cameraOk, cameraClass
            if popper then cameraOk, cameraClass = pcall(require, popper) end
            if not self.destroyed and self.source == source then
                self.playerModule = ok and type(result) == "table" and result or nil
                -- Modern CameraModule returns {}, but its Poppercam class still
                -- supplies the Update method used by the native camera instance.
                self.poppercam = cameraOk and type(cameraClass) == "table" and cameraClass or nil
            end
            self.pending = false
        end)
    end
    local function get(method)
        api:Refresh()
        local playerModule = api.playerModule
        if playerModule and type(playerModule[method]) == "function" then
            local ok, result = pcall(playerModule[method], playerModule)
            if ok then return result end
        end
    end
    function api:GetControls() return get("GetControls") end
    function api:GetCameras()
        local result = get("GetCameras")
        return type(result) == "table" and result or nil
    end
    local function restore()
        if not patch then return end
        local old = patch
        patch = nil
        if old.module.Update == old.replacement then
            local ok = pcall(function() old.module.Update = old.update end)
            if ok and old.enabled then pcall(old.module.Enable, old.module, true) end
        end
    end
    function api:ThroughWalls(enabled)
        if not enabled or self.destroyed then restore(); return false end
        local cameras = self:GetCameras()
        local occlusion = cameras and cameras.activeOcclusionModule or self.poppercam
        if patch and patch.module ~= occlusion then restore() end
        if patch then return patch.module.Update == patch.replacement end
        if type(occlusion) ~= "table" or type(occlusion.Update) ~= "function" then return false end
        local wasEnabled = false
        if type(occlusion.GetEnabled) == "function" and type(occlusion.Enable) == "function" then
            local ok, value = pcall(occlusion.GetEnabled, occlusion)
            wasEnabled = ok and value == true
            if wasEnabled then pcall(occlusion.Enable, occlusion, false) end
        end
        local cleaned = setmetatable({}, {__mode="k"})
        local replacement = function(instance, _, cameraFrame, focus)
            if not cleaned[instance] then
                cleaned[instance] = true
                if type(instance.Cleanup) == "function" then pcall(instance.Cleanup,instance) end
            end
            local extrapolator = instance.focusExtrapolator
            if extrapolator and type(extrapolator.Reset) == "function" then extrapolator:Reset() end
            return cameraFrame, focus
        end
        local update = occlusion.Update
        local ok = pcall(function() occlusion.Update = replacement end)
        if not ok then
            if wasEnabled then pcall(occlusion.Enable, occlusion, true) end
            return false
        end
        patch = {module = occlusion, update = update, replacement = replacement, enabled = wasEnabled}
        return true
    end
    function api:Destroy()
        restore()
        self.destroyed = true
        self.playerModule, self.poppercam = nil, nil
    end
    host.services.cameraAccess = api
    active = api
    Module.Initialized = true
    return api
end
function Module.destroy()
    if active then active:Destroy(); active = nil end
    Module.Initialized = false
end
return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/libraries/CameraAccess.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/libraries/Entity.lua
BYTES: 27268
SHA-256: 0451b6168d2bd2813590de86d78140686da3ab1bbbea91013883817a2e172df3
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
export type Entity = {
    Player: Player?,
    Character: Model,
    Humanoid: Humanoid,
    RootPart: BasePart,
    Head: BasePart?,
    Health: number,
    MaxHealth: number,
    Distance: number,
    Team: Team?,
    IsFriendly: boolean,
    IsNPC: boolean,
    Name: string,
}

export type RangeQuery = {
    Range: number,
    Limit: number?,
    IgnoreTeam: boolean?,
    IgnoreWalls: boolean?,
    IgnoreFriends: boolean?,
    Angle: number?,
    Sort: string?,
    IncludeNPCs: boolean?,
}

export type EntityQuery = {

    Radius: number?,
    MaxDistance: number?,
    IgnoreTeam: boolean?,
    IgnoreWalls: boolean?,
    IgnoreFriends: boolean?,
    Part: string?,
    IncludeNPCs: boolean?,
}

export type EntityLibrary = {
    List: {Entity},
    ByPlayer: {[Player]: Entity},
    NPCList: {Entity},
    ByModel: {[Model]: Entity},
    LocalEntity: Entity?,
    IsAlive: (self: EntityLibrary) -> boolean,
    Refresh: (self: EntityLibrary) -> {Entity},
    Get: (self: EntityLibrary, player: Player) -> Entity?,
    GetNpc: (self: EntityLibrary, model: Model) -> Entity?,
    ForEach: (self: EntityLibrary, callback: (Entity) -> ()) -> (),
    IsFriendly: (self: EntityLibrary, player: Player?) -> boolean,
    IsFriendlyModel: (self: EntityLibrary, model: Model) -> boolean,
    IsEnemy: (self: EntityLibrary, player: Player?) -> boolean?,
    IsProtected: (self: EntityLibrary, player: Player?) -> boolean,
    InRange: (self: EntityLibrary, query: RangeQuery) -> {Entity},
    ClosestToRay: (
        self: EntityLibrary,
        origin: Vector3,
        direction: Vector3,
        query: EntityQuery?
    ) -> Entity?,
    ClosestToCursor: (self: EntityLibrary, query: EntityQuery?) -> Entity?,
    VisibleFrom: (self: EntityLibrary, entity: Entity) -> boolean,
    Rig: (self: EntityLibrary, entity: Entity) -> {{BasePart}},
    Destroy: (self: EntityLibrary) -> (),
}

local Module = {
    Name = "Entity",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local R15_BONES: {{string}} = {
    {"Head", "UpperTorso"},
    {"UpperTorso", "LowerTorso"},
    {"UpperTorso", "LeftUpperArm"},
    {"LeftUpperArm", "LeftLowerArm"},
    {"LeftLowerArm", "LeftHand"},
    {"UpperTorso", "RightUpperArm"},
    {"RightUpperArm", "RightLowerArm"},
    {"RightLowerArm", "RightHand"},
    {"LowerTorso", "LeftUpperLeg"},
    {"LeftUpperLeg", "LeftLowerLeg"},
    {"LeftLowerLeg", "LeftFoot"},
    {"LowerTorso", "RightUpperLeg"},
    {"RightUpperLeg", "RightLowerLeg"},
    {"RightLowerLeg", "RightFoot"},
}

local R6_BONES: {{string}} = {
    {"Head", "Torso"},
    {"Torso", "Left Arm"},
    {"Torso", "Right Arm"},
    {"Torso", "Left Leg"},
    {"Torso", "Right Leg"},
}

local activeLibrary: EntityLibrary? = nil
```

```
function Module.init(context: any): EntityLibrary
    local host: any = context
    local players: Players = host.Players
    local localPlayer: Player = host.LocalPlayer
    local currentWorkspace: Workspace = host.workspace or workspace

    local library: any = {
        List = {},
        ByPlayer = {},
        NPCList = {},
        ByModel = {},
        LocalEntity = nil,
        Connections = {},

        lastRefresh = -1,
        pool = setmetatable({}, {__mode = "k"}),
        npcPool = setmetatable({}, {__mode = "k"}),
        npcModels = setmetatable({}, {__mode = "k"}),
        rigs = setmetatable({}, {__mode = "k"}),
    }

    local function characterParts(player: Player): (Model?, Humanoid?, BasePart?)
        local character: Model? = player.Character
        if not character or not character.Parent then
            for _, containerName: string in ipairs({"PlayersContainer", "Characters", "Live", "Players"}) do
                local container: Instance? = currentWorkspace:FindFirstChild(containerName)
                local candidate: Instance? = container and container:FindFirstChild(player.Name)
                if candidate and candidate:IsA("Model") then
                    character = candidate :: Model
                    break
                end
            end
        end
        if not character or not character.Parent then
            return nil, nil, nil
        end
        local humanoid: Humanoid? = character:FindFirstChildOfClass("Humanoid") :: Humanoid?
        local root: BasePart? = character:FindFirstChild("HumanoidRootPart") :: BasePart?
            or character.PrimaryPart
            or character:FindFirstChildWhichIsA("BasePart") :: BasePart?
        return character, humanoid, root
    end

    local function localRoot(): BasePart?
        local _, _, root = characterParts(localPlayer)
        return root
    end

    local TEAM_ATTRIBUTE_NAMES: {string} = {
        "Team",
        "TeamId",
        "TeamID",
        "Faction",
        "FactionId",
        "FactionID",
    }

    local function normalizeTeamValue(value: any): string?
        if value == nil then
            return nil
        end
        if typeof(value) == "Instance" then
            return "name:" .. string.lower((value :: Instance).Name)
        end
        if type(value) == "table" and type(value.Name) == "string" then
            return "name:" .. string.lower(value.Name)
        end
        if type(value) == "table"
            and type(value.R) == "number"
            and type(value.G) == "number"
            and type(value.B) == "number" then
            return string.format(
                "colour:%d:%d:%d",
                math.round(value.R * 255),
                math.round(value.G * 255),
                math.round(value.B * 255)
            )
        end
        if type(value) == "table" and value.Color ~= nil then
            local nested: string? = normalizeTeamValue(value.Color)
            if nested then
                return nested
            end
        end
        if type(value) == "string" then
            return "name:" .. string.lower(value)
        end
        local text: string = tostring(value)
        if text == "" or text == "nil" then
            return nil
        end
        return "value:" .. string.lower(text)
    end

    local function teamToken(instance: Instance): string?
        for _, attributeName: string in ipairs(TEAM_ATTRIBUTE_NAMES) do
            local ok: boolean, value: any = pcall(
                instance.GetAttribute,
                instance,
                attributeName
            )
            if ok then
                local normalized: string? = normalizeTeamValue(value)
                if normalized then
                    return normalized
                end
            end
        end

        local directOk: boolean, direct: any = pcall(function(): any
            return (instance :: any).Team
        end)
        if directOk then
            local normalizedDirect: string? = normalizeTeamValue(direct)
            if normalizedDirect then
                return normalizedDirect
            end
        end
        local colourOk: boolean, colour: any = pcall(function(): any
            return (instance :: any).TeamColor
        end)
        if colourOk then
            local normalizedColour: string? = normalizeTeamValue(colour)
            if normalizedColour then
                return normalizedColour
            end
        end
        local child: Instance? = instance:FindFirstChild("Team")
        if child then
            local valueOk: boolean, childValue: any = pcall(function(): any
                return (child :: any).Value
            end)
            return normalizeTeamValue(valueOk and childValue or child.Name)
        end
        return nil
    end

    local function localTeamToken(): string?
        return teamToken(localPlayer)
    end

    local function teamResolver(): any
        local hostState: any = host.state
        if hostState and hostState.gameBridge then
            return hostState.gameBridge.teamProvider
        end
        return nil
    end

    local function providerTeamToken(instance: Instance): string?
        local provider: any = teamResolver()
        if not provider or type(provider.Get) ~= "function" then
            return nil
        end
        local ok: boolean, token: any = pcall(provider.Get, instance)
        if ok and type(token) == "string" and token ~= "" then
            return token
        end
        return nil
    end

    local function providerRelation(instance: Instance): (boolean, string?)
        local provider: any = teamResolver()
        if not provider or type(provider.GetRelation) ~= "function" then
            return false, nil
        end
        local ok: boolean, relation: any = pcall(provider.GetRelation, instance)
        if ok and type(relation) == "string" then
            return true, relation
        end
        -- A relation-aware provider owns the decision even when data is late;
        -- falling back to a common lobby Team would reintroduce false friends.
        return true, nil
    end

    local function advancedVerdict(player: Player): any
        local checker: any = host.state and host.state.teamCheck
        if checker and type(checker.analyze) == "function" then
            local ok: boolean, verdict: any = pcall(checker.analyze, player)
            if ok and type(verdict) == "table" then return verdict end
        end
        return nil
    end

    function library:IsFriendly(player: Player?): boolean
        if not player or player == localPlayer then
            return false
        end
        local verdict: any = advancedVerdict(player)
        -- A generic unknown is not authoritative: preserve legacy Entity
        -- token/child detection. Provider/role unknowns must still fail closed.
        if verdict and verdict.method ~= "unknown" then return verdict.sameTeam == true end

        local handled: boolean, relation: string? = providerRelation(player)
        if handled then
            return relation == "Teammate"
        end

        local provider: any = teamResolver()
        if provider and type(provider.IsFriendly) == "function" then
            local ok: boolean, friendly: any = pcall(provider.IsFriendly, player)
            if ok and type(friendly) == "boolean" then
                return friendly
            end
        end

        local providerToken: string? = providerTeamToken(player)
        if providerToken ~= nil then
            local localProviderToken: string? = providerTeamToken(localPlayer)
            if localProviderToken ~= nil then
                return localProviderToken == providerToken
            end
        end

        if provider == nil
            and (player.Neutral == true or localPlayer.Neutral == true) then
            return false
        end
        local localToken: string? = localTeamToken()
        local targetToken: string? = teamToken(player)
        return localToken ~= nil and targetToken ~= nil and localToken == targetToken
    end

    function library:IsFriendlyModel(model: Model): boolean
        if model == localPlayer.Character then
            return false
        end
        local handled: boolean, relation: string? = providerRelation(model)
        if handled then
            return relation == "Teammate"
        end
        local providerToken: string? = providerTeamToken(model)
        if providerToken ~= nil then
            local localProviderToken: string? = providerTeamToken(localPlayer)
            if localProviderToken ~= nil then
                return localProviderToken == providerToken
            end
        end
        local localToken: string? = localTeamToken()
        local targetToken: string? = teamToken(model)
        return localToken ~= nil and targetToken ~= nil and localToken == targetToken
    end

    -- Tri-state target decision for relation-aware games. nil means that the
    -- legacy provider has no relation API and callers keep their old fallback.
    function library:IsEnemy(player: Player?): boolean?
        if not player or player == localPlayer then
            return false
        end
        local verdict: any = advancedVerdict(player)
        if verdict then
            if type(verdict.hostile) == "boolean" then return verdict.hostile end
            if verdict.method == "provider-unknown" or verdict.method == "roles-unknown"
                or verdict.method == "attributes-unknown" then return false end
            return nil
        end
        local handled: boolean, relation: string? = providerRelation(player)
        if handled then
            return relation == "Enemy"
        end
        return nil
    end

    local function passesTeamFilter(entity: Entity, ignoreTeam: boolean?): boolean
        if ignoreTeam then
            return true
        end
        if entity.Player then
            local hostile: boolean? = library:IsEnemy(entity.Player)
            if hostile ~= nil then
                return hostile
            end
        end
        return not entity.IsFriendly
    end

    local function npcParts(model: Model): (Humanoid?, BasePart?)
        local humanoid: Humanoid? =
            model:FindFirstChildOfClass("Humanoid") :: Humanoid?
        local root: BasePart? =
            model:FindFirstChild("HumanoidRootPart") :: BasePart?
                or model.PrimaryPart
                or model:FindFirstChildWhichIsA("BasePart") :: BasePart?
        return humanoid, root
    end

    local function isNpcModel(model: Model): boolean
        if not model.Parent or model == localPlayer.Character then
            return false
        end
        if players:GetPlayerFromCharacter(model) ~= nil then
            return false
        end

        if model:GetAttribute("WurstDisguise") == true then
            return false
        end
        local humanoid: Humanoid?, root: BasePart? = npcParts(model)
        return humanoid ~= nil and root ~= nil
    end

    local function modelFor(instance: Instance): Model?
        if instance:IsA("Model") then
            return instance :: Model
        end
        return instance:FindFirstAncestorOfClass("Model") :: Model?
    end

    local function indexNpc(instance: Instance): ()
        local model: Model? = modelFor(instance)
        if model and isNpcModel(model) then
            library.npcModels[model] = true
        end
    end

    local function unindexNpc(instance: Instance): ()
        local model: Model? = modelFor(instance)
        if model and (instance == model or not model.Parent) then
            library.npcModels[model] = nil
            library.ByModel[model] = nil
        end
    end

    local function refreshNpcs(originPosition: Vector3): ()
        local list: {Entity} = library.NPCList
        local byModel: {[Model]: Entity} = library.ByModel
        table.clear(list)
        table.clear(byModel)
        for model: Model in pairs(library.npcModels) do
            if not isNpcModel(model) then
                library.npcModels[model] = nil
                continue
            end
            local humanoid: Humanoid?, root: BasePart? = npcParts(model)
            if not humanoid or not root or humanoid.Health <= 0 then
                continue
            end
            local entity: any = library.npcPool[model]
            if not entity then
                entity = {}
                library.npcPool[model] = entity
            end
            entity.Player = nil
            entity.Character = model
            entity.Humanoid = humanoid
            entity.RootPart = root
            entity.Head = model:FindFirstChild("Head") :: BasePart?
            entity.Health = humanoid.Health
            entity.MaxHealth = math.max(humanoid.MaxHealth, 1)
            entity.Distance = (originPosition - root.Position).Magnitude
            entity.Team = nil
            entity.IsFriendly = library:IsFriendlyModel(model)
            entity.IsNPC = true
            entity.Name = model.Name
            byModel[model] = entity
            table.insert(list, entity)
        end
    end

    function library:Refresh(force: boolean?): {Entity}

        local now: number = os.clock()
        if not force and now - library.lastRefresh < 1 / 30 then
            return library.List
        end
        library.lastRefresh = now

        local origin: BasePart? = localRoot()
        local originPosition: Vector3 = origin and origin.Position or Vector3.zero
        local list: {Entity} = library.List
        table.clear(list)
        local byPlayer: {[Player]: Entity} = library.ByPlayer
        table.clear(byPlayer)

        for _, player: Player in ipairs(players:GetPlayers()) do
            local character, humanoid, root = characterParts(player)
            if not character or not humanoid or not root then
                continue
            end
            if humanoid.Health <= 0 then
                continue
            end

            local entity: any = library.pool[player]
            if not entity then
                entity = {}
                library.pool[player] = entity
            end
            entity.Player = player
            entity.Character = character
            entity.Humanoid = humanoid
            entity.RootPart = root
            entity.Head = character:FindFirstChild("Head") :: BasePart?
            entity.Health = humanoid.Health
            entity.MaxHealth = math.max(humanoid.MaxHealth, 1)
            entity.Distance = (originPosition - root.Position).Magnitude
            entity.Team = player.Team
            entity.IsFriendly = library:IsFriendly(player)
            entity.IsNPC = false
            entity.Name = player.Name

            byPlayer[player] = entity
            if player == localPlayer then
                library.LocalEntity = entity
            else
                table.insert(list, entity)
            end
        end

        if not byPlayer[localPlayer] then
            library.LocalEntity = nil
        end
        refreshNpcs(originPosition)
        return list
    end

    function library:IsAlive(): boolean
        local _, humanoid, root = characterParts(localPlayer)
        return humanoid ~= nil and root ~= nil and humanoid.Health > 0
    end

    function library:Get(player: Player): Entity?
        return library.ByPlayer[player]
    end

    function library:GetNpc(model: Model): Entity?
        return library.ByModel[model]
    end

    function library:ForEach(callback: (Entity) -> ()): ()
        for _, entity: Entity in ipairs(library.List) do
            callback(entity)
        end
    end

    function library:VisibleFrom(entity: Entity): boolean
        local camera: Camera? = currentWorkspace.CurrentCamera
        if not camera then
            return false
        end
        local parameters: RaycastParams = RaycastParams.new()
        parameters.FilterType = Enum.RaycastFilterType.Exclude
        parameters.IgnoreWater = true
        parameters.FilterDescendantsInstances = {
            entity.Character,
            localPlayer.Character :: any,
            camera,
        }
        local origin: Vector3 = camera.CFrame.Position
        local target: Vector3 = entity.RootPart.Position
        local result: RaycastResult? = currentWorkspace:Raycast(
            origin,
            target - origin,
            parameters
        )
        return result == nil
    end

    function library:Rig(entity: Entity): {{BasePart}}
        local character: Model = entity.Character
        local cached: any = library.rigs[character]
        if cached then
            return cached
        end
        local bones: {{BasePart}} = {}
        local definitions: {{string}} =
            character:FindFirstChild("UpperTorso") and R15_BONES or R6_BONES
        for _, pair: {string} in ipairs(definitions) do
            local first: BasePart? = character:FindFirstChild(pair[1]) :: BasePart?
            local second: BasePart? = character:FindFirstChild(pair[2]) :: BasePart?
            if first and second then
                table.insert(bones, {first, second})
            end
        end
        library.rigs[character] = bones
        return bones
    end

    function library:IsProtected(player: Player?): boolean
        if not player then
            return false
        end
        local targets: any = host.services and host.services.protectedTargets
        if targets and type(targets.isProtected) == "function" then
            return targets.isProtected(player) == true
        end
        return false
    end

    function library:InRange(query: RangeQuery): {Entity}
        local origin: BasePart? = localRoot()
        if not origin then
            return {}
        end
        local resolvedOrigin: BasePart = origin :: BasePart
        local facing: Vector3 = resolvedOrigin.CFrame.LookVector
            * Vector3.new(1, 0, 1)
        local halfAngle: number? = query.Angle and math.rad(query.Angle) / 2 or nil
        local candidates: {Entity} = {}

        local function consider(entity: Entity): ()
            if entity.Distance > query.Range then
                return
            end
            if not passesTeamFilter(entity, query.IgnoreTeam) then
                return
            end
            if not query.IgnoreFriends and library:IsProtected(entity.Player) then
                return
            end
            if halfAngle then
                local delta: Vector3 = (entity.RootPart.Position - resolvedOrigin.Position)
                    * Vector3.new(1, 0, 1)
                if delta.Magnitude > 0.001 and facing.Magnitude > 0.001 then
                    local dot: number = facing.Unit:Dot(delta.Unit)
                    if math.acos(math.clamp(dot, -1, 1)) > halfAngle then
                        return
                    end
                end
            end
            if not query.IgnoreWalls and not library:VisibleFrom(entity) then
                return
            end
            table.insert(candidates, entity)
        end

        for _, entity: Entity in ipairs(library.List) do
            consider(entity)
        end
        if query.IncludeNPCs then
            for _, entity: Entity in ipairs(library.NPCList) do
                consider(entity)
            end
        end

        local sortMode: string = query.Sort or "Distance"
        table.sort(candidates, function(first: Entity, second: Entity): boolean
            if sortMode == "Health" then
                return first.Health < second.Health
            end
            if sortMode == "Threat" then

                return first.Distance / math.max(first.Health, 1)
                    > second.Distance / math.max(second.Health, 1)
            end
            return first.Distance < second.Distance
        end)

        local limit: number = query.Limit or #candidates
        while #candidates > limit do
            table.remove(candidates)
        end
        return candidates
    end

    function library:ClosestToRay(
        origin: Vector3,
        direction: Vector3,
        query: EntityQuery?
    ): Entity?
        local options: EntityQuery = query or {}
        local unit: Vector3 = direction.Magnitude > 0
            and direction.Unit
            or Vector3.new(0, 0, -1)
        local maxDistance: number = options.MaxDistance or math.huge
        local best: Entity? = nil
        local bestScore: number = math.huge

        local function consider(entity: Entity): ()
            if not passesTeamFilter(entity, options.IgnoreTeam) then
                return
            end
            if entity.Distance > maxDistance then
                return
            end
            if not options.IgnoreFriends and library:IsProtected(entity.Player) then
                return
            end
            local part: BasePart = entity.RootPart
            if options.Part == "Head" and entity.Head then
                part = entity.Head :: BasePart
            end
            local offset: Vector3 = part.Position - origin
            local along: number = offset:Dot(unit)
            if along <= 0 then
                return
            end

            local perpendicular: number = (offset - unit * along).Magnitude
            if perpendicular < bestScore
                and (options.IgnoreWalls or library:VisibleFrom(entity)) then
                best = entity
                bestScore = perpendicular
            end
        end
        for _, entity: Entity in ipairs(library.List) do
            consider(entity)
        end
        if options.IncludeNPCs then
            for _, entity: Entity in ipairs(library.NPCList) do
                consider(entity)
            end
        end

        local radius: number? = options.Radius
        if radius and bestScore > radius then
            return nil
        end
        return best
    end

    function library:ClosestToCursor(query: EntityQuery?): Entity?
        local camera: Camera? = currentWorkspace.CurrentCamera
        if not camera then
            return nil
        end
        local ray: Ray? = host.services and host.services.aim.getRay()
        local origin: Vector3 = ray and ray.Origin or camera.CFrame.Position
        local direction: Vector3 = ray and ray.Direction or camera.CFrame.LookVector
        return library:ClosestToRay(origin, direction, query)
    end

    function library:Destroy(): ()
        for _, connection: RBXScriptConnection in ipairs(library.Connections) do
            pcall(function(): ()
                connection:Disconnect()
            end)
        end
        library.Connections = {}
        library.List = {}
        library.ByPlayer = {}
        library.NPCList = {}
        library.ByModel = {}
        library.npcModels = setmetatable({}, {__mode = "k"})
        library.LocalEntity = nil
        if activeLibrary == library then
            activeLibrary = nil
        end
    end

    for _, descendant: Instance in ipairs(currentWorkspace:GetDescendants()) do
        indexNpc(descendant)
    end
    table.insert(
        library.Connections,
        currentWorkspace.DescendantAdded:Connect(indexNpc)
    )
    table.insert(
        library.Connections,
        currentWorkspace.DescendantRemoving:Connect(unindexNpc)
    )

    table.insert(
        library.Connections,
        players.PlayerAdded:Connect(function(player: Player): ()
            table.insert(
                library.Connections,
                player.CharacterAdded:Connect(function(): ()
                    library:Refresh(true)
                end)
            )
            library:Refresh(true)
        end)
```

```
    )
    table.insert(
        library.Connections,
        players.PlayerRemoving:Connect(function(player: Player): ()
            library.ByPlayer[player] = nil
            library:Refresh(true)
        end)
    )
    for _, player: Player in ipairs(players:GetPlayers()) do
        table.insert(
            library.Connections,
            player.CharacterAdded:Connect(function(): ()
                library:Refresh(true)
            end)
        )
    end
    library:Refresh(true)

    activeLibrary = (library :: any) :: EntityLibrary
    Module.Initialized = true
    return (library :: any) :: EntityLibrary
end

function Module.destroy(): ()
    local library: EntityLibrary? = activeLibrary
    if library then
        library:Destroy()
    end
    activeLibrary = nil
    Module.Initialized = false
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/libraries/Entity.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/libraries/Framework.lua
BYTES: 31444
SHA-256: a0705591a469141f6c542bb0b13f738c1f07f48e14933cd5b8a570a97a3da37ac
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
export type CleanupItem = any

export type MovementInputService = {
    getVector: () -> (number, number),
    isJumpHeld: () -> boolean,
    onJumpRequest: ((() -> ())) -> RBXScriptConnection,
}

export type OwnershipService<T> = {
    set: (ownerId: string, value: T?) -> (),
    apply: (instance: Instance?) -> (),
}

export type ModuleServices = {
    movementInput: MovementInputService,
    aim: {getRay: () -> Ray?},
    screenCapture: {
        isAvailable: () -> boolean,
        capture: (string) -> (boolean, string?),
        writeText: (string, string) -> boolean,
    },
    menu: {
        isVisible: () -> boolean,
        isCapturingInput: () -> boolean,
        setVisible: (boolean) -> (),
    },
    mobileActions: {bindPlacement: (...any) -> any},
    protectedTargets: {
        isProtected: (Player?) -> boolean,
        setProvider: (((Player?) -> boolean)?) -> (),
    },
    fovOwnership: OwnershipService<number>,
    platformStandOwnership: OwnershipService<boolean>,
    activity: {
        set: (string, boolean) -> (),
        isActive: (string) -> boolean,
    },
    spoofAvatar: any,
    projectileCalibration: any,
    gameBridge: any,
    shortcuts: any,
    registries: any,
}

export type ModuleContext = {
    framework: Framework,
    entity: any,
    weapons: any,
    render: any,
    libraries: {[string]: any},
    host: any,
    services: ModuleServices,
}

export type Option = {
    Name: string,

    Value: any,
    Object: any,
    SetVisible: (self: Option, visible: boolean) -> (),
}

export type Module = {
    Name: string,
    Category: string,
    Enabled: boolean,
    Feature: any,
    Options: {[string]: Option},
    OptionMetadata: {any},

    CreateToggle: (self: Module, definition: any) -> Option,
    CreateSlider: (self: Module, definition: any) -> Option,
    CreateTwoSlider: (self: Module, definition: any) -> Option,
    CreateDropdown: (self: Module, definition: any) -> Option,
    CreateBind: (self: Module, definition: any) -> Option,
    CreateTextBox: (self: Module, definition: any) -> Option,
    CreateColor: (self: Module, definition: any) -> Option,
    CreateList: (self: Module, definition: any) -> Option,
    CreateButton: (self: Module, definition: any) -> Option,
    CreateSection: (self: Module, name: string) -> Option,
    CreateNote: (self: Module, text: string) -> (),

    Clean: (self: Module, item: CleanupItem) -> CleanupItem,
    Loop: (self: Module, callback: (number) -> ()) -> any,
    Render: (self: Module, callback: (number) -> ()) -> any,
    Event: (self: Module, signal: any, callback: (...any) -> ()) -> any,
    CleanAll: (self: Module) -> (),
    Toggle: (self: Module, value: boolean?) -> (),
    Notify: (self: Module, message: string) -> (),
    SetStatus: (self: Module, status: string?) -> (),
}

export type Category = {
    Name: string,
    Modules: {Module},
    CreateModule: (self: Category, definition: any) -> Module,
}

export type Framework = {
    Categories: {[string]: Category},
    Modules: {[string]: Module},
    Order: {string},
    Host: any,
    GetCategory: (self: Framework, name: string) -> Category,
    GetModule: (self: Framework, name: string) -> Module?,
    DisableAll: (self: Framework) -> (),
    Destroy: (self: Framework) -> (),
}

local Module = {
    Name = "Framework",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local KNOWN_CATEGORIES: {string} = {
    "Exploits", "Utilities", "World", "Render", "Blocks", "Items",
}
local activeFramework: Framework? = nil
local function isKnownCategory(name: string): boolean
    return table.find(KNOWN_CATEGORIES, name) ~= nil
end
local CATEGORY_ALIASES: {[string]: string} = {
    Combat = "Exploits", Blatant = "Exploits", Movement = "World", Other = "Utilities",
    Fun = "Utilities", Visuals = "Render", Protection = "Utilities",
    Utility = "Utilities", Spoof = "Utilities", General = "Utilities", Player = "World",
}

local function canonicalCategory(name: string): string
    if isKnownCategory(name) then
        return name
    end
    return CATEGORY_ALIASES[name] or "Utilities"
end

local function disposeItem(item: CleanupItem): ()
    if item == nil then
        return
    end
    local itemType: string = typeof(item)
    if itemType == "function" then
        pcall(item :: any)
        return
    end
    if itemType == "RBXScriptConnection" then
        pcall(function(): ()
            (item :: any):Disconnect()
        end)
        return
    end
    if itemType == "Instance" then
        pcall(function(): ()
            (item :: any):Destroy()
        end)
        return
    end
    if itemType == "table" then
        local candidate: any = item
        if type(candidate.Disconnect) == "function" then
            pcall(candidate.Disconnect, candidate)
            return
        end
        if type(candidate.Destroy) == "function" then
            pcall(candidate.Destroy, candidate)
            return
        end
        if type(candidate.disconnect) == "function" then
            pcall(candidate.disconnect, candidate)
            return
        end
    end
end

function Module.init(context: any): Framework
    local host: any = context
    local framework: any = {
        Categories = {},
        Modules = {},
        Order = {},
        Host = host,
    }

    local function normalizeSearchName(value: any): string
        return string.lower(tostring(value or "")):gsub("[%s%-%_]", "")
    end
    local moduleSearch: any = {
        entries = {},
        aliases = {
            esp = "playeresp",
            playeresp = "playeresp",
            itemesp = "itemesp",
            autoclicker = "autoclicker",
            aimassist = "aimbot",
            killaura = "killaura",
            fly = "flight",
            speed = "speedhack",
            nofall = "nofall",
        },
    }
    function moduleSearch.Register(feature: any): ()
        if type(feature) ~= "table" then
            return
        end
        local normalized: string = normalizeSearchName(feature.name)
        local binding: any = host.state.shortcutBindings
            and host.state.shortcutBindings[feature.configKey]
        local entry: any = {
            normalized = normalized,
            displayName = feature.name,
            configKey = feature.configKey,
            actionId = feature.configKey,
            aliases = {},
            kind = feature.kind,
            searchable = feature.searchable ~= false,
            feature = feature,
            activate = feature.activate,
            invoke = feature.activate,
        }
        function entry.currentKey(): any
            return binding and binding.key or Enum.KeyCode.Unknown
        end
        function entry.conflict(): boolean
            local key: any = entry.currentKey()
            if key == nil or key == Enum.KeyCode.Unknown then
                return false
            end
            local uses: number = 0
            for _, candidate: any in pairs(host.state.shortcutBindings or {}) do
                if candidate.key == key then
                    uses += 1
                end
            end
            return uses > 1
        end
        function entry.capture(key: Enum.KeyCode): boolean
            return binding ~= nil and binding.assign(key) == true
        end
        function entry.remove(): ()
            if binding and binding.key ~= Enum.KeyCode.Unknown then
                binding.assign(binding.key)
            end
        end
        for alias: string, target: string in pairs(moduleSearch.aliases) do
            if target == normalized then
                table.insert(entry.aliases, alias)
            end
        end
        table.sort(entry.aliases)
        function entry.enabled(): boolean
            return feature.enabled == true
        end
        table.insert(moduleSearch.entries, entry)
    end
    function moduleSearch.Unregister(feature: any): ()
        for index: number = #moduleSearch.entries, 1, -1 do
            if moduleSearch.entries[index].feature == feature then
                table.remove(moduleSearch.entries, index)
            end
        end
    end
    function moduleSearch.Resolve(query: any): any
        local normalized: string = normalizeSearchName(query)
        if normalized == "" then
            return {status = "not_found"}
        end
        normalized = moduleSearch.aliases[normalized] or normalized
        local matches: {any} = {}
        for _, entry: any in ipairs(moduleSearch.entries) do
            if entry.searchable and entry.normalized == normalized then
                table.insert(matches, entry)
            end
        end
        if #matches == 0 then
            return {status = "not_found"}
        end
        if #matches > 1 then
            return {status = "ambiguous", matches = matches}
        end
        return {status = "success", entry = matches[1]}
    end
    function moduleSearch.Execute(query: any): any
        local result: any = moduleSearch.Resolve(query)
        if result.status ~= "success" then
            return result
        end
        local entry: any = result.entry
        if entry.kind ~= "toggle" and entry.kind ~= "action" then
            return {status = "not_toggleable", entry = entry}
        end
        if type(entry.activate) ~= "function" then
            return {status = "not_toggleable", entry = entry}
        end
        entry.activate()
        return {
            status = "success",
            entry = entry,
            enabled = entry.enabled(),
        }
    end
    host.state.moduleSearch = moduleSearch

    local function newOption(
        name: string,
        initialValue: any,
        optionType: string,
        definition: any?
    ): any
        local source: any = definition or {}
        local defaultValue: any = initialValue
        if type(initialValue) == "table" then
            defaultValue = table.clone(initialValue)
        end
        local step: number? = source.Step
        if step == nil and (optionType == "slider" or optionType == "range") then
            local integerOnly: boolean = type(source.Min) == "number"
                and type(source.Max) == "number"
                and source.Min % 1 == 0
                and source.Max % 1 == 0
                and ((type(initialValue) == "number" and initialValue % 1 == 0)
                    or (type(initialValue) == "table"
                        and initialValue.Min % 1 == 0
                        and initialValue.Max % 1 == 0))
            step = integerOnly and 1 or 0
        end
        local option: any = {
            Name = name,
            Value = initialValue,
            Object = nil,
            Metadata = {
                id = "",
                configKey = "",
                label = name,
                type = optionType,
                default = defaultValue,
                current = initialValue,
                min = source.Min,
                max = source.Max,
                step = step,
                values = source.List and table.clone(source.List) or nil,
                callback = source.Function,
                show = source.Show,
                visible = source.Show == nil,
                action = optionType == "action",
                bind = optionType == "bind",
                color = optionType == "color",
                rangeLow = source.DefaultMin,
                rangeHigh = source.DefaultMax,
            },
        }
        function option:SetValue(value: any): ()
            self.Value = value
            self.Metadata.current = value
            if type(value) == "table" and value.Min ~= nil and value.Max ~= nil then
                self.Metadata.rangeLow = value.Min
                self.Metadata.rangeHigh = value.Max
            end
        end
        function option:SetVisible(visible: boolean): ()
            self.Metadata.visible = visible
            local row: any = self.Object
            if typeof(row) == "Instance" and row.ClassName ~= "Frame" then
                row = row.Parent
            end
            if host.setOptionVisible and row then
                host.setOptionVisible(self.Feature, row, visible)
            end
        end
        return option
    end

    local function createModule(category: any, definition: any): Module
        local moduleName: string = definition.Name
        -- Keep a stable internal module/config identity when a game adapts the
        -- visible card. This also names its mobile shortcut correctly at boot.
        local displayName: string = definition.DisplayName or moduleName
        local categoryName: string = definition.Category or category.Name
        local created: any = {
            Name = moduleName,
            Category = categoryName,
            Enabled = false,
            Options = {},
            OptionMetadata = {},
            Cleanup = {},
        }

        local function onToggle(enabled: boolean?): ()
            local nextState: boolean = enabled == true
            created.Enabled = nextState
            if not nextState then
                created:CleanAll()
            end
            local handler: any = definition.Function
            if type(handler) == "function" then
                handler(nextState)
            end
            if not nextState then

                created:CleanAll()
            end
        end

        local kind: string = definition.Kind
            or (definition.Action == true and "action")
            or "toggle"
        local feature: any = host.createUniversalFeature(
            displayName,
            definition.Tooltip or definition.Description or "",
            definition.Order or (#category.Modules + 1),
            kind == "action" and function(): ()
                if type(definition.Function) == "function" then
                    definition.Function(true)
                end
            end or onToggle,
            {
                categoryName = categoryName,
                kind = kind,
                silentAction = definition.Silent == true,
                searchable = definition.Searchable ~= false,
                registry = definition.Registry,
                parent = definition.Parent,
                configKey = definition.ConfigKey,
            }
        )
        created.Feature = feature
        feature.frameworkModule = true

        local function register(option: any, definition2: any?): Option
            option.Feature = feature
            local optionId: string = option.Name:gsub("%W", "")
            option.Metadata.id = optionId
            option.Metadata.configKey = feature.configKey .. "." .. optionId
            local gated: boolean = false
            if definition2 and type(definition2.Show) == "table" then
                option.Show = definition2.Show
                gated = true
            end
            created.Options[option.Name] = option
            table.insert(created.OptionMetadata, option.Metadata)
            feature.optionMetadata = created.OptionMetadata

            if gated then
                created:RefreshVisibility()
            end
            return option
        end

        local function ruleHolds(rule: any): boolean
            local source: any = created.Options[rule.Option]
            local holds: boolean = false
            if source ~= nil then
                if type(rule.Values) == "table" then
                    for _, wanted: any in ipairs(rule.Values) do
                        if source.Value == wanted then
                            holds = true
                            break
                        end
                    end
                else
                    holds = source.Value == true
                end
            end
            if rule.Invert == true then
                return not holds
            end
            return holds
        end

        function created:RefreshVisibility(): ()
            for _, option: any in pairs(created.Options) do
                local rule: any = option.Show
                local available: boolean = created.OptionFilter == nil or created.OptionFilter(option.Name)
                if type(rule) ~= "table" and available and not option.FilteredOut then continue end
                option.FilteredOut = not available
                local visible: boolean = available
                if type(rule) ~= "table" then
                    -- Explicitly unavailable in the current game adapter.
                elseif rule.Option ~= nil then
                    visible = visible and ruleHolds(rule)
                else
                    for _, nested: any in ipairs(rule) do
                        if not ruleHolds(nested) then
                            visible = false
                            break
                        end
                    end
                end
                if option.Visible ~= visible then
                    option.Visible = visible
                    option:SetVisible(visible)
                end
            end
        end

        local function valueCallback(option: any, definition2: any): (any) -> ()
            local function changed(value: any): ()
                option:SetValue(value)
                if type(definition2.Function) == "function" then
                    definition2.Function(value)
                end
                created:RefreshVisibility()
            end
            option.Metadata.callback = changed
            return changed
        end

        function created:CreateToggle(definition2: any): Option
            local option: any = newOption(definition2.Name, definition2.Default == true, "toggle", definition2)
            option.Object = host.addToggleOption(
                feature,
                definition2.Name,
                definition2.Default == true,
                valueCallback(option, definition2),
                definition2.Tooltip,
                definition2.Bindable == true
            )
            return register(option, definition2)
        end

        function created:CreateSlider(definition2: any): Option
            local default: number = definition2.Default or definition2.Min or 0
            local option: any = newOption(definition2.Name, default, "slider", definition2)
            option.Object = host.addNumberOption(
                feature,
                definition2.Name,
                default,
                definition2.Min or 0,
                definition2.Max or 100,
                valueCallback(option, definition2),
                definition2.Tooltip,
                definition2.Step
            )
            return register(option, definition2)
        end

        function created:CreateTwoSlider(definition2: any): Option
            local minimum: number = definition2.Min or 0
            local maximum: number = definition2.Max or 100
            local low: number = definition2.DefaultMin or minimum
            local high: number = definition2.DefaultMax or maximum
            local option: any = newOption(definition2.Name, {Min = low, Max = high}, "range", definition2)
            local builder: any = host.addRangeOption
                or (host.state and host.state.addRangeOption)
            option.GetRandomValue = function(_self: any): number
                local band: any = option.Value
                local lowValue: number = band.Min
                local highValue: number = math.max(band.Max, lowValue)
                return lowValue + math.random() * (highValue - lowValue)
            end
            if type(builder) ~= "function" then
                return register(option)
            end
            local function rangeChanged(newLow: number, newHigh: number): ()
                option:SetValue({Min = newLow, Max = newHigh})
                if type(definition2.Function) == "function" then
                    definition2.Function(newLow, newHigh)
                end
                created:RefreshVisibility()
            end
            option.Metadata.callback = rangeChanged
            option.Object = builder(feature, definition2.Name, {
                min = minimum,
                max = maximum,
                step = definition2.Step,
                defaultLow = low,
                defaultHigh = high,
                integer = definition2.Decimal == nil,
                hint = definition2.Tooltip,
                onChanged = rangeChanged,
            })
            return register(option, definition2)
        end

        function created:CreateDropdown(definition2: any): Option
            local list: {string} = definition2.List or {}
            local index: number = definition2.Index or 1
            local option: any = newOption(definition2.Name, list[index], "dropdown", definition2)
            option.Object = host.addCycleOption(
                feature,
                definition2.Name,
                list,
                index,
                valueCallback(option, definition2),
                definition2.Tooltip
            )
            return register(option, definition2)
        end

        function created:CreateBind(definition2: any): Option
            local default: Enum.KeyCode = definition2.Default or Enum.KeyCode.Unknown
            local option: any = newOption(definition2.Name, default, "bind", definition2)
            option.Object = host.addKeyOption(
                feature,
                definition2.Name,
                default,
                valueCallback(option, definition2),
                definition2.Tooltip
            )
            return register(option, definition2)
        end

        function created:CreateTextBox(definition2: any): Option
            local default: string = definition2.Default or ""
            local option: any = newOption(definition2.Name, default, "text", definition2)
            option.Object = host.addTextOption(
                feature,
                definition2.Name,
                default,
                valueCallback(option, definition2),
                definition2.Persist ~= false,
                definition2.Tooltip
            )
            if option.Object and definition2.Placeholder then
                option.Object.PlaceholderText = definition2.Placeholder
            end
            option.Set = function(self: any, value: string): ()
                self.Metadata.callback(value)
                if self.Object then
                    self.Object.Text = value
                end
```

```
        end
        return register(option, definition2)
    end

    function created:CreateColor(definition2: any): Option
        local default: Color3 = definition2.Default or Color3.new(1, 1, 1)
        local option: any = newOption(definition2.Name, default, "color", definition2)
        option.Object = host.addColorOption(
            feature,
            definition2.Name,
            default,
            valueCallback(option, definition2),
            definition2.Tooltip
        )
        return register(option, definition2)
    end

    function created:CreateList(definition2: any): Option
        local option: any = newOption(definition2.Name, {}, "list", definition2)
        option.Selected = {}

        local builder: any = host.addListOption
            or (host.state and host.state.addListOption)
        if type(builder) ~= "function" then

            return register(option)
        end
        local optionKey: string = feature.configKey
            .. "."
            .. definition2.Name:gsub("%W", "")
        local function listChanged(selected: any, names: {string}): ()
            option:SetValue(selected)
            option.Selected = names
            if type(definition2.Function) == "function" then
                definition2.Function(selected, names)
            end
            created:RefreshVisibility()
        end
        option.Metadata.callback = listChanged
        option.Object = builder(feature, definition2.Name, {
            items = definition2.Items,
            single = definition2.Single == true,
            onRemove = definition2.OnRemove,
            default = definition2.Default,
            hint = definition2.Tooltip,
            emptyText = definition2.EmptyText,
            onChanged = listChanged,
        })

        local controls: any = host.state
            and host.state.listControls
            and host.state.listControls[optionKey]
        option.Set = function(_self: any, item: string, on: boolean): ()
            if controls then
                controls.set(item, on)
            end
        end
        option.Refresh = function(_self: any): ()
            if controls then
                controls.refresh()
            end
        end
        option.IsSelected = function(_self: any, item: string): boolean
            if controls then
                return controls.selected[item] == true
            end
            return false
        end
        return register(option, definition2)
    end

    function created:CreateButton(definition2: any): Option
        if definition2.Hold then
            assert(type(definition2.OnRrelease) == "function", "A held action requires OnRrelease")
        end
        local option: any = newOption(definition2.Name, nil, "action", definition2)
        option.Metadata.hold = definition2.Hold == true
        local function run(): ()
            if option.Metadata.visible == false then return end
            if type(definition2.Function) == "function" then
                definition2.Function()
            end
        end
        option.Metadata.callback = run
        option.Object = host.addActionOption(
            feature,
            definition2.Name,
            run,
            definition2.Tooltip,
            definition2.Hold and {hold = true, onRelease = definition2.OnRelease} or nil
        )
        return register(option, definition2)
    end

    function created:CreateSection(name: string): Option
        local option: any = newOption(name, nil, "section", nil)
        option.Object = host.addSectionOption
            and host.addSectionOption(feature, name)
            or nil

        return register(option)
    end

    function created:CreateNote(text: string): ()
        if host.addInformationOption then
            host.addInformationOption(feature, text)
        end
    end

    function created:Clean(item: CleanupItem): CleanupItem
        table.insert(created.Cleanup, item)
        return item
    end

    -- v139: the scheduler is told which module owns the task, so a frame
    -- time warning names the card instead of "task".
    function created:Loop(callback: (number) -> ()): any
        return created:Clean(host.TaskManager:Connect(callback, nil, moduleName))
    end

    function created:Render(callback: (number) -> ()): any
        return created:Clean(host.TaskManager:Connect(callback, "render", moduleName))
    end

    function created:Event(signal: any, callback: (...any) -> ()): any
        return created:Clean(signal:Connect(callback))
    end

    function created:CleanAll(): ()
        local items: {CleanupItem} = created.Cleanup
        created.Cleanup = {}

        for index: number = #items, 1, -1 do
            disposeItem(items[index])
        end
    end

    function created:Toggle(value: boolean?): ()
        if value ~= nil and value == created.Enabled then
            return
        end
        -- A real state change: run the module's Function with the wanted
        -- state so it can stop loops and restore whatever it changed.
        -- (feature.activate only flips the card UI and must not be used
        -- here - module.destroy() relies on Toggle(false) actually
        -- disabling the module.)
        onToggle(value == nil and not created.Enabled or value == true)
    end

    function created:Reveal(): ()
        if feature and type(feature.activate) == "function" then
            feature.activate()
        end
    end

    function created:SetStatus(status: string?): ()
        created.Status = status
        if feature and type(feature.SetStatus) == "function" then
            feature:SetStatus(status)
        end
    end

    function created:Notify(message: string): ()
        if host.notify then
            host.notify(moduleName .. ": " .. message)
        end
    end

    table.insert(category.Modules, created)
    framework.Modules[moduleName] = created
    table.insert(framework.Order, moduleName)
    return (created :: any) :: Module
end

function framework:GetCategory(name: string): Category
    local existing: any = framework.Categories[name]
    if existing then
        return existing
    end

    local canonical: string = canonicalCategory(name)
    local category: any = framework.Categories[canonical]
    if not category then
        category = {
            Name = canonical,
            Modules = {},
        }
        function category:CreateModule(definition: any): Module
            return createModule(category, definition)
        end
        framework.Categories[canonical] = category
    end
    if name ~= canonical then
        framework.Categories[name] = category
    end
    return (category :: any) :: Category
end

function framework:GetModule(name: string): Module?
    return framework.Modules[name]
end

function framework:DisableAll(): ()
    for _, moduleName: string in ipairs(framework.Order) do
        local entry: any = framework.Modules[moduleName]
        if entry and entry.Enabled then
            pcall(function(): ()
                entry:Toggle(false)
            end)
        end
        if entry then
            pcall(function(): ()
                entry:CleanAll()
            end)
        end
    end
end

function framework:Destroy(): ()
    framework:DisableAll()
    framework.Categories = {}
    framework.Modules = {}
    framework.Order = {}
    if host.state.moduleSearch == moduleSearch then
        host.state.moduleSearch = nil
    end
    table.clear(moduleSearch.entries)
    if activeFramework == framework then
        activeFramework = nil
    end
end

for _, name: string in ipairs(KNOWN_CATEGORIES) do
    framework:GetCategory(name)
end
for oldName: string in pairs(CATEGORY_ALIASES) do
    framework:GetCategory(oldName)
end

activeFramework = (framework :: any) :: Framework
Module.Initialized = true
return (framework :: any) :: Framework
end

function Module.destroy(): ()
    local framework: Framework? = activeFramework
    if framework then
        framework:Destroy()
    end
    activeFramework = nil
    Module.Initialized = false
end

return Module
```

FIN DEL ARCHIVO: src/libraries/Framework.lua

ARCHIVO: src/libraries/GameRoles.lua
BYTES: 5327
SHA-256: 3782a7a178df0f90f7d5c0d8f0437727cfdd1392680286a3e92eb7928ae1bb50
INICIO DEL ARCHIVO

```
-- Shared role presentation contract. Games register data; render modules own
-- their palettes/filters. A Chams colour must never overwrite PlayerESP's.
local Module = {Name = "GameRoles", Initialized = false}
local active: any = nil

function Module.init(host: any): any
    local bridge = host.services.gameBridge
    local service = {bindings = {}}

    function service:Register(provider: any): () -> ()
        assert(type(provider) == "table" and type(provider.Get) == "function", "GameRoles needs Get(player)")
        assert(type(provider.Roles) == "table", "GameRoles needs a Roles array")
        bridge.registerRoles(provider)
        return function() bridge.unregisterRoles(provider) end
    end

    function service:Bind(card: any, settings: any?): any
        settings = settings or {}
        local binding = {provider = nil, rows = {}, roles = {}, dead = false}
        local function visibility(option, visible)
            if visible then option.Show = nil else option.Show = {Option = "__GameRolesAbsent", Values = {true}} end
            if option.SetVisible then option:SetVisible(visible) end
        end
        local function update(provider)
            if binding.dead then return end
            binding.provider = type(provider) == "table" and provider or nil
            table.clear(binding.roles)
            if binding.provider then
                for _, role in ipairs(type(provider.Roles) == "table" and provider.Roles or {}) do
                    if type(role) == "string" and role ~= "" and #binding.roles < 32 and not table.find(binding.roles, role) then
                        table.insert(binding.roles, role)
                    end
                end
            end
            if settings.palette ~= false and #binding.roles > 0 and not card.Options["Role colours"] then
                binding.rows["Role colours"] = card:CreateToggle({Name = "Role colours", Default = true})
            end
            for _, role in ipairs(binding.roles) do
                local colourName = role .. " colour"
                if settings.palette ~= false and not binding.rows[colourName] then
                    local colour = provider.Colors and provider.Colors[role]
                    if type(provider.GetColor) == "function" then
                        local ok, value = pcall(provider.GetColor, role)
                        if ok and typeof(value) == "Color3" then colour = value end
                    end
                    binding.rows[colourName] = card:CreateColor({Name = colourName,
                        Default = typeof(colour) == "Color3" and colour or Color3.fromRGB(236,236,240),
                        Show = {Option = "Role colours"}})
                end
                if settings.filters and not binding.rows[role .. " ESP"] then
                    binding.rows[role .. " ESP"] = card:CreateToggle({Name = role .. " ESP", Default = true})
                end
            end
            for name, option in pairs(binding.rows) do
                local present = name == "Role colours" and #binding.roles > 0
                for _, role in ipairs(binding.roles) do
                    if name == role .. " colour" or name == role .. " ESP" then present = true; break end
                end
                visibility(option, present)
                if present and name ~= "Role colours" and string.sub(name, -7) == " colour" then
                    option.Show = {Option = "Role colours"}
                end
            end
            card:RefreshVisibility()
        end
        function binding:Role(player)
            if not self.provider then return nil end
            local getter = bridge.playerRole or self.provider.Get
            local ok, role = pcall(getter, player)
            return ok and type(role) == "string" and table.find(self.roles, role) and role or nil
        end
        function binding:Resolve(player, fallback)
            local role = self:Role(player)
            local option = role and card.Options[role .. " colour"]
            local enabled = card.Options["Role colours"]
            return option and enabled and enabled.Value and option.Value or fallback, role
        end
        function binding:Allows(player)
            local role = self:Role(player)
            local option = role and card.Options[role .. " ESP"]
            return not option or option.Value ~= false
        end
        local disconnect = bridge.onEvent(function(kind, provider)
            if kind == "roles" then update(provider) end
        end)
        function binding:Destroy()
            if self.dead then return end
            self.dead = true
            if disconnect then disconnect(); disconnect = nil end
            self.provider = nil
            local index = table.find(service.bindings, self)
            if index then table.remove(service.bindings, index) end
        end
        table.insert(service.bindings, binding)
        return binding
    end
    function service:Destroy()
        for _, binding in ipairs(table.clone(self.bindings)) do binding:Destroy() end
    end
    host.services.gameRoles = service
    active = service
    Module.Initialized = true
    return service
end
function Module.destroy()
    if active then active:Destroy(); active = nil end
    Module.Initialized = false
end
return Module
```

FIN DEL ARCHIVO: src/libraries/GameRoles.lua

ARCHIVO: src/libraries/Manifest.lua
BYTES: 11454
SHA-256: 508e74b29950d975ce6df47a1f1057a8b100d34d48c5490cbeae5596ae683526
INICIO DEL ARCHIVO

```
export type ModuleEntry = {
    path: string,
    name: string,
    category: string,
}

export type GamePlaceEntry = {
    placeId: number,
    module: string,
    displayName: string,
}

export type CategoryCatalog = {
    order: {string},
    aliases: {[string]: string},
    default: string,
}

export type Manifest = {
    version: number,
    categories: CategoryCatalog,
    cards: {[string]: string},
    widgets: string,
    core: {string},
    libraries: {string},
    modules: {ModuleEntry},
    gamePlaces: {GamePlaceEntry},
    gameModules: {[string]: {string}},
}

local Manifest: Manifest = {
    version = 1,

    -- v138.9: the catalog also owns the windows. Wurst.lua and ClickGui.lua used
    -- to keep their own copies of these two lists, which is exactly how a card
    -- ended up in one window in the menu and another one in the code.
    categories = {
        order = {
            "Exploits",
            "Utilities",
            "World",
            "Render",
            "Blocks",
            "Items",
        },
        aliases = {
            ["Combat"] = "Exploits",
            ["Blatant"] = "Exploits",
            ["Movement"] = "World",
            ["Other"] = "Utilities",
            ["Visuals"] = "Render",
            ["Protection"] = "Utilities",
            ["Utility"] = "Utilities",
            ["Spoof"] = "Utilities",
            ["Fun"] = "Utilities",
            ["General"] = "Utilities",
            ["Player"] = "World",
        },
        default = "Utilities",
    },

    -- Display name of a card -> window. Only consulted when the card does not
    -- pass categoryName itself.
    cards = {
        -- <arandommenu:cards> managed by tools/card.py
        ["Aimbot"] = "Exploits",
        ["Anti AFK"] = "World",
        ["Anti Fling"] = "Utilities",
        ["Anti Void"] = "Utilities",
        ["Anti-AFK"] = "Other",
        ["Anti-Fling"] = "Movement",
        ["AntiAFK"] = "Other",
        ["Auto Buy"] = "Other",
        ["Auto Dribble"] = "Exploits",
        ["Auto Generator"] = "Other",
        ["Auto Header"] = "Exploits",
        ["Auto Parry / Auto Block"] = "Movement",
        ["Auto Pickup"] = "Other",
        ["Auto Shoot"] = "Utilities",
        ["Auto Swing"] = "Exploits",
        ["Auto Tackle"] = "Exploits",
        ["Ball Trajectory"] = "Render",
        ["Bed Nuker"] = "Exploits",
        ["Best Goal Visual"] = "Render",
        ["Blurt Roles"] = "World",
        ["Cooldown Modifier"] = "Exploits",
        ["Damage Boost"] = "Exploits",
        ["Animations"] = "World",
        ["Disguise"] = "World",
        ["FOV"] = "Render",
        ["Fast Vault Optimizer"] = "Movement",
        ["Fling"] = "Exploits",
        ["Fly"] = "Exploits",
        ["Freeze Animations"] = "Utilities",
        ["Fullbright"] = "Render",
        ["Goalkeeper Assist"] = "Exploits",
        ["Gravity"] = "World",
        ["Gun Aimbot"] = "Exploits",
        ["Improve FPS"] = "World",
        ["Invisibility"] = "Utilities",
        ["Invisible"] = "Exploits",
        ["Jump Settings"] = "Exploits",
        ["KillAura"] = "Exploits",
        ["Killer Hit Aura"] = "Exploits",
        ["Killer Power Cooldowns"] = "Render",
        ["Killer Proximity Visualizer"] = "Render",
        ["Knife Stab"] = "Exploits",
        ["Knife Throw"] = "Exploits",
        ["Lag Switch"] = "Exploits",
        ["Large Shoot"] = "Exploits",
        ["Line of Sight ESP"] = "Render",
        ["Locker & Hideout Detector"] = "Render",
        ["Noclip"] = "Exploits",
        ["Noclip (Vaults & Pallets)"] = "Exploits",
        ["Notifications"] = "Utilities",
        ["Optimize Blind / Flash Protection"] = "Movement",
        ["Pass Assist"] = "Exploits",
        ["Player ESP"] = "Render",
        ["Power Shot"] = "Exploits",
        ["Proximity Prompts"] = "Exploits",
        ["Quick Actions"] = "Other",
        ["Rejoin Server"] = "Other",
        ["Remote Logger"] = "Other",
        ["Round Info"] = "Render",
        ["Scaffold"] = "Movement",
        ["Server Aura"] = "Exploits",
        ["Server Miner"] = "Other",
        ["Server Scaffold"] = "Movement",
        ["Server Settings"] = "Utilities",
        ["Shoot Assist"] = "Exploits",
        ["Silent Aim"] = "Exploits",
        ["Silent Steps / Anti-Sound"] = "Movement",
        ["SilentAIM"] = "Exploits",
        ["Speed Settings"] = "Exploits",
        ["Spider"] = "Exploits",
        ["Teleport"] = "Utilities",
        ["Time Changer"] = "Render",
        ["VD Field of View"] = "Render",
        ["VD Fullbright"] = "Render",
        ["Walk Speed"] = "World",
        ["WalkSpeed Hack"] = "Exploits",
        ["X-Ray"] = "Render",
        ["Zoom Unlocker"] = "Render",
        -- </arandommenu:cards>
    },

    widgets = "src/guis/Wurst/Code/Widgets.lua",
    core = {
        "src/libraries/Framework.lua",
    },
    libraries = {
        "src/libraries/Entity.lua",
        "src/libraries/Targeting.lua",
        "src/libraries/Weapons.lua",
        "src/libraries/Render.lua",
        "src/libraries/GameRoles.lua",
        "src/libraries/CameraAccess.lua",

        "src/guis/Wurst/Code/Cards.lua",

        "src/guis/Wurst/Code/WindowManager.lua",

        "src/guis/Wurst/Code/ClickGui.lua",

        "src/guis/Wurst/Code/FloatingWindows.lua",

        "src/guis/Wurst/Code/SettingsPage.lua",

        "src/guis/Wurst/Code/MobileActions.lua",

        "src/guis/Wurst/Code/Furniture.lua",
    },
    modules = {
        -- <arandommenu:modules> managed by tools/card.py
        {path = "src/games/universal/Combat/AimAssist.lua", name = "Aimbot", category = "Exploits"},
        {path = "src/games/universal/Blatant/Swim.lua", name = "Air Swim", category = "Exploits"},
        {path = "src/games/universal/Movement/AirWalk.lua", name = "Air Walk", category = "Exploits"},
        {path = "src/games/universal/Other/AntiAfk.lua", name = "Anti AFK", category = "World"},
        {path = "src/games/universal/Movement/AntiFling.lua", name = "Anti Fling", category = "Utilities"},
        {path = "src/games/universal/Render/Chams.lua", name = "Chams", category = "Render"},
        {path = "src/games/universal/Other/Animations.lua", name = "Animations", category = "World"},
        {path = "src/games/universal/Other/Disguise.lua", name = "Disguise", category = "World"},
        {path = "src/games/universal/Blatant/Fly.lua", name = "Fly", category = "Exploits"},
        {path = "src/games/universal/Render/FieldOfView.lua", name = "FOV", category = "Render"},
        {path = "src/games/universal/Render/FreeCam.lua", name = "FreeCam", category = "World"},
        {path = "src/games/universal/Movement/FreezeMovements.lua", name = "Freeze Animations", category = "Utilities"},
        {path = "src/games/universal/Render/Fullbright.lua", name = "Fullbright", category = "Render"},
        {path = "src/games/universal/Movement/Gravity.lua", name = "Gravity", category = "World"},
        {path = "src/games/universal/Render/HP.lua", name = "HP", category = "Render"},
        {path = "src/games/universal/Other/ImproveFps.lua", name = "Improve FPS", category = "World"},
        {path = "src/games/universal/Movement/InfiniteJump.lua", name = "Infinite Jump", category = "Exploits"},
        {path = "src/games/universal/Blatant/Invisible.lua", name = "Invisibility", category = "Utilities"},
        {path = "src/games/universal/Render/ItemRender.lua", name = "ItemESP", category = "Render"},
        {path = "src/games/universal/Movement/Jump.lua", name = "Jump Settings", category = "Exploits"},
        {path = "src/games/universal/Combat/KillAura.lua", name = "KillAura", category = "Exploits"},
        {path = "src/games/universal/Render/Markers.lua", name = "Markers", category = "Render"},
        {path = "src/games/universal/Render/Nametags.lua", name = "Nametags", category = "Render"},
        {path = "src/games/universal/Blatant/Noclip.lua", name = "Noclip", category = "Exploits"},
        {path = "src/games/universal/Render/NPCESP.lua", name = "NPCESP", category = "Render"},
        {path = "src/games/universal/Render/Arrows.lua", name = "Player Pointers", category = "Render"},
        {path = "src/games/universal/Render/PlayerESP.lua", name = "PlayerESP", category = "Render"},
        {path = "src/games/universal/Other/ProximityPrompts.lua", name = "Proximity Prompts", category = "Exploits"},
        {path = "src/games/universal/Other/RejoinServer.lua", name = "Rejoin Server", category = "Utilities"},
        {path = "src/games/universal/Other/ServerSettings.lua", name = "Server Settings", category = "Utilities"},
        {path = "src/games/universal/Render/Spectate.lua", name = "Spectate", category = "World"},
        {path = "src/games/universal/Movement/Speed.lua", name = "Speed Settings", category = "Exploits"},
        {path = "src/games/universal/Movement/ClickTeleport.lua", name = "Teleport", category = "Utilities"},
        {path = "src/games/universal/Blatant/WalkFling.lua", name = "Walk Fling", category = "Exploits"},
        {path = "src/games/universal/Blatant/Spider.lua", name = "Wall Crawling", category = "Exploits"},
        {path = "src/games/universal/Render/XRay.lua", name = "X-Ray", category = "Render"},
        {path = "src/games/universal/Render/ZoomUnlocker.lua", name = "Zoom", category = "Render"},
        -- </arandommenu:modules>
    },

    gamePlaces = {
        {placeId = 71480482338212, module = "BedFight", displayName = "Bed Fight"},
        {placeId = 142823291, module = "MM2", displayName = "Murder Mystery 2"},
        {placeId = 14315258385, module = "TRS", displayName = "The Regular Show"},
        {placeId = 93978595733734, module = "VD", displayName = "Virtual Dealer"},
        {placeId = 92876937625630, module = "X2Duels", displayName = "X2 Duels (Legacy)"},
        {placeId = 13585690811593l, module = "X2Duels", displayName = "X2 Duels"},
    },

    gameModules = {
        MM2 = {
            "src/games/MM2/base.lua",
            "src/games/MM2/Prediction.lua",
            "src/games/MM2/Blatant/Shoot.lua",
            "src/games/MM2/Blatant/KnifeAura.lua",
            "src/games/MM2/Combat/AutoShoot.lua",
            "src/games/MM2/Combat/KnifeThrow.lua",
            "src/games/MM2/Render/RoundEsp.lua",
            "src/games/MM2/Render/CoinEsp.lua",
            "src/games/MM2/Movement/Sprint.lua",
            "src/games/MM2/Movement/Teleport.lua",
            "src/games/MM2/Fun/BlurtRoles.lua",
            "src/games/MM2/Other/Notifications.lua",
            "src/games/MM2/Other/AutoUnbox.lua",
            "src/games/MM2/Other/FlingEngine.lua",
            "src/games/MM2/Other/Fling.lua",
            "src/games/MM2/Other/AutoFarm.lua",
            "src/games/MM2/Other/AutoVote.lua",
            "src/games/MM2/Other/CoinsAura.lua",
            "src/games/MM2/Other/GunAura.lua",
            "src/games/MM2/Other/AutoWin.lua",
            "src/games/MM2/Other/GetAllEmotes.lua",
        },
        VD = {
            "src/games/VD/base.lua",
        },
        BedFight = {
            "src/games/BedFight/base.lua",
            "src/games/BedFight/Combat/AutoSwing.lua",
            "src/games/BedFight/Render/RoundInfo.lua",
        },
        X2Duels = {
            "src/games/X2Duels/Rules.lua",
            "src/games/X2Duels/base.lua",
            "src/games/X2Duels/Other/AutoFarm.lua",
        },
        TRS = {
            "src/games/TRS/base.lua",
        },
    },
}

return Manifest
```

FIN DEL ARCHIVO: src/libraries/Manifest.lua

ARCHIVO: src/libraries/Render.lua
BYTES: 22127
SHA-256: 0f28014967b093aa785397f00891999b2ef2a0dd416bf5238fec8fdabf632b99
INICIO DEL ARCHIVO

```
export type Rect = {
```

```
    left: number,
    right: number,
    top: number,
    bottom: number,
    width: number,
    height: number,
    centreX: number,
    centreY: number,
}

export type DrawingSet = {
    root: Frame,
    Line: (self: DrawingSet, group: string, index: number, from: Vector2, to: Vector2, thickness: number, colour: Color3) -> (),
    Box: (self: DrawingSet, rect: Rect, mode: string, thickness: number, colour: Color3) -> (),
    Fill: (self: DrawingSet, rect: Rect?, colour: Color3, transparency: number) -> (),
    Label: (self: DrawingSet, slot: string, text: string, at: Vector2, anchorY: number, size: number, colour: Color3, plate: boolean) -> (),
    HideLabel: (self: DrawingSet, slot: string) -> (),
    HideLines: (self: DrawingSet, group: string, first: number, last: number) -> (),
    Bar: (self: DrawingSet, at: Vector2, height: number, fraction: number) -> (),
    HideBar: (self: DrawingSet) -> (),
    Show: (self: DrawingSet, visible: boolean) -> (),
    Highlight: (self: DrawingSet, adornee: Instance?, colour: Color3, fill: number, mode: string) -> (),
    Destroy: (self: DrawingSet) -> (),
}

export type RenderLibrary = {
    Layer: (self: RenderLibrary, name: string) -> Frame,
    Set: (self: RenderLibrary, layer: Frame, key: any) -> DrawingSet,
    Release: (self: RenderLibrary, layer: Frame, key: any) -> (),
    ReleaseAll: (self: RenderLibrary, layer: Frame) -> (),
    Project: (self: RenderLibrary, camera: Camera, position: Vector3) -> Vector2?,
    ModelRect: (self: RenderLibrary, camera: Camera, model: Model) -> Rect?,
    Destroy: (self: RenderLibrary) -> (),
}

local Module = {
    Name = "Render",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local activeLibrary: RenderLibrary? = nil

function Module.init(context: any): RenderLibrary
    local host: any = context
    local create: any = host.create
    local screenGui: Instance = host.ScreenGui

    local library: any = {}

    local pools: any = setmetatable({}, {__mode = "k"})

    function library:Layer(name: string): Frame
        local existing: Instance? = screenGui:FindFirstChild(name)
        if existing and existing:IsA("Frame") then
            return existing :: Frame
        end
        local layer: Frame = create("Frame", {
            Parent = screenGui,
            Name = name,
            BackgroundTransparency = 1,
            BorderSizePixel = 0,
            Size = UDim2.fromScale(1, 1),
            Visible = false,

            ZIndex = 0,
        }) :: Frame
        pools[layer] = {}
        return layer
    end

    local function newSet(layer: Frame, key: any): DrawingSet
        local root: Frame = create("Frame", {
            Parent = layer,
            Name = typeof(key) == "Instance" and (key :: Instance).Name or tostring(key),
            BackgroundTransparency = 1,
            BorderSizePixel = 0,
            Size = UDim2.fromScale(1, 1),
            ZIndex = 0,
        }) :: Frame

        local set: any = {
            root = root,
            lines = {} :: {[string]: {Frame}},
            labels = {} :: {[string]: TextLabel},
            fill = nil :: Frame?,
            highlight = nil :: Highlight?,
        }

        local function line(group: string, index: number): Frame
            local bucket: {Frame} = set.lines[group]
            if not bucket then
                bucket = {}
                set.lines[group] = bucket
            end
            local existing: Frame? = bucket[index]
            if existing then
                return existing :: Frame
            end
            local frame: Frame = create("Frame", {
                Parent = root,

                Name = group .. tostring(index),
                AnchorPoint = Vector2.new(0.5, 0.5),
                BorderSizePixel = 0,
                Size = UDim2.fromOffset(0, 1),
                Visible = false,
                ZIndex = 1,
            }) :: Frame
            bucket[index] = frame
            return frame
        end

        function set:Line(
            group: string,
            index: number,
            from: Vector2,
            to: Vector2,
            thickness: number,
            colour: Color3
        ): ()
            local frame: Frame = line(group, index)
            local delta: Vector2 = to - from
            local length: number = delta.Magnitude
            if length < 0.5 then
                frame.Visible = false
                return
            end
            frame.Visible = true
            frame.BackgroundColor3 = colour
            frame.Position = UDim2.fromOffset(
                math.round((from.X + to.X) * 0.5),
                math.round((from.Y + to.Y) * 0.5)
            )
            frame.Size = UDim2.fromOffset(math.round(length), thickness)
            frame.Rotation = math.deg(math.atan2(delta.Y, delta.X))
        end

        function set:HideLines(group: string, first: number, last: number): ()
            local bucket: {Frame}? = set.lines[group]
            if not bucket then
                return
            end
            for index: number = first, last do
                local frame: Frame? = (bucket :: {Frame})[index]
                if frame then
                    (frame :: Frame).Visible = false
                end
            end
        end

        function set:Box(
            rect: Rect,
            mode: string,
            thickness: number,
            colour: Color3
        ): ()
            if mode == "Off" or rect.width <= 0 or rect.height <= 0 then
                set:HideLines("Box", 1, 8)
                set:HideLines("BoxOutline", 1, 8)
                return
            end
            local function edge(index, from, to)
                set:Line("BoxOutline", index, from, to, thickness + 2, Color3.fromRGB(8,10,15))
                set:Line("Box", index, from, to, thickness, colour)
                set.lines.BoxOutline[index].ZIndex = 1
                set.lines.Box[index].ZIndex = 2
            end
            local corners: {Vector2} = {
                Vector2.new(rect.left, rect.top),
                Vector2.new(rect.right, rect.top),
                Vector2.new(rect.right, rect.bottom),
                Vector2.new(rect.left, rect.bottom),
            }
            if mode == "Full" then
                for index: number = 1, 4 do
                    edge(index, corners[index], corners[index % 4 + 1])
                end
                set:HideLines("Box", 5, 8)
                set:HideLines("BoxOutline", 5, 8)
                return
            end
            local segment: number =
                math.min(math.max(math.min(rect.width, rect.height) * 0.28, 2), rect.width * 0.5, rect.height * 0.5, 18)
            local index: number = 1
            for corner: number = 1, 4 do
                local from: Vector2 = corners[corner]
                local nextCorner: Vector2 = corners[corner % 4 + 1]
                local previousCorner: Vector2 = corners[(corner + 2) % 4 + 1]
                edge(index, from, from + (nextCorner - from).Unit * segment)
                edge(index + 1, from, from + (previousCorner - from).Unit * segment)
                index += 2
            end
        end

        function set:Fill(rect: Rect?, colour: Color3, transparency: number): ()
            if not rect and not set.fill then return end
            if not set.fill then
                set.fill = create("Frame", {
                    Parent = root,
                    Name = "BoxFill",
                    BackgroundColor3 = colour,
                    BackgroundTransparency = transparency,
                    BorderSizePixel = 0,
                    Visible = false,
                    ZIndex = 0,
                }) :: Frame
            end
            local frame: Frame = set.fill :: Frame
            if not rect then
                frame.Visible = false
                return
            end
            local resolved: Rect = rect :: Rect
            frame.Visible = true
            frame.BackgroundColor3 = colour
            frame.BackgroundTransparency = transparency
            frame.Position =
                UDim2.fromOffset(math.round(resolved.left), math.round(resolved.top))
            frame.Size =
                UDim2.fromOffset(math.round(resolved.width), math.round(resolved.height))
        end

        function set:Label(
            slot: string,
            text: string,
            at: Vector2,
            anchorY: number,
            size: number,
            colour: Color3,
            plate: boolean
        ): ()
            local label: TextLabel? = set.labels[slot]
            if not label then
                label = create("TextLabel", {
                    Parent = root,
                    Name = slot,
                    AnchorPoint = Vector2.new(0.5, anchorY),
                    AutomaticSize = Enum.AutomaticSize.X,
                    BackgroundColor3 = Color3.fromRGB(0, 0, 0),
                    BackgroundTransparency = 1,
                    BorderSizePixel = 0,
                    Text = "",
                    Font = Enum.Font.Arial,
                    RichText = false,
                    TextStrokeColor3 = Color3.fromRGB(0,0,0),
                    TextStrokeTransparency = 1,
                    TextColor3 = Color3.fromRGB(236, 236, 240),
                    TextSize = size,
                    -- v139.1: the height is set once here (and when TextSize
                    -- changes), never on every frame: a Size write makes the
                    -- UI re-layout, and this ran for every label of every ESP.
                    Size = UDim2.fromOffset(0, size + 6),
                    TextTruncate = Enum.TextTruncate.None,
                    TextXAlignment = Enum.TextXAlignment.Center,
                    TextYAlignment = Enum.TextYAlignment.Center,
                    Visible = false,
                    ZIndex = 2,
                }) :: TextLabel
                create("UIPadding", {
                    Parent = label,
                    PaddingLeft = UDim.new(0, 4),
                    PaddingRight = UDim.new(0, 4),
                })
                set.labels[slot] = label
            end
            local resolved: TextLabel = label :: TextLabel
            resolved.Visible = true
            if resolved.Text ~= text then resolved.Text = text end
            if resolved.TextColor3 ~= colour then resolved.TextColor3 = colour end
            if resolved.TextSize ~= size then
                resolved.TextSize = size
                resolved.Size = UDim2.fromOffset(resolved.Size.X.Offset, size + 6)
            end
            local plateTransparency: number = plate and 0.5 or 1
            if resolved.BackgroundTransparency ~= plateTransparency then
                resolved.BackgroundTransparency = plateTransparency
            end
            local x: number = math.round(at.X)
            local y: number = math.round(at.Y)
            local position: UDim2 = resolved.Position
            if position.X.Offset ~= x or position.Y.Offset ~= y then
                resolved.Position = UDim2.fromOffset(x, y)
            end
        end

        function set:HideLabel(slot: string): ()
            local label: TextLabel? = set.labels[slot]
            if label then
                (label :: TextLabel).Visible = false
            end
        end

        function set:Bar(at: Vector2, height: number, fraction: number): ()
            if not set.barTrack then
                set.barTrack = create("Frame", {
                    Parent = root,
                    Name = "HealthTrack",
                    AnchorPoint = Vector2.new(1, 0),
                    BackgroundColor3 = Color3.fromRGB(0, 0, 0),
                    BackgroundTransparency = 0.35,
                    BorderSizePixel = 0,
                    Visible = false,
                    ZIndex = 1,
                }) :: Frame
                set.barFill = create("Frame", {
                    Parent = set.barTrack,
                    Name = "HealthFill",
                    AnchorPoint = Vector2.new(0, 1),
                    BackgroundColor3 = Color3.fromRGB(120, 220, 140),
                    BorderSizePixel = 0,
                    Position = UDim2.new(0, 1, 1, 0),
                    Size = UDim2.new(0, 1, 1, 0),
                    ZIndex = 2,
                }) :: Frame
            end
            local track: Frame = set.barTrack :: Frame
            local fill: Frame = set.barFill :: Frame
            local clamped: number = fraction == fraction and math.clamp(fraction, 0, 1) or 0
            track.Visible = true
            track.Position = UDim2.fromOffset(math.round(at.X), math.round(at.Y))
            track.Size = UDim2.fromOffset(3, math.max(1, math.round(height)))
            fill.Size = UDim2.new(0, 1, clamped, 0)
            local low, mid, high = Color3.fromRGB(255,70,70), Color3.fromRGB(255,220,60), Color3.fromRGB(85,255,85)
            fill.BackgroundColor3 = clamped < 0.5 and low:Lerp(mid, clamped * 2) or mid:Lerp(high, (clamped - 0.5) * 2)
        end

        function set:HideBar(): ()
            if set.barTrack then
                (set.barTrack :: Frame).Visible = false
            end
        end

        function set:Show(visible: boolean): ()
            root.Visible = visible
            if set.highlight then
                (set.highlight :: Highlight).Enabled = visible
            end
        end

        function set:Highlight(
            adornee: Instance?,
            colour: Color3,
            fill: number,
            mode: string
        ): ()
            if not adornee or mode == "Off" then
                if set.highlight then
                    (set.highlight :: Highlight):Destroy()
                    set.highlight = nil
                end
                return
            end
            local highlight: Highlight? = set.highlight
            if not highlight or not (highlight :: Highlight).Parent then
                highlight = create("Highlight", {
                    Parent = adornee,
                    Name = "Wurst_Chams",
                    DepthMode = Enum.HighlightDepthMode.AlwaysOnTop,
                }) :: Highlight
                set.highlight = highlight
            end
            local resolved: Highlight = highlight :: Highlight
            resolved.Adornee = adornee
            resolved.Parent = adornee
            resolved.Enabled = root.Visible
            resolved.DepthMode = mode == "Occluded"
                and Enum.HighlightDepthMode.Occluded
                or Enum.HighlightDepthMode.AlwaysOnTop
            resolved.FillColor = colour
            resolved.OutlineColor = colour
            resolved.FillTransparency = mode == "Outline" and 1 or fill
            resolved.OutlineTransparency = 0
        end

        function set:Destroy(): ()
            if set.highlight then
                pcall(function(): ()
                    (set.highlight :: Highlight):Destroy()
                end)
            end
            pcall(function(): ()
                root:Destroy()
            end)
        end

        return (set :: any) :: DrawingSet
    end

    function library:Set(layer: Frame, key: any): DrawingSet
        local pool: any = pools[layer]
        if not pool then
            pool = {}
            pools[layer] = pool
        end
        local existing: any = pool[key]
        if existing then
            return existing
        end
        local set: DrawingSet = newSet(layer, key)
        pool[key] = set
        return set
    end

    function library:Release(layer: Frame, key: any): ()
        local pool: any = pools[layer]
        local set: any = pool and pool[key]
        if not set then
            return
        end
        pool[key] = nil
        set:Destroy()
    end

    function library:ReleaseAll(layer: Frame): ()
        local pool: any = pools[layer]
        if not pool then
            return
        end
        for key: any, set: any in pairs(pool) do
            pool[key] = nil
            set:Destroy()
        end
    end

    function library:Project(camera: Camera, position: Vector3): Vector2?

        local point: Vector3 = camera:WorldToViewportPoint(position)
        if point.Z <= 0 then
            return nil
        end
        return Vector2.new(point.X, point.Y)
    end

    local function rectFromPoints(
        camera: Camera,
        viewport: Vector2,
        points: {Vector3},
        minimumAspect: number?
    ): Rect?
        local minimumX: number, minimumY: number = math.huge, math.huge
        local maximumX: number, maximumY: number = -math.huge, -math.huge
        local inFront: number = 0
        for _, position: Vector3 in ipairs(points) do
            local point: Vector3 = camera:WorldToViewportPoint(position)
            if point.Z > 0 then
                inFront += 1
                if point.X < minimumX then minimumX = point.X end
                if point.Y < minimumY then minimumY = point.Y end
                if point.X > maximumX then maximumX = point.X end
                if point.Y > maximumY then maximumY = point.Y end
            end
        end
        if inFront < 2 then
            return nil
        end
        minimumX -= 2
        maximumX += 2
        minimumY -= 2
        maximumY += 2
        local width: number = maximumX - minimumX
        local height: number = maximumY - minimumY
        if minimumAspect and height > 0 and height * minimumAspect > width then
            width = height * minimumAspect
            local centreX: number = (minimumX + maximumX) * 0.5
            minimumX = centreX - width * 0.5
            maximumX = centreX + width * 0.5
        end
        if maximumX < 0 or minimumX > viewport.X
            or maximumY < 0 or minimumY > viewport.Y then
            return nil
        end
        local left: number = math.max(minimumX, 0)
        local right: number = math.min(maximumX, viewport.X)
        local top: number = math.max(minimumY, 0)
        local bottom: number = math.min(maximumY, viewport.Y)
        if right - left < 1 or bottom - top < 1 then
            return nil
        end
        return {
            left = left,
            right = right,
            top = top,
            bottom = bottom,
            width = right - left,
            height = bottom - top,
            centreX = (left + right) * 0.5,
            centreY = (top + bottom) * 0.5,
        }
    end

    function library:ModelRect(camera: Camera, model: Model): Rect?
        if not model.Parent then
            return nil
        end
        local viewport: Vector2 = camera.ViewportSize
        local humanoid: Humanoid? = model:FindFirstChildOfClass("Humanoid") :: Humanoid?
        local root: BasePart? = model:FindFirstChild("HumanoidRootPart") :: BasePart?
            or model.PrimaryPart
        if humanoid and root then

            local head: BasePart? = model:FindFirstChild("Head") :: BasePart?
            local rootPosition: Vector3 = root.Position
            local topPosition: Vector3 = head
                and (head.Position + Vector3.new(0, head.Size.Y * 0.5 + 0.1, 0))
                or (rootPosition + Vector3.new(0, 2.3, 0))

            local leftFoot = model:FindFirstChild("LeftFoot") or model:FindFirstChild("Left Leg")
            local rightFoot = model:FindFirstChild("RightFoot") or model:FindFirstChild("Right Leg")
            local lowestY: number = rootPosition.Y - math.max(humanoid.HipHeight + root.Size.Y * 0.5, 2.9)
            if leftFoot and leftFoot:IsA("BasePart") then
                lowestY = math.min(lowestY, leftFoot.Position.Y - leftFoot.Size.Y * 0.5)
            end
            if rightFoot and rightFoot:IsA("BasePart") then
                lowestY = math.min(lowestY, rightFoot.Position.Y - rightFoot.Size.Y * 0.5)
            end
            local bottomPosition: Vector3 = Vector3.new(rootPosition.X, lowestY, rootPosition.Z)

            local rootCFrame: CFrame = root.CFrame
            return rectFromPoints(camera, viewport, {
                topPosition,
                bottomPosition,
                rootPosition,
                rootCFrame * Vector3.new(1.35, 0, 0),
                rootCFrame * Vector3.new(-1.35, 0, 0),
                rootCFrame * Vector3.new(0, 0, 0.75),
                rootCFrame * Vector3.new(0, 0, -0.75),
            }, 0.38)
        end

        local boundsOk: boolean, boundsCFrame: any, boundsSize: any =
            pcall(model.GetBoundingBox, model)
        if not boundsOk or typeof(boundsSize) ~= "Vector3" or boundsSize.Magnitude <= 0 then
            return nil
        end
        local half: Vector3 = boundsSize * 0.5
        return rectFromPoints(camera, viewport, {
            boundsCFrame * Vector3.new(half.X, half.Y, half.Z),
            boundsCFrame * Vector3.new(-half.X, half.Y, half.Z),
            boundsCFrame * Vector3.new(half.X, -half.Y, half.Z),
            boundsCFrame * Vector3.new(-half.X, -half.Y, half.Z),
            boundsCFrame * Vector3.new(half.X, half.Y, -half.Z),
            boundsCFrame * Vector3.new(-half.X, half.Y, -half.Z),
            boundsCFrame * Vector3.new(half.X, -half.Y, -half.Z),
            boundsCFrame * Vector3.new(-half.X, -half.Y, -half.Z),
        })
    end

    function library:Destroy(): ()
        for layer: any, pool: any in pairs(pools) do
            for key: any, set: any in pairs(pool) do
                pool[key] = nil
                pcall(function(): ()
                    set:Destroy()
                end)
            end
            pcall(function(): ()
                (layer :: Frame):Destroy()
            end)
        end
        if activeLibrary == library then
            activeLibrary = nil
        end
    end

    activeLibrary = (library :: any) :: RenderLibrary
    Module.Initialized = true
    return (library :: any) :: RenderLibrary
end

function Module.destroy(): ()
    local library: RenderLibrary? = activeLibrary
    if library then
        library:Destroy()
    end
    activeLibrary = nil
    Module.Initialized = false
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/libraries/Render.lua
--------------------------------------------------------------------------------
--------------------------------------------------------------------------------
ARCHIVO: src/libraries/Targeting.lua
BYTES: 8883
SHA-256: 17822b94fd697c7b34db20745751857 6b08d2e9189858b573e12059cc969e84
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
export type Query = {
    FOV: number?,
    MaxDistance: number?,
    TargetPart: string?,
    Prediction: number?,
    TeamCheck: boolean?,
    VisibilityCheck: boolean?,
    Priority: string?,
    IncludeNPCs: boolean?,
    AllowOffscreen: boolean?,
}

export type Target = {
    player: Player?,
    entity: any,
    character: Model,
    part: BasePart,
    position: Vector3,
    screenPosition: Vector2,
    distance: number,
    cursorDistance: number,
}

export type Library = {
    List: (self: Library, entities: any, query: Query?) -> {Target},
    Select: (self: Library, entities: any, query: Query?) -> Target?,
    Destroy: (self: Library) -> (),
}

local Module = {
    Name = "Targeting",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local activeLibrary: Library? = nil

local function asBasePart(instance: Instance?): BasePart?
    if instance and instance:IsA("BasePart") then
        return instance :: BasePart
    end
    return nil
end

function Module.init(context: any): Library
    local host: any = context.host or context
    local userInput: UserInputService? = host.UserInputService
    local currentWorkspace: Workspace = host.workspace or workspace
    local localPlayer: Player? = host.LocalPlayer
    local services: any = context.services or host.services

    local library: any = {}

    local function camera(): Camera?
        return currentWorkspace.CurrentCamera
    end

    local function cursor(cameraObject: Camera): Vector2
        if userInput and userInput.MouseEnabled then
            local ok: boolean, location: any = pcall(
                (userInput :: UserInputService).GetMouseLocation,
                userInput
            )
            if ok and typeof(location) == "Vector2" then
                return location :: Vector2
            end
        end
        local viewport: Vector2 = cameraObject.ViewportSize
        return Vector2.new(viewport.X / 2, viewport.Y / 2)
    end
```

```
    local function targetParts(entity: any, requested: string?): {BasePart}
        local character: Model = entity.Character
        local wanted: string = requested or "Closest"
        if wanted == "Head" then
            return {
                asBasePart(character:FindFirstChild("Head")) or entity.RootPart,
            }
        end
        if wanted == "Torso" then
            return {
                asBasePart(character:FindFirstChild("UpperTorso"))
                    or asBasePart(character:FindFirstChild("Torso"))
                    or entity.RootPart,
            }
        end
        if wanted ~= "Closest" then
            return {entity.RootPart}
        end
        local head: BasePart? = asBasePart(character:FindFirstChild("Head"))
        if head then
            return {head, entity.RootPart}
        end
        return {entity.RootPart}
    end

    local function protected(player: Player?): boolean
        if not player then
            return false
        end
        local policy: any = services and services.protectedTargets
        if policy and type(policy.isProtected) == "function" then
            return policy.isProtected(player) == true
        end
        return false
    end

    local function visibleFrom(
        cameraObject: Camera,
        entity: any,
        position: Vector3
    ): boolean
        local origin: Vector3 = cameraObject.CFrame.Position
        local parameters: RaycastParams = RaycastParams.new()
        parameters.FilterType = Enum.RaycastFilterType.Exclude
        local ignored: {Instance} = {entity.Character, cameraObject}
        if localPlayer and localPlayer.Character then
            table.insert(ignored, localPlayer.Character)
        end
        parameters.FilterDescendantsInstances = ignored
        parameters.IgnoreWater = true
        local ok: boolean, result: any = pcall(
            currentWorkspace.Raycast,
            currentWorkspace,
            origin,
            position - origin,
            parameters
        )
        return ok and result == nil
    end

    local function candidateForPart(
        cameraObject: Camera,
        pointer: Vector2,
        entity: any,
        part: BasePart,
        prediction: number,
        query: Query
    ): Target?
        local velocity: Vector3 = part.AssemblyLinearVelocity or Vector3.zero
        local predicted: Vector3 = part.Position + velocity * prediction
        local projected: Vector3, onScreen: boolean =
            cameraObject:WorldToViewportPoint(predicted)
        if (not onScreen or projected.Z <= 0) and not query.AllowOffscreen then
            return nil
        end
        local screenPosition: Vector2 = Vector2.new(projected.X, projected.Y)
        local cursorDistance: number = onScreen
                and (screenPosition - pointer).Magnitude
            or math.huge
        if query.FOV ~= nil and cursorDistance > (query.FOV :: number) then
            return nil
        end
        if query.VisibilityCheck == true
            and not visibleFrom(cameraObject, entity, predicted) then
            return nil
        end
        return {
            player = entity.Player,
            entity = entity,
            character = entity.Character,
            part = part,
            position = predicted,
            screenPosition = screenPosition,
            distance = entity.Distance,
            cursorDistance = cursorDistance,
        }
    end

    function library:List(entities: any, query: Query?): {Target}
        local options: Query = query or {}
        local cameraObject: Camera? = camera()
        if not cameraObject or not entities then
            return {}
        end
        if type(entities.Refresh) == "function" then
            entities:Refresh()
        end
        local pointer: Vector2 = cursor(cameraObject :: Camera)
        local found: {Target} = {}
        local maxDistance: number = options.MaxDistance or math.huge
        local prediction: number = math.max(options.Prediction or 0, 0)

        local function consider(entity: any): ()
            if entity.Distance > maxDistance then
                return
            end
            if options.TeamCheck ~= false then
                local hostile: boolean? = nil
                if entity.Player and type(entities.IsEnemy) == "function" then
                    hostile = entities:IsEnemy(entity.Player)
                end
                if hostile == false or hostile == nil and entity.IsFriendly then
                    return
                end
            end
            if protected(entity.Player) then
                return
            end

            local bestForEntity: Target? = nil
            for _, part: BasePart in ipairs(targetParts(entity, options.TargetPart)) do
                local candidate: Target? = candidateForPart(
                    cameraObject :: Camera,
                    pointer,
                    entity,
                    part,
                    prediction,
                    options
                )
                if candidate
                    and (not bestForEntity
                        or candidate.cursorDistance < (bestForEntity :: Target).cursorDistance) then
                    bestForEntity = candidate
                end
            end
            if bestForEntity then
                table.insert(found, bestForEntity :: Target)
            end
        end
        for _, entity: any in ipairs(entities.List or {}) do
            consider(entity)
        end
        if options.IncludeNPCs then
            for _, entity: any in ipairs(entities.NPCList or {}) do
                consider(entity)
            end
        end

        local priority: string = options.Priority or "Cursor"
        table.sort(found, function(first: Target, second: Target): boolean
            if priority == "Distance" then
                if first.distance == second.distance then
                    return first.cursorDistance < second.cursorDistance
                end
                return first.distance < second.distance
            end
            if first.cursorDistance == second.cursorDistance then
                return first.distance < second.distance
            end
            return first.cursorDistance < second.cursorDistance
        end)
        return found
    end

    function library:Select(entities: any, query: Query?): Target?
        return self:List(entities, query)[1]
    end

    function library:Destroy(): ()
        if activeLibrary == library then
            activeLibrary = nil
        end
    end

    activeLibrary = (library :: any) :: Library
    Module.Initialized = true
    return (library :: any) :: Library
end

function Module.destroy(): ()
    local library: Library? = activeLibrary
    if library then
        library:Destroy()
    end
    activeLibrary = nil
    Module.Initialized = false
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/libraries/Targeting.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/libraries/Weapons.lua
BYTES: 15670
SHA-256: 3a4d59ac97449237c66eda8a1817654bdb08d9878634235845 43d3f6cff8558d
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
export type Candidate = {
    id: string,
    label: string,
    kind: string,
    melee: boolean,
    instance: Instance,
}

export type GameSource = {

    scan: () -> {{string}},

    press: (label: string, target: any?) -> boolean,
}

export type WeaponLibrary = {
    Scan: (self: WeaponLibrary, query: any?) -> {Candidate},
    Labels: (self: WeaponLibrary, query: any?) -> {string},
    Find: (self: WeaponLibrary, label: string) -> Candidate?,
    IsMelee: (self: WeaponLibrary, name: string) -> boolean,
    Activate: (self: WeaponLibrary, candidate: Candidate, target: any?) -> boolean,
    Best: (self: WeaponLibrary, query: any?) -> Candidate?,
    Invalidate: (self: WeaponLibrary) -> (),
    SetAllowEquip: (self: WeaponLibrary, allowed: boolean) -> (),
    Swing: (self: WeaponLibrary) -> boolean,
    RegisterGameSource: (self: WeaponLibrary, source: GameSource?) -> (),
    PressButton: (self: WeaponLibrary, button: GuiButton) -> boolean,
    Destroy: (self: WeaponLibrary) -> (),
}

local Module = {
    Name = "Weapons",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local WEAPON_WORDS: {string} = {
    "sword",
    "katana",
    "dagger",
    "blade",
    "knife",
    "axe",
    "mace",
    "spear",
    "scythe",
    "sabre",
    "saber",
    "machete",
    "hammer",
    "club",
    "bat",
    "staff",
    "pickaxe",
    "fist",
    "punch",
    "attack",
    "swing",
    "hit",
    "melee",
    "weapon",
    "combat",
}

local MELEE_WORDS: {string} = {
    "sword",
    "katana",
    "dagger",
    "blade",
    "knife",
    "axe",
    "mace",
    "spear",
    "scythe",
    "sickle",
    "sabre",
    "saber",
    "rapier",
    "cutlass",
    "machete",
    "glaive",
    "halberd",
    "hammer",
    "club",
    "baton",
    "bat",
    "pickaxe",
    "fist",
    "knuckle",
}

local RANGED_WORDS: {string} = {
    "bow",
    "crossbow",
    "arrow",
    "gun",
    "rifle",
    "pistol",
    "revolver",
    "shotgun",
    "sniper",
    "launcher",
    "rocket",
    "grenade",
    "snowball",
    "projectile",
    "throw",
    "wand",
    "staff",
    "gauntlet",
    "fireball",
}

local RIG_WORDS: {string} = {
    "hand",
    "arm",
    "leg",
    "foot",
    "torso",
    "head",
    "upper",
    "lower",
    "root",
    "limb",
    "joint",
    "motor",
    "weld",
    "attachment",
    "rig",
    "bone",
    "hitbox",
    "collider",
    "humanoid",
    "camera",
    "viewmodel",
    "idle",
    "walk",
    "run",
    "sprint",
    "sneak",
    "crouch",
    "jump",
    "fall",
    "land",
    "swim",
    "pose",
    "track",
    "sequence",
}

local NOT_WEAPON_WORDS: {string} = {
    "emote",
    "animation",
    "anim",
    "dance",
    "cosmetic",
    "cape",
    "skin",
    "kit",
    "shop",
    "setting",
    "gift",
    "trade",
    "coin",
    "gem",
    "wool",
    "block",
    "plank",
    "brick",
    "glass",
    "ladder",
    "wood ",
    "sand",
    "dirt",
    "obsidian",
    "potion",
    "food",
    "apple",
    "bed",
    "chest",
    "music",
    "sound",
    "menu",
    "close",
    "back",
    "shopkeeper",
}

local INVENTORY_WORDS: {string} = {
    "hotbar",
    "backpack",
    "inventory",
    "toolbar",
    "slots",
    "items",
}

local TOUCH_WORDS: {string} = {
    "mobile",
    "touch",
    "buttons",
    "controls",
}

local activeLibrary: WeaponLibrary? = nil

local function containsWord(text: string, words: {string}): boolean
    local lowered: string = string.lower(text)
    for _, word: string in ipairs(words) do
        if string.find(lowered, word, 1, true) then
            return true
        end
    end
    return false
end

local function isMeleeName(name: string): boolean
    if containsWord(name, NOT_WEAPON_WORDS)
        or containsWord(name, RANGED_WORDS)
        or containsWord(name, RIG_WORDS) then
        return false
    end
    return containsWord(name, MELEE_WORDS)
end

local function ancestryMatches(instance: Instance, words: {string}, depth: number): boolean
    local current: Instance? = instance
    local steps: number = 0
    while current and steps < depth do
        if containsWord(current.Name, words) then
            return true
        end
        current = current.Parent
        steps += 1
    end
    return false
end

function Module.init(context: any): WeaponLibrary
    local host: any = context
    local localPlayer: Player = host.LocalPlayer
    local currentWorkspace: Workspace = host.workspace or workspace
    local menuGui: Instance? = host.ScreenGui

    local library: any = {

        cache = {},
        cachedAt = -1,
        cachedAll = false,
        cachedMelee = false,

        allowEquip = false,
    }

    function library:PressButton(button: GuiButton): boolean
        local environment: any = getfenv()
        local connectionsOf: any = environment.getconnections
        local fireSignal: any = environment.firesignal
        local pressed: boolean = false

        if type(connectionsOf) == "function" then
            for _, signal: any in
                ipairs({
                    button.MouseButton1Down,
                    button.MouseButton1Click,
                    (button :: any).Activated,
                    button.MouseButtonUp,
                })
            do
                local ok: boolean, connections: any = pcall(connectionsOf, signal)
                if not ok or type(connections) ~= "table" then
                    continue
                end
                for _, connection: any in ipairs(connections) do
                    local fired: boolean = pcall(function(): ()
                        connection:Fire()
                    end)
                    pressed = pressed or fired
                end
            end
        end
        if not pressed and type(fireSignal) == "function" then
            pressed = pcall(fireSignal, (button :: any).Activated) == true
        end
        return pressed
    end

    local function isMenuOwned(instance: Instance): boolean
        return menuGui ~= nil and instance:IsDescendantOf(menuGui :: Instance)
    end

    function library:Invalidate(): ()
        library.cachedAt = -1
    end

    function library:RegisterGameSource(source: any): ()
        library.gameSource = source
        library.cachedAt = -1
    end

    function library:SetAllowEquip(allowed: boolean): ()
        library.allowEquip = allowed == true
    end

    function library:Scan(query: any?): {Candidate}
        local options: any = query or {}
        local everything: boolean = options.All == true
        local now: number = os.clock()
        if now - library.cachedAt < 1
            and library.cachedAll == everything
            and library.cachedMelee == (options.Melee == true) then
            return library.cache
        end
        local found: {Candidate} = {}
        local seen: {[string]: boolean} = {}

        local meleeOnly: boolean = options.Melee == true
        local function add(kind: string, label: string, instance: Instance): ()
            local id: string = kind .. ":" .. label
            if seen[id] then
                return
            end
            local melee: boolean = isMeleeName(label)
            if meleeOnly and not melee then
                return
            end
            seen[id] = true
            table.insert(found, {
                id = id,
                label = label,
                kind = kind,
                melee = melee,
                instance = instance,
            })
        end

        if library.gameSource and type(library.gameSource.scan) == "function" then
            local ok: boolean, listed: any = pcall(library.gameSource.scan)
            if ok and type(listed) == "table" then
                for _, entry: any in ipairs(listed) do
                    if type(entry) == "table" and entry[1] and entry[2] then
                        add("Game", tostring(entry[1]), entry[2])
                    end
                end
            end
        end

        local character: Model? = localPlayer.Character
        if character then
            for _, child: Instance in ipairs(character:GetChildren()) do
                if child:IsA("Tool") then
                    add("Tool", child.Name, child)
                end
            end
        end
        local backpack: Instance? = localPlayer:FindFirstChild("Backpack")
        if backpack then
            for _, child: Instance in ipairs(backpack:GetChildren()) do
                if child:IsA("Tool") then
                    add("Tool", child.Name, child)
                end
            end
        end

        local viewRoots: {Instance} = {}
        local camera: Camera? = currentWorkspace.CurrentCamera
        if camera then
            table.insert(viewRoots, camera)
        end
        if character then
            table.insert(viewRoots, character)
        end
        for _, root: Instance in ipairs(viewRoots) do
            for _, child: Instance in ipairs(root:GetChildren()) do
                if not child:IsA("Model") then
                    continue
                end
                if containsWord(child.Name, WEAPON_WORDS)
                    and not containsWord(child.Name, NOT_WEAPON_WORDS) then
                    add("ViewModel", child.Name, child)
                end
                if string.find(string.lower(child.Name), "viewmodel", 1, true) then

                    for _, held: Instance in ipairs(child:GetChildren()) do
                        if not held:IsA("Model") and not held:IsA("BasePart") then
                            continue
                        end
                        if not isMeleeName(held.Name) then
                            continue
                        end
                        add("ViewModel", held.Name, held)
                    end
                end
            end
        end

        local playerGui: Instance? = localPlayer:FindFirstChild("PlayerGui")
        if playerGui then
            for _, descendant: Instance in ipairs(playerGui:GetDescendants()) do
                if not descendant:IsA("GuiButton") then
                    continue
                end
                if isMenuOwned(descendant) then

                    continue
                end
                if containsWord(descendant.Name, NOT_WEAPON_WORDS) then
                    continue
                end
                local named: boolean = containsWord(descendant.Name, WEAPON_WORDS)
                if not named then

                    if not everything then
                        continue
                    end
                    if not ancestryMatches(descendant, TOUCH_WORDS, 4) then
                        continue
                    end
                    add("Button", descendant.Name, descendant)
                    continue
                end
                local inInventory: boolean =
                    ancestryMatches(descendant, INVENTORY_WORDS, 4)
                add(inInventory and "Slot" or "Button", descendant.Name, descendant)
            end
        end

        library.cache = found
        library.cachedAt = now
        library.cachedAll = everything
        library.cachedMelee = meleeOnly
        return found
    end

    function library:Labels(query: any?): {string}
        local labels: {string} = {}
        for _, candidate: Candidate in ipairs(library:Scan(query)) do
            table.insert(labels, candidate.label .. "  ·  " .. candidate.kind)
        end
        table.sort(labels)
        return labels
    end

    function library:IsMelee(name: string): boolean
        return isMeleeName(name)
    end

    function library:Find(label: string): Candidate?
        local wanted: string = string.lower(label)
        for _, candidate: Candidate in ipairs(library:Scan({All = true})) do
            if string.lower(candidate.label) == wanted then
                return candidate
```

```
            end
            if string.lower(candidate.label .. "  ·  " .. candidate.kind) == wanted then
                return candidate
            end
        end
        return nil
    end

    function library:Best(query: any?): Candidate?
        local found: {Candidate} = library:Scan(query or {All = false})

        local order: {string} = {"Game", "Tool", "Button", "Slot", "ViewModel"}
        local character: Model? = localPlayer.Character
        for _, kind: string in ipairs(order) do
            for _, candidate: Candidate in ipairs(found) do
                if candidate.kind ~= kind then
                    continue
                end
                if kind == "Tool"
                    and character
                    and candidate.instance.Parent ~= character
                    and not library.allowEquip then

                    continue
                end
                return candidate
            end
        end
        -- Do not return a backpack tool that was explicitly skipped above.
        return nil
    end

    function library:Swing(): boolean

        local candidate: Candidate? = library:Best({Melee = true})
            or library:Best()
        if not candidate then
            return false
        end
        return library:Activate(candidate :: Candidate)
    end

    function library:Activate(candidate: Candidate, target: any?): boolean
        local instance: Instance = candidate.instance
        if candidate.kind == "Game" then

            if not library.gameSource or type(library.gameSource.press) ~= "function" then
                return false
            end
            local ok: boolean, pressed: any =
                pcall(library.gameSource.press, candidate.label, target)
            return ok and pressed == true
        end
        if not instance.Parent then
            return false
        end
        if candidate.kind == "Tool" then
            local tool: Tool = instance :: Tool

            if library.allowEquip and tool.Parent ~= localPlayer.Character then
                local humanoid: Humanoid? = localPlayer.Character
                    and localPlayer.Character:FindFirstChildOfClass("Humanoid") :: Humanoid?
                if humanoid then
                    pcall(function(): ()
                        (humanoid :: Humanoid):EquipTool(tool)
                    end)
                end
            end
            -- Tool:Activate on a disabled or unequipped tool can be a
            -- successful no-op; report failure so KillAura can try a held tool.
            if tool.Parent ~= localPlayer.Character or tool.Enabled == false then
                return false
            end
            return pcall(function(): ()
                tool:Activate()
            end)
        end
        if candidate.kind == "Button" or candidate.kind == "Slot" then
            return library:PressButton(instance :: GuiButton)
        end
        if candidate.kind == "ViewModel" then

            for _, other: Candidate in ipairs(library:Scan({All = true})) do
                if other.kind == "Button" then
                    return library:PressButton(other.instance :: GuiButton)
                end
            end
            return false
        end
        return false
    end

    function library:Destroy(): ()
        if activeLibrary == library then
            activeLibrary = nil
        end
    end

    activeLibrary = (library :: any) :: WeaponLibrary
    Module.Initialized = true
    return (library :: any) :: WeaponLibrary
end

function Module.destroy(): ()
    local library: WeaponLibrary? = activeLibrary
    if library then
        library:Destroy()
    end
    activeLibrary = nil
    Module.Initialized = false
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/libraries/Weapons.lua
--------------------------------------------------------------------------------

================================================================================
CODIGO COMPARTIDO DE INTERFAZ
================================================================================

--------------------------------------------------------------------------------
ARCHIVO: src/guis/Wurst/Code/Cards.lua
BYTES: 41045
SHA-256: 1ee0beb170c91c3bd9b1fce9ea2309256046891ccd672b062c60938beaf0c1ed
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
local Module = {
    Name = "Cards",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local moduleCleanup: () -> () = function(): () end

function Module.init(context: any): any
    local host: any = context
    local state: any = host.state
    local configData: any = host.configData
    local queueConfigSave: any = host.queueConfigSave
    local flushConfigSave: any = host.flushConfigSave or queueConfigSave
    local create: any = host.create
    local makeButton: any = host.makeButton
    local makeTextLabel: any = host.makeTextLabel
    local notify: any = host.notify
    local Theme: any = host.Theme
    local CONTROL_FONT: any = host.CONTROL_FONT
    local trackUiConnection: any = host.trackUiConnection
    local beginKeyCapture: any = host.beginKeyCapture
    local UserInputService: any = host.UserInputService
    local applyKeySlotStyle: any = host.applyKeySlotStyle
    local bindActionShortcut: any = host.bindActionShortcut
    local getFeatureCardParent: any = host.getFeatureCardParent

    local FEATURE_CATEGORIES: any = host.FEATURE_CATEGORIES
    local DEFAULT_FEATURE_CATEGORY: any = host.DEFAULT_FEATURE_CATEGORY
    local normalizeFeatureCategory: any = host.normalizeFeatureCategory

    local universalFeatures: any = host.universalFeatures
    local movementFeatures: any = host.movementFeatures
    local mm2Features: any = host.mm2Features
    local allFeatures: any = host.allFeatures
    local registeredSearchFeatures: {any} = {}

    local applyFeaturePresentation: any = host.applyFeaturePresentation
    local refreshFeatureToggle: any = host.refreshFeatureToggle
    local addFeatureTooltip: any = host.addFeatureTooltip

    local drawPixelIcon: any = host.drawPixelIcon
        or function(parent: Instance, _kind: string, _color: Color3?): Frame
            local existing: Instance? = (parent :: any):FindFirstChild("PixelIcon")
            if existing then
                existing:Destroy()
            end
            return create("Frame", {
                Parent = parent,
                Name = "PixelIcon",
                BackgroundTransparency = 1,
            }) :: Frame
        end

    -- v138.9: the cards inside each window are ALPHABETICAL. The random rank of
    -- v138.6-v138.8 made a module hard to find again; ClickGui.sortCategoryRows
    -- reads FeatureSortName (set below) and orders every window by it.
    local function createUniversalFeature(
        name: string,
        description: string,
        layoutOrder: number,
        onToggle: any,
        configuration: any?
    ): any
        configuration = configuration or {}
        -- No curated slot table any more (v138.9): the window sorts by name, so
        -- the per-card LayoutOrder is only the creation fallback.
        local registry: any = configuration.registry or universalFeatures
        local sectionName: string = configuration.section
            or (registry == mm2Features and "MM2")
            or (registry == state.trsFeatures and "TRS")
            or (registry == state.vdFeatures and "VD")
            or (registry == state.bedFightFeatures and "BedFight")
            or (registry == state.bedWarsFeatures and "BedWars")
            or (registry == state.mvsdFeatures and "MVSD")
            or (registry == movementFeatures and "Movement")
            or "Universal"
        local configKey: string = configuration.configKey
            or (sectionName .. "." .. name:gsub("%W", ""))

        local rawCategory: any = type(configuration.categoryName) == "string"
            and configuration.categoryName
            or FEATURE_CATEGORIES[name]
            or DEFAULT_FEATURE_CATEGORY
        local categoryName: string = type(normalizeFeatureCategory) == "function"
            and normalizeFeatureCategory(rawCategory)
            or tostring(rawCategory)

        local feature: any = {
            name = name,
            description = description,
            categoryName = categoryName,
            enabled = false,
            expanded = false,

            kind = configuration.kind
                or (configuration.action == true and "action")
                or (configuration.holdAction == true and "hold")
                or (configuration.category == true and "group")
                or "toggle",
            silentAction = configuration.silentAction == true,
            searchable = configuration.searchable ~= false,
            onHold = configuration.onHold,
            optionCount = 0,
            optionRows = {},
            collapsedHeight = state.layout.cardHeight,
            onToggle = onToggle,
            configKey = configKey,
            registry = registry,
            sectionName = sectionName,
            searchText = string.lower(
                name
                    .. " "
                    .. description
                    .. " "
                    .. sectionName
                    .. " "
                    .. categoryName
            ),
            tooltipText = description,
            tooltipBound = false,
        }

        feature.isAction = feature.kind == "action"
        feature.isCategory = feature.kind == "group"
        feature.isHoldAction = feature.kind == "hold"
        feature.isAlwaysOn = feature.kind == "hold" or feature.kind == "group"

        local requestedParent: Instance? =
            configuration.parent :: any
        local fallback: Instance = (host :: any).ScreenGui or (host :: any).PopupLayer or ({} :: any)
        local rowParent: Instance = getFeatureCardParent(requestedParent or fallback)
        if type(rowParent) ~= "table" or (rowParent :: any).Children == nil then
            rowParent = fallback
        end
        local row: Frame = (create("Frame", {
            Parent = rowParent,
            Name = name:gsub("%W", "") .. "Feature",
            Active = true,
            BackgroundColor3 = state.guiStyle == "Default"
                and Theme.surface
                or Theme.background,
            BackgroundTransparency = state.guiStyle == "Default" and 0.82 or 1,
            BorderSizePixel = 0,
            ClipsDescendants = true,
            LayoutOrder = layoutOrder,
            Size = UDim2.new(1, 0, 0, feature.collapsedHeight),
        }) :: any) :: Frame
        feature.row = row

        row:SetAttribute("FeatureCategory", categoryName)
        -- Alphabetical key for ClickGui.sortCategoryRows.
        row:SetAttribute("FeatureSortName", string.lower(name))

        local bitmapText: any = state.bitmapText
        local title: TextLabel? = nil
        if bitmapText and type(bitmapText.draw) == "function" then
            feature.drawTitle = function(color: Color3?, transparency: number?): ()
                if color ~= nil then
                    feature.titleColor = color
                end
                if transparency ~= nil then
                    feature.titleTransparency = transparency
                end
                bitmapText.draw(row, {
                    {
                        text = feature.name,
                        color = feature.titleColor or Theme.text,
                    },
                }, {
                    Name = "Title",
                    TextSize = state.layout.titleFontSize,
                    ZIndex = 7,
                    align = "Center",
                    transparency = feature.titleTransparency or 0,

                    maxWidth = 140,
                })
            end
            feature.drawTitle(Theme.text, 0)
        else
            title = (makeTextLabel(row, name, state.layout.titleFontSize) :: any) :: TextLabel
            local label: TextLabel = title :: TextLabel
            label.FontFace = CONTROL_FONT

            label.Position = UDim2.fromOffset(28, 0)
            label.Size = UDim2.new(1, -56, 0, feature.collapsedHeight)
            label.TextXAlignment = Enum.TextXAlignment.Center
            label.TextTruncate = Enum.TextTruncate.AtEnd
            label.TextColor3 = state.guiStyle == "Default"
                and Theme.text
                or Theme.text
            feature.title = label
        end

        local arrow: TextButton? = nil
        local toggleExpansion: () -> () = function(): () end

        function feature.hasOptions(): boolean
            if configuration.noOptions == true then
                return false
            end
            for _, optionRow: GuiObject in ipairs(feature.optionRows) do
                if optionRow.Visible then
                    return true
                end
            end
            return false
        end

        local function syncArrow(): ()
            local expandable: boolean = feature.hasOptions()
            feature.hasSettings = expandable
            if expandable and not arrow then
                local built: TextButton = (create("TextButton", {
                    TextTruncate = Enum.TextTruncate.AtEnd,
                    Parent = row,
                    Name = "Expand",
                    Active = true,
                    AnchorPoint = Vector2.new(1, 0),
                    AutoButtonColor = false,
                    BackgroundTransparency = 1,
                    BorderSizePixel = 0,
                    FontFace = CONTROL_FONT,
                    Position = UDim2.new(1, 0, 0, 0),
                    Size = UDim2.fromOffset(22, feature.collapsedHeight),
                    Text = "",
                    TextSize = 10,
                    ZIndex = 9,
                }) :: any) :: TextButton
                trackUiConnection(built.MouseButton1Click:Connect(function(): ()
                    toggleExpansion()
                end))
                arrow = built
                feature.arrow = built
            end
            if arrow then
                (arrow :: TextButton).Visible = expandable

                drawPixelIcon(
                    arrow :: TextButton,
                    feature.expanded and "up" or "down",
                    nil,
                    "arrows"
                )
            end
        end
        feature.syncArrow = syncArrow
        -- Palette changes must repaint the expand arrows immediately. The
        -- per-pixel theme binding covers this too, but the arrow icon is
        -- destroyed and rebuilt on every expansion toggle, and a redraw hook
        -- guarantees the chosen colour lands on every copy ("tengo que
        -- tocarlas para que se aplica el color").
        if state and type(state.registerThemeRefresh) == "function" then
            state.registerThemeRefresh(function(): ()
                if row.Parent and arrow then
                    syncArrow()
                end
            end, {"arrows"})
        end

        local activationTarget: TextButton = create("TextButton", {
            TextTruncate = Enum.TextTruncate.AtEnd,
            ClipsDescendants = true,
            Parent = row,
            Name = "ActivationTarget",
            Active = true,
            AutoButtonColor = false,
            BackgroundTransparency = 1,
            BorderSizePixel = 0,
            Position = UDim2.fromOffset(0, 0),
            Size = UDim2.new(1, 0, 0, feature.collapsedHeight),
            Text = "",
            ZIndex = 6,
        }) :: TextButton
        feature.activationTarget = activationTarget
        if title then
            (title :: TextLabel).ZIndex = 7
        end

        trackUiConnection(row.MouseEnter:Connect(function(): ()
            state.applyCardSkin(feature, true)
        end))
        trackUiConnection(row.MouseLeave:Connect(function(): ()
            state.applyCardSkin(feature, false)
        end))

        local options: Frame = (create("Frame", {
            Parent = row,
            Name = "Options",
            Active = true,
            BackgroundTransparency = 1,
            BorderSizePixel = 0,
            Position = UDim2.fromOffset(4, feature.collapsedHeight),
            Size = UDim2.new(1, -8, 0, 0),
            Visible = false,
            ZIndex = row.ZIndex + 10,
        }) :: any) :: Frame
        feature.options = options
        create("UIPadding", {
            Parent = options,
            PaddingBottom = UDim.new(0, 4),
            PaddingLeft = UDim.new(0, 0),
            PaddingRight = UDim.new(0, 0),
            PaddingTop = UDim.new(0, 4),
        })
        local optionsLayout: UIListLayout = (create("UIListLayout", {
            Parent = options,
            FillDirection = Enum.FillDirection.Vertical,
            HorizontalAlignment = Enum.HorizontalAlignment.Center,
            Padding = UDim.new(0, 4),
            SortOrder = Enum.SortOrder.LayoutOrder,
        }) :: any) :: UIListLayout

        local function updateExpandedSize(_animate: boolean?): ()

            local visibleOptions: number = 0
            local declaredHeight: number = 0
            for _, optionRow: GuiObject in ipairs(feature.optionRows) do
                if optionRow.Visible then
                    visibleOptions += 1
                    declaredHeight += optionRow.Size.Y.Offset
                end
            end
            local expectedHeight: number = if visibleOptions > 0
                then declaredHeight + math.max(0, visibleOptions - 1) * 4
                else 0
            local measuredHeight: number = math.max(
                optionsLayout.AbsoluteContentSize.Y,
                expectedHeight
            )
            local singleOptionExtra: number = if visibleOptions == 1 then 8 else 0

            local optionsHeight: number = measuredHeight + 8 + singleOptionExtra
            feature.optionsHeight = optionsHeight
            options.Size = UDim2.new(1, 0, 0, optionsHeight)
            syncArrow()
            local settingsWindow: any = feature.settingsWindow
            if settingsWindow and settingsWindow.resizeToOptions then
                settingsWindow.resizeToOptions()
            end
        end
        feature.updateExpandedSize = updateExpandedSize
        trackUiConnection(
            optionsLayout:GetPropertyChangedSignal("AbsoluteContentSize")
                :Connect(function(): ()
                    updateExpandedSize(false)
                end)
        )

        toggleExpansion = function(): ()
            if not feature.hasOptions() then

                syncArrow()
                return
            end

            local windows: any = state.windows
            if windows and type(windows.OpenFeatureSettings) == "function" then
                -- Pressing the arrow again closes the options window (the
                -- same path as the window's X button), so the arrows are a
                -- real open/close toggle.
                local window: any = feature.settingsWindow
                if window
                    and window.root
                    and window.root.Visible == true then
                    window.root.Visible = false
                    feature.expanded = false
                    if feature.syncArrow then
                        feature.syncArrow()
                    end
                    return
                end
                windows.OpenFeatureSettings(feature)
            end
        end
        feature.toggleExpansion = toggleExpansion

        function feature.setHeld(held: boolean): ()
            if feature.held == held then
                return
            end
            feature.held = held
            local handler: any = feature.onHold
            if type(handler) ~= "function" then
                return
            end
            local ok: boolean, failure: any = pcall(handler, held)
            if not ok then
                warn("[Random Testing Menu] " .. name .. ": " .. tostring(failure))
                return
            end
            feature.active = held
            refreshFeatureToggle(feature)
        end

        local activationGeneration: number = 0
        local function paintToggle(): ()
            local ok: boolean, failure: any = pcall(refreshFeatureToggle, feature)
            if not ok then warn("[Random Testing Menu] " .. name .. " UI: " .. tostring(failure)) end
        end
        local function activateFeature(): ()
            if feature.isHoldAction then

                return
            end
            if feature.isCategory then
                if not feature.hasOptions() then
                    return
                end
                feature.expanded = not feature.expanded
                updateExpandedSize(true)
                return
            end
            if feature.isAction then
                if feature.actionRunning then
                    return
                end
                feature.actionRunning = true
                task.spawn(function(): ()
                    local success: boolean, errorMessage: any = pcall(feature.onToggle)
                    feature.actionRunning = false
                    if not success then
                        warn("[Random Testing Menu] " .. name .. ": " .. tostring(errorMessage))
                        notify("Could not run " .. name)
                    end
                end)
                return
            end

            local requestedState: boolean = not feature.enabled
            activationGeneration += 1
            local generation: number = activationGeneration
            feature.enabled = requestedState
            if requestedState then feature.enabledAt = os.clock() end
            -- Handlers and profile writes can yield in an executor. Repaint all
            -- views immediately, and ignore an older handler that finishes
            -- after a newer toggle or after teardown changed the feature.
            paintToggle()
            local success: boolean, errorMessage: any = pcall(feature.onToggle, requestedState)
            if generation ~= activationGeneration or feature.enabled ~= requestedState then return end
            if success then
                configData.states[feature.configKey] = requestedState
                flushConfigSave()
            else
                warn("[Random Testing Menu] " .. name .. ": " .. tostring(errorMessage))
                feature.enabled = false
                paintToggle()
                pcall(feature.onToggle, false)
                if generation ~= activationGeneration then return end
                local reason: string = tostring(errorMessage)
                if #reason > 90 then
                    reason = string.sub(reason, 1, 90)
                end
                notify("Could not toggle " .. name .. ": " .. reason)
            end
        end
        feature.activate = activateFeature

        state.applyCardSkin(feature, false)
        if feature.isAction then

            trackUiConnection(activationTarget.MouseButton1Down:Connect(function(): ()
                if feature.row then
                    (feature.row :: Frame).BackgroundTransparency = 0.46
                end
            end))
            trackUiConnection(activationTarget.MouseButton1Up:Connect(function(): ()
                state.applyCardSkin(feature, true)
            end))
        end

        trackUiConnection(activationTarget.MouseButton1Click:Connect(activateFeature))

        trackUiConnection(activationTarget.InputBegan:Connect(function(input: InputObject): ()
            if input.UserInputType == Enum.UserInputType.MouseButton2 then
                toggleExpansion()
            end
        end))

        local shortcutButton: TextButton? = nil
        if state.isMobile then
            local slot: TextButton =
                (makeButton(row, "KEY", 9) :: any) :: TextButton
            slot.Name = "Shortcut"

            slot.AnchorPoint = Vector2.new(0, 0.5)
            slot.Position = UDim2.new(0, 4, 0, feature.collapsedHeight / 2)
            slot.Size = UDim2.fromOffset(22, 22)
            slot.ZIndex = 9
            applyKeySlotStyle(slot)
            slot.TextScaled = true
            create("UITextSizeConstraint", {
                Parent = slot,
                MaxTextSize = 11,
                MinTextSize = 7,
            })
            shortcutButton = slot
        end
        feature.shortcutButton = shortcutButton
        if feature.kind == "group" then

            if shortcutButton then
                (shortcutButton :: TextButton).Visible = false
            end
        elseif feature.kind == "hold" then

            bindActionShortcut(
                shortcutButton,
                feature.configKey,
                name,
                function(): () end,
                {
                    hold = true,
                    onPress = function(): ()
                        feature.setHeld(true)
                    end,
                    onRelease = function(): ()
                        feature.setHeld(false)
                    end,
                }
            )
        else
            bindActionShortcut(
                shortcutButton,
                feature.configKey,
                name,
                activateFeature,
                {
                    -- Read the live state after the handler ran, so the toast
                    -- matches the card even for a module that refuses to start.
                    feedback = function(): string
                        return name .. (feature.enabled == true and " on" or " off")
                    end,
                }
```

```
        )
    end

    -- Mobile: the little square is not a key slot, it is the touch button
    -- for this feature. It previews the configured shape and colour, "+"
    -- when the button does not exist yet, and opens the editor on tap.
    if state.isMobile and shortcutButton and state.mobileUi then
        local slot: TextButton = shortcutButton :: TextButton
        local mobileUi: any = state.mobileUi
        local actionId: string = mobileUi.actionId(feature.configKey)
        local function syncMobileSlot(): ()
            local action: any = mobileUi.actions[actionId]
            local corner: UICorner? = slot:FindFirstChildOfClass("UICorner")
            if action then
                slot.Text = ""
                slot.BackgroundColor3 = mobileUi.colorFromHex(action.color)
                    or Theme.surfaceRaised
                slot.BackgroundTransparency = 0.12
                if corner then
                    corner.CornerRadius = action.shape == "Circle"
                        and UDim.new(1, 0)
                        or action.shape == "Rounded"
                            and UDim.new(0, 10)
                            or UDim.new(0, 2)
                end
            else
                slot.Text = "+"
                slot.BackgroundColor3 = Theme.surfaceRaised
                slot.BackgroundTransparency = 0.35
                if corner then corner.CornerRadius = UDim.new(0, 6) end
            end
        end
        syncMobileSlot()
        if type(mobileUi.onChanged) == "function" then
            mobileUi.onChanged(syncMobileSlot)
        end
        local binding: any = state.shortcutBindings
            and state.shortcutBindings[feature.configKey]
        if binding then
            local previous: any = binding.refresh
            binding.refresh = function(): ()
                if type(previous) == "function" then previous() end
                syncMobileSlot()
            end
        end
    end

    if (feature.kind == "toggle"
            or feature.kind == "action"
            or feature.kind == "hold")
        and not state.isMobile
        and type(beginKeyCapture) == "function"
        and type(state.shortcutBindings) == "table"
        and state.shortcutBindings[feature.configKey] ~= nil then
        local binding: any = state.shortcutBindings[feature.configKey]
        local UNBOUND_BG: Color3 = Color3.fromRGB(0, 0, 0)
        local square: TextButton = (create("TextButton", {
            TextTruncate = Enum.TextTruncate.AtEnd,
            Parent = row,
            Name = "BindSquare",
            Active = true,
            AnchorPoint = Vector2.new(0, 0.5),
            AutoButtonColor = false,
            BackgroundColor3 = UNBOUND_BG,
            BackgroundTransparency = 0.55,
            BorderSizePixel = 0,
            FontFace = CONTROL_FONT,
            Position = UDim2.new(0, 4, 0.5, 0),
            Size = UDim2.fromOffset(18, 18),
            Text = "",
            TextColor3 = Theme.text,
            TextTransparency = 0,
            TextSize = 8,
            ZIndex = 8,
        }) :: any) :: TextButton
        square:SetAttribute("CaptureIdleText", "")
        local edge: UIStroke = (create("UIStroke", {
            Parent = square,
            Name = "StyleStroke",
            Color = Theme.outline,
            Thickness = 1,
            Transparency = 0.6,
        }) :: any) :: UIStroke

        local COMPACT_KEY_NAMES: {[string]: string} = {
            LeftShift = "LS",
            RightShift = "RS",
            LeftControl = "LC",
            RightControl = "RC",
            LeftAlt = "LA",
            RightAlt = "RA",
            CapsLock = "CL",
            Backspace = "Bk",
            Return = "En",
            Escape = "Es",
            Space = "Sp",
            PageUp = "PU",
            PageDown = "PD",
            Insert = "Ins",
            Delete = "Del",
            Home = "Hm",
            End = "End",
        }
        local function keyCaption(): string
            if binding.key == nil or binding.key == Enum.KeyCode.Unknown then
                return ""
            end
            local name: string = tostring(binding.key.Name or "")
            local compact: string? = COMPACT_KEY_NAMES[name]
            if compact then
                return compact
            end
            local display: string = state.keyDisplayName
                and tostring(state.keyDisplayName(name))
                or name
            return string.sub(display, 1, 3)
        end

        local capturing: boolean = false
        local function conflictWith(): any
            if binding.key == nil or binding.key == Enum.KeyCode.Unknown then
                return nil
            end
            for _, other: any in pairs(state.shortcutBindings) do
                if other ~= binding and other.key == binding.key then
                    return other
                end
            end
            return nil
        end
        local function syncSquare(): ()
            if capturing then
                return
            end
            local caption: string = keyCaption()
            square.Text = caption
            square:SetAttribute("CaptureIdleText", caption)
            local bound: boolean = binding.key ~= nil
                and binding.key ~= Enum.KeyCode.Unknown
            -- The letter communicates that a key is assigned. A green
            -- square must not imply an OFF module is still enabled.
            local conflict: any = bound and conflictWith() or nil
            if conflict then
                square.BackgroundColor3 = Theme.negative
                square.BackgroundTransparency = 0.35
                edge.Color = Theme.negative
                edge.Transparency = 0.15
            elseif bound and feature.enabled == true then
                square.BackgroundColor3 = Theme.enabled
                square.BackgroundTransparency = 0.35
                edge.Color = Theme.enabled
                edge.Transparency = 0.25
            else
                square.BackgroundColor3 = UNBOUND_BG
                square.BackgroundTransparency = 0.55
                edge.Color = Theme.outline
                edge.Transparency = if bound then 0.25 else 0.6
            end
        end
        feature.bindSquare = square
        feature.syncBindSquare = syncSquare
        if state.registerThemeRefresh then
            state.registerThemeRefresh(function(): ()
                if square.Parent then
                    syncSquare()
                end
            end, {"enabled", "outline"})
        end
        local refreshBinding: any = binding.refresh
        binding.refresh = function(): ()
            if refreshBinding then
                refreshBinding()
            end
            syncSquare()
        end
        syncSquare()

        local function syncKeyboard(): ()
            square.Visible = UserInputService == nil
                or UserInputService.KeyboardEnabled ~= false
        end
        syncKeyboard()
        pcall(function(): ()
            trackUiConnection(
                UserInputService:GetPropertyChangedSignal("KeyboardEnabled")
                    :Connect(syncKeyboard)
            )
        end)

        local function tipText(): string
            local bound: boolean = binding.key ~= nil
                and binding.key ~= Enum.KeyCode.Unknown
            if not bound then
                return "Bind: click, then press a key"
            end
            local conflict: any = conflictWith()
            if conflict then
                return binding.key.Name .. " — also bound to "
                    .. tostring(conflict.id)
            end
            return "Bound: " .. binding.key.Name
        end
        trackUiConnection(square.MouseEnter:Connect(function(): ()
            local tooltip: any = state.featureTooltip
            if not tooltip then
                return
            end
            state.tooltipToken = (state.tooltipToken or 0) + 1
            local token: number = state.tooltipToken
            task.delay(0.4, function(): ()
                if state.tooltipToken ~= token or not square.Parent then
                    return
                end
                local rawText: string = tipText()
                local bitmap: any = state.bitmapText
                local measured: number = bitmap
                    and type(bitmap.measure) == "function"
                    and tonumber(bitmap.measure({{text = rawText}}, 16))
                    or #rawText * 8
                local width: number = math.clamp(measured + 24, 140, 360)
                local wrapped: string = rawText
                local lines: number = 1
                if bitmap and type(bitmap.wrap) == "function" then
                    wrapped, lines = bitmap.wrap(rawText, width - 24, 16)
                end
                tooltip.Text = wrapped
                tooltip.TextTransparency = 1
                tooltip.Size = UDim2.fromOffset(width, math.max(30, lines * 18 + 12))
                tooltip.Position = UDim2.fromOffset(
                    square.AbsolutePosition.X,
                    square.AbsolutePosition.Y + square.AbsoluteSize.Y + 8
                )
                if state.bitmapText and type(state.bitmapText.draw) == "function" then
                    state.bitmapText.draw(tooltip, {
                        {text = tooltip.Text},
                    }, {
                        Name = "Glyphs",
                        TextSize = 16,
                        maxWidth = width - 24,
                        ellipsis = true,
                        ZIndex = 251,
                    })
                end
                tooltip.Visible = true
            end)
        end))
        trackUiConnection(square.MouseLeave:Connect(function(): ()
            state.tooltipToken = (state.tooltipToken or 0) + 1
            if state.featureTooltip then
                state.featureTooltip.Visible = false
            end
        end))

        local outsideConnection: RBXScriptConnection? = nil
        local function cleanupCapture(): ()
            capturing = false
            if outsideConnection then
                pcall(function(): ()
                    (outsideConnection :: RBXScriptConnection):Disconnect()
                end)
                outsideConnection = nil
            end
            syncSquare()

            task.delay(0.2, function(): ()
                if square.Parent and not capturing then
                    syncSquare()
                end
            end)
        end
        trackUiConnection(square.MouseButton1Click:Connect(function(): ()
            if capturing then

                if state.keyCaptureCancel then
                    state.keyCaptureCancel()
                else
                    cleanupCapture()
                end
                return
            end
            capturing = true
            beginKeyCapture(square, function(keyCode: Enum.KeyCode): ()

                local backspaceKey: any = (Enum.KeyCode :: any).Backspace
                local deleteKey: any = (Enum.KeyCode :: any).Delete
                if keyCode == backspaceKey or keyCode == deleteKey then
                    if binding.key ~= Enum.KeyCode.Unknown
                        and type(binding.assign) == "function" then
                        binding.assign(binding.key)
                    end
                elseif type(binding.assign) == "function" then
                    binding.assign(keyCode)
                end
                cleanupCapture()
            end)

            local baseCancel: any = state.keyCaptureCancel
            state.keyCaptureCancel = function(): ()
                if baseCancel then
                    baseCancel()
                end
                cleanupCapture()
            end

            pcall(function(): ()
                outsideConnection = UserInputService.InputBegan:Connect(function(
                    input: InputObject
                ): ()
                    if input.UserInputType ~= Enum.UserInputType.MouseButton1
                        and input.UserInputType ~= Enum.UserInputType.Touch then
                        return
                    end
                    local px: number = input.Position.X
                    local py: number = input.Position.Y
                    local ax: number = square.AbsolutePosition.X
                    local ay: number = square.AbsolutePosition.Y
                    if px < ax or px > ax + square.AbsoluteSize.X
                        or py < ay or py > ay + square.AbsoluteSize.Y then
                        if state.keyCaptureCancel then
                            state.keyCaptureCancel()
                        end
                    end
                end)
            end)
        end))
    end

    feature.applyLayout = function(): ()
        local metrics: LayoutMetrics = state.layout
        feature.collapsedHeight = metrics.cardHeight
        if title then
            state.setTextMetric(title, metrics.titleFontSize)
        elseif feature.drawTitle then

            feature.drawTitle(nil, nil)
        end
        if arrow then
            (arrow :: TextButton).Size =
                UDim2.fromOffset(22, metrics.cardHeight)
        end
        if shortcutButton then
            (shortcutButton :: TextButton).Size = UDim2.fromOffset(22, 22);
            (shortcutButton :: TextButton).Position =
                UDim2.new(0, 4, 0, metrics.cardHeight / 2)
        end
        activationTarget.Size = UDim2.new(1, 0, 0, metrics.cardHeight)
        options.Position = UDim2.fromOffset(4, metrics.cardHeight)
        syncArrow()
        updateExpandedSize(false)
    end

    feature.applyLayout()

    function feature:SetStatus(status: string?): ()
        feature.status = status
    end

    -- Used when a game is identified after the universal card was made.
    -- configKey and framework identity deliberately stay unchanged.
    function feature:SetName(nextName: string): ()
        if type(nextName) ~= "string" or nextName == "" then return end
        local previous: string = feature.name
        if previous ~= nextName and registry[previous] == feature then
            registry[previous] = nil
            registry[nextName] = feature
        end
        feature.name = nextName
        feature.searchText = string.lower(nextName .. " " .. (feature.description or "")
            .. " " .. sectionName .. " " .. categoryName)
        row:SetAttribute("FeatureSortName", string.lower(nextName))
        if feature.title then feature.title.Text = nextName end
        if feature.drawTitle then feature.drawTitle() end
        local search: any = state.moduleSearch
        if search and type(search.Unregister) == "function" and type(search.Register) == "function" then
            search.Unregister(feature)
            search.Register(feature)
        end
        if state.clickGui and type(state.clickGui.Resort) == "function" then
            state.clickGui.Resort(feature)
        end
        if state.floatingUi and type(state.floatingUi.registerFeature) == "function" then
            state.floatingUi.registerFeature(feature)
        end
    end

    registry[name] = feature
    table.insert(allFeatures, feature)

    if state.floatingUi then
        state.floatingUi.registerFeature(feature)
    end

    if state.clickGui then
        state.clickGui.place(feature)
    end
    applyFeaturePresentation(feature, state.guiStyle == "Default")

    if type(addFeatureTooltip) == "function" and description ~= "" then
        addFeatureTooltip(feature, description)
    end

    local search: any = state.moduleSearch
    if search and type(search.Register) == "function" then
        search.Register(feature)
        table.insert(registeredSearchFeatures, feature)
    end

    if feature.isAlwaysOn then
        task.defer(function(): ()
            local ok: boolean, failure: any = pcall(feature.onToggle, true)
            if not ok then
                warn("[Random Testing Menu] " .. name .. ": " .. tostring(failure))
            end
        end)
    end

    if not feature.isAction
        and not feature.isCategory
        and not feature.isHoldAction
        and configuration.restore ~= false
        and configData.states[feature.configKey] == true then
        task.defer(function(): ()
            -- Do not let an old deferred restore undo a more recent input.
            if activationGeneration ~= 0 or configData.states[feature.configKey] ~= true then return end
            activationGeneration += 1
            local generation: number = activationGeneration
            feature.enabled = true
            feature.enabledAt = os.clock()
            paintToggle()
            local success: boolean, errorMessage: any = pcall(feature.onToggle, true)
            if generation ~= activationGeneration or feature.enabled ~= true then return end
            if not success then
                feature.enabled = false
                paintToggle()
                configData.states[feature.configKey] = false
                queueConfigSave()
                warn(
                    "[Random Testing Menu] Could not restore "
                        .. name
                        .. ": "
                        .. tostring(errorMessage)
                )
            end
        end)
    end
    return feature
end

moduleCleanup = function(): ()
    local search: any = state.moduleSearch
    if search and type(search.Unregister) == "function" then
        for _, feature: any in ipairs(registeredSearchFeatures) do
            search.Unregister(feature)
        end
    end
    table.clear(registeredSearchFeatures)
end
Module.Initialized = true
return {createUniversalFeature = createUniversalFeature}
end

function Module.destroy(): ()
    moduleCleanup()
    moduleCleanup = function(): () end
    Module.Initialized = false
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/guis/Wurst/Code/Cards.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/guis/Wurst/Code/ClickGui.lua
BYTES: 15772
SHA-256: 9c2a059981f2eec49e024961cd3e2fb31c55d33e1ea2d727a0a2ffe6a8757c9e
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
export type CategoryWindow = {
    name: string,
    window: any,
    layout: UIListLayout,
    count: number,
}

local Module = {
    Name = "ClickGui",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local activeCleanup: (() -> ())? = nil

local WINDOW_WIDTH: number = 200
local WINDOW_MAX_HEIGHT: number = 300

local SCALE_MINIMUM: number = 0.7
local SCALE_MAXIMUM: number = 1.6

local CALIBRATION_WIDTH: number = 1600
local CALIBRATION_HEIGHT: number = 900
local GRID_MARGIN: number = 10
local WINDOW_GAP: number = 2

local GRID_GAP: number = 10
local TITLE_HEIGHT: number = 26

-- v138.9: the window list and the aliases live in ONE place, Manifest.lua
-- (categories.order / categories.aliases). This file used to carry its own copy
-- with a different alias set than the loader's. The table below is only the
-- last-resort default for a hand-made state that has no catalog.
local DEFAULT_CATEGORY_ORDER: {string} = {
    "Exploits", "Utilities", "World", "Render", "Blocks", "Items",
}
local categories: any = {
    order = DEFAULT_CATEGORY_ORDER,
    aliases = {Combat = "Exploits"},
    default = "Utilities",
}

local function useCatalog(hostState: any): ()
    local catalog: any = hostState and hostState.categories
    if type(catalog) == "table" and type(catalog.order) == "table"
        and #catalog.order > 0 then
        categories = catalog
    end
end

local function order(): {string}
    return categories.order or DEFAULT_CATEGORY_ORDER
end

local function fallbackNormalize(raw: any): string
    local name: string = tostring(raw or categories.default or "Utilities")
    if table.find(order(), name) ~= nil then
        return name
    end
    local aliases: any = categories.aliases
    if type(aliases) == "table" and type(aliases[name]) == "string" then
        return aliases[name]
    end
    return categories.default or "Utilities"
end

function Module.init(context: any): any
    local host: any = context
    local state: any = host.state
    useCatalog(state)
    local create: any = host.create
    local allFeatures: {any} = host.allFeatures
    local normalizeFeatureCategory: any = host.normalizeFeatureCategory

    local function viewportSize(): Vector2
        -- Wurst stores the last valid viewport during camera swaps, so a
        -- transient 0x0 camera cannot collapse the menu or its window grid.
        local saved: any = state.layoutViewport
        if saved and saved.X > 1 and saved.Y > 1 then return saved end
        local camera: Camera? = workspace.CurrentCamera
        local current: any = camera and camera.ViewportSize
        if current and current.X > 1 and current.Y > 1 then return current end
        return Vector2.new(CALIBRATION_WIDTH, CALIBRATION_HEIGHT)
    end

    local function viewportScale(): number
        local shape: any = host.ThemeEngine and host.ThemeEngine.shape
        local wanted: number = shape and tonumber(shape.scale) or 1
        local viewport: Vector2 = viewportSize()
        -- Clamp the automatic fit, not the product. Clamping the product to
        -- 0.7 made every slider value on a small phone yield exactly 0.7.
        -- Keeping the preference as a multiplier also makes both endpoints
        -- responsive on larger screens without changing the default fit.
        local fit: number = math.clamp(math.min(
            viewport.X / CALIBRATION_WIDTH,
            viewport.Y / CALIBRATION_HEIGHT
        ), SCALE_MINIMUM, SCALE_MAXIMUM)
        if wanted ~= wanted then wanted = 1 end
        return fit * math.clamp(wanted, 0.7, 1.3)
    end

    local function logicalViewport(): Vector2
        local viewport: Vector2 = viewportSize()
        local factor: number = viewportScale()
        return Vector2.new(viewport.X / factor, viewport.Y / factor)
    end

    local windows: any = state.windows
    if not windows then

        error("ClickGui requires the window manager")
    end

    local ClickGui: any = {
        categories = {} :: {[string]: CategoryWindow},
        order = {} :: {string},

        connections = {} :: {RBXScriptConnection},
    }

    -- v192: no hardcoded build number anywhere. The chip prints whatever the
    -- product line carries, which is derived from SOURCE_STAMP, so the header
    -- and the UI Settings window can never disagree.
    local productVersion: string = tostring(
        (host.PRODUCT and host.PRODUCT.version)
            or ((host.BUILD_LABEL or "0.0") .. " Release")
    )
    local LOGO_CHIP_WIDTH: number = 142 + 10 + (8 + #productVersion * 15) + 4
    local LOGO_CHIP_HEIGHT: number = 44
    local TOP_ROW_X: number = 14 + LOGO_CHIP_WIDTH + GRID_GAP
    local TOP_ROW_Y: number = 10
    local nextX: number = TOP_ROW_X
    local nextY: number = TOP_ROW_Y
    local rowHeight: number = LOGO_CHIP_HEIGHT

    local function nextSlot(height: number): Vector2
        local viewport: Vector2 = logicalViewport()
        if nextX + WINDOW_WIDTH > viewport.X - GRID_MARGIN then
            nextX = GRID_MARGIN
            nextY += rowHeight + GRID_GAP
            rowHeight = 0
        end
        local slot: Vector2 = Vector2.new(nextX, nextY)
        nextX += WINDOW_WIDTH + GRID_GAP
        rowHeight = math.max(rowHeight, height)
        return slot
    end

    local function categoryWindow(name: string): CategoryWindow
        local existing: CategoryWindow? = ClickGui.categories[name]
        if existing then
            return existing :: CategoryWindow
        end

        local height: number = TITLE_HEIGHT + 8
        local window: any = windows.Create({
            id = "ClickGui_" .. name,
            title = name,
            size = Vector2.new(WINDOW_WIDTH, height),
            position = nextSlot(WINDOW_MAX_HEIGHT * 0.5),
            maxHeight = WINDOW_MAX_HEIGHT,

            closable = false,
        })

        local layout: UIListLayout = create("UIListLayout", {
            Parent = window.body,
            Padding = UDim.new(0, WINDOW_GAP),
            SortOrder = Enum.SortOrder.LayoutOrder,
        }) :: UIListLayout
        create("UIPadding", {
            Parent = window.body,
            PaddingTop = UDim.new(0, WINDOW_GAP),
            PaddingBottom = UDim.new(0, WINDOW_GAP),
            PaddingLeft = UDim.new(0, WINDOW_GAP),
            PaddingRight = UDim.new(0, WINDOW_GAP),
        })

        local record: CategoryWindow = {
            name = name,
            window = window,
            layout = layout,
            count = 0,
        }
```

```
        ClickGui.categories[name] = record
        table.insert(ClickGui.order, name)
        table.sort(ClickGui.order, function(left: string, right: string): boolean
            local leftRank: number = table.find(order(), left) or math.huge
            local rightRank: number = table.find(order(), right) or math.huge
            if leftRank == rightRank then
                return left < right
            end
            return leftRank < rightRank
        end)

        local function resize(): ()

            local limit: number = tonumber(state.uiMaxHeight) or WINDOW_MAX_HEIGHT
            if limit <= 0 then
                limit = math.huge
            end

            limit = math.min(limit, logicalViewport().Y - 12)

            local declaredHeight: number = 0
            local visibleRows: number = 0
            for _, child: Instance in ipairs(window.body:GetChildren()) do
                if child:IsA("GuiObject") and (child :: GuiObject).Visible then
                    visibleRows += 1
                    declaredHeight += (child :: GuiObject).Size.Y.Offset
                end
            end
            declaredHeight += math.max(visibleRows - 1, 0) * WINDOW_GAP
            local contentHeight: number = math.max(
                layout.AbsoluteContentSize.Y,
                declaredHeight
            )
            local wanted: number = math.min(
                TITLE_HEIGHT + contentHeight + 4,
                limit
            )
            window.fullHeight = wanted
            if not window.collapsed then
                window.root.Size = UDim2.fromOffset(WINDOW_WIDTH, wanted)
            end

            if windows.ClampWindow then
                windows.ClampWindow(window)
            end
        end
        table.insert(
            ClickGui.connections,
            layout:GetPropertyChangedSignal("AbsoluteContentSize"):Connect(resize)
        )
        resize()
        record.resize = resize :: any
        return record
    end

    local function sortCategoryRows(record: CategoryWindow): ()
        local rows: {GuiObject} = {}
        for _, child: Instance in ipairs(record.window.body:GetChildren()) do
            if not child:IsA("GuiObject") then
                continue
            end
            local key: any = child:GetAttribute("FeatureSortName")
            if type(key) == "string" then
                table.insert(rows, child :: GuiObject)
            end
        end
        -- v138.9: every window is ALPHABETICAL by card name (FeatureSortName is
        -- the lowercased display name), so a module is always where you expect
        -- it. Ties (same name in two sections) fall back to the row name so the
        -- order is stable instead of moving between refreshes.
        table.sort(rows, function(left: GuiObject, right: GuiObject): boolean
            local leftKey: string = tostring(left:GetAttribute("FeatureSortName") or "")
            local rightKey: string = tostring(right:GetAttribute("FeatureSortName") or "")
            if leftKey ~= rightKey then
                return leftKey < rightKey
            end
            return tostring(left.Name) < tostring(right.Name)
        end)
        for index: number, row: GuiObject in ipairs(rows) do
            row.LayoutOrder = index
        end
    end

    -- An adapter may change a card's display name after delayed game detection.
    -- Reorder without placing/counting the same row a second time.
    function ClickGui.Resort(feature: any): ()
        local record: CategoryWindow? = feature and ClickGui.categories[feature.categoryName]
        if record then
            sortCategoryRows(record :: CategoryWindow)
        end
    end

    function ClickGui.place(feature: any): ()
        local row: Frame? = feature and feature.row
        if not row then
            return
        end

        local raw: any = (row :: Frame):GetAttribute("FeatureCategory")
        local name: string = type(normalizeFeatureCategory) == "function"
            and normalizeFeatureCategory(raw)
            or fallbackNormalize(raw)
        local record: CategoryWindow = categoryWindow(name)
        record.count += 1;
        (row :: Frame).Parent = record.window.body;
        (row :: Frame).Size = UDim2.new(1, 0, 0, feature.collapsedHeight or 22)
        sortCategoryRows(record)
        if record.resize then
            (record.resize :: () -> ())()
        end
        local userMoved: boolean = false
        for _, category: string in ipairs(ClickGui.order) do
            local candidate: CategoryWindow? = ClickGui.categories[category]
            if candidate and candidate.window.userMoved == true then
                userMoved = true
                break
            end
        end
        if not userMoved and type(ClickGui.Retile) == "function" then
            ClickGui.Retile()
        end
    end

    function ClickGui.Retile(): ()

        nextX = TOP_ROW_X
        nextY = TOP_ROW_Y
        rowHeight = LOGO_CHIP_HEIGHT
        for _, name: string in ipairs(ClickGui.order) do
            local record: CategoryWindow = ClickGui.categories[name]
            if record then
                local slot: Vector2 = nextSlot(
                    record.window.fullHeight or WINDOW_MAX_HEIGHT * 0.5
                )
                record.window.root.Position = UDim2.fromOffset(slot.X, slot.Y)
                if type(windows.SetHome) == "function" then
                    windows.SetHome(record.window, slot)
                end
                if type(windows.ClampWindow) == "function" then
                    windows.ClampWindow(record.window)
                end
            end
        end
    end

    function ClickGui.RefreshHeights(): ()
        for _, name: string in ipairs(ClickGui.order) do
            local record: CategoryWindow = ClickGui.categories[name]
            if record and record.resize then
                (record.resize :: () -> ())()
            end
        end
    end

    function ClickGui.SetVisible(visible: boolean): ()
        for _, name: string in ipairs(ClickGui.order) do
            local record: CategoryWindow = ClickGui.categories[name]

            if not visible and record.window.pinned then
                continue
            end
            record.window:SetVisible(visible and record.window.userVisible ~= false)
        end
    end

    local function resolvedCategory(feature: any): string
        local raw: any = feature and feature.row
            local raw: any = row and row:GetAttribute("FeatureCategory")
            if type(normalizeFeatureCategory) == "function" then
                return normalizeFeatureCategory(raw)
            end
            return fallbackNormalize(raw)
        end
        for _, name: string in ipairs(order()) do
            for _, feature: any in ipairs(allFeatures) do
                if resolvedCategory(feature) == name then
                    categoryWindow(name)
                    break
                end
            end
        end

        for _, feature: any in ipairs(allFeatures) do
            ClickGui.place(feature)
        end

        local scale: UIScale = create("UIScale", {
            Parent = host.PopupLayer,
            Name = "ClickGuiScale",
            Scale = viewportScale(),
        }) :: UIScale

        state.onLayout(function(): ()
            scale.Scale = viewportScale()
            ClickGui.RefreshHeights()
            local userMoved: boolean = false
            for _, name: string in ipairs(ClickGui.order) do
                local record: CategoryWindow? = ClickGui.categories[name]
                if record and record.window.userMoved == true then
                    userMoved = true
                    break
                end
            end
            if not userMoved then
                ClickGui.Retile()
            end
            -- WindowManager's listener can run before this one: reclamp again
            -- after the new scale is installed, using the correct logical viewport.
            if type(windows.Reclamp) == "function" then
                windows.Reclamp()
            end
        end)
        ClickGui.scale = scale

        -- Only update UIScale during the drag. Re-measure/reclamp after it settles.
        ClickGui.ApplyScale = function(): ()
            if ClickGui.scale then
                ClickGui.scale.Scale = viewportScale()
            end
        end

        state.clickGui = ClickGui
        state.addMenuVisibilityListener(function(visible: boolean): ()
            ClickGui.SetVisible(visible)
        end)
        ClickGui.SetVisible(state.visible == true)

        activeCleanup = function(): ()
            for _, name: string in ipairs(ClickGui.order) do
                local record: CategoryWindow? = ClickGui.categories[name]
                if record then
                    pcall(function(): ()
                        (record :: CategoryWindow).window:Destroy()
                    end)
                end
            end
            for _, connection: RBXScriptConnection in ipairs(ClickGui.connections) do
                pcall(function(): ()
                    connection:Disconnect()
                end)
            end
            ClickGui.connections = {}
            ClickGui.categories = {}
            ClickGui.order = {}
            state.clickGui = nil
        end
        Module.Initialized = true
        return ClickGui
    end

    function Module.destroy(): ()
        if activeCleanup then
            pcall(activeCleanup)
        end
        activeCleanup = nil
        Module.Initialized = false
    end

    return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/guis/Wurst/Code/ClickGui.lua
--------------------------------------------------------------------------------


--------------------------------------------------------------------------------
ARCHIVO: src/guis/Wurst/Code/FloatingWindows.lua
BYTES: 72095
SHA-256: 2718cd782a511ba880ed95585de2ff739a8369b00a86341d54506c4c1cfc8db5
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
export type FloatingWindowRecord = {
    id: string,
    root: Frame,
    body: Frame,
    closeButton: TextButton,
}

local activeCleanup: (() -> ())? = nil

local Module = {
    Name = "FloatingWindows",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

function Module.init(context: any): any
    local host: any = context
    local state: any = host.state
    local configData: any = host.configData
    local queueConfigSave: any = host.queueConfigSave
    local create: any = host.create
    local makeButton: any = host.makeButton
    local makeTextBox: any = host.makeTextBox
    local makeTextLabel: any = host.makeTextLabel
    local Theme: any = host.Theme
    local ThemeEngine: any = host.ThemeEngine
    local CONTROL_FONT: any = host.CONTROL_FONT
    local THIN_FONT: any = host.THIN_FONT
    local VALVE_FONT: any = host.VALVE_FONT
    local PopupLayer: any = host.PopupLayer
    local UI_RADIUS: any = host.UI_RADIUS
    local cornerRadius: any = host.cornerRadius
    local trackUiConnection: any = host.trackUiConnection
    local createToggleSwitch: any = host.createToggleSwitch
    local setToggleSwitch: any = host.setToggleSwitch
    local TweenService: any = host.TweenService
    local UserInputService: any = host.UserInputService
    local Players: any = host.Players
    local LocalPlayer: any = host.LocalPlayer
    local Stats: any = host.Stats
    local TaskManager: any = host.TaskManager
    local allFeatures: any = host.allFeatures

    local function bind(instance: Instance, property: string, token: string): ()
        if ThemeEngine and type(ThemeEngine.Bind) == "function" then
            ThemeEngine.Bind(instance, property, token)
        else
            (instance :: any)[property] = Theme[token]
        end
    end

    local ownedTasks: {any} = {}

    local FloatingUi: any = {
        favorites = {},
        favoriteRows = {},
        overlays = {},
        featureRows = {},
        windows = {},
    }
    state.floatingUi = FloatingUi

    if type(configData.ui.favorites) ~= "table" then
        configData.ui.favorites = {}
    end
    if type(configData.ui.overlays) ~= "table" then
        configData.ui.overlays = {}
    end
    if type(configData.ui.floatingPositions) ~= "table" then
        configData.ui.floatingPositions = {}
    end
    if type(configData.ui.overlaySettings) ~= "table" then
        configData.ui.overlaySettings = {}
    end
    local overlaySettings: any = configData.ui.overlaySettings
    overlaySettings.TextGUI = type(overlaySettings.TextGUI) == "table"
        and overlaySettings.TextGUI
        or {}
    overlaySettings.SessionInfo = type(overlaySettings.SessionInfo) == "table"
        and overlaySettings.SessionInfo
        or {}
    overlaySettings.TargetInfo = type(overlaySettings.TargetInfo) == "table"
        and overlaySettings.TargetInfo
        or {}
    overlaySettings.Radar = type(overlaySettings.Radar) == "table"
        and overlaySettings.Radar
        or {}
    local textGuiSettings: any = overlaySettings.TextGUI
    textGuiSettings.sort = textGuiSettings.sort or "Alphabetical"
    textGuiSettings.font = textGuiSettings.font or "Builder Sans"
    textGuiSettings.colorMode = textGuiSettings.colorMode or "Match GUI"
    textGuiSettings.scale = math.clamp(tonumber(textGuiSettings.scale) or 1, 0.75, 2)
    textGuiSettings.shadow = textGuiSettings.shadow ~= false
    textGuiSettings.gradient = textGuiSettings.gradient ~= true
    textGuiSettings.v4Gradient = textGuiSettings.v4Gradient == true
    textGuiSettings.animations = textGuiSettings.animations ~= false
    textGuiSettings.watermark = textGuiSettings.watermark == true
    textGuiSettings.background = textGuiSettings.background == true
    textGuiSettings.hideModules = textGuiSettings.hideModules == true
    textGuiSettings.hideRender = textGuiSettings.hideRender == true
    textGuiSettings.customText = tostring(textGuiSettings.customText or "")
    local sessionSettings: any = overlaySettings.SessionInfo
    sessionSettings.showPerformance = sessionSettings.showPerformance ~= false
    sessionSettings.showServer = sessionSettings.showServer ~= false
    sessionSettings.showRole = sessionSettings.showRole ~= false
    sessionSettings.showDeaths = sessionSettings.showDeaths ~= false
    sessionSettings.showWins = sessionSettings.showWins ~= false
    local targetSettings: any = overlaySettings.TargetInfo
    targetSettings.mode = targetSettings.mode or "Smart Priority"
    targetSettings.showAvatar = targetSettings.showAvatar ~= false
    targetSettings.showHealth = targetSettings.showHealth ~= false
    targetSettings.showDistance = targetSettings.showDistance ~= false
    local radarSettings: any = overlaySettings.Radar
    radarSettings.range = math.clamp(tonumber(radarSettings.range) or 250, 25, 2000)
    radarSettings.size = math.clamp(tonumber(radarSettings.size) or 190, 140, 280)
    radarSettings.rotation = radarSettings.rotation or "Camera"
    radarSettings.colorMode = radarSettings.colorMode or "Smart"
    radarSettings.outOfRange = radarSettings.outOfRange or "Clamp"
    radarSettings.showNames = radarSettings.showNames == true
    radarSettings.showDistance = radarSettings.showDistance == true
    radarSettings.showTeam = radarSettings.showTeam ~= false
    radarSettings.aliveOnly = radarSettings.aliveOnly ~= false
    radarSettings.grid = radarSettings.grid ~= false
    radarSettings.rings = radarSettings.rings ~= false

    local function getFloatingViewport(): Vector2
        local stable: any = state.layoutViewport
        if stable and stable.X > 1 and stable.Y > 1 then return stable end
        local camera: Camera? = workspace.CurrentCamera
        local current: any = camera and camera.ViewportSize
        if current and current.X > 1 and current.Y > 1 then return current end
        return Vector2.new(1280, 720)
    end

    local function clampFloatingPosition(
        position: Vector2,
        size: Vector2
    ): Vector2
        local viewport: Vector2 = getFloatingViewport()
        return Vector2.new(
            math.clamp(position.X, 6, math.max(6, viewport.X - size.X - 6)),
            math.clamp(position.Y, 6, math.max(6, viewport.Y - size.Y - 6))
        )
    end

    local function bringFloatingToFront(record: FloatingWindowRecord): ()
        state.windows.Raise(record.id)
    end

    local function createFloatingWindow(
        id: string,
        titleText: string,
        size: Vector2,
        defaultPosition: Vector2,
        onClose: (() -> ())?,
        chromeless: boolean?
    ): FloatingWindowRecord
        local bare: boolean = chromeless == true
        local headerHeight: number = bare and 0 or 30
        local saved: any = configData.ui.floatingPositions[id]
        local position: Vector2 = type(saved) == "table"
            and Vector2.new(
                tonumber(saved[1]) or defaultPosition.X,
                tonumber(saved[2]) or defaultPosition.Y
            )
            or defaultPosition
        position = clampFloatingPosition(position, size)

        local root: Frame = create("Frame", {
            Parent = PopupLayer,
            Name = "Floating_" .. id,
            Active = true,
            BackgroundColor3 = Theme.background,
            BackgroundTransparency = bare and 1 or 0.08,
            BorderSizePixel = 0,
            ClipsDescendants = not bare,
            Position = UDim2.fromOffset(position.X, position.Y),
            Size = UDim2.fromOffset(size.X, size.Y),
            Visible = false,
            ZIndex = 150,
        }) :: Frame
        bind(root, "BackgroundColor3", "background")
        if not bare then
            create("UICorner", {
                Parent = root,
                CornerRadius = cornerRadius(UI_RADIUS.panel),
            })
            local edge: UIStroke = create("UIStroke", {
                Parent = root,
                Name = "WindowOutline",
                Color = Theme.outline,
                Transparency = 0.08,
                Thickness = 1,
            }) :: UIStroke
            bind(edge, "Color", "outline")

            local shade: Frame = create("Frame", {
                Parent = root,
                Name = "Readability",
                BackgroundColor3 = Theme.background,
                BackgroundTransparency = 0.28,
                BorderSizePixel = 0,
                Size = UDim2.fromScale(1, 1),
            }) :: Frame
            bind(shade, "BackgroundColor3", "background")
            shade:SetAttribute("FloatingDepth", 2)
        end

        local header: Frame = create("Frame", {
            Parent = root,
            Name = "Header",
            Active = not bare,
            BackgroundColor3 = Theme.surface,
            BackgroundTransparency = 0.24,
            BorderSizePixel = 0,
            Size = UDim2.new(1, 0, 0, headerHeight),
            Visible = not bare,
        }) :: Frame
        bind(header, "BackgroundColor3", "surface")
        header:SetAttribute("FloatingDepth", 3)

        local closeButton: TextButton
        if bare then

            closeButton = create("TextButton", {
                TextTruncate = Enum.TextTruncate.AtEnd,
                ClipsDescendants = true,
                Parent = header,
                Name = "Close",
                BackgroundTransparency = 1,
                Text = "",
                Visible = false,
            }) :: TextButton
        else
            local accent: Frame = create("Frame", {
                Parent = header,
                BackgroundColor3 = Theme.outline,
                BackgroundTransparency = 0.62,
                BorderSizePixel = 0,
                Position = UDim2.fromOffset(0, headerHeight - 1),
                Size = UDim2.new(1, 0, 0, 1),
            }) :: Frame
            bind(accent, "BackgroundColor3", "outline")
            accent:SetAttribute("FloatingDepth", 1)
            local title: TextLabel = makeTextLabel(header, titleText, 10)
            title.FontFace = CONTROL_FONT
            title.Position = UDim2.fromOffset(10, 0)
            title.Size = UDim2.new(1, -40, 1, 0)
            title.TextColor3 = Theme.text
            bind(title, "TextColor3", "text")
            title:SetAttribute("FloatingDepth", 2)

            closeButton = makeButton(header, "x", 9)
            closeButton.Name = "Close"
            closeButton.AnchorPoint = Vector2.new(1, 0.5)
            closeButton.BackgroundTransparency = 1
            closeButton.Position = UDim2.new(1, -6, 0.5, 0)
            closeButton.Size = UDim2.fromOffset(22, 22)
            closeButton.TextColor3 = Theme.textMuted
            closeButton:SetAttribute("ThemeRole", "Icon")
            closeButton:SetAttribute("FloatingDepth", 3)
            local closeStroke: UIStroke? =
                closeButton:FindFirstChild("StyleStroke") :: UIStroke?
            if closeStroke then
                closeStroke.Enabled = false
            end
        end

        local body: Frame = create("Frame", {
            Parent = root,
            Name = "Body",
            Active = false,
            BackgroundTransparency = 1,
            BorderSizePixel = 0,
            Position = UDim2.fromOffset(0, headerHeight),
            Size = UDim2.new(1, 0, 1, -headerHeight),
        }) :: Frame
        body:SetAttribute("FloatingDepth", 4)

        if bare then
            root.Active = state.visible == true

            create("UICorner", {
                Parent = root,
                CornerRadius = cornerRadius(UI_RADIUS.control),
            })
            local hint: UIStroke = create("UIStroke", {
                Parent = root,
                Name = "MoveHint",
                Color = Theme.outline,
                Thickness = 1,
                Transparency = state.visible and 0.72 or 1,
            }) :: UIStroke
            bind(hint, "Color", "outline")
            state.addMenuVisibilityListener(function(visible: boolean): ()
                if not root.Parent then
                    return
                end
                root.Active = visible
                hint.Transparency = visible and 0.72 or 1
            end)
        end

        local record: FloatingWindowRecord = {
            id = id,
            root = root,
            body = body,
            closeButton = closeButton,
        }

        state.windows.Adopt(root, {
            id = id,
            body = body,
            header = header,
            closeButton = closeButton,
            headerHeight = headerHeight,
            dragHandle = bare and root or header,
            requiresMenuOpen = bare,
            layered = true,
            onClose = onClose,
        })
        return record
    end

    function FloatingUi.reclamp(): ()
        state.windows.Reclamp()
    end

    state.onLayout(function(): ()
        FloatingUi.reclamp()
    end)

    local FavoritesWindow: FloatingWindowRecord = createFloatingWindow(
        "Favorites",
        "Favorites",
        Vector2.new(state.layout.compact and 240 or 270, 236),
        Vector2.new(40, 90),
        function(): ()
            configData.ui.favoritesWindowOpen = false
            queueConfigSave()
        end
    )
    local FavoritesList: ScrollingFrame = create("ScrollingFrame", {
        Parent = FavoritesWindow.body,
        Name = "FavoriteModules",
        Active = true,
        AutomaticCanvasSize = Enum.AutomaticSize.Y,
        BackgroundTransparency = 1,
        BorderSizePixel = 0,
        CanvasSize = UDim2.fromOffset(0, 0),
        Position = UDim2.fromOffset(8, 8),
        ScrollBarImageColor3 = Theme.outline,
        ScrollBarThickness = 3,
        Size = UDim2.new(1, -16, 1, -16),
    }) :: ScrollingFrame
    FavoritesList:SetAttribute("FloatingDepth", 2)
    create("UIListLayout", {
        Parent = FavoritesList,
        Padding = UDim.new(0, 5),
        SortOrder = Enum.SortOrder.LayoutOrder,
    })

    local function syncFavoriteButton(feature: any): ()
        if not feature.favoriteButton then
            return
        end
        local selected: boolean = configData.ui.favorites[feature.configKey] == true
        feature.favoriteButton.Text = selected and "★" or "☆"
        feature.favoriteButton.TextColor3 = selected
            and Theme.accent
            or Theme.textMuted
        feature.favoriteButton.TextTransparency = selected and 0 or 0.34
    end

    local function refreshFavorites(): ()
        for _, row: GuiObject in ipairs(FloatingUi.favoriteRows) do
            row:Destroy()
        end
        table.clear(FloatingUi.favoriteRows)
        local order: number = 0
        for _, feature: any in ipairs(allFeatures) do
            if configData.ui.favorites[feature.configKey] == true then
                order += 1
                local row: TextButton = create("TextButton", {
                    TextTruncate = Enum.TextTruncate.AtEnd,
                    ClipsDescendants = true,
                    Parent = FavoritesList,
                    Name = feature.configKey:gsub("%W", ""),
                    AutoButtonColor = false,
                    BackgroundColor3 = Theme.surface,
                    BackgroundTransparency = 0.38,
                    BorderSizePixel = 0,
                    FontFace = THIN_FONT,
```

```
                    LayoutOrder = order,
                    Size = UDim2.new(1, -4, 0, 36),
                    Text = feature.name,
                    TextColor3 = feature.enabled
                        and Theme.accent
                        or Theme.text,
                    TextSize = 13,
                    TextXAlignment = Enum.TextXAlignment.Left,
                }) :: TextButton
                create("UICorner", {
                    Parent = row,
                    CornerRadius = cornerRadius(8),
                })
                row:SetAttribute("FloatingDepth", 3)
                create("UIPadding", {
                    Parent = row,
                    PaddingLeft = UDim.new(0, 12),
                    PaddingRight = UDim.new(0, 38),
                })
                local status: Frame = create("Frame", {
                    Parent = row,
                    Name = "Status",
                    AnchorPoint = Vector2.new(1, 0.5),
                    BackgroundColor3 = feature.enabled
                        and Theme.accent
                        or Theme.textMuted,
                    BackgroundTransparency = feature.enabled and 0 or 0.58,
                    BorderSizePixel = 0,
                    Position = UDim2.new(1, -14, 0.5, 0),
                    Size = UDim2.fromOffset(feature.enabled and 9 or 7, feature.enabled and 9 or 7),
                }) :: Frame
                create("UICorner", {Parent = status, CornerRadius = UDim.new(1, 0)})
                status:SetAttribute("FloatingDepth", 2)
                row.MouseButton1Click:Connect(function(): ()
                    if feature.isCategory then
                        state.setMenuVisible(true)
                    end
                    feature.activate()
                end)
                table.insert(FloatingUi.favoriteRows, row)
            end
        end
        if order == 0 then
            local empty: TextLabel = create("TextLabel", {
                TextTruncate = Enum.TextTruncate.AtEnd,
                ClipsDescendants = true,
                Parent = FavoritesList,
                BackgroundTransparency = 1,
                FontFace = THIN_FONT,
                LayoutOrder = 1,
                Size = UDim2.new(1, -4, 0, 52),
                Text = "Star a module to keep it here",
                TextColor3 = Theme.textMuted,
                TextSize = 13,
                TextTransparency = 0.28,
                TextXAlignment = Enum.TextXAlignment.Center,
            }) :: TextLabel
            empty:SetAttribute("FloatingDepth", 3)
            table.insert(FloatingUi.favoriteRows, empty)
        end
    end

    function FloatingUi.toggleFavorite(feature: any): ()
        local nextValue: boolean = configData.ui.favorites[feature.configKey] ~= true
        configData.ui.favorites[feature.configKey] = nextValue and true or nil
        syncFavoriteButton(feature)
        refreshFavorites()
        queueConfigSave()
    end

    function FloatingUi.registerFeature(feature: any): ()
        FloatingUi.favorites[feature.configKey] = feature
        syncFavoriteButton(feature)
        if configData.ui.favorites[feature.configKey] == true then
            refreshFavorites()
        end
    end

    function FloatingUi.unregisterFeature(feature: any): ()
        FloatingUi.favorites[feature.configKey] = nil
        if configData.ui.favorites[feature.configKey] == true then
            refreshFavorites()
        end
    end

    function FloatingUi.refreshFeature(feature: any): ()
        if configData.ui.favorites[feature.configKey] == true then
            refreshFavorites()
        end
    end

    local OverlayMenu: FloatingWindowRecord = createFloatingWindow(
        "OverlayMenu",
        "Overlays",
        Vector2.new(state.layout.compact and 232 or 252, 220),
        Vector2.new(340, 90),
        function(): ()
            configData.ui.overlayMenuOpen = false
            queueConfigSave()
        end
    )
    local OverlayList: Frame = create("Frame", {
        Parent = OverlayMenu.body,
        BackgroundTransparency = 1,
        Position = UDim2.fromOffset(8, 8),
        Size = UDim2.new(1, -16, 1, -16),
    }) :: Frame
    OverlayList:SetAttribute("FloatingDepth", 2)
    create("UIListLayout", {
        Parent = OverlayList,
        Padding = UDim.new(0, 5),
        SortOrder = Enum.SortOrder.LayoutOrder,
    })

    local overlayDefinitions: {any} = {
        {id = "SessionInfo", title = "Session Info", size = Vector2.new(210, 118), position = Vector2.new(24, 300)},
        {id = "TextGUI", title = "Text GUI", size = Vector2.new(240, 220), position = Vector2.new(250, 300)},
        {id = "TargetInfo", title = "Target Info", size = Vector2.new(260, 86), position = Vector2.new(510, 300)},
        {id = "Radar", title = "Radar", size = Vector2.new(radarSettings.size, radarSettings.size), position = Vector2.new(820, 280)},
    }

    local function setOverlayEnabled(id: string, enabled: boolean): ()
        local overlay: any = FloatingUi.overlays[id]
        if not overlay then
            return
        end
        overlay.enabled = enabled
        overlay.window.root.Visible = enabled
        setToggleSwitch(overlay.toggle, enabled, true)
        configData.ui.overlays[id] = enabled
        queueConfigSave()
        if enabled then
            bringFloatingToFront(overlay.window)
        end
    end

    FloatingUi.settingsWindows = {}
    FloatingUi.openSettings = function(id: string): ()
        local window: FloatingWindowRecord? = FloatingUi.settingsWindows[id]
        if window then
            window.root.Visible = not window.root.Visible
            if window.root.Visible then
                bringFloatingToFront(window)
            end
        end
    end

    for index: number, definition: any in ipairs(overlayDefinitions) do
        local overlayId: string = definition.id
        local overlayWindow: FloatingWindowRecord
        overlayWindow = createFloatingWindow(
            overlayId,
            definition.title,
            definition.size,
            definition.position,
            function(): ()
                setOverlayEnabled(overlayId, false)
            end,
            true
        )
        local managerRow: Frame = create("Frame", {
            Parent = OverlayList,
            BackgroundColor3 = Theme.surface,
            BackgroundTransparency = 0.48,
            BorderSizePixel = 0,
            LayoutOrder = index,
            Size = UDim2.new(1, 0, 0, 40),
        }) :: Frame
        managerRow:SetAttribute("FloatingDepth", 3)
        create("UICorner", {
            Parent = managerRow,
            CornerRadius = cornerRadius(8),
        })
        local managerLabel: TextLabel = makeTextLabel(managerRow, definition.title, 10)
        managerLabel.Position = UDim2.fromOffset(10, 0)
        managerLabel.Size = UDim2.new(1, -112, 1, 0)
        managerLabel.TextColor3 = Theme.textMuted
        managerLabel:SetAttribute("FloatingDepth", 1)
        local settingsButton: TextButton = makeButton(managerRow, "", 9)
        settingsButton.Name = "Settings"

        state.applyLinesGlyph(settingsButton, false)
        settingsButton.AnchorPoint = Vector2.new(1, 0.5)
        settingsButton.BackgroundTransparency = 1
        settingsButton.Position = UDim2.new(1, -58, 0.5, 0)
        settingsButton.Size = UDim2.fromOffset(28, 28)
        settingsButton:SetAttribute("FloatingDepth", 2)
        local settingsStroke: UIStroke? =
            settingsButton:FindFirstChild("StyleStroke") :: UIStroke?
        if settingsStroke then
            settingsStroke.Enabled = false
        end
        local toggle: TextButton = createToggleSwitch(managerRow, false, 42)
        toggle.AnchorPoint = Vector2.new(1, 0.5)
        toggle.Position = UDim2.new(1, -7, 0.5, 0)
        toggle:SetAttribute("FloatingDepth", 2)
        FloatingUi.overlays[overlayId] = {
            window = overlayWindow,
            toggle = toggle,
            enabled = false,
        }
        trackUiConnection(toggle.MouseButton1Click:Connect(function(): ()
            setOverlayEnabled(overlayId, not FloatingUi.overlays[overlayId].enabled)
        end))
        trackUiConnection(settingsButton.MouseButton1Click:Connect(function(): ()
            FloatingUi.openSettings(overlayId)
        end))
    end

    local function createOverlaySettingsWindow(
        id: string,
        title: string,
        height: number,
        position: Vector2
    ): (FloatingWindowRecord, ScrollingFrame)

        local window: FloatingWindowRecord = createFloatingWindow(
            id .. "Settings",
            title,
            Vector2.new(
                state.layout.compact and 268 or 292,
                math.min(height, math.max(180, getFloatingViewport().Y - 24))
            ),
            position,
            nil
        )
        local scroll: ScrollingFrame = create("ScrollingFrame", {
            Parent = window.body,
            Active = true,
            AutomaticCanvasSize = Enum.AutomaticSize.Y,
            BackgroundTransparency = 1,
            BorderSizePixel = 0,
            CanvasSize = UDim2.fromOffset(0, 0),
            Position = UDim2.fromOffset(8, 8),
            ScrollBarImageColor3 = Theme.accentDim,
            ScrollBarThickness = 3,
            Size = UDim2.new(1, -16, 1, -16),
        }) :: ScrollingFrame
        scroll:SetAttribute("FloatingDepth", 2)
        create("UIListLayout", {
            Parent = scroll,
            Padding = UDim.new(0, 5),
            SortOrder = Enum.SortOrder.LayoutOrder,
        })
        FloatingUi.settingsWindows[id] = window
        return window, scroll
    end

    local function createCompactSettingRow(parent: Instance, labelText: string): Frame
        local row: Frame = create("Frame", {
            Parent = parent,
            BackgroundColor3 = Theme.surface,
            BackgroundTransparency = 0.44,
            BorderSizePixel = 0,
            Size = UDim2.new(1, -4, 0, 38),
        }) :: Frame
        create("UICorner", {Parent = row, CornerRadius = cornerRadius(8)})
        local label: TextLabel = makeTextLabel(row, labelText, 9)
        label.Position = UDim2.fromOffset(10, 0)
        label.Size = UDim2.new(0.48, -10, 1, 0)
        label.TextColor3 = Theme.textMuted
        return row
    end

    local function addCompactCycle(
        parent: Instance,
        labelText: string,
        values: {string},
        currentValue: string,
        callback: (string) -> ()
    ): ()
        local row: Frame = createCompactSettingRow(parent, labelText)
        local index: number = table.find(values, currentValue) or 1
        local button: TextButton = makeButton(row, values[index], 8)
        button.Position = UDim2.new(0.48, 4, 0, 4)
        button.Size = UDim2.new(0.52, -8, 1, -8)
        trackUiConnection(button.MouseButton1Click:Connect(function(): ()
            index = index % #values + 1
            button.Text = values[index]
            callback(values[index])
            queueConfigSave()
        end))
    end

    local function addCompactToggle(
        parent: Instance,
        labelText: string,
        initial: boolean,
        callback: (boolean) -> ()
    ): ()
        local row: Frame = createCompactSettingRow(parent, labelText)
        local enabled: boolean = initial
        local toggle: TextButton = createToggleSwitch(row, enabled, 42)
        toggle.AnchorPoint = Vector2.new(1, 0.5)
        toggle.Position = UDim2.new(1, -8, 0.5, 0)
        trackUiConnection(toggle.MouseButton1Click:Connect(function(): ()
            enabled = not enabled
            setToggleSwitch(toggle, enabled, true)
            callback(enabled)
            queueConfigSave()
        end))
    end

    local function addCompactText(
        parent: Instance,
        labelText: string,
        initial: string,
        callback: (string) -> ()
    ): ()
        local row: Frame = createCompactSettingRow(parent, labelText)
        local box: TextBox = makeTextBox(row, initial)
        box.Position = UDim2.new(0.48, 4, 0, 4)
        box.Size = UDim2.new(0.52, -8, 1, -8)
        box.PlaceholderText = "Custom text"
        box.TextXAlignment = Enum.TextXAlignment.Left
        trackUiConnection(box.FocusLost:Connect(function(): ()
            callback(string.sub(box.Text, 1, 80))
            queueConfigSave()
        end))
    end

    do
        local textSettingsScroll: ScrollingFrame
        local _textSettingsWindow: FloatingWindowRecord
        _textSettingsWindow, textSettingsScroll = createOverlaySettingsWindow(
            "TextGUI",
            "Text GUI settings",
            state.layout.compact and 500 or 540,
            Vector2.new(300, 70)
        )
        addCompactCycle(textSettingsScroll, "Sort", {"Alphabetical", "Length", "Enabled order", "Section"}, textGuiSettings.sort, function(value: string): () textGuiSettings.sort = value end)
        addCompactCycle(textSettingsScroll, "Font", {"Builder Sans", "Source Sans", "Gotham", "Code", "Valve"}, textGuiSettings.font, function(value: string): () textGuiSettings.font = value end)
        addCompactCycle(textSettingsScroll, "Color mode", {"Match GUI", "Solid white", "Greyscale", "Rainbow", "By section"}, textGuiSettings.colorMode, function(value: string): () textGuiSettings.colorMode = value end)
        addCompactCycle(textSettingsScroll, "Scale", {"0.75", "1", "1.25", "1.5", "2"}, tostring(textGuiSettings.scale), function(value: string): () textGuiSettings.scale = tonumber(value) or 1 end)
        addCompactToggle(textSettingsScroll, "Shadow", textGuiSettings.shadow, function(value: boolean): () textGuiSettings.shadow = value end)
        addCompactToggle(textSettingsScroll, "Gradient", textGuiSettings.gradient, function(value: boolean): () textGuiSettings.gradient = value end)
        addCompactToggle(textSettingsScroll, "V4 gradient", textGuiSettings.v4Gradient, function(value: boolean): () textGuiSettings.v4Gradient = value end)
        addCompactToggle(textSettingsScroll, "Animations", textGuiSettings.animations, function(value: boolean): () textGuiSettings.animations = value end)
        addCompactToggle(textSettingsScroll, "Watermark", textGuiSettings.watermark, function(value: boolean): () textGuiSettings.watermark = value end)
        addCompactToggle(textSettingsScroll, "Render background", textGuiSettings.background, function(value: boolean): () textGuiSettings.background = value end)
        addCompactToggle(textSettingsScroll, "Hide modules", textGuiSettings.hideModules, function(value: boolean): () textGuiSettings.hideModules = value end)
        addCompactToggle(textSettingsScroll, "Hide render", textGuiSettings.hideRender, function(value: boolean): () textGuiSettings.hideRender = value end)
        addCompactText(textSettingsScroll, "Custom text", textGuiSettings.customText, function(value: string): () textGuiSettings.customText = value end)
    end

    do
        local radarSettingsScroll: ScrollingFrame
        local _radarSettingsWindow: FloatingWindowRecord
        _radarSettingsWindow, radarSettingsScroll = createOverlaySettingsWindow(
            "Radar",
            "Radar settings",
            500,
            Vector2.new(630, 70)
        )
        addCompactCycle(radarSettingsScroll, "Range", {"75", "150", "250", "500", "1000", "2000"}, tostring(radarSettings.range), function(value: string): () radarSettings.range = tonumber(value) or 250 end)
        addCompactCycle(radarSettingsScroll, "Radar size", {"160", "190", "220", "250", "280"}, tostring(radarSettings.size), function(value: string): () radarSettings.size = tonumber(value) or 190 end)
        addCompactCycle(radarSettingsScroll, "Rotation", {"Camera", "Player", "North"}, radarSettings.rotation, function(value: string): () radarSettings.rotation = value end)
        addCompactCycle(radarSettingsScroll, "Color mode", {"Smart", "Team", "Health", "MM2 role", "Spectrum"}, radarSettings.colorMode, function(value: string): () radarSettings.colorMode = value end)
        addCompactCycle(radarSettingsScroll, "Out of range", {"Clamp", "Hide"}, radarSettings.outOfRange, function(value: string): () radarSettings.outOfRange = value end)
        addCompactToggle(radarSettingsScroll, "Alive only", radarSettings.aliveOnly, function(value: boolean): () radarSettings.aliveOnly = value end)
        addCompactToggle(radarSettingsScroll, "Show team", radarSettings.showTeam, function(value: boolean): () radarSettings.showTeam = value end)
        addCompactToggle(radarSettingsScroll, "Show names", radarSettings.showNames, function(value: boolean): () radarSettings.showNames = value end)
        addCompactToggle(radarSettingsScroll, "Show distance", radarSettings.showDistance, function(value: boolean): () radarSettings.showDistance = value end)
        addCompactToggle(radarSettingsScroll, "Grid", radarSettings.grid, function(value: boolean): () radarSettings.grid = value end)
        addCompactToggle(radarSettingsScroll, "Range rings", radarSettings.rings, function(value: boolean): () radarSettings.rings = value end)
    end

    do
        local sessionSettingsScroll: ScrollingFrame
        local _sessionSettingsWindow: FloatingWindowRecord
        _sessionSettingsWindow, sessionSettingsScroll = createOverlaySettingsWindow(
            "SessionInfo",
            "Session Info settings",
            310,
            Vector2.new(960, 70)
        )
        addCompactToggle(sessionSettingsScroll, "Performance", sessionSettings.showPerformance, function(value: boolean): () sessionSettings.showPerformance = value end)
        addCompactToggle(sessionSettingsScroll, "Server data", sessionSettings.showServer, function(value: boolean): () sessionSettings.showServer = value end)
        addCompactToggle(sessionSettingsScroll, "Role and team", sessionSettings.showRole, function(value: boolean): () sessionSettings.showRole = value end)
        addCompactToggle(sessionSettingsScroll, "Deaths", sessionSettings.showDeaths, function(value: boolean): () sessionSettings.showDeaths = value end)
        addCompactToggle(sessionSettingsScroll, "Wins / rounds", sessionSettings.showWins, function(value: boolean): () sessionSettings.showWins = value end)
    end

    do
        local targetSettingsScroll: ScrollingFrame
        local _targetSettingsWindow: FloatingWindowRecord
        _targetSettingsWindow, targetSettingsScroll = createOverlaySettingsWindow(
            "TargetInfo",
            "Target Priority",
            280,
            Vector2.new(960, 395)
        )
        addCompactCycle(targetSettingsScroll, "Selection", {"Smart Priority", "Nearest cursor", "Nearest distance"}, targetSettings.mode, function(value: string): () targetSettings.mode = value end)
        addCompactToggle(targetSettingsScroll, "Avatar portrait", targetSettings.showAvatar, function(value: boolean): () targetSettings.showAvatar = value end)
        addCompactToggle(targetSettingsScroll, "Health bar", targetSettings.showHealth, function(value: boolean): () targetSettings.showHealth = value end)
        addCompactToggle(targetSettingsScroll, "Distance", targetSettings.showDistance, function(value: boolean): () targetSettings.showDistance = value end)
    end

    local SessionShadow: TextLabel = makeTextLabel(
        FloatingUi.overlays.SessionInfo.window.body,
        "Session initializing...",
        9
    )
    SessionShadow.Name = "SessionShadow"
    SessionShadow.Position = UDim2.fromOffset(1, 1)
    SessionShadow.Size = UDim2.fromScale(1, 1)
    SessionShadow.TextColor3 = Color3.fromRGB(0, 0, 0)
    SessionShadow.TextTransparency = 0.25
    SessionShadow.TextYAlignment = Enum.TextYAlignment.Top
    SessionShadow:SetAttribute("FloatingDepth", 1)

    local SessionText: TextLabel = makeTextLabel(
        FloatingUi.overlays.SessionInfo.window.body,
        "Session initializing...",
        9
    )
    SessionText.Position = UDim2.fromOffset(0, 0)
    SessionText.Size = UDim2.fromScale(1, 1)
    SessionText.TextColor3 = Theme.text
    SessionText.TextYAlignment = Enum.TextYAlignment.Top
    SessionText:SetAttribute("FloatingDepth", 2)

    local TextGuiBody: Frame = FloatingUi.overlays.TextGUI.window.body
    local TextGuiShadow: TextLabel = makeTextLabel(
        TextGuiBody,
        "No active modules",
        10
    )
    TextGuiShadow.Position = UDim2.fromOffset(1, 1)
    TextGuiShadow.Size = UDim2.fromScale(1, 1)
    TextGuiShadow.TextColor3 = Color3.fromRGB(0, 0, 0)
    TextGuiShadow.TextTransparency = 0.16
    TextGuiShadow.TextWrapped = true
    TextGuiShadow.TextYAlignment = Enum.TextYAlignment.Top
    TextGuiShadow:SetAttribute("FloatingDepth", 1)
    local TextGuiLabel: TextLabel = makeTextLabel(
        FloatingUi.overlays.TextGUI.window.body,
        "No active modules",
        10
    )
    TextGuiLabel.Position = UDim2.fromOffset(0, 0)
    TextGuiLabel.Size = UDim2.fromScale(1, 1)
    TextGuiLabel.TextColor3 = Theme.accent
    TextGuiLabel.TextWrapped = true
    TextGuiLabel.TextYAlignment = Enum.TextYAlignment.Top
    TextGuiLabel:SetAttribute("FloatingDepth", 2)
    local TextGuiGradient: UIGradient = create("UIGradient", {
        Parent = TextGuiLabel,
        Color = ColorSequence.new({
            ColorSequenceKeypoint.new(0, Theme.accent),
            ColorSequenceKeypoint.new(0.5, Theme.accentDim),
            ColorSequenceKeypoint.new(1, Theme.accentFaint),
        }),
        Rotation = 90,
        Enabled = false,
    }) :: UIGradient

    local TargetBody: Frame = FloatingUi.overlays.TargetInfo.window.body

    local TargetPortraitFrame: Frame = create("Frame", {
        Parent = TargetBody,
        BackgroundColor3 = Theme.surface,
        BackgroundTransparency = 0.18,
        BorderSizePixel = 0,
        Position = UDim2.fromOffset(0, 0),
        Size = UDim2.fromOffset(56, 56),
    }) :: Frame
    TargetPortraitFrame:SetAttribute("FloatingDepth", 2)
    create("UICorner", {Parent = TargetPortraitFrame, CornerRadius = cornerRadius(UI_RADIUS.panel)})
    create("UIStroke", {
        Parent = TargetPortraitFrame,
        Color = Theme.outline,
        Transparency = 0.55,
        Thickness = 1,
    })
    local TargetPortrait: ImageLabel = create("ImageLabel", {
        Parent = TargetPortraitFrame,
        BackgroundTransparency = 1,
        BorderSizePixel = 0,
        Image = "",
        Position = UDim2.fromOffset(4, 4),
        ScaleType = Enum.ScaleType.Crop,
        Size = UDim2.new(1, -8, 1, -8),
        ZIndex = 3,
    }) :: ImageLabel
    TargetPortrait:SetAttribute("FloatingDepth", 3)
    create("UICorner", {Parent = TargetPortrait, CornerRadius = cornerRadius(10)})
    local TargetName: TextLabel = makeTextLabel(
        TargetBody,
        "No target",
        11
    )
    TargetName.FontFace = CONTROL_FONT
    TargetName.Position = UDim2.fromOffset(64, 0)
    TargetName.Size = UDim2.new(1, -64, 0, 22)
    TargetName:SetAttribute("FloatingDepth", 2)
    local TargetRole: TextLabel = makeTextLabel(TargetBody, "No priority target", 8)
    TargetRole.Position = UDim2.fromOffset(64, 21)
    TargetRole.Size = UDim2.new(1, -64, 0, 18)
    TargetRole.TextColor3 = Theme.accentDim
    TargetRole:SetAttribute("FloatingDepth", 2)
    local TargetHealthTrack: Frame = create("Frame", {
        Parent = TargetBody,
        BackgroundColor3 = Theme.background,
        BackgroundTransparency = 0.18,
        BorderSizePixel = 0,
        Position = UDim2.fromOffset(64, 42),
        Size = UDim2.new(1, -64, 0, 6),
    }) :: Frame
    TargetHealthTrack:SetAttribute("FloatingDepth", 2)
    create("UICorner", {Parent = TargetHealthTrack, CornerRadius = UDim.new(1, 0)})
    local TargetHealthFill: Frame = create("Frame", {
        Parent = TargetHealthTrack,
        BackgroundColor3 = Theme.accent,
        BorderSizePixel = 0,
        Size = UDim2.fromScale(0, 1),
    }) :: Frame
    TargetHealthFill:SetAttribute("FloatingDepth", 1)
    create("UICorner", {Parent = TargetHealthFill, CornerRadius = UDim.new(1, 0)})
    local TargetDetails: TextLabel = makeTextLabel(
        TargetBody,
        "Waiting for a nearby player",
        9
    )
    TargetDetails.Position = UDim2.fromOffset(0, 58)
    TargetDetails.Size = UDim2.new(1, 0, 0, 28)
    TargetDetails.TextColor3 = Theme.textMuted
    TargetDetails:SetAttribute("FloatingDepth", 2)

    local RadarSurface: Frame = create("Frame", {
        Parent = FloatingUi.overlays.Radar.window.body,
        Name = "Surface",
        AnchorPoint = Vector2.new(0.5, 0.5),
        BackgroundColor3 = Theme.surface,
        BackgroundTransparency = 0.34,
        BorderSizePixel = 0,
        Position = UDim2.fromScale(0.5, 0.5),
        Size = UDim2.fromOffset(radarSettings.size, radarSettings.size),
    }) :: Frame
    RadarSurface:SetAttribute("FloatingDepth", 2)
    create("UICorner", {Parent = RadarSurface, CornerRadius = UDim.new(1, 0)})
    create("UIStroke", {
        Parent = RadarSurface,
        Color = Theme.outline,
        Transparency = 0.55,
    })
    local radarAxes: {Frame} = {}
    for _, rotation: number in ipairs({0, 90}) do
        local axis: Frame = create("Frame", {
            Parent = RadarSurface,
            AnchorPoint = Vector2.new(0.5, 0.5),
            BackgroundColor3 = Theme.outline,
            BackgroundTransparency = 0.72,
            BorderSizePixel = 0,
            Position = UDim2.fromScale(0.5, 0.5),
            Rotation = rotation,
            Size = UDim2.new(1, -16, 0, 1),
        }) :: Frame
        axis:SetAttribute("FloatingDepth", 1)
        table.insert(radarAxes, axis)
    end
    local radarRings: {Frame} = {}
    for _, scale: number in ipairs({0.34, 0.67}) do
        local ring: Frame = create("Frame", {
            Parent = RadarSurface,
            AnchorPoint = Vector2.new(0.5, 0.5),
            BackgroundTransparency = 1,
            Position = UDim2.fromScale(0.5, 0.5),
            Size = UDim2.fromScale(scale, scale),
        }) :: Frame
        ring:SetAttribute("FloatingDepth", 1)
        create("UICorner", {Parent = ring, CornerRadius = UDim.new(1, 0)})
        create("UIStroke", {
            Parent = ring,
            Color = Theme.outline,
            Transparency = 0.7,
            Thickness = 1,
        })
        table.insert(radarRings, ring)
    end
    local RadarNorth: TextLabel = makeTextLabel(RadarSurface, "N", 8)
    RadarNorth.AnchorPoint = Vector2.new(0.5, 0)
    RadarNorth.Position = UDim2.new(0.5, 0, 0, 5)
    RadarNorth.Size = UDim2.fromOffset(20, 18)
    RadarNorth.TextColor3 = Theme.accent
    RadarNorth.TextXAlignment = Enum.TextXAlignment.Center
    RadarNorth:SetAttribute("FloatingDepth", 3)
    local RadarCenter: Frame = create("Frame", {
        Parent = RadarSurface,
        AnchorPoint = Vector2.new(0.5, 0.5),
        BackgroundColor3 = Theme.accent,
        BorderSizePixel = 0,
        Position = UDim2.fromScale(0.5, 0.5),
        Size = UDim2.fromOffset(7, 7),
    }) :: Frame
    RadarCenter:SetAttribute("FloatingDepth", 3)
    create("UICorner", {Parent = RadarCenter, CornerRadius = UDim.new(1, 0)})
    local radarBlips: {[Player]: any} = {}
    local sessionStartedAt: number = os.clock()
    local overlayElapsed: number = 1
    local sessionFps: number = 60
    local sessionStats: any = {
        deaths = 0,
        wins = 0,
        rounds = 0,
        winsAvailable = false,
        roundsAvailable = false,
        role = nil,
    }
    state.sessionInfo = {
        setRole = function(role: string?): () sessionStats.role = role end,
        addWin = function(): ()
            sessionStats.winsAvailable = true
            sessionStats.wins += 1
        end,
        addRound = function(): ()
            sessionStats.roundsAvailable = true
            sessionStats.rounds += 1
        end,
    }
    local deathConnection: RBXScriptConnection? = nil
    local sessionBindGeneration: number = 0
    local sessionTrackingActive: boolean = true
    local function bindSessionCharacter(character: Model): ()
        sessionBindGeneration += 1
        local generation: number = sessionBindGeneration
        if deathConnection then
            deathConnection:Disconnect()
            deathConnection = nil
        end
        local humanoid: Humanoid? = character:FindFirstChildOfClass("Humanoid")
            or character:WaitForChild("Humanoid", 8) :: Humanoid?
        if humanoid
            and sessionTrackingActive
            and generation == sessionBindGeneration
            and character == LocalPlayer.Character then
            deathConnection = humanoid.Died:Connect(function(): ()
                sessionStats.deaths += 1
            end)
        end
    end
    if LocalPlayer.Character then
        task.defer(bindSessionCharacter, LocalPlayer.Character)
    end
    trackUiConnection(LocalPlayer.CharacterAdded:Connect(bindSessionCharacter))

    state.destroySessionTracking = function(): ()
        sessionTrackingActive = false
        sessionBindGeneration += 1
        if deathConnection then
            deathConnection:Disconnect()
            deathConnection = nil
        end
    end
    local thumbnailCache: {[number]: string} = {}
    local thumbnailGeneration: number = 0
    local displayedTargetUserId: number = 0
    local currentOverlayTarget: Player? = nil

    local function requestTargetThumbnail(player: Player?): ()
        local userId: number = player and player.UserId or 0
        if userId == displayedTargetUserId then
            return
        end
```

```
        displayedTargetUserId = userId
        thumbnailGeneration += 1
        local generation: number = thumbnailGeneration
        TargetPortrait.Image = userId > 0 and (thumbnailCache[userId] or "") or ""
        if userId <= 0 or thumbnailCache[userId] then
            return
        end
        task.spawn(function(): ()
            local success: boolean, image: any, ready: any = pcall(
                Players.GetUserThumbnailAsync,
                Players,
                userId,
                Enum.ThumbnailType.HeadShot,
                Enum.ThumbnailSize.Size180x180
            )
            if success and ready == true and type(image) == "string" and image ~= "" then
                thumbnailCache[userId] = image
                if generation == thumbnailGeneration and TargetPortrait.Parent then
                    TargetPortrait.Image = image
                end
            elseif generation == thumbnailGeneration then
                task.delay(1, function(): ()
                    if generation == thumbnailGeneration then
                        displayedTargetUserId = 0
                    end
                end)
            end
        end)
    end

    local function getTextGuiFont(): Font
        if textGuiSettings.font == "Source Sans" then
            return Font.fromEnum(Enum.Font.SourceSans)
        elseif textGuiSettings.font == "Gotham" then
            return Font.fromEnum(Enum.Font.Gotham)
        elseif textGuiSettings.font == "Code" then
            return Font.fromEnum(Enum.Font.Code)
        elseif textGuiSettings.font == "Valve" then
            return VALVE_FONT
        end
        return CONTROL_FONT
    end

    -- v190: `StatsItem:GetValue()` needs the Plugin capability and Delta
    -- rejects it ("The current thread cannot call 'GetValue'"), one error per
    -- overlay sample. The overlay reads the engine value instead, and a failed
    -- read simply leaves the row blank.
    local function getSessionPing(): number?
        local success: boolean, value: any = pcall(function(): number
            local player: any = LocalPlayer or Players.LocalPlayer
            local direct: any = player and player.GetNetworkPing
            if type(direct) ~= "function" then
                return 0
            end
            local seconds: any = direct(player)
            return if type(seconds) == "number" then seconds * 1000 else 0
        end)
        if not success then return nil end
        local milliseconds: number? = tonumber(value)
        if not milliseconds or milliseconds <= 0 then return nil end
        return milliseconds
    end

    local lastTextGuiText: string = ""
    table.insert(ownedTasks, TaskManager:Connect(function(deltaTime: number): ()
        -- v139: the FPS average is computed once, in the scheduler
        -- (state.performance), so this overlay no longer duplicates the work.
        local performance: any = state.performance
        if performance and type(performance.fps) == "number" then
            sessionFps = performance.fps
        else
            sessionFps += ((1 / math.max(deltaTime, 1 / 240)) - sessionFps) * 0.12
        end
        -- v139: nothing to draw when every overlay is off. This used to keep
        -- rebuilding the session lines and the radar 8 times a second forever.
        if not (FloatingUi.overlays.SessionInfo.enabled
            or FloatingUi.overlays.TextGUI.enabled
            or FloatingUi.overlays.TargetInfo.enabled
            or FloatingUi.overlays.Radar.enabled) then
            overlayElapsed = 0
            return
        end
        overlayElapsed += deltaTime
        if overlayElapsed < 0.12 then
            return
        end
        overlayElapsed = 0
        local now: number = os.clock()
        local elapsed: number = math.max(0, math.floor(now - sessionStartedAt))

        if FloatingUi.overlays.SessionInfo.enabled then
            local role: string? = sessionStats.role
            if state.gameRoleProvider then
                local success: boolean, resolved: any = pcall(
                    state.gameRoleProvider,
                    LocalPlayer
                )
                if success and type(resolved) == "string" then
                    role = resolved
                end
            end
            local lines: {string} = {
                string.format(
                    "Injected  %02d:%02d:%02d",
                    math.floor(elapsed / 3600),
                    math.floor(elapsed / 60) % 60,
                    elapsed % 60
                ),
            }
            if sessionSettings.showDeaths then
                table.insert(lines, "Deaths    " .. tostring(sessionStats.deaths))
            end
            if sessionSettings.showWins then
                table.insert(lines, "Wins      " .. (sessionStats.winsAvailable and tostring(sessionStats.wins) or "N/A"))
                table.insert(lines, "Rounds    " .. (sessionStats.roundsAvailable and tostring(sessionStats.rounds) or "N/A"))
            end
            if sessionSettings.showRole then
                table.insert(lines, "Role      " .. tostring(role or "N/A"))
                table.insert(lines, "Team      " .. tostring(LocalPlayer.Team and LocalPlayer.Team.Name or "None"))
            end
            if sessionSettings.showPerformance then
                local ping: number? = getSessionPing()
                table.insert(lines, string.format("FPS       %.0f", math.clamp(sessionFps, 0, 999)))
                table.insert(lines, "Ping      " .. (ping and string.format("%.0f ms", ping) or "N/A"))
                -- Sampled scheduler costs; the worst callback in the window
                -- is independent of the longest frame interval.
                local cost: any = state.performance
                if cost and type(cost.tasksMs) == "number" then
                    table.insert(lines, string.format(
                        "Menu sample %.2f+%.2f ms",
                        cost.tasksMs,
                        cost.renderMs
                    ))
                end
                if cost and (cost.spikeCount or 0) > 0 then
                    table.insert(lines, string.format(
                        "Spikes    %d · worst %.0f ms",
                        cost.spikeCount,
                        cost.worstSpikeMs or 0
                    ))
                end
                if cost and cost.peakTask then
                    table.insert(lines, string.format(
                        "Task peak %s · %.2f ms",
                        tostring(cost.peakTask),
                        tonumber(cost.peakTaskMs) or 0
                    ))
                end
            end
            if sessionSettings.showServer then
                table.insert(lines, string.format("Players   %d/%d", #Players:GetPlayers(), Players.MaxPlayers))
                table.insert(lines, "Game      " .. tostring(state.activeGameName or "Universal"))
            end
            SessionText.Text = table.concat(lines, "\n")
        end

        if FloatingUi.overlays.TextGUI.enabled then
            local enabledFeatures: {any} = {}
            if not textGuiSettings.hideModules then
                for _, feature: any in ipairs(allFeatures) do
                    local lowerName: string = string.lower(feature.name)
                    local renderLike: boolean = string.find(lowerName, "esp", 1, true) ~= nil
                        or string.find(lowerName, "visual", 1, true) ~= nil
                        or string.find(lowerName, "x-ray", 1, true) ~= nil
                        or string.find(lowerName, "fov", 1, true) ~= nil
                        or string.find(lowerName, "fullbright", 1, true) ~= nil
                    if feature.enabled
                        and not feature.isCategory
                        and not feature.isAction
                        and (not textGuiSettings.hideRender or not renderLike) then
                        table.insert(enabledFeatures, feature)
                    end
                end
            end
            table.sort(enabledFeatures, function(left: any, right: any): boolean
                if textGuiSettings.sort == "Length" then
                    return #left.name > #right.name
                elseif textGuiSettings.sort == "Enabled order" then
                    return (left.enabledAt or 0) < (right.enabledAt or 0)
                elseif textGuiSettings.sort == "Section" then
                    local leftKey: string = left.sectionName .. left.name
                    local rightKey: string = right.sectionName .. right.name
                    return leftKey < rightKey
                end
                return string.lower(left.name) < string.lower(right.name)
            end)
            local textLines: {string} = {}
            if textGuiSettings.watermark then
                table.insert(textLines, "Wurst")
            end
            if textGuiSettings.customText ~= "" then
                table.insert(textLines, textGuiSettings.customText)
            end
            for _, feature: any in ipairs(enabledFeatures) do
                table.insert(textLines, feature.name)
            end
            if #textLines == 0 then
                table.insert(textLines, "No active modules")
            end
            local nextText: string = table.concat(textLines, "\n")
            if nextText ~= lastTextGuiText then
                lastTextGuiText = nextText
                TextGuiLabel.Text = nextText
                TextGuiShadow.Text = nextText
                if textGuiSettings.animations then
                    TextGuiLabel.TextTransparency = 0.5
                    TweenService:Create(TextGuiLabel, TweenInfo.new(0.2), {
                        TextTransparency = 0,
                    }):Play()
                end
            end
            TextGuiLabel.FontFace = getTextGuiFont()
            TextGuiShadow.FontFace = TextGuiLabel.FontFace
            TextGuiLabel.TextSize = math.round(14 * textGuiSettings.scale)
            TextGuiShadow.TextSize = TextGuiLabel.TextSize
            TextGuiShadow.Visible = textGuiSettings.shadow
            TextGuiBody.BackgroundColor3 = Theme.background
            TextGuiBody.BackgroundTransparency = textGuiSettings.background and 0.42 or 1
            local textWindowRoot: Frame = FloatingUi.overlays.TextGUI.window.root
            local textHeader: Frame? = textWindowRoot:FindFirstChild("Header") :: Frame?
            local textShade: Frame? = textWindowRoot:FindFirstChild("Readability") :: Frame?
            local textStroke: UIStroke? = textWindowRoot:FindFirstChildOfClass("UIStroke")
            if textHeader then textHeader.Visible = state.visible end
            if textShade then textShade.Visible = state.visible or textGuiSettings.background end
            if textStroke then textStroke.Transparency = state.visible and 0.38 or 1 end
            textWindowRoot.Active = state.visible
            textWindowRoot.BackgroundTransparency = state.visible and 0.08 or 1
            TextGuiBody.Position = state.visible and UDim2.fromOffset(0, 36)
                or UDim2.fromOffset(0, 0)
            TextGuiBody.Size = state.visible and UDim2.new(1, 0, 1, -36)
                or UDim2.fromScale(1, 1)
            local gradientEnabled: boolean = textGuiSettings.gradient
                or textGuiSettings.v4Gradient
                or textGuiSettings.colorMode == "Greyscale"
                or textGuiSettings.colorMode == "Rainbow"
            TextGuiGradient.Enabled = gradientEnabled
            if textGuiSettings.v4Gradient then
                TextGuiGradient.Color = ColorSequence.new({
                    ColorSequenceKeypoint.new(0, Theme.accent),
                    ColorSequenceKeypoint.new(0.34, Theme.accentDim),
                    ColorSequenceKeypoint.new(0.68, Theme.accentFaint),
                    ColorSequenceKeypoint.new(1, Theme.accent),
                })
                TextGuiGradient.Rotation = (now * 45) % 360
            elseif textGuiSettings.colorMode == "Rainbow" then
                local hue: number = (now * 0.12) % 1
                TextGuiLabel.TextColor3 = Color3.fromHSV(hue, 0.72, 1)
                TextGuiGradient.Enabled = false
            elseif textGuiSettings.colorMode == "Greyscale" then
                local pulse: number = 0.72 + 0.28 * math.sin(now * 1.6)
                TextGuiLabel.TextColor3 = Color3.fromHSV(0, 0, pulse)
                TextGuiGradient.Enabled = false
            elseif textGuiSettings.colorMode == "Solid white" then
                TextGuiLabel.TextColor3 = Theme.accent
            elseif textGuiSettings.colorMode == "By section" then
                TextGuiLabel.TextColor3 = Theme.accentDim
            else
                TextGuiLabel.TextColor3 = Theme.accent
                TextGuiGradient.Rotation = 90
            end

            local targetInfoEnabled: boolean = FloatingUi.overlays.TargetInfo.enabled
            local localCharacter: Model? = LocalPlayer.Character
            local localRoot: BasePart? = localCharacter
                and localCharacter:FindFirstChild("HumanoidRootPart") :: BasePart?
            local camera: Camera? = workspace.CurrentCamera
            local selectedTarget: Player? = nil
            local targetRole: string? = nil
            local targetSource: string = "Nearest player"
            local targetAccent: Color3 = Theme.accentDim
            if targetInfoEnabled
                and targetSettings.mode == "Smart Priority"
                and state.gameTargetProvider then
                local success: boolean, snapshot: any = pcall(state.gameTargetProvider)
                if success then
                    if typeof(snapshot) == "Instance" and snapshot:IsA("Player") then
                        selectedTarget = snapshot
                    elseif type(snapshot) == "table"
                        and typeof(snapshot.player) == "Instance"
                        and snapshot.player:IsA("Player") then
                        selectedTarget = snapshot.player
                        targetRole = snapshot.role
                        targetSource = tostring(snapshot.source or "Game priority")
                        if typeof(snapshot.accent) == "Color3" then
                            targetAccent = snapshot.accent
                        end
                    end
                end
            end
            if targetInfoEnabled and not selectedTarget and localRoot then
                local bestScore: number = math.huge
                local pointer: Vector2 = state.isMobile and getFloatingViewport() / 2
                    or UserInputService:GetMouseLocation()
                for _, player: Player in ipairs(Players:GetPlayers()) do
                    if player ~= LocalPlayer then
                        local character: Model? = player.Character
                        local humanoid: Humanoid? = character
                            and character:FindFirstChildOfClass("Humanoid") :: Humanoid?
                        local root: BasePart? = character
                            and character:FindFirstChild("HumanoidRootPart") :: BasePart?
                        if humanoid and humanoid.Health > 0 and root then
                            local distance: number = (root.Position - localRoot.Position).Magnitude
                            local score: number = distance
                            if targetSettings.mode ~= "Nearest distance" and camera then
                                local screenPoint: Vector3, visible: boolean =
                                    camera:WorldToViewportPoint(root.Position)
                                score = visible and (Vector2.new(screenPoint.X, screenPoint.Y) - pointer).Magnitude
                                    or math.huge
                            end
                            if score < bestScore then
                                bestScore = score
                                selectedTarget = player
                            end
                        end
                    end
                end
                targetSource = targetSettings.mode
            end
            currentOverlayTarget = targetInfoEnabled and selectedTarget or nil
            if selectedTarget and state.gameRoleProvider and not targetRole then
                local success: boolean, role: any = pcall(state.gameRoleProvider, selectedTarget)
                targetRole = success and type(role) == "string" and role or nil
            end
            if targetRole then
                targetAccent = targetRole == "Murderer" and Theme.negative
                    or ((targetRole == "Sheriff" or targetRole == "Hero")
                        and Theme.accent
                        or Theme.positive)
            end
            if targetInfoEnabled then
                local targetCharacter: Model? = selectedTarget and selectedTarget.Character
                local targetHumanoid: Humanoid? = targetCharacter
                    and targetCharacter:FindFirstChildOfClass("Humanoid") :: Humanoid?
                local targetRoot: BasePart? = targetCharacter
                    and targetCharacter:FindFirstChild("HumanoidRootPart") :: BasePart?
                if selectedTarget and targetHumanoid then
                    local ratio: number = math.clamp(
                        targetHumanoid.Health / math.max(1, targetHumanoid.MaxHealth),
                        0,
                        1
                    )
                    local distance: number? = localRoot and targetRoot
                        and (targetRoot.Position - localRoot.Position).Magnitude
                        or nil
                    TargetName.Text = selectedTarget.DisplayName .. "  @" .. selectedTarget.Name
                    TargetRole.Text = tostring(targetRole or "Player") .. "  ·  " .. targetSource
                    TargetRole.TextColor3 = targetAccent
                    TargetDetails.Text = string.format(
                        "Health %.0f / %.0f%s",
                        targetHumanoid.Health,
                        targetHumanoid.MaxHealth,
                        targetSettings.showDistance and distance
                            and string.format("  ·  %.0f studs", distance)
                            or ""
                    )
                    TargetHealthTrack.Visible = targetSettings.showHealth
                    TargetHealthFill.Size = UDim2.fromScale(ratio, 1)
                    TargetHealthFill.BackgroundColor3 = ratio > 0.5
                        and Theme.accent
                        or (ratio > 0.25 and Theme.accentDim or Theme.accentFaint)
                    TargetPortraitFrame.Visible = targetSettings.showAvatar
                    requestTargetThumbnail(selectedTarget)
                else
                    TargetName.Text = "No target"
                    TargetRole.Text = "No priority target"
                    TargetDetails.Text = "Waiting for a valid player"
                    TargetHealthFill.Size = UDim2.fromScale(0, 1)
                    TargetHealthTrack.Visible = targetSettings.showHealth
                    TargetPortraitFrame.Visible = targetSettings.showAvatar
                    requestTargetThumbnail(nil)
                end
            end
        end

        if FloatingUi.overlays.Radar.enabled then
            local radarSize: number = radarSettings.size
            RadarSurface.Size = UDim2.fromOffset(radarSize, radarSize)

            local radarWindow: FloatingWindowRecord = FloatingUi.overlays.Radar.window
            local nextRadarSize: UDim2 = UDim2.fromOffset(radarSize, radarSize)
            if radarWindow.root.Size ~= nextRadarSize then
                radarWindow.root.Size = nextRadarSize
                FloatingUi.reclamp()
            end
            for _, axis: Frame in ipairs(radarAxes) do axis.Visible = radarSettings.grid end
            for _, ring: Frame in ipairs(radarRings) do ring.Visible = radarSettings.rings end
            RadarNorth.Visible = radarSettings.rotation == "North"
            local activePlayers: {[Player]: boolean} = {}
            local basis: CFrame? = localRoot and localRoot.CFrame or nil
            if localRoot and radarSettings.rotation == "Camera" and camera then
                local look: Vector3 = Vector3.new(camera.CFrame.LookVector.X, 0, camera.CFrame.LookVector.Z)
                if look.Magnitude > 0.01 then
                    basis = CFrame.lookAt(localRoot.Position, localRoot.Position + look.Unit)
                end
            elseif localRoot and radarSettings.rotation == "North" then
                basis = CFrame.new(localRoot.Position)
            end
            for _, player: Player in ipairs(Players:GetPlayers()) do
                if player ~= LocalPlayer and localRoot and basis then
                    local character: Model? = player.Character
                    local humanoid: Humanoid? = character
                        and character:FindFirstChildOfClass("Humanoid") :: Humanoid?
                    local targetRoot: BasePart? = character
                        and character:FindFirstChild("HumanoidRootPart") :: BasePart?
                    local sameTeam: boolean = LocalPlayer.Team ~= nil and player.Team == LocalPlayer.Team
                    local allowed: boolean = targetRoot ~= nil
                        and (not radarSettings.aliveOnly or (humanoid ~= nil and humanoid.Health > 0))
                        and (radarSettings.showTeam or not sameTeam)
                    if allowed and targetRoot then
                        local relative: Vector3 = (basis :: CFrame):PointToObjectSpace(targetRoot.Position)
                        local distance: number = Vector2.new(relative.X, relative.Z).Magnitude
                        if distance <= radarSettings.range or radarSettings.outOfRange == "Clamp" then
                            activePlayers[player] = true
                            local record: any = radarBlips[player]
                            if not record then
                                local blip: Frame = create("Frame", {
                                    Parent = RadarSurface,
                                    AnchorPoint = Vector2.new(0.5, 0.5),
                                    BackgroundColor3 = Theme.accentFaint,
                                    BorderSizePixel = 0,
                                    Size = UDim2.fromOffset(7, 7),
                                }) :: Frame
                                blip:SetAttribute("FloatingDepth", 4)
                                create("UICorner", {Parent = blip, CornerRadius = UDim.new(1, 0)})
                                local targetRing: UIStroke = create("UIStroke", {
                                    Parent = blip,
                                    Color = Theme.accent,
                                    Transparency = 1,
                                    Thickness = 2,
                                }) :: UIStroke
                                local label: TextLabel = makeTextLabel(blip, "", 7)
                                label.AnchorPoint = Vector2.new(0.5, 0)
                                label.Position = UDim2.new(0.5, 0, 1, 3)
                                label.Size = UDim2.fromOffset(100, 18)
                                label.TextColor3 = Theme.text
                                label.TextXAlignment = Enum.TextXAlignment.Center
                                record = {blip = blip, label = label, ring = targetRing}
                                radarBlips[player] = record
                            end
                            local radius: number = radarSize / 2 - 12
                            local offset: Vector2 = Vector2.new(relative.X, relative.Z)
                                * (radius / radarSettings.range)
                            if offset.Magnitude > radius then offset = offset.Unit * radius end
                            record.blip.Position = UDim2.fromOffset(
                                radarSize / 2 + offset.X,
                                radarSize / 2 + offset.Y
                            )
                            local role: string? = nil
                            if state.gameRoleProvider then
                                local roleOk: boolean, roleValue: any = pcall(state.gameRoleProvider, player)
                                role = roleOk and type(roleValue) == "string" and roleValue or nil
                            end
                            local color: Color3 = Theme.accentFaint
                            if radarSettings.colorMode == "Health" and humanoid then
                                local healthRatio: number = humanoid.Health / math.max(1, humanoid.MaxHealth)
                                color = Color3.fromHSV(healthRatio * 0.34, 0.8, 1)
                            elseif radarSettings.colorMode == "Team" or radarSettings.colorMode == "Smart" then
                                color = sameTeam and Theme.positive or Theme.negative
                            end
                            if (radarSettings.colorMode == "MM2 role" or radarSettings.colorMode == "Smart") and role then
                                color = role == "Murderer" and Theme.negative
                                    or ((role == "Sheriff" or role == "Hero") and Theme.accent
                                        or Theme.positive)
                            elseif radarSettings.colorMode == "Spectrum" then
                                color = Color3.fromHSV((now * 0.1 + player.UserId % 10 / 10) % 1, 0.7, 1)
                            end
                            record.blip.BackgroundColor3 = color
                            record.ring.Transparency = player == currentOverlayTarget and 0 or 1
                            record.blip.Size = UDim2.fromOffset(
                                player == currentOverlayTarget and 10 or 7,
                                player == currentOverlayTarget and 10 or 7
                            )
                            record.label.Visible = radarSettings.showNames or radarSettings.showDistance
                            record.label.Text = (radarSettings.showNames and player.DisplayName or "")
                                .. (radarSettings.showDistance and string.format(" %.0f", distance) or "")
                        end
                    end
                end
            end
            for player: Player, record: any in pairs(radarBlips) do
                if not activePlayers[player] then
                    record.blip:Destroy()
                    radarBlips[player] = nil
                end
            end
        end
    end, nil, "Floating overlays"))

    FavoritesWindow.root.Visible = configData.ui.favoritesWindowOpen == true
    OverlayMenu.root.Visible = configData.ui.overlayMenuOpen == true
    for overlayId: string, overlay: any in pairs(FloatingUi.overlays) do
        setOverlayEnabled(overlayId, configData.ui.overlays[overlayId] == true)
    end
    refreshFavorites()

    activeCleanup = function(): ()
        for _, handle: any in ipairs(ownedTasks) do
            if handle and handle.Disconnect then
                handle:Disconnect()
            end
        end
        table.clear(ownedTasks)
        if type(state.destroySessionTracking) == "function" then
            state.destroySessionTracking()
        end
    end
    Module.Initialized = true
    return state.floatingUi
end

function Module.destroy(): ()
    if activeCleanup then
        pcall(activeCleanup)
    end
    activeCleanup = nil
    Module.Initialized = false
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/guis/Wurst/Code/FloatingWindows.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/guis/Wurst/Code/Furniture.lua
BYTES: 61199
SHA-256: 08a6ef60b88136f102275d280afd4063681ab63ac0c6edd7e9e1f597a2b970a6
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
local Module = {
    Name = "Furniture",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local activeCleanup: (() -> ())? = nil

local WURSTLOGO_BACKGROUND: Color3 = Color3.fromRGB(255, 255, 255)
local WURSTLOGO_TEXT: Color3 = Color3.fromRGB(0, 0, 0)

local FURNITURE_MARGIN_X: number = 14
local FURNITURE_MARGIN_Y: number = 10

local LOGO_HEIGHT: number = 36
local LOGO_WIDTH: number = 142

local HACKLIST_MODE: string = "Auto"
local HACKLIST_POSITION: string = "Left"
local HACKLIST_COLOR: Color3 = Color3.fromRGB(255, 255, 255)
local HACKLIST_SORT_BY: string = "Name"
local HACKLIST_REVERSE: boolean = false
local HACKLIST_ANIMATIONS: boolean = true

local HACKLIST_FONT_SIZE: number = 16
local HACKLIST_LINE_HEIGHT: number = 18
local HACKLIST_TOP: number = 56

local HACKLIST_SLIDE: number = 24

local WURSTLOGO_VISIBILITY: string = "Always"

local PILL_BUTTON: number = 32
local PILL_PADDING_X: number = 7
local PILL_PADDING_Y: number = 5
local PILL_GAP: number = 2
local TOOLTIP_DELAY_MS: number = 400

function Module.init(context: any): any
    local host: any = context
    local state: any = host.state
    local create: any = host.create
    local PRODUCT: any = host.PRODUCT
    local Theme: any = host.Theme
    local ThemeEngine: any = host.ThemeEngine
    local TaskManager: any = host.TaskManager
    local PopupLayer: any = host.PopupLayer
    local TITLE_FONT: any = host.TITLE_FONT
    local CONTROL_FONT: any = host.CONTROL_FONT
    local TweenService: any = host.TweenService
    local allFeatures: {any} = host.allFeatures
    local configData: any = host.configData
    local queueConfigSave: any = host.queueConfigSave
    local UserInputService: any = host.UserInputService
    local colorFromConfig: any = host.colorFromConfig
    local ScreenGui: any = host.ScreenGui
    local makeTextBox: any = host.makeTextBox
    local notify: any = host.notify or function(_message: any): () end

    local Furniture: any = {
        handles = {} :: {any},
        connections = {} :: {RBXScriptConnection},
    }

    for _, ghostName: string in ipairs({
        "WurstLogo",
        "HackList",
        "Navigator",
        "Pill",
    }) do
        local ghost: Instance? = PopupLayer:FindFirstChild(ghostName)
        while ghost do
            ghost:Destroy()
            ghost = PopupLayer:FindFirstChild(ghostName)
        end

        if ScreenGui then
            local screenGhost: Instance? = ScreenGui:FindFirstChild(ghostName)
            while screenGhost do
                screenGhost:Destroy()
                screenGhost = ScreenGui:FindFirstChild(ghostName)
            end
        end
    end

    local versionText: string = tostring(PRODUCT.version)
    local versionWidth: number = 8 + #versionText * 15
    local chipWidth: number = LOGO_WIDTH + 10 + versionWidth + 4
    local chip: Frame = (create("Frame", {
        Parent = PopupLayer,
        Name = "WurstLogo",
        Active = false,
        BackgroundTransparency = 1,
        BorderSizePixel = 0,
        Position = UDim2.fromOffset(FURNITURE_MARGIN_X, FURNITURE_MARGIN_Y),
        Size = UDim2.fromOffset(chipWidth, LOGO_HEIGHT + 8),
        ZIndex = 20,
    }) :: any) :: Frame

    local logoStripe: Frame = (create("Frame", {
        Parent = chip,
        Name = "BackgroundStripe",
        BackgroundColor3 = WURSTLOGO_BACKGROUND,
        BackgroundTransparency = 0.5,
        BorderSizePixel = 0,
        Position = UDim2.fromOffset(0, 12),
        Size = UDim2.fromOffset(chipWidth, 22),
        ZIndex = 20,
    }) :: any) :: Frame

    local logoImage: ImageLabel = (create("ImageLabel", {
        Parent = chip,
        Name = "Logo",
        BackgroundTransparency = 1,
        BorderSizePixel = 0,
        Image = "",
        Position = UDim2.fromOffset(4, 4),
        ScaleType = Enum.ScaleType.Fit,
        Size = UDim2.fromOffset(LOGO_WIDTH, LOGO_HEIGHT),
        Visible = false,
        ZIndex = 21,
    }) :: any) :: ImageLabel

    local logoFallback: TextLabel = (create("TextLabel", {
        TextTruncate = Enum.TextTruncate.AtEnd,
        Parent = chip,
        Name = "LogoFallback",
        BackgroundTransparency = 1,
        FontFace = TITLE_FONT,
        Position = UDim2.fromOffset(4, 4),
        Size = UDim2.fromOffset(LOGO_WIDTH, LOGO_HEIGHT),
        Text = string.upper(tostring(PRODUCT.name)),
        TextColor3 = WURSTLOGO_TEXT,
```

```
        TextSize = 14,
        TextXAlignment = Enum.TextXAlignment.Center,
        ZIndex = 21,
    }) :: any) :: TextLabel

    local versionChip: Frame = (create("Frame", {
        Parent = chip,
        Name = "VersionChip",
        BackgroundTransparency = 1,
        BorderSizePixel = 0,
        Position = UDim2.fromOffset(LOGO_WIDTH + 10, 12),
        Size = UDim2.fromOffset(versionWidth, 22),
        ZIndex = 20,
    }) :: any) :: Frame

    local bitmapText: any = state.bitmapText
    local drawsVersion: boolean = bitmapText ~= nil
        and type(bitmapText.draw) == "function"
    local versionColor: Color3 = WURSTLOGO_TEXT
    local versionLabel: TextLabel? = nil
    local function drawVersion(): ()
        if drawsVersion then
            bitmapText.draw(versionChip, {
                {text = versionText, color = versionColor},
            }, {
                Name = "Version",
                TextSize = 20,
                ZIndex = 21,
                align = "Left",
            })
        end
    end
    if drawsVersion then
        drawVersion()
    else
        versionLabel = (create("TextLabel", {
            TextTruncate = Enum.TextTruncate.AtEnd,
            Parent = versionChip,
            Name = "Version",
            BackgroundTransparency = 1,
            FontFace = TITLE_FONT,
            Position = UDim2.fromOffset(0, 0),
            Size = UDim2.fromScale(1, 1),
            Text = versionText,
            TextColor3 = WURSTLOGO_TEXT,
            TextSize = 20,
            TextXAlignment = Enum.TextXAlignment.Left,
        }) :: any) :: TextLabel
    end

    local function fitLogoBand(): ()
        local measured: number = 0
        if drawsVersion and type(bitmapText.measure) == "function" then

            measured = tonumber(bitmapText.measure(
                {{text = versionText}}, 20
            )) or 0
        elseif versionLabel then
            local bounds: any = (versionLabel :: any).TextBounds
            measured = bounds and tonumber(bounds.X) or 0
        end
        if measured <= 0 then
            return
        end
        local fittedVersion: number = 8 + measured
        local fittedChip: number = LOGO_WIDTH + 10 + fittedVersion + 4
        versionChip.Size = UDim2.fromOffset(fittedVersion, 22)
        chip.Size = UDim2.fromOffset(fittedChip, LOGO_HEIGHT + 8)
        logoStripe.Size = UDim2.fromOffset(fittedChip, 22)
    end
    if versionLabel then
        table.insert(
            Furniture.connections,
            (versionLabel :: TextLabel):GetPropertyChangedSignal("TextBounds")
                :Connect(fitLogoBand)
        )
    end
    if drawsVersion and type(bitmapText.onReady) == "table" then

        table.insert(bitmapText.onReady, fitLogoBand)
    end
    task.defer(fitLogoBand)

    if type(state.applyAssetImage) == "function" then
        state.applyAssetImage("wurstLogo", logoImage, function(): ()
            logoImage.Visible = true
            logoFallback.Visible = false
        end)
    end

    local logoAlways: boolean = true
    local function refreshLogoVisibility(): ()

        chip.Visible = logoAlways and state.visible == true
    end
    local function setLogoBackground(color: Color3): ()
        logoStripe.BackgroundColor3 = color
    end
    local function setLogoTextColor(color: Color3): ()
        versionColor = color
        if drawsVersion then
            drawVersion()
        elseif versionLabel then
            (versionLabel :: TextLabel).TextColor3 = color
        end
        logoFallback.TextColor3 = color
    end
    local function setLogoVisibility(value: string): ()
        logoAlways = value == "Always"
        refreshLogoVisibility()
    end
    setLogoBackground(colorFromConfig(
        configData.values["WurstLogo.Background"],
        WURSTLOGO_BACKGROUND
    ))
    setLogoTextColor(colorFromConfig(
        configData.values["WurstLogo.Text"],
        WURSTLOGO_TEXT
    ))
    local storedLogoVisibility: any = configData.values["WurstLogo.Visibility"]
    setLogoVisibility(type(storedLogoVisibility) == "string"
        and storedLogoVisibility
        or WURSTLOGO_VISIBILITY)

    state.wurstLogo = {
        root = chip,
        SetBackground = setLogoBackground,
        SetTextColor = setLogoTextColor,
        SetVisibility = setLogoVisibility,
    }

    local hudFrame: Frame = (create("Frame", {
        Parent = PopupLayer,
        Name = "HackList",
        Active = false,
        AnchorPoint = Vector2.new(1, 0),
        BackgroundTransparency = 1,
        BorderSizePixel = 0,
        Position = UDim2.new(1, -12, 0, HACKLIST_TOP),
        Size = UDim2.new(0, 260, 1, -(HACKLIST_TOP + 12)),
        ZIndex = 20,
    }) :: any) :: Frame

    local hudLayout: UIListLayout = (create("UIListLayout", {
        Parent = hudFrame,
        FillDirection = Enum.FillDirection.Vertical,
        HorizontalAlignment = Enum.HorizontalAlignment.Right,
        Padding = UDim.new(0, 1),
        SortOrder = Enum.SortOrder.LayoutOrder,
    }) :: any) :: UIListLayout

    local hudMode: string = type(configData.values["HackList.Mode"]) == "string"
        and configData.values["HackList.Mode"]
        or HACKLIST_MODE
    local hudPosition: string = type(configData.values["HackList.Position"]) == "string"
        and configData.values["HackList.Position"]
        or HACKLIST_POSITION
    local hudColor: Color3 = colorFromConfig(
        configData.values["HackList.Color"],
        HACKLIST_COLOR
    )
    local hudSortBy: string = type(configData.values["HackList.Sortby"]) == "string"
        and configData.values["HackList.Sortby"]
        or HACKLIST_SORT_BY
    local hudReverse: boolean = HACKLIST_REVERSE
    if configData.states["HackList.Reversesorting"] ~= nil then
        hudReverse = configData.states["HackList.Reversesorting"] == true
    end
    local hudAnimations: boolean = HACKLIST_ANIMATIONS
    if configData.states["HackList.Animations"] ~= nil then
        hudAnimations = configData.states["HackList.Animations"] == true
    end
    local hudSize: string = type(configData.values["HackList.Size"]) == "string"
        and configData.values["HackList.Size"]
        or "Normal"
    local hudRainbow: boolean = configData.states["HackList.Rainbow"] == true
    local HUD_SIZES: {[string]: {font: number, line: number}} = {
        Small = {font = 12, line = 14},
        Normal = {font = HACKLIST_FONT_SIZE, line = HACKLIST_LINE_HEIGHT},
        Large = {font = 20, line = 23},
    }
    local function hudMetrics(): {font: number, line: number}
        return HUD_SIZES[hudSize] or HUD_SIZES.Normal
    end

    local function hudOnLeft(): boolean

        return hudPosition == "Left" or hudPosition == "Custom"
    end

    local navigatorBottom: () -> number = function(): number
        return FURNITURE_MARGIN_Y + LOGO_HEIGHT + 8
    end
    local hudCustomX: number? = tonumber(configData.values["HackList.CustomX"])
    local hudCustomY: number? = tonumber(configData.values["HackList.CustomY"])

    local function logicalViewport(): Vector2
        local windows: any = state.windows
        if windows and type(windows.LogicalViewport) == "function" then
            local ok: boolean, viewport: any = pcall(windows.LogicalViewport)
            if ok and viewport then
                return viewport
            end
        end
        local raw: Vector2 = ScreenGui and ScreenGui.AbsoluteSize
            or Vector2.new(1280, 720)
        return raw
    end

    local function clampHudCustom(x: number, y: number): (number, number)
        local viewport: Vector2 = logicalViewport()
        return math.clamp(x, 0, math.max(viewport.X - 260, 0)),
            math.clamp(y, 0, math.max(viewport.Y - 32, 0))
    end

    local function placeHudList(): ()
        if hudPosition == "Custom" then
            local x: number = hudCustomX or FURNITURE_MARGIN_X
            local y: number = hudCustomY or navigatorBottom()
            x, y = clampHudCustom(x, y)
            hudCustomX, hudCustomY = x, y
            hudFrame.AnchorPoint = Vector2.new(0, 0)
            hudFrame.Position = UDim2.fromOffset(x, y)
            hudLayout.HorizontalAlignment = Enum.HorizontalAlignment.Left
        elseif hudOnLeft() then
            hudFrame.AnchorPoint = Vector2.new(0, 0)
            hudFrame.Position = UDim2.new(
                0,
                FURNITURE_MARGIN_X,
                0,
                navigatorBottom()
            )
            hudLayout.HorizontalAlignment = Enum.HorizontalAlignment.Left
        else

            local windows: any = state.windows
            local logicalWidth: number? = nil
            if windows and type(windows.LogicalViewport) == "function" then
                local ok: boolean, logical: any = pcall(windows.LogicalViewport)
                if ok and logical then
                    logicalWidth = logical.X
                end
            end
            hudFrame.AnchorPoint = Vector2.new(1, 0)
            if logicalWidth then
                hudFrame.Position = UDim2.new(0, logicalWidth - 12, 0, HACKLIST_TOP)
            else
                hudFrame.Position = UDim2.new(1, -12, 0, HACKLIST_TOP)
            end
            hudLayout.HorizontalAlignment = Enum.HorizontalAlignment.Right
        end
    end
    placeHudList()
    state.onLayout(placeHudList)

    local hudRows: {[string]: any} = {}

    local hudDragInput: InputObject? = nil
    local hudDragStart: Vector2 = Vector2.zero
    local hudDragOrigin: Vector2 = Vector2.zero
    local function hudHitTest(point: Vector2): boolean
        if hudPosition ~= "Custom" or state.visible ~= true then
            return false
        end
        for _, row: any in pairs(hudRows) do
            local target: GuiObject? = row.holder or row.container
            if target and target.Parent and target.Visible then
                local at: Vector2 = target.AbsolutePosition
                local size: Vector2 = target.AbsoluteSize
                if size.X > 0 and size.Y > 0
                    and point.X >= at.X and point.X <= at.X + size.X
                    and point.Y >= at.Y and point.Y <= at.Y + size.Y then
                    return true
                end
            end
        end
        return false
    end
    state.surfaceDragHitTest = hudHitTest
    hudFrame.Active = false
    hudFrame.ZIndex = 20

    table.insert(Furniture.connections, UserInputService.InputBegan:Connect(function(
        input: InputObject
    ): ()
        if state.surfaceDragOwner ~= nil
            or (input.UserInputType ~= Enum.UserInputType.MouseButton1
                and input.UserInputType ~= Enum.UserInputType.Touch)
            or not hudHitTest(Vector2.new(input.Position.X, input.Position.Y)) then
            return
        end
        hudDragInput = input
        state.surfaceDragOwner = hudFrame
        hudDragStart = Vector2.new(input.Position.X, input.Position.Y)
        hudDragOrigin = Vector2.new(
            hudFrame.Position.X.Offset,
            hudFrame.Position.Y.Offset
        )
    end))
    table.insert(Furniture.connections, UserInputService.InputChanged:Connect(function(
        input: InputObject
    ): ()
        if not hudDragInput
            or (input ~= hudDragInput
                and not (hudDragInput.UserInputType == Enum.UserInputType.MouseButton1
                    and input.UserInputType == Enum.UserInputType.MouseMovement)) then
            return
        end
        local pointer: Vector2 = Vector2.new(input.Position.X, input.Position.Y)
        local delta: Vector2 = pointer - hudDragStart
        local windows: any = state.windows
        if windows and type(windows.ScreenToLogical) == "function" then
            delta = windows.ScreenToLogical(delta)
        end
        local x: number, y: number = clampHudCustom(
            hudDragOrigin.X + delta.X,
            hudDragOrigin.Y + delta.Y
        )
        hudCustomX, hudCustomY = x, y
        hudFrame.Position = UDim2.fromOffset(x, y)
    end))
    table.insert(Furniture.connections, UserInputService.InputEnded:Connect(function(
        input: InputObject
    ): ()
        if not hudDragInput
            or (input ~= hudDragInput
                and input.UserInputType ~= Enum.UserInputType.MouseButton1) then
            return
        end
        hudDragInput = nil
        if state.surfaceDragOwner == hudFrame then
            state.surfaceDragOwner = nil
        end
        if hudCustomX and hudCustomY then
            configData.values["HackList.CustomX"] = hudCustomX
            configData.values["HackList.CustomY"] = hudCustomY
            queueConfigSave()
        end
    end))

    local function animationsOn(): boolean
        return hudAnimations and state.animationsEnabled ~= false
    end

    local hudRainbowClock: number = 0
    local function hudEntryColor(order: number?): Color3

        if hudRainbow then
            local hue: number = (hudRainbowClock * 0.35 + (order or 0) * 0.05) % 1
            return Color3.fromHSV(hue, 0.85, 1)
        end
        if animationsOn() then
            local wave: number = 0.5 + 0.5 * math.sin(
                hudRainbowClock * 2.4 - (order or 0) * 0.65
            )
            local brightness: number = 0.45 + 0.55 * wave
            return Color3.new(
                hudColor.R * brightness,
                hudColor.G * brightness,
                hudColor.B * brightness
            )
        end
        return hudColor
    end
    local function drawHudEntry(row: any, name: string, status: string?): ()
        row.name = name
        row.status = status
        local segments: {any} = {{text = name, color = hudEntryColor(row.order)}}
        if status and status ~= "" then
            table.insert(segments, {
                text = " [" .. tostring(status) .. "]",
                color = Theme.textMuted,
            })
        end
        local bitmapText: any = state.bitmapText
        if bitmapText and type(bitmapText.draw) == "function" then
            row.holder = bitmapText.draw(row.container, segments, {
                Name = "Entry",
                TextSize = hudMetrics().font,
                ZIndex = 20,
                align = hudOnLeft() and "Left" or "Right",
            })
        end
    end

    local function newHudRow(name: string): any
        local container: Frame = (create("Frame", {
            Parent = hudFrame,
            Name = name,
            BackgroundTransparency = 1,
            BorderSizePixel = 0,
            Size = UDim2.new(1, 0, 0, hudMetrics().line),
            ZIndex = 20,
        }) :: any) :: Frame
        return {container = container}
    end

    local function slideHudRow(row: any): ()
        if not animationsOn() or not row.holder then
            return
        end
        local holder: Frame = row.holder :: Frame
        local resting: UDim2 = holder.Position

        holder.Position = UDim2.new(
            resting.X.Scale,
            resting.X.Offset + HACKLIST_SLIDE,
            resting.Y.Scale,
            resting.Y.Offset
        )
        TweenService:Create(
            holder,
            TweenInfo.new(0.25, Enum.EasingStyle.Back, Enum.EasingDirection.Out),
            {Position = resting}
        ):Play()
    end

    local function dropHudRow(row: any): ()
        if animationsOn() and row.holder then
            local holder: Frame = row.holder :: Frame
            local slid: UDim2 = UDim2.new(
                holder.Position.X.Scale,
                holder.Position.X.Offset + HACKLIST_SLIDE,
                holder.Position.Y.Scale,
                holder.Position.Y.Offset
            )
            local tween: Tween = TweenService:Create(
                holder,
                TweenInfo.new(0.15, Enum.EasingStyle.Quad, Enum.EasingDirection.In),
                {Position = slid}
            )
            tween.Completed:Connect(function(): ()
                row.container:Destroy()
            end)
            tween:Play()
        else
            row.container:Destroy()
        end
    end

    local hudSignature: string = ""
    local hudElapsed: number = 0
    -- Which side the HUD is drawn on; a change re-draws every row because the
    -- text alignment changes with it (v139.1).
    local hudSide: string = hudOnLeft() and "left" or "right"

    -- The overlay is opted into, not implied by the menu being open. Before
    -- v181 "Show" only meant "keep it after closing", so the list appeared on
    -- every open without any switch behind it.
    -- Only the switch in Settings -> UI -> HackList -> Show. A stale
    -- "ClickGUI.ShowHackList" key from old builds used to keep the overlay on
    -- with nothing visibly enabled, so it no longer counts.
    local hudAlways: boolean = configData.states["HackList.Show"] == true
    local hudVisible: boolean = hudAlways == true

    local function refreshHudList(): ()
        if not hudVisible then
            return
        end
        local entries: {any} = {}
        for _, feature: any in ipairs(allFeatures) do
            if feature.enabled == true and not feature.isCategory then
                table.insert(entries, {
                    name = tostring(feature.name),
                    status = feature.status,
                })
            end
        end

        local function before(left: any, right: any): boolean
            if hudSortBy == "Width" then
                local leftWidth: number = #left.name
                    + (left.status and #tostring(left.status) + 3 or 0)
                local rightWidth: number = #right.name
                    + (right.status and #tostring(right.status) + 3 or 0)
                if leftWidth ~= rightWidth then
                    return leftWidth > rightWidth
                end
            end
            return left.name < right.name
        end
        table.sort(entries, function(left: any, right: any): boolean
            if hudReverse then
                return before(right, left)
            end
            return before(left, right)
        end)

        if hudMode == "Hidden" then
            entries = {}
        elseif #entries > 0 then

            local screenHeight: number
            local manager: any = state.windows
            if manager and type(manager.LogicalViewport) == "function" then
                screenHeight = manager.LogicalViewport().Y
            else
                local camera: Camera? = workspace.CurrentCamera
                screenHeight = camera
                    and (camera :: Camera).ViewportSize.Y
                    or 720
            end
            local available: number = screenHeight - HACKLIST_TOP - 12
            local overflows: boolean =
                #entries * (hudMetrics().line + 1) > available
            if hudMode == "Count" or (hudMode == "Auto" and overflows) then
                entries = {{
                    name = tostring(#entries) .. " hacks active",
                    status = nil,
                }}
            end
        end

        local signature: {string} = {}
        for _, entry: any in ipairs(entries) do
            table.insert(
                signature,
                entry.name .. "\1" .. tostring(entry.status or "")
            )
        end
        local onLeft: boolean = hudOnLeft()
        local composed: string = table.concat(signature, "\2")
            .. (onLeft and "\3left" or "\3right")
        if composed == hudSignature then
            return
        end
        hudSignature = composed
        -- The side decides the text alignment, so moving the HUD redraws all.
        local side: string = onLeft and "left" or "right"
        local redrawAll: boolean = side ~= hudSide
        hudSide = side

        local wanted: {[string]: boolean} = {}
        for order: number, entry: any in ipairs(entries) do
            wanted[entry.name] = true
            local row: any = hudRows[entry.name]
            local isNew: boolean = row == nil
            if not row then
                row = newHudRow(entry.name)
                hudRows[entry.name] = row
            end
            row.container.LayoutOrder = order
            row.order = order
            -- v139.1: only the row that actually changed is re-drawn. One
            -- module updating its status used to rebuild every HUD entry in
            -- the same frame (a visible hitch when the list is long).
            if isNew or redrawAll
                or row.name ~= entry.name or row.status ~= entry.status then
                drawHudEntry(row, entry.name, entry.status)
            end
            if isNew then
                slideHudRow(row)
            end
        end
        for name: string, row: any in pairs(hudRows) do
            if not wanted[name] then
                hudRows[name] = nil
                dropHudRow(row)
            end
        end
    end

    -- v139: the HUD only animates while a rainbow or the brightness pulse is
    -- actually on, and each row keeps ONE colour table that gets mutated in
    -- place. Before this the loop rewrote every visible row 20 times a second
    -- forever, even with the HUD static, and allocated a fresh table per row
    -- per tick (a steady garbage stream for nothing).
    local hudRainbowSince: number = 0
    local function hudReanimate(): ()
        local bitmap = state.bitmapText
        local canRecolor = bitmap ~= nil and bitmap.recolor ~= nil
        for _, row: any in pairs(hudRows) do
            if row.name and row.holder then
                local colors: {Color3} = row.colors
                if not colors then
                    colors = { hudColor, Theme.textMuted }
                    row.colors = colors
                end
                colors[1] = hudEntryColor(row.order)
                colors[2] = Theme.textMuted
                if not canRecolor or not bitmap.recolor(row.holder, colors) then
                    drawHudEntry(row, row.name, row.status)
                end
            end
        end
    end

    local function hudAnimating(): boolean
        return hudVisible and (hudRainbow or animationsOn())
    end
    table.insert(Furniture.handles, TaskManager:Connect(function(
        deltaTime: number
    ): ()

        hudElapsed += deltaTime
        if hudElapsed >= 0.25 then
            hudElapsed %= 0.25
            refreshHudList()
        end
        if not hudAnimating() then
            -- Parked: no rainbow, no pulse, nothing to redraw this frame.
            return
        end
        hudRainbowClock += deltaTime
        hudRainbowSince += deltaTime
        local interval: number = if hudRainbow then 0.05 else 0.1
        if hudRainbowSince >= interval then
            hudRainbowSince %= interval
            hudReanimate()
        end
    end, nil, "HUD animations"))
    refreshHudList()

    local function forceHudRefresh(): ()
        hudSignature = "\0stale"
        refreshHudList()
    end

    if state.bitmapText and type(state.bitmapText.onReady) == "table" then
        table.insert(state.bitmapText.onReady, forceHudRefresh)
    end

    hudFrame.Visible = hudVisible
    local function setHackListVisible(value: boolean): ()
        hudVisible = value ~= false
        hudFrame.Visible = hudVisible
        if hudVisible then
            forceHudRefresh()

            for _, row: any in pairs(hudRows) do
                slideHudRow(row)
            end
        end
    end
    Furniture.SetHackListVisible = setHackListVisible

    local function applyHudVisibility(): ()
        setHackListVisible(hudAlways == true)
    end
    if type(state.addMenuVisibilityListener) == "function" then
        -- Opening or closing the menu never turns the overlay on by itself.
        state.addMenuVisibilityListener(function(_visible: boolean): ()
            if not hudAlways then
                setHackListVisible(false)
            end
        end)
    end

    state.hudList = {
        frame = hudFrame,
        refresh = refreshHudList,
        SetVisible = setHackListVisible,
        SetAlways = function(value: boolean): ()
            hudAlways = value == true
            applyHudVisibility()
        end,

        SetMode = function(value: string): ()
            hudMode = value
            forceHudRefresh()
        end,
        SetPosition = function(value: string): ()
            hudPosition = value
            placeHudList()
            if value == "Custom" then
```

```
            notify("Drag the HackList while the menu is open")
        end
        forceHudRefresh()
    end,
    SetColor = function(color: Color3): ()

        hudColor = color
        forceHudRefresh()
    end,
    SetSortBy = function(value: string): ()
        hudSortBy = value
        forceHudRefresh()
    end,
    SetReverse = function(value: boolean): ()
        hudReverse = value
        forceHudRefresh()
    end,
    SetAnimations = function(value: boolean): ()
        hudAnimations = value
        forceHudRefresh()
    end,
    SetSize = function(value: string): ()
        hudSize = value
        for _, row: any in pairs(hudRows) do
            row.container.Size = UDim2.new(1, 0, 0, hudMetrics().line)
        end
        forceHudRefresh()
    end,
    SetRainbow = function(value: boolean): ()
        hudRainbow = value == true
        if not hudRainbow then
            forceHudRefresh()
        end
    end,
}

local NAV_COLUMNS: number = 3
local NAV_CELL_WIDTH: number = 150
local NAV_CELL_HEIGHT: number = 22
local NAV_CELL_GAP: number = 3
local NAV_WIDTH: number = NAV_COLUMNS * NAV_CELL_WIDTH
    + (NAV_COLUMNS - 1) * NAV_CELL_GAP

local navScreen: Frame = (create("Frame", {

    Parent = ScreenGui or PopupLayer,
    Name = "Navigator",

    Active = false,
    BackgroundColor3 = Color3.fromRGB(0, 0, 0),

    BackgroundTransparency = 1,
    BorderSizePixel = 0,
    Size = UDim2.fromScale(1, 1),
    Visible = false,
    ZIndex = 246,
}) :: any) :: Frame

do
    local bitmap: any = state.bitmapText
    if bitmap and type(bitmap.adoptTree) == "function" then
        table.insert(Furniture.connections, bitmap.adoptTree(navScreen))
    end
end

local navColumn: Frame = (create("Frame", {
    Parent = navScreen,
    Name = "Column",
    AnchorPoint = Vector2.new(0.5, 0),
    BackgroundColor3 = Color3.fromRGB(0, 0, 0),
    BackgroundTransparency = 0.35,
    BorderSizePixel = 0,
    Position = UDim2.new(0.5, 0, 0, 48),
    Size = UDim2.fromOffset(NAV_WIDTH, 32),
    ZIndex = 247,
}) :: any) :: Frame
local navStroke: UIStroke = (create("UIStroke", {
    Parent = navColumn,
    Name = "NavigatorStroke",
    Color = Theme.outline,
    Thickness = 1,
    Transparency = 0.35,
}) :: any) :: UIStroke

local navSearch: TextBox = (create("TextBox", {
    TextTruncate = Enum.TextTruncate.AtEnd,
    Parent = navColumn,
    Name = "Search",
    BackgroundColor3 = Color3.fromRGB(0, 0, 0),
    BackgroundTransparency = 0.35,
    BorderSizePixel = 0,
    ClearTextOnFocus = false,
    FontFace = CONTROL_FONT,
    PlaceholderText = "Search…",
    Position = UDim2.fromOffset(0, 0),
    Size = UDim2.new(1, 0, 0, 32),
    Text = "",
    TextColor3 = Theme.text,
    TextSize = 14,
    TextXAlignment = Enum.TextXAlignment.Left,
    ZIndex = 248,
}) :: any) :: TextBox
create("UIPadding", {
    Parent = navSearch,
    PaddingLeft = UDim.new(0, 6),
    PaddingRight = UDim.new(0, 6),
})

local navResults: ScrollingFrame = (create("ScrollingFrame", {
    Parent = navColumn,
    Name = "Results",
    Active = true,
    AutomaticCanvasSize = Enum.AutomaticSize.None,
    BackgroundTransparency = 1,
    BorderSizePixel = 0,
    CanvasSize = UDim2.new(),
    Position = UDim2.fromOffset(0, 38),
    ScrollBarThickness = 3,
    ScrollingDirection = Enum.ScrollingDirection.Y,
    Size = UDim2.new(1, 0, 1, -38),
    ZIndex = 247,
}) :: any) :: ScrollingFrame
create("UIGridLayout", {
    Parent = navResults,
    CellPadding = UDim2.fromOffset(NAV_CELL_GAP, NAV_CELL_GAP),
    CellSize = UDim2.fromOffset(NAV_CELL_WIDTH, NAV_CELL_HEIGHT),
    SortOrder = Enum.SortOrder.LayoutOrder,
})

local function navPrefs(): any
    if type(configData.values.NavigatorPrefs) ~= "table" then
        configData.values.NavigatorPrefs = {}
    end
    return configData.values.NavigatorPrefs
end

local navOrdered: {any} = {}
local navEntries: {TextButton} = {}
local navIndex: {[any]: number} = {}
local navSelected: number = 1

local navCellConnections: {RBXScriptConnection} = {}
local rebuildNavigator: () -> () = function(): () end

local function paintNavEntry(index: number): ()
    local entry: TextButton? = navEntries[index]
    local feature: any = navOrdered[index]
    if not entry or not feature then
        return
    end
    local on: boolean = feature.enabled == true
    (entry :: TextButton).BackgroundColor3 =
        on and Theme.enabled or Color3.fromRGB(0, 0, 0);
    (entry :: TextButton).BackgroundTransparency = on and 0 or 0.35;
    (entry :: TextButton).TextColor3 = on and Theme.enabledText or Theme.text
    local ring: UIStroke? =
        (entry :: TextButton):FindFirstChild("SelectionRing") :: UIStroke?
    if ring then
        (ring :: UIStroke).Transparency =
            index == navSelected and 0 or 1
    end
end

local function paintNavSelection(): ()
    for index: number in ipairs(navEntries) do
        paintNavEntry(index)
    end
end

local function navActivate(index: number): ()
    local feature: any = navOrdered[index]
    if not feature then
        return
    end
    if type(feature.activate) == "function" then
        feature.activate()
    end
    navPrefs()[tostring(feature.name)] =
        (tonumber(navPrefs()[tostring(feature.name)]) or 0) + 1
    queueConfigSave()
    paintNavEntry(index)
    refreshHudList()
end

local closeNavigator: () -> () = function(): () end

local function navOpenSettings(index: number): ()
    local feature: any = navOrdered[index]
    if not feature or type(feature.hasOptions) ~= "function"
        or not feature.hasOptions() then
        return
    end
    closeNavigator()
    if state.visible ~= true then
        state.setMenuVisible(true)
    end
    if not feature.expanded and type(feature.toggleExpansion) == "function" then
        feature.toggleExpansion()
    end
end

rebuildNavigator = function(): ()
    for _, connection: RBXScriptConnection in ipairs(navCellConnections) do
        pcall(function(): ()
            connection:Disconnect()
        end)
    end
    table.clear(navCellConnections)
    for _, child: Instance in ipairs(navResults:GetChildren()) do
        if child:IsA("GuiObject") then
            child:Destroy()
        end
    end
    local query: string = string.lower(navSearch.Text or "")
    local matches: {any} = {}
    for _, feature: any in ipairs(allFeatures) do
        if query == ""
            or string.find(tostring(feature.searchText), query, 1, true) then
            table.insert(matches, feature)
        end
    end
    local prefs: any = navPrefs()
    table.sort(matches, function(left: any, right: any): boolean
        local leftUses: number = tonumber(prefs[left.name]) or 0
        local rightUses: number = tonumber(prefs[right.name]) or 0
        if leftUses ~= rightUses then
            return leftUses > rightUses
        end
        return tostring(left.name) < tostring(right.name)
    end)
    navOrdered = {}
    navEntries = {}
    navIndex = {}
    for index: number, feature: any in ipairs(matches) do
        table.insert(navOrdered, feature)
        navIndex[feature] = index
        local entry: TextButton = (create("TextButton", {
            TextTruncate = Enum.TextTruncate.AtEnd,
            Parent = navResults,
            Name = "Nav_" .. tostring(feature.name):gsub("%W", ""),
            Active = true,
            AutoButtonColor = false,
            BorderSizePixel = 0,
            FontFace = CONTROL_FONT,
            LayoutOrder = index,
            Text = tostring(feature.name),
            TextSize = 14,
            ZIndex = 248,
        }) :: any) :: TextButton
        create("UIStroke", {
            Parent = entry,
            Name = "SelectionRing",
            Color = Theme.enabled,
            Thickness = 1,
            Transparency = 1,
        })
        navEntries[index] = entry
        table.insert(navCellConnections, entry.MouseButton1Click:Connect(function(): ()
            navSelected = index
            navActivate(index)
            paintNavSelection()
        end))
    end
    navSelected = math.clamp(navSelected, 1, math.max(#navOrdered, 1))
    local rows: number = math.ceil(#navOrdered / NAV_COLUMNS)
    local resultHeight: number = rows * NAV_CELL_HEIGHT
        + math.max(rows - 1, 0) * NAV_CELL_GAP
    local screenHeight: number = ScreenGui
            and tonumber(ScreenGui.AbsoluteSize.Y)
        or 0
    if screenHeight <= 0 then
        screenHeight = 480
    end
    local columnHeight: number = math.min(
        38 + resultHeight,
        math.max(160, screenHeight - 96)
    )
    navColumn.Size = UDim2.fromOffset(NAV_WIDTH, columnHeight)
    navResults.Size = UDim2.fromOffset(NAV_WIDTH, columnHeight - 38)
    navResults.CanvasSize = UDim2.fromOffset(0, resultHeight)
    paintNavSelection()
end

local navigatorHidMenu: boolean = false
local function openNavigator(): ()
    if state.visible == true and type(state.setMenuVisible) == "function" then
        navigatorHidMenu = true
        state.setMenuVisible(false)
    else
        navigatorHidMenu = false
    end
    navScreen.Visible = true
    navSelected = 1
    rebuildNavigator()

end

closeNavigator = function(): ()
    navScreen.Visible = false
    navSearch:ReleaseFocus()
    navSearch.Text = ""
    if navigatorHidMenu then
        navigatorHidMenu = false
        if type(state.setMenuVisible) == "function" then
            state.setMenuVisible(true)
        end
    end
end

table.insert(
    Furniture.connections,
    navSearch:GetPropertyChangedSignal("Text"):Connect(function(): ()
        if navScreen.Visible then
            rebuildNavigator()
        end
    end)
)
table.insert(Furniture.connections, navSearch.FocusLost:Connect(function(
    enterPressed: boolean
): ()
    if enterPressed and navScreen.Visible then
        navActivate(navSelected)

    end
end))

table.insert(Furniture.connections, UserInputService.InputBegan:Connect(function(
    input: InputObject,
    _gameProcessed: boolean
): ()
    if not navScreen.Visible then
        return
    end
    local key: Enum.KeyCode = input.KeyCode
    local step: number = 0
    if key == Enum.KeyCode.Escape then
        closeNavigator()
        return
    elseif key == Enum.KeyCode.Backspace then
        if navSearch.Text == "" then
            closeNavigator()
        end
        return
    elseif key == Enum.KeyCode.Down then
        step = NAV_COLUMNS
    elseif key == Enum.KeyCode.Up then
        step = -NAV_COLUMNS
    elseif key == Enum.KeyCode.Right
        or key == Enum.KeyCode.Tab then
        step = 1
    elseif key == Enum.KeyCode.Left then
        step = -1
    elseif key == Enum.KeyCode.Space then
        if navSearch:IsFocused() then
            return
        end
        navOpenSettings(navSelected)
        return
    else
        return
    end
    if #navOrdered > 0 then
        navSelected = ((navSelected - 1 + step) % #navOrdered) + 1
        paintNavSelection()
    end
end))

state.navigator = {
    root = navScreen,
    refresh = rebuildNavigator,
    refreshFeature = function(feature: any): ()
        local index: number? = navIndex[feature]
        if navScreen.Visible and index then paintNavEntry(index) end
    end,
    Open = openNavigator,
    Close = closeNavigator,
}

local touchPrimary: boolean = state.isMobile == true
local DOCK_SEARCH_WIDTH: number = 190
local DOCK_BOTTOM_MARGIN: number = 70
local dockSearchWidth: number = touchPrimary and 0 or DOCK_SEARCH_WIDTH
local dockSearchGap: number = touchPrimary and 0 or PILL_GAP
local menuButtonX: number = PILL_PADDING_X
    + dockSearchWidth
    + dockSearchGap
local navButtonX: number = menuButtonX + PILL_BUTTON + PILL_GAP
local pillWidth: number = PILL_PADDING_X * 2
    + dockSearchWidth
    + dockSearchGap
    + PILL_BUTTON
    + PILL_GAP
    + PILL_BUTTON

local pillFrame: Frame = (create("Frame", {
    Parent = ScreenGui or PopupLayer,
    Name = "Pill",
    Active = true,
    AnchorPoint = Vector2.new(0.5, 1),
    BackgroundColor3 = Color3.fromRGB(16, 18, 28),
    BackgroundTransparency = 0.28,
    BorderSizePixel = 0,
    Position = UDim2.new(0.5, 0, 1, -DOCK_BOTTOM_MARGIN),
    Size = UDim2.fromOffset(pillWidth, PILL_BUTTON + PILL_PADDING_Y * 2),

    ZIndex = 249,
}) :: any) :: Frame
create("UICorner", {
    Parent = pillFrame,
    CornerRadius = UDim.new(1, 0),
})
create("UIStroke", {
    Parent = pillFrame,
    Color = Color3.fromRGB(255, 255, 255),
    Thickness = 1,
    Transparency = 0.91,
})

local PILL_IDLE_TEXT: Color3 = Color3.fromRGB(232, 232, 238)

local function showPillTip(anchor: GuiObject, text: string): ()
    local tooltip: TextLabel? = state.featureTooltip
    if not tooltip then
        return
    end
    state.tooltipToken = state.tooltipToken + 1
    local bitmap: any = state.bitmapText
    local measured: number = bitmap
            and type(bitmap.measure) == "function"
            and tonumber(bitmap.measure({{text = text}}, 16))
        or #text * 8
    local width: number = math.clamp(measured + 24, 120, 320)
    local wrapped: string = text
    local lines: number = 1
    if bitmap and type(bitmap.wrap) == "function" then
        wrapped, lines = bitmap.wrap(text, width - 24, 16)
    end
    local resolvedTooltip: TextLabel = tooltip :: TextLabel
    resolvedTooltip.Text = wrapped
    resolvedTooltip.TextTransparency = 1
    resolvedTooltip.Size = UDim2.fromOffset(
        width,
        math.max(30, lines * 18 + 12)
    )
    resolvedTooltip.Position = UDim2.fromOffset(
        anchor.AbsolutePosition.X,

        anchor.AbsolutePosition.Y - (math.max(30, lines * 18 + 12) + 8)
    )
    if bitmap and type(bitmap.draw) == "function" then
        bitmap.draw(tooltip, {{text = wrapped}}, {
            Name = "Glyphs",
            TextSize = 16,
            maxWidth = width - 24,
            ellipsis = true,
            ZIndex = 251,
        })
    else
        (tooltip :: TextLabel).TextTransparency = 0
    end
    (tooltip :: TextLabel).Visible = true
end

local function hidePillTip(): ()
    local tooltip: TextLabel? = state.featureTooltip
    if not tooltip then
        return
    end
    state.tooltipToken = state.tooltipToken + 1;
    (tooltip :: TextLabel).Visible = false
end

local function pillButton(
    order: number,
    offsetX: number,
    glyph: string,
    textSize: number,
    tip: string,
    onClick: () -> ()
): TextButton
    local button: TextButton = (create("TextButton", {
        TextTruncate = Enum.TextTruncate.AtEnd,
        Parent = pillFrame,
        Name = "PillButton" .. tostring(order),
        AutoButtonColor = false,
        BackgroundColor3 = Color3.fromRGB(255, 255, 255),
        BackgroundTransparency = 1,
        BorderSizePixel = 0,
        FontFace = CONTROL_FONT,
        Position = UDim2.fromOffset(offsetX, PILL_PADDING_Y),
        Size = UDim2.fromOffset(PILL_BUTTON, PILL_BUTTON),
        Text = glyph,
        TextColor3 = PILL_IDLE_TEXT,
        TextSize = textSize,
        ZIndex = 241,
    }) :: any) :: TextButton
    create("UICorner", {
        Parent = button,
        CornerRadius = UDim.new(1, 0),
    })
    table.insert(Furniture.connections, button.MouseEnter:Connect(function(): ()
        if not UserInputService.MouseEnabled then
            return
        end
        state.tooltipToken = state.tooltipToken + 1
        local token: number = state.tooltipToken
        task.delay(TOOLTIP_DELAY_MS / 1000, function(): ()
            if state.tooltipToken == token and button.Parent then
                showPillTip(button, tip)
            end
        end)
    end))
    table.insert(Furniture.connections, button.MouseLeave:Connect(function(): ()
        hidePillTip()
    end))
    table.insert(Furniture.connections, button.MouseButton1Click:Connect(function(): ()
        hidePillTip()
        onClick()
    end))
    return button
end

local menuButton: TextButton = pillButton(
    1,
    menuButtonX,
    "≡",
    16,
    "Show or hide the menu",
    function(): ()

        if navScreen.Visible and type(state.SetMenuStyle) == "function" then
            state.SetMenuStyle("Wurst", true)
            return
        end
        if type(state.openMenuSurface) == "function" then
            state.openMenuSurface()
        else
            state.setMenuVisible(not state.visible)
        end
    end
)

local navButton: TextButton = pillButton(
    2,
    navButtonX,
    "",
    16,
    "Navigator",
    function(): ()
        local navigator: any = state.navigator
        if not navigator then
            return
        end
        if navigator.root and navigator.root.Visible then
            if type(navigator.Close) == "function" then
                navigator.Close()
            end
        elseif type(navigator.Open) == "function" then
            navigator.Open()
        end
    end
)
do
    local ring: Frame = (create("Frame", {
        Parent = navButton,
        Name = "LupaRing",
        AnchorPoint = Vector2.new(0.5, 0.5),
        BackgroundTransparency = 1,
        BorderSizePixel = 0,
        Position = UDim2.new(0.5, -2, 0.5, -2),
        Size = UDim2.fromOffset(11, 11),
        ZIndex = navButton.ZIndex + 1,
    }) :: any) :: Frame
    create("UICorner", {
        Parent = ring,
        CornerRadius = UDim.new(1, 0),
    })
    create("UIStroke", {
        Parent = ring,
        Color = PILL_IDLE_TEXT,
        Thickness = 1.6,
        Transparency = 0.1,
    })
    local handle: Frame = (create("Frame", {
        Parent = navButton,
        Name = "LupaHandle",
        AnchorPoint = Vector2.new(0.5, 0.5),
        BackgroundColor3 = PILL_IDLE_TEXT,
        BorderSizePixel = 0,
        Position = UDim2.new(0.5, 5, 0.5, 5),
        Rotation = 45,
        Size = UDim2.fromOffset(2, 7),
        ZIndex = navButton.ZIndex + 1,
    }) :: any) :: Frame
    local _ = handle
end

local searchShell: Frame = (create("Frame", {
    Parent = pillFrame,
    Name = "DockSearch",
    BackgroundColor3 = Color3.fromRGB(255, 255, 255),
    BackgroundTransparency = 0.94,
    BorderSizePixel = 0,
    Position = UDim2.fromOffset(PILL_PADDING_X, PILL_PADDING_Y),
    Size = UDim2.fromOffset(dockSearchWidth, PILL_BUTTON),
    Visible = not touchPrimary,
    ZIndex = 241,
}) :: any) :: Frame
create("UICorner", {
    Parent = searchShell,
    CornerRadius = UDim.new(1, 0),
})
if not touchPrimary and type(makeTextBox) == "function" then
    local searchBox: TextBox = (makeTextBox(searchShell, "") :: any) :: TextBox
    searchBox.Name = "DockSearchBox"
    searchBox.BackgroundTransparency = 1
    searchBox.ClearTextOnFocus = false
    searchBox.PlaceholderText = "Search module..."
    searchBox.Position = UDim2.fromOffset(12, 0)
    searchBox.Size = UDim2.new(1, -18, 1, 0)
    searchBox.TextColor3 = PILL_IDLE_TEXT
    searchBox.TextSize = 14
    searchBox.TextXAlignment = Enum.TextXAlignment.Left
    searchBox.ZIndex = 242
    local boxCorner: UICorner? = searchBox:FindFirstChild("StyleCorner") :: UICorner?
    if boxCorner then
        boxCorner:Destroy()
    end
    local boxStroke: UIStroke? = searchBox:FindFirstChild("StyleStroke") :: UIStroke?
    if boxStroke then
        boxStroke:Destroy()
    end

    local function submitSearch(): ()
        local query: string = tostring(searchBox.Text or "")
        if query:gsub("%s", "") == "" then
            return
        end
        searchBox.Text = ""
        local search: any = state.moduleSearch
        if type(search) ~= "table" or type(search.Execute) ~= "function" then
            notify("module search unavailable")
            return
        end
        local result: any = search.Execute(query)
        local status: string = tostring(result and result.status)
        if status == "success" then
            searchBox.Text = ""
            local entryName: string = tostring(
                result.entry and result.entry.displayName or query
            )
            notify(entryName .. ": "
                .. (result.enabled and "enabled" or "disabled"))
        elseif status == "ambiguous" then
            local names: {string} = {}
            for index: number, match: any in ipairs(result.matches or {}) do
                if index > 4 then
                    table.insert(names, "…")
                    break
                end
                table.insert(names, tostring(match.displayName))
            end
            notify("ambiguous: " .. table.concat(names, ", "))
        elseif status == "not_toggleable" then
```

```
            notify(tostring(
                result.entry and result.entry.displayName or query
            ) .. " has nothing to toggle")
        else
            notify("no module matches \"" .. query .. "\"")
        end
    end
    table.insert(
        Furniture.connections,
        searchBox.FocusLost:Connect(function(enterPressed: boolean): ()
            if enterPressed == true then
                submitSearch()
            end
        end)
    )

    pcall(function(): ()
        table.insert(
            Furniture.connections,
            (searchBox :: any).ReturnPressedFromOnScreenKeyboard:Connect(
                submitSearch
            )
        )
    end)
end

local function setDockVisible(value: boolean): ()
    if touchPrimary then
        pillFrame.Visible = true
        return
    end
    pillFrame.Visible = value ~= false
end
if state.uiShowDock == nil then state.uiShowDock = touchPrimary end
setDockVisible(state.uiShowDock == true)
Furniture.SetDockVisible = setDockVisible
state.dock = {
    root = pillFrame,
    SetVisible = setDockVisible,
}

local moving = false
local moveConnections, moveHandle, moveHint = {}, nil, nil
local moveOrigin, dragInput, dragStart, dragAnchor
local function dockAt(x,y)
    local viewport, size = ScreenGui.AbsoluteSize, pillFrame.AbsoluteSize
    if viewport.X<1 or viewport.Y<1 then return end
    if size.X<1 or size.Y<1 then size=Vector2.new(pillWidth,PILL_BUTTON+PILL_PADDING_Y*2) end
    x=math.clamp(x,size.X*0.5,math.max(size.X*0.5,viewport.X-size.X*0.5))
    y=math.clamp(y,size.Y,math.max(size.Y,viewport.Y))
    pillFrame.Position=UDim2.fromScale(x/math.max(1,viewport.X),y/math.max(1,viewport.Y))
end
local savedDock = configData.ui.dockPosition
if type(savedDock)=="table" and type(savedDock.x)=="number" and type(savedDock.y)=="number"
    and savedDock.x>=savedDock.x and savedDock.y==savedDock.y then
    dockAt(math.clamp(savedDock.x,0,1)*ScreenGui.AbsoluteSize.X,math.clamp(savedDock.y,0,1)*ScreenGui.AbsoluteSize.Y)
end
if type(savedDock)=="table" and type(savedDock.x)=="number" and type(savedDock.y)=="number"
    and savedDock.x>=savedDock.x and savedDock.y==savedDock.y then
    task.defer(function()
        if pillFrame.Parent then dockAt(math.clamp(savedDock.x,0,1)*ScreenGui.AbsoluteSize.X,math.clamp(savedDock.y,0,1)*ScreenGui.AbsoluteSize.Y) end
    end)
end
local function finishDockMove(save)
    if not moving then return end
    moving=false;dragInput=nil
    for _,connection in ipairs(moveConnections) do connection:Disconnect() end
    table.clear(moveConnections)
    if moveHandle then moveHandle:Destroy();moveHandle=nil end
    if moveHint then moveHint:Destroy();moveHint=nil end
    if save then
        configData.ui.dockPosition={x=pillFrame.Position.X.Scale,y=pillFrame.Position.Y.Scale}
        queueConfigSave()
    else pillFrame.Position=moveOrigin end
    setDockVisible(state.uiShowDock==true)
end
local function beginDockMove()
    if moving then finishDockMove(true);return end
    moving=true;moveOrigin=pillFrame.Position;setDockVisible(true)
    moveHandle=create("TextButton",{Name="DockMoveHandle",Parent=pillFrame,Size=UDim2.fromScale(1,1),
        Text="Drag dock",FontFace=CONTROL_FONT,TextSize=14,TextColor3=Theme.text,
        BackgroundColor3=Theme.surface,BackgroundTransparency=0.15,BorderSizePixel=0,Active=true,ZIndex=450})
    moveHint=create("TextLabel",{Name="DockMoveHint",Parent=ScreenGui,AnchorPoint=Vector2.new(0.5,0),
        Position=UDim2.new(0.5,0,0,48),Size=UDim2.fromOffset(330,26),Text="Drag and release · Enter saves · Esc cancels",
        FontFace=CONTROL_FONT,TextSize=13,TextColor3=Theme.text,BackgroundColor3=Theme.surface,
        BackgroundTransparency=0.22,BorderSizePixel=0,ZIndex=450})
    local function connect(signal,fn) table.insert(moveConnections,signal:Connect(fn)) end
    connect(moveHandle.InputBegan,function(input)
        if input.UserInputType==Enum.UserInputType.MouseButton1 and input.UserInputType==Enum.UserInputType.Touch then return end
        dragInput=input;dragStart=Vector2.new(input.Position.X,input.Position.Y)
        dragAnchor=pillFrame.AbsolutePosition+Vector2.new(pillFrame.AbsoluteSize.X*0.5,pillFrame.AbsoluteSize.Y)
    end)
    connect(UserInputService.InputChanged,function(input)
        if not dragInput then return end
        if input~=dragInput and not (dragInput.UserInputType==Enum.UserInputType.MouseButton1 and input.UserInputType==Enum.UserInputType.MouseMovement) then return end
        local delta=Vector2.new(input.Position.X,input.Position.Y)-dragStart
        dockAt(dragAnchor.X+delta.X,dragAnchor.Y+delta.Y)
    end)
    connect(UserInputService.InputEnded,function(input)
        if input==dragInput or dragInput and dragInput.UserInputType==Enum.UserInputType.MouseButton1 and input.UserInputType==Enum.UserInputType.MouseButton1 then
            finishDockMove(true)
        end
    end)
    connect(UserInputService.InputBegan,function(input)
        if input.KeyCode==Enum.KeyCode.Escape then finishDockMove(false)
        elseif input.KeyCode==Enum.KeyCode.Return then finishDockMove(true) end
    end)
    connect(UserInputService.WindowFocusReleased,function() finishDockMove(false) end)
end
Furniture.BeginDockMove=beginDockMove
state.dock.BeginMove=beginDockMove
state.dock.CancelMove=function() finishDockMove(false) end
table.insert(Furniture.connections,ScreenGui:GetPropertyChangedSignal("AbsoluteSize"):Connect(function()
    dockAt(pillFrame.Position.X.Scale*ScreenGui.AbsoluteSize.X+pillFrame.Position.X.Offset,
        pillFrame.Position.Y.Scale*ScreenGui.AbsoluteSize.Y+pillFrame.Position.Y.Offset)
end))
state.addMenuVisibilityListener(function(visible) if not visible then finishDockMove(false) end end)

local function restyleMenuButton(): ()
    if not pillFrame.Parent then
        return
    end
    local open: boolean = state.visible == true
    menuButton.BackgroundColor3 = Theme.accent
    menuButton.BackgroundTransparency = open and 0 or 1
    menuButton.TextColor3 = open and Theme.accentText or PILL_IDLE_TEXT
end
restyleMenuButton()
state.addMenuVisibilityListener(function(visible: boolean): ()
    restyleMenuButton()

    refreshLogoVisibility()
    placeHudList()

    if visible then
        task.defer(function(): ()
            local manager: any = state.windows
            if not manager or type(manager.ReflowWindow) ~= "function" then
                return
            end
            for _, window: any in ipairs(manager.list or {}) do
                if (window.featureSettings or window.managementWindow)
                    and window.root.Visible
                    and not window.userMoved then
                    manager.ReflowWindow(window)
                end
            end
        end)
    end
end)
refreshLogoVisibility()
ThemeEngine.OnChange(function(): ()
    restyleMenuButton()

    navSearch.TextColor3 = Theme.text
    navStroke.Color = Theme.outline
    for _, entry: TextButton in ipairs(navEntries) do
        local ring: UIStroke? = entry:FindFirstChild("SelectionRing") :: UIStroke?
        if ring then
            ring.Color = Theme.enabled
        end
    end
    paintNavSelection()
end)

state.pill = {
    root = pillFrame,
    restyle = restyleMenuButton,
}

if type(state.refreshLauncher) == "function" then
    state.refreshLauncher()
end

activeCleanup = function(): ()
    finishDockMove(false)
    state.dock = nil
    for _, handle: any in ipairs(Furniture.handles) do
        pcall(function(): ()
            handle:Disconnect()
        end)
    end
    Furniture.handles = {}
    for _, connection: RBXScriptConnection in ipairs(Furniture.connections) do
        pcall(function(): ()
            connection:Disconnect()
        end)
    end
    Furniture.connections = {}
    for _, connection: RBXScriptConnection in ipairs(navCellConnections) do
        pcall(function(): ()
            connection:Disconnect()
        end)
    end
    table.clear(navCellConnections)
    if state.surfaceDragOwner == hudFrame then
        state.surfaceDragOwner = nil
    end
    if state.surfaceDragHitTest == hudHitTest then
        state.surfaceDragHitTest = nil
    end
    chip:Destroy()
    hudFrame:Destroy()
    navScreen:Destroy()
    pillFrame:Destroy()
    state.hudList = nil
    state.wurstLogo = nil
    state.navigator = nil
    state.pill = nil

    if type(state.refreshLauncher) == "function" then
        pcall(state.refreshLauncher)
    end
end
Module.Initialized = true
return Furniture
end

function Module.destroy(): ()
    if activeCleanup then
        pcall(activeCleanup)
    end
    activeCleanup = nil
    Module.Initialized = false
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/guis/Wurst/Code/Furniture.lua
--------------------------------------------------------------------------------


--------------------------------------------------------------------------------
ARCHIVO: src/guis/Wurst/Code/MobileActions.lua
BYTES: 33408
SHA-256: 72a2f383228123f3adbc59ab55cf9c968f5742b3107de3074d144d2734b3d3d1
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
export type MobileHoldCallbacks = {

    isActive: (() -> boolean)?,
    onPress: (() -> ())?,
    onRelease: (() -> ())?,
}

export type MobileActionRecord = {
    id: string,
    label: string,
    button: TextButton,
    callback: () -> (),

    hold: MobileHoldCallbacks?,
    holding: boolean,
    activeInput: InputObject?,
    startPointer: Vector2,
    startPosition: Vector2,
    moved: boolean,
    holdToken: number,
    size: number,
    opacity: number,
    shape: string,
    color: string,
    connections: {RBXScriptConnection},
}

export type PendingMobilePlacement = {
    id: string,
    label: string,
    callback: () -> (),
    hold: MobileHoldCallbacks?,
    restoreMenu: boolean,
}

local Module = {
    Name = "MobileActions",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

function Module.init(context: any): any
    local host: any = context
    local state: any = host.state
    local configData: any = host.configData
    local queueConfigSave: any = host.queueConfigSave
    local create: any = host.create
    local makeTextLabel: any = host.makeTextLabel
    local notify: any = host.notify
    local Theme: any = host.Theme
    local ThemeEngine: any = host.ThemeEngine
    local CONTROL_FONT: any = host.CONTROL_FONT
    local ScreenGui: any = host.ScreenGui
    local PopupLayer: any = host.PopupLayer
    local TweenService: any = host.TweenService
    local UserInputService: any = host.UserInputService
    local UI_MOTION: any = host.UI_MOTION
    local cornerRadius: any = host.cornerRadius
    local trackUiConnection: any = host.trackUiConnection

    local initialViewport: any = host.initialViewport or Vector2.new(1280, 720)

    type MobileActionRecord = {
        id: string,
        label: string,
        button: TextButton,
        callback: () -> (),

        hold: MobileHoldCallbacks?,
        holding: boolean,
        activeInput: InputObject?,
        startPointer: Vector2,
        startPosition: Vector2,
        moved: boolean,
        holdToken: number,
        size: number,
        opacity: number,
        shape: string,
        color: string,
        connections: {RBXScriptConnection},
    }

    type PendingMobilePlacement = {
        id: string,
        label: string,
        callback: () -> (),
        hold: MobileHoldCallbacks?,
        restoreMenu: boolean,
    }

    local mobileActions: {[string]: MobileActionRecord} = {}
    local pendingMobilePlacement: PendingMobilePlacement? = nil

    local MobilePlacementOverlay: Frame = (create("Frame", {
        Parent = PopupLayer,
        Name = "MobilePlacementOverlay",
        Active = true,
        BackgroundColor3 = Theme.background,
        BackgroundTransparency = 0.2,
        BorderSizePixel = 0,
        Size = UDim2.fromScale(1, 1),
        Visible = false,
        ZIndex = 200,
    }) :: any) :: Frame
    local MobilePlacementLabel = makeTextLabel(
        MobilePlacementOverlay,
        "TOUCH WHERE THE ACTION BUTTON SHOULD GO",
        16
    )
    MobilePlacementLabel.FontFace = CONTROL_FONT
    MobilePlacementLabel.TextXAlignment = Enum.TextXAlignment.Center
    MobilePlacementLabel.Position = UDim2.new(0.08, 0, 0.08, 0)
    MobilePlacementLabel.Size = UDim2.new(0.84, 0, 0, 54)
    MobilePlacementLabel.ZIndex = 201

    local MOBILE_SHAPES: {string} = {"Square", "Rounded", "Circle"}
    local MOBILE_PALETTE: {string} = {
        "#F2F2F2", "#2B2B2B", "#0A84FF", "#30D158",
        "#FF9F0A", "#FF375F", "#BF5AF2", "#64D2FF",
    }
    local SHAPE_CORNERS: {[string]: UDim} = {
        Square = UDim.new(0, 2),
        Rounded = UDim.new(0, 12),
        Circle = UDim.new(1, 0),
    }
    local changeListeners: {() -> ()} = {}

    local function colorToHex(color: Color3): string
        local ok: boolean, hex: any = pcall(function(): any
            return (Color3 :: any).toHex(color)
        end)
        if ok and type(hex) == "string" then
            if string.sub(hex, 1, 1) ~= "#" then
                hex = "#" .. hex
            end
            return string.upper(hex)
        end
        return string.format(
            "#%02X%02X%02X",
            math.floor(color.R * 255 + 0.5),
            math.floor(color.G * 255 + 0.5),
            math.floor(color.B * 255 + 0.5)
        )
    end

    local function colorFromHex(value: string): Color3?
        if type(value) ~= "string" or value == "" then
            return nil
        end
        if string.sub(value, 1, 1) ~= "#" then
            value = "#" .. value
        end
        local ok: boolean, color: any = pcall(function(): any
            return (Color3 :: any).fromHex(value)
        end)
        if ok and typeof(color) == "Color3" then
            return color
        end
        return nil
    end

    local function readableTextColor(background: Color3): Color3
        local luminance: number =
            0.2126 * background.R + 0.7152 * background.G + 0.0722 * background.B
        return luminance > 0.55
            and Color3.fromRGB(18, 18, 18)
            or Color3.fromRGB(242, 242, 242)
    end

    local function ensureMobileConfig(): {[string]: any}
        if type(configData.ui.mobileActions) ~= "table" then
            configData.ui.mobileActions = {}
        end
        return configData.ui.mobileActions
    end

    -- Defaults for every button created from now on: shape and colour are
    -- user choices, not hardcoded squares. Existing saved buttons keep their
    -- own values (or fall back to these when they predate the feature).
    local function ensureMobileDefaults(): {[string]: any}
        if type(configData.ui.mobileButtons) ~= "table" then
            configData.ui.mobileButtons = {}
        end
        local defaults: {[string]: any} = configData.ui.mobileButtons
        if type(defaults.shape) ~= "string"
            or not table.find(MOBILE_SHAPES, defaults.shape) then
            defaults.shape = "Square"
        end
        if type(defaults.color) ~= "string"
            or not colorFromHex(defaults.color) then
            defaults.color = colorToHex(Theme.surfaceRaised)
        end
        return defaults
    end

    local function applyMobileStyle(record: MobileActionRecord): ()
        local corner: UICorner? = record.button:FindFirstChildOfClass("UICorner")
        if corner then
            corner.CornerRadius = SHAPE_CORNERS[record.shape]
                or SHAPE_CORNERS.Rounded
        end
        local color: Color3? = colorFromHex(record.color)
        if color then
            record.button.BackgroundColor3 = color
            record.button.TextColor3 = readableTextColor(color)
        end
    end

    local function notifyMobileChanged(): ()
        for _, listener: () -> () in ipairs(changeListeners) do
            pcall(listener)
        end
    end

    local function clampMobileButtonPosition(
        position: Vector2,
        size: number
    ): Vector2
        local camera: Camera? = workspace.CurrentCamera
        local viewport: Vector2 = camera
            and camera.ViewportSize
            or Vector2.new(1280, 720)
        local radius: number = size / 2
        return Vector2.new(
            math.clamp(position.X, radius + 6, math.max(radius + 6, viewport.X - radius - 6)),
            math.clamp(position.Y, radius + 6, math.max(radius + 6, viewport.Y - radius - 6))
        )
    end

    local function saveMobileActionPosition(record: MobileActionRecord): ()
        local mobileConfig: {[string]: any} = ensureMobileConfig()
        -- The first four slots keep the original saved format; shape and
        -- colour are appended so older configs still load unchanged.
        mobileConfig[record.id] = {
            record.button.Position.X.Offset,
            record.button.Position.Y.Offset,
            record.size,
            record.opacity,
            record.shape,
            record.color,
        }
        queueConfigSave()
        notifyMobileChanged()
    end

    local function removeMobileAction(id: string): ()
        local record: MobileActionRecord? = mobileActions[id]
        if not record then
            return
        end
        mobileActions[id] = nil
        ensureMobileConfig()[id] = nil
        for _, connection: RBXScriptConnection in ipairs(record.connections) do
            connection:Disconnect()
        end
        table.clear(record.connections)
        if record.button.Parent then
            record.button:Destroy()
        end
        queueConfigSave()
        notifyMobileChanged()
        notify(record.label .. " mobile button removed.")
    end

    local editingRecord: MobileActionRecord? = nil
    local editorOverlay: Frame = (create("Frame", {
        Parent = ScreenGui,
        Name = "MobileActionEditor",
        Active = true,
        BackgroundColor3 = Color3.fromRGB(0, 0, 0),
        BackgroundTransparency = 0.45,
        BorderSizePixel = 0,
        Size = UDim2.fromScale(1, 1),
        Visible = false,
        ZIndex = 270,
    }) :: any) :: Frame
    local editorPanel: Frame = (create("Frame", {
        Parent = editorOverlay,
        Name = "Panel",
        AnchorPoint = Vector2.new(0.5, 0.5),
        BackgroundColor3 = Theme.surfaceRaised,
        BackgroundTransparency = 0.04,
        BorderSizePixel = 0,
        Position = UDim2.fromScale(0.5, 0.5),
        Size = UDim2.fromOffset(300, 300),
        ZIndex = 271,
    }) :: any) :: Frame
    create("UICorner", {
        Parent = editorPanel,
        CornerRadius = UDim.new(0, 12),
    })
    local editorStroke: UIStroke = (create("UIStroke", {
        Parent = editorPanel,
        Name = "WindowOutline",
        Color = Theme.outline,
        Thickness = 1,
        Transparency = 0.08,
    }) :: any) :: UIStroke
    if ThemeEngine and type(ThemeEngine.Bind) == "function" then
        ThemeEngine.Bind(editorStroke, "Color", "outline")
    end
    local editorTitle: TextLabel = (makeTextLabel(
        editorPanel,
        "Mobile button",
        16
    ) :: any) :: TextLabel
    editorTitle.Position = UDim2.fromOffset(14, 8)
    editorTitle.Size = UDim2.new(1, -28, 0, 28)
    editorTitle.TextXAlignment = Enum.TextXAlignment.Left
    editorTitle.ZIndex = 272

    local function editorLabel(text: string, y: number): TextLabel
        local label: TextLabel = (makeTextLabel(editorPanel, text, 14) :: any) :: TextLabel
        label.Position = UDim2.fromOffset(14, y)
        label.Size = UDim2.fromOffset(76, 36)
        label.TextXAlignment = Enum.TextXAlignment.Left
        label.ZIndex = 272
        return label
    end
    editorLabel("Size", 46)
    editorLabel("Opacity", 88)
    editorLabel("Shape", 130)
    editorLabel("Colour", 172)

    local function editorButton(
        name: string,
        text: string,
        x: number,
        y: number,
        width: number
    ): TextButton
        local button: TextButton = (create("TextButton", {
            TextTruncate = Enum.TextTruncate.AtEnd,
            Parent = editorPanel,
            Name = name,
            Active = true,
            AutoButtonColor = false,
            BackgroundColor3 = Theme.surface,
            BackgroundTransparency = 0.08,
            BorderSizePixel = 0,
            FontFace = CONTROL_FONT,
            Position = UDim2.fromOffset(x, y),
            Size = UDim2.fromOffset(width, 34),
            Text = text,
            TextColor3 = Theme.text,
            TextSize = 14,
            ZIndex = 272,
        }) :: any) :: TextButton
        create("UICorner", {
            Parent = button,
            CornerRadius = UDim.new(0, 7),
        })
        return button
    end

    local sizeDown: TextButton = editorButton("SizeDown", "-", 92, 47, 40)
    local sizeValue: TextButton = editorButton("SizeValue", "52 px", 138, 47, 72)
    local sizeUp: TextButton = editorButton("SizeUp", "+", 216, 47, 40)
    sizeValue.Active = false
    local opacityDown: TextButton = editorButton("OpacityDown", "-", 92, 89, 40)
    local opacityValue: TextButton = editorButton("OpacityValue", "92%", 138, 89, 72)
    local opacityUp: TextButton = editorButton("OpacityUp", "+", 216, 89, 40)
    opacityValue.Active = false
    local shapeButton: TextButton = editorButton("Shape", "Square", 92, 131, 164)
    shapeButton.TextXAlignment = Enum.TextXAlignment.Center
    local removeButton: TextButton = editorButton("Remove", "Remove", 14, 214, 78)
    removeButton.BackgroundColor3 = Color3.fromRGB(120, 28, 28)
    local resetButton: TextButton = editorButton("Reset", "Reset", 101, 214, 78)
    local doneButton: TextButton = editorButton("Done", "Done", 188, 214, 78)
    local applyAllButton: TextButton = editorButton("ApplyAll", "Apply to all buttons", 14, 256, 272)
    doneButton.BackgroundColor3 = Theme.enabled
    doneButton.TextColor3 = Theme.enabledText
    if ThemeEngine and type(ThemeEngine.Bind) == "function" then
        ThemeEngine.Bind(doneButton, "BackgroundColor3", "enabled")
        ThemeEngine.Bind(doneButton, "TextColor3", "enabledText")
    end

    local function closeEditor(): ()
        editingRecord = nil
        editorOverlay.Visible = false
    end

    -- Forward declaration: the swatch handlers below repaint the editor, and
    -- the editor body is defined after them.
    local refreshEditor: () -> () = function(): () end
    -- Colour swatches: eight presets, repainted with a live ring around the
    -- one the button is currently using.
    local swatches: {[string]: TextButton} = {}
    for index: number, hex: string in ipairs(MOBILE_PALETTE) do
        local color: Color3 = colorFromHex(hex) or Theme.surfaceRaised
        local swatch: TextButton = (create("TextButton", {
            Parent = editorPanel,
            Name = "Colour" .. tostring(index),
            Active = true,
            AutoButtonColor = false,
            BackgroundColor3 = color,
            BorderSizePixel = 0,
            Position = UDim2.fromOffset(92 + (index - 1) * 25, 168),
            Size = UDim2.fromOffset(22, 22),
            Text = "",
            ZIndex = 272,
        }) :: any) :: TextButton
        create("UICorner", {
            Parent = swatch,
            CornerRadius = UDim.new(1, 0),
        })
        local ring: UIStroke = (create("UIStroke", {
            Parent = swatch,
            Name = "SelectedRing",
            Color = Theme.text,
            Thickness = 1,
            Transparency = 0.85,
        }) :: any) :: UIStroke
        swatch:SetAttribute("Hex", hex)
        swatches[hex] = swatch
        trackUiConnection(swatch.MouseButton1Click:Connect(function(): ()
            local record: MobileActionRecord? = editingRecord
            if not record then
                return
            end
            record.color = hex
            refreshEditor()
        end))
        local _ = ring
    end
```

```
    local function refreshSwatches(selected: string): ()
        for hex: string, swatch: TextButton in pairs(swatches) do
            local ring: UIStroke? = swatch:FindFirstChild("SelectedRing") :: UIStroke?
            if ring then
                local active: boolean = string.upper(hex) == string.upper(selected)
                ring.Thickness = active and 2 or 1
                ring.Transparency = active and 0 or 0.85
                ring.Color = active and Theme.text or Theme.outline
            end
        end
    end

    refreshEditor = function(): ()
        local record: MobileActionRecord? = editingRecord
        if not record then
            return
        end
        editorTitle.Text = record.label
        sizeValue.Text = tostring(math.round(record.size)) .. " px"
        opacityValue.Text = tostring(math.round(record.opacity * 100)) .. "%"
        shapeButton.Text = record.shape
        record.button.Size = UDim2.fromOffset(record.size, record.size)
        record.button.BackgroundTransparency = 1 - record.opacity
        applyMobileStyle(record)
        refreshSwatches(record.color)
        local clamped: Vector2 = clampMobileButtonPosition(
            Vector2.new(record.button.Position.X.Offset, record.button.Position.Y.Offset),
            record.size
        )
        record.button.Position = UDim2.fromOffset(clamped.X, clamped.Y)
        saveMobileActionPosition(record)
    end

    local function openEditor(record: MobileActionRecord): ()
        editingRecord = record
        editorOverlay.Visible = true
        refreshEditor()
    end

    trackUiConnection(sizeDown.MouseButton1Click:Connect(function(): ()
        if editingRecord then
            editingRecord.size = math.clamp(editingRecord.size - 8, 36, 120)
            refreshEditor()
        end
    end))
    trackUiConnection(sizeUp.MouseButton1Click:Connect(function(): ()
        if editingRecord then
            editingRecord.size = math.clamp(editingRecord.size + 8, 36, 120)
            refreshEditor()
        end
    end))
    trackUiConnection(opacityDown.MouseButton1Click:Connect(function(): ()
        if editingRecord then
            editingRecord.opacity = math.clamp(editingRecord.opacity - 0.1, 0.2, 1)
            refreshEditor()
        end
    end))
    trackUiConnection(opacityUp.MouseButton1Click:Connect(function(): ()
        if editingRecord then
            editingRecord.opacity = math.clamp(editingRecord.opacity + 0.1, 0.2, 1)
            refreshEditor()
        end
    end))
    trackUiConnection(shapeButton.MouseButton1Click:Connect(function(): ()
        local record: MobileActionRecord? = editingRecord
        if not record then
            return
        end
        local index: number = table.find(MOBILE_SHAPES, record.shape) or 1
        record.shape = MOBILE_SHAPES[index % #MOBILE_SHAPES + 1]
        refreshEditor()
    end))
    trackUiConnection(resetButton.MouseButton1Click:Connect(function(): ()
        local record: MobileActionRecord? = editingRecord
        if record then
            local defaults: {[string]: any} = ensureMobileDefaults()
            record.size = 52
            record.opacity = 0.92
            record.shape = defaults.shape
            record.color = defaults.color
            refreshEditor()
        end
    end))
    trackUiConnection(applyAllButton.MouseButton1Click:Connect(function(): ()
        local source: MobileActionRecord? = editingRecord
        if not source then
            return
        end
        for _, record: MobileActionRecord in pairs(mobileActions) do
            record.size = source.size
            record.opacity = source.opacity
            record.shape = source.shape
            record.color = source.color
            record.button.Size = UDim2.fromOffset(record.size, record.size)
            record.button.BackgroundTransparency = 1 - record.opacity
            applyMobileStyle(record)
            saveMobileActionPosition(record)
        end
        notify("mobile button style applied to all.")
    end))
    trackUiConnection(removeButton.MouseButton1Click:Connect(function(): ()
        local record: MobileActionRecord? = editingRecord
        closeEditor()
        if record then
            removeMobileAction(record.id)
        end
    end))
    trackUiConnection(doneButton.MouseButton1Click:Connect(closeEditor))

    local function setMobileActionVisibility(menuVisible: boolean): ()
        if menuVisible then
            closeEditor()
        end
        for _, record: MobileActionRecord in pairs(mobileActions) do
            record.button.Visible = state.isMobile and not menuVisible
        end
    end

    local function createMobileAction(
        id: string,
        label: string,
        callback: () -> (),
        requestedPosition: Vector2?,
        hold: MobileHoldCallbacks?
    ): MobileActionRecord?
        if not state.isMobile then
            return nil
        end

        local existing: MobileActionRecord? = mobileActions[id]
        if existing then
            existing.callback = callback
            existing.label = label
            existing.hold = hold
            return existing
        end

        local stored: any = ensureMobileConfig()[id]
        local size: number = math.clamp(
            type(stored) == "table" and tonumber(stored[3])
                or tonumber(state.mobileButtonSize)
                or 52,
            36,
            120
        )
        local opacity: number = math.clamp(
            type(stored) == "table" and tonumber(stored[4]) or 0.92,
            0.2,
            1
        )
        local defaults: {[string]: any} = ensureMobileDefaults()
        local shape: string = type(stored) == "table"
            and type(stored[5]) == "string"
            and table.find(MOBILE_SHAPES, stored[5])
            and stored[5]
            or defaults.shape
        local color: string = type(stored) == "table"
            and type(stored[6]) == "string"
            and colorFromHex(stored[6])
            and stored[6]
            or defaults.color
        local actionCount: number = 0
        for _ in pairs(mobileActions) do
            actionCount += 1
        end
        local defaultPosition: Vector2 = Vector2.new(
            74 + (actionCount * (size + 10)),
            math.max(size, initialViewport.Y - size - 70)
        )
        local selectedPosition: Vector2 = requestedPosition or (
            type(stored) == "table"
                and Vector2.new(
                    tonumber(stored[1]) or defaultPosition.X,
                    tonumber(stored[2]) or defaultPosition.Y
                )
                or defaultPosition
        )
        selectedPosition = clampMobileButtonPosition(selectedPosition, size)

        local button = create("TextButton", {
            TextTruncate = Enum.TextTruncate.AtEnd,
            ClipsDescendants = true,
            Parent = ScreenGui,
            Name = "MobileAction_" .. id:gsub("%W", ""),
            Active = true,
            AutoButtonColor = false,
            BackgroundColor3 = Theme.surfaceRaised,
            BackgroundTransparency = 1 - opacity,
            BorderSizePixel = 0,
            FontFace = CONTROL_FONT,
            Position = UDim2.fromOffset(selectedPosition.X, selectedPosition.Y),
            AnchorPoint = Vector2.new(0.5, 0.5),
            Size = UDim2.fromOffset(size, size),
            Text = label,
            TextColor3 = Theme.text,

            TextScaled = true,
            TextWrapped = true,
            Visible = not state.isMobile and not menuVisible
            ZIndex = 180,
        }) :: TextButton
        create("UIPadding", {
            Parent = button,
            PaddingBottom = UDim.new(0, 4),
            PaddingLeft = UDim.new(0, 4),
            PaddingRight = UDim.new(0, 4),
            PaddingTop = UDim.new(0, 4),
        })
        create("UICorner", {
            Parent = button,
            CornerRadius = cornerRadius(12),
        })
        local buttonOutline: UIStroke = (create("UIStroke", {
            Parent = button,
            Name = "MobileActionOutline",
            Color = Theme.outline,
            Transparency = 0.35,
            Thickness = 1,
        }) :: any) :: UIStroke
        if ThemeEngine and type(ThemeEngine.Bind) == "function" then
            ThemeEngine.Bind(buttonOutline, "Color", "outline")
        end
        create("UITextSizeConstraint", {
            Parent = button,
            MaxTextSize = 12,
            MinTextSize = 6,
        })

        local record: MobileActionRecord = {
            id = id,
            label = label,
            button = button,
            callback = callback,
            hold = hold,
            holding = false,
            activeInput = nil,
            startPointer = Vector2.zero,
            startPosition = selectedPosition,
            moved = false,
            holdToken = 0,
            size = size,
            opacity = opacity,
            shape = shape,
            color = color,
            connections = {},
        }
        mobileActions[id] = record
        applyMobileStyle(record)
        saveMobileActionPosition(record)

        table.insert(record.connections, trackUiConnection(
            button.InputBegan:Connect(function(input: InputObject): ()
            if input.UserInputType ~= Enum.UserInputType.Touch then
                return
            end
            record.activeInput = input
            record.startPointer = Vector2.new(input.Position.X, input.Position.Y)
            record.startPosition = Vector2.new(
                button.Position.X.Offset,
                button.Position.Y.Offset
            )
            record.moved = false

            local hold: MobileHoldCallbacks? = record.hold
            local holdActive: boolean = hold ~= nil
                and hold.onPress ~= nil
                and (hold.isActive == nil or (hold.isActive :: () -> boolean)())
            if holdActive and not record.holding then
                record.holding = true
                button.BackgroundTransparency = math.max(0, 1 - record.opacity - 0.1)
                task.spawn((hold :: MobileHoldCallbacks).onPress :: () -> ())
            end
            record.holdToken += 1
            local token: number = record.holdToken
            task.delay(0.75, function(): ()
                if mobileActions[id] ~= record
                    or record.activeInput ~= input
                    or record.moved
                    or record.holdToken ~= token then
                    return
                end
                record.activeInput = nil
                record.holdToken += 1
                if record.holding then
                    record.holding = false
                    if record.hold and record.hold.onRelease then
                        task.spawn(record.hold.onRelease)
                    end
                end
                button.BackgroundTransparency = 1 - record.opacity
                openEditor(record)
            end)
        end)))

        table.insert(record.connections, trackUiConnection(
            UserInputService.InputEnded:Connect(function(
            input: InputObject
        ): ()
            if input.UserInputType ~= Enum.UserInputType.Touch
                or record.activeInput ~= input then
                return
            end
            record.activeInput = nil
            record.holdToken += 1
            if record.holding then
                record.holding = false
                button.BackgroundTransparency = 1 - record.opacity
                if record.hold and record.hold.onRelease then
                    task.spawn(record.hold.onRelease)
                end
                if record.moved then
                    saveMobileActionPosition(record)
                end
            elseif record.moved then
                saveMobileActionPosition(record)
            else
                TweenService:Create(
                    button,
                    TweenInfo.new(0.08, Enum.EasingStyle.Quad),
                    {BackgroundTransparency = math.max(0, 1 - record.opacity - 0.1)}
                ):Play()
                task.delay(0.09, function(): ()
                    if button.Parent then
                        TweenService:Create(
                            button,
                            TweenInfo.new(UI_MOTION.fast, Enum.EasingStyle.Quad),
                            {BackgroundTransparency = 1 - record.opacity}
                        ):Play()
                    end
                end)
                task.spawn(record.callback)
            end
        end)))

        return record
    end

    trackUiConnection(UserInputService.InputChanged:Connect(function(
        input: InputObject
    ): ()
        if input.UserInputType ~= Enum.UserInputType.Touch then
            return
        end
        for _, record: MobileActionRecord in pairs(mobileActions) do
            if record.activeInput == input then
                local pointer: Vector2 = Vector2.new(
                    input.Position.X,
                    input.Position.Y
                )
                local delta: Vector2 = pointer - record.startPointer
                if delta.Magnitude >= 7 then
                    record.moved = true
                    if record.holding then

                        record.holding = false
                        record.button.BackgroundTransparency = 1 - record.opacity
                        if record.hold and record.hold.onRelease then
                            task.spawn(record.hold.onRelease)
                        end
                    end
                end
                if record.moved then
                    local position: Vector2 = clampMobileButtonPosition(
                        record.startPosition + delta,
                        record.button.AbsoluteSize.X
                    )
                    record.button.Position = UDim2.fromOffset(
                        position.X,
                        position.Y
                    )
                end
            end
        end
    end))

    local function beginMobileActionPlacement(
        id: string,
        label: string,
        callback: () -> (),
        hold: MobileHoldCallbacks?
    ): ()
        if not state.isMobile then
            return
        end
        if mobileActions[id] then
            openEditor(mobileActions[id])
            return
        end

        pendingMobilePlacement = {
            id = id,
            label = label,
            callback = callback,
            hold = hold,
            restoreMenu = state.visible,
        }
        MobilePlacementLabel.Text = "TOUCH TO PLACE " .. string.upper(label)
            .. (hold
                and "\nHOLD THE BUTTON TO KEEP IT ACTIVE"
                or "\nHOLD THE BUTTON LATER TO CUSTOMIZE IT")
        MobilePlacementOverlay.Visible = true

        state.setMenuVisible(false)

        local placement: PendingMobilePlacement =
            pendingMobilePlacement :: PendingMobilePlacement
        task.delay(0, function(): ()
            if pendingMobilePlacement == placement then
                pendingMobilePlacement = nil
                MobilePlacementOverlay.Visible = false
                if placement.restoreMenu then
                    state.setMenuVisible(true)
                end
                notify("Mobile button placement cancelled.")
            end
        end)
    end

    trackUiConnection(MobilePlacementOverlay.InputBegan:Connect(function(
        input: InputObject
    ): ()
        local placement: PendingMobilePlacement? = pendingMobilePlacement
        if not placement
            or input.UserInputType ~= Enum.UserInputType.Touch then
            return
        end
        pendingMobilePlacement = nil
        MobilePlacementOverlay.Visible = false
        createMobileAction(
            placement.id,
            placement.label,
            placement.callback,
            Vector2.new(input.Position.X, input.Position.Y),
            placement.hold
        )
        if placement.restoreMenu then
            state.setMenuVisible(true)
        end
        notify(placement.label .. " mobile button placed.")
    end))

    local function bindMobileActionPlacement(
        source: TextButton,
        id: string,
        label: string,
        callback: () -> (),
        hold: MobileHoldCallbacks?
    ): ()
        if not state.isMobile then
            return
        end

        trackUiConnection(source.MouseButton1Click:Connect(function(): ()
            -- A placed button opens its editor; only a missing one starts the
            -- placement flow. Tapping the card slot is therefore "configure".
            local existing: MobileActionRecord? = mobileActions[id]
            if existing then
                openEditor(existing)
            else
                beginMobileActionPlacement(id, label, callback, hold)
            end
        end))

        if ensureMobileConfig()[id] ~= nil then
            createMobileAction(id, label, callback, nil, hold)
        end
    end
    state.bindMobileActionPlacement = bindMobileActionPlacement

    local function resetMobileActions(): ()
        local ids: {string} = {}
        for id: string in pairs(mobileActions) do
            table.insert(ids, id)
        end
        for _, id: string in ipairs(ids) do
            removeMobileAction(id)
        end
        configData.ui.mobileActions = {}
        queueConfigSave()
    end
    state.mobileUi = {
        actions = mobileActions,
        setActionVisibility = setMobileActionVisibility,
        resetActions = resetMobileActions,
        clampButtonPosition = clampMobileButtonPosition,
        saveActionPosition = saveMobileActionPosition,
        editor = editorOverlay,
        openEditor = openEditor,
        closeEditor = closeEditor,
        place = beginMobileActionPlacement,
        defaults = ensureMobileDefaults,
        shapes = MOBILE_SHAPES,
        palette = MOBILE_PALETTE,
        style = applyMobileStyle,
        colorFromHex = colorFromHex,
        colorToHex = colorToHex,
        notifyChanged = notifyMobileChanged,
        onChanged = function(listener: () -> ()): ()
            table.insert(changeListeners, listener)
        end,
        actionId = function(shortcutId: string): string
            return "Shortcut" .. tostring(shortcutId):gsub("%W", "")
        end,
    }

    state.addMenuVisibilityListener(setMobileActionVisibility)

    Module.Initialized = true
    return state.mobileUi
end

function Module.destroy(): ()
    Module.Initialized = false
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/guis/Wurst/Code/MobileActions.lua
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
ARCHIVO: src/guis/Wurst/Code/SettingsPage.lua
BYTES: 84679
SHA-256: c53b97fcc702b50b08be87cea314f11d1f6a26440749bf1fd45df57e8dac461a
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
local Module = {
    Name = "SettingsPage",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local activeCleanup: (() -> ())? = nil

local WINDOW_WIDTH: number = 214
local ROW_HEIGHT: number = 22
local WINDOW_GAP: number = 2

local TITLE_HEIGHT: number = 26

function Module.init(context: any): any

    local OPACITY: number = 0.85
    local TOOLTIP_OPACITY: number = 0.75
    local MAX_HEIGHT: number = 200
    local MAX_SETTINGS_HEIGHT: number = 200
    local HACKLIST_MODE: string = "Auto"
    local HACKLIST_POSITION: string = "Left"
    local HACKLIST_COLOR: Color3 = Color3.fromRGB(255, 255, 255)
    local HACKLIST_SORT_BY: string = "Name"
    local HACKLIST_REVERSE: boolean = false
    local HACKLIST_ANIMATIONS: boolean = true
    local HACKLIST_SIZE: string = "Normal"
    local HACKLIST_RAINBOW: boolean = false

    local host: any = context
    local state: any = host.state
    local configData: any = host.configData
    local queueConfigSave: any = host.queueConfigSave
    local create: any = host.create
    local makeButton: any = host.makeButton
    local makeTextBox: any = host.makeTextBox
    local notify: any = host.notify
    local Theme: any = host.Theme
    local ThemeEngine: any = host.ThemeEngine
    local CONTROL_FONT: any = host.CONTROL_FONT
    local UserInputService: any = host.UserInputService
    local trackUiConnection: any = host.trackUiConnection
    local beginKeyCapture: any = host.beginKeyCapture
    local setKeySlotCapture: any = host.setKeySlotCapture
    local bindActionShortcut: any = host.bindActionShortcut
    local GAME_CHECK: any = host.GAME_CHECK
    local ScreenGui: any = host.ScreenGui
    local allFeatures: {any} = host.allFeatures or {}
    local REPOSITORY_RAW_BASE: any = host.REPOSITORY_RAW_BASE
    local RUNTIME_SAFETY_SOURCE_URL: any = host.RUNTIME_SAFETY_SOURCE_URL
    local RUNTIME_COMPATIBILITY_MARKER: any = host.RUNTIME_COMPATIBILITY_MARKER
    local SOURCE_STAMP: any = host.SOURCE_STAMP

    local addToggleOption: any = host.addToggleOption
    local addNumberOption: any = host.addNumberOption
    local addCycleOption: any = host.addCycleOption
    local addColorOption: any = host.addColorOption

    local drawPixelIcon: any = host.drawPixelIcon
        or function(parent: Instance, _kind: string, _color: Color3?): Frame
            local existing: Instance? = (parent :: any):FindFirstChild("PixelIcon")
            if existing then
                existing:Destroy()
            end
            return create("Frame", {
                Parent = parent,
                Name = "PixelIcon",
                BackgroundTransparency = 1,
            }) :: Frame
        end

    local windows: any = state.windows
    if not windows then
        error("SettingsPage requires the window manager")
    end

    local Settings: any = {
        records = {} :: {any},
        connections = {} :: {RBXScriptConnection},
    }

    local function viewport(): Vector2

        if type(windows.LogicalViewport) == "function" then
            return windows.LogicalViewport()
        end
        local camera: Camera? = workspace.CurrentCamera
        return camera and (camera :: Camera).ViewportSize or Vector2.new(1280, 720)
    end

    local maxSettingsHeight: number = MAX_SETTINGS_HEIGHT
    state.uiMaxHeight = MAX_HEIGHT

    local function textWidth(text: any): number
        local bitmapText: any = state.bitmapText
        if bitmapText and type(bitmapText.measure) == "function" then
            local measured: number = tonumber(bitmapText.measure(
                {{text = tostring(text)}}, 16
            )) or 0
            if measured > 0 then
                return measured
            end
        end
        return #tostring(text) * 12
    end
    local function packedWidth(rows: {{any}}): number
        local widest: number = 150
        for _, row: {any} in ipairs(rows) do
            local labelWidth: number = textWidth(row[1])
            local valueWidth: number = row[2] ~= nil and textWidth(row[2]) or 0
            local extra: number = tonumber(row[3]) or 0
            widest = math.max(widest, 8 + labelWidth + 24 + valueWidth + extra)
        end
        return widest
    end

    local function configWindow(
        id: string,
        title: string,
        definition: any
    ): any

        local packRows: {{any}}? = definition.packRows
        local width: number = packRows and packedWidth(packRows)
            or tonumber(definition.width)
            or WINDOW_WIDTH
        local window: any = windows.Create({
            id = id,
            title = title,
            size = Vector2.new(width, 30),
            position = definition.position,
            closable = definition.closable ~= false,
        })

        window.managementWindow = true
```

```
    if definition.anchor ~= nil then
        window.placementAnchor = definition.anchor
    end

    do
        local bitmapText: any = state.bitmapText
        if bitmapText and type(bitmapText.adoptTree) == "function" then
            table.insert(
                window.connections,
                bitmapText.adoptTree(window.body)
            )
        end
    end
    local layout: UIListLayout = (create("UIListLayout", {
        Parent = window.body,
        Padding = UDim.new(0, WINDOW_GAP),
        SortOrder = Enum.SortOrder.LayoutOrder,
    }) :: any) :: UIListLayout
    create("UIPadding", {
        Parent = window.body,
        PaddingTop = UDim.new(0, WINDOW_GAP),
        PaddingBottom = UDim.new(0, WINDOW_GAP),
        PaddingLeft = UDim.new(0, WINDOW_GAP),
        PaddingRight = UDim.new(0, WINDOW_GAP),
    })

    local record: any = {
        window = window,
        layout = layout,
        open = false,
        width = width,
        packRows = packRows,
    }
    local function resize(): ()
        local limit: number
        if definition.settingsSized then
            limit = maxSettingsHeight
        else
            limit = tonumber(state.uiMaxHeight) or MAX_HEIGHT
        end
        if limit <= 0 then
            limit = math.huge
        end

        if type(windows.LogicalViewport) == "function" then
            limit = math.min(limit, windows.LogicalViewport().Y - 12)
        end
        local wanted: number = math.min(
            TITLE_HEIGHT + layout.AbsoluteContentSize.Y + WINDOW_GAP * 2,
            limit
        )
        window.fullHeight = wanted
        if not window.collapsed then
            window.root.Size = UDim2.fromOffset(record.width, wanted)
        end

        if windows.ReflowWindow then
            windows.ReflowWindow(window)
        elseif windows.ClampWindow then
            windows.ClampWindow(window)
        end
    end
    record.resize = resize

    record.repack = function(rowsOverride: {{any}}?): ()
        if rowsOverride ~= nil then
            record.packRows = rowsOverride
        end
        if record.packRows == nil then
            return
        end
        local packed: number = packedWidth(record.packRows)
        if packed ~= record.width then
            record.width = packed
            resize()
        end
    end
    table.insert(
        Settings.connections,
        layout:GetPropertyChangedSignal("AbsoluteContentSize"):Connect(resize)
    )
    resize()
    table.insert(Settings.records, record)
    return record
end

local function panelFor(record: any, configKey: string): any
    local panel: any = {
        options = record.window.body,
        optionCount = 0,
        optionRows = {},
        configKey = configKey,
        searchText = "",
    }
    panel.updateExpandedSize = function(): ()
        record.resize()
    end
    return panel
end

local function actionRow(
    record: any,
    order: number,
    labelText: string,
    onClick: () -> ()
): TextButton
    local row: TextButton = (create("TextButton", {
        TextTruncate = Enum.TextTruncate.AtEnd,
        Parent = record.window.body,
        Name = "Row_" .. labelText:gsub("%W", ""),
        Active = true,
        AutoButtonColor = false,
        BackgroundColor3 = Color3.fromRGB(0, 0, 0),
        BackgroundTransparency = 0.5,
        BorderSizePixel = 0,
        FontFace = CONTROL_FONT,
        LayoutOrder = order,
        Size = UDim2.new(1, 0, 0, ROW_HEIGHT),
        Text = "",
        ZIndex = record.window.body.ZIndex + 1,
    }) :: any) :: TextButton
    create("TextLabel", {
        TextTruncate = Enum.TextTruncate.AtEnd,
        Parent = row,
        Name = "Label",
        BackgroundTransparency = 1,
        FontFace = CONTROL_FONT,
        Position = UDim2.fromOffset(6, 0),
        Size = UDim2.new(1, -28, 1, 0),
        Text = labelText,
        TextColor3 = Theme.text,
        TextSize = 16,
        TextXAlignment = Enum.TextXAlignment.Left,
        ZIndex = row.ZIndex + 1,
    })
    local arrowBox: Frame = (create("Frame", {
        Parent = row,
        Name = "Arrow",
        AnchorPoint = Vector2.new(1, 0),
        BackgroundTransparency = 1,
        BorderSizePixel = 0,
        Position = UDim2.new(1, -4, 0, 0),
        Size = UDim2.fromOffset(22, ROW_HEIGHT),
        ZIndex = row.ZIndex + 1,
    }) :: any) :: Frame

    drawPixelIcon(arrowBox, "right", nil, "arrows")
    trackUiConnection(row.MouseEnter:Connect(function(): ()
        row.BackgroundTransparency = 0.3
    end))
    trackUiConnection(row.MouseLeave:Connect(function(): ()
        row.BackgroundTransparency = 0.5
    end))
    trackUiConnection(row.MouseButton1Click:Connect(onClick))
    record.resize()
    return row
end

local function raiseOpen(record: any): ()
    record.open = true

    if not record.placed or record.window.stalePlacement then
        record.placed = true
        record.window.stalePlacement = nil
        if windows.PlaceWindow then
            windows.PlaceWindow(record.window)
        end
    end
    record.window:SetVisible(true)
end

local uiRecord: any = nil

local hackListRecord: any = nil
local function openHackList(): ()
    if hackListRecord then
        raiseOpen(hackListRecord)
        return
    end
    local screen: Vector2 = viewport()
    hackListRecord = configWindow("HackList", "HackList", {
        position = Vector2.new(screen.X - WINDOW_WIDTH * 2 - 32, 180),
        anchor = uiRecord and uiRecord.window,
        settingsSized = true,
        packRows = {
            {"Show", nil, 30},
            {"Mode", "Hidden", 26},
            {"Position", "Right", 26},
            {"Color", "#FFFFFF"},
            {"Sort by", "Width", 26},
            {"Reverse sorting", nil, 30},
            {"Animations", nil, 30},
            {"Size", "Normal", 26},
            {"Rainbow", nil, 30},
        },
    })
    local panel: any = panelFor(hackListRecord, "HackList")
    local showStored: boolean = configData.states["HackList.Show"] == true
    addToggleOption(
        panel,
        "Show",
        showStored,
        function(value: boolean): ()
            state.hudListAlways = value
            -- v181: one real switch behind the overlay. It is saved, so the
            -- list no longer shows up on an open menu with nothing enabled.
            configData.states["HackList.Show"] = value
            queueConfigSave()
            if state.hudList and type(state.hudList.SetAlways) == "function" then
                state.hudList.SetAlways(value)
            end
            if state.hudList and type(state.hudList.SetVisible) == "function" then
                state.hudList.SetVisible(value)
            end
        end,
        "Show the hack list. OFF keeps it off the screen, even with the menu open."
    )
    state.hudListAlways = showStored
    if state.hudList and type(state.hudList.SetAlways) == "function" then
        state.hudList.SetAlways(showStored)
    end
    local modes: {string} = {"Auto", "Count", "Hidden"}
    addCycleOption(
        panel,
        "Mode",
        modes,
        table.find(modes, HACKLIST_MODE) or 1,
        function(value: string): ()
            if state.hudList then
                state.hudList.SetMode(value)
            end
        end
    )
    local positions: {string} = {"Left", "Right", "Custom"}
    addCycleOption(
        panel,
        "Position",
        positions,
        table.find(positions, HACKLIST_POSITION) or 1,
        function(value: string): ()
            if state.hudList then
                state.hudList.SetPosition(value)
            end
        end
    )
    addColorOption(panel, "Color", HACKLIST_COLOR, function(color: Color3): ()
        if state.hudList then
            state.hudList.SetColor(color)
        end
    end)
    local sorts: {string} = {"Name", "Width"}
    addCycleOption(
        panel,
        "Sort by",
        sorts,
        table.find(sorts, HACKLIST_SORT_BY) or 1,
        function(value: string): ()
            if state.hudList then
                state.hudList.SetSortBy(value)
            end
        end
    )
    addToggleOption(panel, "Reverse sorting", HACKLIST_REVERSE, function(value: boolean): ()
        if state.hudList then
            state.hudList.SetReverse(value)
        end
    end)
    addToggleOption(panel, "Animations", HACKLIST_ANIMATIONS, function(value: boolean): ()
        if state.hudList then
            state.hudList.SetAnimations(value)
        end
    end)
    local sizes: {string} = {"Small", "Normal", "Large"}
    addCycleOption(
        panel,
        "Size",
        sizes,
        table.find(sizes, HACKLIST_SIZE) or 2,
        function(value: string): ()
            if state.hudList and type(state.hudList.SetSize) == "function" then
                state.hudList.SetSize(value)
            end
        end
    )
    addToggleOption(panel, "Rainbow", HACKLIST_RAINBOW, function(value: boolean): ()
        if state.hudList and type(state.hudList.SetRainbow) == "function" then
            state.hudList.SetRainbow(value)
        end
    end)
    raiseOpen(hackListRecord)
end

local keybindsRecord: any = nil
local profilesRecord: any = nil
local keybindsList: Frame? = nil

local function featureLabel(bindingId: string): string
    for _, feature: any in ipairs(allFeatures) do
        if feature.configKey == bindingId then
            return tostring(feature.name)
        end
    end
    return bindingId
end

local function keyLabel(name: any): string
    if type(state.keyDisplayName) == "function" then
        return state.keyDisplayName(name)
    end
    return tostring(name or "")
end

local function persistBindingKey(binding: any): ()
    local optionKey: string = "Shortcut." .. tostring(binding.id):gsub("%W", "")
    if binding.key == Enum.KeyCode.Unknown then
        configData.values[optionKey] = nil
    else
        configData.values[optionKey] = binding.key.Name
    end
    queueConfigSave()
end

local rebuildKeybinds: () -> () = function(): () end

local function sortedBindings(): {any}
    local list: {any} = {}
    for _, binding: any in pairs(state.shortcutBindings) do
        table.insert(list, binding)
    end
    table.sort(list, function(left: any, right: any): boolean
        return featureLabel(tostring(left.id)) < featureLabel(tostring(right.id))
    end)
    return list
end

rebuildKeybinds = function(): ()
    local container: Frame? = keybindsList
    if not container then
        return
    end
    for _, child: Instance in ipairs((container :: Frame):GetChildren()) do
        if child:IsA("GuiObject") then
            child:Destroy()
        end
    end

    local keyUses: {[string]: number} = {}
    for _, binding: any in pairs(state.shortcutBindings) do
        if binding.key and binding.key ~= Enum.KeyCode.Unknown then
            local name: string = binding.key.Name
            keyUses[name] = (keyUses[name] or 0) + 1
        end
    end

    local order: number = 0

    local function keybindRow(
        labelText: string,
        slotText: string,
        conflicted: boolean,
        onSlot: (TextButton) -> (),
        onRemove: (() -> ())?
    ): ()
        order += 1
        local row: Frame = (create("Frame", {
            Parent = container,
            Name = "Bind_" .. labelText:gsub("%W", ""),
            BackgroundTransparency = 1,
            BorderSizePixel = 0,
            LayoutOrder = order,
            Size = UDim2.new(1, 0, 0, ROW_HEIGHT),
            ZIndex = (container :: Frame).ZIndex + 1,
        }) :: any) :: Frame
        create("TextLabel", {
            TextTruncate = Enum.TextTruncate.AtEnd,
            Parent = row,
            Name = "Label",
            BackgroundTransparency = 1,
            FontFace = CONTROL_FONT,
            Position = UDim2.fromOffset(6, 0),
            Size = UDim2.new(1, -160, 1, 0),
            Text = labelText,
            TextColor3 = Theme.text,
            TextSize = 16,
            TextXAlignment = Enum.TextXAlignment.Left,
            ZIndex = row.ZIndex + 1,
        })
        local slot: TextButton = (create("TextButton", {
            TextTruncate = Enum.TextTruncate.AtEnd,
            Parent = row,
            Name = "Key",
            Active = true,
            AnchorPoint = Vector2.new(1, 0),
            AutoButtonColor = false,
            BackgroundTransparency = 1,
            BorderSizePixel = 0,
            FontFace = CONTROL_FONT,
            Position = UDim2.new(1, -6, 0, 0),
            Size = UDim2.fromOffset(150, ROW_HEIGHT),
            Text = slotText,
            TextColor3 = conflicted and Theme.negative or Theme.textMuted,
            TextSize = 16,
            TextXAlignment = Enum.TextXAlignment.Right,
            ZIndex = row.ZIndex + 1,
        }) :: any) :: TextButton
        slot:SetAttribute("CaptureIdleText", slotText)
        trackUiConnection(slot.MouseButton1Click:Connect(function(): ()
            onSlot(slot)
        end))

        if onRemove then
            trackUiConnection(slot.InputBegan:Connect(function(
                input: InputObject
            ): ()
                if input.UserInputType == Enum.UserInputType.MouseButton2 then
                    (onRemove :: () -> ())()
                end
            end))
        end
    end

    if not UserInputService.TouchEnabled then
        keybindRow("Open menu", keyLabel(state.toggleKey.Name), false, function(slot: TextButton): ()
            if state.keyCaptureCancel then
                state.keyCaptureCancel()
            end
            state.keybindButton = slot
            state.waitingForKey = true
            slot.Text = "..."
            setKeySlotCapture(slot, true)
        end)
    end

    for _, binding: any in ipairs(sortedBindings()) do
        local bound: boolean = binding.key ~= nil
            and binding.key ~= Enum.KeyCode.Unknown
        if not bound then
            continue
        end
        local slotText: string = keyLabel(binding.key.Name)
        local conflicted: boolean = (keyUses[binding.key.Name] or 0) > 1
        keybindRow(featureLabel(tostring(binding.id)), slotText, conflicted, function(slot: TextButton): ()
            beginKeyCapture(slot, function(keyCode: Enum.KeyCode): ()

                binding.key = keyCode == binding.key
                    and Enum.KeyCode.Unknown
                    or keyCode
                persistBindingKey(binding)
                if binding.refresh then
                    binding.refresh()
                end
                rebuildKeybinds()
            end)
        end, function(): ()

            binding.key = Enum.KeyCode.Unknown
            persistBindingKey(binding)
            if binding.refresh then
                binding.refresh()
            end
            rebuildKeybinds()
            notify(featureLabel(tostring(binding.id)) .. " unbound")
        end)
    end

    if keybindsRecord then

        local rows: {{any}} = {
            {"Open menu", "RShift", 12},
            {"Reset keybinds", nil, 26},
            {"Add", nil, 26},
            {"Profiles…", nil, 26},
        }
        for _, binding: any in ipairs(sortedBindings()) do
            if binding.key and binding.key ~= Enum.KeyCode.Unknown then
                table.insert(
                    rows,
                    {featureLabel(tostring(binding.id)), keyLabel(binding.key.Name), 12}
                )
            end
        end
        if keybindsRecord.repack then
            keybindsRecord.repack(rows)
        end
        keybindsRecord.resize()
    end
end

local function profileStore(): any
    if type(configData.values.Profiles) ~= "table" then
        configData.values.Profiles = {}
    end
    return configData.values.Profiles
end

-- Profile stores must never be captured inside a profile.
local PROFILE_STORE_KEYS: {[string]: boolean} = {
    Profiles = true,
    KeybindProfiles = true,
}

local function captureProfile(): any
    local states: {[string]: any} = {}
    for key: string, value: any in pairs(configData.states) do
        states[key] = value
    end
    local values: {[string]: any} = {}
    for key: string, value: any in pairs(configData.values) do
        if not PROFILE_STORE_KEYS[key] then
            values[key] = value
        end
    end
    local shortcuts: {[string]: string} = {}
    for id: string, binding: any in pairs(state.shortcutBindings) do
        if binding.key and binding.key ~= Enum.KeyCode.Unknown then
            shortcuts[id] = binding.key.Name
        end
    end
    local enabledModules: number = 0
    for _, feature: any in ipairs(allFeatures) do
        if configData.states[feature.configKey] == true then
            enabledModules += 1
        end
    end
    local optionCount: number = 0
    for _ in pairs(values) do
        optionCount += 1
    end
    return {
        savedAt = os.time(),
        states = states,
        values = values,
        shortcuts = shortcuts,
        enabledModules = enabledModules,
        optionCount = optionCount,
    }
end

local function applyProfile(name: string): boolean
    local snapshot: any = profileStore()[name]
    if type(snapshot) ~= "table" then
        return false
    end
    -- 1) Saved config first: states are replaced wholesale, values merge
    --    (the profile stores themselves are never captured).
    for key: string in pairs(configData.states) do
        configData.states[key] = nil
    end
    for key: string, value: any in pairs(snapshot.states) do
        configData.states[key] = value
    end
    for key: string, value: any in pairs(snapshot.values) do
        if not PROFILE_STORE_KEYS[key] then
            configData.values[key] = value
        end
    end
    -- 2) Keybinds.
    for _, binding: any in pairs(state.shortcutBindings) do
        local savedName: any = snapshot.shortcuts
            and snapshot.shortcuts[tostring(binding.id)]
        local savedKey: any = Enum.KeyCode.Unknown
        if type(savedName) == "string" and savedName ~= "" then
            local resolved: boolean, candidate: any = pcall(function(): any
                return (Enum.KeyCode :: any)[savedName]
            end)
            if resolved then
                savedKey = candidate
            end
        end
        if savedKey == Enum.KeyCode.Unknown then
            if binding.key ~= Enum.KeyCode.Unknown
                and type(binding.assign) == "function" then
                binding.assign(binding.key)
            end
        elseif binding.key ~= savedKey
            and type(binding.assign) == "function" then
            binding.assign(savedKey)
        elseif type(binding.refresh) == "function" then
            binding.refresh()
        end
    end
    -- 3) Module masters: the cards flip with full UI sync.
    for _, feature: any in ipairs(allFeatures) do
        local wanted: boolean = snapshot.states[feature.configKey] == true
        if feature.enabled ~= wanted
            and type(feature.activate) == "function" then
            pcall(feature.activate)
        end
    end
    -- 4) Option toggles.
    for key: string, control: any in pairs(state.toggleControls or {}) do
        local saved: any = snapshot.states[key]
        if saved ~= nil and type(control.setChecked) == "function" then
            pcall(control.setChecked, saved == true)
        end
    end
    -- 5) Option values (sliders, colours, cycles, text).
    for key: string, control: any in pairs(state.valueControls or {}) do
        local saved: any = snapshot.values[key]
        if saved ~= nil and type(control.apply) == "function" then
            pcall(control.apply, saved)
        end
    end
    queueConfigSave()
    return true
end

local function panicReset(): ()
    -- Every module off (cards flip with full UI sync).
    for _, feature: any in ipairs(allFeatures) do
        if feature.enabled and type(feature.activate) == "function" then
            pcall(feature.activate)
        end
    end
    -- Every option toggle off.
    for _, control: any in pairs(state.toggleControls or {}) do
        if type(control.setChecked) == "function" then
            pcall(control.setChecked, false)
        end
    end
    -- Every option value back to its default, live on screen.
    for _, control: any in pairs(state.valueControls or {}) do
        if control.default ~= nil and type(control.apply) == "function" then
            pcall(control.apply, control.default)
        end
    end
    -- Wipe the saved config. Profile stores and keybinds survive so a
    -- saved profile can bring everything back.
    for key: string in pairs(configData.states) do
        configData.states[key] = nil
    end
    for key: string in pairs(configData.values) do
        if not PROFILE_STORE_KEYS[key]
            and not string.match(key, "^Shortcut%.") then
            configData.values[key] = nil
        end
    end
    queueConfigSave()
    notify("panic: every module and option is back to defaults")
end

local rebuildProfiles: () -> () = function(): () end
local profilesList: Frame? = nil

local function openProfiles(): ()
    if profilesRecord then
        raiseOpen(profilesRecord)
        rebuildProfiles()
        return
    end
    local screen: Vector2 = viewport()
    profilesRecord = configWindow("Profiles", "Profiles", {
        position = Vector2.new(screen.X - WINDOW_WIDTH * 2 - 32, 260),
        anchor = keybindsRecord and keybindsRecord.window,
        settingsSized = true,
    })

    local entryRow: Frame = (create("Frame", {
        Parent = profilesRecord.window.body,
        Name = "NewProfile",
        BackgroundTransparency = 1,
        BorderSizePixel = 0,
        LayoutOrder = 1,
        Size = UDim2.new(1, 0, 0, ROW_HEIGHT),
```

```
        ZIndex = profilesRecord.window.body.ZIndex + 1,
    }) :: any) :: Frame
    local nameBox: TextBox = (makeTextBox(entryRow, "") :: any) :: TextBox
    nameBox.PlaceholderText = "New profile name"
    nameBox.Position = UDim2.fromOffset(0, 0)
    nameBox.Size = UDim2.new(1, -170, 1, 0)
    nameBox.TextSize = 14
    nameBox.ZIndex = entryRow.ZIndex + 1
    local saveButton: TextButton = (makeButton(entryRow, "SAVE", 10) :: any) :: TextButton
    saveButton.AnchorPoint = Vector2.new(1, 0.5)
    saveButton.Position = UDim2.new(1, -66, 0.5, 0)
    saveButton.Size = UDim2.fromOffset(56, 20)
    saveButton.ZIndex = entryRow.ZIndex + 1
    local panicButton: TextButton = (makeButton(entryRow, "PANIC", 10) :: any) :: TextButton
    panicButton.AnchorPoint = Vector2.new(1, 0.5)
    panicButton.Position = UDim2.new(1, 0, 0.5, 0)
    panicButton.Size = UDim2.fromOffset(56, 20)
    panicButton.ZIndex = entryRow.ZIndex + 1
    trackUiConnection(panicButton.MouseButton1Click:Connect(function(): ()
        panicReset()
        rebuildProfiles()
    end))

    profilesList = (create("Frame", {
        Parent = profilesRecord.window.body,
        Name = "ProfileList",
        AutomaticSize = Enum.AutomaticSize.Y,
        BackgroundTransparency = 1,
        BorderSizePixel = 0,
        LayoutOrder = 2,
        Size = UDim2.new(1, 0, 0, 0),
        ZIndex = profilesRecord.window.body.ZIndex + 1,
    }) :: any) :: Frame
    create("UIListLayout", {
        Parent = profilesList,
        Padding = UDim.new(0, WINDOW_GAP),
        SortOrder = Enum.SortOrder.LayoutOrder,
    })

    rebuildProfiles = function(): ()
        local container: Frame? = profilesList
        if not container then
            return
        end
        for _, child: Instance in ipairs((container :: Frame):GetChildren()) do
            if child:IsA("GuiObject") then
                child:Destroy()
            end
        end
        local names: {string} = {}
        for name: string in pairs(profileStore()) do
            table.insert(names, name)
        end
        table.sort(names, function(a: string, b: string): boolean
            local left: any = profileStore()[a]
            local right: any = profileStore()[b]
            local leftAt: number = type(left) == "table"
                and tonumber(left.savedAt) or 0
            local rightAt: number = type(right) == "table"
                and tonumber(right.savedAt) or 0
            if leftAt ~= rightAt then
                return leftAt > rightAt
            end
            return a < b
        end)
        for index: number, name: string in ipairs(names) do
            local snapshot: any = profileStore()[name]
            local row: Frame = (create("Frame", {
                Parent = container,
                Name = "Profile_" .. name:gsub("%W", ""),
                BackgroundTransparency = 1,
                BorderSizePixel = 0,
                LayoutOrder = index,
                Size = UDim2.new(1, 0, 0, 44),
                ZIndex = (container :: Frame).ZIndex + 1,
            }) :: any) :: Frame
            create("TextLabel", {
                TextTruncate = Enum.TextTruncate.AtEnd,
                Parent = row,
                Name = "Label",
                BackgroundTransparency = 1,
                FontFace = CONTROL_FONT,
                Position = UDim2.fromOffset(6, 2),
                Size = UDim2.new(1, -180, 0, 18),
                Text = name,
                TextColor3 = Theme.text,
                TextSize = 14,
                TextXAlignment = Enum.TextXAlignment.Left,
                ZIndex = row.ZIndex + 1,
            })
            local detail: string = "profile"
            if type(snapshot) == "table" then
                local savedAt: number = tonumber(snapshot.savedAt) or 0
                local when: string = savedAt > 0
                    and os.date("%Y-%m-%d %H:%M", savedAt)
                    or "unknown date"
                detail = when
                    .. " · "
                    .. tostring(snapshot.enabledModules or 0)
                    .. " modules on · "
                    .. tostring(snapshot.optionCount or 0)
                    .. " options"
            end
            create("TextLabel", {
                TextTruncate = Enum.TextTruncate.AtEnd,
                Parent = row,
                Name = "Detail",
                BackgroundTransparency = 1,
                FontFace = CONTROL_FONT,
                Position = UDim2.fromOffset(6, 22),
                Size = UDim2.new(1, -180, 0, 16),
                Text = detail,
                TextColor3 = Theme.textMuted,
                TextSize = 12,
                TextXAlignment = Enum.TextXAlignment.Left,
                ZIndex = row.ZIndex + 1,
            })
            local loadButton: TextButton =
                (makeButton(row, "LOAD", 10) :: any) :: TextButton
            loadButton.AnchorPoint = Vector2.new(1, 0.5)
            loadButton.Position = UDim2.new(1, -56, 0.5, 0)
            loadButton.Size = UDim2.fromOffset(48, 18)
            loadButton.ZIndex = row.ZIndex + 1
            trackUiConnection(loadButton.MouseButton1Click:Connect(function(): ()
                if applyProfile(name) then
                    notify("loaded profile " .. name)
                    rebuildProfiles()
                else
                    notify("profile " .. name .. " is corrupted")
                end
            end))
            local updateButton: TextButton =
                (makeButton(row, "UPD", 10) :: any) :: TextButton
            updateButton.AnchorPoint = Vector2.new(1, 0.5)
            updateButton.Position = UDim2.new(1, -108, 0.5, 0)
            updateButton.Size = UDim2.fromOffset(44, 18)
            updateButton.ZIndex = row.ZIndex + 1
            trackUiConnection(updateButton.MouseButton1Click:Connect(function(): ()
                profileStore()[name] = captureProfile()
                queueConfigSave()
                notify("updated profile " .. name)
                rebuildProfiles()
            end))
            local deleteButton: TextButton = (create("TextButton", {
                Parent = row,
                Name = "Delete",
                AutoButtonColor = false,
                BackgroundColor3 = Color3.fromRGB(0, 0, 0),
                BackgroundTransparency = 0.5,
                BorderSizePixel = 0,
                FontFace = CONTROL_FONT,
                AnchorPoint = Vector2.new(1, 0.5),
                Position = UDim2.new(1, -6, 0.5, 0),
                Size = UDim2.fromOffset(18, 18),
                Text = "x",
                TextColor3 = Theme.negative,
                TextSize = 12,
                ZIndex = row.ZIndex + 1,
            }) :: any) :: TextButton
            trackUiConnection(deleteButton.MouseButton1Click:Connect(function(): ()
                profileStore()[name] = nil
                queueConfigSave()
                notify("deleted profile " .. name)
                rebuildProfiles()
            end))
        end
    if profilesRecord then
            if profilesRecord.repack then
                local rows: {{any}} = {{"New profile name", nil, 60}}
                for _, name: string in ipairs(names) do
                    table.insert(rows, {name, nil, 64})
                end
                profilesRecord.repack(rows)
            end
            profilesRecord.resize()
        end
    end

    trackUiConnection(saveButton.MouseButton1Click:Connect(function(): ()
        local name: string = nameBox.Text:gsub("^%s+", ""):gsub("%s+$", "")
        if name == "" then
            notify("enter a profile name")
            return
        end
        profileStore()[name] = captureProfile()
        queueConfigSave()
        nameBox.Text = ""
        rebuildProfiles()
        notify("saved profile " .. name)
    end))

    rebuildProfiles()
    raiseOpen(profilesRecord)
end

local addRecord: any = nil
local addList: Frame? = nil
local pendingConflict: any = nil
local rebuildAddList: () -> () = function(): () end

local function applyBinding(
    binding: any,
    keyCode: Enum.KeyCode,
    clearOther: any
): ()
    binding.key = keyCode
    persistBindingKey(binding)
    if binding.refresh then
        binding.refresh()
    end
    if clearOther then
        clearOther.key = Enum.KeyCode.Unknown
        persistBindingKey(clearOther)
        if clearOther.refresh then
            clearOther.refresh()
        end
    end
    pendingConflict = nil
    rebuildKeybinds()
    rebuildAddList()
    notify(featureLabel(tostring(binding.id)) .. " bound to " .. keyCode.Name)
end

rebuildAddList = function(): ()
    local container: Frame? = addList
    if not container then
        return
    end
    for _, child: Instance in ipairs((container :: Frame):GetChildren()) do
        if child:IsA("GuiObject") then
            child:Destroy()
        end
    end
    local order: number = 0
    local function addRow(
        labelText: string,
        valueText: string,
        warning: boolean,
        onClick: ((TextButton) -> ())?
    ): ()
        order += 1
        local row: TextButton = (create("TextButton", {
            TextTruncate = Enum.TextTruncate.AtEnd,
            Parent = container,
            Name = "Add_" .. labelText:gsub("%W", ""),
            Active = onClick ~= nil,
            AutoButtonColor = false,
            BackgroundColor3 = Color3.fromRGB(0, 0, 0),
            BackgroundTransparency = onClick ~= nil and 0.5 or 1,
            BorderSizePixel = 0,
            FontFace = CONTROL_FONT,
            LayoutOrder = order,
            Size = UDim2.new(1, 0, 0, ROW_HEIGHT),
            Text = "",
            ZIndex = (container :: Frame).ZIndex + 1,
        }) :: any) :: TextButton
        create("TextLabel", {
            TextTruncate = Enum.TextTruncate.AtEnd,
            Parent = row,
            Name = "Label",
            BackgroundTransparency = 1,
            FontFace = CONTROL_FONT,
            Position = UDim2.fromOffset(6, 0),
            Size = UDim2.new(1, -120, 1, 0),
            Text = labelText,
            TextColor3 = warning and Theme.negative or Theme.text,
            TextSize = 16,
            TextXAlignment = Enum.TextXAlignment.Left,
            ZIndex = row.ZIndex + 1,
        })
        local slot: TextButton = (create("TextButton", {
            TextTruncate = Enum.TextTruncate.AtEnd,
            Parent = row,
            Name = "Key",
            Active = onClick ~= nil,
            AnchorPoint = Vector2.new(1, 0),
            AutoButtonColor = false,
            BackgroundTransparency = 1,
            BorderSizePixel = 0,
            FontFace = CONTROL_FONT,
            Position = UDim2.new(1, -6, 0, 0),
            Size = UDim2.fromOffset(110, ROW_HEIGHT),
            Text = valueText,
            TextColor3 = Theme.textMuted,
            TextSize = 16,
            TextXAlignment = Enum.TextXAlignment.Right,
            ZIndex = row.ZIndex + 1,
        }) :: any) :: TextButton
        slot:SetAttribute("CaptureIdleText", valueText)
        if onClick then
            trackUiConnection(row.MouseButton1Click:Connect(function(): ()
                (onClick :: (TextButton) -> ())(slot)
            end))
            trackUiConnection(slot.MouseButton1Click:Connect(function(): ()
                (onClick :: (TextButton) -> ())(slot)
            end))
            trackUiConnection(row.MouseEnter:Connect(function(): ()
                row.BackgroundTransparency = 0.3
            end))
            trackUiConnection(row.MouseLeave:Connect(function(): ()
                row.BackgroundTransparency = 0.5
            end))
        end
    end

    if pendingConflict then

        local conflict: any = pendingConflict
        addRow(
            conflict.key.Name .. " already binds "
                .. featureLabel(tostring(conflict.other.id)),
            "",
            true,
            nil
        )
        addRow("Replace it", "", false, function(_slot: TextButton): ()
            applyBinding(conflict.binding, conflict.key, conflict.other)
        end)
        addRow("Keep both", "", false, function(_slot: TextButton): ()
            applyBinding(conflict.binding, conflict.key, nil)
        end)
        addRow("Cancel", "", false, function(_slot: TextButton): ()
            pendingConflict = nil
            rebuildAddList()
        end)
    else
        addRow("Click an action, then press a key", "", false, nil)
        for _, binding: any in ipairs(sortedBindings()) do
            local bound: boolean = binding.key ~= nil
                and binding.key ~= Enum.KeyCode.Unknown
            addRow(
                featureLabel(tostring(binding.id)),
                bound and keyLabel(binding.key.Name) or "—",
                false,
                function(slot: TextButton): ()
                    beginKeyCapture(slot, function(keyCode: Enum.KeyCode): ()
                        local other: any = nil
                        for _, candidate: any in pairs(state.shortcutBindings) do
                            if candidate ~= binding
                                and candidate.key == keyCode then
                                other = candidate
                                break
                            end
                        end
                        if other then
                            pendingConflict = {
                                binding = binding,
                                key = keyCode,
                                other = other,
                            }
                            rebuildAddList()
                        else
                            applyBinding(binding, keyCode, nil)
                        end
                    end)
                end
            )
        end
    end

    if addRecord then
        if addRecord.repack then
            local rows: {{any}} = {
                {"Click an action, then press a key", nil, 12},
            }
            for _, binding: any in ipairs(sortedBindings()) do
                table.insert(rows, {
                    featureLabel(tostring(binding.id)),
                    binding.key and binding.key ~= Enum.KeyCode.Unknown
                        and binding.key.Name
                        or "—",
                    12,
                })
            end
            addRecord.repack(rows)
        end
        addRecord.resize()
    end
end

local function openAddKeybind(): ()
    pendingConflict = nil
    if addRecord then
        rebuildAddList()
        raiseOpen(addRecord)
        return
    end
    local screen: Vector2 = viewport()
    addRecord = configWindow("KeybindAdd", "Add Keybind", {
        position = Vector2.new(screen.X - WINDOW_WIDTH * 2 - 32, 200),
        anchor = keybindsRecord and keybindsRecord.window,
        settingsSized = true,
        packRows = {{"Click an action, then press a key", nil, 12}},
    })
    addList = (create("Frame", {
        Parent = addRecord.window.body,
        Name = "AddList",
        AutomaticSize = Enum.AutomaticSize.Y,
        BackgroundTransparency = 1,
        BorderSizePixel = 0,
        LayoutOrder = 1,
        Size = UDim2.new(1, 0, 0, 0),
        ZIndex = addRecord.window.body.ZIndex + 1,
    }) :: any) :: Frame
    create("UIListLayout", {
        Parent = addList,
        Padding = UDim.new(0, WINDOW_GAP),
        SortOrder = Enum.SortOrder.LayoutOrder,
    })
    rebuildAddList()
    raiseOpen(addRecord)
end

local function openKeybinds(): ()
    if keybindsRecord then
        raiseOpen(keybindsRecord)
        rebuildKeybinds()
        return
    end
    local screen: Vector2 = viewport()
    local keybindRows: {{any}} = {
        {"Reset keybinds", nil, 26},
        {"Open menu", keyLabel(state.toggleKey.Name), 12},
    }
    for _, binding: any in pairs(state.shortcutBindings) do
        if binding.key and binding.key ~= Enum.KeyCode.Unknown then
            table.insert(
                keybindRows,
                {featureLabel(tostring(binding.id)), binding.key.Name, 12}
            )
        end
    end
    keybindsRecord = configWindow("Keybinds", "Keybinds", {
        position = Vector2.new(screen.X - WINDOW_WIDTH * 2 - 32, 140),
        anchor = uiRecord and uiRecord.window,
        settingsSized = true,
        packRows = keybindRows,
    })
    keybindsList = (create("Frame", {
        Parent = keybindsRecord.window.body,
        Name = "KeybindList",
        AutomaticSize = Enum.AutomaticSize.Y,
        BackgroundTransparency = 1,
        BorderSizePixel = 0,
        LayoutOrder = 1,
        Size = UDim2.new(1, 0, 0, 0),
        ZIndex = keybindsRecord.window.body.ZIndex + 1,
    }) :: any) :: Frame
    create("UIListLayout", {
        Parent = keybindsList,
        Padding = UDim.new(0, WINDOW_GAP),
        SortOrder = Enum.SortOrder.LayoutOrder,
    })
    actionRow(keybindsRecord, 99, "Add", openAddKeybind)
    actionRow(keybindsRecord, 100, "Reset keybinds", function(): ()
        for _, binding: any in pairs(state.shortcutBindings) do
            binding.key = Enum.KeyCode.Unknown
            persistBindingKey(binding)
            if binding.refresh then
                binding.refresh()
            end
        end
        rebuildKeybinds()
        notify("keybinds reset")
    end)
    actionRow(keybindsRecord, 101, "Profiles…", openProfiles)
    rebuildKeybinds()
    raiseOpen(keybindsRecord)
end

-----------------------------------------------------------------------
-- Mobile controls. Keybinds need a keyboard, so on a touch device the
-- same slot opens the buttons page instead: every shortcut the menu
-- exposes can be placed as an on-screen button and restyled (shape,
-- colour, size, opacity) without editing the config by hand.
-----------------------------------------------------------------------
local buttonsRecord: any = nil
local buttonsList: Frame? = nil
local buttonsAddRecord: any = nil
local buttonsAddList: Frame? = nil

local function mobileCallbacks(binding: any): (() -> (), any)
    local hold: boolean = binding.hold == true
    local function press(): ()
        if type(binding.onPress) == "function" then
            binding.onPress()
        end
    end
    local function release(): ()
        if type(binding.onRelease) == "function" then
            binding.onRelease()
        end
    end
    local callback: () -> () = function(): ()
        if not hold then press() end
    end
    if not hold then
        return callback, nil
    end
    return callback, {
        isActive = function(): boolean return true end,
        onPress = press,
        onRelease = release,
    }
end

local function mobileUiOrNil(): any
    return state.mobileUi
end

local function buttonRow(
    parent: Frame,
    order: number,
    labelText: string,
    valueText: string,
    onClick: () -> ()
): TextButton
    local row: TextButton = (create("TextButton", {
        TextTruncate = Enum.TextTruncate.AtEnd,
        Parent = parent,
        Name = "ButtonRow_" .. labelText:gsub("%W", ""),
        Active = true,
        AutoButtonColor = false,
        BackgroundColor3 = Color3.fromRGB(0, 0, 0),
        BackgroundTransparency = 0.5,
        BorderSizePixel = 0,
        FontFace = CONTROL_FONT,
        LayoutOrder = order,
        Size = UDim2.new(1, 0, 0, 22),
        Text = "",
        ZIndex = parent.ZIndex + 1,
    }) :: any) :: TextButton
    create("TextLabel", {
        TextTruncate = Enum.TextTruncate.AtEnd,
        Parent = row,
        Name = "Label",
        BackgroundTransparency = 1,
        FontFace = CONTROL_FONT,
        Position = UDim2.fromOffset(6, 0),
        Size = UDim2.new(0.6, -6, 1, 0),
        Text = labelText,
        TextColor3 = Theme.text,
        TextSize = 14,
        TextXAlignment = Enum.TextXAlignment.Left,
        ZIndex = row.ZIndex + 1,
    })
    create("TextLabel", {
        TextTruncate = Enum.TextTruncate.AtEnd,
        Parent = row,
        Name = "Value",
        BackgroundTransparency = 1,
        FontFace = CONTROL_FONT,
        Position = UDim2.new(0.6, 0, 0, 0),
        Size = UDim2.new(0.4, -8, 1, 0),
        Text = valueText,
        TextColor3 = Theme.textMuted,
        TextSize = 13,
        TextXAlignment = Enum.TextXAlignment.Right,
        ZIndex = row.ZIndex + 1,
    })
    trackUiConnection(row.MouseButton1Click:Connect(onClick))
    return row
end

local rebuildButtons: () -> () = function(): () end
local openAddMobileButton: () -> () = function(): () end

rebuildButtons = function(): ()
    local container: Frame? = buttonsList
    local ui: any = mobileUiOrNil()
    if not container or not ui then
        return
    end
    container:ClearAllChildren()
    local order: number = 1
    local placed: number = 0
    local ids: {string} = {}
    for id: string in pairs(ui.actions) do
        table.insert(ids, id)
    end
    table.sort(ids)
    for _, id: string in ipairs(ids) do
        local record: any = ui.actions[id]
        if record then
            placed += 1
            local shape: string = tostring(record.shape or "Square")
            buttonRow(
                container,
                order,
                tostring(record.label),
                tostring(record.shape),
                function(): ()
                    ui.openEditor(record)
                end
            )
            order += 1
        end
    end
    if placed == 0 then
        buttonRow(container, order, "No buttons yet", "tap Add button", function(): () end)
        order += 1
    end
    local defaults: any = ui.defaults and ui.defaults()
    if defaults then
        buttonRow(
            container,
            order,
            "Default shape",
            tostring(defaults.shape),
            function(): ()
                local shapes: {string} = ui.shapes
                local index: number = table.find(shapes, defaults.shape) or 1
                defaults.shape = shapes[index % #shapes + 1]
                queueConfigSave()
                rebuildButtons()
                notify("new buttons: " .. defaults.shape)
            end
        )
        order += 1
        buttonRow(
            container,
            order,
            "Default colour",
            tostring(defaults.color),
            function(): ()
                local palette: {string} = ui.palette
                local index: number = table.find(palette, string.upper(defaults.color)) or 1
                defaults.color = palette[index % #palette + 1]
                queueConfigSave()
                rebuildButtons()
                notify("new buttons: " .. defaults.color)
            end
        )
        order += 1
    end
    if buttonsRecord then
        buttonsRecord.resize()
    end
end

local function openButtons(): ()
    local ui: any = mobileUiOrNil()
    if not ui then
        notify("touch controls are not ready yet")
        return
    end
    if buttonsRecord then
        raiseOpen(buttonsRecord)
        rebuildButtons()
        return
    end
    local screen: Vector2 = viewport()
    buttonsRecord = configWindow("Buttons", "Buttons", {
        position = Vector2.new(screen.X - WINDOW_WIDTH * 2 - 32, 140),
        anchor = uiRecord and uiRecord.window,
        settingsSized = true,
        packRows = {{"Add button", nil, 26}, {"Reset buttons", nil, 26}},
    })
    buttonsList = (create("Frame", {
        Parent = buttonsRecord.window.body,
        Name = "ButtonList",
        AutomaticSize = Enum.AutomaticSize.Y,
        BackgroundTransparency = 1,
```

```
            BorderSizePixel = 0,
            LayoutOrder = 1,
            Size = UDim2.new(1, 0, 0, 0),
            ZIndex = buttonsRecord.window.body.ZIndex + 1,
        }) :: any) :: Frame
        create("UIListLayout", {
            Parent = buttonsList,
            Padding = UDim.new(0, WINDOW_GAP),
            SortOrder = Enum.SortOrder.LayoutOrder,
        })
        actionRow(buttonsRecord, 90, "Add button", openAddMobileButton)
        actionRow(buttonsRecord, 91, "Reset buttons", function(): ()
            ui.resetActions()
            rebuildButtons()
            notify("mobile buttons reset")
        end)
        rebuildButtons()
        raiseOpen(buttonsRecord)
    end

    openAddMobileButton = function(): ()
        local ui: any = mobileUiOrNil()
        if not ui then
            return
        end
        if buttonsAddRecord then
            raiseOpen(buttonsAddRecord)
            return
        end
        local screen: Vector2 = viewport()
        buttonsAddRecord = configWindow("ButtonAdd", "Add button", {
            position = Vector2.new(screen.X - WINDOW_WIDTH * 2 - 32, 200),
            anchor = buttonsRecord and buttonsRecord.window,
            settingsSized = true,
            packRows = {{"Tap an action to place its button", nil, 12}},
        })
        buttonsAddList = (create("Frame", {
            Parent = buttonsAddRecord.window.body,
            Name = "ButtonAddList",
            AutomaticSize = Enum.AutomaticSize.Y,
            BackgroundTransparency = 1,
            BorderSizePixel = 0,
            LayoutOrder = 1,
            Size = UDim2.new(1, 0, 0, 0),
            ZIndex = buttonsAddRecord.window.body.ZIndex + 1,
        }) :: any) :: Frame
        create("UIListLayout", {
            Parent = buttonsAddList,
            Padding = UDim.new(0, WINDOW_GAP),
            SortOrder = Enum.SortOrder.LayoutOrder,
        })
        local order: number = 1
        for id: string, binding: any in pairs(state.shortcutBindings) do
            local actionId: string = ui.actionId(id)
            if not ui.actions[actionId] then
                local label: string = featureLabel(tostring(id))
                buttonRow(buttonsAddList :: Frame, order, label, "place", function(): ()
                    local callback: () -> (), hold: any = mobileCallbacks(binding)
                    ui.place(actionId, label, callback, hold)
                end)
                order += 1
            end
        end
        if order == 1 then
            buttonRow(buttonsAddList :: Frame, order, "Every action already has a button", "", function(): () end)
        end
        buttonsAddRecord.resize()
        raiseOpen(buttonsAddRecord)
    end

    local windowsRecord: any = nil
    local windowsPanel: any = nil
    local windowOptions: {[string]: boolean} = {}

    local function openWindows(): ()
        local clickGui: any = state.clickGui
        if not windowsRecord then
            local screen: Vector2 = viewport()
            windowsRecord = configWindow("Windows", "Windows", {
                position = Vector2.new(screen.X - WINDOW_WIDTH * 3 - 44, 140),
                anchor = uiRecord and uiRecord.window,
                settingsSized = true,
                packRows = {{"Combat", "ON", 20}},
            })
            windowsPanel = panelFor(windowsRecord, "ClickGUI.Windows")
        end
        if clickGui and windowsPanel then
            for _, name: string in ipairs(clickGui.order or {}) do
                if not windowOptions[name] then
                    windowOptions[name] = true
                    local category: any = clickGui.categories[name]
                    addToggleOption(windowsPanel, name, category.window.userVisible ~= false, function(value: boolean): ()
                        category.window.userVisible = value
                        category.window:SetVisible(value and state.visible == true)
                    end)
                end
            end
        end
        raiseOpen(windowsRecord)
    end

    local function applySavedWindowVisibility(): boolean
        local clickGui: any = state.clickGui
        if not clickGui or type(clickGui.order) ~= "table" then
            return false
        end
        for _, name: string in ipairs(clickGui.order) do
            local category: any = clickGui.categories
                and clickGui.categories[name]
            if category and category.window then
                local optionKey: string = "ClickGUI.Windows."
                    .. tostring(name):gsub("%W", "")
                if configData.states[optionKey] == false then
                    category.window.userVisible = false
                    pcall(function(): ()
                        category.window:SetVisible(false)
                    end)
                end
            end
        end
        return true
    end

    task.spawn(function(): ()

        for _ = 1, 15 do
            applySavedWindowVisibility()
            task.wait(1)
        end
    end)

    local optionsRecord: any = nil

    local actionEntries: {any} = {}
    local actionIndex: {[string]: any} = {}
    local actionSequence: number = 0
    local refreshActionRows: () -> () = function(): () end

    state.wurstOptions = {
        RegisterAction = function(id: any, label: any, callback: any, options: any?): any
            local key: string = tostring(id)
            local configuration: any = options or {}
            local entry: any = actionIndex[key]
            if entry == nil then
                actionSequence += 1
                entry = {id = key, order = actionSequence}
                actionIndex[key] = entry
                table.insert(actionEntries, entry)
            end
            entry.label = tostring(label)
            entry.callback = callback
            entry.enabled = configuration.enabled ~= false
            entry.visible = configuration.visible ~= false
            entry.static = configuration.static == true
            if entry.handle == nil then
                local handle: any = {}
                function handle.SetLabel(text: any): ()
                    entry.label = tostring(text)
                    if entry.rowLabel and entry.rowLabel.Parent then
                        entry.rowLabel.Text = entry.label
                    end
                end
                function handle.SetEnabled(value: boolean): ()
                    entry.enabled = value == true
                    refreshActionRows()
                end
                function handle.SetVisible(value: boolean): ()
                    entry.visible = value == true
                    refreshActionRows()
                end
                function handle.Unregister(): ()
                    state.wurstOptions.UnregisterAction(entry.id)
                end
                entry.handle = handle
            end
            if configuration.shortcut == true
                and entry.shortcutBound ~= true
                and type(bindActionShortcut) == "function" then
                entry.shortcutBound = true
                bindActionShortcut(nil, "Action " .. key, entry.label, function(): ()
                    state.wurstOptions.InvokeAction(key)
                end)
            end
            refreshActionRows()
            return entry.handle
        end,
        UnregisterAction = function(id: any): boolean
            local key: string = tostring(id)
            local entry: any = actionIndex[key]
            if entry == nil then
                return false
            end
            actionIndex[key] = nil
            local at: number? = table.find(actionEntries, entry)
            if at then
                table.remove(actionEntries, at)
            end
            refreshActionRows()
            return true
        end,
        InvokeAction = function(id: any): boolean
            local entry: any = actionIndex[tostring(id)]
            if entry == nil
                or entry.enabled ~= true
                or entry.visible ~= true
                or type(entry.callback) ~= "function" then
                return false
            end
            local ok: boolean, failure: any = pcall(entry.callback)
            if not ok then
                warn("[Wurst] Action " .. tostring(id) .. ": " .. tostring(failure))
            end
            return ok
        end,
        List = function(): {string}
            local ids: {string} = {}
            for _, entry: any in ipairs(actionEntries) do
                table.insert(ids, entry.id)
            end
            return ids
        end,
    }

    refreshActionRows = function(): ()
        if not optionsRecord then
            return
        end
        local body: any = optionsRecord.window.body
        for _, child: Instance in ipairs(body:GetChildren()) do
            if child:IsA("GuiObject")
                and child:GetAttribute("RegisteredAction") == true then
                child:Destroy()
            end
        end
        local packRows: {{any}} = {}
        for _, entry: any in ipairs(actionEntries) do
            if entry.visible then
                table.insert(packRows, {entry.label, nil, 26})
                if entry.static then

                    local label: TextLabel = (create("TextLabel", {
                        TextTruncate = Enum.TextTruncate.AtEnd,
                        Parent = body,
                        Name = "Static_" .. entry.id:gsub("%W", ""),
                        BackgroundTransparency = 1,
                        FontFace = CONTROL_FONT,
                        LayoutOrder = entry.order,
                        Size = UDim2.new(1, 0, 0, ROW_HEIGHT),
                        Text = "  " .. entry.label,
                        TextColor3 = Theme.textMuted,
                        TextSize = 14,
                        TextXAlignment = Enum.TextXAlignment.Left,
                        ZIndex = body.ZIndex + 1,
                    }) :: any) :: TextLabel
                    label:SetAttribute("RegisteredAction", true)
                    entry.rowLabel = label
                else
                    local row: TextButton = actionRow(
                        optionsRecord,
                        entry.order,
                        entry.label,
                        function(): ()
                            state.wurstOptions.InvokeAction(entry.id)
                        end
                    )
                    row:SetAttribute("RegisteredAction", true)
                    local rowLabel: any = row:FindFirstChild("Label")
                    entry.rowLabel = rowLabel
                    if not entry.enabled then
                        row.Active = false
                        if rowLabel then
                            rowLabel.TextColor3 = Theme.textMuted
                        end
                    end
                end
            end
        end
        if optionsRecord.repack then
            optionsRecord.repack(packRows)
        end
        optionsRecord.resize()
    end

    local reinjectBusy: boolean = false
    local reinjectHandle: any = nil
    local function runReinject(): ()
        local function setReinjectLabel(text: string): ()
            if reinjectHandle then
                reinjectHandle.SetLabel(text)
            end
        end
        if reinjectBusy then
            return
        end
        reinjectBusy = true
        setReinjectLabel("Downloading…")
        local sourceUrls: {string} = {
            REPOSITORY_RAW_BASE .. "src/Wurst.lua",
            RUNTIME_SAFETY_SOURCE_URL,
        }
        local sourceOrError: any = nil
        local downloaded: boolean = false
        local downloadFailures: {string} = {}
        for _, candidateUrl: string in ipairs(sourceUrls) do
            local requestUrl: string = candidateUrl
                .. "?reinject=v3&v="
                .. SOURCE_STAMP
            local requestOk: boolean, bodyOrError: any = pcall(function(): string
                return (game :: any):HttpGet(requestUrl, true)
            end)
            local current: boolean = requestOk
                and type(bodyOrError) == "string"
                and #bodyOrError >= 1024
                and bodyOrError:find(RUNTIME_COMPATIBILITY_MARKER, 1, true) ~= nil
            if current then
                downloaded = true
                sourceOrError = bodyOrError
                break
            end
            table.insert(downloadFailures, tostring(bodyOrError))
        end
        if not downloaded then
            reinjectBusy = false
            setReinjectLabel("Reinject latest")
            notify("download failed")
            warn("[Wurst] Reinject download: "
                .. table.concat(downloadFailures, " | "))
            return
        end
        local compiled: any = nil
        local compileError: any = nil
        local compiledOk: boolean, compileResult: any, returnedError: any = pcall(
            compiler,
            sourceOrError,
            "@src/Wurst.lua"
        )
        if compiledOk then
            compiled = compileResult
            compileError = returnedError
        else
            compileError = compileResult
        end
        if type(compiled) ~= "function" then
            reinjectBusy = false
            setReinjectLabel("Reinject latest")
            notify("compile failed")
            warn("[Wurst] Reinject compile: " .. tostring(compileError))
            return
        end
        setReinjectLabel("Restarting…")
        task.defer(function(): ()
            local executed: boolean, executionError: any = xpcall(
                compiled,
                debug.traceback
            )
            if not executed then
                reinjectBusy = false
                setReinjectLabel("Reinject latest")
                warn("[Wurst] Reinject runtime: " .. tostring(executionError))
            end
        end)
    end

    local destructHandle: any = nil
    local function runDestruct(): ()
        if destructHandle then
            destructHandle.SetLabel("Removing…")
        end
        task.defer(function(): ()
            local destruct: any = state.destruct
            if type(destruct) ~= "function" then

                pcall(function(): ()
                    ScreenGui:Destroy()
                end)
                return
            end
            local ok: boolean, failure: any = pcall(destruct)
            if not ok then
                warn("[Wurst] Destruct: " .. tostring(failure))
                pcall(function(): ()
                    ScreenGui:Destroy()
                end)
            end
        end)
    end

    local function runResetLayout(): ()

        configData.ui.windows = {}
        queueConfigSave()
        for _, window: any in ipairs(windows.list or {}) do
            window.userMoved = nil
        end
        if state.clickGui and state.clickGui.Retile then
            state.clickGui.Retile()
        end
        for _, window: any in ipairs(windows.list or {}) do
            if window.featureSettings or window.managementWindow then
                if window.root.Visible and windows.PlaceWindow then
                    windows.PlaceWindow(window, {force = true})
                else
                    window.stalePlacement = true
                end
            end
        end
        for _, record: any in ipairs(Settings.records) do
            if not record.window.root.Visible then
                record.placed = false
            end
        end
        notify("layout reset")
    end

    local recognised: string = GAME_CHECK.MM2Active and "MM2"
        or GAME_CHECK.TRSActive and "TRS"
        or GAME_CHECK.VDActive and "VD"
        or GAME_CHECK.BedFightActive and "BedFight"
        or "Universal only"

    reinjectHandle = state.wurstOptions.RegisterAction(
        "Reinject", "Reinject latest", runReinject
    )
    destructHandle = state.wurstOptions.RegisterAction(
        "Destruct", "Destruct", runDestruct
    )
    state.wurstOptions.RegisterAction("ResetLayout", "Reset layout", runResetLayout)

    local _ = recognised

    local function openWurstOptions(): ()
        if optionsRecord then
            raiseOpen(optionsRecord)
            return
        end
        local screen: Vector2 = viewport()
        optionsRecord = configWindow("WurstOptions", "Wurst Options", {
            position = Vector2.new(screen.X - WINDOW_WIDTH * 2 - 32, 220),
            anchor = uiRecord and uiRecord.window,
            settingsSized = true,
            packRows = {{"Reinject latest", nil, 26}},
        })
        refreshActionRows()
        raiseOpen(optionsRecord)
    end

    local screen: Vector2 = viewport()
    -- Small build watermark on the window the user opens first: the build
    -- number alone so a screenshot says which build it is.
    -- v192: the player asked for "0.191", not "Version 0.191", and for that
    -- number to be the SAME one the product line prints as "0.191 Release".
    -- Both now come from host.BUILD_LABEL, i.e. from SOURCE_STAMP: one stamp,
    -- every label.
    local versionTag: string = tostring(host.BUILD_LABEL or (function(): string
        local buildNumber: string? = string.match(
            tostring(SOURCE_STAMP or ""),
            "^v(%d+)"
        )
        return if buildNumber then "0." .. buildNumber else "0.0"
    end)())
    uiRecord = configWindow("UISettings", "UI Settings · " .. versionTag, {
        position = Vector2.new(screen.X - WINDOW_WIDTH - 12, 64),
        closable = false,
        settingsSized = false,
        packRows = {
            {"HackList", nil, 26},
            {state.isMobile and "Buttons" or "Keybinds", nil, 26},
            {"Windows", nil, 26},
            {"WurstOptions", nil, 26},
            {"Background", "#404040"},
            {"Accent", "#101010"},
            {"Text", "#F0F0F0"},
            {"Opacity", "0.85"},
        },
    })
    uiRecord.open = true

    actionRow(uiRecord, 1, "HackList", openHackList)
    -- A phone has no keyboard: the row that would open keybinds opens the
    -- touch-button page instead. Desktop keeps the keybinds window untouched.
    if state.isMobile then
        actionRow(uiRecord, 2, "Buttons", openButtons)
    else
        actionRow(uiRecord, 2, "Keybinds", openKeybinds)
    end
    actionRow(uiRecord, 3, "Windows", openWindows)
    actionRow(uiRecord, 4, "WurstOptions", openWurstOptions)

    local uiPanel: any = panelFor(uiRecord, "ClickGUI")

    uiPanel.optionCount = 4

    local uiSettingsValues: {[string]: any} = {}
    state.uiSettings = uiSettingsValues
    -- Palette defaults come from the LIVE theme so the swatches always show
    -- what the UI is actually drawing (the old constants drifted from the
    -- real preset and misled the colour pickers).
    local CLICKGUI_ENABLED: Color3 = Theme.enabled
    -- Theme.arrows is a v26 token: older saved themes (and the default
    -- "Wurst" preset before v26.1) never wrote it. Missing token must
    -- never abort this page — fall back to the enabled colour.
    local CLICKGUI_ARROWS: Color3 = Theme.arrows or Theme.enabled
        or Color3.fromRGB(0, 204, 0)
    local uiPaletteRows: {[string]: Frame} = {}
    local function paintPalette(labelText: string, color: Color3): ()
        local option: Frame? = uiPaletteRows[labelText]
        if not option then
            return
        end
        local bar: Frame? = option:FindFirstChild("ColorBar") :: Frame?
        local hex: TextLabel? = option:FindFirstChild("HexValue") :: TextLabel?
        if bar then
            bar.BackgroundColor3 = color
        end
        if hex then
            hex.Text = string.format(
                "#%02X%02X%02X",
                math.floor(color.R * 255 + 0.5),
                math.floor(color.G * 255 + 0.5),
                math.floor(color.B * 255 + 0.5)
            )
        end
    end
    -- A single bad palette row (missing theme token, malformed saved
    -- colour) must never kill the whole Settings window — v26 regressed
    -- exactly here when Theme.arrows was nil and hexOf(nil) aborted
    -- SettingsPage.init before the "Show dock" toggle could restore the
    -- saved dock state. Isolate the rows and keep building the page.
    local PALETTE_ROWS: {any} = {
        {label = "Background", default = Theme.background, tokens = {"surface", "background"}},
        {label = "Accent", default = Theme.accent, tokens = {"accent", "outline"}},
        {label = "Text", default = Theme.text, tokens = {"text", "accentText"}},
        {label = "Enabled", default = Theme.enabled, tokens = {"enabled"}},
        -- v191: this row is the "Arrows" token, but it now also paints the
        -- window buttons (the Pin chip uses it too), so it is labelled
        -- "Buttons". `key` keeps the v26 config key so a saved colour
        -- is not lost by the rename.
        {label = "Buttons", key = "Arrows", default = Theme.arrows, tokens = {"arrows"}},
    }
    -- Re-apply the saved palette when the page builds: the theme starts
    -- from the preset on every execution, so without this the chosen
    -- colours were forgotten on rejoin and the UI came back with preset
    -- colours while the swatches still showed the saved ones.
    if ThemeEngine and type(ThemeEngine.SetMany) == "function" then
        for _, row: any in ipairs(PALETTE_ROWS) do
            local saved: Color3? = colorFromConfig(
                configData.values[
                    uiPanel.configKey .. "."
                        .. (row.key or row.label):gsub("%W", "")
                ],
                nil
            )
            if saved then
                local updates: {[string]: Color3} = {}
                for _, token: string in ipairs(row.tokens) do
                    updates[token] = saved
                end
                ThemeEngine.SetMany(updates)
            end
        end
    end
    local paletteOk: boolean, paletteError: string? = pcall(function(): ()
        for _, row: any in ipairs(PALETTE_ROWS) do
            local tokens: {string} = row.tokens
            local labelText: string = row.label
            local swatch: TextButton? = addColorOption(
                uiPanel,
                labelText,
                row.default or Theme.text,
                function(color: Color3): ()
                    uiSettingsValues[labelText] = color

                    if ThemeEngine and type(ThemeEngine.SetMany) == "function" then
                        local updates: {[string]: Color3} = {}
                        for _, token: string in ipairs(tokens) do
                            updates[token] = color
                        end
                        ThemeEngine.SetMany(updates)
                    elseif ThemeEngine and type(ThemeEngine.Set) == "function" then
                        for _, token: string in ipairs(tokens) do
                            ThemeEngine.Set(token, color)
                        end
                    end
                end,
                row.key
            )
            if swatch and swatch.Parent and swatch.Parent:IsA("Frame") then
                uiPaletteRows[labelText] = swatch.Parent :: Frame
            end
        end
    end)
    if not paletteOk then
        warn("[Wurst:Settings] palette rows skipped: " .. tostring(paletteError))
    end

    addNumberOption(uiPanel, "Opacity", OPACITY, 0.15, 1, function(value: number): ()
        local wanted: number = math.clamp(tonumber(value) or OPACITY, 0.15, 1)
        uiSettingsValues["Opacity"] = wanted
        -- v191: background only. windows.SetOpacity fades the panel surface;
        -- the title, labels, icons and controls stay at full opacity.
        windows.SetOpacity(wanted)
    end, "How see-through the menu BACKGROUND is. Text and controls stay fully readable at any value.", 0.01, true)

    uiSettingsValues["Tooltip opacity"] = TOOLTIP_OPACITY
    if state.featureTooltip then
        state.featureTooltip.BackgroundTransparency = 1 - TOOLTIP_OPACITY
    end

    uiSettingsValues["Max height"] = MAX_HEIGHT
    state.uiMaxHeight = MAX_HEIGHT
    uiSettingsValues["Max settings height"] = MAX_SETTINGS_HEIGHT
    maxSettingsHeight = MAX_SETTINGS_HEIGHT
    state.uiMaxSettingsHeight = MAX_SETTINGS_HEIGHT

    uiSettingsValues["Menu style"] = "Wurst"

    -- v162: the menu's own scale, so "make it like the older build" is a
    -- number and not a code change. It only touches the ClickGui UIScale; the
    -- HUD, the dock and the tooltips keep their own sizes.
    local themeShape: any = ThemeEngine and ThemeEngine.shape
    local initialScale: number = themeShape and tonumber(themeShape.scale) or 1
    uiSettingsValues["Menu scale"] = initialScale
    -- Dragging this used to run a full relayout on every single step, so the
    -- menu flickered and the windows piled on top of each other mid-drag.
    -- v181: the UIScale follows the slider immediately, and the reflow (reclamp
    -- every window into the new logical viewport) runs once the drag settles.
    local scaleReflowToken: number = 0
    addNumberOption(uiPanel, "Menu scale", initialScale, 0.7, 1.3,
        function(value: number): ()
            local shape: any = ThemeEngine and ThemeEngine.shape
            if not shape then
                return
            end
            local wanted: number = math.clamp(tonumber(value) or 1, 0.7, 1.3)
            shape.scale = wanted
            uiSettingsValues["Menu scale"] = wanted
            configData.values["UI.MenuScale"] = wanted
            queueConfigSave()
            if state.clickGui and type(state.clickGui.ApplyScale) == "function" then
                state.clickGui.ApplyScale()
            end
            scaleReflowToken += 1
            local token: number = scaleReflowToken
            task.delay(0.15, function(): ()
                if token ~= scaleReflowToken then
                    return
                end
                if type(state.refreshLayout) == "function" then
                    state.refreshLayout(true)
                end
            end)
        end, "Adjust the size of menu windows on this device.", 0.01, true)

    addToggleOption(uiPanel, "Keep after teleport", true, function(value: boolean): ()
        uiSettingsValues["Keep after teleport"] = value
        local persist: any = state.teleportPersist
        if persist and type(persist.SetEnabled) == "function" then
            persist.SetEnabled(value)
```

```
            if value and persist.supported ~= true then
                notify("queue on teleport unavailable — "
                    .. "the menu cannot follow teleports here")
            end
        end
    end)

    local touchPrimary: boolean = state.isMobile == true
    if not touchPrimary then
        addToggleOption(uiPanel, "Show dock", false, function(value: boolean): ()
            uiSettingsValues["Show dock"] = value
            state.uiShowDock = value
            if state.dock and type(state.dock.SetVisible) == "function" then
                state.dock.SetVisible(value)
            end
        end)
    end

    -- Dock placement is a touch affordance; desktop does not need this action.
    if touchPrimary then
        host.addActionOption(uiPanel, "Move dock", function(): ()
            if state.dock and state.dock.BeginMove then state.dock.BeginMove() end
        end, "Drag and release the dock to save its position.")
    end

    uiRecord.placed = true
    if windows.ReflowWindow then
        windows.ReflowWindow(uiRecord.window)
    end

    if state.bitmapText and type(state.bitmapText.onReady) == "table" then
        table.insert(state.bitmapText.onReady, function(): ()
            for _, record: any in ipairs(Settings.records) do
                if record.repack then
                    record.repack()
                end
            end
        end)
    end

    state.addMenuVisibilityListener(function(visible: boolean): ()
        for _, record: any in ipairs(Settings.records) do
            if not visible and record.window.pinned then
                continue
            end
            if visible then
                record.window:SetVisible(record.open)
            else
                record.open = record.window.root.Visible
                record.window:SetVisible(false)
            end
        end
    end)
    for _, record: any in ipairs(Settings.records) do
        record.window:SetVisible(state.visible == true and record.open)
    end

    Settings.ui = uiRecord
    Settings.openKeybinds = openKeybinds
    Settings.openProfiles = openProfiles
    Settings.openHackList = openHackList
    Settings.openWurstOptions = openWurstOptions

    activeCleanup = function(): ()
        for _, record: any in ipairs(Settings.records) do
            pcall(function(): ()
                record.window:Destroy()
            end)
        end
        Settings.records = {}
        for _, connection: RBXScriptConnection in ipairs(Settings.connections) do
            pcall(function(): ()
                connection:Disconnect()
            end)
        end
        Settings.connections = {}
        state.uiSettings = nil
        state.wurstOptions = nil
    end
    Module.Initialized = true
    return Settings
end

function Module.destroy(): ()
    if activeCleanup then
        pcall(activeCleanup)
    end
    activeCleanup = nil
    Module.Initialized = false
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/guis/Wurst/Code/SettingsPage.lua
--------------------------------------------------------------------------------


--------------------------------------------------------------------------------
ARCHIVO: src/guis/Wurst/Code/Widgets.lua
BYTES: 113668
SHA-256: b0448dd8bebcd89e4f66945b25a0a8fa1cbf346e0487918b1eea6313a3221fd0
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
export type OptionDefinition = {
    kind: string,
    label: string,
    default: any?,
    minimum: number?,
    maximum: number?,
    values: {string}?,
    callback: any,
    persist: boolean?,
}

export type FeatureDefinition = {
    name: string,
    description: string,
    order: number,
    onToggle: any,
    configuration: any?,
    options: {OptionDefinition}?,
}

export type WidgetLibrary = {
    addToggleOption: any,
    addActionOption: any,
    addNumberOption: any,
    addRangeOption: any,
    addCycleOption: any,
    addKeyOption: any,
    addTextOption: any,
    addColorOption: any,
    addListOption: any,
    addSectionOption: any,
    addInformationOption: any,
    setOptionVisible: any,
}

local Module = {
    Name = "Widgets",
    PlaceId = 0,
    Events = {} :: {[string]: any},
    Initialized = false,
}

local boundState: any = nil
local toggleThemeRefreshers: {() -> ()} = {}

local function runToggleThemeRefreshers(changed: any): ()
    for _, refresh: any in ipairs(toggleThemeRefreshers) do
        pcall(refresh, changed)
    end
end

local LABEL_FONT_SIZE: number = 16
local SLIDER_BAR_HEIGHT: number = 6
local SLIDER_KNOB_WIDTH: number = 16
local SLIDER_KNOB_HEIGHT: number = 16

local CHILD_INSET: number = 4
local CHILD_GAP: number = 4

function Module.init(context: any): WidgetLibrary

    local WURST_GREEN: Color3 = Color3.fromRGB(0, 204, 0)
    local host: any = context
    local UserInputService: any = host.UserInputService
    local state: any = host.state
    boundState = state
    table.clear(toggleThemeRefreshers)
    if state then
        state.optionThemeRefresh = runToggleThemeRefreshers
        -- Other UI modules (card bind squares, window chrome) register their
        -- repainters here so palette changes reach everything on screen.
        state.registerThemeRefresh = function(refresh: any, tokens: {string}?): ()
            table.insert(toggleThemeRefreshers, function(changed)
                if tokens and changed then
                    local relevant = false
                    for _, token in ipairs(tokens) do if changed[token] then relevant = true; break end end
                    if not relevant then return end
                end
                refresh()
            end)
        end
        -- Live-apply registry for option VALUES (sliders, colours, cycles,
        -- text). Profiles and the panic reset drive the whole UI through it
        -- instead of only rewriting the saved config.
        state.valueControls = {}
    end
    local configData: any = host.configData
    local queueConfigSave: any = host.queueConfigSave
    local create: any = host.create
    local makeButton: any = host.makeButton
    local makeTextBox: any = host.makeTextBox

    local function flatTextBox(parent: Instance, initial: any): TextBox
        local box: TextBox = (makeTextBox(parent, initial) :: any) :: TextBox
        local corner: UICorner? = box:FindFirstChild("StyleCorner") :: UICorner?
        if corner then
            corner:Destroy()
        end
        return box
    end
    local makeTextLabel: any = host.makeTextLabel
    local notify: any = host.notify
    local Theme: any = host.Theme
    local ThemeEngine: any = host.ThemeEngine
    local CONTROL_FONT: any = host.CONTROL_FONT
    local ScreenGui: any = host.ScreenGui

    local PopupLayer: any = host.PopupLayer
    local trackUiConnection: any = host.trackUiConnection
    local applyKeySlotStyle: any = host.applyKeySlotStyle
    local beginKeyCapture: any = host.beginKeyCapture
    local bindActionShortcut: any = host.bindActionShortcut
    local runActionButton: any = host.runActionButton
    local colorFromConfig: any = host.colorFromConfig
    local colorToConfig: any = host.colorToConfig

    local function drawPixelIcon(
        parent: Instance,
        kind: string,
        color: Color3?,
        token: string?
    ): Frame
        local themed: boolean = token ~= nil
            and ThemeEngine ~= nil
            and Theme[token or ""] ~= nil
        local paint: Color3 = (themed and Theme[token :: string])
            or color
            or WURST_GREEN
        local existing: Instance? = (parent :: any):FindFirstChild("PixelIcon")
        if existing and existing:GetAttribute("PixelKind") == kind
            and existing:GetAttribute("PixelToken") == (token or "") then
            if not themed then
                for _, pixel in ipairs(existing:GetChildren()) do
                    if pixel:IsA("Frame") then pixel.BackgroundColor3 = paint end
                end
            end
            return existing :: Frame
        end
        if existing then existing:Destroy() end
        local sizes: {[string]: {number}} = {
            down = {14, 8},
            up = {14, 8},
            right = {8, 14},
            cross = {10, 10},
            pin = {10, 10},
            check = {12, 10},
        }
        local size: {number} = sizes[kind] or {10, 10}
        local container: Frame = (create("Frame", {
            Parent = parent,
            Name = "PixelIcon",
            AnchorPoint = Vector2.new(0.5, 0.5),
            BackgroundTransparency = 1,
            BorderSizePixel = 0,
            Position = UDim2.fromScale(0.5, 0.5),
            Size = UDim2.fromOffset(size[1], size[2]),
            ZIndex = (tonumber((parent :: any).ZIndex) or 1) + 1,
        }) :: any) :: Frame
        local function px(x: number, y: number, w: number, h: number): ()
            local pixel: any = create("Frame", {
                Parent = container,
                BackgroundColor3 = paint,
                BorderSizePixel = 0,
                Position = UDim2.fromOffset(x * 2, y * 2),
                Size = UDim2.fromOffset(w * 2, h * 2),
                ZIndex = container.ZIndex + 1,
            })
            if themed then
                -- Token binding; Destroying removes obsolete pixels from the registry.
                ThemeEngine.Bind(pixel, "BackgroundColor3", token :: string)
            end
        end
        container:SetAttribute("PixelKind", kind)
        container:SetAttribute("PixelToken", token or "")
        if kind == "down" then
            px(0, 0, 7, 1); px(1, 1, 5, 1); px(2, 2, 3, 1); px(3, 3, 1, 1)
        elseif kind == "up" then
            px(3, 0, 1, 1); px(2, 1, 3, 1); px(1, 2, 5, 1); px(0, 3, 7, 1)
        elseif kind == "right" then
            px(0, 0, 1, 7); px(1, 1, 1, 5); px(2, 2, 1, 3); px(3, 3, 1, 1)
        elseif kind == "cross" then
            px(0, 0, 1, 1); px(4, 0, 1, 1)
            px(1, 1, 1, 1); px(3, 1, 1, 1)
            px(2, 2, 1, 1)
            px(1, 3, 1, 1); px(3, 3, 1, 1)
            px(0, 4, 1, 1); px(4, 4, 1, 1)
        elseif kind == "pin" then
            px(1, 0, 3, 3); px(2, 3, 1, 2)
        elseif kind == "check" then
            px(0, 2, 1, 1); px(1, 3, 1, 1); px(2, 4, 1, 1)
            px(3, 3, 1, 1); px(4, 2, 1, 1); px(5, 1, 1, 1)
        end
        return container
    end

    state.layoutOptionControl = function(
        option: Frame,
        control: GuiObject,
        widthScale: number?
    ): ()
        local top: number = tonumber(option:GetAttribute("ControlTop")) or 0
        local height: number = tonumber(option:GetAttribute("ControlHeight"))
            or option.Size.Y.Offset

        local scale: number = widthScale or 0.5
        control.AnchorPoint = Vector2.new(1, 0)
        control.Position = UDim2.new(1, -10, 0, top)
        control.Size = UDim2.new(scale, -10, 0, height)
        control.ZIndex = math.max(control.ZIndex, option.ZIndex + 2)
    end

    local function createOptionRow(
        feature: any,
        labelText: string,
        _hintText: string?
    ): Frame
        feature.optionCount += 1
        feature.searchText ..= " " .. string.lower(labelText)
        local rowHeight: number = state.layout.optionRowHeight

        local option: Frame = (create("Frame", {
            Parent = feature.options,
            Name = "Option_" .. labelText:gsub("%W", ""),
            BackgroundTransparency = 1,
            BorderSizePixel = 0,
            LayoutOrder = feature.optionCount,
            Size = UDim2.new(1, 0, 0, rowHeight),
            ZIndex = feature.options.ZIndex + 1,
        }) :: any) :: Frame
        table.insert(feature.optionRows, option)

        local label: TextLabel = (create("TextLabel", {
            TextTruncate = Enum.TextTruncate.AtEnd,
            Parent = option,
            Name = "OptionLabel",
            BackgroundTransparency = 1,
            FontFace = CONTROL_FONT,
            Position = UDim2.fromOffset(CHILD_INSET, 0),
            Size = UDim2.new(0.55, -CHILD_INSET, 0, rowHeight),
            Text = labelText,
            TextColor3 = Theme.text,
            TextSize = LABEL_FONT_SIZE,
            TextXAlignment = Enum.TextXAlignment.Left,
            ZIndex = option.ZIndex + 1,
        }) :: any) :: TextLabel
        label.Visible = true

        option:SetAttribute("ControlTop", 3)
        option:SetAttribute("ControlHeight", rowHeight - 6)

        feature.updateExpandedSize()
        return option
    end

    -- A real heading, not an invisible spacer: a capitalised caption with a
    -- hairline that runs to the right edge. Every card can group its rows
    -- without inventing its own label and drifting from the rest.
    state.addSectionOption = function(feature: any, titleText: string): Frame
        feature.optionCount += 1
        local section: Frame = (create("Frame", {
            Parent = feature.options,
            Name = "Section_" .. titleText:gsub("%W", ""),
            BackgroundTransparency = 1,
            BorderSizePixel = 0,
            LayoutOrder = feature.optionCount,
            Size = UDim2.new(1, 0, 0, state.layout.sectionRowHeight or 22),
            Visible = true,
            ZIndex = feature.options.ZIndex + 1,
        }) :: any) :: Frame
        local caption: TextLabel = (create("TextLabel", {
            Parent = section,
            Name = "SectionLabel",
            BackgroundTransparency = 1,
            FontFace = CONTROL_FONT,
            Position = UDim2.fromOffset(CHILD_INSET, 0),
            Size = UDim2.new(1, -CHILD_INSET * 2, 0, section.Size.Y.Offset),
            Text = string.upper(titleText),
            TextColor3 = Theme.textMuted,
            TextSize = 10,
            TextXAlignment = Enum.TextXAlignment.Left,
            ZIndex = section.ZIndex + 1,
        }) :: any) :: TextLabel
        caption:SetAttribute("SectionTitle", titleText)
        -- The divider sits after the caption, so long titles never overlap it.
        create("Frame", {
            Parent = section,
            Name = "SectionLine",
            AnchorPoint = Vector2.new(0, 0.5),
            BackgroundColor3 = Theme.outline,
            BackgroundTransparency = 0.55,
            BorderSizePixel = 0,
            Position = UDim2.new(0, 4 + math.clamp(#string.upper(titleText) * 7 + 8, 24, 220), 0.5, 0),
            Size = UDim2.new(1, -(4 + math.clamp(#string.upper(titleText) * 7 + 8, 24, 220)) - 4, 0, 1),
            ZIndex = section.ZIndex,
        })
        section:SetAttribute("SectionTitle", titleText)
        table.insert(feature.optionRows, section)
        feature.updateExpandedSize()
        return section
    end

    state.addStatOption = function(
        feature: any,
        labelText: string,
        initialValue: string?
    ): any
        local option: Frame = createOptionRow(feature, labelText, nil)
        local optionLabel: TextLabel? =
            option:FindFirstChild("OptionLabel") :: TextLabel?
        if optionLabel then
            (optionLabel :: TextLabel).Size =
                UDim2.new(0.62, -CHILD_INSET, 1, 0);
            (optionLabel :: TextLabel).TextColor3 = Theme.textMuted
        end
        local valueLabel: TextLabel = (create("TextLabel", {
            TextTruncate = Enum.TextTruncate.AtEnd,
            Parent = option,
            Name = "StatValue",
            AnchorPoint = Vector2.new(1, 0),
            BackgroundTransparency = 1,
            FontFace = CONTROL_FONT,
            Position = UDim2.new(1, -CHILD_INSET, 0, 0),
            Size = UDim2.new(0.38, -CHILD_INSET, 1, 0),
            Text = tostring(initialValue or "0"),
            TextColor3 = Theme.text,
            TextSize = LABEL_FONT_SIZE,
            TextXAlignment = Enum.TextXAlignment.Right,
            ZIndex = option.ZIndex + 2,
        }) :: any) :: TextLabel
        local lastText: string = tostring(initialValue or "0")
        return {
            row = option,
            label = valueLabel,
            -- v160: modules refresh their stat rows from a per-frame task. A
            -- Text assignment re-runs the label layout, so writing the same
            -- string 60 times a second was pure waste. Only a change is
            -- written now; the first call always goes through.
            set = function(newText: string): ()
                local nextText: string = tostring(newText)
                if nextText == lastText then
                    return
                end
                lastText = nextText
                if valueLabel and valueLabel.Parent then
                    valueLabel.Text = nextText
                end
            end,
        }
    end

    local function setOptionVisible(
        feature: any,
        option: GuiObject,
        visible: boolean
    ): ()
        -- Hiding the options container itself empties the whole card: every row
        -- disappears and nothing can bring them back except a rebuild. Callers
        -- that pass a control instead of its row hit exactly that, so the
        -- container is never a valid target.
        if option == nil or option == feature.options then
            warn("[Random Testing Menu] setOptionVisible refused the options container")
            return
        end
        -- addToggleOption/addCycleOption return the control inside the row while
        -- addNumberOption/addRangeOption return the row itself. Both are valid
        -- arguments, so the row is resolved here and only ever that row is
        -- hidden.
        local row: GuiObject = option
        local parent: any = option.Parent
        local container: any = feature.options
        if container ~= nil
            and parent ~= nil
            and parent ~= container
            and parent.Parent ~= nil
            and parent.Parent == container then
            row = parent
        end
        row.Visible = visible
        feature.updateExpandedSize()
    end

    local function addToggleOption(
        feature: any,
        labelText: string,
        defaultValue: boolean,
        callback: (boolean) -> (),
        hintText: string?,
        bindable: boolean?
    ): TextButton
        local optionKey: string =
            feature.configKey .. "." .. labelText:gsub("%W", "")
        local enabled: boolean? = configData.states[optionKey]
        if enabled == nil then
            enabled = defaultValue == true
        end

        local option: Frame = createOptionRow(feature, labelText, hintText)
        local button: TextButton = create("TextButton", {
            TextTruncate = Enum.TextTruncate.AtEnd,
            ClipsDescendants = true,
            Parent = option,
            Name = "OptionClickTarget",
            Active = true,
            AutoButtonColor = false,
            BackgroundTransparency = 1,
            BorderSizePixel = 0,
            Size = UDim2.fromScale(1, 1),
            Text = "",
            ZIndex = option.ZIndex + 2,
        }) :: TextButton

        local box: Frame = (create("Frame", {
            Parent = option,
            Name = "OptionCheckbox",
            AnchorPoint = Vector2.new(0, 0.5),
            BackgroundColor3 = Color3.fromRGB(0, 0, 0),
            BackgroundTransparency = 0.5,
            BorderSizePixel = 0,
            Position = UDim2.new(0, CHILD_INSET, 0.5, 0),
            Size = UDim2.fromOffset(22, 22),
            ZIndex = option.ZIndex + 3,
        }) :: any) :: Frame

        local boxStroke: UIStroke = (create("UIStroke", {
            Parent = box,
            ApplyStrokeMode = Enum.ApplyStrokeMode.Border,
            Color = Theme.outline,
            Thickness = 1,
            Transparency = 0.35,
        }) :: any) :: UIStroke

        -- The checkmark contrasts with the Enabled colour (which fills the
        -- box), so it stays visible whatever palette the user picks.
        local function markContrast(color: Color3): Color3
            local luminance: number =
                0.299 * color.R + 0.587 * color.G + 0.114 * color.B
            if luminance > 0.45 then
                return Color3.fromRGB(12, 12, 12)
            end
            return Color3.fromRGB(245, 245, 247)
        end
        local mark: Frame = drawPixelIcon(
            box,
            "check",
            markContrast(Theme.enabled or WURST_GREEN)
        )
        mark.Visible = false

        local function paintMark(): ()
            local paint: Color3 =
                markContrast(Theme.enabled or WURST_GREEN)
            for _, pixel: any in ipairs(mark:GetChildren()) do
                pcall(function(): ()
                    pixel.BackgroundColor3 = paint
                end)
            end
        end
        local optionLabel: TextLabel? =
            option:FindFirstChild("OptionLabel") :: TextLabel?
        if optionLabel then
            (optionLabel :: TextLabel).Position =
                UDim2.fromOffset(CHILD_INSET + 22 + CHILD_GAP, 0);
            (optionLabel :: TextLabel).Size =
                UDim2.new(1, -(CHILD_INSET + 22 + CHILD_GAP), 1, 0)
        end

        local function refreshOptionState(): ()
            local on: boolean = enabled :: boolean
            box.BackgroundColor3 = on and Theme.enabled or Theme.surface
            box.BackgroundTransparency = on and 0 or 0.5
            boxStroke.Color = on and Theme.enabled or Theme.outline
            paintMark()
            mark.Visible = on
        end

        -- Palette changes must repaint existing checkboxes, not only the
        -- ones toggled afterwards.
        table.insert(toggleThemeRefreshers, refreshOptionState)

        local function toggleOption(): ()
            if not option.Visible then return end
            enabled = not enabled
            refreshOptionState()
            configData.states[optionKey] = enabled
            callback(enabled :: boolean)
            queueConfigSave()
            notify(labelText .. ((enabled :: boolean) and " enabled" or " disabled"))
        end
        if bindable then
            -- Use the same persisted/capture-aware shortcut path as actions.
            -- Keyboard and touch toggle the checkbox, callback and saved value
            -- together; a key slot must not be swallowed by the row hit target.
            button.Size = UDim2.new(0.72, 0, 1, 0)
            if optionLabel then
                optionLabel.Size = UDim2.new(0.72, -(CHILD_INSET + 22 + CHILD_GAP), 1, 0)
            end
            local keyButton: TextButton = makeButton(option, "KEY", 9)
            keyButton.Name = "OptionKeybind"
            state.layoutOptionControl(option, keyButton, 0.24)
            keyButton.Position = UDim2.new(0.74, -16, keyButton.Position.Y.Scale, keyButton.Position.Y.Offset)
            keyButton.ZIndex = option.ZIndex + 5
            applyKeySlotStyle(keyButton)
            keyButton.TextScaled = true
            create("UITextSizeConstraint", {Parent = keyButton, MaxTextSize = 14, MinTextSize = 8})
            bindActionShortcut(keyButton, feature.configKey .. ".labelText, labelText, toggleOption)
        end
        trackUiConnection(button.MouseButton1Click:Connect(toggleOption))

        -- Programmatic state sync (e.g. a card master switch turning its
        -- sub-options off). Mirrors the listControls pattern: the visual, the
        -- persisted state and the module callback all move together.
        state.toggleControls = state.toggleControls or {}
        state.toggleControls[optionKey] = {
            -- The live checkbox, so callers (and the boot tests) can read the
            -- option without assuming what its default was.
            getChecked = function(): boolean
                return enabled :: boolean
            end,
            setChecked = function(checked: boolean): ()
                if not button.Parent then
                    return
                end
                local wanted: boolean = checked == true
                if wanted == (enabled :: boolean) then
                    return
                end
                enabled = wanted
                refreshOptionState()
                configData.states[optionKey] = enabled
                callback(enabled :: boolean)
                queueConfigSave()
            end,
        }

        refreshOptionState()
        callback(enabled :: boolean)
        return button
    end
    local function addActionOption(
        feature: any,
        labelText: string,
        callback: () -> (),
        hintText: string?,
        interaction: any?
    ): TextButton
        local option = createOptionRow(feature, labelText, hintText)
        local button = makeButton(option, interaction and interaction.hold and "HOLD" or "RUN", 11)
        button.BackgroundColor3 = state.guiStyle == "Default"
            and Theme.buttonHover
            or Theme.accentSoft

        state.layoutOptionControl(option, button, 0.26)
        local function executeAction(): ()
            if not option.Visible then return end
            runActionButton(
                button,
                "RUN",
                callback,
                labelText .. " completado",
                "[Random Testing Menu] " .. labelText
```

```
        )
    end
    local hold: boolean = interaction ~= nil and interaction.hold == true
    local held = {button = false, shortcut = false}
    local active: boolean = false
    local buttonInput: InputObject? = nil
    local function setHeld(source: string, value: boolean): ()
        if value and not option.Visible then return end
        held[source] = value
        local wanted: boolean = held.button or held.shortcut
        if active == wanted then return end
        active = wanted
        button.Text = wanted and "HELD" or "HOLD"
        if wanted then callback()
        elseif interaction and interaction.onRelease then interaction.onRelease() end
    end
    if hold then
        trackUiConnection(button.InputBegan:Connect(function(input: InputObject): ()
            if input.UserInputType == Enum.UserInputType.MouseButton1 or input.UserInputType == Enum.UserInputType.Touch then
                if buttonInput == nil then buttonInput = input; setHeld("button", true) end
            end
        end))
        trackUiConnection(UserInputService.InputEnded:Connect(function(input: InputObject): ()
            if input == buttonInput or (buttonInput and buttonInput.UserInputType == Enum.UserInputType.MouseButton1
                and input.UserInputType == Enum.UserInputType.MouseButton1) then
                buttonInput = nil
                setHeld("button", false)
            end
        end))
        trackUiConnection(UserInputService.WindowFocusReleased:Connect(function(): ()
            held.button = false
            buttonInput = nil
            setHeld("shortcut", false)
        end))
    else
        trackUiConnection(button.MouseButton1Click:Connect(executeAction))
    end
    local shortcutButton: TextButton =
        (makeButton(option, "KEY", 9) :: any) :: TextButton
    shortcutButton.BackgroundColor3 = state.guiStyle == "Default"
        and Theme.button
        or Theme.surfaceRaised

    state.layoutOptionControl(option, shortcutButton, 0.24)
    shortcutButton.Position = UDim2.new(
        0.74,
        -16,
        shortcutButton.Position.Y.Scale,
        shortcutButton.Position.Y.Offset
    )
    applyKeySlotStyle(shortcutButton)
    shortcutButton.TextScaled = true
    create("UITextSizeConstraint", {
        Parent = shortcutButton,
        MaxTextSize = 14,
        MinTextSize = 8,
    })
    bindActionShortcut(
        shortcutButton,
        feature.configKey .. labelText,
        labelText,
        hold and function(): () setHeld("shortcut", true) end or executeAction,
        hold and {hold = true, onRelease = function(): () setHeld("shortcut", false) end} or nil
    )
    return button :: TextButton
end

type ActiveSlider = {
    update: (number) -> (),
    finish: () -> (),
}

local activeSlider: ActiveSlider? = nil

trackUiConnection(UserInputService.InputChanged:Connect(function(
    input: InputObject
): ()
    if activeSlider
        and (input.UserInputType == Enum.UserInputType.MouseMovement
            or input.UserInputType == Enum.UserInputType.Touch) then
        activeSlider.update(input.Position.X)
    end
end))

trackUiConnection(UserInputService.InputEnded:Connect(function(
    input: InputObject
): ()
    if activeSlider
        and (input.UserInputType == Enum.UserInputType.MouseButton1
            or input.UserInputType == Enum.UserInputType.Touch) then
        local slider: ActiveSlider = activeSlider
        activeSlider = nil
        slider.finish()
    end
end))

-- Accepts all real numbers including +/- math.huge so sliders can
-- express "infinite" (e.g. Fullbright Brightness = inf → white screen).
local function isNumber(value: any): boolean
    return type(value) == "number" and value == value
end

local function formatCompactNumber(value: number): string
    if value == math.huge then return "inf" end
    if value == -math.huge then return "-inf" end
    local magnitude: number = math.abs(value)
    if magnitude >= 1e9 or (magnitude > 0 and magnitude < 1e-4) then
        return string.format("%.3e", value)
    end
    return string.format("%.12g", value)
end

-- When a slider's maximum is infinite, the linear position formula
-- (value-min)/(max-min) collapses to 0 for every finite value. Use a
-- practical display cap so the knob still moves, while the actual
-- value (and any typed infinity) passes through untouched.
local function effectiveMax(minimum: number, maximum: number): number
    if maximum ~= math.huge then return maximum end
    return math.max(100, math.abs(minimum) * 10)
end

-- Parse text-box input that may contain infinity keywords.
local function parseNumericText(text: string): number?
    local cleaned = string.lower(text:gsub("%s+", ""))
    if cleaned == "inf" or cleaned == "infinity" or cleaned == "+inf" then
        return math.huge
    end
    if cleaned == "-inf" or cleaned == "-infinity" then
        return -math.huge
    end
    return tonumber(text)
end

local function addNumberOption(
    feature: any,
    labelText: string,
    defaultValue: number,
    minimum: number,
    maximum: number,
    callback: (number) -> (),
    hintText: string?,
    stepValue: number?,
    boundTyped: boolean?
): Frame
    local optionKey: string =
        feature.configKey .. "." .. labelText:gsub("%W", "")
    local storedValue: number? = tonumber(configData.values[optionKey])

    -- The interval is normally just a drag guide, not a limit on typed or
    -- profile values. UI controls may explicitly opt into bounded input.
    local value: number = isNumber(storedValue)
        and (storedValue :: number)
        or defaultValue
    if boundTyped then
        if value == math.huge or value == -math.huge then value = defaultValue end
        value = math.clamp(value, minimum, maximum)
    end
    local integerOnly: boolean = defaultValue % 1 == 0
        and minimum % 1 == 0
        and maximum % 1 == 0

    local option: Frame = createOptionRow(feature, labelText, hintText)

    option.Size = UDim2.new(1, 0, 0, 44)
    feature.updateExpandedSize()
    local headerLabel: TextLabel? =
        option:FindFirstChild("OptionLabel") :: TextLabel?
    if headerLabel then
        (headerLabel :: TextLabel).Size =
            UDim2.new(0.55, -CHILD_INSET, 0, 22)
    end

    local requestedStep: number? = tonumber(stepValue)
    if requestedStep ~= nil and requestedStep <= 0 then
        requestedStep = nil
    end

    local step: number = requestedStep or (integerOnly and 1 or 0.01)
    local precisionFactor: number = 1
    for _ = 1, 6 do
        if math.abs(step * precisionFactor - math.round(step * precisionFactor)) < 1e-7 then
            break
        end
        precisionFactor *= 10
    end

    option:SetAttribute("Min", minimum)
    option:SetAttribute("Max", maximum)
    option:SetAttribute("Step", step)
    option:SetAttribute("FreeNumeric", true)

    local slider: TextButton = (create("TextButton", {
        TextTruncate = Enum.TextTruncate.AtEnd,
        Parent = option,
        Name = "Slider",
        Active = true,
        AutoButtonColor = false,
        BackgroundTransparency = 1,
        BorderSizePixel = 0,
        Position = UDim2.new(0, 0, 0, 22),
        Size = UDim2.new(1, 0, 0, 22),
        Text = "",
        ZIndex = option.ZIndex + 1,
    }) :: any) :: TextButton

    local minText: string = formatCompactNumber(minimum)
    local maxText: string = formatCompactNumber(maximum)

    local bitmapContract: any = state.bitmapText
    local function endLabelWidth(endText: string): number
        if bitmapContract and type(bitmapContract.measure) == "function" then
            return bitmapContract.measure({{text = endText}}, 14) + 4
        end
        return #endText * 11 + 4
    end
    local minWidth: number = endLabelWidth(minText)
    local maxWidth: number = endLabelWidth(maxText)
    create("TextLabel", {
        TextTruncate = Enum.TextTruncate.AtEnd,
        Parent = slider,
        Name = "MinLabel",
        BackgroundTransparency = 1,
        FontFace = CONTROL_FONT,
        Position = UDim2.new(0, CHILD_INSET, 0, 0),
        Size = UDim2.new(0, minWidth, 1, 0),
        Text = minText,
        TextColor3 = Theme.textMuted,
        TextSize = 14,
        TextXAlignment = Enum.TextXAlignment.Left,
        ZIndex = option.ZIndex + 1,
    })
    create("TextLabel", {
        TextTruncate = Enum.TextTruncate.AtEnd,
        Parent = slider,
        Name = "MaxLabel",
        AnchorPoint = Vector2.new(1, 0),
        BackgroundTransparency = 1,
        FontFace = CONTROL_FONT,
        Position = UDim2.new(1, -CHILD_INSET, 0, 0),
        Size = UDim2.new(0, maxWidth, 1, 0),
        Text = maxText,
        TextColor3 = Theme.textMuted,
        TextSize = 14,
        TextXAlignment = Enum.TextXAlignment.Right,
        ZIndex = option.ZIndex + 1,
    })

    local track: Frame = (create("Frame", {
        Parent = slider,
        Name = "Track",
        BackgroundColor3 = Color3.fromRGB(255, 255, 255),
        BackgroundTransparency = 0.9,
        BorderSizePixel = 0,
        Position = UDim2.new(
            0,
            CHILD_INSET + minWidth + CHILD_GAP,
            0,
            (22 - SLIDER_BAR_HEIGHT) // 2
        ),
        Size = UDim2.new(
            1,
            -(CHILD_INSET * 2 + minWidth + maxWidth + CHILD_GAP * 2),
            0,
            SLIDER_BAR_HEIGHT
        ),
        ZIndex = option.ZIndex + 1,
    }) :: any) :: Frame

    local fill: Frame = (create("Frame", {
        Parent = track,
        Name = "Fill",
        BackgroundColor3 = Theme.accent,
        BorderSizePixel = 0,
        Size = UDim2.fromScale(0, 1),
        ZIndex = track.ZIndex + 1,
    }) :: any) :: Frame

    local knob: Frame = (create("Frame", {
        Parent = track,
        Name = "Knob",
        AnchorPoint = Vector2.new(0.5, 0.5),
        BackgroundColor3 = Theme.text,
        BorderSizePixel = 0,
        Position = UDim2.fromScale(0, 0.5),
        Size = UDim2.fromOffset(SLIDER_KNOB_WIDTH, SLIDER_KNOB_HEIGHT),
        ZIndex = track.ZIndex + 2,
    }) :: any) :: Frame

    local valueInput: TextBox = (flatTextBox(slider, "") :: any) :: TextBox
    valueInput.Name = "NumericValue"
    valueInput.AnchorPoint = Vector2.new(1, 0)
    valueInput.Position = UDim2.new(1, -CHILD_INSET, 0, -22)
    valueInput.Size = UDim2.new(0.42, 0, 0, 22)
    valueInput.BackgroundTransparency = 1
    valueInput.TextColor3 = Theme.textMuted
    valueInput.TextSize = LABEL_FONT_SIZE
    valueInput.TextXAlignment = Enum.TextXAlignment.Right
    valueInput.TextTruncate = Enum.TextTruncate.AtEnd
    valueInput.ZIndex = option.ZIndex + 2
    local valueStroke: UIStroke? =
        valueInput:FindFirstChild("StyleStroke") :: UIStroke?
    if valueStroke then
        (valueStroke :: UIStroke).Transparency = 1
    end

    valueInput.PlaceholderText = boundTyped
        and (formatCompactNumber(minimum) .. "–" .. formatCompactNumber(maximum))
        or "Any number (type inf for infinity)"

    local editing: boolean = false

    local function quantize(raw: number, fromSlider: boolean): number
        if not fromSlider then
            -- Most gameplay sliders intentionally accept values outside
            -- their drag range. Bounded UI controls opt in explicitly so
            -- the displayed/saved value matches what they can apply.
            return boundTyped and math.clamp(raw, minimum, maximum) or raw
        end
        -- When maximum is infinite, skip the upper clamp and stepping
        -- so an infinity typed or dragged in passes through untouched.
        if maximum == math.huge then
            if raw < minimum then raw = minimum end
            return raw
        end
        raw = math.clamp(raw, minimum, maximum)
        local stepped: number = minimum
            + math.round((raw - minimum) / step) * step
        stepped = math.round(stepped * precisionFactor) / precisionFactor
        return math.clamp(stepped, minimum, maximum)
    end
    local function setValue(
        nextValue: number,
        persist: boolean,
        _fromSlider: boolean
    ): ()
        if not isNumber(nextValue) then
            return
        end
        -- Infinity is only valid when the slider explicitly opts in
        -- by having an infinite maximum (e.g. Fullbright Brightness).
        -- On finite-range sliders like Speed, reject it so a typed
        -- "inf" or "1e999" does not corrupt the previous value.
        if (nextValue == math.huge or nextValue == -math.huge)
            and maximum ~= math.huge then
            return
        end
        value = quantize(nextValue, _fromSlider)
        -- When the slider maximum is infinite, use the display cap
        -- so the knob still moves for finite values. Infinity itself
        -- snaps the knob to the far right.
        local sliderMax: number = effectiveMax(minimum, maximum)
        local alpha: number = sliderMax == minimum
            and 0
            or math.clamp((value - minimum) / (sliderMax - minimum), 0, 1)
        fill.Size = UDim2.fromScale(alpha, 1)
        knob.Position = UDim2.fromScale(alpha, 0.5)
        if not editing then
            valueInput.Text = formatCompactNumber(value)
        end
        callback(value)
        if persist then
            configData.values[optionKey] = value
            queueConfigSave()
        end
    end

    local function updateFromScreenX(screenX: number, origin: number, width: number): ()
        local alpha: number = math.clamp((screenX - origin) / width, 0, 1)
        setValue(minimum + (effectiveMax(minimum, maximum) - minimum) * alpha, true, true)
    end

    if state.valueControls then
        state.valueControls[optionKey] = {
            apply = function(stored: any): ()
                setValue(tonumber(stored) or defaultValue, true)
            end,
            default = defaultValue,
        }
    end

    trackUiConnection(slider.InputBegan:Connect(function(
        input: InputObject
    ): ()
        if input.UserInputType ~= Enum.UserInputType.MouseButton1
            and input.UserInputType ~= Enum.UserInputType.Touch then
            return
        end
        if input.Position.X > track.AbsolutePosition.X + track.AbsoluteSize.X then
            return
        end
        -- Rescaling the menu moves the track under the pointer. Keep the
        -- starting screen interval fixed for the duration of this drag.
        local dragOrigin: number = track.AbsolutePosition.X
        local dragWidth: number = math.max(track.AbsoluteSize.X, 1)
        local function updateDrag(screenX: number): ()
            updateFromScreenX(screenX, dragOrigin, dragWidth)
        end
        updateDrag(input.Position.X)
        activeSlider = {
            update = updateDrag,
            finish = function(): ()
                notify(
                    labelText
                        .. ": "
                        .. formatCompactNumber(value)
                )
            end,
        }
    end))

    trackUiConnection(valueInput.Focused:Connect(function(): ()
        editing = true
        valueInput.Text = tostring(value)
        valueInput.CursorPosition = #valueInput.Text + 1
    end))

    trackUiConnection(valueInput.FocusLost:Connect(function(): ()
        local parsed: number? = parseNumericText(valueInput.Text)
        editing = false
        if not isNumber(parsed) then
            valueInput.Text = formatCompactNumber(value)
            notify(labelText .. ": invalid numeric value")
            return
        end
        setValue(parsed :: number, true, false)
        notify(labelText .. ": " .. formatCompactNumber(value))
    end))

    task.defer(function(): ()
        if option.Parent then
            -- Migrate stale out-of-range UI values too; gameplay controls
            -- deliberately leave their old open-range profiles intact.
            local corrected: boolean = boundTyped == true and storedValue ~= nil
                and storedValue ~= value
            setValue(value, corrected, false)
        end
    end)
    return option
end

state.addRangeOption = function(
    feature: any,
    labelText: string,
    definition: any
): Frame
    local minimum: number = tonumber(definition.min) or 0
    local maximum: number = tonumber(definition.max) or 100
    local baseKey: string =
        feature.configKey .. "." .. labelText:gsub("%W", "")
    local storedLow: number? = tonumber(configData.values[baseKey .. ".Low"])
    local storedHigh: number? = tonumber(configData.values[baseKey .. ".High"])

    local low: number = isNumber(storedLow)
        and (storedLow :: number)
        or tonumber(definition.defaultLow)
        or minimum
    local high: number = isNumber(storedHigh)
        and (storedHigh :: number)
        or tonumber(definition.defaultHigh)
        or maximum
    if low > high then
        low, high = high, low
    end
    local integerOnly: boolean = definition.integer ~= false

    local option: Frame = createOptionRow(feature, labelText, definition.hint)

    option.Size = UDim2.new(1, 0, 0, 44)
    feature.updateExpandedSize()
    local rangeLabel: TextLabel? =
        option:FindFirstChild("OptionLabel") :: TextLabel?
    if rangeLabel then
        (rangeLabel :: TextLabel).Size =
            UDim2.new(0.55, -CHILD_INSET, 0, 22)
    end
    local slider: TextButton = (create("TextButton", {
        TextTruncate = Enum.TextTruncate.AtEnd,
        Parent = option,
        Name = "RangeSlider",
        Active = true,
        AutoButtonColor = false,
        BackgroundTransparency = 1,
        BorderSizePixel = 0,
        Position = UDim2.new(0, 0, 0, 22),
        Size = UDim2.new(1, 0, 0, 22),
        Text = "",
        ZIndex = option.ZIndex + 1,
    }) :: any) :: TextButton

    local track: Frame = (create("Frame", {
        Parent = option,
        Name = "Track",
        BackgroundColor3 = Color3.fromRGB(255, 255, 255),
        BackgroundTransparency = 0.9,
        BorderSizePixel = 0,
        Position = UDim2.new(0, CHILD_INSET, 0, 22 + (22 - SLIDER_BAR_HEIGHT) // 2),
        Size = UDim2.new(1, -CHILD_INSET * 2, 0, SLIDER_BAR_HEIGHT),
        ZIndex = option.ZIndex + 1,
    }) :: any) :: Frame

    local span: Frame = (create("Frame", {
        Parent = track,
        Name = "Span",
        BackgroundColor3 = Theme.accent,
        BorderSizePixel = 0,
        Position = UDim2.fromScale(0, 0),
        Size = UDim2.fromScale(1, 1),
        ZIndex = track.ZIndex + 1,
    }) :: any) :: Frame

    local function makeKnob(name: string): Frame
        local knob: Frame = (create("Frame", {
            Parent = track,
            Name = name,
            AnchorPoint = Vector2.new(0.5, 0.5),
            BackgroundColor3 = Theme.text,
            BorderSizePixel = 0,
            Position = UDim2.fromScale(0, 0.5),
            Size = UDim2.fromOffset(SLIDER_KNOB_WIDTH, SLIDER_KNOB_HEIGHT),
            ZIndex = track.ZIndex + 2,
        }) :: any) :: Frame
        return knob
    end
    local lowKnob: Frame = makeKnob("Low")
    local highKnob: Frame = makeKnob("High")

    local step: number? = tonumber(definition.step)
    if step ~= nil and step <= 0 then
        step = nil
    end
    option:SetAttribute("Min", minimum)
    option:SetAttribute("Max", maximum)
    option:SetAttribute("Step", step or (integerOnly and 1 or 0))
    option:SetAttribute("FreeNumeric", true)
    local highBox: TextBox = (flatTextBox(option, "") :: any) :: TextBox
    highBox.Name = "HighValue"
    highBox.AnchorPoint = Vector2.new(1, 0)
    highBox.Position = UDim2.new(1, -CHILD_INSET, 0, 0)
    highBox.Size = UDim2.fromOffset(52, 22)
    highBox.BackgroundTransparency = 1
    highBox.TextColor3 = Theme.textMuted
    highBox.TextSize = LABEL_FONT_SIZE
    highBox.TextXAlignment = Enum.TextXAlignment.Right
    highBox.TextTruncate = Enum.TextTruncate.AtEnd
    highBox.ZIndex = option.ZIndex + 2
    local _dash: TextLabel = (create("TextLabel", {
        TextTruncate = Enum.TextTruncate.AtEnd,
        Parent = option,
        Name = "RangeDash",
        AnchorPoint = Vector2.new(1, 0),
        BackgroundTransparency = 1,
        FontFace = CONTROL_FONT,
        Position = UDim2.new(1, -CHILD_INSET - 52, 0, 0),
        Size = UDim2.fromOffset(14, 22),
        Text = "–",
        TextColor3 = Theme.textMuted,
        TextSize = LABEL_FONT_SIZE,
        TextXAlignment = Enum.TextXAlignment.Center,
        ZIndex = option.ZIndex + 2,
    }) :: any) :: TextLabel
    local lowBox: TextBox = (flatTextBox(option, "") :: any) :: TextBox
    lowBox.Name = "LowValue"
    lowBox.AnchorPoint = Vector2.new(1, 0)
    lowBox.Position = UDim2.new(1, -CHILD_INSET - 52 - 14, 0, 0)
    lowBox.Size = UDim2.fromOffset(52, 22)
    lowBox.BackgroundTransparency = 1
    lowBox.TextColor3 = Theme.textMuted
    lowBox.TextSize = LABEL_FONT_SIZE
    lowBox.TextXAlignment = Enum.TextXAlignment.Right
    lowBox.TextTruncate = Enum.TextTruncate.AtEnd
    lowBox.ZIndex = option.ZIndex + 2
    for _, box: TextBox in ipairs({lowBox, highBox}) do
        local stroke: UIStroke? = box:FindFirstChild("StyleStroke") :: UIStroke?
        if stroke then
            (stroke :: UIStroke).Transparency = 1
        end
    end
    local editingLow: boolean = false
    local editingHigh: boolean = false

    local function alphaOf(value: number): number
        if maximum == minimum then
            return 0
        end
        return math.clamp((value - minimum) / (maximum - minimum), 0, 1)
    end

    local function refreshRange(persist: boolean): ()
        local lowAlpha: number = alphaOf(low)
        local highAlpha: number = alphaOf(high)
        span.Position = UDim2.fromScale(lowAlpha, 0)
        span.Size = UDim2.fromScale(math.max(highAlpha - lowAlpha, 0), 1)
        lowKnob.Position = UDim2.fromScale(lowAlpha, 0.5)
        highKnob.Position = UDim2.fromScale(highAlpha, 0.5)
        if not editingLow then
            lowBox.Text = formatCompactNumber(low)
        end
        if not editingHigh then
            highBox.Text = formatCompactNumber(high)
        end
        if type(definition.onChanged) == "function" then
            definition.onChanged(low, high)
        end
        if persist then
            configData.values[baseKey .. ".Low"] = low
            configData.values[baseKey .. ".High"] = high
            queueConfigSave()
        end
    end

    local dragging: string = "High"
    local function valueAtScreenX(screenX: number): number
        local width: number = math.max(track.AbsoluteSize.X, 1)
        local alpha: number = math.clamp(
            (screenX - track.AbsolutePosition.X) / width,
            0,
            1
        )
        local value: number = minimum + (maximum - minimum) * alpha
        if step ~= nil then
            value = minimum
                + math.round((value - minimum) / (step :: number))
                    * (step :: number)
        elseif integerOnly then
            value = math.round(value)
        end
        return math.clamp(value, minimum, maximum)
    end

    local function quantizeTyped(raw: number): number
        return raw
    end

    local function updateFromScreenX(screenX: number): ()
        local value: number = valueAtScreenX(screenX)
        if dragging == "Low" then
            low = math.min(value, high)
        else
            high = math.max(value, low)
        end
        refreshRange(true)
    end

    trackUiConnection(slider.InputBegan:Connect(function(
        input: InputObject
    ): ()
        if input.UserInputType ~= Enum.UserInputType.MouseButton1
            and input.UserInputType ~= Enum.UserInputType.Touch then
            return
        end
        if input.Position.X > track.AbsolutePosition.X + track.AbsoluteSize.X then
            return
        end
        local value: number = valueAtScreenX(input.Position.X)
        dragging = math.abs(value - low) <= math.abs(value - high)
            and "Low"
            or "High"
        updateFromScreenX(input.Position.X)
        activeSlider = {
            update = updateFromScreenX,
            finish = function(): ()
                notify(
                    labelText
                        .. ": "
```

```
                        .. formatCompactNumber(low)
                        .. " – "
                        .. formatCompactNumber(high)
                )
            end,
        }
    end))

    trackUiConnection(lowBox.Focused:Connect(function(): ()
        editingLow = true
        lowBox.Text = tostring(low)
        lowBox.CursorPosition = #lowBox.Text + 1
    end))
    trackUiConnection(lowBox.FocusLost:Connect(function(): ()
        local parsed: number? = parseNumericText(lowBox.Text)
        editingLow = false
        if not isNumber(parsed) then
            lowBox.Text = formatCompactNumber(low)
            notify(labelText .. ": invalid numeric value")
            return
        end
        low = math.min(quantizeTyped(parsed :: number), high)
        refreshRange(true)
        notify(labelText .. ": "
            .. formatCompactNumber(low) .. " – " .. formatCompactNumber(high))
    end))
    trackUiConnection(highBox.Focused:Connect(function(): ()
        editingHigh = true
        highBox.Text = tostring(high)
        highBox.CursorPosition = #highBox.Text + 1
    end))
    trackUiConnection(highBox.FocusLost:Connect(function(): ()
        local parsed: number? = parseNumericText(highBox.Text)
        editingHigh = false
        if not isNumber(parsed) then
            highBox.Text = formatCompactNumber(high)
            notify(labelText .. ": invalid numeric value")
            return
        end
        high = math.max(quantizeTyped(parsed :: number), low)
        refreshRange(true)
        notify(labelText .. ": "
            .. formatCompactNumber(low) .. " – " .. formatCompactNumber(high))
    end))

    task.defer(function(): ()
        if option.Parent then
            refreshRange(false)
        end
    end)
    refreshRange(false)
    return option
end

state.closeChoiceList = function(): () end
state.choiceListAnchor = nil :: TextButton?

local function watchPopupOwner(
    owner: any,
    onInvalid: () -> ()
): {RBXScriptConnection}
    local connections: {RBXScriptConnection} = {}
    if owner == nil then
        return connections
    end
    local scroller: any = nil
    local windowRoot: any = nil
    local node: any = owner
    while node ~= nil do
        if scroller == nil and type(node.IsA) == "function"
            and node:IsA("ScrollingFrame") then
            scroller = node
            windowRoot = node.Parent
            break
        end
        node = node.Parent
    end
    local function watch(instance: any, property: string, test: () -> boolean): ()
        pcall(function(): ()
            table.insert(
                connections,
                instance:GetPropertyChangedSignal(property):Connect(function(): ()
                    if test() then
                        onInvalid()
                    end
                end)
            )
        end)
    end
    watch(owner, "AbsolutePosition", function() return true end)
    watch(owner, "AbsoluteSize", function() return true end)
    watch(ScreenGui, "AbsoluteSize", function() return true end)
    if windowRoot then
        watch(windowRoot, "Visible", function(): boolean
            return windowRoot.Visible ~= true
        end)
        pcall(function(): ()
            table.insert(
                connections,
                windowRoot.AncestryChanged:Connect(function(): ()
                    if windowRoot.Parent == nil then
                        onInvalid()
                    end
                end)
            )
        end)
    end
    if scroller then

        watch(scroller, "Visible", function(): boolean
            return scroller.Visible ~= true
        end)
        watch(scroller, "CanvasPosition", function(): boolean
            local ok: boolean, clippedOut: any = pcall(function(): boolean
                local top: number = scroller.AbsolutePosition.Y
                local bottom: number = top + scroller.AbsoluteSize.Y
                local rowTop: number = owner.AbsolutePosition.Y
                local rowBottom: number = rowTop + owner.AbsoluteSize.Y
                return rowBottom < top or rowTop > bottom
            end)
            return ok and clippedOut == true
        end)
    end
    return connections
end

local function selectorGeometry(anchor, wantedHeight)
    local viewport = ScreenGui.AbsoluteSize
    local camera = workspace.CurrentCamera
    if (viewport.X < 1 or viewport.Y < 1) and camera then viewport = camera.ViewportSize end
    local row = anchor.Parent
    local owner = row
    while owner and owner ~= ScreenGui do
        if owner:GetAttribute("SelectorOwner") then break end
        owner = owner.Parent
    end
    if not owner or owner == ScreenGui then owner = row end
    local width = math.min(math.max(anchor.AbsoluteSize.X, 160), math.max(1, viewport.X - 16))
    local height = math.min(wantedHeight, math.max(24, viewport.Y - 16))
    local bottom = math.max(anchor.AbsolutePosition.Y + anchor.AbsoluteSize.Y,
        owner.AbsolutePosition.Y + owner.AbsoluteSize.Y)
    local x = math.clamp(anchor.AbsolutePosition.X, 8, math.max(8, viewport.X - width - 8))
    local y = bottom + 6
    -- Prefer below. At the screen edge use a bounded side placement before
    -- falling back above, so a long settings window stays usable.
    if y + height > viewport.Y - 8 then
        local right = owner.AbsolutePosition.X + owner.AbsoluteSize.X + 6
        local left = owner.AbsolutePosition.X - width - 6
        if right + width <= viewport.X - 8 then x = right
        elseif left >= 8 then x = left
        else y = owner.AbsolutePosition.Y - height - 6 end
        if x == right or x == left then y = anchor.AbsolutePosition.Y end
    end
    y = math.clamp(y, 8, math.max(8, viewport.Y - height - 8))
    return UDim2.fromOffset(math.round(x), math.round(y)), UDim2.fromOffset(math.round(width), math.round(height))
end

local function openChoiceList(
    anchor: TextButton,
    values: {string},
    currentIndex: number,
    onPick: (number) -> ()
): ()
    if state.choiceListAnchor == anchor then
        state.closeChoiceList()
        return
    end
    state.closeChoiceList()

    local rowHeight = 24

    local searchable: boolean = #values > 8
    local searchHeight: number = searchable and 24 or 0
    local visibleRows: number = math.min(#values, 6)
    local listHeight: number = visibleRows * rowHeight + 8 + searchHeight
    local listPosition, listSize = selectorGeometry(anchor, listHeight)

    local list: Frame = create("Frame", {
        Parent = ScreenGui,
        Name = "ChoiceList",
        BackgroundColor3 = Theme.surfaceRaised,
        BackgroundTransparency = 0.22,
        BorderSizePixel = 0,
        Position = listPosition,
        Size = listSize,
        ZIndex = 400,
    }) :: Frame
    create("UIStroke", {
        Parent = list,
        Color = Theme.outline,
        Transparency = 0.6,
        Thickness = 1,
    })

    do
        local bitmap: any = state.bitmapText
        if bitmap and type(bitmap.adoptTree) == "function" then
            bitmap.adoptTree(list)
        end
    end

    local filterBox: TextBox? = nil
    if searchable then
        local filterFrame: Frame = create("Frame", {
            Parent = list,
            Name = "Filter",
            BackgroundColor3 = Theme.background,
            BackgroundTransparency = 0.25,
            BorderSizePixel = 0,
            ClipsDescendants = true,
            Position = UDim2.fromOffset(4, 4),
            Size = UDim2.new(1, -8, 0, searchHeight - 4),
            ZIndex = 402,
        }) :: Frame
        local box: TextBox = flatTextBox(filterFrame, "")
        box.Name = "FilterBox"
        box.BackgroundTransparency = 1
        box.PlaceholderText = "Search"
        box.Position = UDim2.fromOffset(8, 0)
        box.Size = UDim2.new(1, -12, 1, 0)
        box.TextXAlignment = Enum.TextXAlignment.Left
        box.TextSize = 14
        box.ZIndex = 403
        local boxCorner: UICorner? =
            box:FindFirstChild("StyleCorner") :: UICorner?
        if boxCorner then
            boxCorner:Destroy()
        end
        local boxStroke: UIStroke? =
            box:FindFirstChild("StyleStroke") :: UIStroke?
        if boxStroke then
            boxStroke:Destroy()
        end
        filterBox = box
    end

    local scroller: ScrollingFrame = create("ScrollingFrame", {
        Parent = list,
        Name = "Items",
        Active = true,
        AutomaticCanvasSize = Enum.AutomaticSize.Y,
        BackgroundTransparency = 1,
        BorderSizePixel = 0,
        CanvasSize = UDim2.new(),
        Position = UDim2.fromOffset(4, 4 + searchHeight),
        ScrollBarThickness = 2,
        ScrollBarImageTransparency = 0.6,
        Size = UDim2.new(1, -8, 1, -(8 + searchHeight)),
        ZIndex = 401,
    }) :: ScrollingFrame
    create("UIListLayout", {
        Parent = scroller,
        Padding = UDim.new(0, 1),
        SortOrder = Enum.SortOrder.LayoutOrder,
    })

    local closed: boolean = false
    local outsideConnection: RBXScriptConnection? = nil
    local ownerConnections: {RBXScriptConnection} = {}
    local function close(): ()
        if closed then
            return
        end
        closed = true
        if outsideConnection then
            outsideConnection:Disconnect()
            outsideConnection = nil
        end
        for _, connection: RBXScriptConnection in ipairs(ownerConnections) do
            pcall(function(): ()
                connection:Disconnect()
            end)
        end
        ownerConnections = {}
        list:Destroy()
        if state.choiceListAnchor == anchor then
            state.choiceListAnchor = nil
        end
        if state.setChoiceBlur then state.setChoiceBlur(false) end
        state.closeChoiceList = function(): () end
    end
    state.closeChoiceList = close
    state.choiceListAnchor = anchor
    if state.setChoiceBlur then state.setChoiceBlur(true) end

    ownerConnections = watchPopupOwner(anchor, close)

    local rowsByValue: {[string]: TextButton} = {}
    for itemIndex: number, value: string in ipairs(values) do
        local item: TextButton = (create("TextButton", {
            TextTruncate = Enum.TextTruncate.AtEnd,
            ClipsDescendants = true,
            Parent = scroller,
            Name = "Choice" .. tostring(itemIndex),
            AutoButtonColor = false,
            BackgroundColor3 = Theme.surfaceHover,
            BackgroundTransparency = itemIndex == currentIndex and 0.25 or 1,
            BorderSizePixel = 0,
            FontFace = CONTROL_FONT,
            LayoutOrder = itemIndex,
            Size = UDim2.new(1, 0, 0, rowHeight - 1),
            Text = value,
            TextColor3 = itemIndex == currentIndex
                and Theme.text
                or Theme.textMuted,
            TextSize = 14,
            ZIndex = 402,
        }) :: any) :: TextButton
        create("UIPadding", {
            Parent = item,
            PaddingLeft = UDim.new(0, 10),
            PaddingRight = UDim.new(0, 8),
        })

        if itemIndex == currentIndex then
            create("Frame", {
                Parent = item,
                Name = "PickedBar",
                BackgroundColor3 = Theme.enabled,
                BorderSizePixel = 0,
                Position = UDim2.fromOffset(-10, 0),
                Size = UDim2.new(0, 3, 1, 0),
                ZIndex = 403,
            })
        end

        local itemStroke: UIStroke = (create("UIStroke", {
            Parent = item,
            Name = "EntryStroke",
            Color = Theme.outline,
            Thickness = 1,
            Transparency = itemIndex == currentIndex and 0.35 or 0.82,
        }) :: any) :: UIStroke
        item.TextXAlignment = Enum.TextXAlignment.Left
        item.MouseEnter:Connect(function(): ()
            if itemIndex ~= currentIndex then
                item.BackgroundTransparency = 0.6
                item.TextColor3 = Theme.text
            end
            itemStroke.Transparency = 0.35
        end)
        item.MouseLeave:Connect(function(): ()
            item.BackgroundTransparency = itemIndex == currentIndex and 0.25 or 1
            item.TextColor3 = itemIndex == currentIndex
                and Theme.text
                or Theme.textMuted
            itemStroke.Transparency = itemIndex == currentIndex and 0.35 or 0.82
        end)
        item.MouseButton1Click:Connect(function(): ()
            close()
            onPick(itemIndex)
        end)
        rowsByValue[value] = item
    end

    if filterBox then
        local box: TextBox = filterBox :: TextBox
        trackUiConnection(box:GetPropertyChangedSignal("Text"):Connect(function(): ()
            local query: string = string.lower(box.Text)
            for value: string, row: TextButton in pairs(rowsByValue) do
                row.Visible = query == ""
                    or string.find(string.lower(value), query, 1, true) ~= nil
            end
        end))
        task.defer(function(): ()
            if not closed then
                box:CaptureFocus()
            end
        end)
    end

    outsideConnection = UserInputService.InputBegan:Connect(function(
        input: InputObject
    ): ()
        if input.KeyCode == Enum.KeyCode.Escape then close();return end
        if input.UserInputType ~= Enum.UserInputType.MouseButton1
            and input.UserInputType ~= Enum.UserInputType.Touch then
            return
        end
        task.defer(function(): ()
            if closed then
                return
            end
            local pointer: Vector2 = Vector2.new(input.Position.X, input.Position.Y)
            local origin: Vector2 = list.AbsolutePosition
            local size: Vector2 = list.AbsoluteSize
            if pointer.X < origin.X
                or pointer.X > origin.X + size.X
                or pointer.Y < origin.Y
                or pointer.Y > origin.Y + size.Y then
                close()
            end
        end)
    end)
    trackUiConnection(outsideConnection :: RBXScriptConnection)
end

state.openChoiceList = openChoiceList

state.addMenuVisibilityListener(function(visible: boolean): ()
    if not visible then
        state.closeChoiceList()
    end
end)

state.openMultiList = function(
    anchor: TextButton,
    items: {string},
    selected: {[string]: boolean},
    onToggle: (string, boolean) -> (),
    onBulk: ((boolean) -> ())?,
    onRemove: ((string) -> ())?
): ()
    if state.choiceListAnchor == anchor then
        state.closeChoiceList()
        return
    end
    state.closeChoiceList()

    local rowHeight: number = 24
    local rows: number = math.min(#items + (onBulk and 1 or 0), 7)
    local listHeight: number = math.max(rows, 1) * rowHeight + 8
    local listPosition, listSize = selectorGeometry(anchor, listHeight)

    local list: Frame = create("Frame", {
        Parent = ScreenGui,
        Name = "MultiList",
        BackgroundColor3 = Theme.surface,
        BackgroundTransparency = 0.22,
        BorderSizePixel = 0,
        Position = listPosition,
        Size = listSize,
        ZIndex = 400,
    }) :: Frame
    create("UIStroke", {
        Parent = list,
        Color = Theme.outline,
        Transparency = 0.6,
        Thickness = 1,
    })

    local scroller: ScrollingFrame = create("ScrollingFrame", {
        Parent = list,
        Name = "Items",
        Active = true,
        AutomaticCanvasSize = Enum.AutomaticSize.Y,
        BackgroundTransparency = 1,
        BorderSizePixel = 0,
        CanvasSize = UDim2.new(),
        Position = UDim2.fromOffset(4, 4),
        ScrollBarThickness = 2,
        ScrollBarImageTransparency = 0.6,
        Size = UDim2.new(1, -8, 1, -8),
        ZIndex = 401,
    }) :: ScrollingFrame
    create("UIListLayout", {
        Parent = scroller,
        Padding = UDim.new(0, 1),
        SortOrder = Enum.SortOrder.LayoutOrder,
    })

    local closed: boolean = false
    local outsideConnection: RBXScriptConnection? = nil
    local ownerConnections: {RBXScriptConnection} = {}
    local function close(): ()
        if closed then
            return
        end
        closed = true
        if outsideConnection then
            outsideConnection:Disconnect()
            outsideConnection = nil
        end
        for _, connection: RBXScriptConnection in ipairs(ownerConnections) do
            pcall(function(): ()
                connection:Disconnect()
            end)
        end
        ownerConnections = {}
        list:Destroy()
        if state.choiceListAnchor == anchor then
            state.choiceListAnchor = nil
        end
        if state.setChoiceBlur then state.setChoiceBlur(false) end
        state.closeChoiceList = function(): () end
    end
    state.closeChoiceList = close
    state.choiceListAnchor = anchor
    if state.setChoiceBlur then state.setChoiceBlur(true) end

    ownerConnections = watchPopupOwner(anchor, close)

    local function makeRow(text: string, order: number): TextButton
        local row: TextButton = (create("TextButton", {
            TextTruncate = Enum.TextTruncate.AtEnd,
            ClipsDescendants = true,
            Parent = scroller,
            Name = "Row" .. tostring(order),
            AutoButtonColor = false,
            BackgroundColor3 = Theme.surfaceHover,
            BackgroundTransparency = 1,
            BorderSizePixel = 0,
            FontFace = CONTROL_FONT,
            LayoutOrder = order,
            Size = UDim2.new(1, 0, 0, rowHeight - 1),
            Text = text,
            TextColor3 = Theme.textMuted,
            TextSize = 14,
            ZIndex = 402,
        }) :: any) :: TextButton
        create("UIPadding", {
            Parent = row,
            PaddingLeft = UDim.new(0, 26),
            PaddingRight = UDim.new(0, 8),
        })
        row.TextXAlignment = Enum.TextXAlignment.Left
        return row
    end

    if onBulk then
        local bulk: TextButton = makeRow("All / none", 0)
        bulk.TextColor3 = Theme.text
        local anyOn: boolean = false
        for _, item: string in ipairs(items) do
            if selected[item] then
                anyOn = true
                break
            end
        end
        bulk.MouseButton1Click:Connect(function(): ()
            close();
            (onBulk :: (boolean) -> ())(not anyOn)
        end)
    end

    for index: number, item: string in ipairs(items) do
        local row: TextButton = makeRow(item, index)

        if onRemove then
            -- Sibling buttons: delete must never toggle selection.
            local wrapper=create("Frame",{Name="Row"..index,Parent=scroller,BackgroundTransparency=1,
                Size=UDim2.new(1,0,0,rowHeight-1),LayoutOrder=index,ZIndex=402})
            row.Name="Select";row.Parent=wrapper;row.Size=UDim2.new(1,-24,1,0)
            row.TextTruncate=Enum.TextTruncate.AtEnd
            local remove=create("TextButton",{Name="Remove",Parent=wrapper,AnchorPoint=Vector2.new(1,0),
                Position=UDim2.fromScale(1,0),Size=UDim2.new(0,24,1,0),BackgroundTransparency=1,
                BorderSizePixel=0,Text="×",Font=Font=CONTROL_FONT,TextSize=16,TextColor3=Theme.textMuted,
                AutoButtonColor=false,ZIndex=403})
            remove.MouseButton1Click:Connect(function() close();onRemove(item) end)
        end

        local mark: Frame = create("Frame", {
            Parent = row,
            Name = "Mark",
            AnchorPoint = Vector2.new(0, 0.5),
            BackgroundTransparency = 1,
            BorderSizePixel = 0,
            Position = UDim2.new(0, -18, 0.5, 0),
            Size = UDim2.fromOffset(11, 11),
            ZIndex = 403,
        }) :: Frame
        local markStroke: UIStroke = (create("UIStroke", {
            Parent = mark,
            Color = Theme.textMuted,
            Transparency = 0.35,
            Thickness = 1,
        }) :: any) :: UIStroke
        local fill: Frame = create("Frame", {
            Parent = mark,
            Name = "Fill",
            AnchorPoint = Vector2.new(0.5, 0.5),
            BackgroundColor3 = Theme.text,
            BackgroundTransparency = 1,
            BorderSizePixel = 0,
            Position = UDim2.fromScale(0.5, 0.5),
            Size = UDim2.fromScale(0, 0),
            ZIndex = 404,
        }) :: Frame

        local function paint(): ()
            local on: boolean = selected[item] == true
            fill.Size = on and UDim2.fromScale(0.6, 0.6) or UDim2.fromScale(0, 0)
            fill.BackgroundTransparency = on and 0.05 or 1
            markStroke.Transparency = on and 0.1 or 0.35
            row.TextColor3 = on and Theme.text or Theme.textMuted
        end
        paint()

        row.MouseEnter:Connect(function(): ()
            row.BackgroundTransparency = 0.72
        end)
        row.MouseLeave:Connect(function(): ()
            row.BackgroundTransparency = 1
        end)
        row.MouseButton1Click:Connect(function(): ()

            selected[item] = not selected[item]
            paint()
            onToggle(item, selected[item] == true)
        end)
    end

    outsideConnection = UserInputService.InputBegan:Connect(function(
        input: InputObject
    ): ()
        if input.KeyCode == Enum.KeyCode.Escape then close();return end
        if input.UserInputType ~= Enum.UserInputType.MouseButton1
            and input.UserInputType ~= Enum.UserInputType.Touch then
            return
        end
        task.defer(function(): ()
            if closed then
                return
            end
            local pointer: Vector2 = Vector2.new(input.Position.X, input.Position.Y)
            local origin: Vector2 = list.AbsolutePosition
            local size: Vector2 = list.AbsoluteSize
            if pointer.X < origin.X
                or pointer.X > origin.X + size.X
                or pointer.Y < origin.Y
                or pointer.Y > origin.Y + size.Y then
                close()
            end
        end)
    end)
    trackUiConnection(outsideConnection :: RBXScriptConnection)
end

state.addListOption = function(feature: any, labelText: string, definition: any): TextButton
    local optionKey: string = feature.configKey .. "." .. labelText:gsub("%W", "")
    local selected: {[string]: boolean} = {}
    local stored: any = configData.values[optionKey]
    if type(stored) == "string" and stored ~= "" then
        for entry: string in string.gmatch(stored, "[^,]+") do
            local trimmed: string = string.match(entry, "^%s*(.-)%s*$") or entry
            if trimmed ~= "" then
                selected[trimmed] = true
            end
        end
    elseif type(definition.default) == "table" then
        for _, entry: string in ipairs(definition.default) do
            selected[entry] = true
        end
    end

    local option: Frame = createOptionRow(feature, labelText, definition.hint)
    local button: TextButton = makeButton(option, "", 11)
    state.layoutOptionControl(option, button, 0.5)
    button.TextXAlignment = Enum.TextXAlignment.Left
    create("UIPadding", {
        Parent = button,
        Name = "ListPadding",
        PaddingLeft = UDim.new(0, 8),
        PaddingRight = UDim.new(0, 10),
    })

    local function currentItems(): {string}
        local provider: any = definition.items
        if type(provider) == "function" then
            local ok: boolean, result: any = pcall(provider)
            if ok and type(result) == "table" then
                return result
            end
```

```
            return {}
        end
        return provider or {}
    end

    local function describe(): ()
        local items: {string} = currentItems()
        local total: number = #items
        local count: number = 0
        local last: string = ""
        for _, item: string in ipairs(items) do
            if selected[item] then
                count += 1
                last = item
            end
        end
        if total == 0 then

            button.Text = definition.emptyText or ""
        elseif count == 0 then
            button.Text = definition.single and "Choose..." or ""
        elseif count == 1 then
            button.Text = last
        elseif count == total then
            button.Text = "all " .. tostring(total)
        else
            button.Text = tostring(count) .. " of " .. tostring(total)
        end
    end

    local function publish(): ()
        local names: {string} = {}
        for name: string, on: boolean in pairs(selected) do
            if on then
                table.insert(names, name)
            end
        end
        table.sort(names)
        configData.values[optionKey] = table.concat(names, ",")
        queueConfigSave()
        describe()
        if type(definition.onChanged) == "function" then
            definition.onChanged(selected, names)
        end
    end

    local function setItem(item: string, on: boolean): ()
        if definition.single and on then table.clear(selected) end
        selected[item] = on or nil
        publish()
    end
    local function openItems(): ()
        local items: {string} = currentItems()
        local bulk = nil
        if not definition.single then
            bulk = function(enableAll: boolean): ()
                for _, item: string in ipairs(items) do selected[item] = enableAll or nil end
                publish()
            end
        end
        state.openMultiList(button, items, selected, function(item, on)
            setItem(item, on)
            if definition.single then state.closeChoiceList() end
        end, bulk, definition.onRemove and function(item)
            selected[item]=nil
            definition.onRemove(item)
            publish()
            if #currentItems()>0 then openItems() end
        end or nil)
    end
    button.MouseButton1Click:Connect(openItems)

    local initialNames = {}
    if definition.single then
        for name, on in pairs(selected) do if on then table.insert(initialNames,name) end end
        table.sort(initialNames)
        if #initialNames > 1 then
            local first = initialNames[1]
            table.clear(selected);selected[first]=true;initialNames={first}
        end
    end
    describe()
    if type(definition.onChanged) == "function" then definition.onChanged(selected, initialNames) end

    button:SetAttribute("ListOption", optionKey)
    state.listSelections = state.listSelections or {}
    state.listSelections[optionKey] = selected
    state.listControls = state.listControls or {}
    state.listControls[optionKey] = {
        set = function(item: string, on: boolean): ()
            setItem(item, on)
        end,
        refresh = function()
            if state.choiceListAnchor == button then state.closeChoiceList() end
            publish()
        end,
        selected = selected,
    }
    button:SetAttribute("ListKey", optionKey)
    return button
end

local function addCycleOption(
    feature: any,
    labelText: string,
    values: {string},
    initialIndex: number?,
    callback: (string) -> (),
    hintText: string?
): TextButton
    local optionKey = feature.configKey .. "." .. labelText:gsub("%W", "")
    local storedValue = configData.values[optionKey]
    local option = createOptionRow(feature, labelText, hintText)
    local index = initialIndex or 1
    if storedValue ~= nil then
        for valueIndex, value in ipairs(values) do
            if value == storedValue then
                index = valueIndex
                break
            end
        end
    end

    local button: TextButton = (create("TextButton", {
        TextTruncate = Enum.TextTruncate.AtEnd,
        Parent = option,
        Name = "ComboValue",
        Active = true,
        AutoButtonColor = false,
        BackgroundTransparency = 1,
        BorderSizePixel = 0,
        FontFace = CONTROL_FONT,
        Position = UDim2.new(0.55, 4, 0, 0),
        Size = UDim2.new(0.45, -(26 + CHILD_INSET), 1, 0),
        ClipsDescendants = true,
        Text = values[index],
        TextColor3 = Theme.text,
        TextSize = LABEL_FONT_SIZE,
        TextXAlignment = Enum.TextXAlignment.Right,
        ZIndex = option.ZIndex + 2,
    }) :: any) :: TextButton

    local arrowBox: TextButton = (create("TextButton", {
        TextTruncate = Enum.TextTruncate.AtEnd,
        Parent = option,
        Name = "ComboArrow",
        Active = true,
        AnchorPoint = Vector2.new(1, 0),
        AutoButtonColor = false,
        BackgroundTransparency = 1,
        BorderSizePixel = 0,
        Position = UDim2.new(1, -CHILD_INSET, 0, 0),
        Size = UDim2.fromOffset(22, 22),
        Text = "",
        ZIndex = option.ZIndex + 2,
    }) :: any) :: TextButton
    drawPixelIcon(arrowBox, "down", nil, "arrows")
    if state and type(state.registerThemeRefresh) == "function" then
        state.registerThemeRefresh(function(): ()
            if arrowBox.Parent then
                drawPixelIcon(arrowBox, "down", nil, "arrows")
            end
        end, {"arrows"})
    end

    local function refreshChoiceVisual(): ()
        if #values > 0 then
            index = math.clamp(index, 1, #values)
        end
        button.Text = values[index] or ""
    end
    refreshChoiceVisual()

    local function openList(): ()
        local control: any = state.valueControls
            and state.valueControls[optionKey]
        if type(control) == "table"
            and type(control.onBeforeOpen) == "function" then
            pcall(control.onBeforeOpen)
            refreshChoiceVisual()
        end
        openChoiceList(button, values, index, function(picked: number): ()
            index = picked
            refreshChoiceVisual()
            callback(values[index])
            configData.values[optionKey] = values[index]
            queueConfigSave()
        end)
    end
    button.MouseButton1Click:Connect(openList)
    arrowBox.MouseButton1Click:Connect(openList)

    if state.valueControls then
        state.valueControls[optionKey] = {
            apply = function(stored: any): ()
                local picked: number? =
                    table.find(values, tostring(stored))
                if picked == nil then
                    return
                end
                index = picked :: number
                refreshChoiceVisual()
                callback(values[index])
                configData.values[optionKey] = values[index]
                queueConfigSave()
            end,
            setIndex = function(newIndex: number): ()
                index = math.clamp(newIndex, 1, math.max(1, #values))
                refreshChoiceVisual()
            end,
            refresh = refreshChoiceVisual,
            default = values[initialIndex or 1],
        }
    end

    callback(values[index])
    return button :: TextButton
end

local function addKeyOption(
    feature: any,
    labelText: string,
    defaultKey: Enum.KeyCode,
    callback: (Enum.KeyCode) -> (),
    hintText: string?
): TextButton

    feature.hasKeyOption = true
    local optionKey = feature.configKey .. "." .. labelText:gsub("%W", "")
    local savedKey = nil
    local savedName = configData.values[optionKey]
    if type(savedName) == "string" and savedName ~= "" then
        pcall(function()
            savedKey = Enum.KeyCode[savedName]
        end)
    end
    local initialKey = savedKey or defaultKey
    local currentKey: Enum.KeyCode = initialKey
    local function keyLabel(name: any): string
        if type(state.keyDisplayName) == "function" then
            return state.keyDisplayName(name)
        end
        return tostring(name or "")
    end
    local option = createOptionRow(feature, labelText, hintText)
    -- A phone cannot press keys. The row is not merely inert: it is
    -- removed from the card so the menu never offers an impossible
    -- control. The action itself stays available through its own card or
    -- the Buttons page.
    if state.isMobile then
        option.Visible = false
        feature.updateExpandedSize()
    end
    local button = makeButton(
        option,
        currentKey == Enum.KeyCode.Unknown and "" or keyLabel(currentKey.Name),
        11
    )
    state.layoutOptionControl(option, button, 0.5)
    applyKeySlotStyle(button)
    button:SetAttribute("CaptureIdleText", button.Text)

    local function assignKey(keyCode: Enum.KeyCode): boolean
        local cleared: boolean = keyCode == currentKey
        currentKey = cleared and Enum.KeyCode.Unknown or keyCode
        button.Text = currentKey == Enum.KeyCode.Unknown
            and ""
            or keyLabel(currentKey.Name)
        button:SetAttribute("CaptureIdleText", button.Text)
        callback(currentKey)
        configData.values[optionKey] = currentKey.Name
        queueConfigSave()
        return not cleared
    end

    button.MouseButton1Click:Connect(function()
        if state.isMobile then
            return
        end
        beginKeyCapture(button, function(keyCode: Enum.KeyCode): ()
            assignKey(keyCode)
        end)
    end)

    callback(currentKey)
    return button :: TextButton
end

-- A card can expose its existing action shortcut as an option without
-- creating a second key or a second persistence entry. This remains in
-- sync with the Shortcuts settings page and old saved bindings.
state.addFeatureShortcutOption = function(feature: any, label: string, hint: string?): any
    local binding: any = state.shortcutBindings and state.shortcutBindings[feature.configKey]
    if not binding or state.isMobile then return nil end
    local option: any = createOptionRow(feature, label, hint)
    local button: TextButton = (makeButton(option, "", 11) :: any) :: TextButton
    state.layoutOptionControl(option, button, 0.5)
    applyKeySlotStyle(button)
    local previousRefresh: any = binding.refresh
    binding.refresh = function(): ()
        if type(previousRefresh) == "function" then previousRefresh() end
        local key: any = binding.key
        button.Text = key and key ~= Enum.KeyCode.Unknown
            and state.keyDisplayName(key.Name) or ""
        button:SetAttribute("CaptureIdleText", button.Text)
    end
    binding.refresh()
    trackUiConnection(button.MouseButton1Click:Connect(function(): ()
        beginKeyCapture(button, function(keyCode: Enum.KeyCode): ()
            binding.assign(keyCode)
        end)
    end))
    return button
end

local function addTextOption(
    feature: any,
    labelText: string,
    defaultText: string,
    callback: (string) -> (),
    persist: boolean?,
    hintText: string?
): TextBox
    local optionKey = feature.configKey .. "." .. labelText:gsub("%W", "")
    local shouldPersist = persist ~= false
    local initialText = shouldPersist and configData.values[optionKey] or nil
    if initialText == nil then
        initialText = defaultText or ""
    end
    local option = createOptionRow(feature, labelText, hintText)
    local box = flatTextBox(option, initialText)
    box.PlaceholderText = "Enter"
    box.TextXAlignment = Enum.TextXAlignment.Center
    state.layoutOptionControl(option, box, 0.5)

    box.FocusLost:Connect(function(enterPressed)
        if enterPressed then
            callback(box.Text)
        end
    end)

    box:GetPropertyChangedSignal("Text"):Connect(function()
        callback(box.Text)
        if shouldPersist then
            configData.values[optionKey] = box.Text
            queueConfigSave()
        end
    end)

    if not shouldPersist and configData.values[optionKey] ~= nil then
        configData.values[optionKey] = nil
        queueConfigSave()
    end

    if state.valueControls then
        state.valueControls[optionKey] = {
            apply = function(stored: any): ()
                box.Text = tostring(stored or "")
                if shouldPersist then
                    configData.values[optionKey] = box.Text
                    queueConfigSave()
                end
            end,
            default = defaultText or "",
        }
    end
    callback(initialText)
    return box :: TextBox
end

local colorPicker = {}

colorPicker.open = function(initialColor, onApply, ownerRow)
    if not colorPicker.panel then
        local panel = create("Frame", {
            Parent = PopupLayer,
            Name = "ColorPicker",
            Active = true,
            BackgroundColor3 = Theme.surface,
            BorderColor3 = Theme.outline,
            BorderSizePixel = 1,
            Position = UDim2.new(0.5, -170, 0.5, -150),
            Size = UDim2.fromOffset(340, 300),
            Visible = false,
            ZIndex = 460,
        })

        do
            local bitmap: any = state.bitmapText
            if bitmap and type(bitmap.adoptTree) == "function" then
                bitmap.adoptTree(panel)
            end
        end

        local title = makeTextLabel(panel, "COLOR", 14)
        title.FontFace = CONTROL_FONT
        title.Position = UDim2.fromOffset(15, 8)
        title.Size = UDim2.new(1, -60, 0, 28)
        title.ZIndex = 81

        local close = makeButton(panel, "X", 12)
        close.AutoButtonColor = false
        close.Position = UDim2.new(1, -42, 0, 8)
        close.Size = UDim2.fromOffset(28, 28)
        close.ZIndex = 82

        local sv = makeButton(panel, "", 10)
        sv.AutoButtonColor = false
        sv:SetAttribute("NoHover", true)
        sv.Position = UDim2.fromOffset(15, 44)
        sv.Size = UDim2.fromOffset(205, 165)
        sv.ZIndex = 81
        local saturationGradient = create("UIGradient", {
            Parent = sv,
            Color = ColorSequence.new(
                Color3.fromRGB(255, 255, 255),
                Color3.fromRGB(255, 0, 0)
            ),
        })
        local shade = create("Frame", {
            Parent = sv,
            BackgroundColor3 = Theme.background,
            BorderSizePixel = 0,
            Size = UDim2.fromScale(1, 1),
            ZIndex = 462,
        })
        create("UIGradient", {
            Parent = shade,
            Rotation = 90,
            Transparency = NumberSequence.new({
                NumberSequenceKeypoint.new(0, 1),
                NumberSequenceKeypoint.new(1, 0),
            }),
        })
        local svCursor = create("Frame", {
            Parent = sv,
            AnchorPoint = Vector2.new(0.5, 0.5),
            BackgroundColor3 = Color3.fromRGB(255, 255, 255),
            BorderColor3 = Theme.surface,
            BorderSizePixel = 2,
            Size = UDim2.fromOffset(12, 12),
            ZIndex = 463,
        })
        create("UICorner", {
            Parent = svCursor,
            CornerRadius = UDim.new(1, 0),
        })

        local hue = makeButton(panel, "", 10)
        hue.AutoButtonColor = false
        hue:SetAttribute("NoHover", true)
        hue.Position = UDim2.fromOffset(232, 44)
        hue.Size = UDim2.fromOffset(22, 165)
        hue.ZIndex = 81
        create("UIGradient", {
            Parent = hue,
            Rotation = 90,
            Color = ColorSequence.new({
                ColorSequenceKeypoint.new(0, Color3.fromHSV(0, 1, 1)),
                ColorSequenceKeypoint.new(0.17, Color3.fromHSV(0.17, 1, 1)),
                ColorSequenceKeypoint.new(0.33, Color3.fromHSV(0.33, 1, 1)),
                ColorSequenceKeypoint.new(0.5, Color3.fromHSV(0.5, 1, 1)),
                ColorSequenceKeypoint.new(0.67, Color3.fromHSV(0.67, 1, 1)),
                ColorSequenceKeypoint.new(0.83, Color3.fromHSV(0.83, 1, 1)),
                ColorSequenceKeypoint.new(1, Color3.fromHSV(1, 1, 1)),
            }),
        })
        local hueCursor = create("Frame", {
            Parent = hue,
            AnchorPoint = Vector2.new(0.5, 0.5),
            BackgroundColor3 = Color3.fromRGB(245, 245, 245),
            BorderColor3 = Theme.surface,
            BorderSizePixel = 1,
            Position = UDim2.new(0.5, 0, 0, 0),
            Size = UDim2.new(1, 6, 0, 5),
            ZIndex = 463,
        })

        local preview = create("Frame", {
            Parent = panel,
            BackgroundColor3 = Color3.fromRGB(255, 255, 255),
            BorderColor3 = Color3.fromRGB(85, 85, 85),
            BorderSizePixel = 1,
            Position = UDim2.fromOffset(273, 45),
            Size = UDim2.fromOffset(48, 48),
            ZIndex = 461,
        })

        local apply = makeButton(panel, "APPLY", 11)
        apply.AutoButtonColor = false
        apply.Position = UDim2.fromOffset(268, 105)
        apply.Size = UDim2.fromOffset(58, 34)
        apply.ZIndex = 81

        local hex = flatTextBox(panel, "#ffffff")
        hex.Name = "HexField"
        hex.Position = UDim2.fromOffset(15, 218)
        hex.Size = UDim2.fromOffset(150, 30)
        hex.PlaceholderText = "#rrggbb"
        hex.TextXAlignment = Enum.TextXAlignment.Center
        hex.ZIndex = 81

        local rgb = flatTextBox(panel, "255, 255, 255")
        rgb.Name = "RgbField"
        rgb.Position = UDim2.fromOffset(174, 218)
        rgb.Size = UDim2.fromOffset(151, 30)
        rgb.PlaceholderText = "r, g, b"
        rgb.TextXAlignment = Enum.TextXAlignment.Center
        rgb.ZIndex = 81

        local hint = makeTextLabel(
            panel,
            "Drag the square for saturation and brightness, the strip for hue",
            10
        )
        hint.TextColor3 = Color3.fromRGB(145, 145, 145)
        hint.Position = UDim2.fromOffset(15, 256)
        hint.Size = UDim2.new(1, -30, 0, 24)
        hint.ZIndex = 81

        colorPicker.panel = panel
        colorPicker.sv = sv
        colorPicker.saturationGradient = saturationGradient
        colorPicker.svCursor = svCursor
        colorPicker.hue = hue
        colorPicker.hueCursor = hueCursor
        colorPicker.preview = preview
        colorPicker.hex = hex
        colorPicker.rgb = rgb
        colorPicker.apply = apply
        colorPicker.drag = nil

        local function updateColor()
            local color = Color3.fromHSV(
                colorPicker.h,
                colorPicker.s,
                colorPicker.v
            )
            colorPicker.current = color
            colorPicker.preview.BackgroundColor3 = color
            colorPicker.saturationGradient.Color = ColorSequence.new(
                Color3.fromRGB(255, 255, 255),
                Color3.fromHSV(colorPicker.h, 1, 1)
            )
            colorPicker.svCursor.Position = UDim2.fromScale(
                colorPicker.s,
                1 - colorPicker.v
            )
            colorPicker.hueCursor.Position = UDim2.new(
                0.5,
                0,
                colorPicker.h,
                0
            )
            local channels = colorToConfig(color)
            colorPicker.hex.Text = string.format(
                "#%02x%02x%02x",
                channels[1],
                channels[2],
                channels[3]
            )
            colorPicker.rgb.Text = string.format(
                "%d, %d, %d",
                channels[1],
                channels[2],
                channels[3]
            )
        end
        colorPicker.update = updateColor

        local function updateFromPointer(kind, position)
            local object = kind == "hue" and hue or sv
            local relative = Vector2.new(position.X, position.Y)
                - object.AbsolutePosition
            if kind == "hue" then
                colorPicker.h = math.clamp(
                    relative.Y / object.AbsoluteSize.Y,
                    0,
                    1
                )
            else
                colorPicker.s = math.clamp(
                    relative.X / object.AbsoluteSize.X,
                    0,
                    1
                )
                colorPicker.v = 1 - math.clamp(
                    relative.Y / object.AbsoluteSize.Y,
                    0,
                    1
                )
            end
            updateColor()
        end

        sv.InputBegan:Connect(function(input)
            if input.UserInputType == Enum.UserInputType.MouseButton1
                or input.UserInputType == Enum.UserInputType.Touch then
                colorPicker.drag = "sv"
                updateFromPointer("sv", input.Position)
            end
        end)
        hue.InputBegan:Connect(function(input)
            if input.UserInputType == Enum.UserInputType.MouseButton1
                or input.UserInputType == Enum.UserInputType.Touch then
                colorPicker.drag = "hue"
                updateFromPointer("hue", input.Position)
            end
        end)
        colorPicker.inputChanged = UserInputService.InputChanged:Connect(function(input)
            if colorPicker.panel.Visible
                and colorPicker.drag
                and (input.UserInputType == Enum.UserInputType.MouseMovement
                    or input.UserInputType == Enum.UserInputType.Touch) then
                updateFromPointer(colorPicker.drag, input.Position)
            end
        end)
        colorPicker.inputEnded = UserInputService.InputEnded:Connect(function(input)
            if input.UserInputType == Enum.UserInputType.MouseButton1
                or input.UserInputType == Enum.UserInputType.Touch then
                colorPicker.drag = nil
            end
        end)

        hex.FocusLost:Connect(function()
            local value = string.match(hex.Text, "^#?(%x%x%x%x%x%x)$")
            if value then
                local r = tonumber(string.sub(value, 1, 2), 16)
                local g = tonumber(string.sub(value, 3, 4), 16)
                local b = tonumber(string.sub(value, 5, 6), 16)
                colorPicker.h, colorPicker.s, colorPicker.v =
                    Color3.fromRGB(r, g, b):ToHSV()
            end

            updateColor()
        end)
        rgb.FocusLost:Connect(function()
            local r, g, b = string.match(
                rgb.Text,
                "^%s*(%d+)%s*[, ]%s*(%d+)%s*[, ]%s*(%d+)%s*$"
            )
            if r and g and b then
                colorPicker.h, colorPicker.s, colorPicker.v = Color3.fromRGB(
                    math.clamp(tonumber(r) or 0, 0, 255),
                    math.clamp(tonumber(g) or 0, 0, 255),
                    math.clamp(tonumber(b) or 0, 0, 255)
                ):ToHSV()
            end
            updateColor()
        end)
        close.MouseButton1Click:Connect(function()
            panel.Visible = false
        end)
        apply.MouseButton1Click:Connect(function()
            if colorPicker.callback then
                colorPicker.callback(colorPicker.current)
            end
            panel.Visible = false
        end)
    end

    colorPicker.h, colorPicker.s, colorPicker.v = initialColor:ToHSV()
    colorPicker.callback = onApply
```

```
        colorPicker.panel.Visible = true
        colorPicker.update()

        for _, connection in ipairs(colorPicker.ownerConnections or {}) do
            pcall(function()
                connection:Disconnect()
            end)
        end
        colorPicker.ownerConnections = watchPopupOwner(ownerRow, function()
            colorPicker.panel.Visible = false
            for _, connection in ipairs(colorPicker.ownerConnections or {}) do
                pcall(function()
                    connection:Disconnect()
                end)
            end
            colorPicker.ownerConnections = {}
        end)
    end

    state.colorPicker = colorPicker

    -- `keyText` (v191) keeps the SAVED config key stable when a row is
    -- renamed: "Arrows" became "Buttons" and the stored colour must survive.
    local function addColorOption(feature, labelText, defaultColor, callback, hintText, keyText)
        local optionKey = feature.configKey .. "."
            .. (keyText or labelText):gsub("%W", "")
        local color = colorFromConfig(configData.values[optionKey], defaultColor)
        -- A nil here (missing saved value AND missing default token) used to
        -- crash hexOf below and take the whole Settings page down with it.
        if color == nil then
            color = Color3.fromRGB(255, 255, 255)
        end
        local option = createOptionRow(feature, labelText, hintText)

        option.Size = UDim2.new(1, 0, 0, 44)
        feature.updateExpandedSize()
        local colorLabel: TextLabel? =
            option:FindFirstChild("OptionLabel") :: TextLabel?
        if colorLabel then
            (colorLabel :: TextLabel).Size =
                UDim2.new(0.55, -CHILD_INSET, 0, 22)
        end

        local hexValue: TextLabel = (create("TextLabel", {
            TextTruncate = Enum.TextTruncate.AtEnd,
            Parent = option,
            Name = "HexValue",
            AnchorPoint = Vector2.new(1, 0),
            BackgroundTransparency = 1,
            FontFace = CONTROL_FONT,
            Position = UDim2.new(1, -CHILD_INSET, 0, 0),
            Size = UDim2.new(0.42, 0, 0, 22),
            Text = "",
            TextColor3 = Theme.textMuted,
            TextSize = LABEL_FONT_SIZE,
            TextXAlignment = Enum.TextXAlignment.Right,
            ZIndex = option.ZIndex + 2,
        }) :: any) :: TextLabel

        local bar: Frame = (create("Frame", {
            Parent = option,
            Name = "ColorBar",
            BackgroundColor3 = color,
            BorderSizePixel = 0,
            Position = UDim2.new(0, CHILD_INSET, 0, 22),
            Size = UDim2.new(1, -CHILD_INSET * 2, 0, 22),
            ZIndex = option.ZIndex + 1,
        }) :: any) :: Frame

        local swatch = create("TextButton", {
            TextTruncate = Enum.TextTruncate.AtEnd,
            Parent = option,
            Name = "ColorClickTarget",
            Active = true,
            AutoButtonColor = false,
            BackgroundTransparency = 1,
            BorderSizePixel = 0,
            Size = UDim2.fromScale(1, 1),
            Text = "",
            ZIndex = option.ZIndex + 3,
        })

        local function hexOf(value: Color3): string
            return string.format(
                "#%02X%02X%02X",
                math.floor(value.R * 255 + 0.5),
                math.floor(value.G * 255 + 0.5),
                math.floor(value.B * 255 + 0.5)
            )
        end
        hexValue.Text = hexOf(color)

        swatch.MouseButton1Click:Connect(function()
            colorPicker.open(color, function(newColor)
                color = newColor
                bar.BackgroundColor3 = color
                hexValue.Text = hexOf(color)
                configData.values[optionKey] = colorToConfig(color)
                callback(color)
                queueConfigSave()
            end, swatch)
        end)

        if state.valueControls then
            state.valueControls[optionKey] = {
                apply = function(stored: any): ()
                    local restored: Color3 =
                        colorFromConfig(stored, defaultColor)
                    color = restored
                    bar.BackgroundColor3 = color
                    hexValue.Text = hexOf(color)
                    configData.values[optionKey] = colorToConfig(color)
                    callback(color)
                    queueConfigSave()
                end,
                default = colorToConfig(defaultColor),
            }
        end

        callback(color)
        return swatch
    end

    local function addInformationOption(feature: any, text: string): ()
        local current: string = feature.tooltipText or feature.description or ""
        local combined: string = current ~= "" and (current .. "\n" .. text) or text
        feature.tooltipText = combined
        feature.description = combined
    end

    type OptionDefinition = {
        kind: string,
        label: string,
        default: any?,
        minimum: number?,
        maximum: number?,
        values: {string}?,
        callback: any,
        persist: boolean?,
    }

    type FeatureDefinition = {
        name: string,
        description: string,
        order: number,
        onToggle: any,
        configuration: any?,
        options: {OptionDefinition}?,
    }

    local function addConfiguredOptions(
        feature: any,
        definitions: {OptionDefinition}
    ): ()
        for _, definition: OptionDefinition in ipairs(definitions) do
            if definition.kind == "toggle" then
                addToggleOption(
                    feature,
                    definition.label,
                    definition.default == true,
                    definition.callback
                )
            elseif definition.kind == "slider" then
                addNumberOption(
                    feature,
                    definition.label,
                    tonumber(definition.default) or 0,
                    definition.minimum or 0,
                    definition.maximum or 100,
                    definition.callback
                )
            elseif definition.kind == "color" then
                addColorOption(
                    feature,
                    definition.label,
                    definition.default :: Color3,
                    definition.callback
                )
            elseif definition.kind == "cycle" then
                addCycleOption(
                    feature,
                    definition.label,
                    definition.values or {},
                    tonumber(definition.default) or 1,
                    definition.callback
                )
            elseif definition.kind == "keybind" then
                addKeyOption(
                    feature,
                    definition.label,
                    definition.default :: Enum.KeyCode,
                    definition.callback
                )
            elseif definition.kind == "text" then
                addTextOption(
                    feature,
                    definition.label,
                    tostring(definition.default or ""),
                    definition.callback,
                    definition.persist
                )
            elseif definition.kind == "action" then
                addActionOption(feature, definition.label, definition.callback)
            elseif definition.kind == "info" then
                addInformationOption(feature, definition.label)
            end
        end
    end

    local function createFeatureFromDefinition(
        definition: FeatureDefinition
    ): any

        local feature: any = host.createUniversalFeature(
            definition.name,
            definition.description,
            definition.order,
            definition.onToggle,
            definition.configuration
        )
        if definition.options then
            addConfiguredOptions(feature, definition.options)
        end
        return feature
    end

    local function createFeaturesFromDefinitions(
        definitions: {FeatureDefinition}
    ): {any}
        local createdFeatures: {any} = {}
        for _, definition: FeatureDefinition in ipairs(definitions) do
            table.insert(
                createdFeatures,
                createFeatureFromDefinition(definition)
            )
        end
        return createdFeatures
    end

    state.components = {
        createFeature = createFeatureFromDefinition,
        createFeatures = createFeaturesFromDefinitions,
        addOptions = addConfiguredOptions,
    }

    -- Lets a module sync a toggle row (visual + saved state + callback)
    -- without clicking it. Returns false when the option does not exist.
    state.setToggleChecked = function(optionKey: string, checked: boolean): boolean
        local control: any = state.toggleControls
            and state.toggleControls[optionKey]
        if type(control) == "table" and type(control.setChecked) == "function" then
            control.setChecked(checked)
            return true
        end
        return false
    end

    -- Panel switching for calibration cards. Each producer
    -- calls begin with its panel name and gets a finish closure; a "Mode"
    -- cycle at the top of the card selects which panel's rows are visible.
    -- Switching views never changes any recorded preference. `pinToFront`
    -- moves the panel to the front of the selector and selects it (used by
    -- the MM2 Calibration panel, which registers after the universal ones).
    state.beginCardPanel = function(
        feature: any,
        name: string,
        pinToFront: boolean?
    ): () -> ()
        local panels: any = feature.cardPanels
        if not panels then
            panels = {selected = name, values = {}, groups = {}}
            feature.cardPanels = panels
            function panels:refresh(): ()
                for group: any, rows: any in pairs(self.groups) do
                    for _, row: any in ipairs(rows) do
                        row.Visible = group == self.selected
                    end
                end
                feature.updateExpandedSize()
            end
            function panels:buildSelector(): ()
                if self.selector then
                    if state.closeChoiceList then
                        state.closeChoiceList()
                    end
                    local oldRow: any = self.selector.Parent
                    local index: number? = table.find(feature.optionRows, oldRow)
                    if index then
                        table.remove(feature.optionRows, index :: number)
                    end
                    oldRow:Destroy()
                end
                self.selector = addCycleOption(
                    feature,
                    "Mode",
                    self.values,
                    table.find(self.values, self.selected) or 1,
                    function(value: string): ()
                        self.selected = value
                        self:refresh()
                    end,
                    "Select controls to display. Recording toggles stay "
                        .. "independent; switching views never discards "
                        .. "data or stops an enabled observer."
                )
                if self.selector then
                    -- addCycleOption restores the persisted value on its own;
                    -- keep the visible panel in sync with what it shows.
                    local restored: any = (self.selector :: any).Text
                    if type(restored) == "string"
                        and table.find(self.values, restored) then
                        self.selected = restored
                    end
                    (self.selector :: any).Parent.LayoutOrder = -1
                end
                self:refresh()
            end
        end
        if not table.find(panels.values, name) then
            if pinToFront then
                table.insert(panels.values, 1, name)
                panels.selected = name
            else
                table.insert(panels.values, name)
            end
            panels:buildSelector()
        end
        local first: number = #feature.optionRows + 1
        return function(): ()
            local rows: {any} = {}
            for index: number = first, #feature.optionRows do
                table.insert(rows, feature.optionRows[index])
                end
                panels.groups[name] = rows
                panels:refresh()
            end
        end

        Module.Initialized = true
        return {
            addToggleOption = addToggleOption,
            addActionOption = addActionOption,
            addNumberOption = addNumberOption,
            addRangeOption = state.addRangeOption,
            addCycleOption = addCycleOption,
            addKeyOption = addKeyOption,
            addTextOption = addTextOption,
            addColorOption = addColorOption,
            addListOption = state.addListOption,
            addSectionOption = state.addSectionOption,
            addInformationOption = addInformationOption,
            setOptionVisible = setOptionVisible,
            drawPixelIcon = drawPixelIcon,
        }
    end

    function Module.destroy(): ()
        Module.Initialized = false
        table.clear(toggleThemeRefreshers)
        if boundState and type(boundState.valueControls) == "table" then
            table.clear(boundState.valueControls)
        end
        if boundState and boundState.optionThemeRefresh == runToggleThemeRefreshers then
            boundState.optionThemeRefresh = nil
        end
        boundState = nil
    end

    return Module

    ----------------------------------------------------------------------------------------------------
    FIN DEL ARCHIVO: src/guis/Wurst/Code/Widgets.lua
    ----------------------------------------------------------------------------------------------------


    ----------------------------------------------------------------------------------------------------
    ARCHIVO: src/guis/Wurst/Code/WindowManager.lua
    BYTES: 51323
    SHA-256: 35649a5cc4c58a9f7398429cc5f1ad352ff6a73b7f0dc60d7defc93f26cfba5c
    INICIO DEL ARCHIVO
    ----------------------------------------------------------------------------------------------------
    export type WindowDefinition = {
        id: string,
        title: string?,
        size: Vector2,
        position: Vector2,

        chromeless: boolean?,
        closable: boolean?,
        collapsible: boolean?,
        pinnable: boolean?,
        maxHeight: number?,
        onClose: (() -> ())?,
    }

    export type Window = {
        id: string,
        root: GuiObject,
        body: GuiObject,
        header: Frame?,
        closeButton: TextButton?,
        collapsed: boolean,
        pinned: boolean,
        SetVisible: (self: Window, visible: boolean) -> (),
        SetCollapsed: (self: Window, collapsed: boolean) -> (),
        SetPinned: (self: Window, pinned: boolean) -> (),
        Raise: (self: Window) -> (),
        Destroy: (self: Window) -> (),
    }

    local Module = {
        Name = "WindowManager",
        PlaceId = 0,
        Events = {} :: {[string]: any},
        Initialized = false,
    }

    local activeCleanup: (() -> ())? = nil

    local SNAP_DISTANCE: number = 8

    local BASE_Z: number = 150
    local MARGIN: number = 6

    local PLACEMENT_GAP: number = 10

    local TITLE_HEIGHT: number = 26

    local TITLE_FONT_SIZE: number = 16

    function Module.init(context: any): any
        local host: any = context
        local state: any = host.state
        local configData: any = host.configData
        local queueConfigSave: any = host.queueConfigSave
        local create: any = host.create
        local Theme: any = host.Theme
        local ThemeEngine: any = host.ThemeEngine
        local CONTROL_FONT: any = host.CONTROL_FONT
        local PopupLayer: any = host.PopupLayer
        local trackUiConnection: any = host.trackUiConnection
        local UserInputService: any = host.UserInputService

        local drawPixelIcon: any = host.drawPixelIcon
            or function(parent: Instance, _kind: string, _color: Color3?): Frame
                local existing: Instance? = (parent :: any):FindFirstChild("PixelIcon")
                if existing then
                    existing:Destroy()
                end
                return create("Frame", {
                    Parent = parent,
                    Name = "PixelIcon",
                    BackgroundTransparency = 1,
                }) :: Frame
            end

        configData.ui = configData.ui or {}
        configData.ui.windows = configData.ui.windows or {}

        local manager: any = {
            list = {} :: {any},
            byId = {} :: {[string]: any},
        }

        local function screenViewport(): Vector2
            local stable: any = state.layoutViewport
            if stable and stable.X > 1 and stable.Y > 1 then return stable end
            local camera: Camera? = workspace.CurrentCamera
            local current: any = camera and camera.ViewportSize
            if current and current.X > 1 and current.Y > 1 then return current end
            return Vector2.new(1280, 720)
        end

        local function layerScale(): number
            local instance: any = PopupLayer
                and (PopupLayer :: Instance):FindFirstChild("ClickGuiScale")
            local value: number = instance and tonumber(instance.Scale) or 1
            if value ~= value or value <= 0 then
                value = 1
            end
            return value
        end

        local function logicalViewport(): Vector2
            local screen: Vector2 = screenViewport()
            local factor: number = layerScale()
            return Vector2.new(screen.X / factor, screen.Y / factor)
        end

        local function screenToLogical(point: Vector2): Vector2
            local factor: number = layerScale()
            return Vector2.new(point.X / factor, point.Y / factor)
        end

        local function logicalToScreen(point: Vector2): Vector2
            local factor: number = layerScale()
            return Vector2.new(point.X * factor, point.Y * factor)
        end

        local function windowRect(window: any): (number, number, number, number)
            local root: any = window.root
            return root.Position.X.Offset, root.Position.Y.Offset,
                root.Size.X.Offset, root.Size.Y.Offset
        end

        local function snapPixel(value: number): number
            local factor: number = layerScale()
            return math.round(value * factor) / factor
        end

        -- How much of a window must stay reachable when it cannot fit:
        -- enough title bar to grab, move and close it.
        local MIN_GRAB_X: number = 140
        local MIN_GRAB_Y: number = 40

        local function clamp(position: Vector2, size: Vector2): Vector2
            local screen: Vector2 = logicalViewport()
            -- When the window fits, keep it fully on screen. When it does
            -- not (a small or resized viewport), the old full-fit clamp
            -- pinned every oversized window to the same top-left corner and
            -- they stacked into one pile; instead let the window extend
            -- past the edge and only guarantee the grab strip stays
            -- visible, so part of it simply shows, as requested.
            local x: number
            if size.X <= screen.X - MARGIN * 2 then
                x = math.clamp(position.X, MARGIN, screen.X - size.X - MARGIN)
            else
                x = math.clamp(
                    position.X,
                    MARGIN - (size.X - MIN_GRAB_X),
                    screen.X - MIN_GRAB_X
                )
            end
            local y: number
            if size.Y <= screen.Y - MARGIN * 2 then
                y = math.clamp(position.Y, MARGIN, screen.Y - size.Y - MARGIN)
            else
                y = math.clamp(position.Y, MARGIN, screen.Y - MIN_GRAB_Y)
            end
            return Vector2.new(snapPixel(x), snapPixel(y))
        end

        function manager.ClampWindow(window: any): ()
            local root: any = window and window.root
            if not root then
                return
            end
            local landed: Vector2 = clamp(
                Vector2.new(root.Position.X.Offset, root.Position.Y.Offset),
                Vector2.new(root.Size.X.Offset, root.Size.Y.Offset)
            )
            root.Position = UDim2.fromOffset(landed.X, landed.Y)
        end

        local function rectsOverlap(
            aPos: Vector2,
            aSize: Vector2,
            bPos: Vector2,
            bSize: Vector2
        ): boolean
            local overlapX: number = math.min(aPos.X + aSize.X, bPos.X + bSize.X)
                - math.max(aPos.X, bPos.X)
            local overlapY: number = math.min(aPos.Y + aSize.Y, bPos.Y + bSize.Y)
                - math.max(aPos.Y, bPos.Y)
            if overlapX <= 0 or overlapY <= 0 then
                return false
            end
            return overlapX * overlapY > 0.3 * aSize.X * aSize.Y
        end

        -- A window that opens (nearly) on top of a visible one reads as
        -- the two "glitching into each other": same landing spot, rows
        -- bleeding through. Cascade it a step at a time until its body is
        -- mostly its own, exactly like the reclamp cascade but at open.
        function manager.Deoverlap(window: any): ()
            local root: any = window and window.root
            if not root or window.userMoved then
                -- A manually saved position belongs to the user, even if they
                -- deliberately chose to overlap another window.
                return
            end
            local size: Vector2 = Vector2.new(
                root.Size.X.Offset,
                root.Size.Y.Offset
            )
            local position: Vector2 = Vector2.new(
                root.Position.X.Offset,
                root.Position.Y.Offset
            )
            local initial: Vector2 = position
            for _ = 1, 10 do
                local stacked: boolean = false
                for _, other: any in ipairs(manager.list) do
                    if other ~= window and other.root and other.root.Visible then
                        if rectsOverlap(
                            position,
                            size,
                            Vector2.new(
                                other.root.Position.X.Offset,
                                other.root.Position.Y.Offset
                            ),
                            Vector2.new(
                                other.root.Size.X.Offset,
                                other.root.Size.Y.Offset
                            )
                        ) then
                            stacked = true
                            break
                        end
                    end
                end
                if not stacked then
                    break
                end
                position = clamp(position + Vector2.new(30, 30), size)
            end
            root.Position = UDim2.fromOffset(position.X, position.Y)
            if (position.X ~= initial.X or position.Y ~= initial.Y)
                and type(manager.SetHome) == "function" then
                manager.SetHome(window, position)
            end
        end

        function manager.MeasureWindow(window: any): Vector2
            local root: any = window and window.root
            if not root then
                return Vector2.zero
            end
            local width: number = root.Size.X.Offset
            local height: number
            if window.collapsed then
                height = window.headerHeight or root.Size.Y.Offset
            else
                height = math.max(root.Size.Y.Offset, tonumber(window.fullHeight) or 0)
            end
            return Vector2.new(width, height)
        end

        function manager.AbsoluteRect(window: any): (number, number, number, number)
            local x: number, y: number, w: number, h: number = windowRect(window)
            local origin: Vector2 = logicalToScreen(Vector2.new(x, y))
            local extent: Vector2 = logicalToScreen(Vector2.new(w, h))
            return origin.X, origin.Y, extent.X, extent.Y
        end

        manager.LogicalViewport = logicalViewport
        manager.ScreenToLogical = screenToLogical
        manager.LogicalToScreen = logicalToScreen

        local function bind(instance: Instance, property: string, token: string): ()
            if ThemeEngine and type(ThemeEngine.Bind) == "function" then
                ThemeEngine.Bind(instance, property, token)
            else
                (instance :: any)[property] = Theme[token]
            end
        end

        -- Each window's home position: where the user (or its default)
        -- actually put it. A viewport shrink only borrows a clamped
        -- position; it must never become the window's place, or growing
        -- the screen again cannot put the window back (the pile-up in
        -- the report). Reclamp below always derives from home.
        local homePositions: {[string]: Vector2} = {}

        -- Automatic tiling/placement establishes a new home too. Otherwise the
```

Leer columna izquierda completa y despues la derecha. Texto seleccionable; lineas largas comprimidas. Tabulaciones: 4 espacios. Original adjunto.

```
    -- next scale/viewport refresh snaps windows back to their creation slots.
    function manager.SetHome(window: any, position: Vector2): ()
        if window and window.id and position then
            homePositions[window.id] = Vector2.new(position.X, position.Y)
        end
    end

    local function persist(window: any): ()
        local home: Vector2? = homePositions[window.id]
        configData.ui.windows[window.id] = {
            x = home and home.X or window.root.Position.X.Offset,
            y = home and home.Y or window.root.Position.Y.Offset,
            collapsed = window.collapsed,
            pinned = window.pinned,
        }
        queueConfigSave()
    end

    local function snap(window: any, position: Vector2, size: Vector2): Vector2
        local screen: Vector2 = logicalViewport()
        local xLines: {number} = {MARGIN, screen.X - size.X - MARGIN}
        local yLines: {number} = {MARGIN, screen.Y - size.Y - MARGIN}
        for _, other: any in ipairs(manager.list) do
            if other ~= window and other.root.Visible then
                local otherX: number, otherY: number, otherW: number, otherH: number =
                    windowRect(other)
                table.insert(xLines, otherX)
                table.insert(xLines, otherX + otherW)
                table.insert(xLines, otherX - size.X)
                table.insert(yLines, otherY)
                table.insert(yLines, otherY + otherH)
                table.insert(yLines, otherY - size.Y)
            end
        end
        local x: number, y: number = position.X, position.Y
        for _, line: number in ipairs(xLines) do
            if math.abs(x - line) <= SNAP_DISTANCE then
                x = line
                break
            end
        end
        for _, line: number in ipairs(yLines) do
            if math.abs(y - line) <= SNAP_DISTANCE then
                y = line
                break
            end
        end
        return Vector2.new(snapPixel(x), snapPixel(y))
    end

    local function raise(window: any): ()
        local index: number? = table.find(manager.list, window)
        if index then
            table.remove(manager.list, index)
        end
        table.insert(manager.list, window)
        for order: number, entry: any in ipairs(manager.list) do

            local base: number = math.min(BASE_Z + order, 239)
            entry.root.ZIndex = base

            if entry.layered then
                for _, descendant: Instance in ipairs(entry.root:GetDescendants()) do
                    if descendant:IsA("GuiObject") then
                        local relative: number =
                            tonumber(descendant:GetAttribute("FloatingDepth")) or 1
                        descendant.ZIndex = base + relative
                    end
                end
            end
        end

        function manager.Raise(id: string): ()
            local window: any = manager.byId[id]
            if window then
                raise(window)
            end
        end

        function manager.Adopt(root: GuiObject, definition: any): any
            root:SetAttribute("SelectorOwner", true)
            local id: string = definition.id
            local saved: any = configData.ui.windows[id]
            if type(saved) ~= "table" then

                local legacy: any = (configData.ui.floatingPositions or {})[id]
                if type(legacy) == "table" then
                    saved = {x = tonumber(legacy[1]), y = tonumber(legacy[2])}
                end
            end

            local window: any = {
                id = id,
                root = root,
                body = definition.body,
                header = definition.header,
                closeButton = definition.closeButton,
                collapsed = type(saved) == "table" and saved.collapsed == true,
                pinned = type(saved) == "table" and saved.pinned == true,
                fullHeight = root.Size.Y.Offset,
                headerHeight = definition.headerHeight or 0,
                layered = definition.layered == true,

                connections = {} :: {RBXScriptConnection},
            }

            local function own(connection: RBXScriptConnection): RBXScriptConnection
                table.insert(window.connections, connection)
                trackUiConnection(connection)
                return connection
            end

            local savedHome: Vector2? = nil
            if type(saved) == "table" and saved.x and saved.y then
                local savedX: number? = tonumber(saved.x)
                local savedY: number? = tonumber(saved.y)
                if savedX and savedY then
                    savedHome = Vector2.new(savedX, savedY)
                    local landed: Vector2 = clamp(
                        savedHome,
                        Vector2.new(root.Size.X.Offset, root.Size.Y.Offset)
                    )
                    root.Position = UDim2.fromOffset(landed.X, landed.Y)
                    window.userMoved = true
                end
            end
            -- Do not replace an off-screen saved position with a temporary clamp;
            -- it becomes reachable again when the viewport grows.
            homePositions[id] = savedHome or Vector2.new(
                root.Position.X.Offset,
                root.Position.Y.Offset
            )

            function window:SetVisible(visible: boolean): ()
                root.Visible = visible
                if visible then
                    raise(window)
                    manager.Deoverlap(window)
                end
            end

            function window:SetCollapsed(collapsed: boolean): ()
                window.collapsed = collapsed
                if window.body then
                    (window.body :: any).Visible = not collapsed
                end

                root.Size = collapsed
                    and UDim2.fromOffset(root.Size.X.Offset, window.headerHeight)
                    or UDim2.fromOffset(root.Size.X.Offset, window.fullHeight)
                persist(window)
            end

            function window:SetPinned(pinned: boolean): ()
                window.pinned = pinned

                if not pinned and state.visible ~= true then
                    root.Visible = false
                end
                persist(window)
            end

            function window:Raise(): ()
                raise(window)
            end

            function window:Destroy(): ()
                if state.surfaceDragOwner == root then
                    state.surfaceDragOwner = nil
                end
                local index: number? = table.find(manager.list, window)
                if index then
                    table.remove(manager.list, index)
                end
                manager.byId[id] = nil
                for _, connection: RBXScriptConnection in ipairs(window.connections) do
                    pcall(function(): ()
                        connection:Disconnect()
                    end)
                end
                window.connections = {}
                root:Destroy()
            end

            local dragHandle: GuiObject = definition.dragHandle or root
            local requiresMenuOpen: boolean = definition.requiresMenuOpen == true
            local dragging: boolean = false
            local dragInput: InputObject? = nil
            local dragStart: Vector2 = Vector2.zero
            local startPosition: Vector2 = Vector2.new(
                root.Position.X.Offset,
                root.Position.Y.Offset
            )

            own(root.InputBegan:Connect(function(): ()
                raise(window)
            end))
            own(dragHandle.InputBegan:Connect(function(
                input: InputObject
            ): ()
                if requiresMenuOpen and not state.visible then
                    return
                end
                local pointer: Vector2 = Vector2.new(input.Position.X, input.Position.Y)
                if state.surfaceDragHitTest
                    and state.surfaceDragHitTest(pointer) then
                    return
                end
                if state.surfaceDragOwner ~= nil
                    and state.surfaceDragOwner ~= root then
                    return
                end
                if input.UserInputType ~= Enum.UserInputType.MouseButton1
                    and input.UserInputType ~= Enum.UserInputType.Touch then
                    return
                end

                local absoluteSize: Vector2 = dragHandle.AbsoluteSize
                if absoluteSize.X > 0 and absoluteSize.Y > 0 then
                    local absolutePosition: Vector2 = dragHandle.AbsolutePosition
                    if input.Position.X < absolutePosition.X
                        or input.Position.X > absolutePosition.X + absoluteSize.X
                        or input.Position.Y < absolutePosition.Y
                        or input.Position.Y > absolutePosition.Y + absoluteSize.Y then
                        return
                    end
                end
                dragging = true
                dragInput = input
                state.surfaceDragOwner = root
                dragStart = Vector2.new(input.Position.X, input.Position.Y)
                startPosition = Vector2.new(
                    root.Position.X.Offset,
                    root.Position.Y.Offset
                )
                raise(window)
            end))
            own(UserInputService.InputChanged:Connect(function(
                input: InputObject
            ): ()
                if not dragging then
                    return
                end
                if input == dragInput
                    or (dragInput
                        and dragInput.UserInputType == Enum.UserInputType.MouseButton1
                        and input.UserInputType == Enum.UserInputType.MouseMovement) then

                    local pointer: Vector2 = Vector2.new(input.Position.X, input.Position.Y)
                    local delta: Vector2 = screenToLogical(pointer - dragStart)
                    local size: Vector2 = Vector2.new(
                        root.Size.X.Offset,
                        root.Size.Y.Offset
                    )
                    local wanted: Vector2 = startPosition + delta
                    local landed: Vector2 = snap(window, clamp(wanted, size), size)
                    root.Position = UDim2.fromOffset(landed.X, landed.Y)
                end
            end))
            own(UserInputService.InputEnded:Connect(function(
                input: InputObject
            ): ()
                if dragging
                    and (input == dragInput
                        or input.UserInputType == Enum.UserInputType.MouseButton1) then
                    dragging = false
                    dragInput = nil
                    if state.surfaceDragOwner == root then
                        state.surfaceDragOwner = nil
                    end

                    window.userMoved = true
                    homePositions[window.id] = Vector2.new(
                        root.Position.X.Offset,
                        root.Position.Y.Offset
                    )
                    persist(window)
                end
            end))
            if window.closeButton then
                own(
                    (window.closeButton :: TextButton).MouseButton1Click:Connect(function(): ()
                        root.Visible = false
                        if definition.onClose then
                            definition.onClose()
                        end
                    end)
                )
            end

            manager.byId[id] = window
            raise(window)
            return window
        end

        function manager.Create(definition: WindowDefinition): any
            local id: string = definition.id
            local bare: boolean = definition.chromeless == true
            local headerHeight: number = bare and 0 or TITLE_HEIGHT
            local saved: any = configData.ui.windows[id]

            local position: Vector2 = definition.position
            if type(saved) == "table" then
                position = Vector2.new(
                    tonumber(saved.x) or position.X,
                    tonumber(saved.y) or position.Y
                )
            end
            position = clamp(position, definition.size)
            homePositions[id] = position

            -- A single alpha is applied after composing the panel, rather than
            -- independently fading overlapping rows, labels, icons and strokes.
            -- Keep chromeless/floating overlays as Frames: CanvasGroup clips its
            -- descendants and should not change those overlays' separate styling.
            local root: any = create(bare and "Frame" or "CanvasGroup", {
                Parent = PopupLayer,
                Name = "Window_" .. id,
                Active = true,
                BackgroundTransparency = bare and 1 or 0,
                BorderSizePixel = 0,
                ClipsDescendants = not bare,
                Position = UDim2.fromOffset(position.X, position.Y),
                Size = UDim2.fromOffset(definition.size.X, definition.size.Y),
                Visible = false,
                ZIndex = BASE_Z,
            })
            if not bare then
                -- v191: the panel surface is what Opacity fades (see SetOpacity).
                -- The group itself stays opaque so labels, icons and strokes keep
                -- full contrast at every slider position.
                root.GroupTransparency = 0
                root.BackgroundTransparency = 1 - (manager.opacity or 0.85)
                root:SetAttribute("WurstPanelSurface", true)
            end
            bind(root, "BackgroundColor3", "background")

            local window: any = nil
            local paintChrome: {() -> ()} = {}

            local chromeConnections: {RBXScriptConnection} = {}
            local header: Frame? = nil
            local closeButton: TextButton? = nil

            if not bare then

                local edge: UIStroke = create("UIStroke", {
                    Parent = root,
                    Name = "WindowOutline",
                    Color = Theme.outline,
                    Transparency = 0.08,
                    Thickness = 1,
                }) :: any) :: UIStroke
                bind(edge, "Color", "outline")

                local bar: Frame = create("Frame", {
                    Parent = root,
                    Name = "Header",
                    BackgroundTransparency = 0,
                    BorderSizePixel = 0,
                    Size = UDim2.new(1, 0, 0, headerHeight),
                    ZIndex = 2,
                }) :: Frame
                bind(bar, "BackgroundColor3", "accent")

                local bitmapText: any = state.bitmapText
                if bitmapText and type(bitmapText.draw) == "function" then
                    local function paintTitle(): ()
                        if root.Parent == nil then
                            return
                        end

                        bitmapText.draw(bar, {
                            {text = definition.title or id, color = Theme.accentText},
                        }, {
                            Name = "Title",
                            TextSize = TITLE_FONT_SIZE,
                            ZIndex = 3,
                            AnchorPoint = Vector2.new(0, 0.5),
                            Position = UDim2.new(0, 8, 0.5, 0),
                            maxWidth = math.max(40, root.Size.X.Offset - 68),
                        })
                    end
                    paintTitle()
                    if ThemeEngine and type(ThemeEngine.OnChange) == "function" then
                        ThemeEngine.OnChange(paintTitle, {"accentText"})
                    end

                    pcall(function(): ()
                        table.insert(
                            chromeConnections,
                            root:GetPropertyChangedSignal("Size"):Connect(paintTitle)
                        )
                    end)
                else
                    local title: TextLabel = (create("TextLabel", {
                        TextTruncate = Enum.TextTruncate.AtEnd,
                        Parent = bar,
                        Name = "Title",
                        BackgroundTransparency = 1,
                        FontFace = CONTROL_FONT,
                        Text = definition.title or id,
                        TextSize = TITLE_FONT_SIZE,
                    }) :: any) :: TextLabel
                    title.AnchorPoint = Vector2.new(0, 0.5)
                    title.Position = UDim2.new(0, 8, 0.5, 0)
                    title.Size = UDim2.new(1, -60, 1, 0)
                    title.TextXAlignment = Enum.TextXAlignment.Left
                    title.ZIndex = 3
                    bind(title, "TextColor3", "accentText")
                end

                local buttonX: number = -5

                local function chromeButton(kind: string, paint: Color3, token: string?): TextButton
                    local button: TextButton = (create("TextButton", {
                        TextTruncate = Enum.TextTruncate.AtEnd,
                        Parent = bar,
                        Active = true,
                        AutoButtonColor = false,
                        -- v191: a name per kind ("pin", "up", "cross") so the
                        -- offline suite can find the exact chip it asserts on.
                        Name = kind,
                        BackgroundColor3 = Theme.surface,
                        BackgroundTransparency = 0.72,
                        BorderSizePixel = 0,
                        FontFace = CONTROL_FONT,
                        Text = "",
                        TextSize = 11,
                    }) :: any) :: TextButton
                    local stroke = create("UIStroke", {
                        Parent = button,
                        Color = Theme.outline,
                        Thickness = 1,
                        Transparency = 0.55,
                    })
                    bind(button, "BackgroundColor3", "surface")
                    bind(stroke, "Color", "outline")
                    button.AnchorPoint = Vector2.new(1, 0.5)
                    button.Position = UDim2.new(1, buttonX, 0.5, 0)
                    button.Size = UDim2.fromOffset(22, 22)
                    button.ZIndex = 3
                    drawPixelIcon(button, kind, paint, token)

                    table.insert(chromeConnections, button.MouseEnter:Connect(function(): ()
                        button.BackgroundTransparency = 0.45
                    end))
                    table.insert(chromeConnections, button.MouseLeave:Connect(function(): ()
                        button.BackgroundTransparency = 0.72
                    end))
                    buttonX -= 25
                    return button
                end

                if definition.closable ~= false then
                    closeButton = chromeButton("cross", Theme.negative, "negative")
                end
                if definition.pinnable ~= false then

                    -- v191: the pin chip is themed by the SAME token the expand
                    -- chevrons use ("arrows", labelled "Buttons" in Settings), so
                    -- one palette row colours every small window button. Pinned is
                    -- shown by an arrows-coloured tint behind the icon instead of
                    -- by overwriting the button's own theme-bound background.
                    local pin: TextButton = chromeButton("pin", Theme.arrows, "arrows")
                    local pinTint: Frame = (create("Frame", {
                        Parent = pin,
                        Name = "PinTint",
                        BackgroundColor3 = Theme.arrows,
                        BackgroundTransparency = 1,
                        BorderSizePixel = 0,
                        Size = UDim2.fromScale(1, 1),
                        ZIndex = pin.ZIndex,
                    }) :: any) :: Frame
                    bind(pinTint, "BackgroundColor3", "arrows")
                    local function paintPin(): ()
                        pin.BackgroundTransparency = window.pinned and 0.35 or 0.72
                        pinTint.BackgroundTransparency = window.pinned and 0.55 or 1
                    end
                    table.insert(paintChrome, paintPin)
                    table.insert(chromeConnections, pin.MouseButton1Click:Connect(function(): ()
                        window:SetPinned(not window.pinned)
                        paintPin()
                    end))
                end
                if definition.collapsible ~= false then

                    local chevron: TextButton =
                        chromeButton("up", Theme.arrows, "arrows")
                    local function paintChevron(): ()
                        drawPixelIcon(
                            chevron,
                            window.collapsed and "down" or "up",
                            Theme.arrows,
                            "arrows"
                        )
                    end
                    table.insert(chromeConnections, chevron.MouseButton1Click:Connect(function(): ()
                        window:SetCollapsed(not window.collapsed)
                        paintChevron()
                    end))
                    -- Right-click anywhere on the header bar also collapses and
                    -- expands the window, alongside pressing the arrow itself.
                    table.insert(chromeConnections, bar.InputBegan:Connect(function(input: InputObject): ()
                        if input.UserInputType == Enum.UserInputType.MouseButton2 then
                            window:SetCollapsed(not window.collapsed)
                            paintChevron()
                        end
                    end))
                    table.insert(paintChrome, paintChevron)
                end
                header = bar
            end

            local body: ScrollingFrame = create("ScrollingFrame", {
                Parent = root,
                Name = "Body",
                Active = true,
                AutomaticCanvasSize = Enum.AutomaticSize.Y,
                BackgroundTransparency = 1,
                BorderSizePixel = 0,
                CanvasSize = UDim2.new(),
                Position = UDim2.fromOffset(0, headerHeight),
                ScrollBarThickness = 3,
                ScrollingDirection = Enum.ScrollingDirection.Y,
                Size = UDim2.new(1, 0, 1, -headerHeight),
                ZIndex = 2,
            }) :: ScrollingFrame
            bind(body, "ScrollBarImageColor3", "accentDim")

            local adopted: any = manager.Adopt(root, {
                id = id,
                body = body,
                header = header,
                closeButton = closeButton,
                headerHeight = headerHeight,
                dragHandle = bare and root or header,
                requiresMenuOpen = bare,
                onClose = definition.onClose,
            })
            adopted.fullHeight = definition.size.Y
            for _, connection: RBXScriptConnection in ipairs(chromeConnections) do
                table.insert(adopted.connections, connection)
                trackUiConnection(connection)
            end
            window = adopted
            for _, paint: () -> () in ipairs(paintChrome) do
                paint()
            end
            -- Palette changes must repaint the chrome (chevrons, pin) instantly.
            -- Before this, the arrow icons kept the colour they were created
            -- with until they were clicked ("tengo que tocarlas para que puedan
            -- cambiarse de color").
            if state and type(state.registerThemeRefresh) == "function" then
                state.registerThemeRefresh(function(): ()
                    if root.Parent == nil then
                        return
                    end
                    for _, paint: () -> () in ipairs(paintChrome) do
                        paint()
                    end
                end, {"arrows", "enabled", "surface"})
            end

            if window.collapsed then
                window:SetCollapsed(true)
            end

            manager.byId[id] = window
            raise(window)
            return window
        end

        function manager.Get(id: string): any
            return manager.byId[id]
        end

        function manager.SetMenuVisible(visible: boolean): ()
            for _, window: any in ipairs(manager.list) do
                if not visible and window.pinned then
                    continue
                end
                window.root.Visible = visible and window.wasVisible == true
                if visible == false then
                    window.wasVisible = window.root.Visible
                end
            end
        end

        local function intersects(
            ax: number, ay: number, aw: number, ah: number,
            bx: number, by: number, bw: number, bh: number
        ): boolean
            return ax < bx + bw and bx < ax + aw and ay < by + bh and by < ay + ah
        end

        local function obstacleRects(excludeWindow: any): {{number}}
            local rects: {{number}} = {}
            for _, other: any in ipairs(manager.list) do
                local root: any = other.root
                if other ~= excludeWindow and root and root.Visible ~= false then
                    local size: Vector2 = manager.MeasureWindow(other)
                    table.insert(rects, {
                        root.Position.X.Offset,
                        root.Position.Y.Offset,
                        size.X,
                        size.Y,
                    })
                end
            end
            return rects
        end

        local function rectIsFree(
            x: number, y: number, width: number, height: number,
            obstacles: {{number}}
        ): boolean
            local screen: Vector2 = logicalViewport()
            if x < MARGIN or y < MARGIN
                or x + width > screen.X - MARGIN
                or y + height > screen.Y - MARGIN then
                return false
            end
            for _, rect: {number} in ipairs(obstacles) do
                if intersects(
                    x, y, width, height,
                    rect[1] - PLACEMENT_GAP, rect[2] - PLACEMENT_GAP,
                    rect[3] + PLACEMENT_GAP * 2, rect[4] + PLACEMENT_GAP * 2
                ) then
                    return false
                end
            end
            return true
        end

        local function anchorRect(preferredAnchor: any): {number}?
            if type(preferredAnchor) ~= "table" then
                return nil
            end
            local root: any = preferredAnchor.root
            if root then
                if root.Parent == nil then
                    return nil
                end
                local size: Vector2 = manager.MeasureWindow(preferredAnchor)
                return {root.Position.X.Offset, root.Position.Y.Offset, size.X, size.Y}
            end
            local x: number? = tonumber(preferredAnchor.x)
            local y: number? = tonumber(preferredAnchor.y)
            if x and y then
                return {
                    x, y,
                    tonumber(preferredAnchor.width) or 0,
                    tonumber(preferredAnchor.height) or 0,
                }
            end
            return nil
        end

        function manager.FindFreeRect(
            desiredSize: Vector2,
            preferredAnchor: any?,
            excludeWindow: any?
```

```
): Vector2
    local width: number = desiredSize.X
    local height: number = desiredSize.Y
    local obstacles: {{number}} = obstacleRects(excludeWindow)
    local screen: Vector2 = logicalViewport()

    local function fitY(y: number): number
        return math.max(MARGIN, math.min(y, screen.Y - height - MARGIN))
    end

    local anchor: {number}? = anchorRect(preferredAnchor)
    if anchor then
        local candidates: {{number}} = {
            {anchor[1] + anchor[3] + PLACEMENT_GAP, fitY(anchor[2])},
            {anchor[1] - width - PLACEMENT_GAP, fitY(anchor[2])},
            {anchor[1], anchor[2] + anchor[4] + PLACEMENT_GAP},
        }
        for _, candidate: {number} in ipairs(candidates) do
            if rectIsFree(candidate[1], candidate[2], width, height, obstacles) then
                return Vector2.new(candidate[1], candidate[2])
            end
        end
    end

    local step: number = 40
    local yLimit: number = math.max(64, screen.Y - height - MARGIN)
    local xLimit: number = math.max(MARGIN, screen.X - width - MARGIN)
    local y: number = 64
    while y <= yLimit do
        local x: number = MARGIN
        while x <= xLimit do
            if rectIsFree(x, y, width, height, obstacles) then
                return Vector2.new(x, y)
            end
            x += step
        end
        y += step
    end
    return clamp(Vector2.new(MARGIN, 64), desiredSize)
end

function manager.PlaceWindow(window: any, options: any?): ()
    if not window or not window.root then
        return
    end
    local opts: any = options or {}
    if opts.anchor ~= nil then
        window.placementAnchor = opts.anchor
    end
    if window.userMoved and opts.force ~= true then
        manager.ClampWindow(window)
        return
    end
    local size: Vector2 = manager.MeasureWindow(window)
    local landed: Vector2 = manager.FindFreeRect(
        size,
        window.placementAnchor,
        window
    )
    window.root.Position = UDim2.fromOffset(landed.X, landed.Y)
    manager.ClampWindow(window)
    manager.SetHome(window, Vector2.new(
        window.root.Position.X.Offset,
        window.root.Position.Y.Offset
    ))
end

function manager.ReflowWindow(window: any): ()
    if not window or not window.root then
        return
    end
    if window.userMoved then
        manager.ClampWindow(window)
        return
    end
    local size: Vector2 = manager.MeasureWindow(window)
    local x: number = window.root.Position.X.Offset
    local y: number = window.root.Position.Y.Offset
    if rectIsFree(x, y, size.X, size.Y, obstacleRects(window)) then
        return
    end
    local landed: Vector2 = manager.FindFreeRect(
        size,
        window.placementAnchor,
        window
    )
    window.root.Position = UDim2.fromOffset(landed.X, landed.Y)
    manager.ClampWindow(window)
    manager.SetHome(window, Vector2.new(
        window.root.Position.X.Offset,
        window.root.Position.Y.Offset
    ))
end

local function categoryAnchor(feature: any): any
    local row: any = feature and feature.row
    local rowRoot: any = row and row.Parent and (row.Parent :: any).Parent
    if rowRoot then
        for _, entry: any in ipairs(manager.list) do
            if entry.root == rowRoot then
                return entry
            end
        end
    end
    local gui: any = state.clickGui
    if row and gui and type(gui.categories) == "table"
        and type(row.GetAttribute) == "function" then
        local raw: any = row:GetAttribute("FeatureCategory")
        local name: any = raw
        if type(host.normalizeFeatureCategory) == "function" then
            name = host.normalizeFeatureCategory(raw)
        end
        local record: any = name and gui.categories[name]
        if record and record.window then
            return record.window
        end
    end
    return nil
end

function manager.OpenFeatureSettings(feature: any): any
    if type(feature) ~= "table" or feature.row == nil then
        return nil
    end
    local id: string = "FeatureSettings_"
        .. tostring(feature.configKey or feature.name):gsub("%W", "")
    local existing: any = manager.byId[id]
    if existing then
        existing.root.Visible = true

        if existing.stalePlacement then
            existing.stalePlacement = nil
            manager.PlaceWindow(existing, {force = true})
        end
        feature.expanded = true
        if feature.syncArrow then
            feature.syncArrow()
        end
        raise(existing)
        return existing
    end

    local bitmapText: any = state.bitmapText
    local width: number = 200
    for _, optionRow: any in ipairs(feature.optionRows or {}) do
        local label: any = optionRow:FindFirstChild("OptionLabel")
        local text: string = label and tostring(label.Text) or ""
        local labelWidth: number
        if bitmapText and type(bitmapText.measure) == "function" then
            labelWidth = bitmapText.measure({{text = text}}, 16)
        else
            labelWidth = #text * 12
        end
        width = math.max(width, 8 + labelWidth + 24 + 72)
    end
    width = math.min(width, 420)

    local expectedLimit: number = tonumber(state.uiMaxSettingsHeight) or 200
    if expectedLimit <= 0 then
        expectedLimit = 600
    end
    local expectedHeight: number = math.min(
        TITLE_HEIGHT + (feature.optionsHeight or 40),
        expectedLimit
    )
    local anchor: any = categoryAnchor(feature)
    local slot: Vector2 = manager.FindFreeRect(
        Vector2.new(width, expectedHeight),
        anchor
    )

    local window: any = manager.Create({
        id = id,
        title = tostring(feature.name) .. " Settings",
        size = Vector2.new(width, TITLE_HEIGHT + 40),
        position = slot,
        closable = true,

        collapsible = false,
        onClose = function(): ()
            feature.expanded = false
            if feature.syncArrow then
                feature.syncArrow()
            end
        end,
    })
    window.featureSettings = true
    window.placementAnchor = anchor
    feature.settingsWindow = window

    local options: any = feature.options
    if feature.updateExpandedSize then
        feature.updateExpandedSize(false)
    end
    if options then
        options.Visible = true
        options.Position = UDim2.fromOffset(0, 0)
        options.Size = UDim2.new(1, 0, 0, feature.optionsHeight or 40)
        options.Parent = window.body
        if bitmapText and type(bitmapText.adoptTree) == "function" then
            table.insert(
                window.connections,
                bitmapText.adoptTree(options)
            )
        end
    end

    local function resizeToOptions(): ()
        if options then
            options.Size = UDim2.new(1, 0, 0, feature.optionsHeight or 40)
        end
        local wanted: number = TITLE_HEIGHT + (feature.optionsHeight or 40)
        local limit: number = tonumber(state.uiMaxSettingsHeight) or 200
        if limit <= 0 then
            limit = math.huge
        end
        wanted = math.min(wanted, limit)

        wanted = math.min(wanted, logicalViewport().Y - MARGIN * 2)
        window.fullHeight = wanted
        if not window.collapsed then
            window.root.Size = UDim2.fromOffset(width, wanted)
        end

        manager.ReflowWindow(window)
    end
    window.resizeToOptions = resizeToOptions
    resizeToOptions()

    local function repackWidth(): ()
        if not window.root.Parent then
            return
        end
        local bitmapText: any = state.bitmapText
        local widest: number = 0
        for _, optionRow: any in ipairs(feature.optionRows or {}) do
            local label: any = optionRow:FindFirstChild("OptionLabel")
            if label then
                local measured: number = 0
                if bitmapText and type(bitmapText.measure) == "function" then
                    measured = tonumber(bitmapText.measure(
                        {{text = label.Text}},
                        tonumber(label.TextSize) or 16
                    )) or 0
                end
                if measured <= 0 then
                    local bounds: any = label.TextBounds
                    measured = bounds and tonumber(bounds.X) or 0
                end
                if measured > 0 then
                    widest = math.max(widest, 8 + measured + 24 + 72)
                end
            end
        end
        if widest > 0 then
            local packed: number = math.clamp(widest, 160, 420)
            if packed ~= width then
                width = packed
                resizeToOptions()
            end
        end
    end
    window.repackWidth = repackWidth
    task.defer(repackWidth)

    feature.expanded = true
    if feature.syncArrow then
        feature.syncArrow()
    end
    window.root.Visible = true
    raise(window)
    return window
end

function manager.BringToFront(window: any): ()
    if window then
        raise(window)
    end
end

function manager.DebugRects(): any
    local rects: {any} = {}
    for _, window: any in ipairs(manager.list) do
        local x: number, y: number, w: number, h: number = windowRect(window)
        table.insert(rects, {
            id = window.id,
            x = x, y = y, width = w, height = h,
            visible = window.root.Visible == true,
            userMoved = window.userMoved == true,
            collapsed = window.collapsed == true,
        })
    end
    return {
        scale = layerScale(),
        viewport = logicalViewport(),
        windows = rects,
    }
end

do
    local bitmapText: any = state.bitmapText
    if bitmapText and type(bitmapText.onReady) == "table" then
        table.insert(bitmapText.onReady, function(): ()
            for _, window: any in ipairs(manager.list) do
                if window.featureSettings and window.repackWidth then
                    window.repackWidth()
                end
            end
        end)
    end
end

state.addMenuVisibilityListener(function(visible: boolean): ()
    for _, window: any in ipairs(manager.list) do
        if window.featureSettings then
            if not visible and window.pinned then
                continue
            end
            if visible then
                window.root.Visible = window.wasOpenWithMenu == true
            else
                window.wasOpenWithMenu = window.root.Visible
                window.root.Visible = false
            end
        end
    end
end)

manager.opacity = 0.85
-- Kept as a public value for callers which display the selected alpha.
manager.bodyTransparency = 1 - manager.opacity
-- v191: Opacity means "how see-through the BACKGROUND is", not "fade the
-- whole window". v190 and earlier drove root.GroupTransparency, which
-- alpha-blended the panel together with its title, labels and icons: the
-- lower the slider, the harder the text was to read. The panel surface is
-- now the only thing that gets transparent (root.BackgroundTransparency),
-- so text and controls stay fully opaque at every setting.
function manager.SetOpacity(alpha: number): ()
    local wanted: number = tonumber(alpha) or 0.85
    if wanted ~= wanted then wanted = 0.85 end
    wanted = math.clamp(wanted, 0.15, 1)
    manager.opacity = wanted
    manager.bodyTransparency = 1 - wanted
    for _, window: any in ipairs(manager.list) do
        local root: any = window.root
        -- Not every record in this list is a full GuiObject (a bare
        -- overlay may be a plain table while it is being built), so the
        -- surface test is defensive: never assume the methods exist.
        if root and type(root.IsA) == "function" then
            if root:IsA("CanvasGroup") then
                -- Content is never faded: the group stays fully opaque.
                root.GroupTransparency = 0
            end
            local surface: boolean = type(root.GetAttribute) == "function"
                and root:GetAttribute("WurstPanelSurface") == true
            if surface then
                root.BackgroundTransparency = 1 - wanted
            end
        end
    end
end

function manager.Reclamp(): ()
    -- After landing, windows that ended on (nearly) the same spot
    -- cascade a step down-right each, so a shrink never leaves a
    -- pile of identical title bars in one corner.
    local placed: {Vector2} = {}
    for _, window: any in ipairs(manager.list) do
        local x: number, y: number, w: number, h: number = windowRect(window)
        local home: Vector2? = homePositions[window.id]
        local base: Vector2 = home or Vector2.new(x, y)
        local landed: Vector2 = clamp(base, Vector2.new(w, h))
        local attempts: number = 0
        while attempts < 8 do
            local stacked: boolean = false
            for _, other: Vector2 in ipairs(placed) do
                if math.abs(other.X - landed.X) < 6
                    and math.abs(other.Y - landed.Y) < 6 then
                    stacked = true
                    break
                end
            end
            if not stacked then
                break
            end
            attempts += 1
            landed = clamp(
                landed + Vector2.new(26, 26),
                Vector2.new(w, h)
            )
        end
        table.insert(placed, landed)
        window.root.Position = UDim2.fromOffset(landed.X, landed.Y)
    end
end

state.onLayout(function(): ()
    manager.Reclamp()
end)

state.windows = manager

activeCleanup = function(): ()
    for index: number = #manager.list, 1, -1 do
        local window: any = manager.list[index]
        pcall(function(): ()
            window.root:Destroy()
        end)
    end
    manager.list = {}
    manager.byId = {}
    state.windows = nil
    end
    Module.Initialized = true
    return manager
end

function Module.destroy(): ()
    if activeCleanup then
        pcall(activeCleanup)
    end
    activeCleanup = nil
    Module.Initialized = false
end

return Module

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/guis/Wurst/Code/WindowManager.lua
--------------------------------------------------------------------------------


================================================================================
CARGADOR Y RUNTIME
================================================================================

--------------------------------------------------------------------------------
ARCHIVO: src/Wurst.lua
BYTES: 259370
SHA-256: cd6b97cb48bab9218acba3990d65b36cd2f782c06ec7136e27b5ca23bf5e1d7f
INICIO DEL ARCHIVO
--------------------------------------------------------------------------------
local RUNTIME_COMPATIBILITY_MARKER: string =
    "Initialization error — check executor console"
-- v196: `src/Wurst.lua` IS the loader. A player pastes its raw URL into the
-- executor, so the file a re-inject downloads has to be this same branch.
-- This constant used to hardcode an older session branch, which meant the
-- Reinject button in Settings could pull a different (older) menu than the one
-- already running. It is no longer a second address: RUNTIME_SAFETY_SOURCE_URL
-- below is built from RUNTIME_BRANCH, exactly like RUNTIME_RAW_BASE.

local DEFAULT_FEATURE_CATEGORY: string = "Utilities"

-- v138.9: the window order, the aliases and the card -> window map are read from
-- Manifest.lua ("categories" and "cards") right after the catalog is fetched.
-- These are only the loader's last-resort defaults, not a second catalog.
local OFFICIAL_CATEGORIES: {string} = {
    "Exploits",
    "Utilities",
    "World",
    "Render",
    "Blocks",
    "Items",
}

local CATEGORY_ALIASES: {[string]: string} = {}

local FEATURE_CATEGORIES: {[string]: string} = {}

local UI_RADIUS: {[string]: number} = {
    shell = 8,
    panel = 5,
    card = 4,
    control = 3,
    chip = 3,
}

local UI_MOTION: {[string]: number} = {
    fast = 0.1,
    base = 0.16,
    slow = 0.22,
}

type LayoutMetrics = {
    mode: string,
    compact: boolean,
    touch: boolean,
    contentPadding: number,
    cardSpacing: number,
    cardHeight: number,
    cardCorner: number,
    controlHeight: number,
    optionRowHeight: number,
    sectionRowHeight: number,
    switchWidth: number,
    switchHeight: number,
    iconButton: number,
    keySlotWidth: number,
    touchTarget: number,
    titleFontSize: number,
    bodyFontSize: number,
    captionFontSize: number,
    scrollBar: number,
}

local function computeLayoutMetrics(
    _viewport: Vector2,
    touchEnabled: boolean
): LayoutMetrics
    local mode: string = "Desktop"
    if touchEnabled then
        mode = "Mobile"
    end
    local mobile: boolean = mode == "Mobile"
    return {
        mode = mode,
        compact = mobile,
        touch = touchEnabled,
        contentPadding = 10,
        cardSpacing = 2,
        cardHeight = 22,
        cardCorner = UI_RADIUS.card,
        controlHeight = 20,
        optionRowHeight = 22,
        sectionRowHeight = 22,
        switchWidth = 26,
        switchHeight = 14,
        iconButton = 16,
        keySlotWidth = 34,
        touchTarget = touchEnabled and 32 or 24,
        titleFontSize = 16,
        bodyFontSize = 16,
        captionFontSize = 12,
        scrollBar = 3,
    }
end

-- v195: the branch this session publishes to. It used to point at an older
-- session branch, and the loader in the README pointed at yet another one: a
-- player following the docs asked GitHub for a branch that had never been
-- pushed, got "404: Not Found" back, and `loadstring` turned that into
-- `:2: attempt to call a nil value` with no cause. One branch, one default.
local RUNTIME_BRANCH: string = "arena/0ce50d99-arandommenu"
do
    local override: any = nil
    local okGenv, genv = pcall(function()
        return (getgenv :: any)()
    end)
    if okGenv and type(genv) == "table" and type(genv.WURST_BRANCH) == "string" and genv.WURST_BRANCH ~= "" then
        override = genv.WURST_BRANCH
    elseif type(_G) == "table" and type((_G :: any).WURST_BRANCH) == "string" and (_G :: any).WURST_BRANCH ~= "" then
        override = (_G :: any).WURST_BRANCH
    else
        local fenv: any = (getfenv() :: any)
        if type(fenv) == "table" and type(fenv.WURST_BRANCH) == "string" and fenv.WURST_BRANCH ~= "" then
            override = fenv.WURST_BRANCH
        end
    end
    if type(override) == "string" and override ~= "" then
        RUNTIME_BRANCH = (override:gsub("^refs/heads/", ""))
    end
end
local RUNTIME_RAW_BASE: string =
    "https://raw.githubusercontent.com/XzAngel19/ARandomMenu/refs/heads/"
        .. RUNTIME_BRANCH
        .. "/"
local RUNTIME_SAFETY_SOURCE_URL: string = RUNTIME_RAW_BASE .. "src/Wurst.lua"

-- v192: the build number shown in the menu has ONE source, this stamp. "v193"
-- renders as "0.192" in the window titles and as "0.192 Release" next to the
-- product name, so bumping the stamp moves every label at once -- the player
-- asked for "0.191" (no "Version" word) tied to the product line that used to
-- carry a different number, and for the two to never drift apart again.
local SOURCE_STAMP: string = "v199-20261108-aim-camera-fire"

local BUILD_NUMBER: string? = string.match(SOURCE_STAMP, "^v(%d+)")
local BUILD_LABEL: string = if BUILD_NUMBER then "0." .. BUILD_NUMBER else "0.0"

local BOOT_STARTED_AT: number = os.clock()
local BOOT_FINGERPRINT_SECONDS: number = 0
local PENDING_FINGERPRINT: boolean = false

local PRODUCT = {
    name = "Wurst",
    -- Derived, never hardcoded: "0.192 Release" for the stamp above.
    version = BUILD_LABEL .. " Release",
    logPrefix = "Wurst",
    guiName = "Wurst",
    blurName = "WurstBlur",
    storageFolder = "Wurst",
}

-- v195: the destruct runtime used to sit at the end of `bootstrap()`. Luau allows
-- 200 live locals per function and the main chunk is one function, so the five
-- locals this block declares (`runtimeCleaned`, `runtimeDestructed`,
-- `runtimeHandoff`, `cleanupStep`, `cleanupRuntime`) landed on top of the 196
-- already open and pushed the frame to 201. The player's executor then refused
-- to compile the file at all:
--     Out of local registers when trying to allocate cleanupRuntime
-- They now live in their own function frame. Everything they read from
-- `bootstrap()` arrives as a parameter, the returned closures are handed back
-- to `bootstrap()` which stores them on `state`, and nothing about the order of
-- cleanup, the flags or the genv handshake changed.
local function createRuntimeDestructor(deps: any): any
    local state = deps.state
    local MenuBlur = deps.MenuBlur
    local ScreenGui = deps.ScreenGui
    local allFeatures = deps.allFeatures
    local featureConnections = deps.featureConnections
    local TaskManager = deps.TaskManager
    local cleanupMM2Runtime = deps.cleanupMM2Runtime
    local closeConfigSave = deps.closeConfigSave
    local setMenuCursorUnlocked = deps.setMenuCursorUnlocked
    local disconnectFeatureConnection = deps.disconnectFeatureConnection
    local persistenceGenv = deps.persistenceGenv
    local logPrint: any = deps.logPrint
    local reportError: any = deps.warn

    local runtimeCleaned: boolean = false
    local runtimeDestructed: boolean = false
    local runtimeHandoff: any = nil

    local function cleanupStep(label: string, step: () -> ()): ()
        local ok: boolean, failure: any = pcall(step)
        if not ok then
            reportError("[" .. PRODUCT.name .. "] destruct/" .. label .. ": " .. tostring(failure))
        end
    end

    local function cleanupRuntime(): ()
        if runtimeCleaned then
            return
        end
        runtimeCleaned = true
    cleanupStep("teleport persistence", function(): ()
        if state.teleportPersist then
            state.teleportPersist.Close()
        end
    end)
    cleanupStep("blur", function(): ()
        if MenuBlur then
            MenuBlur:Destroy()
            MenuBlur = nil
        end
    end)
    cleanupStep("cursor", function(): ()
        setMenuCursorUnlocked(false)
    end)
    cleanupStep("session", function(): ()
        if type(state.destroySessionTracking) == "function" then
            state.destroySessionTracking()
        end
        state.sessionInfo = nil
    end)

    for _, feature in ipairs(allFeatures) do
        if feature.enabled then
            cleanupStep("feature " .. tostring(feature.name), function(): ()
                feature.onToggle(false)
            end)
            feature.enabled = false
        end
    end
```

```
    cleanupStep("held keybinds", function(): ()
        for _, binding: any in pairs(state.shortcutBindings) do
            if binding.hold and binding.pressed and binding.onRelease then
                binding.pressed = false
                binding.onRelease()
            end
        end
        table.clear(state.shortcutBindings)
    end)
    cleanupStep("feature connections", function(): ()
        for name in pairs(featureConnections) do
            disconnectFeatureConnection(name)
        end
    end)
    cleanupStep("tooltips", function(): ()
        for _, connection: RBXScriptConnection in ipairs(state.tooltipConnections) do
            connection:Disconnect()
        end
        table.clear(state.tooltipConnections)
    end)
    cleanupStep("ui connections", function(): ()
        for _, connection: RBXScriptConnection in ipairs(state.uiConnections) do
            connection:Disconnect()
        end
        table.clear(state.uiConnections)
    end)
    cleanupStep("projectile calibration", function(): ()
        local services: any = state.frameworkEnvironment
            and state.frameworkEnvironment.services
        local registry: any = services and services.projectileCalibration
        local calibration: any = registry and registry.get()
        if calibration and type(calibration.destroy) == "function" then
            calibration:destroy()
        end
        if registry then
            registry.set(nil)
        end
    end)
    cleanupStep("task manager", function(): ()
        TaskManager:Destroy()
    end)
    cleanupStep("global input", function(): ()
        for _, key: string in ipairs({
            "globalInputChangedConnection",
            "globalInputEndedConnection",
            "globalInputBeganConnection",
            "globalShortcutBeganConnection",
            "globalShortcutEndedConnection",
        }) do
            local connection: any = state[key]
            if connection then
                connection:Disconnect()
                state[key] = nil
            end
        end
    end)
    cleanupStep("colour picker", function(): ()

        local picker: any = state.colorPicker
        if not picker then
            return
        end
        if picker.inputChanged then
            picker.inputChanged:Disconnect()
        end
        if picker.inputEnded then
            picker.inputEnded:Disconnect()
        end
    end)
    cleanupStep("MM2 runtime", cleanupMM2Runtime)
    for _, key: string in ipairs({
        "cleanupFrameworkRuntime",
        "cleanupTRSRuntime",
        "cleanupVDRuntime",
        "cleanupBedFightRuntime",
        "cleanupX2DuelsRuntime",
    }) do
        local cleanup: any = state[key]
        if type(cleanup) == "function" then
            cleanupStep(key, cleanup)
        end
    end
    cleanupStep("config", closeConfigSave)
    local genv: any = persistenceGenv()
    if genv and genv.WURST_RUNTIME == runtimeHandoff then
        genv.WURST_RUNTIME = nil
    end
end


    local function destructRuntime(): ()
        if runtimeDestructed then
            return
        end
        runtimeDestructed = true
        -- Revoke before any other cleanup can yield. Stop is still useful after
        -- ordinary GUI teardown has preserved a queued teleport.
        cleanupStep("session kill switch", function(): ()
            if state.teleportPersist then
                state.teleportPersist.Stop()
            end
        end)
        if runtimeCleaned then
            return
        end
        cleanupStep("runtime", cleanupRuntime)
        cleanupStep("screen gui", function(): ()
            if ScreenGui.Parent then
                ScreenGui:Destroy()
            end
        end)

        logPrint("[" .. PRODUCT.name .. "] Destruct complete.")
    end

    local function bindHandoff(): ()
        runtimeHandoff = {Destruct = destructRuntime}
        local activeGenv: any = persistenceGenv()
        if activeGenv then
            activeGenv.WURST_RUNTIME = runtimeHandoff
        end
        ScreenGui.AncestryChanged:Connect(function(_: Instance, parent: Instance?): ()
            if not parent then
                cleanupRuntime()
            end
        end)
    end

    return {
        cleanupRuntime = cleanupRuntime,
        destructRuntime = destructRuntime,
        bindHandoff = bindHandoff,
    }
end

local function bootstrap(): ()
local executorGlobals: any = getfenv()

local DEBUG_LOGS: boolean = (function(): boolean
    local getGenv: any = executorGlobals.getgenv
    if type(getGenv) == "function" then
        local ok: boolean, genv: any = pcall(getGenv)
        if ok and type(genv) == "table" then
            return genv.WURST_DEBUG == true
        end
    end
    return false
end)()
local realPrint: any = executorGlobals.print
local realWarn: any = executorGlobals.warn
local print = function(...: any): ()
    if DEBUG_LOGS then
        realPrint(...)
    end
end
local warn = function(...: any): ()
    if DEBUG_LOGS then
        realWarn(...)
    end
end

local function cloneReference(instance: any): any
    if type(executorGlobals.cloneref) == "function" then
        local success: boolean, cloned: any = pcall(executorGlobals.cloneref, instance)
        if success then
            return cloned
        end
    end
    return instance
end

local Players: Players = cloneReference(game:GetService("Players"))
local UserInputService: UserInputService = cloneReference(game:GetService("UserInputService"))
local TweenService: TweenService = cloneReference(game:GetService("TweenService"))
local Lighting: Lighting = cloneReference(game:GetService("Lighting"))
local RunService: RunService = cloneReference(game:GetService("RunService"))
local HttpService: HttpService = cloneReference(game:GetService("HttpService"))
local CollectionService: CollectionService = cloneReference(game:GetService("CollectionService"))
local Stats: Stats = cloneReference(game:GetService("Stats"))
local LocalPlayer: Player = (function(): Player

    local player: any = Players.LocalPlayer
    while player == nil do
        task.wait(0.1)
        player = Players.LocalPlayer
    end
    return player :: Player
end)()
local PlayerGui: PlayerGui = LocalPlayer:WaitForChild("PlayerGui") :: PlayerGui

local GUI_NAME = PRODUCT.guiName
local BLUR_NAME = PRODUCT.blurName
local CONFIG_FOLDER = PRODUCT.storageFolder
local CONFIG_PROFILE_FOLDER = CONFIG_FOLDER .. "/Profiles"
local CONFIG_GAME_ID: string = tostring(game.GameId > 0 and game.GameId or game.PlaceId)
local CONFIG_FILE_LUA = CONFIG_PROFILE_FOLDER .. "/Game_" .. CONFIG_GAME_ID .. ".lua"
local CONFIG_FILE_BAK = CONFIG_FILE_LUA .. ".bak"
local CONFIG_FILE = CONFIG_PROFILE_FOLDER
    .. "/Game_"
    .. CONFIG_GAME_ID
    .. ".Config"
local REPOSITORY_RAW_BASE = RUNTIME_RAW_BASE

local GAME_MODULES: {[number]: string} = {
    [71480482338212] = "BedFight",
    [142823291] = "MM2",
    [143152583851 = "TRS",
    [93978595733734] = "VD",
    [92876937625630] = "X2Duels",
    [135856908115931] = "X2Duels",
}

type GameFingerprint = {
    module: string,
    required: number,
    markers: {(ReplicatedStorage) -> boolean},
}

local GAME_FINGERPRINTS: {GameFingerprint} = {
    {
        module = "MM2",
        required = 4,
        markers = {

            function(replicated: ReplicatedStorage): boolean
                local remotes: Instance? = replicated:FindFirstChild("Remotes")
                local gameplay: Instance? = remotes
                    and remotes:FindFirstChild("Gameplay")
                if not gameplay then
                    return false
                end
                local found: number = 0
                for _, name: string in ipairs({
                    "RoundStart",
                    "RoundEndFade",
                    "GetCurrentPlayerData",
                    "PlayerDataChanged",
                }) do
                    if gameplay:FindFirstChild(name) then
                        found += 1
                    end
                end
                return found >= 3
            end,

            function(replicated: ReplicatedStorage): boolean
                local services: Instance? =
                    replicated:FindFirstChild("ClientServices")
                local weaponService: Instance? = services
                    and services:FindFirstChild("WeaponService")
                return weaponService ~= nil
                    and weaponService:FindFirstChild("GunFired") ~= nil
            end,

            function(replicated: ReplicatedStorage): boolean
                local mainGui: Instance? = replicated:FindFirstChild("MainGUI")
                local gameFrame: Instance? = mainGui
                    and mainGui:FindFirstChild("Game")
                return gameFrame ~= nil
                    and gameFrame:FindFirstChild("Timer") ~= nil
            end,

            function(_replicated: ReplicatedStorage): boolean
                local ok: boolean, groups: any = pcall(function(): any
                    return game:GetService("PhysicsService")
                        :GetRegisteredCollisionGroups()
                end)
                if not ok or type(groups) ~= "table" then
                    return false
                end
                for _, group: any in ipairs(groups) do
                    if type(group) == "table" and group.name == "Murderer" then
                        return true
                    end
                end
                return false
            end,

            function(_replicated: ReplicatedStorage): boolean
                for _, object: Instance in ipairs(workspace:GetChildren()) do
                    if object:IsA("Model")
                        and object.Name ~= "Lobby"
                        and (object:FindFirstChild("CoinContainer")
                            or object:FindFirstChild("CoinAreas")) then
                        return true
                    end
                end
                return false
            end,
        },
    },
    {
        module = "X2Duels",
        required = 3,
        markers = {
            function(replicated: ReplicatedStorage): boolean
                local packages: Instance? = replicated:FindFirstChild("Packages")
                local networking: Instance? = packages
                    and packages:FindFirstChild("Networking")
                local re: Instance? = networking and networking:FindFirstChild("RE")
                local match: Instance? = re and re:FindFirstChild("Match")
                if not match then
                    return false
                end
                local found: number = 0
                for _, name: string in ipairs({
                    "MatchResults", "WinNotification", "PlayerRespawned", "SetDead",
                }) do
                    if match:FindFirstChild(name) then
                        found += 1
                    end
                end
                return found >= 2
            end,
            function(replicated: ReplicatedStorage): boolean
                local packages: Instance? = replicated:FindFirstChild("Packages")
                local networking: Instance? = packages
                    and packages:FindFirstChild("Networking")
                local re: Instance? = networking and networking:FindFirstChild("RE")
                local rematch: Instance? = re and re:FindFirstChild("Rematch")
                return rematch ~= nil
                    and rematch:FindFirstChild("RematchVote") ~= nil
                    and rematch:FindFirstChild("RematchTeleport") ~= nil
            end,
            function(replicated: ReplicatedStorage): boolean
                local packages: Instance? = replicated:FindFirstChild("Packages")
                local net: Instance? = packages and packages:FindFirstChild("Net")
                local re: Instance? = net and net:FindFirstChild("RE")
                if not re then
                    return false
                end
                local found: number = 0
                for _, name: string in ipairs({
                    "ThrowEvent", "SlashEvent", "GunKill", "KnifeKill",
                }) do
                    if re:FindFirstChild(name) then
                        found += 1
                    end
                end
                return found >= 3
            end,
            function(replicated: ReplicatedStorage): boolean
                local client: Instance? = replicated:FindFirstChild("Client")
                local controllers: Instance? = client
                    and client:FindFirstChild("Controllers")
                return controllers ~= nil
                    and controllers:FindFirstChild("MatchController") ~= nil
                    and controllers:FindFirstChild("GunCrosshairController") ~= nil
            end,
            -- Current MVS capture: Packages.Networking RE/Combat plus the
            -- legacy client controllers. These markers do not depend on the
            -- hashed knife remote names or on the older X2 rematch schema.
            function(replicated: ReplicatedStorage): boolean
                local packages: Instance? = replicated:FindFirstChild("Packages")
                local networking: Instance? = packages
                    and packages:FindFirstChild("Networking")
                local re: Instance? = networking and networking:FindFirstChild("RE")
                local combat: Instance? = re and re:FindFirstChild("Combat")
                return combat ~= nil
                    and combat:FindFirstChild("Beam") ~= nil
                    and combat:FindFirstChild("PlaySound") ~= nil
            end,
            function(replicated: ReplicatedStorage): boolean
                local client: Instance? = replicated:FindFirstChild("Client")
                local legacy: Instance? = client
                    and client:FindFirstChild("LegacyClientScripts")
                return legacy ~= nil
                    and legacy:FindFirstChild("BeamHandle") ~= nil
                    and client:FindFirstChild("ClientRuntime") ~= nil
            end,
        },
    },

}

local function detectGameModuleByStructure(): (string?, number)
    local replicated: ReplicatedStorage = game:GetService("ReplicatedStorage")
    for _, fingerprint: GameFingerprint in ipairs(GAME_FINGERPRINTS) do
        local score: number = 0
        for _, marker: (ReplicatedStorage) -> boolean in ipairs(fingerprint.markers) do
            local ok: boolean, matched: any = pcall(marker, replicated)
            if ok and matched == true then
                score += 1
            end
        end
        if score >= fingerprint.required then
            return fingerprint.module, score
        end
    end
    return nil, 0
end

local ACTIVE_GAME_MODULE: string? = GAME_MODULES[game.PlaceId]
local ACTIVE_GAME_SOURCE: string = ACTIVE_GAME_MODULE and "PlaceId" or "none"
PENDING_FINGERPRINT = ACTIVE_GAME_MODULE == nil
if not ACTIVE_GAME_MODULE then

    local fingerprintAt: number = os.clock()
    local detected: string?, score: number = detectGameModuleByStructure()
    BOOT_FINGERPRINT_SECONDS = os.clock() - fingerprintAt
    if detected then
        ACTIVE_GAME_MODULE = detected
        ACTIVE_GAME_SOURCE = "fingerprint " .. tostring(score) .. " markers"
        PENDING_FINGERPRINT = false
    end
end

local GAME_CHECK = {
    MM2 = 142823291,
    BedFight = 71480482338212,
    TRS = 143152583851,
    VD = 93978595733734,
    X2Duels = 92876937625630,
    MM2Active = ACTIVE_GAME_MODULE == "MM2",
    TRSActive = ACTIVE_GAME_MODULE == "TRS",
    VDActive = ACTIVE_GAME_MODULE == "VD",
    BedFightActive = ACTIVE_GAME_MODULE == "BedFight",
    X2DuelsActive = ACTIVE_GAME_MODULE == "X2Duels",
}

function GAME_CHECK.matches(identifier: number): boolean
    if game.PlaceId == identifier then
        return true
    end

    for placeId: number, moduleName: string in pairs(GAME_MODULES) do
        if placeId == identifier then
            return ACTIVE_GAME_MODULE == moduleName
        end
    end
    return false
end

GAME_CHECK.source = ACTIVE_GAME_SOURCE

export type ThemeShape = {
    opacity: number,
    tooltipOpacity: number,
    radius: number,
    rowHeight: number,
    borderThickness: number,
    maxHeight: number,
    scale: number,
}

local THEME_PRESETS: {[string]: {[string]: Color3}} = {

    Wurst = {
        background = Color3.fromRGB(64, 64, 64),
        sidebar = Color3.fromRGB(48, 48, 48),
        surface = Color3.fromRGB(64, 64, 64),
        surfaceRaised = Color3.fromRGB(74, 74, 74),
        surfaceHover = Color3.fromRGB(86, 86, 86),
        surfacePressed = Color3.fromRGB(96, 96, 96),
        button = Color3.fromRGB(56, 56, 56),
        buttonHover = Color3.fromRGB(72, 72, 72),
        buttonPressed = Color3.fromRGB(84, 84, 84),
        outline = Color3.fromRGB(16, 16, 16),
        accent = Color3.fromRGB(16, 16, 16),
        accentDim = Color3.fromRGB(36, 36, 36),
        accentFaint = Color3.fromRGB(56, 56, 56),
        accentSoft = Color3.fromRGB(40, 40, 40),
        accentText = Color3.fromRGB(240, 240, 240),

        enabled = Color3.fromRGB(32, 175, 32),
        enabledText = Color3.fromRGB(240, 240, 240),
        arrows = Color3.fromRGB(0, 204, 0),
        text = Color3.fromRGB(240, 240, 240),
        textMuted = Color3.fromRGB(168, 168, 168),
        tooltip = Color3.fromRGB(24, 24, 24),
    },

    Monochrome = {
        background = Color3.fromRGB(8, 8, 9),
        sidebar = Color3.fromRGB(13, 13, 14),
        surface = Color3.fromRGB(26, 26, 28),
        surfaceRaised = Color3.fromRGB(32, 32, 35),
        surfaceHover = Color3.fromRGB(40, 40, 44),
        surfacePressed = Color3.fromRGB(48, 48, 52),
        button = Color3.fromRGB(32, 32, 35),
        buttonHover = Color3.fromRGB(46, 46, 50),
        buttonPressed = Color3.fromRGB(56, 56, 61),
        outline = Color3.fromRGB(92, 92, 98),
        accent = Color3.fromRGB(236, 236, 238),
        accentDim = Color3.fromRGB(196, 196, 202),
        accentFaint = Color3.fromRGB(160, 160, 168),
        accentSoft = Color3.fromRGB(52, 52, 56),
        accentText = Color3.fromRGB(8, 8, 9),
        enabled = Color3.fromRGB(236, 236, 238),
        enabledText = Color3.fromRGB(8, 8, 9),
        arrows = Color3.fromRGB(0, 204, 0),
        text = Color3.fromRGB(245, 245, 247),
        textMuted = Color3.fromRGB(158, 158, 166),
        tooltip = Color3.fromRGB(16, 16, 18),
    },
}

local Theme: {[string]: Color3} = {}
for token: string, value: Color3 in pairs(THEME_PRESETS.Wurst) do
    Theme[token] = value
end

Theme.positive = Color3.fromRGB(126, 196, 138)
Theme.negative = Color3.fromRGB(214, 106, 106)

local ThemeEngine = {
    name = "Wurst",
    presets = THEME_PRESETS,
    tokens = Theme,
    shape = {
        opacity = 0.86,
        tooltipOpacity = 0.95,
        radius = 0,
        rowHeight = 22,
        borderThickness = 1,
        maxHeight = 340,
        scale = 1,
    } :: ThemeShape,
}

-- Strong, token-indexed bindings. Destroying releases entries immediately;
-- changing Accent never walks every text/arrow/pixel binding in the menu.
local themeBindings: any = {}
local themeBuckets: any = {}
local themeWatches: any = {}
local themeListeners: {any} = {}
local function unbindTheme(instance: any)
    local record = themeBindings[instance]
    if record then
        for _, token in pairs(record) do
            if themeBuckets[token] then themeBuckets[token][instance] = nil end
        end
    end
    themeBindings[instance] = nil
    local watch = themeWatches[instance]
    themeWatches[instance] = nil
    if watch then watch:Disconnect() end
end
local function repaintThemeBindings(values: {[string]: Color3}): ()
    for token, value in pairs(values) do
        for instance in pairs(themeBuckets[token] or {}) do
            local ok = pcall(function()
                if instance.Parent == nil then unbindTheme(instance); return end
                for property, boundToken in pairs(themeBindings[instance] or {}) do
                    if boundToken == token then instance[property] = value end
                end
            end)
            if not ok then unbindTheme(instance) end
        end
    end
end
function ThemeEngine.Bind(instance: Instance, property: string, token: string): Instance
    local record = themeBindings[instance]
    if not record then
        record = {}; themeBindings[instance] = record
        local ok, watch = pcall(function()
            return instance.Destroying:Connect(function() unbindTheme(instance) end)
        end)
        if ok then themeWatches[instance] = watch end
    end
    local old = record[property]
    record[property] = token
    if old and old ~= token and themeBuckets[old] then
        local stillUsed = false
        for _, bound in pairs(record) do if bound == old then stillUsed = true; break end end
        if not stillUsed then themeBuckets[old][instance] = nil end
    end
    themeBuckets[token] = themeBuckets[token] or {}
    themeBuckets[token][instance] = true
    if Theme[token] then (instance :: any)[property] = Theme[token] end
    return instance
end
function ThemeEngine.OnChange(callback: any, tokens: {string}?): ()
    table.insert(themeListeners, function(name, changed)
        if tokens and changed then
            local relevant = false
            for _, token in ipairs(tokens) do if changed[token] then relevant = true; break end end
            if not relevant then return end
        end
        callback(name, changed)
    end)
end
function ThemeEngine.SetMany(input: {[string]: Color3}): ()
    local values = table.clone(input)
    if values.accent ~= nil or values.background ~= nil then
        local accent = values.accent or Theme.accent
        local background = values.background or Theme.background
        for token, amount in pairs({accentDim = 0.72, accentSoft = 0.58, accentFaint = 0.42}) do
            values[token] = values[token] or Color3.new(background.R + (accent.R - background.R) * amount,
                background.G + (accent.G - background.G) * amount, background.B + (accent.B - background.B) * amount)
        end
    end
    local changed = {}
    for token, value in pairs(values) do
        if Theme[token] ~= value then changed[token] = value; Theme[token] = value end
    end
    if next(changed) == nil then return end
    repaintThemeBindings(changed)
    for _, listener in ipairs(themeListeners) do pcall(listener, ThemeEngine.name, changed) end
end

function ThemeEngine.Set(token: string, value: Color3): ()
    ThemeEngine.SetMany({[token] = value})
end

function ThemeEngine.Apply(name: string): boolean
    local preset: {[string]: Color3}? = THEME_PRESETS[name]
    if not preset then
        return false
    end
    for token: string, value: Color3 in pairs(preset :: {[string]: Color3}) do
        Theme[token] = value
    end
    ThemeEngine.name = name
    repaintThemeBindings(Theme)
    for _, listener: (string) -> () in ipairs(themeListeners) do
        pcall(listener, name)
    end
    return true
end

local DEFAULT_HEADER_COLOR: Color3 = Theme.text
local DEFAULT_MENU_COLOR: Color3 = Theme.background
local DEFAULT_SURFACE_COLOR: Color3 = Theme.surface
local DEFAULT_BORDER_COLOR: Color3 = Theme.outline

local TITLE_FONT: Font = Font.new(
    "rbxasset://fonts/families/BuilderSans.json",
    Enum.FontWeight.Bold,
    Enum.FontStyle.Normal
)
local THIN_FONT: Font = Font.new(
    "rbxasset://fonts/families/BuilderSans.json",
    Enum.FontWeight.Regular,
    Enum.FontStyle.Normal
)
local CONTROL_FONT: Font = Font.new(
    "rbxasset://fonts/families/BuilderSans.json",
    Enum.FontWeight.SemiBold,
    Enum.FontStyle.Normal
)
local VALVE_FONT: Font = CONTROL_FONT
local TEXT_BASE_MULTIPLIER: number = 1.35
local TEXT_SCALE_MINIMUM: number = 0.8
local TEXT_SCALE_MAXIMUM: number = 1.2
local IS_MOBILE_DEVICE: boolean = UserInputService.TouchEnabled
    and not UserInputService.KeyboardEnabled

-- v192/v193: a Windows touchscreen laptop reports TouchEnabled = true, so
-- Roblox spawns its phone controls (PlayerGui.TouchGui) and games switch their
-- own HUD to the touch layout. The player's screenshot showed Murder Mystery 2
-- doing exactly that: its `TouchInteractButtons` ScreenGui (Throw / Equip /
-- Perk) is a StarterGui template with ResetOnSpawn = false, so it sits in
-- PlayerGui and gets Enabled = true the moment the game believes the device is
-- a phone.
--
-- A machine with a keyboard is a desktop, so the two GUIs that ARE the phone
-- controls are hidden: Roblox's `TouchGui` and the game's `TouchInteractButtons`.
-- This is deliberately the least intrusive fix available:
--
--   * it writes ONLY `ScreenGui.Enabled` on those two GUIs, never anything the
--     game reads as state (v192 wrote the `Device` attribute of PlayerGui, which
--     changes game logic and can make the game rebuild its whole HUD -- removed);
```

```
--   * it never writes TouchEnabled (a client cannot), never touches the mouse
--     icon, MouseBehavior, the camera or a keybind;
--   * on a device without a keyboard nothing happens at all.
--
-- Re-applied whenever one of those GUIs appears and whenever the game turns one
-- back on, because Murder Mystery 2 does that on every HUD refresh.
local DESKTOP_HIDDEN_GUIS: {[string]: boolean} = {
    TouchGui = true,
    TouchInteractButtons = true,
}

local function isDesktopMachine(): boolean
    if not UserInputService.KeyboardEnabled then
        return false
    end
    if UserInputService.MouseEnabled then
        return true
    end
    -- No mouse reported (a few touch drivers say so): a mouse or keyboard key
    -- as the last input still means the player is not using thumbs.
    local ok: boolean, lastInput: any = pcall(function(): any
        return UserInputService:GetLastInputType()
    end)
    if not ok then
        return false
    end
    return lastInput == Enum.UserInputType.MouseButton1
        or lastInput == Enum.UserInputType.MouseMovement
        or lastInput == Enum.UserInputType.MouseWheel
        or lastInput == Enum.UserInputType.Keyboard
end

local function suppressTouchControls(): ()
    if not UserInputService.TouchEnabled or not isDesktopMachine() then
        return
    end
    for name in pairs(DESKTOP_HIDDEN_GUIS) do
        local ok: boolean, gui: any = pcall(function(): any
            return PlayerGui:FindFirstChild(name)
        end)
        if ok and gui and gui:IsA("ScreenGui") and gui.Enabled then
            pcall(function(): ()
                gui.Enabled = false
            end)
        end
    end
end

-- Keep watching the GUIs that the game turns back on. The handler calls the
-- same guarded write, so the property change it causes finds nothing to do and
-- there is no loop.
pcall(function(): ()
    local function watch(gui: Instance): ()
        pcall(function(): ()
            gui:GetPropertyChangedSignal("Enabled"):Connect(function(): ()
                if gui.Enabled then
                    task.defer(suppressTouchControls)
                end
            end)
        end)
    end

    suppressTouchControls()
    PlayerGui.ChildAdded:Connect(function(child: Instance): ()
        if DESKTOP_HIDDEN_GUIS[child.Name] then
            watch(child)
            task.defer(suppressTouchControls)
        end
    end)
    for _, child: Instance in ipairs(PlayerGui:GetChildren()) do
        if DESKTOP_HIDDEN_GUIS[child.Name] then
            watch(child)
        end
    end
end)

local configData: any = {
    version = 1,
    states = {},
    values = {},
    ui = {},
}
local configSaveQueued = false
local configSaveClosed = false
local configWriteAllowed: any = nil

local function serializeLuaValue(value: any, depth: number): string
    local kind: string = typeof(value)
    if kind == "string" then
        return string.format("%q", value)
    elseif kind == "number" then
        if value ~= value or value == math.huge or value == -math.huge then
            return "0"
        end
        return string.format("%.14g", value)
    elseif kind == "boolean" then
        return value and "true" or "false"
    elseif kind == "table" and depth < 6 then
        local parts: {string} = {}
        local length: number = #value
        for index: number = 1, length do
            table.insert(parts, serializeLuaValue(value[index], depth + 1))
        end
        for key: any, entry: any in pairs(value) do
            local keyType: string = typeof(key)
            local keyText: string? = nil
            if keyType == "string" then
                keyText = "[" .. string.format("%q", key) .. "]"
            elseif keyType == "number" then
                if key % 1 ~= 0 or key < 1 or key > length then
                    keyText = "[" .. string.format("%.14g", key) .. "]"
                end
            elseif keyType == "boolean" then
                keyText = "[" .. tostring(key) .. "]"
            end
            if keyText then
                table.insert(
                    parts,
                    keyText .. " = " .. serializeLuaValue(entry, depth + 1)
                )
            end
        end
        if #parts == 0 then
            return "{}"
        end
        return "{" .. table.concat(parts, ", ") .. "}"
    end
    return "nil"
end

local function decodeLuaConfig(source: string): any
    local compiler: any = (getfenv() :: any).loadstring
    if type(compiler) ~= "function" or type(source) ~= "string" then
        return nil
    end
    local built: boolean, chunk: any = pcall(compiler, source, "@WurstConfig")
    if not built or type(chunk) ~= "function" then
        return nil
    end
    local ran: boolean, result: any = pcall(chunk)
    if not ran or type(result) ~= "table" then
        return nil
    end
    return result
end

local function applyLoadedConfig(loaded: any): boolean
    if type(loaded) ~= "table" then
        return false
    end
    configData.version = type(loaded.version) == "number" and loaded.version or 1
    configData.states = type(loaded.states) == "table" and loaded.states or {}
    configData.values = type(loaded.values) == "table" and loaded.values or {}
    configData.ui = type(loaded.ui) == "table" and loaded.ui or {}
    return true
end

local function loadConfig()
    if type(isfile) ~= "function" or type(readfile) ~= "function" then
        return
    end

    local function readLuaConfig(path: string): any
        local existsOk: boolean, exists: any = pcall(isfile, path)
        if not existsOk or exists ~= true then
            return nil
        end
        local readOk: boolean, source: any = pcall(readfile, path)
        if not readOk or type(source) ~= "string" then
            return nil
        end
        return decodeLuaConfig(source)
    end

    local loaded: any = readLuaConfig(CONFIG_FILE_LUA)
    if not applyLoadedConfig(loaded) then
        loaded = readLuaConfig(CONFIG_FILE_BAK)
    end
    if not applyLoadedConfig(loaded) then
        pcall(function()
            if isfile(CONFIG_FILE) then
                local decoded: any = HttpService:JSONDecode(readfile(CONFIG_FILE))
                if applyLoadedConfig(decoded) then
                    loaded = decoded
                end
            end
        end)
    end

    configData.ui.windowPosition = nil
    configData.ui.windowSize = nil
end

local function saveConfigNow()
    if configSaveClosed or type(writefile) ~= "function"
        or (type(configWriteAllowed) == "function" and not configWriteAllowed()) then
        return
    end

    pcall(function()
        if type(isfolder) == "function"
            and type(makefolder) == "function"
            and not isfolder(CONFIG_FOLDER) then
            makefolder(CONFIG_FOLDER)
        end
        if type(isfolder) == "function"
            and type(makefolder) == "function"
            and not isfolder(CONFIG_PROFILE_FOLDER) then
            makefolder(CONFIG_PROFILE_FOLDER)
        end
        local payload: string = "-- Wurst config for game "
            .. CONFIG_GAME_ID
            .. " - modules, options, keybinds and UI.\n"
            .. "return {\n"
            .. "    version = "
            .. serializeLuaValue(configData.version, 0)
            .. ",\n"
            .. "    states = "
            .. serializeLuaValue(configData.states, 0)
            .. ",\n"
            .. "    values = "
            .. serializeLuaValue(configData.values, 0)
            .. ",\n"
            .. "    ui = "
            .. serializeLuaValue(configData.ui, 0)
            .. ",\n"
            .. "}\n"

        if type(isfile) == "function" and type(readfile) == "function" then
            local existsOk: boolean, exists: any = pcall(isfile, CONFIG_FILE_LUA)
            if existsOk and exists == true then
                local readOk: boolean, previous: any =
                    pcall(readfile, CONFIG_FILE_LUA)
                if readOk and type(previous) == "string" and #previous > 0 then
                    writefile(CONFIG_FILE_BAK, previous)
                end
            end
        end
        if type(configWriteAllowed) == "function" and not configWriteAllowed() then
            return
        end
        writefile(CONFIG_FILE_LUA, payload)
    end)
end

local configSaveGeneration: number = 0
local function queueConfigSave()
    if configSaveClosed or configSaveQueued then
        return
    end
    configSaveQueued = true
    local generation: number = configSaveGeneration + 1
    configSaveGeneration = generation
    task.delay(0.35, function()
        if configSaveGeneration ~= generation then
            return
        end
        configSaveQueued = false
        saveConfigNow()
    end)
end

local function flushConfigSave()
    configSaveGeneration += 1
    configSaveQueued = false
    saveConfigNow()
end

-- Retired runtimes must not write an old profile from a delayed task or their
-- still-registered BindToClose callback after a manual reinjection.
local function closeConfigSave()
    if configSaveClosed then
        return
    end
    flushConfigSave()
    configSaveClosed = true
end

pcall(function(): ()
    (game :: any):BindToClose(function(): ()
        saveConfigNow()
    end)
end)

local function colorFromConfig(value, fallback)
    if type(value) == "table" then
        return Color3.fromRGB(
            math.clamp(tonumber(value[1]) or 0, 0, 255),
            math.clamp(tonumber(value[2]) or 0, 0, 255),
            math.clamp(tonumber(value[3]) or 0, 0, 255)
        )
    end
    return fallback
end

local function colorToConfig(color)
    return {
        math.round(color.R * 255),
        math.round(color.G * 255),
        math.round(color.B * 255),
    }
end

local function sharedRuntimeGenv(): any
    local getGenv: any = (getfenv() :: any).getgenv
    if type(getGenv) == "function" then
        local ok: boolean, genv: any = pcall(getGenv)
        if ok and type(genv) == "table" then
            return genv
        end
    end
    return nil
end

-- Tear down the previous instance synchronously, before reading its profile.
-- This also works when Roblox defers AncestryChanged delivery.
local previousGenv: any = sharedRuntimeGenv()
local previousRuntime: any = previousGenv and previousGenv.WURST_RUNTIME
if type(previousRuntime) == "table" and type(previousRuntime.Destruct) == "function" then
    pcall(previousRuntime.Destruct)
end

local function findExistingGui()
    local locations = {PlayerGui}

    local coreGuiOk, coreGui = pcall(function()
        return game:GetService("CoreGui")
    end)

    if coreGuiOk and coreGui then
        table.insert(locations, coreGui)
    end

    if type(gethui) == "function" then
        local hiddenGuiOk, hiddenGui = pcall(gethui)
        if hiddenGuiOk and hiddenGui then
            table.insert(locations, hiddenGui)
        end
    end

    for _, location in ipairs(locations) do

        local existing = location:FindFirstChild(GUI_NAME)
        while existing do
            existing:Destroy()
            existing = location:FindFirstChild(GUI_NAME)
        end
    end
end

findExistingGui()
loadConfig()

local oldBlur = Lighting:FindFirstChild(BLUR_NAME)
if oldBlur then
    oldBlur:Destroy()
end

local MenuBlur: any = nil
pcall(function(): ()
    MenuBlur = Instance.new("BlurEffect")
    MenuBlur.Name = BLUR_NAME
    MenuBlur.Size = 14
    MenuBlur.Enabled = false
    MenuBlur.Parent = Lighting
end)

local ScreenGui = Instance.new("ScreenGui")
ScreenGui.Name = GUI_NAME
ScreenGui.ResetOnSpawn = false
ScreenGui.IgnoreGuiInset = true

ScreenGui.AutoLocalize = false
ScreenGui.DisplayOrder = 999999
ScreenGui.ZIndexBehavior = Enum.ZIndexBehavior.Sibling
ScreenGui:SetAttribute("BuildId", "0001-S17")

local function parentScreenGui()

    local protector: any = (getfenv() :: any).syn
    if type(protector) == "table" and type(protector.protect_gui) == "function" then
        pcall(protector.protect_gui, ScreenGui)
    end
    if type(gethui) == "function" then
        local hiddenGuiOk, hiddenGui = pcall(gethui)
        if hiddenGuiOk and hiddenGui then
            local parented = pcall(function()
                ScreenGui.Parent = hiddenGui
            end)
            if parented then
                return
            end
        end
    end

    local coreGuiOk, coreGui = pcall(function()
        return game:GetService("CoreGui")
    end)

    if coreGuiOk and coreGui then
        local parented = pcall(function()
            ScreenGui.Parent = coreGui
        end)
        if parented then
            return
        end
    end

    ScreenGui.Parent = PlayerGui
end

parentScreenGui()

local PopupLayer: Frame = Instance.new("Frame")
PopupLayer.Name = "PopupLayer"
PopupLayer.Active = false
PopupLayer.BackgroundTransparency = 1
PopupLayer.BorderSizePixel = 0
PopupLayer.Size = UDim2.fromScale(1, 1)
PopupLayer.ZIndex = 150
PopupLayer.Parent = ScreenGui

-- One-time reset of old automatic key defaults. Future explicit assignments
-- remain persisted; movement controls such as Fly's up/down are not shortcuts.
if configData.ui.unboundKeyDefaultsVersion ~= 1 then
    for key: string, value: any in pairs(configData.values) do
        if string.sub(key, 1, 9) == "Shortcut."
            or (type(value) == "string" and string.match(string.lower(key), "%.%w*key$")) then
            configData.values[key] = "Unknown"
        end
    end
    configData.ui.unboundKeyDefaultsVersion = 1
end
-- Restore the menu-only default removed in v34; module shortcuts stay unbound.
if configData.ui.menuKeyRestoreVersion ~= 1 then
    if not configData.ui.toggleKey or configData.ui.toggleKey == "Unknown" then
        configData.ui.toggleKey = "RightShift"
    end
    configData.ui.menuKeyRestoreVersion = 1
end
local savedToggleKey = Enum.KeyCode.RightShift
if type(configData.ui.toggleKey) == "string" then
    pcall(function()
        savedToggleKey = Enum.KeyCode[configData.ui.toggleKey]
            or Enum.KeyCode.RightShift
    end)
end

local state: any = {

    -- Diagnostics use the same boot flag in release and debug builds.
    debug = DEBUG_LOGS,
    visible = false,
    isMobile = IS_MOBILE_DEVICE,
    guiStyle = "Default",
    mobileMenuButtonVisible = true,
    toggleKey = savedToggleKey,
    waitingForKey = false,
    keyCaptureCallback = nil,
    headerColor = DEFAULT_HEADER_COLOR,
    menuColor = DEFAULT_MENU_COLOR,
    hookFunction = (getfenv() :: any).hookfunction,
    hookMetamethod = (getfenv() :: any).hookmetamethod,
    getNamecallMethod = (getfenv() :: any).getnamecallmethod,
    checkCaller = (getfenv() :: any).checkcaller,
    newCClosure = (getfenv() :: any).newcclosure,
    fireTouchInterest = (getfenv() :: any).firetouchinterest,
    requireModule = require,
    uiConnections = {} :: {RBXScriptConnection},
    styledButtons = {} :: {TextButton},
    styledTextBoxes = {} :: {TextBox},
    gameModulePaths = {} :: {[string]: {string}},
}

state.activeGameName = ACTIVE_GAME_MODULE

state.textScale = configData.ui.textScaleCalibrationVersion == 2
    and math.clamp(
        tonumber(configData.ui.textScale) or 1,
        TEXT_SCALE_MINIMUM,
        TEXT_SCALE_MAXIMUM
    )
    or 1
state.animationsEnabled = configData.ui.animationsEnabled ~= false

state.mobileButtonSize = math.clamp(
    tonumber(configData.ui.mobileButtonSize) or 52,
    28,
    120
)
configData.ui.guiStyle = nil

if configData.ui.wurstSchema ~= 3 then
    if configData.ui.wurstSchema ~= 2 then

        configData.values["ClickGUI.Maxheight"] = nil
        configData.values["ClickGUI.Maxsettingsheight"] = nil
        configData.values["ClickGUI.Opacity"] = nil
        configData.values["ClickGUI.Tooltipopacity"] = nil
        configData.values["WurstLogo.Background"] = nil
        configData.values["WurstLogo.Text"] = nil
        configData.values["WurstLogo.Visibility"] = nil
    end

    configData.ui.wurstSchema = 3
    configData.ui.windows = {}
    queueConfigSave()
end
configData.ui.textScaleCalibrationVersion = 2

state.menuStyle = "Wurst"

-- v162: the menu's own scale is remembered. It is a plain number, applied to
-- the same ThemeEngine.shape.scale the ClickGui already uses for its UIScale,
-- so nothing else in the UI has to know about it.
do
    local savedScale: any = configData.values["UI.MenuScale"]
    if type(savedScale) == "number" and savedScale == savedScale then
        ThemeEngine.shape.scale = math.clamp(savedScale, 0.7, 1.3)
    end
end

if configData.values["UI.CodeFaceMigration"] ~= 1 then
    configData.values["UI.CodeFaceMigration"] = 1
    if configData.values.interfaceFont ~= nil
        and configData.values.interfaceFont ~= "Code" then
        configData.values.interfaceFont = "Code"
    end
    if configData.values["UI.FontPack"] == nil
        and configData.values["UI.FontPack"] ~= "Code" then
        configData.values["UI.FontPack"] = "Code"
    end
    queueConfigSave()
end

state.fontPack = "Code"

local persistenceEnvironment: any = getfenv()
local persistenceGenv = sharedRuntimeGenv

-- These failures affect Destruct/Keep even in a release build, whose normal
-- debug warnings are muted. Keep them visible in the executor console.
local function teleportWarn(message: string): ()
    local logger: any = type(realWarn) == "function" and realWarn or warn
    logger("[" .. PRODUCT.logPrefix .. "] " .. message)
end

local function resolveTeleportQueue(): any
    for _, name: string in ipairs({"queue_on_teleport", "queueonteleport"}) do
        local queue: any = persistenceEnvironment[name]
        if type(queue) == "function" then
            return queue
        end
    end
    for _, providerName: string in ipairs({"syn", "fluxus"}) do
        local provider: any = persistenceEnvironment[providerName]
        if type(provider) == "table"
            and type(provider.queue_on_teleport) == "function" then
            return provider.queue_on_teleport
        end
    end
    return nil
end

-- The authorization lives beside the per-universe profile, not in getgenv:
-- executor environments may be recreated during a teleport. When storage
-- works, each boot/toggle gets a new token for its guarded queued snippet.
local teleportTokenPath: string = CONFIG_PROFILE_FOLDER
    .. "/Teleport_" .. CONFIG_GAME_ID .. ".txt"
local function newTeleportOwner(): string
    local ok: boolean, guid: any = pcall(function(): string
        return HttpService:GenerateGUID(false)
    end)
    if ok and type(guid) == "string" and #guid > 0 then
        return guid
    end
    return tostring(os.time()) .. ":" .. tostring(os.clock()) .. ":"
        .. tostring(math.random(1, 1073741824)) .. ":" .. tostring({})
end

local teleportOwner: string = newTeleportOwner()
local teleportSerial: number = 0
local teleportToken: string = ""
local teleportGuarded: boolean = false
local function writeTeleportToken(value: string): boolean
    if type(readfile) ~= "function" or type(writefile) ~= "function" then
        return false
    end
    local ok: boolean = pcall(function(): ()
        if type(isfolder) == "function" and type(makefolder) == "function" then
            if not isfolder(CONFIG_FOLDER) then
                makefolder(CONFIG_FOLDER)
            end
            if not isfolder(CONFIG_PROFILE_FOLDER) then
                makefolder(CONFIG_PROFILE_FOLDER)
            end
        end
        writefile(teleportTokenPath, value)
    end)
    if not ok then
        return false
    end
    local readOk: boolean, stored: any = pcall(readfile, teleportTokenPath)
    return readOk and stored == value
end

local function claimTeleportToken(): ()
    teleportSerial += 1
    teleportToken = teleportOwner .. ":" .. tostring(teleportSerial)
    teleportGuarded = writeTeleportToken(teleportToken)
end

local function revokeTeleportToken(): ()
    if not teleportGuarded then
        return
    end
    if type(readfile) ~= "function" then
        teleportWarn("could not read the teleport token; revocation is unverified")
        return
    end
    local ok: boolean, stored: any = pcall(readfile, teleportTokenPath)
    if not ok then
        teleportWarn("could not read the teleport token; revocation is unverified")
        return
    end
    if stored ~= teleportToken then
        return -- A newer runtime already owns this universe's authorization.
    end
    if not writeTeleportToken("stopped:" .. teleportToken) then
        -- If overwriting stopped working, try deleting only our own token.
        local checked: boolean, current: any = pcall(readfile, teleportTokenPath)
        local removed: boolean = false
        if checked and current == teleportToken and type(delfile) == "function" then
            local deleted: boolean, result: any = pcall(delfile, teleportTokenPath)
            removed = deleted and result ~= false
        end
        local stillOk: boolean, remaining: any = pcall(readfile, teleportTokenPath)
        if (stillOk and remaining == teleportToken) or (not stillOk and not removed) then
            teleportWarn("could not revoke the teleport token")
        end
    end
end

local function buildTeleportSnippet(token: string, guarded: boolean): string
    -- Arm on GameId, not PlaceId: instanced places in one game share a universe.
    local armedGameId: number = game.GameId
    local guardLines: {string} = guarded and {
        "    if type(readfile) ~= \"function\" then return false end",
        "    local ok, value = pcall(readfile, " .. string.format("%q", teleportTokenPath) .. ")",
        "    return ok and value == " .. string.format("%q", token),
    } or {"    return true"}
    return table.concat({
        "local genv = (type(getgenv) == \"function\" and getgenv()) or {}",
        "local function authorized()",
        table.concat(guardLines, "\n"),
        "end",
        "local function allowed()",
        "    return genv.WURST_STOPPED ~= true and game.GameId == "
            .. tostring(armedGameId) .. " and authorized()",
        "end",
        "if genv.WURST_STOPPED == true or not authorized() then return end",
        "if not game:IsLoaded() then game.Loaded:Wait() end",
        "if not allowed() then return end",
        "task.wait(1)",
```

```
        "if not allowed() then return end",
        "local ok, err = pcall(function()",
        "    local source = game:HttpGet("
            .. string.format("%q", RUNTIME_RAW_BASE .. "src/Wurst.lua?t=")
            .. " .. tostring(os.time()))",
        "    if not allowed() then return end",
        "    genv.WURST_BRANCH = " .. string.format("%q", RUNTIME_BRANCH),
        "    loadstring(source)()",
        "end)",
        "if not ok then warn(\"[Wurst] teleport reinject failed: \" .. tostring(err)) end",
    }, "\n")
end

local queueTeleport: any = resolveTeleportQueue()
local contract: any = {
    supported = type(queueTeleport) == "function",
    enabled = false,
    closed = false,
    queued = false,
}
state.teleportPersist = contract
local teleportConnection: any = nil
local appliedTeleportSetting: boolean? = nil
local initialGenv: any = persistenceGenv()
if initialGenv then
    initialGenv.WURST_TP_OWNER = teleportOwner
end
claimTeleportToken()

local function ownsTeleportSession(): boolean
    local genv: any = persistenceGenv()
    return not genv or genv.WURST_TP_OWNER == teleportOwner
end

-- A previous VM can still have delayed profile/telemetry writers even when
-- getgenv was replaced. Do not let them overwrite the new universe owner.
local function ownsRuntimeData(): boolean
    local genv: any = persistenceGenv()
    if genv and genv.WURST_TP_OWNER ~= nil
        and genv.WURST_TP_OWNER ~= teleportOwner then
        return false
    end
    if teleportGuarded and type(readfile) == "function" then
        local ok: boolean, value: any = pcall(readfile, teleportTokenPath)
        if ok then
            return value == teleportToken or value == "stopped:" .. teleportToken
        end
    end
    -- If the storage API becomes unreadable, ownership cannot be verified
    -- across VMs; retain the existing best-effort getgenv behaviour.
    return true
end
configWriteAllowed = ownsRuntimeData
state.ownsRuntimeData = ownsRuntimeData

local function disconnectTeleport(): ()
    if teleportConnection then
        pcall(function(): ()
            teleportConnection:Disconnect()
        end)
        teleportConnection = nil
    end
end
local function markTeleportStopped(): ()
    local genv: any = persistenceGenv()
    if genv and genv.WURST_TP_OWNER == teleportOwner then
        genv.WURST_STOPPED = true
        genv.WURST_TP_OWNER = nil
    end
end

local function warnUnguardedTeleport(): ()
    if not teleportGuarded and not contract._unguardedSaid then
        contract._unguardedSaid = true
        teleportWarn("teleport token storage unavailable; "
            .. "a queue already submitted during teleport may not be revocable")
    end
end

local function onTeleport(status: any): ()
    if contract.closed or not contract.enabled or not ownsTeleportSession() then
        return
    end
    if status == Enum.TeleportState.Failed then
        if contract.queued then
            -- A failed teleport may leave its old source in an executor queue.
            contract.queued = false
            revokeTeleportToken()
            claimTeleportToken()
        end
        return
    end
    if status ~= Enum.TeleportState.Started
        and status ~= Enum.TeleportState.WaitingForServer
        and status ~= Enum.TeleportState.InProgress then
        return
    end
    if contract.queued then
        return
    end
    warnUnguardedTeleport()
    local ok: boolean, result: any = pcall(
        queueTeleport,
        buildTeleportSnippet(teleportToken, teleportGuarded)
    )
    if ok and result ~= false then
        contract.queued = true
    else
        teleportWarn("teleport queue failed: " .. tostring(result))
    end
end

contract.SetEnabled = function(enabled: boolean): ()
    if contract.closed or not ownsTeleportSession() then
        return
    end
    local wanted: boolean = enabled == true
    contract.enabled = wanted
    if appliedTeleportSetting == wanted then
        return
    end
    if appliedTeleportSetting ~= nil then
        revokeTeleportToken()
        claimTeleportToken()
    end
    appliedTeleportSetting = wanted
    contract.queued = false
    disconnectTeleport()
    local genv: any = persistenceGenv()
    if genv and genv.WURST_TP_OWNER == teleportOwner then
        genv.WURST_STOPPED = not wanted
    end
    if not wanted then
        return
    end
    if not contract.supported then
        if not contract._unsupportedSaid then
            contract._unsupportedSaid = true
            teleportWarn("queue_on_teleport is not available; "
                .. "the menu cannot follow teleports here")
        end
        return
    end
    warnUnguardedTeleport()
    local connected: boolean, result: any = pcall(function(): any
        return LocalPlayer.OnTeleport:Connect(onTeleport)
    end)
    if connected then
        teleportConnection = result
    else
        contract.supported = false
        teleportWarn("could not observe teleport: " .. tostring(result))
    end
end
contract.Sync = function(): ()
    contract.SetEnabled(contract.enabled)
end

-- Ordinary GUI teardown during a teleport must leave the submitted token
-- intact. Explicit Destruct always revokes it, including after teardown.
contract.Close = function(): ()
    if contract.closed then
        return
    end
    local preserve: boolean = contract.queued and contract.enabled and ownsTeleportSession()
    contract.closed = true
    disconnectTeleport()
    if not preserve then
        revokeTeleportToken()
        markTeleportStopped()
    end
    contract.enabled = false
end
contract.Stop = function(): ()
    contract.closed = true
    contract.enabled = false
    disconnectTeleport()
    revokeTeleportToken()
    markTeleportStopped()
end
contract.SetEnabled(configData.states["ClickGUI.Keepafterteleport"] ~= false)

state.menuStyleListeners = {} :: {(string) -> ()}

state.SetMenuStyle = function(style: string, openImmediately: boolean?): ()
    if style == "Wurst" and style ~= "Navigator" then
        return
    end
    state.menuStyle = style
    configData.values["UI.MenuStyle"] = style
    queueConfigSave()
    local navigator: any = state.navigator
    local navigatorOpen: boolean = navigator ~= nil
        and navigator.root ~= nil
        and navigator.root.Visible == true
    if style == "Wurst" then
        if navigatorOpen and type(navigator.Close) == "function" then
            navigator.Close()
        end
        if openImmediately and not state.visible then
            state.setMenuVisible(true)
        end
    else
        if state.visible then
            state.setMenuVisible(false)
        end
        if openImmediately and navigator
            and type(navigator.Open) == "function" then
            navigator.Open()
        end
    end
    for _, listener: (string) -> () in ipairs(state.menuStyleListeners) do
        pcall(listener, style)
    end
end

state.openMenuSurface = function(): ()
    if state.menuStyle == "Navigator" then
        local navigator: any = state.navigator
        if navigator and navigator.root and navigator.root.Visible then
            navigator.Close()
            return
        end

        if state.visible then
            state.setMenuVisible(false)
        end
        if navigator and type(navigator.Open) == "function" then
            navigator.Open()
            return
        end
    end

    local navigator: any = state.navigator
    if navigator and navigator.root and navigator.root.Visible
        and type(navigator.Close) == "function" then
        navigator.Close()
    end
    state.setMenuVisible(not state.visible)
end

-- Camera replacement can briefly expose a zero-sized viewport. Keep the last
-- valid size so the menu is not rescaled or moved during that transition.
local function validLayoutViewport(viewport: any): boolean
    return viewport ~= nil and type(viewport.X) == "number" and type(viewport.Y) == "number"
        and viewport.X > 1 and viewport.Y > 1
        and viewport.X < math.huge and viewport.Y < math.huge
end
local bootViewport: any = workspace.CurrentCamera and workspace.CurrentCamera.ViewportSize
state.layoutViewport = if validLayoutViewport(bootViewport) then bootViewport else Vector2.new(1280, 720)
state.layout = computeLayoutMetrics(state.layoutViewport, UserInputService.TouchEnabled)
state.layoutListeners = {} :: {(LayoutMetrics) -> ()}

state.onLayout = function(listener: (LayoutMetrics) -> ()): ()
    table.insert(state.layoutListeners, listener)
end

state.refreshLayout = function(force: boolean?): boolean
    local camera: Camera? = workspace.CurrentCamera
    local viewport: any = camera and camera.ViewportSize
    if not validLayoutViewport(viewport) then
        if force then state.layoutNeedsRefresh = true end
        return false
    end
    local nextLayout: LayoutMetrics = computeLayoutMetrics(viewport, UserInputService.TouchEnabled)
    local previous: LayoutMetrics = state.layout
    local previousViewport: Vector2 = state.layoutViewport
    local forceRefresh: boolean = force == true or state.layoutNeedsRefresh == true
    state.layoutNeedsRefresh = false
    if not forceRefresh
        and previous.mode == nextLayout.mode
        and previous.cardHeight == nextLayout.cardHeight
        and previousViewport.X == viewport.X and previousViewport.Y == viewport.Y then
        return true
    end
    state.layoutViewport = viewport
    state.layout = nextLayout
    for _, listener: (LayoutMetrics) -> () in ipairs(state.layoutListeners) do
        local ok: boolean, failure: any = pcall(listener, nextLayout)
        if not ok then
            warn("[" .. PRODUCT.name .. "] layout listener: " .. tostring(failure))
        end
    end
    return true
end

local function trackUiConnection(
    connection: RBXScriptConnection
): RBXScriptConnection
    table.insert(state.uiConnections, connection)
    return connection
end

local menuCursorState: any = {
    owned = false,
    applying = false,
    behavior = nil,
    iconEnabled = nil,
    -- v162: how many times this menu session has re-asserted the cursor. A
    -- game that keeps setting it back (shift lock does exactly that, every
    -- frame) is not a bug to fight, it is the player's own state.
    fights = 0,
    fightStart = 0,
    yielded = false,
}
-- Re-assertions allowed before the menu lets the game own the cursor again.
local CURSOR_FIGHT_LIMIT: number = 8
local CURSOR_FIGHT_WINDOW: number = 1

-- v162 — SHIFT LOCK. Two rules, both learned from a reported regression:
--
-- 1. Only undo OUR OWN write. The old release path always wrote a behavior
--    back, so a shift lock the game had just enabled (pressing the same key
--    that closes the menu) was overwritten with Default on the way out: the
--    mouse stayed free and shift lock looked broken from then on.
-- 2. Never write LockCenter. If the player was shift-locked when the menu
--    opened, restoring that value would lock the camera without the game's own
--    shift-lock state, which is worse than releasing to Default.
local function setMenuCursorUnlocked(visible: boolean): ()
    if menuCursorState.applying then
        return
    end
    if visible then
        if menuCursorState.yielded then
            return
        end
        if not menuCursorState.owned then
            menuCursorState.behavior = UserInputService.MouseBehavior
            menuCursorState.iconEnabled = UserInputService.MouseIconEnabled
            menuCursorState.owned = true
        end
        menuCursorState.applying = true
        pcall(function(): ()
            UserInputService.MouseBehavior = Enum.MouseBehavior.Default
            UserInputService.MouseIconEnabled = true
        end)
        menuCursorState.applying = false
        return
    end
    if not menuCursorState.owned then
        menuCursorState.yielded = false
        return
    end
    menuCursorState.applying = true
    pcall(function(): ()
        -- Rule 1: if the cursor is not where we left it, somebody else (the
        -- game) moved it on purpose and that state stays.
        if UserInputService.MouseBehavior == Enum.MouseBehavior.Default then
            local saved: any = menuCursorState.behavior
            if saved == Enum.MouseBehavior.LockCenter or saved == nil then
                saved = Enum.MouseBehavior.Default
            end
            UserInputService.MouseBehavior = saved
        end
        if UserInputService.MouseIconEnabled == true
            and menuCursorState.iconEnabled ~= nil then
            UserInputService.MouseIconEnabled = menuCursorState.iconEnabled
        end
    end)
    menuCursorState.applying = false
    menuCursorState.behavior = nil
    menuCursorState.iconEnabled = nil
    menuCursorState.owned = false
    menuCursorState.fights = 0
    menuCursorState.yielded = false
end

local function enforceMenuCursor(): ()
    if menuCursorState.applying then
        return
    end
    if state.visible ~= true or not menuCursorState.owned then
        return
    end

    local rightClicking: boolean = false
    pcall(function(): ()
        rightClicking = UserInputService:IsMouseButtonPressed(Enum.UserInputType.MouseButton2)
    end)
    if rightClicking then
        return
    end
    if UserInputService.MouseBehavior == Enum.MouseBehavior.Default
        and UserInputService.MouseIconEnabled == true then
        return
    end
    -- Rule 3: a cursor that keeps being taken back is the player's shift lock.
    -- Re-assert a few times (a camera script may lock it once on open) and then
    -- stop, instead of fighting it sixty times a second for as long as the
    -- menu is open.
    local now: number = os.clock()
    if now - menuCursorState.fightStart > CURSOR_FIGHT_WINDOW then
        menuCursorState.fightStart = now
        menuCursorState.fights = 0
    end
    menuCursorState.fights += 1
    if menuCursorState.fights > CURSOR_FIGHT_LIMIT then
        menuCursorState.yielded = true
        menuCursorState.fights = 0
        return
    end
    setMenuCursorUnlocked(true)
end
pcall(function(): ()
    trackUiConnection(UserInputService:GetPropertyChangedSignal("MouseBehavior"):Connect(enforceMenuCursor))
    trackUiConnection(UserInputService:GetPropertyChangedSignal("MouseIconEnabled"):Connect(enforceMenuCursor))
    trackUiConnection(UserInputService.InputEnded:Connect(function(input: InputObject): ()
        if input.UserInputType == Enum.UserInputType.MouseButton2 and state.visible then
            task.defer(function(): ()
                if state.visible and not UserInputService:IsMouseButtonPressed(Enum.UserInputType.MouseButton2) then
                    setMenuCursorUnlocked(true)
                end
            end)
        end
    end))
    -- v162: a teleport/respawn can reset the cursor under us; releasing is
    -- always safe, holding a stale ownership is not.
    if LocalPlayer then
        pcall(function(): ()
            trackUiConnection(LocalPlayer.CharacterAdded:Connect(function(): ()
                if state.visible ~= true then
                    menuCursorState.owned = false
                    menuCursorState.yielded = false
                end
            end))
        end)
    end
end)

-- Roblox blurs the 3D scene, not GUI text; share the existing effect rather
-- than allocating a new BlurEffect every time a selector opens.
state.setChoiceBlur = function(active: boolean): ()
    state.choiceBlur = active
    if MenuBlur then
        MenuBlur.Enabled = state.visible == true or active
    end
end

local menuVisibilityListeners: {(boolean) -> ()} = {}
state.addMenuVisibilityListener = function(listener: (boolean) -> ()): ()
    table.insert(menuVisibilityListeners, listener)
end

local function setMenuVisibleCore(visible: boolean): ()
    state.visible = visible
    setMenuCursorUnlocked(visible)
    if MenuBlur then
        pcall(function(): ()
            MenuBlur.Enabled = visible == true or state.choiceBlur == true
        end)
    end
    for _, listener: (boolean) -> () in ipairs(menuVisibilityListeners) do
        pcall(listener, visible)
    end
end
state.setMenuVisible = setMenuVisibleCore

type AssetDefinition = {
    url: string,
    fileName: string,
    fallback: string,
    version: number?,
    assetType: string?,
}

do
local generatedEnvironment: any = nil
if type(executorGlobals.getgenv) == "function" then
    local environmentOk: boolean, resolvedEnvironment: any = pcall(
        executorGlobals.getgenv
    )
    if environmentOk and type(resolvedEnvironment) == "table" then
        generatedEnvironment = resolvedEnvironment
    end
end

local function getEnvironmentValue(name: string): any
    if generatedEnvironment and generatedEnvironment[name] ~= nil then
        return generatedEnvironment[name]
    end
    return executorGlobals[name]
end

local AssetManager: any = {
    cacheRoot = CONFIG_FOLDER,
    cacheFolder = CONFIG_FOLDER .. "/Assets",
    resolved = {} :: {[string]: string},
    resolving = {} :: {[string]: boolean},

    manifest = {

        valveFont = {
            url = REPOSITORY_RAW_BASE
                .. "src/guis/Wurst/Assets/Typography/ValveBD.otf",
            fileName = "valve-bd-v1.otf",
            fallback = "",
            version = 1,
            assetType = "font/otf",
        },

        wurstLogo = {
            url = REPOSITORY_RAW_BASE .. "src/guis/Wurst/Assets/Wurst/wurst_128.png",
            fileName = "wurst-logo-v1.png",
            fallback = "",
            version = 1,
            assetType = "image/png",
        },

        bitmapFontAtlas = {
            url = REPOSITORY_RAW_BASE .. "src/guis/Wurst/Assets/Font/monocraft-16.png",
            fileName = "monocraft-16-v1.png",
            fallback = "",
            version = 1,
            assetType = "image/png",
        },

        bitmapFontAtlas8 = {
            url = REPOSITORY_RAW_BASE .. "src/guis/Wurst/Assets/Font/monocraft-8.png",
            fileName = "monocraft-8-v1.png",
            fallback = "",
            version = 1,
            assetType = "image/png",
        },
        bitmapFontAtlas24 = {
            url = REPOSITORY_RAW_BASE .. "src/guis/Wurst/Assets/Font/monocraft-24.png",
            fileName = "monocraft-24-v1.png",
            fallback = "",
            version = 1,
            assetType = "image/png",
        },
        bitmapFontAtlas32 = {
            url = REPOSITORY_RAW_BASE .. "src/guis/Wurst/Assets/Font/monocraft-32.png",
            fileName = "monocraft-32-v1.png",
            fallback = "",
            version = 1,
            assetType = "image/png",
        },

    } :: {[string]: AssetDefinition},
}

local function selectHttpRequest(): (any, string)
    local synEnvironment: any = getEnvironmentValue("syn")
    local candidates: {{any}} = {
        {getEnvironmentValue("request"), "request"},
        {getEnvironmentValue("http_request"), "http_request"},
        {getEnvironmentValue("httprequest"), "httprequest"},
        {synEnvironment and synEnvironment.request, "syn.request"},
    }
    for _, candidate: {any} in ipairs(candidates) do
        if type(candidate[1]) == "function" then
            return candidate[1], tostring(candidate[2])
        end
    end
    return nil, "game:HttpGet"
end

local function validateAssetBytes(
    bytes: any,
    definition: AssetDefinition
): (boolean, string?)
    if type(bytes) ~= "string" then
        return false, "non-binary response: " .. type(bytes)
    end
    if #bytes < 64 then
        return false, "response too short: " .. tostring(#bytes) .. " bytes"
    end
    local prefix: string = string.lower(string.sub(bytes, 1, 128))
    if string.find(prefix, "<!doctype html", 1, true)
        or string.find(prefix, "<html", 1, true)
        or string.find(prefix, "404: not found", 1, true) then
        return false, "the response contains HTML/404"
    end
    if definition.assetType == "font/otf" then
        local fontSignature: string = string.sub(bytes, 1, 4)
        if fontSignature ~= "OTTO" and fontSignature ~= string.char(0, 1, 0, 0) then
            return false, "invalid OpenType/TrueType signature"
        end
        return true, nil
    end
    local isPng: boolean = string.sub(bytes, 1, 8) == "\137PNG\r\n\26\n"
    local isJpeg: boolean = string.byte(bytes, 1) == 0xFF
        and string.byte(bytes, 2) == 0xD8
        and string.byte(bytes, 3) == 0xFF
    if not isPng and not isJpeg then
        return false, "invalid PNG signature"
    end
    return true, nil
end

function AssetManager.ensureCacheFolder(): (boolean, string?)
    local isFolder: any = getEnvironmentValue("isfolder")
    local makeFolder: any = getEnvironmentValue("makefolder")
    local function ensureOne(path: string): (boolean, string?)
        if type(isFolder) == "function" then
            local checked: boolean, exists: any = pcall(isFolder, path)
            if checked and exists then
                return true, nil
            end
        end
        if type(makeFolder) ~= "function" then
            return false, "makefolder is unavailable"
        end
        local created: boolean, createError: any = pcall(makeFolder, path)
        if not created then
            return false, tostring(createError)
        end
        return true, nil
    end

    local rootOk: boolean, rootError: string? = ensureOne(AssetManager.cacheRoot)
    if not rootOk then
        return false, "carpeta principal: " .. tostring(rootError)
    end
    local cacheOk: boolean, cacheError: string? = ensureOne(AssetManager.cacheFolder)
    if not cacheOk then
        return false, "carpeta Assets: " .. tostring(cacheError)
    end
    return true, nil
end

function AssetManager.sweepStaleCache(): number
    local listFiles: any = getEnvironmentValue("listfiles")
    local deleteFile: any = getEnvironmentValue("delfile")
    if type(listFiles) ~= "function" or type(deleteFile) ~= "function" then
        return 0
    end
    local expected: {[string]: boolean} = {}
    for _, definition: any in pairs(AssetManager.manifest) do
        expected[tostring(definition.fileName)] = true
    end
    local listed: boolean, entries: any = pcall(listFiles, AssetManager.cacheFolder)
    if not listed or type(entries) ~= "table" then
        return 0
    end
    local removed: number = 0
    for _, path: any in ipairs(entries) do
        local entry: string = tostring(path)
        local fileName: string = entry:gsub("\\", "/"):match("([^/]+)$") or entry
        if not expected[fileName] then
            if pcall(deleteFile, entry) then
                removed += 1
            end
        end
    end
    return removed
end

function AssetManager.readVerifiedCache(
    path: string,
    definition: AssetDefinition
): (string?, string?)
    local isFile: any = getEnvironmentValue("isfile")
    local readFile: any = getEnvironmentValue("readfile")
    if type(isFile) ~= "function" or type(readFile) ~= "function" then
        return nil, "isfile/readfile are unavailable"
    end
    local checked: boolean, exists: any = pcall(isFile, path)
    if not checked or not exists then
        return nil, "file not found"
    end
    local readOk: boolean, bytes: any = pcall(readFile, path)
    if not readOk then
        return nil, "readfile: " .. tostring(bytes)
```

```
    end
    local valid: boolean, validationError: string? = validateAssetBytes(
        bytes,
        definition
    )
    if not valid then
        return nil, validationError
    end
    return bytes, nil
end

function AssetManager.download(definition: AssetDefinition): (string?, string?)
    local requestFunction: any, _requestName: string = selectHttpRequest()

    local lastError: string = "no response"
    for attempt: number = 1, 3 do
        local requestOk: boolean, response: any
        if requestFunction then
            requestOk, response = pcall(requestFunction, {
                Url = definition.url,
                Method = "GET",
                Headers = {Accept = definition.assetType or "*/*"},
            })
        else
            requestOk, response = pcall(function(): any
                return {
                    StatusCode = 200,
                    Body = (game :: any):HttpGet(definition.url, true),
                }
            end)
        end

        if requestOk then
            local statusCode: number = 0
            local bytes: any = nil
            if type(response) == "string" then
                statusCode = 200
                bytes = response
            elseif type(response) == "table" then
                statusCode = tonumber(
                    response.StatusCode or response.Status or response.status_code
                ) or 0
                bytes = response.Body or response.body or response.ResponseBody
            end
            local valid: boolean, validationError: string? = validateAssetBytes(
                bytes,
                definition
            )
            if statusCode >= 200 and statusCode < 300 and valid then
                return bytes, nil
            end
            response = "HTTP "
                .. tostring(statusCode)
                .. ": "
                .. tostring(validationError)
        end

        lastError = "attempt "
            .. tostring(attempt)
            .. ": "
            .. tostring(response)
        if attempt < 3 then
            task.wait(0.35 * attempt)
        end
    end
    return nil, "download failed after three attempts; " .. lastError
end

function AssetManager.resolve(logicalName: string): (string?, string?)
    local alreadyResolved: string? = AssetManager.resolved[logicalName]
    if alreadyResolved then
        return alreadyResolved, nil
    end
    local definition: AssetDefinition? = AssetManager.manifest[logicalName]
    if not definition then
        return nil, "asset desconocido: " .. logicalName
    end
    local assetFunction: any = getEnvironmentValue("getcustomasset")
        or getEnvironmentValue("getsynasset")
    local writeFile: any = getEnvironmentValue("writefile")
    if type(assetFunction) ~= "function" then
        return nil, "getcustomasset/getsynasset is unavailable"
    end
    if type(writeFile) ~= "function" then
        return nil, "writefile is unavailable"
    end

    local folderOk: boolean, folderError: string? =
        AssetManager.ensureCacheFolder()
    if not folderOk then
        return nil, folderError
    end
    local path: string = AssetManager.cacheFolder .. "/" .. definition.fileName
    local waitTime: number = 0
    while AssetManager.resolving[logicalName] and waitTime < 12 do
        task.wait(0.05)
        waitTime += 0.05
    end
    alreadyResolved = AssetManager.resolved[logicalName]
    if alreadyResolved then
        return alreadyResolved, nil
    end
    if AssetManager.resolving[logicalName] then
        return nil, "timed out waiting for " .. logicalName
    end
    AssetManager.resolving[logicalName] = true
    local cachedBytes: string?, cacheError: string? =
        AssetManager.readVerifiedCache(path, definition)
    if not cachedBytes then
        print("[" .. PRODUCT.logPrefix .. ":Assets] cache cannot be reused: " .. tostring(cacheError))
        local downloaded: string?, downloadError: string? =
            AssetManager.download(definition)
        if not downloaded then
            AssetManager.resolving[logicalName] = false
            return nil, downloadError
        end
        local written: boolean, writeError: any = pcall(writeFile, path, downloaded)
        if not written then
            AssetManager.resolving[logicalName] = false
            return nil, "writefile: " .. tostring(writeError)
        end
        local verified: string?, verifyError: string? =
            AssetManager.readVerifiedCache(path, definition)
        if not verified then
            AssetManager.resolving[logicalName] = false
            return nil, "post-write verification: " .. tostring(verifyError)
        end
    end

    local assetOk: boolean, assetId: any = pcall(assetFunction, path)
    if not assetOk or type(assetId) ~= "string" or assetId == "" then
        AssetManager.resolving[logicalName] = false
        return nil, "getcustomasset: " .. tostring(assetId)
    end
    AssetManager.resolved[logicalName] = assetId
    AssetManager.resolving[logicalName] = false
    return assetId, nil
end

function AssetManager.resolveFont(
    logicalName: string,
    fallbackFont: Font,
    weight: Enum.FontWeight,
    style: Enum.FontStyle
): Font
    local assetId: string?, loadError: string? = AssetManager.resolve(logicalName)
    if not assetId then
        warn(
            "[" .. PRODUCT.logPrefix .. ":Assets] fallback font for "
                .. logicalName
                .. ": "
                .. tostring(loadError)
        )
        return fallbackFont
    end
    local fontOk: boolean, fontFace: any = pcall(
        Font.new,
        assetId,
        weight,
        style
    )
    if not fontOk or typeof(fontFace) ~= "Font" then
        warn(
            "[" .. PRODUCT.logPrefix .. ":Assets] Font.new fallback para "
                .. logicalName
                .. ": "
                .. tostring(fontFace)
            )
        return fallbackFont
    end
    print("[" .. PRODUCT.logPrefix .. ":Assets] font loaded: " .. logicalName)
    return fontFace
end

    VALVE_FONT = AssetManager.resolveFont(
        "valveFont",
        CONTROL_FONT,
        Enum.FontWeight.Bold,
        Enum.FontStyle.Normal
    )

state.customFontAssets = {

    ["Valve"] = "valveFont",
}

state.listInterfaceFonts = function(): {string}
    local names: {string} = {"Menu default"}

    for _, item: EnumItem in ipairs(Enum.Font:GetEnumItems()) do
        if item ~= Enum.Font.Unknown then
            table.insert(names, item.Name)
        end
    end
    for name: string in pairs(state.customFontAssets) do
        table.insert(names, name)
    end
    table.sort(names, function(first: string, second: string): boolean
        if first == "Menu default" then
            return true
        end
        if second == "Menu default" then
            return false
        end
        return first < second
    end)
    return names
end

state.resolveInterfaceFont = function(name: string): Font?
    if name == "Menu default" then
        return Font.new(
            "rbxasset://fonts/families/BuilderSans.json",
            Enum.FontWeight.Regular,
            Enum.FontStyle.Normal
        )
    end
    local customAsset: string? = state.customFontAssets[name]
    if customAsset then

        return AssetManager.resolveFont(
            customAsset,
            CONTROL_FONT,
            Enum.FontWeight.Regular,
            Enum.FontStyle.Normal
        )
    end
    local enumOk: boolean, enumItem: any = pcall(function(): any
        return (Enum.Font :: any)[name]
    end)
    if not enumOk or typeof(enumItem) ~= "EnumItem" then
        return nil
    end
    local fontOk: boolean, face: any = pcall(Font.fromEnum, enumItem)
    if not fontOk then
        return nil
    end
    return face :: Font
end

state.applyInterfaceFont = function(name: string, persist: boolean?): ()
    local base: Font? = state.resolveInterfaceFont(name)
    if not base then
        return
    end
    local resolvedBase: Font = base :: Font
    local family: string = resolvedBase.Family
    local style: Enum.FontStyle = resolvedBase.Style

    local hasWeights: boolean =
        string.find(family, "fonts/families/", 1, true) ~= nil
    local titleWeight: Enum.FontWeight = hasWeights
        and Enum.FontWeight.Bold
        or resolvedBase.Weight
    local controlWeight: Enum.FontWeight = hasWeights
        and Enum.FontWeight.SemiBold
        or resolvedBase.Weight
    local bodyWeight: Enum.FontWeight = hasWeights
        and Enum.FontWeight.Regular
        or resolvedBase.Weight
    TITLE_FONT = Font.new(family, titleWeight, style)
    CONTROL_FONT = Font.new(family, controlWeight, style)
    THIN_FONT = Font.new(family, bodyWeight, style)
    VALVE_FONT = CONTROL_FONT

    for _, descendant: Instance in ipairs(ScreenGui:GetDescendants()) do
        if descendant:IsA("TextLabel")
            or descendant:IsA("TextButton")
            or descendant:IsA("TextBox") then

            local current: Font? = (descendant :: any).FontFace
            if current then
                local weight: Enum.FontWeight = hasWeights
                    and (current :: Font).Weight
                    or resolvedBase.Weight
                local ok: boolean = pcall(function(): ()
                    (descendant :: any).FontFace =
                        Font.new(family, weight, (current :: Font).Style)
                end)
                if not ok then
                    (descendant :: any).FontFace = resolvedBase
                end
            else
                (descendant :: any).FontFace = resolvedBase
            end
        end
    end

    if persist ~= false then
        configData.values.interfaceFont = name
        queueConfigSave()
    end
end

state.resolveAsset = function(logicalName: string): (string?, string?)
    return AssetManager.resolve(logicalName)
end

state.sweepStaleAssets = function(): number
    return AssetManager.sweepStaleCache()
end

state.applyAssetImage = function(
    logicalName: string,
    image: ImageLabel,
    onApplied: (() -> ())?
): ()
    task.spawn(function(): ()
        local assetId: string?, failure: string? = AssetManager.resolve(logicalName)
        if not assetId then
            warn("[" .. PRODUCT.logPrefix .. ":Assets] " .. logicalName .. ": " .. tostring(failure))
            return
        end
        if not image.Parent then
            return
        end
        image.Image = assetId :: string
        if onApplied then
            onApplied()
        end
    end)
end
end

local registeredText = {}
local SectionManager = {
    entries = {},
}

function SectionManager.register(sectionName, builder)
    SectionManager.entries[sectionName] = {
        builder = builder,
        loaded = false,
        loading = false,
    }
end

function SectionManager.initialize(sectionName: string): (boolean, string?)
    local entry = SectionManager.entries[sectionName]
    if not entry or entry.loaded or entry.loading then
        return entry ~= nil and entry.loaded == true, nil
    end

    entry.loading = true

    local success, errorMessage = xpcall(entry.builder, function(message)
        local text = tostring(message)
        if debug and type(debug.traceback) == "function" then
            return debug.traceback(text, 2)
        end
        return text
    end)

    entry.loading = false
    if success then
        entry.loaded = true
        return true, nil
    end

    warn(
        "[" .. PRODUCT.name .. "] "
            .. sectionName
            .. " failed to initialize: "
            .. tostring(errorMessage)
    )
    return false, tostring(errorMessage)
end

function SectionManager.initializeAllAsync(): ()
    local sectionNames: {string} = {}
    for sectionName: string in pairs(SectionManager.entries) do
        table.insert(sectionNames, sectionName)
    end
    table.sort(sectionNames)

    local total: number = #sectionNames
    local completed: number = 0
    local failures: number = 0

    local function finishLoading(): ()
        print("[" .. PRODUCT.logPrefix .. ":Bootstrap] Load complete · failures=" .. tostring(failures))
        -- With no game-specific section, the universal menu is already built.
        -- It still needs to release its launcher, especially on mobile.
        local function finish(): ()
            state.menuReady = true
            if type(state.refreshLauncher) == "function" then
                pcall(state.refreshLauncher)
            end
        end
        if type(task) == "table" and type(task.defer) == "function" then
            task.defer(finish)
        else
            finish()
        end
    end

    if total == 0 then
        finishLoading()
        return
    end

    for _, sectionName: string in ipairs(sectionNames) do
        task.spawn(function(): ()
            print("[" .. PRODUCT.logPrefix .. ":Bootstrap] Initializing section " .. sectionName)
            local success: boolean, errorMessage: string? =
                SectionManager.initialize(sectionName)
            if not success then
                failures += 1
                local environment=state.frameworkEnvironment
                if environment and type(environment.notify)=="function" then
                    environment.notify("Could not load "..sectionName.." — check the console")
                end
                warn(
                    "[" .. PRODUCT.logPrefix .. ":Bootstrap] Section "
                        .. sectionName
                        .. " failed: "
                        .. tostring(errorMessage)
                )
            end
            completed += 1
            if completed >= total then
                finishLoading()
            end
        end)
    end
end

local function create(className, properties)
    local object = Instance.new(className)

    for property, value in pairs(properties or {}) do
        if property ~= "Parent" then
            object[property] = value
        end
    end

    if object:IsA("TextLabel")
        or object:IsA("TextButton")
        or object:IsA("TextBox") then
        object.AutoLocalize = false
    end

    if properties and properties.Parent then
        object.Parent = properties.Parent
    end

    return object
end

local function cornerRadius(_radius: number): UDim
    return UDim.new(0, 0)
end

local function addRoundedStyle(
    object: GuiObject,
    radius: number,
    strokeTransparency: number?
): (UICorner, UIStroke)
    local corner: UICorner = Instance.new("UICorner")
    corner.Name = "StyleCorner"
    corner.CornerRadius = cornerRadius(radius)
    corner.Parent = object

    local stroke: UIStroke = Instance.new("UIStroke")
    stroke.Name = "StyleStroke"
    stroke.ApplyStrokeMode = Enum.ApplyStrokeMode.Border
    stroke.Color = Theme.outline
    stroke.Thickness = 1
    stroke.Transparency = strokeTransparency or 0.45
    stroke.Parent = object
    return corner, stroke
end

state.bitmapText = {
    size = 16,
    lineHeight = 18,

    advance = 12,

    ascent = 14,

    source = "monocraft-fallback",

    atlas = {
        ready = false,
        image = nil :: string?,
        size = 16,
        first = 32,
        last = 126,
        columns = 16,
        cellWidth = 11,
        cellHeight = 17,
        advance = 11,
        ascent = 15,
        descent = 2,

        generation = 0,

        fallbackGlyph = 63,
    },
    onReady = {} :: {() -> ()},

    stats = {
        draws = 0,
        skipped = 0,
        glyphsCreated = 0,
        glyphsReused = 0,
        lastDrawMs = 0,
    },

    _live = setmetatable({}, {__mode = "k"}) :: any,

    _adopted = setmetatable({}, {__mode = "k"}) :: any,

    _pendingAdopt = setmetatable({}, {__mode = "k"}) :: any,

    width = function(text: any): number
        return state.bitmapText.measure({{text = tostring(text)}}, 16)
    end,
    make = function(properties: any): TextLabel
        local resolved: any = {}
        for key: string, value: any in pairs(properties or {}) do
            resolved[key] = value
        end
        resolved.FontFace = CONTROL_FONT
        resolved.TextSize = resolved.TextSize or 16
        if resolved.TextTruncate == nil then
            resolved.TextTruncate = Enum.TextTruncate.AtEnd
        end
        return (create("TextLabel", resolved) :: any) :: TextLabel
    end,
}

local fontEnvironment: any = getfenv()
state.bitmapText.capabilities = {
    canHttp = type((game :: any).HttpGet) == "function",
    canRead = type(fontEnvironment.readfile) == "function",
    canWrite = type(fontEnvironment.writefile) == "function",
    canCustomAsset = type(fontEnvironment.getcustomasset) == "function",
    canSynAsset = type(fontEnvironment.getsynasset) == "function",
    mobile = state.isMobile == true,
    desktop = state.isMobile ~= true,
}

state.bitmapText.family = {
    [8] = {
        ready = false, image = nil :: string?, size = 8, columns = 16,
        cellWidth = 5, cellHeight = 9, advance = 5, ascent = 8, descent = 1,
        asset = "bitmapFontAtlas8",
    },
    [16] = state.bitmapText.atlas,
    [24] = {
        ready = false, image = nil :: string?, size = 24, columns = 16,
        cellWidth = 16, cellHeight = 25, advance = 16, ascent = 22, descent = 3,
        asset = "bitmapFontAtlas24",
    },
    [32] = {
        ready = false, image = nil :: string?, size = 32, columns = 16,
        cellWidth = 21, cellHeight = 33, advance = 21, ascent = 29, descent = 4,
        asset = "bitmapFontAtlas32",
    },
}

state.bitmapText.layerScale = function(): number
    local instance: any = PopupLayer:FindFirstChild("ClickGuiScale")
    local value: number = instance and tonumber(instance.Scale) or 1
    if value ~= value or value <= 0 then
        value = 1
    end
    return value
end

state.bitmapText.pickAtlas = function(physicalSize: number): any
    local contract: any = state.bitmapText

    if contract.rasterFamilyEnabled == false then
        return contract.atlas
    end
    local smallestCovering: any = nil
    local largest: any = nil
    for _, member: any in pairs(contract.family) do
        if member.ready and member.image then
            if member.size >= physicalSize
                and (smallestCovering == nil
                    or member.size < smallestCovering.size) then
                smallestCovering = member
            end
            if largest == nil or member.size > largest.size then
                largest = member
            end
        end
    end
    return smallestCovering or largest or contract.atlas
end

state.bitmapText.baselineFor = function(textSize: number?): number
    local contract: any = state.bitmapText
    return contract.atlas.ascent * ((tonumber(textSize) or 16) / contract.atlas.size)
end

state.bitmapText.eachCode = function(text: string, visit: (number) -> ()): ()
    if utf8.len(text) ~= nil then
        for _, code: number in utf8.codes(text) do
            visit(code)
        end
    else
        for position: number = 1, #text do
            visit(string.byte(text, position) or 63)
        end
    end
end

local function contractAdvanceFor(atlas: any, code: number, scale: number): number
    local advances: any = atlas.advances
    local specific: any = advances and (advances[code] or advances[tostring(code)])
    return (tonumber(specific) or atlas.advance) * scale
end

state.bitmapText.measure = function(segments: {any}, textSize: number?): number
    local contract: any = state.bitmapText
    local atlas: any = contract.atlas
    local scale: number = (tonumber(textSize) or 16) / contract.size
    local widest: number = 0
    local x: number = 0
    for _, segment: any in ipairs(segments) do
        contract.eachCode(tostring(segment.text or ""), function(code: number): ()
            if code == 10 then
                x = 0
            else
                if atlas.ready then
                    x += contractAdvanceFor(atlas, code, scale)
                else
                    x += contract.advance * scale
                end
                widest = math.max(widest, x)
            end
        end)
    end
    return widest
end

state.bitmapText.wrap = function(
    text: any,
    wrapWidth: number,
    textSize: number?
): (string, number)
    local contract: any = state.bitmapText
    local lines: {string} = {}
    local current: string = ""
    for word: string in string.gmatch(tostring(text or ""), "%S+") do
        local candidate: string = current == "" and word
            or (current .. " " .. word)
        if contract.measure({{text = candidate}}, textSize) <= wrapWidth
            or current == "" then
            current = candidate
        else
            table.insert(lines, current)
            current = word
        end
    end
    if current ~= "" then
        table.insert(lines, current)
    end
    if #lines == 0 then
```

```
        lines = {""}
    end
    return table.concat(lines, "\n"), #lines
end

state.bitmapText.draw = function(
    parent: Instance,
    segments: {any},
    options: any
): Frame
    local startedAt: number = os.clock()
    local contract: any = state.bitmapText
    local resolvedOptions: any = options or {}
    local name: string = tostring(resolvedOptions.Name or "BitmapText")
    local textSize: number = tonumber(resolvedOptions.TextSize) or 16
    local zIndex: number = tonumber(resolvedOptions.ZIndex) or 20
    local align: string = tostring(resolvedOptions.align or "Left")
    local shadow: boolean = resolvedOptions.shadow == true
    local maxWidth: number? = tonumber(resolvedOptions.maxWidth)
    local wantsEllipsis: boolean = resolvedOptions.ellipsis == true
    local transparency: number = tonumber(resolvedOptions.transparency) or 0
    local atlas: any = contract.atlas

    local deviceScale: number = contract.layerScale()
    local raster: any = contract.pickAtlas(textSize * deviceScale)
    local parts: {string} = {
        tostring(textSize), align, tostring(shadow),
        tostring(maxWidth), tostring(atlas.ready),
        tostring(transparency), tostring(wantsEllipsis),
        string.format("%.4f", deviceScale),
        tostring(raster and raster.size),
        tostring(atlas.generation),
    }
    for _, segment: any in ipairs(segments) do
        local color: Color3 = segment.color or Theme.text
        table.insert(parts, tostring(segment.text or ""))
        table.insert(parts, string.format(
            "%02X%02X%02X",
            math.floor(color.R * 255 + 0.5),
            math.floor(color.G * 255 + 0.5),
            math.floor(color.B * 255 + 0.5)
        ))
    end
    local signature: string = table.concat(parts, "\1")

    local holder: any = (parent :: any):FindFirstChild(name)
    local record: any = holder and contract._live[holder]
    if holder and not record then

        holder:Destroy()
        holder = nil
    end
    if not holder then
        holder = create("Frame", {
            Parent = parent,
            Name = name,
            BackgroundTransparency = 1,
            BorderSizePixel = 0,
        })
        record = {glyphs = {} :: {ImageLabel}, fallback = nil :: TextLabel?}
        contract._live[holder] = record
    end
    record.request = {parent = parent, segments = segments, options = resolvedOptions}

    holder.ZIndex = zIndex
    if resolvedOptions.AnchorPoint or resolvedOptions.Position then
        holder.AnchorPoint = resolvedOptions.AnchorPoint or Vector2.new(0, 0.5)
        holder.Position = resolvedOptions.Position or UDim2.new(0, 0, 0.5, 0)
    elseif align == "Right" then
        holder.AnchorPoint = Vector2.new(1, 0.5)
        holder.Position = UDim2.new(1, 0, 0.5, 0)
    elseif align == "Center" then
        holder.AnchorPoint = Vector2.new(0.5, 0.5)
        holder.Position = UDim2.new(0.5, 0, 0.5, 0)
    else
        holder.AnchorPoint = Vector2.new(0, 0.5)
        holder.Position = UDim2.new(0, 0, 0.5, 0)
    end

    if record.signature == signature then
        contract.stats.skipped += 1
        return holder :: Frame
    end
    record.signature = signature
    record.paintMap = {}

    local used: number = 0
    local function nextGlyph(): ImageLabel
        used += 1
        local glyph: ImageLabel? = record.glyphs[used]
        if glyph then
            contract.stats.glyphsReused += 1
        else
            glyph = (create("ImageLabel", {
                Parent = holder,
                Name = "G",
                BackgroundTransparency = 1,
                BorderSizePixel = 0,
            }) :: any) :: ImageLabel

            pcall(function(): ()
                (glyph :: any).ResampleMode = Enum.ResamplerMode.Pixelated
            end)
            record.glyphs[used] = glyph
            contract.stats.glyphsCreated += 1
        end
        (glyph :: ImageLabel).Visible = true
        return glyph :: ImageLabel
    end

    local width: number = 0
    local height: number = 0
    if atlas.ready and atlas.image then
        if next(contract._pendingAdopt) ~= nil then
            contract.flushPendingAdopt()
        end
        if record.fallback then
            (record.fallback :: TextLabel).Visible = false
        end

        local scale: number = textSize / atlas.size
        local function snapPx(value: number): number
            return math.round(value * deviceScale) / deviceScale
        end
        local rasterScale: number = textSize / raster.size
        local pixelStep: number = 1 / deviceScale
        local glyphWidth: number = math.max(
            pixelStep,
            snapPx(raster.cellWidth * rasterScale)
        )
        local glyphHeight: number = math.max(
            pixelStep,
            snapPx(raster.cellHeight * rasterScale)
        )
        local lineAdvance: number = contract.lineHeight * scale
        local shadowOffset: number = math.max(pixelStep, snapPx(2 * scale))
        local x: number = 0
        local y: number = 0
        local lineCount: number = 1
        local truncated: boolean = false
        -- Reserve ellipsis space BEFORE painting; drawing dots on top of
        -- already-painted trailing glyphs makes long option values look corrupt.
        local contentWidth: number? = maxWidth
        if maxWidth ~= nil and wantsEllipsis
            and contract.measure(segments, textSize) > maxWidth then
            contentWidth = math.max(0, maxWidth - 2 * contractAdvanceFor(atlas, 46, scale))
        end
        local lastColor: Color3 = Theme.text
        local currentSegment = 1
        local function blit(code: number, paint: Color3, dx: number, dy: number, z: number): ()
            local index: number = code - atlas.first
            local glyph: ImageLabel = nextGlyph()
            record.paintMap[used] = {segment = currentSegment, factor = z == zIndex + 1 and 0.25 or 1}
            glyph.Image = raster.image
            glyph.ImageColor3 = paint
            glyph.ImageTransparency = transparency
            glyph.ImageRectOffset = Vector2.new(
                (index % raster.columns) * raster.cellWidth,
                (index // raster.columns) * raster.cellHeight
            )
            glyph.ImageRectSize = Vector2.new(raster.cellWidth, raster.cellHeight)
            glyph.Position = UDim2.fromOffset(snapPx(dx), snapPx(dy))
            glyph.Size = UDim2.fromOffset(glyphWidth, glyphHeight)
            glyph.ZIndex = z
        end
        for segmentIndex, segment: any in ipairs(segments) do
            currentSegment = segmentIndex
            local color: Color3 = segment.color or Theme.text

            local shade: Color3 = Color3.new(
                color.R * 0.25, color.G * 0.25, color.B * 0.25
            )
            lastColor = color
            contract.eachCode(tostring(segment.text or ""), function(code: number): ()
                if truncated then return end
                if code == 10 then
                    x = 0
                    y += lineAdvance
                    lineCount += 1
                    return
                end
                local advance: number = contractAdvanceFor(atlas, code, scale)
                if contentWidth ~= nil and x + advance > contentWidth then
                    truncated = true
                    return
                end
                if code ~= 32 then
                    local mapped: number = code
                    if code < atlas.first or code > atlas.last then
                        mapped = atlas.fallbackGlyph
                    end
                    if shadow then
                        blit(mapped, shade, x + shadowOffset, y + shadowOffset, zIndex + 1)
                    end
                    blit(mapped, color, x, y, zIndex + 2)
                end
                x += advance
                width = math.max(width, x)
            end)
        end

        if truncated and wantsEllipsis then
            local dot: number = 46
            local dotAdvance: number = contractAdvanceFor(atlas, dot, scale)
            local budget: number = (maxWidth :: number)
            local dotX: number = math.min(x, budget - dotAdvance * 2)
            if dotX >= 0 then
                if shadow then
                    blit(dot, Color3.new(lastColor.R * 0.25, lastColor.G * 0.25, lastColor.B * 0.25), dotX + shadowOffset, y + shadowOffset, zIndex + 1)
                    blit(dot, Color3.new(lastColor.R * 0.25, lastColor.G * 0.25, lastColor.B * 0.25), dotX + dotAdvance + shadowOffset, y + shadowOffset, zIndex + 1)
                end
                blit(dot, lastColor, dotX, y, zIndex + 2)
                blit(dot, lastColor, dotX + dotAdvance, y, zIndex + 2)
                width = math.max(width, dotX + dotAdvance * 2)
            end
        end
        height = glyphHeight + (lineCount - 1) * lineAdvance
    else
        local composed: {string} = {}
        local estimated: number = 0
        for _, segment: any in ipairs(segments) do
            local text: string = tostring(segment.text or "")
                :gsub("&", "&amp;"):gsub("<", "&lt;"):gsub(">", "&gt;")
            local color: Color3 = segment.color or Theme.text
            table.insert(composed, string.format(
                '<font color="#%02X%02X%02X">%s</font>',
                math.floor(color.R * 255 + 0.5),
                math.floor(color.G * 255 + 0.5),
                math.floor(color.B * 255 + 0.5),
                text
            ))
            estimated += #tostring(segment.text or "")
        end
        local fallback: TextLabel? = record.fallback
        if not fallback then
            fallback = contract.make({
                Parent = holder,
                Name = "Fallback",
                BackgroundTransparency = 1,
                RichText = true,
                Size = UDim2.new(1, 0, 1, 0),
                TextColor3 = Theme.text,
            })
            record.fallback = fallback

            contract._adopted[fallback] = true
        end
        local label: TextLabel = fallback :: TextLabel
        label.Visible = true
        label.Text = table.concat(composed)
        label.TextSize = textSize
        label.TextTransparency = transparency
        label.TextXAlignment = align == "Right" and Enum.TextXAlignment.Right
            or align == "Center" and Enum.TextXAlignment.Center
            or Enum.TextXAlignment.Left
        label.ZIndex = zIndex + 1
        width = estimated * math.floor(textSize * 0.75)
        height = math.floor(textSize * 1.125)
    end

    for index: number = used + 1, #record.glyphs do
        (record.glyphs[index] :: ImageLabel).Visible = false
    end

    if maxWidth ~= nil then
        width = math.min(width, maxWidth)
    end
    holder.ClipsDescendants = maxWidth ~= nil
    holder.Size = UDim2.fromOffset(width, height)
    holder:SetAttribute("MeasuredWidth", width)
    contract.stats.draws += 1
    contract.stats.lastDrawMs = (os.clock() - startedAt) * 1000
    return holder :: Frame
end

-- Colour-only animation: no measuring, glyph layout, atlas selection or
-- holder position resets (which would interrupt entry slide tweens).
state.bitmapText.recolor = function(holder: any, colors: {Color3}): boolean
    local contract = state.bitmapText
    local record = holder and contract._live[holder]
    if not record or not record.request then return false end
    local segments = record.request.segments
    local dirty = false
    for index, segment in ipairs(segments) do
        if colors[index] and segment.color ~= colors[index] then
            segment.color = colors[index]
            dirty = true
        end
    end
    if not dirty then return true end
    record.signature = nil
    if record.fallback and record.fallback.Visible then
        local composed = {}
        for _, segment in ipairs(segments) do
            local text = tostring(segment.text or ""):gsub("&", "&amp;"):gsub("<", "&lt;"):gsub(">", "&gt;")
            local color = segment.color or Theme.text
            table.insert(composed, string.format('<font color="#%02X%02X%02X">%s</font>',
                math.floor(color.R * 255 + 0.5), math.floor(color.G * 255 + 0.5), math.floor(color.B * 255 + 0.5), text))
        end
        record.fallback.Text = table.concat(composed)
    else
        for index, binding in ipairs(record.paintMap or {}) do
            local glyph = record.glyphs[index]
            local color = segments[binding.segment].color or Theme.text
            local factor = binding.factor
            local paint = factor == 1 and color or Color3.new(color.R * factor, color.G * factor, color.B * factor)
            if glyph and glyph.Parent and glyph.ImageColor3 ~= paint then glyph.ImageColor3 = paint end
        end
    end
    contract.stats.recolors = (contract.stats.recolors or 0) + 1
    return true
end

state.bitmapText.release = function(holder: Instance?): ()
    if holder then
        state.bitmapText._live[holder] = nil
        holder:Destroy()
    end
end

state.bitmapText.adopt = function(label: any, options: any?): ()
    local contract: any = state.bitmapText
    if label == nil
        or type(label.IsA) ~= "function"
        or label:IsA("TextBox")
        or not (label:IsA("TextLabel") or label:IsA("TextButton")) then
        return
    end
    if label:GetAttribute("NoBitmap") == true then
        return
    end
    if contract._adopted[label] then
        return
    end

    if label.Parent ~= nil and contract._live[label.Parent] ~= nil then
        return
    end

    if not (contract.atlas.ready and contract.atlas.image) then
        contract._pendingAdopt[label] = options or true
        return
    end
    contract._pendingAdopt[label] = nil
    local resolvedOptions: any = options or {}
    contract._adopted[label] = true
    label.TextTransparency = 1
    label.ClipsDescendants = true
    local function paint(): ()
        if label.Parent == nil then
            return
        end
        local alignment: any = label.TextXAlignment
        contract.draw(label, {
            {text = label.Text, color = label.TextColor3},
        }, {
            Name = "Glyphs",
            TextSize = label.TextSize,
            ZIndex = (tonumber(label.ZIndex) or 1) + 1,
            align = alignment == Enum.TextXAlignment.Right and "Right"
                or alignment == Enum.TextXAlignment.Center and "Center"
                or "Left",
            shadow = resolvedOptions.shadow == true,
            maxWidth = math.max(0, label.AbsoluteSize.X),
            ellipsis = label.TextTruncate == Enum.TextTruncate.AtEnd,
        })
    end
    paint()

    pcall(function(): ()
        label:GetPropertyChangedSignal("Text"):Connect(paint)
        label:GetPropertyChangedSignal("TextColor3"):Connect(paint)
        label:GetPropertyChangedSignal("TextSize"):Connect(paint)
        label:GetPropertyChangedSignal("AbsoluteSize"):Connect(paint)
        label:GetPropertyChangedSignal("TextXAlignment"):Connect(paint)
    end)
end

state.bitmapText.flushPendingAdopt = function(): ()
    local contract: any = state.bitmapText
    if not (contract.atlas.ready and contract.atlas.image) then
        return
    end
    local parked: {Instance} = {}
    for label: any in pairs(contract._pendingAdopt) do
        table.insert(parked, label)
    end
    for _, label: any in ipairs(parked) do
        local options: any = contract._pendingAdopt[label]
        contract._pendingAdopt[label] = nil
        if label.Parent ~= nil then
            contract.adopt(label, type(options) == "table" and options or nil)
        end
    end
end

state.bitmapText.adoptTree = function(root: Instance, options: any?): any
    local contract: any = state.bitmapText
    pcall(function(): ()
        for _, descendant: Instance in ipairs(root:GetDescendants()) do
            contract.adopt(descendant, options)
        end
    end)
    local connected: boolean, connection: any = pcall(function(): any
        return root.DescendantAdded:Connect(function(descendant: Instance): ()
            contract.adopt(descendant, options)
        end)
    end)
    if connected and connection then
        return connection
    end
    return {
        Connected = false,
        Disconnect = function(): () end,
    }
end

state.bitmapText.loadExactAtlas = function(image: string, metrics: any): ()
    local atlas: any = state.bitmapText.atlas
    if type(image) ~= "string" or type(metrics) ~= "table" then
        return
    end
    for _, key: string in ipairs({
        "size", "first", "last", "columns", "cellWidth", "cellHeight", "advance",
    }) do
        if tonumber(metrics[key]) then
            atlas[key] = tonumber(metrics[key])
        end
    end

    if type(metrics.advances) == "table" then
        atlas.advances = metrics.advances
    else
        atlas.advances = nil
    end
    atlas.image = image
    atlas.ready = true
    atlas.generation += 1
    state.bitmapText.rasterFamilyEnabled = false
    state.bitmapText.source = "minecraft-exact"
    for holder: any, record: any in pairs(state.bitmapText._live) do
        if holder.Parent ~= nil and record.request ~= nil then
            pcall(function(): ()
                state.bitmapText.draw(
                    record.request.parent,
                    record.request.segments,
                    record.request.options
                )
            end)
        end
    end
    for _, listener: () -> () in ipairs(state.bitmapText.onReady) do
        pcall(listener)
    end
end

state.bitmapText._fallback = {
    size = 16, first = 32, last = 126, columns = 16,
    cellWidth = 11, cellHeight = 17, advance = 11, ascent = 15, descent = 2,
}
state.bitmapText.rasterFamilyEnabled = true

state.bitmapText.restoreFallbackAtlas = function(): ()
    local contract: any = state.bitmapText
    local atlas: any = contract.atlas
    for key: string, value: number in pairs(contract._fallback) do
        atlas[key] = value
    end
    atlas.advances = nil
    atlas.image = contract._fallbackImage
    atlas.ready = contract._fallbackImage ~= nil
    atlas.generation += 1
    contract.rasterFamilyEnabled = true
    contract.source = atlas.ready and "monocraft-fallback" or "unavailable"
end

state.bitmapText.packs = {
    ["Minecraft 1.18.1"] = {
        folder = "MinecraftFont",
        version = "1.18.1",
        source = "minecraft-exact",
    },
    ["Minecraft 26.2"] = {
        folder = "MinecraftFont-26.2",
        version = "26.2",
        source = "minecraft-26.2-local",
    },
}

state.bitmapText.probePack = function(selection: string): boolean
    local contract: any = state.bitmapText
    local pack: any = contract.packs[selection]
    if not pack then
        return false
    end
    local loaded: boolean = false
    pcall(function(): ()
        if type(isfile) ~= "function" or type(readfile) ~= "function" then
            return
        end
        local manifestPath: string = pack.folder .. "/manifest.json"
        if not isfile(manifestPath) then
            return
        end
        local manifest: any = HttpService:JSONDecode(readfile(manifestPath))
        if type(manifest) ~= "table"
            or manifest.format ~= "wurst-minecraft-font-v1"
            or manifest.version ~= pack.version
            or type(manifest.runtime) ~= "table" then
            warn("[" .. PRODUCT.logPrefix .. ":Assets] " .. pack.folder
                .. "/ is present but not a Minecraft " .. pack.version
                .. " pack; ignoring")
            return
        end
        local imagePath: string = pack.folder .. "/"
            .. tostring(manifest.runtime.image or "runtime-ascii.png")
        if not isfile(imagePath) then
            return
        end
        local registerAsset: any = fontEnvironment.getcustomasset
            or fontEnvironment.getsynasset
        if type(registerAsset) ~= "function" then
            return
        end
        local assetOk: boolean, assetId: any = pcall(registerAsset, imagePath)
        if assetOk and type(assetId) == "string" and assetId ~= "" then
            contract.loadExactAtlas(assetId, manifest.runtime)
            contract.source = pack.source
            loaded = true
        end
    end)
    return loaded
end

state.bitmapText.applyFontPack = function(selection: string): ()
    local contract: any = state.bitmapText
    state.fontPack = selection
    contract.restoreFallbackAtlas()
    if selection == "Code" then

        contract.atlas.ready = false
        contract.atlas.generation += 1
        contract.source = "code-vector"
    end
    if selection ~= "Monocraft" and selection ~= "Code"
        and contract.packs[selection] then
        if not contract.probePack(selection)
            and contract._packWarned ~= selection then
            contract._packWarned = selection
            warn("[" .. PRODUCT.logPrefix .. ":Assets] font pack \""
                .. selection
                .. "\" is not installed; rendering the Monocraft fallback (OFL)")
        end
    end
    contract.flushPendingAdopt()
    for holder: any, record: any in pairs(contract._live) do
        if holder.Parent ~= nil and record.request ~= nil then
            pcall(function(): ()
                contract.draw(
                    record.request.parent,
                    record.request.segments,
                    record.request.options
                )
            end)
        end
    end
    for _, listener: () -> () in ipairs(contract.onReady) do
        pcall(listener)
    end
end

task.spawn(function(): ()
    local contract: any = state.bitmapText

    if state.fontPack == "Code" then
        contract.source = "code-vector"
        return
    end
    if type(state.resolveAsset) ~= "function" then
        state.bitmapText.source = "unavailable"
        return
    end

    local packLoaded: boolean = false
    if state.fontPack == "Monocraft" then
        packLoaded = contract.probePack(state.fontPack)
        if packLoaded then
            print("[" .. PRODUCT.logPrefix .. ":Assets] font pack loaded: "
                .. tostring(state.fontPack))
        elseif contract._packWarned ~= state.fontPack then
            contract._packWarned = state.fontPack
            warn("[" .. PRODUCT.logPrefix .. ":Assets] font pack \""
                .. tostring(state.fontPack)
                .. "\" is not installed; rendering the Monocraft fallback (OFL)")
        end
    end

    local assetId: string?, atlasError: string? =
        state.resolveAsset("bitmapFontAtlas")
    if assetId then
        contract._fallbackImage = assetId
        if not packLoaded then
            contract.atlas.image = assetId
            contract.atlas.ready = true
            contract.atlas.generation += 1
            contract.source = "monocraft-fallback"
        end

        for _, member: any in pairs(contract.family) do
            if member ~= contract.atlas and member.asset then
                pcall(function(): ()
                    local memberId: string? = (state.resolveAsset(member.asset))
                    if memberId then
                        member.image = memberId
                        member.ready = true
                    end
                end)
            end
        end
    elseif not packLoaded then
        contract.source = "unavailable"
        warn("[" .. PRODUCT.logPrefix .. ":Assets] no bitmap atlas ("
            .. tostring(atlasError)
            .. "); text renders on the vector stand-in")
        return
    end
    contract.flushPendingAdopt()
    for holder: any, record: any in pairs(contract._live) do
        if holder.Parent ~= nil and record.request ~= nil then
            pcall(function(): ()
                contract.draw(
                    record.request.parent,
                    record.request.segments,
                    record.request.options
                )
            end)
        end
    end
    for _, listener: () -> () in ipairs(contract.onReady) do
        pcall(listener)
    end
end)

state.onLayout(function(): ()
    for holder: any, record: any in pairs(state.bitmapText._live) do
        if holder.Parent ~= nil and record.request ~= nil then
            pcall(function(): ()
                state.bitmapText.draw(
                    record.request.parent,
                    record.request.segments,
                    record.request.options
                )
            end)
        end
    end
end)

state.registeredTextIndex = setmetatable({}, {__mode = "k"}) :: any

local function registerText(
    object: TextLabel | TextButton | TextBox,
    normalSize: number
): TextLabel | TextButton | TextBox
    local entry: any = {
```

```
        object = object,
        normalSize = normalSize,
    }
    table.insert(registeredText, entry)
    state.registeredTextIndex[object] = entry

    object.TextSize = math.round(
        normalSize * TEXT_BASE_MULTIPLIER * state.textScale
    )
    return object
end

local function fitTextSize(object: any, requested: number): number
    local height: number = object.AbsoluteSize.Y
    if height <= 0 then
        local vertical: UDim = object.Size.Y
        height = vertical.Scale == 0 and vertical.Offset or 0
    end
    if height <= 0 then
        return requested
    end
    return math.max(7, math.min(requested, math.floor((height - 2) / 1.2)))
end
state.fitTextSize = fitTextSize

state.setTextMetric = function(object: any, normalSize: number): ()
    local entry: any = state.registeredTextIndex[object]
    if entry then
        entry.normalSize = normalSize
    end
    object.TextSize = fitTextSize(
        object,
        math.round(normalSize * TEXT_BASE_MULTIPLIER * state.textScale)
    )
end

local function makeTextLabel(
    parent: Instance,
    text: string,
    normalSize: number?
): TextLabel

    return registerText(create("TextLabel", {
        Parent = parent,
        BackgroundTransparency = 1,
        BorderSizePixel = 0,
        ClipsDescendants = true,
        FontFace = THIN_FONT,
        Text = text,
        TextColor3 = Theme.text,
        TextTruncate = Enum.TextTruncate.AtEnd,
        TextXAlignment = Enum.TextXAlignment.Left,
        TextYAlignment = Enum.TextYAlignment.Center,
    }), normalSize or 14) :: TextLabel
end

local function makeButton(
    parent: Instance,
    text: string,
    normalSize: number?
): TextButton
    local isDefaultStyle: boolean = state.guiStyle == "Default"
    local button = registerText(create("TextButton", {
        Parent = parent,
        AutoButtonColor = false,
        ClipsDescendants = true,
        TextTruncate = Enum.TextTruncate.AtEnd,
        BackgroundColor3 = isDefaultStyle
            and Theme.button
            or DEFAULT_SURFACE_COLOR,
        BorderSizePixel = 0,
        FontFace = CONTROL_FONT,
        Text = text,
        TextColor3 = isDefaultStyle
            and Theme.text
            or Theme.text,
        TextXAlignment = Enum.TextXAlignment.Center,
        TextYAlignment = Enum.TextYAlignment.Center,
    }), normalSize or 14)

    addRoundedStyle(button, 9, 0.48)
    button:SetAttribute(
        "IdleColor",
        isDefaultStyle and Theme.button or DEFAULT_SURFACE_COLOR
    )
    button:SetAttribute(
        "HoverColor",
        isDefaultStyle and Theme.buttonHover or Theme.surfaceHover
    )
    button:SetAttribute(
        "PressedColor",
        isDefaultStyle and Theme.buttonPressed or Theme.surfacePressed
    )
    table.insert(state.styledButtons, button)

    trackUiConnection(button.MouseEnter:Connect(function()
        if button:GetAttribute("NoHover") or button:GetAttribute("Disabled") then
            return
        end
        local configuredHover: any = button:GetAttribute("HoverColor")
        local hoverColor: Color3 = typeof(configuredHover) == "Color3"
            and configuredHover
            or Theme.surfaceHover
        TweenService:Create(
            button,
            TweenInfo.new(UI_MOTION.base, Enum.EasingStyle.Quad, Enum.EasingDirection.Out),
            {BackgroundColor3 = hoverColor}
        ):Play()
    end))

    trackUiConnection(button.MouseLeave:Connect(function()
        if button:GetAttribute("NoHover") then
            return
        end
        local selectedColor: any = button:GetAttribute("SelectedColor")
        local configuredIdle: any = button:GetAttribute("IdleColor")
        local targetColor: Color3 = button:GetAttribute("Selected")
            and (typeof(selectedColor) == "Color3" and selectedColor or Theme.accentSoft)
            or (typeof(configuredIdle) == "Color3"
                and configuredIdle
                or DEFAULT_SURFACE_COLOR)

        TweenService:Create(
            button,
            TweenInfo.new(UI_MOTION.base, Enum.EasingStyle.Quad, Enum.EasingDirection.Out),
            {BackgroundColor3 = targetColor}
        ):Play()
    end))

    trackUiConnection(button.MouseButton1Down:Connect(function(): ()
        if button:GetAttribute("Disabled")
            or button:GetAttribute("NoPressEffect") then
            return
        end
        local configuredPressed: any = button:GetAttribute("PressedColor")
        local pressedColor: Color3 = typeof(configuredPressed) == "Color3"
            and configuredPressed
            or Theme.surfacePressed
        TweenService:Create(
            button,
            TweenInfo.new(0.08, Enum.EasingStyle.Quad),
            {BackgroundColor3 = pressedColor}
        ):Play()
    end))

    trackUiConnection(button.MouseButton1Up:Connect(function(): ()
        if button:GetAttribute("Disabled")
            or button:GetAttribute("NoPressEffect") then
            return
        end
        local selectedColor: any = button:GetAttribute("SelectedColor")
        local configuredHover: any = button:GetAttribute("HoverColor")
        local targetColor: Color3 = button:GetAttribute("Selected")
            and (typeof(selectedColor) == "Color3" and selectedColor or Theme.accentSoft)
            or (typeof(configuredHover) == "Color3"
                and configuredHover
                or Theme.surfaceHover)
        TweenService:Create(
            button,
            TweenInfo.new(UI_MOTION.fast, Enum.EasingStyle.Quad),
            {BackgroundColor3 = targetColor}
        ):Play()
    end))

    return button :: TextButton
end

local function setToggleSwitch(button: TextButton, enabled: boolean, animate: boolean?): ()
    button:SetAttribute("Checked", enabled)
    local frame: Frame? = button:FindFirstChild("SwitchTrack") :: Frame?
    local core: Frame? = frame and frame:FindFirstChild("Knob") :: Frame?
    if not frame or not core then
        return
    end
    local stroke: UIStroke? = frame:FindFirstChild("MarkStroke") :: UIStroke?
    local targetSize: UDim2 = enabled
        and UDim2.fromScale(0.62, 0.62)
        or UDim2.fromScale(0, 0)
    local targetTransparency: number = enabled and 0.05 or 1
    if animate and state.animationsEnabled then
        TweenService:Create(core, TweenInfo.new(UI_MOTION.base, Enum.EasingStyle.Quart), {
            Size = targetSize,
            BackgroundTransparency = targetTransparency,
        }):Play()
    else
        core.Size = targetSize
        core.BackgroundTransparency = targetTransparency
    end
    if stroke then
        stroke.Transparency = enabled and 0.15 or 0.45
    end
end

local function createToggleSwitch(
    parent: Instance,
    enabled: boolean,
    width: number?
): TextButton
    local markSize: number = math.clamp(width or 15, 12, 18)
    local button: TextButton = create("TextButton", {
        TextTruncate = Enum.TextTruncate.AtEnd,
        ClipsDescendants = true,
        Parent = parent,
        Active = true,
        AutoButtonColor = false,
        BackgroundTransparency = 1,
        BorderSizePixel = 0,

        Size = UDim2.fromOffset(math.max(markSize + 14, 28), 30),
        Text = "",
    }) :: TextButton
    local frame: Frame = create("Frame", {
        Parent = button,
        Name = "SwitchTrack",
        AnchorPoint = Vector2.new(0.5, 0.5),
        BackgroundColor3 = Theme.text,
        BackgroundTransparency = 1,
        BorderSizePixel = 0,
        Position = UDim2.fromScale(0.5, 0.5),
        Size = UDim2.fromOffset(markSize, markSize),
        ZIndex = button.ZIndex + 1,
    }) :: Frame
    create("UIStroke", {
        Parent = frame,
        Name = "MarkStroke",
        Color = Theme.outline,
        Transparency = 0.45,
        Thickness = 1,
    })

    local _core: Frame = create("Frame", {
        Parent = frame,
        Name = "Knob",
        AnchorPoint = Vector2.new(0.5, 0.5),
        BackgroundColor3 = Theme.text,
        BackgroundTransparency = 1,
        BorderSizePixel = 0,
        Position = UDim2.fromScale(0.5, 0.5),
        Size = UDim2.fromScale(0, 0),
        ZIndex = frame.ZIndex + 1,
    }) :: Frame
    setToggleSwitch(button, enabled, false)
    return button
end

local function applyKeySlotStyle(button: TextButton): ()
    button.ZIndex = math.max(2, button.ZIndex)
    button.BackgroundColor3 = Theme.surface
    button.BackgroundTransparency = 0.54
    button.TextColor3 = Theme.textMuted
    button:SetAttribute("ThemeRole", "KeySlot")
    button:SetAttribute("IdleColor", Theme.surface)
    button:SetAttribute("HoverColor", Theme.surfaceHover)
    button:SetAttribute("PressedColor", Theme.buttonPressed)
    local buttonStroke: UIStroke? = button:FindFirstChildOfClass("UIStroke")
    if buttonStroke then
        buttonStroke.Color = Theme.outline
        buttonStroke.Transparency = 0.78
    end
end

state.applyLinesGlyph = function(
    button: TextButton,
    dimmed: boolean,
    active: boolean?
): ()
    button.Text = ""
    local existing: Instance? = button:FindFirstChild("LinesGlyph")
    local glyph: Frame
    if existing and existing:IsA("Frame") then
        glyph = existing :: Frame
    else
        glyph = create("Frame", {
            Parent = button,
            Name = "LinesGlyph",
            AnchorPoint = Vector2.new(0.5, 0.5),
            BackgroundTransparency = 1,
            BorderSizePixel = 0,
            Position = UDim2.fromScale(0.5, 0.5),
            Size = UDim2.fromOffset(12, 8),
            ZIndex = button.ZIndex + 1,
        }) :: Frame
        for index: number = 1, 3 do
            local _bar: Frame = create("Frame", {
                Parent = glyph,
                Name = "Line" .. tostring(index),
                AnchorPoint = Vector2.new(0.5, 0.5),
                BackgroundColor3 = Theme.text,
                BorderSizePixel = 0,
                Position = UDim2.new(0.5, 0, 0, (index - 1) * 3.5 + 0.5),

                Size = UDim2.fromOffset(index == 2 and 9 or 12, 1),
                ZIndex = button.ZIndex + 2,
            }) :: Frame
        end
    end
    local barTransparency: number = dimmed and 0.74 or (active and 0.02 or 0.3)
    for _, bar: Instance in ipairs(glyph:GetChildren()) do
        if bar:IsA("Frame") then
            bar.BackgroundTransparency = barTransparency
        end
    end
end

state.restyleButtons = function(isDefault: boolean): ()
    for _, button: TextButton in ipairs(state.styledButtons) do
        if button.Parent then
            local role: any = button:GetAttribute("ThemeRole")
            if role ~= "Navigation" and role ~= "Icon" then
                local idleColor: Color3 = isDefault
                    and Theme.button
                    or DEFAULT_SURFACE_COLOR
                button:SetAttribute("IdleColor", idleColor)
                button:SetAttribute(
                    "HoverColor",
                    isDefault and Theme.buttonHover or Theme.surfaceHover
                )
                button:SetAttribute(
                    "PressedColor",
                    isDefault and Theme.buttonPressed or Theme.surfacePressed
                )
                if button:GetAttribute("Selected") ~= true then
                    button.BackgroundColor3 = idleColor
                end
                button.TextColor3 = isDefault and Theme.text or Theme.text
                local stroke: UIStroke? =
                    button:FindFirstChild("StyleStroke") :: UIStroke?
                if stroke then
                    stroke.Enabled = true
                    stroke.Color = isDefault and Theme.outline or Theme.outline
                    stroke.Transparency = isDefault and 0.32 or 0.48
                end
                if role == "KeySlot" then
                    button.BackgroundColor3 = Theme.surface
                    button.BackgroundTransparency = 0.54
                    button.TextColor3 = Theme.textMuted
                    local keyStroke: UIStroke? =
                        button:FindFirstChild("StyleStroke") :: UIStroke?
                    if keyStroke then
                        keyStroke.Color = Theme.outline
                        keyStroke.Transparency = 0.78
                    end
                end
            end
        end
    end
end

local KEY_DISPLAY_NAMES: {[string]: string} = {
    RightShift = "RShift", LeftShift = "LShift",
    RightControl = "RCtrl", LeftControl = "LCtrl",
    RightAlt = "RAlt", LeftAlt = "LAlt",
    Backspace = "Back", CapsLock = "Caps",
    Return = "Enter", Escape = "Esc",
    PageUp = "PgUp", PageDown = "PgDn",
    Insert = "Ins", Delete = "Del",
    RightMeta = "RMeta", LeftMeta = "LMeta",
}
state.keyDisplayName = function(name: any): string
    local resolved: string = tostring(name or "")
    return KEY_DISPLAY_NAMES[resolved] or resolved
end

local function setKeySlotCapture(button: TextButton, active: boolean): ()
    local stroke: UIStroke? = button:FindFirstChild("StyleStroke") :: UIStroke?
    TweenService:Create(
        button,
        TweenInfo.new(UI_MOTION.fast, Enum.EasingStyle.Quad, Enum.EasingDirection.Out),
        {
            BackgroundColor3 = active
                and Theme.buttonPressed
                or Theme.surface,
            TextColor3 = active
                and Theme.text
                or Theme.textMuted,
        }
    ):Play()
    if stroke then
        TweenService:Create(
            stroke,
            TweenInfo.new(UI_MOTION.fast, Enum.EasingStyle.Quad, Enum.EasingDirection.Out),
            {Transparency = active and 0.38 or 0.78}
        ):Play()
    end
end

local function beginKeyCapture(
    button: TextButton,
    callback: (Enum.KeyCode) -> ()
): ()
    if state.waitingForKey then
        state.waitingForKey = false
        if state.keybindButton then
            state.keybindButton.Text = state.keyDisplayName(state.toggleKey.Name)
            setKeySlotCapture(state.keybindButton, false)
        end
    end
    if state.keyCaptureCancel then
        state.keyCaptureCancel()
    end
    button.Text = "..."
    setKeySlotCapture(button, true)
    local cancelled: boolean = false
    state.keyCaptureCancel = function(): ()
        if cancelled then
            return
        end
        cancelled = true
        state.keyCaptureCancel = nil
        state.keyCaptureCallback = nil
        setKeySlotCapture(button, false)
        if button.Parent and button:GetAttribute("CaptureIdleText") then
            button.Text = tostring(button:GetAttribute("CaptureIdleText"))
        end
    end
    state.keyCaptureCallback = function(keyCode: Enum.KeyCode): ()
        if cancelled then
            return
        end
        cancelled = true
        state.keyCaptureCancel = nil
        setKeySlotCapture(button, false)
        callback(keyCode)
    end
end

local function makeTextBox(parent: Instance, defaultText: any): TextBox
    local isDefaultStyle: boolean = state.guiStyle == "Default"
    local textBox: TextBox = (registerText(create("TextBox", {
        Parent = parent,
        BackgroundColor3 = isDefaultStyle
            and Theme.buttonPressed
            or Theme.surface,
        BorderSizePixel = 0,
        ClearTextOnFocus = false,

        ClipsDescendants = true,
        TextTruncate = Enum.TextTruncate.AtEnd,
        FontFace = THIN_FONT,
        PlaceholderColor3 = isDefaultStyle
            and Theme.textMuted
            or Theme.textMuted,
        Text = tostring(defaultText),
        TextColor3 = isDefaultStyle
            and Theme.text
            or Theme.text,
        TextXAlignment = Enum.TextXAlignment.Center,
    }), 13) :: any) :: TextBox
    addRoundedStyle(textBox, 9, 0.5)
    table.insert(state.styledTextBoxes, textBox)
    trackUiConnection(textBox.Focused:Connect(function(): ()
        TweenService:Create(
            textBox,
            TweenInfo.new(UI_MOTION.base, Enum.EasingStyle.Quad),
            {BackgroundColor3 = state.guiStyle == "Default"
                and Theme.buttonHover
                or Theme.surfaceRaised}
        ):Play()
    end))
    trackUiConnection(textBox.FocusLost:Connect(function(): ()
        TweenService:Create(
            textBox,
            TweenInfo.new(UI_MOTION.base, Enum.EasingStyle.Quad),
            {BackgroundColor3 = state.guiStyle == "Default"
                and Theme.buttonPressed
                or Theme.surface}
        ):Play()
    end))
    return textBox
end

state.restyleTextBoxes = function(isDefault: boolean): ()
    for _, textBox: TextBox in ipairs(state.styledTextBoxes) do
        if textBox.Parent then
            textBox.BackgroundColor3 = isDefault
                and Theme.buttonPressed
                or Theme.surface
            textBox.TextColor3 = isDefault and Theme.text or Theme.text
            textBox.PlaceholderColor3 = isDefault
                and Theme.textMuted
                or Theme.textMuted
            local stroke: UIStroke? =
                textBox:FindFirstChild("StyleStroke") :: UIStroke?
            if stroke then
                stroke.Color = isDefault and Theme.outline or Theme.outline
                stroke.Transparency = isDefault and 0.3 or 0.5
            end
        end
    end
end

local ToastContainer: Frame = (create("Frame", {

    Parent = ScreenGui,
    Name = "ToastContainer",
    AnchorPoint = Vector2.new(1, 1),
    BackgroundTransparency = 1,
    BorderSizePixel = 0,
    Position = UDim2.new(1, -12, 1, -12),
    Size = UDim2.fromOffset(state.layout.compact and 220 or 300, 300),
    ZIndex = 300,
}) :: any) :: Frame

state.onLayout(function(metrics: LayoutMetrics): ()
    ToastContainer.Size = UDim2.fromOffset(metrics.compact and 220 or 300, 300)
end)

create("UIListLayout", {
    Parent = ToastContainer,
    FillDirection = Enum.FillDirection.Vertical,
    HorizontalAlignment = Enum.HorizontalAlignment.Right,
    Padding = UDim.new(0, 8),
    SortOrder = Enum.SortOrder.LayoutOrder,
    VerticalAlignment = Enum.VerticalAlignment.Bottom,
})

local toastSequence: number = 0
local lastToastText: string = ""
local lastToastAt: number = -math.huge

-- v191 notification pass: the toast is the only channel the player reads
-- during a round, so it now
--   * stays on screen long enough to read a long message (length-based
--     lifetime instead of a flat 3.1 s),
--   * never stacks into a wall that covers the play area (oldest toast is
--     retired once the stack is full), and
--   * paints its accent bar by kind ("danger" red, "good" green, otherwise
--     the theme accent), so a warning does not look like the fifth "Loaded"
--     line of the session.
local TOAST_STACK_LIMIT: number = 5
local activeToasts: {any} = {}

local function toastAccent(kind: string?): Color3
    if kind == "danger" then
        return Theme.negative
    end
    if kind == "good" then
        return Theme.enabled
    end
    return Theme.accent
end

local function toastLifetime(text: string): number
    -- ~55 characters per second of comfortable reading, floor 2.6 s and a
    -- ceiling so a stack cannot pile up behind one long message.
    return math.clamp(2.6 + #text * 0.022, 2.6, 7)
end

local function retireToast(toast: any): ()
    for index: number = #activeToasts, 1, -1 do
        if activeToasts[index] == toast then
            table.remove(activeToasts, index)
        end
    end
end

local function notify(message: any, kind: string?): ()
    local text: string = tostring(message)
    local now = os.clock()
    if text == lastToastText and now - lastToastAt < 3 then return end
    lastToastText, lastToastAt = text, now
    print("[" .. PRODUCT.name .. "] " .. text)
    toastSequence += 1

    local toast: CanvasGroup = Instance.new("CanvasGroup")
    toast.Name = "Toast"
    toast.AutomaticSize = Enum.AutomaticSize.Y
    toast.BackgroundColor3 = state.guiStyle == "Default"
        and Theme.surface
        or Theme.surfaceRaised
    toast.BackgroundTransparency = state.guiStyle == "Default" and 0.12 or 0.04
    toast.BorderSizePixel = 0
    toast.GroupTransparency = 1
    toast.LayoutOrder = toastSequence
    toast.Size = UDim2.new(1, 0, 0, state.layout.compact and 46 or 50)
    toast.ZIndex = 301
    toast.Parent = ToastContainer

    create("UICorner", {
        Parent = toast,
        CornerRadius = cornerRadius(UI_RADIUS.panel),
    })
    create("UIStroke", {
        Parent = toast,
        Color = state.guiStyle == "Default"
            and Theme.outline
            or Theme.outline,
        Transparency = state.guiStyle == "Default" and 0.42 or 0.22,
        Thickness = 1,
    })
    local _accent: Frame = (create("Frame", {
        Parent = toast,
        BackgroundColor3 = toastAccent(kind),
        BorderSizePixel = 0,
        Size = UDim2.new(0, 4, 1, 0),
        ZIndex = 302,
    }) :: any) :: Frame

    local label: TextLabel = (makeTextLabel(toast, text, 14) :: any)
        :: TextLabel
    label.AutomaticSize = Enum.AutomaticSize.Y
    label.Position = UDim2.fromOffset(16, 10)
    label.Size = UDim2.new(1, -28, 0, 36)
    label.TextColor3 = state.guiStyle == "Default"
        and Theme.text
        or Theme.text
    label.TextWrapped = true
    label.ZIndex = 303

    TweenService:Create(
        toast,
        TweenInfo.new(0.22, Enum.EasingStyle.Quart, Enum.EasingDirection.Out),
        {GroupTransparency = 0}
    ):Play()
    table.insert(activeToasts, toast)
    while #activeToasts > TOAST_STACK_LIMIT do
        local oldest: any = table.remove(activeToasts, 1)
        if oldest and oldest ~= toast and oldest.Parent then
            pcall(function(): ()
                oldest:Destroy()
            end)
        end
    end
    task.delay(toastLifetime(text), function(): ()
        retireToast(toast)
        if not toast.Parent then
            return
        end
        local tween: Tween = TweenService:Create(
            toast,
            TweenInfo.new(
                0.2,
                Enum.EasingStyle.Quad,
                Enum.EasingDirection.In
            ),
            {GroupTransparency = 1}
        )
        tween:Play()
        tween.Completed:Once(function(): ()
            toast:Destroy()
        end)
    end)
end

-- A small centred modal with two answers. The MM2 calibration prompt uses it so
-- nothing is written to the executor workspace before the player says Yes.
-- Deliberately tiny: one question, two buttons, no scrolling and no state.
local function confirmDialog(options: any): Frame?
    if type(options) ~= "table" then
        return nil
    end
    local title: string = tostring(options.title or "Question")
    local message: string = tostring(options.message or "")
    local yesText: string = tostring(options.yes or "Yes")
    local noText: string = tostring(options.no or "No")
    local onYes: any = options.onYes
    local onNo: any = options.onNo
    local rememberLabel: any = options.rememberLabel
    local remember: boolean = options.rememberDefault == true

    -- PopupLayer has the menu's UIScale and is anchored at the top left.
    -- A 50% position inside it therefore is not the viewport centre when the
    -- menu is scaled. Put the modal directly in the unscaled ScreenGui.
    local previous: Instance? = ScreenGui:FindFirstChild("ConfirmDialog")
```

```
    while previous do
        previous:Destroy()
        previous = ScreenGui:FindFirstChild("ConfirmDialog")
    end

    local backdrop: TextButton = (create("TextButton", {
        Parent = ScreenGui,
        Name = "ConfirmDialog",
        Text = "",
        AutoButtonColor = false,
        Active = true,
        BackgroundColor3 = Color3.new(0, 0, 0),
        BackgroundTransparency = 0.4,
        BorderSizePixel = 0,
        Size = UDim2.fromScale(1, 1),
        ZIndex = 400,
    }) :: any) :: TextButton

    local panel: Frame = (create("Frame", {
        Parent = backdrop,
        Name = "Panel",
        Active = true,
        AnchorPoint = Vector2.new(0.5, 0.5),
        Position = UDim2.fromScale(0.5, 0.5),
        Size = UDim2.new(1, -32, 0, 0),
        AutomaticSize = Enum.AutomaticSize.Y,
        BackgroundColor3 = Theme.surface,
        BorderSizePixel = 0,
        ZIndex = 401,
    }) :: any) :: Frame
    create("UISizeConstraint", {
        Parent = panel,
        MaxSize = Vector2.new(340, 10000),
    })
    create("UICorner", {
        Parent = panel,
        CornerRadius = cornerRadius(UI_RADIUS.panel),
    })
    create("UIStroke", {
        Parent = panel,
        Color = Theme.outline,
        Thickness = 1,
    })
    create("UIListLayout", {
        Parent = panel,
        FillDirection = Enum.FillDirection.Vertical,
        SortOrder = Enum.SortOrder.LayoutOrder,
        Padding = UDim.new(0, 6),
    })
    create("UIPadding", {
        Parent = panel,
        PaddingTop = UDim.new(0, 14),
        PaddingBottom = UDim.new(0, 14),
        PaddingLeft = UDim.new(0, 16),
        PaddingRight = UDim.new(0, 16),
    })

    local titleLabel: TextLabel = (create("TextLabel", {
        Parent = panel,
        Name = "Title",
        LayoutOrder = 1,
        BackgroundTransparency = 1,
        FontFace = CONTROL_FONT,
        Text = title,
        TextColor3 = Theme.text,
        TextSize = 15,
        TextWrapped = true,
        TextXAlignment = Enum.TextXAlignment.Left,
        Size = UDim2.new(1, 0, 0, 20),
        ZIndex = 402,
    }) :: any) :: TextLabel

    local messageLabel: TextLabel = (create("TextLabel", {
        Parent = panel,
        Name = "Message",
        LayoutOrder = 2,
        AutomaticSize = Enum.AutomaticSize.Y,
        BackgroundTransparency = 1,
        FontFace = CONTROL_FONT,
        Text = message,
        TextColor3 = Theme.textMuted,
        TextSize = 13,
        TextWrapped = true,
        TextXAlignment = Enum.TextXAlignment.Left,
        TextYAlignment = Enum.TextYAlignment.Top,
        Size = UDim2.new(1, 0, 0, 0),
        ZIndex = 402,
    }) :: any) :: TextLabel

    if type(rememberLabel) == "string" and rememberLabel ~= "" then
        local row: TextButton = (create("TextButton", {
            Parent = panel,
            Name = "RememberChoice",
            LayoutOrder = 3,
            Size = UDim2.new(1, 0, 0, 26),
            BackgroundTransparency = 1,
            AutoButtonColor = false,
            Text = "",
            ZIndex = 402,
        }) :: any) :: TextButton
        local box: Frame = (create("Frame", {
            Parent = row,
            Name = "Box",
            AnchorPoint = Vector2.new(0, 0.5),
            Position = UDim2.new(0, 0, 0.5, 0),
            Size = UDim2.fromOffset(16, 16),
            BackgroundColor3 = Theme.button,
            BorderSizePixel = 0,
            ZIndex = 403,
        }) :: any) :: Frame
        create("UICorner", {Parent = box, CornerRadius = cornerRadius(UI_RADIUS.control)})
        create("UIStroke", {Parent = box, Color = Theme.outline, Thickness = 1})
        local mark: TextLabel = (create("TextLabel", {
            Parent = box,
            Name = "Check",
            BackgroundTransparency = 1,
            Size = UDim2.fromScale(1, 1),
            FontFace = CONTROL_FONT,
            Text = "✓",
            TextColor3 = Theme.accentText,
            TextSize = 14,
            Visible = remember,
            ZIndex = 404,
        }) :: any) :: TextLabel
        create("TextLabel", {
            Parent = row,
            Name = "Label",
            BackgroundTransparency = 1,
            Position = UDim2.fromOffset(24, 0),
            Size = UDim2.new(1, -24, 1, 0),
            FontFace = CONTROL_FONT,
            Text = rememberLabel,
            TextColor3 = Theme.text,
            TextSize = 13,
            TextXAlignment = Enum.TextXAlignment.Left,
            ZIndex = 403,
        })
        local function refreshRemember(): ()
            mark.Visible = remember
            box.BackgroundColor3 = remember and Theme.accent or Theme.button
        end
        refreshRemember()
        row.MouseButton1Click:Connect(function(): ()
            remember = not remember
            refreshRemember()
        end)
    end

    local actions: Frame = (create("Frame", {
        Parent = panel,
        Name = "Actions",
        LayoutOrder = 4,
        BackgroundTransparency = 1,
        Size = UDim2.new(1, 0, 0, 30),
        ZIndex = 402,
    }) :: any) :: Frame
    create("UIListLayout", {
        Parent = actions,
        FillDirection = Enum.FillDirection.Horizontal,
        HorizontalAlignment = Enum.HorizontalAlignment.Right,
        VerticalAlignment = Enum.VerticalAlignment.Center,
        SortOrder = Enum.SortOrder.LayoutOrder,
        Padding = UDim.new(0, 8),
    })

    local answered: boolean = false
    local function close(answer: boolean?): ()
        if answered then
            return
        end
        answered = true
        if backdrop.Parent then
            backdrop:Destroy()
        end
        local callback: any = answer == true and onYes or onNo
        if type(callback) == "function" then
            local ok: boolean, failure: any = pcall(callback, remember)
            if not ok then
                warn("[" .. PRODUCT.name .. "] confirm dialog: " .. tostring(failure))
            end
        end
    end

    local function answerButton(text: string, order: number, primary: boolean): TextButton
        local button: TextButton = (create("TextButton", {
            Parent = actions,
            Name = text,
            LayoutOrder = order,
            AutoButtonColor = false,
            BackgroundColor3 = primary and Theme.accent or Theme.button,
            BorderSizePixel = 0,
            Size = UDim2.fromOffset(primary and 92 or 78, 28),
            Text = text,
            FontFace = CONTROL_FONT,
            TextColor3 = primary and Theme.accentText or Theme.text,
            TextSize = 13,
            ZIndex = 403,
        }) :: any) :: TextButton
        create("UICorner", {
            Parent = button,
            CornerRadius = cornerRadius(UI_RADIUS.control),
        })
        button.MouseButton1Click:Connect(function(): ()
            close(primary)
        end)
        return button
    end

    -- No sits on the left, the answer that writes files on the right.
    answerButton(noText, 1, false)
    answerButton(yesText, 2, true)
    return panel
end

state.confirmDialog = confirmDialog

local CardBin: Frame = (create("Frame", {
    Parent = ScreenGui,
    Name = "CardBin",
    BackgroundTransparency = 1,
    BorderSizePixel = 0,
    Size = UDim2.fromScale(1, 1),
    Visible = false,
}) :: any) :: Frame

state.universalScroll = CardBin
state.trsScroll = CardBin
state.vdScroll = CardBin
state.mvsdScroll = CardBin
state.bedFightScroll = CardBin
state.spoofScroll = CardBin

print("[" .. PRODUCT.logPrefix .. ":Bootstrap] Base GUI visible; building universal features")

local universalFeatures = {}
local movementFeatures = {}

state.movementFeatures = movementFeatures
local mm2Features = {}
state.trsFeatures = {}

state.spoofFeatures = {}
state.vdFeatures = {}
state.bedFightFeatures = {}
state.bedWarsFeatures = {}
state.mvsdFeatures = {}
local allFeatures: {any} = {}
local featureConnections = {}

state.invokeModuleSearch = function(query: string): boolean
    local search: any = state.moduleSearch
    if type(search) ~= "table" or type(search.Execute) ~= "function" then
        return false
    end
    local result: any = search.Execute(query)
    return type(result) == "table" and result.status == "success"
end

state.onLayout(function(): ()
    for _, feature: any in ipairs(allFeatures) do
        if type(feature.applyLayout) == "function" then
            local ok: boolean, failure: any = pcall(feature.applyLayout)
            if not ok then
                warn("[" .. PRODUCT.name .. "] card layout: " .. tostring(failure))
            end
        end
    end
end)
state.tooltipConnections = {} :: {RBXScriptConnection}
local MobileMenuButton = create("TextButton", {
    TextTruncate = Enum.TextTruncate.AtEnd,
    ClipsDescendants = true,
    Parent = ScreenGui,
    Name = "MobileMenuButton",
    AutoButtonColor = false,
    BackgroundColor3 = Theme.surface,
    BackgroundTransparency = 0.12,
    BorderSizePixel = 0,
    FontFace = CONTROL_FONT,
    Position = UDim2.new(1, -18, 0, 18),
    AnchorPoint = Vector2.new(1, 0),
    Size = UDim2.fromOffset(52, 52),
    Text = "≡",
    TextColor3 = Theme.text,
    TextSize = 20,
    -- Hidden until the whole menu has finished loading: showing it at injection
    -- flashed the old circular launcher for a few seconds before the GUI existed.
    Visible = false,
    ZIndex = 190,
}) :: TextButton
create("UICorner", {
    Parent = MobileMenuButton,
    CornerRadius = UDim.new(1, 0),
})
create("UIStroke", {
    Parent = MobileMenuButton,
    Color = Theme.outline,
    Transparency = 0.55,
    Thickness = 1,
})

do
    local dragInput: InputObject? = nil
    local dragStart: Vector2 = Vector2.zero
    local startPosition: UDim2 = MobileMenuButton.Position
    local travelled: number = 0

    local storedX: number? = tonumber(configData.ui.launcherX)
    local storedY: number? = tonumber(configData.ui.launcherY)
    if storedX and storedY then
        MobileMenuButton.Position = UDim2.fromOffset(storedX, storedY)
        MobileMenuButton.AnchorPoint = Vector2.new(0, 0)
    end

    local launcherScale: UIScale = create("UIScale", {
        Parent = MobileMenuButton,
        Name = "LauncherScale",
        Scale = 1,
    }) :: UIScale

    local function pressLauncher(pressed: boolean): ()
        if not state.animationsEnabled then
            launcherScale.Scale = 1
            return
        end
        TweenService:Create(
            launcherScale,
            TweenInfo.new(
                pressed and UI_MOTION.fast or 0.34,
                pressed and Enum.EasingStyle.Quart or Enum.EasingStyle.Back,
                Enum.EasingDirection.Out
            ),
            {Scale = pressed and 0.9 or 1}
        ):Play()
    end

    trackUiConnection(MobileMenuButton.InputBegan:Connect(function(
        input: InputObject
    ): ()
        if input.UserInputType ~= Enum.UserInputType.MouseButton1
            and input.UserInputType ~= Enum.UserInputType.Touch then
            return
        end
        dragInput = input
        dragStart = Vector2.new(input.Position.X, input.Position.Y)
        startPosition = MobileMenuButton.Position
        travelled = 0
        pressLauncher(true)
    end))

    trackUiConnection(UserInputService.InputChanged:Connect(function(
        input: InputObject
    ): ()
        if not dragInput then
            return
        end
        if input.UserInputType ~= Enum.UserInputType.MouseMovement
            and input.UserInputType ~= Enum.UserInputType.Touch then
            return
        end
        local pointer: Vector2 = Vector2.new(input.Position.X, input.Position.Y)
        local delta: Vector2 = pointer - dragStart
        travelled = math.max(travelled, delta.Magnitude)
        if travelled < 6 then

            return
        end

        MobileMenuButton.AnchorPoint = Vector2.new(0, 0)
        local viewport: Vector2 = ScreenGui.AbsoluteSize
        local size: Vector2 = MobileMenuButton.AbsoluteSize
        local baseX: number = startPosition.X.Scale * viewport.X + startPosition.X.Offset
        local baseY: number = startPosition.Y.Scale * viewport.Y + startPosition.Y.Offset
        MobileMenuButton.Position = UDim2.fromOffset(
            math.clamp(math.round(baseX + delta.X), 0, math.max(viewport.X - size.X, 0)),
            math.clamp(math.round(baseY + delta.Y), 0, math.max(viewport.Y - size.Y, 0))
        )
    end))

    trackUiConnection(UserInputService.InputEnded:Connect(function(
        input: InputObject
    ): ()
        if input ~= dragInput then
            return
        end
        dragInput = nil
        pressLauncher(false)
        if travelled >= 6 then

            local viewport: Vector2 = ScreenGui.AbsoluteSize
            local size: Vector2 = MobileMenuButton.AbsoluteSize
            local currentX: number = MobileMenuButton.Position.X.Offset
            local currentY: number = MobileMenuButton.Position.Y.Offset
            local margin: number = 14
            local nearestX: number = currentX + size.X * 0.5 < viewport.X * 0.5
                and margin
                or math.max(margin, viewport.X - size.X - margin)
            local restingY: number = math.clamp(
                currentY,
                margin,
                math.max(margin, viewport.Y - size.Y - margin)
            )
            local resting: UDim2 = UDim2.fromOffset(
                math.round(nearestX),
                math.round(restingY)
            )
            if state.animationsEnabled then
                TweenService:Create(
                    MobileMenuButton,
                    TweenInfo.new(0.42, Enum.EasingStyle.Back, Enum.EasingDirection.Out),
                    {Position = resting}
                ):Play()
            else
                MobileMenuButton.Position = resting
            end
            configData.ui.launcherX = resting.X.Offset
            configData.ui.launcherY = resting.Y.Offset
            queueConfigSave()
        end
    end))

    trackUiConnection(MobileMenuButton.MouseButton1Click:Connect(function(): ()

        if travelled >= 6 then
            return
        end
        state.openMenuSurface()
    end))
end

local function refreshLauncherVisibility(): ()
    -- Only once every section has initialized (state.menuReady) and one frame has
    -- passed, so the launcher can never appear before the menu it opens.
    MobileMenuButton.Visible = state.menuReady == true
        and (state.isMobile or state.toggleKey == Enum.KeyCode.Unknown)
        and state.pill == nil
        and not state.visible
end

state.refreshLauncher = refreshLauncherVisibility
refreshLauncherVisibility()
state.addMenuVisibilityListener(function(): ()
    refreshLauncherVisibility()
end)

type TaskHandle = {
    Connected: boolean,
    Disconnect: (self: TaskHandle) -> (),
    label: string,
}

type ScheduledTask = {
    callback: (...any) -> (),
    handle: TaskHandle,
    failures: number,
    label: string,
    costMs: number,
    calls: number,
    warned: boolean,
}

-- Scheduler callbacks are timed individually. These elapsed durations can
-- include yielding/scheduling delays and cannot by themselves prove why a
-- frame was long. The frame interval and the maximum callback are reported
-- separately; no frame is attributed to whichever callback happens to run next.
local TASK_WARN_MS: number = 3
local PERFORMANCE_EVERY: number = 0.5

local TaskManager: any = {
    tasks = {} :: {ScheduledTask},
    renderTasks = {} :: {ScheduledTask},
    taskCount = 0,
    renderCount = 0,
    nextId = 0,
    connection = nil :: RBXScriptConnection?,
    renderConnection = nil :: RBXScriptConnection?,
    tasksMs = 0,
    renderMs = 0,
}

state.performance = {
    fps = 60,
    frameMs = 1000 / 60,
    tasksMs = 0,
    renderMs = 0,
    taskCount = 0,
    renderCount = 0,
    slowestTask = nil :: (string | nil),
    slowestMs = 0,
    -- Heartbeat intervals and callback durations are independent measurements.
    spikeMs = 0,
    spikeCount = 0,
    spikeAgo = -1,
    spikeAt = nil :: (number | nil),
    spikeWarnedAt = nil :: (number | nil),
    worstSpikeMs = 0,
    peakTask = nil :: (string | nil),
    peakTaskMs = 0,
    peakTaskPhase = nil :: (string | nil),
    peakTaskAt = nil :: (number | nil),
    observedSince = os.clock(),
}

local function describeTask(callback: any): string
    -- "n" is the inferred function name (usually present for a named local
    -- function), "s" is the chunk/source. A chunk path ("something/file.rs:12")
    -- is not a module name, so those are skipped instead of filling the
    -- performance report with noise.
    for _, kind: string in ipairs({"n", "s"}) do
        local ok: boolean, info: any = pcall(debug.info, callback, kind)
        if ok and type(info) == "string" and info ~= "" then
            local name: string = tostring(info)
            if not string.find(name, "/", 1, true)
                and not string.find(name, "@", 1, true)
                and string.find(name, ".rs:", 1, true) == nil then
                if #name > 64 then
                    name = string.sub(name, 1, 64)
                end
                return name
            end
        end
    end
    return "task"
end

-- Swap-remove keeps the buckets as dense arrays, so the per-frame loop is an
-- ipairs over contiguous memory instead of pairs over a hash with holes.
local function removeFromBucket(bucket: {ScheduledTask}, index: number): ()
    local last: number = #bucket
    if index ~= last then
        bucket[index] = bucket[last]
        bucket[last].handle.index = index
    end
    bucket[last] = nil
end

function TaskManager:Connect(
    callback: (...any) -> (),
    phase: string?,
    label: string?
): TaskHandle
    self.nextId += 1
    local bucket: {ScheduledTask} = phase == "render"
        and self.renderTasks
        or self.tasks
    local handle: TaskHandle
    handle = {
        Connected = true,
        label = label or describeTask(callback),
        index = 0,
        Disconnect = function(selfHandle: any): ()
            if not selfHandle.Connected then
                return
            end
            selfHandle.Connected = false
            local current: {ScheduledTask} = selfHandle.bucket
            local index: number = selfHandle.index
            if current and index >= 1 and index <= #current
                and current[index] ~= nil and current[index].handle == selfHandle then
                removeFromBucket(current, index)
            end
        end,
    } :: any
    handle.bucket = bucket
    handle.index = #bucket + 1
    bucket[handle.index] = {
        callback = callback,
        handle = handle,
        failures = 0,
        label = handle.label,
        costMs = 0,
        calls = 0,
        warned = false,
    }
    if phase == "render" then
        self.renderCount = #self.renderTasks
    else
        self.taskCount = #self.tasks
    end
    return handle
end

function TaskManager:Stats(): any
    local slowest: any = nil
    local list: {any} = {}
    for _, bucket: {ScheduledTask} in ipairs({self.tasks, self.renderTasks}) do
        for _, scheduled: ScheduledTask in ipairs(bucket) do
            table.insert(list, {
                label = scheduled.label,
                ms = scheduled.costMs,
                calls = scheduled.calls,
            })
            if not slowest or scheduled.costMs > slowest.ms then
                slowest = {label = scheduled.label, ms = scheduled.costMs}
            end
        end
    end
    table.sort(list, function(left: any, right: any): boolean
        return left.ms > right.ms
    end)
    return {
        taskCount = #self.tasks,
        renderCount = #self.renderTasks,
        tasksMs = self.tasksMs,
        renderMs = self.renderMs,
        slowest = slowest,
        top = list,
    }
end

function TaskManager:Destroy(): ()
    for _, bucket: {ScheduledTask} in ipairs({self.tasks, self.renderTasks}) do
        for _, scheduled: ScheduledTask in ipairs(bucket) do
            scheduled.handle.Connected = false
        end
        table.clear(bucket)
    end
    self.taskCount = 0
    self.renderCount = 0
    if self.connection then
        self.connection:Disconnect()
        self.connection = nil
    end
    if self.renderConnection then
        self.renderConnection:Disconnect()
        self.renderConnection = nil
    end
end

local function runBucket(
    bucket: {ScheduledTask},
    deltaTime: number,
    phase: string
): number
    local total: number = 0
    local index: number = 1
    while index <= #bucket do
        local scheduled: ScheduledTask = bucket[index]
        local startedAt: number = os.clock()
        local success: boolean, message: any = pcall(
            scheduled.callback,
            deltaTime
        )
        -- Elapsed time, not a CPU profiler; a yielding callback may include
        -- time spent waiting. Retain the peak for the internal session metrics.
        local finishedAt: number = os.clock()
        local cost: number = math.max(0, (finishedAt - startedAt) * 1000)
        local performance: any = state.performance
        if cost > (performance.peakTaskMs or 0) then
            performance.peakTaskMs = cost
            performance.peakTask = scheduled.label
            performance.peakTaskPhase = phase
            performance.peakTaskAt = finishedAt
        end
        total += cost
        scheduled.calls += 1
        -- Fast attack, slow decay: a single spike is visible in the report but
        -- does not fire the warning; a task that is genuinely heavy will.
        if cost > scheduled.costMs then
            scheduled.costMs = cost
        else
            scheduled.costMs += (cost - scheduled.costMs) * 0.05
        end
        if not success then
```

```
            scheduled.failures += 1
            warn(
                "[" .. PRODUCT.name .. "] scheduled task "
                    .. scheduled.label
                    .. " failed: "
                    .. tostring(message)
            )
            if scheduled.failures >= 3 then
                scheduled.handle.Connected = false
            end
        else
            -- A recovered task should not lose its failure budget forever.
            -- Only three consecutive failing frames disable it.
            scheduled.failures = 0
        end
        if not scheduled.warned and scheduled.costMs >= TASK_WARN_MS then
            scheduled.warned = true
            warn(
                "[" .. PRODUCT.name .. ":Perf] "
                    .. scheduled.label
                    .. string.format(" recent callback score %.2f ms", scheduled.costMs)
                    .. " (decaying peak; elapsed may include waits). Compare with the module off."
            )
        end
        if not scheduled.handle.Connected then
            -- Disabled by its third failure (a real Disconnect already took it
            -- out of the bucket). Drop it now and let the task that the
            -- swap-remove moved into this slot run in this same frame.
            if bucket[index] == scheduled then
                removeFromBucket(bucket, index)
            end
        else
            -- It ran from this slot; the next task is index + 1. If it is no
            -- longer here, another task's disconnect swapped our slot and it
            -- already ran, so advancing is still correct.
            index += 1
        end
    end
    return total
end

local function publishPerformance(deltaTime: number): ()
    local performance: any = state.performance
    local frameMs: number = math.max(deltaTime, 1 / 1000) * 1000
    performance.frameMs += (frameMs - performance.frameMs) * 0.1
    performance.fps = 1000 / math.max(performance.frameMs, 1 / 1000)
    local stats: any = TaskManager:Stats()
    performance.taskCount = stats.taskCount
    performance.renderCount = stats.renderCount
    performance.tasksMs = stats.tasksMs
    performance.renderMs = stats.renderMs
    performance.slowestTask = stats.slowest and stats.slowest.label or nil
    performance.slowestMs = stats.slowest and stats.slowest.ms or 0
    performance.spikeAgo = if performance.spikeAt
        then math.max(0, os.clock() - performance.spikeAt)
        else -1
    performance.updatedAt = os.clock()
end

local performanceElapsed: number = 0

-- v139.1: a spike is one frame far over budget (a hitch), not a steady cost.
-- 45 ms is ~2.7 frames at 60 FPS: low enough to catch a real stutter, high
-- enough that ordinary jitter does not spam the console.
local SPIKE_MS: number = 45
local SPIKE_WARN_EVERY: number = 5

TaskManager.connection = RunService.Heartbeat:Connect(function(
    deltaTime: number
): ()
    -- deltaTime describes the interval since the previous frame; tasks run
    -- below now. Do not pair this delta with the current frame's task label.
    local total: number = runBucket(TaskManager.tasks, deltaTime, "heartbeat")
    TaskManager.tasksMs = total
    performanceElapsed += deltaTime
    if performanceElapsed >= PERFORMANCE_EVERY then
        performanceElapsed = 0
        publishPerformance(deltaTime)
    end

    local frameMs: number = math.max(0, deltaTime) * 1000
    local performance: any = state.performance
    if frameMs >= SPIKE_MS then
        performance.spikeCount += 1
        performance.spikeMs = frameMs
        performance.spikeAt = os.clock()
        if frameMs > performance.worstSpikeMs then
            performance.worstSpikeMs = frameMs
        end
        local now: number = os.clock()
        if now - (performance.spikeWarnedAt or -math.huge) >= SPIKE_WARN_EVERY then
            performance.spikeWarnedAt = now
            warn(string.format(
                "[%s:Perf] frame interval %.0f ms (cause not determined)",
                PRODUCT.name,
                frameMs
            ))
        end
    end
end)

TaskManager.renderConnection = RunService.RenderStepped:Connect(function(
    deltaTime: number
): ()
    local total: number = runBucket(TaskManager.renderTasks, deltaTime, "render")
    TaskManager.renderMs = total
end)

state.taskManager = TaskManager

local function disconnectFeatureConnection(name)
    local connection = featureConnections[name]
    if connection then
        connection:Disconnect()
        featureConnections[name] = nil
    end
end

local function getCharacterParts()
    local character = LocalPlayer.Character
    if not character then
        return nil, nil, nil
    end

    return character,
        character:FindFirstChildOfClass("Humanoid"),
        character:FindFirstChild("HumanoidRootPart")
end

do

local cachedControls: any = nil
local controlsFetchedAt: number = -math.huge

local function getControls(): any
    if cachedControls ~= nil or os.clock() - controlsFetchedAt < 2 then
        return cachedControls
    end
    controlsFetchedAt = os.clock()
    local ok: boolean, controls: any = pcall(function(): any
        local scripts: Instance? = LocalPlayer:FindFirstChildOfClass("PlayerScripts")
        local playerModule: Instance? = scripts and scripts:FindFirstChild("PlayerModule")
        if not playerModule then
            return nil
        end
        return (state.requireModule(playerModule) :: any):GetControls()
    end)
    if ok and type(controls) == "table" then
        cachedControls = controls
    end
    return cachedControls
end

state.getMoveInput = function(): (number, number)
    local controls: any = getControls()
    if controls then
        local ok: boolean, move: any = pcall(controls.GetMoveVector, controls)
        if ok and typeof(move) == "Vector3" and move.Magnitude > 0.001 then

            return -move.Z, move.X
        end
    end

    local _, humanoid: Humanoid? = getCharacterParts()
    if humanoid and humanoid.MoveDirection.Magnitude > 0.001 then
        local camera: Camera? = workspace.CurrentCamera
        if camera then
            local forward: Vector3 = camera.CFrame.LookVector
            local right: Vector3 = camera.CFrame.RightVector
            local flatForward: Vector3 = Vector3.new(forward.X, 0, forward.Z)
            local flatRight: Vector3 = Vector3.new(right.X, 0, right.Z)
            if flatForward.Magnitude > 0.001 and flatRight.Magnitude > 0.001 then
                local direction: Vector3 = humanoid.MoveDirection
                return direction:Dot(flatForward.Unit), direction:Dot(flatRight.Unit)
            end
        end
    end

    local forwardScalar: number = 0
    local rightScalar: number = 0
    if UserInputService.KeyboardEnabled then
        if UserInputService:IsKeyDown(Enum.KeyCode.W) then
            forwardScalar += 1
        end
        if UserInputService:IsKeyDown(Enum.KeyCode.S) then
            forwardScalar -= 1
        end
        if UserInputService:IsKeyDown(Enum.KeyCode.D) then
            rightScalar += 1
        end
        if UserInputService:IsKeyDown(Enum.KeyCode.A) then
            rightScalar -= 1
        end
    end
    return forwardScalar, rightScalar
end

local lastJumpRequestAt: number = -math.huge
trackUiConnection(UserInputService.JumpRequest:Connect(function(): ()
    lastJumpRequestAt = os.clock()
end))

state.onJumpRequest = function(callback: () -> ()): RBXScriptConnection
    return UserInputService.JumpRequest:Connect(function(): ()
        if state.keyCaptureCallback
            or state.waitingForKey
            or UserInputService:GetFocusedTextBox() then
            return
        end
        callback()
    end)
end

state.isJumpHeld = function(): boolean
    if UserInputService:GetFocusedTextBox() then
        return false
    end
    if UserInputService.KeyboardEnabled
        and UserInputService:IsKeyDown(Enum.KeyCode.Space) then
        return true
    end
    if UserInputService.GamepadEnabled
        and UserInputService:IsKeyDown(Enum.KeyCode.ButtonA) then
        return true
    end
    local _, humanoid: Humanoid? = getCharacterParts()
    if humanoid and humanoid.Jump then
        return true
    end

    return os.clock() - lastJumpRequestAt < 0.2
end

state.getAimRay = function(): Ray?
    local camera: Camera? = workspace.CurrentCamera
    if not camera then
        return nil
    end
    local pointer: Vector2
    if UserInputService.MouseEnabled then
        pointer = UserInputService:GetMouseLocation()
    else
        pointer = camera.ViewportSize / 2
    end
    return camera:ViewportPointToRay(pointer.X, pointer.Y)
end
end

local function getFeatureCardParent(_container: Instance): Instance
    return CardBin
end

local function applyFeaturePresentation(feature: any, isDefault: boolean): ()
    if feature.kind == "hold" then
        feature.enabled = feature.active == true
    end
    if state.applyCardSkin then
        state.applyCardSkin(feature, false)
    end
    if feature.options then
        feature.options.BackgroundTransparency = 1
    end

    if feature.shortcutButton then
        feature.shortcutButton.BackgroundColor3 = isDefault
            and Theme.surface
            or Theme.surfaceRaised

        local inertSlot: boolean = feature.kind == "group"
            and 0.86
            or (isDefault and 0.54 or 1)
        feature.shortcutButton.TextTransparency = inertSlot and 1 or 0
        feature.shortcutButton.TextColor3 = isDefault
            and Theme.text
            or Theme.text
    end
end
end
state.restyleAllFeatures = function(isDefault: boolean): ()
    for _, feature: any in ipairs(allFeatures) do
        applyFeaturePresentation(feature, isDefault)
    end
end

state.applyCardSkin = function(feature: any, hovered: boolean?): ()
    local row: Frame? = feature.row
    if not row then
        return
    end
    local resolvedRow: Frame = row :: Frame
    local isHovered: boolean = hovered == true
    local isOn: boolean = feature.enabled == true

    local surface: number
    local titleTransparency: number = 0
    local titleColor: Color3 = Theme.text

    local fill: Color3 = Theme.surface

    if feature.isCategory then

        surface = isHovered and 0.3 or 0.5
        titleColor = Theme.text
        titleTransparency = 0
    elseif feature.isAction then
        surface = isHovered and 0.3 or 0.5
    elseif feature.isHoldAction then

        fill = isOn and Theme.enabled or Theme.surface
        surface = isOn and 0 or (isHovered and 0.3 or 0.5)
        titleColor = isOn and Theme.enabledText or Theme.text
    else
        fill = isOn and Theme.enabled or Theme.surface
        surface = isOn and 0 or (isHovered and 0.3 or 0.5)
        titleColor = isOn and Theme.enabledText or Theme.text
    end

    resolvedRow.BackgroundColor3 = fill
    resolvedRow.BackgroundTransparency = surface

    if feature.drawTitle then
        -- Rainbow changes fills, not glyph topology. Reuse existing bitmap
        -- titles unless their actual colour/opacity changed.
        if feature.lastTitleColor ~= titleColor or feature.lastTitleTransparency ~= titleTransparency then
            feature.lastTitleColor = titleColor
            feature.lastTitleTransparency = titleTransparency
            feature.drawTitle(titleColor, titleTransparency)
        end
    elseif feature.title then
        local titleLabel: TextLabel = feature.title :: TextLabel
        titleLabel.TextColor3 = titleColor
        titleLabel.TextTransparency = titleTransparency
    end
end

ThemeEngine.OnChange(function(_name, changed): ()
    for _, feature: any in ipairs(allFeatures) do
        if feature.row then
            pcall(state.applyCardSkin, feature, false)
        end
    end
    -- Palette changes (Background/Accent/Text/Enabled) must also repaint the
    -- option checkboxes that were already on screen.
    if type(state.optionThemeRefresh) == "function" then
        pcall(state.optionThemeRefresh, changed)
    end
end)

local function refreshFeatureToggle(feature: any): ()
    if feature.isHoldAction then
        feature.enabled = feature.active == true
    end
    state.applyCardSkin(feature, false)
    if type(feature.syncBindSquare) == "function" then
        feature.syncBindSquare()
    end
    if state.floatingUi then
        state.floatingUi.refreshFeature(feature)
    end
    if state.navigator and state.navigator.refreshFeature then
        state.navigator.refreshFeature(feature)
    end
end

local function runActionButton(
    button: TextButton,
    idleText: string,
    callback: () -> (),
    successMessage: string,
    errorPrefix: string
): ()
    if button:GetAttribute("Running") == true then
        return
    end
    button:SetAttribute("Running", true)
    button:SetAttribute("Disabled", true)
    button.Active = false
    button.Text = "Running…"
    button.BackgroundColor3 = state.guiStyle == "Default"
        and Theme.buttonHover
        or Theme.accentSoft

    task.spawn(function(): ()
        local running: boolean = true
        task.spawn(function(): ()
            local frames: {string} = {"Running·", "Running··", "Running···"}
            local frameIndex: number = 1
            while running and button.Parent do
                button.Text = frames[frameIndex]
                frameIndex = frameIndex % #frames + 1
                task.wait(0.22)
            end
        end)

        local success: boolean, errorMessage: any = pcall(callback)
        running = false
        if button.Parent then
            button.Text = success and "Done" or "Failed"
            button.BackgroundColor3 = success and Theme.accentSoft or Theme.negative
        end
        if success then
            notify(successMessage)
        else
            warn(errorPrefix .. ": " .. tostring(errorMessage))
            notify("Error in " .. errorPrefix)
        end
        task.wait(0.45)
        if button.Parent then
            button.Text = idleText
            button.Active = true
            button:SetAttribute("Disabled", false)
            button:SetAttribute("Running", false)
            button.BackgroundColor3 = idleText == "RUN"
                and Theme.accentSoft
                or Theme.surface
        end
    end)
end

local function compactShortcutLabel(label: string): string
    local trimmed: string = string.upper(
        (string.gsub(string.gsub(label, "%s+", " "), "^%s*(.-)%s*$", "%1"))
    )
    if trimmed == "" then
        return "ACTION"
    end
    if #trimmed <= 18 then
        return trimmed
    end
    local cut: string = string.sub(trimmed, 1, 18)
    local lastSpace: number? = string.find(string.reverse(cut), " ")
    if lastSpace and lastSpace < 8 then
        cut = string.sub(cut, 1, 18 - lastSpace)
    end
    return cut
end

state.shortcutBindings = {} :: {[string]: any}

local function shortcutSlotText(binding: any): string

    if binding.key == Enum.KeyCode.Unknown then
        return ""
    end
    return binding.key.Name
end

local function setShortcutPressed(binding: any, pressed: boolean): ()
    if binding.pressed == pressed then
        return
    end
    binding.pressed = pressed
    if pressed then
        if binding.onPress then
            task.spawn(binding.onPress)
        end
    elseif binding.onRelease then
        task.spawn(binding.onRelease)
    end
end

local function bindActionShortcut(
    button: TextButton?,
    shortcutId: string,
    label: string,
    callback: () -> (),
    options: any?
): ()
    local configuration: any = options or {}
    local optionKey: string = "Shortcut." .. shortcutId:gsub("%W", "")
    local binding: any = {
        id = shortcutId,
        button = button,
        label = label,
        hold = configuration.hold == true,
        key = Enum.KeyCode.Unknown,
        pressed = false,
        onPress = configuration.onPress or callback,
        onRelease = configuration.onRelease,
        -- v176: a shortcut fired with the menu closed used to change state with
        -- no visible proof. The toast is how the user (and MM2, where several
        -- keys belong to the game) can tell the bind actually ran.
        feedback = configuration.feedback,
    }
    local savedName: any = configData.values[optionKey]
    if type(savedName) == "string" and savedName ~= "" then

        local resolved: boolean, savedKey: any = pcall(function(): any
            return (Enum.KeyCode :: any)[savedName]
        end)
        if resolved and savedKey then
            binding.key = savedKey
        end
    end
    state.shortcutBindings[shortcutId] = binding

    local function refreshButtonText(): ()
        if not button then
            return
        end
        (button :: TextButton).Text = shortcutSlotText(binding);
        (button :: TextButton):SetAttribute("CaptureIdleText", (button :: TextButton).Text)
    end
    binding.refresh = refreshButtonText
    refreshButtonText()

    binding.assign = function(keyCode: Enum.KeyCode): boolean
        local oldKey: Enum.KeyCode = binding.key
        local cleared: boolean = keyCode == oldKey
        binding.key = cleared and Enum.KeyCode.Unknown or keyCode
        setShortcutPressed(binding, false)
        configData.values[optionKey] = binding.key.Name
        queueConfigSave()
        -- Refresh both ends of any conflict, as well as this key's square.
        for _, other: any in pairs(state.shortcutBindings) do
            if other == binding
                or (oldKey ~= Enum.KeyCode.Unknown and other.key == oldKey)
                or (binding.key ~= Enum.KeyCode.Unknown and other.key == binding.key) then
                if type(other.refresh) == "function" then other.refresh() end
            end
        end
        return not cleared
    end

    if state.isMobile then

        if button and state.bindMobileActionPlacement then
            state.bindMobileActionPlacement(
                button,
                "Shortcut" .. shortcutId:gsub("%W", ""),
                compactShortcutLabel(label),
                function(): ()
                    if not binding.hold and binding.onPress then
                        binding.onPress()
                    end
                end,
                {
                    isActive = function(): boolean
                        return binding.hold == true
                    end,
                    onPress = function(): ()
                        if binding.hold and binding.onPress then
                            binding.onPress()
                        end
                    end,
                    onRelease = function(): ()
                        if binding.hold and binding.onRelease then
                            binding.onRelease()
                        end
                    end,
                }
            )
        end
        return
    end

    if button then
        trackUiConnection((button :: TextButton).MouseButton1Click:Connect(function(): ()
            beginKeyCapture(button, function(keyCode: Enum.KeyCode): ()
                binding.assign(keyCode)
            end)
        end))
    end
end

state.bindFeatureActivationKey = function(
    feature: any,
    defaultKey: Enum.KeyCode?,
    onPress: () -> (),
    onRelease: (() -> ())?
): ()
    local binding: any = state.shortcutBindings[feature.configKey]
    if not binding then
        return
    end
    binding.onPress = onPress
    binding.onRelease = onRelease
    binding.hold = onRelease ~= nil
    -- Activation registration never assigns a key. Only user key capture does.
    if binding.refresh then
        binding.refresh()
    end
end

-- v176: gameProcessed only says that the place (or the core GUI) already used
-- the input. Games such as MM2 bind their own actions to common keys, and one
-- of them was enough to make a menu keybind look dead while a card click
-- worked. The menu now honours its own shortcut even then; typing is still
-- blocked, because a focused TextBox keeps the key for itself.
do
local function shortcutInputAllowed(_gameProcessed: boolean): boolean
    return not state.keyCaptureCallback
        and not state.waitingForKey
        and UserInputService:GetFocusedTextBox() == nil
end

state.globalShortcutBeganConnection = UserInputService.InputBegan:Connect(
    function(input: InputObject, gameProcessed: boolean): ()
        if input.UserInputType ~= Enum.UserInputType.Keyboard
            or not shortcutInputAllowed(gameProcessed) then
            return
        end

        if input.KeyCode == Enum.KeyCode.Unknown then
            return
        end
        for _, binding: any in pairs(state.shortcutBindings) do
            if binding.key == input.KeyCode then
                if binding.hold then
                    setShortcutPressed(binding, true)
                elseif binding.onPress then
                    task.spawn(function(): ()
                        binding.onPress()
                        if type(binding.feedback) == "function" then
                            local ok: boolean, text: any = pcall(binding.feedback)
                            if ok and type(text) == "string" and text ~= "" then
                                notify(text)
                            end
                        end
                    end)
                end
            end
        end
    end
)

state.globalShortcutEndedConnection = UserInputService.InputEnded:Connect(
    function(input: InputObject): ()
        if input.UserInputType ~= Enum.UserInputType.Keyboard
            or input.KeyCode == Enum.KeyCode.Unknown then
            return
        end
        for _, binding: any in pairs(state.shortcutBindings) do
            if binding.hold and binding.key == input.KeyCode then
                setShortcutPressed(binding, false)
            end
        end
    end
)

trackUiConnection(UserInputService.WindowFocusReleased:Connect(function(): ()
    for _, binding: any in pairs(state.shortcutBindings) do
        if binding.hold then
            setShortcutPressed(binding, false)
        end
    end
end))
end

state.itemEspProviders = {}
state.itemEspListeners = {}

state.gameBridge = {
    roleProvider = nil :: any,

    teamProvider = nil :: any,
    extras = {} :: {any},
    listeners = {} :: {(string, any) -> ()},
}

state.onGameBridge = function(listener: (string, any) -> ()): () -> ()
    table.insert(state.gameBridge.listeners, listener)
```

```
    if state.gameBridge.roleProvider then
        pcall(listener, "roles", state.gameBridge.roleProvider)
    end
    if state.gameBridge.teamProvider then
        pcall(listener, "team", state.gameBridge.teamProvider)
    end
    for _, extra: any in ipairs(state.gameBridge.extras) do
        pcall(listener, "extra", extra)
    end

    return function(): ()
        local index: number? = table.find(state.gameBridge.listeners, listener)
        if index then
            table.remove(state.gameBridge.listeners, index)
        end
    end
end

state.registerTeamProvider = function(provider: any): ()
    if type(provider) ~= "table"
        or (type(provider.IsFriendly) ~= "function"
            and type(provider.Get) ~= "function"
            and type(provider.GetRelation) ~= "function") then
        return
    end
    state.gameBridge.teamProvider = provider
    state.teamProvider = provider
    for _, listener: (string, any) -> () in ipairs(state.gameBridge.listeners) do
        pcall(listener, "team", provider)
    end
end

state.registerRoleProvider = function(provider: any): ()
    if type(provider) ~= "table" or type(provider.Get) ~= "function" then
        return
    end
    state.gameBridge.roleProvider = provider
    if type(state.clearRoleMemo) == "function" then
        state.clearRoleMemo()
    end
    for _, listener: (string, any) -> () in ipairs(state.gameBridge.listeners) do
        pcall(listener, "roles", provider)
    end
end

-- Ownership-aware removal: an old game's cleanup cannot erase a newer provider.
state.unregisterRoleProvider = function(provider: any): ()
    if state.gameBridge.roleProvider ~= provider then return end
    state.gameBridge.roleProvider = nil
    if state.clearRoleMemo then state.clearRoleMemo() end
    for _, listener in ipairs(state.gameBridge.listeners) do pcall(listener, "roles", nil) end
end

state.registerEspExtra = function(extra: any): ()
    if type(extra) ~= "table" or type(extra.Name) ~= "string" then
        return
    end
    table.insert(state.gameBridge.extras, extra)
    for _, listener: (string, any) -> () in ipairs(state.gameBridge.listeners) do
        pcall(listener, "extra", extra)
    end
end

local roleMemo: {[Player]: {role: string?, at: number}} =
    setmetatable({}, {__mode = "k"}) :: any
local roleColorMemo: {[Player]: {colour: Color3?, at: number}} =
    setmetatable({}, {__mode = "k"}) :: any
local ROLE_MEMO_SECONDS: number = 1 / 30

-- Advanced team analysis. Always enabled: it is not a module and currently
-- cannot be turned off. Every query tries several independent methods and
-- returns the first that can decide:
--   1. the game's own team provider (registered via registerTeamProvider)
--   2. role sides, when the game registers roles (MM2: Innocent/Hero/Sheriff
--      side together, the Murderer alone)
--   3. the Roblox Teams service (player.Team)
--   4. team colours, but only when the game actually uses teams
--   5. neutral players are never teammates
-- Anything the methods cannot decide is reported as unknown, never guessed.
local TEAM_SIDE_BY_ROLE: {[string]: string} = {
    ["murderer"] = "hostile",
    ["killer"] = "hostile",
    ["traitor"] = "hostile",
    ["zombie"] = "hostile",
    ["infected"] = "hostile",
    ["sheriff"] = "guard",
    ["hero"] = "guard",
    ["detective"] = "guard",
    ["innocent"] = "guard",
    ["survivor"] = "guard",
    ["crewmate"] = "guard",
    ["dead"] = "none",
}

state.teamCheck = {}

-- Team/faction attributes are explicit game data, not Roblox's shared
-- lobby Team. Check the same common names used by the entity library.
local TEAM_ATTRIBUTE_NAMES: {string} = {"Team", "TeamId", "TeamID", "Faction", "FactionId", "FactionID"}
local function normalizeTeamToken(value: any): string?
    if typeof(value) == "Instance" then value = value.Name end
    if type(value) ~= "string" and type(value) ~= "number" then return nil end
    local token: string = string.lower(string.gsub(tostring(value), "^%s*(.-)%s*$", "%1"))
    if token == "" or token == "none" or token == "unknown" or token == "outside" then return nil end
    return token
end

local function teamAttribute(player: Player): string?
    for _, name: string in ipairs(TEAM_ATTRIBUTE_NAMES) do
        local ok: boolean, value: any = pcall(player.GetAttribute, player, name)
        local token: string? = ok and normalizeTeamToken(value) or nil
        if token then return token end
    end
    -- Some games expose a StringValue/ObjectValue named Team instead of a
    -- Roblox Team property; retain Entity's existing support for that form.
    local ok: boolean, value: any = pcall(function(): any
        local child: Instance? = player:FindFirstChild("Team")
        return child and (child :: any).Value or nil
    end)
    return ok and normalizeTeamToken(value) or nil
end

local function providerToken(provider: any, player: Player): string?
    local ok: boolean, value: any = pcall(provider.Get, player)
    return ok and normalizeTeamToken(value) or nil
end

local function isFfaToken(token: string?): boolean
    return token == "ffa" or token == "freeforall"
end

function state.teamCheck.analyze(player: Player?): {[string]: any}
    if not player or player == LocalPlayer then
        return {sameTeam = false, method = "self", hostile = false}
    end

    local provider: any = state.gameBridge.teamProvider
    if provider and type(provider.GetRelation) == "function" then
        local ok: boolean, relation: any = pcall(provider.GetRelation, player)
        if ok and relation == "Teammate" then
            return {sameTeam = true, method = "provider", hostile = false}
        end
        if ok and relation == "Enemy" then
            return {sameTeam = false, method = "provider", hostile = true}
        end
        -- Unknown/Outside are not hostile confirmations. Never reinterpret
        -- them using a shared Roblox lobby Team.
        return {sameTeam = nil, method = "provider-unknown", hostile = nil}
    end
    if provider and type(provider.IsFriendly) == "function" then
        local ok: boolean, friendly: any = pcall(provider.IsFriendly, player)
        if ok and type(friendly) == "boolean" then
            return {sameTeam = friendly, method = "provider", hostile = not friendly}
        end
    end
    if provider and type(provider.Get) == "function" then
        local mine: string? = providerToken(provider, LocalPlayer)
        local theirs: string? = providerToken(provider, player :: Player)
        if mine and theirs then
            if isFfaToken(mine) and isFfaToken(theirs) then
                return {sameTeam = false, method = "provider-ffa", hostile = true}
            end
            if not isFfaToken(mine) and not isFfaToken(theirs) then
                return {sameTeam = mine == theirs, method = "provider", hostile = mine ~= theirs}
            end
        end
        -- A provider owns this decision even while its data is incomplete.
        return {sameTeam = nil, method = "provider-unknown", hostile = nil}
    end
    if provider then
        return {sameTeam = nil, method = "provider-unknown", hostile = nil}
    end

    local roleProvider: any = state.gameBridge.roleProvider
    if roleProvider and type(state.playerRole) == "function" then
        local myRole: any = state.playerRole(LocalPlayer)
        local theirRole: any = state.playerRole(player :: Player)
        if type(myRole) == "string" and type(theirRole) == "string" then
            local mySide: string? = TEAM_SIDE_BY_ROLE[string.lower(myRole)]
            local theirSide: string? = TEAM_SIDE_BY_ROLE[string.lower(theirRole)]
            if mySide and theirSide and mySide ~= "none" and theirSide ~= "none" then
                return {sameTeam = mySide == theirSide, method = "roles", hostile = mySide ~= theirSide}
            end
            if not mySide and not theirSide and myRole == theirRole
                and myRole ~= "Unknown" and myRole ~= "None" then
                return {sameTeam = true, method = "roles", hostile = false}
            end
        end
        if roleProvider.ReplacesTeams == true then
            return {sameTeam = nil, method = "roles-unknown", hostile = nil}
        end
    end

    local myAttribute: string? = teamAttribute(LocalPlayer)
    local theirAttribute: string? = teamAttribute(player :: Player)
    if myAttribute or theirAttribute then
        if not myAttribute or not theirAttribute then
            return {sameTeam = nil, method = "attributes-unknown", hostile = nil}
        end
        if isFfaToken(myAttribute) and isFfaToken(theirAttribute) then
            return {sameTeam = false, method = "attributes-ffa", hostile = true}
        end
        if isFfaToken(myAttribute) or isFfaToken(theirAttribute) then
            return {sameTeam = nil, method = "attributes-unknown", hostile = nil}
        end
        return {sameTeam = myAttribute == theirAttribute, method = "attributes", hostile = myAttribute ~= theirAttribute}
    end

    -- Neutral is checked before Team/TeamColor. An assigned but inactive Team
    -- must not make two neutral players allies.
    if LocalPlayer.Neutral == true or (player :: Player).Neutral == true then
        return {sameTeam = false, method = "neutral", hostile = true}
    end
    local myTeam: Team? = LocalPlayer.Team
    local theirTeam: Team? = (player :: Player).Team
    if myTeam ~= nil or theirTeam ~= nil then
        if myTeam == nil or theirTeam == nil then
            return {sameTeam = false, method = "teams", hostile = true}
        end
        return {
            sameTeam = (myTeam :: Team) == (theirTeam :: Team),
            method = "teams",
            hostile = (myTeam :: Team) ~= (theirTeam :: Team),
        }
    end

    local teamsService: any = game:FindService("Teams")
    if teamsService and #teamsService:GetTeams() > 0 then
        local sameColour: boolean = LocalPlayer.TeamColor == (player :: Player).TeamColor
        return {sameTeam = sameColour, method = "teamcolor", hostile = not sameColour}
    end
    return {sameTeam = nil, method = "unknown", hostile = nil}
end

function state.teamCheck.isSameTeam(player: Player?): boolean
    local verdict: any = state.teamCheck.analyze(player)
    return verdict.sameTeam == true
end

function state.teamCheck.isEnemy(player: Player?): boolean
    local verdict: any = state.teamCheck.analyze(player)
    if verdict.hostile ~= nil then
        return verdict.hostile
    end
    if verdict.sameTeam == false then
        return true
    end
    return false
end

state.playerRole = function(player: Player): string?
    local provider: any = state.gameBridge.roleProvider
    if not provider then
        return nil
    end
    local now: number = os.clock()
    local cached: any = roleMemo[player]
    if cached and now - cached.at < ROLE_MEMO_SECONDS then
        return cached.role
    end
    local resolved: string? = nil
    local ok: boolean, role: any = pcall(provider.Get, player)
    if ok and type(role) == "string" and role ~= "" then
        resolved = role
    end
    roleMemo[player] = {role = resolved, at = now}
    return resolved
end

state.playerRoleColor = function(player: Player): Color3?
    local provider: any = state.gameBridge.roleProvider
    if not provider then
        return nil
    end
    local now: number = os.clock()
    local cached: any = roleColorMemo[player]
    if cached and now - cached.at < ROLE_MEMO_SECONDS then
        return cached.colour
    end
    local role: string? = state.playerRole(player)
    local resolved: Color3? = nil
    if role then
        if type(provider.GetColor) == "function" then
            local ok: boolean, colour: any = pcall(provider.GetColor, role)
            if ok and typeof(colour) == "Color3" then
                resolved = colour
            end
        end
        if not resolved then
            local colour: any = provider.Colors and provider.Colors[role]
            if typeof(colour) == "Color3" then
                resolved = colour
            end
        end
    end
    roleColorMemo[player] = {colour = resolved, at = now}
    return resolved
end

state.playerPerk = function(player: Player): string?
    local provider: any = state.gameBridge.roleProvider
    if not provider or type(provider.GetPerk) ~= "function" then
        return nil
    end
    local ok: boolean, perk: any = pcall(provider.GetPerk, player)
    if ok and type(perk) == "string" and perk ~= "" then
        return perk
    end
    return nil
end

local function clearRoleMemo(): ()
    table.clear(roleMemo :: any)
    table.clear(roleColorMemo :: any)
end
state.clearRoleMemo = clearRoleMemo

state.invalidateGameRoles = function(): ()
    clearRoleMemo()
    for _, listener: (string, any) -> () in ipairs(state.gameBridge.listeners) do
        pcall(listener, "roleUpdate", state.gameBridge.roleProvider)
    end
end

state.emitGameBridgeEvent = function(eventType: string, payload: any): ()
    for _, listener: (string, any) -> () in ipairs(state.gameBridge.listeners) do
        pcall(listener, eventType, payload)
    end
end

state.featureTooltip = create("TextLabel", {
    TextTruncate = Enum.TextTruncate.AtEnd,
    ClipsDescendants = true,
    Parent = PopupLayer,
    Name = "FeatureTooltip",
    Active = false,
    AutomaticSize = Enum.AutomaticSize.None,
    BackgroundColor3 = Theme.surface,
    BackgroundTransparency = 0.04,
    BorderColor3 = Color3.fromRGB(58, 58, 58),
    BorderSizePixel = 1,
    FontFace = THIN_FONT,
    Size = UDim2.fromOffset(320, 30),
    Text = "",
    TextColor3 = Color3.fromRGB(205, 205, 205),
    TextSize = 11,
    TextWrapped = true,
    Visible = false,
    ZIndex = 250,
}) :: TextLabel

create("UIPadding", {
    Parent = state.featureTooltip,
    PaddingBottom = UDim.new(0, 5),
    PaddingLeft = UDim.new(0, 8),
    PaddingRight = UDim.new(0, 8),
    PaddingTop = UDim.new(0, 5),
})

state.tooltipToken = 0

local function showFeatureTooltip(row: Frame, text: string, pointer: Vector2?): ()
    state.tooltipToken = state.tooltipToken + 1
    state.featureTooltip.Text = text
    state.featureTooltip.TextTransparency = 1

    local bitmap: any = state.bitmapText
    local innerLimit: number = 488

    local wrapped: string, lineTotal: number = bitmap.wrap(text, innerLimit, 16)
    local measured: number = bitmap.measure({{text = wrapped}}, 16)
    local width: number = math.clamp(measured + 24, 210, 520)
    local height: number = math.max(30, lineTotal * bitmap.lineHeight + 12)
    state.featureTooltip.Size = UDim2.fromOffset(width, height)
    bitmap.draw(state.featureTooltip, {
        {text = wrapped, color = Color3.fromRGB(205, 205, 205)},
    }, {
        Name = "Glyphs",
        TextSize = 16,
        ZIndex = 251,
        AnchorPoint = Vector2.new(0, 0.5),
        Position = UDim2.new(0, 0, 0.5, 0),
    })

    local manager: any = state.windows
    local viewport: Vector2
    if manager and type(manager.LogicalViewport) == "function" then
        viewport = manager.LogicalViewport()
    else
        local camera: Camera? = workspace.CurrentCamera
        viewport = camera and camera.ViewportSize or Vector2.new(1280, 720)
    end
    local anchor: Vector2 = pointer
        or Vector2.new(
            row.AbsolutePosition.X + 16,
            row.AbsolutePosition.Y + row.AbsoluteSize.Y * 0.5
        )
    if manager and type(manager.ScreenToLogical) == "function" then
        anchor = manager.ScreenToLogical(anchor)
    end
    local x: number = math.clamp(anchor.X + 12, 8, math.max(8, viewport.X - width - 8))
    local y: number = math.clamp(
        anchor.Y - height - 8,
        8,
        math.max(8, viewport.Y - height - 8)
    )

    state.featureTooltip.Position = UDim2.fromOffset(x, y)
    state.featureTooltip.Visible = true
end

local function hideFeatureTooltip(): ()
    state.tooltipToken = state.tooltipToken + 1
    state.featureTooltip.Visible = false
end

local COMPACT_FEATURE_CARDS: boolean = true
local function addFeatureTooltip(feature: any, text: string): ()
    if COMPACT_FEATURE_CARDS then
        return
    end
    feature.tooltipText = text
    if feature.tooltipBound then
        return
    end
    feature.tooltipBound = true

    local row: Frame = feature.row :: Frame
    local heldInput: InputObject? = nil
    local heldStart: Vector2? = nil

    table.insert(state.tooltipConnections, row.MouseLeave:Connect(function(): ()
        heldInput = nil
        heldStart = nil
        hideFeatureTooltip()
    end))

    table.insert(state.tooltipConnections, row.InputBegan:Connect(function(input: InputObject): ()
        if input.UserInputType ~= Enum.UserInputType.Touch
            and input.UserInputType ~= Enum.UserInputType.MouseButton1 then
            return
        end

        heldInput = input
        heldStart = Vector2.new(input.Position.X, input.Position.Y)
        state.tooltipToken = state.tooltipToken + 1
        local pressToken: number = state.tooltipToken
        task.delay(0.42, function(): ()
            if heldInput == input and state.tooltipToken == pressToken then
                local pointer: Vector2 = input.UserInputType == Enum.UserInputType.Touch
                    and Vector2.new(input.Position.X, input.Position.Y)
                    or UserInputService:GetMouseLocation()
                showFeatureTooltip(row, feature.tooltipText, pointer)
            end
        end)
    end))

    table.insert(state.tooltipConnections, row.InputChanged:Connect(function(input: InputObject): ()
        if heldInput == input and heldStart then
            local current: Vector2 = Vector2.new(input.Position.X, input.Position.Y)
            if (current - heldStart).Magnitude > 12 then
                heldInput = nil
                heldStart = nil
                hideFeatureTooltip()
            end
        end
    end))

    table.insert(state.tooltipConnections, row.InputEnded:Connect(function(input: InputObject): ()
        if heldInput == input then
            heldInput = nil
            heldStart = nil
            hideFeatureTooltip()
        end
    end))
end

type FlyRuntime = {
    yLevel: number?,
    burstAt: number,
    platform: BasePart?,
    walkSpeed: number?,
}

local findPlayerByText: (string) -> Player? = function(text: string): Player?
    local query: string = string.lower(text)
    if query == "" then
        return nil
    end
    local players: {Player} = Players:GetPlayers()
    for pass: number = 1, 3 do
        for _, player: Player in ipairs(players) do
            local name: string = string.lower(player.Name)
            local displayName: string = string.lower(player.DisplayName)
            local matched: boolean = pass == 1
                and (name == query or displayName == query)
                or pass == 2
                and (name:sub(1, #query) == query
                    or displayName:sub(1, #query) == query)
                or (name:find(query, 1, true) ~= nil
                    or displayName:find(query, 1, true) ~= nil)
            if matched then
                return player
            end
        end
    end
    return nil
end
local performFling: (Player) -> () = function(_target: Player): () end

state.isProtectedTarget = function(_targetPlayer: Player): boolean
    return false
end

local MM2Scroll: Frame = CardBin

local cleanupMM2Runtime = function() end

type GameModule = {
    Name: string,
    PlaceId: number,
    Events: {[string]: any},
    Initialized: boolean,
    init: (runtime: any) -> any,
    destroy: () -> (),
}

type ModuleDiagnostics = {
    uiBindings: number,
    taskBindings: number,
}

state.logBootstrap = function(message: string): ()
    print("[" .. PRODUCT.logPrefix .. ":Bootstrap] " .. message)
end

print("[" .. PRODUCT.logPrefix .. "] build loader-v3 · " .. SOURCE_STAMP)

state.logBootstrapError = function(message: string): ()
    warn("[" .. PRODUCT.logPrefix .. ":Bootstrap] " .. message)
end

local widgetBuilders: any = nil

local function createGameModuleEnvironment(
    diagnostics: ModuleDiagnostics
): any
    local baseEnvironment: any = getfenv()
    local function recordBinding(callback: (...any) -> any): (...any) -> any
        return function(...: any): any
            diagnostics.uiBindings += 1
            return callback(...)
        end
    end
    local environment: any = {
        game = game,
        workspace = workspace,

        print = print,
        warn = warn,
        Players = Players,
        UserInputService = UserInputService,
        TweenService = TweenService,
        Lighting = Lighting,
        StarterGui = game:GetService("StarterGui"),
        RunService = RunService,
        HttpService = HttpService,
        CollectionService = CollectionService,
        Stats = Stats,
        LocalPlayer = LocalPlayer,
        PlayerGui = PlayerGui,
        ScreenGui = ScreenGui,

        PRODUCT = PRODUCT,
        Theme = Theme,

        ThemeEngine = ThemeEngine,

        GAME_CHECK = GAME_CHECK,

        CONTROL_FONT = CONTROL_FONT,
        TITLE_FONT = TITLE_FONT,
        THIN_FONT = THIN_FONT,
        DEFAULT_HEADER_COLOR = DEFAULT_HEADER_COLOR,
        DEFAULT_SURFACE_COLOR = DEFAULT_SURFACE_COLOR,
        DEFAULT_BORDER_COLOR = DEFAULT_BORDER_COLOR,
        state = state,

        services = nil,
        configData = configData,
        queueConfigSave = queueConfigSave,
        flushConfigSave = flushConfigSave,
        create = create,
        BUILD_LABEL = BUILD_LABEL,
        suppressTouchControls = suppressTouchControls,
        makeTextLabel = makeTextLabel,
        makeButton = makeButton,
        makeTextBox = makeTextBox,
        notify = notify,

        isMenuOwned = function(instance: Instance): boolean
            return instance:IsDescendantOf(ScreenGui)
        end,
        TaskManager = TaskManager,
        featureConnections = featureConnections,
        disconnectFeatureConnection = disconnectFeatureConnection,
        getCharacterParts = getCharacterParts,

        addFeatureTooltip = addFeatureTooltip,
        cloneReference = cloneReference,

        PopupLayer = PopupLayer,
        UI_RADIUS = UI_RADIUS,
        UI_MOTION = UI_MOTION,
        cornerRadius = cornerRadius,
        setKeySlotCapture = setKeySlotCapture,

        SettingsPage = CardBin,
        getFeatureCardParent = getFeatureCardParent,

        REPOSITORY_RAW_BASE = REPOSITORY_RAW_BASE,
        RUNTIME_SAFETY_SOURCE_URL = RUNTIME_SAFETY_SOURCE_URL,
        RUNTIME_COMPATIBILITY_MARKER = RUNTIME_COMPATIBILITY_MARKER,
        SOURCE_STAMP = SOURCE_STAMP,

        TEXT_BASE_MULTIPLIER = TEXT_BASE_MULTIPLIER,
        TEXT_SCALE_MINIMUM = TEXT_SCALE_MINIMUM,
        TEXT_SCALE_MAXIMUM = TEXT_SCALE_MAXIMUM,
        registeredText = registeredText,

        initialViewport = initialViewport,

        FEATURE_CATEGORIES = FEATURE_CATEGORIES,
        categories = {
            order = OFFICIAL_CATEGORIES,
            aliases = CATEGORY_ALIASES,
            default = DEFAULT_FEATURE_CATEGORY,
        },
        DEFAULT_FEATURE_CATEGORY = DEFAULT_FEATURE_CATEGORY,

        OFFICIAL_CATEGORIES = OFFICIAL_CATEGORIES,
        normalizeFeatureCategory = normalizeFeatureCategory,
        universalFeatures = universalFeatures,
        movementFeatures = movementFeatures,
        applyFeaturePresentation = applyFeaturePresentation,
        refreshFeatureToggle = refreshFeatureToggle,
        VALVE_FONT = VALVE_FONT,
        trackUiConnection = trackUiConnection,
        createToggleSwitch = createToggleSwitch,
        setToggleSwitch = setToggleSwitch,
        applyKeySlotStyle = applyKeySlotStyle,
        beginKeyCapture = beginKeyCapture,
        bindActionShortcut = bindActionShortcut,
        runActionButton = runActionButton,
        colorFromConfig = colorFromConfig,
        colorToConfig = colorToConfig,

        registerRoleProvider = state.registerRoleProvider,
        registerTeamProvider = state.registerTeamProvider,
        registerEspExtra = state.registerEspExtra,
        onGameBridge = state.onGameBridge,
        playerRole = state.playerRole,
        teamCheck = state.teamCheck,
        findPlayerByText = findPlayerByText,
```

```
        performFling = performFling,
        MM2Scroll = MM2Scroll,
        mm2Features = mm2Features,
        vdFeatures = state.vdFeatures,
        bedFightFeatures = state.bedFightFeatures,
        bedWarsFeatures = state.bedWarsFeatures,
        mvsdFeatures = state.mvsdFeatures,
        allFeatures = allFeatures,
        cleanupMM2Runtime = cleanupMM2Runtime,
    }

    for builderName: string, builder: any in pairs(widgetBuilders or {}) do
        environment[builderName] = recordBinding(builder)
    end
    return setmetatable(environment, {
        __index = baseEnvironment,
    })
end

local function loadGameModule(
    placeId: number
): (GameModule?, any, ModuleDiagnostics, string?)
    local executorEnvironment: any = getfenv()
    local diagnostics: ModuleDiagnostics = {
        uiBindings = 0,
        taskBindings = 0,
    }
    if type(executorEnvironment.loadstring) ~= "function"
        or type(executorEnvironment.setfenv) ~= "function" then
        return nil, nil, diagnostics, "The executor does not support loadstring/setfenv"
    end

    local moduleName: string = GAME_MODULES[placeId]
        or ACTIVE_GAME_MODULE
        or tostring(placeId)
    local paths: {string} = state.gameModulePaths[moduleName]
        or {"src/games/" .. moduleName .. "/base.lua"}
    local moduleEnvironment: any = createGameModuleEnvironment(diagnostics)
    -- Game modules get the same services facade the framework environment
    -- uses, so game modules can access services.gameBridge consistently.
    moduleEnvironment.services = state.moduleServices
    local loadedModules: {GameModule} = {}
    local skipped: {string} = {}
    for _, path: string in paths do
        local source: any = nil
        local failure: any = nil
        if type(state.fetchRepositorySource) == "function" then
            source, failure = state.fetchRepositorySource(path)
        else
            local ok: boolean
            ok, source = pcall(function(): string
                return (game :: any):HttpGet(REPOSITORY_RAW_BASE .. path .. "?v=" .. SOURCE_STAMP, true)
            end)
            if not ok then failure = source source = nil end
        end
        if type(source) ~= "string" or #source < 64 then
            local readFile: any = executorEnvironment.readfile
            local localPath: string = "Wurst/" .. path
            local ok: boolean = false
            if type(readFile) == "function" then ok, source = pcall(readFile, localPath) end
            if not ok or type(source) ~= "string" then
                table.insert(skipped, path .. " -> " .. tostring(failure))
                continue
            end
        end
        local chunk: any, compileError: any = executorEnvironment.loadstring(source, "@" .. path)
        if type(chunk) ~= "function" then
            table.insert(skipped, path .. " -> " .. tostring(compileError))
            continue
        end
        executorEnvironment.setfenv(chunk, moduleEnvironment)
        local ok: boolean, loaded: any = pcall(chunk)
        if not ok or type(loaded) ~= "table" or type(loaded.init) ~= "function" or type(loaded.destroy) ~= "function" then
            table.insert(skipped, path .. " -> " .. tostring(loaded))
            continue
        end
        table.insert(loadedModules, loaded)
    end

    if #loadedModules == 0 then
        return nil, nil, diagnostics,
            "No " .. moduleName .. " file loaded (" .. table.concat(skipped, " | ") .. ")"
    end
    for _, reason: string in ipairs(skipped) do
        state.logBootstrapError("Skipped " .. moduleName .. " file " .. reason)
    end
    local composite: any = {Name = moduleName, Initialized = false}
    function composite.init(runtime: any): any
        local initializedCount: number = 0
        local failures: {string} = {}
        for _, loaded: GameModule in loadedModules do
            local ok: boolean, failure: any = pcall(loaded.init, runtime)
            if ok then
                initializedCount += 1
            else

                pcall(loaded.destroy)
                table.insert(failures, tostring(loaded.Name) .. ": " .. tostring(failure))
                state.logBootstrapError(
                    moduleName .. " module " .. tostring(loaded.Name)
                        .. " failed: " .. tostring(failure)
                )
            end
        end
        if initializedCount == 0 then
            error(table.concat(failures, " | "))
        end
        composite.Initialized = true
        return composite
    end
    function composite.destroy(): ()
        for index: number = #loadedModules, 1, -1 do pcall(loadedModules[index].destroy) end
        composite.Initialized = false
    end
    return composite :: GameModule, moduleEnvironment, diagnostics, nil
end

local function registerPlaceModule(
    tabName: string,
    placeId: number
): ()

    if game.PlaceId ~= placeId
        and ACTIVE_GAME_MODULE ~= GAME_MODULES[placeId] then
        return
    end

    state.logBootstrap(
        "Detected PlaceId: "
            .. tostring(game.PlaceId)
            .. " · module "
            .. tostring(ACTIVE_GAME_MODULE)
            .. " (" .. ACTIVE_GAME_SOURCE .. ")"
    )
    SectionManager.register(tabName, function(): ()
        local loadedModule: GameModule?, _moduleEnvironment: any,
            diagnostics: ModuleDiagnostics, loadError: string? =
            loadGameModule(GAME_MODULES[game.PlaceId] and game.PlaceId or placeId)
        if not loadedModule then
            state.logBootstrapError(
                "Could not load " .. tabName .. ": " .. tostring(loadError)
            )
            error(loadError or "Could not load the module")
        end
        local taskCountBefore: number = TaskManager.nextId
        local featureCountBefore: number = #allFeatures
        local connectionsBefore: {[string]: boolean} = {}
        for connectionName: string in pairs(featureConnections) do
            connectionsBefore[connectionName] = true
        end
        local runtime: any = {
            Menu = {
                Root = CardBin,
                ScreenGui = ScreenGui,
                Pages = {},
                Components = state.components,
                Notify = notify,
            },
            State = state,
            Services = state.frameworkEnvironment.services,
            TaskManager = TaskManager,
            Diagnostics = diagnostics,
        }
        local initialized: boolean, initError: any = pcall(
            loadedModule.init,
            runtime
        )
        if not initialized then
            loadedModule.Initialized = true
            local rolledBack: boolean, rollbackError: any = pcall(
                loadedModule.destroy
            )
            if not rolledBack then
                state.logBootstrapError(
                    tabName .. " rollback failed: " .. tostring(rollbackError)
                )
            end
        end
        for connectionName: string in pairs(featureConnections) do
            if not connectionsBefore[connectionName] then
                disconnectFeatureConnection(connectionName)
            end
        end
        for taskId: number, scheduled: ScheduledTask in pairs(TaskManager.tasks) do
            if taskId > taskCountBefore then
                scheduled.handle:Disconnect()
            end
        end
        for index: number = #allFeatures, featureCountBefore + 1, -1 do
            local feature: any = allFeatures[index]
            if feature.enabled then
                pcall(feature.onToggle, false)
            end
            if feature.registry then
                feature.registry[feature.name] = nil
            end
            if feature.row and feature.row.Parent then
                feature.row:Destroy()
            end

            if feature.settingsWindow then
                pcall(function(): ()
                    feature.settingsWindow:Destroy()
                end)
                feature.settingsWindow = nil
            end
            table.remove(allFeatures, index)

            if state.moduleSearch
                and type(state.moduleSearch.Unregister) == "function" then
                state.moduleSearch.Unregister(feature)
            end

            if feature.configKey and state.shortcutBindings then
                state.shortcutBindings[feature.configKey] = nil
            end
            if state.floatingUi and state.floatingUi.unregisterFeature then
                state.floatingUi.unregisterFeature(feature)
            end
        end
        state.logBootstrapError(tabName .. " init failed: " .. tostring(initError))
        error(initError)
    end
    diagnostics.taskBindings = TaskManager.nextId - taskCountBefore
    state.activeGameModule = loadedModule
    state.logBootstrap(
        tabName
            .. " active · UI callbacks="
            .. tostring(diagnostics.uiBindings)
            .. " · Task callbacks="
            .. tostring(diagnostics.taskBindings)
    )
    local function destroyModule(): ()
        local destroyed: boolean, destroyError: any = pcall(
            loadedModule.destroy
        )
        if not destroyed then
            state.logBootstrapError(
                tabName .. " destroy failed: " .. tostring(destroyError)
            )
        end
    end
    if tabName == "MM2" then
        cleanupMM2Runtime = destroyModule
    elseif tabName == "TRS" then
        state.cleanupTRSRuntime = destroyModule
    elseif tabName == "VD" then
        state.cleanupVDRuntime = destroyModule
    elseif tabName == "BedFight" then
        state.cleanupBedFightRuntime = destroyModule
    elseif tabName == "X2Duels" then
        state.cleanupX2DuelsRuntime = destroyModule
    end
end)
end

state.logBootstrap(
    "Initializing PlaceId="
        .. tostring(game.PlaceId)
        .. " · module="
        .. tostring(ACTIVE_GAME_MODULE or "Universal")
)

registerPlaceModule("MM2", GAME_CHECK.MM2)
registerPlaceModule("TRS", GAME_CHECK.TRS)
registerPlaceModule("VD", GAME_CHECK.VD)
registerPlaceModule("BedFight", GAME_CHECK.BedFight)
registerPlaceModule("X2Duels", GAME_CHECK.X2Duels)

if PENDING_FINGERPRINT then
    task.spawn(function(): ()
        local attempts: number = 0
        while attempts < 6 do
            attempts += 1
            task.wait(0.5)
            local detected: string?, score: number = detectGameModuleByStructure()
            if detected then
                ACTIVE_GAME_MODULE = detected
                ACTIVE_GAME_SOURCE = "fingerprint " .. tostring(score) .. " markers"
                state.activeGameName = detected
                GAME_CHECK.MM2Active = detected == "MM2"
                GAME_CHECK.TRSActive = detected == "TRS"
                GAME_CHECK.VDActive = detected == "VD"
                GAME_CHECK.BedFightActive = detected == "BedFight"
                GAME_CHECK.X2DuelsActive = detected == "X2Duels"
                GAME_CHECK.source = ACTIVE_GAME_SOURCE
                PENDING_FINGERPRINT = false
                registerPlaceModule(detected, GAME_CHECK[detected] or 0)
                SectionManager.initialize(detected)
                break
            end
        end
    end)
end

state.boot = {
    stamp = SOURCE_STAMP,
    bundle = nil :: {[string]: string}?,
    sources = {} :: {[string]: string},
    timing = {} :: {[string]: number},
    started = BOOT_STARTED_AT,
}

state.boot.timing.gui = os.clock() - BOOT_STARTED_AT

state.markBoot = function(phase: string, startedAt: number): ()
    state.boot.timing[phase] = os.clock() - startedAt
end

state.summarizeBoot = function(): ()
    local timing: any = state.boot.timing
    local total: number = os.clock() - (state.boot.started or os.clock())
    print(string.format(
        "[" .. PRODUCT.logPrefix .. ":Timing] fingerprint %.2fs · gui %.2fs · bundle %.2fs · manifest %.2fs · widgets %.2fs · framework %.2fs · libraries %.2fs · modules %.2fs (%d/%d) · game module %.2fs · total %.2fs",
        BOOT_FINGERPRINT_SECONDS,
        tonumber(timing.gui) or 0,
        tonumber(timing.bundle) or 0,
        tonumber(timing.manifest) or 0,
        tonumber(timing.widgets) or 0,
        tonumber(timing.framework) or 0,
        tonumber(timing.libraries) or 0,
        tonumber(timing.modules) or 0,
        tonumber(timing.moduleCount) or 0,
        tonumber(timing.moduleTotal) or 0,
        tonumber(timing.game) or 0,
        total
    ))
end

state.sourceLooksValid = function(source: any, minimum: number): boolean
    if type(source) ~= "string" or #source < minimum then
        return false
    end
    local head: string = string.lower(string.sub(source, 1, 512))
    return string.find(head, "404: not found", 1, true) == nil
        and string.find(head, "<!doctype html", 1, true) == nil
        and string.find(head, "<html", 1, true) == nil
end

-- v195: a failed download used to be reported as whatever body came back, so a
-- wrong branch surfaced as `Bundle: 404: Not Found` (or as nothing at all when
-- the executor threw). The branch is the most common cause -- a branch that was
-- never pushed returns the same text as a missing file -- so the message names
-- it explicitly and the reason is short enough to fit in one console line.
state.describeHttpFailure = function(body: any): string
    local text: string = ""
    if type(body) == "string" then
        text = string.sub(body, 1, 512)
    else
        text = tostring(body)
    end
    local lowered: string = string.lower(text)
    local cause: string
    if string.find(lowered, "404: not found", 1, true) then
        cause = "HTTP 404"
    elseif string.find(lowered, "<!doctype html", 1, true)
        or string.find(lowered, "<html", 1, true) then
        cause = "HTML answer, not Lua"
    elseif #text < 32 then
        cause = "empty answer (" .. tostring(#text) .. " bytes)"
    else
        cause = string.sub(string.gsub(text, "%s+", " "), 1, 160)
    end
    return cause .. " · branch " .. RUNTIME_BRANCH
end

state.httpGetPath = function(path: string): (string?, string?)
    local url: string = REPOSITORY_RAW_BASE .. path .. "?v=" .. SOURCE_STAMP
    local ok: boolean, body: any = pcall(function(): string
        return (game :: any):HttpGet(url, true)
    end)
    if ok and state.sourceLooksValid(body, 32) then
        return body :: string, nil
    end
    if not ok then
        return nil, "HttpGet threw: " .. tostring(body) .. " · branch " .. RUNTIME_BRANCH
    end
    return nil, state.describeHttpFailure(body)
end

state.fetchRepositorySource = function(path: string): (string?, string?)
    local bundled: any = state.boot.bundle and state.boot.bundle[path]
    if type(bundled) == "string" then
        return bundled, nil
    end
    local remembered: string? = state.boot.sources[path]
    if remembered then
        return remembered, nil
    end
    local downloaded: string?, downloadError: string? = state.httpGetPath(path)
    if downloaded then
        state.boot.sources[path] = downloaded
        return downloaded, nil
    end
    return nil, downloadError or ("Download failed: " .. path)
end

-- Old builds wrote one file per source plus a stamp into the executor folder.
-- Nothing reads that any more; clear it once, in the background, so it neither
-- slows the boot nor keeps several hundred KB on the device.
state.purgeLegacySourceCache = function(): number
    local environment: any = getfenv()
    local listFiles: any = environment.listfiles
    local deleteFile: any = environment.delfile
    local isFolder: any = environment.isfolder
    if type(listFiles) ~= "function" or type(deleteFile) ~= "function" then
        return 0
    end
    local removed: number = 0
    local function purge(folder: string): ()
        local listed: boolean, entries: any = pcall(listFiles, folder)
        if not listed or type(entries) ~= "table" then
            return
        end
        for _, raw: any in ipairs(entries) do
            local entry: string = tostring(raw)
            local directory: boolean = false
            if type(isFolder) == "function" then
                local checked: boolean, verdict: any = pcall(isFolder, entry)
                directory = checked and verdict == true
            end
            if directory then
                purge(entry)
            else
                if pcall(deleteFile, entry) then
                    removed += 1
                end
            end
        end
    end
    purge("Wurst/src")
    pcall(deleteFile, "Wurst/.stamp")
    return removed
end

state.compileRepositorySource = function(path: string, source: string): (any, string?)
    local environment: any = getfenv()
    if type(environment.loadstring) ~= "function" then
        return nil, "The executor does not support loadstring"
    end
    local chunk: any, compileError: any = environment.loadstring(source, "@" .. path)
    if type(chunk) ~= "function" then
        return nil, "Compilation failed (" .. path .. "): " .. tostring(compileError)
    end
    if type(environment.setfenv) == "function" and state.frameworkEnvironment then
        environment.setfenv(chunk, state.frameworkEnvironment)
    end
    local ran: boolean, result: any = pcall(chunk)
    if not ran then
        return nil, "Execution failed (" .. path .. "): " .. tostring(result)
    end
    return result, nil
end

state.fetchRepositoryChunk = function(path: string): (any, string?)
    local source: string?, sourceError: string? = state.fetchRepositorySource(path)
    if not source then
        return nil, sourceError
    end
    return state.compileRepositorySource(path, source :: string)
end

state.downloadSourcesParallel = function(paths: {string}): ()
    local pending: number = #paths
    if pending == 0 then
        return
    end
    local index: number = 0
    local workers: number = math.min(8, pending)
    local timedOut: boolean = false
    for _ = 1, workers do
        task.spawn(function(): ()
            while not timedOut do
                index += 1
                local current: number = index
                if current > #paths then
                    return
                end
                local path: string = paths[current]
                if not state.boot.sources[path] then

                    local ok: boolean, source: any = pcall(
                        state.fetchRepositorySource,
                        path
                    )
                    if ok and type(source) == "string" then
                        state.boot.sources[path] = source
                    end
                end
                pending -= 1
            end
        end)
    end
    local deadline: number = os.clock() + 12
    while pending > 0 and os.clock() < deadline do
        task.wait(0.03)
    end
    if pending > 0 then
        timedOut = true
        for _, path: string in ipairs(paths) do
            if not state.boot.sources[path] then
                local source: string? = state.fetchRepositorySource(path)
                if source then
                    state.boot.sources[path] = source
                end
            end
        end
    end
end

do
(function(): ()
local diagnostics: ModuleDiagnostics = {uiBindings = 0, taskBindings = 0}
state.frameworkEnvironment = createGameModuleEnvironment(diagnostics)

type OwnershipService = {
    set: (string, any?) -> (),
    apply: (Instance?) -> (),
}
type ModuleServices = {
    movementInput: any,
    aim: any,
    menu: any,
    mobileActions: any,
    protectedTargets: any,
    fovOwnership: OwnershipService,
    platformStandOwnership: OwnershipService,
    activity: any,
    spoofAvatar: any,
    projectileCalibration: any,
    gameBridge: any,
    shortcuts: any,
    registries: any,
}

local function makePropertyOwnership(propertyName: string): OwnershipService
    local owners: {[string]: any} = {}
    local baselines: any = setmetatable({}, {__mode = "k"})
    local sequence: number = 0
    local service: any = {}
    function service.set(ownerId: string, value: any?): ()
        if value == nil then
            owners[ownerId] = nil
            if next(owners) == nil then
                for instance: any, baseline: any in pairs(baselines) do
                    if instance.Parent then
                        instance[propertyName] = baseline
                    end
                end
                baselines = setmetatable({}, {__mode = "k"})
            end
            return
        end
        local owner: any = owners[ownerId]
        if not owner then
            sequence += 1
            owner = {sequence = sequence, value = value}
            owners[ownerId] = owner
        else
            owner.value = value
        end
    end
    function service.apply(instance: Instance?): ()
        if not instance then
            return
        end
        local target: any = instance
        if baselines[target] == nil then
            baselines[target] = target[propertyName]
        end
        local selected: any = nil
        for _, owner: any in pairs(owners) do
            if not selected or owner.sequence > selected.sequence then
                selected = owner
            end
        end
        if selected then
            target[propertyName] = selected.value
        end
    end
    return service :: OwnershipService
end

local protectedTargetProvider: (Player?) -> boolean = function(player: Player?): boolean
    return player == nil or player == LocalPlayer
end
local activityFlags: {[string]: boolean} = {}
local spoofEmotes: {any} = {}
local spoofDescription: HumanoidDescription? = nil
local spoofRig: Model? = nil
local projectileCalibrationOwner: any = nil

local SCREEN_CAPTURE_NAMES: {string} = {
    "screenshot",
    "captureScreenshot",
    "capture_screenshot",
    "getScreenshot",
}
local function findScreenCapture(): any
    local environment: any = getfenv()
    for _, name: string in ipairs(SCREEN_CAPTURE_NAMES) do
        local candidate: any = environment[name]
        if type(candidate) == "function" then
            return candidate
        end
    end
    return nil
end
local screenCaptureService: any = {
    isAvailable = function(): boolean
        return findScreenCapture() ~= nil
    end,
    capture = function(path: string): (boolean, string?)
        local environment: any = getfenv()
        local capture: any = findScreenCapture()
        if type(capture) ~= "function" then
            return false, "executor has no screen capture API"
        end
        local folder: string? = string.match(path, "^(.*)/[^/]+$")
        local isFolder: any = environment.isfolder
        local makeFolder: any = environment.makefolder
        if folder and type(isFolder) == "function" and type(makeFolder) == "function" then
            local parent: string? = string.match(folder :: string, "^(.*)/[^/]+$")
            if parent and not isFolder(parent) then
                pcall(makeFolder, parent)
            end
            if not isFolder(folder :: string) then
                pcall(makeFolder, folder)
            end
        end

        local ok: boolean, result: any = pcall(capture, path)
        if not ok then
            ok, result = pcall(capture)
        end
        if not ok then
            return false, tostring(result)
        end
        if result == false then
            return false, "screen capture API declined the request"
        end

        if type(result) == "string"
            and result ~= ""
            and type(environment.writefile) == "function" then
            pcall(environment.writefile, path, result)
        end
        return true, nil
    end,
    writeText = function(path: string, contents: string): boolean
        local environment: any = getfenv()
        if type(environment.writefile) ~= "function" then
            return false
        end
        return pcall(environment.writefile, path, contents) == true
    end,
}

local moduleServices: ModuleServices = {
    movementInput = {
        getVector = state.getMoveInput,
        isJumpHeld = state.isJumpHeld,
        onJumpRequest = state.onJumpRequest,
    },
```

```
    aim = {getRay = state.getAimRay},
    screenCapture = screenCaptureService,
    menu = {
        isVisible = function(): boolean return state.visible == true end,
        isCapturingInput = function(): boolean
            return state.keyCaptureCallback ~= nil or state.waitingForKey == true
        end,
        setVisible = function(visible: boolean): () state.setMenuVisible(visible) end,
    },
    mobileActions = {

        bindPlacement = function(...: any): any
            local bindPlacement: any = state.bindMobileActionPlacement
            if type(bindPlacement) == "function" then
                return bindPlacement(...)
            end
            return nil
        end,
    },
    protectedTargets = {
        isProtected = function(player: Player?): boolean
            return protectedTargetProvider(player)
        end,
        setProvider = function(provider: ((Player?) -> boolean)?): ()
            protectedTargetProvider = provider or function(player: Player?): boolean
                return player == nil or player == LocalPlayer
            end
        end,
    },
    fovOwnership = makePropertyOwnership("FieldOfView"),
    platformStandOwnership = makePropertyOwnership("PlatformStand"),
    activity = {
        set = function(name: string, active: boolean): () activityFlags[name] = active end,
        isActive = function(name: string): boolean return activityFlags[name] == true end,
    },
    spoofAvatar = {
        getEmotes = function(): {any} return spoofEmotes end,
        setEmotes = function(emotes: {any}): () spoofEmotes = emotes end,
        getDescription = function(): HumanoidDescription? return spoofDescription end,
        setDescription = function(value: HumanoidDescription?): () spoofDescription = value end,
        getRig = function(): Model? return spoofRig end,
        setRig = function(value: Model?): () spoofRig = value end,
    },
    projectileCalibration = {
        set = function(owner: any): () projectileCalibrationOwner = owner end,
        get = function(): any return projectileCalibrationOwner end,
    },
    itemEsp = {
        register = function(provider: any): (() -> ())?
            if type(provider) ~= "table"
                or type(provider.name) ~= "string"
                or #provider.name == 0
                or type(provider.setEnabled) ~= "function" then
                return nil
            end
            for _, existing: any in ipairs(state.itemEspProviders) do
                if existing.name == provider.name then
                    return nil
                end
            end
            table.insert(state.itemEspProviders, provider)
            for _, listener: () -> () in ipairs(state.itemEspListeners) do
                pcall(listener)
            end
            return function(): ()
                local index: number? =
                    table.find(state.itemEspProviders, provider)
                if index then
                    table.remove(state.itemEspProviders, index :: number)
                    for _, listener: () -> () in ipairs(state.itemEspListeners) do
                        pcall(listener)
                    end
                end
            end
        end,
        providers = function(): {any}
            return state.itemEspProviders
        end,
        onChanged = function(listener: () -> ()): (() -> ())?
            table.insert(state.itemEspListeners, listener)
            return function(): ()
                local index: number? =
                    table.find(state.itemEspListeners, listener)
                if index then
                    table.remove(state.itemEspListeners, index :: number)
                end
            end
        end,
    },
    gameBridge = {
        onEvent = state.onGameBridge,
        teamCheck = state.teamCheck,
        playerRole = state.playerRole,
        playerRoleColor = state.playerRoleColor,
        playerPerk = state.playerPerk,
        registerRoles = state.registerRoleProvider,
        unregisterRoles = state.unregisterRoleProvider,

        silentShot = function(): any
            return state.lastSilentShotInfo
        end,
    },
    shortcuts = {bindActivation = state.bindFeatureActivationKey},
    registries = {
        wurstOptions = function(): any return state.wurstOptions end,
        moduleSearch = function(): any return state.moduleSearch end,
    },
}
state.frameworkEnvironment.services = moduleServices
state.moduleServices = moduleServices


local bundleAt: number = os.clock()
local packed: any, packedError: string? = state.fetchRepositoryChunk("src/libraries/bundle.lua")
if type(packed) == "table"
    and packed.stamp == SOURCE_STAMP
    and type(packed.files) == "table" then
    state.boot.bundle = packed.files
    state.logBootstrap("Bundle accepted · stamp=" .. SOURCE_STAMP)

    -- Off the boot path: drop the legacy per-source cache folder, and sweep
    -- stale assets. Neither blocks the menu.
    task.spawn(function(): ()
        local purgedSources: number = state.purgeLegacySourceCache()
        local staleAssets: number = state.sweepStaleAssets()
        if purgedSources + staleAssets > 0 then
            print("[" .. PRODUCT.logPrefix .. ":Cache] purged "
                .. tostring(purgedSources) .. " legacy source file(s), "
                .. tostring(staleAssets) .. " stale asset(s)")
        end
    end)
else
    if packedError then
        state.logBootstrapError("Bundle: " .. packedError)
    elseif type(packed) == "table" then
        state.logBootstrapError(
            "Bundle stamp mismatch: got "
                .. tostring(packed.stamp)
                .. " expected "
                .. SOURCE_STAMP
        )
    end
    state.boot.bundle = nil
end
state.markBoot("bundle", bundleAt)

local manifestAt: number = os.clock()
local manifest: any, manifestError: string? =
    state.fetchRepositoryChunk("src/libraries/Manifest.lua")
if type(manifest) ~= "table" or type(manifest.modules) ~= "table" then
    -- v138.9: there is no second catalog any more. The old duplicated copy in
    -- this file silently booted a DIFFERENT menu whenever the fetch hiccuped; a
    -- missing catalog is now a loud, single failure.
    if manifestError then
        state.logBootstrapError("Manifest: " .. manifestError)
    end
    state.logBootstrapError(
        "Manifest: src/libraries/Manifest.lua did not load; the menu cannot boot"
    )
    if type(state.notify) == "function" then
        pcall(state.notify, "Wurst: the module catalog could not be loaded")
    end
    return
end
local catalogCategories: any = manifest.categories
if type(catalogCategories) == "table" then
    if type(catalogCategories.order) == "table" and #catalogCategories.order > 0 then
        OFFICIAL_CATEGORIES = catalogCategories.order
    end
    if type(catalogCategories.aliases) == "table" then
        CATEGORY_ALIASES = catalogCategories.aliases
    end
end
if type(manifest.cards) == "table" then
    FEATURE_CATEGORIES = manifest.cards
end
state.categories = {
    order = OFFICIAL_CATEGORIES,
    aliases = CATEGORY_ALIASES,
    default = DEFAULT_FEATURE_CATEGORY,
}
state.markBoot("manifest", manifestAt)

-- Derive game module paths and place IDs from the manifest so nothing is
-- hardcoded in loadGameModule or GAME_MODULES.
state.gameModulePaths = manifest.gameModules or {}
for _, entry: any in ipairs(manifest.gamePlaces or {}) do
    if type(entry) == "table" and type(entry.placeId) == "number"
        and type(entry.module) == "string" then
        GAME_MODULES[entry.placeId] = entry.module
    end
end

local loadList: {string} = {}
table.insert(loadList, manifest.widgets or "src/guis/Wurst/Code/Widgets.lua")
for _, path: string in ipairs(manifest.core or {}) do
    table.insert(loadList, path)
end
for _, path: string in ipairs(manifest.libraries or {}) do
    table.insert(loadList, path)
end
for _, entry: any in ipairs(manifest.modules or {}) do
    table.insert(loadList, type(entry) == "table" and entry.path or tostring(entry))
end
if not state.boot.bundle then
    state.downloadSourcesParallel(loadList)
end

local loaded: {any} = {}
local function loadPiece(path: string): any?
    local piece: any, failure: string? = state.fetchRepositoryChunk(path)
    if type(piece) ~= "table" or type(piece.init) ~= "function" then
        state.logBootstrapError(
            "Invalid module: " .. path .. " · " .. tostring(failure)
        )
        return nil
    end
    return piece
end

local widgetsAt: number = os.clock()
local widgetPath: string = manifest.widgets or "src/guis/Wurst/Code/Widgets.lua"
local widgetPiece: any? = loadPiece(widgetPath)
if widgetPiece then
    local widgetOk: boolean, builders: any = pcall(
        widgetPiece.init,
        state.frameworkEnvironment
    )
    if widgetOk and type(builders) == "table" then
        table.insert(loaded, widgetPiece)
        widgetBuilders = builders
        for builderName: string, builder: any in pairs(builders) do
            state.frameworkEnvironment[builderName] = function(...: any): any
                diagnostics.uiBindings += 1
                return builder(...)
            end
        end
    else
        state.logBootstrapError("Widgets init failed: " .. tostring(builders))
    end
end
state.markBoot("widgets", widgetsAt)
if not widgetBuilders then
    state.logBootstrapError(
        "Widget library unavailable; modules cannot build their settings"
    )
    notify("Could not load the widget library")
    state.summarizeBoot()
    return
end

local frameworkAt: number = os.clock()
local frameworkPiece: any? = loadPiece(manifest.core[1] or "src/libraries/Framework.lua")
if not frameworkPiece then
    state.logBootstrapError("Framework unavailable; external modules skipped")
    state.summarizeBoot()
    return
end
local frameworkOk: boolean, framework: any = pcall(
    frameworkPiece.init,
    state.frameworkEnvironment
)
if not frameworkOk or type(framework) ~= "table" then
    state.logBootstrapError("Framework init failed: " .. tostring(framework))
    state.summarizeBoot()
    return
end
table.insert(loaded, frameworkPiece)
state.framework = framework
state.markBoot("framework", frameworkAt)

local librariesAt: number = os.clock()
local libraries: any = {}
for _, path: string in ipairs(manifest.libraries or {}) do
    local piece: any? = loadPiece(path)
    if not piece then
        continue
    end
    local startedOk: boolean, library: any = pcall(
        piece.init,
        state.frameworkEnvironment
    )
    if not startedOk or type(library) ~= "table" then
        state.logBootstrapError(
            "Library failed (" .. path .. "): " .. tostring(library)
        )
        continue
    end
    table.insert(loaded, piece)
    local key: string = string.lower(
        string.match(path, "([^/]+)%.lua$") or path
    )
    libraries[key] = library
end

local cardLibrary: any = libraries.cards
if cardLibrary and type(cardLibrary.createUniversalFeature) == "function" then
    widgetBuilders.createUniversalFeature = cardLibrary.createUniversalFeature
    state.frameworkEnvironment.createUniversalFeature = function(...: any): any
        diagnostics.uiBindings += 1
        return cardLibrary.createUniversalFeature(...)
    end
else
    state.logBootstrapError("Card factory unavailable; modules cannot create cards")
    notify("Could not load the card factory")
    return
end

local entityLibrary: any = libraries.entity
state.weaponLibrary = libraries.weapons
state.renderLibrary = libraries.render
state.markBoot("libraries", librariesAt)

local context: any = {
    framework = framework,
    entity = entityLibrary,
    weapons = libraries.weapons,
    render = libraries.render,
    libraries = libraries,
    host = state.frameworkEnvironment,
    services = moduleServices,
}
local modulesAt: number = os.clock()
local moduleCount: number = 0
for _, entry: any in ipairs(manifest.modules) do
    local path: string = type(entry) == "table" and entry.path or tostring(entry)
    local piece: any? = loadPiece(path)
    if piece then
        local started: boolean, failure: any = pcall(piece.init, context)
        if started then
            table.insert(loaded, piece)
            moduleCount += 1
            state.logBootstrap("Module loaded: " .. path)
        else
            state.logBootstrapError(
                "Module failed (" .. path .. "): " .. tostring(failure)
            )
        end
    end
end
state.boot.timing.moduleCount = moduleCount
state.boot.timing.moduleTotal = #manifest.modules
state.markBoot("modules", modulesAt)

if moduleCount < #manifest.modules then
    local missing: number = #manifest.modules - moduleCount
    state.logBootstrapError(
        tostring(missing)
            .. " de "
            .. tostring(#manifest.modules)
            .. " modules did not start; look for \"Module failed\" above"
    )
    notify(tostring(missing) .. " modules did not load — check the console")
end

state.cleanupFrameworkRuntime = function(): ()
    for index: number = #loaded, 1, -1 do
        local piece: any = loaded[index]
        if type(piece.destroy) == "function" then
            pcall(piece.destroy)
        end
    end
    table.clear(loaded)
    state.framework = nil
    state.renderLibrary = nil
end
state.summarizeBoot()
end)()
end

do
    local savedFont: string = tostring(configData.values.interfaceFont or "")
    if savedFont == "" or savedFont == "Menu default"

        or savedFont == "Minecraft (Monocraft)"
        or savedFont == "Candy Fruits" then
        savedFont = "Code"
    end
    task.spawn(function(): ()
        state.applyInterfaceFont(savedFont, false)
    end)
end

do
local function initializeWindowInput(): ()

local function reflowForViewport(force: boolean?): ()
    task.defer(function(): ()
        if not ScreenGui.Parent then
            return
        end
        if not state.refreshLayout(force) then return end
        if state.floatingUi and state.floatingUi.reclamp then
            state.floatingUi.reclamp()
        end
    end)
end

local viewportConnection: RBXScriptConnection? = nil
local function bindViewport(): ()
    if viewportConnection then
        viewportConnection:Disconnect()
        viewportConnection = nil
    end
    local camera: Camera? = workspace.CurrentCamera
    if not camera then
        return
    end
    viewportConnection = trackUiConnection(
        camera:GetPropertyChangedSignal("ViewportSize"):Connect(function(): ()
            reflowForViewport(false)
        end)
    )
end

bindViewport()

trackUiConnection(workspace:GetPropertyChangedSignal("CurrentCamera"):Connect(
    function(): ()
        bindViewport()
        reflowForViewport(true)
    end
))

state.globalInputBeganConnection = UserInputService.InputBegan:Connect(function(input)
    if state.keyCaptureCallback then
        if input.UserInputType == Enum.UserInputType.Keyboard
            and input.KeyCode ~= Enum.KeyCode.Unknown then
            if input.KeyCode == Enum.KeyCode.Escape then
                if state.keyCaptureCancel then
                    state.keyCaptureCancel()
                end
                return
            end
            local callback = state.keyCaptureCallback
            callback(input.KeyCode)
            task.defer(function(): ()
                if state.keyCaptureCallback == callback then
                    state.keyCaptureCallback = nil
                end
            end)
        end
        return
    end

    if state.waitingForKey then
        if input.UserInputType == Enum.UserInputType.Keyboard
            and input.KeyCode ~= Enum.KeyCode.Unknown then
            if input.KeyCode == Enum.KeyCode.Escape then
                state.waitingForKey = false
                if state.keybindButton then
                    state.keybindButton.Text = state.keyDisplayName(state.toggleKey.Name)
                    setKeySlotCapture(state.keybindButton, false)
                end
                return
            end
            state.toggleKey = input.KeyCode
            if state.keybindButton then
                state.keybindButton.Text = state.keyDisplayName(input.KeyCode.Name)
                setKeySlotCapture(state.keybindButton, false)
            end
            configData.ui.toggleKey = input.KeyCode.Name
            queueConfigSave()
            notify("Menu key: " .. input.KeyCode.Name)
            task.defer(function(): ()
                state.waitingForKey = false
            end)
        end
        return
    end

    if UserInputService:GetFocusedTextBox() then
        return
    end

    if input.UserInputType == Enum.UserInputType.Keyboard
        and input.KeyCode == state.toggleKey then
        state.openMenuSurface()
    end
end)
end
initializeWindowInput()

task.defer(function(): ()
    state.refreshLayout(true)
end)
end


local runtime: any = createRuntimeDestructor({
    state = state,
    MenuBlur = MenuBlur,
    ScreenGui = ScreenGui,
    allFeatures = allFeatures,
    featureConnections = featureConnections,
    TaskManager = TaskManager,
    cleanupMM2Runtime = cleanupMM2Runtime,
    closeConfigSave = closeConfigSave,
    setMenuCursorUnlocked = setMenuCursorUnlocked,
    disconnectFeatureConnection = disconnectFeatureConnection,
    persistenceGenv = persistenceGenv,
    logPrint = print,
    warn = warn,
})
state.destruct = runtime.destructRuntime
state.cleanupRuntime = runtime.cleanupRuntime
runtime.bindHandoff()

SectionManager.initializeAllAsync()
task.defer(saveConfigNow)

print(
    PRODUCT.name
        .. " v"
        .. PRODUCT.version
        .. " | Toggle key: "
        .. state.toggleKey.Name
        .. " | build "
        .. SOURCE_STAMP
        .. " | branch "
        .. RUNTIME_BRANCH
)

end

local initialized: boolean, bootstrapError: any = xpcall(bootstrap, function(message: any): string
    local text: string = tostring(message)
    if debug and type(debug.traceback) == "function" then
        return debug.traceback("[" .. PRODUCT.logPrefix .. ":Bootstrap] " .. text, 2)
    end
    return "[" .. PRODUCT.logPrefix .. ":Bootstrap] " .. text
end)

if not initialized then

    local reportFatal: any = (getfenv() :: any).warn
    if type(reportFatal) == "function" then
        reportFatal("[" .. PRODUCT.logPrefix .. ":Bootstrap] Initialization stopped:\n" .. tostring(bootstrapError))
    end

    pcall(function(): ()
        local players: any = game:GetService("Players")
        local playerGui: any = players.LocalPlayer:WaitForChild("PlayerGui")
        local gui: any = Instance.new("ScreenGui")
        gui.Name = "WurstBootError"
        gui.ZIndexBehavior = Enum.ZIndexBehavior.Sibling
        gui.DisplayOrder = 9999
        gui.Parent = playerGui
        local box: any = Instance.new("Frame")
        box.BackgroundColor3 = Color3.fromRGB(26, 24, 30)
        box.BorderSizePixel = 0
        box.Size = UDim2.fromOffset(440, 150)
        box.Position = UDim2.new(0.5, -220, 0.06, 0)
        box.Parent = gui
        Instance.new("UICorner", box).CornerRadius = UDim.new(0, 8)
        local label: any = Instance.new("TextLabel")
        label.BackgroundTransparency = 1
        label.Size = UDim2.new(1, -28, 1, -16)
        label.Position = UDim2.fromOffset(14, 8)
        label.Font = Enum.Font.Gotham
        label.TextSize = 13
        label.TextXAlignment = Enum.TextXAlignment.Left
        label.TextYAlignment = Enum.TextYAlignment.Top
        label.TextWrapped = true
        label.TextColor3 = Color3.fromRGB(255, 122, 122)
        label.Text = "[Wurst] no arranco:\n" .. tostring(bootstrapError):sub(1, 400)
        label.Parent = box
    end)
end

--------------------------------------------------------------------------------
FIN DEL ARCHIVO: src/Wurst.lua
--------------------------------------------------------------------------------
```